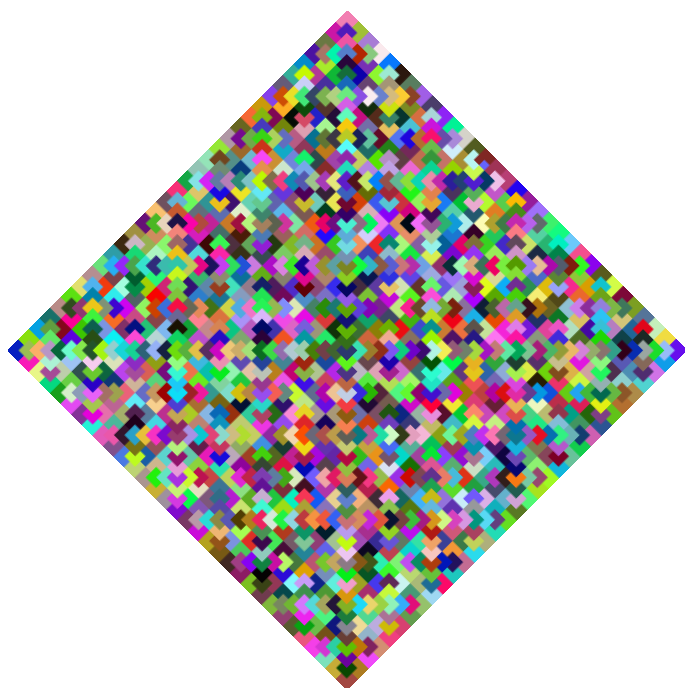




Diseño y Análisis de Algoritmos

Camilo Rocha

Email address: me@camilorocha.info

Para Laura.

© Derechos de autor 2019-2026 Camilo Rocha.
Última actualización 4 de octubre de 2026.
Versión 0.0



Esta obra está bajo una licencia de Creative Commons
Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional.
Este trabajo puede ser copiado y distribuido libremente, como copia electrónica o en
papel. No puede ser vendido por un valor mayor a su costo actual de reproducción,
almacenamiento o transmisión.

Índice general

Prólogo	XIII
¿Para quién es este libro?	XV
¿Por qué leer este libro?	XV
¿Qué contiene este libro y cómo está organizado?	XVII
¿Cómo leer este libro?	XIX
¿Cómo, cuándo y dónde se escribió este libro?	XXI
Agradecimientos	XXII
Capítulo 0. Preliminares	1
0.1. Arreglos: notación y convenciones	2
0.2. Notación de cuantificadores	9
0.3. Problemas algorítmicos y su especificación	18
0.4. Tipos de problemas	28
0.5. El concepto de algoritmo	33
0.6. Especificación de algoritmos	39
0.7. Corrección de algoritmos	44
0.8. Relaciones bien fundadas e inducción	50
0.9. Inducción sobre los números naturales	60
Notas del capítulo y referencias	66
Capítulo 1. Análisis asintótico	69
1.1. Medidas para problemas y algoritmos	70
1.2. Notación asintótica	81

1.3.	Propiedades de la notación asintótica	90
1.4.	Funciones y sumas de referencia	98
1.5.	Análisis de algoritmos iterativos	106
1.6.	Recurrencias y árboles de recurrencia	113
1.7.	El Teorema Maestro	122
1.8.	El modelo de costos en Python	131
1.9.	Análisis amortizado	138
	Notas del capítulo y referencias	144
Capítulo 2.	Dividir, conquistar y combinar	149
2.1.	Ideas clave	152
2.2.	Teselación de tableros de ajedrez	159
2.3.	Una metodología	164
2.4.	Ordenamiento de arreglos	169
2.5.	<i>MergeSort</i> : un ordenamiento de arreglos más eficiente	178
2.6.	<i>QuickSort</i> : ordenar sin combinar	186
2.7.	Búsqueda binaria	198
2.8.	Búsqueda binaria sobre predicados monótonos	206
2.9.	Bisección	213
2.10.	Aplicación: la planeación de una travesía	219
2.11.	Exponenciación rápida	226
	Notas del capítulo y referencias	231
Capítulo 3.	Programación dinámica	235
3.1.	Ideas clave	237
3.2.	Cálculo de los números de Fibonacci	243
3.3.	Una metodología	255
3.4.	Suma máxima de un subarreglo	261
3.5.	Subsecuencia creciente más larga	271
3.6.	El problema del morral	281
3.7.	Subsecuencia común más larga	297
3.8.	Caminos mínimos entre todos los vértices	309
3.9.	El problema del agente viajero	324
	Notas del capítulo y referencias	337
Capítulo 4.	Algoritmos voraces	341

4.1. Ideas clave	343
4.2. Agendamiento de actividades	353
4.3. Una metodología	367
4.4. Cubrimiento de intervalos	372
4.5. Tiempo en el sistema	387
4.6. El morral fraccionario	396
4.7. Árboles de cubrimiento mínimo	405
4.8. Caminos más cortos	439
Notas del capítulo y referencias	452
Capítulo 5. Reintento	455
5.1. Ideas clave	456
5.2. Generación de objetos combinatorios	465
5.3. Una metodología	472
5.4. El problema de las N reinas	476
5.5. Suma exacta de un subconjunto	487
5.6. Conteo de segmentaciones de texto	497
5.7. Ramificación y poda: el problema del morral	511
5.8. El agente viajero, revisitado	522
Notas del capítulo y referencias	532
Capítulo 6. Computabilidad	535
6.1. Problemas de decisión	536
6.2. Un marco universal basado en lenguajes	544
6.3. Aceptación, rechazo y decisión	550
6.4. Funciones computables	556
6.5. Clausura y el teorema de Post	566
6.6. Enumeración	574
6.7. ¿Cuántos problemas se pueden decidir?	580
Notas del capítulo y referencias	594
Capítulo 7. Indecidibilidad	597
7.1. Reducciones	598
7.2. El problema de la parada	605
7.3. Más problemas indecidibles	612
7.4. El teorema de Rice	617

7.5. Dominós: el problema de correspondencia de Post	624
7.6. Reescritura	630
Notas del capítulo y referencias	638
Capítulo 8. Intratabilidad: P vs. NP	641
8.1. La clase P	643
8.2. La clase NP	648
8.3. La clase coNP	656
8.4. Reducciones polinomiales	662
8.5. La clase NPC	668
8.6. CLIQUE es NP-completo	676
8.7. VCOVER es NP-completo	682
8.8. SUBSETSUM es NP-completo	686
8.9. TSP es NP-completo	695
8.10. ¿Qué hacer con un problema NP-completo?	700
Notas del capítulo y referencias	704
Bibliografía	709
Índice alfabético	717

Prólogo

Un algoritmo es un procedimiento descrito en una cantidad finita de pasos precisos que, aplicado mecánicamente, resuelve un problema y termina siempre con una respuesta. La idea es mucho más antigua que los computadores: hacia el año 300 a. C., Euclides describió un procedimiento para calcular el máximo común divisor de dos números que todavía se enseña y se usa, y que suele citarse como uno de los algoritmos más antiguos que se conocen. Lo que sí trajeron los computadores fue la escala. Hoy hay algoritmos detrás del trayecto que sugiere una aplicación de mapas, de la comparación de dos cadenas de ADN, de la compresión de una fotografía y del orden en que un buscador presenta sus resultados. Todos comparten el rasgo que hace interesante la noción: una descripción finita que da cuenta de infinitas situaciones posibles.

Saber qué es un algoritmo no dice cómo encontrar uno. Un problema no viene acompañado de su solución y, para muchos problemas interesantes, el primer procedimiento que suele ocurrírsele a cualquiera consiste en examinar todas las posibilidades hasta dar con la respuesta. Ese procedimiento suele ser correcto, pero la cantidad de posibilidades crece tan rápido con el tamaño de la entrada que, aun para instancias modestas, difícilmente un computador terminaría de examinarlas en un tiempo razonable. Diseñar un algoritmo es, en buena medida, encontrar la manera de evitar ese recorrido exhaustivo.

Las técnicas de diseño son las respuestas que la disciplina ha acumulado ante esa necesidad: formas de razonar que ya han funcionado en muchos problemas y que se pueden ensayar en uno nuevo. Quien las conoce no enfrenta cada problema desde cero, sino con una lista de preguntas concretas: si el problema se puede partir en partes independientes, si sus subproblemas se repiten, si una decisión local se puede tomar sin tener que reconsiderarla o si conviene explorar las alternativas para abandonar temprano las que no conducen a nada. Aprender estas técnicas es, en ese sentido, buena parte de lo que separa a quien conoce algunos algoritmos de quien sabe diseñarlos.

Un algoritmo se juzga, ante todo, por dos criterios. El primero es la corrección: un algoritmo resuelve un problema solo si produce una respuesta adecuada para cada una de sus instancias; no basta con que acierte en la mayoría. Como un problema suele tener infinitas instancias, ninguna colección de pruebas alcanza a examinarlas todas; las pruebas pueden delatar un error, pero en general no pueden certificar que no lo hay. La garantía de que un algoritmo es correcto tiene que venir, entonces, de un argumento matemático. No se trata de un escrúpulo académico. Los algoritmos toman decisiones en sistemas financieros, médicos y de transporte: un error que se manifiesta en una instancia entre millones puede tener consecuencias graves, precisamente porque nadie lo esperaba.

El segundo criterio es la eficiencia, es decir, la cantidad de tiempo y de memoria que un algoritmo requiere en función del tamaño de la entrada. Un algoritmo correcto que tarda siglos en responder tiene, para efectos prácticos, poca más utilidad que uno que no se ha escrito. Podría pensarse que el avance del hardware vuelve irrelevante esta preocupación, pero la experiencia indica lo contrario: un computador mil veces más rápido acelera mil veces cualquier algoritmo, mientras que reemplazar un algoritmo cuadrático por uno casi lineal puede representar, para entradas de millones de datos, una ganancia mucho mayor. Por eso la eficiencia se estudia como una propiedad del algoritmo y no de la máquina en que se ejecuta, con herramientas matemáticas que permiten comparar algoritmos antes de programarlos. A lo largo del texto, corrección y eficiencia van de la mano: un algoritmo eficiente e incorrecto no resuelve el problema; uno correcto e ineficiente rara vez sirve en la práctica.

La aparición de herramientas de inteligencia artificial generativa, capaces de producir código a partir de una descripción en lenguaje natural, podría sugerir que estudiar el diseño de algoritmos ha perdido sentido. Hay buenas razones para pensar lo contrario. Estas herramientas abaratan considerablemente escribir un programa, pero no abaratan en la misma medida saber si ese programa es correcto y eficiente. Un programa generado es, a fin de cuentas, una propuesta: puede funcionar en los ejemplos con los que se ensaya y fallar en una instancia que nadie consideró, o responder bien con entradas pequeñas y volverse inutilizable con entradas grandes. Quien lo va a usar tiene que poder juzgarlo y para eso necesita justamente lo que este texto enseña: especificar con precisión qué se espera, argumentar por qué el programa lo cumple y estimar cuánto le cuesta hacerlo.

Hay, además, preguntas que difícilmente una herramienta vuelve obsoletas. Formular bien un problema sigue siendo trabajo de quien lo plantea y una especificación ambigua tiende a producir, con cualquier herramienta, la solución de un problema distinto. Reconocer que un problema es, según todo lo que se sabe, intratable, o que no existe algoritmo alguno que lo resuelva, evita pedirle a cualquiera –humano o sistema artificial– algo que muy probablemente no puede entregar. Los sistemas de

inteligencia artificial son, ellos mismos, programas que se ejecutan en computadores y están sujetos a los mismos límites que estudia la última parte del libro. La inteligencia artificial generativa difícilmente reemplaza, entonces, el conocimiento de algoritmos; más bien lo vuelve más necesario, porque desplaza buena parte del trabajo de escribir código hacia el de evaluarlo.

¿Para quién es este libro?

Este libro está dirigido, en primer lugar, a estudiantes de ciencias de la computación, de ingeniería de sistemas, de matemáticas y de cualquier otra disciplina en la que los algoritmos sean una herramienta para resolver problemas. Está pensado también para profesores de cursos de algoritmos o de fundamentos de la computación, que encontrarán más adelante varios recorridos posibles para organizar un curso con él. Puede servir, finalmente, a quienes estudian por su cuenta o se preparan para competencias de programación y buscan, además de algoritmos, las razones por las cuales funcionan.

Conviene precisar, además, qué no es este libro. No es un texto para aprender a programar: supone que el lector ya escribe programas con cierta soltura y dirige su atención a qué hacen esos programas y por qué funcionan, más que a cómo se expresan en un lenguaje. Tampoco es un catálogo de algoritmos listos para usar. Aunque incluye muchos algoritmos conocidos, estos aparecen como ejemplos de una manera de razonar y no como piezas para memorizar; quien busque únicamente una implementación encontrará otras fuentes más directas. Finalmente, no es un texto sobre estructuras de datos: las que se usan se presentan en la medida en que algún algoritmo las necesita.

Lo que el texto supone del lector es relativamente poco. Del lado de la programación, experiencia con algún lenguaje imperativo, idealmente Python, que es el lenguaje de los ejemplos, y familiaridad con estructuras de datos básicas como listas, pilas, colas y grafos. Del lado de las matemáticas, lo que suele cubrir un primer curso de matemáticas discretas: lógica proposicional y de predicados, conjuntos, relaciones y funciones, y alguna práctica con demostraciones. Lo demás, es decir, la notación, las técnicas de demostración que se usan extensivamente y las herramientas de análisis, se desarrolla en el propio texto.

¿Por qué leer este libro?

Este no es un libro más de algoritmos. Su enfoque es formal y práctico. Un algoritmo se entiende como una afirmación sobre infinitas instancias que requiere demostración y el diseño se apoya de inicio a fin en un mismo principio, la inducción. Dividir, conquistar y combinar se presenta como la construcción de una demostración inductiva; la programación dinámica parte de definiciones recurrentes; y algoritmos clásicos como los de Kruskal, Prim y Dijkstra, que suelen presentarse

como procedimientos, se formulan aquí también mediante definiciones recurrentes. La misma vocación formal explica otras decisiones que rara vez se encuentran en un texto de algoritmos: la notación de cuantificadores de la escuela de Eindhoven, los invariantes como herramienta de diseño y no solo de verificación, el argumento basado en cardinales infinitos que muestra que existen problemas indecidibles y el uso de sistemas de reescritura para exhibir algunos de ellos.

Ese rigor no está reñido con la práctica, que es la otra mitad del propósito del libro. Los algoritmos se escriben en Python y se pueden ejecutar. Cada técnica viene acompañada de una metodología que conduce paso a paso de la especificación de un problema a un algoritmo y de patrones de demostración con nombre propio que se pueden ensayar en problemas nuevos. Además, varios problemas reaparecen a lo largo del texto, abordados con técnicas distintas. El problema del morral, por ejemplo, se resuelve con programación dinámica, con un algoritmo voraz en su versión fraccionaria, y con ramificación y poda; el del agente viajero regresa en el capítulo de intratabilidad para explicar por qué no se conoce para él una solución eficiente. Comparar las técnicas sobre un mismo problema es, posiblemente, la mejor manera de entender qué aporta cada una y cuándo conviene usarla.

El texto da, también, un tratamiento coherente a las dos formas de expresar un algoritmo: la recursión y los ciclos. Cada una tiene sus propias herramientas y el libro las desarrolla en paralelo. La corrección de un algoritmo recursivo se demuestra por inducción y su costo se obtiene resolviendo una recurrencia, con árboles de recurrencia o con el Teorema Maestro. La corrección de un ciclo se demuestra con un invariante y con una cantidad que disminuye en cada iteración sin poder disminuir indefinidamente, y su costo se obtiene con sumas. Ambos tratamientos provienen del mismo principio de inducción sobre relaciones bien fundadas, de modo que pasar de una forma a la otra, como ocurre al convertir la memorización en tabulación en la programación dinámica, no exige cambiar de manera de razonar.

Cada capítulo cierra, además, con notas y referencias que van más allá de una bibliografía. En ellas se cuenta de dónde provienen las ideas y los algoritmos del capítulo, se sitúan las decisiones del texto frente a las de otros autores y se indica dónde profundizar, tanto en la literatura clásica del área como en fuentes de problemas de la programación competitiva. Junto con las notas históricas que acompañan la introducción de varios capítulos, permiten ver los algoritmos no como resultados aislados, sino como parte de una historia de ideas que todavía está en curso.

Finalmente, el libro está escrito originalmente en castellano. Difícilmente se encuentra en este idioma un texto de algoritmos que combine un tratamiento formal con una orientación práctica como la que se propone aquí, y es frecuente que quienes estudian el área en países hispanohablantes recurran a traducciones o a textos en inglés.

¿Qué contiene este libro y cómo está organizado?

El texto se organiza en tres partes. La primera, formada por los Capítulos 0 y 1, reúne las herramientas con las que se especifican los problemas, se demuestra la corrección de los algoritmos y se mide su eficiencia. La segunda, formada por los Capítulos 2 a 5, estudia cuatro técnicas de diseño. La tercera, formada por los Capítulos 6 a 8, se ocupa de los límites: de lo que ningún algoritmo puede hacer y de lo que, hasta donde se sabe, ningún algoritmo eficiente puede hacer.

El Capítulo 0 fija el lenguaje del resto del texto. Precisa qué es un problema algorítmico, y cómo se especifica en términos de sus entradas y de las salidas esperadas, clasifica los problemas según la clase de respuesta que piden y define qué significa que un algoritmo resuelva un problema. Presenta también las herramientas con las que se demuestra la corrección: la notación de cuantificadores con la que se escriben las especificaciones, los invariantes de ciclo y un principio de inducción basado en relaciones bien fundadas, del cual la inducción sobre los números naturales y los argumentos de terminación resultan casos particulares.

El Capítulo 1 se ocupa de la eficiencia. Parte de mediciones concretas sobre un mismo problema para motivar la notación asintótica, que permite comparar algoritmos sin atarse a una máquina ni a un lenguaje, y desarrolla después las técnicas para obtener el costo de un algoritmo: sumas de referencia para los iterativos, y recurrencias, árboles de recurrencia y el Teorema Maestro para los recursivos. Cierra con el modelo de costos de Python, el lenguaje de los programas del texto, y con una introducción al análisis amortizado. Estas herramientas no se quedan en la primera parte: la mayoría de los algoritmos de los capítulos siguientes se presenta con su especificación, una demostración de su corrección y un análisis de su costo.

La segunda parte es el núcleo del texto. Sus cuatro capítulos estudian las técnicas de diseño que responden, cada una a su manera, a las preguntas planteadas al comienzo de este prólogo, y comparten una misma organización: presentan las ideas clave de la técnica, las ilustran con un problema escogido para mostrarla en su forma más limpia, las recogen en una metodología que conduce de la especificación de un problema a un algoritmo y la aplican después a una serie de ejemplos. Las cuatro técnicas reducen una instancia de un problema a instancias más pequeñas del mismo problema y se distinguen por lo que le agregan a esa reducción.

Dividir, conquistar y combinar (Capítulo 2) parte una instancia en subinstancias independientes, las resuelve por separado y construye con sus soluciones la solución de la instancia original. Es la más elemental de las cuatro técnicas y, en cierto sentido, la base de las demás. El capítulo la desarrolla sobre el ordenamiento y la búsqueda en arreglos, generaliza la búsqueda binaria hasta la bisección sobre predicados monótonos y termina con la exponenciación rápida. Fuera del texto, la técnica está detrás de la multiplicación rápida de números grandes, de la transformada rápida de Fourier, que sostiene buena parte del procesamiento de audio,

imágenes y señales, y de muchos algoritmos paralelos y distribuidos, pues la independencia de las subinstancias permite resolverlas al mismo tiempo en procesadores o máquinas distintas.

La programación dinámica (Capítulo 3) se aplica cuando las subinstancias no son independientes, sino que se repiten. En lugar de resolver muchas veces una misma subinstancia, la resuelve una sola vez y guarda su solución en una memoria auxiliar: parte de una definición recurrente de la función de interés y produce un algoritmo que la calcula, a cambio de espacio. El capítulo la aplica a problemas sobre secuencias, de optimización combinatoria y de caminos mínimos en grafos. Fuera del texto, sus aplicaciones son muy amplias: la comparación de secuencias de ADN y de proteínas en bioinformática, las herramientas que muestran las diferencias entre dos versiones de un archivo, el enrutamiento en redes, y la planeación y asignación de recursos en investigación de operaciones, entre otras.

Los algoritmos voraces (Capítulo 4) llevan la reducción al extremo: en cada paso toman una única decisión, dictada por un criterio de selección local, y no la reconsideran. Cuando funcionan producen algoritmos sencillos y eficientes, pero funcionan con poca frecuencia, de modo que el peso del capítulo recae en demostrar que el criterio escogido conduce a una solución óptima. Los ejemplos incluyen el agendamiento de actividades y tareas, el morral fraccionario, los árboles de cubrimiento mínimo y los caminos más cortos desde un vértice. En ideas de este tipo se apoyan el diseño de redes de comunicación y de distribución de costo mínimo, algunas políticas de planificación de tareas en sistemas operativos, la compresión de datos y el cálculo de rutas en aplicaciones de mapas.

El reintento (Capítulo 5) es la alternativa para los problemas en los que ninguna de las técnicas anteriores está disponible: construye las soluciones una decisión a la vez y abandona una construcción parcial apenas sabe que no puede completarse. En el peor caso examina una cantidad exponencial de posibilidades, pero una poda bien diseñada marca con frecuencia la diferencia entre un programa que no termina en un tiempo razonable y uno que responde en milisegundos. El capítulo lo aplica a la generación de objetos combinatorios, al problema de las N reinas, a la suma exacta de un subconjunto y al conteo de segmentaciones de texto, y lo extiende a problemas de optimización mediante ramificación y poda. En buena medida sobre estas ideas descansan los resolvedores de restricciones y de satisfacibilidad lógica que se usan en planeación, en verificación de hardware y software, y en la construcción de horarios.

La tercera parte cambia de pregunta. Las dos primeras suponen que un problema tiene solución algorítmica y se ocupan de encontrarla y de analizarla; esta se pregunta cuándo esa suposición se justifica. El Capítulo 6 precisa qué significa computar. Restringe el estudio a los problemas de decisión, muestra que esa restricción no es una pérdida de generalidad, los representa como lenguajes formales y fija

a Python como modelo de cómputo, con el respaldo de la tesis de Church y Turing. Con ese marco caracteriza los problemas que sí se pueden resolver y muestra, con un argumento de cardinalidad, que son muy pocos frente a los que no.

El Capítulo 7 exhibe problemas concretos que ningún algoritmo resuelve. Parte del problema de la parada, cuya indecidibilidad se demuestra con un argumento diagonal, y transporta ese resultado a otros problemas mediante reducciones, entre ellos el problema de correspondencia de Post, presentado como un juego de dominós, y problemas sobre sistemas de reescritura. Demuestra también el teorema de Rice, según el cual ninguna propiedad semántica no trivial de los programas es decidable. Para quien programa, la consecuencia es concreta: el analizador estático perfecto, el verificador infalible de equivalencia entre programas o el detector de ciclos infinitos que nunca se equivoca no se pueden construir.

El Capítulo 8 se ocupa de lo que sí se puede computar, pero posiblemente no en un tiempo razonable. Define las clases P , NP y $coNP$, adapta las reducciones para que tengan en cuenta su costo y, con ellas, identifica los problemas NP -completos, que son, en un sentido preciso, los más difíciles de NP . Demuestra que cuatro problemas concretos pertenecen a esa clase, entre ellos el del agente viajero, que aparece también en los capítulos de programación dinámica y de reintento. Cierra con lo que se puede hacer cuando la respuesta exacta y eficiente está fuera de alcance. A diferencia de los dos capítulos anteriores, cuyos resultados son definitivos, aquí los resultados son en su mayoría condicionales: dependen de la pregunta de si P es igual a NP , que nadie ha logrado responder hasta ahora.

¿Cómo leer este libro?

El texto completo cubre el material de dos cursos de un semestre, pero admite también otras lecturas, según el propósito de quien lo tome. Para escoger entre ellas conviene saber cómo dependen las partes entre sí. La primera es requisito de las otras dos, pues sin su notación, sus técnicas de demostración y su análisis de costos el resto del texto difícilmente se puede seguir. La segunda y la tercera, en cambio, dependen poco una de la otra: la tercera toma algunos problemas de la segunda como ejemplos, pero sus argumentos no requieren las técnicas de diseño. Dentro de cada parte conviene leer los capítulos en orden, porque cada técnica de diseño se presenta en contraste con las anteriores y cada capítulo de la tercera parte se apoya en el que lo precede.

De la primera parte, el Capítulo 0 puede leerse de corrido, pero también funciona como referencia: quien ya tenga práctica con especificaciones y demostraciones puede recorrerlo con rapidez, y volver a él cuando una notación o una técnica de demostración aparezca más adelante. El Capítulo 1, en cambio, conviene leerlo con cuidado, porque sus herramientas se usan en el análisis de casi todos los algoritmos del texto.

Un curso de diseño y análisis de algoritmos puede apoyarse en los Capítulos 0 a 5. Es el recorrido que mejor aprovecha el hilo conductor del texto: las cuatro técnicas se presentan en contraste unas con otras y varios problemas se retoman de un capítulo a otro. Si el tiempo es escaso, en cada capítulo de la segunda parte las tres primeras secciones, es decir, las ideas clave, el problema que las ilustra y la metodología constituyen el núcleo, y los ejemplos que siguen pueden escogerse según los intereses del curso.

Un curso de fundamentos de la computación, o la parte de un curso dedicada a la computabilidad y la complejidad, puede apoyarse en el Capítulo 0, en las Secciones 1.1 a 1.3 del Capítulo 1, que presentan la notación asintótica, y en los Capítulos 6 a 8. Las pocas referencias de estos a la segunda parte se pueden seguir, en general, sin haberla leído.

El texto puede servir también de apoyo a cursos más específicos, siempre sobre la base de la primera parte. Un curso sobre árboles y grafos encuentra en la segunda parte varias secciones dedicadas a algoritmos sobre grafos: los caminos mínimos entre todos los vértices y el agente viajero en el Capítulo 3, los árboles de cubrimiento mínimo y los caminos más cortos en el Capítulo 4, y el agente viajero, de nuevo, en el Capítulo 5. Un curso sobre algoritmos para cadenas puede tomar como punto de partida la subsecuencia común más larga y la subsecuencia creciente más larga del Capítulo 3, y la segmentación de texto del Capítulo 5. Un curso de estructuras de datos, por su parte, puede apoyarse en la inducción estructural del Capítulo 0 y en la manera de razonar del Capítulo 2 para definir estructuras recurrentemente, verificarlas y estimar su eficiencia. Para ese curso resulta central el análisis amortizado (Sección 1.9), que en otros recorridos es opcional, y son útiles los conjuntos disyuntos y las colas de prioridad que aparecen en el Capítulo 4.

Quien lea por cuenta propia puede seguir el orden de los capítulos. En una primera lectura puede omitir, sin perder la continuidad, las siguientes secciones, que no se usan más adelante:

- el modelo de costos en Python (Sección 1.8) y el análisis amortizado (Sección 1.9), que cierran el Capítulo 1;
- la aplicación a la planeación de una travesía (Sección 2.10);
- la clausura y el teorema de Post (Sección 6.5) y la enumeración (Sección 6.6);
- la reescritura (Sección 7.6).

En cambio, conviene no omitir los ejercicios al final de cada sección, pues varios completan demostraciones o desarrollan ideas que el texto solo enuncia.

¿Cómo, cuándo y dónde se escribió este libro?

Este texto comenzó como un conjunto de notas de clase. En enero de 2016, al vincularse a la Pontificia Universidad Javeriana Cali, el autor asumió el curso de Análisis y Diseño de Algoritmos y no encontró un texto que se ajustara a lo que buscaba. Empezó entonces a escribir notas a mano y en una tableta, una por cada sesión de clase. Ese curso fue el recorrido para el cual se concibió originalmente el material; los demás usos descritos en la sección anterior llegaron después.

Al cabo de varios semestres, esas notas formaban un compendio, parte del cual ya estaba transcrito a $\text{\LaTeX}$. En 2019 el autor lo reunió en una primera versión con cuatro capítulos: preliminares, notación O , dividir y conquistar, y programación dinámica. A partir de entonces el texto creció a razón de un par de secciones por semestre, con la incorporación de los algoritmos voraces y de los capítulos sobre decidibilidad e intratabilidad. El reintento siguió otro camino: problemas como el de las N reinas, el sudoku o la segmentación de texto fueron habituales en el curso durante años, pero sus notas solo pasaron a $\text{\LaTeX}$ en 2026, cuando el texto alcanzó su forma actual, al cumplirse diez años de dictar el curso. Casi todo el material se escribió durante la vinculación del autor a la Javeriana. La excepción son las Secciones 0.8 y 0.9, sobre relaciones bien fundadas e inducción, que se basan en un escrito corto que el autor preparó hacia 2014 para el curso Teoría de la Programación de la Escuela Colombiana de Ingeniería. Por lo demás, el material se apoya en la literatura clásica del área, que se reconoce en las notas y referencias de cada capítulo.

Las notas no se quedaron en un solo curso. En 2018 el autor diseñó el curso de Árboles y Grafos como antesala de Análisis y Diseño de Algoritmos. En él se estudian estructuras y algoritmos sobre árboles y grafos con el mismo enfoque de este texto: la inducción matemática como principio del diseño algorítmico. Ese curso toma como referencia los capítulos de análisis asintótico y de dividir y conquistar. A su vez, el curso de Estructuras de Datos, que lo precede, ha usado estas notas como referencia para la notación O .

Detrás del texto hay una convicción doble. Como científico de la computación, al autor le importa que los programas sean correctos y eficientes, y que eso se pueda demostrar. Como entrenador de programación competitiva y gestor de maratones de programación durante más de veinte años, sabe también que una técnica vale en la medida en que sirve para resolver problemas. Ha sido un miembro activo de la Maratón Nacional de Programación colombiana, en la que ha propuesto y resuelto problemas, y ha formado parte de delegaciones colombianas en las finales mundiales del ICPC en Tailandia (2016) y en Estados Unidos (2017). Esa experiencia se refleja en la selección de los problemas del texto y en las referencias a fuentes de la programación competitiva que aparecen en las notas de los capítulos.

En la etapa final, la inteligencia artificial generativa sirvió como apoyo para pulir y terminar el texto. Se usó principalmente en la corrección y en la elaboración de figuras. También sirvió para mejorar la redacción, para reflexionar sobre cómo hilar las secciones nuevas con las existentes, para reorganizar ejercicios y para detectar dependencias entre secciones. Su aporte más significativo fue la construcción automática de gráficas, y de simulaciones y de casos de prueba con los que se alimentaron las tablas que comparan algoritmos. Este trabajo fue más intenso en las últimas secciones incorporadas, cuyo material estaba todavía en notas manuscritas o fuera del formato del texto. La experiencia confirmó lo que se argumenta al comienzo de este prólogo: la herramienta aceleró la producción, pero juzgar la corrección de cada resultado siguió siendo trabajo del autor. Todo el contenido fue revisado por él en su totalidad, hasta el último punto y la última coma, y la responsabilidad por ese contenido, incluidos los errores que aún pueda tener, es exclusivamente suya.

Agradecimientos

El autor agradece, en primer lugar, a los estudiantes que han pasado por el curso de Análisis y Diseño de Algoritmos desde el primer semestre de 2016, que según sus cuentas son más de seiscientos: sus preguntas y sus dudas dieron forma a buena parte de estas páginas. Agradece también a Miguel Romero, asistente de docencia del curso entre el segundo semestre de 2018 y el segundo semestre de 2019, mientras iniciaba sus estudios doctorales en Ingeniería y Ciencias Aplicadas, y a Kevin Díaz, asistente de docencia en el primer semestre de 2020, mientras avanzaba en sus estudios de maestría en Ingeniería y Ciencias.

Una mención especial merece Carlos Ramírez, profesor del Departamento de Electrónica y Ciencias de la Computación de la Javeriana Cali, con quien el autor ha dictado el curso durante los últimos seis años y quien, desde el segundo semestre de 2020, ha contribuido a mejorar estas notas con sus sugerencias, y con la detección de errores y gazapos. En este tiempo ha sido mucho más que un compañero de curso: ha sido un amigo. Juntos han discutido ideas, preparado problemas de programación competitiva, diseñando e implementando algoritmos, y generando casos de prueba para que los estudiantes practiquen. Al mismo tiempo han colaborado en proyectos de investigación que van desde la especificación de semánticas de lenguajes de programación en lógica de reescritura hasta el diseño de modelos para entender la polarización en redes sociales basados en algoritmos y aprendizaje mecánico en grafos.

Camilo Rocha
Octubre 2026. Cali, Colombia.

Preliminares

El propósito de un algoritmo es resolver mecánicamente un problema –o, equivalentemente, todas las instancias de un problema. Contar con una noción de algoritmo para este propósito requiere antes precisar qué se entiende por y cómo se define un problema algorítmico. La finalidad de este capítulo es presentar un marco general para la definición de problemas algorítmicos, e introducir convenciones para su posterior diseño y análisis. Hay una segunda finalidad, menos evidente pero igual de importante. Un algoritmo no es una propuesta que se acepta porque funcionó en unos cuantos ejemplos: es una afirmación sobre infinitas instancias y, como tal, requiere demostración. Este capítulo también presenta las herramientas matemáticas con las cuales se demuestra que un algoritmo hace lo que se espera de él, y que se usan una y otra vez en el resto del texto.

Las primeras dos secciones fijan la notación. La Sección 0.1 introduce los arreglos, que son la estructura de datos sobre la cual se plantean la mayoría de los problemas del texto, y la Sección 0.2 presenta la notación de cuantificadores con la que se escriben sus propiedades.

Las siguientes cuatro secciones se ocupan de los problemas y de los algoritmos que los resuelven. La Sección 0.3 precisa qué es un problema algorítmico y cómo se especifica, la Sección 0.4 clasifica los problemas según la clase de respuesta que piden, la Sección 0.5 define qué es un algoritmo y qué significa que resuelva un problema, y la Sección 0.6 describe los lenguajes con los cuales se especifican los algoritmos en este texto, junto con las convenciones de código que se siguen.

Las últimas tres secciones se ocupan de la corrección. La Sección 0.7 distingue la corrección parcial de la total, explica por qué probar un algoritmo con algunas entradas no basta e introduce los invariantes de ciclo. La Sección 0.8 presenta las relaciones bien fundadas y el principio de inducción que se obtiene de ellas, y la Sección 0.9 lo especializa a los números naturales, justifica el método de invariantes y demuestra la corrección del primer algoritmo del texto.

Tres de las decisiones de este capítulo se apartan del tratamiento habitual en los textos de algoritmos y conviene anunciarlas desde el comienzo.

La primera es la notación de cuantificadores de la Sección 0.2. La mayoría de los textos escribe las sumatorias, los mínimos y las cuantificaciones lógicas cada uno con su propia convención, heredada de la matemática clásica. La notación que se usa aquí, desarrollada en los años setenta por el grupo de trabajo de Edsger Dijkstra en Eindhoven y difundida después por David Gries, Anne Kaldewaij y Roland Backhouse, hace explícitos la variable, el rango y el término, y trata todos los cuantificadores con las mismas reglas. El precio es la extrañeza inicial; la ganancia es poder manipular las expresiones con reglas uniformes y traducirlas directamente a ciclos.

La segunda es el papel de los invariantes. Es común presentarlos como una herramienta de verificación: primero se escribe el algoritmo y después se busca un invariante que permita demostrar que funciona. En la tradición de la que proviene este texto, el invariante es ante todo una herramienta de diseño: quien sabe qué propiedad debe mantener un ciclo sabe también cómo escribirlo, y el algoritmo se deriva junto con su demostración en lugar de justificarse a posteriori.

La tercera es el tratamiento de la inducción. Lo usual es postular el principio de inducción sobre los números naturales y usarlo como punto de partida. Aquí se parte de una noción más abstracta, la de relación bien fundada, y de ella se obtienen como casos particulares la inducción sobre los naturales, la inducción estructural y los argumentos de terminación. El camino es un poco más largo, pero a cambio un solo principio cubre todos los usos que el texto hace de la inducción.

Las notas al final del capítulo amplían estas tres decisiones e indican dónde encontrar tratamientos más extensos de cada una.

Este capítulo puede leerse de corrido antes de continuar, pero también funciona como referencia: es natural volver a él cuando una notación, una convención o una técnica de demostración aparezca en los capítulos siguientes.

0.1. Arreglos: notación y convenciones

La gran mayoría de los problemas algorítmicos que se estudian en este texto se definen y se estudian con base en arreglos. Un arreglo puede contener números, caracteres o valores Booleanos, entre otros. Los arreglos también se pueden componer entre sí para formar matrices, hipermatrices como cubos, etc. Esta sección fija la notación con la que se escriben los arreglos a lo largo del texto, junto con las operaciones y propiedades básicas que se usan con más frecuencia.

Definición 0.1.1

Un *arreglo* de *tamaño* N es una estructura de datos que almacena una colección de elementos (o valores), cada uno identificado por un *índice* correspondiente a un número natural en el rango $0..N - 1$.

No es de interés en este texto estudiar los detalles de implementación de los arreglos, ni en los lenguajes de programación en general ni en uno en particular. Más bien, se supondrá que el acceso a cualquier elemento de un arreglo, dado su índice, toma tiempo constante, como sucede en la práctica de la programación. Este supuesto hace parte del modelo de costo que se precisa en el Capítulo 1.

0.1.1. Notación básica. Se usa una notación similar a la de los intervalos de números reales, con corchetes $[]$ y paréntesis $()$, para especificar arreglos y para definir operaciones sobre ellos. El corchete indica que el extremo se incluye y el paréntesis que se excluye.

Nota 0.1.1

La expresión $A[0..N)$ indica que A es un arreglo de N elementos con índices en el rango $0..N - 1$. El *tamaño* (o, equivalentemente, la cantidad de índices) de un arreglo A se denota $|A|$; así, si $A[0..N)$, entonces $|A| = N$. Un arreglo de tamaño 0, es decir, sin índices, se llama *vacío* y se denota $[]$.

Un arreglo concreto se escribe enumerando sus elementos entre corchetes, en orden de índice. Por ejemplo, $A = [1, 0, 25, -1, 8]$ es un arreglo de tamaño 5. La operación más básica sobre un arreglo es consultar el elemento que ocupa una posición dada.

Definición 0.1.2

Sean $A[0..N)$ y n un número natural. Si $n < N$, la *indexación* $A[n]$ denota el elemento de A con índice n . Si $n \geq N$, la expresión $A[n]$ es un error.

Por ejemplo, con $A = [1, 0, 25, -1, 8]$ se tiene $A[3] = -1$, mientras que $A[5]$ es un error. En particular, cualquier indexación en el arreglo vacío es un error.

Con frecuencia es necesario referirse a una porción contigua de un arreglo, por ejemplo a su primera mitad o a los elementos que un algoritmo aún no ha procesado. La siguiente definición precisa esta idea.

Definición 0.1.3

Sean $A[0..N]$ y a, b números naturales. Si $a \leq b \leq N$, la *sección* (o el *subarreglo*) $A[a..b]$ es el arreglo B de tamaño $b - a + 1$ tal que

$$B[i] = A[a + i], \quad 0 \leq i < b - a + 1.$$

Si no se cumple $a \leq b \leq N$, la expresión $A[a..b]$ es un error.

En palabras, la sección $A[a..b]$ contiene, en el mismo orden, los elementos de A con índices desde a hasta $b - 1$ (el índice b no se incluye), re-indexados a partir de 0. Por ejemplo, con $A = [1, 0, 25, -1, 8]$ se tiene $A[1..3] = [0, 25]$ y $A[2..2] = []$, mientras que $A[3..7]$ y $A[4..2]$ son errores.

La Definición 0.1.3 trata una sección fuera de rango del mismo modo que la Definición 0.1.2 trata un índice fuera de rango: como un error. Una consecuencia es que $A[a..b]$ es vacía exactamente cuando $a = b$.

La notación de intervalo semiabierto tiene una variante cerrada, que se usa cuando resulta más natural nombrar el último índice que el primero excluido. Abusando un poco de la notación, las expresiones $A[0..N)$ y $A[0..N - 1]$ denotan el mismo arreglo y, en general, $A[a..b]$ denota la sección $A[a..b + 1)$.

0.1.2. Propiedades de las secciones. Varios razonamientos a lo largo del texto consisten en partir un arreglo en dos secciones consecutivas y razonar sobre cada una por separado. Así se procede, por ejemplo, en el ordenamiento por mezcla, en la búsqueda binaria y en los invariantes de ciclo de la Sección 0.7. Para enunciar estos hechos con precisión se requieren dos nociones adicionales: la igualdad de arreglos y su concatenación.

Definición 0.1.4

Dos arreglos A y B son *iguales*, y se escribe $A = B$, si y solo si $|A| = |B|$ y $A[i] = B[i]$ para todo índice $0 \leq i < |A|$.

La concatenación de dos arreglos es el arreglo que resulta de escribir los elementos del primero seguidos de los elementos del segundo.

Definición 0.1.5

Sean $B[0..M)$ y $C[0..K)$. La *concatenación* $B \mathbin{++} C$ es el arreglo D de tamaño $M + K$ tal que

$$D[i] = \begin{cases} B[i], & \text{si } 0 \leq i < M, \\ C[i - M], & \text{si } M \leq i < M + K. \end{cases}$$

Por ejemplo, $[1, 0] ++ [25, -1, 8] = [1, 0, 25, -1, 8]$ y $[] ++ [7] = [7]$. Con estas dos nociones se pueden enunciar las propiedades de las secciones que se usan más adelante.

Teorema 0.1.1

Sean $A[0..N]$ y a, b, c números naturales tales que $a \leq c \leq b \leq N$. Entonces:

1. $|A[a..b]| = b - a$.
2. $A[0..N] = A$ y $A[a..a] = []$.
3. Si $a < N$, entonces $A[a..a + 1] = [A[a]]$.
4. $A[a..b] = A[a..c] ++ A[c..b]$.

Las tres primeras propiedades son consecuencia directa de la Definición 0.1.3; la cuarta requiere un argumento corto, que se presenta a continuación.

Demostración

Las propiedades (1)–(3) se siguen de la Definición 0.1.3 con $b - a$, $N - 0$, $a - a$ y $(a + 1) - a$ como tamaños, respectivamente.

Para (4), sean $L = A[a..b]$ y $R = A[a..c] ++ A[c..b]$. Por (1) y la Definición 0.1.5, $|R| = (c - a) + (b - c) = b - a = |L|$. Sea $0 \leq i < b - a$. Si $i < c - a$, entonces $R[i] = A[a..c][i] = A[a + i] = L[i]$. Si $c - a \leq i < b - a$, entonces

$$R[i] = A[c..b][i - (c - a)] = A[c + i - (c - a)] = A[a + i] = L[i].$$

Por la Definición 0.1.4, $L = R$.

La Figura 1 ilustra la cuarta propiedad del Teorema 0.1.1: el punto c parte la sección $A[a..b]$ en dos secciones consecutivas, sin que ningún elemento quede por fuera ni aparezca en ambas.

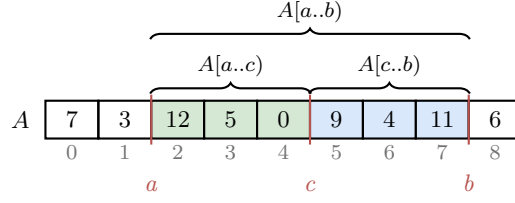


Figura 1. Un arreglo $A[0..9]$ y la sección $A[a..b]$ con $a = 2$ y $b = 8$, partida en $c = 5$ como $A[a..c] ++ A[c..b]$. Los índices aparecen debajo de cada celda; las marcas a , c y b se ubican en las fronteras entre celdas.

0.1.3. Matrices y cadenas. La notación para arreglos se extiende de manera natural a más dimensiones. Una matriz A de tamaño $N \times M$ se especifica como $A[0..N][0..M]$: es un arreglo de N filas, cada una de las cuales es un arreglo de M elementos. La expresión $A[i][j]$ denota el elemento en la fila i y la columna j , siempre y cuando $0 \leq i < N$ y $0 \leq j < M$. Del mismo modo se especifican arreglos de tres o más dimensiones.

Una cadena de tamaño N se representa como un arreglo de N caracteres. Todo lo dicho en esta sección sobre indexación, secciones y concatenación aplica por igual a las cadenas.

0.1.4. Los arreglos como funciones. Los arreglos también se pueden concebir como objetos matemáticos. De esta forma es posible referirse a ellos de manera abstracta y formal, sin atarse a su implementación en un lenguaje de programación particular, y enfocar el diseño y el análisis en las ideas.

Nota 0.1.2

Matemáticamente, un arreglo $A[0..N)$ cuyos elementos son de un tipo T es una función

$$A : \{0, 1, \dots, N - 1\} \rightarrow T.$$

El dominio de A es el conjunto de sus índices y el codominio es el tipo T de sus elementos. El conjunto $\{A[0], A[1], \dots, A[N - 1]\}$ de valores que efectivamente aparecen en A es la *imagen* de la función. El arreglo vacío es la única función con dominio vacío.

Por ejemplo, el arreglo $A = [1, 0, 25, -1, 8]$ corresponde a la función

$$A : \{0, 1, 2, 3, 4\} \rightarrow \mathbb{Z},$$

cuya imagen es el conjunto $\{-1, 0, 1, 8, 25\}$. De igual manera, un arreglo de N valores Booleanos es una función $A : \{0, \dots, N - 1\} \rightarrow \mathbb{B}$.

Bajo esta lectura, la igualdad de la Definición 0.1.4 es la igualdad usual de funciones: mismo dominio y mismo valor en cada punto. El tipo T de los elementos no siempre se menciona explícitamente; en esos casos se entenderá del contexto o se indicará cuando sea necesario.

0.1.5. Permutaciones. Muchos problemas, empezando por el de ordenamiento, exigen que la salida contenga exactamente los mismos elementos que la entrada, tal vez en otro orden. La vista funcional de los arreglos permite expresar esta condición con precisión: basta con reordenar los índices.

Definición 0.1.6

Sean $A[0..N)$ y $B[0..N)$. Se dice que B es una *permutación* de A si y solo si existe una función biyectiva $\sigma : \{0, \dots, N-1\} \rightarrow \{0, \dots, N-1\}$ tal que $B[i] = A[\sigma(i)]$ para todo $0 \leq i < N$; es decir, si $B = A \circ \sigma$.

La función σ indica, para cada posición de B , de qué posición de A proviene su elemento. Que σ sea biyectiva garantiza que cada elemento de A se usa exactamente una vez. Note que la definición exige que ambos arreglos tengan el mismo tamaño.

Ejemplo 0.1.1

Sean $A = [1, 0, 25, -1, 8]$ y $B = [-1, 0, 1, 8, 25]$. El arreglo B es una permutación de A , como lo atestigua la función σ dada por

$$\sigma(0) = 3, \quad \sigma(1) = 1, \quad \sigma(2) = 0, \quad \sigma(3) = 4, \quad \sigma(4) = 2.$$

Por ejemplo, $B[0] = A[\sigma(0)] = A[3] = -1$. En cambio, $[-1, 0, 1, 8, 8]$ no es una permutación de A : el valor 25 no aparece y el valor 8 aparece dos veces.

Cuando un arreglo tiene valores repetidos, la función σ que atestigua una permutación no es única. Por ejemplo, $[5, 5]$ es permutación de sí mismo mediante la identidad y también mediante la función que intercambia los índices 0 y 1.

0.1.6. Arreglos en Python. El lenguaje de programación de este texto es Python, cuyas listas cumplen el papel de arreglos. La notación de esta sección fue escogida de modo que su correspondencia con Python sea directa.

Nota 0.1.3

Para una lista A de Python:

- $|A|$ corresponde a `len(A)`;

- la indexación $A[n]$ corresponde a $A[n]$, que produce un error (`IndexError`) si $n \geq |A|$;
- la sección $A[a..b]$ corresponde a $A[a:b]$;
- la concatenación $B ++ C$ corresponde a $B + C$.

Hay dos diferencias que conviene tener presentes. Primero, Python recorta silenciosamente las secciones fuera de rango en lugar de producir un error: con $A = [1, 0, 25, -1, 8]$, la expresión $A[3:7]$ vale $[-1, 8]$ y $A[4:2]$ vale $[]$. Segundo, Python admite índices negativos, que cuentan desde el final: $A[-1]$ vale 8 . Ninguno de estos dos comportamientos tiene contraparte en la notación de esta sección.

Una tercera diferencia concierne al costo y no al significado: en Python, $A[a:b]$ construye una lista nueva, lo cual toma tiempo proporcional a $b - a$. En la notación matemática, en cambio, una sección es solo una forma de nombrar una porción de un arreglo. Las convenciones de código del texto se detallan en la Sección 0.6 y el costo de las operaciones de Python, en el Capítulo 1.

Ejercicios

1. Considere dos arreglos $A[0..M)$ y $B[0..N)$.
 - a) Describa e implemente un algoritmo que permita determinar si los dos arreglos son iguales en el sentido de la Definición 0.1.4.
 - b) Describa e implemente un algoritmo que permita determinar si B es una sección de A , es decir, si existen a y b tales que $A[a..b) = B$.
2. Demuestre que la concatenación es asociativa y que el arreglo vacío es su elemento neutro, es decir, que para arreglos B , C y D cualesquiera se cumplen $(B ++ C) ++ D = B ++ (C ++ D)$ y $[] ++ B = B = B ++ []$.
3. Sean $A[0..N)$ y $a \leq b \leq N$. Demuestre que si $i \leq j \leq b - a$, entonces

$$A[a..b)[i..j) = A[a + i..a + j).$$

Es decir, una sección de una sección es una sección del arreglo original.

4. Demuestre que la relación “ B es una permutación de A ” es una relación de equivalencia, es decir, que es reflexiva, simétrica y transitiva. Sugerencia: use la función identidad, la función inversa y la composición de funciones.
5. Demuestre que si B es una permutación de A , entonces cada valor aparece en B la misma cantidad de veces que en A . ¿Es cierto el recíproco? Justifique su respuesta.
6. Considere la lista $A = [1, 0, 25, -1, 8]$ de Python.

- a) Evalúe $A[3:10]$, $A[4:2]$, $A[-2:]$ y $A[1:-1]$. Para cada una, indique si la expresión correspondiente en la notación de esta sección es un error o qué arreglo denota.
 - b) Implemente una función `section(A, a, b)` que retorne $A[a:b]$ cuando $0 \leq a \leq b \leq |A|$ y que, en cualquier otro caso, produzca un error, de acuerdo con la Definición 0.1.3.
7. Investigue sobre árboles binarios casi llenos (e.g., aquellos que se usan en la implementación de *HeapSort* como implementación de colas de prioridad). Explique brevemente cómo representar un árbol binario casi lleno (e.g., de números) en un arreglo. ¿Qué relación hay entre los índices del arreglo y la profundidad de los elementos del árbol?
 8. En el lenguaje de programación Python, los arreglos son en realidad listas indexadas. Investigue cuál es la principal diferencia entre los arreglos de Python y los arreglos de C/C++/Java. ¿Qué ventajas tienen los arreglos de Python sobre los arreglos en estos otros lenguajes? ¿Tienen alguna desventaja?
 9. Cuando están disponibles en un lenguaje de programación imperativo, los arreglos se pueden pasar como parámetros de funciones o procedimientos. Hay diferentes formas de pasar un arreglo como parámetro: por ejemplo, por valor o por referencia. Responda las siguientes preguntas para los lenguajes de programación Python, C, C++ y Java:
 - a) ¿Cuál es el mecanismo por defecto al pasar un arreglo como parámetro?
 - b) ¿Se puede modificar el comportamiento por defecto? De ser posible, ¿cómo?
 - c) ¿Cuáles son las principales ventajas y desventajas de cada uno de los mecanismos identificados anteriormente?

0.2. Notación de cuantificadores

La notación matemática tradicional usa convenciones distintas para operaciones que, en el fondo, tienen la misma forma. Una sumatoria se escribe $\sum_{i=0}^{N-1} A[i]$, una cuantificación universal $\forall i. P(i)$ y un mínimo $\min\{A[i] \mid 0 \leq i < N\}$. En cada caso hay una variable que recorre un rango de valores, un término que se evalúa para cada uno de ellos y una operación que combina los resultados. Sin embargo, cada notación marca estos tres elementos de manera distinta y, en algunas, el rango queda implícito.

Esta sección presenta una notación uniforme para todas estas operaciones, conocida como *notación de Eindhoven*, que es la que se usa a lo largo del texto. Su virtud principal es que hace explícitas la variable, el rango y el término. Esto facilita manipular expresiones con rangos de índices como los de la Sección 0.1 y, como se verá al final de la sección, traducirlas directamente a código iterativo.

0.2.1. Forma general. Una expresión cuantificada combina, mediante un operador binario, los valores que toma un término cuando una o más variables recorren los valores que satisfacen una condición.

Definición 0.2.1

Sea $\oplus$ un operador binario asociativo y conmutativo, con elemento neutro e . Una *expresión cuantificada* se escribe

$$(\oplus x \mid R(x) : T(x))$$

en donde x es la *variable ligada* (o una lista de variables ligadas), $R(x)$ es una expresión Booleana llamada el *rango* y $T(x)$ es una expresión llamada el *término*. Su valor es el resultado de combinar con $\oplus$ los valores $T(x)$ para todos los x que satisfacen $R(x)$. Si ningún x satisface $R(x)$, su valor es e .

La asociatividad y la conmutatividad de $\oplus$ garantizan que el valor de la expresión no depende del orden en que se combinen los términos, de modo que la definición no necesita fijarlo. El elemento neutro da sentido al caso en que el rango no contiene ningún valor, que se discute más adelante.

La variable ligada es local a la expresión: su nombre no tiene importancia fuera de ella y puede cambiarse por otro que no aparezca en la expresión sin alterar su valor. Por ejemplo, $(+i \mid 0 \leq i < N : A[i])$ y $(+j \mid 0 \leq j < N : A[j])$ denotan el mismo número. Las demás variables que aparecen en la expresión, como A y N en el ejemplo, se llaman *libres* y su valor proviene del contexto.

Dos convenciones abrevian la escritura de las expresiones cuantificadas. La primera es declarar la variable ligada junto con el conjunto al que pertenece:

$$(\forall x \in A \mid R(x) : T(x)) \quad \text{abrevia} \quad (\forall x \mid x \in A \wedge R(x) : T(x)).$$

La segunda es omitir el rango cuando es *true*, dejando vacío el espacio entre la barra y los dos puntos, como en $(\forall x \in A : T(x))$, que afirma que todos los elementos de A satisfacen T .

Hay una restricción que conviene fijar desde ya. Una suma o un mínimo sobre infinitos valores puede no estar definido, mientras que una afirmación sobre infinitos valores sí lo está. Por ello, se adopta la siguiente convención.

Nota 0.2.1

Los cuantificadores aritméticos (suma, producto, mínimo, máximo y conteo) se usan únicamente con rangos finitos. Los cuantificadores lógicos $\forall$ y $\exists$ admiten rangos infinitos: la expresión $(\forall x \mid R(x) : T(x))$ afirma que $T(x)$ se cumple

para todo x que satisface $R(x)$, y $(\exists x \mid R(x) : T(x))$ afirma que se cumple para al menos uno.

Los cuantificadores sobre rangos infinitos serán necesarios en la Sección 0.8, en donde se afirman propiedades de todos los elementos de un conjunto que puede ser infinito.

0.2.2. Catálogo de cuantificadores. Cada operador asociativo y conmutativo con elemento neutro da lugar a un cuantificador. La Tabla 1 reúne los que se usan en el texto, con su operador, su elemento neutro y la notación tradicional correspondiente. Allí, máx y mín denotan el máximo y el mínimo de dos valores.

Cuantificador	Operador	Neutro	Notación tradicional
$(\forall x \mid R : T)$	$\wedge$	<i>true</i>	$\forall x. R \Rightarrow T$
$(\exists x \mid R : T)$	$\vee$	<i>false</i>	$\exists x. R \wedge T$
$(+ x \mid R : T)$	$+$	0	$\sum_R T$
$(\cdot x \mid R : T)$	$\cdot$	1	$\prod_R T$
$(\downarrow x \mid R : T)$	mín	$+\infty$	$\text{mín}\{T \mid R\}$
$(\uparrow x \mid R : T)$	máx	$-\infty$	$\text{máx}\{T \mid R\}$

Tabla 1. Cuantificadores usados en el texto, con el operador que combina los términos, su elemento neutro y la notación tradicional correspondiente.

Contar cuántos valores de la variable satisfacen una condición es una operación tan frecuente que merece su propio cuantificador. No se obtiene de un operador nuevo, sino de la suma.

Definición 0.2.2

El *cuantificador de conteo* se define como

$$(\# x \mid R(x) : T(x)) = (+ x \mid R(x) \wedge T(x) : 1),$$

en donde $T(x)$ es una expresión Booleana. Su valor es la cantidad de valores de x que satisfacen $R(x)$ y $T(x)$.

El siguiente ejemplo evalúa algunas expresiones cuantificadas sobre un arreglo concreto.

Ejemplo 0.2.1

Sea $A = [1, 0, 25, -1, 8]$, de modo que $N = 5$. Entonces:

- $(+ i \mid 0 \leq i < N : A[i]) = 1 + 0 + 25 + (-1) + 8 = 33$;
- $(\cdot i \mid 2 \leq i < 4 : A[i]) = 25 \cdot (-1) = -25$;
- $(\downarrow i \mid 0 \leq i < N : A[i]) = -1$ y $(\uparrow i \mid 0 \leq i < N : A[i]) = 25$;
- $(\# i \mid 0 \leq i < N : A[i] > 0) = 3$;
- $(\exists i \mid 0 \leq i < N : A[i] = 0)$ es verdadera, mientras que $(\forall i \mid 0 \leq i < N : A[i] \geq 0)$ es falsa.

0.2.3. Rango vacío. La Definición 0.2.1 asigna un valor a toda expresión cuantificada, incluso cuando ningún valor de la variable satisface el rango. Este caso aparece con mucha frecuencia: por ejemplo, al sumar los elementos de un arreglo que resulta ser vacío o al afirmar algo sobre la sección $A[0..0)$.

Nota 0.2.2

Si el rango R es vacío, es decir, si ningún valor de la variable lo satisface, entonces:

$$\begin{aligned}
 (\forall x \mid R : T) &= \text{true}, & (+ x \mid R : T) &= 0, \\
 (\exists x \mid R : T) &= \text{false}, & (\cdot x \mid R : T) &= 1, \\
 (\downarrow x \mid R : T) &= +\infty, & (\uparrow x \mid R : T) &= -\infty, \\
 (\# x \mid R : T) &= 0.
 \end{aligned}$$

Estos valores no son arbitrarios. Si un rango se parte en dos y una de las partes es vacía, el resultado debe coincidir con el de la otra parte; para ello, la parte vacía debe aportar el elemento neutro del operador. En el caso de $\forall$, el valor *true* corresponde a la lectura usual: una afirmación sobre todos los elementos de un conjunto vacío no tiene contraejemplos. En el caso de $\downarrow$, el valor $+\infty$ es el que representa la constante INF en el código de este texto.

0.2.4. Reglas de manipulación. Razonar con expresiones cuantificadas requiere transformarlas. Las siguientes reglas bastan para los razonamientos del texto. Se aceptan sin demostración, pues son consecuencia de la asociatividad, la conmutatividad y el elemento neutro del operador; tratamientos completos se encuentran en los textos citados al final del capítulo.

Nota 0.2.3

Sea $\oplus$ un operador asociativo y conmutativo con elemento neutro e .

1. *Rango vacío:* $(\oplus x \mid \text{false} : T(x)) = e$.
2. *Un punto:* $(\oplus x \mid x = E : T(x)) = T(E)$, si la expresión E no contiene a x .
3. *Partición del rango:* si $R(x)$ y $S(x)$ no son verdaderas simultáneamente para ningún x , entonces

$$(\oplus x \mid R(x) \vee S(x) : T(x)) = (\oplus x \mid R(x) : T(x)) \oplus (\oplus x \mid S(x) : T(x)).$$

4. *Negación:*

$$\neg(\forall x \mid R(x) : T(x)) \equiv (\exists x \mid R(x) : \neg T(x)),$$

$$\neg(\exists x \mid R(x) : T(x)) \equiv (\forall x \mid R(x) : \neg T(x)).$$

Para $\forall$ y $\exists$, la partición del rango vale incluso si $R(x)$ y $S(x)$ se solapan, porque $\wedge$ y $\vee$ son idempotentes: combinar dos veces el mismo término no cambia el resultado. Para la suma, en cambio, el solapamiento contaría dos veces los términos comunes.

Un caso particular de la partición aparece constantemente con arreglos. Si $a \leq c \leq b$, el rango $a \leq i < b$ es la unión disjunta de $a \leq i < c$ y $c \leq i < b$, de modo que

$$(\oplus i \mid a \leq i < b : T(i)) = (\oplus i \mid a \leq i < c : T(i)) \oplus (\oplus i \mid c \leq i < b : T(i)).$$

Es la contraparte, para cuantificadores, de la partición de secciones del Teorema 0.1.1.

Otra consecuencia de las reglas es la que permite extender un rango en una unidad. Se enuncia como teorema porque es la que se usa con más frecuencia, tanto en los invariantes de la Sección 0.7 como en las demostraciones por inducción de la Sección 0.9.

Teorema 0.2.1: Separación de un término

Sean $\oplus$ un operador asociativo y conmutativo con elemento neutro, y $a \leq b$ números enteros. Entonces

$$(\oplus i \mid a \leq i < b + 1 : T(i)) = (\oplus i \mid a \leq i < b : T(i)) \oplus T(b).$$

La demostración muestra las reglas en acción: se parte el rango y se aplica la regla de un punto a la parte que tiene un solo elemento.

Demostración

Como $a \leq b$ y la variable i es entera, el rango $a \leq i < b + 1$ equivale a $a \leq i < b \vee i = b$, y las dos partes no son verdaderas simultáneamente. Por la partición del rango y la regla de un punto,

$$\begin{aligned} & (\oplus i \mid a \leq i < b + 1 : T(i)) \\ &= (\oplus i \mid a \leq i < b : T(i)) \oplus (\oplus i \mid i = b : T(i)) \\ &= (\oplus i \mid a \leq i < b : T(i)) \oplus T(b). \end{aligned}$$

0.2.5. Ejemplos con arreglos. Con la notación de esta sección es posible expresar con precisión propiedades de arreglos que hasta ahora solo se han descrito en palabras. La más importante para lo que sigue es la de estar ordenado, que aparece en la especificación del problema de ordenamiento en la Sección 0.3.

Definición 0.2.3

Un arreglo $A[0..N)$ está *ordenado ascendentemente* si y solo si

$$(\forall i \mid 1 \leq i < N : A[i - 1] \leq A[i]).$$

Si $N \leq 1$, el rango $1 \leq i < N$ es vacío y, por la Nota 0.2.2, la expresión es verdadera: el arreglo vacío y los arreglos de un elemento están ordenados, como cabe esperar.

La Definición 0.2.3 compara únicamente elementos consecutivos. Una formulación alternativa compara todos los pares:

$$(\forall i, j \mid 0 \leq i < j < N : A[i] \leq A[j]).$$

Las dos formulaciones son equivalentes, pero demostrarlo requiere inducción; se propone como ejercicio en la Sección 0.9. El siguiente ejemplo reúne otras propiedades usuales.

Ejemplo 0.2.2

Sean $A[0..N)$ y v un valor.

- El valor v aparece en A : $(\exists i \mid 0 \leq i < N : A[i] = v)$.
- La cantidad de apariciones de v en A : $(\# i \mid 0 \leq i < N : A[i] = v)$.
- El menor elemento de A : $(\downarrow i \mid 0 \leq i < N : A[i])$, que vale $+\infty$ si A es vacío.

- La suma de la sección $A[a..b]$: $(+ i \mid a \leq i < b : A[i])$.
- Todos los elementos de A son distintos: $(\forall i, j \mid 0 \leq i < j < N : A[i] \neq A[j])$.

0.2.6. De la notación al código iterativo. Una ventaja importante de esta notación es que traducir una expresión cuantificada a código iterativo es casi mecánico. Las partes que la notación separa corresponden una a una con las partes de un ciclo, como se muestra en la Tabla 2.

En la expresión	En el ciclo
Variable ligada x	Variable del ciclo
Rango $R(x)$	Condición para repetir una iteración
Término $T(x)$	Argumento de la operación en cada iteración
Operador $\oplus$	Operación de acumulación
Elemento neutro e	Valor inicial del acumulador

Tabla 2. Correspondencia entre las partes de una expresión cuantificada $(\oplus x \mid R(x) : T(x))$ y las partes del ciclo que la calcula.

Por ejemplo, la suma $(+ i \mid a \leq i < b : A[i])$ se traduce así: el acumulador empieza en 0, la variable i empieza en a , el ciclo se repite mientras $i < b$ y en cada iteración se suma el término $A[i]$ y se avanza i en uno.

Implementación 0.2.1: Suma de una sección

```

1 def section_sum(A, a, b):
2     acc, i = 0, a
3     while i < b:
4         acc = acc + A[i]
5         i = i + 1
6     return acc

```

Se prefiere aquí el ciclo **while** sobre el **for** porque hace explícitos los tres elementos del rango: el valor inicial de la variable, la condición para repetir y el avance. Esto será conveniente al razonar sobre el ciclo al final de esta subsección.

El mismo esquema sirve para los demás cuantificadores, cambiando el valor inicial y la operación: para $\downarrow$ el acumulador empieza en INF y la operación es min ;

para $\forall$ empieza en **True** y la operación es **and**. A partir de este esquema básico se obtienen tres variantes frecuentes:

1. *Rangos con filtro*. Si el rango tiene la forma $a \leq i < b \wedge P(i)$, la condición $P(i)$ se traduce en un condicional dentro del ciclo. Así se implementa, por ejemplo, el conteo $(\#i \mid 0 \leq i < N : A[i] = v)$: un ciclo sobre $0 \leq i < N$ que suma 1 solo cuando $A[i] = v$.
2. *Varias variables ligadas*. Cada variable ligada da lugar a un ciclo, y los ciclos quedan anidados. La formulación de “ordenado” que compara todos los pares produce dos ciclos anidados, mientras que la Definición 0.2.3 produce uno solo. Ambas calculan lo mismo, pero la primera hace del orden de N^2 comparaciones y la segunda del orden de N ; esta diferencia de costo es el tema del Capítulo 1.
3. *Salida temprana*. Para $\forall$, el acumulador no puede recuperarse una vez que vale *false*, pues $false \wedge p = false$ para cualquier p . Por ello, el ciclo puede terminar en cuanto encuentra un contraejemplo. Lo análogo vale para $\exists$ y el valor *true*.

La salida temprana no requiere abandonar la función en la mitad del ciclo. Basta con que el acumulador Booleano actúe como *centinela*: la condición para repetir exige, además de que el rango no se haya agotado, que el acumulador no haya alcanzado el valor absorbente. Aplicada a la Definición 0.2.3, esta idea produce la siguiente implementación.

Implementación 0.2.2: Verificación de orden

```

1 def is_sorted(A):
2     ans, i = True, 1
3     while i < len(A) and ans:
4         ans = A[i-1] <= A[i]
5         i = i + 1
6     return ans

```

Según el esquema, la actualización del acumulador sería

`ans = ans and A[i-1] <= A[i]`

Como la condición del ciclo garantiza que `ans` es verdadero al entrar en cada iteración, esa conjunción se simplifica a la asignación de la línea 4. El ciclo tiene así un único punto de salida, lo cual simplifica el razonamiento sobre su corrección en la Sección 0.7.

La correspondencia entre la notación y el ciclo tiene una consecuencia que se aprovechará en la Sección 0.7. Al comienzo de cada iteración del ciclo de la Implementación 0.2.1, el acumulador cumple

$$acc = (+j \mid a \leq j < i : A[j]).$$

Antes de la primera iteración, con $i = a$, el rango es vacío y ambos lados valen 0. Cada iteración suma $A[i]$ al acumulador y avanza i en uno, lo cual preserva la igualdad por el Teorema 0.2.1. Cuando el ciclo termina, se tiene $i = b$ (pues i parte de $a \leq b$ y avanza de uno en uno) y, por lo tanto, $acc = (+j \mid a \leq j < b : A[j])$, que es precisamente lo que se quería calcular. Una propiedad que se cumple al inicio y que cada iteración preserva se llama un *invariante* del ciclo; la Sección 0.7 desarrolla esta idea.

Finalmente, Python ofrece funciones que implementan directamente los cuantificadores más comunes. Se usan cuando su brevedad no oculta la idea del algoritmo.

Nota 0.2.4

Con rango $a \leq i < b$, término $T(i)$ y condición Booleana $P(i)$, los cuantificadores corresponden a las siguientes expresiones de Python, en donde g abrevia la expresión generadora $(T(i) \text{ for } i \text{ in range}(a, b))$ y p abrevia $(P(i) \text{ for } i \text{ in range}(a, b))$:

```

 $\forall$   all(p)
 $\exists$   any(p)
+    sum(g)
 $\cdot$   math.prod(g)
 $\downarrow$  min(g, default=INF)
 $\uparrow$  max(g, default=-INF)
#    sum(1 for i in range(a, b) if P(i))

```

Las funciones `all` y `any` implementan la salida temprana. Note, además, que `min` y `max` producen un error sobre una secuencia vacía a menos que se indique el valor `default`; con `default=INF`, el comportamiento coincide con la Nota 0.2.2.

Ejercicios

- Sea $A = [3, 1, 4, 1, 5]$. Evalúe las siguientes expresiones:
 - $(+i \mid 1 \leq i < 4 : A[i])$.
 - $(\cdot i \mid 0 \leq i < 5 \wedge A[i] \neq 1 : A[i])$.
 - $(\uparrow i \mid 3 \leq i < 3 : A[i])$.
 - $(\#i, j \mid 0 \leq i < j < 5 : A[i] = A[j])$.
 - $(\forall i \mid 0 \leq i < 5 : A[i] > 0) \wedge (\exists i \mid 0 \leq i < 5 : A[i] > 4)$.
- Sea $A[0..N]$ un arreglo de números enteros. Escriba con la notación de esta sección cada una de las siguientes propiedades:

- a) El máximo de A aparece exactamente una vez.
 - b) A es una permutación de $[0, 1, \dots, N - 1]$.
 - c) A se lee igual de izquierda a derecha que de derecha a izquierda (es un palíndromo).
 - d) Todo elemento de $A[0..n)$ es menor o igual que todo elemento de $A[n..N)$, para un n dado con $0 \leq n \leq N$.
 - e) Ninguna sección de A de tamaño K tiene suma mayor que un valor S dado.
3. Use las reglas de la Nota 0.2.3 para demostrar la versión izquierda del Teorema 0.2.1: si $a \leq b$, entonces
- $$(\oplus i \mid a - 1 \leq i < b : T(i)) = T(a - 1) \oplus (\oplus i \mid a \leq i < b : T(i)).$$
4. Use la regla de negación para escribir, sin el símbolo $\neg$ al frente, qué significa que un arreglo *no* esté ordenado ascendentemente.
5. Demuestre, usando el Teorema 0.2.1, que $A[0..N + 1)$ está ordenado ascendentemente si y solo si $A[0..N)$ lo está y, además, $N = 0$ o $A[N - 1] \leq A[N]$.
6. Suponga que se decidiera que el valor de $(\downarrow x \mid R : T)$ con rango vacío fuera 0 en lugar de $+\infty$. Muestre, con un ejemplo concreto, que la regla de partición del rango dejaría de ser válida.
7. Implemente en Python una función `quantify(op, e, a, b, T)` que calcule $(\oplus i \mid a \leq i < b : T(i))$, en donde `op` es una función de dos argumentos, `e` su elemento neutro y `T` una función de un argumento. Úsela para calcular la suma, el mínimo y la verificación de orden de un arreglo.
8. Implemente la verificación de orden con la formulación que compara todos los pares, usando dos ciclos anidados y salida temprana con un centinela, sin retornar en la mitad de la función. ¿Cuántas comparaciones hace, en el peor caso, sobre un arreglo de tamaño N ? Compárelo con la cantidad que hace la Implementación 0.2.2.
9. Traduzca a código iterativo cada una de las expresiones del Ejercicio 2, identificando en cada caso la variable del ciclo, la condición para repetir, la operación de acumulación y el valor inicial.

0.3. Problemas algorítmicos y su especificación

Un computador puede ser utilizado para asistir en la mecanización de un sinnúmero de tareas, de diferentes envergaduras y con propósitos muy diversos. Por ejemplo, un computador puede ser utilizado para encontrar la ruta más rápida (o corta) para conducir de un lugar a otro en una ciudad. También puede ser utilizado para almacenar y analizar grandes volúmenes de datos como los asociados al

genoma humano. Hoy en día, los computadores son la herramienta de trabajo en muchas profesiones y se requieren para una gran cantidad de actividades.

Generalmente, es necesario diseñar piezas algorítmicas pequeñas y con propósitos muy específicos que, al componerse con otras piezas, permitan abordar tareas de mayor dificultad, como las mencionadas anteriormente. En algunos lenguajes de programación estas piezas reciben el nombre de funciones, procedimientos o módulos.

Nota 0.3.1

El propósito de este manuscrito es presentar técnicas de diseño y análisis de algoritmos para construir satisfactoriamente piezas algorítmicas fundamentales, que junto con otras piezas pueden conformar grandes sistemas de cómputo, información o procesamiento.

Conceptualmente, se puede aceptar que un problema es una pregunta formulada y a la cual se desea encontrar respuesta. De esta forma, se puede concebir un problema algorítmico como un problema cuya respuesta puede ser encontrada con la asistencia de un computador que ejecute mecánicamente unas instrucciones. Para ello, el problema debe formularse con precisión.

0.3.1. Especificación. Formular un problema con precisión significa decir con qué datos se cuenta y qué se desea obtener a partir de ellos.

Definición 0.3.1

La *especificación* de un problema algorítmico está conformada por dos partes: la descripción de la *entrada* y la descripción de la *salida*.

En un problema algorítmico, la descripción de la entrada identifica cuáles son los datos relevantes que hacen parte del planteamiento del problema y qué se supone sobre ellos. La descripción de la salida identifica la pregunta que se desea responder con base en esos datos. Lo importante es que la descripción de la salida no es una instrucción, sino una *condición* que relaciona la salida con la entrada: dada una entrada y una salida candidata, debe ser posible decidir si la segunda es correcta para la primera. Las dos partes de una especificación reciben nombres propios.

Definición 0.3.2

La *precondición* de un problema algorítmico es la condición que se supone sobre la entrada. La *poscondición* es la condición que debe cumplir la salida con respecto a la entrada.

En este texto las especificaciones se presentan en un formato fijo, en el cual la precondición se describe junto a la palabra “Entrada” y la poscondición junto a la palabra “Salida”.

Nota 0.3.2

Se usa el siguiente formato para presentar problemas algorítmicos:

Entrada: ...

Salida: ...

en donde a la derecha de “Entrada:” se describe la entrada del problema y a la derecha de “Salida:” la salida del problema.

El Ejemplo 0.3.1 presenta la especificación del problema de ordenamiento de un arreglo de números. Como el arreglo se modifica en el mismo lugar, la salida debe poder referirse al valor que tenía el arreglo antes de ser modificado. Para ello se adopta la convención de nombrar ese valor inicial con el subíndice 0: si la entrada es el arreglo A , su valor inicial se denota A_0 .

Ejemplo 0.3.1

Considere el problema de ordenar un arreglo de números. El problema algorítmico asociado puede definirse de la siguiente manera:

Entrada: Un arreglo $A[0..N)$, con $N \geq 0$, de números enteros, cuyo valor inicial se denota A_0 .

Salida: El arreglo A modificado de modo que esté ordenado ascendentemente y sea una permutación de A_0 .

La poscondición del Ejemplo 0.3.1 tiene dos partes y ambas son necesarias. La primera usa la Definición 0.2.3. La segunda usa la Definición 0.1.6 y descarta salidas que están ordenadas pero que no tienen nada que ver con la entrada: sin ella, llenar el arreglo con ceros sería una respuesta correcta.

Observe, además, que al especificar el problema se tomaron varias decisiones:

1. Se indica concretamente la estructura de datos que contiene los números y cómo se llama: un arreglo llamado A , con N números enteros indexados desde 0.

2. Se identifica qué arreglos se consideran: cualquier arreglo de números enteros, incluyendo el arreglo vacío (¿por qué?).
3. Se establece que el objetivo es ordenar el arreglo dado y no una copia suya, es decir, se plantea un ordenamiento *in situ*. Esta decisión es la que obliga a nombrar el valor inicial A_0 .

0.3.2. Parámetros e instancias. Un problema algorítmico puede contener “variables” a las cuales no se han asignado valores concretos en su especificación. En el Ejemplo 0.3.1, este es el caso de A y N : indican que la entrada del problema puede ser *cualquier* arreglo (llamado A por conveniencia o por falta de imaginación) de *cualquier* tamaño. La intención de emplear variables en la especificación de un problema algorítmico es transmitir el mensaje de que su formulación es de carácter general y no particular.

Definición 0.3.3

Un *parámetro* es una variable que aparece en la especificación de un problema algorítmico y no tiene un valor concreto.

Dado que la especificación de un problema puede tener parámetros, un problema algorítmico en realidad representa una colección de problemas concretos: uno por cada asignación de valores a sus parámetros que cumpla con la precondition. Es decir, un problema algorítmico con parámetros es una expresión simbólica que representa una colección de problemas concretos (o instancias), aquellos que específicamente se desea resolver en la práctica.

Definición 0.3.4

Una *instancia* de un problema algorítmico es una asignación de valores concretos a los parámetros de su especificación que cumple con la precondition.

Un problema algorítmico, en general, tiene tantas instancias como asignaciones admitan sus parámetros. De esta forma, hay problemas algorítmicos que representan una cantidad finita de instancias y otros una cantidad infinita de ellas.

Ejemplo 0.3.2

Considere el problema algorítmico presentado en el Ejemplo 0.3.1. Las siguientes son algunas de sus instancias:

- $A = []$ y $N = 0$.

- $A = [1, 2, 3]$ y $N = 3$.
- $A = [2, 7, 1, 9, 8]$ y $N = 5$.

La noción de instancia no es exclusiva de la computación: está presente, sin que se le dé ese nombre, desde la formación básica en matemáticas.

Ejemplo 0.3.3

Al establecer la ecuación $(a + b)^2 = a^2 + 2ab + b^2$, con a y b números reales, lo que realmente se está indicando es que para cualquier par de números reales (i.e., instancias de a y b) se puede calcular su suma al cuadrado calculando independientemente sus cuadrados y sumando el doble de su producto.

0.3.3. Soluciones. Aún no se ha definido qué se entiende por responder la pregunta que plantea una instancia. Gracias a que la poscondición es una condición que relaciona la salida con la entrada, la definición es directa.

Definición 0.3.5

Una *solución* de una instancia de un problema algorítmico es una salida que satisface la poscondición del problema para los valores de esa instancia.

Una instancia puede tener exactamente una solución, varias o ninguna. La Figura 2 ilustra los tres casos con un problema de búsqueda.

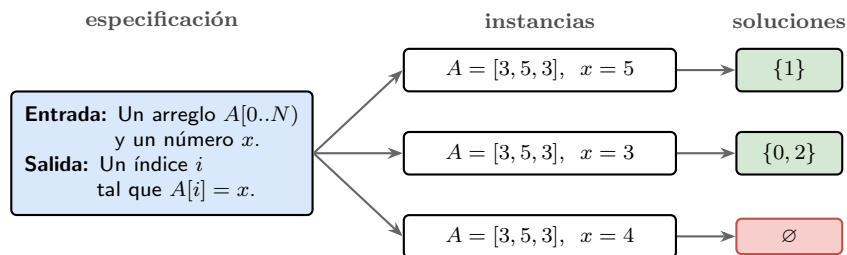


Figura 2. Una especificación, tres de sus instancias y el conjunto de soluciones de cada una. Como $x = 3$ aparece dos veces en A , la segunda instancia tiene dos soluciones; como $x = 4$ no aparece, la tercera no tiene ninguna.

Resolver *una* instancia específica y resolver *todas* las instancias de un problema son tareas cuya dificultad puede ser abismalmente distinta. Una instancia pequeña

puede resolverse sin ingenio alguno: ordenar un arreglo con cinco números es fácil, mientras que ordenar otro con un millón puede ser más demandante en recursos. Además, ciertas instancias pueden “esconder” la complejidad real del problema: ordenar un arreglo que ya está ordenado es trivial, sin importar su tamaño, mientras que el caso general no lo es.

El estudio de algoritmos es interesante precisamente cuando su propósito es resolver todas las instancias de un problema algorítmico. El reto del diseño algorítmico está en definir un algoritmo (e.g., unas instrucciones que un computador pueda ejecutar) que, para cada instancia, produzca una de sus soluciones. Esta noción de resolver un problema se precisa en la Sección 0.5.

Ejemplo 0.3.4

Para el problema de ordenamiento presentado en el Ejemplo 0.3.1, las siguientes son las soluciones de las instancias del Ejemplo 0.3.2:

- $A = []$.
- $A = [1, 2, 3]$.
- $A = [1, 2, 7, 8, 9]$.

En los dos primeros casos, el arreglo A estaba originalmente ordenado ascendentemente. En los tres casos la solución es única: un arreglo ordenado queda determinado por los valores que contiene.

La unicidad del Ejemplo 0.3.4 es una propiedad del problema de ordenamiento y no de los problemas en general. A continuación se presenta un problema cuya única instancia tiene más de una solución.

Ejemplo 0.3.5

Considere la siguiente especificación:

Entrada: Un tablero de ajedrez de 8×8 vacío.

Salida: Ocho celdas del tablero dado en las cuales se pueden ubicar reinas sin que se ataquen entre ellas.

Se propone como ejercicio al lector mostrar que la única instancia de este problema algorítmico, conocido como el “problema de las 8 reinas”, tiene más de una solución.

El problema de las 8 reinas se generaliza naturalmente a un tablero de $N \times N$ en el cual se deben ubicar N reinas. Esta generalización tiene infinitas instancias, una por cada $N \geq 1$, y algunas de ellas no tienen solución. Por ejemplo, en un

tablero de 2×2 cualquier par de celdas está en la misma fila, la misma columna o la misma diagonal, de modo que no es posible ubicar dos reinas sin que se ataquen; lo mismo ocurre, con un argumento algo más largo, en un tablero de 3×3 .

Una instancia sin solución indica que la especificación está incompleta: no dice qué debe responderse en ese caso. La manera de completarla es incluir ese caso en la poscondición.

Ejemplo 0.3.6

La siguiente especificación del problema de las N reinas indica qué responder cuando no hay solución:

Entrada: Un tablero de ajedrez de $N \times N$ vacío, con $N \geq 1$.

Salida: N celdas del tablero dado en las cuales se pueden ubicar reinas sin que se ataquen entre ellas, si existen; de lo contrario, una indicación de que no existen.

Con esta especificación, toda instancia tiene al menos una solución. El problema se estudia en detalle en la Sección 5.4.

Completar una especificación de esta manera es lo que permite, en la Sección 0.5, exigir que un algoritmo responda *todas* las instancias de un problema sin excepción.

Hay, por último, una diferencia en los niveles de abstracción empleados en las especificaciones de los ejemplos 0.3.1 y 0.3.5. Para el ordenamiento, la entrada es un arreglo, que puede ser considerado como un elemento primitivo en un lenguaje de programación. En cambio, la entrada del problema de las 8 reinas es un tablero de ajedrez, que difícilmente es parte de los elementos primitivos de un lenguaje de programación. Esta diferencia no indica que alguno de los dos estilos de especificación esté mal. Sugiere, más bien, que en algunas ocasiones es conveniente especificar sin detallar la forma de los parámetros, porque hacerlo se considera irrelevante en ese momento o porque hay una razón de fondo para mantener un nivel de abstracción alto. En el caso de las 8 reinas, el tablero puede representarse fácilmente con una matriz de 8×8 llena de ceros; sin embargo, la especificación asume que no hay necesidad de entrar en ese tipo de detalles para entender la esencia del problema y pensar en una forma de resolverlo.

0.3.4. Buenas y malas especificaciones. Especificar problemas algorítmicos requiere experiencia que se construye con la práctica. Puede ser retador adquirir esta disciplina, pero merece la pena: un problema bien especificado está más cerca de ser resuelto. En la Nota 0.3.3 se presentan algunas sugerencias para la especificación de problemas algorítmicos.

Nota 0.3.3: Regla de la mano derecha para especificar problemas

Un problema bien especificado formula cómo determinar si una salida dada es correcta para una entrada dada. Para ello, es recomendable:

- usar notación y términos claros y concisos, evitando argot especializado ajeno a la computación;
- si es necesario incluir argot especializado, definirlo formalmente (y preferiblemente antes de ser usado);
- indicar qué debe responderse para las instancias en las que no existe una salida con las características deseadas;
- *no* mencionar
 - ni detalles de la fuente de datos de la cual proviene la entrada (e.g., base de datos, archivo de texto o un microservicio en la nube),
 - ni una estrategia de solución o detalles de su implementación;
- si la especificación *puede* ser simplificada, entonces *debe* ser simplificada.

Toda especificación debe ser precisa, corta y elegante.

El problema del Ejemplo 0.3.1 está bien especificado de acuerdo con la regla de la mano derecha de la Nota 0.3.3. El siguiente ejemplo muestra cómo aplicarla para corregir una especificación defectuosa.

Ejemplo 0.3.7

Considere la siguiente especificación:

Entrada: Una secuencia de líneas de texto; en la primera un número x y en la segunda un arreglo A .

Salida: ¿Está x en A ?

La entrada menciona la fuente de los datos (líneas de texto), que es irrelevante para el problema; además, no dice de qué tipo son los elementos de A ni nombra su tamaño. La salida, aunque clara en este caso, puede escribirse como una condición precisa con la notación de la Sección 0.2. Al corregir estos defectos se obtiene:

Entrada: Un arreglo $A[0..N)$ de números enteros y un número entero x .

Salida: $\neg(\exists i \mid 0 \leq i < N : A[i] = x)$?

Esta sección concluye con más ejemplos de buenas y malas especificaciones de problemas algorítmicos.

Ejemplo 0.3.8

A continuación se presentan buenas especificaciones de problemas algorítmicos.

Entrada: Un grafo dirigido G .

Salida: Un orden topológico de G , si existe; de lo contrario, una lista vacía.

Entrada: Un grafo $G = (V, E)$ y un conjunto $C \subseteq V$.

Salida: ¿El subgrafo de G inducido por C es un clique (i.e., un grafo en donde cualquier par de vértices está conectado)?

Entrada: Un conjunto finito P de puntos del plano cartesiano, con al menos dos puntos.

Salida: La mínima distancia entre dos puntos distintos de P .

Las malas especificaciones del siguiente ejemplo incumplen, cada una, al menos una de las recomendaciones de la regla de la mano derecha.

Ejemplo 0.3.9

A continuación se presentan malas especificaciones de problemas algorítmicos; las razones por las cuales son consideradas malas especificaciones, en cada caso, se proponen como ejercicio para el lector.

Entrada: Un grafo G con peso en los arcos.

Salida: El mejor camino para ir de u a v .

Entrada: Una colección de cadenas.

Salida: La supercadena maximal de todas las cadenas de la entrada.

Entrada: Un arreglo A de números.

Salida: El arreglo A ordenado usando el método de la burbuja.

Ejercicios

1. Considere el problema de encontrar un número en un arreglo de números. Especifique los siguientes problemas asociados e identifique cuáles son sus parámetros:
 - a) Determinar si el número está en el arreglo.
 - b) Suponiendo que el número está en el arreglo, determinar un índice en donde está el número en el arreglo.
 - c) Si el número está en el arreglo, determinar un índice en el arreglo en donde está el número; de lo contrario, identificar que no aparece.
 - d) Suponiendo que el número está en el arreglo, determinar el índice más pequeño en donde está el número en el arreglo.
 - e) Suponiendo que el número está en el arreglo, determinar el índice más grande en donde está el número en el arreglo.
2. Escriba las poscondiciones de los problemas del Ejercicio 1 con la notación de la Sección 0.2. ¿En cuáles de ellos la precondition no es trivial?
3. Investigue y especifique el problema de buscar un valor numérico en un arreglo de números *ordenados* ascendentemente. ¿Qué diferencia fundamental hay entre este problema y los especificados en el Ejercicio 1?
4. Especifique el problema de ordenar ascendentemente un arreglo de números enteros dejando el resultado en un arreglo nuevo, sin modificar el arreglo dado. ¿Sigue siendo necesario nombrar el valor inicial de la entrada? Justifique su respuesta.
5. Identifique la precondition y la poscondición de cada una de las especificaciones del Ejemplo 0.3.8.
6. Especifique el problema de calcular en $A[0..N)$ el valor de los primeros N números factoriales.
7. Especifique el problema de multiplicar dos matrices de números. ¿Qué restricciones deben cumplir las dimensiones de las matrices?
8. ¿Cuántas instancias tiene el problema algorítmico del Ejemplo 0.3.1? Justifique su respuesta.
9. Investigue acerca de cada uno de los siguientes problemas y especifíquelos como problemas algorítmicos:
 - a) El problema de primalidad de números.
 - b) El último teorema de Fermat.
 - c) La conjetura de Goldbach.
 - d) El problema de satisfacibilidad proposicional.
 - e) El problema de la parada.

10. Clasifique los problemas del Ejercicio 1 de acuerdo a si tienen instancias con solución única o no. Ilustre su respuesta con ejemplos.
11. Clasifique los problemas del Ejercicio 9 de acuerdo a si tienen instancias con solución única o no. Ilustre su respuesta con ejemplos.
12. ¿Cuántas soluciones tiene el problema de las 8 reinas del Ejemplo 0.3.5? Investigue y formule tres soluciones.
13. Demuestre que el problema de las N reinas no tiene solución para $N = 3$.
14. Considere el siguiente problema:

Entrada: Un tablero de Sudoku parcialmente lleno.

Salida: Una solución del tablero de Sudoku dado.

- a) ¿Cuántas instancias tiene el problema? Detalle los cálculos hechos.
 - b) Especifique una instancia del problema que no tiene solución.
 - c) Corrija la especificación para que toda instancia tenga al menos una solución.
 - d) ¿Es un problema para el cual cada instancia tiene a lo sumo una solución? Justifique su respuesta.
15. Investigue y especifique tres problemas algorítmicos que tengan una cantidad finita de instancias.
 16. Investigue y especifique tres problemas algorítmicos que tengan una cantidad infinita de instancias.
 17. ¿Existe un problema algorítmico con múltiples instancias, pero para el cual todas las instancias tienen la misma solución (i.e., independientemente de la entrada, la salida siempre es la misma)? Justifique su respuesta.
 18. Explique brevemente por qué las especificaciones del Ejemplo 0.3.9 no son consideradas buenas especificaciones de problemas algorítmicos y corríjelas, como en el Ejemplo 0.3.7.
-

0.4. Tipos de problemas

En la Sección 0.3, la poscondición de un problema determina qué clase de respuesta se espera: un sí o un no, un objeto con ciertas características, un número o una colección de objetos. Esta sección clasifica los problemas algorítmicos de acuerdo con esa forma. La clasificación no es un fin en sí misma: cada una de las técnicas de diseño del texto se adapta mejor a unos tipos de problemas que a otros; reconocer el tipo de un problema es a menudo el primer paso para escoger cómo abordarlo.

0.4.1. Testigos. Muchos problemas que a primera vista son distintos comparten una misma estructura: para cada instancia hay una colección de objetos que cumplen cierta propiedad y lo que cambia de un problema a otro es qué se pregunta sobre esa colección.

Definición 0.4.1

Suponga que para cada instancia x de un problema se ha fijado un conjunto finito $W(x)$ de objetos. Los elementos de $W(x)$ se llaman *testigos* de la instancia x .

La Definición 0.4.1 es deliberadamente general: el conjunto $W(x)$ lo determina cada problema. El siguiente ejemplo fija el que se usará a lo largo de la sección.

Ejemplo 0.4.1

Considere un arreglo $A[0..N)$ de números enteros positivos y un número entero X . Para un conjunto de índices $T \subseteq \{0, \dots, N-1\}$, sea

$$\sigma(T) = (+i \mid i \in T : A[i])$$

la suma de los elementos de A con índices en T . Los testigos de la instancia son los conjuntos de índices cuyos elementos suman exactamente X :

$$W(A, X) = \{ T \subseteq \{0, \dots, N-1\} \mid \sigma(T) = X \}.$$

Por ejemplo, con $A = [3, 1, 4, 2, 5]$ y $X = 5$, los testigos son $\{4\}$, $\{0, 3\}$ y $\{1, 2\}$, pues $A[4] = 5$, $A[0] + A[3] = 3 + 2 = 5$ y $A[1] + A[2] = 1 + 4 = 5$.

Este problema, conocido como el de la *suma de subconjuntos*, es el mismo con el que se abre la Sección 5.1. En la terminología de ese capítulo, los testigos son las soluciones de un problema de búsqueda combinatoria.

0.4.2. Cinco tipos de problemas. Fijado el conjunto de testigos $W(x)$ de cada instancia, hay cinco preguntas naturales que pueden hacerse sobre él. Cada una da lugar a un tipo de problema.

Definición 0.4.2

Sea x una instancia con conjunto de testigos $W(x)$.

1. En un *problema de decisión*, la salida es el valor Booleano de $(\exists w \mid w \in W(x) : \text{true})$, es decir, si existe algún testigo.

2. En un *problema de búsqueda*, la salida es un testigo $w \in W(x)$, si existe; de lo contrario, una indicación de que no existe.
3. En un *problema de optimización*, se cuenta además con una *función objetivo* f que asigna un número a cada testigo. La salida es un testigo $w \in W(x)$ cuyo valor $f(w)$ es mínimo (o máximo) entre todos los testigos, si existe; de lo contrario, una indicación de que no existe.
4. En un *problema de conteo*, la salida es la cantidad de testigos, $(\# w \mid w \in W(x) : \text{true})$.
5. En un *problema de enumeración*, la salida es el conjunto $W(x)$ de todos los testigos.

El siguiente ejemplo presenta las cinco versiones del problema de la suma de subconjuntos. En todas, la entrada es la misma y lo único que cambia es la salida.

Ejemplo 0.4.2

Sean $A[0..N)$, X y σ como en el Ejemplo 0.4.1, y sea $I = \{0, \dots, N-1\}$. Las cinco versiones del problema tienen la entrada

Entrada: Un arreglo $A[0..N)$ de números enteros positivos y un número entero X .

Salida: ...

y las siguientes salidas:

1. Decisión: $(\exists T \mid T \subseteq I : \sigma(T) = X)$.
2. Búsqueda: un conjunto $T \subseteq I$ con $\sigma(T) = X$, si existe; de lo contrario, una indicación de que no existe.
3. Optimización: un conjunto $T \subseteq I$ con $\sigma(T) = X$ y con la menor cantidad posible de elementos, si existe; de lo contrario, una indicación de que no existe.
4. Conteo: $(\# T \mid T \subseteq I : \sigma(T) = X)$.
5. Enumeración: el conjunto $\{T \subseteq I \mid \sigma(T) = X\}$.

Para la instancia $A = [3, 1, 4, 2, 5]$ y $X = 5$, las respuestas son, respectivamente: verdadero; cualquiera de $\{4\}$, $\{0, 3\}$ o $\{1, 2\}$; $\{4\}$; 3; y $\{\{4\}, \{0, 3\}, \{1, 2\}\}$.

En los problemas de optimización conviene distinguir dos versiones. La Definición 0.4.2 pide un testigo óptimo y esta es la *versión de testigo*. Con frecuencia basta, en cambio, con conocer el valor óptimo de la función objetivo, sin exhibir un testigo que lo alcance; esta es la *versión de valor*. Para la suma de subconjuntos,

la versión de valor es

$$(\downarrow T \mid T \subseteq I \wedge \sigma(T) = X : |T|).$$

Observe que la convención de la Nota 0.2.2 resuelve por sí sola el caso sin testigos: si no hay ninguno, el rango es vacío y el valor es $+\infty$. La distinción entre las dos versiones es importante en el Capítulo 3, en donde es usual calcular primero el valor óptimo y reconstruir después un testigo que lo alcance.

No todo problema se presenta naturalmente con testigos, pero muchos que no parecen hacerlo encajan en la clasificación. El problema de ordenamiento del Ejemplo 0.3.1, por ejemplo, puede verse como un problema de búsqueda en el cual cada instancia tiene exactamente un testigo: el arreglo ordenado que es permutación de A_0 . Cuando todas las instancias tienen exactamente un testigo, buscarlo equivale a calcular una función de la entrada.

0.4.3. Resumen y relaciones. Los cinco tipos difieren en la forma de su salida y en la cantidad de soluciones que admite cada instancia, en el sentido de la Definición 0.3.5. La Tabla 3 resume estas diferencias e indica en qué capítulos del texto cada tipo de problema tiene un papel protagónico.

Tipo	Salida	Soluciones	Capítulos
Decisión	Booleano	exactamente una	6–8
Búsqueda	testigo	una o varias	2 y 5
Optimización (valor)	número	exactamente una	3 y 4
Optimización (testigo)	testigo	una o varias	3 y 4
Conteo	natural	exactamente una	5
Enumeración	conjunto	exactamente una	5

Tabla 3. Los cinco tipos de problemas: forma de la salida, cantidad de soluciones por instancia y capítulos en los que cada tipo tiene un papel protagónico.

La enumeración tiene exactamente una solución porque su salida es un conjunto. Si, en cambio, se pidiera una *lista* con todos los testigos, cada orden posible de la lista sería una solución distinta. Los modos de enumeración, optimización y conteo que se distinguen en la Sección 5.1 corresponden a los tipos del mismo nombre.

Los tipos de problemas no son independientes entre sí: saber resolver uno puede bastar para resolver otro. Por ejemplo, si se cuenta con un algoritmo que resuelve la versión de conteo, también se puede resolver la de decisión: existe un testigo si y solo si la cantidad de testigos es positiva. La Figura 3 reúne estas relaciones.

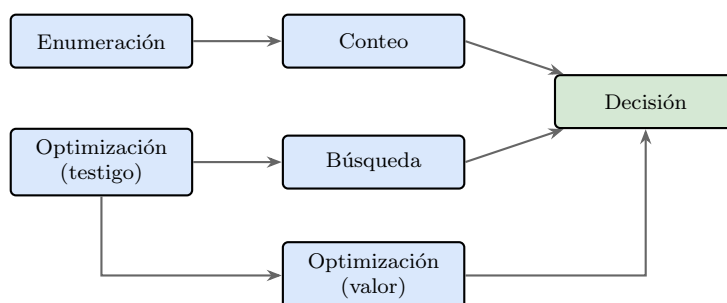


Figura 3. Relaciones entre los tipos de problemas definidos sobre un mismo conjunto de testigos. Una flecha de un tipo a otro indica que un algoritmo para el primero permite resolver el segundo. Las relaciones se componen; por ejemplo, la enumeración permite resolver la decisión.

Cada flecha de la Figura 3 se justifica con un argumento corto:

- de la enumeración al conteo, contando los elementos del conjunto de testigos;
- del conteo a la decisión, comparando la cantidad con 0;
- de la optimización (testigo) a la búsqueda, pues un testigo óptimo es, en particular, un testigo;
- de la búsqueda a la decisión, verificando si se encontró un testigo o se indicó que no existe;
- de la optimización (testigo) a la optimización (valor), evaluando la función objetivo en el testigo encontrado;
- de la optimización (valor) a la decisión, comparando el valor con $+\infty$ (o con $-\infty$, si se maximiza).

Estas relaciones dicen que un tipo de problema permite resolver otro, pero no dicen nada sobre el costo de hacerlo ni sobre las direcciones contrarias. Que un problema de decisión sea fácil no implica que su versión de conteo lo sea: decidir si hay algún camino entre dos vértices de un grafo y contar cuántos caminos simples hay entre ellos son problemas de naturaleza muy distinta. Por ello, al especificar un problema conviene preguntarse cuál de las cinco preguntas se quiere responder realmente, pues la elección cambia el problema. El estudio riguroso de las relaciones entre problemas es uno de los temas de los Capítulos 6 a 8.

Ejercicios

1. Clasifique cada uno de los problemas especificados en el Ejercicio 1 de la Sección 0.3 de acuerdo con los tipos de la Definición 0.4.2. En cada caso, identifique el conjunto de testigos.

2. Para la instancia $A = [2, 3, 5, 1, 4]$ y $X = 6$ del problema de la suma de subconjuntos, calcule las respuestas a las cinco versiones del Ejemplo 0.4.2 y el valor óptimo de la versión de valor.
3. Considere un grafo dirigido G con pesos positivos en los arcos y dos vértices u y v de G . Sea el conjunto de testigos el de los caminos simples de u a v y la función objetivo el peso de un camino. Especifique las cinco versiones del problema, más la versión de valor de la optimización.
4. Considere el problema de las N reinas del Ejemplo 0.3.6. ¿Cuál es el conjunto de testigos de una instancia? Especifique las versiones de decisión, conteo y enumeración. ¿Tiene sentido una versión de optimización? Justifique su respuesta.
5. Justifique que la enumeración permite resolver la búsqueda, aunque la Figura 3 no incluya esa flecha de manera directa.
6. Suponga que cuenta con un algoritmo que resuelve la versión de decisión del problema de la suma de subconjuntos para cualquier instancia. Muestre cómo usarlo para resolver la versión de búsqueda. Sugerencia: decida, para cada índice i , si existe un testigo que no contenga a i . ¿Cuántas veces usa el algoritmo de decisión?
7. Suponga que en la versión de enumeración del Ejemplo 0.4.2 se pidiera una lista con todos los testigos, en lugar del conjunto. ¿Cuántas soluciones tendría la instancia $A = [3, 1, 4, 2, 5]$ y $X = 5$? ¿Cuántas tendría una instancia con k testigos?
8. Considere los enunciados informales “¿hay alguna palabra del diccionario D que sea prefijo de la cadena s ?” y “¿cuántas palabras del diccionario D son prefijo de la cadena s ?”. Especifique ambos problemas, identifique su conjunto de testigos común y explique por qué, aunque se parecen, son problemas distintos.

0.5. El concepto de algoritmo

Donald Knuth, uno de los científicos más destacados de la informática, ha propuesto caracterizar las ciencias de la computación como el estudio de los algoritmos [Knu74]. Sea o no una caracterización completa, los algoritmos son el hilo que conecta buena parte de las áreas de la disciplina. Esta sección precisa qué se entiende por algoritmo en este texto, fija el modelo de computación con el que se trabaja y define qué significa que un algoritmo resuelva un problema algorítmico.

La palabra misma tiene una historia que vale la pena conocer, porque ilustra que la noción de algoritmo nació ligada al cálculo con números mucho antes de que existieran los computadores.

Nota 0.5.1

La palabra *algoritmo* se asocia al nombre del matemático persa Muhammad ibn Musa al-Khwarismi, quien vivió aproximadamente entre los años 780 y 850 y trabajó en la Casa de la Sabiduría de Bagdad. Su tratado sobre el cálculo con los numerales de origen indio, cuyo original árabe se perdió, se conserva en una traducción latina del siglo XII conocida como *Algoritmi de numero Indorum*; allí, *Algoritmi* es la forma latina de su nombre. De ella provienen el latín medieval *algorismus*, que designaba la aritmética con numerales decimales y el castellano antiguo *alguarismo*, del cual deriva *guarismo*. La forma actual de la palabra se habría fijado después, por influencia del griego *arithmós* (número). La Real Academia Española propone una etimología alternativa: quizá del latín tardío *algobarismus*, abreviación del árabe clásico *ḥisābu lǧubār*, “cálculo mediante cifras arábigas”. Otro libro de al-Khwarismi, dedicado a la resolución sistemática de ecuaciones de primer y segundo grado, dio origen a la palabra *álgebra*.

En 1843, Ada Lovelace publicó una traducción comentada de un artículo de Luigi Menabrea sobre la Máquina Analítica diseñada por Charles Babbage [ML43]. La última de sus notas, la Nota G, describe paso a paso cómo la máquina calcularía los números de Bernoulli. La máquina nunca fue construida y el algoritmo no pudo ejecutarse. Con frecuencia se considera que la Nota G contiene el primer programa de computador y que Lovelace fue la primera programadora; algunos historiadores matizan esta atribución, pues Babbage había preparado antes varios programas para su máquina que nunca publicó [Bro82].

0.5.1. Definición. El propósito intuitivo de un algoritmo es resolver un problema algorítmico mediante una secuencia de pasos que puedan ejecutarse mecánicamente. La siguiente definición precisa qué condiciones debe cumplir esa secuencia de pasos.

Definición 0.5.1

Un *algoritmo* es una secuencia de instrucciones que cumple las siguientes condiciones:

1. su descripción es finita;
2. cada instrucción es precisa y no admite ambigüedad;
3. cada instrucción es una operación elemental, que puede ejecutarse en una cantidad finita de tiempo;

4. para toda entrada admisible, su ejecución termina después de una cantidad finita de pasos y produce una salida.

La primera condición exige que un algoritmo pueda escribirse por completo: aunque resuelva infinitas instancias, su descripción ocupa una cantidad finita de espacio. La segunda condición se refiere a que, en cada momento de la ejecución, debe ser claro qué instrucción sigue y qué hace. Por ejemplo, si una instrucción consiste en sumar dos números enteros, ambos números deben estar definidos en ese momento y la operación de adición debe estar identificada sin confusión posible con otra. La tercera condición excluye instrucciones como “encuentre la solución del problema”, que no son operaciones elementales sino el problema mismo. Qué cuenta como operación elemental depende del modelo de computación, que se precisa en la siguiente subsección. La cuarta condición, la terminación, es la más exigente. Un procedimiento que para alguna entrada admisible se ejecuta indefinidamente no es un algoritmo. Esta restricción excluye programas útiles que no están pensados para terminar, como un sistema operativo o un servidor web, pero es la adecuada para el estudio de este texto, en el cual el objetivo de un algoritmo es calcular una respuesta. Demostrar que un algoritmo termina es parte de demostrar que es correcto, como se verá en la Sección 0.7.

La Definición 0.5.1 no restringe la forma de las entradas ni de las salidas: un algoritmo puede recibir cero, uno o más datos de entrada y producir uno o más resultados, siempre que la salida quede determinada por la entrada.

0.5.2. Modelo de computación. La Definición 0.5.1 depende de qué se considere una operación elemental. Antes de que existieran los computadores, varios matemáticos se ocuparon de precisar la noción de “calcular mecánicamente”. En la década de 1930, Alan Turing propuso las máquinas que hoy llevan su nombre y Alonzo Church propuso el cálculo lambda, entre otros modelos. Todos los modelos razonables que se han propuesto resultaron equivalentes: calculan exactamente las mismas funciones.

La *tesis de Church–Turing* afirma que cualquier procedimiento que intuitivamente pueda llamarse mecánico puede ser realizado por una máquina de Turing. No es un teorema, pues la noción de “procedimiento intuitivamente mecánico” no es un objeto matemático, pero hay consenso en aceptarla. Una consecuencia práctica es que se puede escoger como modelo de computación cualquier formalismo equivalente a las máquinas de Turing, sin perder generalidad. Este texto escoge uno que resulte cómodo para diseñar y analizar algoritmos.

Nota 0.5.2

El modelo de computación de este texto es el lenguaje de programación Python, idealizado: se supone que la memoria disponible y la profundidad de la recursión no tienen límite. Las operaciones elementales son las operaciones básicas del lenguaje, cuyo costo se precisa en el Capítulo 1.

Con este modelo, un algoritmo se puede escribir como una función de Python. La elección no es una restricción: por la tesis de Church–Turing, lo que se puede calcular en Python idealizado es lo mismo que se puede calcular en cualquier otro modelo razonable. El Capítulo 6 desarrolla este argumento con detalle.

Conviene distinguir entre un programa y un algoritmo. Un *programa* de Python puede no terminar para alguna entrada; un algoritmo, por la cuarta condición de la Definición 0.5.1, termina siempre. Todo algoritmo puede escribirse como un programa, pero no todo programa es un algoritmo. Los Capítulos 6 a 8 trabajan precisamente con programas que pueden no terminar.

0.5.3. Resolver un problema. La Definición 0.5.1 no menciona problemas algorítmicos, a pesar de que la finalidad de un algoritmo en este texto es resolverlos. La siguiente definición establece la relación entre ambas nociones, apoyándose en la noción de solución de una instancia de la Definición 0.3.5.

Definición 0.5.2

Sean P un problema algorítmico y A un algoritmo. Se dice que A *resuelve* P si y solo si, para cada instancia de P , la ejecución de A termina y produce una solución de esa instancia.

La Definición 0.5.2 no admite excepciones: un algoritmo que produce respuestas correctas para casi todas las instancias, pero no para alguna, no resuelve el problema. Esta exigencia es razonable gracias a que, como se discutió en la Sección 0.3, una especificación bien hecha indica qué responder incluso cuando no existe una salida con las características deseadas; así, toda instancia tiene al menos una solución. Cuando una instancia tiene varias soluciones, basta con que el algoritmo produzca una cualquiera de ellas.

La noción de resolver un problema no depende del lenguaje en el cual se describe el algoritmo ni del lenguaje de programación en el cual se implementa. Tampoco garantiza que todo problema pueda resolverse: hay problemas bien especificados que ningún algoritmo resuelve, como el problema de la parada mencionado en los ejercicios de la Sección 0.3. Este es el tema del Capítulo 6.

El siguiente ejemplo presenta un algoritmo para el problema de ordenamiento, descrito en lenguaje natural. Este algoritmo, conocido como *ordenamiento por selección*, es el ejemplo recurrente de las secciones 0.7 y 0.9.

Ejemplo 0.5.1

Considere el problema de ordenamiento de un arreglo $A[0..N)$ de números enteros, especificado en el Ejemplo 0.3.1. A continuación se presenta un algoritmo especificado en lenguaje natural que “permite” resolver el problema:

1. Para $n = 0, 1, \dots, N - 1$, repita la siguiente secuencia de pasos:
 - a) sea i un índice de A tal que $n \leq i < N$ y $A[i]$ sea mínimo entre $A[n], \dots, A[N - 1]$.
 - b) intercambie $A[n]$ y $A[i]$.
2. Termine; el resultado queda en A .

La Figura 4 muestra la ejecución del algoritmo sobre el arreglo $A = [2, 7, 1, 9, 8]$. En cada iteración, la región $A[0..n)$ ya ordenada crece en un elemento: el mínimo de la región restante $A[n..N)$ se lleva a la posición n .

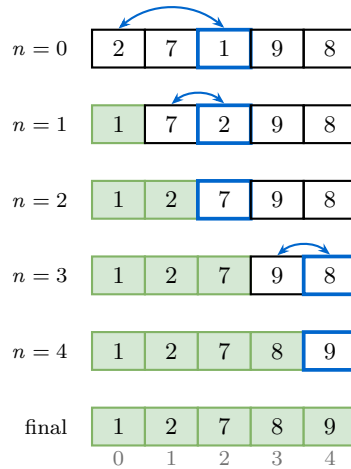


Figura 4. Ejecución del ordenamiento por selección sobre $A = [2, 7, 1, 9, 8]$. Cada fila muestra el arreglo al comienzo de la iteración n : la región $A[0..n)$ aparece en verde y el mínimo de $A[n..N)$ con borde azul. La flecha indica el intercambio de $A[n]$ con ese mínimo, que se omite cuando el mínimo ya está en la posición n .

Aparentemente, el algoritmo del Ejemplo 0.5.1 ordena cualquier arreglo de números y la Figura 4 parece confirmarlo. Sin embargo, una ejecución sobre un arreglo particular no es una certeza matemática de que funcione para *todos*; por

eso se usa la palabra “permite” entre comillas. No es suficiente con creer, es necesario demostrar. La Sección 0.7 explica por qué probar el algoritmo con algunas entradas no basta y la Sección 0.9 demuestra que el algoritmo es correcto.

Ejercicios

1. Elabore un pequeño resumen de las principales contribuciones de Donald Knuth a la informática y las matemáticas, identificando al menos cinco de sus principales manuscritos (e.g., libros y artículos).
2. Investigue sobre la vida de Muhammad ibn Musa al-Khwarismi y explique el origen de su nombre. Además, liste al menos cinco de sus principales aportes a la ciencia.
3. Investigue sobre el algoritmo de la Nota G de Ada Lovelace para la Máquina Analítica de Charles Babbage y explique, a un alto nivel, su diseño.
4. Explique cuáles son las partes de una máquina de Turing y cómo se ejecutan algoritmos en ella. Proponga una máquina de Turing corta que realice alguna tarea sencilla y explique su funcionamiento.
5. Considere el algoritmo propuesto en el Ejemplo 0.5.1 para ordenar ascendente un arreglo de números. Identifique al menos una fuente de ambigüedad en su descripción y proponga cómo corregir este defecto.
6. Considere la siguiente variante del algoritmo del Ejemplo 0.5.1:
 - a) Si $N \geq 2$, entonces repita, para $n = 0, 1, \dots, N - 2$, la siguiente secuencia de pasos:
 - 1) sea i el índice de un valor mínimo en $A[n..N]$;
 - 2) intercambie $A[n]$ y $A[i]$.
 - b) Termine; el resultado queda en A .

Explique por qué parece razonable que esta variante también ordene el arreglo y proponga cómo demostrar que ambos algoritmos resuelven el problema de ordenamiento. Retome este ejercicio después de estudiar la Sección 0.9.

7. Considere el problema de ordenamiento del Ejemplo 0.3.1. Suponga que hay un algoritmo que, dado $A[0..N]$, deja su contenido ordenado en un arreglo nuevo $B[0..N]$, el cual retorna como resultado, sin modificar A . ¿Resuelve dicho algoritmo el problema del Ejemplo 0.3.1? Justifique su respuesta.
8. Considere el siguiente procedimiento, que recibe un número natural $n \geq 1$: mientras $n \neq 1$, si n es par reemplace n por $n/2$ y, si es impar, reemplace n por $3n + 1$; al final, retorne *true*. Investigue sobre la conjetura de Collatz y explique por qué no se sabe si este procedimiento es un algoritmo según la Definición 0.5.1.

9. Considere el problema de la versión de decisión de la suma de subconjuntos del Ejemplo 0.4.2 y el siguiente procedimiento: sume todos los elementos de A y responda *true* si y solo si la suma es mayor o igual que X . Muestre una instancia para la cual el procedimiento produce una solución y otra para la cual no. ¿Resuelve el procedimiento el problema?

0.6. Especificación de algoritmos

En este texto se presentan técnicas de solución de problemas que resultan en algoritmos, recursivos o iterativos. Independientemente de la forma que tome una solución, se debe contar con un lenguaje suficientemente expresivo para escribirla y conciso para entenderla. Para ello existen numerosas opciones, desde lenguaje natural (e.g., el que usamos para comunicarnos verbalmente) hasta lenguajes matemáticos o de programación con una semántica muy precisa y clara. Cada opción viene con beneficios e inconvenientes. Por ejemplo, una ventaja del lenguaje natural es su fácil descripción; el precio que se paga es, como tal, la posibilidad de involuntariamente llegar a una descripción ambigua dada la inherente ambigüedad del lenguaje natural. Una opción que contrarrestaría esa posible ambigüedad sería optar por un lenguaje de programación en lugar del lenguaje natural. Sin embargo, cada lenguaje de programación tiene su propia semántica (e.g., el comando **for** en Python y en Java puede ser interpretado de forma diferente) y esto obliga al lector a aprenderlo. Como estos, hay más ejemplos de beneficios e inconvenientes, animados por un sinnúmero de discusiones. En conclusión, en este texto se reconoce que no hay una solución perfecta, que convenga a cualquier tipo de lector, para el problema de cómo especificar algoritmos.

Por las razones expuestas anteriormente, se usarán varios *lenguajes* de especificación de algoritmos. Dependiendo del contexto, se podrá usar uno u otro. El compromiso del autor con los lectores es el siguiente: en ningún caso habrá ambigüedades ni especificaciones incompletas, sin importar el lenguaje que se use. Eso sí, el espectro de posibilidades se restringe a tres opciones, como se anuncia en la Nota 0.6.1.

0.6.1. Tres lenguajes. Ninguno de los tres lenguajes es mejor que los otros en todas las situaciones. Lo que decide cuál usar es el propósito de la descripción: presentar una idea, razonar sobre ella o ejecutarla.

Nota 0.6.1

En este texto se usan los siguientes lenguajes para especificar un algoritmo, cada uno con el propósito indicado:

- *Enumeraciones estructuradas*, para presentar una idea por primera vez, cuando el detalle técnico estorba, como en el Ejemplo 0.5.1.
- *Notación matemática*, para definir, derivar y demostrar, con las convenciones de las secciones 0.1 y 0.2.
- *El lenguaje de programación Python*, cuando el algoritmo debe poder ejecutarse o cuando los detalles de la implementación importan.

En las enumeraciones estructuradas se usa lenguaje natural (e.g., castellano) en combinación con notación matemática sencilla, si es necesario. En estos casos, los pasos de un algoritmo se describen con frases de palabras, que algunas veces contienen fórmulas. A pesar de ser una opción muy intuitiva, se ha de propender por ser preciso con las palabras porque de lo contrario describir un algoritmo podría fácilmente resultar en pasos ambiguos, como usualmente sucede en las recetas de cocina.

Usar pseudo-código o código puede ayudar a eliminar fuentes de ambigüedad gracias a la sintaxis técnica de los lenguajes de programación, como se explicó anteriormente. De esta forma, un algoritmo es o está muy cerca de ser un programa de computador. La relativa desventaja con esta opción es que la utilidad del algoritmo está supeditada a la semántica del lenguaje de programación elegido y se puede perder –entre tanto detalle técnico– la idea primordial detrás de la solución que encarna. Por ello, a lo largo de este texto se usará esta opción siempre y cuando la especificación del algoritmo no resulte en demasiado detalle técnico o sea extremadamente dependiente de la semántica de Python.

Finalmente, al usar notación matemática se elimina de raíz el problema de la ambigüedad, pero la especificación puede resultar extraña a los ojos poco entrenados o desconocedores de la notación elegida.

0.6.2. Un mismo algoritmo en tres lenguajes. La manera más directa de comparar los tres lenguajes es describir con cada uno el mismo algoritmo. El Ejemplo 0.5.1 ya presentó el ordenamiento por selección como una enumeración estructurada. A continuación se describe en notación matemática y en Python.

En notación matemática, el ciclo se escribe con la variable que lo recorre, la asignación de un índice de un valor mínimo –escrito con el cuantificador de la Sección 0.2– y el intercambio, que se denota con una asignación simultánea.

Algoritmo 0.6.1: Ordenamiento por selección

```

para  $n := 0, 1, \dots, N - 1$  :
     $i :=$  un índice con  $n \leq i < N$  y  $A[i] = (\downarrow j \mid n \leq j < N : A[j])$ 
     $A[n], A[i] := A[i], A[n]$ 

```

La descripción es corta porque delega en la notación de las secciones 0.1 y 0.2 todo lo que no es esencial. En particular, no dice cómo se encuentra el índice i : la expresión cuantificada establece qué valor debe tener, no cómo calcularlo. Al implementar el algoritmo, ese cálculo debe escribirse explícitamente, como sugiere la Sección 0.2.

Implementación 0.6.1: Ordenamiento por selección

```

1 def index_of_min(A, a, b):
2     # pre:  $0 \leq a < b \leq \text{len}(A)$ 
3     i, j = a, a + 1
4     while j < b:
5         if A[j] < A[i]:
6             i = j
7             j = j + 1
8     return i
9
10 def selection_sort(A):
11     N, n = len(A), 0
12     while n < N:
13         i = index_of_min(A, n, N)
14         A[n], A[i] = A[i], A[n]
15         n = n + 1

```

La implementación es más larga que las otras dos descripciones, pero puede ejecutarse. Muestra, además, decisiones que las otras dos ocultan: que el índice del mínimo se calcula con un recorrido, que `index_of_min` supone $a < b$ y que `selection_sort` modifica el arreglo recibido y no retorna nada, de acuerdo con la especificación *in situ* del Ejemplo 0.3.1. La suposición $a < b$ está escrita como un comentario y no se verifica en el código, por la razón que se explica a continuación.

Ninguna de las tres descripciones es más correcta que las otras: describen el mismo algoritmo con distintos niveles de detalle. Lo que cambia es qué se deja implícito y, por lo tanto, qué es fácil de ver en cada una.

0.6.3. Convenciones de código. El código de este texto sigue unas pocas convenciones, para que el lector pueda concentrarse en el algoritmo y no en el estilo.

- Los identificadores y los comentarios se escriben en inglés, como es usual en la programación.
- Las listas de Python cumplen el papel de arreglos, con índices desde 0, de acuerdo con la Sección 0.1.
- La constante INF, definida como `float("inf")`, representa el valor $+\infty$ del rango vacío de la Nota 0.2.2.
- Los ciclos tienen un único punto de salida. Cuando hay salida temprana, se usa un centinela en la condición del ciclo, como en la Implementación 0.2.2, en lugar de retornar desde adentro.
- El nombre de una función describe lo que calcula, no cómo lo calcula.
- Cada función supone su precondition y no la verifica. La precondition se escribe como un comentario que empieza con `pre:`, como en la Implementación 0.6.1, y es responsabilidad de quien llama a la función garantizar que se cumple.

Esta última convención es una decisión deliberada y no un descuido. Verificar la precondition en cada llamada duplica trabajo que el algoritmo ya garantiza – en la Implementación 0.6.1, por ejemplo, el ciclo de `selection_sort` solo llama a `index_of_min` con $n < N$ – y llena el código de comprobaciones que oscurecen la idea. El precio es que una llamada que incumple la precondition produce un resultado sin sentido o un error de ejecución. En un programa destinado a usuarios finales, en el cual los datos pueden venir de cualquier parte, la decisión sería probablemente la contraria. Aquí las preconditiones se demuestran, que es más fuerte que verificarlas: es el tema de la Sección 0.7.

Falta una convención más: qué retorna una función. La respuesta depende del tipo del problema que resuelve, en el sentido de la Sección 0.4, y se resume en la Tabla 4.

Tipo de problema	Valor retornado
Decisión	un bool
Búsqueda	el testigo o None si no existe
Optimización (testigo)	el testigo óptimo o None si no existe
Optimización (valor)	un número, INF si no hay testigos
Conteo	un int, que es 0 si no hay testigos
Enumeración	una lista, vacía si no hay testigos

Tabla 4. Qué retorna la implementación de un algoritmo, según el tipo del problema que resuelve.

Estas convenciones tienen una virtud adicional: hacen que el valor retornado distinga siempre el caso en el que no hay testigos, sin necesidad de un mecanismo especial. Un algoritmo cuya implementación retorna **None** está indicando, en los términos de la Sección 0.3, que la instancia no admite una salida con las características deseadas.

Ejercicios

1. Investigue sobre el formalismo conocido como *diagramas de flujo* y úselo para describir el algoritmo en el Ejemplo 0.5.1.
2. Investigue sobre el formalismo conocido como *lenguaje de comandos guardados* (en inglés, *guarded command language*) y úselo para describir el algoritmo en el Ejemplo 0.5.1.
3. Recuerde el problema de buscar un número en un arreglo de números ordenado ascendentemente (Ejercicio 0.3.3). Para este problema se conoce una solución llamada “búsqueda binaria”. Investigue sobre esta solución y:
 - a) Especifique el algoritmo de búsqueda binaria en lenguaje natural para resolver el problema dado.
 - b) Especifique el algoritmo de búsqueda binaria en diagramas de flujo para resolver el problema dado (ver Ejercicio 1).
 - c) Especifique el algoritmo de búsqueda binaria en el lenguaje de comandos guardados para resolver el problema dado (ver Ejercicio 2).
 - d) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Python para resolver el problema dado.
 - e) Especifique el algoritmo de búsqueda binaria en un lenguaje de programación funcional, como Haskell o Scala.
 - f) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Brainfuck.

- g) Compare las especificaciones anteriores. ¿Cuál hace más evidente la idea del algoritmo? ¿Cuál hace más evidente su costo?
- 4. Considere el problema de calcular la descomposición en factores primos de un número natural. Una aproximación para resolver el problema es la construcción de la Criba de Eratóstenes, para luego factorizar el número dado.
 - a) Investigue sobre la Criba de Eratóstenes. ¿Cuáles son las operaciones básicas en su construcción? Explique su respuesta.
 - b) Especifique el problema de construir la Criba de Eratóstenes en un arreglo $A[0..N)$ para los primeros N números naturales.
 - c) Proponga un algoritmo para resolver el problema dado, utilizando el lenguaje de especificación de su preferencia.
 - d) ¿Cómo puede garantizar que la solución propuesta es un algoritmo y funciona?
- 5. Considere la Implementación 0.6.1.
 - a) ¿Qué ocurre al evaluar `index_of_min(A, a, b)` cuando $a = b$? Explique por qué `selection_sort` nunca produce esa llamada.
 - b) Modifique `selection_sort` para que, en lugar de modificar el arreglo recibido, retorne un arreglo nuevo con los mismos elementos ordenados.
- 6. La Tabla 4 propone retornar **None** cuando un problema de búsqueda no tiene testigos. Otra convención posible es retornar un valor especial del mismo tipo que los testigos, como el índice -1 en una búsqueda sobre un arreglo. Compare las dos convenciones e indique qué ventajas y qué riesgos tiene cada una.

0.7. Corrección de algoritmos

La Sección 0.5 definió qué significa que un algoritmo resuelva un problema: que para cada instancia termine y produzca una solución. Falta responder cómo se sabe que un algoritmo dado cumple esa condición. Esta sección precisa qué hay que demostrar, explica por qué ejecutar el algoritmo sobre algunas entradas no basta e introduce la herramienta con la cual se razona sobre los ciclos.

0.7.1. Corrección parcial y total. Demostrar que un algoritmo resuelve un problema exige dos cosas distintas: que las respuestas que produce sean correctas y que efectivamente produzca una respuesta. Conviene separarlas.

Definición 0.7.1

Sean P un problema algorítmico y A un algoritmo. Se dice que A es *parcialmente correcto* con respecto a P si y solo si, para cada instancia de P , en caso de que la ejecución de A termine, su salida es una solución de esa instancia. Se

dice que A es *totalmente correcto* con respecto a P si y solo si es parcialmente correcto y, además, su ejecución termina para cada instancia de P .

La corrección total es exactamente la noción de la Definición 0.5.2: un algoritmo resuelve un problema si y solo si es totalmente correcto con respecto a él. La corrección parcial, por sí sola, no dice gran cosa: un procedimiento que nunca termina es parcialmente correcto con respecto a cualquier problema, pues la condición “en caso de que la ejecución termine” nunca se cumple. Por eso ambas mitades son necesarias.

La ventaja de separarlas es que cada una se demuestra con una herramienta distinta. La corrección parcial se razona con invariantes, en los algoritmos iterativos, o con inducción, en los recursivos. La terminación se razona exhibiendo una cantidad que disminuye y no puede disminuir indefinidamente. Las dos herramientas son el tema de las secciones 0.8 y 0.9; esta sección introduce la primera y explica por qué hace falta.

0.7.2. Por qué probar no es suficiente. La forma más inmediata de ganar confianza en un algoritmo es ejecutarlo sobre algunas entradas y comparar los resultados con los esperados. Esta práctica es indispensable en la construcción de software, pero tiene un límite de principio: un conjunto de pruebas examina una cantidad finita de instancias, mientras que la mayoría de los problemas algorítmicos tiene infinitas, como se discutió en la Sección 0.3. Aun cuando todas las pruebas pasen, quedan sin examinar infinitas instancias.

El siguiente ejemplo muestra que esta limitación no es solo teórica.

Ejemplo 0.7.1

Considere la siguiente variante de la Implementación 0.2.2, que pretende verificar si un arreglo está ordenado ascendentemente en el sentido de la Definición 0.2.3:

```
def is_sorted(A):  
    ans, i = True, 1  
    while i < len(A) and ans:  
        ans = A[i-1] < A[i]  
        i = i + 1  
    return ans
```

La única diferencia está en la línea 4: la comparación es $<$ en lugar de $\leq$. Considere ahora el siguiente conjunto de pruebas, que examina el arreglo vacío, un arreglo de un elemento, arreglos ordenados y desordenados, y valores

negativos:

$[], [5], [1, 2, 3], [3, 1, 2], [2, 7, 1, 9, 8], [-4, 0, 7]$.

El procedimiento produce la respuesta correcta en las seis pruebas. Sin embargo, sobre el arreglo $[1, 1]$ responde *false*, cuando la respuesta correcta es *true*: un arreglo con valores repetidos puede estar ordenado.

El conjunto de pruebas del Ejemplo 0.7.1 es razonable y, aun así, no toca el único caso en el que el procedimiento falla. Se puede objetar que un conjunto de pruebas mejor habría incluido valores repetidos, y es cierto; pero esa objeción solo puede hacerse cuando ya se conoce el error. Frente a un error desconocido, no hay manera de saber si el conjunto de pruebas lo cubre. Esta es la observación de Edsger Dijkstra: las pruebas pueden mostrar la presencia de errores, pero nunca su ausencia [Dij72].

La acumulación de evidencia tampoco es prueba, ni siquiera cuando la evidencia es abrumadora.

Ejemplo 0.7.2

Considere la afirmación “ $n^2 + n + 41$ es primo para todo número natural n ”. La afirmación se puede verificar para $n = 0, 1, \dots, 39$ y en todos los casos el resultado es primo. Sin embargo, es falsa: para $n = 40$ se obtiene $40^2 + 40 + 41 = 1681 = 41^2$.

0.7.3. El papel de las pruebas. Nada de lo anterior significa que probar sea inútil. Las pruebas son la manera más económica de encontrar errores y, en la práctica, encuentran muchos. Sirven también para detectar que un cambio dañó algo que antes funcionaba.

Hay, además, una tarea que solo las pruebas cubren. Demostrar la corrección de un algoritmo y escribir un programa que lo implemente son cosas distintas: la demostración habla del algoritmo, no del programa. Un algoritmo demostrado puede implementarse mal. Las pruebas son la forma habitual de ganar confianza en que la implementación corresponde al algoritmo que sí se demostró.

Lo que las pruebas no pueden hacer es sustituir la demostración. De ahí la frase con la que termina la Sección 0.5: no es suficiente con creer, es necesario demostrar.

0.7.4. Invariantes. Demostrar que un ciclo hace lo que se espera presenta una dificultad: la cantidad de iteraciones depende de la entrada, de modo que no se puede razonar sobre “todas las iteraciones” una por una. La salida consiste en encontrar una propiedad que no cambie de una iteración a otra.

Definición 0.7.2

Un *invariante* de un ciclo es una propiedad que se cumple cada vez que se evalúa la condición del ciclo: antes de la primera iteración, entre iteraciones consecutivas y al terminar.

Un invariante es útil cuando permite concluir la poscondición al terminar el ciclo. Establecerlo requiere verificar tres condiciones, que se ilustran en la Figura 5.

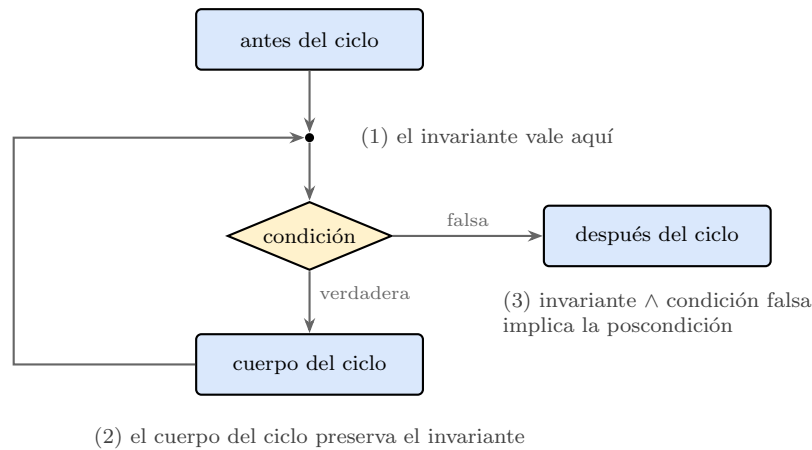


Figura 5. Las tres condiciones del método de invariantes: inicialización, estabilidad y logro. El invariante se evalúa en el punto marcado, justo antes de la condición del ciclo.

Nota 0.7.1: Método de invariantes

Para demostrar que un ciclo establece una poscondición Q , basta con encontrar un invariante I y verificar:

1. *Inicialización:* I se cumple antes de la primera evaluación de la condición del ciclo.
2. *Estabilidad:* si I se cumple y la condición del ciclo es verdadera, entonces I se cumple de nuevo al terminar la iteración.
3. *Logro:* si I se cumple y la condición del ciclo es falsa, entonces Q se cumple.

La inicialización y la estabilidad se ocupan de una sola iteración cada una, sin importar cuántas iteraciones haya. Que esto sea suficiente para concluir que

I se cumple en *todas* las evaluaciones de la condición es un hecho que requiere demostración; la Sección 0.9 la presenta. Observe, además, que el método no dice nada sobre la terminación del ciclo: si el ciclo no termina, el logro nunca se usa.

El primer ejemplo apareció en la Sección 0.2, al traducir un cuantificador a un ciclo.

Ejemplo 0.7.3

Para la Implementación 0.2.1, que calcula $(+i \mid a \leq i < b : A[i])$, la propiedad

$$a \leq i \leq b \wedge acc = (+j \mid a \leq j < i : A[j])$$

es un invariante del ciclo. La inicialización se cumple porque $i = a$ hace vacío el rango y $acc = 0$. La estabilidad se sigue del Teorema 0.2.1: sumar $A[i]$ al acumulador y avanzar i en uno produce de nuevo la igualdad. El logro se obtiene al terminar el ciclo, cuando la condición $i < b$ es falsa: junto con $i \leq b$ se concluye $i = b$ y, por lo tanto, $acc = (+j \mid a \leq j < b : A[j])$, que es la poscondición.

El segundo ejemplo es el ordenamiento por selección del Ejemplo 0.5.1, cuyo invariante tiene tres partes.

Ejemplo 0.7.4

Considere el ciclo del ordenamiento por selección, con variable n . La siguiente propiedad es un invariante del ciclo:

$$\begin{aligned} 0 &\leq n \leq N, \\ A[0..n) &\text{ está ordenado ascendentemente,} \\ (\forall i, j \mid 0 &\leq i < n \leq j < N : A[i] \leq A[j]), \\ A &\text{ es una permutación de } A_0. \end{aligned}$$

La segunda parte dice que la sección ya procesada está ordenada; la tercera, que ninguno de sus elementos supera a los que faltan por procesar; la cuarta, que no se han perdido ni inventado elementos. Las tres corresponden a lo que muestra la Figura 4.

Al terminar el ciclo se tiene $n = N$, de modo que la segunda parte del invariante afirma que $A[0..N)$, es decir, todo el arreglo, está ordenado; junto con la cuarta parte, se obtiene exactamente la poscondición del Ejemplo 0.3.1. La tercera parte no aparece en la poscondición, pero es indispensable: sin ella no se puede demostrar la estabilidad, pues nada garantizaría que el elemento que se agrega a la sección

ordenada no es menor que alguno de los que ya están allí. La demostración completa se presenta en la Sección 0.9.

0.7.5. Terminación. El método de invariantes atiende la corrección parcial. Para la corrección total falta demostrar que el ciclo termina, y para ello se exhibe una cantidad que disminuye en cada iteración y que no puede disminuir indefinidamente.

En el ordenamiento por selección, esa cantidad es $N - n$: cada iteración aumenta n en uno, de modo que $N - n$ disminuye en uno, y el invariante garantiza que $N - n \geq 0$. Una cantidad entera no negativa no puede disminuir indefinidamente, luego el ciclo termina. El mismo argumento, con la cantidad $b - i$, demuestra la terminación del ciclo de la Implementación 0.2.1.

En los algoritmos recursivos el argumento es análogo: se exhibe una cantidad que disminuye en cada llamada recursiva, como en el Capítulo 2. La noción general de “cantidad que no puede disminuir indefinidamente” es la de relación bien fundada, que se presenta en la Sección 0.8.

Ejercicios

1. Proponga un invariante para el ciclo de la función `index_of_min` de la Implementación 0.6.1 y verifique las tres condiciones de la Nota 0.7.1. Proponga también una cantidad que demuestre su terminación.
2. Proponga un invariante para el ciclo de la Implementación 0.2.2 y verifique las tres condiciones. Sugerencia: el invariante debe relacionar el valor de `ans` con una expresión cuantificada cuyo rango dependa de `i`.
3. Considere el procedimiento del Ejemplo 0.7.1.
 - a) Diseñe un conjunto de pruebas que detecte el error.
 - b) ¿Demuestra ese conjunto de pruebas que el procedimiento corregido es correcto? Justifique su respuesta.
 - c) Identifique cuál de las tres condiciones de la Nota 0.7.1 falla al intentar demostrar la corrección del procedimiento con el invariante del Ejercicio 2.
4. Escriba un procedimiento que sea parcialmente correcto con respecto al problema de ordenamiento del Ejemplo 0.3.1 pero que no sea totalmente correcto. ¿Es un algoritmo, en el sentido de la Definición 0.5.1?
5. Retome el procedimiento de Collatz de los ejercicios de la Sección 0.5, que responde *true* para todo número natural $n \geq 1$. Argumente que es parcialmente correcto con respecto al problema cuya salida es siempre *true*. ¿Qué haría falta para demostrar que es totalmente correcto?

6. Un conjunto de pruebas exhaustivo examina todas las instancias de un problema. Considere el problema de ordenar un arreglo de tamaño 3 cuyos elementos son números enteros representables en 64 bits. ¿Cuántas instancias tiene? Estime cuánto tomaría examinarlas todas a razón de mil millones de instancias por segundo.
7. Considere un ciclo cuyo invariante es la propiedad “true”. Verifique que la inicialización y la estabilidad se cumplen siempre. ¿Qué dice esto sobre la utilidad de un invariante?

0.8. Relaciones bien fundadas e inducción

El concepto de relación hace parte de nuestro lenguaje cotidiano, además de ser fundamental en el desarrollo de la informática y las matemáticas. Es difícil pensar en establecer una conversación sin necesidad de invocar una relación. Cuando alguien dice “soy bogotano”, esa persona está declarando una relación muy común que asigna a cada persona su ciudad de nacimiento.

El objetivo de esta sección es presentar los conceptos de relación y algunas propiedades como orden y buena fundación, para finalizar con un teorema de inducción sobre relaciones bien fundadas y su aplicación a la terminación de algoritmos. Con ello se saldan las dos deudas que dejó la Sección 0.7: precisar qué es una cantidad que no puede disminuir indefinidamente y contar con la herramienta con la cual se justifica el método de invariantes.

0.8.1. Relación e imagen.

Definición 0.8.1: Relación

Sean A y B conjuntos. Una *relación (binaria) de A en B* es un conjunto de parejas ordenadas en $A \times B$, es decir, R es una relación binaria de A en B si y solo si $R \subseteq A \times B$. Si $R \subseteq A \times B$, entonces A es llamado el *conjunto de partida* (u *origen*) de R y B el *conjunto de llegada* (o *destino*) de R . Además, si $(a, b) \in A \times B$, entonces se puede denotar $(a, b) \in R$ como $a R b$ y $(a, b) \notin R$ como $a \not R b$.

Note que reducir el tratamiento a relaciones binarias en la Definición 0.8.1 no es en realidad una restricción, dado que el producto cartesiano de una colección de conjuntos es en sí un conjunto. Es decir, dada una colección de conjuntos $A_1, \dots, A_n$, el producto cartesiano $A = A_1 \times \dots \times A_n$ es un conjunto y una relación $R \subseteq A \times B$ sería simplemente una relación $R \subseteq (A_1 \times \dots \times A_n) \times B$.

Dada una relación $R \subseteq A \times B$, es posible definir su relación inversa $R^{-1} \subseteq B \times A$ como el conjunto

$$R^{-1} = \{(b, a) \in B \times A \mid a R b\}.$$

La idea de la relación inversa es común en el lenguaje cotidiano en donde, por ejemplo, la relación “ser nieto(a) de” es la relación inversa de “ser abuelo(a) de”. En algunas ocasiones la relación inversa R^{-1} es sugestivamente denotada por la imagen reflejada $\mathfrak{A}$ del símbolo R . Por ejemplo, $>^{-1}$ sobre $\mathbb{N}$ es denotada por $<$ y $\geq^{-1}$ por $\leq$.

La imagen de un conjunto bajo una relación es exactamente la colección de elementos en el conjunto de llegada que están relacionados con al menos un elemento del conjunto dado. Note que esta construcción tiene sentido únicamente cuando el conjunto dado está contenido en el conjunto de partida de la relación.

Definición 0.8.2: Imagen

Sean A y B conjuntos, y sea $R \subseteq A \times B$. Si $C \subseteq A$, entonces la *imagen* de C bajo R , denotada $R[C]$, es el conjunto definido por

$$R[C] = \{b \in B \mid (\exists x \in C : x R b)\}.$$

Por ejemplo, si $\mathbb{N}$ denota el conjunto de números naturales, $>$ denota la relación “mayor que” y $\geq$ la relación “mayor o igual que” sobre $\mathbb{N}$, entonces

$$\begin{aligned} >[\{4\}] &= \{0, 1, 2, 3\}, \\ \geq[\{4\}] &= \{0, 1, 2, 3, 4\}, \\ >[\mathbb{N}] &= \mathbb{N}. \end{aligned}$$

0.8.2. Orden. Un orden sobre un conjunto A es una relación $R \subseteq A \times A$ que determina cierta “estructura organizacional” entre los elementos de A .

Definición 0.8.3: Orden

Sean A un conjunto y $R \subseteq A \times A$. La relación R es llamada un *orden (estricto)* sobre A si y solo si R es transitiva e irreflexiva. Un *conjunto ordenado* es un conjunto con un orden sobre él, es decir, un par (A, R) con R un orden sobre A .

Ejemplos de conjuntos ordenados están en todas partes. La relación $<$ sobre los naturales $\mathbb{N}$, los enteros $\mathbb{Z}$, los racionales $\mathbb{Q}$ y los reales $\mathbb{R}$ hace de estos conjuntos conjuntos ordenados. De la misma forma, la relación de inclusión estricta $\subset$

sobre conjuntos hace de cualquier conjunto potencia $\wp(A)$ un conjunto ordenado $(\wp(A), \subset)$.

Dado un conjunto ordenado $(A, <)$, la clausura reflexiva $<^=$ de su orden (estricto) $<$ *siempre* se denota por $\preceq$. En general, recuerde que la *clausura reflexiva* $R^=$ de una relación $R \subseteq A \times A$ es la relación $R \cup id_A$, en donde $id_A = \{(a, a) \mid a \in A\}$. Note que gracias a la irreflexividad de $<$ los mapas definidos por

$$\begin{aligned} < &\mapsto \preceq && (\text{en donde } \preceq = < \cup id_A) \\ \preceq &\mapsto < && (\text{en donde } < = \preceq - id_A) \end{aligned}$$

son inversos el uno del otro, de tal modo que se puede obtener el orden estricto $<$ de su versión no estricta $\preceq$ y viceversa. De esta forma se puede hablar de manera equivalente de un conjunto ordenado como un par $(A, <)$ o como un par $(A, \preceq)$, al especificar el orden estricto $<$ o su clausura reflexiva $\preceq$.

Algunas veces un conjunto ordenado $(A, <)$ es llamado *parcialmente ordenado* o *poset* de manera abreviada. Esto se hace para enfatizar que al menos una pareja de elementos en $(A, <)$ puede ser *incomparable*. Por ejemplo, en el poset $(\wp(\{0, 1\}), \subset)$ los conjuntos unitarios $\{0\}$ y $\{1\}$ son incomparables porque $\{0\} \not\subset \{1\}$ y $\{1\} \not\subset \{0\}$. En contraste, en el poset $(\mathbb{N}, <)$ cualquier par de elementos es comparable. Un conjunto ordenado en donde cualquier par de elementos es comparable recibe el nombre de *conjunto linealmente ordenado* (o *conjunto totalmente ordenado* o *cadena*) y su relación de orden recibe el nombre de *orden lineal* (u *orden total*).

Definición 0.8.4: Orden lineal

Un poset $(A, <)$ es llamado *linealmente ordenado* si y solo si satisface la siguiente fórmula:

$$(\forall x, y \in A \mid x \neq y : x < y \vee x > y).$$

Una consecuencia de la “parcialidad” de los conjuntos que no están linealmente ordenados es que, en general, estos tienen elementos que *no* están precedidos por ningún otro elemento (en el conjunto) o que *no* son sucesores de ningún otro elemento (en el conjunto). Por ejemplo, en el poset $(\wp(\{0, 1, 2\}) - \{\{0, 1, 2\}\}, \subset)$, el conjunto $\emptyset$ es exactamente ese elemento que no es precedido por ningún otro elemento en el poset, mientras que los conjuntos $\{0, 1\}$, $\{0, 2\}$ y $\{1, 2\}$ son exactamente esos elementos que no son sucesores de ningún otro elemento en el poset. En un poset, aquellos elementos que no están precedidos por ningún elemento reciben el nombre de *minimales* y aquellos que no son sucesores de ningún otro elemento el nombre de *maximales*.

Definición 0.8.5: Minimal y maximal

Sean A un conjunto y $R \subseteq A \times A$. Entonces:

1. un elemento $a \in A$ es llamado *R-minimal en A* si y solo si satisface la fórmula $(\forall x \in A \mid \neg(x R a))$;
2. un elemento $a \in A$ es llamado *R-maximal en A* si y solo si satisface la fórmula $(\forall x \in A \mid \neg(a R x))$.

Note que la definición de elementos minimales y maximales no supone que R sea un orden sobre A , es decir, esta es una definición general. Sin embargo, las discusiones sobre elementos minimales y maximales sucederán frecuentemente cuando A y R forman un poset (A, R) . En este último caso y cuando $a \in A$ sea un elemento *R-minimal en A* (resp., *R-maximal en A*) se dirá que a es *minimal en* (A, R) (resp., *maximal en* (A, R)).

0.8.3. Buena fundación. Una relación bien fundada sobre un conjunto es, operacionalmente hablando y como su nombre lo indica, una relación que termina. Visto desde un punto de vista más general, una relación bien fundada es una relación que no admite cadenas descendientes infinitas. La importancia de este tipo de relaciones tanto en la informática como en la matemática es indiscutible. Este tipo de relación es el fundamento, por ejemplo, de la inducción, de la recurrencia, de la iteración y de la terminación de algoritmos.

Definición 0.8.6: Buena fundación

Sean A un conjunto y $R \subseteq A \times A$. La pareja (A, R) es *bien fundada* si y solo si cualquier conjunto no vacío $B \subseteq A$ es tal que tiene un elemento minimal en (B, R) . Es decir, (A, R) es bien fundada si y solo si A y R satisfacen la siguiente fórmula:

$$(\forall B \subseteq A \mid B \neq \emptyset : (\exists x \in B \mid x \text{ es minimal en } (B, R))).$$

El siguiente resultado es inmediato a partir de la definición y se usa en la demostración del principio de inducción.

Teorema 0.8.1

Sean A un conjunto y $R \subseteq A \times A$. Si (A, R) es bien fundada y $B \subseteq A$, entonces (B, R) es bien fundada.

Demostración

Sea $C \subseteq B$ no vacío. Como $B \subseteq A$, también $C \subseteq A$ y, por la Definición 0.8.6, C tiene un elemento minimal en (C, R) . Como la condición de ser minimal en (C, R) no depende de A ni de B , ese mismo elemento atestigua lo pedido.

Los siguientes ejemplos ilustran la definición y, sobre todo, lo que la buena fundación *no* exige.

Ejemplo 0.8.1

- $(\mathbb{N}, <)$ es bien fundada: todo conjunto no vacío de números naturales tiene un elemento mínimo, que en particular es minimal.
- $(\mathbb{Z}, <)$ no es bien fundada: el propio $\mathbb{Z}$ no tiene elemento minimal.
- $(\mathbb{R}_{\geq 0}, <)$ no es bien fundada, a pesar de que todos sus elementos son mayores o iguales que 0: el conjunto $\{1/2^n \mid n \in \mathbb{N}\}$ no tiene elemento minimal. Estar acotado por debajo no es suficiente.
- La relación “ser prefijo propio de” sobre las cadenas de un alfabeto es bien fundada, pues cada paso reduce la longitud.
- El orden lexicográfico sobre $\mathbb{N} \times \mathbb{N}$, definido por $(a, b) \prec (c, d)$ si y solo si $a < c$, o bien $a = c$ y $b < d$, es bien fundado. Es el orden con el cual se demuestra la terminación de ciclos anidados.

La lectura operativa de la buena fundación es la ausencia de cadenas descendientes infinitas, es decir, de sucesiones $x_0 \succ x_1 \succ x_2 \succ \dots$ en las cuales cada elemento es estrictamente menor que el anterior.

Teorema 0.8.2

Sean A un conjunto y $R \subseteq A \times A$. Si (A, R) es bien fundada, entonces no existe una sucesión infinita $x_0, x_1, x_2, \dots$ de elementos de A tal que $x_{n+1} R x_n$ para todo $n \in \mathbb{N}$.

Demostración

Hacia una contradicción, suponga que existe tal sucesión y sea $B = \{x_n \mid n \in \mathbb{N}\}$, que no es vacío. Por la Definición 0.8.6, B tiene un elemento minimal en

(B, R) , digamos x_k . Pero $x_{k+1} \in B$ y $x_{k+1} R x_k$, lo cual contradice que x_k sea minimal.

El recíproco del Teorema 0.8.2 también es cierto, pero su demostración requiere construir una cadena descendiente infinita escogiendo, en cada paso, un elemento entre infinitas posibilidades; formalmente, esto requiere el axioma de elección dependiente. Como la dirección demostrada es la que se usa para argumentar terminación, este texto no entra en ese detalle.

0.8.4. Inducción bien fundada. Como consecuencia de la buena fundación de una relación, se obtiene un principio de inducción muy general conocido como el principio de *inducción bien fundada*.

Teorema 0.8.3: Inducción bien fundada

Sean A un conjunto, $\prec \subseteq A \times A$ y $(A, \prec)$ bien fundada. Si cualquier conjunto $B \subseteq A$ es tal que para cualquier $a \in A$, si $\succ[\{a\}] \subseteq B$ entonces $a \in B$, entonces necesariamente $B = A$. En símbolos, la siguiente fórmula es una tautología:

$$(\forall B \subseteq A : (\forall x \in A \mid \succ[\{x\}] \subseteq B : x \in B) \implies B = A).$$

Demostración

Sea $B \subseteq A$ y suponga que B satisface la fórmula

$$(\forall x \in A \mid \succ[\{x\}] \subseteq B : x \in B).$$

El objetivo es demostrar $B = A$. Hacia una contradicción, suponga que $B \neq A$. Dado que $B \subseteq A$ y $B \neq A$, entonces $B \subsetneq A$ y por ende $A - B \neq \emptyset$. Por el Teorema 0.8.1, $(A - B, \prec)$ es bien fundada y, por definición, tiene un elemento minimal, digamos $a \in A - B$. Hay entonces dos casos:

- Caso $\succ[\{a\}] \subseteq B$: por suposición, $a \in B$, lo cual contradice la pertenencia de a en $A - B$.
- Caso $\succ[\{a\}] \not\subseteq B$: entonces $\succ[\{a\}] - B \neq \emptyset$ (note que $\succ[\{a\}] \subseteq A$). Sea $b \in \succ[\{a\}] - B$. Entonces $b \in \succ[\{a\}]$, lo que implica que $b \prec a$, contradiciendo la minimalidad de a en $(A - B, \prec)$.

En cualquiera de los dos casos hay una contradicción, concluyendo que $B = A$.

El principio de inducción bien fundada es muy general y contiene otros principios de inducción como casos especiales, incluyendo el principio de inducción sobre

números naturales, en donde $(A, \prec) = (\mathbb{N}, <)$, que es el tema de la Sección 0.9. El aspecto clave del principio es que, si $(A, \prec)$ es bien fundada, el principio de inducción es útil para demostrar que cualquier subconjunto $B \subseteq A$ es tal que $B = A$. En particular, si

$$P : A \rightarrow \mathbb{B}$$

es un predicado tal que $B = \{x \in A \mid P(x)\}$, entonces se puede usar el principio de inducción bien fundada de $(A, \prec)$ para tratar de demostrar que todos los elementos de A satisfacen el predicado P (es decir, $B = A$). Escrito con predicados, el principio toma la forma de una regla de inferencia: si se demuestra la premisa, se concluye la conclusión.

$$\frac{(\forall x \in A \mid (\forall y \in A \mid y \prec x : P(y)) \implies P(x))}{(\forall x \in A \mid P(x))}$$

En esta formulación, cualquier elemento a minimal en $(A, \prec)$ satisface $\succ[\{a\}] = \emptyset$ y la obligación de demostración sería la pertenencia de a en B , es decir, que $P(a)$ es cierto para todo elemento a minimal en $(A, \prec)$. Estos son los “casos base” que están implícitos en el principio de inducción bien fundada: en ellos el rango $y \prec a$ de la hipótesis es vacío y, por la Nota 0.2.2, la hipótesis se cumple sin aportar información. Los “casos inductivos” corresponden entonces a todos aquellos elementos b que no son minimales en $(A, \prec)$ y la obligación de demostración sería la pertenencia de b en B suponiendo que $\succ[\{b\}] \subseteq B$. La “hipótesis inductiva” para b es entonces suponer que $\succ[\{b\}] \subseteq B$, es decir, que si $x \in \succ[\{b\}]$, entonces x es tal que $P(x)$ es cierto.

Note que los casos inductivos pueden ser potencialmente demasiados. Afortunadamente, los elementos no minimales en una relación $(A, \prec)$ suelen estar caracterizados por “patrones” que permiten describir estos elementos no minimales en pocos casos sintácticos. Por ejemplo, cualquier elemento n no minimal en $(\mathbb{N}, <)$ satisface el patrón sintáctico $n = m + 1$, para algún $m \in \mathbb{N}$. En este caso, no se menciona explícitamente qué son n o m , sino más bien se caracterizan y relacionan unívocamente gracias a la suma de números naturales, y el caso inductivo $P(n)$ supondría que todos los elementos $\{0, 1, \dots, m\}$ satisfacen la propiedad P que se quiere demostrar.

0.8.5. Medidas. En la práctica rara vez se define una relación bien fundada desde cero. Lo usual es asociar a cada elemento una *medida* en un conjunto que ya se sabe bien fundado, casi siempre $(\mathbb{N}, <)$, y heredar de allí la buena fundación.

Teorema 0.8.4: Medidas

Sean A y C conjuntos, $f : A \rightarrow C$ una función y $(C, \prec_C)$ bien fundada. Defina la relación $\prec_A$ sobre A por

$$a \prec_A a' \text{ si y solo si } f(a) \prec_C f(a').$$

Entonces, $(A, \prec_A)$ es bien fundada.

Demostración

Sea $B \subseteq A$ no vacío. Entonces $f[B] \subseteq C$ no es vacío y, por la buena fundación de $(C, \prec_C)$, tiene un elemento minimal c . Sea $a \in B$ tal que $f(a) = c$. Si existiera $b \in B$ con $b \prec_A a$, entonces $f(b) \prec_C c$, lo cual contradice la minimalidad de c . Luego a es minimal en $(B, \prec_A)$.

0.8.6. Terminación. Con las nociones anteriores se puede formalizar el argumento de terminación que la Sección 0.7 usó de manera informal. La idea es asociar a cada estado de la ejecución una medida y verificar que cada paso la hace decrecer.

Teorema 0.8.5: Terminación

Considere un ciclo y sea S el conjunto de sus estados posibles. Si existen un conjunto bien fundado $(C, \prec)$ y una función $f : S \rightarrow C$ tales que, para cada iteración que lleva del estado s al estado s' , se cumple $f(s') \prec f(s)$, entonces el ciclo termina.

Demostración

Si el ciclo no terminara, su ejecución produciría una sucesión infinita de estados $s_0, s_1, s_2, \dots$ y, por hipótesis, la sucesión $f(s_0), f(s_1), f(s_2), \dots$ sería una cadena descendiente infinita en $(C, \prec)$, lo cual es imposible por el Teorema 0.8.2.

La función f del Teorema 0.8.5 recibe el nombre de *función de cota* o *variante* del ciclo. Para el ordenamiento por selección del Ejemplo 0.5.1, la variante es $f(s) = N - n$ con valores en $(\mathbb{N}, <)$: cada iteración aumenta n en uno y el invariante del Ejemplo 0.7.4 garantiza que $n \leq N$, de modo que $N - n$ es un número natural que decrece. Para la Implementación 0.2.1, la variante es $b - i$.

El mismo criterio se aplica a los algoritmos recursivos, en los cuales la medida debe decrecer en cada llamada recursiva; así se demuestra la terminación de los algoritmos del Capítulo 2. Cuando una sola cantidad no decrece en todos los pasos,

se puede recurrir a medidas con valores en otros conjuntos bien fundados, como el orden lexicográfico del Ejemplo 0.8.1.

Ejercicios

1. Sea A un conjunto con m elementos y B un conjunto con n elementos. ¿Cuántas relaciones hay de A en B ? Justifique su respuesta.
2. Sean A y B conjuntos, y $R \subseteq A \times B$. Demuestre que $(R^{-1})^{-1} = R$.
3. Demuestre que $<[\mathbb{N}] = \mathbb{N} - \{0\}$.
4. Demuestre que $>[\mathbb{N}] = \mathbb{N}$.
5. Sean A un conjunto y $R \subseteq A \times A$. Defina en palabras y formalmente las siguientes propiedades:
 - a) R es reflexiva.
 - b) R es irreflexiva.
 - c) R es simétrica.
 - d) R es transitiva.
 - e) R es antisimétrica.
 - f) R es asimétrica.
6. Sean A un conjunto y $R \subseteq A \times A$. Demuestre o refute:
 - a) Si R es transitiva e irreflexiva, entonces R es asimétrica.
 - b) Si R es reflexiva y transitiva, entonces R es antisimétrica.
 - c) Si R es simétrica y antisimétrica, entonces R es reflexiva.
 - d) Si R es antisimétrica y asimétrica, entonces $R = \emptyset$.
 - e) Si R es transitiva y asimétrica, entonces R es irreflexiva.
 - f) Si R es transitiva, entonces R^{-1} es transitiva.
 - g) Si R es antisimétrica, entonces R^{-1} es antisimétrica.
7. Sean A un conjunto y $R \subseteq A \times A$. Demuestre que R es un orden si y solo si R^{-1} es un orden.
8. Sea $A = \{a\}$. ¿Existe una relación $R \subseteq A \times A$ tal que R sea un orden? Explique.
9. Sea $\mid \subseteq \mathbb{N} \times \mathbb{N}$ la relación de divisibilidad estricta entre números naturales, es decir, si $a, b \in \mathbb{N}$, entonces

$$a \mid b \equiv (\exists x \in \mathbb{N} - \{0, 1\} \mid a \cdot x = b).$$
 Demuestre que $(\mathbb{N} - \{0, 1\}, \mid)$ es un orden.
10. Sean A un conjunto y $\prec \subseteq A \times A$ un orden. Demuestre que los mapas $\prec \mapsto \preceq$ y $\preceq \mapsto \prec$ son inversos el uno del otro.
11. Demuestre que el poset $(A, \prec)$ es linealmente ordenado si y solo si $(\forall x, y \in A \mid x \preceq y \vee x \succeq y)$.

12. Defina un orden sobre $\mathbb{N}$ que no sea linealmente ordenado. Explique.
13. Sea $(A, <)$ un poset. Demuestre o refute:
 - a) Si $A = \{a\}$ y a es minimal y maximal en $(A, <)$, entonces $< = \emptyset$.
 - b) Si $B \subseteq A$ y $a \in B$ es minimal en $(B, <)$, entonces a es minimal en $(A, <)$.
14. Sean $(A, <)$ un poset y $a \in A$. Demuestre que a es minimal en $(A, <)$ si y solo si a es maximal en $(A, >)$.
15. Sean $(A, <)$ un poset y $a \in A$. Demuestre que a es minimal en $(A, <)$ si y solo si $>[\{a\}] = \emptyset$.
16. Considere la siguiente afirmación:
 Si el conjunto de elementos maximales de un poset $(A, <)$ no es vacío,
 entonces para cualquier $x \in A$ se puede escoger un elemento maximal
 $m_x \in A$ tal que $m_x \succeq x$.
 ¿Es esta afirmación correcta? La forma de responder esta pregunta es suministrando una demostración de la afirmación o un contraejemplo.
17. Para cada una de las siguientes parejas, determine si es bien fundada y justifique su respuesta:
 - a) $(\mathbb{N}, >)$.
 - b) $(\mathbb{Q}_{\geq 0}, <)$.
 - c) $(\wp(A), \subset)$, con A finito.
 - d) $(\wp(\mathbb{N}), \subset)$.
 - e) La relación “ser subcadena propia de” sobre las cadenas de un alfabeto finito.
18. Demuestre que el orden lexicográfico sobre $\mathbb{N} \times \mathbb{N}$ del Ejemplo 0.8.1 es bien fundado. Sugerencia: considere primero las primeras componentes.
19. Considere el siguiente ciclo, con n y m números naturales: mientras $n > 0$, si $m > 0$ disminuya m en uno; de lo contrario, disminuya n en uno y asigne a m un número natural cualquiera. Muestre que ninguna de las dos variables sirve por sí sola como variante y proponga una que sí funcione.
20. Use el Teorema 0.8.4 para demostrar que la relación sobre los arreglos definida por $A < B$ si y solo si $|A| < |B|$ es bien fundada. ¿Qué medida y qué conjunto bien fundado se usan?
21. Un algoritmo recursivo recibe un arreglo $A[a..b)$ y se llama recursivamente sobre $A[a..c)$ y $A[c..b)$, con $a < c < b$. Proponga una medida que demuestre su terminación y verifique que decrece en ambas llamadas.

0.9. Inducción sobre los números naturales

La relación bien fundada que más se usa en la práctica es $(\mathbb{N}, <)$. Esta sección la estudia como caso particular del principio general de la Sección 0.8, obtiene de él las dos formas de inducción con las que se razona sobre los números naturales y las aplica a las dos deudas pendientes del capítulo: justificar el método de invariantes de la Nota 0.7.1 y demostrar que el ordenamiento por selección es correcto.

0.9.1. Dos principios de inducción. Por el Ejemplo 0.8.1, $(\mathbb{N}, <)$ es bien fundada, de modo que el Teorema 0.8.3 se aplica. Basta con observar que, para $n \in \mathbb{N}$,

$$>[\{n\}] = \{0, 1, \dots, n-1\}$$

es el conjunto de los naturales estrictamente menores que n . El principio resultante se conoce como inducción fuerte.

Teorema 0.9.1: Inducción fuerte

Sea $P : \mathbb{N} \rightarrow \mathbb{B}$ un predicado. Si

$$(\forall n \in \mathbb{N} : (\forall m \mid m < n : P(m)) \implies P(n)),$$

entonces $(\forall n \in \mathbb{N} : P(n))$.

Demostración

Sea $B = \{n \in \mathbb{N} \mid P(n)\}$. La hipótesis afirma que, para cada $n \in \mathbb{N}$, si $>[\{n\}] \subseteq B$ entonces $n \in B$. Como $(\mathbb{N}, <)$ es bien fundada, el Teorema 0.8.3 permite concluir $B = \mathbb{N}$, que es lo pedido.

El Teorema 0.9.1 no tiene caso base separado. Para $n = 0$ el rango $m < 0$ es vacío y, por la Nota 0.2.2, la hipótesis $(\forall m \mid m < 0 : P(m))$ es verdadera sin aportar información: hay que demostrar $P(0)$ sin suponer nada. Los casos base son, como en el principio general, los elementos minimales.

La forma de inducción más conocida se obtiene del mismo teorema general, pero con otra relación: la que asocia a cada natural únicamente su sucesor.

Teorema 0.9.2: Inducción ordinaria

Sea $P : \mathbb{N} \rightarrow \mathbb{B}$ un predicado. Si $P(0)$ y $(\forall n \in \mathbb{N} : P(n) \implies P(n+1))$, entonces $(\forall n \in \mathbb{N} : P(n))$.

Demostración

Sea $\prec_1$ la relación sobre $\mathbb{N}$ definida por $m \prec_1 n$ si y solo si $n = m + 1$. Primero, $(\mathbb{N}, \prec_1)$ es bien fundada: si $B \subseteq \mathbb{N}$ no es vacío, sea n su elemento mínimo con respecto a $<$; si algún $m \in B$ cumpliera $m \prec_1 n$, entonces $m = n - 1 < n$, contradiciendo la minimalidad de n . Luego n es minimal en $(B, \prec_1)$.

Segundo, $\succ_1[\{0\}] = \emptyset$ y $\succ_1[\{n + 1\}] = \{n\}$. Por lo tanto, la premisa del Teorema 0.8.3 para $B = \{n \in \mathbb{N} \mid P(n)\}$ se reduce a dos obligaciones: $0 \in B$, es decir $P(0)$, y $n \in B \implies n + 1 \in B$, es decir $P(n) \implies P(n + 1)$. Ambas son hipótesis del teorema, de donde $B = \mathbb{N}$.

Las dos formas de inducción difieren únicamente en la relación bien fundada escogida, no en el principio. La inducción fuerte permite suponer la propiedad para *todos* los naturales menores; la ordinaria, solo para el inmediatamente anterior. Escoger la relación adecuada es la parte creativa de una demostración por inducción.

Nota 0.9.1

Cuando una propiedad solo se cumple a partir de un valor k , se aplica el mismo principio al conjunto $\{n \in \mathbb{N} \mid n \geq k\}$, que también es bien fundado con $<$ por el Teorema 0.8.1. Las obligaciones se convierten en $P(k)$ y, para $n \geq k$, $P(n) \implies P(n + 1)$.

0.9.2. Justificación del método de invariantes. La Nota 0.7.1 afirma que basta con verificar la inicialización y la estabilidad para concluir que el invariante se cumple en todas las evaluaciones de la condición del ciclo. Esa afirmación quedó sin demostrar. La herramienta para demostrarla es la inducción ordinaria sobre el número de iteraciones completadas.

Teorema 0.9.3

Considere un ciclo y, para $k \in \mathbb{N}$, sea s_k el estado de la ejecución en la $(k + 1)$ -ésima evaluación de la condición del ciclo, si esa evaluación ocurre. Sea I una propiedad de los estados tal que:

1. $I(s_0)$ (inicialización);
2. para todo k , si $I(s_k)$ y la condición es verdadera en s_k , entonces $I(s_{k+1})$ (estabilidad).

Entonces, $I(s_k)$ para todo k tal que s_k existe.

Demostración

Sea $P(k)$ la propiedad “si s_k existe, entonces $I(s_k)$ ”. Se demuestra $(\forall k \in \mathbb{N} \mid P(k))$ por inducción ordinaria (Teorema 0.9.2).

El caso base $P(0)$ es la inicialización. Para el caso inductivo, suponga $P(k)$ y suponga que s_{k+1} existe. Que s_{k+1} exista significa que hubo una $(k+2)$ -ésima evaluación de la condición, de modo que s_k existe y la condición es verdadera en s_k . Por la hipótesis inductiva, $I(s_k)$; por la estabilidad, $I(s_{k+1})$. Luego $P(k+1)$.

Con el Teorema 0.9.3, el método de la Nota 0.7.1 queda justificado. Si además el ciclo termina, la última evaluación de la condición ocurre en un estado en el cual la condición es falsa y en el cual, por el teorema, el invariante se cumple; el logro produce entonces la poscondición. La terminación se obtiene, a su vez, exhibiendo una variante y aplicando el Teorema 0.8.5. Invariante y variante juntos dan la corrección total en el sentido de la Definición 0.7.1.

0.9.3. Corrección del ordenamiento por selección. Ya están disponibles todas las herramientas para saldar la promesa de la Sección 0.5. La demostración se hace en dos partes, una por cada ciclo de la Implementación 0.6.1.

Teorema 0.9.4

Sean $A[0..N)$ y a, b tales que $0 \leq a < b \leq N$. La ejecución de `index_of_min` sobre A , a y b termina y retorna un índice i tal que

$$a \leq i < b \quad \text{y} \quad A[i] = (\downarrow k \mid a \leq k < b : A[k]).$$

Demostración

Considere la propiedad

$$I : \quad a \leq i < j \leq b \wedge A[i] = (\downarrow k \mid a \leq k < j : A[k]).$$

Inicialización. Al evaluar por primera vez la condición se tiene $i = a$ y $j = a + 1$. Como $a < b$, se cumple $a \leq i < j \leq b$. Además, el rango $a \leq k < a + 1$ tiene un solo punto, de modo que por la regla de un punto de la Nota 0.2.3 el mínimo es $A[a] = A[i]$.

Estabilidad. Suponga I y $j < b$. La iteración asigna a i el valor j si $A[j] < A[i]$, y lo deja igual en caso contrario; en ambos casos el nuevo valor

i' cumple $A[i'] = (A[i] \downarrow A[j])$ y $a \leq i' < j + 1$. Por el Teorema 0.2.1,

$$\begin{aligned} (\downarrow k \mid a \leq k < j + 1 : A[k]) &= (\downarrow k \mid a \leq k < j : A[k]) \downarrow A[j] \\ &= A[i] \downarrow A[j], \end{aligned}$$

de modo que I se cumple de nuevo con $j' = j + 1 \leq b$.

Logro. Al terminar el ciclo la condición $j < b$ es falsa y, con $j \leq b$ del invariante, se concluye $j = b$. El invariante da entonces $a \leq i < b$ y $A[i] = (\downarrow k \mid a \leq k < b : A[k])$, que es lo afirmado.

Terminación. La cantidad $b - j$ es un número natural, por el invariante, y cada iteración la disminuye en uno. Por el Teorema 0.8.5 con $(\mathbb{N}, <)$, el ciclo termina.

Teorema 0.9.5

El ordenamiento por selección de la Implementación 0.6.1 es totalmente correcto con respecto al problema de ordenamiento del Ejemplo 0.3.1.

Demostración

Sea A_0 el valor inicial del arreglo y considere el invariante del Ejemplo 0.7.4:

$$\begin{aligned} I : \quad & 0 \leq n \leq N, \\ & A[0..n) \text{ está ordenado ascendentemente,} \\ & (\forall i, j \mid 0 \leq i < n \leq j < N : A[i] \leq A[j]), \\ & A \text{ es una permutación de } A_0. \end{aligned}$$

Inicialización. Al evaluar por primera vez la condición se tiene $n = 0$, de modo que $0 \leq n \leq N$. Las secciones y los rangos involucrados son vacíos: $A[0..0)$ está ordenado y el rango $0 \leq i < 0$ es vacío, luego la tercera parte es verdadera por la Nota 0.2.2. Finalmente, $A = A_0$ es una permutación de A_0 mediante la identidad.

Estabilidad. Suponga I y $n < N$. Por el Teorema 0.9.4, la iteración obtiene un índice i con $n \leq i < N$ y $A[i] = (\downarrow k \mid n \leq k < N : A[k])$, e intercambia $A[n]$ con $A[i]$. Sea A' el arreglo después del intercambio y $n' = n + 1$.

- Como $n < N$, se cumple $0 \leq n' \leq N$.
- El intercambio es una permutación de A y la composición de permutaciones es una permutación, de modo que A' sigue siendo permutación de A_0 .

- El intercambio solo modifica posiciones mayores o iguales que n , así que $A'[0..n) = A[0..n)$, que está ordenado. Además, $A'[n]$ es un elemento de $A[n..N)$ y, si $n > 0$, la tercera parte del invariante da $A[n-1] \leq A'[n]$. Por el Ejercicio 0.2.5, $A'[0..n+1)$ está ordenado.
- Sean $0 \leq i' < n' \leq j < N$. Si $i' < n$, entonces $A'[i'] = A[i']$ y $A'[j]$ es un elemento de $A[n..N)$, luego la tercera parte del invariante da $A'[i'] \leq A'[j]$. Si $i' = n$, entonces $A'[n]$ es el mínimo de $A[n..N)$ y $A'[j]$ es un elemento de esa misma sección, luego $A'[n] \leq A'[j]$.

En consecuencia, I se cumple de nuevo con n' .

Logro. Al terminar el ciclo la condición $n < N$ es falsa y, con $n \leq N$ del invariante, se concluye $n = N$. La segunda parte del invariante afirma entonces que $A[0..N)$, es decir todo el arreglo, está ordenado ascendentemente, y la cuarta que es una permutación de A_0 . Esto es exactamente la poscondición del Ejemplo 0.3.1, luego el algoritmo es parcialmente correcto.

Terminación. La cantidad $N-n$ es un número natural, por la primera parte del invariante, y cada iteración la disminuye en uno. Por el Teorema 0.8.5, el ciclo termina.

Al ser parcialmente correcto y terminar para toda instancia, el algoritmo es totalmente correcto.

La demostración explica, por fin, por qué la tercera parte del invariante es indispensable, aunque no aparezca en la poscondición: es la que permite concluir, en la estabilidad, que el elemento que se agrega a la sección ordenada no es menor que los que ya estaban allí.

0.9.4. Otras instancias. La inducción sobre los naturales no es la única que se usa en el texto. Un algoritmo recursivo sobre secciones de un arreglo, como los del Capítulo 2, se llama a sí mismo sobre secciones más pequeñas de $A[a..b)$. La medida $b-a$ y el Teorema 0.8.4 convierten esas llamadas en una relación bien fundada, y el Teorema 0.8.3 da entonces un principio de inducción: para demostrar que el algoritmo es correcto sobre $A[a..b)$, se puede suponer que lo es sobre toda sección más corta. Esa es la forma que toma la corrección de los algoritmos de dividir y conquistar.

Ejercicios

1. Considere el orden bien fundado $(\mathbb{N}, <)$. Se demostrará, paso a paso y usando el principio de inducción bien fundada, que cualquier número natural $n \in \mathbb{N}$ satisface la fórmula

$$(+i \mid 0 \leq i \leq n : i) = \frac{n \cdot (n + 1)}{2}.$$

- a) Identifique explícitamente a qué corresponden, para este problema, el conjunto A y la relación $\prec$ que aparecen en el Teorema 0.8.3.
 - b) Defina un predicado $P : \mathbb{N} \rightarrow \mathbb{B}$ que corresponda a la propiedad que se quiere demostrar; también defina el conjunto $B \subseteq \mathbb{N}$ de aquellos números naturales que satisfacen P .
 - c) Escriba el principio de inducción para el conjunto A y la relación $\prec$ identificados en el numeral (a), y el conjunto B del numeral (b). Note que el cuantificador universal sobre los subconjuntos de A en la formulación del principio de inducción bien fundada desaparece dado que ya se ha escogido un B en particular.
 - d) Enuncie y demuestre el caso base para P .
 - e) Enuncie y demuestre el caso inductivo para P , enunciando explícitamente la hipótesis inductiva.
2. Demuestre que la inducción fuerte y la inducción ordinaria son interderivables: use el Teorema 0.9.2 para demostrar el Teorema 0.9.1 y viceversa. Sugerencia: para una de las dos direcciones, considere el predicado $Q(n) = (\forall m \mid m \leq n : P(m))$.
 3. Demuestre que las dos formulaciones de arreglo ordenado de la Sección 0.2 son equivalentes, es decir, que para todo $A[0..N]$,

$$(\forall i \mid 1 \leq i < N : A[i-1] \leq A[i])$$

si y solo si $(\forall i, j \mid 0 \leq i < j < N : A[i] \leq A[j]).$

Sugerencia: una de las dos direcciones es inmediata; la otra requiere inducción sobre $j - i$.

4. Demuestre que la variante del ordenamiento por selección de los ejercicios de la Sección 0.5, cuyo ciclo llega hasta $N - 2$, también es totalmente correcta. ¿Qué parte de la demostración del Teorema 0.9.5 cambia?
5. Demuestre, con el invariante que propuso en los ejercicios de la Sección 0.7, que la Implementación 0.2.2 es totalmente correcta con respecto al problema de decidir si un arreglo está ordenado ascendentemente.
6. Demuestre que la Implementación 0.2.1 es totalmente correcta con respecto al problema cuya salida es $(+i \mid a \leq i < b : A[i])$.

7. Considere la siguiente “demostración” de que todos los caballos de un conjunto tienen el mismo color. Sea $P(n)$ la propiedad “en todo conjunto de n caballos, todos tienen el mismo color”. El caso base $P(1)$ es evidente. Para el caso inductivo, suponga $P(n)$ y considere un conjunto de $n + 1$ caballos. Al retirar el último, quedan n caballos que, por hipótesis inductiva, tienen el mismo color; al retirar el primero en cambio, quedan otros n caballos que también tienen el mismo color. Como los dos subconjuntos se traslapan, todos los $n + 1$ caballos tienen el mismo color. Encuentre el error en el argumento y explique exactamente en qué valor de n falla.
8. Demuestre por inducción que $(\#T \mid T \subseteq \{0, \dots, N - 1\} : true) = 2^N$, es decir, que un conjunto de N elementos tiene 2^N subconjuntos. ¿Qué dice este resultado sobre la cantidad de testigos que debe examinar un algoritmo de fuerza bruta para el problema de la Sección 0.4?

Notas del capítulo y referencias

En numerosos textos, los arreglos son considerados como conjuntos de variables elementales e indexadas consecutivamente. Esta concepción coincide con el tratamiento presentado en este capítulo. Autores como E. Dijkstra [Dij76] proponen un tratamiento alejado de la tradición y más cercano a la lógica para los arreglos, introduciendo el concepto de “variable arreglo” y adoptando una notación distinta para su indexación. Un tratamiento más extenso de la relación entre arreglos y funciones es propuesto por A. Kaldewaij [Kal90], incluyendo las nociones de sección e indexación, y notación para operar con ellos.

La notación de cuantificadores de la Sección 0.2 no es la habitual en los textos de algoritmos, que suelen escribir cada cuantificador con la convención heredada de la matemática clásica: $\sum$ para las sumatorias, mín y máx sobre conjuntos, y el $\forall$ de la lógica de predicados. La notación uniforme que se usa aquí surgió en los años setenta del grupo de trabajo de E. Dijkstra en Eindhoven y fue desarrollada como parte de un cálculo de predicados orientado a la derivación de programas; el tratamiento canónico es el de E. Dijkstra y C. Scholten [DS90]. Presentaciones didácticas, con las reglas de manipulación demostradas en detalle, se encuentran en D. Gries y F. Schneider [GS93], A. Kaldewaij [Kal90] y R. Backhouse [Bac03]. En este capítulo las reglas se aceptan sin demostración; los textos citados las demuestran y desarrollan muchas más.

La especificación de problemas a partir de una relación entre la entrada dada y la salida esperada, y la concepción de un problema algorítmico como una colección de instancias, son usuales en la literatura; ver, e.g., [CLRS22, Bha15]. La clasificación de la Sección 0.4 en problemas de decisión, búsqueda, optimización, conteo y enumeración también es común, aunque no siempre se presenta de manera

explícita; las relaciones entre estos tipos de problemas y su dificultad relativa son uno de los temas de los Capítulos 6 a 8.

Las condiciones que debe cumplir una secuencia de instrucciones para ser considerada un algoritmo, en la Definición 0.5.1, siguen de cerca las que enumera D. Knuth [Knu97]: finitud, precisión, entrada, salida y efectividad. Los modelos de computación que dieron origen a la teoría son de A. Turing [Tur36] y A. Church [Chu36b], publicados ambos en 1936; la tesis que lleva sus nombres, así como la equivalencia entre los modelos propuestos, se discute en el Capítulo 6.

Autores como J. Kleinberg y É. Tardos [KET06] usan enumeraciones estructuradas para especificar algoritmos, mientras que autores como T. Cormen et al. [CLRS22] y J. Erickson [Eri19] usan pseudo-código. Autores como H. Bhasin [Bha15] y S. Skiena [Ski08] usan complementariamente pseudo-código y un lenguaje de programación para especificar algoritmos. D. Gries [Gri81] usa el lenguaje de comandos guardados (en inglés, *guarded command language*) para especificar algoritmos iterativos; este puede ser considerado un lenguaje matemático para la especificación de algoritmos.

Las nociones de corrección parcial y total, y el uso de aserciones para demostrarla, provienen de R. Floyd [Flo67] y de C. Hoare [Hoa69]. La diferencia entre la presentación de este capítulo y la de la mayoría de los textos de algoritmos no está en la técnica sino en el orden de las cosas: es común presentar el invariante como una herramienta de verificación, que se busca una vez escrito el algoritmo, mientras que en la tradición de E. Dijkstra [Dij76], D. Gries [Gri81] y A. Kalde-waij [Kal90] el invariante es el punto de partida del diseño: se escoge qué propiedad debe mantener el ciclo y de ella se deriva el ciclo junto con su demostración. La observación sobre los límites de las pruebas de la Sección 0.7 aparece en las notas sobre programación estructurada de E. Dijkstra [Dij72].

El tratamiento de la inducción en las secciones 0.8 y 0.9 también invierte el orden habitual. Lo usual en los textos introductorios es postular el principio de inducción sobre los números naturales, en la tradición de los axiomas de Peano, y presentar la inducción fuerte y la estructural como variantes. Aquí se parte de las relaciones bien fundadas, de donde los tres principios se obtienen escogiendo la relación adecuada. El uso de conjuntos bien fundados para demostrar terminación se remonta también a R. Floyd [Flo67], quien propuso reemplazar la cantidad entera que decrece por un elemento de un conjunto bien fundado, lo cual permite tratar ciclos anidados con órdenes lexicográficos. El estudio sistemático de los órdenes que sirven para demostrar terminación es un área por derecho propio; N. Dershowitz y Z. Manna [DM79] introdujeron los órdenes multiconjunto, y F. Baader y T. Nipkow [BN98] presentan el tema en el contexto de la reescritura de términos.

El ordenamiento por selección, que sirve de ejemplo recurrente desde la Sección 0.5, aparece en prácticamente todos los textos introductorios de algoritmos.

Su análisis de costo se retoma en el Capítulo 1 y se compara allí con el de otros algoritmos de ordenamiento.

Análisis asintótico

Considere por un momento que cuenta con dos algoritmos, digamos A_0 y A_1 , que resuelven un problema algorítmico dado. Al ser soluciones del mismo problema, se está suponiendo que tanto A_0 como A_1 resuelven correctamente todas las instancias del problema. Sin más información y con poca reflexión, cualquiera de las dos soluciones podría adoptarse como “la” solución del problema algorítmico. Sin embargo, en la práctica es importante entender cómo se comportarían A_0 y A_1 para decir objetivamente cuál de las dos soluciones es “mejor” o “más conveniente”. Por ello, entender cuáles son las características que hacen que un algoritmo sea mejor que el otro es clave para diseñar algoritmos.

El *análisis asintótico* es una herramienta matemática que permite comparar algoritmos con base en algunas de sus medidas. En el caso puntual de este manuscrito, el interés está en las medidas que permiten identificar cuántas instrucciones, o cuántas unidades de tiempo o memoria –haciendo suposiciones mínimas sobre el lenguaje de programación en el cual se implemente y la arquitectura de la máquina en donde se despliegue dicha implementación– se requieren para que un algoritmo resuelva las instancias del problema para el cual fue diseñado. Al uso de estos recursos (i.e., tiempo y espacio) que hace un algoritmo se le denomina *eficiencia algorítmica*: es mejor usar menos instrucciones, o menos unidades de tiempo o memoria cuando sea posible.

Un reto fundamental al cual se enfrenta el análisis asintótico tiene que ver con el hecho de que un problema algorítmico puede contar con múltiples parámetros e infinitas instancias. Es decir, el análisis asintótico debe ser capaz de clasificar la eficiencia algorítmica de manera tal que refleje lo que en realidad sucede con cada instancia, pero de manera suficientemente general como para no entrar a distinguir casos muy específicos para cada una de ellas.

El capítulo se apoya de principio a fin en un problema concreto: encontrar la mayor suma de un subarreglo de un arreglo de números enteros. Tres algoritmos

que lo resuelven, medidos primero con un reloj y después contando operaciones, motivan las definiciones de las primeras secciones; más adelante se analizan con precisión y se les suma un cuarto algoritmo, obtenido con la técnica de dividir y conquistar. En conjunto, los cuatro ilustran cómo una idea de diseño puede reducir en un orden de magnitud el costo de un algoritmo.

Al final de este capítulo, el lector estará familiarizado con la notación asintótica y sus propiedades, y con las nociones de complejidad temporal y espacial de un algoritmo en el peor y en el mejor caso. Sabrá obtener la función que mide el costo de un algoritmo, sea este iterativo o recursivo, y clasificarla con sumas de referencia, árboles de recurrencia o el Teorema Maestro. Conocerá también el costo de las operaciones más usadas de Python, el lenguaje en el que se expresan algunos de los algoritmos de los capítulos siguientes. Finalmente, se aborda la idea del análisis amortizado, que permite acotar el costo de una secuencia de operaciones sobre una estructura de datos.

1.1. Medidas para problemas y algoritmos

La cantidad de problemas algorítmicos es inmensa. Por ello, es prácticamente imposible construir un marco matemático lo suficientemente general como para albergar el análisis de cualquier algoritmo. Por ejemplo, ¿cómo analizar las soluciones de un problema que tiene diez parámetros de la misma forma en que se analizan las de otro problema que tiene uno solo? Así como hay heterogeneidad en la cantidad de parámetros, también la hay en el tipo de operaciones que los algoritmos realizan. En algunos casos estas operaciones tienen naturaleza aritmética (e.g., multiplicar dos números enteros o calcular la raíz cuadrada de un número real no negativo); en otros, permiten manipular y construir cadenas de caracteres.

Esta sección establece las suposiciones sobre la forma de los problemas algorítmicos y sobre las instrucciones que un algoritmo puede usar como básicas, de manera que sea posible asociar a cada algoritmo una medida de los recursos que utiliza. Para que estas suposiciones no parezcan arbitrarias, la sección parte de un problema concreto y de mediciones reales sobre tres algoritmos que lo resuelven.

1.1.1. Un problema conductor. Dado un arreglo $A[0..N)$ y $0 \leq i \leq j \leq N$, el *subarreglo* $A[i..j)$ está conformado por las posiciones consecutivas de A desde i (incluida) hasta j (excluida). En particular, cuando $i = j$ el subarreglo es vacío y su suma es 0.

Ejemplo 1.1.1

El problema de la *suma máxima de un subarreglo* consiste en encontrar la mayor suma posible de los elementos de un subarreglo de un arreglo de números enteros:

Entrada: Arreglo $A[0..N)$, con $N \geq 0$, de números enteros.

Salida: $(\uparrow i, j \mid 0 \leq i \leq j \leq N : (+k \mid i \leq k < j : A[k]))$.

Por ejemplo, para $A = [3, -4, 5, -1, 2, -6, 4]$ la respuesta es 6, que corresponde a la suma del subarreglo $A[2..5) = [5, -1, 2]$. Como el subarreglo vacío siempre es un candidato, la respuesta nunca es negativa: si todos los elementos de A son negativos, la respuesta es 0.

A continuación se describen tres algoritmos que resuelven el problema de la suma máxima de un subarreglo. Por ahora basta con entender la idea de cada uno; la forma de expresarlos en Python y el cálculo preciso de su costo se abordarán más adelante en el capítulo.

Algoritmo Exhaustivo. Para cada par (i, j) con $0 \leq i \leq j \leq N$, calcula desde cero la suma de $A[i..j)$ recorriendo sus elementos uno a uno. La respuesta es la mayor de todas las sumas calculadas.

Algoritmo Acumulado. Observa que la suma de $A[i..j+1)$ es la suma de $A[i..j)$ más $A[j]$. Entonces, para cada i fijo, obtiene las sumas de

$$A[i..i), A[i..i+1), \dots, A[i..N)$$

agregando un elemento a la vez a la suma anterior, en lugar de recalcularlas desde cero. La respuesta es, de nuevo, la mayor de todas las sumas calculadas.

Algoritmo Incremental. Recorre el arreglo una sola vez, de izquierda a derecha, manteniendo dos valores: r , la mayor suma de un subarreglo (posiblemente vacío) que termina en la posición actual, y b , la mayor suma encontrada hasta el momento. Ambos empiezan en 0 y, al procesar $A[j]$, se actualizan así:

$$r := (r + A[j]) \uparrow 0 \quad \text{y} \quad b := b \uparrow r.$$

Al terminar el recorrido, la respuesta es b . La corrección de este algoritmo no es evidente: en este capítulo se toma como dada y se justifica en la Sección 3.4, con las herramientas de la programación dinámica (ver también el Ejercicio 8).

1.1.2. Una primera medición: tiempo. La forma más directa de comparar los tres algoritmos es implementarlos y medir cuánto tardan. La Figura 1 y el Ejemplo 1.1.2 presentan los tiempos obtenidos al ejecutarlos sobre arreglos aleatorios de tamaño $N = 2^4, 2^5, \dots$, en dos entornos distintos: implementaciones en Python y

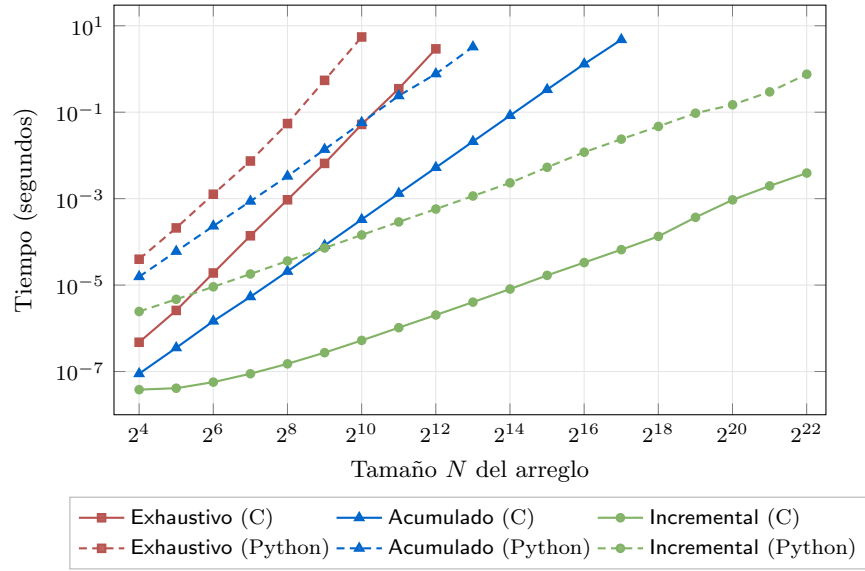


Figura 1. Tiempo de ejecución de los algoritmos Exhaustivo, Acumulado e Incremental en función del tamaño del arreglo, implementados en C (líneas continuas) y en Python (líneas discontinuas). Ambos ejes están en escala logarítmica. Mediciones hechas en un procesador Intel Xeon a 2,1 GHz, con Python 3.12 y C compilado con gcc -O2.

en el lenguaje compilado C. Cada serie se detuvo cuando una ejecución empezaba a tardar varios segundos.

Ejemplo 1.1.2

La siguiente tabla muestra algunos de los tiempos, en segundos, graficados en la Figura 1. Un guion indica que la medición no se hizo porque habría tardado demasiado.

	$N = 2^8$	$N = 2^{10}$	$N = 2^{12}$	$N = 2^{14}$	$N = 2^{20}$
Exhaustivo (C)	$9,4 \cdot 10^{-4}$	$5,2 \cdot 10^{-2}$	2,9	—	—
Acumulado (C)	$2,1 \cdot 10^{-5}$	$3,3 \cdot 10^{-4}$	$5,2 \cdot 10^{-3}$	$8,4 \cdot 10^{-2}$	—
Incremental (C)	$1,5 \cdot 10^{-7}$	$5,2 \cdot 10^{-7}$	$2,0 \cdot 10^{-6}$	$8,1 \cdot 10^{-6}$	$9,4 \cdot 10^{-4}$
Exhaustivo (Python)	$5,5 \cdot 10^{-2}$	5,5	—	—	—
Acumulado (Python)	$3,3 \cdot 10^{-3}$	$5,7 \cdot 10^{-2}$	$7,7 \cdot 10^{-1}$	—	—
Incremental (Python)	$3,6 \cdot 10^{-5}$	$1,4 \cdot 10^{-4}$	$5,7 \cdot 10^{-4}$	$2,3 \cdot 10^{-3}$	$1,5 \cdot 10^{-1}$

De estas mediciones se desprenden algunas observaciones:

1. *Los valores absolutos dependen del entorno.* El mismo algoritmo tarda entre 50 y 300 veces más en Python que en C. Otro procesador, otro compilador u otra versión del intérprete producirían números distintos.
2. *La forma del crecimiento no depende del entorno.* Al duplicar N , el tiempo de Exhaustivo se multiplica aproximadamente por 8, el de Acumulado por 4 y el de Incremental por 2, tanto en C como en Python. En la gráfica esto se ve como pares de rectas paralelas: cambiar de entorno desplaza cada recta, pero no altera su pendiente.
3. *El algoritmo pesa más que el entorno.* Para $N = 2^{12}$, Incremental en Python tarda menos de un milisegundo, mientras que Exhaustivo en C tarda casi tres segundos: una diferencia de más de 5 000 veces a favor del entorno “lento”. Esta diferencia sigue creciendo con N .
4. *Una medición no demuestra nada sobre las entradas que no se midieron.* Las mediciones son finitas y ruidosas: para arreglos pequeños, por ejemplo, el tiempo de Incremental en C casi no cambia entre $N = 2^4$ y $N = 2^5$ porque lo dominan costos fijos ajenos al algoritmo. Ninguna tabla, por grande que sea, garantiza cómo se comportará un algoritmo para todo N .

Las dos primeras observaciones sugieren buscar una medida que ignore el entorno y conserve la forma del crecimiento. La tercera indica que esa medida es la que realmente importa al diseñar algoritmos. La cuarta indica que, para hacer afirmaciones sobre todas las entradas, no bastan las mediciones: se requieren demostraciones, que es el propósito de la notación asintótica de la Sección 1.2.

1.1.3. El tamaño de la entrada. Para muchos problemas algorítmicos es fácil identificar una medida razonable del tamaño de la entrada. En el problema de la suma máxima de un subarreglo, al igual que en el problema de ordenamiento del Ejemplo 0.3.1, la cantidad N de elementos del arreglo es la elección natural (tampoco hay muchas más opciones). En el caso del tablero de ajedrez del Ejemplo 0.3.5, el tamaño de la entrada está dado por las dimensiones del tablero: 8 filas y 8 columnas o, de manera equivalente, 64 casillas.

Nota 1.1.1

Dado un problema algorítmico P , se denomina *tamaño de la entrada de P* a alguna medida razonable sobre los parámetros de P .

La noción de tamaño de la entrada presentada en la Nota 1.1.1 está lejos de ser un concepto matemáticamente preciso; además, no indica las unidades con las cuales se mide. Algunas veces se usará la cantidad de elementos de un arreglo como

unidad de medida y otras veces la cantidad de bits que conforman un número: no hay una única unidad preferida.

Otra “mala” noticia es que no existe una forma automática (i.e., algorítmica) de asignar el tamaño de la entrada a cualquier problema algorítmico. Por ello, es necesario desarrollar rápidamente la destreza de especificar claramente los problemas algorítmicos, para luego identificar cuáles son los parámetros clave que permiten definir el tamaño de su entrada.

Ejemplo 1.1.3

Considere el problema de sumar los elementos de una matriz de números:

Entrada: Matriz $A[0..M][0..N]$, con $M, N \geq 0$, de números enteros.

Salida: $(+i, j \mid 0 \leq i < M \wedge 0 \leq j < N : A[i][j])$.

En este caso, el tamaño de la entrada está dado por la cantidad de filas y la cantidad de columnas de la matriz, es decir, por el par (M, N) .

Los dos ejemplos anteriores muestran los dos casos que se presentan en la práctica: un problema cuyo tamaño se resume en un solo número y otro que necesita dos. La siguiente nota advierte sobre la tentación de resumir siempre en uno.

Nota 1.1.2

Cuando el tamaño de la entrada depende de varios parámetros, como en el Ejemplo 1.1.3, las medidas asociadas a un algoritmo son funciones de varias variables. Es posible resumir varios parámetros en uno solo (e.g., $M \cdot N$, la cantidad de elementos de la matriz), pero no siempre es conveniente: en ocasiones un algoritmo se comporta de manera muy distinta frente a cada parámetro y resumirlos oculta esa diferencia.

Aun en los problemas sencillos de los ejemplos 1.1.1 y 1.1.3, se está ignorando el tamaño de los números que se suman. El tamaño de los números puede llegar a ser un factor importante para entender cómo se comporta un algoritmo: no cuesta lo mismo sumar un par de números de pocos dígitos que otro par con miles de dígitos. Sin embargo, este nivel de detalle resulta innecesario en la mayoría de los casos, dado que se opera con datos cuyos tamaños básicos no son exageradamente grandes.

Nota 1.1.3

Para evitar un detalle excesivo al definir el tamaño de la entrada de un problema, y en la medida de las posibilidades, se supondrá que el tamaño de los números que conforman la entrada es despreciable.

El acuerdo de la Nota 1.1.3 se usará en la mayoría de los problemas abordados en este texto y tiene una justificación en la práctica. Por ejemplo, es común emplear procesadores de 64 bits para ejecutar algoritmos. Esto quiere decir que la comparación, la suma o la multiplicación de números de 64 bits se realizan directamente con operaciones del procesador, sin incurrir en “costos” adicionales. Aunque no siempre es posible, este acuerdo elimina dificultades técnicas y facilita la tarea de analizar un algoritmo.

1.1.4. Un modelo de costos. Las observaciones de la Sección 1.1.2 sugieren reemplazar el tiempo, que depende del entorno, por una medida que solo dependa del algoritmo. La idea es contar pasos de una máquina abstracta en la que ciertas operaciones básicas cuestan una unidad, sin importar el procesador o el lenguaje de programación en el cual se implemente el algoritmo.

Definición 1.1.1: Modelo de costos

Una *operación elemental* es cualquiera de las siguientes:

1. una operación aritmética o una comparación entre números cuyo tamaño es despreciable (en el sentido de la Nota 1.1.3);
2. la lectura o la escritura de una posición de un arreglo, dado su índice;
3. la asignación de un valor a una variable;
4. la evaluación de una condición, un salto en el flujo de control, el llamado a una función o el retorno de una función.

El *costo* de ejecutar un algoritmo A sobre una instancia x , denotado $\text{costo}_A(x)$, es la cantidad de operaciones elementales que efectúa A al procesar x .

Contar exactamente todas las operaciones elementales es tedioso y depende de detalles poco interesantes: ¿la actualización $b := b \uparrow r$ cuesta una operación, o dos (una comparación y una asignación)? En la práctica, basta con contar una operación representativa.

Nota 1.1.4

Una *operación barómetro* de un algoritmo es una operación elemental que, para entradas suficientemente grandes, se ejecuta al menos una fracción constante de las veces que se ejecuta cualquier otra. Contar solo las operaciones barómetro produce un número que difiere del costo por, a lo sumo, un factor constante; la Sección 1.3 justifica formalmente por qué ese factor puede ignorarse.

En los tres algoritmos de la suma máxima de un subarreglo, el acceso a una posición del arreglo A es una operación barómetro: todo el trabajo de cada algoritmo gira alrededor de leer elementos de A y sumarlos.

Ejemplo 1.1.4

La siguiente tabla muestra la cantidad de accesos al arreglo A que hace cada algoritmo, para los mismos tamaños del Ejemplo 1.1.2:

	$N = 2^8$	$N = 2^{10}$	$N = 2^{12}$	$N = 2^{14}$
Exhaustivo	2 829 056	179 481 600	11 461 636 096	733 141 975 040
Acumulado	32 896	524 800	8 390 656	134 225 920
Incremental	256	1 024	4 096	16 384

Estos valores corresponden exactamente a las fórmulas

$$\frac{N(N+1)(N+2)}{6}, \quad \frac{N(N+1)}{2} \quad \text{y} \quad N,$$

respectivamente, que se deducirán en la Sección 1.5.

A diferencia de los tiempos, los conteos del Ejemplo 1.1.4 son exactos y reproducibles: cualquier persona que repita el conteo, en cualquier máquina, obtendrá los mismos números. Aun así, conservan la forma del crecimiento observada en las mediciones: al duplicar N , el conteo de **Exhaustivo** se multiplica aproximadamente por 8, el de **Acumulado** por 4 y el de **Incremental** por 2.

1.1.5. Peor caso, mejor caso y caso promedio. En los tres algoritmos de la suma máxima de un subarreglo, la cantidad de accesos a A depende únicamente de N : el contenido del arreglo no influye. Esto no es lo habitual. Considere el problema de ordenar ascendentemente un arreglo $A[0..N)$ (Ejemplo 0.3.1) y el siguiente algoritmo.

Algoritmo de ordenamiento por inserción. Recorre el arreglo de izquierda a derecha. Al llegar a la posición k , el prefijo $A[0..k)$ ya está ordenado; entonces, desplaza $A[k]$ hacia la izquierda, intercambiándolo con su vecino izquierdo mientras este sea mayor. Al terminar el recorrido, el arreglo está ordenado.

Una operación barómetro para este algoritmo es la comparación entre dos elementos del arreglo. La Figura 2 muestra cuántas comparaciones hace para tres tipos de arreglos de tamaño N . Si el arreglo ya está ordenado, cada elemento se compara una sola vez con su vecino y no se desplaza: $N - 1$ comparaciones en total (para $N \geq 1$). Si el arreglo está invertido, cada elemento debe desplazarse hasta el comienzo del arreglo: $N(N - 1)/2$ comparaciones. Para $N = 200$, esto es 199 frente a 19900 comparaciones: el mismo algoritmo, con entradas del mismo tamaño, puede hacer cien veces más trabajo.

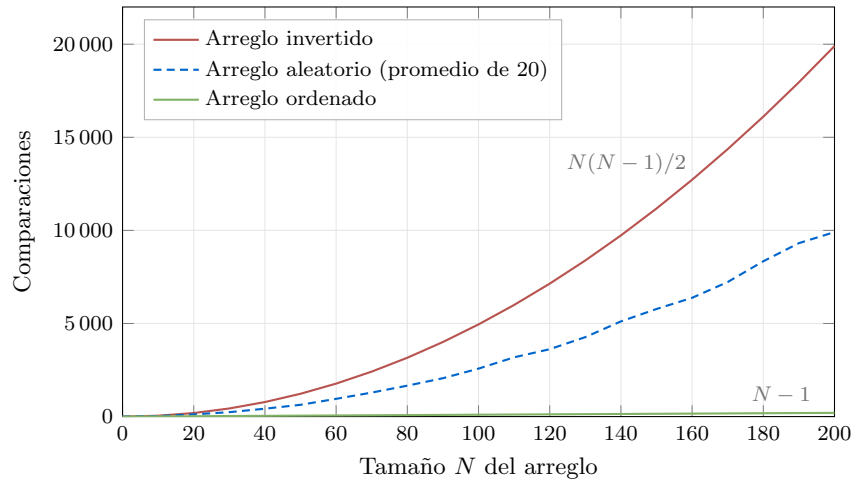


Figura 2. Cantidad de comparaciones entre elementos que hace el ordenamiento por inserción sobre arreglos invertidos, aleatorios y ordenados de tamaño N .

En consecuencia, no tiene sentido hablar de “el” costo de un algoritmo sobre las entradas de tamaño n sin decir de cuál de ellas se trata. Las siguientes definiciones escogen las dos instancias extremas. En ellas, para un problema P y $n \in \mathbb{N}$, I_n denota el conjunto de instancias de P cuyo tamaño es n .

Definición 1.1.2: Peor caso

Sea A un algoritmo que resuelve un problema P . El *costo de A en el peor caso* es la función $T_A^\uparrow : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ definida por:

$$T_A^\uparrow(n) = (\uparrow x \mid x \in I_n : \text{costo}_A(x)).$$

La definición dual, que escoge la instancia más favorable en lugar de la más desfavorable, se obtiene reemplazando el máximo por el mínimo.

Definición 1.1.3: Mejor caso

Sea A un algoritmo que resuelve un problema P . El *costo de A en el mejor caso* es la función $T_A^\downarrow : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ definida por:

$$T_A^\downarrow(n) = (\downarrow x \mid x \in I_n : \text{costo}_A(x)).$$

Para que estas definiciones tengan sentido se supone que, para cada n , el conjunto I_n no es vacío y que los costos de sus instancias están acotados; así, el máximo y el mínimo existen. Esta suposición se cumple para todos los algoritmos estudiados en este texto. Para el ordenamiento por inserción, al que se denota Ins , se tiene $T_{\text{Ins}}^\uparrow(N) = N(N-1)/2$ y $T_{\text{Ins}}^\downarrow(N) = N-1$ (para $N \geq 1$), contando comparaciones.

Nota 1.1.5

Cuando el costo cuenta operaciones elementales, se habla de la *complejidad temporal* de un algoritmo; cuando cuenta unidades de memoria (ver la subsección siguiente), de su *complejidad espacial*. Salvo que se indique lo contrario, la complejidad de un algoritmo A se refiere a su costo en el peor caso y se escribe simplemente T_A en lugar de $T_A^\uparrow$.

La preferencia por el peor caso en la Nota 1.1.5 tiene varias razones. Primero, es una garantía: ninguna instancia de tamaño n cuesta más que $T_A(n)$. Segundo, en muchos algoritmos el peor caso ocurre con frecuencia (e.g., buscar un elemento que no está en un arreglo). Tercero, el mejor caso suele decir poco: casi cualquier algoritmo de ordenamiento puede modificarse para que ordene rápidamente un arreglo que ya está ordenado.

Nota 1.1.6

Entre los dos extremos está el *caso promedio*: el costo esperado de A sobre una instancia de tamaño n escogida al azar según alguna distribución de probabilidad. La curva de arreglos aleatorios de la Figura 2 ilustra el caso promedio del ordenamiento por inserción, que resulta cercano a $N^2/4$ comparaciones. Precisar esta noción requiere teoría de probabilidad y escoger una distribución razonable, lo cual está fuera del alcance de este capítulo; el Capítulo 2 vuelve sobre esta idea.

Las funciones de las definiciones 1.1.2 y 1.1.3 tienen como dominio los números naturales y como codominio los reales no negativos. Esto obedece a que el tamaño de una instancia se mide, generalmente, en unidades enteras (e.g., cantidad de elementos, cantidad de bits) y a que la cantidad de operaciones o de memoria nunca es negativa (por cuestiones técnicas, se prefiere que el codominio sea el de los reales y no el de los naturales). Cuando el tamaño de la entrada depende de k parámetros (Nota 1.1.2), el dominio es $\mathbb{N}^k$ en lugar de $\mathbb{N}$.

Abstraer la cantidad de recursos empleados por un algoritmo con una función matemática tiene varios beneficios:

1. la noción de función es básica no solo en matemáticas sino en cualquier ciencia;
2. hay herramientas matemáticas disponibles para analizar y clasificar funciones y, por tanto, para clasificar algoritmos según la cantidad de recursos que requieren;
3. brinda un nivel de abstracción conveniente para comparar algoritmos, independiente de la velocidad del procesador o de la memoria de la máquina concreta en la cual se ejecuten.

1.1.6. Espacio. El modelo de costos también permite medir la memoria que usa un algoritmo. Para ello se supone que cada variable que almacena un número de tamaño despreciable y cada posición de un arreglo ocupa una unidad de memoria (o *celda*). Conviene distinguir entre el *espacio de la entrada*, que es el ocupado por la instancia misma, y el *espacio auxiliar*, que es el adicional que el algoritmo necesita durante su ejecución. Salvo que se indique lo contrario, la complejidad espacial de un algoritmo se refiere a su espacio auxiliar, en el peor caso.

Los tres algoritmos de la suma máxima de un subarreglo usan una cantidad de celdas auxiliares que no depende de N : unos cuantos índices y acumuladores. Considere, en contraste, la siguiente variante de Exhaustivo.

Ejemplo 1.1.5

El algoritmo Prefijos primero construye un arreglo auxiliar $P[0..N+1]$ con las *sumas prefijas* de A , es decir, $P[j] = (+k \mid 0 \leq k < j : A[k])$ para $0 \leq j \leq N$. Como $P[j+1] = P[j] + A[j]$, este arreglo se construye con un solo recorrido de A . Luego, para cada par (i, j) con $0 \leq i \leq j \leq N$, obtiene la suma de $A[i..j]$ como $P[j] - P[i]$, con una resta, en lugar de recorrer el subarreglo.

Frente a Exhaustivo, este algoritmo emplea $N+1$ celdas auxiliares y, a cambio, calcula la suma de cada subarreglo con una cantidad constante de operaciones. Sin embargo, Acumulado logra un ahorro de tiempo equivalente sin usar el arreglo auxiliar: gastar memoria para ahorrar tiempo es una estrategia útil, pero no siempre necesaria.

Las definiciones 1.1.2 y 1.1.3 se aplican sin cambios al espacio, reemplazando la cantidad de operaciones elementales por la cantidad máxima de celdas auxiliares ocupadas simultáneamente durante la ejecución.

Ejercicios

1. Investigue sobre el problema de determinar la ruta más corta de un vértice a otro en un grafo dirigido *sin* pesos en los arcos. Especifique este problema y determine el tamaño de la entrada.
2. Investigue sobre el problema de determinar la ruta más corta de un vértice a otro en un grafo dirigido *con* pesos en los arcos. Especifique este problema y determine el tamaño de la entrada.
3. Investigue sobre el problema del morral (en inglés, *knapsack*). Especifique este problema y determine el tamaño de la entrada. ¿Se puede expresar el tamaño de la entrada en función de solo una de sus variables?
4. Considere el problema de buscar una palabra de longitud m dentro de un texto de longitud n . Especifique el problema y discuta si conviene resumir el tamaño de la entrada en un solo parámetro (Nota 1.1.2).
5. Para $N = 0, 1, 2, 3$, enumere todos los pares (i, j) que considera el algoritmo **Exhaustivo** y verifique que la cantidad total de accesos al arreglo coincide con $\frac{N(N+1)(N+2)}{6}$.
6. Aplique a mano los algoritmos **Acumulado** e **Incremental** al arreglo del Ejemplo 1.1.1, $A = [3, -4, 5, -1, 2, -6, 4]$. Para **Incremental**, registre los valores de r y b después de procesar cada posición.
7. Suponga que en el problema de la suma máxima de un subarreglo se exige que el subarreglo no sea vacío. Modifique la especificación del Ejemplo 1.1.1 y el algoritmo **Incremental** para que resuelvan esta variante.
8. 🌈 Justifique la corrección del algoritmo **Incremental**. Para ello, demuestre que después de procesar $A[0..j]$ se cumple: r es la mayor suma de un subarreglo de $A[0..j]$ que termina en la posición j , y b es la mayor suma de un subarreglo de $A[0..j]$.
9. El algoritmo **Exhaustivo** considera $\frac{(N+1)(N+2)}{2}$ pares (i, j) , y cada par implica algunas operaciones elementales aunque el subarreglo sea vacío. Determine a partir de qué valor de N la cantidad de accesos al arreglo es mayor o igual que la cantidad de pares. ¿Qué relación tiene esto con la expresión “para entradas suficientemente grandes” de la Nota 1.1.4?
10. La *búsqueda lineal* determina si un valor x está en un arreglo $A[0..N)$ comparándolo con cada elemento, de izquierda a derecha, hasta encontrarlo o agotar

- el arreglo. Contando comparaciones con x , determine el costo de este algoritmo en el peor y en el mejor caso, y describa las instancias que los producen.
11. Justifique que el ordenamiento por inserción hace $N - 1$ comparaciones sobre un arreglo ordenado y $N(N - 1)/2$ comparaciones sobre un arreglo invertido, para $N \geq 1$.
 12. Determine la cantidad de accesos al arreglo A y la cantidad de celdas auxiliares que usa el algoritmo Prefijos del Ejemplo 1.1.5. Compare sus resultados con los de Exhaustivo y Acumulado.
 13. Implemente los tres algoritmos de la suma máxima de un subarreglo en el lenguaje de programación de su preferencia y repita las mediciones de la Sección 1.1.2. ¿Obtiene los mismos tiempos? ¿Obtiene los mismos cocientes al duplicar N ?

1.2. Notación asintótica

Esta sección presenta la definición formal de la notación O (en inglés, *big Oh notation*), y algunas notaciones relacionadas. También se incluyen ejemplos de cómo utilizarlas para clasificar y comparar funciones (matemáticas).

Los conteos obtenidos en la Sección 1.1 para el problema de la suma máxima de un subarreglo ilustran la necesidad de esta notación. Los tres algoritmos hacen

$$\frac{N(N+1)(N+2)}{6}, \quad \frac{N(N+1)}{2} \quad \text{y} \quad N$$

accesos al arreglo, respectivamente. Al compararlos, lo relevante no son los coeficientes ni los términos de menor grado, sino que el primero crece como N^3 , el segundo como N^2 y el tercero como N . La notación asintótica es el lenguaje que permite afirmar esto de manera precisa.

1.2.1. La definición de O . La idea que se quiere capturar es la de una función que, salvo por un factor constante y salvo por lo que ocurra en una cantidad finita de casos iniciales, no supera a otra. Esas dos concesiones, el factor constante y los casos iniciales, aparecen en la definición como dos variables cuantificadas existencialmente: c para el factor y n_0 para el punto a partir del cual se exige la desigualdad.

Definición 1.2.1

Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El *orden de f* , denotado $O(f)$, es el conjunto definido de la siguiente manera:

$$O(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists n_0 \in \mathbb{N})(\exists c \in \mathbb{R}_{>0})(\forall n \in \mathbb{N}) \\ n \geq n_0 \Rightarrow g(n) \leq cf(n)\}.$$

El orden $O(f)$ de una función f es el conjunto de todas las funciones que en el límite están acotadas (por encima) por un factor constante de f . En español, se dice que una función g está en $O(f)$ (i.e., g es del orden de f) si eventualmente g está acotada por encima por un múltiplo constante de f . En este caso, se escribe $g \in O(f)$. Las variables n_0 y c cuantificadas existencialmente en la Definición 1.2.1 son usadas para denotar “eventualmente” y “factor constante”, respectivamente.

La Figura 3 ilustra cuándo una función g está en $O(f)$. La constante c indica cuál es el múltiplo de f con el que se acota a g y la constante n_0 indica a partir de qué punto cf permanece estable como acotamiento de g . Lo que ocurra antes de n_0 , incluido el hecho de que g supere a cf , es irrelevante de acuerdo con la definición.

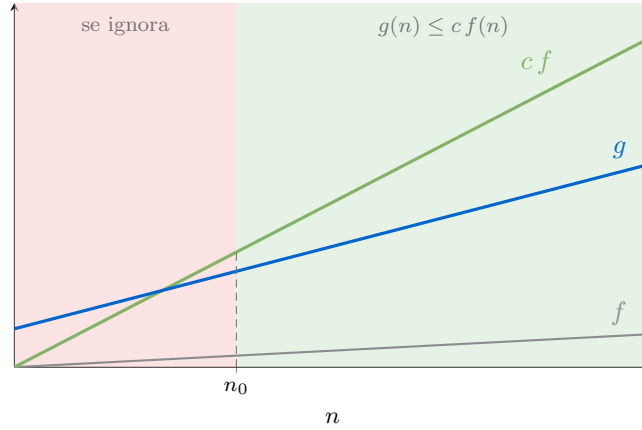


Figura 3. La función g está acotada por un factor constante positivo c de f en el límite (i.e., $g \in O(f)$). Las funciones de la figura son $g(n) = 5n + 10$ y $f(n) = n$, con $c = 10$ y $n_0 = 3$ (ver el Ejemplo 1.2.1).

La notación O puede también definirse sobre funciones con otro tipo. Por ejemplo, se pueden considerar funciones con dominio en los reales o de tuplas de números naturales. Para el propósito de este manuscrito, basta con enfocarse en aquellas funciones que tienen dominio en los naturales y codominio en los reales no negativos. La definición de O , tal y como se presenta con dominio en los naturales y rango en

los reales no negativos, es suficiente para clasificar y comparar una gran cantidad de funciones asociadas a algoritmos.

Nota 1.2.1

Por conveniencia, en algunas ocasiones se abusa la notación de orden permitiendo funciones más generales. Por ejemplo, de vez en cuando, se indica que $g \in O(f)$ aun si $g(n)$ es negativo o indefinido para algunos valores $n < n_0$. También se permite que f sea negativa o indefinida para alguna cantidad finita de valores en su dominio. En estos casos, es necesario escoger un n_0 lo suficientemente grande para que estos comportamientos sean excluidos para cualquier $n \geq n_0$.

El abuso de la Nota 1.2.1 es el que permite escribir $\log n \in O(n)$, a pesar de que $\log n$ no está definido en $n = 0$: basta con tomar $n_0 \geq 1$.

Nota 1.2.2

En este texto, $\log n$ denota el logaritmo de n en base 2, salvo que se indique lo contrario. La escogencia de la base es irrelevante para la notación asintótica: cambiar de base multiplica la función por una constante positiva y, como se verá en la Sección 1.3, los factores constantes no alteran el orden de una función.

Las dos notas siguientes se refieren a la escritura de la notación y a su alcance. La primera advierte sobre una costumbre extendida en la literatura y la segunda extiende la definición a funciones de varias variables.

Nota 1.2.3

Es común encontrar en la literatura la notación $g = O(f)$ en lugar de $g \in O(f)$. Se trata de un abuso de notación: $O(f)$ es un conjunto de funciones y g es una función, por lo que la relación entre ambos es la pertenencia. El abuso es inconveniente porque sugiere una igualdad que no es simétrica: se escribe $n = O(n^2)$, pero nunca $O(n^2) = n$. En este texto siempre se usa $\in$.

La Definición 1.2.1 habla de funciones de una sola variable, pero la Sección 1.1 ya mostró problemas cuyo tamaño necesita varios parámetros.

Nota 1.2.4

Cuando el tamaño de la entrada depende de varios parámetros (Nota 1.1.2), la Definición 1.2.1 se extiende a funciones $f, g : \mathbb{N}^k \rightarrow \mathbb{R}_{\geq 0}$ exigiendo la desigualdad $g(n_1, \dots, n_k) \leq cf(n_1, \dots, n_k)$ para todas las tuplas cuyas componentes sean mayores o iguales que n_0 . Es decir, “eventualmente” pasa a significar que *todos* los parámetros son grandes. Por ejemplo, el algoritmo que suma los elementos de una matriz de M filas y N columnas (Ejemplo 1.1.3) hace MN accesos a la matriz y usa $MN + M + N$ operaciones elementales para recorrerla, de modo que su costo está en $O(MN)$: para $M, N \geq 1$ se tiene $M \leq MN$ y $N \leq MN$, luego $MN + M + N \leq 3MN$.

1.2.2. Demostraciones de pertenencia. La necesidad de introducir una definición formal de la notación O es para establecer observaciones precisas sobre el comportamiento asintótico de algoritmos. En estos casos, se requieren demostraciones rigurosas que permitan clasificarlos. Para establecer $g \in O(f)$ se debe demostrar la existencia de dos valores $n_0 \in \mathbb{N}$ y $c \in \mathbb{R}_{>0}$ de tal forma que $g(n) \leq cf(n)$ sea cierto para cualquier $n \geq n_0$. Una forma alternativa de ver este tipo de demostraciones es a modo de juego contra un oponente que siempre juega de forma perfecta (i.e., si hay forma de ganar, el oponente escoge una opción que le garantiza ganar).

Nota 1.2.5

El jugador con el primer turno es quien tiene el objetivo de establecer $g \in O(f)$. En el primer turno, dicho jugador escoge valores para n_0 y c . En el segundo turno, el oponente “todopoderoso” escoge un valor $n \geq n_0$. El juego lo gana el primer jugador si $g(n) \leq cf(n)$; de lo contrario, gana el oponente.

La justificación de ver una posible demostración de $g \in O(f)$ a modo de juego, como lo propone la Nota 1.2.5, es la siguiente. Si $g \in O(f)$, entonces existen constantes n_0 y c tal que $g(n) \leq cf(n)$ para $n \geq n_0$, y de esta forma el primer jugador tiene una estrategia ganadora. De lo contrario, sin importar cómo el primer jugador escoja n_0 y c , el oponente encontrará (dado que siempre juega de forma perfecta) un $n \geq n_0$ que falsifique la desigualdad y así ganará el juego. Es importante entender que en ocasiones se dice que las variables n_0 y c son constantes. En realidad, este abuso del lenguaje tiene el propósito de dejar claro que estos valores no dependen de n .

Ejemplo 1.2.1

Se desea demostrar que $5n + 10$ está en $O(n)$. Para ello, es necesario escoger valores $n_0 \in \mathbb{N}$ y $c \in \mathbb{R}_{>0}$ tales que para cualquier $n \geq n_0$ se dé $5n + 10 \leq cn$.

El primer jugador puede tomar $n_0 = 3$ y $c = 10$, y se observa que $5n + 10 \leq 10n$ siempre y cuando $2 \leq n$. Como $n_0 = 3$, la desigualdad $5n + 10 \leq 10n$ es cierta cuando $n \geq n_0$. Es decir, no importa qué valor escoja el oponente en su turno, aquel que siempre juega perfecto, pues cualquier escogencia hace que la desigualdad sea cierta. En conclusión,

$$(5n + 10) \in O(n).$$

Los testigos n_0 y c no son únicos: cualquier par que haga cierta la desigualdad sirve para demostrar la pertenencia. Además, no es necesario encontrar los “mejores” testigos posibles, lo cual suele simplificar bastante las demostraciones. El siguiente ejemplo explota esta libertad.

Ejemplo 1.2.2

Se desea demostrar que la cantidad de accesos al arreglo que hace el algoritmo Exhaustivo está en $O(N^3)$, es decir:

$$\frac{N(N+1)(N+2)}{6} \in O(N^3).$$

El primer jugador toma $n_0 = 1$ y $c = 1$. Para $N \geq 1$ se tiene $N + 1 \leq 2N$ y $N + 2 \leq 3N$, de donde

$$\frac{N(N+1)(N+2)}{6} \leq \frac{N \cdot 2N \cdot 3N}{6} = N^3.$$

Nótese que las desigualdades $N+1 \leq 2N$ y $N+2 \leq 3N$ son bastante generosas, pero la conclusión es la misma: el orden no cambia por un factor constante.

1.2.3. Demostraciones de no pertenencia. Un ejercicio común en la comparación asintótica de funciones es el de establecer cuándo una función *no* está en el orden de otra. Para este propósito es indispensable entender muy bien la definición de O y, en particular, la alternancia de cuantificadores en ella. Intuitivamente, $g \notin O(f)$ cuando sin importar con qué factor constante se amplifique a f es imposible que dicho factor constante de f domine a g en el límite. Estas demostraciones tienden a ser posiblemente más retadoras que las de pertenencia dado que se debe demostrar que las constantes n_0 y c no existen. Dicho de otra forma, que sin importar cómo se escojan n_0 y c , al menos un valor de $n \geq n_0$ satisface $g(n) > cf(n)$. En estos casos, resulta conveniente usar la técnica de demostración por contradicción: suponer que dichos valores existen para luego llegar a un sinsentido o absurdo lógico.

Ejemplo 1.2.3

Se desea demostrar que n^2 no está en $O(10000n)$. Para ello, es necesario garantizar que es imposible escoger valores $n_0 \in \mathbb{N}$ y $c \in \mathbb{R}_{>0}$ tales que si $n \geq n_0$ se dé $n^2 \leq 10000cn$.

Hacia una contradicción, suponga que dichos valores existen. Sin pérdida de generalidad, también suponga que $n_0 \geq 1$. Entonces, para $n \geq n_0$ se tiene

$$n^2 \leq 10000cn \quad \text{sii} \quad \frac{n}{10000} \leq c.$$

Es decir, se debe dar que la constante c acote la función $\frac{n}{10000}$, lo cual es imposible (¿por qué?). En conclusión,

$$n^2 \notin O(10000n).$$

En el Ejemplo 1.2.3, la contradicción se obtiene al observar que ninguna constante acota una función que crece sin límite. El siguiente ejemplo exhibe explícitamente el valor de n con el que gana el oponente, que es la otra forma de presentar este tipo de demostraciones.

Ejemplo 1.2.4

Se desea demostrar que la cantidad de accesos al arreglo que hace el algoritmo **Acumulado** no está en $O(N)$, es decir, que dicho algoritmo no es lineal:

$$\frac{N(N+1)}{2} \notin O(N).$$

Hacia una contradicción, suponga que existen $n_0 \in \mathbb{N}$ y $c \in \mathbb{R}_{>0}$ tales que $\frac{N(N+1)}{2} \leq cN$ para cualquier $N \geq n_0$. Sin pérdida de generalidad, suponga que $n_0 \geq 1$. El oponente escoge

$$N = n_0 + \lceil 2c \rceil,$$

que satisface $N \geq n_0$ y $N \geq 2c$. Al dividir la desigualdad supuesta entre N , que es positivo, se obtiene $\frac{N+1}{2} \leq c$; sin embargo,

$$\frac{N+1}{2} \geq \frac{2c+1}{2} > c,$$

lo cual es una contradicción. En conclusión, $\frac{N(N+1)}{2} \notin O(N)$.

La notación asintótica es suficientemente robusta y de alto nivel como para obviar detalles que se desean ignorar y que dependen –por ejemplo– de la arquitectura de máquina o del lenguaje de programación en el cual se implementa un algoritmo. También, es precisa como para poder establecer comparaciones entre diferentes alternativas de alto nivel que resuelvan un mismo problema algorítmico,

especialmente cuando el tamaño de la entrada es grande y se requiere ingenio para plantear un algoritmo que sea práctico.

1.2.4. Cotas inferiores y cotas exactas. La notación O es útil para estimar una cota *superior* del uso de recursos que un algoritmo requiere para resolver un problema. De forma dual, también puede ser interesante estimar una cota *inferior* del uso de estos recursos. A continuación se introduce nueva notación para tal fin.

Definición 1.2.2

Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El conjunto $\Omega(f)$ se define para cualquier $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ como:

$$g \in \Omega(f) \quad \text{sii} \quad f \in O(g).$$

El conjunto $\Omega(f)$ (leído, ‘ómega de f ’) es el dual de $O(f)$, pues es el conjunto de todas aquellas funciones que están acotadas por debajo (en el límite) por un múltiplo positivo de f .

Nota 1.2.6

La definición de Ω en la Definición 1.2.2 no es la habitual. En este texto se opta por esta formulación dado que directamente depende de O . De este modo, basta con recordar solamente la definición de O , y una equivalencia sencilla para utilizar y razonar sobre Ω . El Ejercicio 17 de esta sección presenta la definición usual de Ω (para cualquier $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$)

$$\Omega(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists n_0 \in \mathbb{N})(\exists c \in \mathbb{R}_{> 0})(\forall n \in \mathbb{N}) \\ n \geq n_0 \Rightarrow g(n) \geq cf(n)\}$$

y plantea la equivalencia entre las dos definiciones.

En adición a O y Ω , existe otra noción asintótica para clasificar funciones que cuentan con una función que, simultáneamente en el límite, sirve como cota superior e inferior con factores que pueden ser distintos en cada caso.

Definición 1.2.3

Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El conjunto $\Theta(f)$ se define como:

$$\Theta(f) = O(f) \cap \Omega(f).$$

El conjunto $\Theta(f)$ se denomina el *exacto orden de f* y, como se puede deducir de su definición, es más poderoso (en relación con que brinda más información) que $O(f)$ o $\Omega(f)$ individualmente. Al igual que con la definición de Ω , hay una definición distinta que se usa habitualmente (Ejercicio 18 de esta sección). Por los mismos motivos explicados anteriormente para Ω , se prefiere la caracterización de Θ en la Definición 1.2.3.

Ejemplo 1.2.5

Se desea demostrar, para $n \geq 1$:

$$5n \in \Theta(n + \log n^2).$$

Considere el siguiente razonamiento:

$$\begin{aligned} 5n &\in \Theta(n + \log n^2) \\ \text{sii } 5n &\in \Theta(n + 2 \log n) && (\text{aritmética}) \\ \text{sii } 5n &\in O(n + 2 \log n) \cap \Omega(n + 2 \log n) && (\text{def. de } \Theta) \\ \text{sii } 5n &\in O(n + 2 \log n) \wedge 5n \in \Omega(n + 2 \log n) && (\text{def. de } \cap) \\ \text{sii } 5n &\in O(n + 2 \log n) \wedge (n + 2 \log n) \in O(5n) && (\text{def. de } \Omega) \\ \text{sii } 5n &\in O(5n) \wedge (n + 2 \log n) \in O(n + 2 \log n) && (O(5n) = O(n + 2 \log n)) \\ \text{sii } &\text{true} && (f \in O(f)). \end{aligned}$$

La demostración de la justificación dada en el quinto paso del razonamiento se propone como ejercicio al lector (Ejercicio 9).

Antes de aplicar estas notaciones al costo de un algoritmo conviene detenerse en un punto que suele pasarse por alto y que es fuente frecuente de afirmaciones imprecisas.

Nota 1.2.7

Las notaciones O , Ω y Θ clasifican funciones, no algoritmos. Por ello, antes de clasificar el costo de un algoritmo es indispensable haber fijado de cuál función se habla, i.e., de cuál caso (Nota 1.1.5). Para el ordenamiento por inserción, por ejemplo, se tiene $T_{\text{ins}}^{\uparrow} \in \Theta(N^2)$ y $T_{\text{ins}}^{\downarrow} \in \Theta(N)$ contando comparaciones. En particular, la afirmación $T_{\text{ins}}^{\uparrow} \in \Theta(N^2)$ no dice que el algoritmo haga una cantidad cuadrática de comparaciones sobre cualquier arreglo, sino que la cota superior y la cota inferior del peor caso coinciden.

Ejercicios

1. En la Nota 1.2.1 se indica que en ocasiones se pueden considerar funciones negativas o indefinidas en algunos puntos. Explique por qué los abusos identificados en la Definición 1.2.1 no afectan el uso formal de la notación O .
2. Considere la demostración del Ejemplo 1.2.1. Escoja valores diferentes para n_0 y c de tal forma que se obtenga una nueva demostración.
3. En la demostración por contradicción en el Ejemplo 1.2.3 se indica que sin pérdida de generalidad se puede suponer $n_0 \geq 1$. Explique por qué esta afirmación es correcta y no limita la generalidad de la demostración.
4. Suponga que un algoritmo A realiza $T_A : \mathbb{N} \rightarrow \mathbb{R}_{>0}$ operaciones básicas para ordenar un arreglo (i.e., $T_A(n)$ es la cantidad de operaciones que realiza A para ordenar un arreglo de tamaño n). Como parte de una tarea, Pedrito demostró que $T_A \in O(n^2)$, Juanita que $T_A \in O(n^{3/2})$ y el profesor publicó una solución con una demostración de $T_A \in O(n \log n)$.
 - a) ¿Es posible que las tres demostraciones sean correctas? Justifique su respuesta.
 - b) Si las tres demostraciones fueran correctas, ¿cuál de las tres afirmaciones brinda “más” información sobre el comportamiento de A ? Explique brevemente su respuesta.
5. En el Ejemplo 1.2.3 se usa el argumento de que la función $\frac{n}{10000}$ no puede ser acotada por la constante c . Demuestre formalmente que esta afirmación es cierta.
6. Demuestre, para cualquier $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, que $f \in O(f)$ y que $f \in \Theta(f)$.
7. Demuestre que la cantidad de accesos al arreglo que hace el algoritmo Exhaustivo está en $\Omega(N^3)$ y concluya, junto con el Ejemplo 1.2.2, que está en $\Theta(N^3)$.
8. Considere la búsqueda lineal de un valor x en un arreglo $A[0..N)$, contando comparaciones con x . Determine el orden exacto de su costo en el peor caso y en el mejor caso.
9. Para $n \in \mathbb{N}$, demuestre que $O(5n) = O(n + 2 \log n)$.
10. Para $n \in \mathbb{N}$, demuestre que $2n^2 \in O(n^3)$ y que $n^3 \notin O(2n^2)$.
11. Para $n \in \mathbb{N}$, demuestre o refute: $2^{n+1} \in O(2^n)$.
12. Para $n \in \mathbb{N}$, demuestre o refute: $(n+1)! \in O(n!)$, en donde $_{-}!$ es la función factorial.
13. Para $n \in \mathbb{N}$ y $f : \mathbb{N} \rightarrow \mathbb{R}_{>0}$, demuestre o refute:
 - a) Si $f \in O(n)$, entonces $f^2 \in O(n^2)$.
 - b) Si $f \in O(n)$, entonces $2^f \in O(2^n)$.

14. Considere la extensión de la notación O a funciones de varias variables (Nota 1.2.4). Demuestre que $M + N \in O(MN)$ y refute que $MN \in O(M + N)$.
15. Explique por qué la notación $g = O(f)$ de la Nota 1.2.3 no puede entenderse como una igualdad. Para ello, exhiba dos funciones g y h tales que $g = O(n^2)$ y $h = O(n^2)$, en el sentido abusivo, pero $g \neq h$.
16. Considere la siguiente definición para una función $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

$$O^*(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists c \in \mathbb{R}_{> 0})(\forall n \in \mathbb{N})g(n) \leq cf(n)\}.$$

Demuestre: si $f : \mathbb{N} \rightarrow \mathbb{R}_{> 0}$, entonces $O(f) = O^*(f)$.

17. Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El conjunto $\Omega^*(f)$ se define de la siguiente manera:

$$\Omega^*(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists n_0 \in \mathbb{N})(\exists c \in \mathbb{R}_{> 0})(\forall n \in \mathbb{N}) \\ n \geq n_0 \Rightarrow g(n) \geq cf(n)\}.$$

Demuestre que $\Omega(f) = \Omega^*(f)$.

18. Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El conjunto $\Theta^*(f)$ se define de la siguiente manera:

$$\Theta^*(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists n_0 \in \mathbb{N})(\exists c, d \in \mathbb{R}_{> 0})(\forall n \in \mathbb{N}) \\ n \geq n_0 \Rightarrow cf(n) \leq g(n) \leq df(n)\}.$$

Demuestre que $\Theta(f) = \Theta^*(f)$.

1.3. Propiedades de la notación asintótica

El razonamiento para clasificar y comparar funciones con las notaciones O , Ω , Θ se hace más práctico explotando sus propiedades matemáticas. Esta sección presenta algunas de estas propiedades, enfocándose principalmente en O , y las ilustra con ejemplos. Los ejercicios de la sección incluyen más propiedades de la notación asintótica.

1.3.1. Constantes y términos de menor orden. Los factores constantes en una expresión que determina la cantidad de recursos requeridos para resolver un problema tienden a ser extremadamente dependientes del sistema en donde se desplieguen los algoritmos. Por ello, si no se desean cálculos que estén comprometidos con un lenguaje de programación específico o un entorno de ejecución particular, tiene todo el sentido ignorar los términos constantes al clasificar y comparar asintóticamente las funciones asociadas a los algoritmos.

Teorema 1.3.1: Regla de constantes

Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Si $c \in \mathbb{R}_{> 0}$, entonces $O(f) = O(cf)$.

Demostración

Se supone $c \in \mathbb{R}_{>0}$ y se procede por doble inclusión:

- El objetivo es demostrar $O(f) \subseteq O(cf)$. Si $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es tal que $g \in O(f)$, basta con demostrar $g \in O(cf)$. Si $g \in O(f)$, entonces hay $n_0 \in \mathbb{N}$ y $c_0 \in \mathbb{R}_{>0}$ tales que $g(n) \leq c_0 f(n)$ para $n \geq n_0$. Tome $n_1 = n_0$ y $c_1 = \frac{c_0}{c}$, y note que para $n \geq n_1$ se tiene:

$$\begin{aligned} g(n) &\leq c_0 f(n) && \text{(por suposición)} \\ &= c_1 c f(n) && \text{(por definición de } c_1\text{).} \end{aligned}$$

Luego, $g \in O(cf)$ con testigos n_1 y c_1 .

- El objetivo es demostrar $O(cf) \subseteq O(f)$. Si $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es tal que $g \in O(cf)$, basta con demostrar $g \in O(f)$. Si $g \in O(cf)$, entonces hay $n_0 \in \mathbb{N}$ y $c_0 \in \mathbb{R}_{>0}$ tales que $g(n) \leq c_0 c f(n)$ para $n \geq n_0$. Tome $n_1 = n_0$ y $c_1 = c_0 c$, y note que para $n \geq n_1$ se tiene:

$$\begin{aligned} g(n) &\leq c_0 c f(n) && \text{(por suposición)} \\ &= c_1 f(n) && \text{(por definición de } c_1\text{).} \end{aligned}$$

Luego, $g \in O(f)$ con testigos n_1 y c_1 .

Es importante aclarar que los factores constantes pueden ser determinantes en el diseño de un algoritmo. Puede pensarse que la notación O es una herramienta que permite comparar algoritmos con enfoques fundamentalmente distintos que resuelven un problema dado y determinar, entre ellos, cuál tiene el mejor potencial para escalar a entradas de gran tamaño. Una vez el algoritmo de alto nivel esté claro, es importante esforzarse por reducir el factor constante asociado al algoritmo. De cualquier forma, si el éxito de la solución algorítmica depende de qué tan rápido pueda resolver un problema, claramente este debe funcionar tan rápido como sea posible.

El Teorema 1.3.1 es también la justificación formal de dos acuerdos adoptados en secciones anteriores: contar una operación barómetro en lugar de todas las operaciones elementales (Nota 1.1.4) y escoger una base cualquiera para el logaritmo (Nota 1.2.2). En ambos casos, la diferencia entre lo que se cuenta y lo que se quiere contar es un factor constante.

Los términos que tienen orden bajo, en relación con los demás términos que definen una función, tienden a ser irrelevantes a medida que el tamaño del problema a resolver aumenta.

Teorema 1.3.2: Subsunción por suma

Sea $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Si $g \in O(f)$, entonces $O(f + g) = O(f)$, en donde $f + g$ denota la suma punto a punto de f y g (i.e., $(f + g)(n) = f(n) + g(n)$).

Demostración

Se supone $g \in O(f)$ y se procede por doble inclusión.

- El objetivo es demostrar $O(f + g) \subseteq O(f)$. Si $h : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es tal que $h \in O(f + g)$, basta con demostrar $h \in O(f)$. Primero, por la suposición $g \in O(f)$, hay $n_0 \in \mathbb{N}$ y $c_0 \in \mathbb{R}_{> 0}$ tales que $g(n) \leq c_0 f(n)$ para $n \geq n_0$. Segundo, por la suposición $h \in O(f + g)$, hay $n_1 \in \mathbb{N}$ y $c_1 \in \mathbb{R}_{> 0}$ tales que $h(n) \leq c_1(f + g)(n)$ para $n \geq n_1$. Observe que, para $n \geq n_0 \uparrow n_1$:

$$\begin{aligned}
 h(n) &\leq c_1(f + g)(n) && \text{(por suposición si } n \geq n_1) \\
 &= c_1 f(n) + c_1 g(n) && \text{(por aritmética)} \\
 &\leq c_1 f(n) + c_1 c_0 f(n) && \text{(por suposición si } n \geq n_0) \\
 &= c_1(1 + c_0)f(n) && \text{(por aritmética).}
 \end{aligned}$$

Luego, con testigos $n_2 = n_0 \uparrow n_1$ y $c_2 = c_1(1 + c_0)$, se concluye $h \in O(f)$.

- El caso $O(f) \subseteq O(f + g)$ se propone como ejercicio al lector.

El Ejemplo 1.3.1 presenta un caso en el cual se pueden emplear los teoremas 1.3.1 y 1.3.2 para simplificar la clasificación asintótica de funciones. En conjunto, estas dos propiedades son de gran utilidad práctica pues permiten eliminar factores constantes e ignorar términos irrelevantes de una función.

Ejemplo 1.3.1

Una cota superior para la cantidad de operaciones que toma el algoritmo MergeSort es aproximadamente

$$6n \log n + 6n,$$

en donde n es la cantidad de elementos a ordenar. Entonces, la complejidad temporal de este algoritmo puede abstraerse asintóticamente así:

$$\begin{aligned}
 O(6n \log n + 6n) &= O(6n \log n) && \text{(por el Teorema 1.3.2)} \\
 &= O(n \log n) && \text{(por el Teorema 1.3.1).}
 \end{aligned}$$

El siguiente ejemplo muestra el ahorro de trabajo que representan estas propiedades frente al uso directo de la definición.

Ejemplo 1.3.2

En el Ejemplo 1.2.4 se requirió una demostración por contradicción, con testigos explícitos, para clasificar la cantidad de accesos al arreglo que hace el algoritmo *Acumulado*. Con las dos propiedades anteriores, el mismo resultado es inmediato:

$$\begin{aligned} O\left(\frac{N(N+1)}{2}\right) &= O\left(\frac{N^2}{2} + \frac{N}{2}\right) && \text{(por aritmética)} \\ &= O\left(\frac{N^2}{2}\right) && \text{(por el Teorema 1.3.2)} \\ &= O(N^2) && \text{(por el Teorema 1.3.1).} \end{aligned}$$

El segundo paso usa que $\frac{N}{2} \in O\left(\frac{N^2}{2}\right)$. Como se verá en el Teorema 1.3.4, la igualdad $O\left(\frac{N(N+1)}{2}\right) = O(N^2)$ equivale a $\frac{N(N+1)}{2} \in \Theta(N^2)$.

1.3.2. Comparación de órdenes. Los conjuntos contruidos a partir de la notación O satisfacen una especie de transitividad.

Teorema 1.3.3: Transitividad

Sean $f, g, h : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Si $h \in O(g)$ y $g \in O(f)$, entonces $h \in O(f)$.

Demostración

Se propone como ejercicio al lector.

También es posible usar la notación O para comparar funciones entre sí y, consecuentemente, comparar indirectamente la eficiencia relativa de diferentes algoritmos para resolver un problema dado.

Teorema 1.3.4

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

1. $O(f) = O(g)$ sii $f \in O(g)$ y $g \in O(f)$.
2. $O(f) \subset O(g)$ sii $f \in O(g)$ y $g \notin O(f)$.

Demostración

Se proponen como ejercicio al lector.

Vale la pena precisar qué tipo de relación entre funciones queda determinada por estas dos propiedades, pues el lenguaje habitual (“ f crece más lento que g ”) sugiere un orden que no es tal.

Nota 1.3.1

La relación $f \preceq g$, definida como $f \in O(g)$, es reflexiva y transitiva (Teorema 1.3.3), pero no es antisimétrica: n y $5n$ se acotan mutuamente sin ser la misma función. Es decir, $\preceq$ es un preorden y no un orden parcial. El Teorema 1.3.4 precisa las dos situaciones de interés: $O(f) = O(g)$ es la manera de decir que f y g crecen al mismo ritmo (equivalentemente, $f \in \Theta(g)$), mientras que la contención estricta $O(f) \subset O(g)$ es la manera de decir que f crece estrictamente más lento que g . Esta última es la relación con la que se ordenan funciones en la Sección 1.4.

1.3.3. Reglas de suma y producto. Finalmente, se presentan propiedades de la notación O que son especialmente útiles para analizar la complejidad temporal de algoritmos secuenciales e iterativos.

Teorema 1.3.5: Reglas de suma y producto

Sean $f, f_1, g, g_1 : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

1. $O(f + g) = O(f \uparrow g)$, en donde la suma y el máximo se interpretan punto a punto.
2. Si $f_1 \in O(f)$ y $g_1 \in O(g)$, entonces $f_1 g_1 \in O(fg)$, en donde el producto se interpreta punto a punto.

Demostración

Se demuestra la Propiedad 1; la Propiedad 2 se propone como ejercicio al lector. La observación clave es que, como f y g son no negativas, para cualquier $n \in \mathbb{N}$ se tiene:

$$(f \uparrow g)(n) \leq (f + g)(n) \leq 2(f \uparrow g)(n).$$

La primera desigualdad se da porque el término que no alcanza el máximo es no negativo y la segunda porque cada uno de los dos términos es a lo sumo el máximo. Se procede por doble inclusión.

- Sea $h \in O(f + g)$, con testigos $n_0 \in \mathbb{N}$ y $c_0 \in \mathbb{R}_{>0}$. Para $n \geq n_0$ se tiene $h(n) \leq c_0(f + g)(n) \leq 2c_0(f \uparrow g)(n)$, de donde $h \in O(f \uparrow g)$ con testigos n_0 y $2c_0$.
- Sea $h \in O(f \uparrow g)$, con testigos $n_0 \in \mathbb{N}$ y $c_0 \in \mathbb{R}_{>0}$. Para $n \geq n_0$ se tiene $h(n) \leq c_0(f \uparrow g)(n) \leq c_0(f + g)(n)$, de donde $h \in O(f + g)$ con testigos n_0 y c_0 .

La Propiedad 1 en el Teorema 1.3.5 se conoce como la *regla de la suma*, mientras que la Propiedad 2 como la *regla del producto*. La regla de la suma dice que, al ejecutar dos tareas una después de la otra, el costo asintótico lo determina la más costosa de las dos; la regla del producto, que al repetir una tarea una cantidad de veces dada por otra función, los órdenes se multiplican.

Ejemplo 1.3.3

Considere la función $3n^2 \log n + 10n^2 + 100$. Aplicando las propiedades anteriores:

$$\begin{aligned}
 & O(3n^2 \log n + 10n^2 + 100) \\
 &= O(3n^2 \log n + 10n^2) && \text{(subsunción por suma)} \\
 &= O(3n^2 \log n) && \text{(subsunción por suma)} \\
 &= O(n^2 \log n) && \text{(regla de constantes).}
 \end{aligned}$$

Los dos primeros pasos requieren, respectivamente, $100 \in O(3n^2 \log n + 10n^2)$ y $10n^2 \in O(3n^2 \log n)$; esta última se tiene porque $\log n \geq 1$ para $n \geq 2$.

1.3.4. El criterio del límite. Comparar dos funciones con la definición de O obliga a exhibir testigos o a razonar por contradicción. Cuando las funciones son suficientemente regulares, el cálculo de un límite reemplaza ese trabajo.

Teorema 1.3.6: Criterio del límite

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tales que $g(n) > 0$ para n suficientemente grande. Entonces:

1. Si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \in \mathbb{R}_{>0}$, entonces $O(f) = O(g)$ y, equivalentemente, $f \in \Theta(g)$.

2. Si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, entonces $O(f) \subset O(g)$; en particular, $f \in O(g)$ y $f \notin \Theta(g)$.
3. Si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$, entonces $O(g) \subset O(f)$.

La demostración del Teorema 1.3.6 se obtiene directamente de la definición de límite y se puede consultar en [BB88]. El criterio es una condición suficiente, pero no necesaria: hay funciones del mismo orden para las cuales el límite del cociente no existe (Ejercicio 15).

Ejemplo 1.3.4

Se desea comparar $\log n$ con $\sqrt{n}$. Usando la regla de L'Hôpital sobre las funciones reales correspondientes:

$$\lim_{n \rightarrow \infty} \frac{\log n}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{1/(n \ln 2)}{1/(2\sqrt{n})} = \lim_{n \rightarrow \infty} \frac{2}{\ln 2 \cdot \sqrt{n}} = 0.$$

Por la Propiedad 2 del Teorema 1.3.6, se concluye $O(\log n) \subset O(\sqrt{n})$: el logaritmo crece estrictamente más lento que la raíz cuadrada. Obtener esta misma conclusión con la definición de O requiere bastante más trabajo.

El segundo ejemplo compara un polinomio con una exponencial, una comparación que se repite a lo largo del texto cada vez que se discute si un problema admite una solución práctica.

Ejemplo 1.3.5

Se desea comparar n^{100} con 2^n . Aplicando la regla de L'Hôpital cien veces:

$$\lim_{n \rightarrow \infty} \frac{n^{100}}{2^n} = \lim_{n \rightarrow \infty} \frac{100!}{(\ln 2)^{100} 2^n} = 0,$$

de donde $O(n^{100}) \subset O(2^n)$. Es decir, cualquier algoritmo con complejidad polinomial, así sea de grado 100, es asintóticamente preferible a uno con complejidad exponencial.

1.3.5. Notación asintótica en expresiones. Es común encontrar la notación asintótica dentro de expresiones aritméticas. Por ejemplo, en la Sección 1.7 aparece una desigualdad de la forma

$$T(n) \leq aT(n/b) + O(n^k),$$

en donde $O(n^k)$ ocupa el lugar de un sumando. La siguiente convención le da sentido preciso a este uso.

Nota 1.3.2

Cuando $O(f)$ aparece dentro de una expresión aritmética, denota *alguna* función anónima del conjunto $O(f)$. Así, la expresión $n^2 + O(n)$ denota cualquier función de la forma $n^2 + h(n)$, con $h \in O(n)$, y la desigualdad $T(n) \leq 2T(n/2) + O(n)$ afirma que existe $h \in O(n)$ tal que $T(n) \leq 2T(n/2) + h(n)$. Una igualdad entre dos expresiones de este tipo se lee de izquierda a derecha: significa que toda función denotada por el lado izquierdo también es denotada por el lado derecho.

La lectura de izquierda a derecha de la Nota 1.3.2 es esencial. Por ejemplo, $n + O(\sqrt{n}) = O(n)$ es una afirmación cierta, mientras que $O(n) = n + O(\sqrt{n})$ es falsa. Esta asimetría es la misma que se señaló en la Nota 1.2.3 y explica por qué una cadena de igualdades con notación asintótica no puede recorrerse en ambos sentidos, como lo ilustra el Ejercicio 13.

Ejercicios

1. Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Demuestre que, si $c \in \mathbb{R}_{>0}$, entonces:
 - a) $\Omega(f) = \Omega(cf)$.
 - b) $\Theta(f) = \Theta(cf)$.
2. Complete la demostración del Teorema 1.3.2 con el caso $O(f) \subseteq O(f + g)$.
3. Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tales que $g \in O(f)$. Demuestre o refute:
 - a) $\Omega(f + g) = \Omega(f)$.
 - b) $\Theta(f + g) = \Theta(f)$.
4. Demuestre el Teorema 1.3.3.
5. Sean $f, g, h : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Demuestre o refute:
 - a) Si $h \in \Omega(g)$ y $g \in \Omega(f)$, entonces $h \in \Omega(f)$.
 - b) Si $h \in \Theta(g)$ y $g \in \Theta(f)$, entonces $h \in \Theta(f)$.
6. Demuestre la Propiedad 1 del Teorema 1.3.4.
7. Demuestre la Propiedad 2 del Teorema 1.3.4.
8. Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Demuestre o refute:
 - a) $\Omega(f) = \Omega(g)$ sii $f \in \Omega(g)$ y $g \in \Omega(f)$.
 - b) $\Omega(f) \subset \Omega(g)$ sii $f \in \Omega(g)$ y $g \notin \Omega(f)$.
 - c) $\Theta(f) = \Theta(g)$ sii $f \in \Theta(g)$ y $g \in \Theta(f)$.
 - d) $\Theta(f) \subset \Theta(g)$ sii $f \in \Theta(g)$ y $g \notin \Theta(f)$.
9. Demuestre la Propiedad 2 del Teorema 1.3.5.

10. Formule duales para Ω y Θ de las propiedades de suma y producto en el Teorema 1.3.5. Justifique su respuesta con demostraciones.
11. Proponga funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tales que $f \notin O(g)$ y $g \notin O(f)$. Justifique su respuesta.
12. Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Demuestre que las siguientes propiedades son equivalentes:
 - a) $O(f) = O(g)$.
 - b) $\Theta(f) = \Theta(g)$.
 - c) $f \in \Theta(g)$.
13. Considere la siguiente argumentación:

$$O(n^2) = O(n^3 + (n^2 - n^3)) \quad (\text{por aritmética})$$

$$= O(n^3) \quad (\text{por el Teorema 1.3.2 con } n^2 - n^3 \in O(n^3)).$$
 Claramente $O(n^2) \neq O(n^3)$. Entonces, ¿cuál es el error en la argumentación?
14. Usando el Teorema 1.3.6, compare los siguientes pares de funciones:
 - a) $n \log n$ y $n^{1.1}$.
 - b) 2^n y 3^n .
 - c) $\log n$ y $\log(n^2)$.
15. Encuentre funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tales que $O(f) = O(g)$, pero para las cuales $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ no exista.
16. De acuerdo con la Nota 1.3.2, decida cuáles de las siguientes afirmaciones son ciertas y justifique su respuesta:
 - a) $n^2 + O(n) = O(n^2)$.
 - b) $O(n^2) = n^2 + O(n)$.
 - c) $O(n) + O(n^2) = O(n^2)$.

1.4. Funciones y sumas de referencia

Las secciones anteriores presentan la notación asintótica y sus propiedades, pero no dicen nada sobre cuáles funciones aparecen en la práctica ni sobre cómo se comparan entre sí. Esta sección reúne ese catálogo: la jerarquía de las familias de funciones que surgen al analizar algoritmos, las identidades de logaritmos y factoriales que se usan con más frecuencia, y las sumas que producen los conteos de las secciones 1.5 y 1.6. Es una sección de consulta: varios enunciados se presentan sin demostración cuando esta se reduce a un cálculo, indicando siempre con cuál herramienta de la Sección 1.3 se obtendría.

1.4.1. La jerarquía de crecimiento. Al analizar algoritmos aparece, una y otra vez, un puñado de familias de funciones: constantes, logaritmos, raíces, polinomios, exponenciales y factoriales. El primer resultado de esta sección ordena

esas familias entre sí. Las contenciones son estrictas, de modo que cada una crece efectivamente más rápido que la anterior, en el sentido preciso de la Nota 1.3.1.

Teorema 1.4.1: Jerarquía de crecimiento

Todas las contenciones siguientes son estrictas:

$$\begin{aligned} O(1) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \log n) \\ \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!). \end{aligned}$$

Demostración

Cada contención se obtiene aplicando la Propiedad 2 del Teorema 1.3.6 al par de funciones correspondiente. El Ejemplo 1.3.4 desarrolla el caso $O(\log n) \subset O(\sqrt{n})$; los demás son análogos y se proponen como ejercicio al lector.

Aquí $O(1)$ denota el orden de la función constante 1, es decir, el conjunto de las funciones que eventualmente están acotadas por una constante. Es común escribir “costo constante” en lugar de “costo en $O(1)$ ”.

La cadena del Teorema 1.4.1 cubre los casos más frecuentes, pero no todos. Las siguientes propiedades generales se encargan del resto.

Teorema 1.4.2

Sean $a, b \in \mathbb{R}_{>0}$ y $c \in \mathbb{R}_{>1}$. Todas las contenciones siguientes son estrictas:

1. $O(n^a) \subset O(n^b)$, siempre que $a < b$.
2. $O((\log n)^a) \subset O(n^b)$: cualquier potencia de un logaritmo crece más lento que cualquier potencia positiva de n .
3. $O(n^a) \subset O(c^n)$: cualquier polinomio crece más lento que cualquier exponencial.
4. $O(c^n) \subset O(n!)$: cualquier exponencial crece más lento que el factorial.

Demostración

Las tres primeras propiedades se obtienen con el Teorema 1.3.6, como en los ejemplos 1.3.4 y 1.3.5. Para la última, observe que $\frac{c^n}{n!} = (\times k \mid 1 \leq k \leq n : \frac{c}{k})$ y que cada factor con $k \geq 2c$ es a lo sumo $\frac{1}{2}$, de modo que el producto tiende a 0. Los detalles se proponen como ejercicio al lector.

La Figura 4 muestra estas funciones para valores pequeños de n . Nótese que el eje vertical está en escala logarítmica: sin ella, las curvas de $n!$ y 2^n harían invisibles a todas las demás.

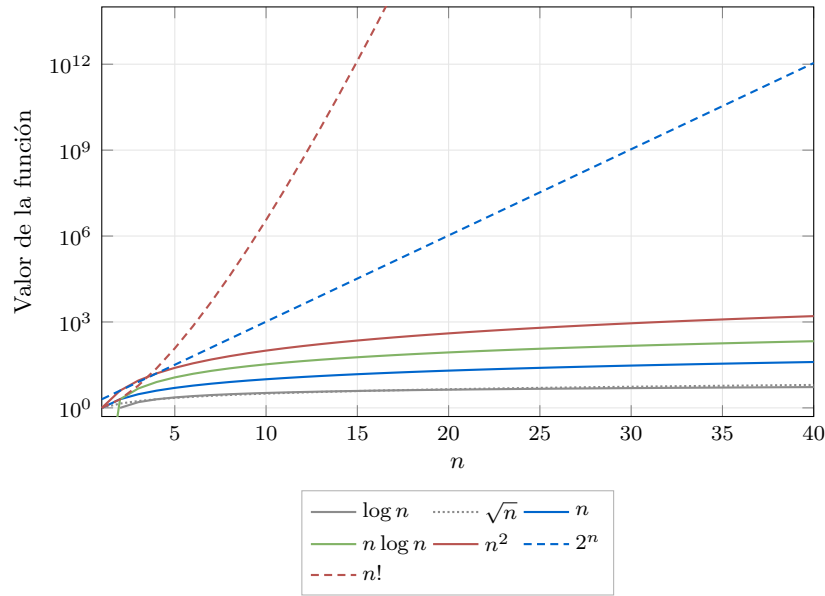


Figura 4. Funciones de referencia para $1 \leq n \leq 40$. El eje vertical está en escala logarítmica.

La diferencia entre estas familias no es una sutileza matemática: es la diferencia entre poder resolver un problema y no poder hacerlo.

Ejemplo 1.4.1

Suponga una máquina que ejecuta 10^9 operaciones elementales por segundo. La siguiente tabla muestra el mayor tamaño de entrada que alcanza a procesar un algoritmo cuyo costo es la función indicada, según el tiempo disponible:

Costo	1 segundo	1 minuto	1 hora
$\log n$	2^{10^9}	$2^{6 \times 10^{10}}$	$2^{3,6 \times 10^{12}}$
$\sqrt{n}$	10^{18}	$3,6 \times 10^{21}$	$1,3 \times 10^{25}$
n	10^9	6×10^{10}	$3,6 \times 10^{12}$
$n \log n$	$3,9 \times 10^7$	$1,9 \times 10^9$	$9,9 \times 10^{10}$
n^2	31 622	244 948	1 897 366
n^3	1 000	3 914	15 326
2^n	29	35	41
$n!$	12	13	15

La lectura interesante es la de las filas inferiores: multiplicar por 3600 el tiempo disponible le agrega apenas doce unidades al tamaño alcanzable por un algoritmo exponencial y tres al de uno factorial. Con esas familias, esperar más tiempo o comprar una máquina más rápida no resuelve el problema: hay que cambiar de algoritmo.

1.4.2. Logaritmos, exponenciales y factoriales. Las tres familias que más aparecen al analizar algoritmos recursivos son también las que más manipulación algebraica exigen. Esta subsección reúne las identidades que se usan a diario y el único resultado no elemental que el texto necesita sobre el factorial.

La primera nota recoge las identidades de logaritmos.

Nota 1.4.1

Las siguientes identidades se usan con frecuencia al analizar algoritmos, para $a, b, c, n \in \mathbb{R}_{>0}$ y $b > 1$:

$$\begin{aligned} \log(ac) &= \log a + \log c, & \log\left(\frac{a}{c}\right) &= \log a - \log c, \\ \log(a^c) &= c \log a, & \log_b n &= \frac{\log n}{\log b}, \\ 2^{\log n} &= n, & a^{\log_b n} &= n^{\log_b a}. \end{aligned}$$

La cuarta identidad, junto con el Teorema 1.3.1, es la que permite ignorar la base del logaritmo: $\log_b n$ y $\log n$ difieren en el factor constante $\frac{1}{\log b} > 0$, de modo que $\Theta(\log_b n) = \Theta(\log n)$.

La segunda nota se refiere a una notación compacta que es cómoda pero se presta a confusión.

Nota 1.4.2

Es habitual escribir $\log^2 n$ para $(\log n)^2$ y, en general, $\log^a n$ para $(\log n)^a$. Conviene no confundir esta notación con $\log n^2$, que es $2 \log n$ por la tercera identidad de la Nota 1.4.1. Las dos funciones tienen órdenes distintos: $\log n^2 \in \Theta(\log n)$, mientras que $\log^2 n \notin O(\log n)$.

El factorial es más difícil de manipular que las otras familias, porque no tiene una expresión cerrada sencilla. La aproximación de Stirling suple esa carencia: expresa a $n!$ en términos de funciones conocidas, con un error que se desvanece.

Teorema 1.4.3: Aproximación de Stirling

Para $n \geq 1$:

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right).$$

El Teorema 1.4.3 se enuncia sin demostración; una se encuentra en [CLRS22]. Su consecuencia más usada en este texto es la siguiente.

Corolario 1.4.4

$\log(n!) \in \Theta(n \log n)$.

Demostración

Al tomar logaritmo en el Teorema 1.4.3 y usar las identidades de la Nota 1.4.1:

$$\log(n!) = n \log n - n \log e + \frac{1}{2} \log(2\pi n) + \Theta\left(\frac{1}{n}\right).$$

Para la cota superior, los tres últimos términos están en $O(n \log n)$, de donde $\log(n!) \leq 2n \log n$ para n suficientemente grande. Para la cota inferior, note que $n \log e \leq \frac{1}{2}n \log n$ cuando $\log n \geq 2 \log e$, i.e., cuando $n \geq 8$; entonces $\log(n!) \geq \frac{1}{2}n \log n$ para n suficientemente grande. Luego $\log(n!) \in \Theta(n \log n)$.

El Corolario 1.4.4 explica por qué el orden $n \log n$ aparece tan seguido en algoritmos de ordenamiento: un algoritmo que ordena comparando elementos debe distinguir entre las $n!$ permutaciones posibles de la entrada y, con cada comparación, cuando mucho divide en dos el conjunto de candidatas. Se necesitan entonces al menos $\log(n!)$ comparaciones, que es $\Theta(n \log n)$.

1.4.3. Sumas de referencia. Analizar un ciclo consiste, casi siempre, en sumar el costo de sus iteraciones, y analizar una función recurrente consiste en sumar

el costo de sus niveles. Por ello, el análisis de algoritmos se apoya en un repertorio reducido de sumas. El siguiente teorema reúne las que se usan en el resto del capítulo, con su fórmula cerrada cuando la tienen y con su orden en todos los casos.

Teorema 1.4.5: Sumas de referencia

Sean $n \in \mathbb{N}$ y $r \in \mathbb{R}_{>0}$ con $r \neq 1$:

1. $(+k \mid 0 \leq k < n : k) = \frac{n(n-1)}{2} \in \Theta(n^2)$.
2. $(+k \mid 0 \leq k < n : k^2) = \frac{(n-1)n(2n-1)}{6} \in \Theta(n^3)$ y, en general, $(+k \mid 0 \leq k < n : k^a) \in \Theta(n^{a+1})$ para $a \in \mathbb{R}_{\geq 0}$.
3. $(+k \mid 0 \leq k \leq n : r^k) = \frac{r^{n+1}-1}{r-1}$. Además, esta suma está en $\Theta(1)$ si $r < 1$ y en $\Theta(r^n)$ si $r > 1$.
4. $H_n = (+k \mid 1 \leq k \leq n : \frac{1}{k}) \in \Theta(\log n)$, denominada la *suma armónica*.
5. $(+k \mid 1 \leq k \leq n : \log k) = \log(n!) \in \Theta(n \log n)$.

Demostración

Las fórmulas cerradas de las Propiedades 1, 2 y 3 se demuestran por inducción sobre n y se proponen como ejercicio al lector. Los órdenes de las Propiedades 2 y 4 se obtienen con la Nota 1.4.4, y el de la Propiedad 5 es el Corolario 1.4.4.

De las cinco sumas anteriores, la geométrica merece un comentario aparte, porque su comportamiento es el que después explica la forma de los árboles de recurrencia.

Nota 1.4.3

La Propiedad 3 del Teorema 1.4.5 merece atención porque su lectura reaparece en la Sección 1.6. Una suma geométrica de razón menor que 1 es, salvo un factor constante, su primer término; una de razón mayor que 1 es, salvo un factor constante, su último término. Es decir, en una suma geométrica siempre domina uno de los dos extremos.

Cuando una suma no está en el repertorio anterior, casi siempre basta con acotarla en lugar de calcularla. La siguiente nota describe la herramienta más útil para hacerlo.

Nota 1.4.4

Cuando $f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ es monótona, una suma de sus valores se acota por integrales. Si f es creciente:

$$\int_0^n f(x) dx \leq (+k \mid 1 \leq k \leq n : f(k)) \leq \int_1^{n+1} f(x) dx,$$

y las desigualdades se invierten si f es decreciente. Cada término de la suma es el área de un rectángulo de base 1 y altura $f(k)$, y las integrales acotan la unión de esos rectángulos por debajo y por encima.

Las dos sumas del siguiente ejemplo ilustran el procedimiento y, de paso, completan la demostración del Teorema 1.4.5.

Ejemplo 1.4.2

Dos aplicaciones de la Nota 1.4.4:

1. Como $\frac{1}{x}$ es decreciente, $\int_1^{n+1} \frac{dx}{x} \leq H_n \leq 1 + \int_1^n \frac{dx}{x}$, es decir, $\ln(n+1) \leq H_n \leq 1 + \ln n$. Por el Teorema 1.3.1, $H_n \in \Theta(\log n)$.
2. Como x^a es creciente para $a \geq 0$, la suma $(+k \mid 1 \leq k \leq n : k^a)$ está entre $\frac{n^{a+1}}{a+1}$ y $\frac{(n+1)^{a+1}}{a+1}$, de donde está en $\Theta(n^{a+1})$.

El siguiente ejemplo muestra de dónde salen los conteos que la Sección 1.1 presentó sin justificar.

Ejemplo 1.4.3

El algoritmo Exhaustivo recorre el subarreglo $A[i..j]$ para cada par (i, j) con $0 \leq i \leq j \leq N$, haciendo $j - i$ accesos al arreglo. La cantidad total de accesos es entonces una suma doble. Sustituyendo $d = j - i$ en la suma interna y luego $m = N - i$ en la externa:

$$\begin{aligned} & (+i, j \mid 0 \leq i \leq j \leq N : j - i) \\ &= (+i \mid 0 \leq i \leq N : (+d \mid 0 \leq d \leq N - i : d)) \\ &= (+i \mid 0 \leq i \leq N : \frac{(N-i)(N-i+1)}{2}) \\ &= (+m \mid 0 \leq m \leq N : \frac{m(m+1)}{2}), \end{aligned}$$

en donde el segundo paso usa la Propiedad 1 del Teorema 1.4.5. La última suma vale $\frac{N(N+1)(N+2)}{6}$, que es el conteo reportado en el Ejemplo 1.1.4. Para

clasificarlo asintóticamente no hace falta la fórmula exacta: por la Propiedad 2, la suma de $\frac{m(m+1)}{2} \in \Theta(m^2)$ sobre $0 \leq m \leq N$ está en $\Theta(N^3)$.

Ejercicios

1. Justificando su respuesta, clasifique ascendentemente por orden asintótico las siguientes funciones:
 2^{2^n} $\log \sqrt{n}$ $n^2 \log n$ $\log(n!)$ $n!$ $n^{2,5}$
2. Justificando su respuesta, clasifique ascendentemente por orden asintótico las siguientes funciones:
 $\sqrt{2n}$ $\left(\frac{3}{2}\right)^n$ $(\log n)^2$ $2^{\sqrt{2 \log n}}$ $n \log n$ $\log(n^2)$
3. Demuestre que $\log_a n \in \Theta(\log_b n)$ para cualesquiera $a, b \in \mathbb{R}_{>1}$.
4. Complete la demostración del Teorema 1.4.1 para los pares $O(n) \subset O(n \log n)$ y $O(n^3) \subset O(2^n)$.
5. Complete la demostración de la Propiedad 4 del Teorema 1.4.2.
6. Ubique las funciones $n^{\log n}$ y $(\log n)^{\log n}$ en la jerarquía del Teorema 1.4.1. ¿Alguna de ellas es polinomial? ¿Alguna es exponencial?
7. Demuestre por inducción las fórmulas cerradas de las Propiedades 1 y 3 del Teorema 1.4.5.
8. Usando la fórmula cerrada de la Propiedad 3 del Teorema 1.4.5, demuestre que la suma está en $\Theta(1)$ cuando $r < 1$ y en $\Theta(r^n)$ cuando $r > 1$.
9. Determine el orden de las siguientes sumas:
 - a) $(+k \mid 1 \leq k \leq n : k^3)$.
 - b) $(+k \mid 1 \leq k \leq n : k 2^k)$.
 - c) $(+k \mid 1 \leq k \leq n : \frac{n}{k})$.
10. Repita la tabla del Ejemplo 1.4.1 suponiendo una máquina diez veces más rápida. ¿Por cuánto se multiplica el tamaño alcanzable en cada fila?
11. Demuestre que, si la velocidad de la máquina se duplica, el tamaño alcanzable por un algoritmo de costo 2^n aumenta en una constante, mientras que el de uno de costo n^2 se multiplica por una constante.
12. Un algoritmo con costo n^2 procesa una entrada de tamaño 10 000 en un segundo. Estime cuánto tardaría con una entrada de tamaño 100 000. Repita el cálculo para costos $n \log n$ y 2^n .
13. Calcule, con la suma doble del Ejemplo 1.4.3, la cantidad de accesos al arreglo que hace el algoritmo *Acumulado* y verifique que coincide con $\frac{N(N+1)}{2}$.

1.5. Análisis de algoritmos iterativos

Las secciones anteriores construyen el lenguaje para comparar funciones y el repertorio de funciones y sumas que aparecen al hacerlo, pero no explican de dónde sale la función asociada a un algoritmo. La Sección 1.1 reportó, por ejemplo, que el algoritmo *Acumulado* hace $\frac{N(N+1)}{2}$ accesos al arreglo, y prometió deducirlo más adelante. Esta sección cumple esa promesa para los algoritmos iterativos, i.e., aquellos cuyo trabajo se organiza en secuencias, condicionales y ciclos.

Los algoritmos se siguen describiendo en prosa o en notación matemática, como en la Sección 1.1; la forma de expresarlos en Python, y el costo de cada construcción de ese lenguaje, se abordan en la Sección 1.8.

1.5.1. De un algoritmo a una función. Asociar una función a un algoritmo iterativo es un proceso mecánico que descompone el algoritmo en sus construcciones y traduce cada una en una expresión de costo. La siguiente metodología resume ese proceso, que en esencia consiste en leer el algoritmo de adentro hacia afuera.

Metodología 1.5.1: Análisis de un algoritmo iterativo

Sea A un algoritmo y x una instancia. El costo $\text{costo}_A(x)$ de la Definición 1.1.1 se calcula así:

1. Una operación elemental cuesta 1.
2. Una secuencia $S_0; S_1$ cuesta la suma de los costos de S_0 y S_1 .
3. Un condicional cuesta la evaluación de la guarda más el costo de la rama que se ejecute. En el peor caso, se toma el máximo de las ramas; en el mejor caso, el mínimo.
4. Un ciclo que ejecuta K iteraciones, en donde la iteración k cuesta c_k y la evaluación de la guarda que precede a esa iteración cuesta g_k , cuesta

$$(+k \mid 0 \leq k \leq K : g_k) + (+k \mid 0 \leq k < K : c_k),$$

en donde la primera suma recorre las $K + 1$ evaluaciones de la guarda: una antes de cada iteración y una final, que es la que termina el ciclo. El costo de la guarda también puede variar entre iteraciones, por lo que en general es una suma y no un producto.

5. Un ciclo anidado dentro de otro produce una suma anidada: el costo c_k de cada iteración del ciclo externo es, a su vez, una suma.

Para clasificar el resultado asintóticamente basta con contar una operación barómetro (Nota 1.1.4) en lugar de todas las operaciones elementales.

Las sumas que produce la Metodología 1.5.1 son precisamente las del Teorema 1.4.5, y su simplificación usa las reglas de la Sección 1.3. En el caso habitual, en el que la guarda consiste en comparar un índice con una cantidad ya calculada, se tiene $g_k \in \Theta(1)$ y la primera suma vale $\Theta(K)$; si además el cuerpo del ciclo cuesta al menos una operación por iteración, la regla de la suma permite ignorarla. Ninguna de las dos condiciones es automática, como lo muestra el siguiente ejemplo.

Ejemplo 1.5.1

Considere el ciclo que, dado un arreglo $A[0..N)$ de números positivos y una cota S , avanza un índice k mientras la suma de $A[0..k)$ sea menor que S y cuyo cuerpo se limita a incrementar k .

Si la guarda se evalúa recalculando la suma desde cero, su costo es $g_k = \Theta(k)$, mientras que el del cuerpo es $c_k \in \Theta(1)$. Por la Metodología 1.5.1, el costo del ciclo es

$$(+k \mid 0 \leq k \leq K : \Theta(k)) + \Theta(K) = \Theta(K^2),$$

de modo que aquí son las guardas, no el cuerpo, las que determinan el costo. Si en cambio la guarda se evalúa manteniendo la suma en una variable que el cuerpo actualiza, entonces $g_k \in \Theta(1)$ y el costo del ciclo baja a $\Theta(K)$.

El paso que más errores produce es el cuarto, cuando se reemplaza alguna de las dos sumas por un producto. La siguiente nota precisa cuándo ese reemplazo es válido.

Nota 1.5.1

Si todas las iteraciones cuestan lo mismo, la suma $(+k \mid 0 \leq k < K : c_k)$ es el producto $K \cdot c$ y la regla del producto (Teorema 1.3.5) permite multiplicar los órdenes; lo mismo aplica a la suma de las guardas. Si el costo varía entre iteraciones, acotar cada c_k (o cada g_k) por el mayor de ellos produce una cota superior correcta, pero no necesariamente ajustada: el resultado es un O , no un Θ . Para afirmar Θ hay que calcular la suma o acotarla también por debajo.

1.5.2. Ciclos de recorrido fijo. Los ciclos más sencillos de analizar son aquellos cuya cantidad de iteraciones depende solo del tamaño de la entrada y no de su contenido. En ellos el peor y el mejor caso coinciden, de modo que la Nota 1.1.5 no exige ninguna precisión adicional. Los tres algoritmos del problema conductor son de este tipo.

Ejemplo 1.5.2

El algoritmo **Incremental** recorre el arreglo una vez y, en cada iteración, hace un acceso a A y una cantidad constante de operaciones adicionales. Por la Metodología 1.5.1, su costo es

$$(+j \mid 0 \leq j \leq N : g) + (+j \mid 0 \leq j < N : c) \in \Theta(N),$$

para constantes g y c , pues tanto la guarda como el cuerpo cuestan una cantidad fija de operaciones. La cantidad de accesos al arreglo es exactamente N .

El algoritmo **Acumulado** tiene dos ciclos anidados: el externo fija el extremo izquierdo i y el interno recorre los extremos derechos j , con $i \leq j < N$, haciendo un acceso a A en cada iteración. La cantidad de accesos es entonces

$$\begin{aligned} & (+i \mid 0 \leq i \leq N : (+j \mid i \leq j < N : 1)) \\ &= (+i \mid 0 \leq i \leq N : N - i) \\ &= (+m \mid 0 \leq m \leq N : m) = \frac{N(N+1)}{2}, \end{aligned}$$

con el cambio de variable $m = N - i$ y la Propiedad 1 del Teorema 1.4.5. Luego su costo está en $\Theta(N^2)$.

El algoritmo **Exhaustivo** agrega un tercer ciclo, el que recorre $A[i..j]$ para calcular su suma desde cero. La suma triple resultante se calculó en el Ejemplo 1.4.3: son $\frac{N(N+1)(N+2)}{6}$ accesos, de modo que su costo está en $\Theta(N^3)$.

Estos tres conteos son los que la Sección 1.1 reportó y los que explican la forma de las curvas de la Figura 1. Vale la pena detenerse en el factor $\frac{1}{2}$ que aparece en el análisis de **Acumulado**, pues es característico de los ciclos anidados en los que el ciclo interno arranca donde va el externo.

La Figura 5 muestra los pares (i, j) que visita el ciclo doble de **Acumulado**. Son los de la mitad superior de la cuadrícula, incluida la diagonal: casi la mitad de los N^2 pares posibles. De ahí el factor $\frac{1}{2}$ y, sobre todo, la conclusión que interesa: un ciclo doble “triangular” es tan cuadrático como uno que recorra toda la cuadrícula, pues las dos cantidades difieren en un factor constante.

Cuando el tamaño de la entrada tiene varios parámetros, cada ciclo aporta el suyo.

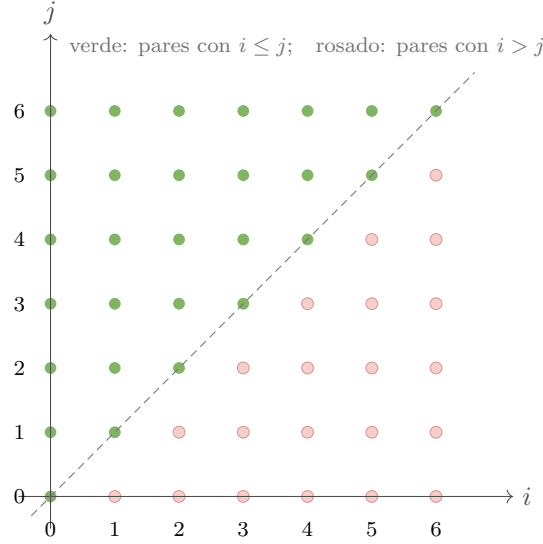


Figura 5. Pares (i, j) con $0 \leq i \leq j < N$, para $N = 7$. El algoritmo **Acumulado** visita los pares verdes, que son aproximadamente la mitad de la cuadrícula.

Ejemplo 1.5.3

El algoritmo que suma los elementos de una matriz $A[0..M][0..N]$ (Ejemplo 1.1.3) recorre las M filas y, para cada una, sus N columnas. La cantidad de accesos es

$$\begin{aligned} & (+i \mid 0 \leq i < M : (+j \mid 0 \leq j < N : 1)) \\ & = (+i \mid 0 \leq i < M : N) = MN, \end{aligned}$$

de modo que su costo está en $\Theta(MN)$, en el sentido de la Nota 1.2.4. Nótese que $\Theta(MN)$ es informativo, mientras que resumir el tamaño en $P = MN$ y decir que el algoritmo es $\Theta(P)$ esconde la estructura de la entrada.

1.5.3. Ciclos que dependen de los datos. Cuando la cantidad de iteraciones depende del contenido de la entrada y no solo de su tamaño, el análisis debe distinguir casos. Aquí reaparecen las definiciones 1.1.2 y 1.1.3: hay que acotar el costo para toda instancia y, además, exhibir instancias que alcancen la cota.

Ejemplo 1.5.4

La búsqueda lineal de un valor x en $A[0..N)$ recorre el arreglo de izquierda a derecha y se detiene al encontrar x . Si x está en la posición p , el ciclo hace $p + 1$ comparaciones; si no está, hace N .

En el peor caso, el ciclo recorre el arreglo completo, lo cual ocurre cuando x no está en A o está únicamente en la última posición: $T^\uparrow(N) = N \in \Theta(N)$. En el mejor caso, x está en la posición 0 y basta una comparación: $T^\downarrow(N) = 1 \in \Theta(1)$.

El ordenamiento por inserción, descrito en la Sección 1.1, es el ejemplo en el que la diferencia entre los dos casos es más pronunciada. Su ciclo interno es el que depende de los datos: la cantidad de desplazamientos de cada elemento depende de qué tan lejos esté de su posición final.

Ejemplo 1.5.5

Al procesar la posición k , con $1 \leq k < N$, el ordenamiento por inserción compara $A[k]$ con su vecino izquierdo y lo intercambia mientras este sea mayor. Sea c_k la cantidad de comparaciones que hace el ciclo interno en ese paso. Cada comparación, salvo a lo sumo la última, produce un desplazamiento y el elemento no puede desplazarse más allá de la posición 0; luego $1 \leq c_k \leq k$. La cantidad total de comparaciones es

$$(+k \mid 1 \leq k < N : c_k).$$

Si el arreglo ya está ordenado, la primera comparación de cada paso falla y ningún elemento se desplaza, i.e., $c_k = 1$ para todo k , de modo que la suma vale $N - 1$. Si el arreglo está invertido, cada elemento se desplaza hasta la posición 0, i.e., $c_k = k$, y la suma vale

$$(+k \mid 1 \leq k < N : k) = \frac{N(N-1)}{2}.$$

Esta última cantidad es también la cota superior que se obtiene al reemplazar cada c_k por k , de modo que ninguna instancia la supera. Se concluye $T^\uparrow(N) = \frac{N(N-1)}{2} \in \Theta(N^2)$ y $T^\downarrow(N) = N - 1 \in \Theta(N)$, que son exactamente los valores medidos en la Figura 2.

El Ejemplo 1.5.5 ilustra el procedimiento general para estos casos y conviene enunciarlo por separado, pues es la fuente de la mayoría de los análisis incorrectos.

Nota 1.5.2

Para establecer que el costo en el peor caso de un algoritmo está en $\Theta(f)$ hacen falta dos argumentos: una cota superior, que se obtiene acotando el costo de cada iteración y sumando, y una instancia concreta de cada tamaño que alcance esa cota. Sin lo segundo solo se justifica $T^\uparrow \in O(f)$, pues la cota superior podría ser inalcanzable (Nota 1.5.1).

1.5.4. Ciclos que no avanzan de uno en uno. Todos los ciclos anteriores recorren sus índices de uno en uno, de modo que la cantidad de iteraciones es del orden del recorrido. Los ciclos en los que el índice se multiplica o se divide por una constante se comportan de manera distinta y son el origen de los logaritmos en el análisis de algoritmos.

Ejemplo 1.5.6

Considere el algoritmo que determina la cantidad de bits en la representación binaria de un número $n \geq 1$: mientras n sea mayor que 0, reemplaza n por $\lfloor n/2 \rfloor$ y cuenta una iteración.

Después de k iteraciones, el valor de la variable es $\lfloor n/2^k \rfloor$, que es 0 por primera vez cuando $2^k > n$, es decir, cuando $k > \log n$. El ciclo hace entonces $\lfloor \log n \rfloor + 1$ iteraciones, cada una de costo constante, y el costo del algoritmo está en $\Theta(\log n)$.

El argumento del Ejemplo 1.5.6 se aplica sin cambios a cualquier ciclo cuyo índice se multiplique o se divida por una constante $b > 1$: la cantidad de iteraciones es $\Theta(\log_b n)$ y, por la Nota 1.2.2, la base es irrelevante.

Nota 1.5.3

Al combinar esta observación con la regla del producto se obtiene el orden $n \log n$, que es el de varios de los algoritmos más usados: un ciclo de $\Theta(\log n)$ iteraciones anidado dentro de uno de $\Theta(n)$ iteraciones, con cuerpo de costo constante, tiene costo $\Theta(n \log n)$. La Sección 1.6 muestra que ese mismo orden aparece, por otra vía, al resolver recurrencias.

1.5.5. Espacio. El análisis del espacio auxiliar sigue las mismas reglas, contando celdas en lugar de operaciones. La diferencia es que las celdas se reutilizan: lo que interesa es la cantidad máxima ocupada simultáneamente, no la cantidad total de asignaciones.

Ejemplo 1.5.7

Los tres algoritmos del Ejemplo 1.5.2 usan una cantidad fija de variables, sin importar N : índices y acumuladores. Su espacio auxiliar está entonces en $\Theta(1)$, y lo mismo ocurre con la búsqueda lineal y con el ordenamiento por inserción, que reordena el arreglo de la entrada mediante intercambios.

El algoritmo Prefijos del Ejemplo 1.1.5 es el único que se aparta: su arreglo auxiliar de sumas prefijas ocupa $N + 1$ celdas, de modo que su espacio auxiliar está en $\Theta(N)$, aunque su costo en tiempo sea del mismo orden que el de Acumulado.

Ejercicios

1. El ordenamiento por selección recorre las posiciones k de 0 a $N - 1$ y, para cada una, busca el menor elemento de $A[k..N)$ y lo intercambia con $A[k]$. Determine la cantidad exacta de comparaciones entre elementos y clasifíquela asintóticamente. ¿Coinciden el peor y el mejor caso?
2. El ordenamiento de burbuja recorre repetidamente el arreglo intercambiando pares adyacentes que estén en desorden, hasta que un recorrido completo no haga ningún intercambio. Determine su costo en el peor y en el mejor caso.
3. Considere el algoritmo que multiplica una matriz de M filas y K columnas por una de K filas y N columnas, calculando cada una de las MN entradas del resultado como una suma de K productos. Determine su costo y su espacio auxiliar.
4. Considere el ciclo que mantiene dos índices, uno que avanza desde 0 y otro que retrocede desde $N - 1$, y que termina cuando se cruzan. Suponga que cada iteración avanza exactamente uno de los dos índices. Determine la cantidad de iteraciones.
5. Un algoritmo cuenta los pares (i, j) con $0 \leq i < j < N$ tales que $A[i] + A[j] = 0$, revisando todos los pares. Determine su costo y compárelo con el del Ejemplo 1.5.2.
6. Determine la cantidad de iteraciones de un ciclo que parte de n y divide por 3 en cada iteración hasta llegar a 0. Justifique su respuesta como en el Ejemplo 1.5.6.
7. 🌈 Determine la cantidad de iteraciones de un ciclo que parte de $i = 1$ y en cada iteración hace $i := i + \lfloor i/2 \rfloor$, hasta que i supere a n .
8. Considere el siguiente análisis del algoritmo Acumulado: “el ciclo externo hace $N + 1$ iteraciones y el interno, a lo sumo, N ; luego el costo está en $\Theta(N^2)$ ”.

Explique por qué la conclusión es correcta pero el argumento es incompleto, según la Nota 1.5.1.

9. Demuestre que la suma de las guardas en la Metodología 1.5.1 puede ignorarse cuando $g_k \in \Theta(1)$ y el cuerpo del ciclo tiene costo al menos 1 por iteración. ¿Qué ocurre si el cuerpo del ciclo puede tener costo 0?
10. Reescriba el ciclo del Ejemplo 1.5.1 de manera que la guarda no recalcula la suma, y verifique que el costo resultante está en $\Theta(K)$. ¿Cambia la respuesta que entrega el algoritmo?
11. Un algoritmo recorre el arreglo $A[0..N)$ y, para cada posición k , ejecuta un ciclo de $\lfloor \log(k+1) \rfloor + 1$ iteraciones de costo constante. Determine su costo exacto y su orden, usando el Teorema 1.4.5.
12. Determine el espacio auxiliar de una variante del algoritmo Incremental que guarde, para cada posición j , la mejor suma que termina en j . Compárelo con el del algoritmo original.
13. Proponga un algoritmo iterativo cuyo costo en el peor caso esté en $\Theta(N^2)$ y cuyo costo en el mejor caso esté en $\Theta(N)$, distinto del ordenamiento por inserción. Justifique ambas cotas.

1.6. Recurrencias y árboles de recurrencia

La Sección 1.5 traduce un algoritmo iterativo en una suma. Un algoritmo recursivo no admite esa traducción directa, porque su costo se define en términos de sí mismo: el costo de resolver una instancia de tamaño n depende del costo de resolver las instancias más pequeñas que genera. El resultado es una ecuación recurrente, o *recurrencia*, y el trabajo de esta sección consiste en convertirla en una función explícita, o al menos en su orden.

La sección presenta tres herramientas complementarias: la expansión, que produce una conjetura; la inducción, que la confirma; y los árboles de recurrencia, que organizan la expansión de manera visual y permiten tratar recurrencias que la expansión directa no resuelve cómodamente. La Sección 1.7 presenta después un teorema que resuelve de una vez toda una familia de recurrencias.

1.6.1. De un algoritmo recursivo a una recurrencia. El primer paso es análogo al de la Metodología 1.5.1: descomponer el algoritmo en sus partes y traducir cada una en una expresión de costo. La diferencia está en que ahora los llamados recurrentes se traducen en la misma función que se está definiendo.

Metodología 1.6.1: Obtención de una recurrencia

Dado un algoritmo recursivo A :

1. Identifique el parámetro que mide el tamaño de la entrada y defina $T(n)$ como el costo de A sobre una instancia de tamaño n , en el caso que interese (Nota 1.1.5).
2. Determine el costo del trabajo propio de un llamado, i.e., el de todas las operaciones excepto los llamados recurrentes, aplicando la Metodología 1.5.1.
3. Por cada llamado recurrente, agregue un término $T(m)$, en donde m es el tamaño del argumento de ese llamado.
4. Escriba aparte los casos base, i.e., los tamaños para los cuales el algoritmo no hace llamados recurrentes.
5. Verifique que todo llamado recurrente reduzca el tamaño y que esa reducción alcance un caso base, pues de lo contrario ni el algoritmo termina ni la recurrencia define una función.

Los términos que aporta el paso segundo se escriben casi siempre con notación asintótica, como en $T(n) = 2T(n/2) + \Theta(n)$. Esta escritura es la de la Nota 1.3.2: el $\Theta(n)$ denota alguna función de ese orden, no una función específica. El siguiente ejemplo aplica la metodología a dos algoritmos de forma muy distinta.

Ejemplo 1.6.1

Considere primero el algoritmo que calcula la suma de los elementos de $A[0..N)$ así: si el arreglo es vacío, la suma es 0; de lo contrario, es $A[0]$ más la suma de $A[1..N)$. Cada llamado hace una cantidad constante de trabajo propio y un llamado recurrente sobre un arreglo con un elemento menos, de modo que

$$T(0) = \Theta(1) \quad \text{y} \quad T(N) = T(N-1) + \Theta(1).$$

Considere ahora la variante que parte el arreglo por la mitad: si el arreglo es vacío o tiene un elemento, la suma es inmediata; de lo contrario, es la suma de la mitad izquierda más la de la mitad derecha. Hay dos llamados recurrentes, cada uno sobre un arreglo de tamaño $N/2$, y el trabajo propio sigue siendo constante:

$$T(1) = \Theta(1) \quad \text{y} \quad T(N) = 2T(N/2) + \Theta(1).$$

Los dos algoritmos hacen el mismo trabajo total, pero sus recurrencias no se parecen en nada. Resolverlas es el propósito del resto de la sección.

Nota 1.6.1

Escribir $T(N/2)$ supone que N es par. En general, un algoritmo que parte el arreglo por la mitad genera subproblemas de tamaños $\lfloor N/2 \rfloor$ y $\lceil N/2 \rceil$, y la recurrencia exacta los incluye. En este texto se omiten los pisos y techos, pues no alteran el orden de la solución; el Ejercicio 9 explora esa afirmación.

1.6.2. Expansión. El método más directo para resolver una recurrencia consiste en aplicarla a sí misma varias veces, reconocer el patrón que se forma y escribirlo como una suma de las de la Sección 1.4.

Ejemplo 1.6.2

Considere $T(n) = T(n-1) + n$ con $T(0) = 0$, que es la recurrencia de un algoritmo que procesa un elemento y hace un recorrido completo antes de llamarse a sí mismo. Al expandirla:

$$\begin{aligned} T(n) &= T(n-1) + n \\ &= T(n-2) + (n-1) + n \\ &= T(n-3) + (n-2) + (n-1) + n. \end{aligned}$$

Después de k pasos se tiene $T(n) = T(n-k) + (+j \mid n-k < j \leq n : j)$. Tomando $k = n$ se llega al caso base y

$$T(n) = T(0) + (+j \mid 1 \leq j \leq n : j) = \frac{n(n+1)}{2} \in \Theta(n^2),$$

por la Propiedad 1 del Teorema 1.4.5.

Cuando cada llamado genera más de un llamado recurrente, la expansión produce una suma geométrica en lugar de una aritmética, y el resultado cambia de familia.

Ejemplo 1.6.3

Considere $T(n) = 2T(n-1) + 1$ con $T(0) = 1$, que corresponde a un algoritmo que, con trabajo constante, se llama dos veces sobre instancias con un elemento menos. Al expandirla:

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 4T(n-2) + 2 + 1 \\ &= 8T(n-3) + 4 + 2 + 1. \end{aligned}$$

Después de k pasos, $T(n) = 2^k T(n - k) + (+j \mid 0 \leq j < k : 2^j)$. Con $k = n$:

$$T(n) = 2^n T(0) + (+j \mid 0 \leq j < n : 2^j) = 2^n + (2^n - 1) \in \Theta(2^n),$$

por la Propiedad 3 del Teorema 1.4.5. Esta es la recurrencia característica de los algoritmos que exploran todas las combinaciones posibles de n decisiones binarias.

Nota 1.6.2

La expansión produce una *conjetura*, no una demostración. El paso “después de k pasos se tiene...” generaliza un patrón observado en los primeros tres o cuatro niveles y supone que se mantiene en todos los demás. Esa suposición es justamente lo que demuestra la inducción de la subsección siguiente.

1.6.3. Verificación por inducción. Confirmar una conjetura obtenida por expansión es un ejercicio de inducción sobre n : se supone que la fórmula vale para los tamaños menores y se verifica que la recurrencia la reproduce para n .

Ejemplo 1.6.4

Se demuestra que la solución de $T(n) = 2T(n - 1) + 1$ con $T(0) = 1$ es $T(n) = 2^{n+1} - 1$, que es la fórmula cerrada obtenida en el Ejemplo 1.6.3.

Para $n = 0$ se tiene $T(0) = 1 = 2^1 - 1$. Suponga ahora que $T(n - 1) = 2^n - 1$ para algún $n \geq 1$. Entonces:

$$T(n) = 2T(n - 1) + 1 = 2(2^n - 1) + 1 = 2^{n+1} - 1,$$

que es lo que se quería demostrar.

Cuando solo interesa el orden y no la fórmula exacta, la inducción se hace sobre una desigualdad, i.e., se demuestra $T(n) \leq cf(n)$ para una constante c fija de antemano. Aquí aparece el error más común de toda la sección.

Nota 1.6.3

Al demostrar $T \in O(f)$ por inducción, la constante c debe fijarse antes de la demostración y ser la misma en todos los pasos. Arrastrar la notación O dentro de la inducción permite “demostrar” afirmaciones falsas. Por ejemplo, para $T(n) = 2T(n/2) + n$, un razonamiento que suponga $T(n/2) \in O(n/2)$ y

concluya

$$T(n) = 2O(n/2) + n = O(n) + n = O(n)$$

establecería que $T \in O(n)$, lo cual es falso: la solución está en $\Theta(n \log n)$. El defecto está en que la constante oculta crece en cada nivel y una cantidad infinita de crecimientos no es una constante. Con la constante explícita el error es visible: de $T(n/2) \leq c \frac{n}{2}$ se obtiene $T(n) \leq cn + n$, que no es $\leq cn$ para ningún c .

En ocasiones la inducción directa no cierra, aun cuando la conjetura es correcta, porque la hipótesis es demasiado débil para absorber los términos de menor orden. El remedio es fortalecerla.

Ejemplo 1.6.5

Considere $T(n) = 2T(n/2) + 1$ con $T(1) = 1$, cuya solución conjeturada es $T \in O(n)$. Al suponer $T(n/2) \leq c \frac{n}{2}$ se obtiene $T(n) \leq cn + 1$, que no es $\leq cn$. La inducción no cierra por culpa del $+1$.

Al fortalecer la hipótesis a $T(n) \leq cn - 1$, en cambio:

$$T(n) \leq 2\left(c \frac{n}{2} - 1\right) + 1 = cn - 1,$$

y la inducción cierra. El caso base exige $T(1) = 1 \leq c - 1$, que se satisface con $c = 2$. Restar un término de menor orden a la hipótesis la hace más fuerte, y por eso funciona.

1.6.4. Árboles de recurrencia. La expansión se vuelve difícil de seguir cuando los llamados recurrentes tienen tamaños distintos entre sí. El árbol de recurrencia organiza la misma información de manera visual y hace evidente cuál parte de la recursión domina el costo.

Metodología 1.6.2: Árbol de recurrencia

Dada una recurrencia, se construye un árbol en el que cada nodo representa un llamado y se etiqueta con el trabajo propio de ese llamado, sin contar sus llamados recurrentes. Los hijos de un nodo son los llamados que este genera. El costo total es la suma de todas las etiquetas, que se organiza por niveles determinando:

1. la cantidad de nodos en el nivel i ;
2. el tamaño del subproblema en el nivel i ;
3. el costo de cada nodo del nivel i , que es el trabajo propio para ese tamaño;

4. la cantidad de niveles, i.e., la altura del árbol, que se obtiene determinando cuándo el tamaño alcanza un caso base.

El costo total es la suma, sobre todos los niveles, del producto de la cantidad de nodos por el costo de cada nodo.

La Figura 6 muestra el árbol de $T(n) = 2T(n/2) + n$, la recurrencia que caracteriza a los algoritmos que parten la entrada en dos mitades y combinan los resultados con un recorrido.

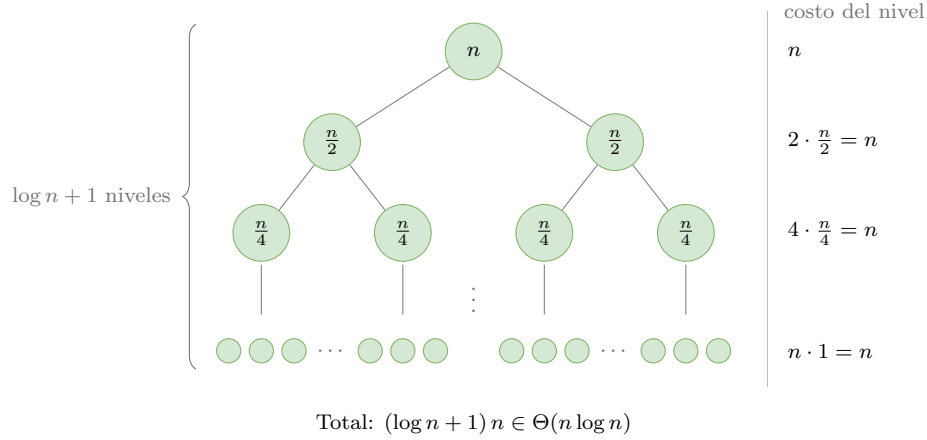


Figura 6. Árbol de recurrencia de $T(n) = 2T(n/2) + n$. Cada nodo está etiquetado con el tamaño de su subproblema, que en esta recurrencia coincide con su trabajo propio.

En el nivel i hay 2^i nodos, cada uno de tamaño $n/2^i$ y con trabajo propio $n/2^i$, de modo que el costo del nivel es $2^i \cdot \frac{n}{2^i} = n$ en todos los niveles. El tamaño llega a 1 cuando $2^i = n$, i.e., en el nivel $\log n$, así que hay $\log n + 1$ niveles y el costo total está en $\Theta(n \log n)$.

Cuando los subproblemas tienen tamaños distintos, el árbol deja de ser uniforme, pero el procedimiento no cambia.

La Figura 7 muestra el árbol de $T(n) = T(n/3) + T(2n/3) + n$. Mientras todos los nodos de un nivel sigan existiendo, el costo de ese nivel es n , pues los tamaños de los subproblemas de un nivel suman n . La rama que divide siempre por 3 llega al caso base en el nivel $\log_3 n$ y la que divide siempre por $\frac{3}{2}$, en el nivel $\log_{3/2} n$. Entonces el costo total es a lo sumo $n \log_{3/2} n$ y al menos $n \log_3 n$, de donde $T \in \Theta(n \log n)$, pues las dos cotas difieren en un factor constante (Nota 1.2.2).

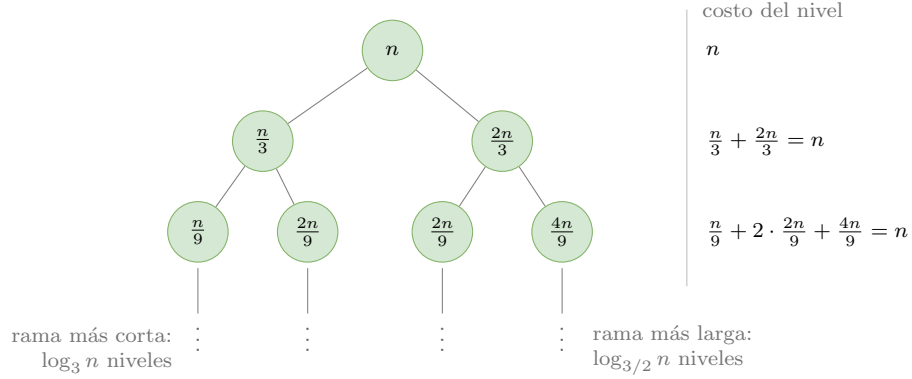


Figura 7. Árbol de recurrencia de $T(n) = T(n/3) + T(2n/3) + n$. Las ramas alcanzan el caso base a profundidades distintas.

Los dos ejemplos anteriores tienen costo constante por nivel, pero esa es solo una de tres posibilidades. La siguiente nota las enumera y es, en la práctica, la forma rápida de leer un árbol.

Nota 1.6.4

Sea s_i el costo del nivel i de un árbol de recurrencia de altura h . Por la Propiedad 3 del Teorema 1.4.5 y la Nota 1.4.3:

1. si los s_i decrecen geométricamente, el costo total está en $\Theta(s_0)$, i.e., domina la raíz;
2. si los s_i son todos del mismo orden, el costo total está en $\Theta(h \cdot s_0)$, i.e., el costo de un nivel por la altura;
3. si los s_i crecen geométricamente, el costo total está en $\Theta(s_h)$, i.e., dominan las hojas.

Nota 1.6.5

Un árbol de recurrencia produce una demostración cuando la suma por niveles se hace de manera exacta, lo cual suele requerir que n sea una potencia de la razón con la que se divide. En los demás casos, el árbol produce una conjetura que se confirma por inducción, como en la Nota 1.6.2.

1.6.5. Restar y dividir. Las recurrencias de esta sección se dividen en dos familias, según cómo se reduzca el tamaño en los llamados recurrentes, y la diferencia entre ellas es la que explica por qué unos algoritmos recursivos escalan y otros no.

Cuando el tamaño se reduce restando una constante, como en $T(n) = aT(n-1) + f(n)$, el árbol tiene profundidad n . Con $a = 1$ el resultado es la suma de f sobre todos los niveles, que para f polinomial da un polinomio de un grado más; con $a \geq 2$ los niveles crecen geométricamente y dominan las hojas, lo que produce un costo exponencial.

Cuando el tamaño se reduce dividiendo por una constante, como en $T(n) = aT(n/b) + f(n)$, el árbol tiene profundidad $\log_b n$ y hay $a^{\log_b n} = n^{\log_b a}$ hojas. Esta es la familia que resuelve el Teorema Maestro de la Sección 1.7. La siguiente tabla resume los casos más frecuentes.

Ejemplo 1.6.6

Recurrencias frecuentes y su orden, suponiendo casos base constantes:

Recurrencia	Orden	Comentario
$T(n) = T(n-1) + \Theta(1)$	$\Theta(n)$	recorrido recursivo
$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$	suma aritmética
$T(n) = 2T(n-1) + \Theta(1)$	$\Theta(2^n)$	dominan las hojas
$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$	domina la altura
$T(n) = T(n/2) + \Theta(n)$	$\Theta(n)$	domina la raíz
$T(n) = 2T(n/2) + \Theta(1)$	$\Theta(n)$	dominan las hojas
$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	costo constante por nivel

Queda por considerar el espacio. A las celdas que ocupa cada llamado hay que agregar las de los llamados que están suspendidos esperando su resultado, pues todos ellos coexisten en memoria. La cantidad de llamados suspendidos es, en el peor caso, la altura del árbol de recurrencia.

Ejemplo 1.6.7

Los dos algoritmos del Ejemplo 1.6.1 usan una cantidad constante de celdas por llamado, pero difieren en el espacio total. El primero, que resta uno al tamaño, tiene un árbol de profundidad N y por tanto usa $\Theta(N)$ celdas. El segundo, que divide por dos, tiene profundidad $\log N$ y usa $\Theta(\log N)$ celdas, aunque haga la misma cantidad de llamados.

Nótese el contraste con la versión iterativa del recorrido, que usa $\Theta(1)$ celdas (Ejemplo 1.5.7): la recursión cobra en espacio lo que ahorra en claridad.

Ejercicios

1. Resuelva por expansión y verifique por inducción, suponiendo casos base constantes:
 - a) $T(n) = T(n-1) + \log n$.
 - b) $T(n) = T(n-1) + n^2$.
 - c) $T(n) = 3T(n-1) + 1$.
2. Dibuje el árbol de recurrencia de $T(n) = 3T(n/2) + n$, determine las cuatro cantidades de la Metodología 1.6.2 y clasifique el régimen según la Nota 1.6.4.
3. Dibuje el árbol de recurrencia de $T(n) = 2T(n/2) + n^2$ y determine su orden. ¿Cuál de los tres regímenes se presenta?
4. Resuelva $T(n) = 2T(\sqrt{n}) + \log n$. 🌈
5. Obtenga la recurrencia del algoritmo que calcula x^n multiplicando x por x^{n-1} , y la del que eleva al cuadrado $x^{n/2}$. Compare los órdenes resultantes.
6. Obtenga la recurrencia del algoritmo que calcula el n -ésimo número de Fibonacci sumando los dos anteriores, calculados recursivamente. Acótela por debajo y por arriba con recurrencias de la forma $T(n) = 2T(n-c) + \Theta(1)$ y concluya que su costo es exponencial.
7. Considere la siguiente “demostración” de que la solución de $T(n) = 4T(n/2) + n$ está en $O(n^2)$: “suponiendo $T(n/2) \leq c(n/2)^2$, se tiene $T(n) \leq 4c \frac{n^2}{4} + n = cn^2 + n$, que está en $O(n^2)$ ”. Explique por qué el argumento es incorrecto y corríjalo fortaleciendo la hipótesis, como en el Ejemplo 1.6.5.
8. Demuestre que la solución de $T(n) = T(n/2) + T(n/3) + n$ está en $\Theta(n)$. Para la cota superior use la Nota 1.6.4.
9. Considere la recurrencia $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n$ con $T(1) = 1$. Demuestre por inducción que $T(n) \leq n \lceil \log n \rceil + n$ y compare con el resultado que se obtiene al ignorar los pisos y techos.
10. Determine el espacio auxiliar del algoritmo del Ejercicio 9, contando los llamados suspendidos.
11. Un algoritmo recursivo hace un llamado sobre un subproblema de tamaño $n-1$ y, además, un recorrido de $\Theta(n)$ para preparar ese llamado. Otro hace dos llamados sobre subproblemas de tamaño $n/2$ y un recorrido de $\Theta(n)$. Determine y compare sus costos en tiempo y en espacio.
12. Use árboles de recurrencia para resolver cada una de las siguientes recurrencias:
 - a) $T(n) = 2T(n/4) + \sqrt{n}$.
 - b) $T(n) = 2T(n/4) + n$.
 - c) $T(n) = 2T(n/4) + n^2$.
13. Use árboles de recurrencia para resolver cada una de las siguientes recurrencias:

- a) $T(n) = T(n/2) + T(n/3) + T(n/6) + n$.
 b) $T(n) = T(n/2) + 2T(n/3) + 3T(n/4) + n^2$.
 c) $T(n) = 2T(n/2) + O(n \log n)$.
 d) $T(n) = 2T(n/2) + O(n/\log n)$.
 e) $T(n) = \sqrt{n}T(\sqrt{n}) + n$.
14. Use árboles de recurrencia para resolver cada una de las siguientes recurrencias:
 a) $T(n) = 2T(n-1) + n \log n$.
 b) $T(n) = 2T(n-1) + n^2$.
15. Proponga una recurrencia cuyo árbol tenga costo por nivel decreciente geométricamente y otra cuyo costo por nivel crezca geométricamente. Determine el orden de sus soluciones y verifíquelo con la Nota 1.6.4.

1.7. El Teorema Maestro

La Sección 1.6 mostró que las recurrencias que dividen el tamaño por una constante producen árboles de profundidad logarítmica, cuyo costo depende de cómo se reparte el trabajo entre los niveles. El Teorema Maestro convierte esa observación en una receta: para toda una familia de recurrencias, basta comparar dos cantidades para conocer el orden de la solución, sin dibujar el árbol ni hacer inducción.

El teorema es de gran utilidad para analizar algoritmos que resultan de aplicar la técnica de dividir y conquistar, que es el tema del Capítulo 2. Esta sección caracteriza las recurrencias a las que se aplica, enuncia el teorema en dos versiones, lo demuestra con un árbol de recurrencia y lo ilustra con ejemplos, entre ellos un cuarto algoritmo para el problema conductor del capítulo.

1.7.1. Funciones simples. El Teorema Maestro no se aplica a cualquier recurrencia. La familia que cubre es la de los algoritmos que, en cada llamado, generan una cantidad fija de subproblemas, todos del mismo tamaño y más pequeños que el original por un factor fijo, y que combinan sus resultados con un trabajo propio acotado por un polinomio. La siguiente definición precisa esa familia.

Definición 1.7.1: Función simple

Se dice que la función $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es *simple* si satisface las siguientes condiciones para $n \in \mathbb{N}$:

- Caso base: para n suficientemente pequeño,

$$T(n) \in O(1).$$

- Caso recurrente: para los demás valores de n y constantes $a, b, k \in \mathbb{R}_{\geq 0}$,

$$T(n) \leq a \cdot T(n/b) + O(n^k)$$

tales que:

- $a \geq 1$ representa la cantidad de llamados recurrentes hechos por el algoritmo asociado a T y
- $b > 1$ representa el factor por el cual se reduce el tamaño de la entrada en cada llamado recurrente de T .

El caso base en una función simple indica que una vez el tamaño de la entrada es suficientemente pequeño como para no hacer llamados recurrentes, entonces el problema se puede resolver en tiempo o espacio constante (i.e., $O(1)$). El caso recurrente asume que se recurre sobre a instancias similares a la instancia dada, pero de tamaño reducido en un factor b , y que estos llamados requieren $O(n^k)$ tiempo o espacio adicional para ser consolidados como respuesta de la instancia inicial. El término $O(n^k)$ se interpreta según la Nota 1.3.2, y el cociente n/b , que puede no ser un número natural, según la Nota 1.6.1.

El siguiente ejemplo muestra cómo se reconoce una función simple a partir de la descripción de un algoritmo.

Ejemplo 1.7.1

Sea $T(n)$ la función que determina la cantidad de operaciones realizadas por *MergeSort* para ordenar un arreglo de tamaño n . Como los casos base, que corresponden a ordenar un arreglo vacío o de un solo elemento, toman tiempo constante, se tiene:

$$T(n) \in O(1), \text{ para } n \leq 1.$$

En el algoritmo hay dos llamados recurrentes (i.e., $a = 2$), cada uno sobre una mitad del arreglo dado (i.e., $b = 2$), y se requiere un cálculo lineal en el tamaño de la entrada (i.e., $k = 1$) para consolidar los resultados obtenidos de los llamados recurrentes. Es decir, para el caso recurrente, se tiene:

$$T(n) \leq 2 \cdot T(n/2) + O(n), \text{ para } n > 1.$$

Luego, T es una función simple.

Cuando una función ha sido declarada como simple, no es necesario fijarse en el valor exacto de sus casos base: como se verá en la demostración, estos solo afectan el costo de las hojas del árbol de recurrencia por un factor constante. La restricción de que todos los llamados recurrentes se hagan sobre subproblemas del mismo tamaño sí es esencial. Las recurrencias con subproblemas de tamaños distintos, como la de la Figura 7, quedan por fuera del teorema y se analizan con árboles de recurrencia.

1.7.2. El enunciado. Con la noción de función simple se puede precisar el enunciado del Teorema Maestro. Sus tres casos provienen de comparar dos cantidades: el número a de subproblemas que genera cada llamado y el factor b^k por el cual se reduce el trabajo propio de un llamado al pasar de un nivel al siguiente.

Teorema 1.7.1: Teorema Maestro

Si $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es una función simple con parámetros $a \geq 1$, $b > 1$ y $k \geq 0$, entonces para $n \in \mathbb{N}$:

$$T(n) \in \begin{cases} O(n^k \log n) & , \text{ si } a = b^k, \\ O(n^k) & , \text{ si } a < b^k, \\ O(n^{\log_b a}) & , \text{ si } a > b^k. \end{cases}$$

Intuitivamente, a representa la tasa con la que se multiplican los subproblemas de un nivel al siguiente, mientras que b^k representa la tasa con la que se reduce el trabajo propio de cada uno. Cuando $a = b^k$, las dos tasas se compensan y el algoritmo hace la misma cantidad de trabajo en cada nivel de la recurrencia: $O(n^k)$ operaciones en cada uno de los $O(\log n)$ niveles, es decir, $O(n^k \log n)$ operaciones en total. Cuando $a < b^k$, la cantidad de trabajo decrece a medida que se profundiza la recurrencia y el trabajo $O(n^k)$ de la raíz domina el total. Cuando $a > b^k$, hay demasiada proliferación de subproblemas: el trabajo por nivel crece con la profundidad y el total lo domina el último nivel, que tiene $a^{\log_b n} = n^{\log_b a}$ hojas (Nota 1.4.1).

El Teorema 1.7.1 da cotas superiores, porque la Definición 1.7.1 solo acota el caso recurrente por encima. Cuando la recurrencia es una igualdad y el trabajo propio es de orden exacto n^k , las mismas tres conclusiones valen con Θ . Esta versión es la que permite afirmar órdenes exactos, como se hace en el Capítulo 2.

Teorema 1.7.2: Teorema Maestro, versión exacta

Sea $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tal que $T(n) \in \Theta(1)$ para n suficientemente pequeño y $T(n) = a \cdot T(n/b) + \Theta(n^k)$ para los demás valores de n , con $a \geq 1$, $b > 1$ y $k \geq 0$. Entonces:

$$T(n) \in \begin{cases} \Theta(n^k \log n) & , \text{ si } a = b^k, \\ \Theta(n^k) & , \text{ si } a < b^k, \\ \Theta(n^{\log_b a}) & , \text{ si } a > b^k. \end{cases}$$

1.7.3. Demostración. La demostración formaliza la intuición anterior con un árbol de recurrencia. Para evitar pisos y techos, se hace para los valores de n que

son potencias de b ; la nota al final de la subsección indica cómo se extiende a los demás valores.

Por la Definición 1.7.1, hay constantes $c, d \in \mathbb{R}_{>0}$ tales que el trabajo propio de un llamado de tamaño n es a lo sumo cn^k y el costo de un caso base es a lo sumo d . Sin pérdida de generalidad, se supone que el caso base es $n = 1$ y que la desigualdad del caso recurrente vale para todo $n > 1$.

Teorema 1.7.3: Expansión de una función simple

Para todo $j \in \mathbb{N}$, si $n = b^j$ y $r = \frac{a}{b^k}$, entonces:

$$T(n) \leq cn^k (+i \mid 0 \leq i < j : r^i) + da^j.$$

Demostración

Para abreviar, sea $S_m = (+i \mid 0 \leq i < m : r^i)$. Se procede por inducción sobre j . Para $j = 0$ se tiene $n = 1$ y $T(1) \leq d = cS_0 + da^0$, pues $S_0 = 0$. Para $j \geq 1$, suponga el resultado para $j - 1$, es decir, para $n/b = b^{j-1}$:

$$\begin{aligned} T(n) &\leq aT(n/b) + cn^k && \text{(caso recurrente)} \\ &\leq ac\left(\frac{n}{b}\right)^k S_{j-1} + ada^{j-1} + cn^k && \text{(hipótesis inductiva)} \\ &= cn^k r S_{j-1} + da^j + cn^k && \text{(pues } a/b^k = r) \\ &= cn^k S_j + da^j && \text{(pues } r S_{j-1} + 1 = S_j). \end{aligned}$$

El Teorema 1.7.3 es la suma por niveles del árbol de la Figura 8: el nivel i tiene a^i nodos de tamaño n/b^i , cuyo trabajo propio suma a lo sumo $a^i c (n/b^i)^k = cn^k r^i$, y el último nivel tiene a^j hojas de costo a lo sumo d . Con $j = \log_b n$ se tiene $a^j = n^{\log_b a}$.

Con este resultado, la demostración del Teorema Maestro (Teorema 1.7.1) se reduce a analizar la suma geométrica del Teorema 1.7.3 según el valor de r , que es exactamente la clasificación de la Nota 1.6.4.

Demostración

Sea $n = b^j$, con $j = \log_b n$, y sea $r = \frac{a}{b^k}$. Se distinguen tres casos, según el Teorema 1.7.3 y la Propiedad 3 del Teorema 1.4.5.

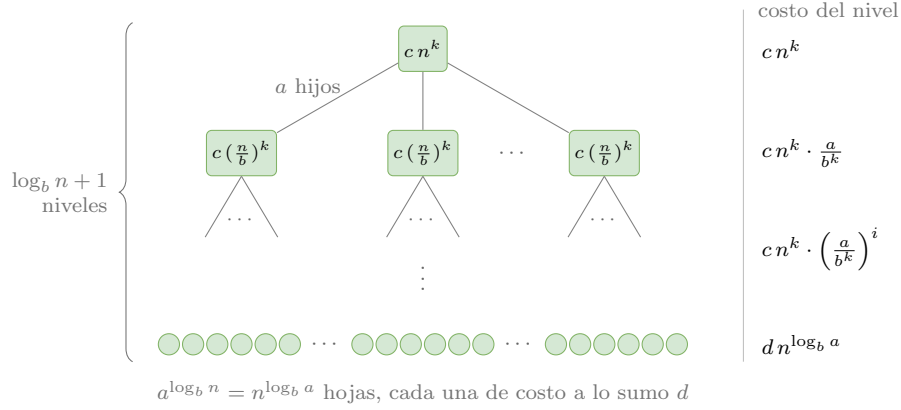


Figura 8. Árbol de recurrencia de una función simple. El costo del nivel i es una progresión geométrica de razón $r = \frac{a}{b^k}$.

- Si $a = b^k$, entonces $r = 1$ y $\log_b a = k$. La suma vale j , de modo que $T(n) \leq cn^k \log_b n + dn^k$, que está en $O(n^k \log n)$. Todos los niveles cuestan lo mismo.
- Si $a < b^k$, entonces $r < 1$ y la suma es a lo sumo $\frac{1}{1-r}$. Además $\log_b a < k$, de modo que $n^{\log_b a} \leq n^k$. Luego $T(n) \leq \left(\frac{c}{1-r} + d\right) n^k$, que está en $O(n^k)$. Domina la raíz.
- Si $a > b^k$, entonces $r > 1$ y la suma es a lo sumo $\frac{r^j}{r-1}$. Como $n^k r^j = b^{jk} \frac{a^j}{b^{jk}} = a^j = n^{\log_b a}$, se tiene $T(n) \leq \left(\frac{c}{r-1} + d\right) n^{\log_b a}$, que está en $O(n^{\log_b a})$. Dominan las hojas.

La versión exacta del Teorema 1.7.2 se demuestra de la misma manera, acotando además el trabajo propio por debajo con una constante $c' > 0$; en cada caso, el término dominante aparece también en la cota inferior. Los detalles se proponen como ejercicio al lector.

Nota 1.7.1

La demostración anterior cubre solo los valores de n que son potencias de b . Para extenderla a todo n basta con que T sea no decreciente, lo cual se cumple para las funciones de costo de los algoritmos de este texto: si $b^j \leq n < b^{j+1}$, entonces $T(b^j) \leq T(n) \leq T(b^{j+1})$, y las cotas obtenidas para b^j y b^{j+1} difieren en un factor constante. Una demostración que trata directamente los pisos y techos se encuentra en [CLRS22].

1.7.4. Ejemplos. Con el teorema demostrado, su uso se reduce a identificar los parámetros a , b y k , comparar a con b^k y leer la conclusión. Los ejemplos siguientes cubren los tres casos.

Ejemplo 1.7.2: Complejidad temporal de MergeSort

La cantidad de operaciones básicas que lleva a cabo el algoritmo MergeSort está definida por la función simple T cuyo caso recurrente corresponde a:

$$T(n) = 2 \cdot T(n/2) + O(n).$$

Luego, $a = 2$, $b = 2$ y $k = 1$. Note que:

$$a = 2 = 2^1 = b^k,$$

lo cual corresponde al primer caso del Teorema Maestro. En consecuencia, $T(n) \in O(n \log n)$. Como el trabajo de combinar las dos mitades es de orden exacto n , la versión exacta del teorema permite concluir además $T(n) \in \Theta(n \log n)$.

La búsqueda binaria, que también cae en el primer caso, muestra que ese caso no siempre produce el factor n^k visible: cuando $k = 0$, el resultado es solo un logaritmo.

Ejemplo 1.7.3: Complejidad temporal de búsqueda binaria

La cantidad de operaciones básicas que lleva a cabo el algoritmo de búsqueda binaria está definida por la función simple T cuyo caso recurrente corresponde a:

$$T(n) = T(n/2) + O(1).$$

Luego, $a = 1$, $b = 2$ y $k = 0$. Note que:

$$a = 1 = 2^0 = b^k,$$

lo cual corresponde al primer caso del Teorema Maestro. En consecuencia, $T(n) \in O(n^0 \log n) = O(\log n)$.

El siguiente ejemplo, ficticio, ilustra el segundo caso, en el cual la raíz del árbol concentra la mayor parte del trabajo.

Ejemplo 1.7.4

Considere una función simple cuyo caso recurrente está dado por:

$$T(n) = 2 \cdot T(n/3) + O(n).$$

Luego, $a = 2$, $b = 3$ y $k = 1$. Note que:

$$a = 2 < 3 = 3^1 = b^k,$$

lo cual corresponde al segundo caso del Teorema Maestro. En consecuencia, $T(n) \in O(n)$.

El tercer caso aparece cuando cada llamado genera muchos subproblemas en relación con cuánto se reduce su tamaño. El ejemplo clásico es la multiplicación de matrices.

Ejemplo 1.7.5: Complejidad temporal de Strassen

El algoritmo de Strassen permite multiplicar dos matrices cuadradas de tamaño n . De una forma ingeniosa, este algoritmo hace 7 llamados recurrentes sobre subproblemas reduciendo en mitades los tamaños de los subproblemas. Las respuestas obtenidas de los llamados recurrentes se consolidan con $O(n^2)$ operaciones:

$$T(n) = 7 \cdot T(n/2) + O(n^2).$$

Luego, $a = 7$, $b = 2$ y $k = 2$. Note que:

$$a = 7 > 4 = 2^2 = b^k,$$

lo cual corresponde al tercer caso del Teorema Maestro. En consecuencia, $T(n) \in O(n^{\log_2 7})$. Como $\log_2 7 \approx 2,807$, se tiene en particular $T(n) \in O(n^{2,81})$, que es asintóticamente mejor que el $\Theta(n^3)$ del algoritmo usual (Ejercicio 3).

El último ejemplo regresa al problema conductor del capítulo. Los algoritmos Exhaustivo, Acumulado e Incremental tienen costos en $\Theta(N^3)$, $\Theta(N^2)$ y $\Theta(N)$; dividir y conquistar produce un cuarto algoritmo, cuyo costo queda entre los dos últimos.

Ejemplo 1.7.6: Suma máxima por dividir y conquistar

El algoritmo Mitades resuelve el problema de la suma máxima de un subarreglo (Ejemplo 1.1.1) así. Si el arreglo tiene a lo sumo un elemento, la respuesta es inmediata. De lo contrario, sea $m = \lfloor N/2 \rfloor$. Todo subarreglo de $A[0..N)$ está contenido en $A[0..m)$, está contenido en $A[m..N)$ o cruza la posición m . Los dos primeros casos se resuelven recursivamente. Para el tercero, la mejor suma de un subarreglo que cruza m es la mejor suma de un sufijo de $A[0..m)$

más la mejor suma de un prefijo de $A[m..N]$, y cada una se calcula con un recorrido. La respuesta es la mayor de las tres.

Hay dos llamados recurrentes sobre mitades del arreglo y el trabajo propio de cada llamado, dominado por los dos recorridos, es de orden exacto N :

$$T(N) = 2 \cdot T(N/2) + \Theta(N).$$

Luego, $a = 2 = 2^1 = b^k$ y, por la versión exacta del Teorema Maestro, $T(N) \in \Theta(N \log N)$.

Con **Mitades**, el problema conductor queda resuelto por cuatro algoritmos de órdenes N^3 , N^2 , $N \log N$ y N , respectivamente. La mejora de cada uno respecto al anterior proviene de una idea distinta: reutilizar sumas parciales, dividir el problema y, en el caso de **Incremental**, mantener la información justa para no tener que volver atrás. Esta última idea es la de la programación dinámica y la corrección de **Incremental** se justifica con esa técnica en la Sección 3.4.

1.7.5. Alcance del teorema. El Teorema Maestro resuelve de manera inmediata una familia amplia de recurrencias, pero no todas las que aparecen en el análisis de algoritmos. Conviene reconocer los casos que quedan por fuera, pues aplicar el teorema en ellos produce conclusiones sin fundamento.

Nota 1.7.2

El Teorema Maestro no se aplica, al menos en la forma del Teorema 1.7.1, en los siguientes casos:

1. Cuando los subproblemas tienen tamaños distintos, como en $T(n) = T(n/3) + T(2n/3) + n$. Estas recurrencias se analizan con árboles de recurrencia (Figura 7).
2. Cuando el trabajo propio no está acotado por un polinomio n^k del orden exacto que se necesita, como en $T(n) = 2T(n/2) + n \log n$. Aquí $n \log n \in O(n^{1+\varepsilon})$ para todo $\varepsilon > 0$, pero usar esa cota da una conclusión que no es ajustada; el árbol de recurrencia da el orden exacto.
3. Cuando el tamaño se reduce restando en lugar de dividiendo, como en $T(n) = 2T(n-1) + 1$. Estas recurrencias tienen árboles de profundidad n y se resuelven como en la Sección 1.6.

Ejercicios

1. Use el Teorema Maestro para calcular $T(n)$ en cada uno de los siguientes casos:
 - a) $T(n) = T(n/2) + 1$
 - b) $T(n) = 3T(n/3) + 8n$
 - c) $T(n) = 2T(4n/5) + n^3$
2. Use el Teorema Maestro para calcular $T(n)$ en cada uno de los siguientes casos:
 - a) $T(n) = 2T(n/3) + n^3$
 - b) $T(n) = 3T(n/3) + 5n$
 - c) $T(n) = 6T(4n/5) + 4n^2$
3. Justifique por qué, para analizar con ayuda del Teorema Maestro el orden de la cantidad de recursos que emplea un algoritmo, no es necesario contar con los valores exactos de los casos base de la función simple asociada.
4. Sean $a, b \in \mathbb{R}$ tales que $a \geq 1$ y $b > 1$. Demuestre, para $n \in \mathbb{N}$, que $n^{\log_b a} = a^{\log_b n}$.
5. Investigue acerca del Algoritmo de Strassen para la multiplicación de matrices. Explique brevemente en qué consiste, cómo reduce la cantidad de llamados recurrentes de 8 a 7 e ilustre su funcionamiento con un ejemplo.
6. Demuestre la versión exacta del Teorema Maestro (Teorema 1.7.2) para n potencia de b , adaptando el Teorema 1.7.3 para obtener también una cota inferior.
7. Use la versión exacta del Teorema Maestro para determinar el orden de:
 - a) $T(n) = 3T(n/2) + \Theta(n)$.
 - b) $T(n) = 4T(n/2) + \Theta(n^2)$.
 - c) $T(n) = 8T(n/2) + \Theta(n^2)$.
8. Para cada una de las siguientes recurrencias, decida si el Teorema Maestro se aplica. Si no, explique por qué y resuélvala con las herramientas de la Sección 1.6:
 - a) $T(n) = 2T(n/2) + n \log n$.
 - b) $T(n) = T(n/4) + T(3n/4) + n$.
 - c) $T(n) = 4T(n/2) + n^2 \log n$.
 - d) $T(n) = T(n-2) + n$.
9. Justifique que la mejor suma de un subarreglo que cruza la posición m en el Ejemplo 1.7.6 es la suma de la mejor suma de un sufijo de $A[0..m)$ y la mejor suma de un prefijo de $A[m..N)$. Tenga en cuenta que ambos pueden ser vacíos.
10. Determine el espacio auxiliar del algoritmo **Mitades** del Ejemplo 1.7.6, contando los llamados suspendidos como en la Sección 1.6. Compárelo con el de **Incremental**.

11. Implemente el algoritmo **Mitades** y agregue sus tiempos a las mediciones de la Sección 1.1.2. ¿Dónde queda su curva respecto a las de los algoritmos **Acumulado** e **Incremental**?

1.8. El modelo de costos en Python

Hasta este punto, los algoritmos del capítulo se han descrito en prosa o en notación matemática, y su costo se ha calculado sobre la máquina abstracta de la Definición 1.1.1. Este texto también expresa sus algoritmos en Python y las mediciones de la Sección 1.1.2 se hicieron sobre implementaciones concretas. Esta sección tiende el puente entre las dos cosas: muestra cómo se escriben en Python algunos de los algoritmos del capítulo, identifica en qué coinciden las construcciones del lenguaje con el modelo abstracto y señala los casos en los que se apartan de él.

El propósito no es describir el funcionamiento interno del intérprete, sino fijar los costos que el resto del texto da por sentados cada vez que analiza un algoritmo escrito en Python.

1.8.1. Del modelo abstracto a Python. La mayoría de las construcciones elementales de Python cuestan lo mismo que en el modelo abstracto: una asignación, una comparación entre números pequeños o el acceso a una posición de una lista son operaciones de costo constante. El riesgo está en las construcciones que tienen la apariencia de una operación elemental pero que, por dentro, recorren una estructura completa. El siguiente ejemplo reúne el costo de las operaciones más frecuentes sobre las estructuras de datos predefinidas del lenguaje.

Ejemplo 1.8.1

La siguiente tabla indica el costo de las operaciones más usadas en este texto, según la documentación del intérprete de referencia de Python [Pyt26]. Aquí N denota la cantidad de elementos de la lista A .

Operación	Ejemplo	Costo
Acceso por índice	<code>A[i], A[i] = x</code>	$\Theta(1)$
Longitud	<code>len(A)</code>	$\Theta(1)$
Agregar o retirar al final	<code>A.append(x), A.pop()</code>	$\Theta(1)$ (amortizado)
Insertar o retirar en i	<code>A.insert(i, x), A.pop(i)</code>	$\Theta(N - i)$
Pertenencia	<code>x in A</code>	$\Theta(N)$
Rebanada	<code>A[i:j]</code>	$\Theta(j - i)$
Concatenación	<code>A + B</code>	$\Theta(N + M)$
Suma, mínimo y máximo	<code>sum(A), min(A)</code>	$\Theta(N)$
Ordenamiento	<code>sorted(A), A.sort()</code>	$\Theta(N \log N)$
Conjuntos y diccionarios	<code>x in S, D[k]</code>	$\Theta(1)$ (promedio)

En la concatenación, M es la cantidad de elementos de B . El costo de la pertenencia en una lista corresponde al peor caso, que se presenta cuando x no está en A . El costo “amortizado” de la tercera fila se explica en la Sección 1.9; para efectos del análisis, significa que una secuencia de K de esas operaciones cuesta $\Theta(K)$ en total.

La tabla del Ejemplo 1.8.1 debe leerse con la misma cautela que el resto del capítulo: las operaciones con costo $\Theta(1)$ son las que corresponden a operaciones elementales del modelo abstracto y las demás esconden un ciclo. Al analizar un programa en Python, cada una de estas últimas debe tratarse como el ciclo que es, aplicando la Metodología 1.5.1.

1.8.2. Los algoritmos del capítulo en Python. Los tres algoritmos del problema conductor se traducen a Python casi línea por línea a partir de su descripción en la Sección 1.1, y sus implementaciones son las que se usaron para las mediciones de la Figura 1. La primera es la del algoritmo Exhaustivo.

Algoritmo 1.8.1: Algoritmo Exhaustivo

```
1 def mss_exhaustive(A):
2     N, best = len(A), 0
3     for i in range(N + 1):
4         for j in range(i, N + 1):
5             s = 0
6             for k in range(i, j):
7                 s += A[k]
8             best = max(best, s)
9     return best
```

Los tres ciclos del Algoritmo 1.8.1 corresponden a las tres sumas del Ejemplo 1.4.3, y el acceso $A[k]$ de la línea 7 es la operación barómetro que allí se contó. Como la función $\max$ recibe dos números, su costo es constante. El algoritmo Acumulado elimina el ciclo más interno.

Algoritmo 1.8.2: Algoritmo Acumulado

```
1 def mss_accumulated(A):
2     N, best = len(A), 0
3     for i in range(N + 1):
4         s = 0
5         for j in range(i, N):
```

```
6     s += A[j]
7     best = max(best, s)
8     return best
```

El algoritmo Incremental recorre la lista una sola vez, manteniendo los valores r y b de su descripción. Nótese que el ciclo recorre directamente los elementos de la lista en lugar de sus índices; el costo es el mismo, pues cada iteración entrega un elemento con costo constante.

Algoritmo 1.8.3: Algoritmo Incremental

```
1 def mss_incremental(A):
2     r = b = 0
3     for x in A:
4         r = max(r + x, 0)
5         b = max(b, r)
6     return b
```

El algoritmo Mitades del Ejemplo 1.7.6 recibe, además de la lista, los extremos del subarreglo sobre el cual trabaja. Esa decisión no es cosmética: como se verá en la subsección siguiente, pasar rebanadas en lugar de índices cambiaría su costo.

Algoritmo 1.8.4: Algoritmo Mitades

```
1 def mss_halves(A, lo, hi):
2     ans = None
3     if hi - lo == 0: ans = 0
4     elif hi - lo == 1: ans = max(A[lo], 0)
5     else:
6         m = (lo + hi) // 2
7         left = mss_halves(A, lo, m)
8         right = mss_halves(A, m, hi)
9         s = suffix = 0
10        for k in range(m - 1, lo - 1, -1):
11            s += A[k]
12            suffix = max(suffix, s)
13        s = prefix = 0
14        for k in range(m, hi):
15            s += A[k]
16            prefix = max(prefix, s)
17        ans = max(left, right, suffix + prefix)
```

```
18 return ans
```

Los dos llamados recurrentes de las líneas 7 y 8 aportan el término $2T(N/2)$ de la recurrencia, y los dos ciclos de las líneas 10 a 16, que calculan la mejor suma de un sufijo de la mitad izquierda y la mejor suma de un prefijo de la mitad derecha, aportan el término $\Theta(N)$. La invocación inicial es `mss_halves(A, 0, len(A))`.

La brevedad de Python invita a escribir los algoritmos con sus funciones predefinidas. El siguiente ejemplo muestra que ese estilo puede esconder el costo real de un programa.

Ejemplo 1.8.2

El algoritmo Exhaustivo se puede escribir en una sola expresión:

```
1 max(sum(A[i:j]) for i in range(N + 1) for j in range(i, N + 1))
```

Esta versión tiene dos ciclos visibles en lugar de tres, pero su costo sigue estando en $\Theta(N^3)$: según la tabla del Ejemplo 1.8.1, la rebanada `A[i:j]` y la suma `sum` cuestan cada una $\Theta(j - i)$. El tercer ciclo no desaparece; solo deja de verse.

Por último, el ordenamiento por inserción de la Sección 1.1 se escribe así. El ciclo `while` es el ciclo interno que depende de los datos, cuyo análisis se hizo en el Ejemplo 1.5.5.

Algoritmo 1.8.5: Ordenamiento por inserción

```
1 def insertion_sort(A):
2     for k in range(1, len(A)):
3         j = k
4         while j > 0 and A[j - 1] > A[j]:
5             A[j - 1], A[j] = A[j], A[j - 1]
6             j -= 1
```

1.8.3. Donde Python se aparta del modelo. Las implementaciones anteriores se ajustan al modelo abstracto porque usan solo operaciones de costo constante. Hay, sin embargo, varias situaciones en las que el comportamiento de Python difiere del modelo y conviene conocerlas para no llegar a conclusiones equivocadas.

Enteros de precisión arbitraria. Los enteros de Python no tienen un tamaño máximo: el intérprete usa tantos bits como haga falta para representar cada número. Esto es cómodo, pero rompe la suposición de la Nota 1.1.3, según la cual el tamaño de los números es despreciable.

Ejemplo 1.8.3

El número 2^{1000} tiene 1001 bits y el número de Fibonacci F_{1000} tiene 694 bits. En general, F_n tiene $\Theta(n)$ bits, de modo que un algoritmo que calcula F_n con n sumas no hace n operaciones elementales: cada suma cuesta un tiempo proporcional a la cantidad de bits de sus operandos y el costo total está en $\Theta(n^2)$ operaciones sobre bits. Mientras los números quepan en una palabra del procesador, en cambio, la suposición de la Nota 1.1.3 es razonable.

Las rebanadas copian. Una rebanada $A[i:j]$ no es una vista sobre la lista original, sino una lista nueva con copia de los elementos, y por eso cuesta $\Theta(j-i)$. Un algoritmo recursivo que pasa rebanadas a sus llamados paga esa copia en cada uno.

Ejemplo 1.8.4

El primer algoritmo del Ejemplo 1.6.1 suma $A[0]$ con la suma de $A[1..N]$. En Python se puede escribir de dos maneras:

```

1 def rsum_index(A, i=0):
2     ans = None
3     if i == len(A): ans = 0
4     else: ans = A[i] + rsum_index(A, i + 1)
5     return ans
6
7 def rsum_slice(A):
8     ans = None
9     if len(A) == 0: ans = 0
10    else: ans = A[0] + rsum_slice(A[1:])
11    return ans

```

La primera versión satisface $T(N) = T(N-1) + \Theta(1)$ y su costo está en $\Theta(N)$. En la segunda, la rebanada $A[1:]$ copia $N-1$ elementos, de modo que $T(N) = T(N-1) + \Theta(N)$ y, por el Ejemplo 1.6.2, su costo está en $\Theta(N^2)$.

La Figura 9 confirma el análisis del Ejemplo 1.8.4 con mediciones hechas en el mismo entorno de la Sección 1.1.2. Al duplicar N , el tiempo de la versión con índices se multiplica aproximadamente por 2 y el de la versión con rebanadas, por 4 o más. Para $N = 2^{14}$, la segunda tarda más de mil veces lo que tarda la primera.

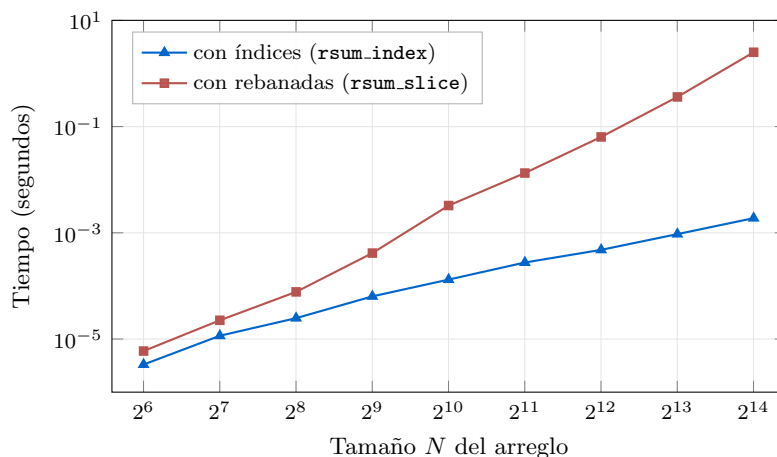


Figura 9. Tiempo de ejecución de las dos versiones de la suma recursiva del Ejemplo 1.8.4. Ambos ejes están en escala logarítmica.

Diccionarios y conjuntos. Las operaciones de consulta, inserción y pertenencia sobre diccionarios y conjuntos cuestan $\Theta(1)$ *en promedio*, no en el peor caso. Estas estructuras se implementan con tablas de dispersión (en inglés, *hash tables*), cuyo buen desempeño depende de que las llaves se repartan de manera uniforme; en el peor caso, una operación puede costar $\Theta(N)$. En la práctica este peor caso es muy improbable y este texto adopta la convención habitual de tratar estas operaciones como de costo constante, recordando que se trata del caso promedio de la Nota 1.1.6.

Concatenación de cadenas. Las cadenas de Python son inmutables: `s + t` construye una cadena nueva y copia los caracteres de ambas. Construir una cadena de longitud N agregando un carácter a la vez con `s += c` puede costar $\Theta(N^2)$, por la misma razón que `rsum_slice`. El intérprete de referencia optimiza algunos casos de este patrón, pero el lenguaje no lo garantiza; la forma idiomática y de costo lineal es acumular las partes en una lista y unir las al final con `''.join(partes)`.

Recursión. Python limita la profundidad de la recursión: por omisión, un programa que acumula más de unos mil llamados suspendidos termina con un error. Las mediciones de la Figura 9, con recursiones de profundidad N , requirieron elevar ese límite con `sys.setrecursionlimit`. Además, cada llamado suspendido ocupa memoria, que es el costo en espacio discutido al final de la Sección 1.6. En este texto se supone, como en el tratamiento de Python como modelo de cómputo de los capítulos de computabilidad, que la profundidad de la recursión no tiene límite; al implementar los algoritmos, sin embargo, conviene tener presente que esa suposición es una idealización.

1.8.4. Las constantes. Las mediciones de la Sección 1.1.2 mostraron que el mismo algoritmo tarda entre 50 y 300 veces más en Python que en C. Con lo visto en esta sección, esa diferencia tiene una explicación precisa: cada operación elemental del modelo abstracto se traduce en varias instrucciones del procesador cuando se ejecuta en C, y en muchas más cuando la ejecuta el intérprete de Python, que debe además verificar tipos, administrar memoria y despachar cada operación. La diferencia es, por tanto, un factor aproximadamente constante por operación elemental.

Esta es exactamente la clase de diferencia que la notación asintótica ignora a propósito (Teorema 1.3.1). Por eso las dos curvas de cada algoritmo en la Figura 1 son paralelas y por eso las conclusiones asintóticas del capítulo valen por igual en los dos lenguajes. Las funciones predefinidas de Python, que están implementadas en C, reducen esa constante: calcular `sum(A)` es más rápido que recorrer A con un ciclo escrito en Python, aunque ambos cuesten $\Theta(N)$. Esa ventaja es real y conviene aprovecharla, pero nunca compensa una diferencia de orden, como lo muestran el Ejemplo 1.8.2 y la tercera observación de la Sección 1.1.2.

Ejercicios

1. Determine el costo de cada uno de los siguientes fragmentos, suponiendo que A es una lista de N números:
 - a) `for x in A: B.insert(0, x).`
 - b) `for x in A: B.append(x).`
 - c) `c = sum(1 for i in range(N) if A[i] in A[:i]).`
 - d) `while A: A.pop(0).`
2. Reescriba el tercer fragmento del ejercicio anterior usando un conjunto, de manera que su costo en promedio esté en $\Theta(N)$.
3. El segundo algoritmo del Ejemplo 1.6.1 parte el arreglo por la mitad. Escriba en Python una versión que pase índices y otra que pase rebanadas, obtenga la recurrencia de cada una y determine sus órdenes.
4. Compare, con mediciones, el costo de la pertenencia `x in A` cuando A es una lista y cuando es un conjunto, para tamaños $N = 10^3, 10^4, \dots, 10^6$.
5. Construya una cadena de N caracteres agregando un carácter a la vez con `+=` y, alternativamente, acumulando los caracteres en una lista y usando `join`. Mida ambas versiones y discuta si los resultados coinciden con el análisis.
6. Implemente el cálculo de F_n con n sumas y mida su tiempo para $n = 10^4, 2 \cdot 10^4, 4 \cdot 10^4, \dots$. ¿Se observa un comportamiento lineal o cuadrático? Explique la respuesta con el Ejemplo 1.8.3.

7. Implemente el algoritmo Prefijos del Ejemplo 1.1.5 en Python y determine su costo y su espacio auxiliar. Compárelos con los del Algoritmo 1.8.2.
8. Escriba una versión de `rsum_index` que no use recursión y compare el tiempo de ambas para $N = 900$. ¿Qué ocurre con la versión recursiva para $N = 2000$ si no se modifica el límite de recursión?
9. Explique por qué el ciclo `for x in A` del Algoritmo 1.8.3 tiene el mismo costo que un ciclo sobre `range(len(A))` que accede a `A[i]`.

1.9. Análisis amortizado

Las técnicas de las secciones anteriores acotan el costo de un algoritmo sobre una instancia. Hay situaciones, sin embargo, en las que interesa el costo de una *secuencia* de operaciones sobre una misma estructura de datos y en las que acotar cada operación por su peor caso produce una cota demasiado pesimista: algunas operaciones son costosas, pero solo pueden ocurrir de vez en cuando, porque las operaciones baratas que las preceden son las que las hacen posibles. El análisis amortizado estudia el costo total de esas secuencias. La palabra “amortizado” viene de las finanzas y se refiere a repartir un gasto grande entre varios períodos.

Un resultado importante de esta sección para el resto del texto es la justificación del costo de `append` en las listas de Python, que la Sección 1.8 presentó como “ $\Theta(1)$ amortizado”. Antes de llegar a él, la sección presenta la definición de costo amortizado y tres métodos para calcularlo, cada uno ilustrado con un ejemplo.

1.9.1. Motivación y definición. El ejemplo más sencillo en el que el análisis operación por operación falla es una pila con una operación que retira varios elementos a la vez.

Ejemplo 1.9.1

Considere una pila con tres operaciones: `push(x)`, que inserta un elemento; `pop()`, que retira el elemento del tope; y `multipop(k)`, que retira elementos del tope hasta haber retirado k o hasta que la pila quede vacía. Como costo se cuenta la cantidad de elementos insertados o retirados.

Las dos primeras operaciones cuestan 1. La tercera cuesta $k \downarrow s$, en donde s es la cantidad de elementos en la pila y puede llegar a costar tanto como la cantidad de operaciones previas. Un análisis que acote cada operación por su peor caso concluye que una secuencia de K operaciones, a partir de la pila vacía, cuesta $O(K^2)$.

Esa cota es correcta, pero está lejos de ser ajustada. Cada elemento se retira a lo sumo una vez después de haber sido insertado, de modo que la cantidad total de elementos retirados, sumando todas las operaciones `pop` y `multipop`, no supera la cantidad de operaciones `push`. Luego, la secuencia completa cuesta a lo sumo $2K$.

El Ejemplo 1.9.1 sugiere asignar a cada operación un costo “promedio” que refleje su participación en el costo total, aunque una ejecución particular cueste más. La siguiente definición precisa esa idea.

Definición 1.9.1: Costo amortizado

Considere una estructura de datos con un estado inicial y un conjunto de operaciones. Se dice que las operaciones tienen costos amortizados $\hat{c}$ si, para toda secuencia de operaciones $o_1, \dots, o_K$ aplicada a partir del estado inicial, se tiene

$$(+i \mid 1 \leq i \leq K : c_i) \leq (+i \mid 1 \leq i \leq K : \hat{c}_i),$$

en donde c_i es el costo real de la operación o_i y $\hat{c}_i$ es el costo amortizado asignado a ella.

En el Ejemplo 1.9.1, por ejemplo, se pueden asignar costos amortizados 2 a `push`, y 0 a `pop` y a `multipop`: la suma de los costos amortizados es el doble de la cantidad de inserciones, que es una cota del costo real.

Nota 1.9.1

El costo amortizado no es un costo en el caso promedio en el sentido de la Nota 1.1.6. No hay en él ninguna distribución de probabilidad: la Definición 1.9.1 exige la desigualdad para *toda* secuencia de operaciones, incluida la más desfavorable. Es, por tanto, una garantía de peor caso, solo que sobre secuencias completas en lugar de operaciones individuales.

1.9.2. El método agregado. La forma más directa de obtener costos amortizados consiste en acotar el costo total de una secuencia arbitraria de K operaciones y dividir el resultado entre K . Fue lo que se hizo en el Ejemplo 1.9.1 y funciona bien cuando todas las operaciones son del mismo tipo.

Ejemplo 1.9.2

Un contador binario de k bits, representado por un arreglo $b[0..k)$ con el bit menos significativo en $b[0]$, se incrementa así: se recorren los bits desde $b[0]$ cambiando cada 1 por 0, hasta encontrar un 0, que se cambia por 1. El costo de un incremento es la cantidad de bits que cambia.

Un incremento puede cambiar los k bits (por ejemplo, al pasar de $01 \dots 1$ a $10 \dots 0$), de modo que un análisis operación por operación da $O(Kk)$ para $K < 2^k$ incrementos a partir de 0. Sin embargo, el bit $b[0]$ cambia en cada incremento, el bit $b[1]$ en uno de cada dos, el bit $b[2]$ en uno de cada cuatro y, en general, el bit $b[i]$ en uno de cada 2^i . La cantidad total de cambios es entonces

$$(+i \mid 0 \leq i < k : \lfloor K/2^i \rfloor) < K(+i \mid 0 \leq i < \infty : 2^{-i}) = 2K,$$

por la Propiedad 3 del Teorema 1.4.5. Luego, cada incremento tiene costo amortizado 2.

La Figura 10 muestra los primeros 16 incrementos de un contador de 5 bits. Los bits que cambian en cada incremento están resaltados, y la columna de la derecha acumula el costo total, que nunca alcanza el doble de la cantidad de incrementos.

valor	b_4	b_3	b_2	b_1	b_0	costo	total
0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	0	0	1	0	2	3
3	0	0	0	1	1	1	4
4	0	0	1	0	0	3	7
5	0	0	1	0	1	1	8
6	0	0	1	1	0	2	10
7	0	0	1	1	1	1	11
8	0	1	0	0	0	4	15
9	0	1	0	0	1	1	16
10	0	1	0	1	0	2	18
11	0	1	0	1	1	1	19
12	0	1	1	0	0	3	22
13	0	1	1	0	1	1	23
14	0	1	1	1	0	2	25
15	0	1	1	1	1	1	26
16	1	0	0	0	0	5	31

Figura 10. Los primeros 16 incrementos de un contador binario. Los bits resaltados son los que cambian en cada incremento; la columna b_i tiene un bit resaltado cada 2^i filas.

1.9.3. El método contable. El método agregado asigna el mismo costo amortizado a todas las operaciones. El método contable permite asignar costos distintos a operaciones distintas y razona en términos de un ahorro: cada operación paga su costo amortizado; si este excede el costo real, la diferencia queda guardada como *crédito* para pagar operaciones futuras. La única condición es que el crédito acumulado nunca sea negativo, pues solo así la suma de los costos amortizados es una cota del costo real.

Ejemplo 1.9.3

En la pila del Ejemplo 1.9.1, asigne costo amortizado 2 a **push**, y 0 a **pop** y a **multipop**. Cada **push** usa una unidad para insertar el elemento y deja la otra depositada sobre él. Cuando ese elemento se retira, ya sea con **pop** o como parte de un **multipop**, la extracción se paga con el crédito depositado.

El invariante es que cada elemento de la pila tiene exactamente una unidad de crédito, de modo que el crédito total es igual a la cantidad de elementos y nunca es negativo. Por la Definición 1.9.1, K operaciones cuestan a lo sumo $2K$.

1.9.4. El método del potencial. En el método contable, el crédito está repartido entre los elementos de la estructura. El método del potencial lo concentra en una sola cantidad: una función que asigna a cada estado de la estructura un número real, su *potencial*, que representa el trabajo que la estructura tiene “almacenado” para pagar operaciones futuras.

Teorema 1.9.1: Método del potencial

Sea Φ una función que asigna a cada estado de una estructura de datos un número real, tal que $\Phi(D_0) = 0$ para el estado inicial D_0 y $\Phi(D) \geq 0$ para todo estado D alcanzable. Si la operación o_i lleva la estructura del estado D_{i-1} al estado D_i con costo real c_i , entonces los valores

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

son costos amortizados en el sentido de la Definición 1.9.1.

Demostración

Al sumar los costos amortizados de una secuencia de K operaciones, los términos de potencial se cancelan en pares consecutivos:

$$\begin{aligned} (+i \mid 1 \leq i \leq K : \hat{c}_i) &= (+i \mid 1 \leq i \leq K : c_i) + \Phi(D_K) - \Phi(D_0) \\ &\geq (+i \mid 1 \leq i \leq K : c_i), \end{aligned}$$

pues $\Phi(D_K) \geq 0 = \Phi(D_0)$.

La dificultad del método está en escoger Φ : debe crecer con las operaciones baratas, para acumular potencial, y decrecer con las costosas lo suficiente como para compensar su costo real. El ejemplo principal de la sección, el arreglo dinámico, muestra una escogencia típica.

Ejemplo 1.9.4

Un *arreglo dinámico* almacena n elementos en un bloque de memoria con capacidad para $cap \geq n$ elementos. La operación `append(x)` escribe x en la posición n si hay espacio. Si el bloque está lleno, i.e., si $n = cap$, primero reserva un bloque nuevo con el doble de capacidad (o con capacidad 1, si $cap = 0$), copia allí los n elementos y luego escribe x . El costo es 1 en el primer caso y $n + 1$ en el segundo.

Tome $\Phi = 2n - cap$, con el estado inicial $n = cap = 0$. Después de la primera operación, la capacidad nunca supera el doble del tamaño, de modo que $\Phi \geq 0$. Para un `append` sin crecimiento, n aumenta en 1 y la capacidad no cambia, luego $\hat{c} = 1 + 2 = 3$. Para un `append` con crecimiento desde $n = cap \geq 1$, el potencial pasa de $2n - n = n$ a $2(n + 1) - 2n = 2$, luego

$$\hat{c} = (n + 1) + 2 - n = 3.$$

El primer `append`, que pasa de capacidad 0 a 1, tiene $\hat{c} = 1 + 1 = 2$. Por el Teorema 1.9.1, cada `append` tiene costo amortizado a lo sumo 3, y K operaciones a partir del arreglo vacío cuestan a lo sumo $3K$.

La Figura 11 muestra el costo real de cada una de las primeras 64 operaciones `append`. Las operaciones que duplican la capacidad aparecen como picos cada vez más altos, pero también cada vez más espaciados, y el costo promedio de las primeras k operaciones se mantiene por debajo de 3.

1.9.5. Consecuencias para el resto del texto. Las listas de Python son arreglos dinámicos y el Ejemplo 1.9.4 es la justificación del costo “ $\Theta(1)$ amortizado” que la tabla del Ejemplo 1.8.1 asigna a `append`. Dos precisiones completan el panorama.

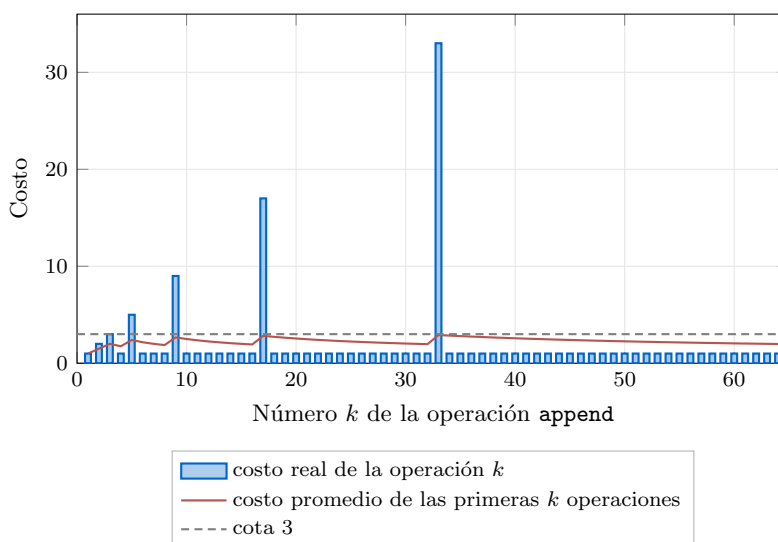


Figura 11. Costo de las primeras 64 operaciones `append` sobre un arreglo dinámico que duplica su capacidad al llenarse. Los picos corresponden a las operaciones que copian el contenido a un bloque nuevo.

Nota 1.9.2

El intérprete de referencia de Python no duplica la capacidad, sino que la aumenta en un factor cercano a $\frac{9}{8}$, lo cual desperdicia menos memoria. El análisis del Ejemplo 1.9.4 se adapta a cualquier factor de crecimiento $\alpha > 1$ y solo cambia la constante del costo amortizado, que crece a medida que α se acerca a 1 (Ejercicio 5). Lo que no funciona es crecer sumando una cantidad fija: si cada crecimiento agrega d celdas, hay un crecimiento cada d operaciones y K operaciones cuestan $\Theta(K^2/d)$, de modo que el costo amortizado de `append` deja de ser constante.

La consecuencia práctica es que una lista de Python puede usarse como pila, con `append` para insertar y `pop()` para retirar, con costo $\Theta(1)$ amortizado por operación. Los algoritmos de backtracking y de búsqueda del texto usan las listas exactamente de esa manera, y su análisis descansa en este resultado.

Por último, el análisis amortizado es también la herramienta con la que se estudian estructuras de datos más elaboradas. El ejemplo clásico es la estructura de conjuntos disjuntos con compresión de caminos, que se usa en el Capítulo 4: una secuencia de K operaciones sobre ella cuesta $O(K \alpha(K))$, en donde α es una función que crece tan lentamente que, en la práctica, no supera 4. Ese análisis está fuera del alcance de este texto y se puede consultar en [CLRS22].

Ejercicios

1. Suponga que el contador binario del Ejemplo 1.9.2 admite también una operación de decremento. Muestre una secuencia de K operaciones que cueste $\Theta(Kk)$ y explique por qué el argumento agregado deja de funcionar.
 2. Use el método del potencial con Φ igual a la cantidad de bits en 1 para demostrar que el incremento del contador binario tiene costo amortizado 2.
 3. Use el método del potencial con Φ igual a la cantidad de elementos en la pila para analizar la pila del Ejemplo 1.9.1. Compare con el Ejemplo 1.9.3.
 4. Una cola se puede implementar con dos pilas: los elementos se insertan en la primera y se retiran de la segunda; cuando la segunda está vacía y se pide retirar, se trasladan todos los elementos de la primera a la segunda. Demuestre que cada operación tiene costo amortizado constante.
 5. Considere un arreglo dinámico cuya capacidad crece en un factor $\alpha > 1$ al llenarse. Determine una cota para el costo amortizado de `append` en función de α y evalúela para $\alpha = 2$ y $\alpha = \frac{9}{8}$.
 6. Demuestre que si un arreglo dinámico crece agregando una cantidad fija d de celdas, K operaciones `append` a partir del arreglo vacío cuestan $\Theta(K^2/d)$.
 7. 🌈 Suponga que un arreglo dinámico también reduce su capacidad a la mitad cuando queda lleno hasta la mitad. Muestre una secuencia de operaciones `append` y `pop` en la que cada operación cueste $\Theta(n)$. Luego demuestre que reducir la capacidad a la mitad solo cuando el arreglo queda lleno hasta la cuarta parte sí da costo amortizado constante.
 8. Mida en Python el tiempo de cada una de 10^6 operaciones `append` consecutivas sobre una lista vacía y grafique los resultados. ¿Se observan los picos de la Figura 11? ¿Qué tan frecuentes son?
-

Notas del capítulo y referencias

El análisis asintótico es una herramienta cuyo alcance va más allá de comparar algoritmos. Con ella se decide también cuál estructura de datos conviene usar para implementar un tipo abstracto de datos, pues un mismo tipo abstracto admite implementaciones cuyas operaciones tienen órdenes muy distintos. El capítulo presenta dos casos: un conjunto de elementos sobre el cual se consulta la pertenencia cuesta $\Theta(N)$ por consulta si se implementa con una lista y $\Theta(1)$ en promedio si se implementa con una tabla de dispersión (Sección 1.8); una pila implementada sobre un arreglo dinámico cuesta $\Theta(1)$ amortizado por operación, siempre que el arreglo crezca multiplicando su capacidad y no sumándole una cantidad fija (Sección 1.9).

En ambos casos el tipo abstracto es el mismo y lo que cambia es el costo de cada operación, que a su vez determina el costo de todo algoritmo que lo use. Los capítulos siguientes toman este tipo de decisiones con frecuencia y en todas ellas el criterio es el que desarrolla este capítulo.

Saber medir, sin embargo, no es un arte sino una disciplina, que requiere precisión y entrenamiento. Cada análisis exige fijar el tamaño de la entrada, el caso que se estudia, la operación que se cuenta y el tipo de cota que se quiere establecer, y un descuido en cualquiera de esas decisiones puede producir una conclusión falsa con apariencia de correcta. El capítulo documenta varios de esos descuidos: afirmar un orden exacto sin exhibir una instancia que alcance la cota (Nota 1.5.2), reemplazar una suma por un producto cuando el costo varía entre iteraciones (Nota 1.5.1), arrastrar la notación O dentro de una inducción (Nota 1.6.3), aplicar el Teorema Maestro fuera de su alcance (Nota 1.7.2) o ignorar que una operación de Python con apariencia de elemental esconde un ciclo (Ejemplo 1.8.2). La única protección contra estos errores es la que ofrece cualquier disciplina: definiciones precisas, demostraciones completas y práctica.

Vistas en conjunto, las secciones del capítulo describen un procedimiento. Primero se especifica el problema y se escoge una medida del tamaño de su entrada; luego se fija el caso que interesa (casi siempre el peor) y una operación barómetro; después se traduce el algoritmo en una función, que es una suma si el algoritmo es iterativo y una recurrencia si es recursivo; por último, se clasifica esa función con las propiedades de la notación asintótica, el criterio del límite, las sumas de referencia, los árboles de recurrencia o el Teorema Maestro, según el caso. Las notas que siguen recorren ese procedimiento en el orden del capítulo, indicando para cada parte su origen y dónde profundizar.

El problema de la suma máxima de un subarreglo, usado como hilo conductor del capítulo, y la comparación entre los algoritmos cúbico, cuadrático, $n \log n$ y lineal que lo resuelven siguen la presentación de J. Bentley en [Ben00], en donde el algoritmo lineal se atribuye a J. Kadane. La noción de operación barómetro se toma de G. Brassard y P. Bratley [BB88]. Las mediciones de la Sección 1.1, y en particular la lectura del cociente entre tiempos al duplicar el tamaño de la entrada, siguen la propuesta experimental de R. Sedgewick y K. Wayne en [SW11], quienes hacen del experimento de duplicación una herramienta sistemática para formular y contrastar hipótesis sobre el orden de un programa.

La notación O es mucho más antigua que la computación. La introdujo P. Bachmann en 1894, en un tratado de teoría de números [Bac94], y la popularizó E. Landau algunos años después [Lan09]; por eso se la conoce también como notación de Landau. Su uso sistemático en el análisis de algoritmos se debe a D. E. Knuth, quien en una carta de 1976 [Knu76] propuso las notaciones Ω y Θ en la forma en que se usan hoy. La motivación de esa carta es la misma de la Nota 1.2.7: Knuth

había encontrado con frecuencia la notación O usada para expresar cotas inferiores, como cuando se descarta un método de ordenamiento “porque su tiempo es $O(n^2)$ ”, y consideró necesario contar con una notación propia para ellas.

La presentación en este texto de Ω y Θ , definidas a partir de O , no es la habitual; presentaciones convencionales de estas nociones se encuentran en, e.g., [CLRS22, KET06]. Los ejercicios de la Sección 1.2 que establecen la equivalencia entre las dos formulaciones están inspirados por la presentación de G. Brassard y P. Bratley en [BB88]. Cormen et al. [CLRS22] presentan también las notaciones o y ω , que son las versiones estrictas de O y Ω , y que no se usan en este texto.

Las propiedades de la Sección 1.3 son bien conocidas y se pueden consultar en varios de los textos de referencia citados aquí. Algunos de sus ejercicios están inspirados o aparecen en [BB88], en donde el criterio del límite se enuncia como ejercicio y su demostración, que se omite en este texto, también se encuentra. Para las sumas, los logaritmos, la aproximación de Stirling y, en general, el cálculo con funciones discretas que requiere la Sección 1.4, la referencia obligada es el texto de R. L. Graham, D. E. Knuth y O. Patashnik [GKP94], escrito precisamente como fundamento matemático para la computación. Los textos [CLRS22, KET06, BB88, Eri19] son una fuente extensa de más propiedades de O , Ω y Θ , y de ejemplos de su uso.

La presentación de los árboles de recurrencia en la Sección 1.6, y varios de sus ejercicios, están inspirados en el tratamiento del tema en [Eri19]. Cormen et al. [CLRS22] incluyen técnicas para resolver recurrencias complementarias a las de este capítulo. El Teorema Maestro, en una forma cercana a la de la Sección 1.7, se debe a J. L. Bentley, D. Haken y J. B. Saxe [BHS80]; el tratamiento de este texto sigue la propuesta de T. Roughgarden [Rou17], cuya demostración también se basa en un árbol de recurrencia; una demostración que trata directamente los pisos y techos se encuentra en [CLRS22]. Las recurrencias con subproblemas de tamaños distintos, que la Nota 1.7.2 deja por fuera del Teorema Maestro, se resuelven en general con el teorema de M. Akra y L. Bazzi [AB98], que lo generaliza.

Los costos de las operaciones de Python de la Sección 1.8 corresponden al intérprete de referencia y se documentan en [Pyt26]; otras implementaciones del lenguaje pueden diferir en algunos casos. El análisis amortizado de la Sección 1.9 fue formalizado por R. E. Tarjan [Tar85], quien reunió en ese artículo los métodos que el capítulo presenta y la motivación que los justifica: el análisis de peor caso por operación puede ser demasiado pesimista y el de caso promedio depende de suposiciones probabilísticas que pueden no cumplirse. El análisis de la estructura de conjuntos disjuntos mencionada al final de esa sección se debe también a Tarjan [Tar75].

El problema conductor deja una lección que vale la pena hacer explícita. Cada uno de sus cuatro algoritmos mejora al anterior en un orden de magnitud gracias a una idea de diseño: reutilizar sumas parciales, dividir el problema en mitades y

mantener la información justa para no tener que volver atrás. Ninguna mejora del procesador o del lenguaje compite con eso, como lo mostraron las mediciones de la Sección 1.1. Además, el algoritmo **Incremental** no admite una mejora asintótica: todo algoritmo correcto debe examinar cada uno de los N elementos, pues de lo contrario un elemento no examinado podría cambiarse por uno muy grande sin que el algoritmo lo advierta, y la respuesta cambiaría. Luego todo algoritmo para el problema hace $\Omega(N)$ operaciones: **Incremental** es asintóticamente óptimo. Este argumento no se refiere a un algoritmo en particular, sino al problema mismo, y establece una distinción que este capítulo solo roza: la complejidad de un algoritmo es una cota sobre los recursos que usa, mientras que la complejidad de un problema es una cota sobre los recursos que necesita cualquier algoritmo que lo resuelva. La cota $\log(n!) \in \Theta(n \log n)$ para los algoritmos de ordenamiento que comparan elementos, esbozada en la Sección 1.4, es otro ejemplo de esta segunda clase. Las ideas de diseño detrás de **Mitades** e **Incremental** son el tema de los Capítulos 2 y 3, y la complejidad de los problemas, el del Capítulo 8.

Conviene, por último, reconocer los límites de la herramienta. La notación asintótica ignora a propósito las constantes y el comportamiento para entradas pequeñas; en la práctica ambas cosas importan: el algoritmo de ordenamiento de la biblioteca de Python, por ejemplo, combina un método de orden $n \log n$ con el ordenamiento por inserción, que es cuadrático en el peor caso, pero más rápido que cualquier otro para arreglos cortos. En el otro extremo, hay algoritmos asintóticamente superiores que nunca se usan porque sus constantes son astronómicas, como varios de los sucesores del algoritmo de Strassen para multiplicar matrices. Tampoco es cierto que todos los accesos a memoria cuesten lo mismo, como supone la Definición 1.1.1: en un computador real, acceder a datos cercanos a los accedidos recientemente es mucho más rápido que acceder a datos lejanos. Por estas razones, el análisis asintótico indica cuáles algoritmos tienen futuro para entradas grandes, pero no reemplaza la medición: la complementa, como lo hicieron las mediciones de la Sección 1.1 al motivar las definiciones del capítulo.

El texto [SF13] de R. Sedgwick y P. Flajolet es una fuente de profundización y temas especializados en el área de análisis de algoritmos; se recomienda como lectura obligatoria (y selectiva) al lector decididamente interesado en el tema.

Dividir, conquistar y combinar

Resolver un problema algorítmico consiste, en el fondo, en recorrer un espacio de soluciones posibles sin tener que –necesariamente– examinarlo por completo. Las técnicas de diseño que presenta este texto son maneras distintas de lograrlo y esta es la primera de ellas: partir el problema en subproblemas independientes, resolver cada uno por separado y construir con sus soluciones la solución del problema original.

La idea no es nueva ni es exclusiva de la computación. Está basada en un principio que la humanidad conoce y usa desde hace siglos para construir demostraciones: el principio de *inducción matemática*. Una instancia de un problema se divide en instancias del mismo problema, pero más pequeñas, que es lo que el nombre de la técnica llama *dividir*; las soluciones de esas instancias se consolidan en una solución de la instancia original, que es *combinar*; y las instancias que ya no hace falta dividir se resuelven directamente, que es *conquistar*. Esas tres fases son, respectivamente, la escogencia de las instancias sobre las cuales se aplica la hipótesis inductiva, el caso inductivo y los casos base de una demostración por inducción. La técnica suele llamarse *dividir y conquistar*; en este texto se usa el nombre completo porque, como se verá, la fase de combinación es con frecuencia en donde recae gran parte del diseño algorítmico y la que decide qué tan bueno resulta el algoritmo.

Vista así, esta técnica es la más elemental de las cuatro que estudia el texto y esa elementalidad es justamente lo que la hace central: las otras tres se leen como refinamientos suyos. Todas reducen una instancia a instancias más pequeñas del mismo problema y se distinguen por lo que le agregan a esa reducción: la programación dinámica recuerda las soluciones ya calculadas, porque sus subinstancias se repiten; los algoritmos voraces generan una sola subinstancia, dictada por un criterio de selección; y el reintento abandona subinstancias sin terminar de resolverlas cuando no aportan a la solución buscada.

Técnica	Capítulo	¿Qué le agrega?
Dividir, conquistar y combinar	2	nada
Programación dinámica	3	memoria
Algoritmos voraces	4	un criterio de selección
Reintento	5	poda

Conviene ser franco con lo que esta tabla afirma y con lo que no. Es un mapa para leer el texto y no un resultado matemático: un algoritmo voraz no es, en ningún sentido formal, un caso particular de dividir, conquistar y combinar. Lo que sí es cierto, lo que el lector debería llevarse, es que el esquema de razonamiento que se presenta aquí reaparece en los tres capítulos siguientes con ingredientes adicionales; quien lo domine en su forma pura tendrá menos que aprender más adelante.

La técnica tiene además una hoja de vida notable. En 1960, A. Kolmogórov conjeturó en un seminario que multiplicar dos números de n dígitos requería del orden de n^2 operaciones elementales. Una semana después, A. Karatsuba, entonces estudiante, refutó la conjetura partiendo cada número en dos mitades y observando que los cuatro productos que parecían necesarios se podían obtener con tres [KO62]. En 1965, J. Cooley y J. Tukey publicaron la transformada rápida de Fourier, que separa una señal en sus componentes de índice par e impar y reduce el cálculo de $O(n^2)$ a $O(n \log n)$ [CT65]; está debajo del audio digital, de la compresión de imágenes y del procesamiento de señales, y es difícil encontrar un algoritmo con mayor impacto práctico en el siglo pasado. Más atrás todavía, el algoritmo de mezcla que estudia la Sección 2.5 se le atribuye a J. von Neumann en 1945, cuando apenas existían los computadores de programa almacenado [Knu88].

Hay una razón de fondo por la cual esta técnica ocupa un lugar central en el pensamiento algorítmico y es la independencia de las subinstancias. Cuando dos subproblemas no comparten nada, no solo se pueden resolver en cualquier orden: se pueden resolver *al mismo tiempo*, en procesadores distintos, en máquinas distintas o en centros de datos distintos. Esa propiedad es la que sostiene buena parte de la computación paralela y distribuida. Un algoritmo diseñado con esta técnica suele admitir una versión paralela casi sin modificarlo, mientras que uno diseñado con las técnicas de los capítulos siguientes rara vez lo permite, porque sus subinstancias dependen unas de otras.

Nota 2.0.1: Dividir fuera de la computación

La idea es anterior a los computadores y aparece en situaciones que nada tienen que ver con ellos. Un torneo de eliminación directa encuentra un campeón entre n equipos en $\log_2 n$ rondas. El conteo de una votación se hace por mesa y

luego se agrega por localidad. Para ubicar la falla de un cable de un kilómetro se mide en la mitad, lo cual descarta medio kilómetro de una sola vez.

Un ejemplo muy instructivo es el de repartir un trabajo entre varias personas, porque trae consigo la lección de fondo: dividir no es gratis. Quien reparte debe después consolidar lo que cada quien hizo y ese costo de coordinación es exactamente lo que en este capítulo se llama combinar. Cuando la consolidación es cara, repartir puede no mejorar nada; cuando es barata, la ganancia es enorme. Las recurrencias que se plantean a lo largo del capítulo presentan diferentes demandas de consolidación de los resultados de las subinstancias.

Nota 2.0.2: ¿Dónde está la dificultad?

Cada técnica de diseño concentra su dificultad en un lugar distinto y vale la pena decir desde ahora dónde está la de esta. No está en la correctitud: una vez escogida la manera de dividir, la demostración es una inducción que sigue usualmente el mismo esquema y que la Sección 2.1 enuncia como punto de partida. Tampoco está en la implementación, que suele ser una transcripción del diseño. Está en dos lugares menos visibles. El primero es la escogencia de la descomposición, pues un mismo problema admite varias y no todas conducen a un algoritmo mejor que el evidente. El segundo es el diseño de la función que combina o que divide, que es un algoritmo en sí misma, con su propia especificación y su propia demostración. A eso hay que sumarle el análisis, que aquí toma la forma de una recurrencia.

Este capítulo comienza con las ideas centrales de la técnica en la Sección 2.1 y con un problema, en la Sección 2.2, que permite apreciarla sin la distracción de un algoritmo: allí la construcción es, literalmente, una demostración por inducción. La Sección 2.3 recoge lo aprendido en una metodología para diseñar este tipo de algoritmos a partir de la especificación de un problema, que es la que guía el resto del capítulo.

Vienen luego los ejemplos, agrupados en tres bloques. El primero son los algoritmos de ordenamiento de las secciones 2.4 a 2.6, que muestran cómo el mismo problema cambia de complejidad según dónde se ponga el trabajo. El segundo son los algoritmos de búsqueda de las secciones 2.7 a 2.10, que parten de la búsqueda binaria en un arreglo ordenado, la generalizan hasta quedarse con lo esencial y terminan aplicándola a un problema que no parece tener nada que ver con ella. El tercero es la Sección 2.11, que lleva la técnica fuera de los arreglos y trabaja directamente con números. El capítulo cierra con notas y referencias. Algunos problemas y demostraciones se proponen como ejercicios para el lector.

2.1. Ideas clave

Considere un arreglo $A[0..N]$ de números, con $N > 0$, y el objetivo de calcular su máximo. Hay al menos dos maneras naturales de reducir este problema a instancias más pequeñas de sí mismo. La primera es separar el primer elemento del resto: el máximo de $A[0..N]$ es el mayor entre $A[0]$ y el máximo de $A[1..N]$. La segunda es partir el arreglo por la mitad: el máximo de $A[0..N]$ es el mayor entre los máximos de $A[0..N/2]$ y de $A[N/2..N]$ (suponiendo por ahora que $N \geq 2$ y dejando para después los detalles de la división entera). Las dos maneras calculan lo mismo y, como se verá, hacen la misma cantidad de comparaciones. Sin embargo, no son el mismo algoritmo: la primera anida N llamados recurrentes y la segunda apenas del orden de $\log_2 N$.

La técnica que se estudia en este capítulo consiste, precisamente, en resolver un problema reduciéndolo a instancias más pequeñas del mismo problema. Lo que el ejemplo anterior anticipa es que la manera de reducirlo es una decisión de diseño y que esa decisión tiene consecuencias.

2.1.1. Descomposiciones. Para hacer precisa la idea de reducir una instancia a otras más pequeñas conviene fijar qué se entiende por *más pequeña*.

Definición 2.1.1: Descomposición

Considere un problema algorítmico con conjunto de instancias I . Una *descomposición* del problema está dada por:

1. un conjunto $B \subseteq I$ de *instancias base*;
2. para cada instancia $x \in I \setminus B$, una secuencia finita y no vacía $x_1, \dots, x_a$ de instancias del problema, llamadas las *subinstancias* de x ; y
3. una *medida* $\mu : I \rightarrow \mathbb{N}$ tal que, para cada $x \in I \setminus B$, se tiene $\mu(x_i) < \mu(x)$ para $1 \leq i \leq a$.

Las instancias base son aquellas que se resuelven sin recurrir a otras instancias. La medida es la que da sentido a la palabra *pequeña*: cada subinstancia tiene medida estrictamente menor que la instancia de la cual proviene. Como la medida toma valores en los números naturales, no existen cadenas infinitas de instancias con medida estrictamente decreciente. Esta es la razón por la cual resolver una instancia a partir de sus subinstancias y estas a partir de las suyas, termina.

En los problemas sobre arreglos de este capítulo las instancias son subarreglos $A[low..hi]$ y la medida es casi siempre su longitud $hi - low$. No siempre es así: en la Sección 2.2 las instancias son tableros y la medida es el exponente de su lado.

Ejemplo 2.1.1

Para el cálculo del máximo, las instancias son los subarreglos $A[low..hi]$ con $low < hi$ y la medida es $\mu = hi - low$. Las instancias base son los subarreglos de un elemento, es decir, aquellos con $\mu = 1$. Las dos maneras de reducir el problema son dos descomposiciones distintas:

- **Por decremento:** la única subinstancia de $A[low..hi]$ es $A[low + 1..hi]$, cuya medida es $\mu - 1$.
- **Por mitades:** con $mid = low + \lfloor (hi - low)/2 \rfloor$, las subinstancias son $A[low..mid]$ y $A[mid..hi]$. Como $\mu \geq 2$, se tiene $low < mid < hi$ y ambas medidas son estrictamente menores que μ (¿por qué?).

La condición $\mu \geq 2$ en la descomposición por mitades no es un detalle. Si se permitiera partir un subarreglo de un elemento, una de las dos mitades sería vacía y la otra sería el mismo subarreglo, con la misma medida. La Definición 2.1.1 prohíbe esa situación y, con ello, un algoritmo que no termina.

La Definición 2.1.1 no captura una propiedad que tienen todas las descomposiciones de este capítulo y que es la que distingue a la técnica.

Nota 2.1.1: Independencia de las subinstancias

En las descomposiciones de este capítulo, las subinstancias de una instancia son *independientes*: resolver una de ellas no requiere la solución de ninguna de las otras y, al desplegar la descomposición, una misma subinstancia no se necesita desde dos lugares distintos. En los problemas sobre arreglos esto se traduce en que las subinstancias de $A[low..hi]$ son subarreglos disjuntos.

Esta es exactamente la propiedad que falla en los problemas de la Sección 3.1, donde las subinstancias se solapan y resolverlas por separado repetiría trabajo. Cuando las subinstancias son independientes no hay nada que reusar y basta con resolver cada una en el momento en que se encuentra.

2.1.2. Las tres fases. Una vez fijada una descomposición, resolver una instancia x consiste en tres fases:

- **Conquistar:** si x es una instancia base, se resuelve directamente.
- **Dividir:** de lo contrario, se construyen las subinstancias $x_1, \dots, x_a$ de x y cada una se resuelve recurrentemente.
- **Combinar:** a partir de las soluciones de las subinstancias se construye una solución de x .

Estas tres fases son las de una demostración por inducción. Conquistar corresponde a los casos base. Dividir corresponde a escoger las instancias sobre las cuales se aplica la hipótesis inductiva, que en el algoritmo toma la forma de llamados recurrentes. Combinar corresponde al caso inductivo, que construye la conclusión a partir de lo que la hipótesis garantiza. De hecho, la inducción no es solo una analogía: es la manera de demostrar que un algoritmo diseñado de esta forma es correcto.

Inducción matemática	Técnica	Algoritmo recurrente
Caso base	Conquistar	Solución directa
Hipótesis inductiva	Dividir	Llamados recurrentes
Caso inductivo	Combinar	Consolidación de las soluciones

La correspondencia de la tabla es una simplificación del principio de inducción matemática, pero es la que conviene tener en mente al diseñar: cada fase del algoritmo responde por una parte de la demostración.

Algoritmo 2.1.1: Esquema general de dividir, conquistar y combinar

```

1 def solve(x):
2   if is_base(x):
3     ans = conquer(x)           # conquer
4   else:
5     xs = divide(x)             # divide
6     sols = [solve(y) for y in xs]
7     ans = combine(x, sols)     # combine
8   return ans

```

Las funciones `is_base`, `conquer`, `divide` y `combine` son las que cambian de un problema a otro. Lo que no cambia es la manera en que el esquema las usa y, en consecuencia, lo que se debe exigir de cada una de ellas para que el esquema sea correcto.

Teorema 2.1.1

Considere un problema con una descomposición en el sentido de la Definición 2.1.1 y suponga que:

1. `is_base(x)` es cierto si y solo si $x \in B$;
2. si $x \in B$, entonces `conquer(x)` es una solución de x ;
3. si $x \notin B$, entonces `divide(x)` retorna las subinstancias $x_1, \dots, x_a$ de x ; y

4. si $x \notin B$ y s_i es una solución de x_i para $1 \leq i \leq a$, entonces el llamado `combine` con argumentos x y $[s_1, \dots, s_a]$ retorna una solución de x .

Entonces, para toda instancia x , el llamado `solve(x)` del Algoritmo 2.1.1 termina y retorna una solución de x .

Demostración

Se procede por inducción fuerte sobre $\mu(x)$. Sea x una instancia y suponga, como hipótesis inductiva, que para toda instancia y con $\mu(y) < \mu(x)$ el llamado `solve(y)` termina y retorna una solución de y .

- Si $x \in B$, por (1) el esquema ejecuta `conquer(x)`, que por (2) es una solución de x . No hay llamados recurrentes, de modo que el llamado termina.
- Si $x \notin B$, por (1) y (3) el esquema construye las subinstancias $x_1, \dots, x_a$. Por la Definición 2.1.1, $\mu(x_i) < \mu(x)$ para cada i , así que la hipótesis inductiva garantiza que cada llamado `solve(xi)` termina y retorna una solución s_i de x_i . Como la secuencia de subinstancias es finita, la construcción de `sols` termina. Por (4), el valor retornado es una solución de x .

Como la medida toma valores en $\mathbb{N}$, el principio de inducción fuerte permite concluir que la propiedad vale para toda instancia.

El Teorema 2.1.1 reparte la corrección en obligaciones locales. Las condiciones (1) y (3) dicen que el algoritmo implementa la descomposición escogida. La condición (2) es la corrección del caso base y casi siempre es inmediata. La condición (4) es la que concentra el esfuerzo: es la corrección de la combinación, suponiendo correctas las soluciones de las subinstancias. Nótese que en ningún lugar de la demostración se razona sobre la ejecución de los llamados recurrentes; basta con lo que la hipótesis inductiva garantiza de ellos.

Nota 2.1.2: Lo que el teorema no dice

El Teorema 2.1.1 garantiza corrección y terminación, pero no dice nada sobre eficiencia. Dos descomposiciones del mismo problema, ambas satisfaciendo las cuatro condiciones, pueden dar lugar a algoritmos con costos muy distintos. De eso se ocupa la subsección siguiente.

2.1.3. Invariantes de ciclo. La inducción es la herramienta con la cual se demuestra que un algoritmo recurrente es correcto. Cuando el mismo algoritmo se

escribe con ciclos en lugar de llamados recurrentes, que es el paso opcional de la metodología de la Sección 2.3, la herramienta es otra: el invariante.

Nota 2.1.3: Invariante de ciclo

Un *invariante* (de ciclo) es una fórmula lógica asociada a un ciclo iterativo que cumple las siguientes dos condiciones:

- ser cierta antes de que inicien las iteraciones del ciclo
- ser cierta después de cualquier iteración del ciclo.

El diseño de algoritmos con invariantes requiere de disciplina, práctica y un entendimiento claro de la estrategia de solución que materializa el ciclo al cual está asociado. Cuando los invariantes están bien planteados y son suficientemente completos, es relativamente fácil obtener un ciclo iterativo correcto. La estrategia en estas notas es diseñar ciclos iterativos a partir de invariantes, es decir, formularlos antes de escribir el ciclo y no después.

Nota 2.1.4: Cómo se verifica un invariante

Para demostrar que un ciclo iterativo C con condición de terminación B es correcto con respecto a un invariante P es necesario demostrar que:

1. [Iniciación] P es cierto antes de la primera iteración de C y
2. [Estabilidad] Si P es cierto antes de una iteración de C , entonces P es cierto después de una iteración de C (i.e., que $P \wedge B$ implica P después de la terminación de una iteración de C).

Adicionalmente, una vez terminen las iteraciones de C , el invariante P debe suministrar información importante acerca del objetivo de C (i.e., $P \wedge \neg B$ deben proveer información sobre la finalidad de C).

Las secciones siguientes usan estas dos herramientas de manera complementaria: la inducción para los algoritmos recurrentes y los invariantes para los ciclos que aparecen dentro de ellos, como los que dividen o combinan.

2.1.4. ¿Dónde está el trabajo? Sea $T(x)$ la cantidad de operaciones que realiza `solve(x)`. Si x no es una instancia base, el Algoritmo 2.1.1 permite leer directamente una recurrencia:

$$T(x) = D(x) + (+i \mid 1 \leq i \leq a : T(x_i)) + C(x),$$

donde $D(x)$ es el costo de dividir y $C(x)$ es el costo de combinar. La recurrencia muestra que hay tres lugares donde puede estar el trabajo: en la división, en la

combinación, y en la cantidad y el tamaño de las subinstancias. Los algoritmos de este capítulo se distinguen, precisamente, por la manera en que reparten el trabajo entre esos tres lugares. La tabla siguiente lo resume; note que compara algoritmos de este capítulo, a diferencia de la tabla de la introducción, que compara entre sí las cuatro técnicas de diseño del texto.

Algoritmo	Sección	Subinstancias	Dividir	Combinar
Ordenamiento	2.4	una, de tamaño $n - 1$	$O(n)$	$O(1)$
<i>MergeSort</i>	2.5	dos, de tamaño $n/2$	$O(1)$	$O(n)$
<i>QuickSort</i>	2.6	dos, de tamaño variable	$O(n)$	$O(1)$
Búsqueda binaria	2.7	una, de tamaño $n/2$	$O(1)$	$O(1)$
Exponenciación	2.11	una, de tamaño $n/2$	$O(1)$	$O(1)$

La tabla contiene varias observaciones que se desarrollan a lo largo del capítulo. El ordenamiento de la Sección 2.4 y *MergeSort* reparten el trabajo de maneras opuestas: el primero trabaja antes de recurrir, al ubicar el mínimo, y el segundo después, al mezclar. *QuickSort* trabaja, como el primero, antes de recurrir, pero lo hace sobre dos subinstancias y no sobre una. La búsqueda binaria, por su parte, casi no trabaja en ninguna de las dos fases: toda su eficiencia proviene de descartar la mitad del espacio en cada paso.

Cuando las a subinstancias tienen el mismo tamaño n/b , y dividir y combinar cuestan en conjunto $O(n^k)$, la recurrencia anterior corresponde a una función simple y su solución la da el Teorema Maestro del Capítulo 1; así se analiza, por ejemplo, la búsqueda binaria en el Ejemplo 1.7.3. Cuando las subinstancias se obtienen restando una constante, y no dividiendo por una, el teorema no aplica y la recurrencia debe resolverse de otra manera.

Ejemplo 2.1.2

Considere las dos descomposiciones del Ejemplo 2.1.1. En ambas, dividir y combinar toman tiempo constante: calcular *mid* y comparar dos números. Para la descomposición por decremento, la recurrencia es

$$T(n) = T(n - 1) + O(1),$$

que no corresponde a una función simple. Desplegándola se obtiene $T(n) \in O(n)$. Para la descomposición por mitades, la recurrencia es

$$T(n) = 2 \cdot T(n/2) + O(1),$$

con $a = 2$, $b = 2$ y $k = 0$. Como $a = 2 > 1 = b^k$, el Teorema Maestro da $T(n) \in O(n^{\log_2 2}) = O(n)$.

Los tiempos coinciden, pero no el espacio. Cada llamado recurrente pendiente ocupa espacio en la pila de ejecución y la cantidad máxima de llamados pendientes es la profundidad de la recurrencia: n para la primera descomposición y del orden de $\log_2 n$ para la segunda. Así, la primera usa espacio $O(n)$ y la segunda espacio $O(\log n)$.

El Ejemplo 2.1.2 cierra el ejemplo con el cual se abrió la sección. La elección de la descomposición no afectó la corrección, que en ambos casos se sigue del Teorema 2.1.1, pero sí el consumo de recursos. En los problemas de las secciones siguientes la diferencia será mayor: la misma idea de pasar de una reducción por decremento a una por mitades es la que lleva de un ordenamiento en tiempo $O(n^2)$ a uno en tiempo $O(n \log n)$.

Ejercicios

1. Implemente en Python las dos maneras de calcular el máximo de un subarreglo $A[low..hi]$ descritas en el Ejemplo 2.1.1. Para cada una, identifique las funciones `is_base`, `conquer`, `divide` y `combine` del Algoritmo 2.1.1 y demuestre, usando el Teorema 2.1.1, que es correcta.
2. En el Ejemplo 2.1.1, justifique que $low < mid < hi$ cuando $hi - low \geq 2$. ¿Qué ocurre si se escoge $mid = low + \lceil (hi - low)/2 \rceil$? ¿Y si las instancias base fueran los subarreglos vacíos?
3. Considere el problema de calcular la suma de los elementos de un arreglo $A[0..N)$, con $N \geq 0$.
 - a) Proponga una descomposición por decremento y otra por mitades, indicando en cada caso las instancias base, las subinstancias y la medida.
 - b) Plantee la recurrencia del tiempo de cada una y resuélvala.
 - c) Determine la complejidad espacial de cada una.
4. Considere el cálculo de a^n , con a un número y $n \in \mathbb{N}$.
 - a) Proponga una descomposición basada en la igualdad $a^n = a \cdot a^{n-1}$ y otra basada en las igualdades $a^{2m} = (a^m)^2$ y $a^{2m+1} = a \cdot (a^m)^2$.
 - b) Para cada una, determine cuántas multiplicaciones se realizan en función de n .
5. La sucesión de Fibonacci satisface $F_n = F_{n-1} + F_{n-2}$ para $n \geq 2$, con $F_0 = 0$ y $F_1 = 1$.
 - a) Proponga una descomposición a partir de esta definición y verifique que satisface la Definición 2.1.1.
 - b) Muestre que la descomposición no satisface la Nota 2.1.1.

- c) Explique por qué el Algoritmo 2.1.1 instanciado con esta descomposición es correcto pero ineficiente. Contraste su respuesta con la Sección 3.1.
6. Muestre con un ejemplo que la condición $\mu(x_i) < \mu(x)$ de la Definición 2.1.1 es necesaria para la terminación del Algoritmo 2.1.1. ¿En qué lugar de la demostración del Teorema 2.1.1 se usa?
7. Suponga que en el cálculo del máximo por mitades se usan las subinstancias $A[low..mid + 1)$ y $A[mid..hi)$, que comparten el elemento $A[mid]$. ¿Sigue siendo correcto el algoritmo? ¿Sigue siendo una descomposición en el sentido de la Definición 2.1.1? ¿Afecta esto su complejidad temporal?
8. 🌐 Una vez leído el capítulo, complete la tabla de la subsección ¿Dónde está el trabajo? con la teselación de la Sección 2.2. Para cada algoritmo de la tabla, plantee la recurrencia de su tiempo e indique si puede resolverse con el Teorema Maestro.

2.2. Teselación de tableros de ajedrez

Esta sección presenta el problema de la teselación de tableros de ajedrez y lo resuelve con la técnica de este capítulo. Es un problema particularmente apropiado para apreciarla, porque en él el algoritmo y su demostración de correctitud son una sola cosa: lo que se construye es una demostración por inducción de que la teselación existe y el algoritmo se lee de ella. En las secciones siguientes las dos cosas serán distintas, pues primero se diseñará un algoritmo y después se demostrará que satisface su especificación.

2.2.1. Especificación del problema. A continuación se especifica el problema de teselación de los tableros de ajedrez generalizados.

Problema 2.2.1: Teselación de un tablero generalizado

Entrada: un tablero cuadrado de lado 2^N , $N \geq 1$, con una celda faltante.

Salida: ¿es posible teselar el tablero con fichas en forma de L que comprenden exactamente tres celdas del tablero?

A continuación se muestran: (a) un tablero generalizado de lado 4; (b) fichas con las cuales se teselan los tableros (de las cuales siempre hay suficientes); y (c) una posible teselación de un tablero de lado 4. Esto se ilustra en la Figura 1.

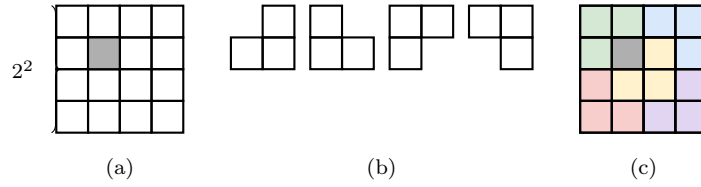


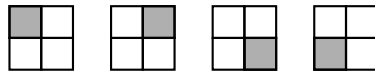
Figura 1. (a) un tablero generalizado de lado 4; (b) las fichas en forma de L ; y (c) una teselación del tablero de (a).

El problema se plantea de manera general, i.e., para cualquier tablero que cumpla con las indicaciones de la entrada. Como N no está acotado, esto significa que la cantidad de tableros es infinita y que no es posible tratar de construir teselaciones enumerando explícitamente todos los tableros. Si se formulase el problema directamente a modo de una fórmula matemática (e.g., $\forall N. N \geq 1 \dots$), sería claro que una opción es proceder por inducción matemática.

2.2.2. Diseño de una solución. Se propone un algoritmo recurrente que está basado en análisis de casos, siguiendo las ideas de una demostración por inducción. Los tableros que pueden ser teselados directamente se tratan como casos base, mientras que los tableros que no son lo suficientemente pequeños como para ser teselados directamente se teselan al componer las teselaciones de tableros generalizados más pequeños. En los términos de la Definición 2.1.1, las instancias son los tableros de lado 2^n con la posición de su hueco, las instancias base son las de lado 2 y la medida es el exponente n , que disminuye en uno al pasar a los subtableros.

Algoritmo 2.2.1

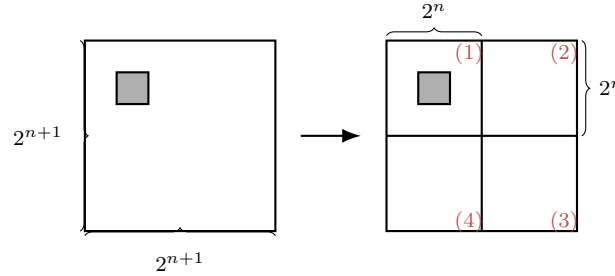
- Los tableros más pequeños se pueden enumerar; estos corresponden al exponente $n = 1$, es decir, a los tableros de lado 2, y son cuatro:



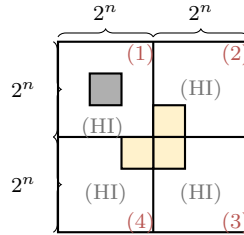
Cada uno de estos tableros se puede teselar con una ficha en forma de L . Esto concluye el *caso base* o, más bien, los *casos base*.

- Los demás tableros tienen lado 2^{n+1} , con $n \geq 1$. Se supondrá, a modo de *hipótesis inductiva*, que todo tablero de lado 2^n con exactamente un hueco se puede teselar, tal cual como se procede por inducción matemática sobre el exponente. La idea es entonces encontrar una forma de teselar un tablero de lado 2^{n+1} que tiene exactamente un hueco. Note que si ese tablero se divide en 4 partes iguales, resultan 4 tableros de lado 2^n (enumerados de 1 a 4 en la gráfica a continuación), lo cual se acerca

a la situación que describe la hipótesis inductiva. Estos dos hechos se representan gráficamente a continuación:



Como el subtablero 1 tiene lado 2^n y un hueco, se puede teselar por la hipótesis inductiva. Sin embargo, los otros 3 subtableros no tienen hueco y por ello no se puede usar la hipótesis inductiva directamente. Pero, el hecho de que no tengan hueco no quiere decir que no se pueda suponer *convenientemente* que tienen un hueco: uno que resulta de ubicar una de las fichas en forma de L en sus tres esquinas concéntricas:



Ahora, los subtableros 2-4 se pueden tratar como si tuvieran un hueco en una de sus esquinas y consecuentemente cumplen con la condición de la hipótesis inductiva (HI). Es decir, cada uno de ellos se puede teselar al tener lado 2^n y exactamente un hueco. Por consiguiente, todo el tablero de lado 2^{n+1} se puede teselar.

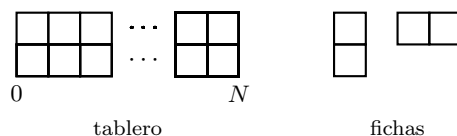
La solución dada en el problema anterior muestra cómo un tablero de ajedrez generalizado y con un hueco puede ser teselado. En realidad, brinda más que eso, pues demuestra constructivamente la posibilidad de encontrar una teselación con ayuda de un algoritmo. En los casos base, el problema se resuelve directamente teselando con una ficha (i.e., conquistar). Mientras que en los casos inductivos: (i) se “recurre” sobre el subtablero en donde está el hueco dado que es una subinstancia del problema inicial (i.e., dividir); y (ii) en los otros tres subtableros se marcan las esquinas concéntricas como si tuvieran un hueco, se recurre en cada subtablero (i.e., dividir) y finalmente se ubica una ficha en forma de L sobre las tres esquinas concéntricas de los subtableros teselados para completar la teselación

(i.e., combinar). Este algoritmo puede ser usado en cualquier tablero por el estilo: está garantizada su correctitud por una demostración por inducción matemática.

Esa correspondencia entre el principio de inducción matemática, la técnica de este capítulo y la estructura de un algoritmo recurrente es la que resume la Sección 2.1.

Ejercicios

1. Con base en la demostración que prueba la existencia de una teselación de un tablero de ajedrez generalizado:
 - a) Diseñe un algoritmo en el lenguaje de programación Python3 que tesele un tablero dado.
 - b) Extienda su implementación para que, además de la teselación, produzca una representación gráfica del tablero teselado en arte ASCII, de modo que el resultado se pueda verificar visualmente para tableros pequeños. Investigue sobre la herramienta `ditaa`, que convierte ese tipo de representaciones en imágenes.
2. Un tablero de ajedrez *doblemente* generalizado es un tablero cuadrado de lado 2^N , $N \geq 1$, al cual le hacen falta dos celdas de colores opuestos (asumiendo que está coloreado como un tablero de ajedrez convencional; por ejemplo, una celda blanca y una negra).
 - a) ¿Es posible teselar dicho tablero con (suficientes) fichas de dominó, en donde cada ficha cubre exactamente dos celdas adyacentes del tablero? Si su respuesta es negativa, suministre un contraejemplo. De lo contrario, construya una demostración siguiendo la estructura propuesta en esta sección.
 - b) 🎨 Considere la misma pregunta del numeral anterior para tableros cuadrados de lado $N \geq 1$ (a los cuales le faltan dos celdas de colores opuestos). Tenga en cuenta que esta es una generalización del problema inicial. ¿Es posible teselar cualquiera de estos tableros?
3. Considere un tablero de ajedrez de 8×8 celdas que ha sido “mutilado” removiendo exactamente dos celdas de esquinas opuestas. Note que, a diferencia del Ejercicio 2, las dos celdas removidas son del mismo color.
 - a) Especifique el problema de determinar si dichos tableros se pueden teselar con (suficientes) fichas de dominó, en donde cada ficha de dominó cubre exactamente dos celdas adyacentes.
 - b) Resuelva el problema especificado en el numeral anterior.
4. Considere tableros de $2 \times N$ celdas y fichas de dominó que cubren exactamente dos celdas adyacentes.



Proponga una función recurrente que determine la *cantidad* de formas de teselar el tablero con (suficientes) fichas de dominó. Justifique su respuesta con una demostración.

5. Las *Torres de Hanói* son un rompecabezas matemático inventado en 1883 por el matemático francés É. Lucas. El juego consta de tres postes fijados a un tablero y de varios discos perforados de radios distintos, apilados de mayor a menor en uno de los postes. El objetivo es trasladar la pila completa a otro poste, con tres reglas: se mueve un disco a la vez, solo se puede mover el disco que esté encima de un poste y nunca se puede colocar un disco sobre otro de menor radio.

En este ejercicio hay dos objetivos: demostrar que el juego tiene solución y que hay un algoritmo que lo resuelve con una cantidad de movimientos dada.

- Especifique el problema de las Torres de Hanói para $N \geq 1$ discos perforados.
 - Demuestre que el problema especificado es soluble para cualquier N .
 - Proponga un algoritmo que lleve a cabo exactamente $2^N - 1$ movimientos de discos para resolver el problema. Demuestre que su algoritmo resuelve el problema dado.
6. Sean k y l números naturales. Se dice que k es *divisible* por l si y solo si existe un $p \in \mathbb{N}$ tal que $k = p \cdot l$ (en este caso, p se llama el *testigo* de la divisibilidad de k entre l). Por ejemplo, 15 es divisible por 3 con testigo 5 porque $15 = 5 \cdot 3$. Use el principio de inducción matemática para demostrar que $11^n - 4^n$ es divisible por 7 para todo $n \in \mathbb{N}$.
7. Demuestre que $x^n - 1$ es divisible por $x - 1$ para todo $n \in \mathbb{N}$ y todo $x \in \mathbb{N}$ con $x \geq 1$.
8. La función $F : \mathbb{N} \rightarrow \mathbb{N}$ de Fibonacci se define inductivamente de la siguiente manera:

$$F(0) = 0 \quad F(1) = 1 \quad F(n) = F(n-2) + F(n-1), \text{ para } n \geq 2.$$

Demuestre que la función F de Fibonacci satisface las siguientes igualdades:

- $F(1) + F(3) + \cdots + F(2(n-1) + 1) = F(2n)$, para $n \geq 1$.
 - $F(0) + F(2) + \cdots + F(2n) = F(2n+1) - 1$, para $n \geq 0$.
9. Demuestre que la función F de Fibonacci satisface, para $n \in \mathbb{N}$, la siguiente igualdad:

$$F(0)^2 + F(1)^2 + \cdots + F(n)^2 = F(n)F(n+1).$$

10. Demuestre que la función F de Fibonacci satisface, para $n \geq 1$, la siguiente igualdad:

$$F(n)^2 - F(n+1)F(n-1) = (-1)^{n+1}.$$

11. Demuestre que la función F de Fibonacci, para $n \geq 1$, satisface

$$F(n) \geq \left(\frac{3}{2}\right)^{n-2}.$$

12. El producto entre dos matrices de dimensión 2×2 se define como:

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \times \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{pmatrix} = \begin{pmatrix} a_{00}b_{00} + a_{01}b_{10} & a_{00}b_{01} + a_{01}b_{11} \\ a_{10}b_{00} + a_{11}b_{10} & a_{10}b_{01} + a_{11}b_{11} \end{pmatrix}.$$

Dada una matriz A de 2×2 , se define A^n para $n \geq 1$ de la siguiente manera:

$$A^1 = A$$

$$A^{n+1} = A \times A^n, \quad n \geq 1.$$

Usando el principio de inducción matemática demuestre, para $n \geq 1$, la siguiente igualdad relacionada con la función F de Fibonacci:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{pmatrix}.$$

2.3. Una metodología

Con base en las ideas de la Sección 2.1 y en el ejemplo desarrollado en la Sección 2.2, se propone una metodología para diseñar algoritmos con dividir, conquistar y combinar a partir de la especificación de un problema. No debe entenderse como una receta infalible, sino como una lista de las preguntas que conviene responder y en qué orden.

La Nota 2.0.2 anunció que la dificultad de esta técnica está en la escogencia de la descomposición. En la lista que sigue, esa es la cuarta pregunta, y es la única que conviene responder varias veces antes de seguir adelante: ensayar dos o tres descomposiciones cuesta poco y decide todo lo demás.

Metodología 2.3.1: Dividir, conquistar y combinar

- **Especificación:** especificar el problema, indicando claramente su entrada y su salida.
- **Generalización:** proponer una función cuyos parámetros describan las instancias del problema y que incluya, como caso particular, la instancia que se desea resolver. Es importante que en este punto la función cuente

con una descripción clara en palabras y no con una definición recurrente. La razón es que esa descripción es la especificación de la función y el diseño que vendrá después es su implementación: tenerlas por separado es lo que permite demostrar que la segunda cumple la primera.

- **Reformulación de la especificación:** verificar que la salida del problema se obtiene de un llamado particular de la función propuesta. Si no es así, hay que volver al paso anterior y proponer otra.
- **Descomposición:** escoger una descomposición en el sentido de la Definición 2.1.1, es decir, cuáles instancias son base, en qué subinstancias se divide una instancia que no lo es y cuál es la medida que decrece. Es el paso que concentra la dificultad y conviene ensayar varias opciones antes de comprometerse con una.
- **Diseño de las tres fases:** para la descomposición escogida, decidir cómo se resuelven directamente las instancias base (conquistar), cómo se construyen las subinstancias (dividir) y cómo se obtiene la solución de la instancia a partir de las soluciones de sus subinstancias (combinar). Es común que dividir o combinar requiera una función auxiliar con especificación propia.
- **Correctitud:** demostrar que el diseño calcula lo que la descripción en palabras dice. Basta con verificar las cuatro condiciones del Teorema 2.1.1, de las cuales las tres primeras suelen ser inmediatas. La cuarta, la de la combinación, es la que exige un argumento propio y con frecuencia se enuncia como un lema sobre la función auxiliar del paso anterior.
- **Implementación:** escribir la función recurrente, que es la transcripción de las tres fases, junto con las funciones auxiliares que se hayan introducido.
- **Versión iterativa (opcional):** reescribir el algoritmo como uno o más ciclos, formular sus invariantes y demostrar que el diseño los satisface. Vale la pena cuando interesa eliminar el espacio que consume la pila de ejecución.
- **Análisis:** plantear la recurrencia del tiempo a partir de la estructura del algoritmo, sumando el costo de dividir, el de los llamados recurrentes y el de combinar, y resolverla con el Teorema Maestro del Capítulo 1 cuando corresponda a una función simple. Para el espacio, tener en cuenta tanto la memoria auxiliar que se reserve como la profundidad de la recurrencia.

2.3.1. Generalizar para poder recurrir. El paso de generalización merece un comentario, pues es el que más sorprende la primera vez. Un problema se especifica sobre una entrada fija, pero un algoritmo recurrente necesita invocarse sobre

instancias que no son la original. La función que se propone en ese paso es la que hace posible ese llamado, y casi siempre resuelve algo más general que el problema planteado.

La forma más frecuente de generalizar consiste en agregar parámetros que describan la subinstancia sobre la cual se trabaja. En los problemas sobre arreglos, esos parámetros son los extremos de un subarreglo: en lugar de una función que ordena $A[0..N)$, se propone una que ordena $A[low..hi)$ y la salida del problema se obtiene con $low = 0$ y $hi = N$. Esta es la generalización que usan las secciones siguientes.

Nota 2.3.1: Fortalecer la hipótesis

Hay una segunda forma de generalizar, menos evidente y más poderosa: resolver un problema *más difícil* que el original, de modo que la hipótesis inductiva entregue más información.

El diseño de la Sección 2.2 es un ejemplo. Al dividir un tablero de lado 2^{n+1} en cuatro subtableros de lado 2^n , solo uno de ellos tiene un hueco y los otros tres no satisfacen la hipótesis inductiva. La salida no es debilitar la hipótesis sino usarla a conveniencia: se ubica una ficha en las tres esquinas concéntricas, con lo cual los otros tres subtableros pasan a tener un hueco, y esa ficha es justamente la combinación.

Parece una paradoja que sea más fácil demostrar algo más fuerte, pero no lo es: al fortalecer el enunciado se fortalece también la hipótesis inductiva y, en un diseño recurrente, esa hipótesis es lo único con lo que se cuenta. Cuando un diseño se atasca porque las soluciones de las subinstancias no bastan para construir la de la instancia, conviene preguntarse qué información adicional haría falta y agregarla a la especificación de la función.

2.3.2. Escoger la descomposición. Los pasos de la descomposición y de las tres fases están ligados: la descomposición escogida determina qué tan costoso es dividir y qué tanto queda por combinar, que es el reparto del trabajo que describe la Sección 2.1.

Por eso, cuando el diseño admite varias descomposiciones, conviene plantear la recurrencia de cada una *antes* de implementar cualquiera. Es un cálculo corto, se hace con la estructura del algoritmo a la vista y evita descubrir tarde que la descomposición escogida no mejora al algoritmo evidente. El Ejercicio 2 de esta sección es, precisamente, un caso en el cual dividir por mitades no mejora nada.

2.3.3. La metodología en la teselación. Los pasos de la Metodología 2.3.1 se ilustrarán con cada uno de los ejemplos que se presentan en el resto del capítulo. De hecho, la Sección 2.2 ya siguió varios de ellos, aunque sin nombrarlos.

Paso	En la Sección 2.2
Especificación	Problema 2.2.1
Generalización	los tableros de lado 2^n con un hueco
Reformulación	el tablero de la entrada es uno de ellos
Descomposición	los cuatro subtableros de lado 2^n
Diseño de las tres fases	Algoritmo 2.2.1
Correctitud	la inducción del Algoritmo 2.2.1
Implementación	se propone como ejercicio
Versión iterativa	no se considera
Análisis	se propone como ejercicio

Dos filas de la tabla merecen un comentario.

La fila de generalización recoge lo que la Nota 2.3.1 describe. El Problema 2.2.1 pregunta si un tablero con un hueco se puede teselar, y el diseño responde algo más: exhibe una teselación. Además, al dividir, tres de los cuatro subtableros reciben un hueco que no tenían, precisamente para que la hipótesis inductiva sea aplicable.

La fila de correctitud es particular, por la razón que advierte la apertura de la Sección 2.2: allí el algoritmo y su demostración son una sola cosa. Es el caso excepcional, y por eso esa sección sirve para presentar la técnica pero no la metodología completa.

Ejercicios

1. Complete las dos filas que la tabla anterior deja pendientes para la Sección 2.2.
 - a) Implemente en Python una función que reciba n y la posición del hueco, y retorne la lista de las fichas de una teselación, cada una descrita por las tres celdas que ocupa.
 - b) Plantee la recurrencia de la cantidad de fichas que ubica su implementación y resuélvala. ¿Se puede usar el Teorema Maestro? Contraste el resultado con la cantidad de celdas del tablero.
2. Considere el problema de determinar si un valor x aparece en un arreglo $A[0..N)$, sin suponer nada sobre su orden. Siga los pasos de la Metodología 2.3.1 con una descomposición por mitades hasta el paso de análisis. ¿Mejora el resultado al algoritmo evidente que recorre el arreglo? ¿Qué le dice esto sobre la técnica?
3. Considere el problema de contar las ocurrencias de un valor x en un arreglo $A[0..N)$.
 - a) Siga los pasos de la Metodología 2.3.1 con una descomposición por decremento.
 - b) Repita el ejercicio con una descomposición por mitades.
 - c) Compare las dos recurrencias y las dos complejidades espaciales.
4. Considere el problema de determinar si un arreglo $A[0..N)$ está ordenado de manera ascendente.
 - a) Proponga una descomposición por mitades y explique por qué saber que cada mitad está ordenada no basta para concluir que el arreglo lo está.
 - b) Siguiendo la Nota 2.3.1, fortalezca la especificación de la función de modo que la combinación sea posible. ¿Qué información adicional debe retornar?
 - c) Implemente el algoritmo resultante y analícelo.
5. Un arreglo $A[0..N)$ de números, con $N \geq 1$, admite una *suma máxima de segmento*: el mayor valor de $\sum_{k=i}^{j-1} A[k]$ sobre todos los subarreglos no vacíos $A[i..j)$.
 - a) Proponga una descomposición por mitades. ¿Por qué no basta con las sumas máximas de las dos mitades?
 - b) 🌈 Fortalezca la especificación de la función, siguiendo la Nota 2.3.1, hasta que la combinación se pueda hacer en tiempo constante. Plantee la recurrencia resultante y resuélvala.
 - c) La Sección 3.4 resuelve este mismo problema con programación dinámica. Compare ambas soluciones en tiempo, en espacio y en la dificultad de sus demostraciones.

6. El paso de generalización de la Metodología 2.3.1 exige que la función propuesta incluya la instancia original como caso particular. Explique qué sale mal si se propone una función que no la incluye y dé un ejemplo.

2.4. Ordenamiento de arreglos

Ordenar un arreglo de números enteros es un problema fundamental en computación.

Problema 2.4.1: Ordenamiento de un arreglo de números

- Entrada:** Un arreglo $A[0..N)$, con $N \geq 0$, de números, cuyo valor inicial se denota A_0 .
- Salida:** El arreglo $A[0..N)$ modificado de modo que esté ordenado ascendentemente y sea una permutación de A_0 (i.e., $A[0..N)$ ordenado ascendentemente).

A pesar de la simplicidad de su planteamiento y de contar con soluciones muy intuitivas, es un problema que ha sido estudiado extensivamente. La principal razón, pareciera, es la dificultad para resolverlo eficientemente por medio de un algoritmo cuando no se cuenta con información adicional sobre el arreglo dado.

Esta sección lo resuelve dos veces con el mismo diseño: primero como un algoritmo recurrente y después como uno iterativo. Es la única sección del capítulo que recorre los nueve pasos de la Metodología 2.3.1, incluido el paso opcional, y por eso sirve también para ilustrar cómo se pasa de una forma a la otra.

2.4.1. Diseño recurrente. Siguiendo la Metodología 2.3.1, se generaliza el problema con una función que ordena $A[n..N)$ para $0 \leq n \leq N$, de modo que la salida se obtiene con $n = 0$. La descomposición escogida es por decremento: la instancia base es el subarreglo vacío, la única subinstancia de $A[n..N)$ es $A[n + 1..N)$ y la medida es $N - n$. Queda por decidir qué hacer antes de recurrir para que las dos soluciones se puedan juntar sin trabajo adicional y la respuesta es ubicar en $A[n]$ el menor elemento del subarreglo.

Algoritmo 2.4.1

1. Si $n = N$, entonces no se hace nada.
2. Si $n \neq N$, entonces hay al menos un elemento en $A[n..N)$:
 - a) identificar el índice $n \leq m < N$ con el mínimo valor en $A[n..N)$,

- b) intercambiar $A[n]$ y $A[m]$, y
- c) recurrir sobre $A[n+1..N]$.

Note que el caso $n = N$, identifica la situación en la cual $A[n..N]$ es vacío (y está ordenado). En el caso $n \neq N$ es posible que el arreglo no esté ordenado, por lo cual se ubica en $A[n]$ el elemento más pequeño en $A[n..N]$; se recurre ordenando $A[n+1..N]$. La intuición es que en este último caso, dado que $A[n]$ es el mínimo valor en $A[n..N]$ y $A[n+1..N]$ está ordenado ascendentemente, necesariamente $A[n..N]$ queda ordenado ascendentemente.

Implementación 2.4.1: Ordenamiento basado en el Algoritmo 2.4.1

```

1 def sort(A, n, N):
2     if n==N: pass
3     else:
4         m = n
5         for i in range(n+1, N):
6             if A[i] < A[m]: m = i
7         A[n],A[m] = A[m],A[n]
8         sort(A, n+1, N)

```

Ejemplo 2.4.1

La función `sort` se usa a continuación con algunos ejemplos:

```

1 A = [8, -10, -2, 3, -50, 2, 10, 8, 4 ]
2 sort(A, 0, len(A))
3 print(A)
4
5 A = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ]
6 sort(A, 0, len(A))
7 print(A)
8
9 A = [ ]
10 sort(A, 0, len(A))
11 print(A)
12
13 A = [ 5 ]
14 sort(A, 0, len(A))
15 print(A)
16

```

El resultado de esta ejecución es el siguiente:

```

1 [-50, -10, -2, 2, 3, 4, 8, 8, 10]
2 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
3 []
4 [5]
```

Se han mostrado ejemplos que sugieren indicar que funciona bien para algunas instancias del problema, mas no una demostración de que el algoritmo funciona correctamente (i.e., resuelve adecuadamente todas las instancias del problema). Para ello es necesario demostrar que en realidad el código hace lo que debe hacer, es decir, ordenar un arreglo ascendentemente. Dicho de otra forma: se desea demostrar que `sort` ordena ascendentemente cualquier arreglo de números.

2.4.2. Correctitud.

Teorema 2.4.1: Correctitud de `sort`

Para $N = \text{len}(A)$ y $0 \leq n \leq N$, el llamado `sort(A, n, N)` ordena ascendentemente el arreglo $A[n..N)$. En particular, `sort(A, 0, N)` ordena ascendentemente a $A[0..N)$.

Demostración

Se procede por inducción sobre n :

- Caso base ($n = N$): en este caso, el algoritmo no hace ninguna operación. Note que $A[n..N)$ es el arreglo vacío, el cual por definición está ordenado (pues no tiene un par de elementos en desorden).
- Caso inductivo ($n \neq N$): en este caso, n es un índice dentro del arreglo y $A[n]$ representa el valor en esa posición. El ciclo **for** y la asignación posterior al ciclo ubican en $A[n]$ el mínimo valor en $A[n..N)$. Por la hipótesis inductiva, $A[n+1..N)$ está ordenado ascendentemente (este es el llamado recurrente de `sort`). Como ningún elemento en $A[n+1..N)$ es menor que $A[n]$ y $A[n+1..N)$ está ordenado ascendentemente, necesariamente $A[n..N)$ está ordenado ascendentemente.

Finalmente, como el resultado es cierto para cualquier n que satisface $0 \leq n \leq N$, en particular vale para $n = 0$ y de esa forma se tiene que `sort(A, 0, N)` ordena ascendentemente a $A[0..N)$.

2.4.3. Complejidad. La complejidad temporal del algoritmo `sort` se establece asintóticamente a continuación.

Teorema 2.4.2: Complejidad temporal de sort

Para $N = \text{len}(A)$ y $0 \leq n \leq N$, el llamado `sort(A, n, N)` toma tiempo $O((N - n)^2)$. En particular, el llamado `sort(A, 0, N)` toma tiempo $O(N^2)$.

Demostración

Sin pérdida de generalidad, se supondrá que las operaciones aritméticas, de asignación y de control toman exactamente una unidad de tiempo. Además, se define $k = N - n$ para simplificar la manipulación aritmética en los cálculos. Note que en cada llamado recurrente se realizan $N - n = k$ iteraciones del ciclo `for` y cada una de ellas toma tiempo constante. Entonces, la cantidad de operaciones está dada por:

$$\begin{aligned} & (N - n) + (N - (n + 1)) + \cdots + (N - (N - 1)) + (N - N) \\ &= k + (k - 1) + \cdots + 1 + 0 \\ &= \frac{k(k + 1)}{2} \\ &= \frac{(N - n)(N - n + 1)}{2} \in O((N - n)^2). \end{aligned}$$

En particular, para $n = 0$, se tiene que la cantidad de operaciones está dada por $O(N^2)$.

La complejidad espacial se debe calcular con cuidado, en especial bajo la ausencia de información acerca de detalles de implementación de la recurrencia. A continuación, se presenta un estimado que supone que la creación del ambiente de ejecución de cada uno de los llamados recurrentes es de orden constante.

Teorema 2.4.3: Complejidad espacial de sort

Para $N = \text{len}(A)$ y $0 \leq n \leq N$, el llamado `sort(A, n, N)` usa espacio $O(N - n)$. En particular, el llamado `sort(A, 0, N)` usa espacio $O(N)$.

Demostración

En cada llamado recurrente de `sort` se crean a lo sumo dos variables, llamadas `m` e `i`. Como hay a lo sumo $N - n$ llamados recurrentes en donde se crean estas dos variables (únicamente se evita su creación en el caso base), la complejidad

espacial de $\text{sort}(A, n, N)$ es $O(N - n)$. Luego, la complejidad espacial de $\text{sort}(A, 0, N)$ es $O(N)$.

Es importante aclarar que en el Teorema 2.4.3 se está suponiendo que las variables que se crean en cada llamado recurrente son independientes entre los llamados. Sin embargo, si se pudieran reutilizar de tal forma que fuera una cantidad constante para todos los llamados recurrentes, la complejidad espacial del algoritmo sería de orden constante.

2.4.4. Una versión iterativa. Esa última observación sugiere escribir el algoritmo sin llamados recurrentes, que es el paso opcional de la Metodología 2.3.1. Para ello se usa la noción de invariante de la Nota 2.1.3, que es la que guía el diseño de un ciclo.

Se utilizarán los siguientes invariantes para representar el avance del algoritmo hacia el ordenamiento del arreglo dado. Para evitar confusiones porque el ordenamiento se hará en el mismo arreglo (i.e., es un ordenamiento in-situ) se identificará con $\bar{A}$ al estado inicial del arreglo, antes de que el algoritmo a ser diseñado lo manipule para lograr su ordenamiento. Es decir, el estado inicial del arreglo A se llama $\bar{A}$.

P_0 : $A[0..N)$ es una permutación de $\bar{A}[0..N)$.

P_1 : $A[0..n)$ tiene los n elementos de $\bar{A}[0..N)$ más pequeños y ordenados ascendentemente.

P_2 : $0 \leq n \leq N$.

Hay tres invariantes, llamados P_0, P_1, P_2 . El invariante P_0 indica que los elementos de A son los mismos que los de $\bar{A}$, posiblemente en diferente orden. El invariante P_1 indica no solo que $A[0..N)$ está parcialmente ordenado en las posiciones $0..n-1$, sino que los elementos allí corresponden a los más pequeños en $\bar{A}$. El invariante P_2 indica que el valor de n varía entre 0 y N , inclusive.

Antes de continuar con el diseño de los casos del algoritmo, es clave pensar en cuál sería la situación ideal que –con base en los invariantes dados– garantizará que el arreglo A está ordenado. En este sentido, el invariante P_1 es clave, pues cuando n sea N (lo cual es permitido por el invariante P_2), se tendría la siguiente situación:

$A[0..N)$ tiene los n elementos de $\bar{A}[0..N)$ más pequeños y ordenados ascendentemente.

Es decir, $A[0..N)$ tendría sus elementos ordenados ascendentemente; no faltaría ni sobraría elemento alguno gracias al invariante P_0 . De esta forma, se sugiere que la estrategia es ir avanzando n , que puede iniciar desde 0, hasta que su valor sea N , mientras se mantienen los invariantes formulados. Una representación gráfica de

los invariantes P_1 y P_2 , y de esta idea de solución algorítmica, se muestra en la Figura 2.

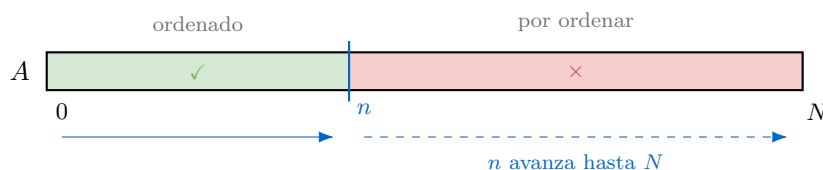


Figura 2. Idea de solución algorítmica para el ordenamiento.

Con base en los invariantes y en la idea gráfica de diseño, se propone el Algoritmo 2.4.2 para ordenar iterativa y ascendentemente un arreglo de números.

Algoritmo 2.4.2

Para $n = 0, 1, \dots, N - 1$:

1. identificar el índice $n \leq m < N$ con el mínimo valor en $A[n..N)$ e
2. intercambiar $A[n]$ y $A[m]$.

La propuesta del Algoritmo 2.4.2 es similar a la del Algoritmo 2.4.1 recurrente. La gran diferencia es que después del paso (2) no se recurre sino que se continúa con las iteraciones del ciclo. Este diseño resulta en la siguiente implementación.

Implementación 2.4.2: Ordenamiento basado en el Algoritmo 2.4.2

```

1 def sortiter(A):
2     N = len(A)
3     #  $P_0 \wedge P_1 \wedge P_2$ 
4     for n in range(0, N):
5         m = n
6         for i in range(n+1, N):
7             if A[i] < A[m]: m = i
8         A[n], A[m] = A[m], A[n]
```

Note que el ciclo interno, el que selecciona el valor mínimo en $A[n..N)$, es exactamente el mismo de la Implementación 2.4.1. Lo único que cambia entre las dos versiones es qué ocupa el lugar del llamado recurrente: aquí, la siguiente iteración del ciclo externo.

2.4.5. Correctitud del ciclo. La demostración de que un algoritmo iterativo es correcto está directamente ligada a la noción de invariante, en los términos que describe la Nota 2.1.4.

A manera de ejemplo de cómo establecer la correctitud del ciclo principal de la Implementación 2.4.2 con respecto a sus invariantes, se presentan demostraciones de iniciación y estabilidad. Además, se muestra cómo los invariantes ayudan a determinar qué logra la ejecución del algoritmo. Dado que hay un ciclo anidado dentro del ciclo principal, se supondrá que dicho ciclo cumple su objetivo: calcular en la variable m el índice del valor mínimo en $A[n..N]$.

Teorema 2.4.4

Los invariantes P_0, P_1, P_2 son ciertos antes de la ejecución del ciclo (principal) de la Implementación 2.4.2.

Demostración

Antes de la primera iteración del ciclo, las variables n y N tienen los valores 0 y $\text{len}(A)$, respectivamente. Note que:

- Dado que el arreglo A no ha sido modificado, claramente P_0 es cierto.
- Como $n = 0$, el arreglo $A[0..n)$ es vacío y por tanto no tiene elementos. Esto concuerda con el invariante P_1 .
- Con $n = 0$ y $N = \text{len}(A)$ se establecen trivialmente las desigualdades $0 \leq n \leq N$.

Teorema 2.4.5

Si los invariantes P_0, P_1, P_2 son ciertos y se itera una vez el ciclo (principal) de la Implementación 2.4.2, entonces P_0, P_1, P_2 siguen siendo ciertos.

Demostración

Lo que cambia en una iteración del ciclo son la variable n y (posiblemente) el arreglo $A[n..N)$. Para proceder, se supone que P_0, P_1, P_2 son ciertos antes de que estos cambios surtan efecto. Además, como es necesario que la condición $0 \leq n < N$ (esta es la guarda del ciclo) se cumpla para que dicha iteración se dé, se supondrá también que es cierta. Observe:

- Como $A[0..N)$ es una permutación de $\bar{A}[0..N)$ (suposición de que P_0 es cierto antes de la iteración) y como lo único que afecta a A en una iteración es el intercambio $A[n], A[m] = A[m], A[n]$, el invariante P_0 es cierto después de una iteración del ciclo.
- Por P_1 , se tiene que $A[0..n)$ está ordenado ascendentemente con los n elementos más pequeños de $\bar{A}[0..N)$. En una iteración, $A[n]$ es actualizado con el elemento más pequeño de $A[n..N)$. Luego, $A[0..n+1)$ está ordenado ascendentemente con los $n+1$ elementos más pequeños de $\bar{A}[0..N)$ (este es el invariante P_1 después de una iteración del ciclo).
- Como $0 \leq n < N$ y n aumenta en uno con una iteración del ciclo, entonces claramente $0 \leq n+1 \leq N$, lo cual corresponde al invariante P_2 después de incrementar n .

Finalmente, note que el ciclo termina cuando $n = N$. Como el ciclo es correcto con respecto a los invariantes P_0, P_1, P_2 , se tiene que $A[0..N)$ tiene los N elementos de $\bar{A}[0..N)$ más pequeños y ordenados. Es decir, `sortiter(A)` ordena ascendentemente el arreglo A . El análisis de las complejidades temporal y espacial, al igual que la corrección del ciclo interno de la Implementación 2.4.2, se proponen como ejercicio al lector.

Ejercicios

1. El algoritmo de ordenamiento diseñado e implementado en esta sección es muy conocido y tiene un nombre. ¿Cuál es?
2. Demuestre la siguiente versión, más general, del teorema de corrección de la función `sort`

Teorema. Para $N = \text{len}(A)$ y $0 \leq n \leq m \leq N$, el llamado de `sort` con argumentos A, n y m ordena ascendentemente el arreglo $A[n..m)$.
3. Se propone el siguiente algoritmo para ordenar un arreglo $A[0..N)$ de números: mientras que $A[0..N)$ no esté ordenado, permutarlo.
 - a) ¿Puede estimar la complejidad temporal de este algoritmo?
 - b) ¿Puede demostrar que funciona?

Si es necesario contar con algunos supuestos, indique cuáles son. Justifique sus respuestas.
4. Considere la siguiente variación del Algoritmo 2.4.1:
 - Si $n = N$, entonces no se hace nada.
 - Si $n = N - 1$, entonces no se hace nada.
 - Si $n \neq N$, entonces hay al menos un elemento en $A[n..N)$:
 - identificar el índice $n \leq m < N$ con el mínimo valor en $A[n..N)$,

- intercambiar $A[n]$ y $A[m]$, y
- recurrir sobre $A[n+1..N]$.

Con base en este algoritmo:

- a) Modifique la función `sort` en la Implementación 2.4.1 para que considere el segundo caso base planteado anteriormente.
 - b) Formule el teorema de corrección y demuestre que es correcto para esta nueva versión del algoritmo.
 - c) ¿Cuál es la complejidad temporal y espacial del algoritmo? Justifique su respuesta.
5. En el caso recurrente del Algoritmo 2.4.1, para ordenar el arreglo $A[n..N]$ se usa la estrategia de identificar el índice $n \leq m < N$ con el mínimo valor e intercambiarlo con $A[n]$ antes de recurrir sobre $A[n+1..N]$. Considere las dos variaciones siguientes:
- a) una estrategia que identifique valores *máximos* y no mínimos; y
 - b) el problema de ordenar *descendentemente* un arreglo de números, que debe especificarse primero.
- Para cada una de ellas, diseñe un algoritmo recurrente que use dividir, conquistar y combinar, impleméntelo en el lenguaje de programación Python, enuncie y demuestre su teorema de corrección, y calcule su complejidad temporal.
6. Complete el análisis de la versión iterativa.
- a) Demuestre que el ciclo interno de la Implementación 2.4.2 termina.
 - b) Proponga invariantes para ese ciclo interno y demuestre que su diseño es correcto.
 - c) Determine las complejidades temporal y espacial de `sortiter` y compárelas con las de `sort`.

7. Considere la siguiente función `sortitergen`:

```

1 def sortitergen(A, low, hi):
2     for n in range(low, hi):
3         m = n
4         for i in range(n+1, hi):
5             if A[i] < A[m]: m = i
6         A[n], A[m] = A[m], A[n]
```

Esta es una versión generalizada de la función `sortiter` en la Implementación 2.4.2 que –dados $A[0..N]$, low , hi , con $0 \leq low \leq hi \leq N$ – ordena ascendentemente el subarreglo $A[low..hi]$.

Proponga invariantes para el ciclo externo y demuestre la correctitud de la función (suponiendo que el ciclo interno es correcto). Compare esta generalización con la del paso correspondiente de la Metodología 2.3.1.

8. Considere el problema del ordenamiento de un arreglo de números. Suponga que únicamente se consideran arreglos cuyos valores están entre 0 y 10000.

- a) Especifique el problema.
 - b) Diseñe un algoritmo que resuelva el problema dado, y cuyas complejidades temporal y espacial sean $O(N)$, en donde N es el tamaño del arreglo.
9. Considere la siguiente colección de problemas algorítmicos sobre un arreglo de números:
- a) Calcular la suma de los elementos del arreglo.
 - b) Determinar el mínimo valor en el arreglo.
 - c) Determinar el máximo valor en el arreglo.
 - d) Calcular la cantidad de apariciones de un número dado en el arreglo.
- Para cada uno de estos problemas: especifique el problema dado; diseñe un algoritmo recurrente con dividir, conquistar y combinar y otro iterativo con sus invariantes; demuestre que ambos son correctos con respecto a la especificación; calcule sus complejidades temporales; e impleméntelos en el lenguaje de programación Python.
10. La siguiente es una enumeración de algoritmos de ordenamiento iterativos ampliamente conocidos:
- a) *BubbleSort*
 - b) *InsertionSort*
 - c) *ShellSort*

Para cada uno de estos algoritmos en el caso de ordenamiento de un arreglo de números:

- a) Plasme el diseño del algoritmo al estilo del Algoritmo 2.4.2.
 - b) Proponga invariantes para los ciclos que materializan los algoritmos iterativos.
 - c) Demuestre que son correctos con respecto a la especificación propuesta.
 - d) Determine la complejidad temporal y espacial del algoritmo.
-

2.5. MergeSort: un ordenamiento de arreglos más eficiente

La idea principal detrás del algoritmo *MergeSort* no es muy distinta a la de los algoritmos de ordenamiento presentados en la Sección 2.4, pues su diseño se puede también explicar con base en dividir, conquistar y combinar. La principal característica de *MergeSort* es que usa la hipótesis inductiva dos veces en cada paso del “combinar”: el subarreglo se divide en dos “mitades” que, siendo problemas más pequeños que el original, se puede suponer que están ordenadas ascendentemente (i.e., se hace un llamado recurrente para cada una de ellas), para posteriormente combinarlas a modo de cremallera, manteniendo el orden de los elementos al ser consolidados en el arreglo original. Esta idea general se presenta en la Figura 3.

2.5.1. Diseño. Siguiendo la Metodología 2.3.1, la generalización es la misma de la Sección 2.4, pero con dos parámetros: una función que ordena $A[low..hi]$, de modo que el problema se resuelve con $low = 0$ y $hi = N$. La descomposición, en cambio, es distinta: las subinstancias son las dos mitades del subarreglo, la medida es su longitud $hi - low$ y las instancias base son los subarreglos con a lo sumo un elemento. Como dividir no cuesta nada, todo el trabajo queda para la combinación.

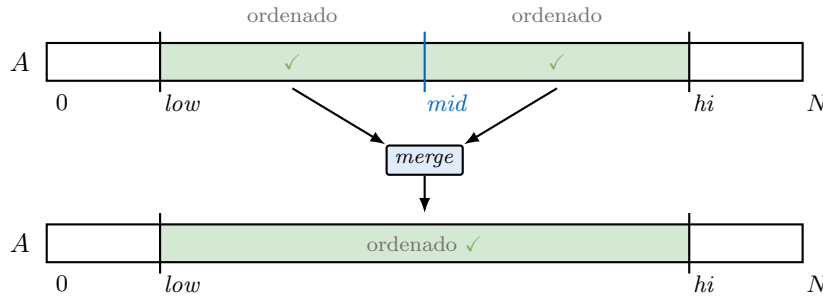


Figura 3. Idea general del diseño de *MergeSort*.

A cada momento, el algoritmo mantiene un “cerco” para el ordenamiento de un subarreglo de A con base en las variables low y hi : el objetivo es ordenar $A[low..hi]$. Si esto es posible para cualesquiera low y hi , en particular se tiene resuelto el ordenamiento de todo el arreglo $A[0..N]$.

Antes de continuar explicando la imagen anterior que resume la idea de *MergeSort*, se introducen las siguientes condiciones:

C_0 : $A[0..N]$ es una permutación de $\bar{A}[0..N]$.

C_1 : $0 \leq low \leq hi \leq N$.

Note que la idea de ordenar el arreglo dado *in situ* se captura con la condición C_0 , donde $\bar{A}$ denota el estado inicial del arreglo, como en la Sección 2.4. La condición C_1 brinda pistas de cómo se puede avanzar con low y hi ordenando por partes:

- Si $A[low..hi]$ tiene a lo sumo un elemento, entonces no hay mucho que hacer pues el arreglo vacío y cualquier arreglo unitario están ordenados por definición. Estos casos suceden cuando $low + 1 \geq hi$.
- Si $A[low..hi]$ tiene al menos dos elementos, entonces se puede “partir” en dos mitades (aproximadas) $A[low..mid]$ y $A[mid..hi]$ ordenando cada una de ellas recurrentemente y posteriormente combinándolas. Estos casos suceden cuando $low + 1 < hi$.

2.5.2. La función merge. La idea principal detrás de la función `mergesort` en la Implementación 2.5.1 debería estar clara en este punto. Por ende, el foco estará en entender cómo la función `merge` combina las dos partes del arreglo que están ordenadas garantizando el ordenamiento general del arreglo $A[low..hi]$.

Implementación 2.5.1: MergeSort en Python

```

1 def mergesort(A, low, hi):
2     if low+1<hi:
3         mid = low+((hi-low)>>1) # mid = (low+hi)//2
4         mergesort(A, low, mid) # induction hypothesis on the first half
5         mergesort(A, mid, hi)  # induction hypothesis on the second half
6         merge(A, low, mid, hi) # combine the two halves preserving the order
7
8 def merge(A, low, mid, hi):
9     global tmp # a global array at least the size of A
10    for i in range(low, hi): tmp[i] = A[i] # copy A[low..hi) to tmp[low..hi)
11    l,r = low,mid
12    for n in range(low, hi):
13        if l==mid: A[n],r = tmp[r],r+1 # only process the right half
14        elif r==hi: A[n],l = tmp[l],l+1 # only process the left half
15        else:
16            # the first pending element of each half needs to be compared
17            if tmp[l]<=tmp[r]: A[n],l = tmp[l],l+1 # choose the one on the left
18            else: A[n],r = tmp[r],r+1 # choose the one on the right

```

Para entender cómo encajan las piezas del algoritmo, se proponen los siguientes invariantes para el segundo ciclo de la función `merge`, en el sentido de la Nota 2.1.3:

- P_0 : $A[low..n]$ está ordenado y contiene exactamente los elementos de $tmp[low..l)$ y $tmp[mid..r)$; y ninguno de los pendientes, los de $tmp[l..mid)$ y $tmp[r..hi)$, es menor que ellos.
- P_1 : $low \leq l \leq mid \leq r \leq hi$.
- P_2 : $low \leq n \leq hi$.
- P_3 : $n - low = (l - low) + (r - mid)$.

El invariante P_0 indica que en $A[low..n]$ se van almacenando, en orden, los elementos de $tmp[low..l)$ y $tmp[mid..r)$ que han sido procesados, y que lo que falta por procesar no altera ese orden. Los invariantes P_1 y P_2 indican entre qué rangos pueden tomar valores las variables l , r y n . El invariante P_3 relaciona las tres: cada elemento ubicado en A proviene de exactamente una de las dos mitades.

Intuitivamente, las variables l y r pueden entenderse como dos (dedos) índices que señalan el siguiente elemento de la mitad izquierda y de la derecha, respectivamente, que debe ser procesado. Cuando $l = mid$ se tiene que la mitad izquierda ha sido agotada; de la misma forma, cuando $r = hi$ se tiene que la mitad derecha ha sido agotada. En cualquiera de estos dos casos no hay que comparar nada y basta con copiar a A los elementos pendientes en la mitad correspondiente. Si hay al menos un elemento en cada una de las mitades, entonces es necesario compararlos para determinar cuál de los dos se copiará a $A[n]$: el menor de los dos o, en caso de igualdad, el de la mitad izquierda. La Figura 4 resume visualmente el rol de cada una de las variables y de los arreglos involucrados en `merge`.

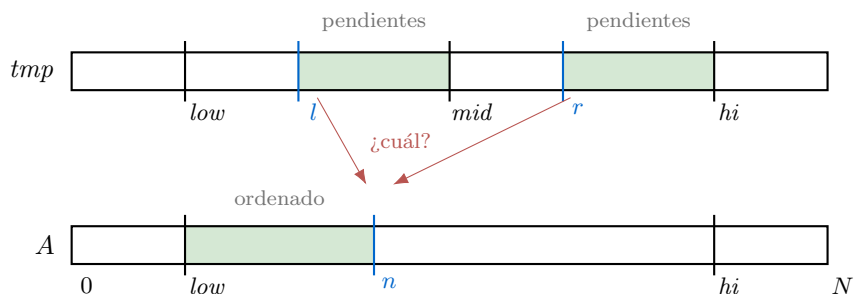


Figura 4. El rol de las variables y de los arreglos involucrados en `merge`.

Antes de proceder a demostrar que `merge` y `mergesort` son correctos con respecto a su especificación, se presentan ejemplos de cómo el algoritmo ordena algunos arreglos de números.

Ejemplo 2.5.1

La función `mergesort` se usa a continuación con algunos ejemplos:

```

1 tmp = [ None for _ in range(10) ]
2 A = [ 8, -10, -2, 3, -50, 2, 10, 8, 4 ]
3 mergesort(A, 0, len(A))
4 print(A)
5
6 A = [ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ]
7 mergesort(A, 0, len(A))
8 print(A)
9
```

```

10 A = [ ]
11 mergesort(A, 0, len(A))
12 print(A)
13
14 A = [ 5 ]
15 mergesort(A, 0, len(A))
16 print(A)

```

El resultado de esta ejecución es el siguiente:

```

1 [-50, -10, -2, 2, 3, 4, 8, 8, 10]
2 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
3 []
4 [5]

```

2.5.3. Correctitud. A continuación se presenta y demuestra el teorema de correctitud de `merge`, que es la pieza sobre la cual descansa la de `mergesort`.

Teorema 2.5.1: Correctitud de `merge`

Sean $0 \leq low \leq mid \leq hi \leq N$ y suponga que $A[low..mid]$ y $A[mid..hi]$ están ordenados ascendentemente. Entonces el llamado `merge(A, low, mid, hi)` termina y deja en $A[low..hi]$ una permutación de su contenido inicial, ordenada ascendentemente; el resto de A no se modifica.

Demostración

El primer ciclo copia $A[low..hi]$ en $tmp[low..hi]$, de modo que al comenzar el segundo ciclo los subarreglos $tmp[low..mid]$ y $tmp[mid..hi]$ son copias de las dos mitades y, por hipótesis, están ordenados ascendentemente. La demostración para el segundo ciclo sigue el esquema de la Nota 2.1.4.

Iniciación. Antes de la primera iteración se tiene $n = low$, $l = low$ y $r = mid$. El subarreglo $A[low..n]$ es vacío, así que P_0 se satisface trivialmente; P_1 y P_2 se siguen de $low \leq mid \leq hi$; y P_3 se reduce a $0 = 0 + 0$.

Estabilidad. Suponga que P_0 a P_3 son ciertos al comenzar una iteración, de modo que $n < hi$. Por P_3 ,

$$(l - low) + (r - mid) = n - low < hi - low = (mid - low) + (hi - mid),$$

así que, por P_1 , no puede ocurrir simultáneamente $l = mid$ y $r = hi$: hay al menos un elemento pendiente. Sea x el valor que la iteración ubica en $A[n]$.

Hay tres casos:

- Si $l = mid$, la mitad izquierda está agotada y, por lo anterior, $r < hi$. Se toma $x = tmp[r]$ y se incrementa r .
- Si $r = hi$, la mitad derecha está agotada y, por lo anterior, $l < mid$. Se toma $x = tmp[l]$ y se incrementa l .
- De lo contrario hay elementos pendientes en ambas mitades y se toma $x = tmp[l]$ si $tmp[l] \leq tmp[r]$, incrementando l , o $x = tmp[r]$ en caso contrario, incrementando r . En ambos casos x es el menor de los dos elementos pendientes que encabezan las mitades.

En los tres casos x es el menor de los elementos pendientes: en los dos primeros porque la mitad que sobrevive está ordenada, y en el tercero porque cada mitad está ordenada y se comparan sus dos primeros elementos pendientes. Por P_0 , ningún elemento pendiente es menor que los ya ubicados, de modo que x no es menor que ellos y $A[low..n+1)$ queda ordenado ascendentemente. Además, los elementos pendientes después de la iteración son los de antes menos x , y cada uno de ellos es mayor o igual que x por lo recién dicho; por lo tanto P_0 se mantiene con los nuevos valores de n , l y r . El invariante P_1 se mantiene porque el índice que se incrementa no sobrepasa su cota, y P_2 porque $n+1 \leq hi$. Finalmente, P_3 se mantiene porque n y exactamente uno de l o r se incrementan en uno.

Terminación. El segundo ciclo realiza exactamente $hi - low$ iteraciones, pues n recorre ese rango.

Conclusión. Al terminar se tiene $n = hi$, así que P_3 da

$$(l - low) + (r - mid) = hi - low = (mid - low) + (hi - mid).$$

Como P_1 garantiza $l \leq mid$ y $r \leq hi$, ninguno de los dos sumandos puede quedarse corto y, por lo tanto, $l = mid$ y $r = hi$. Con esto, P_0 dice que $A[low..hi)$ está ordenado ascendentemente y contiene exactamente los elementos de $tmp[low..mid)$ y $tmp[mid..hi)$, que son los que tenía $A[low..hi)$ al comenzar el llamado. Las posiciones fuera de ese rango nunca se escriben.

Con base en la correctitud (o corrección) de `merge`, se puede establecer la correctitud de `mergesort`.

Teorema 2.5.2: Correctitud de mergesort

Para $N = \text{len}(A)$ y $0 \leq low \leq hi \leq N$, el llamado `mergesort(A, low, hi)` ordena ascendentemente $A[low..hi)$.

Demostración

Se procede por inducción sobre el tamaño de $A[low..hi]$:

- Caso base ($low + 1 \geq hi$): en este caso, `mergesort` no modifica a A . Note que cuando $low + 1 \geq hi$, $A[low..hi]$ es vacío o tiene exactamente un elemento, por lo cual está ordenado ascendentemente.
- Caso inductivo ($low + 1 < hi$): en este caso, el problema se divide en dos subproblemas: ordenar $A[low..mid]$ y $A[mid..hi]$. Bajo la condición dada, el código garantiza que se cumple la desigualdad $low < mid < hi$ (esto se debe demostrar y se propone como ejercicio para el lector). Esto garantiza que cada uno de los subarreglos tiene al menos un elemento y ambos tienen menor tamaño que $A[low..hi]$: como son más pequeños, por hipótesis inductiva, se puede suponer que después de los llamados recurrentes `mergesort(A, low, mid)` y `mergesort(A, mid, hi)` ordenan $A[low..mid]$ y $A[mid..hi]$, respectivamente. Dado que `merge` es correcto (Teorema 2.5.1), el llamado `merge(A, low, mid, hi)` ordena ascendentemente a $A[low..hi]$.

En cualquiera de los dos casos, $A[low..hi]$ resulta ordenado ascendentemente por `mergesort(A, low, hi)`.

2.5.4. Complejidad. La ventaja que tiene el algoritmo `mergesort` con respecto al primer algoritmo de ordenamiento estudiado, es que su comportamiento asintótico en función de la cantidad de operaciones que lleva a cabo para ordenar un arreglo es mejor (i.e., crece menos rápido).

Teorema 2.5.3: Complejidad temporal de mergesort

Para $N = \text{len}(A)$ y $0 \leq low \leq hi \leq N$, el llamado `mergesort(A, low, hi)` toma tiempo $O((hi - low) \log(hi - low))$. En consecuencia, `mergesort(A, 0, N)` toma tiempo $O(N \log N)$.

Demostración

Sea $n = hi - low$ (note que $n \geq 0$). A modo de ejercicio para el lector, se puede establecer que `merge(A, low, mid, hi)` toma tiempo $O(hi - low) = O(n)$ (esto es lo que cuesta combinar las soluciones a los subproblemas de `mergesort`). Entonces, la ecuación de recurrencia que define la cantidad de

operaciones básicas está dada por

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

pues se recurre una vez en cada una de las dos mitades y cuesta $O(n)$ combinar estas soluciones. De acuerdo con el Teorema Maestro del Capítulo 1, se tiene que

$$T(n) \in O(n \log n).$$

Note que el llamado `mergesort(A, 0, N)` corresponde a $low = 0$ y $hi = N$, de modo que $n = N$ y, consecuentemente, $T(N) \in O(N \log N)$.

Finalmente, se analiza la complejidad espacial de `mergesort`.

Teorema 2.5.4: Complejidad espacial de mergesort

Para $N = \text{len}(A)$ y suponiendo que la variable `tmp` es suministrada para `merge`, el llamado `mergesort(A, 0, N)` usa $O(\lceil \log N \rceil)$ espacio, en donde $\lceil \cdot \rceil$ es la función techo.

Demostración

Se propone como ejercicio para el lector.

El Teorema 2.5.4 cuenta únicamente el espacio de la pila de ejecución. Si se cuenta también el arreglo `tmp`, que tiene el tamaño de A , el espacio total es $O(N)$. Esa memoria auxiliar es el precio que `mergesort` paga por combinar, y es la diferencia con el algoritmo de la sección siguiente, que ordena sin memoria adicional.

Ejercicios

1. La demostración del Teorema 2.5.1 usa el invariante P_3 para concluir que, al terminar el segundo ciclo de `merge`, se tiene $l = mid$ y $r = hi$.
 - a) Verifique en detalle que P_3 es un invariante del ciclo, revisando los tres casos de una iteración.
 - b) ¿Qué se podría concluir al terminar el ciclo si solo se contara con P_0 , P_1 y P_2 ?
 2. Proponga invariantes para el primer ciclo de `merge`, el que copia $A[low..hi]$ en $tmp[low..hi]$, y demuestre que cumple su objetivo.
 3. Demuestre que la desigualdad $low < mid < hi$ es cierta en `mergesort`.
 4. Demuestre que el llamado `merge(A, low, mid, hi)` toma tiempo $O(hi - low)$ y complete con ello la demostración del Teorema 2.5.3.
 5. Demuestre el Teorema 2.5.4.
 6. Considere la función `merge` que es llamada desde la función `mergesort` en la Implementación 2.5.1. Allí se usa la variable global `tmp` para almacenar una copia de partes de A en cada llamado. Suponiendo que dicha variable no es global y que en cada llamado recurrente se crea un arreglo de tamaño $hi - low$ para la copia temporal, demuestre que `mergesort(A, 0, N)` usa espacio $O(N)$. Justifique su respuesta.
 7. La cantidad de inversiones de un arreglo es un indicador de qué tan desordenado está: si este valor es 0, entonces el arreglo está completamente ordenado. Si el arreglo está ordenado descendientemente, entonces este valor es el máximo. Formalmente, una *inversión* en un arreglo $A[0..N)$, $N \geq 0$, es una pareja de índices $0 \leq i < j < N$ tales que $A[i] > A[j]$. Por ejemplo, en el arreglo $[3, 1, 2]$ hay dos inversiones: $(0, 1)$ y $(0, 2)$.
 - a) Diseñe un algoritmo que calcule la cantidad de inversiones en $A[0..N)$ en tiempo $O(N \log N)$.
 - b) Demuestre que el algoritmo propuesto es correcto con respecto a la especificación.
-

2.6. QuickSort: ordenar sin combinar

El algoritmo *MergeSort* de la Sección 2.5 divide sin esfuerzo, pues basta con calcular *mid*, y deja todo el trabajo para la combinación, que hace la función `merge`. La pregunta natural es si el reparto se puede invertir: hacer el trabajo *antes* de recurrir, de modo que al regresar de los llamados recurrentes no quede nada por

combinar. El algoritmo *QuickSort*, propuesto por C. A. R. Hoare [Hoa62], responde que sí.

La idea es escoger un elemento del subarreglo, llamado el *pivote*, y reorganizar $A[low..hi]$ de manera que el pivote quede en una posición p con todos los elementos menores que él a su izquierda y todos los que no lo son a su derecha. Una vez hecho esto, el pivote ya está en la posición que le corresponde en el arreglo ordenado y solo falta ordenar, por separado, lo que quedó a cada lado. La idea general del diseño se presenta en la Figura 5.

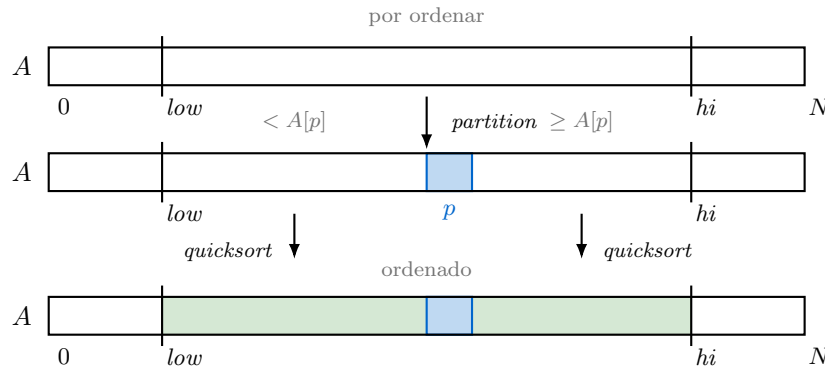


Figura 5. Idea general del diseño de *QuickSort*.

Este es el punto en el cual se aprecia la diferencia con *MergeSort*: no hay una fase de combinación. Los dos subarreglos que se ordenan recurrentemente están separados por el pivote y ya ocupan las posiciones que les corresponden, así que yuxtaponerlos no requiere ninguna operación. Dicho en los términos de la Sección 2.1, el trabajo de este algoritmo está en la división.

2.6.1. Diseño. Se sigue la Metodología 2.3.1. La especificación es la del Problema 2.4.1 y la generalización es la misma de *MergeSort*: una función *quicksort* que recibe A , low y hi , y ordena ascendentemente el subarreglo $A[low..hi]$. El problema se resuelve entonces con el llamado *quicksort*(A , 0 , N).

La descomposición es la que introduce la novedad. Las instancias base son los subarreglos con a lo sumo un elemento, es decir, aquellos con $low + 1 \geq hi$, que están ordenados por definición. Para las demás, se supone disponible una función *partition*(A , low , hi) con la siguiente especificación en palabras: reorganiza $A[low..hi]$ y retorna una posición p , con $low \leq p < hi$, tal que ningún elemento de $A[low..p]$ es mayor o igual que $A[p]$ y ningún elemento de $A[p+1..hi]$ es menor que $A[p]$. Las subinstancias son entonces $A[low..p]$ y $A[p+1..hi]$, y la medida es,

como siempre, la longitud del subarreglo. Como p pertenece al subarreglo, ambas subinstancias son estrictamente más cortas que $A[low..hi]$.

Note que los tamaños de las dos subinstancias no se conocen de antemano: dependen de dónde quede el pivote y, por lo tanto, del contenido del arreglo. Esta es la segunda diferencia con *MergeSort*, cuyas dos mitades tienen siempre tamaños conocidos, y es la que obligará a analizar el algoritmo con más cuidado.

2.6.2. La partición. Falta diseñar *partition*, que es el corazón del algoritmo. Se escoge como pivote el primer elemento del subarreglo, $A[low]$, y se recorre el resto de izquierda a derecha manteniendo dos zonas: la de los elementos que resultaron menores que el pivote y la de los que no. Las variables m e i marcan los límites de esas zonas, como se muestra en la Figura 6.

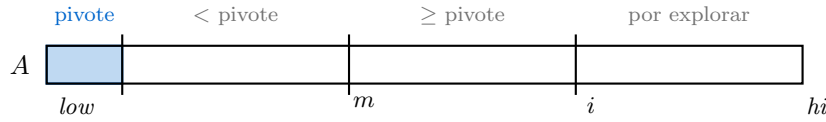


Figura 6. Relación entre las variables en el diseño de la partición.

Los invariantes del ciclo son los siguientes:

- Q_0 : $low + 1 \leq m \leq i \leq hi$.
- Q_1 : $A[k] < A[low]$ para todo k con $low < k < m$.
- Q_2 : $A[k] \geq A[low]$ para todo k con $m \leq k < i$.
- Q_3 : $A[low..hi]$ es una permutación de su versión inicial
y $A[low]$ no ha cambiado.

El invariante Q_0 indica entre qué rangos toman valores las dos variables. Los invariantes Q_1 y Q_2 describen las dos zonas ya procesadas: $A[low + 1..m)$ contiene los elementos menores que el pivote y $A[m..i)$ los que no. El subarreglo $A[i..hi)$ es el que falta por explorar. El invariante Q_3 registra que el arreglo solo se modifica intercambiando elementos y que el pivote permanece en $A[low]$ mientras dura el ciclo.

Con estos invariantes, cada iteración es inmediata. Si $A[i] \geq A[low]$, el elemento ya está donde debe estar, al final de la segunda zona, y basta con avanzar i . Si $A[i] < A[low]$, hay que llevarlo a la primera zona; para no perder lo que allí había, se intercambia con $A[m]$, que es el primer elemento de la segunda zona, y se avanzan m e i . Al terminar el ciclo se tiene $i = hi$ y solo falta ubicar el pivote entre las dos zonas, lo que se logra intercambiándolo con $A[m - 1]$.

Implementación 2.6.1: Quicksort en Python

```

1 def quicksort(A, low, hi):
2     if low+1<hi:
3         p = partition(A, low, hi) # divide: the pivot reaches its final
           ↪ position
4         quicksort(A, low, p)      # induction hypothesis on the smaller
           ↪ elements
5         quicksort(A, p+1, hi)     # induction hypothesis on the other ones
6                                   # nothing is left to combine
7
8 def partition(A, low, hi):
9     m = low+1                     # A[low+1..m) holds the elements < A[low]
10    for i in range(low+1, hi):    # A[m..i) holds the elements >= A[low]
11        if A[i] < A[low]:
12            A[i],A[m] = A[m],A[i] # move A[i] to the first zone
13            m = m+1
14    A[low],A[m-1] = A[m-1],A[low] # the pivot reaches its position
15    return m-1

```

Ejemplo 2.6.1

Considere el llamado `partition(A, 0, 6)` sobre el arreglo $A = [5, 8, 3, 9, 1, 7]$, de modo que el pivote es $A[0] = 5$. La siguiente tabla muestra el valor de $A[0..6)$ al comenzar cada iteración del ciclo, con las dos zonas separadas por barras:

i	m	$A[i]$	$A[0..6)$
1	1	8	5 8 3 9 1 7
2	1	3	5 8 3 9 1 7
3	2	9	5 3 8 9 1 7
4	2	1	5 3 8 9 1 7
5	3	7	5 3 1 9 8 7
al terminar			5 3 1 9 8 7

Al terminar el ciclo se tiene $m = 3$, así que el pivote se intercambia con $A[2] = 1$ y el arreglo queda $[1, 3, 5, 9, 8, 7]$. La función retorna $p = 2$, que es la posición que el 5 ocupa en el arreglo ordenado.

2.6.3. Correctitud. La correctitud del algoritmo se establece en dos etapas que corresponden a las dos herramientas de la Sección 2.1. La primera es la de

partition, que es un ciclo y por lo tanto se demuestra con sus invariantes, en el esquema de la Nota 2.1.4. La segunda es la de **quicksort**, que es recurrente y por lo tanto se demuestra por inducción, tomando la primera como hipótesis. Ese reparto es el que hace manejable el argumento: ninguna de las dos demostraciones tiene que razonar sobre lo que hace la otra.

Se comienza por la partición. El enunciado recoge las tres cosas que **quicksort** necesita de ella: que termine, que no pierda ni invente elementos y que la posición retornada separe el subarreglo en los dos bloques anunciados.

Teorema 2.6.1: Correctitud de partition

Para $N = \text{len}(A)$ y $0 \leq \text{low} < \text{hi} \leq N$, el llamado **partition**(A , low , hi) termina, deja en $A[\text{low}..\text{hi})$ una permutación de su versión inicial y retorna una posición p , con $\text{low} \leq p < \text{hi}$, tal que todo elemento de $A[\text{low}..p)$ es menor que $A[p]$ y todo elemento de $A[p+1..\text{hi})$ es mayor o igual que $A[p]$.

Demostración

Los invariantes Q_0 a Q_3 son ciertos al comenzar el ciclo, con $m = i = \text{low} + 1$: las dos zonas son vacías, de modo que Q_1 y Q_2 se satisfacen trivialmente, y el arreglo no se ha modificado.

Suponga que los invariantes son ciertos al comenzar una iteración con $i < \text{hi}$ y sea $v = A[\text{low}]$ el pivote, que por Q_3 no ha cambiado. Hay dos casos:

- Si $A[i] \geq v$, solo se incrementa i . La zona $A[\text{low} + 1..m)$ no cambia, de modo que Q_1 se mantiene, y la zona $A[m..i)$ crece con un elemento que satisface la condición de Q_2 .
- Si $A[i] < v$, se intercambian $A[i]$ y $A[m]$ y se incrementan ambas variables. Por Q_0 se tiene $m \leq i$, así que el intercambio es legítimo. Si $m = i$ el intercambio no hace nada y el elemento, que es menor que v , se agrega a la primera zona. Si $m < i$, por Q_2 el valor $A[m]$ es mayor o igual que v y pasa a ocupar la posición i , que es el nuevo final de la segunda zona; el valor menor que v pasa a la posición m , que es el nuevo final de la primera. En ambos casos Q_1 y Q_2 se mantienen. El invariante Q_3 se mantiene porque solo se intercambiaron dos posiciones, ninguna de ellas low , dado que $m > \text{low}$ e $i > \text{low}$.

En ambos casos i crece en uno y m no supera a i , de modo que Q_0 se mantiene. Como i crece en cada iteración y está acotada por hi , el ciclo termina y lo hace con $i = \text{hi}$. Los invariantes garantizan entonces que $A[\text{low} + 1..m)$ contiene

elementos menores que v y que $A[m..hi]$ contiene elementos mayores o iguales que v .

Falta el intercambio final. Por Q_0 se tiene $m - 1 \geq low$, así que $p = m - 1$ satisface $low \leq p < hi$. Si $m = low + 1$, la primera zona es vacía, el intercambio ocurre entre $A[low]$ y él mismo, y el pivote queda en $p = low$ con $A[low..p)$ vacío. Si $m > low + 1$, el valor $A[m - 1]$ es, por Q_1 , menor que v y pasa a la posición low , que queda dentro de la primera zona; el pivote pasa a la posición $m - 1$. En ambos casos, al terminar se tiene que $A[low..p)$ contiene los elementos menores que $A[p] = v$ y $A[p + 1..hi]$ los mayores o iguales, y el subarreglo sigue siendo una permutación de su versión inicial.

Con la partición establecida, la correctitud de quicksort es una inducción sobre la longitud del subarreglo. Vale la pena anticipar dónde se usa cada hipótesis. El Teorema 2.6.1 entra dos veces: una para garantizar que p está dentro del subarreglo, lo cual hace que las dos subinstancias sean estrictamente más cortas y habilita la hipótesis inductiva, y otra para garantizar la separación en menores y mayores, que es lo que permite concluir sin combinar.

Teorema 2.6.2: Correctitud de quicksort

Para $N = \text{len}(A)$ y $0 \leq low \leq hi \leq N$, el llamado $\text{quicksort}(A, low, hi)$ ordena ascendentemente $A[low..hi]$.

Demostración

Se procede por inducción fuerte sobre $n = hi - low$.

- Caso base ($low + 1 \geq hi$): el subarreglo es vacío o tiene exactamente un elemento, por lo cual está ordenado ascendentemente, y el algoritmo no lo modifica.
- Caso inductivo ($low + 1 < hi$): por el Teorema 2.6.1, el llamado a **partition** retorna p con $low \leq p < hi$, de modo que los subarreglos $A[low..p)$ y $A[p + 1..hi]$ tienen longitudes $p - low$ y $hi - p - 1$, ambas estrictamente menores que n . Por hipótesis inductiva, los dos llamados recurrentes los ordenan ascendentemente. Como ambos llamados permutan únicamente elementos dentro de su propio subarreglo, al terminar sigue siendo cierto que todo elemento de $A[low..p)$ es menor que $A[p]$ y que todo elemento de $A[p + 1..hi]$ es mayor o igual que $A[p]$. Por lo tanto, $A[low..hi]$ está ordenado ascendentemente.

La demostración anterior hace explícito por qué no hay combinación: todo el argumento del caso inductivo consiste en observar que las soluciones de las subinstancias ya están en su lugar. Lo que en *MergeSort* exige el Teorema 2.5.1, aquí lo garantiza la partición antes de recurrir.

2.6.4. Complejidad y escogencia del pivote. El tiempo que toma quicksort depende de dónde quede el pivote en cada llamado y, por lo tanto, del contenido del arreglo. Es fácil ver que `partition(A, low, hi)` toma tiempo $O(hi - low)$, pues realiza una iteración por cada elemento del subarreglo y cada una toma tiempo constante. Con $n = hi - low$ y con q la longitud de la primera subinstancia, la recurrencia es

$$T(n) = T(q) + T(n - q - 1) + O(n),$$

donde $0 \leq q \leq n - 1$. Es una recurrencia cuyo comportamiento depende de q y, por eso, no basta con aplicarle el Teorema Maestro: hay que estudiar los casos.

Teorema 2.6.3: Complejidad temporal de quicksort en el peor caso

El llamado `quicksort(A, 0, N)` toma tiempo $O(N^2)$ y existen arreglos de tamaño N para los cuales toma tiempo $\Omega(N^2)$.

Demostración

La cota superior se obtiene notando que cada llamado hace a lo sumo $O(n)$ trabajo propio, que la suma de las longitudes de las subinstancias de un llamado es $n - 1$ y que la profundidad de la recurrencia es a lo sumo N .

Para la cota inferior, considere un arreglo ordenado ascendentemente. El pivote $A[low]$ es el mínimo del subarreglo, así que ningún elemento pasa a la primera zona, el ciclo de `partition` termina con $m = low + 1$ y la función retorna $p = low$. La primera subinstancia es vacía y la segunda es $A[low + 1..hi]$, de longitud $n - 1$. La recurrencia se convierte en

$$T(n) = T(n - 1) + O(n),$$

que es exactamente la del algoritmo `sort` de la Sección 2.4. El cálculo de la demostración del Teorema 2.4.2 da entonces $T(N) \in \Theta(N^2)$.

Nota 2.6.1: Quicksort con mal pivote es el ordenamiento por selección

La demostración anterior dice algo más que una cota. Cuando el pivote resulta ser siempre el mínimo, `partition` hace el trabajo de recorrer el subarreglo para

dejar su menor elemento de primero y `quicksort` recurre sobre el resto: es el Algoritmo 2.4.1 de la Sección 2.4. El peor caso de *QuickSort* no es un algoritmo nuevo, sino el primer ordenamiento que se estudió en este capítulo. Resulta además llamativo que ese peor caso se alcance con un arreglo ya ordenado, que es justamente la entrada que uno esperaría más fácil.

En el otro extremo está el mejor caso. Si el pivote siempre queda en la mitad del subarreglo, las dos subinstancias tienen longitud aproximada $n/2$ y la recurrencia es

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n),$$

que es la misma de *MergeSort* y cuya solución, por el Teorema Maestro, es $T(n) \in O(n \log n)$.

Entre los dos extremos hay mucho espacio y la buena noticia es que no hace falta acertar con la mitad exacta. Basta con que las particiones no sean demasiado desbalanceadas.

Teorema 2.6.4: Particiones balanceadas

Sea α un número real con $0 < \alpha \leq 1/2$. Si en todo llamado de `quicksort` sobre un subarreglo de longitud $n \geq 2$ cada una de las dos subinstancias tiene longitud a lo sumo $(1 - \alpha)n$, entonces `quicksort(A, 0, N)` toma tiempo $O(N \log N)$.

Demostración

Considere el árbol de llamados. En el nivel 0 está el llamado inicial, sobre un subarreglo de longitud N ; en el nivel j están los llamados que se originan tras j particiones.

Primero, el trabajo por nivel. Los subarreglos de un mismo nivel son disjuntos, pues las subinstancias de un llamado lo son y están contenidas en su subarreglo. Por lo tanto, la suma de sus longitudes es a lo sumo N y, como cada llamado hace trabajo proporcional a la longitud de su subarreglo, el trabajo total del nivel j es $O(N)$.

Segundo, la cantidad de niveles. Por hipótesis, la longitud de un subarreglo del nivel j es a lo sumo $(1 - \alpha)^j N$. Un llamado con subarreglo de longitud menor que 2 no genera llamados recurrentes, de modo que los niveles no vacíos satisfacen $(1 - \alpha)^j N \geq 1$, es decir, $j \leq \log_{1/(1-\alpha)} N$.

Multiplicando ambas cosas, el tiempo total es $O(N \log_{1/(1-\alpha)} N)$. Como α es una constante, el cambio de base solo introduce un factor constante y el tiempo es $O(N \log N)$.

El Teorema 2.6.4 explica por qué *QuickSort* se comporta bien en la práctica pese a su peor caso cuadrático: la catástrofe requiere un desbalance extremo y sostenido, no un desbalance cualquiera. Incluso una partición que siempre deje una décima parte de los elementos a un lado y nueve décimas al otro da tiempo $O(N \log N)$.

Lo que sigue es garantizar ese balance. La Implementación 2.6.1 escoge siempre el primer elemento del subarreglo, de modo que el balance queda a merced de cómo venga dispuesta la entrada, sobre la cual no se tiene control. La salida consiste en quitarle al pivote esa dependencia: se escoge al azar.

Implementación 2.6.2: Quicksort con pivote aleatorio

```

1 from random import randrange
2
3 def rquicksort(A, low, hi):
4     if low+1<hi:
5         r = randrange(low, hi)           # a uniformly chosen position
6         A[low],A[r] = A[r],A[low]         # the pivot is moved to A[low]
7         p = partition(A, low, hi)
8         rquicksort(A, low, p)
9         rquicksort(A, p+1, hi)

```

Teorema 2.6.5: Complejidad temporal esperada

Sea $A[0..N)$ un arreglo cuyos elementos son distintos par a par. El llamado `rquicksort(A, 0, N)` toma tiempo esperado $O(N \log N)$, donde el valor esperado se toma sobre las escogencias aleatorias del algoritmo.

La demostración de este teorema requiere herramientas de probabilidad que no se presentan en estas notas y puede consultarse en el texto de T. Cormen et al. [CLRS22]; el Ejercicio 9 propone reconstruirla a quien cuente con ellas. El Teorema 2.6.4 da, de todas maneras, la intuición de por qué es cierto: con elementos distintos, para que el algoritmo se degrade tendría que escoger pivotes extremos casi siempre y eso es improbable.

La hipótesis sobre los elementos no es un tecnicismo de la demostración: sin ella el enunciado es falso. Considere un arreglo cuyos elementos son todos iguales. La guarda `A[i] < A[low]` es falsa para todo i , de modo que `partition` retorna siempre

$p = low$ y se cae en la recurrencia del Teorema 2.6.3. Lo determinante es que la aleatorización no puede intervenir: el intercambio de $A[low]$ con $A[r]$ ocurre entre dos valores iguales y por lo tanto no altera el arreglo. El tiempo es $\Theta(N^2)$ para toda secuencia de escogencias aleatorias; no por mala suerte, sino con certeza. El Ejercicio 5 pide trabajar los detalles y muestra, además, que el fenómeno no exige que todos los elementos sean iguales: basta con que la cantidad de valores distintos permanezca acotada mientras N crece.

Nota 2.6.2: ¿Qué cambia con la aleatorización?

Conviene precisar entonces qué compra el Teorema 2.6.5 y qué no. El peor caso sigue siendo $O(N^2)$: nada impide que el algoritmo escoja siempre el peor pivote. Lo que cambia es de qué depende ese peor caso, y la distinción útil es entre el *orden* de la entrada y su *contenido*.

El peor caso de la Implementación 2.6.1 es una propiedad del orden: un arreglo ordenado ascendentemente es malo por cómo están dispuestos sus elementos, no por cuáles son. Escoger el pivote al azar equivale a desordenar la entrada y por eso desarma esa clase de peor caso: entre los arreglos de elementos distintos ya no hay entradas malas, solo escogencias desafortunadas.

El caso de los elementos repetidos es de otra naturaleza. Allí el problema es una propiedad del multiconjunto de valores y ninguna permutación lo altera, razón por la cual la aleatorización es impotente. La cura no está en escoger mejor el pivote sino en cambiar la partición: con la partición en tres zonas que propone el Ejercicio 5, la cual separa los elementos menores, los iguales y los mayores que el pivote, todos los duplicados del pivote quedan resueltos de una sola vez y el tiempo esperado vuelve a ser $O(N \log N)$ sin hipótesis alguna sobre los valores.

Finalmente, el espacio.

Teorema 2.6.6: Complejidad espacial de quicksort

Suponiendo que la creación del ambiente de ejecución de cada llamado recurrente es de orden constante, el llamado `quicksort(A, 0, N)` usa espacio $O(N)$ en el peor caso.

Demostración

Se propone como ejercicio para el lector.

Nota 2.6.3: QuickSort frente a MergeSort

Los dos algoritmos ordenan con la misma técnica y reparten el trabajo de maneras opuestas, lo cual se refleja en sus características. *MergeSort* garantiza tiempo $O(N \log N)$ para toda entrada, pero necesita un arreglo auxiliar. *QuickSort* trabaja sobre el arreglo dado, sin memoria auxiliar más allá de la pila de ejecución, pero su peor caso es cuadrático. Hay una tercera diferencia, que se explora en el Ejercicio 6: *merge* preserva el orden relativo de los elementos iguales, mientras que *partition* no.

Ejercicios

1. Ejecute a mano `quicksort(A, 0, 6)` sobre el arreglo $A = [5, 8, 3, 9, 1, 7]$ del Ejemplo 2.6.1, indicando el valor de A y la posición retornada después de cada partición. ¿Cuántos llamados a `partition` se hacen en total?
2. Demuestre el Teorema 2.6.6. Concluya que, en el peor caso, `quicksort` usa más espacio que *MergeSort*.
3. Modifique la Implementación 2.6.1 de modo que el llamado recurrente se haga primero sobre la subinstancia más corta y el trabajo sobre la otra se realice con un ciclo, actualizando *low* o *hi*, en lugar de un llamado recurrente.
 - a) Proponga el invariante del ciclo y demuestre que el diseño es correcto.
 - b) Demuestre que la cantidad de llamados recurrentes pendientes es a lo sumo $O(\log N)$ y concluya que el espacio utilizado también lo es.
4. Exhiba, para cada $N \geq 1$, un arreglo de tamaño N sobre el cual el llamado inicial toma tiempo $\Theta(N^2)$. ¿Cuántos arreglos con esa propiedad hay? Responda la misma pregunta suponiendo que el pivote escogido es $A[hi - 1]$ en lugar de $A[low]$.
5. Considere un arreglo cuyos elementos son todos iguales.
 - a) ¿Qué retorna `partition` en ese caso? ¿Cuánto tiempo toma el ordenamiento completo? ¿Cambiaría algo si la guarda fuera $A[i] \leq A[low]$?
 - b) Demuestre que la Implementación 2.6.2 toma tiempo $\Theta(N^2)$ sobre ese arreglo para toda secuencia de escogencias aleatorias. Concluya que la hipótesis del Teorema 2.6.5 no se puede omitir.
 - c) Considere ahora un arreglo con k valores distintos, cada uno con multiplicidad N/k . Estime la cantidad de comparaciones que realiza la Implementación 2.6.2 y verifique su estimación de manera experimental para $k = 2$, $k = 10$ y $k = 100$. ¿De qué tamaño debe ser k para que el algoritmo deje de ser cuadrático?

- d) Diseñe una partición en tres zonas que reorganice $A[low..hi]$ en los elementos menores, los iguales y los mayores que el pivote, y retorne las dos posiciones que las separan. Proponga los invariantes de su ciclo y demuestre que el diseño es correcto. Este problema y su solución se conocen como el problema de la bandera nacional holandesa, propuesto por E. W. Dijkstra [Dij76].
 - e) Modifique `quicksort` para que use la partición del numeral anterior y determine cuánto tiempo toma sobre los arreglos de los numerales (a) y (c).
6. Un algoritmo de ordenamiento es *estable* si dos elementos iguales aparecen, al terminar, en el mismo orden relativo en el que aparecían inicialmente.
- a) Exhiba un arreglo que muestre que `quicksort` no es estable. Para distinguir elementos iguales, use parejas y compare únicamente su primera componente.
 - b) Argumente por qué *MergeSort* sí lo es.
7. La *selección* del k -ésimo menor elemento de un arreglo $A[0..N]$, con $0 \leq k < N$, consiste en determinar el valor que ocuparía la posición k si el arreglo estuviera ordenado ascendentemente.
- a) Diseñe un algoritmo que use `partition` y que recurra sobre una sola de las dos subinstancias. Justifique cómo se decide cuál.
 - b) Demuestre que su algoritmo es correcto.
 - c) Determine su complejidad temporal en el mejor y en el peor caso. Compárela con la de ordenar el arreglo y leer $A[k]$.
8. Una variante frecuente de la Implementación 2.6.2 consiste en escoger al azar una cantidad constante de candidatos a pivote, por ejemplo cinco, y usar como pivote la mediana de esos candidatos, con la esperanza de evitar los pivotes extremos.
- a) Implemente esta variante. Note que, como la cantidad de candidatos es constante, calcular su mediana toma tiempo constante.
 - b) Exhiba, para cada N , una secuencia de escogencias aleatorias que lleve al algoritmo a tiempo $\Theta(N^2)$. Concluya que el peor caso no mejora.
 - c) Muestre que, sobre arreglos de elementos distintos, el tiempo esperado sigue siendo $O(N \log N)$ y que, por lo tanto, el análisis asintótico no cambia. ¿Qué es, entonces, lo que mejora esta variante? Relacione su respuesta con el Teorema 2.6.4.
 - d) ¿Cambiaría la respuesta si la cantidad de candidatos creciera con N , por ejemplo $\sqrt{N}$? Discuta qué costo tendría calcular la mediana en ese caso.
 - e) ¿Ayuda esta variante con el arreglo de elementos iguales del Ejercicio 5? Justifique su respuesta.

9. 🌈 Demuestre el Teorema 2.6.5. Puede ser útil considerar, para cada pareja de elementos, la probabilidad de que sean comparados en algún momento de la ejecución. Señale en qué punto del argumento se usa que los elementos son distintos par a par.

2.7. Búsqueda binaria

El problema de búsqueda binaria consiste en encontrar un valor en un arreglo ordenado.

Problema 2.7.1: Búsqueda binaria

Entrada: Un arreglo $A[0..N)$ de números ordenado ascendentemente, $N \geq 0$, y un número X .

Salida: ¿Está X en $A[0..N)$?

Esta versión del problema corresponde al problema de decisión, es decir, el que requiere responder *sí* o *no*. Hay otras versiones en las cuales se desea averiguar más información como, por ejemplo, un índice en el arreglo en el cual aparece el valor dado (si aparece) o la cantidad de ocurrencias de este. También hay variantes en las cuales el arreglo dado no está ordenado ascendentemente sino descendientemente. Cualquiera de estas variantes se puede resolver con modificaciones (en su mayoría sencillas) del algoritmo que se presenta en esta sección.

2.7.1. Diseño. Siguiendo la Metodología 2.3.1, la generalización es una función que busca a X en $A[low..hi)$, de modo que el problema se resuelve con $low = 0$ y $hi = N$. La descomposición, sin embargo, es distinta de todas las anteriores del capítulo: una instancia tiene una sola subinstancia, que es una de las dos mitades del subarreglo, y la otra mitad simplemente se descarta. La medida vuelve a ser la longitud $hi - low$. Como hay un solo llamado recurrente y su respuesta es la respuesta de la instancia, no hay nada que combinar: toda la eficiencia proviene de lo que se descarta.

La idea general es explotar la información que se tiene sobre el arreglo $A[0..N)$. En particular, el hecho de que esté ordenado (ascendentemente, aunque –como se explica anteriormente– puede estar ordenado de otra forma) brinda la posibilidad de evitar hacer comparaciones y operaciones innecesarias. La idea general es la siguiente: suponga que se compara el valor dado X con el valor $A[n]$ del arreglo ($0 \leq n < N$). Allí hay tres opciones:

- Si $A[n] = x$, entonces X está en $A[0..N)$ y se responde afirmativamente.

- Si $A[n] < x$, entonces X necesariamente NO está en $A[0..n)$ dado que el arreglo está ordenado; en este caso, solo tiene sentido buscar a partir del índice $n + 1$. Es decir, se puede recurrir con un problema de tamaño menor.
- Si $A[n] > x$, entonces X necesariamente NO está en $A[n..N)$ dado que el arreglo está ordenado; en este caso, solo tiene sentido buscar antes del índice n . Es decir, se puede recurrir con un problema de tamaño menor.

Cualquier caso, bajo las suposiciones hechas en la especificación del problema, cae en una de las tres opciones enumeradas anteriormente. Así, se puede explicar de manera más precisa la idea central detrás del algoritmo de búsqueda: descartar rápidamente porciones del arreglo de entrada durante el proceso de búsqueda teniendo la certeza de que la porción del espacio de búsqueda que se preserva tiene suficiente información para responder la pregunta. La razón por la cual el algoritmo tiene la palabra “binaria” en su nombre es por la forma en cual el espacio restante para la búsqueda se divide en dos “mitades”, de las cuales una se mantiene y la otra se descarta.

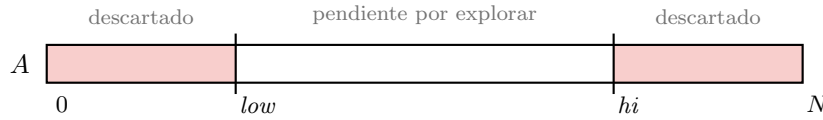


Figura 7. Espacio de búsqueda activo y porciones descartadas.

El diseño general del algoritmo se presenta en la Figura 7. El algoritmo mantiene dos *centinelas* low y hi que definen las fronteras del espacio de búsqueda activo $A[low..hi)$ para el valor X . Las otras dos porciones $A[0..low)$ y $A[hi..N)$ han sido descartadas y se tiene certeza de que no es necesario seguir buscando allí. Este diseño usa las siguientes condiciones:

$$C_0 : (\exists i \mid 0 \leq i < N : A[i] = X) \equiv (\exists i \mid low \leq i < hi : A[i] = X).$$

$$C_1 : 0 \leq low \leq hi \leq N.$$

La lectura de la condición C_0 se puede facilitar si la equivalencia lógica se interpreta como una doble implicación: si el valor está en el arreglo, entonces está en el espacio de búsqueda pendiente por explorar (implicación de izquierda a derecha); si el valor está en el espacio pendiente por explorar, entonces (claramente) está en el arreglo (derecha a izquierda). De manera alternativa y apelando a la contraposición proposicional, esta segunda implicación se puede interpretar de la siguiente forma: si el valor NO está en el arreglo, entonces NO está en el espacio de búsqueda pendiente por explorar. La condición C_1 es auxiliar y limita los valores que las

variables *low* y *hi* pueden tomar. Es importante hacer dos observaciones sobre estas variables. Primero, *low* nunca debe sobrepasar a *hi*. Segundo, bajo estas restricciones necesariamente *low* es un índice de $A[0..N)$ cuando el arreglo no es vacío (i.e., cuando $N \neq 0$), pero este no es el caso con *hi* (¿por qué?). Estas propiedades son importantes pues la corrección y terminación del algoritmo dependen, en gran medida, de que sean ciertas.

La idea del algoritmo de búsqueda binaria es estrechar incrementalmente el espacio pendiente por explorar. Es decir, la idea es que las variables *low* y *hi* se acerquen de tal manera que la cantidad de valores pendientes por explorar $A[low..hi)$ sea cada vez más pequeña. ¿En qué casos indican estas variables que se ha estrechado suficientemente el cerco? De acuerdo con la condición C_1 , estos casos están dados por $low = hi$ y $low + 1 = hi$: o no hay espacio por explorar o hay exactamente una posición de A por ser explorada. En el primer caso, claramente la respuesta es negativa pues un arreglo vacío no contiene valor alguno. Como se verá en los algoritmos que se diseñarán, este caso base solo es posible cuando el arreglo $A[0..N)$ es vacío. En el segundo caso, esto quiere decir que la máxima reducción que se puede hacer del espacio de búsqueda, de acuerdo con los invariantes propuestos, es a $A[low..low + 1)$ que corresponde a $A[low]$. Si se combina esta observación con la condición C_0 , entonces se habría logrado reducir la búsqueda en todo el arreglo a solo un punto: para determinar si X está en $A[0..N)$, basta con determinar si X es el valor en $A[low]$.

La siguiente pregunta a responder es la siguiente: ¿en qué casos (y cómo) se debe estrechar el cerco establecido por *low* y *hi* sobre $A[0..N)$? Estos son en realidad los casos inductivos y están caracterizados por la desigualdad $low + 1 < hi$. Note que bajo esta suposición, existe un número entero *mid* que satisface $low < mid < hi$. Esto en combinación con la condición C_1 , cuando $N \neq 0$, garantiza que *mid* es un índice del arreglo $A[0..N)$ y por ende, se puede consultar el valor almacenado allí: es precisamente contra $A[mid]$ que se compara X para decidir cuál de las dos partes $A[low..mid)$ o $A[mid..hi)$ descartar directamente. El cómo estrechar el cerco se responde de manera oportunista: como se ha de descartar una de dos partes, es conveniente que (sin saber a priori cuál) las dos partes sean lo suficientemente grandes como para estrechar el cerco lo más ajustado posible. Es de esta forma como se llega a la conclusión de que es conveniente que *mid* sea un punto medio (puede haber más de uno, ¿por qué?) entre *low* y *hi*, como se muestra en la Figura 8.

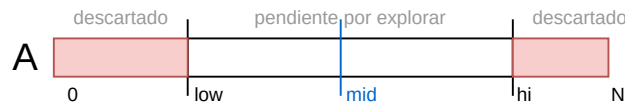


Figura 8. Escogencia de *mid* como punto medio entre los centinelas.

Algoritmo 2.7.1

1. si $low = hi$, entonces responder negativamente;
2. si $low + 1 = hi$, entonces responder afirmativamente si $A[low] = x$ y negativamente de lo contrario; y
3. si $low + 1 < hi$, entonces sea *mid* un punto medio entre *low* y *hi* y:
 - a) si $A[mid] = x$, entonces responder afirmativamente;
 - b) si $A[mid] < x$, entonces recurrir sobre $A[mid..hi)$; y
 - c) si $A[mid] > x$, entonces recurrir sobre $A[low..mid)$.

Con base en este diseño se propone la siguiente implementación:

Implementación 2.7.1: Búsqueda binaria basada en el Algoritmo 2.7.1

```

1 def binsearch(A, X, low, hi):
2     ans = None
3     if low==hi: ans = False
4     elif low+1==hi: ans = A[low]==X
5     else:
6         mid = low+((hi-low)>>1)      # mid = (low+hi)//2
7         if A[mid]==X: ans = True
8         elif A[mid]<X: ans = binsearch(A, X, mid, hi)
9         else: ans = binsearch(A, X, low, mid)
10    return ans

```

La función `binsearch` identifica tres casos. Si el espacio de búsqueda es vacío, entonces responde negativamente. Si el espacio de búsqueda ha sido reducido a una sola posición, entonces se responde con base en lo que hay en esa posición: si X está allí, entonces se responde afirmativamente; de lo contrario, negativamente. Si el espacio de búsqueda tiene al menos dos elementos, entonces se calcula la “mitad” *mid* del subarreglo y se recurre con base en la relación existente entre X y el elemento $A[mid]$, cuando el elemento no se encuentra allí. Esta es una traducción literal del Algoritmo 2.7.1 al lenguaje de programación Python.

2.7.2. Correctitud y terminación. A continuación se presenta la demostración de que la función `binsearch` es correcta con respecto a su especificación.

Teorema 2.7.1

Para $N = \text{len}(A)$ y $0 \leq \text{low} \leq \text{hi} \leq N$, el llamado `binsearch(A, X, low, hi)` determina si X está en $A[\text{low}..\text{hi}]$. En consecuencia, `binsearch(A, X, 0, N)` determina si X está en $A[0..N)$.

Demostración

Se procede por inducción sobre la distancia entre low y hi :

- Caso base ($\text{low} = \text{hi}$): el espacio de búsqueda es vacío, luego se responde negativamente
- Caso base ($\text{low} + 1 = \text{hi}$): el espacio de búsqueda tiene una sola posición $A[\text{low}]$, entonces se determina directamente si X corresponde a ese valor.
- Caso inductivo ($\text{low} + 1 < \text{hi}$): el espacio de búsqueda tiene al menos dos posiciones. Por la forma en que se calcula mid , se puede demostrar que: (i) $\text{low} < \text{mid} < \text{hi}$ y (ii) mid es un índice del arreglo A (estas demostraciones se proponen como ejercicio para el lector). Con base en estas observaciones, se tiene que la búsqueda de X en $A[\text{low}..\text{mid})$ y $A[\text{mid}..\text{hi}]$, respectivamente, son subproblemas de la búsqueda en $A[\text{low}..\text{hi}]$. Note que si $A[\text{mid}] < X$, es imposible que X esté en $A[\text{low}..\text{mid})$ (¿por qué?). De manera similar, si $A[\text{mid}] > X$, es imposible que X esté en $A[\text{mid}..\text{hi}]$ (¿por qué?). En cada uno de los casos se recurre sobre la “mitad” que tiene el potencial de contener a X .

En consecuencia, el llamado `binsearch(A, X, 0, N)` resuelve el problema de búsqueda de X en el arreglo A ordenado ascendentemente.

La demostración de que la función `binsearch` termina se propone como ejercicio para el lector.

Teorema 2.7.2

Para $N = \text{len}(A)$ y $0 \leq \text{low} \leq \text{hi} \leq N$, el llamado `binsearch(A, X, low, hi)` termina.

Demostración

Se propone como ejercicio para el lector.

2.7.3. Complejidad.**Teorema 2.7.3**

Para $N = \text{len}(A)$, el llamado `binsearch(A, x, 0, N)` toma $O(\lceil \log N \rceil)$ tiempo, en donde $\lceil \cdot \rceil$ es la función techo.

Demostración

Ver el Ejemplo 1.7.3.

2.7.4. Una versión iterativa. Una versión iterativa del algoritmo de búsqueda binaria se presenta a continuación. Inicialmente identifica dos casos. Si el arreglo dado es vacío, la respuesta necesariamente es negativa. De lo contrario, iterativamente se estrecha el cerco de la búsqueda tratando de acercar `low` y `hi` hasta que sean números consecutivos. En esta segunda parte, en la cual en realidad se lleva a cabo la búsqueda, note que la inicialización `low, hi = 0, N` hace que las condiciones C_0 y C_1 , vistas como invariantes del ciclo iterativo, sean ciertas trivialmente: inicialmente el espacio de búsqueda es todo el arreglo.

Implementación 2.7.2: Búsqueda binaria iterativa

```

1 def binsearch(A, X):
2     N, ans = len(A), False
3     if N!=0:
4         low, hi = 0, N
5         #  $C_0 \wedge C_1$ 
6         while low+1!=hi:
7             mid = low+((hi-low)>>1)    # mid = (low+hi)//2
8             if A[mid]<=X: low = mid
9             else: hi = mid
10        ans = A[low]==X
11    return ans

```

La correctitud y el análisis asintótico de la versión iterativa de `binsearch` se proponen como ejercicio para el lector.

Para terminar, vale la pena mirar con cuidado dónde se usó la hipótesis de que el arreglo está ordenado. Se usó en un solo lugar: al afirmar que, si $A[mid] < X$, entonces X no puede estar a la izquierda de mid , y simétricamente. Dicho de otra forma, lo que el algoritmo necesita no es el orden del arreglo, sino poder decidir, con una sola consulta, hacia cuál lado continuar. La Sección 2.8 lleva esta observación hasta sus últimas consecuencias y muestra que la búsqueda binaria funciona en situaciones donde no hay arreglo alguno.

Ejercicios

1. Simule la ejecución del llamado `binsearch(A, X, 0, len(A))` para

$A = [-4, 2, 2, 7, 11, 14, 18, 23, 100]$ y $X = 23$.

Explique gráficamente qué sucede en cada llamado recurrente en relación con las variables `low`, `hi` y `mid`.

2. En la demostración del Teorema 2.7.1, caso inductivo, se hacen las afirmaciones (i) y (ii). Proponga demostraciones para cada una de ellas.
3. Demuestre el Teorema 2.7.2.
4. Demuestre el Teorema 2.7.3.
5. Proponga y demuestre el teorema de corrección del algoritmo de búsqueda binaria iterativa en la Implementación 2.7.2. Demuestre que el algoritmo termina.
6. Especifique el problema de búsqueda binaria en un arreglo ordenado *descendentemente*.
7. La siguiente lista de problemas tienen como entrada un arreglo de números $A[0..N]$, $N \geq 0$, ordenado ascendentemente y un número X .
 - I. Identificar el primer índice en el que aparece X en $A[0..N]$.
 - II. Identificar el último índice en el que aparece X en $A[0..N]$.
 - III. Determinar la cantidad de veces que aparece X en $A[0..N]$.

En cada uno de los problemas anteriores, la respuesta debe ser -1 cuando la búsqueda falla. Con base en esta información y para cada uno de los problemas:

 - a) Diseñe un algoritmo que resuelva el problema en tiempo $O(\log N)$.
 - b) Demuestre que el algoritmo es correcto.
 - c) Justifique por qué el algoritmo propuesto termina.
8. En un arreglo $A[0..N]$ de números, un número X es *mayoritario* si aparece más de $N/2$ veces. Suponiendo que $A[0..N]$ está ordenado ascendentemente:
 - a) Diseñe un algoritmo que determine si X es mayoritario en tiempo $O(\log N)$.
 - b) Demuestre que el algoritmo es correcto.
 - c) Demuestre que el algoritmo termina.

- d) ¿Cómo puede resolver el problema si el arreglo no necesariamente está ordenado?
9. Un *punto fijo* en un arreglo es un índice que almacena su mismo valor. Por ejemplo, 3 es un punto fijo en $[-2, 5, 1, 3, 0, 0]$.
- Especifique el problema de encontrar un punto fijo en un arreglo $A[0..N)$ ordenado ascendentemente.
 - Diseñe un algoritmo que resuelva el problema dado en tiempo $O(\log N)$.
 - Demuestre que el algoritmo es correcto y termina.
10. En el ámbito de la estadística, la mediana representa el valor de la variable de posición central en un conjunto de datos ordenados. Por ejemplo, en el conjunto $\{7, 8, 9, 10, 11, 12\}$ la mediana es 9,5. Dado un arreglo de números ordenado ascendentemente, la mediana se puede calcular en tiempo $O(1)$. Un problema interesante, más bien, es determinar la mediana de un conjunto de datos cuando estos están divididos en dos grupos de igual tamaño.

Entrada: Arreglos $A[0..N)$ y $B[0..N)$, $N \geq 0$, de números y ordenados ascendentemente.

Salida: Mediana del conjunto de datos representado por los elementos de $A[0..N)$ y $B[0..N)$.

Diseñe y analice un algoritmo que en tiempo $O(\log N)$ resuelva el problema dado. Justifique su respuesta.

11. Un arreglo de números no repetidos es *bitónico* si está conformado por una secuencia ascendente con al menos dos elementos seguida de una secuencia descendente con al menos dos elementos, o si puede ser rotado circularmente para que cumpla esta condición. Por ejemplo, $[1, 2, 3, 0, -1]$ y $[5, 4, 3, 2]$ son bitónicos, mientras que $[0, 2]$, $[2, 4, 3, 5]$ y $[1, 2, 1]$ no lo son. Suponiendo que los números son distintos, diseñe y analice un algoritmo que determine si un arreglo dado $A[0..N)$ es bitónico en tiempo $O(\log N)$.
12. Considere un arreglo $A[0..N)$ con la propiedad especial de que $A[0] \geq A[1]$ y $A[N-2] \leq A[N-1]$. Se dice que un índice n de $A[0..N)$ es un *mínimo local* cuando $A[n-1] \geq A[n] \leq A[n+1]$ (necesariamente $0 < n < N-1$). Con las condiciones dadas, el arreglo debe tener al menos un mínimo local. Por ejemplo, en $[7, 2, 2, 4, -1, 5, 6]$ los mínimos locales son 1, 2 y 4. Diseñe y analice un algoritmo que en tiempo $O(\log N)$ encuentre un mínimo local en $A[0..N)$.
13. Considere un arreglo $A[0..N)$ ordenado ascendentemente que ha sido rotado circularmente k pasos, con $1 \leq k < N$. Por ejemplo, el arreglo $[3, 7, 10, 1, 2]$ está rotado 3 pasos.
- Diseñe y analice un algoritmo que en tiempo $O(\log N)$ calcule la cantidad de pasos que ha sido rotado $A[0..N)$.

- b) Diseñe y analice un algoritmo que en tiempo $O(\log N)$ determine si un número X está en $A[0..N)$.

2.8. Búsqueda binaria sobre predicados monótonos

La búsqueda binaria de la Sección 2.7 necesita un arreglo ordenado. Esta sección muestra que esa hipótesis es más fuerte de lo necesario y que lo que el algoritmo realmente explota es una propiedad mucho más común. Para llegar allí conviene comenzar por un problema que no tiene arreglos.

Problema 2.8.1: Raíz cuadrada entera

Entrada: Un número natural N .

Salida: El mayor número natural R tal que $R^2 \leq N$, es decir, $\lfloor \sqrt{N} \rfloor$.

La solución evidente consiste en ensayar $R = 0, 1, 2, \dots$ hasta que R^2 sobrepase a N , lo cual toma tiempo proporcional a $\sqrt{N}$. Para N del orden de 10^{18} eso son mil millones de iteraciones. La pregunta es si se puede hacer algo mejor y, más interesante aún, por qué debería ser posible si aquí no hay ningún arreglo que recorrer ni ordenar.

La observación clave es la siguiente. Considere la propiedad

$$P(R) : R^2 > N$$

sobre los números naturales. Esta propiedad no es arbitraria: si es cierta para un valor, es cierta para todos los valores mayores, pues elevar al cuadrado preserva el orden entre naturales. Los naturales quedan entonces partidos en dos tramos, uno donde P es falso y otro, a su derecha, donde es cierto. Además, la respuesta del Problema 2.8.1 es justamente el último valor del primer tramo. Encontrar esa frontera es lo mismo que buscó la Sección 2.7 y se puede hacer con la misma idea de descartar la mitad del espacio en cada paso.

2.8.1. Predicados monótonos y la búsqueda de la frontera.

Definición 2.8.1: Predicado monótono

Sean a y b números enteros con $a \leq b$. Un predicado P definido sobre los enteros del intervalo $[a, b]$ es *monótono* si para todo par de enteros i y j con $a \leq i \leq j \leq b$ se tiene que $P(i)$ implica $P(j)$.

Dicho en palabras: una vez que P se vuelve cierto, permanece cierto. Si además se sabe que $P(a)$ es falso y que $P(b)$ es cierto, entonces existe un punto exacto donde P cambia de valor y ese punto es único.

Teorema 2.8.1: Existencia y unicidad de la frontera

Sea P un predicado monótono sobre $[a, b]$, con $a < b$, tal que $P(a)$ es falso y $P(b)$ es cierto. Entonces existe un único entero f , con $a < f \leq b$, tal que $P(f)$ es cierto y $P(f-1)$ es falso. Además, $P(i)$ es falso para todo $i \in [a, f)$ y cierto para todo $i \in [f, b]$.

Demostración

El conjunto $S = \{i \in [a, b] \mid P(i)\}$ no es vacío, pues contiene a b , y está acotado inferiormente. Sea f su menor elemento. Como $P(a)$ es falso, $a \notin S$ y por lo tanto $f > a$. Por definición de f , el valor $P(f)$ es cierto y $P(f-1)$ es falso, pues $f-1 \in [a, b]$ y $f-1 \notin S$. La monotonía de P garantiza que $P(i)$ es cierto para todo $i \geq f$ en el intervalo; y $P(i)$ debe ser falso para $i < f$, pues de lo contrario i estaría en S y sería menor que su mínimo. La unicidad es inmediata: si f' satisface las mismas condiciones, entonces $f' \in S$ y $f'-1 \notin S$, de donde $f' = f$.

El algoritmo que calcula esa frontera mantiene dos centinelas low y hi que acorralan el punto buscado. La diferencia con la Sección 2.7 es que lo que se descarta no son posiciones de un arreglo, sino valores para los cuales ya se sabe qué responde P , como lo ilustra la Figura 9.

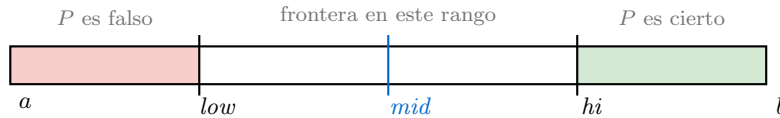


Figura 9. Centinelas de la búsqueda de la frontera de un predicado monótono.

Las condiciones del diseño son las siguientes:

$$R_0 : P(low) \text{ es falso y } P(hi) \text{ es cierto.}$$

$$R_1 : a \leq low < hi \leq b.$$

La condición R_0 es la que da sentido al procedimiento: por el Teorema 2.8.1 aplicado al intervalo $[low, hi]$, la frontera está en $(low, hi]$ y, en particular, el trabajo termina cuando $low + 1 = hi$, caso en el cual la frontera es hi . La condición R_1 acota las dos variables y exige que los centinelas no se crucen.

Mientras haya espacio por explorar, se escoge un valor mid estrictamente entre los dos centinelas y se evalúa $P(mid)$, que es la única operación del algoritmo que depende del problema concreto. Si resulta cierto, entonces la frontera no está a la derecha de mid y se puede tomar $hi = mid$; si resulta falso, la frontera está a la derecha y se puede tomar $low = mid$. En ambos casos se preserva R_0 y el espacio pendiente se reduce aproximadamente a la mitad.

Implementación 2.8.1: Búsqueda de la frontera

```

1 def boundary(P, low, hi):
2     # Pre: low < hi and P is monotone on [low..hi]
3     # Pre: P(low) is false and P(hi) is true
4     #  $R_0 \wedge R_1$ 
5     while low+1 != hi:
6         mid = low + ((hi-low) >> 1) # mid = (low+hi)//2
7         if P(mid): hi = mid         # the boundary is at mid or before it
8         else: low = mid             # the boundary is after mid
9     return hi

```

Teorema 2.8.2: Correctitud de boundary

Sea P un predicado monótono sobre $[a, b]$ y sean low_0 y hi_0 enteros con $a \leq low_0 < hi_0 \leq b$ tales que $P(low_0)$ es falso y $P(hi_0)$ es cierto. El llamado $boundary(P, low_0, hi_0)$ termina y retorna la frontera f del Teorema 2.8.1 para el intervalo $[low_0, hi_0]$. Además, realiza a lo sumo $\lceil \log_2(hi_0 - low_0) \rceil$ evaluaciones de P .

Demostración

Las condiciones R_0 y R_1 son ciertas antes de la primera iteración, por la hipótesis sobre low_0 y hi_0 .

Suponga que son ciertas al comenzar una iteración, de modo que $low+1 \neq hi$ y, con R_1 , $low+1 < hi$. El valor calculado satisface $low < mid < hi$ (se propone como ejercicio, igual que en la Sección 2.5). Si $P(mid)$ es cierto, la asignación $hi = mid$ preserva R_0 , pues el nuevo hi satisface el predicado,

y preserva R_1 , pues $mid > low$. Si $P(mid)$ es falso, la asignación $low = mid$ preserva R_0 y R_1 por el mismo argumento. En ambos casos la cantidad $hi - low$ disminuye estrictamente y, como es un entero positivo, el ciclo termina.

Al terminar se tiene $low + 1 = hi$ junto con R_0 . Por lo tanto $P(hi)$ es cierto y $P(hi - 1)$ es falso, que es exactamente la caracterización de la frontera dada por el Teorema 2.8.1, que también garantiza su unicidad.

En cuanto a las evaluaciones de P , si $d = hi - low$ al comenzar una iteración, entonces después de la asignación el nuevo valor es $\lfloor d/2 \rfloor$ o $\lceil d/2 \rceil$, en ambos casos a lo sumo $\lceil d/2 \rceil$. El ciclo termina cuando $d = 1$, lo cual ocurre a más tardar tras $\lceil \log_2(hi_0 - low_0) \rceil$ iteraciones, y cada una evalúa P una vez.

Nota 2.8.1: ¿Qué cuesta una búsqueda binaria?

El Teorema 2.8.2 cuenta evaluaciones de P y no operaciones. Esto es deliberado: el costo total del algoritmo es el producto del logaritmo del tamaño del intervalo por el costo de evaluar P una vez. En la Sección 2.7 evaluar el predicado cuesta tiempo constante, así que ese producto es $O(\log N)$; en otras aplicaciones la evaluación puede costar bastante más y es ella la que domina el análisis.

2.8.2. La raíz cuadrada entera. Con *boundary* disponible, el Problema 2.8.1 se resuelve identificando el predicado y el intervalo. El predicado es $P(R) : R^2 > N$, que es monótono sobre los naturales. Como cota inferior sirve $R = 0$, pues $0^2 = 0 \leq N$ y por lo tanto $P(0)$ es falso. Como cota superior sirve $R = N + 1$, pues $(N + 1)^2 = N^2 + 2N + 1 > N$ para todo $N \geq 0$ y por lo tanto $P(N + 1)$ es cierto. La frontera es el menor R con $R^2 > N$, de modo que la respuesta del problema es ese valor menos uno.

Implementación 2.8.2: Raíz cuadrada entera

```
1 def isqrt(N):
2     # Pre: N ≥ 0
3     return boundary(lambda R: R * R > N, 0, N + 1) - 1
```

Teorema 2.8.3: Correctitud de isqrt

Para todo $N \geq 0$, el llamado `isqrt(N)` retorna $\lfloor \sqrt{N} \rfloor$ y realiza $O(\log N)$ multiplicaciones.

Demostración

El predicado $P(R) : R^2 > N$ es monótono sobre los naturales: si $R \leq S$ y $R^2 > N$, entonces $S^2 \geq R^2 > N$. Como se argumentó antes, $P(0)$ es falso y $P(N+1)$ es cierto, así que el Teorema 2.8.2 garantiza que el llamado a `boundary` retorna el menor natural f con $f^2 > N$ y que realiza a lo sumo $\lceil \log_2(N+1) \rceil$ evaluaciones del predicado, cada una con una multiplicación. Por el Teorema 2.8.1, $(f-1)^2 \leq N$ y todo $R \geq f$ satisface $R^2 > N$; por lo tanto $f-1$ es el mayor natural cuyo cuadrado no supera a N , es decir, $\lfloor \sqrt{N} \rfloor$.

Nota 2.8.2: ¿Por qué no usar la raíz cuadrada de punto flotante?

Es tentador resolver el Problema 2.8.1 calculando la raíz cuadrada en punto flotante y tomando su parte entera. El inconveniente es que esa raíz se calcula con precisión finita y que, para valores grandes de N , el resultado puede quedar del lado equivocado de un entero. Por ejemplo, con aritmética de doble precisión, la expresión `int(sqrt(N))` entrega un valor incorrecto para algunos N cercanos a 10^{18} . El algoritmo de la Implementación 2.8.2, en cambio, solo usa aritmética entera y es exacto para todo N .

2.8.3. La búsqueda binaria como caso particular. La Sección 2.7 resulta ser una instancia de este esquema. Sea $A[0..N)$ un arreglo ordenado ascendentemente, sea X un número y considere el predicado

$$P(i) : i = N \vee A[i] \geq X,$$

definido sobre los enteros del intervalo $[-1, N]$, con la convención de que $P(-1)$ es falso. El valor -1 es un centinela: allí el predicado se declara falso por convención y nunca se evalúa, pues la Implementación 2.8.1 solo consulta valores estrictamente entre los centinelas. Vale la pena notarlo, porque en Python la expresión `A[-1]` no produce un error sino el último elemento del arreglo. Este predicado es monótono precisamente porque el arreglo está ordenado: si $A[i] \geq X$ y $j \geq i$, entonces $A[j] \geq A[i] \geq X$. Su frontera es el primer índice donde podría aparecer X , de modo que X está en el arreglo si y solo si esa frontera es distinta de N y allí el valor es X .

Implementación 2.8.3: Búsqueda binaria vía la frontera

```
1 def lower_bound(A, X):
2     # the first index i such that A[i]>=X, or len(A) if there is none
3     N = len(A)
4     return boundary(lambda i: i==N or A[i]>=X, -1, N)
```

```
5
6 def binsearch(A, X):
7     N = len(A)
8     i = lower_bound(A, X)
9     return i!=N and A[i]==X
```

La función `binsearch` anterior resuelve el Problema 2.7.1, igual que la de la Sección 2.7, pero sin caso base propio ni llamados recurrentes: toda su lógica está en el predicado. De hecho, responde más que el problema, pues `lower_bound` entrega la primera posición donde aparece X cuando este está presente. Con el predicado $P(i) : i = N \vee A[i] > X$ se obtiene, de la misma manera, la primera posición después de la última ocurrencia de X , y la diferencia entre ambas fronteras es la cantidad de ocurrencias. Los tres primeros problemas del Ejercicio 7 quedan así resueltos de un solo golpe.

Nota 2.8.3: Qué hipótesis hace falta realmente

Vale la pena insistir en lo que este cambio de punto de vista revela. La Sección 2.7 exige que el arreglo esté ordenado, pero lo que el algoritmo necesita es que el predicado con el cual se decide hacia dónde continuar sea monótono. El orden del arreglo es apenas una manera, muy frecuente, de conseguirlo. Por eso hay problemas sin arreglos, como la raíz cuadrada entera, que se resuelven con búsqueda binaria, y problemas con arreglos ordenados que no se pueden resolver así, porque el predicado que interesa no es monótono.

2.8.4. Buscar en el espacio de las respuestas. La aplicación más útil del esquema aparece en problemas de optimización, es decir, aquellos que piden el menor (o el mayor) valor que satisface alguna condición. La estrategia consiste en cambiar la pregunta de optimización por una de decisión y buscar la frontera entre las respuestas imposibles y las posibles. Sus pasos son los siguientes:

1. **Formular la pregunta de decisión.** Para un candidato a respuesta x , definir el predicado $P(x)$ que dice si x es suficiente para satisfacer la condición del problema.
2. **Verificar la monotonía.** Argumentar que si x es suficiente, entonces todo candidato mayor también lo es (o, en los problemas de maximización, todo candidato menor). Este paso no es un trámite: si la monotonía falla, el esquema no aplica.
3. **Acotar el espacio de búsqueda.** Exhibir un candidato low_0 que con certeza no sirve y uno hi_0 que con certeza sirve. Suelen ser valores extremos y groseros;

no importa que estén lejos de la respuesta, pues solo entran al costo dentro de un logaritmo.

4. **Diseñar el algoritmo de decisión.** Es la evaluación de P y, por la Nota 2.8.1, es lo que domina el costo total.

Lo que hace atractivo el esquema es que decidir suele ser mucho más fácil que optimizar: la pregunta *¿alcanza con x ?* se responde a menudo con un recorrido, mientras que la pregunta *¿cuál es el mejor x ?* parece exigir considerar todas las posibilidades. La Sección 2.10 desarrolla en detalle un problema de este tipo y los ejercicios de esta sección ofrecen otros más sencillos.

Ejercicios

1. Demuestre que en la Implementación 2.8.1 se satisface $low < mid < hi$ siempre que $low + 1 < hi$.
2. Simule el llamado `isqrt(50)`, indicando el valor de low , hi y mid en cada iteración. ¿Cuántas evaluaciones del predicado se hacen?
3. ¿Qué ocurre con la Implementación 2.8.2 si $N = 0$? ¿Y si se reemplaza la cota superior $N + 1$ por N ? Justifique su respuesta en términos de las hipótesis del Teorema 2.8.2.
4. Generalice la Implementación 2.8.2 a la raíz k -ésima entera, es decir, al mayor natural R tal que $R^k \leq N$, para $k \geq 1$ dado.
 - a) Especifique el problema e identifique el predicado y las cotas.
 - b) Implemente el algoritmo y demuestre que es correcto.
 - c) Determine su complejidad temporal en función de N y de k . Tenga en cuenta el costo de calcular R^k .
5. Considere el problema de calcular el cociente entero $\lfloor n/d \rfloor$ de dos naturales n y d , con $d > 0$, sin usar las operaciones de división del lenguaje.
 - a) Identifique un predicado monótono y unas cotas apropiadas.
 - b) Implemente el algoritmo y determine cuántas multiplicaciones realiza.
6. Reescriba la búsqueda binaria de la Implementación 2.7.2 usando `boundary` y compare ambas versiones. ¿Cuál es más fácil de demostrar correcta? ¿Por qué?
7. Con base en la Implementación 2.8.3, implemente y demuestre correctos los algoritmos que determinan el último índice en el que aparece x en $A[0..N)$ y la cantidad de veces que aparece.
8. Exhiba un arreglo ordenado ascendentemente y un predicado sobre sus índices que no sea monótono. Muestre qué retorna `boundary` con ese predicado y explique cuál hipótesis del Teorema 2.8.2 se está violando.

9. Un sistema de control de versiones guarda una secuencia de N versiones de un programa. Se sabe que la versión 0 funciona correctamente, que la versión $N - 1$ tiene un error y que, una vez introducido, el error persiste en todas las versiones posteriores. Verificar si una versión tiene el error requiere compilar el programa y ejecutar sus pruebas, lo que toma tiempo C .
- Diseñe un algoritmo que identifique la primera versión defectuosa y determine su costo en términos de N y C .
 - ¿Cuántas versiones habría que probar si $N = 1000$?
 - Discuta qué sucede si el error es intermitente, es decir, si el predicado deja de ser monótono.
10. Dado un arreglo $A[0..N)$ de naturales, con $N \geq 1$, y un natural $H \geq N$, considere el problema de determinar el menor natural $v \geq 1$ tal que
- $$P(v) : (+i \mid 0 \leq i < N : \lceil A[i]/v \rceil) \leq H.$$
- Demuestre que el predicado $P(v)$ es monótono.
 - Exhiba cotas low_0 y hi_0 apropiadas y justifíquelas.
 - Implemente el algoritmo y determine su complejidad temporal.
11. 🌈 Sea $f : \mathbb{N} \rightarrow \mathbb{Z}$ una función creciente de la cual se desconoce todo salvo que existe un natural m con $f(m) \geq 0$. Se puede evaluar f en cualquier punto, pero cada evaluación es costosa. Diseñe un algoritmo que encuentre el menor natural m con $f(m) \geq 0$ realizando $O(\log m)$ evaluaciones de f . Note que el problema no suministra una cota superior: hay que construirla.

2.9. Bisección

La Sección 2.8 dejó la búsqueda binaria reducida a su ingrediente esencial: un predicado monótono y dos centinelas que acorralan la frontera. Nada en esa idea obliga a que los candidatos sean números enteros. Esta sección la lleva a los números reales, con lo cual aparece un problema nuevo: entre dos reales distintos siempre hay otro real, de modo que el espacio de búsqueda nunca se agota y el ciclo nunca termina por haberse quedado sin candidatos. La terminación tendrá que venir de otro lado.

El problema que sirve de guía es el de encontrar una raíz de una función, y la herramienta que lo hace posible es un resultado clásico del análisis.

Teorema 2.9.1: Teorema del valor intermedio

Sea $f : [a, b] \rightarrow \mathbb{R}$ una función continua con $f(a) < 0 < f(b)$. Entonces existe $r \in (a, b)$ tal que $f(r) = 0$.

Este teorema es el que garantiza que la búsqueda tiene sentido: si los extremos del intervalo tienen signos distintos, hay una raíz adentro y buscarla no es una empresa vana. Note además que su enunciado tiene la misma forma que la hipótesis de la Sección 2.8, con el predicado $P(x) : f(x) > 0$ en el papel del predicado monótono: es falso en a y cierto en b . La diferencia es que, sin monotonía de f , ese predicado no tiene por qué ser monótono y puede haber varias raíces. El algoritmo encontrará una de ellas.

Problema 2.9.1: Raíz de una función continua

Entrada: Una función continua $f : [a, b] \rightarrow \mathbb{R}$ con $f(a) < 0 < f(b)$, y una tolerancia $\varepsilon > 0$.

Salida: Un número $r \in [a, b]$ tal que $|r - r^*| \leq \varepsilon$ para alguna raíz r^* de f .

Note que la salida no pide una raíz, sino un valor suficientemente cercano a una. Esta es la primera consecuencia de trabajar con reales. Una raíz puede ser un número irracional, que no es representable de manera exacta, así que pedirla sería pedir lo imposible; la tolerancia ε es la que vuelve razonable el problema.

2.9.1. El diseño. El algoritmo mantiene dos centinelas *low* y *hi* que encierran una raíz, con las siguientes condiciones:

$$S_0 : f(\text{low}) < 0 \leq f(\text{hi}).$$

$$S_1 : a \leq \text{low} < \text{hi} \leq b.$$

Por el Teorema 2.9.1 aplicado al intervalo $[\text{low}, \text{hi}]$, la condición S_0 garantiza que en ese intervalo hay una raíz de f . Todo lo que hace el algoritmo es estrechar el intervalo preservando esa garantía: se evalúa f en el punto medio y se descarta la mitad en la cual los dos extremos tendrían el mismo signo, como lo ilustra la Figura 10.

Implementación 2.9.1: Bisección

```

1 def bisection(f, low, hi, eps):
2     # Pre: f is continuous on [low,hi]
3     # Pre: f(low)<0<=f(hi) and eps>0
4     # S0 ∧ S1
5     while hi-low>eps:
6         mid = (low+hi)/2

```

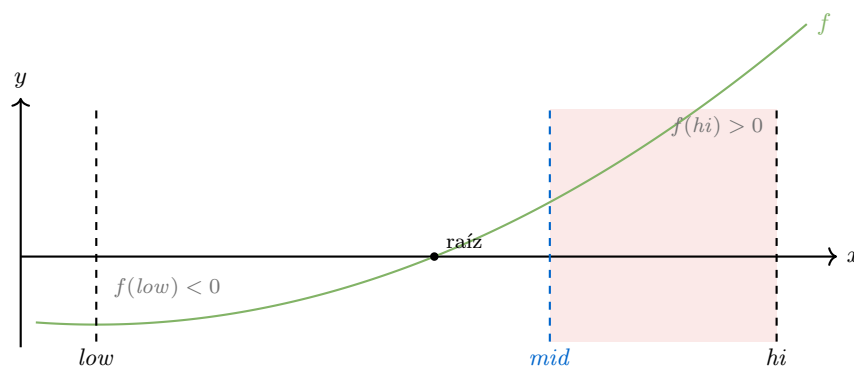


Figura 10. Una iteración de la bisección: se descarta la mitad en la cual los extremos tienen el mismo signo.

```

7   if f(mid)<0: low = mid    # the root is to the right of mid
8   else: hi = mid          # the root is at mid or to its left
9   return (low+hi)/2

```

Teorema 2.9.2: Correctitud de bisección

Sea $f : [a, b] \rightarrow \mathbb{R}$ continua con $f(a) < 0 \leq f(b)$ y sea $\varepsilon > 0$. El llamado `bisect(f, a, b, eps)` termina, realiza $0 \uparrow \lceil \log_2((b-a)/\varepsilon) \rceil$ evaluaciones de f y retorna un valor r para el cual existe una raíz r^* de f con $|r - r^*| \leq \varepsilon/2$.

Demostración

Si $b - a \leq \varepsilon$, entonces el ciclo no se ejecuta y hay 0 evaluaciones de f . Suponga $b - a > \varepsilon$. Las condiciones S_0 y S_1 son ciertas antes de la primera iteración, por las hipótesis sobre a y b .

Suponga que son ciertas al comenzar una iteración. Como $low < mid < hi$, la asignación preserva S_1 . Si $f(mid) < 0$, el nuevo intervalo es $[mid, hi]$ y S_0 se preserva porque el extremo izquierdo sigue teniendo imagen negativa y el derecho no cambia. Si $f(mid) \geq 0$, el nuevo intervalo es $[low, mid]$ y S_0 se preserva por la razón simétrica.

En cada iteración la longitud del intervalo se reduce exactamente a la mitad, de modo que tras k iteraciones vale $(b-a)/2^k$. El ciclo termina cuando esa longitud no supera a ε , lo cual ocurre para $k = \lceil \log_2((b-a)/\varepsilon) \rceil > 0$ por la suposición, y cada iteración evalúa f una sola vez.

Al terminar, S_0 sigue siendo cierta. Si $f(hi) = 0$, entonces hi es una raíz de f ; de lo contrario $f(low) < 0 < f(hi)$ y el Teorema 2.9.1, aplicado a $[low, hi]$, garantiza una raíz en (low, hi) . En ambos casos hay una raíz r^* en $[low, hi]$. Como el valor retornado es el punto medio del intervalo y este tiene longitud a lo sumo ε , se concluye que $|r - r^*| \leq \varepsilon/2$.

El teorema anterior exhibe la diferencia esencial con la Sección 2.8. Allí el ciclo terminaba porque el espacio de candidatos se agotaba, y la respuesta era exacta. Aquí el espacio nunca se agota, pues siempre hay reales entre low y hi ; el ciclo termina porque quien plantea el problema declara, con ε , cuándo el intervalo es suficientemente pequeño. Esta es la razón por la cual el costo del algoritmo depende de la precisión pedida y no del tamaño de una entrada.

Ejemplo 2.9.1

Para hallar una raíz en el intervalo $[0, 1]$ con tolerancia $\varepsilon = 10^{-9}$, el Teorema 2.9.2 da $\lceil \log_2 10^9 \rceil = 30$ evaluaciones de f . Exigir el doble de dígitos correctos, es decir $\varepsilon = 10^{-18}$, cuesta apenas 30 evaluaciones más: cada iteración agrega un bit de precisión.

2.9.2. La raíz cuadrada, otra vez. El Problema 2.8.1 pedía la raíz cuadrada entera de un natural. Su versión real se resuelve con el mismo esquema. Dado $N \geq 0$, la función $f(R) = R^2 - N$ es continua y creciente sobre los reales no negativos, y $\sqrt{N}$ es su única raíz allí. Como cotas sirven $low = 0$, donde $f(0) = -N$, y $hi = \max(1, N)$, donde f no es negativa (¿por qué hace falta el 1?).

Implementación 2.9.2: Raíz cuadrada real

```
1 def rsqrt(N, eps):
2     # Pre: N>0 and eps>0
3     return bisect(lambda R: R*R-N, 0, max(1,N), eps)
```

Vale la pena comparar esta solución con la de la Sección 2.8. Las dos recorren el mismo camino: un predicado monótono, dos centinelas y una reducción a la mitad en cada paso. Lo que cambia es el criterio de parada y la naturaleza de la respuesta: con enteros el algoritmo se detiene cuando los centinelas quedan adyacentes y la respuesta es exacta; con reales se detiene cuando el intervalo es más corto que ε y la respuesta es una aproximación.

Nota 2.9.1: Cuando la respuesta es entera

Esta distinción tiene una consecuencia práctica. Si por la naturaleza del problema se sabe que la respuesta es un número entero, conviene buscar sobre los enteros con el esquema de la Sección 2.8 y no sobre los reales con bisección. La razón es que la primera opción entrega la respuesta exacta y la segunda solo la acorrala; para recuperar el entero habría que redondear el resultado, lo cual obliga a verificar aparte que el redondeo cayó del lado correcto. Esta misma distinción reaparece en la Sección 6.1, al pasar de un problema de optimización con datos reales a uno con datos enteros.

Nota 2.9.2: Bisección y punto flotante

Al codificar la Implementación 2.9.1 con números de punto flotante hay que tener cuidado con dos asuntos.

El primero es la terminación. Los reales de la máquina son finitos y la distancia entre dos consecutivos crece con su magnitud. Si ε es menor que esa distancia, la condición $hi - low > \varepsilon$ nunca se vuelve falsa y el ciclo no termina. Por eso, en la práctica, se acostumbra fijar de antemano la cantidad de iteraciones: cien iteraciones bastan para cualquier precisión alcanzable en doble precisión, por el Ejemplo 2.9.1.

El segundo es la interpretación de ε . Una tolerancia absoluta como 10^{-9} es exigente si la raíz es del orden de 10^{-20} y es imposible de satisfacer si es del orden de 10^{20} . Cuando la magnitud de la respuesta no se conoce de antemano, es preferible una tolerancia relativa, es decir, detenerse cuando $hi - low \leq \varepsilon \cdot |low|$.

Ejercicios

1. Justifique por qué en la Implementación 2.9.2 la cota superior es $\max(1, N)$ y no simplemente N . ¿Qué ocurre con $N = 1/4$?
2. Simule el llamado `bisect(f, 0, 2, 0.25)` con $f(r) = r^2 - 2$, indicando el intervalo en cada iteración. Compare la cantidad de iteraciones con la cota del Teorema 2.9.2.
3. Modifique la Implementación 2.9.1 para que reciba la cantidad de iteraciones en lugar de la tolerancia, como sugiere la Nota 2.9.2. Enuncie y demuestre el teorema de correctitud de esta versión.
4. Suponga que se retira la hipótesis $f(a) < 0 \leq f(b)$ y solo se sabe que f es continua.
 - a) Exhiba una función continua con una raíz en $[a, b]$ sobre la cual la Implementación 2.9.1 no la encuentra.
 - b) ¿Qué se puede afirmar cuando $f(a)$ y $f(b)$ tienen el mismo signo?
5. Sea f continua y estrictamente creciente en $[a, b]$ con $f(a) < 0 < f(b)$.
 - a) Demuestre que el predicado $P(x) : f(x) \geq 0$ es monótono en el sentido de la Definición 2.8.1, extendido a los reales.
 - b) Demuestre que la raíz es única.
 - c) Concluya que, en este caso, el valor retornado por `bisect` aproxima la raíz y no solo alguna raíz.
6. Dado un real $c > 0$ y un natural $k \geq 1$, diseñe un algoritmo que calcule $c^{1/k}$ con tolerancia ε . Identifique la función, sus cotas y la cantidad de evaluaciones que realiza.
7. Considere el problema de calcular la función inversa de una función continua y estrictamente creciente $g : [a, b] \rightarrow \mathbb{R}$, es decir, dado y en la imagen de g , hallar x con $g(x) = y$ hasta una tolerancia ε . Expresé el problema como una búsqueda de raíz y escriba el algoritmo.
8. Una función $g : [a, b] \rightarrow \mathbb{R}$ es *unimodal* si existe $m \in [a, b]$ tal que g es estrictamente creciente en $[a, m]$ y estrictamente decreciente en $[m, b]$.
 - a) Muestre que el predicado que usa la bisección no sirve para hallar m , pues no es monótono.
 - b) Sean $u < v$ dos puntos en (a, b) . Demuestre que si $g(u) < g(v)$, entonces $m \in (u, b]$, y que si $g(u) > g(v)$, entonces $m \in [a, v)$.
 - c) Con base en lo anterior, diseñe un algoritmo que acorrale m con tolerancia ε evaluando g dos veces por iteración. Este algoritmo se conoce como *búsqueda ternaria*.
 - d) ¿Cuántas evaluaciones de g realiza? Compárelo con la bisección.
 - e) Relacione este algoritmo con el Ejercicio 11.

9. 🌈 El *método de Newton* aproxima una raíz de una función derivable con la iteración $x_{k+1} = x_k - f(x_k)/f'(x_k)$.
- Implemente el método para $f(r) = r^2 - n$ y compare experimentalmente la cantidad de iteraciones con la de `bisect` para $n = 2$ y $\varepsilon = 10^{-12}$.
 - Exhiba una función y un punto inicial para los cuales el método no converge. ¿Puede ocurrir algo así con la bisección?
 - Discuta qué gana y qué pierde cada método. Tenga en cuenta que la bisección solo requiere continuidad y comparaciones de signo.

2.10. Aplicación: la planeación de una travesía

Las secciones anteriores desarrollaron un esquema y lo aplicaron a problemas cuya solución era, en el fondo, evidente desde el principio: la raíz cuadrada entera se puede calcular ensayando, y la búsqueda binaria ya se había estudiado. Esta sección aplica el mismo esquema a un problema que no lo sugiere en absoluto y en el cual la búsqueda binaria aparece donde uno menos la espera.

El problema es el siguiente. Un grupo de excursionistas recorrerá un sendero en el cual hay N sitios de campamento. El sendero queda dividido por ellos en $N + 1$ tramos, de los cuales se conoce la longitud. La travesía durará varios días y cada noche se acampa en uno de los sitios, de modo que escoger K sitios equivale a partir el recorrido en $K + 1$ jornadas. Se quiere que la jornada más larga sea lo más corta posible.

Problema 2.10.1: Planeación de una travesía

Entrada: Un arreglo $D[0..M)$ de naturales positivos con $M \geq 1$, que representa las longitudes de los tramos consecutivos del sendero, y un natural K con $0 \leq K < M$, que representa la cantidad de noches.

Salida: El menor natural X tal que $D[0..M)$ se puede partir en a lo sumo $K + 1$ bloques contiguos, cada uno con suma a lo sumo X .

Ejemplo 2.10.1

Considere $D[0..5) = [7, 2, 6, 4, 5]$ con $K = 3$ noches, es decir, cuatro jornadas. La respuesta es 8, que se alcanza con la partición

$$\underbrace{7}_{7} \mid \underbrace{2 \ 6}_{8} \mid \underbrace{4}_{4} \mid \underbrace{5}_{5}.$$

Con $X = 7$ no es posible: el tramo de longitud 6 no cabe junto con ninguno de sus vecinos, ni el de longitud 4 junto con el de longitud 5, de modo que harían falta cinco jornadas.

Antes de seguir conviene notar por qué el problema no invita a pensar en búsqueda binaria. Lo que se pide es una partición óptima de un arreglo en bloques contiguos, que es un problema de optimización combinatoria. La cantidad de particiones en $K + 1$ bloques es $\binom{M-1}{K}$, así que enumerarlas no es viable. La primera idea razonable es resolverlo con programación dinámica sobre prefijos del arreglo y cantidad de jornadas disponibles, lo cual se puede hacer en tiempo $O(KM^2)$ con las técnicas del capítulo siguiente. Nada en el enunciado sugiere que haya algo ordenado sobre lo cual buscar; de hecho, el arreglo de entrada ni siquiera está ordenado y ordenarlo carecería de sentido, pues los bloques deben ser contiguos.

2.10.1. De la optimización a la decisión. El giro consiste en aplicar el esquema de la Sección 2.8 no sobre el arreglo, sino sobre el espacio de las respuestas. En lugar de preguntar cuál es la mejor jornada máxima, se pregunta si un valor dado alcanza:

$P(X)$: $D[0..M)$ se puede partir en a lo sumo $K + 1$ bloques contiguos, cada uno con suma a lo sumo X .

Con esta formulación, la salida del Problema 2.10.1 es exactamente la frontera de P y los cuatro pasos de la Sección 2.8 se pueden seguir uno por uno. Los dos primeros son los siguientes.

Teorema 2.10.1: Monotonía y cotas

Sea $m = (\uparrow i \mid 0 \leq i < M : D[i])$ y sea $s = (+i \mid 0 \leq i < M : D[i])$. Entonces:

1. P es monótono sobre los enteros del intervalo $[m - 1, s]$;
2. $P(m - 1)$ es falso; y
3. $P(s)$ es cierto.

Demostración

Para la monotonía, sean $X \leq Y$ enteros y suponga que $P(X)$ es cierto. La partición que atestigua $P(X)$ tiene a lo sumo $K + 1$ bloques, cada uno con suma a lo sumo $X \leq Y$, de modo que esa misma partición atestigua $P(Y)$.

Para (2), sea j un índice con $D[j] = m$ y considere cualquier partición de $D[0..M)$ en bloques contiguos. El bloque que contiene a $D[j]$ tiene suma

al menos $m > m - 1$, pues las longitudes son positivas. Por lo tanto ninguna partición sirve y $P(m - 1)$ es falso.

Para (3), la partición en un solo bloque tiene suma s y usa $1 \leq K + 1$ bloques, de modo que atestigua $P(s)$.

Las cotas del teorema anterior son deliberadamente bondadosas. La búsqueda binaria solo paga el logaritmo del tamaño del intervalo, así que no vale la pena afinarlas: pasar de s a una cota diez veces mejor ahorra apenas unas tres evaluaciones del predicado.

2.10.2. El algoritmo de decisión. Falta el paso que concentra el trabajo: decidir, para un X dado, si $P(X)$ es cierto. La observación es que no hay que examinar particiones, sino construir una sola: la que acumula tramos en la jornada actual mientras quepan y solo acampa cuando el siguiente tramo no cabe.

Implementación 2.10.1: Cantidad mínima de jornadas

```

1 def days(D, X):
2     # Pre: X >= max(D)
3     used, acc = 1, 0           # one day is under way, with nothing walked
4     for d in D:
5         if acc + d <= X: acc = acc + d # the leg fits in the current day
6         else: used, acc = used + 1, d # a night is spent here
7     return used

```

Teorema 2.10.2: Correctitud de days

Sea $X \geq m$. El llamado `days(D, X)` retorna la menor cantidad de bloques contiguos en los que se puede partir $D[0..M)$ de modo que cada uno tenga suma a lo sumo X . En consecuencia, $P(X)$ es cierto si y solo si `days(D, X) ≤ K + 1`.

Demostración

Sea g_j el índice donde termina el j -ésimo bloque construido por `days` y sea c_j el índice donde termina el j -ésimo bloque de cualquier otra partición válida. Se demuestra por inducción sobre j que $g_j \geq c_j$.

Para $j = 1$, el primer bloque de `days` se extiende mientras la suma no sobrepase x , de modo que ningún bloque válido que comience en 0 puede terminar más allá; es decir, $g_1 \geq c_1$. Para el paso inductivo, suponga $g_j \geq c_j$.

El bloque $j + 1$ de la otra partición comienza en $c_j + 1$ y el de **days** comienza en $g_j + 1 \geq c_j + 1$; como parte de una posición no anterior y se extiende todo lo que puede, termina en una posición no anterior, esto es, $g_{j+1} \geq c_{j+1}$.

Si la otra partición usa t bloques, entonces $c_t = M - 1$ y por lo tanto $g_t \geq M - 1$, lo cual significa que **days** ya ha cubierto todo el arreglo con a lo sumo t bloques. Así, **days** nunca usa más bloques que cualquier partición válida, y como la que construye es válida, su cantidad de bloques es la mínima.

Para la segunda afirmación, si $P(X)$ es cierto existe una partición válida con a lo sumo $K + 1$ bloques y, por lo anterior, $\text{days}(D, X) \leq K + 1$. Recíprocamente, la partición que construye **days** es válida, así que si usa a lo sumo $K + 1$ bloques entonces $P(X)$ es cierto.

Nota 2.10.1: El espíritu de days

La función **days** decide sin considerar alternativas: en cada paso toma la decisión que parece mejor en ese momento, que es caminar tanto como se pueda antes de acampar, y nunca la revisa. Esa es la forma de los algoritmos *voraces*, y la demostración del Teorema 2.10.2 es típica de ellos: se compara la solución construida con una solución cualquiera y se muestra que la primera nunca va por detrás de la segunda. El capítulo dedicado a esa técnica, el Capítulo 4, estudia el asunto con el cuidado que merece; aquí basta con el teorema.

2.10.3. El algoritmo completo. Con el predicado decidido, el algoritmo es un llamado a **boundary** de la Sección 2.8.

Implementación 2.10.2: Planeación de una travesía

```
1 def trip(D, K):
2     # Pre: len(D)>=1, 0<=K<len(D), all distances are positive
3     return boundary(lambda X: days(D,X)<=K+1, max(D)-1, sum(D))
```

Ejemplo 2.10.2

Para $D[0..5] = [7, 2, 6, 4, 5]$ y $K = 3$ se tiene $m = 7$ y $s = 24$, de modo que la búsqueda comienza con $low = 6$ y $hi = 24$. La siguiente tabla muestra cada iteración:

<i>low</i>	<i>hi</i>	<i>mid</i>	<i>days</i>	jornadas construidas
6	24	15	2	7 2 6 4 5
6	15	10	3	7 2 6 4 5
6	10	8	4	7 2 6 4 5
6	8	7	5	7 2 6 4 5

En las tres primeras iteraciones la cantidad de jornadas no supera a $K+1 = 4$, de modo que el predicado es cierto y *hi* se actualiza. En la última, con *mid* = 7, hacen falta cinco jornadas, el predicado es falso y *low* pasa a 7. El ciclo termina con *low* = 7 y *hi* = 8, y se retorna 8, en coincidencia con el Ejemplo 2.10.1.

Teorema 2.10.3: Correctitud y complejidad de trip

El llamado `trip(D, K)` retorna la salida del Problema 2.10.1 y toma tiempo $O(M \log s)$, donde s es la suma de las longitudes de los tramos.

Demostración

Por el Teorema 2.10.1, el predicado es monótono en $[m-1, s]$, falso en $m-1$ y cierto en s , que son las hipótesis del Teorema 2.8.2. Por el Teorema 2.10.2, la función que se le pasa a `boundary` decide P correctamente para todo $x \geq m$, y en particular para los valores en los cuales se evalúa (véase la Nota 2.10.2). Entonces `boundary` retorna el menor X con $P(X)$ cierto, que es la salida del problema.

En cuanto al costo, la búsqueda realiza a lo sumo $\lceil \log_2(s - m + 1) \rceil$ evaluaciones del predicado, es decir, $O(\log s)$. Cada evaluación ejecuta `days`, que recorre el arreglo una vez en tiempo $O(M)$.

Nota 2.10.2: ¿Dónde se evalúa el predicado?

La precondition de `days` exige $X \geq m$, mientras que la cota inferior de la búsqueda es $m-1$. No hay contradicción: la Implementación 2.8.1 solo evalúa el predicado en valores *estrictamente* entre los dos centinelas y, por lo tanto, nunca en $m-1$. Que $P(m-1)$ sea falso es un hecho matemático, demostrado en el Teorema 2.10.1, y no algo que el programa deba verificar. Esta situación es frecuente cuando se busca sobre el espacio de las respuestas: los

extremos suelen ser valores imposibles o absurdos que existen solo para anclar el invariante.

El contraste con la solución por programación dinámica vale la pena. Aquella explora el espacio de las particiones y toma tiempo $O(KM^2)$; esta explora el espacio de las respuestas y toma tiempo $O(M \log s)$, es decir, esencialmente el de leer la entrada unas pocas decenas de veces. La diferencia no está en una implementación más astuta, sino en haber cambiado la pregunta: decidir si un valor alcanza resultó mucho más fácil que determinar cuál es el mejor.

Nota 2.10.3: Dos ambigüedades del enunciado original

El Problema 2.10.1 toma dos decisiones que el enunciado original del problema deja abiertas y que conviene hacer explícitas, pues ilustran el papel de una especificación cuidadosa.

La primera es que el problema habla de escoger exactamente K sitios, mientras que aquí se pide partir en *a lo sumo* $K+1$ bloques. Las dos versiones tienen la misma respuesta, lo cual se propone demostrar en el Ejercicio 1.

La segunda es qué significa que haya más noches que sitios disponibles. La especificación lo evita exigiendo $K < M$, es decir, que las noches no superen la cantidad de sitios de campamento. Sin esa condición habría que decidir si se permite acampar dos noches en el mismo sitio, lo cual no cambiaría la respuesta pero sí el significado de la entrada.

Ejercicios

1. Demuestre que la respuesta del Problema 2.10.1 no cambia si se exige que la partición tenga exactamente $K + 1$ bloques. Puede ser útil observar que $K + 1 \leq M$ y que un bloque con dos o más tramos se puede partir sin aumentar la suma máxima.
2. ¿Qué retorna `trip(D, K)` cuando $K = 0$? ¿Y cuando $K = M - 1$? Verifique que en ambos casos la respuesta coincide con lo que dicta el sentido común.
3. Simule `trip(D, 2)` sobre $D[0..5] = [7, 2, 6, 4, 5]$, construyendo una tabla como la del Ejemplo 2.10.2.
4. Suponga que se reemplaza la cota inferior $m - 1$ por 0 en la Implementación 2.10.2.
 - a) ¿Sigue siendo cierto que el predicado es falso en la cota inferior?
 - b) ¿Qué ocurre al ejecutar `days(D, X)` con $X < m$? Corrija la función para que responda correctamente también en ese caso y discuta si la corrección afecta la complejidad.
5. Modifique el algoritmo para que, además del valor óptimo, retorne una partición que lo alcance, es decir, los sitios donde se debe acampar. ¿Cambia la complejidad?
6. Suponga que las longitudes de los tramos pueden ser números reales positivos y que se pide la respuesta con tolerancia ε .
 - a) Adapte el algoritmo usando la Sección 2.9.
 - b) Explique por qué, con longitudes enteras, es preferible la versión de esta sección. Relacione su respuesta con la Nota 2.9.1.
7. Considere la variante en la cual se quiere *maximizar* la jornada más corta, con las mismas restricciones.
 - a) Formule el predicado apropiado y demuestre que es monótono. Tenga en cuenta que la dirección de la monotonía se invierte.
 - b) Diseñe el algoritmo de decisión y demuestre que es correcto.
8. Una biblioteca tiene M libros en un estante, el i -ésimo con $D[i]$ páginas, y $K + 1$ lectores. Los libros se reparten en bloques contiguos, uno por lector, y se quiere minimizar la cantidad máxima de páginas asignadas a un lector. Relacione este problema con el Problema 2.10.1 y escriba su solución.
9. Una banda transportadora debe despachar M paquetes, en orden, a lo largo de H días; el i -ésimo paquete pesa $D[i]$. Cada día se despacha un bloque contiguo de paquetes cuyo peso total no supere la capacidad del camión. Determine la menor capacidad que permite despachar todos los paquetes en H días.

10. 🌈 Con las técnicas del capítulo siguiente, diseñe un algoritmo que resuelva el Problema 2.10.1 en tiempo $O(K \cdot M^2)$ y demuestre que es correcto. Compare ambas soluciones de manera experimental.

2.11. Exponenciación rápida

Todos los problemas del capítulo, hasta ahora, han tenido arreglos o tableros como entrada. Esta última sección muestra que la técnica no depende de eso: su entrada son dos números y su ganancia es tan grande como la de la búsqueda binaria.

Problema 2.11.1: Potencia

Entrada: Un número A y un natural N .

Salida: El valor A^N , con la convención $A^0 = 1$.

La definición usual de la potencia, $A^N = A \cdot A^{N-1}$ para $N > 0$, ya es un diseño recurrente: su descomposición tiene como instancias base las de exponente 0, como única subinstancia la de exponente $N - 1$ y como medida el exponente mismo. Es una descomposición por decremento, en el sentido del Ejemplo 2.1.1, y realiza N multiplicaciones.

La descomposición por mitades se obtiene de una igualdad elemental, que vale para todo $m \geq 0$:

$$A^{2m} = (A^m)^2 \quad \text{y} \quad A^{2m+1} = (A^m)^2 \cdot A.$$

Dicho en palabras: para elevar A a un exponente, basta con elevarlo a la mitad del exponente y elevar al cuadrado ese resultado, multiplicando por A una vez más si el exponente es impar. Lo notable es que la mitad se calcula *una sola vez*: el mismo valor A^m se usa dos veces en el producto. Por eso, aunque hay dos factores, solo hay una subinstancia y el exponente se reduce a la mitad en cada paso.

2.11.1. El diseño. Siguiendo la Metodología 2.3.1, la función que generaliza el problema es la que calcula a^n . Las instancias base son las de exponente 0, cuya respuesta es 1. Para $n > 0$, la única subinstancia es la de exponente $\lfloor n/2 \rfloor$, cuya medida es estrictamente menor. La combinación es elevar al cuadrado y, si a es impar, multiplicar por n .

Implementación 2.11.1: Exponenciación rápida

```

1 def power(a, n):
2     # Pre: n>=0
3     if n==0: ans = 1
4     else:
5         half = power(a, n>>1) # half =  $a^{\lfloor n/2 \rfloor}$ 
6         ans = half*half
7         if n&1: ans = ans*a    # n is odd
8     return ans

```

Teorema 2.11.1: Correctitud de power

Para todo número a y todo natural n , el llamado `power(a, n)` termina y retorna a^n .

Demostración

Se procede por inducción fuerte sobre n .

- Caso base ($n = 0$): el algoritmo retorna 1, que es a^0 por la convención del Problema 2.11.1.
- Caso inductivo ($n > 0$): sea $m = \lfloor n/2 \rfloor$. Como $n > 0$, se tiene $m < n$ y la hipótesis inductiva garantiza que el llamado recurrente retorna a^m . Si n es par, entonces $n = 2m$ y el algoritmo retorna $a^m \cdot a^m = a^{2m} = a^n$. Si n es impar, entonces $n = 2m+1$ y el algoritmo retorna $a^m \cdot a^m \cdot a = a^{2m+1} = a^n$.

La terminación se sigue de que la medida, el exponente, decrece estrictamente en cada llamado y está acotada inferiormente por 0.

Teorema 2.11.2: Cantidad de multiplicaciones

Para $n \geq 1$, el llamado `power(a, n)` realiza $\lfloor \log_2 n \rfloor + 1 + s(n)$ multiplicaciones, donde $s(n)$ es la cantidad de unos en la representación binaria de n . En particular, realiza $O(\log n)$ multiplicaciones.

Demostración

Sea $M(n)$ la cantidad de multiplicaciones del llamado. Del algoritmo se lee que $M(0) = 0$ y que, para $n > 0$,

$$M(n) = M(\lfloor n/2 \rfloor) + 1 + (n \bmod 2),$$

pues siempre se eleva al cuadrado y solo se multiplica por a cuando n es impar. Los llamados recurrentes corresponden a los valores n , $\lfloor n/2 \rfloor$, $\lfloor n/4 \rfloor$, y así sucesivamente hasta 1 y 0, cuya cantidad de elementos no nulos es $\lfloor \log_2 n \rfloor + 1$; cada uno aporta una multiplicación por el cuadrado. Además, el bit menos significativo de $\lfloor n/2^j \rfloor$ es el j -ésimo bit de n , de modo que la cantidad de llamados con exponente impar es exactamente $s(n)$, y cada uno aporta una multiplicación adicional. Sumando ambas contribuciones se obtiene el resultado. Como $s(n) \leq \lfloor \log_2 n \rfloor + 1$, se concluye que $M(n) \leq 2\lfloor \log_2 n \rfloor + 2 \in O(\log n)$.

Ejemplo 2.11.1

Para $N = 1000$ la definición usual realiza 1000 multiplicaciones. Como 1000 en binario es 1111101000, que tiene seis unos, el Teorema 2.11.2 da $9 + 1 + 6 = 16$ multiplicaciones. Para $N = 10^{18}$ la diferencia es entre un trillón de multiplicaciones y menos de cien.

2.11.2. La versión iterativa. El paso opcional de la Metodología 2.3.1 sugiere buscar una versión iterativa, que aquí elimina la pila de ejecución y expone con claridad el papel de la representación binaria del exponente. El invariante es

$$U : \quad \text{ans} \cdot \text{base}^e = a^n \quad \text{con } e \geq 0,$$

que se lee así: lo ya calculado, multiplicado por lo que falta, reproduce la respuesta. En cada iteración el exponente pendiente se parte por la mitad, lo cual obliga a elevar la base al cuadrado, y si el exponente pendiente es impar se traslada un factor a ans .

Implementación 2.11.2: Exponenciación rápida iterativa

```

1 def ipower(a, n):
2     # Pre: n>=0
3     ans, base, e = 1, a, n
4     while e!=0:                # U
5         if e&1: ans = ans*base
6         base, e = base*base, e>>1
7     return ans

```

Teorema 2.11.3: Correctitud de ipower

Para todo número a y todo natural n , el llamado `ipower(a, n)` termina y retorna a^n .

Demostración

El invariante U es cierto antes de la primera iteración, pues $1 \cdot a^n = a^n$.

Suponga que U es cierto al comenzar una iteración, con $e > 0$, y sean ans' , $base'$ y e' los valores al terminarla. Si e es par, entonces $e = 2m$, $ans' = ans$, $base' = base^2$ y $e' = m$, de modo que

$$ans' \cdot (base')^{e'} = ans \cdot (base^2)^m = ans \cdot base^{2m} = ans \cdot base^e = a^n.$$

Si e es impar, entonces $e = 2m + 1$, $ans' = ans \cdot base$, $base' = base^2$ y $e' = m$, de modo que

$$ans' \cdot (base')^{e'} = ans \cdot base \cdot base^{2m} = ans \cdot base^e = a^n.$$

En ambos casos e decrece estrictamente, pues $e > 0$, así que el ciclo termina y lo hace con $e = 0$. El invariante da entonces $ans \cdot base^0 = ans = a^n$.

2.11.3. Lo que el algoritmo realmente necesita. Conviene revisar qué propiedades de la multiplicación se usaron. En ninguna de las dos demostraciones se usó que los números sean reales, ni que el producto sea conmutativo: solo se usó que el producto es asociativo y que 1 es su elemento neutro. En consecuencia, el mismo algoritmo calcula la potencia n -ésima de cualquier operación con esas dos propiedades, lo cual amplía bastante su alcance.

Ejemplo 2.11.2: Exponenciación modular

Calcular $a^n \bmod m$ aparece en criptografía y en pruebas de primalidad, con exponentes de cientos de dígitos. Si se calcula primero a^n y luego el residuo, el valor intermedio es astronómico. En cambio, como el residuo del producto es el producto de los residuos, basta con tomar el residuo después de cada multiplicación. Así, todos los valores intermedios quedan acotados por m y el costo total es de $O(\log n)$ multiplicaciones entre números menores que m .

Ejemplo 2.11.3: Fibonacci en tiempo logarítmico

El producto de matrices de 2×2 es asociativo y tiene elemento neutro, así que `power` calcula potencias de matrices. El Ejercicio 12 establece que

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{pmatrix},$$

de modo que el n -ésimo número de Fibonacci se obtiene con $O(\log n)$ multiplicaciones de matrices, cada una con una cantidad constante de operaciones entre números. Esto es notablemente mejor que el recorrido que calcula los términos uno por uno.

Nota 2.11.1: ¿Qué se está contando?

Los teoremas de esta sección cuentan multiplicaciones y no operaciones elementales. La distinción importa porque los valores involucrados crecen: el resultado a^n tiene del orden de $n \log a$ dígitos, así que ni siquiera escribirlo toma tiempo constante, y las últimas multiplicaciones operan sobre números enormes. Contar multiplicaciones es la medida adecuada cuando los operandos caben en una palabra de la máquina, como ocurre en el Ejemplo 2.11.2, donde todos los valores intermedios son menores que m . En los demás casos, al costo hay que sumarle el de operar con números grandes.

Esta sección cierra el capítulo con el mismo mensaje con el cual empezó, en el Ejemplo 2.1.1: entre reducir una instancia en uno y reducirla a la mitad hay la distancia que separa lo imposible de lo inmediato. Lo que cambia no es el problema, sino la descomposición que se escoge para resolverlo.

Ejercicios

1. Simule `power(2, 13)` indicando el exponente de cada llamado recurrente y la cantidad de multiplicaciones. Verifique el resultado con el Teorema 2.11.2.
2. Determine la cantidad de multiplicaciones que realiza `ipower(A, N)` y compárela con la de `power(A, N)`.
3. ¿Qué ocurre con `power(A, N)` si N es negativo? Modifique el algoritmo para admitir exponentes enteros cuando $A \neq 0$, e indique qué propiedad adicional de la operación se necesita.
4. Implemente la exponenciación modular del Ejemplo 2.11.2 y demuestre que es correcta. Puede ser útil la igualdad $(x \cdot y) \bmod m = ((x \bmod m) \cdot (y \bmod m)) \bmod m$.

5. Implemente el cálculo de $F(N)$ del Ejemplo 2.11.3 y determine su complejidad temporal, suponiendo que las operaciones entre números toman tiempo constante. Compárelo experimentalmente con el cálculo iterativo que obtiene los términos uno por uno.
6. La *multiplicación del campesino ruso* calcula $A \cdot N$, para N natural, usando únicamente sumas, duplicaciones y divisiones por dos.
 - a) Enuncie las igualdades análogas a las de esta sección para el producto.
 - b) Escriba el algoritmo, demuestre que es correcto y determine cuántas sumas realiza.
 - c) Explique en qué se parece este algoritmo al de la Implementación 2.11.2.
7. Considere la variante que recorre la representación binaria del exponente de izquierda a derecha, es decir, que calcula a^n a partir de $a^{\lfloor n/2 \rfloor}$ empezando por el bit más significativo.
 - a) Escriba el algoritmo iterativo correspondiente y proponga su invariante.
 - b) Demuestre que es correcto y determine cuántas multiplicaciones realiza.
8. Muestre que `power` no siempre usa la mínima cantidad posible de multiplicaciones.
 - a) Calcule cuántas multiplicaciones usa para $n = 15$.
 - b) Exhiba una secuencia de cinco multiplicaciones que calcule a^{15} .
 - c) 🌈 El problema de hallar la mínima cantidad de multiplicaciones para un exponente dado se conoce como el problema de las *cadena de adición*. Investigue por qué no se conoce un algoritmo eficiente que lo resuelva.
9. Sea A una matriz cuadrada de $k \times k$. Determine la complejidad temporal de calcular A^n con `power`, suponiendo que multiplicar dos matrices de $k \times k$ toma tiempo $O(k^3)$. ¿Para qué valores de k y n conviene este algoritmo frente a multiplicar $n - 1$ veces?

Notas del capítulo y referencias

El planteamiento en este capítulo de dividir, conquistar y combinar es similar al propuesto por T. Cormen et al. [CLRS22]. J. Kleinberg y É. Tardos [KET06], al igual que en este texto, introducen la técnica con *MergeSort*, y diseñan algoritmos para resolver problemas como los de encontrar pares de puntos más cercanos y la transformada rápida de Fourier. Estas dos referencias son fuentes variadas de ejercicios y problemas, sugeridas para profundizar en el tema. La presentación de dividir y conquistar como una *técnica de diseño*, y no como una colección de algoritmos ingeniosos, se remonta al menos a un artículo divulgativo de J. Bentley [Ben84], de donde proviene también el problema de la suma máxima de segmento que propone el Ejercicio 5 de la Sección 2.3.

Otros autores como H. Bhasin [Bha15] y A. Levitin [Lev12] distinguen diferentes versiones de dividir y conquistar. Por ejemplo, incluyen *decrementar* y conquistar, y *transformar* y conquistar. Por decrementar se entiende un caso de dividir y conquistar en el cual cada subproblema resulta de reducir el problema inicial en una cantidad constante y no en un factor constante (e.g., como el algoritmo de ordenamiento en la Sección 2.4 en donde cada subproblema tiene una posición menos que ordenar). Por transformar se entiende que las instancias iniciales son pre-procesadas antes de aplicar dividir y conquistar. Por ejemplo, si se desea determinar si un elemento en un arreglo es único, se puede ordenar el arreglo para luego determinar la unicidad del elemento. Estos textos también son una buena fuente de ejemplos y problemas algorítmicos.

Los algoritmos de ordenamiento tienen una literatura propia y extensa; el volumen 3 de D. Knuth [Knu88] sigue siendo la referencia de conjunto, y allí se encuentran tanto la historia de *MergeSort*, que se le atribuye a J. von Neumann en 1945, como el catálogo de variantes de los algoritmos iterativos que se proponen como ejercicio en la Sección 2.4. El algoritmo de la Sección 2.6 fue propuesto por C. A. R. Hoare [Hoa62] y su análisis en el caso promedio, que aquí solo se enuncia, se encuentra en el texto de T. Cormen et al. [CLRS22]; un tratamiento más fino, con constantes y no solo con órdenes de magnitud, está en R. Sedgewick y P. Flajolet [SF13]. La partición en tres zonas que se propone como ejercicio en esa misma sección es el problema de la bandera nacional holandesa de E. Dijkstra [Dij76].

El diseño de ciclos iterativos basado en invariantes, que en este capítulo se presenta en la Sección 2.1 y se usa desde la Sección 2.4 en adelante, es común en la literatura; ver, e.g., [CLRS22]. La disciplina de plantear invariantes a modo de predicados formales proviene de enfoques como los de E. Dijkstra [Dij76], D. Gries [Gri81], E. Cohen [Coh90] y A. Kaldewaij [Kal90]. En Castellano, el uso de predicados para la derivación de algoritmos iterativos y recurrentes es propuesto por J. Bohórquez [Boh06].

La búsqueda binaria es, en apariencia, el algoritmo más simple del capítulo y, en la práctica, uno de los más fáciles de implementar incorrectamente. Su historia la relata D. Knuth en el volumen 3 de su obra [Knu88]: aparece impresa por primera vez en la década de 1940, pero las primeras versiones publicadas que funcionan para arreglos de cualquier tamaño son bastante posteriores. La lectura en términos de predicados monótonos de la Sección 2.8, y el uso de la búsqueda sobre el espacio de las respuestas de la Sección 2.10, son parte del folclor de la programación competitiva; el texto de S. Halim et al. [HHE20] las presenta así y es una buena fuente de problemas de esa familia. El problema de la Sección 2.10 es una adaptación del problema 907 del juez en línea *UVa Online Judge*. La bisección de la Sección 2.9,

por su parte, es un método clásico del análisis numérico, anterior a los computadores, y suele presentarse junto con métodos de convergencia más rápida, como el de Newton, que se propone comparar en los ejercicios de esa sección.

La exponenciación rápida de la Sección 2.11 es mucho más antigua que la computación: el método binario para elevar a una potencia aparece en textos de la India varios siglos antes de nuestra era. Su versión modular es hoy un ingrediente básico de la criptografía de clave pública y de las pruebas de primalidad, incluida la prueba determinística de M. Agrawal, N. Kayal y N. Saxena [AKS04]. La pregunta por la mínima cantidad de multiplicaciones necesarias para un exponente dado, que se propone como ejercicio, es el problema de las cadenas de adición, para el cual no se conoce un algoritmo eficiente.

La inmensa mayoría de problemas propuestos como ejercicios en esta sección hacen parte del folclor de computación y algoritmos, y como tal se pueden encontrar en varias fuentes como libros, artículos científicos y divulgativos, e internet. El Ejercicio 2b de la Sección 2.2 es un resultado conocido como el Teorema de Gomory y cuya demostración usa ciclos Hamiltonianos. Los ejercicios 11, 12 y 13 de la Sección 2.7 están inspirados por sendos problemas en [Eri19].

Programación dinámica

La técnica de diseño presentada hasta ahora responde a una pregunta: ¿cómo evitar recorrer por completo el espacio de soluciones de un problema? Dividir, conquistar y combinar (Capítulo 2) lo parte en subespacios independientes que se resuelven por separado y cuyas soluciones se combinan al final. Este capítulo presenta una segunda respuesta: comprimir el espacio de soluciones, aprovechando que muchas de sus partes se repiten. Al descomponer un problema en subproblemas es frecuente que una misma subinstancia aparezca en varias ramas del cálculo y se resuelva una y otra vez; la *programación dinámica* organiza el cálculo de manera que eso no suceda. De manera más precisa, la programación dinámica parte de una función recurrente que resuelve un problema algorítmico –de decisión, de conteo o de optimización– y produce un algoritmo que la computa resolviendo cada subinstancia una sola vez.

La apuesta que hace la técnica para lograrlo es emplear memoria para ahorrar tiempo. El cálculo de la función de interés dispone de una memoria auxiliar que funciona como un registro: cada valor que se calcula se escribe allí una sola vez y se consulta después tantas veces como sea necesario. Esa apuesta no siempre se puede hacer y precisar cuándo es posible es el asunto con el que abre el capítulo.

Nota 3.0.1: Sobre el nombre

Richard E. Bellman (1920–1984) fue un matemático aplicado que acuñó, hacia 1950, el término *programación dinámica*.

Conviene aclarar de entrada que ‘programación’ no significa aquí escribir código. El término se usa en el sentido que tiene en ‘programación lineal’ o en ‘programación de una obra’: planificar, decidir por etapas. La técnica es anterior a que programar computadores fuera una actividad corriente y el nombre no alude a ella.

El nombre tampoco se escogió por razones técnicas, sino presupuestas. Bellman trabajaba en la Corporación RAND, financiada por la Fuerza Aérea de los Estados Unidos, en un momento en que el responsable de esa financiación en Washington sentía una hostilidad declarada hacia las palabras ‘investigación’ y ‘matemáticas’. Lo que Bellman necesitaba era un rótulo que ocultara que lo que hacía dentro de RAND eran matemáticas y al que ningún congresista pudiera objetar. El siguiente es un apartado traducido de su autobiografía (ver notas y referencias al final de capítulo) en el cual relata la ‘curiosa’ historia detrás de la intención de este nombre.

Una pregunta interesante es, “¿de dónde viene el nombre ‘programación dinámica’?” Los años 1950s no eran buenos para la investigación en matemáticas. Tuvimos un personaje muy interesante en Washington, llamado Wilson. Él era el Secretario de Defensa, y realmente tenía miedo y odio patológicos de la palabra ‘investigación’. No estoy usando estos adjetivos ligeramente; los estoy usando precisamente. Su cara se descomponía, se ponía roja, y se tornaba violento si alguien usaba el término ‘investigación’ en su presencia. Se podrán imaginar cómo se sentía, entonces, acerca del término ‘matemático’. La Corporación RAND estaba empleada por la Fuerza Aérea, la cual tenía, esencialmente, a Wilson como jefe. Por esto, sentí la necesidad de hacer algo para proteger de Wilson y de la Fuerza Aérea el hecho de que en realidad estaba haciendo matemáticas al interior de la Corporación RAND. ¿Qué título, qué nombre, podía escoger? En primer lugar, estaba interesado en planeación (en inglés, *planning*), en toma de decisiones, en pensar. Pero la palabra ‘planeación’ no era una buena elección por varias razones. Por ello decidí usar el término ‘programación’. Quería lograr la idea de que esto era dinámico, con múltiples escenarios, que dependía del tiempo; entonces se me ocurrió matar dos pájaros con una sola piedra. Usemos una palabra con un significado preciso y absoluto, es decir ‘dinámica’, en el sentido físico clásico. Esta palabra también tiene propiedades interesantes como adjetivo dado que es imposible usarla en un sentido peyorativo. Trate de pensar en una combinación que le haga tomar un significado peyorativo. Es imposible. Luego, pensé que ‘programación dinámica’ era un buen nombre. Era algo que ni siquiera un congresista podría objetar. Y ese fue el término que usé como sombrilla para mis actividades.

El relato tiene, eso sí, una inconsistencia que vale la pena señalar. El Wilson al que Bellman se refiere, Charles Erwin Wilson, fue Secretario de Defensa entre 1953 y 1957, mientras que la escogencia del nombre se sitúa en 1950.

Puede tratarse de otro Charles E. Wilson, que en esos años dirigía la Oficina de Movilización de la Defensa o simplemente de un recuerdo mezclado treinta años después. S. Dreyfus reprodujo y comentó estos pasajes en [Dre02].

Nota 3.0.2: Antes que los algoritmos, el nombre

A diferencia de lo que ocurre con las demás técnicas de este texto, aquí el nombre llegó antes que la mayoría de los algoritmos. La programación dinámica no nació en la algorítmica, sino en el control óptimo y la investigación de operaciones: Bellman llegó a la Corporación RAND en 1949 a trabajar sobre procesos de decisión de múltiples etapas, es decir, sobre problemas en los cuales hay que decidir por etapas y cada decisión condiciona las siguientes. En 1957 publicó el libro que fija la técnica y que enuncia el *principio de optimalidad*, antepasado directo de lo que en la Sección 3.1 se llama propiedad de subestructura óptima. Al año siguiente publicó, además, un algoritmo de caminos mínimos que hoy lleva su nombre junto con el de L. Ford.

Casi todos los algoritmos que este capítulo estudia son posteriores. La programación dinámica se incorporó a la algorítmica más tarde, cuando la disciplina empezó a organizar sus resultados por técnica de diseño y no por problema. Varios algoritmos que habían sido publicados de manera independiente quedaron reconocidos como instancias de una misma idea (ver las notas al final del capítulo).

Este capítulo presenta las ideas principales que respaldan la programación dinámica, incluyendo técnicas de implementación, una metodología para su uso y varios ejemplos, en dominios como la comparación de secuencias, la optimización combinatoria y los caminos mínimos en grafos.

3.1. Ideas clave

La programación dinámica es una técnica que puede ser útil para resolver problemas algorítmicos. Sin embargo, no en todos los problemas algorítmicos tiene sentido usarla. Aquellos problemas que son susceptibles de abordar usando la técnica exhiben ciertas propiedades en común que relacionan la solución de instancias con subinstancias.

Nota 3.1.1: Propiedades necesarias

Considere un problema algorítmico y suponga fijada una manera de descomponer cada una de sus instancias en subinstancias del mismo problema. Las

dos propiedades siguientes son necesarias para resolver el problema algorítmico con programación dinámica:

- *Propiedad de subestructura*: la solución de una instancia se puede calcular a partir de las soluciones de una cantidad acotada de subinstancias estrictamente menores.
- *Propiedad de solapamiento*: al desplegar la descomposición, una misma subinstancia se requiere desde varios lugares distintos. Dicho de otro modo, la cantidad de subinstancias diferentes es mucho menor que la cantidad de veces que se necesitan.

Conviene subrayar que estas dos propiedades no se predicen de un problema aislado sino de un problema *junto con* una manera de descomponerlo. Un mismo problema admite descomposiciones distintas y puede ocurrir que unas exhiban las propiedades y otras no. Por eso la Metodología 3.3.1 comienza por escoger una *función objetivo*: esa elección es la que fija la descomposición y, con ella, la que decide si la técnica es aplicable. En varias de las secciones de este capítulo la dificultad principal del diseño consiste, precisamente, en dar con la descomposición adecuada.

Toda descomposición debe cumplir además una condición para que tenga sentido hablar de subinstancias: debe existir una medida concreta y bien definida —el tamaño de un arreglo, la cantidad de elementos disponibles, la longitud de un prefijo— que decrezca estrictamente al pasar de una instancia a sus subinstancias. Sin esa medida, la descomposición puede no terminar y, en consecuencia, no define nada. Cuál es la medida apropiada depende del problema.

Un ejemplo conocido de la propiedad de subestructura es el problema de ordenamiento de un arreglo de números (Problema 2.4.1). Descomponga $A[0..N]$ en las dos mitades $A[0..n]$ y $A[n..N]$, con n la mitad de N . El arreglo ordenado se obtiene mezclando las dos mitades ordenadas, de modo que hay subestructura, y la medida que decrece es la longitud del subarreglo. Lo que no hay es solapamiento: las dos mitades no comparten ningún índice y lo mismo ocurre al desplegar la descomposición, así que ninguna subinstancia se necesita dos veces. Un algoritmo que resuelva cada subinstancia apenas la encuentre no repite trabajo, y esa es la razón por la cual el ordenamiento se resuelve con dividir, conquistar y combinar, y no con programación dinámica.

La propiedad de subestructura se puede especializar dependiendo de la naturaleza del problema que se desea resolver (e.g., decisión, conteo u optimización). El caso de los problemas de optimización es el más frecuente en este capítulo y tiene nombre propio.

Definición 3.1.1: Subestructura óptima

Un problema de optimización, junto con una descomposición fijada, tiene la *propiedad de subestructura óptima* si y solo si se cumplen las dos condiciones siguientes:

- *Descomposición*: para cada instancia existe una solución óptima cuyas partes son soluciones óptimas de las subinstancias correspondientes.
- *Composición*: al combinar soluciones óptimas de las subinstancias se obtiene una solución de la instancia, cuyo valor es el que la descomposición predice.

Nota 3.1.2: Las dos mitades

Las dos condiciones de la Definición 3.1.1 son independientes y ambas hacen falta. La de descomposición garantiza que la recurrencia no se le escapa el óptimo; la de composición, que todo lo que la recurrencia construye es una solución legítima.

Esa separación explica la forma que tienen todas las demostraciones de correctitud de este capítulo. Cuando se demuestra que una función objetivo ϕ calcula el óptimo, el argumento se parte invariablemente en dos desigualdades y cada una verifica una de las dos mitades: la que acota por composición exhibe soluciones construidas a partir de las subinstancias, y la que acota por descomposición toma una solución óptima cualquiera y la parte. El lector que llegue a la Sección 3.5, a la Sección 3.7 o a la Sección 3.8 reconocerá el patrón en cada una.

En la mitad de descomposición el cuantificador importa. En algunos problemas la propiedad vale en forma universal: *toda* solución óptima se descompone en soluciones óptimas de sus subinstancias, como sucede en el Ejemplo 3.1.1. En otros vale únicamente en forma existencial y, entonces, una demostración de correctitud no puede partir de una solución óptima cualquiera: debe exhibir la adecuada o transformar una cualquiera hasta obtenerla. La demostración del Teorema 3.7.1, en la Sección 3.7, es un ejemplo de lo segundo. Enunciar la propiedad en forma universal sería, por lo tanto, demasiado fuerte.

Esta propiedad es la versión algorítmica del *principio de optimalidad* que R. Bellman enunció en 1957 (Nota 3.0.2) y los dos nombres se usan indistintamente en la literatura. La propiedad reaparece en el Capítulo 4: es una

condición necesaria, aunque no suficiente, para que una estrategia voraz produzca soluciones óptimas.

Ejemplo 3.1.1

Considere el problema de determinar la ruta más corta de un punto A a un punto B en una ciudad. Note que si una ruta óptima (porque puede haber más de una) de A a B pasa por un punto C , entonces necesariamente en esa ruta la subruta de A a C también debe ser óptima. De lo contrario, la ruta de A a C no sería óptima: se podría llegar a C “más rápido” de otra forma y, en consecuencia, a B más rápido usando otra ruta. Pero esto riñe con la suposición de que la ruta de A a B que pasa por C es óptima; i.e., no puede haber una ruta mejor.

Observe que el argumento no escogió una ruta óptima en particular sino que partió de una cualquiera. Este problema exhibe, entonces, la forma universal de la propiedad. Para este problema, ¿pueden existir solapamientos entre subproblemas?

A simple vista pareciera que la propiedad de subestructura es inocua por su sencillez y obviedad; sin embargo, no todos los problemas cuentan con esta propiedad. Basta con cambiar ‘más corta’ por ‘más larga’ en el ejemplo anterior para que deje de valer.

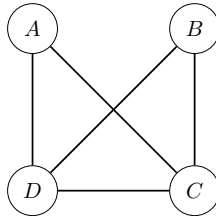


Figura 1. Una ciudad con cuatro sitios y cinco tramos. Entre A y B no hay un tramo directo.

Ejemplo 3.1.2

Considere el problema de determinar la ruta *simple* más larga de un punto A a un punto B en una ciudad. Simple se refiere a que no se visita un sitio más

de una vez; sin esa restricción el problema estaría indefinido, pues bastaría con dar vueltas indefinidamente. La longitud de una ruta es la cantidad de tramos que la componen.

Considere la instancia de la Figura 1. La ruta simple más larga de A a B tiene longitud 3 y una de ellas es $A \rightarrow C \rightarrow D \rightarrow B$. Esa ruta pasa por C , de manera que el argumento del Ejemplo 3.1.1 exigiría que su tramo inicial, de A a C , fuera una ruta simple más larga de A a C . No lo es: ese tramo tiene longitud 1, mientras que la ruta simple más larga de A a C es $A \rightarrow D \rightarrow B \rightarrow C$, de longitud 3. La propiedad de subestructura óptima falla.

La razón de fondo es que las partes óptimas no se dejan recomponer. La ruta simple más larga de A a C es $A \rightarrow D \rightarrow B \rightarrow C$ y la ruta simple más larga de C a B es $C \rightarrow A \rightarrow D \rightarrow B$; al concatenarlas usando C como punto “pivote” se obtiene

$$A \rightarrow D \rightarrow B \rightarrow C \rightarrow A \rightarrow D \rightarrow B,$$

que no es una ruta simple. Ser simple es una restricción global: no se puede verificar mirando las partes por separado y, por eso, las soluciones óptimas de las subinstancias no se dejan combinar.

Note que las dos mitades de la Definición 3.1.1 fallan en este problema y que cada uno de los dos párrafos anteriores refuta una: el primero, la de descomposición; el segundo, la de composición.

Falta ver la propiedad de solapamiento en acción, pues hasta aquí solo ha aparecido un problema que no la tiene. El ejemplo más sencillo es el de los números de Fibonacci, tema de la Sección 3.2: como $fib(n)$ se define en términos de $fib(n-1)$ y $fib(n-2)$, y el cálculo de $fib(n-1)$ vuelve a necesitar $fib(n-2)$, ese mismo valor se requiere desde dos lugares distintos. Al desplegar la recurrencia hay apenas n subinstancias diferentes, mientras que la cantidad de veces que se piden crece exponencialmente. Esa desproporción es la propiedad de solapamiento y es lo que la programación dinámica aprovecha.

La siguiente tabla resume los problemas mencionados en esta sección, cada uno con la descomposición que se discutió para él.

Problema	Subestructura	Solapamiento
Ordenamiento de un arreglo	sí	no
Números de Fibonacci	sí	sí
Ruta más corta	sí	sí
Ruta simple más larga	no	—

Únicamente la segunda y la tercera fila corresponden a problemas que se dejan abordar con programación dinámica. La primera se resuelve con dividir, conquistar

y combinar: sin solapamiento no hay nada que reusar y la memoria auxiliar sería un gasto sin contrapartida. La última no se deja abordar con ninguna de las dos, pues sin subestructura no hay siquiera una recurrencia que plantear; su casilla de solapamiento queda vacía porque no tiene sentido preguntar por ella.

Ejercicios

1. Justificando su respuesta, responda la pregunta formulada al final del Ejemplo 3.1.1.
2. Considere el problema de determinar si dada una colección de números $A[0..N]$ y un número x , es posible escoger algunos elementos de A cuya suma es x . Determine si este problema exhibe las propiedades de subestructura y solapamiento. Justifique e ilustre su respuesta con un ejemplo.
3. Considere el problema de buscar un valor en un arreglo ordenado, junto con la descomposición que descarta la mitad del arreglo en cada paso. Determine cuál de las dos propiedades exhibe y cuál no, y explique por qué eso lo convierte en un candidato para dividir, conquistar y combinar y no para programación dinámica.
4. En la Sección 3.4 se resuelve un mismo problema de dos maneras distintas: con dividir, conquistar y combinar y con programación dinámica. Después de leer esa sección, identifique la descomposición que usa cada una de las dos y explique por qué solo la segunda exhibe la propiedad de solapamiento. Concluya en qué sentido las dos propiedades dependen de la descomposición escogida y no únicamente del problema.
5. Suponga que se cuenta con el registro de precios de una acción a lo largo de N días, un precio por día. Se desea determinar la longitud de la mejor “corrida” de la acción durante los N días: es decir, la mayor cantidad de días (no necesariamente consecutivos) en los cuales la acción aumentó de precio. ¿Exhibe este problema alguna de las propiedades de subestructura o solapamiento? Justifique e ilustre su respuesta con un ejemplo.
6. Un empresario desea hacer una gira por varias ciudades conduciendo su vehículo (debe iniciar y terminar en la ciudad de origen, visitando cada una de las demás ciudades exactamente una vez). Para cada trayecto cuenta con la cantidad de dinero que cuesta el desplazamiento. Suponiendo que existe un camino directo entre cualquier par de ciudades, indique si determinar el menor costo posible de la gira exhibe alguna de las propiedades de subestructura o solapamiento. Justifique e ilustre su respuesta con un ejemplo.
7. En el Ejemplo 3.1.2 se exhibe una instancia en la cual la propiedad de subestructura óptima falla para el problema de la ruta simple más larga. Exhiba una

instancia con cuatro sitios en la cual la propiedad sí se cumpla y explique por qué un solo ejemplo favorable no permite concluir que el problema la tiene.

3.2. Cálculo de los números de Fibonacci

Se deja de lado, por un momento, el proceso de plantear una función recurrente que resuelva un problema con las propiedades de subestructura y solapamiento. El objetivo de esta sección es presentar las dos principales formas, *memorización* y *tabulación*, de instrumentar la programación dinámica. Para ilustrarlas, se parte de una recurrencia bien conocida: la función que calcula los números de Fibonacci.

Definición 3.2.1: Función de Fibonacci

Para $n \in \mathbb{N}$, la definición de la función *fib* es la siguiente:

$$fib(n) = \begin{cases} 0 & , \text{ si } n = 0, \\ 1 & , \text{ si } n = 1, \\ fib(n-2) + fib(n-1) & , \text{ si } n \geq 2. \end{cases}$$

El problema que se desea resolver es el siguiente.

Problema 3.2.1: Cálculo de la función de Fibonacci

Entrada: Un número entero N , $N \geq 0$.

Salida: $fib(N)$.

El Problema 3.2.1 no es de decisión, ni de conteo, ni de optimización: es la evaluación de una función que ya viene dada. Eso es deliberado. La Sección 3.3 se ocupará de cómo llegar a la función recurrente a partir de un problema; aquí la función se recibe hecha para poder concentrarse en cómo calcularla.

Vale la pena anotar que *fib* sí proviene de un problema de conteo. Leonardo de Pisa, conocido como Fibonacci, lo plantea en su *Liber Abaci* de 1202: una pareja de conejos recién nacida tarda un mes en madurar y a partir de entonces produce una pareja nueva cada mes; si ninguno muere, ¿cuántas parejas hay al cabo de n meses? La respuesta es exactamente $fib(n)$ con la definición anterior. La sucesión, sin embargo, había sido estudiada siglos antes en la India, en relación con el conteo de patrones métricos en la poesía sánscrita.

Hay dos casos base: para $N = 0$ o $N = 1$, se tiene que $\text{fib}(N) = N$. El caso recurrente se plantea para cualquier otro valor de N recurriendo sobre los dos valores inmediatamente anteriores, y por eso son necesarios los dos casos base.

La descomposición está fijada por la propia definición de *fib*: la instancia n se descompone en las subinstancias $n - 1$ y $n - 2$. Con esa descomposición, el Problema 3.2.1 tiene las dos propiedades de la Nota 3.1.1. Hay subestructura, pues el valor de una instancia se calcula a partir del de dos subinstancias y la medida que decrece estrictamente es el propio n . Y hay solapamiento, pues dos llamados recurrentes distintos terminan pidiendo la misma subinstancia: en el cálculo de $\text{fib}(4)$ se recalcula $\text{fib}(2)$, como se aprecia en la Figura 2. Note que la Definición 3.1.1 no interviene aquí, pues no se trata de un problema de optimización y no hay soluciones que comparar.

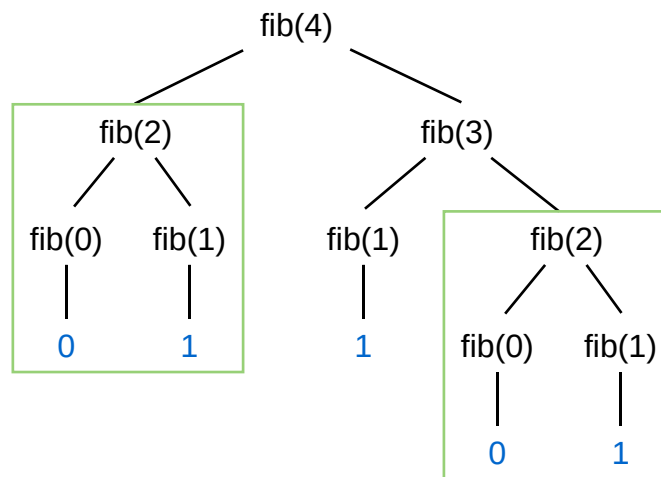


Figura 2. Árbol de llamados recurrentes en el cálculo de $\text{fib}(4)$. El subárbol correspondiente a $\text{fib}(2)$ aparece dos veces.

Una implementación directa de esta función en el lenguaje de programación Python 3 se presenta a continuación.

Algoritmo 3.2.1: Cálculo de la función fib

```

1 def fib(n):
2     ans = None
3     if n<=1: ans = n
4     else: ans = fib(n-2)+fib(n-1)

```

```
5 return ans
```

La función `fib` se usa a continuación con algunos ejemplos:

```
1 print('fib(0):', fib(0))
2 print('fib(1):', fib(1))
3 print('fib(2):', fib(2))
4 print('fib(10):', fib(10))
5 print('fib(15):', fib(15))
```

El resultado de esta ejecución es el siguiente:

```
1 fib(0): 0
2 fib(1): 1
3 fib(2): 1
4 fib(10): 55
5 fib(15): 610
```

Si la implementación de la función `fib` es tan sencilla y básicamente resulta de transcribir la definición matemática como código, ¿por qué hay que preocuparse por usar programación dinámica? La respuesta es que este algoritmo, para valores de N apenas moderados, no termina en ningún plazo razonable. Los llamados recurrentes se solapan y, en consecuencia, hay valores que se calculan una y otra vez.

Se puede ser preciso al respecto. El llamado `fib(n)` hace exactamente $2 \text{fib}(n+1) - 1$ llamados a `fib` (Ejercicio 2). Para $n = 100$ son alrededor de $1,15 \times 10^{21}$ llamados; aun suponiendo, con enorme optimismo, mil millones de llamados por segundo, el cálculo tardaría del orden de treinta mil años. No se trata de tener paciencia. La complejidad temporal de `fib(n)` es, entonces, $O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$, expresión que resulta de resolver la ecuación característica $X^2 - X - 1 = 0$ asociada a la recurrencia (los detalles de una demostración que soporte esta afirmación se proponen como ejercicio para el lector). Es decir, la complejidad temporal de `fib` es exponencial, lo cual no es una buena noticia si el objetivo es contar con un algoritmo eficiente para calcular `fib`.

La forma en que resulta útil la programación dinámica es precisamente almacenando el resultado de los cálculos, ahorrando tiempo de cómputo cuando sea necesario visitar una y otra vez dichas instancias en cada uno de los subproblemas.

Nota 3.2.1: Memorización y tabulación

Hay dos formas usualmente empleadas para diseñar un algoritmo con programación dinámica y que permiten implementar eficientemente la función recurrente asociada a su solución:

Memorización: se calcula la función objetivo *por demanda* de modo tal que los valores intermedios, resultado de los llamados recurrentes, se van calculando en la medida que sea necesario. Generalmente, resultan algoritmos recurrentes muy parecidos a la función objetivo. Hay una memoria compartida que se usa: para una instancia específica del problema se consulta si ha sido resuelta antes (i.e., el valor está en la memoria compartida), entonces se usa el valor registrado en la memoria; de lo contrario, se calcula el valor correspondiente a dicha instancia, ya bien sea directa o recurrentemente, y se almacena en la memoria compartida para que pueda ser usado posteriormente de ser necesario.

Tabulación: se calcula la función objetivo *exhaustivamente* considerando todos los casos posibles para implementar la función dada con base en los parámetros de interés. La memoria compartida se va “llenando” incrementalmente con todos estos casos hasta obtener el valor deseado. Generalmente, resultan algoritmos iterativos.

En lo que resta de la sección se ilustra cómo usar memorización y tabulación para obtener una implementación eficiente de la función *fib*.

3.2.1. Memorización. La programación dinámica con memorización es conocida en inglés como programación dinámica *top-down*. Básicamente, se extiende la función a implementar con una memoria compartida que puede ser leída y modificada en los llamados recurrentes. Esto, específicamente, puede hacerse con una variable global o aumentando la lista de parámetros de la función con una variable que es pasada por referencia. Una de las ventajas de las implementaciones con memorización es que la estructura del código resultante es básicamente la misma que la de una transcripción de la función objetivo.

La idea general es la siguiente:

- Si el valor que se desea calcular está en la memoria compartida (i.e., ha sido calculado previamente), este valor se usa.
- De lo contrario, se calcula directa o recurrentemente el valor deseado y se almacena en la memoria compartida. En las invocaciones recurrentes para los subproblemas también se hace uso de la memoria compartida de modo tal que los cálculos intermedios queden almacenados en ella.

La implementación de *fib* con memorización se presenta en el Algoritmo 3.2.2 con la función `fib_memo`; un diccionario representa la memoria compartida y es identificado como `mem`.

Algoritmo 3.2.2: Cálculo de fib con memorización

```

1 def fib_memo(n, mem):
2     ans = None
3     if n in mem: ans = mem[n]    # if the value is available, use it!
4     else:          # otherwise, it needs to be computed
5         if n <= 1: ans = n
6         else: ans = fib_memo(n-2, mem) + fib_memo(n-1, mem)
7         mem[n] = ans            # store the value in the shared memory
8     return ans

```

Cuando el valor que se quiere calcular está disponible en `mem`, el algoritmo lo retorna. De lo contrario, el valor se calcula (incluyendo los casos base) y, una vez se cuenta con este valor, se almacena antes de retornarlo. Note que los llamados recurrentes también hacen uso de la memoria compartida: dado que `mem` se pasa por referencia, todos los cálculos que se hagan en los llamados recurrentes quedan almacenados en `mem`.

Es importante observar que la función `fib_memo` se comporta correctamente siempre y cuando se respeten algunas condiciones sobre sus parámetros. En particular, se debe garantizar que `mem` no contiene “basura” en el sentido de que los valores almacenados allí son correctos. De lo contrario, no habría garantía acerca de los cálculos realizados ni del valor retornado. Por ello, es importante contar con condiciones de representación que apoyen el diseño del algoritmo.

$$C_0 : (\forall k \mid k \in \mathbb{N} : k \in \text{mem} \Rightarrow \text{mem}[k] = \text{fib}(k)).$$

La condición C_0 indica que si k es una de las llaves del diccionario, necesariamente dicha llave está asociada al valor $\text{fib}(k)$; es decir, lo que está almacenado en el diccionario (suponiendo que solamente hay números naturales como llaves) son valores correctos de Fibonacci.

Conviene distinguir esta condición de una precondition. La condición C_0 se predica de la memoria compartida y debe valer tanto al entrar como al salir de cada llamado: es la que permite reusar sin verificar lo que está almacenado. Que n sea un número natural, en cambio, es una precondition sobre el parámetro: restringe cómo se puede invocar la función, no cambia durante la ejecución y no dice nada sobre `mem`.

Teorema 3.2.1

Sea $n \in \mathbb{N}$ y sea `mem` un diccionario que satisface la condición C_0 . Entonces el llamado `fib_memo(n, mem)` es tal que:

1. calcula $fib(n)$; y
2. al terminar su ejecución `mem` sigue satisfaciendo C_0 .

Demostración

Se procede por inducción fuerte sobre n , con análisis de casos sobre el cuerpo de `fib_memo`:

- Caso $n \in mem$: no se modifica `mem`, por lo cual C_0 sigue siendo cierto. Como C_0 se supone y $n \in mem$, entonces $mem[n] = fib(n)$. Luego, `fib_memo(n, mem)` calcula $fib(n)$.
- Caso $n \notin mem$: si $n = 0$ o $n = 1$, entonces `ans` toma el valor n , que es $fib(n)$, y la única modificación a `mem` agrega la llave n asociada a ese valor, de modo que C_0 se preserva. Si $n \geq 2$, los dos llamados recurrentes son sobre $n - 2$ y $n - 1$, ambos estrictamente menores que n . Por hipótesis inductiva, el primero calcula $fib(n - 2)$ y deja `mem` satisfaciendo C_0 ; en consecuencia el segundo se hace bajo las suposiciones del enunciado y calcula $fib(n - 1)$, dejando de nuevo C_0 cierto. Luego `ans` toma el valor $fib(n - 2) + fib(n - 1)$, que es $fib(n)$ por definición, y aumentar `mem` con $mem[n] = ans$ preserva C_0 .

Resta por hacer explícito cómo `fib_memo` resuelve el problema planteado inicialmente.

Teorema 3.2.2

El llamado `fib_memo(N, {})`, en donde `{}` representa el diccionario vacío, calcula $fib(N)$.

Demostración

La demostración es un corolario del Teorema 3.2.1 y se propone como ejercicio al lector.

A continuación se muestra el resultado de ejecución de `fib_memo`, incluyendo el cálculo para $n = 100$, y el contenido de la memoria compartida después de este llamado. Note que todos los valores hasta 100 han sido calculados y almacenados en la memoria compartida. Una ventaja práctica de la memorización es que la memoria compartida puede ser calculada incrementalmente: por ejemplo, hacer el llamado

para calcular `fib_memo(15, mem)` y posteriormente hacer el llamado para calcular `fib_memo(100, mem)` de tal forma que en el segundo llamado se cuente con algunos valores previamente (y correctamente) calculados en `mem`.

```

1 print('fib(0):', fib_memo(0, dict()))
2 print('fib(1):', fib_memo(1, dict()))
3 print('fib(2):', fib_memo(2, dict()))
4 print('fib(10):', fib_memo(10, dict()))
5 print('fib(15):', fib_memo(15, dict()))
6
7 mem = dict()
8 print('fib(100):', fib_memo(100, mem))
9 print(mem)

```

El resultado de esta ejecución es el siguiente:

```

1 fib(0): 0
2 fib(1): 1
3 fib(2): 1
4 fib(10): 55
5 fib(15): 610
6 fib(100): 354224848179261915075
7 {0: 0, 1: 1, 2: 1, 3: 2, 4: 3, 5: 5, 6: 8, 7: 13, 8: 21, 9: 34, 10: 55,
8  11: 89, 12: 144, 13: 233, 14: 377, 15: 610, 16: 987, 17: 1597,
9  ...
10 98: 135301852344706746049, 99: 218922995834555169026,
11 100: 354224848179261915075}

```

Finalmente, se establece el orden de la complejidad temporal de `fib_memo`.

Teorema 3.2.3

Sea $n \in \mathbb{N}$. Si el acceso y modificación de `mem` con `mem[n]` toma tiempo $O(\psi(n))$, entonces el llamado `fib_memo(n, mem)` toma tiempo $O(n\psi(n))$.

Demostración

Se propone como ejercicio al lector.

Si el acceso y modificación de diccionarios es de orden constante, entonces la complejidad temporal de `fib_memo` es de orden lineal en n . Su complejidad espacial también lo es: al terminar, la memoria compartida tiene una llave por cada valor entre 0 y n , a lo cual hay que sumar la pila de llamados recurrentes, que alcanza

profundidad n . Es importante resaltar que se está suponiendo algo que en la práctica no es necesariamente cierto: operar números grandes es de orden constante. En realidad, como la función fib crece rápidamente, es fácil encontrar valores de n no muy grandes para los cuales $fib(n)$ sobrepasa la cota de 2^{63} para números enteros en un procesador de 64 bits. En el caso particular de `fib_memo`, Python “esconde” del usuario esta complicación dado que sus enteros básicos son de precisión arbitraria. Este no necesariamente es el caso en otros lenguajes de programación, y deben usarse librerías especiales o implementar las operaciones de enteros grandes partiendo de cero. Con cualquiera de estas opciones, las operaciones aritméticas dejan de ser de orden constante en el peor de los casos.

3.2.2. Tabulación. Al igual que la memorización, la tabulación es una alternativa para diseñar algoritmos con programación dinámica. Esta técnica es conocida en inglés como *bottom-up*. Aunque sirven el mismo propósito, los algoritmos con tabulación son intrínsecamente distintos a los que resultan con memorización. Primero, los algoritmos con tabulación son iterativos por naturaleza. Segundo, con memorización los valores de la función objetivo se calculan por demanda, mientras que con tabulación se pueden calcular más valores intermedios de los que son necesarios para calcular la función para el objetivo final. Tercero, en algunos casos las tabulaciones pueden ser optimizadas para ahorrar espacio. Esta última característica hace que la tabulación sea preferida, en muchas situaciones, sobre la memorización.

A modo de ejemplo, se diseñará un algoritmo por tabulación que permita calcular la función fib . Esto se hará con la ayuda de una variable que almacene el cálculo parcial de los valores de la función hasta obtener el valor deseado.

$$P_0 : (\forall i \mid 0 \leq i < n : tab[i] = fib(i)).$$

$$P_1 : 0 \leq n \leq N + 1.$$

La variable de almacenamiento es el arreglo $tab[0..N + 1]$ que permite guardar valores de fib . En particular, $tab[0..n)$ almacena los primeros n valores de fib . La variable n indica qué valores se han calculado; note que cuando $n = N + 1$, el valor deseado está en $tab[N]$. Visualmente, estos invariantes se pueden representar como se muestra en el *diagrama de necesidades* de la Figura 3.

En una tabulación, el diagrama de necesidades indica cómo se inicializa la memoria adicional (en este caso $tab[0] = 0$ y $tab[1] = 1$) y cómo los valores que se calculan recurrentemente (i.e., $tab[n]$ para $n \geq 2$) se pueden determinar a partir de valores previamente calculados en la memoria (en este caso, $tab[n - 2]$ y $tab[n - 1]$). Note que de acuerdo con el invariante P_0 , los valores de fib están calculados correctamente en $tab[0..n)$, y n varía entre 0 y $N + 1$, inclusive.

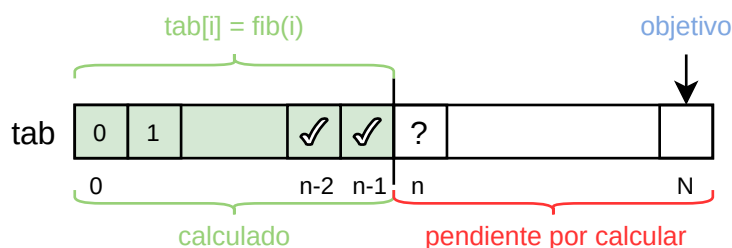


Figura 3. Diagrama de necesidades para la tabulación de *fib*. Las posiciones 0 y 1 de *tab* se inicializan directamente; cada valor posterior se obtiene de los dos inmediatamente anteriores.

Con base en estos invariantes y en el diagrama de necesidades, se propone la función *fib_tab* en el Algoritmo 3.2.3 para calcular iterativamente el valor de la función *fib*.

Algoritmo 3.2.3: Cálculo de fib con tabulación

```

1 def fib_tab(N):
2     ans = None
3     if N<=1: ans = N
4     else:
5         tab,n = [ None for _ in range(N+1) ],2
6         tab[0],tab[1] = 0,1
7         #  $P_0 \wedge P_1$ 
8         while n!=N+1:
9             tab[n],n = tab[n-2]+tab[n-1],n+1
10        ans = tab[N]
11    return ans

```

La función *fib_tab* determina el valor de retorno distinguiendo dos casos. Si $N \leq 1$, que corresponde a los casos base, el valor se calcula directamente. De lo contrario, se crea la memoria auxiliar en *tab*[0..N] inicializada con valores indefinidos; excepto por *tab*[0] y *tab*[1] que son 0 y 1, respectivamente, dado que *fib*(0) = 0 y *fib*(1) = 1. Para cualquier valor a partir de 2, el valor *tab*[n] se calcula sumando los dos valores inmediatamente anteriores en *tab*: por P_0 , estos valores coinciden con *fib*(n-2) y *fib*(n-1), luego se esperaría que *tab*[n] corresponda a *fib*(n) cuando se actualice la tabulación.

Para demostrar que el algoritmo iterativo de tabulación *fib_tab* funciona, es necesario demostrar que el ciclo satisface los invariantes P_0 y P_1 .

Teorema 3.2.4

El ciclo en `fib_tab` satisface los invariantes P_0 y P_1 . Es decir,

1. P_0 y P_1 son ciertos antes de la (posible) primera iteración del ciclo.
2. P_0 y P_1 son ciertos antes de una iteración del ciclo, entonces son ciertos después de una iteración del ciclo.

Demostración

Las propiedades se demostrarán para P_0 ; las demostraciones para P_1 se proponen como ejercicios para el lector.

1. Antes de la primera iteración del ciclo se tiene $n = 2$, de modo que P_0 exige únicamente que $tab[0] = fib(0)$ y que $tab[1] = fib(1)$; es decir, que $tab[0] = 0$ y $tab[1] = 1$. Eso es exactamente lo que hace la inicialización, luego P_0 es cierto antes de la primera iteración del ciclo.
2. Suponga que P_0 es cierto; se desea demostrar que P_0 es cierto después de una iteración del ciclo (es decir, se supone que P_0 es cierto para n y se demuestra que es cierto para $n + 1$). Note que al suponer que P_0 es cierto antes de una iteración del ciclo, se está suponiendo que $tab[0..n]$ tiene los primeros n valores de fib . En el cuerpo del ciclo se modifican $tab[n]$ y n con los valores $tab[n - 2] + tab[n - 1]$ y $n + 1$, respectivamente; como la asignación es simultánea, los lados derechos se evalúan antes de modificar nada y, por lo tanto, $tab[n - 2]$ y $tab[n - 1]$ son los valores que cubre la suposición. Además, el ciclo solo se ejecuta con $n \geq 2$, así que vale $fib(n) = fib(n - 2) + fib(n - 1)$. Se concluye que $tab[n] = fib(n)$ y, como n incrementa en una unidad, es cierto que:

$$(\forall i \mid 0 \leq i < n + 1 : tab[i] = fib(i)).$$

Con base en el Teorema 3.2.4 es fácil demostrar que la función `fib_tab` es correcta con respecto a la especificación del Problema 3.2.1.

Teorema 3.2.5

El llamado `fib_tab(N)` calcula $fib(N)$.

Demostración

Recuerde que $N \in \mathbb{N}$, como se indica en la formulación del problema. La demostración de terminación del algoritmo se propone como ejercicio al lector. Si $N = 0$ o $N = 1$, entonces el algoritmo responde 0 o 1, respectivamente. De lo contrario (i.e., $N \geq 2$), suponga que la ejecución del ciclo termina para cualquier $N \geq 2$. Note que esto sucede cuando $n = N + 1$. En este caso, el invariante P_0 es equivalente a:

$$(\forall i \mid 0 \leq i < N + 1 : \text{tab}[i] = \text{fib}(i)).$$

En particular, $\text{tab}[N] = \text{fib}(N)$. Es decir, en el caso $N \geq 2$, `fib_tab(N)` responde con el valor de $\text{fib}(N)$.

En cualquiera de los casos `fib_tab(N)` calcula el valor de $\text{fib}(N)$.

Los cálculos de las complejidades temporal y espacial de `fib_tab` se proponen como ejercicios al lector (¿cuáles son estas complejidades?).

Hay una versión iterativa para calcular *fib* con tabulación que toma tiempo $O(N)$ y espacio $O(1)$. Diseñar este algoritmo se propone como ejercicio al lector (note que en el caso recurrente solo depende de dos valores).

3.2.3. Comparación. Las dos técnicas presentadas en esta sección calculan lo mismo y con la misma complejidad temporal, pero no son intercambiables. La siguiente tabla resume sus diferencias.

	Memorización	Tabulación
En inglés	<i>top-down</i>	<i>bottom-up</i>
Forma del algoritmo	recurrente	iterativo
Valores que calcula	solo los necesarios	todos los del rango
Relación con la definición	transcripción casi literal	exige un orden de llenado
Memoria compartida	diccionario	arreglo o matriz
Reducción del espacio	difícil	con frecuencia posible

Las dos primeras filas explican por qué la memorización suele ser el primer diseño que se intenta: se obtiene casi copiando la definición recurrente. La tercera señala su ventaja propia, que es no calcular lo que no se necesita; hay problemas en los cuales el rango de la función es enorme pero solo una fracción pequeña de sus valores interviene, y allí la tabulación desperdiciaría todo el trabajo restante.

La última fila es la que más pesa en este capítulo. Una vez fijado el orden de llenado de una tabulación, es posible preguntarse qué parte de la tabla sigue haciendo falta y descartar el resto; con memorización eso es mucho más difícil, porque los valores se piden en un orden que depende de la instancia y no se sabe cuándo deja de necesitarse uno. El ejercicio que acaba de proponerse –calcular *fib* en espacio $O(1)$ – es el caso más simple de esa idea, que reaparece en la Sección 3.4,

en la Sección 3.6, en la Sección 3.7 y en la Sección 3.8, cada vez con una forma distinta. Esa es la razón por la cual la reducción del espacio es un paso de la Metodología 3.3.1.

Ejercicios

1. Al estudiar la función `fib`, se afirmó que su complejidad temporal es

$$O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right),$$

en donde n es valor que se desea calcular. También se indicó que esta expresión resulta de resolver la ecuación de diferencia $X^2 - X - 1 = 0$ con variable X .

- a) Investigue sobre ecuaciones de recurrencia y explique de dónde resulta la ecuación dada.
 - b) Investigue sobre técnicas para resolver ecuaciones de recurrencia y aplique una de ellas para demostrar que la complejidad temporal de `fib(n)` es $O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$.
2. Demuestre que el llamado `fib(n)` hace exactamente $2\text{fib}(n+1) - 1$ llamados a `fib`. Concluya que la cantidad de llamados es impar para todo n y calcule cuántos se hacen para $n = 100$.
 3. Demuestre que el llamado `fib_memo(n, dict())` hace exactamente $2n - 1$ llamados a `fib_memo` cuando $n \geq 1$. Compare esta cantidad con la del ejercicio anterior.
 4. Demuestre el Teorema 3.2.2.
 5. Demuestre el Teorema 3.2.3. Enuncie y demuestre además la afirmación correspondiente sobre la complejidad espacial de `fib_memo`, teniendo en cuenta tanto la memoria compartida como la pila de llamados recurrentes.
 6. En el Teorema 3.2.2 se concluye que el llamado `fib_memo(N, {})`, en donde `{}` representa el diccionario vacío, calcula $\text{fib}(N)$. Proponga (justificando su respuesta) otra forma de invocar `fib_memo` para que calcule $\text{fib}(N)$.
 7. Demuestre que el ciclo en `fib_tab` satisface el invariante P_1 .
 8. Demuestre que `fib_tab` termina.
 9. Para $N \in \mathbb{N}$, demuestre:
 - a) La complejidad temporal de `fib_tab(N)` es $O(N)$.
 - b) La complejidad espacial de `fib_tab(N)` es $O(N)$.
 10. Diseñe un algoritmo iterativo que use la técnica de programación dinámica con tabulación que permita calcular la función fib en tiempo $O(N)$ y espacio

$O(1)$. Formule invariantes y demuestre que el algoritmo es correcto. ¿Cuál es el diagrama de necesidades?

11. Un *coeficiente binomial* es un número entero positivo que corresponde a la expansión polinomial de la potencia de un binomio. Para $n, k \in \mathbb{N}$,

$$\binom{n}{k}$$

es el coeficiente del término x^k en la expansión de $(1+x)^n$. El valor numérico de la expresión $\binom{n}{k}$, leída “de n se toman k ”, corresponde a la cantidad de subconjuntos de k elementos de un conjunto de n elementos (si $k > n$, este valor es 0). Los coeficientes binomiales satisfacen la siguiente recurrencia:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

Con base en esta definición recurrente:

- Explique por qué la programación dinámica es útil para resolver el problema de calcular coeficientes binomiales.
- Diseñe un algoritmo recurrente que calcule $\binom{n}{k}$ usando memorización.
- 🌈 Diseñe un algoritmo iterativo que calcule $\binom{n}{k}$ usando tabulación.

3.3. Una metodología

Con base en las ideas de la Sección 3.1 y en el ejemplo desarrollado en la Sección 3.2, se propone una metodología para diseñar algoritmos con programación dinámica a partir de la especificación de un problema. No debe entenderse como una receta infalible, sino como una lista de las preguntas que conviene responder y en qué orden.

Se anuncia de una vez cuál de los pasos concentra la dificultad. No es la implementación, que suele ser breve, ni el análisis, que suele ser inmediato: es el planteamiento de la función objetivo. Encontrar qué generalizar y con cuántos parámetros es lo que decide si la técnica funciona, y es también lo que la Sección 3.1 anticipó al señalar que las propiedades de subestructura y solapamiento se predicaban de un problema junto con una descomposición. Escoger la función objetivo es escoger esa descomposición.

Metodología 3.3.1: Programación dinámica

Especificación: especificar el problema, indicando claramente su entrada y su salida. De esa lectura debe quedar claro si lo que se desea es decidir, contar u optimizar, pues de ello depende cuál de las propiedades de la Sección 3.1 hay que verificar: la subestructura general de la Nota 3.1.1 en

los dos primeros casos y la subestructura óptima de la Definición 3.1.1 en el tercero.

Función objetivo: formular una función objetivo ϕ que generalice la salida del problema o que, en su defecto, permita calcularla. Comúnmente se obtiene al generalizar los parámetros del problema que hacen parte de la salida de la especificación, y puede involucrar uno o más parámetros dependiendo del caso. Es importante que en este punto ϕ cuente con una descripción clara en palabras y *no* con una definición formal. La razón es que esa descripción es la especificación de ϕ y la definición recurrente que vendrá después es su diseño: tenerlas por separado es lo que permite demostrar que la segunda cumple la primera. Si se escribe primero la recurrencia, no queda nada contra qué contrastarla.

Reformulación de la especificación: verificar que la salida del problema se puede reescribir en términos de ϕ , de manera directa o indirecta. En algunos problemas basta con evaluar ϕ en los parámetros de la instancia; en otros hace falta además una cuantificación sobre sus valores. Si la salida no se deja reescribir de ninguna de las dos maneras, hay que volver al paso anterior y proponer otra función.

Planteamiento recurrente: presentar una definición recurrente de ϕ . Esta definición es la base de la programación dinámica y, como tal, debe ser completa, cubriendo todas las instancias, y no ser ambigua. Debe además decrecer en una medida bien fundada, en el sentido de la Sección 3.1, para que el cálculo termine.

Correctitud de la función objetivo: demostrar que la definición recurrente calcula lo que la descripción en palabras dice. Este paso es el que verifica que el problema tiene subestructura con respecto a la descomposición escogida, y su forma habitual se discute en la subsección siguiente.

¿Memorización o tabulación?: decidir cómo se instrumenta la recurrencia. La tabla de la Sección 3.2 resume las diferencias entre las dos opciones; en la práctica, conviene memorizar cuando solo hace falta una fracción pequeña de los valores de ϕ o cuando la recurrencia es difícil de ordenar, y tabular en los demás casos, sobre todo si se aspira a reducir el espacio. Dependiendo de la elección:

- Con memorización, determinar qué estructura de datos representa la memoria compartida y formular las *condiciones de representación* que debe satisfacer, como se hizo con C_0 en la Sección 3.2. Conviene además estimar la profundidad de la recurrencia, que en algunos lenguajes de programación puede provocar errores de ejecución.

- Con tabulación, diseñar un *diagrama de necesidades* que muestre de qué celdas depende cada celda, como el de la Figura 3. El diagrama fija el orden de llenado y de él se leen los invariantes de los ciclos.

Reducción del espacio (opcional): si se escogió tabular, mirar el diagrama de necesidades y preguntarse qué parte de la tabulación se puede descartar sin perder lo que hará falta después. Formular los invariantes de la versión reducida, que típicamente describen una memoria en la cual conviven valores del presente y del pasado.

Reconstrucción de una solución óptima (opcional): si el problema pide exhibir una solución y no solo su valor, decidir cómo recuperarla. La tabulación guarda, además del óptimo, el rastro de las decisiones que lo produjeron y recorrerla en sentido contrario permite reconstruirlas. Tenga presente que este paso puede reñir con el anterior.

Implementación: instanciar el Algoritmo 3.3.1 si se escogió memorización o escribir los ciclos que dicta el diagrama de necesidades si se escogió tabulación, y demostrar que satisfacen los invariantes formulados.

Análisis: establecer las complejidades temporal y espacial. Con tabulación, el tiempo suele ser el tamaño de la tabla multiplicado por el costo de llenar una celda; con memorización, la cantidad de llaves distintas multiplicada por ese mismo costo. Verifique que la cota resultante sea polinomial en el tamaño de la entrada y no solamente en los valores numéricos que aparecen en ella.

Los pasos anteriores se ilustrarán con cada uno de los ejemplos que se presentan en el resto del capítulo. De hecho, la Sección 3.2 ya siguió varios de ellos, aunque sin nombrarlos.

Paso	En la Sección 3.2
Especificación	Problema 3.2.1
Función objetivo	<i>fib</i> , que viene dada
Reformulación	la salida es <i>fib</i> (<i>N</i>)
Planteamiento recurrente	Definición 3.2.1
Correctitud	inmediata: la recurrencia es la función
¿Memorización o tabulación?	las dos, a modo de comparación
Reducción del espacio	se propone como ejercicio
Reconstrucción	no aplica
Implementación	algoritmos 3.2.2 y 3.2.3
Análisis	Teorema 3.2.3 y ejercicios

La fila de correctitud merece un comentario. En la Sección 3.2 no hubo nada que demostrar porque la función objetivo *era* la definición recurrente: no había una descripción en palabras separada de la fórmula. Ese es el caso excepcional y por eso esa sección sirve para presentar las técnicas de instrumentación, pero no la metodología completa. En todas las secciones siguientes las dos cosas son distintas y la demostración es indispensable.

3.3.1. El esquema general. Los algoritmos con memorización de este capítulo son todos el mismo algoritmo con distintos rellenos. Conviene, entonces, dejarlo escrito una vez.

Algoritmo 3.3.1: Esquema general de memorización

Para una función objetivo ϕ y una memoria compartida mem indexada por los parámetros p de ϕ , el cálculo de $\phi(p)$ procede así:

1. Si p es una llave de mem , retornar $mem[p]$.
2. De lo contrario:
 - a) si p corresponde a un caso base, calcular el valor directamente;
 - b) de lo contrario, invocar recurrentemente el esquema sobre cada una de las subinstancias que exige la definición de ϕ y combinar los resultados según esa definición;
 - c) almacenar el valor obtenido en $mem[p]$ y retornarlo.

Tres observaciones sobre el Algoritmo 3.3.1.

La estructura es la de ϕ : Quitando el paso 1 y el paso 2(c), lo que queda es la transcripción literal de la definición recurrente. Esa es la ventaja de la memorización y la razón por la cual conviene ensayarla primero: el algoritmo se obtiene casi sin decisiones de diseño.

La memoria nunca se corrige: Un valor almacenado en el paso 2(c) no vuelve a modificarse. Por eso basta con una condición de representación que diga que todo lo almacenado es correcto y por eso el paso 1 puede confiar en lo que encuentra sin verificar nada. Si la definición de ϕ es ambigua o incompleta, esta confianza se pierde y el esquema deja de funcionar.

La profundidad es la de la recurrencia: El esquema consume pila proporcional a la longitud de la cadena más larga de subinstancias, aunque la cantidad de llaves distintas sea pequeña. Esa es una diferencia real con la tabulación, que es iterativa, y es la responsable de los errores de ejecución que se mencionan en el paso correspondiente de la metodología.

3.3.2. El patrón de las dos desigualdades. El paso de correctitud de la función objetivo es el que exige un argumento propio en cada problema. Lo que no cambia de un problema a otro es la forma del argumento y vale la pena describirla una vez.

Sea ϕ la definición recurrente propuesta y sea L el valor que la descripción en palabras dice que ϕ debe calcular. Lo que hay que demostrar es “ $\phi = L$ ”, y eso se hace por inducción sobre la medida que decrece, separando la igualdad en dos desigualdades. Cada una corresponde a una de las dos mitades de la Definición 3.1.1:

Composición: cada alternativa que la recurrencia considera corresponde a una solución legítima de la instancia. En un problema de minimización esto da “ $\phi \geq L$ ” y en uno de maximización “ $\phi \leq L$ ”. El argumento consiste en construir, a partir de las soluciones óptimas de las subinstancias, una solución de la instancia, y verificar que es factible y que su valor es el que la recurrencia predice. Es la mitad “obvia” y la que se olvida con más frecuencia.

Descomposición: ninguna solución se le escapa a la recurrencia. Da la desigualdad contraria. El argumento consiste en tomar una solución óptima de la instancia, partirla según la descomposición escogida y reconocer sus pedazos como soluciones de las subinstancias, para luego invocar la hipótesis inductiva. Es la mitad “trabajosa”, y es donde el cuantificador de la Definición 3.1.1 se vuelve delicado: si la propiedad solo vale en forma existencial, no se puede partir de una solución óptima cualquiera sino que hay que transformarla primero.

Reconocer este patrón ahorra trabajo. Al leer las demostraciones de las secciones siguientes conviene identificar, en cada una, cuál de las dos mitades se está verificando: la Sección 3.5, la Sección 3.7 y la Sección 3.8 demuestran las dos desigualdades de manera explícita, y en la Sección 3.7 se ve con claridad el caso en el cual hay que transformar una solución óptima antes de poder partirla.

Ejercicios

1. El problema de multiplicación de una secuencia de matrices consiste en minimizar la cantidad de operaciones básicas (sumas y multiplicaciones escalares, básicamente) para multiplicar $N \geq 0$ matrices. Este algoritmo es uno de los ejemplos clásicos de programación dinámica.
 - a) Investigue sobre este problema y formule su especificación.
 - b) Investigue sobre su solución con programación dinámica y, paso a paso, identifique el uso en ella de la metodología propuesta en esta sección. Indique claramente en qué consiste cada uno de los pasos y, en caso tal de que alguno de los pasos no esté presente, justifique esta situación.
2. El problema del corte de un lingote de oro (en inglés conocido como *rod cutting*) consiste en maximizar la ganancia que se puede obtener de cortar verticalmente el lingote en varios pedazos (para venderlos individualmente).

Entrada: La longitud $N \geq 0$ de un lingote y una tabla $T[1..N]$ especificando la ganancia a obtener por cada tamaño vendido.

Salida: Máxima ganancia que se puede obtener vendiendo el lingote de tamaño N , posiblemente en varios pedazos, en relación con la tabla $T[1..N]$.

Considere la siguiente función objetivo, para $0 \leq n \leq N$:

$\phi(n)$: “Máxima ganancia que se puede obtener vendiendo el lingote de tamaño n , posiblemente en varios pedazos, en relación con la tabla $T[1..N]$ ”.

Como definición de ϕ , se propone la siguiente función recurrente para $0 \leq n \leq N$:

$$\phi(n) = \begin{cases} 0 & , \text{ si } n = 0, \\ (\uparrow i \mid 1 \leq i \leq n : T[i] + \phi(n - i)) & , \text{ si } n \neq 0. \end{cases}$$

El caso base indica que no hay ganancia alguna cuando no hay nada que vender. El caso recurrente indica que se escoge aquella opción que resulta de cortar el lingote en un pedazo de tamaño i y lo mejor que se pueda hacer con el tamaño restante $n - i$ (¿por qué esto cubre todos los casos?).

Con base en este planteamiento, siga la metodología propuesta en esta sección para diseñar:

- a) Una demostración de que ϕ calcula lo que su descripción dice, siguiendo el patrón de las dos desigualdades. Ilustre con un ejemplo la propiedad de solapamiento.
- b) Un algoritmo con memorización para ϕ , instanciando el Algoritmo 3.3.1.

- c) Un algoritmo con tabulación para ϕ (¿se puede hacer reducción de la tabulación?)
- d) Un algoritmo que, además del valor óptimo, retorne los tamaños de los pedazos en que conviene cortar el lingote.

En cada caso debe resultar código en Python ejecutable.

3. En la tabla de esta sección, la fila de correctitud dice que en la Sección 3.2 no hubo nada que demostrar. Proponga para *fib* una descripción en palabras que no sea la propia definición recurrente –por ejemplo, la que resulta del problema de los conejos– y demuestre, con el patrón de las dos desigualdades, que la Definición 3.2.1 la calcula.

3.4. Suma máxima de un subarreglo

El problema del subarreglo de suma máxima consiste en calcular la máxima suma posible de un subarreglo en un arreglo de números dado.

Problema 3.4.1: Subarreglo de suma máxima

Entrada: Un arreglo $A[0..N)$, con $N \geq 0$, de números.

Salida: Suma máxima de un subarreglo de $A[0..N)$.

Este ha sido un problema estudiado exhaustivamente desde su formulación en 1977 por Ulf Grenander (inicialmente se planteó para una matriz y no para un arreglo unidimensional). En el procesamiento de imágenes, por ejemplo, este problema tiene aplicación en la identificación de áreas brillantes en mapas de bits. También tiene aplicaciones en biología computacional y en finanzas.

Ejemplo 3.4.1

Considere el arreglo $A = [-1, 3, -2, 1, 4, -2, 1, 0]$. El subarreglo de suma máxima es $A[1..5) = [3, -2, 1, 4]$ con suma 6.

Este problema se puede resolver trivialmente para algunas instancias. Por ejemplo, si el arreglo dado únicamente tiene números no negativos, la respuesta es la suma de sus elementos. También, si todos sus elementos son números negativos, la respuesta es 0, pues el arreglo vacío es subarreglo de cualquier arreglo y tiene suma 0. El caso general se puede resolver exhaustiva pero ineficientemente, como se presenta en el Algoritmo 3.4.1.

Algoritmo 3.4.1: Fuerza bruta

1. Para cada pareja (i, j) que satisface $0 \leq i \leq j \leq N$, determinar la suma de $A[i..j]$.
2. Retornar aquella suma que sea máxima.

La complejidad temporal del Algoritmo 3.4.1 es $O(N^3)$ pues calcular todas las parejas (i, j) toma tiempo $O(N^2)$ y para cada una de ellas la suma de $A[i..j]$ toma tiempo $O(N)$. Esta complejidad se puede reducir a $O(N^2)$ con un pre-proceso que toma tiempo y espacio $O(N)$. En particular, se pueden calcular las sumas de los prefijos (o sufijos) de $A[0..N]$ de tal forma que la suma de cada subarreglo $A[i..j]$ se obtiene en tiempo $O(1)$, reduciendo de $O(N^3)$ a $O(N^2)$ el tiempo que toma la solución global.

Aplicando la técnica dividir, conquistar y combinar se puede obtener una solución más eficiente (o menos ineficiente).

Algoritmo 3.4.2: Dividir, conquistar y combinar

Suponga que low y hi son tales que $0 \leq low \leq hi \leq N$. El objetivo de este algoritmo es calcular la suma máxima de un subarreglo en $A[low..hi]$.

1. Si $low = hi$, entonces retornar $-\infty$.
2. Si $low + 1 = hi$, entonces retornar $A[low]$.
3. De lo contrario:
 - a) sea mid un punto medio entre low y hi ,
 - b) recurrir sobre $A[low..mid]$ para determinar la máxima suma de un subarreglo de $A[low..mid]$,
 - c) recurrir sobre $A[mid..hi]$ para determinar la máxima suma de un subarreglo de $A[mid..hi]$,
 - d) determinar la suma máxima de un subarreglo de $A[low..hi]$ que incluye el valor en mid (i.e., que cruza de izquierda a derecha), y
 - e) retornar el máximo valor obtenido en los pasos (3b),(3c),(3d).

El rango vacío del primer paso retorna $-\infty$, que es la identidad del máximo, porque sobre un rango vacío no hay ningún subarreglo no vacío que considerar. Si se desea incluir el subarreglo vacío entre las opciones, basta con maximizar con 0 el valor que retorna el algoritmo.

Suponiendo que el Algoritmo 3.4.2 es correcto (es un ejercicio para el lector), su llamado con $low = 0$ y $hi = N$ resuelve el problema del subarreglo de suma máxima en $A[0..N]$. Además, si el paso (3d) se calcula en tiempo lineal en función

de $hi - low$, con peor caso $O(N)$, entonces la cantidad de operaciones básicas que el algoritmo realiza para resolver el problema está dada por la recurrencia $T(N)$ definida en su caso recurrente por

$$T(N) = 2T(N/2) + O(N).$$

Es decir, bajo la suposición hecha, el Algoritmo 3.4.2 toma tiempo $O(N \log N)$ en resolver el problema de suma máxima de un subarreglo. Su complejidad espacial es $O(\log N)$, que corresponde a la profundidad de la recurrencia; la del Algoritmo 3.4.1 es $O(1)$ o bien $O(N)$ en la versión con sumas de prefijos.

Si se cuenta con un algoritmo relativamente eficiente que resuelve el problema sin usar programación dinámica, ¿qué hace este problema en este capítulo? Pues bien, lo que sucede es que se puede diseñar una solución que toma tiempo $O(N)$ si se usa programación dinámica. Ese es el objetivo de lo que resta de esta sección: diseñar dicha solución.

Nota 3.4.1: El algoritmo de Kadane

Los tres algoritmos que esta sección presenta reproducen, en orden, la historia del problema. Grenander lo planteó en 1977, en el contexto de la estimación de máxima verosimilitud de patrones en imágenes digitalizadas y dio para él una solución $O(N^2)$ con sumas de prefijos. Michael Shamos oyó del problema y en una sola noche produjo la solución $O(N \log N)$ por dividir, conquistar y combinar. Poco después, Shamos expuso el problema y su historia en un seminario en Carnegie Mellon; entre los asistentes estaba el estadístico Jay Kadane, quien diseñó la solución lineal en menos de un minuto. Por eso esa solución recibe el nombre de *algoritmo de Kadane* y es el diseño al cual se llega al final de esta sección.

La anécdota se conoce sobre todo por una columna de J. Bentley de 1984, escrita unos cinco años después del seminario, lo cual explica que a veces se le atribuya esa fecha. El propio Kadane relató los hechos en 2023 (ver las notas al final del capítulo).

Que el algoritmo sea asintóticamente óptimo no es una conjetura sino un hecho demostrable. Cualquier algoritmo que resuelva el problema debe leer los N elementos del arreglo: si alguno no se lee, se puede modificar ese elemento hasta cambiar la respuesta sin que el algoritmo se entere. Luego $\Omega(N)$ es una cota inferior y el algoritmo de Kadane la alcanza.

El mismo algoritmo se ha obtenido por caminos muy distintos del que sigue esta sección. D. Gries lo derivó en 1982 aplicando la estrategia de desarrollo de programas a partir de invariantes y R. Bird lo obtuvo en 1989 por manipulación algebraica del algoritmo de fuerza bruta.

El caso de la suma máxima de un subarreglo en un arreglo vacío es trivial, como se ha explicado anteriormente. Igualmente, considerar el arreglo vacío como uno de los posibles subarreglos (aún si el arreglo dado no es vacío) resulta en restringir la salida de la solución a números no negativos. Los casos interesantes se presentan, en realidad, al fijar la atención en aquellos subarreglos que tienen al menos un elemento. El desarrollo que se presenta a continuación se enfoca, entonces, en determinar la suma máxima de un subarreglo no vacío, dando por descontado el caso en el cual $A[0..N)$ tiene tamaño $N = 0$. Posteriormente, se presentará la solución para el caso más general en el cual el arreglo puede ser vacío.

Un arreglo de suma máxima (no vacío) termina en alguno de los índices de $A[0..N)$. Es decir, para algún índice $0 \leq n < N$, un subarreglo de suma máxima debe terminar en n (i.e., incluyendo a $A[n]$ como último elemento de la suma de elementos contiguos). Si se contara con la información de las mejores sumas que terminan en cada uno de los índices $0 \leq n < N$, entonces el problema de determinar la mejor suma de un subarreglo (no vacío) en $A[0..N)$ estaría resuelto: bastaría con maximizar sobre todos estos valores. Esta es la estrategia de diseño que se adopta para resolver el problema en cuestión. Se sigue la Metodología 3.3.1 para diseñar una solución de programación dinámica con tabulación; el paso de especificación ya quedó cumplido con el Problema 3.4.1.

Función objetivo. Para $0 \leq n < N$, se define:

$\phi(n)$: “suma máxima de un subarreglo de $A[0..N)$ que termina en n .”

La función ϕ tiene como dominio los índices de A y co-dominio a todos los números. Una forma alternativa (y equivalente) de interpretar ϕ es la siguiente: $\phi(n)$ es la suma máxima de un sufijo no vacío de $A[0..n]$.

Reformulación de la especificación. Note que, con base en ϕ , se puede especificar la salida del problema (restringido) como la siguiente maximatoria:

$$(\uparrow n \mid 0 \leq n < N : \phi(n)).$$

Si por convención se adopta que esta maximatoria es 0 cuando $N = 0$, entonces esta expresión también abarca el caso en el cual el arreglo dado es vacío. Es más, si también se desea considerar el subarreglo vacío como opción entre los subarreglos de A , basta con extender la maximatoria de la siguiente manera (¿por qué?):

$$(\uparrow n \mid 0 \leq n < N : \phi(n)) \uparrow 0.$$

En cualquier caso, es evidente que la función objetivo permite reformular el problema de interés.

Planteamiento recurrente. El objetivo es obtener una definición recurrente para ϕ . Siguiendo los principios de dividir, conquistar y combinar, el primer paso es identificar aquellos casos que se pueden resolver directamente sin necesidad de

recurrir (i.e., conquistar). Dada la definición de la función objetivo, ese caso corresponde a $\phi(0)$: ¿cuál es la suma máxima de un subarreglo de $A[0..N]$ que termina en 0? Solo $A[0..0]$ satisface esta condición. Es decir, $\phi(0) = A[0]$. ¿Qué sucede cuando $n \neq 0$? Para responder esta pregunta, suponga que $A[k..n]$ es un arreglo de suma máxima que inicia en algún $0 \leq k \leq n$ y termina en n . Es decir, el objetivo es que $\phi(n)$ tenga como valor la suma de los elementos de este subarreglo. Si $k = n$, entonces esta suma máxima es $A[n]$. Si $k \neq n$ (i.e., $k < n$), entonces esta suma máxima tiene la forma $A[k] + \dots + A[n-1] + A[n]$ o, de manera equivalente, $\phi(n-1) + A[n]$ (¿por qué? Ayuda: pensar en la propiedad de la subestructura óptima).

Con base en el análisis anterior, se propone la siguiente definición para ϕ , con $0 \leq n < N$ y $N \neq 0$:

$$\phi(n) = \begin{cases} A[0] & , \text{ si } n = 0, \\ A[n] \uparrow (A[n] + \phi(n-1)) & , \text{ si } n \neq 0. \end{cases}$$

El razonamiento anterior es una *derivación*: sirve para descubrir la recurrencia, no para justificarla. En él se usó ϕ con su significado pretendido –“la suma máxima que termina en n ”– antes de que hubiera una definición que lo garantizara, lo cual es legítimo mientras se esté explorando pero no como demostración. La justificación viene ahora, y sigue el patrón de las dos desigualdades de la Sección 3.3.

Correctitud de la función objetivo. La correctitud de la definición recurrente de ϕ se presenta como el Teorema 3.4.1.

Teorema 3.4.1

Sea $0 \leq n < N$. Si $N \neq 0$, entonces $\phi(n)$ es la suma máxima de un subarreglo de $A[0..N]$ que termina en n .

Demostración

Sea $L(n)$ la suma máxima de un subarreglo no vacío de $A[0..N]$ que termina en n ; es decir, el máximo de las sumas de los subarreglos de la forma $A[k..n]$ con $0 \leq k \leq n$. El objetivo es demostrar que $\phi(n) = L(n)$ y se procede por inducción sobre n .

Caso base $n = 0$: el único subarreglo no vacío que termina en 0 es $A[0..0]$, cuya suma es $A[0]$. Luego $L(0) = A[0] = \phi(0)$.

Caso inductivo $n \neq 0$: suponga que $\phi(n-1) = L(n-1)$. Se demuestran las dos desigualdades por separado.

- *Composición:* $\phi(n) \leq L(n)$. Basta ver que los dos argumentos del máximo en la definición de $\phi(n)$ son sumas de subarreglos que terminan en n . El primero, $A[n]$, es la suma de $A[n..n]$, luego $A[n] \leq L(n)$. Para el segundo, sea $A[k..n-1]$ un subarreglo de suma máxima que termina en $n-1$, de modo que su suma es $L(n-1)$; entonces $A[k..n]$ es un subarreglo que termina en n y su suma es $L(n-1) + A[n]$, de donde

$$A[n] + \phi(n-1) = A[n] + L(n-1) \leq L(n),$$

usando la hipótesis inductiva. Como los dos argumentos no exceden a $L(n)$, tampoco lo hace su máximo.

- *Descomposición:* hay que ver que $\phi(n) \geq L(n)$. Tome un subarreglo $A[k..n]$ de suma máxima que termine en n , con $0 \leq k \leq n$. Si $k = n$, entonces $L(n) = A[n] \leq \phi(n)$, pues $A[n]$ es uno de los dos argumentos del máximo. Si $k < n$, entonces $A[k..n-1]$ es un subarreglo no vacío que termina en $n-1$ y, por lo tanto, su suma no excede a $L(n-1)$. Entonces se tiene:

$$\begin{aligned} L(n) &= (A[k] + \cdots + A[n-1]) + A[n] && \text{(aritmética)} \\ &\leq L(n-1) + A[n] && \text{(observación)} \\ &= \phi(n-1) + A[n] && \text{(hipótesis inductiva)} \\ &\leq \phi(n) && \text{(uno de los argumentos).} \end{aligned}$$

En cualquier caso, $\phi(n) = L(n)$.

Con base en el planteamiento recurrente de la función objetivo, es fácil ver con ejemplos que existen solapamientos entre diferentes instancias de la solución. Se propone al lector, como ejercicio, encontrar instancias para las cuales esto pasa y explicar en qué consisten los solapamientos con esos casos en particular.

¿Memorización o tabulación? Se prefiere, dado que uno de los objetivos del diseño de la solución es ahorrar espacio, implementar la técnica de programación dinámica para la función objetivo con tabulación.

De acuerdo con la definición recurrente de ϕ , se propone el diagrama de necesidades de la Figura 4. En el diagrama se usa la memoria de tabulación $tab[0..N)$

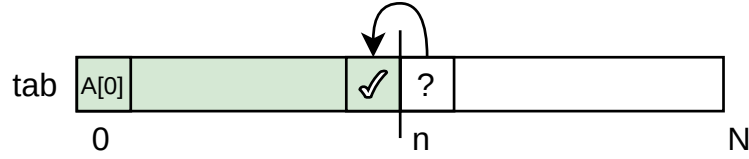


Figura 4. Diagrama de necesidades para la tabulación de ϕ en el problema del subarreglo de suma máxima. El valor $tab[n]$ depende únicamente de $tab[n-1]$.

para almacenar los valores de la función y una variable n para indicar el índice de la tabla que se debe calcular en un momento dado. Esta tabla en la posición 0 es inicializada con $A[0]$, como lo indica la definición de ϕ . Para calcular el valor para $n \geq 1$ (i.e., en el caso recurrente) en la tabla se depende directamente del valor inmediatamente anterior (esto sucede porque $\phi(n)$ depende de $\phi(n-1)$). Esto quiere decir que el valor en $tab[n-1]$ debe calcularse antes de intentar calcular $tab[n]$. Estas observaciones son importantes para diseñar el algoritmo que llena la tabla con los valores de ϕ .

El diseño del algoritmo iterativo para llenar la memoria de tabulación se basa en los siguientes invariantes:

$$P_0 : (\forall i \mid 0 \leq i < n : tab[i] = \phi(i)).$$

$$P_1 : 0 \leq n \leq N.$$

El invariante P_0 indica que el subarreglo $tab[0..n)$ tiene, en orden, los primeros n valores de ϕ : el valor en la i -ésima posición de tab almacena el valor $\phi(i)$. El segundo invariante indica en qué rango toma valores la variable n . Note que se pudo restringir a 1 como límite inferior para n , pero no es necesario. Con estos invariantes la estrategia está clara: llenar tab de izquierda a derecha. Con esta tabla completamente llena es posible, entonces, resolver el problema de determinar la suma máxima de los subarreglos de $A[0..N)$ al calcular el valor máximo en tab . Esto último se hará con una segunda pasada sobre tab , usando el siguiente invariante adicional.

$$P_2 : ans = (\uparrow i \mid 0 \leq i < n : \phi(i)) \uparrow 0.$$

Implementación. Con base en este diseño, se propone el Algoritmo 3.4.3.

Algoritmo 3.4.3: Tabulación para ϕ

```

1 def mss(A):
2     ans, N = 0, len(A)
3     if N != 0:
4         tab = [ None for _ in range(N) ]

```



```

5     tab[0] = A[0]
6     #  $P_0 \wedge P_1$ 
7     for n in range(1, N):
8         tab[n] = max(A[n], tab[n-1]+A[n])
9         #  $P_0 \wedge P_1 \wedge P_2$ 
10    for n in range(N):
11        ans = max(ans, tab[n])
12    return ans

```

El caso en el cual el arreglo dado es vacío se trata de manera especial con ayuda de una instrucción condicional.

Análisis. Resta demostrar que `mss` es correcta con respecto a la especificación dada y determinar sus complejidades temporal y espacial.

Teorema 3.4.2

El llamado `mss(A)`:

1. Calcula la suma máxima de un subarreglo de $A[0..N)$, incluyendo el subarreglo vacío entre las opciones.
2. Toma tiempo $O(N)$.
3. Toma espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

Reducción del espacio. En el diagrama de necesidades se puede observar que para el cálculo de $tab[n]$ solo es necesario contar con el valor precalculado en $tab[n-1]$. Es decir, de manera más general, para los casos recurrentes solo es necesario contar con un valor pre-calculado y no con un prefijo completo de tab . Con base en esta observación, se propone el invariante P_3 que introduce la variable entera $prev$ para reemplazar el arreglo tab :

$$P_3 : \quad prev = \phi(n-1).$$

La variable $prev$ representa la mejor suma de $A[0..N)$ que termina en la posición inmediatamente anterior a n . Por ello, la idea es usar este valor en el cuerpo del ciclo iterativo e ignorar todos los valores para índices inferiores a $n-1$. Además, es necesario incluir el cálculo del valor máximo de ϕ dentro del cuerpo del ciclo principal puesto que sería imposible recuperarlo de otra forma habiendo reducido

el espacio de la tabulación (no hay arreglo que recorrer para calcular el máximo). Esta estrategia resulta en la función `mss_opt` que se presenta en el Algoritmo 3.4.4.

Algoritmo 3.4.4: Reducción del espacio de la tabulación

```

1 def mss_opt(A):
2     ans, N = 0, len(A)
3     if N!=0:
4         prev, ans = A[0], max(ans, A[0])
5         #  $P_1 \wedge P_2 \wedge P_3$ 
6         for n in range(1, N):
7             prev = max(A[n], prev+A[n])
8             ans = max(ans, prev)
9     return ans

```

Si se demuestra correcto, el algoritmo optimizado resolvería el problema de interés en tiempo $O(N)$ y en espacio $O(1)$.

Teorema 3.4.3

El llamado `mss_opt(A)`:

1. Calcula la suma máxima de un subarreglo de $A[0..N)$, incluyendo el subarreglo vacío entre las opciones.
2. Toma tiempo $O(N)$.
3. Toma espacio $O(1)$.

Demostración

Se propone como ejercicio al lector.

Finalmente, el Algoritmo de Kadane se puede obtener del Algoritmo 3.4.4 bajo ciertos supuestos o acuerdos. La observación clave es notar que las asignaciones a las variables `prev` y `ans` en las líneas 4 y 7-8 son básicamente las mismas: `prev` se actualiza con el siguiente “mejor” y `ans` lleva registro del mejor encontrado entre los primeros n valores de la función. Con esa observación se puede iniciar el ciclo en $n = 0$, si se acepta que `curr` vale 0 al entrar en él. Ese valor inicial admite una lectura precisa: es $\phi(-1)$, la suma máxima de un subarreglo que termina inmediatamente antes del arreglo, que no puede ser otro que el subarreglo vacío. Con esa convención, la primera iteración calcula $A[0] \uparrow (A[0] + 0) = A[0]$, que es justamente $\phi(0)$. La

transformación del Algoritmo 3.4.4 bajo las observaciones anteriores resulta en el Algoritmo de Kadane, el cual se presenta en el Algoritmo 3.4.5.

Algoritmo 3.4.5: Algoritmo de Kadane

```

1 def kadane(A):
2     N, curr, ans = len(A), 0, 0
3     for n in range(N):
4         curr = max(A[n], curr+A[n])
5         ans = max(ans, curr)
6     return ans

```

Al igual que `mss_opt`, la función `kadane` resuelve el problema de suma máxima de un subarreglo en un arreglo usando tiempo lineal y espacio constante. El diseño de los invariantes para el ciclo en `kadane` y su corrección se proponen como ejercicio para el lector.

Ejercicios

1. Diseñe una función `mss_bf` en Python que implemente el diseño del Algoritmo 3.4.1. Corra la implementación con arreglos de números generados aleatoriamente de tamaños 10, 50, 100 y 250. ¿Cuál es la diferencia en tiempo entre las ejecuciones?
2. Después de presentar el Algoritmo 3.4.1, se propone una mejora para disminuir su complejidad de $O(N^3)$ a $O(N^2)$. Diseñe una función `mss_bf_opt` en Python que implemente este diseño optimizado. Corra la implementación con arreglos de números generados aleatoriamente de tamaños 10, 100, 1000 y 5000. ¿Cuál es la diferencia en tiempo entre las ejecuciones?
3. Diseñe una función `mss_dcc` en Python que implemente el diseño del Algoritmo 3.4.2. Corra la implementación con arreglos de números generados aleatoriamente de tamaños 10, 100, 1000, 10000 y 100000. ¿Cuál es la diferencia en tiempo entre las ejecuciones?
4. En la reformulación de la especificación con base en ϕ , se afirma que la expresión

$$(\uparrow n \mid 0 \leq n < N : \phi(n)) \uparrow 0.$$

permite considerar el subarreglo vacío como opción entre los subarreglos de A . Proponga un ejemplo de un arreglo $A[0..N)$ en el cual el resultado de la maximatoria sobre n sea negativo.

5. En el planteamiento recurrente de ϕ , se supone que $A[k..n]$ es un arreglo de suma máxima que inicia en algún $0 \leq k \leq n$ y termina en n . En el caso $k < n$

se indica que la suma $A[k] + \dots + A[n-1] + A[n]$ es igual a $\phi(n-1) + A[n]$. Ilustre esta afirmación con un ejemplo y elabore una justificación de por qué la igualdad es cierta (no es necesaria una demostración).

6. Elabore un ejemplo en el cual se exhiba la propiedad de solapamiento para ϕ . Explique brevemente su respuesta.
7. Diseñe un algoritmo con memorización para la función ϕ , incluyendo los predicados correspondientes a las condiciones que debe satisfacer el algoritmo. Demuestre que el algoritmo es correcto con respecto a la especificación dada, y estime las complejidades temporal y espacial.
8. Demuestre el Teorema 3.4.2.
9. Demuestre el Teorema 3.4.3.
10. Proponga invariantes para el Algoritmo de Kadane (Algoritmo 3.4.5) y demuestre que es correcto. También determine las complejidades temporal y espacial del algoritmo.
11. Modifique el Algoritmo de Kadane (Algoritmo 3.4.5) para que no se tenga en cuenta el arreglo vacío entre los subarreglos de A al calcular la suma máxima. Explique brevemente por qué funciona el algoritmo propuesto.
12. Modifique cualquiera de los algoritmos propuestos en esta sección que toman tiempo lineal para que, además de calcular la suma máxima de un subarreglo de $A[0..N]$, retorne k, n tales que el subarreglo $A[k..n]$ es de suma máxima (la complejidad del algoritmo resultante también debe ser lineal en función del tamaño del arreglo dado).

3.5. Subsecuencia creciente más larga

El problema de la subsecuencia creciente más larga consiste en determinar la longitud de la subsecuencia creciente de mayor tamaño en un arreglo de números dado. Antes de especificarlo es necesario precisar la noción de subsecuencia, que no debe confundirse con la de subarreglo usada en la Sección 3.4.

Definición 3.5.1: Subsecuencia

Sea $A[0..N]$ un arreglo de números. Una *subsecuencia* de $A[0..N]$ de longitud k , con $0 \leq k \leq N$, es una sucesión

$$A[i_0], A[i_1], \dots, A[i_{k-1}]$$

determinada por índices $0 \leq i_0 < i_1 < \dots < i_{k-1} < N$. La subsecuencia de longitud 0 se llama *vacía*. Cuando $k \neq 0$ se dice que la subsecuencia *termina* en i_{k-1} .

Una subsecuencia es *creciente* si y solo si

$$A[i_0] < A[i_1] < \cdots < A[i_{k-1}];$$

las subsecuencias de longitud 0 y 1 son crecientes por vacuidad.

Note que una subsecuencia queda determinada por un conjunto de índices y no por sus valores: dos subsecuencias distintas pueden tener los mismos elementos. El problema que se desea resolver es el siguiente.

Problema 3.5.1: Subsecuencia creciente más larga

Entrada: Un arreglo $A[0..N]$, con $N \geq 0$, de números.

Salida: Longitud de una subsecuencia creciente más larga de $A[0..N]$.

Se habla de *una* subsecuencia creciente más larga, y no de *la*, porque puede haber varias; lo que es único es su longitud.

Ejemplo 3.5.1

Considere el arreglo $A = [3, 1, 4, 1, 5, 9, 2, 6]$. La subsecuencia determinada por los índices 0, 2, 4, 5 es 3, 4, 5, 9 y es creciente de longitud 4. También lo son 1, 4, 5, 9 (índices 1, 2, 4, 5) y 1, 4, 5, 6 (índices 1, 2, 4, 7). Ninguna subsecuencia creciente de A tiene longitud 5, de modo que la respuesta para esta instancia es 4.

En contraste, la subsecuencia 3, 1, 4 (índices 0, 1, 2) no es creciente, y la sucesión 4, 3 no es subsecuencia de A porque sus índices no aparecen en orden.

Nota 3.5.1: Subsecuencias y subarreglos

Todo subarreglo de $A[0..N]$ es una subsecuencia –la que resulta de tomar índices consecutivos– pero no toda subsecuencia es un subarreglo. La diferencia se paga en la cantidad de objetos que hay que considerar: $A[0..N]$ tiene $N(N+1)/2 + 1$ subarreglos y 2^N subsecuencias, una por cada subconjunto de índices. Un algoritmo de fuerza bruta que enumere las subsecuencias para quedarse con la más larga que sea creciente toma, entonces, tiempo exponencial en el tamaño del arreglo.

La diferencia también se paga en el diseño, y esto es lo que hace que este problema no sea una variación menor del de la Sección 3.4. Allí, un

subarreglo de suma máxima que termina en n o bien es $A[n..n]$ o bien extiende un subarreglo que termina en $n - 1$: la contigüidad obliga a que el elemento anterior sea exactamente $A[n - 1]$ y por eso $\phi(n)$ dependía únicamente de $\phi(n - 1)$. En una subsecuencia creciente que termina en n , el elemento anterior a $A[n]$ puede ser cualquier $A[k]$ con $k < n$ y $A[k] < A[n]$. No hay un único candidato sino tantos como índices haya a la izquierda de n .

A diferencia del Problema 3.2.1, este es un problema de optimización, de modo que la propiedad que hay que verificar es la subestructura óptima de la Definición 3.1.1 y no solamente la subestructura general. El Problema 3.5.1 la exhibe, junto con la de solapamiento, así que se sigue la Metodología 3.3.1; el paso de especificación ya quedó cumplido con el enunciado del problema.

Función objetivo. Para $0 \leq n < N$, se define:

$\phi(n)$: “longitud de una subsecuencia creciente más larga de $A[0..N)$ que termina en n .”

La función ϕ tiene como dominio los índices de A y como co-dominio los enteros positivos. Note que $\phi(n) \geq 1$ para cualquier índice n , pues la subsecuencia determinada por el único índice n es creciente y termina en n .

Reformulación de la especificación. Toda subsecuencia creciente no vacía de $A[0..N)$ termina en alguno de sus índices. Luego, la salida del problema se puede escribir como la maximatoria:

$$(\uparrow n \mid 0 \leq n < N : \phi(n)) \uparrow 0.$$

El 0 adicional cubre el caso en el cual el arreglo dado es vacío, pues allí la maximatoria se toma sobre un rango vacío. A diferencia de lo que sucede en la Sección 3.4, ese es el *único* caso en el que el 0 interviene: como $\phi(n) \geq 1$ siempre, la maximatoria nunca es negativa.

Vale la pena señalar una diferencia con el problema del morral. Allí, la función objetivo evaluada en los parámetros de la instancia, $\phi(N, X)$, era ya la salida del problema. Aquí la función objetivo es apenas un ingrediente: la maximatoria externa es indispensable y no se puede prescindir de ella. Esta es la misma situación de la Sección 3.4 y es la razón por la cual la salida se reformula en lugar de simplemente evaluarse.

Planteamiento recurrente. Sea $0 \leq n < N$ y considere una subsecuencia creciente más larga de $A[0..N)$ que termina en n ; sea k el índice del elemento inmediatamente anterior a $A[n]$ en dicha subsecuencia, si es que existe. Hay dos posibilidades. Si la subsecuencia tiene longitud 1, entonces no hay tal k y la subsecuencia es simplemente $A[n]$. Si tiene longitud mayor, entonces $k < n$ y, por ser creciente, $A[k] < A[n]$; además, al quitar $A[n]$ queda una subsecuencia creciente

que termina en k , la cual debe ser lo más larga posible (de lo contrario, se podría alargar la original –¿por qué?). Es la propiedad de subestructura óptima.

Ahora bien, no se sabe de antemano cuál de los índices a la izquierda de n juega el papel de k . Se apuesta a todos los caballos: se consideran todos los k admisibles y se prefiere el que produzca la mayor longitud. Con base en este análisis se propone la siguiente definición para ϕ , con $0 \leq n < N$:

$$\phi(n) = 1 + (\uparrow k \mid 0 \leq k < n \wedge A[k] < A[n] : \phi(k)).$$

Nota 3.5.2: Una recurrencia sin caso base

La definición de ϕ consta de una sola ecuación y no distingue casos. Esto puede sorprender, pues todas las funciones objetivo vistas hasta ahora tienen al menos un caso base explícito. Lo que sucede es que el caso base está escondido en la convención de que una maximatoria sobre un rango vacío vale 0, la misma convención que permitió obtener el Algoritmo de Kadane en la Sección 3.4. En efecto, cuando $n = 0$ el rango es vacío y se obtiene $\phi(0) = 1 + 0 = 1$, que es la respuesta correcta.

Lo mismo ocurre, y esto es lo interesante, cuando $n \neq 0$ pero ningún índice a la izquierda de n satisface $A[k] < A[n]$: el rango vuelve a ser vacío y $\phi(n) = 1$. Es decir, una única ecuación cubre a la vez el caso base y un caso recurrente degenerado. Escribir $\phi(0) = 1$ como caso aparte no sería incorrecto, pero sí redundante.

La recurrencia tampoco es ambigua ni incompleta: para cada n el lado derecho está definido en términos de valores $\phi(k)$ con $k < n$, y la definición se puede leer como una definición por inducción fuerte sobre n .

Correctitud de la función objetivo. Como siempre, la observación intuitiva debe ser respaldada por un razonamiento formal.

Teorema 3.5.1

Sea $0 \leq n < N$. Entonces $\phi(n)$ es la longitud de una subsecuencia creciente más larga de $A[0..N)$ que termina en n .

Demostración

Sea $L(n)$ la longitud de una subsecuencia creciente más larga de $A[0..N)$ que termina en n ; note que $L(n) \geq 1$ y que el objetivo es demostrar $\phi(n) = L(n)$.

Se procede por inducción fuerte sobre n , suponiendo $\phi(k) = L(k)$ para todo $0 \leq k < n$. Se demuestran las dos desigualdades por separado.

- *Descomposición:* $L(n) \leq \phi(n)$. Sea $A[i_0], \dots, A[i_{m-1}]$ una subsecuencia creciente de longitud $m = L(n)$ que termina en n (i.e., $i_{m-1} = n$). Si $m = 1$, entonces $L(n) = 1 \leq \phi(n)$ dado que la maximatoria en la definición de ϕ nunca es negativa. Si $m \geq 2$, sea $k = i_{m-2}$. Entonces $k < n$ y $A[k] < A[n]$ por ser creciente la subsecuencia, luego k está en el rango de la maximatoria. Además, $A[i_0], \dots, A[i_{m-2}]$ es una subsecuencia creciente de longitud $m - 1$ que termina en k , de donde $L(k) \geq m - 1$. Entonces se tiene:

$$\begin{aligned}
 \phi(n) &\geq 1 + \phi(k) && (k \text{ está en el rango}) \\
 &= 1 + L(k) && (\text{hipótesis inductiva}) \\
 &\geq 1 + (m - 1) && (\text{observación anterior}) \\
 &= L(n) && (\text{aritmética}).
 \end{aligned}$$

- *Composición:* $\phi(n) \leq L(n)$. Si el rango de la maximatoria es vacío, entonces $\phi(n) = 1 \leq L(n)$. De lo contrario, como el rango es finito y no vacío, la maximatoria se alcanza en algún k con $k < n$ y $A[k] < A[n]$; es decir, $\phi(n) = 1 + \phi(k)$. Por hipótesis inductiva, $\phi(k) = L(k)$ y existe una subsecuencia creciente de longitud $L(k)$ que termina en k . Al extenderla con $A[n]$ se obtiene una subsecuencia de longitud $L(k) + 1$: es subsecuencia porque $k < n$ y es creciente porque $A[k] < A[n]$. Como termina en n , se concluye que $L(n) \geq L(k) + 1 = \phi(n)$.

En cualquier caso, $\phi(n) = L(n)$.

La propiedad de solapamiento es evidente en esta formulación: el cálculo de $\phi(n)$ invoca $\phi(k)$ para varios índices k y cada uno de esos llamados invoca a su vez índices menores. Encontrar instancias concretas en las cuales un mismo $\phi(k)$ se requiere desde dos llamados distintos se propone como ejercicio al lector.

¿Memorización o tabulación? Se opta por una tabulación. El diagrama de necesidades correspondiente se presenta en la Figura 5 y tiene una forma que no había aparecido antes en este capítulo. En el diagrama se usa la memoria de tabulación $tab[0..N]$ para almacenar los valores de la función y una variable n para indicar el índice de la tabla que se debe calcular en un momento dado. Hasta ahora, cada celda dependía de una cantidad fija de celdas previamente calculadas: una en la Sección 3.4 y dos en la Sección 3.6. Aquí, en cambio, $tab[n]$ puede depender de cualquiera de las n celdas que lo preceden. Esto tiene dos consecuencias: llenar la

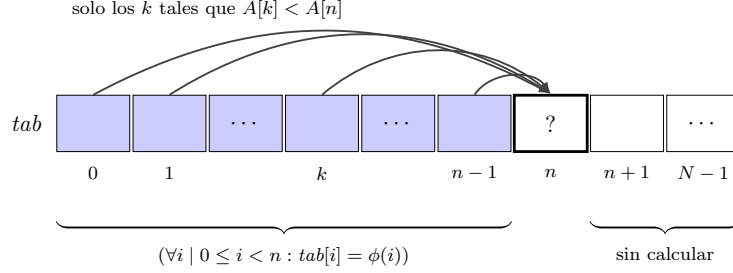


Figura 5. Diagrama de necesidades para la tabulación de ϕ en el problema de la subsecuencia creciente más larga. El valor $tab[n]$ depende de todo el prefijo $tab[0..n)$ y no de una cantidad acotada de celdas; cuáles de ellas intervienen efectivamente depende de los valores de A y no solo del índice n .

celda n ya no toma tiempo constante sino tiempo $O(n)$ y la reducción del espacio deja de ser posible, como se discute más adelante.

La estrategia sigue siendo llenar tab de izquierda a derecha. Los invariantes del ciclo principal son:

$$\begin{aligned} P_0 : & \quad (\forall i \mid 0 \leq i < n : tab[i] = \phi(i)). \\ P_1 : & \quad 0 \leq n \leq N. \\ P_2 : & \quad ans = (\uparrow i \mid 0 \leq i < n : \phi(i)) \uparrow 0. \end{aligned}$$

Los invariantes P_0 y P_1 son los mismos de la Sección 3.4: el subarreglo $tab[0..n)$ tiene, en orden, los primeros n valores de ϕ y n toma valores entre 0 y N . El invariante P_2 acumula la maximatoria de la reformulación sobre los valores ya calculados; a diferencia de lo hecho en la Sección 3.4, aquí conviene calcularla en el mismo ciclo y no en una segunda pasada, pues el ciclo ya no es de tiempo constante por iteración y una pasada adicional no cambia el orden del algoritmo.

El cálculo de $tab[n]$ requiere, a su vez, un ciclo que recorra los índices a la izquierda de n . Es el primer ciclo anidado del capítulo, y como tal necesita sus propios invariantes. Para un n fijo, y con k la variable del ciclo interno:

$$\begin{aligned} Q_0 : & \quad tab[n] = 1 + (\uparrow i \mid 0 \leq i < k \wedge A[i] < A[n] : \phi(i)). \\ Q_1 : & \quad 0 \leq k \leq n. \end{aligned}$$

El invariante Q_0 indica que $tab[n]$ contiene, en todo momento, el valor que tendría $\phi(n)$ si el rango de su maximatoria se restringiera a los índices menores que k . Cuando $k = 0$ el rango es vacío y $tab[n] = 1$, que es precisamente el valor con el cual se inicializa la celda; cuando $k = n$ el rango es el de la definición de ϕ y, por lo tanto, $tab[n] = \phi(n)$. Con base en este diseño se propone el Algoritmo 3.5.1.

Implementación. En el Algoritmo 3.5.1 se presenta una tabulación para la función objetivo. Note las finalidades de los dos ciclos y cómo los invariantes permiten entender cada una de ellas.

Algoritmo 3.5.1: Tabulación para ϕ

```

1 def lis(A):
2     N, ans = len(A), 0
3     tab = [ None for _ in range(N) ]
4     #  $P_0 \wedge P_1 \wedge P_2$ 
5     for n in range(N):
6         tab[n] = 1
7         #  $Q_0 \wedge Q_1$ 
8         for k in range(n):
9             if A[k] < A[n]: tab[n] = max(tab[n], tab[k]+1)
10        ans = max(ans, tab[n])
11    return ans

```

A diferencia de las funciones `mss` y `mss_opt` de la Sección 3.4, la función `lis` no requiere tratar de manera especial el caso en el cual el arreglo dado es vacío. Cuando $N = 0$, ninguno de los dos ciclos se ejecuta y se retorna el valor inicial de `ans`, que es 0: exactamente lo que indica la reformulación de la especificación.

A continuación se ilustra el uso de la función con algunas instancias:

```

1 >>> print(lis([3, 1, 4, 1, 5, 9, 2, 6]))
2 4
3 >>> print(lis([]))
4 0
5 >>> print(lis([7, 7, 7]))
6 1
7 >>> print(lis([5, 4, 3, 2, 1]))
8 1
9 >>> print(lis([1, 2, 3, 4, 5]))
10 5

```

La tercera instancia es un buen recordatorio de que la subsecuencia debe ser creciente en sentido estricto: en `[7, 7, 7]` ninguna subsecuencia de longitud 2 lo es.

Análisis. El Teorema 3.5.2 cierra el diseño de la solución del problema con el criterio de correctitud y de análisis asintótico.

Teorema 3.5.2

El llamado $\text{lis}(A)$, con $\text{len}(A) = N$:

1. Calcula la longitud de una subsecuencia creciente más larga de $A[0..N)$.
2. Toma tiempo $O(N^2)$.
3. Usa espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

Reducción del espacio. El paso opcional de la Metodología 3.3.1 invita a explorar si la memoria de tabulación se puede reducir. En las dos secciones anteriores la respuesta fue afirmativa y en ambos casos el argumento se leyó directamente en el diagrama de necesidades: bastaba con conservar una celda en la Sección 3.4 y una columna en la Sección 3.6. Aquí la respuesta es negativa, y conviene entender por qué.

Nota 3.5.3: Por qué la tabulación no se puede reducir

Una reducción del tipo de las secciones anteriores consiste en conservar solo una ventana de tamaño fijo de la tabulación. Para que eso funcione debe existir una cota w , independiente de la instancia, tal que $\phi(n)$ solo dependa de los valores $\phi(k)$ con $n - k \leq w$. En este problema tal cota no existe.

Considere, para $N \geq 4$, el arreglo

$$A = [1, 2, 3, \underbrace{0, 0, \dots, 0}_{N-4}, 4].$$

Se tiene $\phi(0) = 1$, $\phi(1) = 2$ y $\phi(2) = 3$; cada una de las posiciones ocupadas por un 0 tiene $\phi(n) = 1$; y para la última posición, $\phi(N-1) = 1 + 3 = 4$, valor que se alcanza únicamente en el índice 2. Es decir, el cálculo de la última celda necesita el valor de una celda que está a distancia $N-3$, la cual puede hacerse tan grande como se quiera.

Conviene precisar qué es exactamente lo que falla, porque es fácil equivocarse de criterio. Lo que impide la reducción no es que las dependencias dependan de los datos de la instancia. En este problema dependen de ellos —el rango de la maximatoria está restringido por la condición $A[k] < A[n]$ — pero eso, por sí solo, no sería obstáculo. La Sección 3.7 presenta un problema en el cual cuáles celdas se consultan también depende de los datos y allí la reducción del espacio funciona sin dificultad.

Lo que decide el asunto es si la ventana está acotada. En la Sección 3.4 cada celda depende de una celda anterior, en la Sección 3.6 de dos y en la Sección 3.7 de tres, siempre vecinas y siempre a distancia acotada, sin importar cuál de ellas intervenga en cada caso concreto. Aquí la ventana es todo el prefijo.

Esto no significa que el espacio $O(N)$ sea desperdiciado. Significa que $tab[0..N)$ es, en este problema, información que el algoritmo realmente necesita conservar completa.

Nota 3.5.4: Una solución en $O(N \log N)$

La cota $O(N^2)$ del Algoritmo 3.5.1 no es la mejor posible. Existe un algoritmo que resuelve el Problema 3.5.1 en tiempo $O(N \log N)$ y espacio $O(N)$, basado en una idea distinta: en lugar de calcular para cada índice la longitud de la mejor subsecuencia que termina allí, se mantiene, para cada longitud ℓ , el menor valor con el cual puede terminar una subsecuencia creciente de longitud ℓ dentro del prefijo procesado. Ese registro es siempre una sucesión creciente, lo cual permite actualizarlo con una búsqueda binaria por cada elemento del arreglo.

Ese algoritmo suele describirse como una combinación de una decisión voraz con una búsqueda binaria y no como programación dinámica en el sentido de este capítulo: no computa una función recurrente evitando recalculer subinstancias. Su diseño se propone como ejercicio.

Vale la pena anotar, por último, que este problema ha recibido mucha más atención de la que su enunciado sugiere, por razones que poco tienen que ver con la algorítmica. En las notas del final del capítulo se dan referencias, incluyendo un libro dedicado exclusivamente a él.

Ejercicios

1. Demuestre que un arreglo $A[0..N]$ tiene exactamente 2^N subsecuencias y $N(N+1)/2 + 1$ subarreglos. ¿Para qué valores de N coinciden estas dos cantidades?
2. En el planteamiento recurrente de ϕ se afirma que, al quitar $A[n]$ de una subsecuencia creciente más larga que termina en n , la subsecuencia que queda debe ser lo más larga posible entre las que terminan en k . Justifique esta afirmación.
3. Elabore un ejemplo en el cual se exhiba la propiedad de solapamiento para ϕ . Explique brevemente su respuesta.
4. Demuestre que el ciclo interno del Algoritmo 3.5.1 satisface los invariantes Q_0 y Q_1 .
5. Demuestre el Teorema 3.5.2.
6. Una subsecuencia es *no decreciente* si cada uno de sus elementos es menor o igual que el siguiente.
 - a) Especifique el problema de determinar la longitud de una subsecuencia no decreciente más larga de $A[0..N]$.
 - b) Modifique la definición de ϕ y el Algoritmo 3.5.1 para resolverlo. ¿Qué responde su algoritmo para el arreglo $[7, 7, 7]$?
7. Considere ahora el problema de determinar la longitud de una subsecuencia *decreciente* más larga de $A[0..N]$.
 - a) Resuélvalo modificando la definición de ϕ .
 - b) Resuélvalo sin modificar el Algoritmo 3.5.1, es decir, transformando solo el arreglo de entrada. Justifique que la transformación es correcta.
8. Diseñe un algoritmo con memorización para la función ϕ instanciando el Algoritmo 3.3.1, e incluya las condiciones de representación que debe satisfacer la memoria compartida. Demuestre que el algoritmo es correcto y estime sus complejidades temporal y espacial.
9. Modifique el Algoritmo 3.5.1 para que, además de calcular la longitud de una subsecuencia creciente más larga de $A[0..N]$, retorne los índices que la determinan. La complejidad temporal del algoritmo resultante debe seguir siendo $O(N^2)$. ¿Cuál es su complejidad espacial?
10. En la Nota 3.5.3 se exhibe, para cada $N \geq 4$, un arreglo en el cual el cálculo de $\phi(N-1)$ requiere el valor de $\phi(2)$. Construya una familia de arreglos en la cual el cálculo de $\phi(N-1)$ requiera los valores de $\phi(k)$ para *todos* los índices $k < N-1$.
11. Modifique el Algoritmo 3.5.1 para que, en lugar de la longitud de una subsecuencia creciente más larga, calcule la *cantidad* de subsecuencias crecientes de esa longitud. Formule los invariantes correspondientes y justifique el diseño.

12. Resuelva el Ejercicio 4 de la Sección 3.1, en el cual se desea determinar la mayor cantidad de días, no necesariamente consecutivos, en los cuales el precio de una acción aumentó a lo largo de N días. Explique en qué sentido ese problema es una instancia del Problema 3.5.1.
13. 🌐 Diseñe el algoritmo de tiempo $O(N \log N)$ esbozado en la Nota 3.5.4. Formule los invariantes del ciclo, demuestre que el registro que se mantiene es siempre una sucesión creciente y concluya que el algoritmo es correcto.

3.6. El problema del morral

El problema del morral es un problema de optimización combinatoria.

Nota 3.6.1: El nombre del problema

El nombre del problema del morral (en inglés, *knapsack* o *KS*) se deriva de una analogía en la cual se desea escoger de una colección de elementos para cargar en un morral que tiene una restricción de peso (o volumen). El objetivo es escoger dicha combinación de elementos maximizando su valor total sin exceder la capacidad del morral.

Este es un problema que surge naturalmente en situaciones de asignación de recursos en las cuales quienes toman las decisiones deben escoger proyectos o tareas indivisibles bajo una restricción presupuestal o de tiempo.

De forma general, dada una colección de elementos, cada uno con un peso y un valor asociados, el problema consiste en determinar cuál es la suma máxima de valores que se puede obtener al tomar algunos de los elementos sin que su suma exceda una restricción de peso global.

Problema 3.6.1: Problema del morral

Entrada: Arreglos $V[0..N]$ y $W[0..N]$, con $N \geq 0$, de valores (números positivos) y de pesos (números enteros positivos), y un número entero $X \geq 0$.

Salida: Suma máxima de elementos con valores en $V[0..N]$ cuyos pesos en $W[0..N]$ no exceden X .

La condición de salida del problema supone que el valor y el peso del i -ésimo elemento están dados por $V[i]$ y $W[i]$, respectivamente.

La restricción a valores y pesos positivos no es un tecnicismo. Que un elemento tenga valor o peso negativo no tiene interpretación en el problema, y admitir el 0 solo agrega instancias degeneradas: un elemento de peso 0 se puede llevar siempre y uno de valor 0 nunca cambia la respuesta. Excluirlos tiene, además, dos consecuencias que vale la pena anunciar. La primera es que este enunciado queda alineado con el del morral fraccionario de la Sección 4.6, del cual este problema es la restricción a decisiones de todo o nada. La segunda es que la variante con copias ilimitadas, que se propone como ejercicio al final de la sección, queda bien planteada: un elemento de peso 0 y valor positivo se podría llevar cuantas veces se quisiera y el óptimo no existiría.

La naturaleza combinatoria del problema del morral se revela al notar que se requiere una exploración sobre los subconjuntos de índices de $V[0..N]$. Para cualquier $S \subseteq \{0, \dots, N-1\}$, se definen las funciones *value* y *weight*:

$$\text{value}(S) = (+i \mid i \in S : V[i]) \quad \text{y} \quad \text{weight}(S) = (+i \mid i \in S : W[i]).$$

La expresión $\text{value}(S)$ representa el valor total de los elementos indexados por S , mientras que $\text{weight}(S)$ el peso total de los elementos indexados por S . Así, la salida del problema del morral corresponde a la expresión:

$$(\uparrow S \mid S \subseteq \{0, \dots, N-1\} \wedge \text{weight}(S) \leq X : \text{value}(S)).$$

Como hay 2^N subconjuntos S de $\{0, \dots, N-1\}$ (¿por qué?), se dice que el problema del morral es de naturaleza combinatoria, pues debe considerar todos aquellos subconjuntos S para determinar la mejor combinación posible bajo la restricción dada. Debe ser claro, entonces, que un algoritmo de fuerza bruta para resolver el problema del morral tomará tiempo exponencial en la cantidad de elementos entre los cuales se elige.

Ejemplo 3.6.1

Considere 4 elementos con valores y pesos dados por, respectivamente:

$$V = [4, 6, 5, 1] \quad \text{y} \quad W = [3, 5, 5, 2].$$

- Con $X = 15$, la mejor opción es llevar todos los elementos, obteniendo como valor total 16 y peso total 15.
- Con $X = 11$, el valor máximo es 11. Esto resulta de dos configuraciones cuyo peso total es 10: llevar los elementos 0, 1, 3 o los elementos 1, 2.
- Con $X = 5$, la mejor opción es llevar el elemento 1, con valor total 6 y peso total 5.
- Con $X = 1$, la mejor (y única) opción es no echar nada al morral.

Este es un problema de optimización, de modo que la propiedad que hay que verificar es la subestructura óptima de la Definición 3.1.1. El problema del morral la exhibe, junto con la de solapamiento (¿por qué?), así que se seguirá la Metodología 3.3.1; el paso de especificación ya quedó cumplido con el Problema 3.6.1.

Función objetivo. Hay dos variables que permiten generalizar la salida del problema: N y X . Por una parte, la cantidad de elementos disponibles se puede hacer variar para considerar diferentes prefijos de $V[0..N]$ y $W[0..N]$ (también se pueden hacer variar los sufijos, resultando en un planteamiento similar). Por otra parte, el peso disponible en el morral cambiaría cada vez que se decide escoger un elemento. Con base en estas observaciones, se propone la siguiente función objetivo para $0 \leq n \leq N$ y $0 \leq x \leq X$:

$$\phi(n, x) : \quad \text{“suma máxima de elementos con valores en } V[0..n] \\ \text{cuyos pesos en } W[0..n] \text{ no exceden } x\text{.”}$$

Reformulación de la especificación. El objetivo del problema es calcular $\phi(N, X)$.

Planteamiento recurrente. La variable n se toma como “pivote”: para razonar por casos, estos se estructuran primero con base en condiciones sobre n y, de ser necesario, luego se dividen en subcasos con base en condiciones sobre la segunda variable x . Es importante tener en cuenta que $\phi(n, x)$ debe ser definida para todos los valores de n y x que satisfacen $0 \leq n \leq N$ y $0 \leq x \leq X$.

Se procede por casos sobre n y x :

- No hay elementos para escoger (i.e., $n = 0$). Luego, sin importar la capacidad x del morral, lo máximo (y único) que se puede obtener es 0.
- Hay al menos un elemento para escoger (i.e., $n > 0$). Entonces están las opciones de que el elemento $n - 1$ (este es el n -ésimo elemento) pese más que la capacidad x del morral o que no:
 - si $W[n - 1] > x$, no hay más opción que ignorar ese elemento y escoger lo mejor posible con los elementos que restan, sin modificar la capacidad del morral; es decir, se recurre con $n - 1$ y x .
 - si $W[n - 1] \leq x$, entonces están las opciones de adicionar el elemento al morral o no. Como el objetivo es tomar la mejor decisión, se apuesta a los dos caballos. No escogerlo resulta en la misma situación del caso anterior (i.e., se recurre con $n - 1$ y x). Escogerlo tiene el efecto de disminuir la capacidad del morral a $x - W[n - 1]$ y de aumentar el beneficio en $V[n - 1]$ unidades; es decir, se recurre con $n - 1$ y $x - W[n - 1]$, y se acumula $V[n - 1]$ beneficio.

En cada uno de los casos recurrentes se respetan las condiciones $0 \leq n \leq N$ y $0 \leq x \leq X$ (¿por qué?), lo cual es clave para contar con una buena definición de ϕ .

Vale la pena observar, además, que como los pesos son positivos, en el segundo caso recurrente se tiene $x - W[n-1] < x$: decrecen los dos parámetros y no solamente n . La medida bien fundada que exige la Sección 3.1 se puede tomar, entonces, sobre cualquiera de los dos (Ejercicio 4). La naturaleza exponencial de la búsqueda de “la” mejor elección de elementos se ve en el segundo caso recurrente: allí se apuesta a llevar y a no llevar el elemento, es decir, se consideran todos los subconjuntos de índices pendientes sin acumular el n -ésimo elemento y todos los subconjuntos de índices pendientes acumulando el n -ésimo elemento.

La definición formal de ϕ se presenta para $0 \leq n \leq N$ y $0 \leq x \leq X$:

$$\phi(n, x) = \begin{cases} 0 & , \text{ si } n = 0, \\ \phi(n-1, x) & , \text{ si } n \neq 0 \wedge W[n-1] > x, \\ \phi(n-1, x) \uparrow \\ (\phi(n-1, x - W[n-1]) + V[n-1]) & , \text{ si } n \neq 0 \wedge W[n-1] \leq x. \end{cases}$$

Correctitud de la función objetivo. Es necesario demostrar que la definición de ϕ permite resolver el problema del morral.

Teorema 3.6.1

Si $0 \leq n \leq N$ y $0 \leq x \leq X$, entonces $\phi(n, x)$ es la suma máxima de elementos con valores en $V[0..n]$ cuyos pesos en $W[0..n]$ no exceden x .

Demostración

Diga que un conjunto $S \subseteq \{0, \dots, n-1\}$ es *admisibile para* (n, x) si $\text{weight}(S) \leq x$, y sea

$$L(n, x) = (\uparrow S \mid S \text{ es admisibile para } (n, x) : \text{value}(S)).$$

Note que $L(n, x)$ siempre existe, pues $\emptyset$ es admisibile para cualquier (n, x) y hay una cantidad finita de conjuntos. El objetivo es demostrar que $\phi(n, x) = L(n, x)$, y se procede por inducción sobre n .

Caso base $n = 0$: el único subconjunto de $\emptyset$ es $\emptyset$, cuyo valor es 0. Luego $L(0, x) = 0 = \phi(0, x)$ para cualquier x .

Caso inductivo $n \neq 0$: suponga que $\phi(n-1, y) = L(n-1, y)$ para todo $0 \leq y \leq X$. Se distinguen los dos casos de la definición de ϕ y, en cada uno, se demuestran las dos desigualdades.

- Caso $W[n-1] > x$. En este caso el elemento $n-1$ no cabe en el morral, de modo que ningún conjunto admisibile para (n, x) lo contiene: los conjuntos admisibles para (n, x) son exactamente los admisibles para $(n-1, x)$.

Luego $L(n, x) = L(n-1, x)$ y, por hipótesis inductiva, $L(n, x) = \phi(n-1, x) = \phi(n, x)$.

- Caso $W[n-1] \leq x$. Se demuestran las dos desigualdades por separado.
 - *Composición*: $\phi(n, x) \leq L(n, x)$. Basta ver que los dos argumentos del máximo son valores de conjuntos admisibles para (n, x) . Por hipótesis inductiva, $\phi(n-1, x) = L(n-1, x) = \text{value}(S_1)$ para algún S_1 admisible para $(n-1, x)$; como $S_1 \subseteq \{0, \dots, n-2\}$, también es admisible para (n, x) . De nuevo por hipótesis inductiva, $\phi(n-1, x-W[n-1]) = \text{value}(S_2)$ para algún S_2 admisible para $(n-1, x-W[n-1])$; entonces $S_2 \cup \{n-1\}$ es admisible para (n, x) , pues su peso es $\text{weight}(S_2) + W[n-1] \leq x$, y su valor es $\text{value}(S_2) + V[n-1]$. Como los dos argumentos son valores de conjuntos admisibles, ninguno excede a $L(n, x)$ y tampoco lo hace su máximo.
 - *Descomposición*: $\phi(n, x) \geq L(n, x)$. Tome S admisible para (n, x) con $\text{value}(S) = L(n, x)$. Si $n-1 \notin S$, entonces S es admisible para $(n-1, x)$ y, por hipótesis inductiva, $L(n, x) = \text{value}(S) \leq L(n-1, x) = \phi(n-1, x) \leq \phi(n, x)$. Si $n-1 \in S$, entonces $S \setminus \{n-1\}$ es admisible para $(n-1, x-W[n-1])$, pues su peso es $\text{weight}(S) - W[n-1] \leq x - W[n-1]$. Entonces se tiene:

$$\begin{aligned}
 L(n, x) &= \text{value}(S \setminus \{n-1\}) + V[n-1] && (n-1 \in S) \\
 &\leq L(n-1, x-W[n-1]) + V[n-1] && (\text{admisibilidad}) \\
 &= \phi(n-1, x-W[n-1]) + V[n-1] && (\text{hipótesis inductiva}) \\
 &\leq \phi(n, x) && (\text{uno de los argumentos}).
 \end{aligned}$$

En cualquier caso, $\phi(n, x) = L(n, x)$.

Note que la demostración nunca necesitó preguntarse cuándo la solución óptima es vacía. La pregunta parece natural –si no se lleva nada, ¿qué se puede concluir?– y bajo la hipótesis de que los valores son positivos tiene, de hecho, una respuesta: $\emptyset$ es óptimo si y solo si no hay elementos o ninguno de ellos cabe. Pero esa respuesta hay que demostrarla y, sobre todo, depende de una hipótesis que el argumento anterior no usó en ninguna parte: basta permitir elementos de valor 0 para que la caracterización deje de valer, mientras que la demostración sigue intacta (Ejercicio 6). Separar el argumento en las dos mitades evita el problema de raíz, porque en ninguna de las dos hace falta caracterizar la solución óptima: en la de composición se construyen conjuntos admisibles y en la de descomposición se parte uno cualquiera.

La demostración puede parecer larga, pero su estructura es la misma que la del Teorema 3.4.1 y la del Teorema 3.5.1. Lo único que la alarga es que hay dos casos en la definición de ϕ y que, en el segundo, el máximo tiene dos argumentos. Vale la pena señalar que lo que se demuestra es que ϕ calcula el *valor* óptimo y no

que identifique una configuración que lo alcance; puede haber más de una, como lo muestra el Ejemplo 3.6.1 con $X = 11$.

¿Memorización o tabulación? Se diseñará una tabulación para implementar eficientemente ϕ con base en el diagrama de necesidades de la Figura 6. La

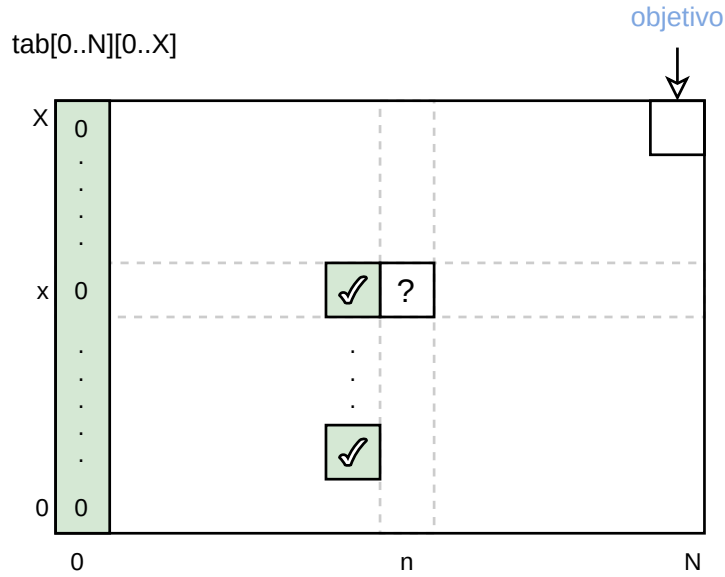


Figura 6. Diagrama de necesidades para la tabulación de ϕ en el problema del morral. El valor $tab[n][x]$ depende de dos celdas de la columna $n - 1$, una por cada llamado recurrente.

tabulación se implementa con una matriz $tab[0..N][0..X]$ con $N + 1$ columnas y $X + 1$ filas. El objetivo es calcular el valor correspondiente a $tab[N][X]$ dado que el problema se resuelve con $\phi(N, X)$. Cuando $n = 0$, sin importar el valor de x , el valor $\phi(n, x)$ es 0. Por ello, la primera columna de la matriz tiene únicamente el valor 0. Para calcular $\phi(n, x)$, en el primer caso recurrente se depende de $\phi(n - 1, x)$ y en el segundo caso recurrente además de $\phi(n - 1, x - W[n - 1])$. Por ello, en la matriz se identificaron dos celdas de la columna $n - 1$: cada una corresponde a cada uno de los llamados recurrentes, para los dos posibles valores de la capacidad restante en el morral. Estas observaciones indican que se puede llenar la tabulación por columnas, comenzando por la columna $n = 1$ (esta no es la única forma de hacerlo, ¿cierto?). Esta decisión se ve reflejada en los siguientes invariantes:

$$P_0 : (\forall i, j \mid 0 \leq i < n \wedge 0 \leq j \leq X : \text{tab}[i][j] = \phi(i, j)).$$

$$P_1 : (\forall j \mid 0 \leq j < x : \text{tab}[n][j] = \phi(n, j)).$$

$$P_2 : 0 \leq n \leq N + 1.$$

$$P_3 : 0 \leq x \leq X + 1.$$

El invariante P_0 indica que todas las columnas a la izquierda de la columna n han sido llenadas con los valores correspondientes de ϕ . El invariante P_1 especifica que los valores en la columna n , por debajo de la fila x , corresponden a los valores de $\phi(n, \cdot)$. Los invariantes P_2 y P_3 son de apoyo: el primero indica que los valores de n varían entre 0 y $N + 1$, inclusive; el segundo, que los valores de x varían entre 0 y $X + 1$, inclusive. Los dos invariantes de apoyo sugieren una forma de llenar la tabla: por columnas, de izquierda a derecha, hasta llenar la columna N . Visualmente, esta decisión se puede incluir en el diagrama de necesidades, como lo muestra la Figura 7.

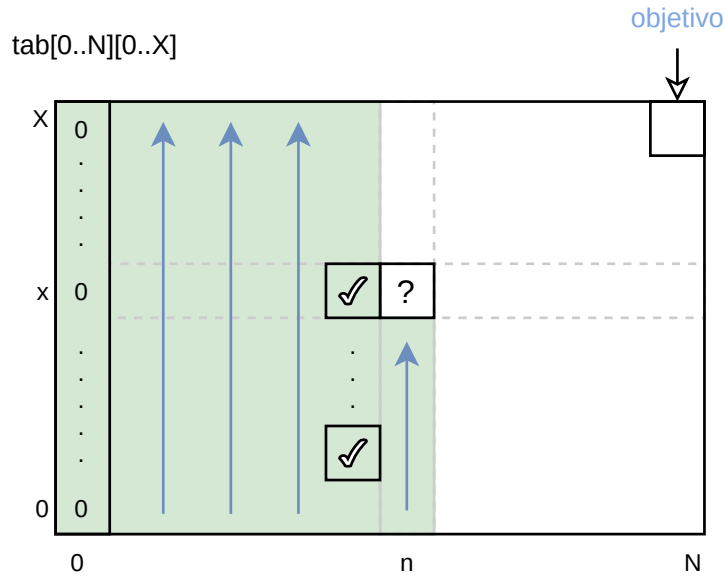


Figura 7. Orden de llenado de la tabulación en el problema del morral: por columnas, de izquierda a derecha.

Para implementar el algoritmo de tabulación, basta con seguir los invariantes y tener clara la imagen mental de ellos en el diagrama de necesidades. La idea es entonces, una vez creada la tabla e inicializada su primera columna, iniciar a llenarla

desde la columna $n = 1$. Esto se hará como con las máquinas de escribir: se llena una columna, se procesa la siguiente. Intencionalmente se permite que la variable x llegue a $X + 1$, aunque esta fila no haga parte de la tabla: con $x = X + 1$ se sabe que se ha llenado la columna actual y se procede a iniciar la siguiente. Algo similar sucede con n : se habrá llenado la tabla por completo cuando se tenga $n = N + 1$.

Implementación. El Algoritmo 3.6.1 presenta la tabulación diseñada para la función ϕ .

Algoritmo 3.6.1: Tabulación para el problema del morral

```

1 def ks_tab(V, W, X):
2     N = len(V)
3     tab = [ [ None for _ in range(X+1) ] for _ in range(N+1) ]
4     for x in range(X+1): tab[0][x] = 0
5     n, x = 1, 0
6     #  $P_0 \wedge P_1 \wedge P_2 \wedge P_3$ 
7     while n != N+1:
8         if x == X+1: n, x = n+1, 0
9         else:
10            if x < W[n-1]: tab[n][x] = tab[n-1][x]
11            else: tab[n][x] = max(tab[n-1][x], tab[n-1][x-W[n-1]]+V[n-1])
12            x += 1
13     return tab[N][X]
```

Análisis. La demostración de la corrección del algoritmo es rutina con base en los invariantes dados y la especificación del problema. La complejidad espacial está dominada por el tamaño de la tabla, y la temporal por su creación y actualización. Antes de introducir formalmente estos resultados, se presenta un ejemplo de la ejecución del algoritmo.

```

1 >>> V = [4, 6, 5, 1]
2 >>> W = [3, 5, 5, 2]
3 >>> print(ks_tab(V, W, 15))
4 16
5 >>> print(ks_tab(V, W, 11))
6 11
7 >>> print(ks_tab(V, W, 5))
8 6
9 >>> print(ks_tab(V, W, 1))
10 0
```

Teorema 3.6.2

El llamado `ks_tab(V, W, X)`, con $\text{len}(V) = \text{len}(W) = N$:

1. Calcula la suma máxima de elementos con valores en $V[0..N)$ cuyos pesos en $W[0..N)$ no exceden X .
2. Toma tiempo $O(NX)$.
3. Usa espacio $O(NX)$.

Demostración

Se propone como ejercicio al lector.

La complejidad temporal del algoritmo propuesto es polinomial en función de N y X . Sin embargo, es importante tener en cuenta que la cota X *no* es polinomial en función de N ; a esto se le llama tiempo pseudo-polinomial. Por ejemplo, si $X = 2^{32}$, basta con 32 bits para representar X . Sin embargo, el algoritmo propuesto usa 2^{32} filas para construir y llenar la tabla. Por ello, es usual indicar que la complejidad temporal (y espacial) de la función `ks_tab` es $O(N \cdot 2^{\text{bits en } X})$.

3.6.1. Primera optimización. El diagrama de necesidades sugiere que el diseño de la tabulación se puede optimizar. En realidad, el cálculo de la columna $n \geq 1$ solo depende de valores en la columna $n - 1$. Además, al iniciar con la columna $n = 1$, la columna $n = 0$ está completamente inicializada. Esta observación permite hacer una optimización importante en el espacio usado por la tabulación.

El diagrama de necesidades de la Figura 8 reduce la tabulación a dos columnas: una para el presente (que corresponde a la columna n) y otra para el pasado inmediatamente anterior (que corresponde a la columna $n - 1$). La noción de “presente” se representa con el arreglo *curr* y la de “pasado inmediato” con *prev*. Un reto surge ante la pregunta de qué hacer cuando *curr* haya sido llenado. Note que en el caso en el cual exista la necesidad de seguir procesando la tabulación (i.e., $n \neq N + 1$), este arreglo pasará a ser el pasado inmediato y no habrá uso para los valores almacenados en *prev*. Por ello, y para evitar la creación de memoria adicional y copias innecesarias, se puede decidir reutilizar *prev* para el rol de nuevo presente. Para facilitar esta decisión en el código final, se prefiere que la tabulación sea una matriz de dos columnas, cada una de ellas de tamaño $X + 1$, y usar dos variables (abusando la notación, *prev* y *curr*) que alternen opuestamente con los

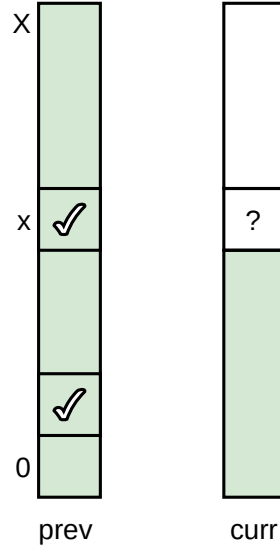


Figura 8. Diagrama de necesidades de la primera optimización: la tabulación se reduce a la columna del presente y la del pasado inmediato.

valores 0 y 1: cuando $prev$ sea 0, necesariamente $curr$ es 1, y viceversa.

$$Q_0 : (\forall j \mid 0 \leq j \leq X : tab[prev][j] = \phi(n-1, j)).$$

$$Q_1 : (\forall j \mid 0 \leq j < x : tab[curr][j] = \phi(n, j)).$$

$$Q_2 : 0 \leq prev \leq 1.$$

$$Q_3 : 0 \leq curr \leq 1.$$

$$Q_4 : prev + curr = 1.$$

$$P_2 : 0 \leq n \leq N + 1.$$

$$P_3 : 0 \leq x \leq X + 1.$$

La matriz para la tabulación es $tab[0..2][0..X]$. El invariante Q_0 indica que $tab[prev]$ contiene todos los valores del pasado inmediatamente anterior, mientras que el arreglo $tab[curr][0..x)$ los del presente (aún faltan por procesar valores del presente en $tab[curr][x..X]$), de acuerdo con Q_1 . Los invariantes Q_2 y Q_3 especifican que los valores de $prev$ y $curr$ solo pueden ser 0 o 1, y el invariante Q_4 que los valores de estas variables no deben coincidir. Los invariantes P_2 y P_3 corresponden a los de la especificación del algoritmo sin optimizar. Con base en este diseño se presenta la función `ks_tab_opt1` en el Algoritmo 3.6.2.

Algoritmo 3.6.2: Primera optimización del espacio

```

1 def ks_tab_opt1(V, W, X):
2     N = len(V)
3     tab = [ [ 0 for _ in range(X+1) ] for _ in range(2) ]
4     n,x,prev,curr = 1,0,0,1
5     #  $Q_0 \wedge Q_1 \wedge Q_2 \wedge Q_3 \wedge Q_4 \wedge P_2 \wedge P_3$ 
6     while n!=N+1:
7         if x==X+1: n,x,prev,curr = n+1,0,1-prev,1-curr
8         else:
9             tab[curr][x] = tab[prev][x]
10            if W[n-1]<=x:
11                tab[curr][x] = max(tab[curr][x], tab[prev][x-W[n-1]]+V[n-1])
12            x += 1
13     return tab[prev][X]

```

Además de los cambios derivados de la nueva forma de hacer la tabulación, la función `ks_tab_opt1` incluye una actualización en la inicialización de la tabla: en el momento de su creación, todos los valores en ella son 0. Esto evita tener que procesar la primera columna después de su creación (es decir, se está ahorrando una pasada por una columna completa). Inicialmente, *prev* es asignada el índice 0 y *curr* el índice 1. Es decir, el pasado está registrado en la columna 0 de *tab* y el presente en la columna 1. El cuerpo del ciclo tiene dos partes: una de cambio de columna (línea 7) y otra de cambio de fila (líneas 8-12). El cambio de columna se hace de manera similar al algoritmo sin optimización, incrementando *n* en una unidad y asignando 0 a *x*. Las variables *prev* y *curr* invierten sus valores (¿por qué la resta funciona?). Se pudo optar por intercambiar directamente los valores de estas dos variables apelando a la sustitución “simultánea” ofrecida por Python. Sin embargo, esto no es posible en lenguajes de programación imperativos como C, C++ o Java. Por esto, se prefiere hacer el “complemento” con 1, dado que esta operación aritmética se puede implementar con una asignación individual sin la necesidad de una variable intermedia/temporal. El cuerpo del ciclo se encarga de encontrar el mejor valor posible para $\phi(n, x)$ de acuerdo con su definición recurrente. Observe que el valor que se retorna está al final de la columna *prev* y no en la columna *curr* (¿por qué?).

Se ilustra el uso de la función `ks_tab_opt1` con algunas instancias del problema:

```

1 >>> V = [4, 6, 5, 1]
2 >>> W = [3, 5, 5, 2]
3 >>> print(ks_tab_opt1(V, W, 15))
4 16
5 >>> print(ks_tab_opt1(V, W, 11))
6 11
7 >>> print(ks_tab_opt1(V, W, 5))

```



```

8 6
9 >>> print(ks_tab_opt1(V, W, 1))
10 0

```

El diseño de la tabulación para `ks_tab_opt1` permite reducir el espacio de la tabulación a $O(2X) = O(X)$, mientras que el tiempo de ejecución se mantiene en $O(NX)$.

Teorema 3.6.3

El llamado `ks_tab_opt1(V, W, X)`, con $\text{len}(V) = \text{len}(W) = N$:

1. Calcula la suma máxima de elementos con valores en $V[0..N)$ cuyos pesos en $W[0..N)$ no exceden X .
2. Toma tiempo $O(NX)$.
3. Usa espacio $O(X)$.

Demostración

Se propone como ejercicio al lector.

3.6.2. Segunda optimización. El espacio de la tabulación puede ser reducido aún más. Note que los valores que se utilizan para actualizar la columna n (i.e., la del presente) están en un espacio acotado dentro de la columna $n - 1$ (i.e., la del pasado inmediatamente anterior). Si de alguna forma se pudiera consolidar la información de los dos momentos en el tiempo en una sola columna, esto permitiría reducir el espacio de $2X$ a X posiciones de memoria. La observación clave es que para tabular $\phi(n, x)$, solo se requieren valores de $\phi(n - 1, y)$, con $0 \leq y \leq x$. Es decir, en la tabulación deseada se podría “partir” esa columna de tal forma que la parte inferior al índice x tuviera solo valores del pasado inmediato y en la parte superior de x solo valores del presente. El diagrama de necesidades de la Figura 9 resume visualmente estas observaciones. Como la parte inferior de $\text{tab}[0..X]$ debe tener disponibles los valores del pasado, necesariamente la forma de llenarla debe iniciar por su parte superior. Los siguientes invariantes hacen explícita esta y las

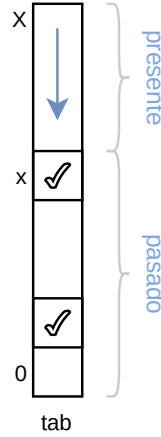


Figura 9. Diagrama de necesidades de la segunda optimización: una única columna en la cual la parte inferior al índice x tiene valores del pasado inmediato y la parte superior valores del presente.

observaciones anteriores.

$$R_0 : (\forall j \mid 0 \leq j \leq x : \text{tab}[j] = \phi(n-1, j)).$$

$$R_1 : (\forall j \mid x < j \leq X : \text{tab}[j] = \phi(n, j)).$$

$$P_2 : 0 \leq n \leq N+1.$$

$$P_3 : -1 \leq x \leq X.$$

El invariante R_0 especifica que el subarreglo $\text{tab}[0..x]$ tiene los valores del pasado inmediato (i.e., desde $\phi(n-1, 0)$ hasta $\phi(n-1, x)$). El invariante R_1 , a su vez, especifica que el subarreglo $\text{tab}[x+1..X]$ tiene los valores del presente (i.e., desde $\phi(n, x+1)$ hasta $\phi(n, X)$). Los invariantes P_2 y P_3 son los mismos de las dos versiones anteriores de los algoritmos de tabulación de ϕ .

Algoritmo 3.6.3: Segunda optimización del espacio

```

1 def ks_tab_opt2(V, W, X):
2     N = len(V)
3     tab = [ 0 for _ in range(X+1) ]
4     n, x = 1, X
5     #  $R_0 \wedge R_1 \wedge P_2 \wedge P_3$ 
6     while n != N+1:
7         if x == -1: n, x = n+1, X
8         else:
9             if W[n-1] <= x:
```

```

10         tab[x] = max(tab[x], tab[x-W[n-1]]+V[n-1])
11         x -= 1
12     return tab[X]

```

Inicialmente, $tab[0..X]$ es creado con valores 0. A partir de la columna $n = 1$ y hasta la columna $n = N$, se hace una de dos cosas. O bien se detecta que se ha procesado una nueva columna (i.e., $x = -1$) y se avanza a la (posible) siguiente columna para comenzar el proceso desde la fila X . De lo contrario, el valor en $tab[x]$ es $\phi(n-1, x)$. Si se tiene capacidad disponible en el morral (i.e., $W[n-1] \leq x$) y hay mayor beneficio en llevar el elemento a no llevarlo, entonces se actualiza $tab[x]$ con $\phi(n-1, x - W[n-1]) + V[n-1]$.

Teorema 3.6.4

El llamado `ks_tab_opt2(V, W, X)`, con $len(V) = len(W) = N$:

1. Calcula la suma máxima de elementos con valores en $V[0..N)$ cuyos pesos en $W[0..N)$ no exceden X .
2. Toma tiempo $O(NX)$.
3. Usa espacio $O(X)$.

Demostración

Se propone como ejercicio al lector.

Asintóticamente, los algoritmos 3.6.2 y 3.6.3 usan la misma cantidad de espacio. Sin embargo, el espacio se reduce exactamente a la mitad en `ks_tab_opt2` y, además, el código es más sucinto y fácil de seguir. Por estas razones debería preferirse esta función a `ks_tab_opt1`.

Ejercicios

1. Demuestre que la cantidad de subconjuntos de un conjunto de N elementos es 2^N .
2. Considere los 4 elementos con pesos y valores dados en el Ejemplo 3.6.1. Enumere los 16 conjuntos de índices correspondientes a esta instancia del problema del morral, y a cada uno de ellos asocie el peso y valor total. Verifique que la conclusión es correcta para cada X en dicho ejemplo.
3. Justifique con un ejemplo que el problema del morral exhibe las propiedades de subestructura óptima y solapamiento.
4. Demuestre que la definición recurrente de $\phi(n, x)$, para $0 \leq n \leq N$ y $0 \leq x \leq X$, en cualquier llamado recurrente mantiene los valores de n y x en estos rangos. Demuestre además que en el segundo caso recurrente decrecen estrictamente los dos parámetros, e indique en dónde usa que los pesos son positivos.
5. Demuestre que si $S \subseteq \{0, \dots, n-1\}$ es admisible para (n, x) —es decir, si $\text{weight}(S) \leq x$ — entonces S tiene a lo sumo x elementos. Concluya que una solución óptima del Problema 3.6.1 lleva a lo sumo $\min(N, X)$ elementos e indique en dónde usa que los pesos son positivos.
6. Una demostración tentadora del Teorema 3.6.1 empieza por caracterizar la solución óptima S y observa que, si $S = \emptyset$, entonces no hay elementos para escoger (i.e., $n = 0$) o todos los elementos disponibles pesan más que x (i.e., $W[i] > x$ para $0 \leq i < n$).
 - a) Demuestre que, bajo la hipótesis del Problema 3.6.1 de que los valores son positivos, esa caracterización es correcta.
 - b) Suponga ahora que se permiten elementos de valor 0. Exhiba una instancia en la cual $\emptyset$ sea una solución óptima sin que se cumpla ninguna de las dos condiciones.
 - c) Revise la demostración del Teorema 3.6.1 y determine si sigue siendo válida bajo esa relajación. Concluya por qué conviene no partir de una caracterización de la solución óptima, incluso cuando se tiene una a mano.
7. En la tabulación propuesta para ϕ en `ks_tab` se opta por llenar `tab[0..N][0..X]` por columnas. Diseñe una tabulación para ϕ que resulte de llenar `tab` por filas. Bajo esta decisión de llenar la matriz, ¿es posible optimizar el espacio de la tabulación? Justifique su respuesta.
8. Demuestre el Teorema 3.6.2.
9. Diseñe un algoritmo con memorización para ϕ . Demuestre que es correcto con respecto a la especificación hecha. ¿Cuáles son sus complejidades temporal y espacial? Justifique su respuesta.

10. Investigue acerca de tiempo pseudo-polinomial y explique su relación con algoritmos numéricos. En particular, explique su relación con el algoritmo común para determinar si un número es primo.
11. Demuestre el Teorema 3.6.3.
12. En la función `ks_tab_opt1`, los papeles de presente y pasado se intercambian con la asignación `prev, curr = 1-prev, 1-curr`. Demuestre que si a es un número entero que satisface $0 \leq a \leq 1$, entonces $0 \leq 1 - a \leq 1$, y explique por qué eso es lo que garantiza que `prev` y `curr` sigan siendo índices válidos de la tabla.
13. En la función `ks_tab_opt1` se retorna el valor `tab[prev][X]`. Explique por qué sería incorrecto retornar el valor `tab[curr][X]`.
14. Demuestre el Teorema 3.6.4.
15. En el planteamiento de la función objetivo, la cantidad de elementos disponibles se hizo variar para considerar diferentes prefijos de los índices de $V[0..N]$. Plantee una definición alternativa a ϕ en la cual se hagan variar los diferentes sufijos de esos índices.
16. Hay una variante del problema del morral en la cual de cada elemento hay una cantidad ilimitada de copias (y se permite llevar cuantas copias sean deseadas de cada elemento).
 - a) Especifique esta variante del problema del morral.
 - b) Explique por qué la hipótesis de que los pesos son positivos es aquí indispensable y no solamente conveniente.
 - c) Diseñe una solución con tabulación para el problema especificado usando la Metodología 3.3.1. Si es posible reducir el espacio de la tabulación, redúzcalo al máximo.
17. La siguiente especificación corresponde al problema, comúnmente denominado, suma exacta de un subconjunto (en inglés, *Subset Sum*):

Entrada: Arreglo $A[0..N]$, con $N \geq 0$, de números enteros positivos y un número entero $X \geq 0$.

Salida: ¿Hay un subconjunto de elementos de $A[0..N]$ cuya suma es X ?

- a) En la literatura se indica que el problema de la suma exacta de un subconjunto es una instancia particular del problema del morral. Justifique por qué esta afirmación es cierta e ilustre su respuesta con ejemplos. ¿En dónde usa que los elementos de $A[0..N]$ son positivos?
- b) Diseñe un algoritmo con tabulación que permita resolver el problema de suma exacta de un subconjunto.
- c) El problema se puede plantear también sobre arreglos de números enteros cualesquiera. Explique qué parte de su respuesta al numeral (a) deja de funcionar en ese caso. Esa versión general es el tema de la Sección 5.5.

3.7. Subsecuencia común más larga

El problema de la subsecuencia común más larga consiste en determinar cuánto se parecen dos arreglos, midiendo ese parecido por la mayor cantidad de elementos que aparecen en ambos, en el mismo orden y no necesariamente de manera contigua. La noción de subsecuencia es la de la Definición 3.5.1; lo nuevo es que ahora hay dos arreglos y se busca algo que sea subsecuencia de los dos a la vez.

Definición 3.7.1: Subsecuencia común

Sean $A[0..M)$ y $B[0..N)$ arreglos. Una sucesión s es una *subsecuencia común* de A y B si y solo si s es subsecuencia de $A[0..M)$ y s es subsecuencia de $B[0..N)$, en el sentido de la Definición 3.5.1. La subsecuencia vacía es común a cualquier par de arreglos, de modo que siempre existe al menos una subsecuencia común.

Note que una subsecuencia común está determinada por *dos* conjuntos de índices, uno en cada arreglo, y que esos dos conjuntos pueden ser distintos entre sí.

Problema 3.7.1: Subsecuencia común más larga

Entrada: Arreglos $A[0..M)$ y $B[0..N)$, con $M \geq 0$ y $N \geq 0$.

Salida: Longitud de una subsecuencia común más larga de $A[0..M)$ y $B[0..N)$.

Ejemplo 3.7.1

Considere los arreglos de caracteres $A = \text{ALGORITMO}$ y $B = \text{LOGARITMO}$. La sucesión LGRITMO es una subsecuencia común: en A la determinan los índices 1, 2, 4, 5, 6, 7, 8 y en B los índices 0, 2, 4, 5, 6, 7, 8. Tiene longitud 7 y es una subsecuencia común más larga.

No es la única: LORITMO también es común a A y B y también tiene longitud 7. Como es usual en los problemas de optimización, lo que es único es el valor óptimo y no la configuración que lo alcanza.

Nota 3.7.1: Dónde aparece este problema

Este problema es el corazón de las herramientas que comparan archivos de texto línea por línea, como el programa `diff` y los sistemas de control de versiones: las líneas que pertenecen a una subsecuencia común más larga son las que se reportan como no modificadas, y todas las demás como insertadas o eliminadas. En biología computacional aparece al comparar secuencias de ADN o de proteínas, en donde el parecido entre dos secuencias es evidencia de parentesco evolutivo. Variantes cercanas del problema, como la distancia de edición, se usan en correctores ortográficos y en motores de búsqueda.

Una solución de fuerza bruta consiste en enumerar las 2^M subsecuencias de A y verificar, para cada una, si es también subsecuencia de B . Esa verificación se puede hacer en tiempo $O(N)$, de modo que el costo total es $O(2^M \cdot N)$: exponencial, como en los demás problemas de este capítulo. El objetivo es obtener una solución polinomial con programación dinámica.

Este es un problema de optimización, de modo que la propiedad que hay que verificar es la subestructura óptima de la Definición 3.1.1. Se sigue entonces la Metodología 3.3.1, cuyo paso de especificación ya quedó cumplido con el Problema 3.7.1.

Función objetivo. Como en el problema del morral, hay dos parámetros que se pueden hacer variar y en este caso son los tamaños de los dos arreglos. Para $0 \leq m \leq M$ y $0 \leq n \leq N$, se define:

$\phi(m, n)$: “longitud de una subsecuencia común más larga de $A[0..m]$ y $B[0..n]$.”

Es decir, se generaliza el problema a todos los pares de prefijos de los arreglos dados.

Reformulación de la especificación. El objetivo del problema es calcular $\phi(M, N)$. A diferencia de lo que ocurre en la Sección 3.4 y en la Sección 3.5, aquí no hace falta ninguna maximatoria externa: la función objetivo evaluada en los parámetros de la instancia es ya la salida del problema, como sucedía en la Sección 3.6.

Planteamiento recurrente. Se procede por casos sobre m y n . Si alguno de los dos prefijos es vacío no hay nada que compartir y la única subsecuencia común es la vacía; por lo tanto $\phi(m, n) = 0$ cuando $m = 0$ o $n = 0$.

Suponga entonces que $m \neq 0$ y $n \neq 0$, y fije la atención en los últimos elementos de los dos prefijos, $A[m-1]$ y $B[n-1]$. Hay dos posibilidades:

- Si $A[m-1] = B[n-1]$, conviene emparejar esos dos elementos y resolver el problema para los prefijos que quedan al quitarlos; es decir, $\phi(m, n) = 1 + \phi(m-1, n-1)$. Que esto sea correcto no es evidente: podría ocurrir que alguna subsecuencia común más larga no emparejara $A[m-1]$ con $B[n-1]$. Lo que se

demuestra más adelante es que, si eso ocurre, siempre se puede modificar esa subsecuencia para que sí lo haga, sin cambiar su longitud.

- Si $A[m-1] \neq B[n-1]$, entonces esos dos elementos no se pueden emparejar entre sí y, en consecuencia, al menos uno de los dos sobra. Como no se sabe cuál, se apuesta a los dos caballos: se descarta $A[m-1]$ y se recurre con $m-1$ y n , se descarta $B[n-1]$ y se recurre con m y $n-1$, y se prefiere la opción que produzca la mayor longitud.

En los tres casos los llamados recurrentes respetan las condiciones $0 \leq m \leq M$ y $0 \leq n \leq N$. La definición formal de ϕ se presenta para $0 \leq m \leq M$ y $0 \leq n \leq N$:

$$\phi(m, n) = \begin{cases} 0 & , \text{ si } m = 0 \vee n = 0, \\ 1 + \phi(m-1, n-1) & , \text{ si } m, n \neq 0 \wedge A[m-1] = B[n-1], \\ \phi(m-1, n) \uparrow \phi(m, n-1) & , \text{ si } m, n \neq 0 \wedge A[m-1] \neq B[n-1]. \end{cases}$$

Observe que el segundo caso tiene un solo llamado recurrente mientras que el tercero tiene dos. Es en el tercero donde se esconde el crecimiento exponencial que la programación dinámica va a evitar.

Correctitud de la función objetivo. El Teorema 3.7.1 presenta el criterio de correctitud de la función ϕ .

Teorema 3.7.1

Si $0 \leq m \leq M$ y $0 \leq n \leq N$, entonces $\phi(m, n)$ es la longitud de una subsecuencia común más larga de $A[0..m]$ y $B[0..n]$.

Demostración

Sea $L(m, n)$ la longitud de una subsecuencia común más larga de $A[0..m]$ y $B[0..n]$. Se procede por inducción sobre $m+n$, suponiendo que $\phi(i, j) = L(i, j)$ para todo par (i, j) con $i+j < m+n$.

Si $m = 0$ o $n = 0$, la única subsecuencia común es la vacía, luego $L(m, n) = 0 = \phi(m, n)$. Suponga entonces que $m \neq 0$ y $n \neq 0$, y sea s una subsecuencia común de longitud $k = L(m, n)$. Se distinguen dos casos.

- Caso $A[m-1] = B[n-1]$. Se procede por doble desigualdad.

Composición: $L(m, n) \geq 1 + L(m-1, n-1)$. Al tomar una subsecuencia común más larga de $A[0..m-1]$ y $B[0..n-1]$ y extenderla con el elemento $A[m-1]$ se obtiene una subsecuencia común de $A[0..m]$ y $B[0..n]$, pues $A[m-1] = B[n-1]$ y los índices usados son menores que $m-1$ y $n-1$, respectivamente.

Descomposición: $L(m, n) \leq 1 + L(m-1, n-1)$. Note que $k \geq 1$, ya que el elemento $A[m-1]$ por sí solo es una subsecuencia común. Sean i y j los índices en A y en B del último elemento de s . Si $i \neq m-1$ o $j \neq n-1$, considere la sucesión s' que resulta de quitarle a s su último elemento y agregarle $A[m-1]$: como los índices que s usa antes de su último elemento son menores que $i \leq m-1$ y que $j \leq n-1$, la sucesión s' es una subsecuencia común de la misma longitud k cuyo último elemento está emparejado en los índices $m-1$ y $n-1$. Sin pérdida de generalidad se puede suponer, entonces, que $i = m-1$ y $j = n-1$. Al quitarle a s su último elemento queda una subsecuencia común de $A[0..m-1)$ y $B[0..n-1)$ de longitud $k-1$, de donde $L(m-1, n-1) \geq k-1$.

De las dos desigualdades y de la hipótesis inductiva se concluye $\phi(m, n) = 1 + \phi(m-1, n-1) = 1 + L(m-1, n-1) = L(m, n)$.

- Caso $A[m-1] \neq B[n-1]$. Se procede por doble desigualdad.

Composición: $L(m, n) \geq L(m-1, n)$ y $L(m, n) \geq L(m, n-1)$, pues toda subsecuencia común de un par de prefijos lo es también de cualquier par de prefijos más largos.

Descomposición: $L(m, n) \leq L(m-1, n) \uparrow L(m, n-1)$. Si $k = 0$ la desigualdad es inmediata. De lo contrario, sean i y j los índices en A y en B del último elemento de s ; se tiene $A[i] = B[j]$ y, como $A[m-1] \neq B[n-1]$, no puede suceder a la vez que $i = m-1$ y $j = n-1$. Si $i < m-1$, entonces s es subsecuencia común de $A[0..m-1)$ y $B[0..n)$, luego $k \leq L(m-1, n)$. Si $j < n-1$, un argumento simétrico da $k \leq L(m, n-1)$.

De las dos desigualdades y de la hipótesis inductiva se concluye:

$$\begin{aligned}\phi(m, n) &= \phi(m-1, n) \uparrow \phi(m, n-1) \\ &= L(m-1, n) \uparrow L(m, n-1) \\ &= L(m, n).\end{aligned}$$

Note que las desigualdades de esta demostración se establecen entre L y combinaciones de L , y solo al final se invoca la hipótesis inductiva para pasar a ϕ ; en las secciones anteriores se establecen directamente entre ϕ y L . Las dos formas caben en el patrón de la Sección 3.3 y cada una resulta más cómoda según el problema, pero conviene reconocerlas como la misma cosa.

La propiedad de solapamiento se observa con facilidad: $\phi(m-1, n)$ y $\phi(m, n-1)$ invocan ambos a $\phi(m-1, n-1)$. Es exactamente la misma situación del Problema 3.2.1, con dos llamados que confluyen en un tercero: esto hace crecer exponencialmente una implementación que se limite a transcribir la definición de ϕ .

3.7.1. Tabulación. El siguiente paso es determinar si se usa memorización o tabulación.

¿Memorización o tabulación? Se diseñará una tabulación con base en el diagrama de necesidades de la Figura 10. La tabulación se implementa con una

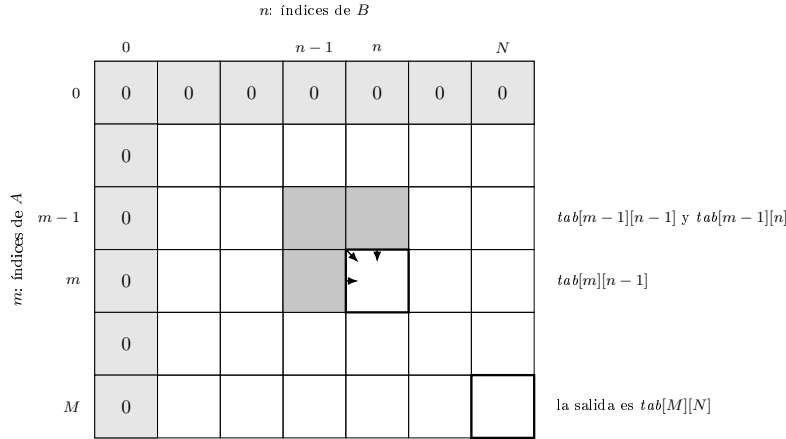


Figura 10. Diagrama de necesidades para la tabulación de ϕ en el problema de la subsecuencia común más larga. La primera fila y la primera columna corresponden a los casos base. El valor $tab[m][n]$ depende de las tres celdas vecinas que lo preceden: la diagonal, la superior y la izquierda.

matriz $tab[0..M][0..N]$ con $M + 1$ filas y $N + 1$ columnas. El objetivo es calcular el valor correspondiente a $tab[M][N]$, dado que el problema se resuelve con $\phi(M, N)$. La primera fila y la primera columna corresponden al caso base y tienen únicamente el valor 0. Para calcular $tab[m][n]$ en los casos recurrentes se depende de $tab[m - 1][n - 1]$, de $tab[m - 1][n]$ y de $tab[m][n - 1]$; cuál de ellos se usa depende de si $A[m - 1]$ y $B[n - 1]$ coinciden o no.

Las tres celdas de las que depende $tab[m][n]$ están en la fila $m - 1$ o a la izquierda en la fila m . Esto indica que se puede llenar la tabulación por filas, de arriba hacia abajo y, dentro de cada fila, de izquierda a derecha. Esta decisión se ve reflejada en los siguientes invariantes:

$$P_0 : (\forall i, j \mid 0 \leq i < m \wedge 0 \leq j \leq N : tab[i][j] = \phi(i, j)).$$

$$P_1 : (\forall j \mid 0 \leq j < n : tab[m][j] = \phi(m, j)).$$

$$P_2 : 0 \leq m \leq M + 1.$$

$$P_3 : 0 \leq n \leq N + 1.$$

El invariante P_0 indica que todas las filas por encima de la fila m han sido llenadas con los valores correspondientes de ϕ . El invariante P_1 especifica que los valores en la fila m , a la izquierda de la columna n , corresponden a los valores de ϕ .

Los invariantes P_2 y P_3 son de apoyo y, como en la Sección 3.6, permiten que las variables tomen un valor por fuera del rango de la tabla para señalar que se terminó de llenar una fila o la tabla completa. Con base en este diseño se presenta la función `lcs_tab` en el Algoritmo 3.7.1.

Implementación. El Algoritmo 3.7.1 presenta una implementación de la tabulación diseñada para ϕ .

Algoritmo 3.7.1: Tabulación para ϕ

```

1 def lcs_tab(A, B):
2     M,N = len(A),len(B)
3     tab = [ [ None for _ in range(N+1) ] for _ in range(M+1) ]
4     for n in range(N+1): tab[0][n] = 0
5     m,n = 1,0
6     #  $P_0 \wedge P_1 \wedge P_2 \wedge P_3$ 
7     while m!=M+1:
8         if n==N+1: m,n = m+1,0
9         else:
10            if n==0: tab[m][n] = 0
11            elif A[m-1]==B[n-1]: tab[m][n] = tab[m-1][n-1]+1
12            else: tab[m][n] = max(tab[m-1][n], tab[m][n-1])
13            n += 1
14     return tab[M][N]
```

La estructura de `lcs_tab` es la misma de `ks_tab` en la Sección 3.6, con los papeles de filas y columnas intercambiados. El cuerpo del ciclo tiene dos partes: una de cambio de fila, cuando ya se llenó la fila actual, y otra que calcula una celda distinguiendo los tres casos de la definición de ϕ . Note que la primera columna se trata dentro del ciclo, con la condición `n==0`, mientras que la primera fila se inicializa antes de entrar en él.

Se ilustra el uso de la función con algunas instancias:

```

1 >>> print(lcs_tab('ALGORITMO', 'LOGARITMO'))
2 7
3 >>> print(lcs_tab('ABC', 'ABC'))
4 3
5 >>> print(lcs_tab('ABC', 'XYZ'))
6 0
7 >>> print(lcs_tab('', 'ABC'))
8 0
```

Análisis. La correctitud y el análisis asintótico del Algoritmo 3.7.1 se enuncian en el Teorema 3.7.2.

Teorema 3.7.2

El llamado `lcs_tab(A, B)`, con $\text{len}(A) = M$ y $\text{len}(B) = N$:

1. Calcula la longitud de una subsecuencia común más larga de $A[0..M)$ y $B[0..N)$.
2. Toma tiempo $O(MN)$.
3. Usa espacio $O(MN)$.

Demostración

Se propone como ejercicio al lector.

A diferencia de lo que ocurre en la Sección 3.6, esta complejidad es genuinamente polinomial en el tamaño de la entrada: M y N son las longitudes de los arreglos y no valores numéricos que aparecen en ellos, de modo que no hay aquí nada parecido al tiempo pseudo-polinomial.

3.7.2. Reducción del espacio. El diagrama de necesidades muestra que el cálculo de la fila m solo depende de valores en la fila $m - 1$ y de valores ya calculados en la fila m misma. Como en la Sección 3.6, esto sugiere que no hace falta conservar la tabla completa. Ahora bien, la reducción no es idéntica a la de aquella sección y conviene entender por qué.

Nota 3.7.2: Por qué no funciona el recorrido en reversa

En la Sección 3.6 el espacio se redujo a un solo arreglo recorriéndolo en sentido contrario al del llenado. El truco funciona allí porque el cálculo de una celda depende únicamente de celdas del pasado inmediato: al recorrer en reversa, las posiciones que aún no se han sobrescrito son exactamente las que se necesitan.

Aquí ese truco falla, porque $\phi(m, n)$ depende también de $\phi(m, n - 1)$, que es un valor del *presente*. Recorrer la fila de derecha a izquierda dejaría ese valor sin calcular. Y recorrerla de izquierda a derecha, sobre un único arreglo, destruye el valor $\phi(m - 1, n - 1)$ justo un paso antes de necesitarlo: al escribir en la posición $n - 1$ se pierde el dato que hará falta al procesar la posición n .

La solución es conservar ese único valor en una variable auxiliar antes de sobrescribirlo. Es un patrón que vale la pena reconocer: cuando una reducción

de espacio falla por un solo dato, casi siempre basta con una variable adicional en lugar de renunciar a la reducción.

Vale la pena observar, además, que en este problema *cuáles* de las tres celdas vecinas se consultan depende de los datos: si $A[m-1] = B[n-1]$ se usa la diagonal y si no, las otras dos. Eso no impide la reducción, y es justamente el punto que señala la Nota 3.5.3: lo que decide el asunto no es si las dependencias dependen de la instancia sino si la ventana está acotada. Aquí lo está, en tres celdas vecinas, aunque no siempre sean las mismas tres.

Con base en esa observación se propone el diagrama de necesidades de la Figura 11, en el cual la tabulación es un único arreglo $row[0..N]$ acompañado de una variable $prev$. Los invariantes del ciclo interno, para una fila m fija, hacen explícitas

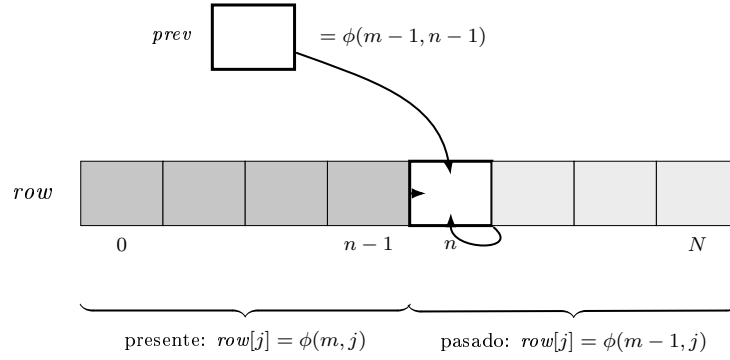


Figura 11. Diagrama de necesidades de la reducción del espacio en el problema de la subsecuencia común más larga. El arreglo row tiene, a la izquierda de la posición n , valores del presente y, de la posición n en adelante, valores del pasado inmediato. La variable $prev$ conserva el valor $\phi(m-1, n-1)$, que ya fue sobrescrito en row .

estas observaciones:

$$R_0 : (\forall j \mid 0 \leq j < n : row[j] = \phi(m, j)).$$

$$R_1 : (\forall j \mid n \leq j \leq N : row[j] = \phi(m-1, j)).$$

$$R_2 : prev = \phi(m-1, n-1).$$

$$R_3 : 1 \leq n \leq N+1.$$

El invariante R_0 especifica que el subarreglo $row[0..n)$ tiene los valores del presente y el invariante R_1 que el subarreglo $row[n..N]$ tiene los del pasado inmediato. En particular, $row[n]$ es $\phi(m-1, n)$ y $row[n-1]$ es $\phi(m, n-1)$: los dos valores que

requiere el tercer caso de la definición de ϕ . El invariante R_2 conserva el tercer valor necesario, $\phi(m-1, n-1)$, que ya no está disponible en *row*. El resultado es la función `lcs_tab_opt` del Algoritmo 3.7.2.

Algoritmo 3.7.2: Reducción del espacio de la tabulación

```

1 def lcs_tab_opt(A, B):
2     M, N = len(A), len(B)
3     row = [ 0 for _ in range(N+1) ]
4     m = 1
5     while m != M+1:
6         prev, n = 0, 1
7         #  $R_0 \wedge R_1 \wedge R_2 \wedge R_3$ 
8         while n != N+1:
9             curr = row[n]
10            if A[m-1] == B[n-1]: row[n] = prev+1
11            else: row[n] = max(row[n], row[n-1])
12            prev, n = curr, n+1
13        m += 1
14    return row[N]
```

La variable `curr` guarda el valor de `row[n]` antes de sobrescribirlo, es decir, guarda $\phi(m-1, n)$. Al final de la iteración ese valor pasa a `prev`, que en la iteración siguiente –en la cual la variable n vale uno más– contendrá exactamente $\phi(m-1, n-1)$, como lo exige R_2 . La asignación inicial `prev, n = 0, 1` establece R_2 al comenzar cada fila, pues $\phi(m-1, 0) = 0$ para cualquier m .

Teorema 3.7.3

El llamado `lcs_tab_opt(A, B)`, con $\text{len}(A) = M$ y $\text{len}(B) = N$:

1. Calcula la longitud de una subsecuencia común más larga de $A[0..M)$ y $B[0..N)$.
2. Toma tiempo $O(MN)$.
3. Usa espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

Como el espacio utilizado es $O(N)$ y el problema es simétrico en sus dos argumentos, conviene invocar la función con el arreglo más corto en el segundo parámetro, obteniendo espacio $O(\min(M, N))$.

3.7.3. Reconstrucción de una solución óptima. Hasta este punto, todos los algoritmos del capítulo calculan el *valor* de una solución óptima y ninguno exhibe una solución que lo alcance. Para el problema del morral se obtiene el mejor beneficio posible pero no qué elementos llevar; para el problema del agente viajero, el costo del mejor circuito pero no el circuito. Esta sección cierra esa brecha, y lo hace aquí porque es donde la reconstrucción entra en conflicto con la reducción del espacio que acaba de hacerse.

Problema 3.7.2: Reconstrucción de una subsecuencia común más larga

Entrada: Arreglos $A[0..M]$ y $B[0..N]$, con $M \geq 0$ y $N \geq 0$.

Salida: Una subsecuencia común más larga de $A[0..M]$ y $B[0..N]$.

La idea es que la tabulación contiene, además del valor óptimo, el rastro de las decisiones que lo produjeron. Basta con recorrerla en sentido contrario, desde la celda $tab[M][N]$ hacia la celda $tab[0][0]$, preguntando en cada paso cuál de los tres casos de la definición de ϕ fue el que se aplicó:

- Si $A[m-1] = B[n-1]$, se aplicó el segundo caso. Ese elemento hace parte de la subsecuencia que se está reconstruyendo y el recorrido continúa en $tab[m-1][n-1]$.
- Si $A[m-1] \neq B[n-1]$, se aplicó el tercer caso y el valor proviene del mayor entre $tab[m-1][n]$ y $tab[m][n-1]$. El recorrido continúa en aquella de las dos celdas que alcanza el máximo; si las dos lo alcanzan, cualquiera de ellas sirve.
- El recorrido termina al llegar a la fila 0 o a la columna 0, pues allí la subsecuencia común es vacía.

Como los elementos se descubren de derecha a izquierda, se obtienen en orden inverso y hay que invertirlos al final. El Algoritmo 3.7.3 implementa esta estrategia. Recibe la tabulación completa como parámetro, la cual se obtiene de `lcs_tab` cambiando su última línea por `return tab`.

Algoritmo 3.7.3: Reconstrucción de una solución óptima

```

1 def lcs_rec(A, B, tab):
2     ans, m, n = [], len(A), len(B)
3     while m != 0 and n != 0:
```

```

4     if A[m-1]==B[n-1]:
5         ans.append(A[m-1])
6         m,n = m-1,n-1
7     elif tab[m-1][n]>=tab[m][n-1]: m -= 1
8     else: n -= 1
9     ans.reverse()
10    return ans

```

Se ilustra el uso de la función con la instancia del Ejemplo 3.7.1:

```

1 >>> A,B = 'ALGORITMO', 'LOGARITMO'
2 >>> print(''.join(lcs_rec(A, B, lcs_tab(A, B))))
3 LGRITMO

```

Teorema 3.7.4

Sea tab la tabulación construida por el Algoritmo 3.7.1 para los arreglos $A[0..M]$ y $B[0..N]$. El llamado $lcs_rec(A, B, tab)$:

1. Calcula una subsecuencia común más larga de $A[0..M]$ y $B[0..N]$.
2. Toma tiempo $O(M + N)$.
3. Usa espacio $O(M + N)$, sin contar el de la tabulación.

Demostración

Se propone como ejercicio al lector.

El costo de la reconstrucción es despreciable frente al de construir la tabla: en cada iteración del ciclo disminuye m , disminuye n o disminuyen ambos, de modo que a lo sumo se ejecutan $M + N$ iteraciones. Lo que no es despreciable es el espacio.

Nota 3.7.3: La tensión entre reducir y reconstruir

El Algoritmo 3.7.3 necesita la tabulación completa y por eso no se puede combinar con el Algoritmo 3.7.2: la reducción del espacio conserva el valor óptimo pero destruye el rastro de las decisiones que lo produjeron. Es decir, para este problema se puede calcular el valor óptimo en espacio $O(\min(M, N))$, o exhibir una solución óptima en espacio $O(MN)$, pero los dos algoritmos presentados no permiten hacer ambas cosas a la vez.

Esta tensión conviene tenerla presente cada vez que se reduzca el espacio de una tabulación: la pregunta pertinente no es solamente si el valor óptimo se puede seguir calculando, sino también qué se pierde con la información que se descarta. No es, eso sí, inevitable: en la Sección 3.8 se reconstruyen las soluciones óptimas sin costo adicional de espacio, porque allí el rastro de las decisiones cabe en un solo dato por celda.

Vale la pena señalar que la tensión sí se puede resolver para este problema en particular. Existe un algoritmo, debido a D. Hirschberg, que reconstruye una subsecuencia común más larga en tiempo $O(MN)$ y espacio $O(\min(M, N))$, combinando la programación dinámica de esta sección con la técnica de dividir, conquistar y combinar. La idea es calcular, con el algoritmo de espacio reducido, por dónde cruza una solución óptima la fila central de la tabla, y recurrir sobre las dos mitades que quedan. En las notas del final del capítulo se indican referencias.

Ejercicios

1. Demuestre que si $A[0..M)$ y $B[0..N)$ no tienen ningún elemento en común, entonces todas las entradas de la tabulación son 0.
2. Demuestre que ϕ es monótona en cada uno de sus argumentos; es decir, que $\phi(m-1, n) \leq \phi(m, n)$ y $\phi(m, n-1) \leq \phi(m, n)$ siempre que los índices estén en rango. Concluya que dos celdas vecinas de la tabulación difieren en 0 o en 1.
3. Elabore un ejemplo en el cual se exhiba la propiedad de solapamiento para ϕ . Explique brevemente su respuesta.
4. Demuestre el Teorema 3.7.2.
5. Demuestre el Teorema 3.7.3. En particular, demuestre que el ciclo interno satisface los invariantes R_0 , R_1 , R_2 y R_3 .
6. Demuestre el Teorema 3.7.4.
7. Diseñe un algoritmo con memorización para ϕ instanciando el Algoritmo 3.3.1, e incluya las condiciones de representación que debe satisfacer la memoria compartida. ¿Cuáles son sus complejidades temporal y espacial?
8. En el Ejemplo 3.7.1 se exhiben dos subsecuencias comunes más largas de ALGORITMO y LOGARITMO.
 - a) ¿Cuál de las dos retorna el Algoritmo 3.7.3? Explique por qué.
 - b) Modifique la línea 5 del algoritmo cambiando $\geq$ por $>$. ¿Cambia la longitud de la respuesta? ¿Cambia la subsecuencia retornada?

9. Modifique el Algoritmo 3.7.1 para que, además de la longitud, calcule la *cantidad* de subsecuencias comunes más largas distintas que hay. Cuidado: dos conjuntos de índices distintos pueden determinar la misma sucesión de valores; decida explícitamente cuál de las dos cantidades está calculando.
 10. El problema de la *subcadena* común más larga pide la mayor longitud de un subarreglo que aparezca en $A[0..M)$ y en $B[0..N)$; es decir, la versión contigua de este problema.
 - a) Especifique el problema.
 - b) Explique por qué la definición de ϕ de esta sección no lo resuelve, e ilustre su explicación con un ejemplo.
 - c) Diseñe una función objetivo y una definición recurrente apropiadas, en el espíritu de la función objetivo de la Sección 3.4, y proponga una tabulación.
 11. La *distancia de edición* entre dos arreglos es la mínima cantidad de operaciones de inserción, eliminación y sustitución de un elemento que se requieren para transformar el primero en el segundo.
 - a) Especifique el problema de calcular la distancia de edición.
 - b) Siga la Metodología 3.3.1 para diseñar una solución con tabulación. Debe resultar código en Python ejecutable.
 - c) ¿Se puede reducir el espacio de la tabulación? ¿Se puede reconstruir una secuencia de operaciones de costo mínimo?
 12. Suponga que $A[0..N)$ es un arreglo cuyos elementos son todos distintos y sea $B[0..N)$ el arreglo que resulta de ordenar A de manera creciente. Demuestre que una subsecuencia común más larga de A y B es una subsecuencia creciente más larga de A (Sección 3.5). ¿Qué complejidad temporal tiene el algoritmo que resulta y cómo se compara con la del Algoritmo 3.5.1? ¿Qué sucede si A tiene elementos repetidos?
 13. Generalice el problema a tres arreglos $A[0..M)$, $B[0..N)$ y $C[0..K)$. Proponga una función objetivo y su definición recurrente, y determine las complejidades temporal y espacial de la tabulación correspondiente. ¿Cuántos casos tiene la definición recurrente?
 14. 🌐 Diseñe el algoritmo de Hirschberg esbozado en la Nota 3.7.3, que reconstruye una subsecuencia común más larga en tiempo $O(MN)$ y espacio $O(\min(M, N))$. Justifique la cota de espacio y explique por qué la cota de tiempo se mantiene a pesar de la recurrencia.
-

3.8. Caminos mínimos entre todos los vértices

El problema de caminos mínimos entre todos los pares consiste en calcular, para cada par de vértices de un grafo dirigido y pesado, el costo del camino más

barato que los une. Es el problema con el cual se abrió la Sección 3.1 –allí se usó la ruta más corta entre dos puntos de una ciudad para ilustrar la propiedad de subestructura óptima– y esta sección lo resuelve en su forma general.

Nota 3.8.1: Grafos dirigidos y pesados

Un *grafo* G es una pareja (V, E) de *vértices* V y de *arcos* $E \subseteq V \times V$ sobre los vértices. Si $(u, v) \in E$ se dice que hay un arco entre u y v . El grafo puede ser *dirigido* o *no-dirigido*. En un grafo dirigido los arcos tienen dirección, es decir, un origen y un destino. En un grafo no-dirigido los arcos no tienen dirección, únicamente conectan dos vértices. En este texto, mientras no se haga la aclaración, los grafos se suponen no-dirigidos; en esta sección, en cambio, se suponen dirigidos.

Un *camino* de u a v en G es una sucesión de vértices $u = x_0, x_1, \dots, x_\ell = v$, con $\ell \geq 0$, tal que $(x_i, x_{i+1}) \in E$ para $0 \leq i < \ell$. Los vértices $x_1, \dots, x_{\ell-1}$ se llaman *intermedios*. El camino es *simple* si no repite vértices. Un *ciclo* es un camino de un vértice a sí mismo con $\ell \geq 1$. Note que para cualquier vértice u existe el camino de u a u con $\ell = 0$, llamado *camino vacío*.

Dada una función de peso $w : E \rightarrow \mathbb{R}$, el *costo* de un camino es la suma de los pesos de los arcos que lo componen; el camino vacío tiene costo 0.

Las relaciones de amistad o de seguimiento en redes sociales se pueden modelar como grafos, al igual que sistemas biológicos como redes de co-expresión genética o redes de interacción entre proteínas.

A diferencia de lo que sucede en otras secciones de este capítulo, los pesos aquí pueden ser negativos. Esa libertad tiene un precio y es la única hipótesis que hay que imponer sobre la entrada.

Problema 3.8.1: Caminos mínimos entre todos los pares

Entrada: Un grafo dirigido $G = (V, E)$, con $N = |V| \geq 0$ vértices y función de peso $w : E \rightarrow \mathbb{R}$, sin ciclos de peso negativo.

Salida: El costo mínimo de un camino de u a v , para cada par de vértices $u, v \in V$.

Nota 3.8.2: ¿Por qué se excluyen los ciclos de peso negativo?

Suponga que en G hay un ciclo de peso negativo y que dicho ciclo es alcanzable desde u y alcanza a v . Entonces no existe un camino de costo mínimo de u a v : dado cualquier camino, se puede obtener otro más barato recorriendo el ciclo una vez más y así indefinidamente. El ínfimo de los costos es $-\infty$ y el problema queda mal planteado. Por eso la hipótesis aparece en la especificación y no como un detalle técnico.

Al excluirlos se gana algo más que buen planteamiento. Si no hay ciclos de peso negativo, entonces todo ciclo tiene costo no negativo y, en consecuencia, quitarle a un camino los ciclos que contenga no aumenta su costo. Luego, si existe algún camino de u a v , entonces existe uno *simple* de costo mínimo. Como en un grafo de N vértices hay una cantidad finita de caminos simples, el mínimo se alcanza y el problema tiene solución. La demostración de esta afirmación se propone como ejercicio al lector.

Vale la pena contrastar esta situación con la del Ejemplo 3.1.2, en donde exigir que los caminos fueran simples destruía la propiedad de subestructura óptima. Aquí la simplicidad de los caminos no es una restricción impuesta desde afuera sino una consecuencia de la hipótesis sobre los pesos y por eso no causa el mismo problema.

Se supondrá que $V = \{0, 1, \dots, N - 1\}$ y que el grafo se representa con una matriz de adyacencia $w[0..N][0..N]$ tal que $w[u][v]$ es el peso del arco (u, v) cuando este existe, $w[u][u] = 0$ y $w[u][v] = +\infty$ cuando no hay arco de u a v . Note que esta matriz *no* es necesariamente simétrica, pues el grafo es dirigido.

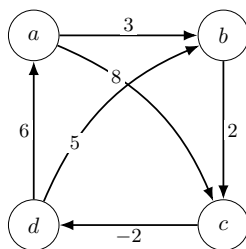


Figura 12. Un grafo dirigido y pesado con cuatro vértices y un arco de peso negativo. No tiene ciclos de peso negativo.

Ejemplo 3.8.1

Considere el grafo dirigido con 4 vértices de la Figura 12, en el cual el arco de c a d tiene peso negativo. El grafo no tiene ciclos de peso negativo: el único ciclo es $b \rightarrow c \rightarrow d \rightarrow b$, de costo $2 + (-2) + 5 = 5$. La matriz de costos mínimos entre todos los pares es la siguiente:

	a	b	c	d
a	0	3	5	3
b	6	0	2	0
c	4	3	0	-2
d	6	5	7	0

Observe que el costo mínimo de a a c es 5 y no 8: conviene ignorar el arco directo y pasar por b . Del mismo modo, el costo mínimo de c a a es 4, alcanzado por el camino $c \rightarrow d \rightarrow a$ que aprovecha el arco de peso negativo.

Nota 3.8.3: ¿Por qué no basta con un algoritmo voraz?

El problema de caminos mínimos desde un único origen y el algoritmo voraz de Dijkstra que lo resuelve son el tema de la Sección 4.8. Ejecutar ese algoritmo una vez por cada vértice resolvería el problema de esta sección y en muchos casos sería la mejor opción disponible.

Sin embargo, la correctitud del algoritmo de Dijkstra depende de que los pesos sean no negativos: su decisión voraz consiste en dar por definitiva la distancia del vértice más cercano aún no procesado y un arco de peso negativo puede invalidar esa decisión más adelante. Con pesos arbitrarios, entonces, la estrategia deja de servir. El algoritmo de esta sección no impone esa restricción y esa es precisamente su ventaja. En los ejercicios se explora una tercera vía, debida a D. Johnson, que reformula los pesos para poder usar Dijkstra incluso cuando hay arcos negativos.

Nota 3.8.4: Sobre el nombre del algoritmo

El algoritmo que se obtiene al final de esta sección se conoce universalmente como *algoritmo de Floyd–Warshall*, por R. Floyd y S. Warshall, quienes lo publicaron de manera independiente en 1962. El registro histórico, sin embargo, es más generoso: B. Roy había descrito el mismo esquema en 1959, tres años antes. Por esa razón, en este texto se prefiere llamarlo *algoritmo de Roy–Floyd–Warshall*, con los tres nombres en orden cronológico.

Conviene precisar en qué consistió el aporte de cada uno. Roy y Warshall se ocuparon de la *clausura transitiva* de un grafo, es decir, de determinar qué vértices son alcanzables desde cuáles; Floyd fue quien observó que el mismo esquema de recurrencia sirve para calcular distancias mínimas y no solamente alcanzabilidad. No se trata del mismo problema, sino de la misma idea aplicada a dos problemas distintos. En los ejercicios y en las notas del final del capítulo se explora hasta dónde llega esa idea.

Una advertencia práctica para el lector: en la literatura, en los cursos y en internet, el algoritmo aparece casi siempre bajo el nombre de Floyd–Warshall. Al buscar material complementario conviene usar ese nombre.

Este es un problema de optimización, de modo que la propiedad que hay que verificar es la subestructura óptima de la Definición 3.1.1. El problema la exhibe, junto con la de solapamiento, así que se sigue la Metodología 3.3.1; el paso de especificación ya quedó cumplido con el Problema 3.8.1.

Función objetivo. La salida del problema tiene dos parámetros, los vértices u y v , pero hacerlos variar no basta: es lo que ya hace la especificación y no produce subproblemas más simples. Hace falta un tercer parámetro que mida el progreso y la elección acertada –la de Roy, Warshall y Floyd– consiste en restringir cuáles vértices se permiten como intermedios. Para $0 \leq k \leq N$ y $u, v \in V$, se define:

$\phi(k, u, v)$: “costo mínimo de un camino de u a v cuyos vértices intermedios están todos en $\{0, \dots, k-1\}$.”

Es decir, k cuenta cuántos vértices están habilitados para ser usados de paso, comenzando por el vértice 0. Cuando $k = 0$ no se permite ningún vértice intermedio y cuando $k = N$ se permiten todos. Note que la restricción recae sobre los vértices *intermedios*: los extremos u y v son siempre libres.

Reformulación de la especificación. El objetivo del problema es calcular $\phi(N, u, v)$ para cada par de vértices $u, v \in V$, pues con $k = N$ la restricción sobre los vértices intermedios desaparece.

Planteamiento recurrente. El caso base es $k = 0$: sin vértices intermedios, el único camino posible de u a v es el arco directo, cuyo costo es $w[u][v]$ o el camino vacío cuando $u = v$, cuyo costo es 0. La convención adoptada para la matriz de adyacencia cubre las tres situaciones a la vez.

Suponga ahora que $k \neq 0$ y considere un camino de costo mínimo de u a v con vértices intermedios en $\{0, \dots, k-1\}$. Por la Nota 3.8.2 se puede suponer que es simple. La pregunta que organiza el análisis es si ese camino usa o no el vértice $k-1$, que es el que se acaba de habilitar:

- Si no lo usa, entonces sus vértices intermedios están en $\{0, \dots, k-2\}$ y su costo es $\phi(k-1, u, v)$.

- Si lo usa, entonces lo usa exactamente una vez, por ser simple. Eso permite partir el camino en dos: el tramo de u a $k-1$ y el tramo de $k-1$ a v . Ninguno de los dos tramos usa $k-1$ como vértice intermedio, de modo que ambos tienen sus intermedios en $\{0, \dots, k-2\}$, y cada uno debe ser de costo mínimo –de lo contrario el camino completo no lo sería–. Su costo es entonces $\phi(k-1, u, k-1) + \phi(k-1, k-1, v)$.

Como no se sabe de antemano cuál de las dos posibilidades se da, se prefiere la más barata. La definición formal de ϕ se presenta para $0 \leq k \leq N$ y $u, v \in V$:

$$\phi(k, u, v) = \begin{cases} w[u][v] & , \text{ si } k = 0, \\ \phi(k-1, u, v) \downarrow \\ (\phi(k-1, u, k-1) + \phi(k-1, k-1, v)) & , \text{ si } k \neq 0. \end{cases}$$

La definición es notablemente más simple que la del problema del morral o la del agente viajero: un solo caso recurrente, sin condiciones sobre los datos. Toda la dificultad está en haber encontrado el tercer parámetro.

Correctitud de la función objetivo. La correctitud de la función objetivo se presenta en el Teorema 3.8.1.

Teorema 3.8.1

Sea G un grafo dirigido sin ciclos de peso negativo. Si $0 \leq k \leq N$ y $u, v \in V$, entonces $\phi(k, u, v)$ es el costo mínimo de un camino de u a v cuyos vértices intermedios están todos en $\{0, \dots, k-1\}$.

Demostración

Sea $L(k, u, v)$ el costo mínimo de un camino de u a v con vértices intermedios en $\{0, \dots, k-1\}$, con el convenio de que $L(k, u, v) = +\infty$ cuando no existe tal camino. Se procede por inducción sobre k .

Caso $k = 0$: los caminos de u a v sin vértices intermedios son el arco (u, v) , si existe, y el camino vacío cuando $u = v$. Por la convención adoptada para la matriz de adyacencia, $L(0, u, v) = w[u][v] = \phi(0, u, v)$.

Caso $k \neq 0$: suponga que $\phi(k-1, x, y) = L(k-1, x, y)$ para cualesquiera $x, y \in V$. Se demuestran las dos desigualdades. Note que, por tratarse de un problema de minimización, la mitad de composición produce la desigualdad $\phi \geq L$ y la de descomposición la contraria; en las secciones anteriores, que son de maximización, los papeles están invertidos.

- *Composición:* $\phi(k, u, v) \geq L(k, u, v)$. Basta ver que $L(k, u, v)$ no excede a ninguno de los dos argumentos del mínimo. Si alguno de ellos vale $+\infty$,

la desigualdad correspondiente es inmediata, así que se pueden suponer finitos.

Para el primero: por hipótesis inductiva, $\phi(k-1, u, v) = L(k-1, u, v)$ es el costo de un camino de u a v con intermedios en $\{0, \dots, k-2\}$, que también los tiene en $\{0, \dots, k-1\}$; luego $L(k, u, v) \leq \phi(k-1, u, v)$.

Para el segundo: de nuevo por hipótesis inductiva, hay un camino de u a $k-1$ de costo $\phi(k-1, u, k-1)$ y otro de $k-1$ a v de costo $\phi(k-1, k-1, v)$, ambos con intermedios en $\{0, \dots, k-2\}$. Al concatenarlos se obtiene una caminata de u a v cuyos vértices interiores están en $\{0, \dots, k-1\}$, aunque puede repetir vértices. Quitarle los ciclos produce un camino simple de u a v cuyo costo no es mayor –pues ningún ciclo tiene costo negativo– y cuyos vértices intermedios son un subconjunto de los anteriores. Ese camino es admisible, de donde $L(k, u, v) \leq \phi(k-1, u, k-1) + \phi(k-1, k-1, v)$.

- *Descomposición:* $\phi(k, u, v) \leq L(k, u, v)$. Si no hay camino admisible, entonces se tiene $L(k, u, v) = +\infty$ y la desigualdad es inmediata. De lo contrario, sea P un camino simple de u a v de costo $L(k, u, v)$ con intermedios en $\{0, \dots, k-1\}$, el cual existe por la Nota 3.8.2. Si $k-1$ no es intermedio de P , entonces P tiene sus intermedios en $\{0, \dots, k-2\}$ y, por hipótesis inductiva, $\phi(k-1, u, v) \leq \text{cost}(P) = L(k, u, v)$; como $\phi(k, u, v) \leq \phi(k-1, u, v)$, se concluye. Si $k-1$ sí es intermedio de P , entonces aparece exactamente una vez por ser P simple, y P se parte en P_1 , de u a $k-1$, y P_2 , de $k-1$ a v . Ninguno de los dos tiene a $k-1$ como intermedio, luego ambos tienen sus intermedios en $\{0, \dots, k-2\}$ y, por hipótesis inductiva,

$$\phi(k-1, u, k-1) + \phi(k-1, k-1, v) \leq \text{cost}(P_1) + \text{cost}(P_2) = L(k, u, v),$$

de donde $\phi(k, u, v) \leq L(k, u, v)$.

El solapamiento es masivo en esta formulación: el cálculo de la capa k consulta N^2 veces la fila y la columna $k-1$ de la capa anterior. Es, de hecho, el problema de este capítulo en el cual una implementación que se limite a transcribir la definición de ϕ resulta más catastrófica.

3.8.1. Tabulación. Como se explica a continuación, la tabulación es una opción conveniente para computar ϕ .

¿**Memorización o tabulación?** La decisión es más fácil aquí que en las secciones anteriores. La ventaja propia de la memorización es calcular solo los valores que hacen falta, pero en este problema hacen falta casi todos: la salida pide $\phi(N, u, v)$ para *cada* par de vértices y eso arrastra prácticamente los $N \cdot N \cdot (N+1)$ estados de la función. No hay nada que ahorrar, y a cambio habría que pagar la profundidad

de la recurrencia y el costo de un diccionario con esa cantidad de llaves. Se opta, entonces, por una tabulación.

De acuerdo con la definición recurrente de ϕ , se propone el diagrama de necesidades de la Figura 13. La tabulación es tridimensional: hay $N + 1$ capas, una por

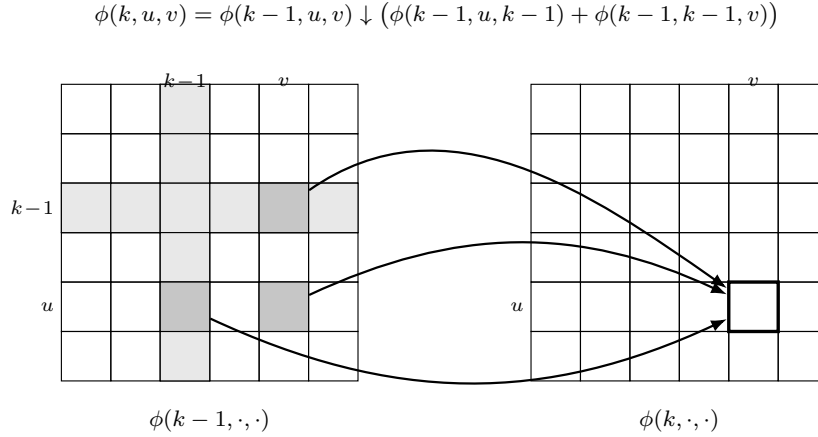


Figura 13. Diagrama de necesidades para la tabulación de ϕ en el problema de caminos mínimos entre todos los pares. La celda (u, v) de la capa k depende de tres celdas de la capa $k-1$: la misma celda (u, v) , la celda $(u, k-1)$ de la columna $k-1$ y la celda $(k-1, v)$ de la fila $k-1$.

cada valor de k , y cada capa es una matriz de $N \times N$ celdas indexada por parejas de vértices. La capa 0 es la matriz de adyacencia. Cada capa posterior se calcula por completo a partir de la anterior, lo cual sugiere llenar la tabulación capa por capa. Los invariantes del ciclo principal son:

$$P_0 : (\forall i, u, v \mid 0 \leq i < k \wedge u, v \in V : \text{tab}[i][u][v] = \phi(i, u, v)).$$

$$P_1 : 0 \leq k \leq N + 1.$$

El invariante P_0 indica que todas las capas anteriores a la capa k han sido llenadas con los valores correspondientes de ϕ . Los invariantes de los dos ciclos internos, que recorren los pares de vértices dentro de una capa, se proponen como ejercicio al lector. El diseño resulta en la función `apsp_tab` del Algoritmo 3.8.1.

Implementación. En la implementación se distinguen claramente la creación de la tabulación, sin valores, un primer paso para establecer los casos base y, finalmente, el proceso de calcular completamente la recurrencia en la tabla de la tabulación.

Algoritmo 3.8.1: Tabulación para ϕ

```

1 INF = float('inf')
2
3 def apsp_tab(N, w):
4     tab = [ [ [ None for _ in range(N) ] for _ in range(N) ]
5             for _ in range(N+1) ]
6     for u in range(N):
7         for v in range(N): tab[0][u][v] = w[u][v]
8     #  $P_0 \wedge P_1$ 
9     for k in range(1, N+1):
10        for u in range(N):
11            for v in range(N):
12                tab[k][u][v] = min(tab[k-1][u][v],
13                                   tab[k-1][u][k-1]+tab[k-1][k-1][v])
14    return tab[N]

```

La variable k recorre los valores de 1 a N y en la iteración k el vértice que se habilita como intermedio es $k-1$. Vale la pena insistir en esta correspondencia, pues es la fuente de errores más común al implementar el algoritmo. El valor de retorno es la última capa, que corresponde a $\phi(N, \cdot, \cdot)$.

Se ilustra el uso de la función con el grafo del Ejemplo 3.8.1:

```

1 >>> w = [ [ 0, 3, 8, INF ],
2 ...      [ INF, 0, 2, INF ],
3 ...      [ INF, INF, 0, -2 ],
4 ...      [ 6, 5, INF, 0 ] ]
5 >>> for row in apsp_tab(4, w): print(row)
6 [0, 3, 5, 3]
7 [6, 0, 2, 0]
8 [4, 3, 0, -2]
9 [6, 5, 7, 0]

```

Análisis. El análisis del algoritmo que tabula a ϕ se presenta en el Teorema 3.8.2.

Teorema 3.8.2

El llamado `apsp_tab(N, w)`:

1. Calcula el costo mínimo de un camino de u a v , para cada par de vértices $u, v \in V$.

2. Toma tiempo $O(N^3)$.
3. Usa espacio $O(N^3)$.

Demostración

Se propone como ejercicio al lector.

3.8.2. Reducción del espacio. El espacio $O(N^3)$ es escandaloso: guarda $N+1$ matrices cuando la respuesta es una sola. Como cada capa depende únicamente de la anterior, el argumento de la Sección 3.6 sugiere conservar dos capas y alternarlas, lo cual daría espacio $O(N^2)$. Resulta, sin embargo, que se puede hacer algo mejor y más simple: una sola matriz, sobrescrita en el lugar. Que esto sea correcto no es evidente, porque al sobrescribir se destruyen valores de la capa $k-1$ que todavía se van a necesitar. El siguiente resultado explica por qué no importa.

Teorema 3.8.3

Sea $1 \leq k \leq N$ y sean $u, v \in V$. Entonces

$$\phi(k, u, k-1) = \phi(k-1, u, k-1),$$

$$\phi(k, k-1, v) = \phi(k-1, k-1, v).$$

Es decir, la fila $k-1$ y la columna $k-1$ no cambian al pasar de la capa $k-1$ a la capa k .

Demostración

Observe primero que $\phi(k-1, k-1, k-1) = 0$. En efecto, el camino vacío de $k-1$ a $k-1$ tiene costo 0 y es admisible, y cualquier otro camino de $k-1$ a $k-1$ es un ciclo, cuyo costo es no negativo por hipótesis sobre G . Escriba $a = \phi(k-1, u, k-1)$ y $b = \phi(k-1, k-1, k-1)$ para abreviar; se acaba de observar que $b = 0$. Entonces se tiene:

$$\begin{aligned} \phi(k, u, k-1) &= a \downarrow (a + b) && \text{(definición de } \phi) \\ &= a \downarrow (a + 0) && (b = 0) \\ &= a \downarrow a && \text{(aritmética)} \\ &= a && \text{(idempotencia del mínimo).} \end{aligned}$$

La demostración de la segunda igualdad es análoga y se propone como ejercicio al lector.

La consecuencia práctica del Teorema 3.8.3 es inmediata. Los únicos valores de la capa $k-1$ que se consultan al calcular la capa k , además de la propia celda (u, v) , están en la fila $k-1$ y en la columna $k-1$. Si esas dos no cambian, entonces da lo mismo leerlas de la capa $k-1$ o de la capa k ya parcialmente escrita. Se puede, por lo tanto, trabajar sobre una única matriz. El diagrama de necesidades correspondiente se presenta en la Figura 14. Los invariantes de los dos ciclos internos, para un valor

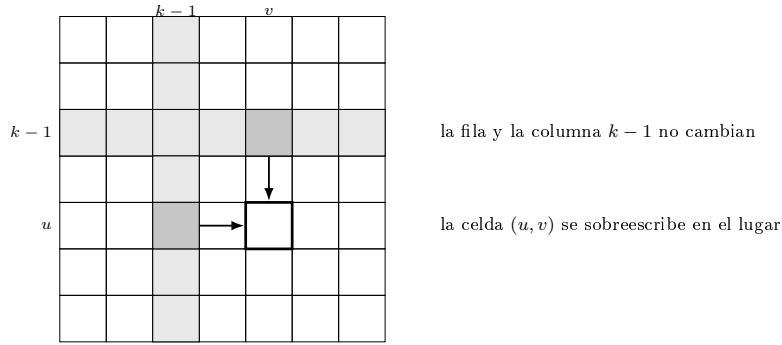


Figura 14. Diagrama de necesidades de la reducción del espacio en el problema de caminos mínimos entre todos los pares. Los valores que se consultan están en la cruz que forman la fila $k-1$ y la columna $k-1$, y por el Teorema 3.8.3 esa cruz no cambia al pasar de una capa a la siguiente.

fijo de k , describen una matriz que mezcla presente y pasado:

$$Q_0 : (\forall i, j \mid 0 \leq i < u \wedge j \in V : tab[i][j] = \phi(k, i, j)).$$

$$Q_1 : (\forall j \mid 0 \leq j < v : tab[u][j] = \phi(k, u, j)).$$

$$Q_2 : (\forall j \mid v \leq j < N : tab[u][j] = \phi(k-1, u, j)).$$

$$Q_3 : (\forall i, j \mid u < i < N \wedge j \in V : tab[i][j] = \phi(k-1, i, j)).$$

Las filas anteriores a la u ya tienen valores del presente (Q_0), las posteriores todavía tienen valores del pasado (Q_3) y la fila u está partida en dos por la columna v (Q_1 y Q_2). Sin el Teorema 3.8.3 esta mezcla sería fatal; con él, es inofensiva. El resultado es la función `apsp` del Algoritmo 3.8.2.

Algoritmo 3.8.2: Algoritmo de Roy–Floyd–Warshall

```

1 def apsp(N, w):
2     tab = [ [ w[u][v] for v in range(N) ] for u in range(N) ]
3     for k in range(1, N+1):

```

```

4  #  $Q_0 \wedge Q_1 \wedge Q_2 \wedge Q_3$ 
5  for u in range(N):
6      for v in range(N):
7          tab[u][v] = min(tab[u][v], tab[u][k-1]+tab[k-1][v])
8  return tab

```

Son seis líneas de código y esa brevedad explica buena parte de la popularidad del algoritmo. Note que la matriz de adyacencia se copia en lugar de modificarse: es una cortesía hacia quien invoca la función, que probablemente no espera que su grafo quede alterado.

Teorema 3.8.4

El llamado `apsp(N, w)`:

1. Calcula el costo mínimo de un camino de u a v , para cada par de vértices $u, v \in V$.
2. Toma tiempo $O(N^3)$.
3. Usa espacio $O(N^2)$.

Demostración

Se propone como ejercicio al lector.

Vale la pena registrar que esta es la tercera forma de reducción del espacio que aparece en el capítulo y que las tres son distintas. En la Sección 3.6 se conservó una sola columna recorriéndola en reversa; en la Sección 3.7, una sola fila más una variable auxiliar; aquí, la tabulación completa de una capa, sobreescrita en el lugar. Lo que las tres tienen en común es el método: mirar el diagrama de necesidades y preguntarse qué se puede descartar sin perder lo que hará falta.

3.8.3. Reconstrucción de una solución óptima. Como en la Sección 3.7, el algoritmo calcula el costo de los caminos óptimos pero no los exhibe. La reconstrucción es aquí particularmente elegante y no cuesta más que una matriz adicional.

La idea es mantener, junto con los costos, una matriz $nxt[0..N][0..N]$ tal que $nxt[u][v]$ es el vértice que sigue a u en un camino de costo mínimo de u a v . Inicialmente, cuando los únicos caminos disponibles son los arcos directos, $nxt[u][v]$ es v si el arco (u, v) existe y no está definido en caso contrario. Cada vez que la actualización de $tab[u][v]$ mejora el costo pasando por el vértice $k-1$, el camino de u a v empieza como el camino de u a $k-1$; por lo tanto, $nxt[u][v]$ debe tomar el

valor de $nxt[u][k-1]$. Con esa matriz, recuperar un camino consiste en seguir los apuntadores desde u hasta llegar a v .

Algoritmo 3.8.3: Reconstrucción de los caminos óptimos

```

1 def apsp_wit(N, w):
2     tab = [ [ w[u][v] for v in range(N) ] for u in range(N) ]
3     nxt = [ [ v if w[u][v]!=INF else None for v in range(N) ]
4             for u in range(N) ]
5     for k in range(1, N+1):
6         for u in range(N):
7             for v in range(N):
8                 if tab[u][k-1]+tab[k-1][v] < tab[u][v]:
9                     tab[u][v] = tab[u][k-1]+tab[k-1][v]
10                    nxt[u][v] = nxt[u][k-1]
11     return tab,nxt
12
13 def path(u, v, nxt):
14     ans = []
15     if nxt[u][v]!=None:
16         ans = [u]
17         while u!=v:
18             u = nxt[u][v]
19             ans.append(u)
20     return ans

```

El llamado `path(u, v, nxt)` retorna la lista vacía cuando no hay camino de u a v y de lo contrario la sucesión de vértices de un camino de costo mínimo, incluyendo los dos extremos. Con el grafo del Ejemplo 3.8.1, en el cual los vértices a, b, c, d se identifican con 0, 1, 2, 3:

```

1 >>> tab,nxt = apsp_wit(4, w)
2 >>> print(path(0, 2, nxt), tab[0][2])
3 [0, 1, 2] 5
4 >>> print(path(2, 0, nxt), tab[2][0])
5 [2, 3, 0] 4

```

Teorema 3.8.5

El llamado `apsp_wit(N, w)` toma tiempo $O(N^3)$ y espacio $O(N^2)$. Si `tab` y `nxt` son los valores que retorna, entonces el llamado `path(u, v, nxt)` retorna un camino de u a v de costo $tab[u][v]$ y toma tiempo $O(N)$.

Demostración

Se propone como ejercicio al lector.

A diferencia de lo que sucede en la Nota 3.7.3, aquí no hay tensión entre reducir el espacio y reconstruir una solución: la matriz `nxt` ocupa $O(N^2)$, que es exactamente lo que ya ocupa la tabulación reducida. La razón de fondo es que el rastro de las decisiones se puede resumir en un solo vértice por par, mientras que en el problema de la subsecuencia común más larga hacía falta la tabla entera.

Nota 3.8.5: Detección de ciclos de peso negativo

La especificación del Problema 3.8.1 excluye los grafos con ciclos de peso negativo, pero nada impide ejecutar el Algoritmo 3.8.2 sobre uno de ellos. Los valores que se obtienen ya no corresponden a ϕ , pues ϕ no está bien definida en ese caso. Sin embargo, la diagonal de la matriz resultante permite detectar la situación: hay un ciclo de peso negativo en G si y solo si $tab[u][u] < 0$ para algún vértice u .

Esto convierte al algoritmo en una herramienta de verificación, además de una de cálculo, y es la manera usual de comprobar que la hipótesis de la especificación se cumple antes de confiar en el resultado. La demostración de esta afirmación se propone como ejercicio al lector.

Ejercicios

1. Demuestre la afirmación de la Nota 3.8.2: si G no tiene ciclos de peso negativo y existe algún camino de u a v , entonces existe un camino simple de u a v de costo mínimo.
2. Formule los invariantes de los dos ciclos internos del Algoritmo 3.8.1 y demuestre el Teorema 3.8.2.
3. Complete la demostración del Teorema 3.8.3 con la igualdad $\phi(k, k-1, v) = \phi(k-1, k-1, v)$.
4. Demuestre el Teorema 3.8.4. En particular, demuestre que los ciclos internos satisfacen los invariantes Q_0 , Q_1 , Q_2 y Q_3 .
5. Demuestre el Teorema 3.8.5.
6. Demuestre la afirmación de la Nota 3.8.5.
7. En el Algoritmo 3.8.2 el ciclo sobre k es el más externo de los tres. Explique con un ejemplo concreto por qué el algoritmo deja de ser correcto si se coloca como el más interno, es decir, si se recorre **for** u ... **for** v ... **for** k . ¿Qué invariante deja de cumplirse?
8. La *clausura transitiva* de un grafo dirigido $G = (V, E)$ es la relación que indica, para cada par de vértices u, v , si existe algún camino de u a v . Este es el problema que resolvieron Roy y Warshall (Nota 3.8.4).
 - a) Especifique el problema.
 - b) Adapte la función objetivo ϕ de esta sección reemplazando los costos por valores Booleanos. ¿En qué se convierten el mínimo y la suma?
 - c) Escriba el algoritmo resultante. ¿Cuántas líneas de código lo separan del Algoritmo 3.8.2?
9. Suponga ahora que el peso de un arco representa la capacidad de un canal de comunicación y que la capacidad de un camino es la del arco de menor capacidad que lo compone. El problema de capacidad máxima pide, para cada par de vértices, la mayor capacidad posible de un camino que los une.
 - a) Especifique el problema y adapte la función objetivo.
 - b) Escriba el algoritmo resultante.
10. Compare las definiciones recurrentes de esta sección y de los dos ejercicios anteriores. En los tres casos la recurrencia tiene la misma forma, con dos operaciones que cambian de un problema a otro. Identifique esas dos operaciones en cada caso y determine cuál es el elemento neutro de cada una. Investigue qué es un *semianillo* y explique en qué sentido los tres problemas son instancias de un mismo problema algebraico.

11. Investigue sobre el algoritmo de Bellman–Ford, que resuelve el problema de caminos mínimos desde un único origen admitiendo pesos negativos.
 - a) Identifique en él la metodología de la Sección 3.3: ¿cuál es su función objetivo y cuál su definición recurrente?
 - b) ¿Cuál es su complejidad temporal? Compárela con la de ejecutarlo una vez por cada vértice del grafo frente a la del Algoritmo 3.8.2. ¿Para qué grafos conviene cada opción?
12. ¿Qué sucede si el grafo de entrada es no-dirigido? En particular, explique qué le ocurre a la hipótesis de la especificación cuando hay una arista de peso negativo, y concluya para qué clase de grafos no-dirigidos tiene sentido el problema.
13. 🌐 Investigue sobre el algoritmo de Johnson, mencionado en la Nota 3.8.3, que resuelve el problema de esta sección con pesos negativos ejecutando el algoritmo de Dijkstra una vez por cada vértice, después de reformular los pesos.
 - a) Explique en qué consiste la reformulación de los pesos y por qué preserva los caminos de costo mínimo.
 - b) Determine su complejidad temporal y compárela con la del Algoritmo 3.8.2. ¿Para qué grafos es preferible?

3.9. El problema del agente viajero

El problema del agente viajero es un problema de optimización combinatoria en grafos. A diferencia del problema de la Sección 3.8, aquí no se busca el costo de un camino entre dos vértices sino el de un recorrido que pase por todos ellos.

Nota 3.9.1: Grafos completos y circuitos

Las nociones de grafo, de arco y de camino son las de la Nota 3.8.1. A diferencia de lo que sucede en la Sección 3.8, en esta sección los grafos se suponen no-dirigidos.

Se dice que un grafo es *completo* cuando hay un arco entre cualquier par de vértices (distintos). Un *circuito* en un grafo es un camino que visita exactamente una vez cada uno de sus vértices y al finalizar regresa al vértice de origen.

Nota 3.9.2: El problema del agente viajero

El problema del agente viajero (en inglés, *travelling salesman problem* o *TSP*) se ocupa de responder la siguiente pregunta: dada una colección de ciudades

y las distancias entre cada par de ellas, ¿cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y al finalizar regresa a la ciudad origen?

K. Menger parece ser el primer matemático en ocuparse del problema, en Viena hacia 1930; la formulación moderna se le atribuye a H. Whitney, a partir de una charla que dio en Princeton en 1934. Desde entonces ha sido estudiado exhaustivamente en las áreas de optimización y teoría de la complejidad, hasta el punto de ser uno de los problemas más trabajados de la disciplina.

De forma general, el sistema de ciudades y rutas en el problema del agente viajero se puede abstraer en un grafo en el cual las ciudades se modelan como vértices y los caminos entre ciudades como arcos. En este problema se menciona una noción de medida para las rutas. Para incluir este elemento en el modelado del problema, se puede optar por tener una función de distancia sobre los arcos del grafo. Note que en el problema del agente viajero se desea encontrar un circuito de costo mínimo en un grafo completo con respecto a una función de distancia sobre sus arcos.

Problema 3.9.1: El problema del agente viajero

Entrada: Un grafo $G = (V, E)$ completo, con $|V| \geq 3$ vértices y función de peso sobre los arcos $w : E \rightarrow \mathbb{R}_{\geq 0}$.

Salida: El costo mínimo (con respecto a w) de un circuito en G .

La condición $|V| \geq 3$ no es un tecnicismo. Como se verá enseguida, un circuito está compuesto por exactamente $|V|$ arcos distintos y un grafo completo con dos vértices tiene un solo arco; con menos de tres vértices, entonces, no hay circuitos y el problema no tiene sentido.

Note además que la salida es el *costo* de un circuito óptimo y no el circuito mismo. Como en la Sección 3.7 y en la Sección 3.8, exhibir una solución óptima es un problema distinto, que se enuncia por separado.

Problema 3.9.2: Reconstrucción de un circuito de costo mínimo

Entrada: Un grafo $G = (V, E)$ completo, con $|V| \geq 3$ vértices y función de peso sobre los arcos $w : E \rightarrow \mathbb{R}_{\geq 0}$.

Salida: Un circuito de costo mínimo (con respecto a w) en G .

El Problema 3.9.2 se propone como ejercicio al final de la sección, una vez resuelto el Problema 3.9.1.

La salida del Problema 3.9.1 se puede precisar matemáticamente y exhibir así, entre otras cosas, su estructura combinatoria. La primera observación importante es que en un circuito *no* es importante identificar cuál es el vértice de “inicio”, pues cualquiera de los vértices en dicho circuito puede ser considerado de inicio (¿por qué?). Entonces, para el modelado del problema es natural volcar el interés en los arcos del grafo. Suponga que se cuenta con un predicado *circuit* que indica, para cualquier $C \subseteq E$, si C es un circuito en G . Es decir, $circuit(G, C)$ (o, simplemente, $circuit(C)$ cuando G se entiende del contexto) es cierto únicamente cuando C es un circuito en G . Además, se define la función *cost* para cualquier subconjunto $C \subseteq E$ y función de peso $w : E \rightarrow \mathbb{R}_{\geq 0}$ de la siguiente manera:

$$cost(C) = (\sum e \mid e \in C : w(e)).$$

Es decir, $cost(C)$ es la suma de los pesos de los arcos en C . Con base en estas definiciones, se formula la salida del problema del agente viajero de la siguiente manera:

$$(\downarrow C \mid C \subseteq E \wedge circuit(C) : cost(C)).$$

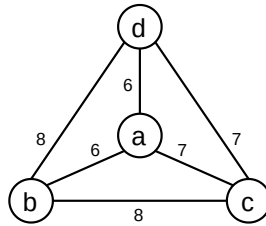
Esta fórmula expresa que se desea calcular el mínimo costo entre todos los circuitos en G . Note que la cantidad de posibles circuitos está acotada por $2^{|E|}$, la cantidad de subconjuntos de E . Más precisamente, está acotada por la cantidad de subconjuntos de E de tamaño $|V|$, dado que un circuito tiene exactamente $|V|$ arcos; es decir, por

$$\binom{|E|}{|V|},$$

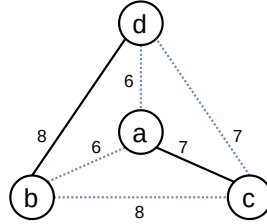
que denota la cantidad de subconjuntos de tamaño $|V|$ en un conjunto de tamaño $|E|$. En esta observación yace la naturaleza combinatoria del problema del agente viajero.

Ejemplo 3.9.1

A continuación se presenta un grafo completo con 4 vértices a, b, c, d .



El circuito $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$ es óptimo y tiene costo 27:



3.9.1. Diseño de una función objetivo. Necesariamente hay un circuito de costo mínimo cuando G tiene al menos tres vértices; suponga que $C \subseteq E$ es un circuito óptimo de G en este sentido. Sin pérdida de generalidad, también suponga que

$$C = \{(u_0, u_1), (u_1, u_2), \dots, (u_{N-1}, u_0)\},$$

en donde $N = |V|$ y $V = \{u_0, u_1, u_2, \dots, u_{N-1}\}$. Observe que el circuito

$$u_0, u_1, u_2, \dots, u_{N-1}, u_0$$

es óptimo si y solo si el camino de u_0 a u_{N-1} en este circuito es óptimo, entre todos los caminos posibles de u_0 a u_{N-1} que incluyen los demás vértices de G (de lo contrario, no sería de costo mínimo —¿por qué?). Esta es la propiedad de la subestructura óptima que se “explotará” para formular una función objetivo que ayude a resolver el problema del agente viajero usando programación dinámica.

Función objetivo. Dado que en un circuito no importa desde dónde se inicie el recorrido, se fija arbitrariamente un vértice $s \in V$ como punto de partida. Se usa el hecho de que G tiene al menos tres vértices; de lo contrario el problema es trivial, ya que dicho circuito no existe.

Para $X \subseteq V$ y $u \in X$:

$\phi(u, X)$: “mínimo costo de un camino (simple) que inicia en s , visita cada uno de los vértices en X y termina en u .”

Basada en la propiedad de la subestructura óptima, la observación importante para aceptar la formulación de ϕ como (potencialmente) útil es que todo circuito óptimo se puede construir con base en rutas óptimas que visiten todos los vértices del grafo dado.

Reformulación de la especificación. El objetivo es entonces calcular el valor de la siguiente expresión:

$$(\downarrow u \mid u \in V \setminus \{s\} : \phi(u, V \setminus \{s\}) + w(u, s)).$$

Entre todos los posibles caminos que inician en s , y que incluyen todos los demás vértices del grafo y terminan en u , se desea calcular aquel camino que sumando la

ruta de regreso directa de u a s sea de costo mínimo. Note que cuando $V = \{s\}$ la minimatoria se toma sobre un rango vacío y, en consecuencia, su valor es $+\infty$: la identidad del mínimo. Esto refleja correctamente que un grafo con un único vértice no tiene circuitos.

El caso $|V| = 2$ merece una advertencia y es la razón por la cual la especificación exige $|V| \geq 3$. Si $V = \{s, u\}$, la expresión anterior se reduce a $\phi(u, \{u\}) + w(u, s) = 2w(s, u)$, que es un valor finito. Pero ese valor no corresponde a ningún circuito: el recorrido $s \rightarrow u \rightarrow s$ usa dos veces el único arco disponible, mientras que un circuito consta de $|V|$ arcos *distintos*. La reformulación, entonces, solo es fiel a la especificación cuando el grafo tiene al menos tres vértices.

Planteamiento recurrente. Una ruta de costo mínimo de s a cualquier otro vértice debe estar conformada por subrutas óptimas. Es decir, cada ruta que inicie en s en dicho camino, también debe ser de costo mínimo.

La definición formal de ϕ se presenta para $X \subseteq V \setminus \{s\}$ y $u \in V \setminus \{s\}$:

$$\phi(u, X) = \begin{cases} +\infty & , \text{ si } u \notin X, \\ w(s, u) & , \text{ si } X = \{u\}, \\ (\downarrow v \mid v \in X \setminus \{u\} : \phi(v, X \setminus \{u\}) + w(v, u)) & , \text{ si } u \in X \wedge |X| \geq 2. \end{cases}$$

Se consideran tres casos para definir ϕ . Hay dos opciones entre u y X : que u sea elemento de X o no y, cuando lo es, que sea el único o no. Es imposible que haya un camino que visite todos los elementos de X y que termine en u cuando $u \notin X$; la forma de expresar que esto es imposible es con el valor $+\infty$ dado que es la identidad del mínimo en los reales. Si u es el único elemento de X , solo hay un camino de s a u : el camino directo entre s y u , el cual tiene costo $w(s, u)$. Cuando en X hay otros elementos en adición a u , se apuesta por todos los caballos: se prefiere aquél vértice $v \in X \setminus \{u\}$ que en dicho camino conecte directamente con u y para el cual el camino desde s permita construir un camino hasta u de costo mínimo.

Correctitud de la función objetivo. El Teorema 3.9.1 presenta la propiedad de correctitud del diseño propuesto para ϕ .

Teorema 3.9.1

Sean $s \in V$ y $X \subseteq V \setminus \{s\}$. Si $u \in X$, entonces $\phi(u, X)$ es el costo mínimo de un camino (simple) que inicia en s , visita cada uno de los vértices en X y termina en u .

Demostración

Sea $L(u, X)$ el costo mínimo de un camino simple que inicia en s , visita cada uno de los vértices de X y termina en u , con el convenio de que $L(u, X) = +\infty$ cuando no existe tal camino. El objetivo es demostrar que $\phi(u, X) = L(u, X)$, y se procede por inducción sobre $|X|$.

Si $u \notin X$ no hay ningún camino que termine en u y visite exactamente los vértices de X , luego $L(u, X) = +\infty = \phi(u, X)$. Si $X = \{u\}$, el único camino que inicia en s , visita únicamente a u y termina en u es el arco (s, u) , de costo $w(s, u)$; luego $L(u, \{u\}) = w(s, u) = \phi(u, \{u\})$.

Suponga entonces que $u \in X$ y $|X| \geq 2$, y que $\phi(v, Y) = L(v, Y)$ para todo Y con $|Y| < |X|$ y todo v . Escriba $Y = X \setminus \{u\}$ y note que $|Y| = |X| - 1$. Se demuestran las dos desigualdades.

- *Composición:* $\phi(u, X) \geq L(u, X)$. Basta ver que $L(u, X)$ no excede a ninguno de los valores sobre los cuales se toma la minimatoria. Sea $v \in Y$ y suponga que $\phi(v, Y)$ es finito, pues de lo contrario la desigualdad correspondiente es inmediata. Por hipótesis inductiva hay un camino simple P que inicia en s , visita exactamente los vértices de Y , termina en v y tiene costo $\phi(v, Y)$. Como $u \notin Y$, el camino que resulta de extender P con el arco (v, u) sigue siendo simple, inicia en s , visita exactamente los vértices de X y termina en u ; su costo es $\phi(v, Y) + w(v, u)$. Luego $L(u, X) \leq \phi(v, Y) + w(v, u)$ para cada $v \in Y$.
- *Descomposición:* $\phi(u, X) \leq L(u, X)$. Si no hay camino admisible la desigualdad es inmediata. De lo contrario, sea P un camino simple de costo $L(u, X)$ que inicia en s , visita exactamente los vértices de X y termina en u . Como $|X| \geq 2$, el vértice que precede a u en P es algún $v \in Y$. Al quitarle a P su último arco queda un camino simple que inicia en s , visita exactamente los vértices de Y y termina en v , de modo que su costo no es menor que $L(v, Y)$. Entonces se tiene:

$$\begin{aligned}
 L(u, X) &= L(u, X) - w(v, u) + w(v, u) && \text{(aritmética)} \\
 &\geq L(v, Y) + w(v, u) && \text{(observación anterior)} \\
 &= \phi(v, Y) + w(v, u) && \text{(hipótesis inductiva)} \\
 &\geq \phi(u, X) && (v \in Y).
 \end{aligned}$$

En cualquier caso, $\phi(u, X) = L(u, X)$.

¿Memorización o tabulación? La función ϕ es distinta a las que se han usado para resolver los problemas anteriores. En particular, tiene como parámetro un conjunto. Como tal, esta definición se puede implementar directamente en un lenguaje de programación (usando conjuntos disponibles en sus librerías o implementando la estructura de datos correspondiente). No resulta fácil pensar cómo tabular cuando en una de las dos dimensiones hay un conjunto. Tampoco resulta

fácil pensar cómo implementar una memorización eficiente cuando el acceso a los datos depende de una colección y que esta colección puede “mutar” entre los distintos llamados recurrentes. En realidad, estas son noticias *parcialmente* malas. La noticia parcialmente buena es que bajo ciertas suposiciones acerca de la cantidad de vértices en el grafo, los conjuntos se pueden representar con números naturales y las operaciones básicas sobre ellos (al menos las que se requieren para calcular la función objetivo) se pueden realizar en tiempo constante. Entonces, antes de proponer una solución con programación dinámica, se estudia cómo representar conjuntos y algunas de sus operaciones con números naturales.

3.9.2. Especificación de conjuntos con máscaras de bits. El propósito de esta sección es mostrar cómo, bajo ciertas suposiciones, se pueden usar máscaras y operaciones de bits para representar conjuntos y operaciones sobre ellos. En particular, para un conjunto de N elementos, es deseable contar con las siguientes operaciones:

- Representación del conjunto completo o *universal*, junto con todos sus subconjuntos.
- Sustraer un elemento de un conjunto.
- Consultar si un elemento hace parte de un conjunto.
- Determinar si un conjunto es el conjunto unitario determinado por un elemento dado.

Estando disponible y de manera eficiente, la funcionalidad enumerada sería suficiente para implementar un algoritmo con programación dinámica que permita calcular la función ϕ eficientemente. Al menos, esta es la intención.

Las máscaras de bits (i.e., secuencias de bits) ofrecen una posibilidad si la cantidad de elementos N está acotada: un conjunto universal con N elementos se puede representar con N bits. Es más, cualquier conjunto con a lo sumo N elementos se puede también representar con una secuencia de bits de tamaño N . Sin pérdida de generalidad, se puede suponer que hay un orden entre los elementos del conjunto universal que se quiere representar. Bajo esta suposición, se puede asumir que el conjunto a codificar es $U = \{0, 1, \dots, N - 1\}$ de tal forma que 0 identifica el primer elemento de dicho conjunto, 1 el segundo y así, con $N - 1$ representando el último elemento en este orden arbitrario. Una secuencia de bits de tamaño N se puede, entonces, usar como función característica para representar exactamente un subconjunto de U : el n -ésimo bit, yendo del menos significativo al más significativo, indica si el n -ésimo elemento de U hace parte del subconjunto, para $0 \leq n < N$.

Ejemplo 3.9.2

Considere el conjunto $\{a, b, c, d\}$ de etiquetas. Suponga que las etiquetas tienen el siguiente orden:

$$a < b < c < d.$$

Entonces:

- La secuencia 1111 representa el conjunto $\{a, b, c, d\}$.
- La secuencia 1010 representa el conjunto $\{b, d\}$.
- La secuencia 0100 representa el conjunto unitario $\{c\}$.
- La secuencia 0000 representa el conjunto vacío $\emptyset$.

Entonces, en el problema del agente viajero se puede suponer que el conjunto de vértices $V = \{v_0, v_1, \dots, v_{N-1}\}$ y sus subconjuntos, se pueden representar con máscaras de bits de tamaño N (fijando algún orden sobre V).

En un lenguaje de programación, como Python, se puede construir rápidamente un conjunto universo para cualquier $N \in \mathbb{N}$ haciendo corrimiento de bits y una resta (ojo, no es la única manera de hacerlo). A continuación se presenta la función `universe` que dado dicho N , retorna el número entero cuya representación en base 2 es una secuencia de exactamente N unos:

```
1 def universe(N): return (1<<N)-1
```

Por ejemplo, `universe(3)` es 7 dado que su representación en base 2 es 111. La expresión `1<<N` calcula el número natural 2^N . Al restarle 1, se “apaga” el único bit existente y todos los 0 a su derecha de “encienden”, resultando en una secuencia de N unos.

El siguiente reto es determinar si un bit específico está encendido o apagado en una máscara de bits. Suponiendo que se cuenta con una máscara de N bits, para cualquier $0 \leq n < N$, se puede identificar si el n -ésimo bit está encendido con una disyunción bit a bit.

```
1 def is_elt(n, X): return (X|(1<<n))==X
```

El llamado `is_elt(n, X)` es cierto cuando el n -ésimo bit del código binario del número natural X es 1, suponiendo que n es una posición en la máscara de bits correspondiente a X .

De una manera muy similar, se puede apagar el n -ésimo bit en una máscara de bits.

```
1 def remove_elt(n, X): return X-(1<<n) if is_elt(n, X) else X
```


Si ese bit está encendido, entonces se apaga; de lo contrario, la máscara no se modifica.

Finalmente, determinar si una máscara representa un conjunto unitario dado es fácil de calcular: en la máscara correspondiente al conjunto unitario $\{n\}$ únicamente hay un bit encendido, exactamente el n -ésimo.

```
1 def singleton(n, X): return X==(1<<n)
```

Estas cuatro operaciones, basadas en comparación y aritmética de bits, son eficientes en la práctica cuando N no supera el tamaño de la palabra en un procesador. Por ejemplo, en procesadores de 32 bits estas operaciones se pueden implementar directa y eficientemente siempre y cuando $N \leq 32$. Lo mismo sucede con procesadores de 64 bits, etc. De lo contrario, se deberá incurrir en procedimientos adicionales que difícilmente serán de orden constante.

3.9.3. Diseño de un algoritmo con memorización. Bajo la suposición de que se pueden representar los subconjuntos de V con máscaras de bits, se obtiene casi directamente un algoritmo para implementar ϕ con memorización. Concretamente, se supondrá que G tiene N vértices $V = \{0, 1, \dots, N-1\}$ y que su representación es una matriz de adyacencia $w[0..N][0..N]$ tal que para cualquier $u, v \in V$, se tiene $w[u][v] = w[v][u] = w(u, v)$ y $w[u][u] = 0$. Es decir, los vértices se identifican con los primeros N números naturales y la función de peso entre vértices se especifica con una matriz cuadrada y simétrica en donde cada entrada indica el peso del arco entre los vértices correspondientes; esta matriz es 0 en su diagonal.

El Algoritmo 3.9.1 presenta las funciones `phi_memo` y `tsp` que resuelven el problema del agente viajero para un grafo de N vértices y función w de peso en los arcos. La función principal es `tsp`, la cual recibe estos dos parámetros (se pudo también optar por tener estos dos parámetros como variables globales, simplificando –pero oscureciendo en este caso en particular– el código). La función `phi_memo` es la solución por memorización que implementa la función ϕ . Note que el vértice 0 juega el papel del vértice s en la especificación de la función ϕ .

Implementación.

Algoritmo 3.9.1: Memorización para el agente viajero

```
1 INF = float('inf')
2
3 def phi_memo(N, w, u, X, mem):
4     ans, key = None, (u, X)
5     if key in mem: ans = mem[key]
6     else:
7         if not(is_elt(u, X)): ans = INF
```

```

8     elif singleton(u, X): ans = w[0][u]
9     else:
10         ans, Y = INF, remove_elt(u, X)
11         for v in range(1, N):
12             if is_elt(v, Y):
13                 ans = min(ans, phi_memo(N, w, v, Y, mem)+w[v][u])
14         mem[key] = ans
15     return ans
16
17 def tsp(N, w):
18     ans = INF
19     X = remove_elt(0, universe(N))
20     mem = dict()
21     for u in range(1, N):
22         ans = min(ans, phi_memo(N, w, u, X, mem)+w[u][0])
23     return ans

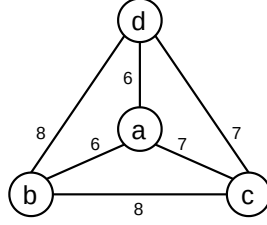
```

Un llamado `tsp(N, w)` inicialmente construye en X el conjunto de todos los vértices excepto el vértice de partida, es decir, el conjunto $\{1, \dots, N-1\}$. Iniciando con la memoria vacía, se itera sobre cada uno de los vértices en este conjunto tratando de encontrar aquella combinación que permita construir un circuito de costo mínimo que inicie en 0 y termine en 0, agotando todos los demás vértices en X . Esto corresponde, tal cual, con la reformulación de la especificación en la Sección 3.9.1.

El llamado `phi_memo(N, w, u, X, mem)` calcula la función $\phi(u, X)$ para el grafo de N vértices con función de peso $w[0..N][0..N]$, usando la memorización en el diccionario `mem`. Las llaves del diccionario son parejas en donde el primer argumento corresponde a un identificador de vértice y el segundo a una máscara de bits (representando un conjunto de vértices). Si el valor asociado a la llave (u, X) está presente en el diccionario, se usa directamente como valor de retorno. De lo contrario, se calcula el valor correspondiente a esta llave y se almacena en el diccionario antes de retornarlo. Note que se calcula este valor con base en cada uno de los tres casos que hacen parte de la definición recurrente de ϕ .

Ejemplo 3.9.3

A continuación se presenta el grafo completo con 4 vértices a, b, c, d del Ejemplo 3.9.1.



Este es un grafo con $N = 4$ vértices y función de peso w dada por la siguiente matriz de adyacencia:

```

1 w = [ [ 0, 6, 7, 6 ],
2       [ 6, 0, 8, 8 ],
3       [ 7, 8, 0, 7 ],
4       [ 6, 8, 7, 0 ] ]

```

El valor calculado por la función `tsp` en este caso es 27:

```

1 >>> tsp(4, w)
2 27

```

Análisis. La correctitud de `tsp` depende directamente de la correctitud de `phi_memo`, bajo las siguientes condiciones:

$$C_0 : (\forall u, X \mid (u, X) \in \text{mem} : \text{mem}[(u, X)] = \phi(u, X)).$$

$$C_1 : (\forall u, X \mid (u, X) \in \text{mem} : 1 \leq u < N \wedge 0 \leq X < 2^N).$$

Las condiciones C_0 y C_1 son condiciones de representación, respectivamente, de la memorización y de conjuntos con máscaras de bits. La condición C_0 indica que los valores almacenados en la memoria compartida son correctos en relación con ϕ . La condición C_1 indica que los valores utilizados como índice de la memoria son los esperados: un vértice distinto al de inicio y una máscara de bits para el conjunto de N vértices. La correctitud de `phi_memo` consiste, básicamente, en garantizar que estas dos condiciones se mantienen a lo largo de todos los llamados recurrentes.

Teorema 3.9.2

Para $V = \{0, 1, \dots, N - 1\}$ y función de peso $w[0..N][0..N]$, el llamado `phi_memo(N, w, u, X, mem)` con $u \in V \setminus \{0\}$ y $0 \leq X < 2^N$:

1. Preserva las condiciones C_0 y C_1 , y
2. Calcula $\phi(u, X)$.

Demostración

Se propone como ejercicio para el lector.

El diccionario `mem` tiene a lo sumo $N \cdot 2^N$ llaves y el valor asociado a cada una de ellas se calcula exactamente una vez. Suponiendo que el acceso al diccionario y las operaciones sobre la máscara de bits son de orden constante, ese cálculo toma tiempo $O(N)$ por las líneas de código 11-13. Luego, el trabajo total realizado por todos los llamados a `phi_memo` es $O(N^2 \cdot 2^N)$. Como el ciclo en `tsp` comparte una única memoria `mem`, este trabajo total no se multiplica por la cantidad de iteraciones y la complejidad temporal del llamado `tsp(N, w)` es también $O(N^2 \cdot 2^N)$. La complejidad espacial es $O(N \cdot 2^N)$, dominada por la cantidad de llaves del diccionario. Los detalles de estas afirmaciones se proponen como ejercicio para el lector.

Teorema 3.9.3

El llamado `tsp(N, w)` toma tiempo $O(N^2 \cdot 2^N)$ y espacio $O(N \cdot 2^N)$.

Demostración

Se propone como ejercicio para el lector.

Nota 3.9.3: El algoritmo de Bellman–Held–Karp

El algoritmo desarrollado en esta sección se conoce como *algoritmo de Held–Karp* o, con más justicia, como *algoritmo de Bellman–Held–Karp*. Fue publicado en 1962 de manera independiente por R. Bellman, por un lado, y por M. Held y R. Karp, por el otro.

Que sea de Bellman cierra un arco que el capítulo abrió en la Nota 3.0.1. La Nota 3.0.2 señala que en programación dinámica el nombre llegó antes que casi todos los algoritmos; este es la excepción, pues lo diseñó quien puso el nombre, doce años después de haberlo escogido.

Vale la pena anotar, por último, que la cota $O(N^2 \cdot 2^N)$ sigue siendo, más de sesenta años después, la mejor conocida para el caso general del problema. Bajar la base de la exponencial por debajo de 2 es un problema abierto célebre, que ha resistido todos los intentos.

Ejercicios

1. Encuentre 5 ejemplos cotidianos de grafos, y explique cuáles son los vértices y arcos en cada uno de ellos. En cada caso, explique si el grafo es dirigido o no-dirigido, y si es completo. Si hay una noción de peso, distancia o importancia, ¿cuál es?
2. Suponga que se cuenta con un circuito en un grafo $G = (V, E)$. Explique por qué cada uno de los vértices en dicho circuito puede ser considerado como vértice inicial del circuito.
3. Proponga una definición formal para el predicado $circuit(C)$ relativa a un grafo $G = (V, E)$ tal que $C \subseteq E$. Si prefiere, puede tratar $circuit$ como una función Booleana $circuit : 2^E \rightarrow \mathbb{B}$, en donde 2^E denota la colección de subconjuntos de E .
4. Explique por qué la cantidad máxima de circuitos en un grafo completo $G = (V, E)$ está acotada por $\binom{|E|}{|V|}$. ¿Son circuitos todos los subconjuntos de E de tamaño $|V|$? ¿Por qué?
5. Enumere todos los circuitos del grafo en el Ejemplo 3.9.1 sin distinguir entre rotaciones (¿cuántos hay?). Justifique que el circuito de costo mínimo allí es 27.
6. En la definición recurrente de ϕ se incluye como caso base $\phi(u, X) = +\infty$ cuando $u \notin X$. En realidad, este caso se incluye por completitud de ϕ dado que $X \subseteq V \setminus \{s\}$ y $u \in V \setminus \{s\}$. Explique por qué este caso no es necesario para calcular la salida del problema. En particular, justifique cómo ϕ se puede definir, sin introducir inconsistencias matemáticas, únicamente para aquellos casos en los cuales $u \in X$.
7. Es posible obtener una versión distinta de la función `remove_elt` usando exclusivamente operaciones sobre bits (sin operaciones aritméticas). Investigue acerca de esta operación y proponga una versión alternativa de `remove_elt`.
8. ¿Cómo se puede iterar sobre todos los subconjuntos de un conjunto de tamaño N con máscaras de bits? Diseñe un algoritmo que itere sobre todas las posibles máscaras de 2 bits y calcule la tabla de verdad para la disyunción y conjunción.
9. Demuestre el Teorema 3.9.2.
10. Demuestre el Teorema 3.9.3.
11. Resuelva el Problema 3.9.2 extendiendo el Algoritmo 3.9.1 para que, además del costo de un circuito óptimo, retorne un circuito que lo alcance. ¿Cuánto espacio adicional requiere? Compare su respuesta con lo que sucede en la Sección 3.7 y en la Sección 3.8 (Nota 3.7.3).

12. Diseñe un algoritmo de programación dinámica con tabulación para ϕ y un algoritmo iterativo para resolver el problema del agente viajero. Estime su complejidad temporal y espacial.

Notas del capítulo y referencias

La noción de programación dinámica cobra un aire más general cuando se considera más allá de la computación y del estudio de algoritmos eficientes para implementar cierto tipo de funciones recurrentes. Para el estudio de fenómenos y problemas económicos, como originalmente nace, es un método recursivo para resolver problemas de decisión secuenciales. Se le conoce también como *inducción reversa* (en inglés, *backward induction*), y es empleada para encontrar reglas de decisión óptimas en juegos, equilibrios perfectos de juegos multi-agente y equilibrios competitivos en modelos económicos dinámicos. El texto autobiográfico de R. Bellman [Bel84] incluye apartes de los orígenes de la programación dinámica en este contexto más general, al igual que algunos trabajos más recientes en Economía como el de J. Rust [Rus06]. El aparte textual que se presenta en la Nota 3.0.1 es tomado de [Bel84, p. 159]; la traducción al castellano es responsabilidad del autor. Las dos obras que menciona la Nota 3.0.2 son el libro de 1957 en el cual Bellman fija la técnica y enuncia el principio de optimalidad [Bel57], y el artículo del año siguiente en el cual aparece el algoritmo de caminos mínimos que hoy lleva su nombre junto con el de L. Ford [Bel58].

Las propiedades de subestructura y de solapamiento se plantean casi siempre para estudiar qué problemas son candidatos a resolverse con programación dinámica, y lo mismo sucede con las dos técnicas de implementación, la memorización (en inglés, *top-down*) y la tabulación (en inglés, *bottom-up*), que se comparan en la Sección 3.2. Estos pares de temas se encuentran explicados, de manera similar a la seguida en este capítulo, en [CLRS22] y [KET06]. La metodología propuesta en la Sección 3.3 es original, pero de ninguna forma alejada a otras propuestas como la encontrada en [Eri19]. En el caso de este texto, el enfoque se basa en la definición de una función objetivo que posteriormente se define formalmente de manera recurrente y en separar esas dos cosas para que la segunda se pueda demostrar correcta respecto de la primera.

Los problemas escogidos para ilustrar el uso de la programación dinámica también son extensamente conocidos en la comunidad de algoritmos. Posiblemente, el enfoque de anotar los diseños de memorización con condiciones y de tabulación con invariantes explícitos no sea tan popular. En este capítulo se ha hecho el esfuerzo de exponer las principales propiedades de los problemas y los algoritmos usando una notación matemática sencilla. El uso de invariantes à la Dijkstra & Scholten también es planteado por J. Bohórquez [Boh06]. El algoritmo que implementa la

reducción a una sola columna en la Sección 3.6 está inspirado en observaciones hechas en [Eri19].

La mayoría de los algoritmos presentados calcula el valor de una solución óptima y no una solución que lo alcance. La reconstrucción de una solución óptima se desarrolla en la Sección 3.7 y en la Sección 3.8, y se propone como ejercicio en las demás secciones. Se trata de una técnica general y no de un truco propio de un problema en particular: la tabulación guarda, además del valor óptimo, el rastro de las decisiones que lo produjeron y recorrerla en sentido contrario permite recuperarla. Los textos citados en estas notas la presentan bajo nombres como *traceback* o *backtracking of the table*, que no deben confundirse con la técnica de reintento, tema de otro capítulo de este texto.

La historia del problema de la Sección 3.4, resumida en la Nota 3.4.1, se conoce sobre todo por una columna de J. Bentley [Ben84], en la cual aparecen tanto la anécdota como los tres algoritmos. J. Kadane publicó su propia versión de los hechos más de cuarenta años después [Kad23], y allí señala, además, que el algoritmo que Bentley reconstruyó en su columna no coincide exactamente con el que él tenía en mente. El mismo algoritmo lineal se ha obtenido por caminos independientes del seguido en este capítulo: D. Gries [Gri82] lo derivó aplicando la estrategia estándar de desarrollo de programas a partir de invariantes –el mismo estilo que se sigue aquí– y R. Bird [Bir89] lo obtuvo por manipulación algebraica del algoritmo de fuerza bruta.

El problema de la subsecuencia creciente más larga de la Sección 3.5 es, entre los que se estudian en este capítulo, el que más atención ha recibido por fuera de la algorítmica. La pregunta por la longitud típica de la subsecuencia creciente más larga de una permutación aleatoria se conoce como el problema de Ulam y ha resultado tener conexiones profundas con áreas en apariencia distantes: permutaciones aleatorias, matrices aleatorias, tableaux de Young y modelos de crecimiento. Tanto así que el problema cuenta con un libro dedicado exclusivamente a él, el de D. Romik [Rom15], en donde se desarrollan el teorema de Vershik–Kerov–Logan–Shepp sobre la forma límite, el teorema de Baik–Deift–Johansson y la distribución de Tracy–Widom. Nada de eso hace falta para leer la Sección 3.5, pero sí ilustra bien que un problema algorítmico sencillo de enunciar puede esconder matemáticas de gran riqueza. El algoritmo de tiempo $O(N \log N)$ mencionado en la Nota 3.5.4 se remonta al trabajo de C. Schensted [Sch61], en donde aparece como subproducto de una correspondencia entre permutaciones y tableaux; su presentación moderna en términos del solitario ‘paciencia’ (en inglés, *patience sorting*) se encuentra en el primer capítulo de [Rom15].

El problema de la subsecuencia común más larga de la Sección 3.7, en cambio, es conocido sobre todo por sus aplicaciones. El algoritmo de espacio lineal que

reconstruye una solución óptima, mencionado en la Nota 3.7.3, se debe a D. Hirschberg [Hir75] y combina la programación dinámica de esa sección con dividir, conquistar y combinar. La variante conocida como distancia de edición, propuesta como ejercicio en la misma sección, fue estudiada por R. Wagner y M. Fischer [WF74]; la solución que allí se presenta es esencialmente la tabulación de la Sección 3.7 con un caso adicional. Vale la pena anotar que, para ninguno de los dos problemas, se conoce un algoritmo cuyo tiempo sea sustancialmente mejor que el cuadrático y que hay evidencia de que tal algoritmo no existe: bajo hipótesis estándar de la teoría de la complejidad, no hay soluciones en tiempo $O((MN)^{1-\epsilon})$ para ningún $\epsilon > 0$.

El algoritmo de la Sección 3.8 tiene una historia enredada que vale la pena contar completa. B. Roy lo describió en 1959 [Roy59] y S. Warshall en 1962 [War62], ambos para el problema de la clausura transitiva; ese mismo año, R. Floyd [Flo62] publicó la versión para caminos mínimos que es la que se estudia aquí. Ni siquiera la formulación con tres ciclos anidados, que es la que todo el mundo memoriza, es de alguno de los tres: se debe a P. Ingerman [Ing62], también de 1962. El nombre usual, algoritmo de Floyd–Warshall, omite a Roy a pesar de haber sido el primero, y por eso en este texto se prefiere Roy–Floyd–Warshall, como se explica en la Nota 3.8.4. A. Schrijver documenta en detalle las diversas atribuciones [Sch04, Sec. 8.6g].

Los últimos ejercicios de esa sección apuntan a algo más profundo que una curiosidad histórica. La misma recurrencia resuelve tres problemas distintos, a saber, alcanzabilidad, distancias mínimas y capacidad máxima, cambiando únicamente las dos operaciones que combinan los valores. Y va más lejos: S. Kleene había usado ese mismo esquema en 1956 [Kle56] para convertir un autómata finito en una expresión regular, con la unión y la concatenación en el papel de esas dos operaciones. Lo que las cuatro instancias comparten es una estructura algebraica, la de *semianillo*, y el problema general de calcular esta clase de recurrencias sobre un semianillo arbitrario se conoce como el *problema algebraico de caminos*. Es un buen ejemplo de que la programación dinámica no es un catálogo de trucos sino una manera de reconocer estructura: cuatro problemas que nadie agruparía a primera vista resultan ser el mismo cálculo.

El problema de la Sección 3.9 es, probablemente, el problema de optimización combinatoria más estudiado que existe. K. Menger fue el primer matemático en ocuparse de él, hacia 1930, y la formulación moderna se le atribuye a H. Whitney a partir de una charla que dio en Princeton en 1934. El algoritmo que la sección desarrolla fue publicado en 1962 de manera independiente por R. Bellman [Bel62], por un lado, y por M. Held y R. Karp [HK62], por el otro; de ahí el nombre de algoritmo de Bellman–Held–Karp que se usa en la Nota 3.9.3. Su cota de $O(N^2 \cdot 2^N)$ sigue siendo, más de sesenta años después, la mejor conocida para el caso general del

problema. Obtener una solución exacta en tiempo $O(c^N)$ con $c < 2$ es un problema abierto célebre.

Los textos de T. Cormen et al. [CLRS22], J. Kleinberg y É. Tardos [KET06], J. Erickson [Eri19], H. Bhasin [Bha15] y A. Levitin [Lev12] son fuentes extensas de ejemplos de soluciones con programación dinámica y ejercicios para ser resueltos con esta técnica. Textos de entrenamiento diseñados para programación competitiva como los de S. Skiena [Ski08] y S. Halim et al. [HHE20] contienen ejemplos complementarios, además de ‘trucos’ de implementación, al igual que páginas de internet asociadas a programación competitiva y algoritmos.

Además de las técnicas de reducción de espacio vistas para tabulaciones, existen técnicas para la reducción del tiempo de cómputo para algunos problemas que admiten programación dinámica como solución. Entre ellas se encuentran las basadas en monotonías y desigualdades propuestas por D. Knuth en [Knu88]. Existen otras basadas en envolvimientos convexos, y en dividir y conquistar; se sugiere al lector [Bei13] para un recuento más detallado y ejemplos.

Algoritmos voraces

Las dos técnicas de diseño presentadas hasta ahora responden, cada una a su manera, a una misma pregunta: ¿cómo evitar recorrer por completo el espacio de soluciones de un problema? Dividir, conquistar y combinar lo parte en subespacios independientes que se resuelven por separado; la programación dinámica lo comprime, aprovechando que muchas subinstancias se repiten. Ambas respuestas conservan una característica común: el espacio de soluciones se sigue explorando, solo que de manera organizada. Este capítulo presenta una tercera respuesta y es la más radical de todas: no explorar, sino ir a la fija. Un algoritmo voraz recorre un único camino del espacio de soluciones, aquel que le dicta un criterio de selección fijado de antemano y nunca se desvía de él, para resolver –generalmente– un problema de optimización. De manera más precisa, un *algoritmo voraz* (en inglés, *greedy algorithm*) construye una solución tomando una secuencia de decisiones, cada una de las cuales optimiza un criterio local e inmediato, sin considerar sus consecuencias a largo plazo y sin reconsiderarla después. Lo notable, cuando la técnica funciona, es que esa sucesión de elecciones miopes resulta ser una solución globalmente óptima.

En la práctica, la optimización local rara vez conduce a un óptimo global. Es decir, en el espacio de los algoritmos de optimización, los voraces son realmente excepciones. Entonces, ¿por qué estudiar este tipo de algoritmos si raramente han de funcionar? La razón principal es que cuando los algoritmos voraces existen, estos son elegantes en su diseño, sencillos de programar y eficientes computacionalmente. Hay una frase anónima que resume bien esta situación, casi paradójica:

Los algoritmos voraces no funcionan, pero cuando sí, lo hacen muy bien.

–Autor desconocido.

De esa excepcionalidad se desprende el rasgo que distingue a este capítulo de los anteriores. El diseño de un algoritmo voraz debe estar *siempre* acompañado de una demostración de su corrección, i.e., de que logra resolver el problema de optimización planteado. No se trata de una precaución metodológica, sino de una

necesidad: para casi cualquier problema de optimización se puede proponer un criterio de selección plausible, escribir en pocas líneas el algoritmo voraz correspondiente y ejecutarlo sin que nada falle; el programa responderá y responderá –muy posiblemente– mal. Un algoritmo voraz sin demostración no es un algoritmo con una laguna teórica, es una conjetura.

Nota 4.0.1: ¿Dónde está la dificultad?

Vale la pena contrastar este capítulo con el Capítulo 3. En programación dinámica, el trabajo difícil está en *encontrar* la función objetivo y su definición recurrente; una vez planteadas, la demostración de corrección suele ser una inducción rutinaria y la implementación una transcripción. En los algoritmos voraces sucede exactamente lo contrario: el algoritmo es casi siempre inmediato –ordenar y recorrer, o escoger repetidamente el mínimo o máximo– y todo el esfuerzo intelectual está en demostrar que el criterio escogido es el correcto. El lector que mida el avance por la cantidad de código escrito tendrá la impresión de que este capítulo es más fácil que el anterior.

Ahora bien, estas demostraciones no se inventan desde cero cada vez: siguen un puñado de patrones que se repiten con notable regularidad. Identificarlos, nombrarlos y saber cuál ensayar en cada situación es el objetivo central del capítulo y es también la razón por la cual los ejemplos se presentan en el orden en que se presentan. Se estudiarán tres patrones: el *argumento de intercambio*, que transforma una solución óptima cualquiera en otra que contiene la elección voraz; el *argumento de inversión*, su variante para problemas cuya solución es un ordenamiento; y el argumento de que *el voraz va adelante*, que compara paso a paso lo construido por el algoritmo contra cualquier otra solución. Los tres se introducen sobre ejemplos concretos y se sistematizan en la Sección 4.3.

Es importante aprender a plantear este tipo de demostraciones, más allá de solo tener familiaridad con ellas o inclusive de entenderlas. Las demostraciones de *optimización local*, como se llamarán en el desarrollo del capítulo, intrínsecamente incluyen el diseño del algoritmo voraz correspondiente: cómo elegir el “siguiente” elemento que permitirá, finalmente, contar con una solución óptima (i.e., globalmente óptima). Este patrón ayudará, además, a encontrar un argumento formal –a veces no tan evidente– para justificar por qué un algoritmo voraz funciona.

Nota 4.0.2: Sobre el nombre

En la literatura en inglés esta técnica se conoce como *greedy*, adjetivo que describe a quien toma todo lo que puede en el momento sin pensar en lo

que vendrá después. En castellano circulan las traducciones *voraz*, *ávido* y, con menos fortuna, *glotón*. En este texto se prefiere *voraz* por ser la más extendida, pero conviene no perder de vista lo que el nombre describe: no la calidad del resultado, sino la actitud del algoritmo frente a cada decisión. Elegir lo mejor *ahora* es una descripción del procedimiento; que eso produzca lo mejor *al final* es precisamente lo que hay que demostrar.

Nota 4.0.3: Antes que el nombre, los algoritmos

Como ocurre con el reintento, los algoritmos voraces son bastante más antiguos que la idea de estudiarlos como una técnica de diseño. En 1926, Otakar Borůvka publicó un algoritmo para construir redes eléctricas de costo mínimo que hoy se reconoce como el primer algoritmo para el problema del árbol de cubrimiento mínimo. A ese le siguieron los de Joseph Kruskal (1956) y Robert Prim (1957) para el mismo problema –este último redescubriendo un algoritmo que Vojtěch Jarník había publicado en 1930–, el de David Huffman (1952) para códigos de longitud variable y el de Edsger Dijkstra (1959) para caminos más cortos. Todos ellos son anteriores a cualquier intento de explicar qué tienen en común. Ese intento llegó con la teoría de matroides, en particular con el trabajo de Jack Edmonds a comienzos de la década de 1970, que caracteriza una clase de problemas en los cuales la estrategia voraz es óptima. La caracterización es elegante pero parcial: varios de los problemas de este capítulo se resuelven vorazmente sin caer dentro de ella (ver las notas al final del capítulo).

Este capítulo presenta las ideas principales que respaldan los algoritmos voraces, una metodología para su uso y varios ejemplos, en dominios como el agendamiento óptimo, las redes de costo mínimo y los caminos más cortos.

4.1. Ideas clave

Considere el problema de pagar una cantidad exacta con la menor cantidad posible de monedas, disponiendo de una cantidad ilimitada de monedas de cada denominación. Con las denominaciones de 1, 5, 10 y 25 unidades, y una cantidad de 30 unidades por pagar, la respuesta es inmediata: una moneda de 25 y una de 5, dos monedas en total. Y el razonamiento que produce esa respuesta también lo es: se toma la moneda más grande que quepa, se descuenta y se repite con lo que falte. Este razonamiento es un algoritmo voraz, y se escribe en tres líneas.

Algoritmo 4.1.1

```
1 def coins(D, M, X, m):
2     # D[0..M] holds denominations in decreasing order
3     # and D[M-1] == 1
4     ans = None
5     if X == 0: ans = 0
6     else: ans = X // D[m] + coins(D, M, X % D[m], m + 1)
7     return ans
```

La suposición de que $D[M - 1] = 1$ garantiza que el llamado termina siempre con $X = 0$ (¿por qué?), de modo que la función está bien definida. Y funciona: con $D = [25, 10, 5, 1]$ la función responde el mínimo para cualquier cantidad.

Ejemplo 4.1.1

Con las denominaciones usuales, la función responde lo esperado:

```
1 >>> coins([25, 10, 5, 1], 4, 30, 0)
2 2
3 >>> coins([25, 10, 5, 1], 4, 63, 0)
4 6
```

Ahora considere las denominaciones 1, 3 y 4, y la cantidad 6:

```
1 >>> coins([4, 3, 1], 3, 6, 0)
2 3
```

La respuesta corresponde a las monedas $4 + 1 + 1$. Pero $6 = 3 + 3$ se paga con dos monedas, y por lo tanto la respuesta correcta es 2.

El Ejemplo 4.1.1 merece una mirada cuidadosa del detalle porque contiene en miniatura todo lo que hay que entender sobre esta técnica. El algoritmo no cambió; el código es exactamente el mismo. No hubo error de programación, ni de índices ni un caso base mal planteado. Tampoco hubo una ejecución fallida: la función terminó, retornó un número y ese número es la cantidad de monedas de un pago legítimo de 6 unidades. Lo único que cambió fue el conjunto de denominaciones y con ese cambio desapareció la corrección.

Nota 4.1.1: Fallar en silencio

Fallar en silencio es una forma muy incómoda de fallar que puede tener un algoritmo. Un algoritmo de reintento mal podado se demora; uno de programación dinámica con una recurrencia equivocada suele producir disparates visibles. Un algoritmo voraz con un criterio equivocado produce respuestas plausibles, del tipo correcto, frecuentemente cercanas al óptimo y algunas veces iguales a él. Ninguna cantidad de pruebas con casos pequeños distingue con seguridad un criterio correcto de uno que no lo es –con $D = [4, 3, 1]$, las cantidades $0, \dots, 5$ dan la respuesta correcta y solo la sexta falla. Por eso la demostración no es opcional.

El propósito de esta sección es precisar qué hay que demostrar. Se introduce el vocabulario mínimo para hablar de problemas de optimización y de la manera en que un algoritmo voraz construye sus soluciones, se plantea un esquema general que todos los algoritmos del capítulo instancian y se aíslan las dos propiedades de las cuales depende que ese esquema sea correcto.

4.1.1. Problemas de optimización. Los problemas que se resuelven con algoritmos voraces son, casi siempre, problemas de optimización. La estructura común a todos ellos es la siguiente.

Definición 4.1.1: Problema de optimización

Un *problema de optimización* está dado por un conjunto $\mathcal{F}$ finito y no vacío de *soluciones factibles*, una *función objetivo* $c : \mathcal{F} \rightarrow \mathbb{R}$ y un *sentido*, que es maximizar o minimizar. Una solución factible $s \in \mathcal{F}$ es *óptima* si $c(s) \geq c(s')$ para toda $s' \in \mathcal{F}$ cuando el sentido es maximizar, o $c(s) \leq c(s')$ para toda $s' \in \mathcal{F}$, cuando es minimizar.

Los tres elementos de la Definición 4.1.1 deben leerse en la especificación del problema y conviene practicar esa lectura desde ahora. En el problema de las monedas, $\mathcal{F}$ es el conjunto de las colecciones de monedas que suman exactamente la cantidad pedida, c cuenta cuántas monedas tiene la colección y el sentido es minimizar. Note que la factibilidad –sumar exactamente la cantidad– y la optimalidad –usar pocas monedas– son requisitos de naturaleza distinta: el primero es una condición que se cumple o no se cumple, mientras el segundo es una comparación entre las que sí la cumplen. Perder de vista esa distinción es la fuente más común de confusión al plantear un problema de optimización.

Nota 4.1.2: Sobre la unicidad

La Definición 4.1.1 habla de *una* solución óptima, no de *la* solución óptima, y la diferencia importa. Puede haber varias soluciones factibles distintas con el mismo valor óptimo de la función objetivo; lo único que necesariamente es único es ese valor. Todos los teoremas de este capítulo están enunciados de manera de admitir esa multiplicidad y varias de sus demostraciones dependen precisamente de poder reemplazar una solución óptima por otra igual de buena.

4.1.2. Candidatos, decisiones y criterio de selección. Un algoritmo voraz no examina $\mathcal{F}$: lo construye por partes. Para hacerlo necesita que las soluciones factibles se puedan describir como colecciones de piezas más simples.

Definición 4.1.2: Candidatos y soluciones parciales

Considere un problema de optimización con soluciones factibles $\mathcal{F}$. Un *conjunto de candidatos* para el problema es un conjunto finito C tal que $\mathcal{F} \subseteq \mathcal{P}(C)$. Un subconjunto $S \subseteq C$ es una *solución parcial* sii $S \subseteq S'$ para alguna $S' \in \mathcal{F}$; es decir, sii S todavía se puede extender hasta una solución factible.

Con este vocabulario, la tarea de un algoritmo voraz se describe en una frase: partir de la solución parcial vacía y agregarle candidatos, uno a la vez, hasta obtener una solución factible. Lo que caracteriza a la técnica es cómo se decide cuál candidato agregar.

Definición 4.1.3: Criterio de selección

Un *criterio de selección* sobre un conjunto de candidatos C es un preorden total $\preceq$ sobre C . Un candidato $c \in C'$ es *óptimo* en $C' \subseteq C$ según el criterio sii $c \preceq c'$ para todo $c' \in C'$.

Que el criterio sea un preorden *total* garantiza que siempre haya un candidato óptimo entre los disponibles; que sea un preorden y no un orden admite que haya empates y por lo tanto varios candidatos óptimos a la vez. Ninguna de las dos cosas es un detalle técnico. La primera es lo que hace que el algoritmo nunca se quede sin saber qué hacer. La segunda obliga a que las demostraciones de este capítulo funcionen para *cualquiera* de los candidatos óptimos, no para uno escogido a conveniencia.

Ejemplo 4.1.2

Los criterios de selección de las secciones de este capítulo, enunciados de manera informal:

- En el problema de las monedas, el candidato es una moneda y el criterio prefiere la de mayor denominación.
- En el agendamiento de actividades (Sección 4.2), el candidato es una actividad y el criterio prefiere la que termine más temprano.
- En el cubrimiento de intervalos (Sección 4.4), el candidato es un intervalo y el criterio prefiere, entre los que cubren el extremo pendiente, el que se extienda más lejos.
- En el árbol de cubrimiento mínimo (Sección 4.7), el candidato es un arco y el criterio prefiere el de menor peso.

Observe que en los cuatro casos el criterio se enuncia en una línea y no menciona el objetivo global del problema.

4.1.3. El esquema general. Con los ingredientes anteriores, el esquema que instancian todos los algoritmos voraces del capítulo es el siguiente.

Algoritmo 4.1.2: Esquema general de un algoritmo voraz

Para un conjunto de candidatos C y un criterio de selección $\preceq$:

1. $S = \emptyset$.
2. Mientras $S \notin \mathcal{F}$ y $C \neq \emptyset$:
 - a) escoger $c \in C$ óptimo según $\preceq$ y removerlo de C ;
 - b) si $S \cup \{c\}$ es una solución parcial, entonces $S = S \cup \{c\}$.
3. Si $S \in \mathcal{F}$, retornar S ; de lo contrario, no hay solución.

Tres observaciones sobre el Algoritmo 4.1.2, cada una de las cuales lo separa de las técnicas ya estudiadas.

No hay reintento: Un candidato removido de C en el paso 2(a) no vuelve a considerarse y un candidato agregado a S en el paso 2(b) no vuelve a retirarse. Comparado con el Capítulo 5, donde toda decisión es provisional y puede deshacerse, aquí toda decisión es definitiva.

No hay exploración: En cada iteración se examina un único candidato, el que dicta el criterio. La programación dinámica presentada en el Capítulo 3, en la misma situación, evaluaría todas las alternativas y se quedaría con la mejor; el algoritmo voraz apuesta a una y sigue.

El ciclo termina: Cada iteración remueve un elemento de C , que es finito. Por eso el esquema siempre termina, incluso cuando el criterio de selección es completamente equivocado. Esta es justamente la razón por la cual un criterio equivocado no se manifiesta como una ejecución fallida sino como una respuesta incorrecta (o falla en silencio).

Nota 4.1.3: Criterios estáticos y dinámicos

El paso 2(a) del Algoritmo 4.1.2 pide el candidato óptimo entre los que quedan, y la manera de conseguirlo depende de si el criterio cambia o no durante la ejecución.

Cuando $\preceq$ no depende de S –el peso de un arco, la denominación de una moneda, el tiempo de finalización de una actividad– se dice que el criterio es *estático*. En ese caso el paso 2(a) no requiere estructura alguna: basta ordenar C una sola vez al comienzo y recorrerlo en orden. Esta es la razón por la cual tantos algoritmos voraces tienen la forma *ordenar y recorrer*, y también la razón por la cual su complejidad temporal suele quedar dominada por el $O(|C| \log |C|)$ del ordenamiento.

Cuando $\preceq$ sí depende de la solución parcial construida hasta el momento, el criterio es *dinámico* y hay que recalcularlo, o al menos actualizarlo, en cada iteración. Ordenar de antemano ya no sirve y se necesita una estructura de datos que entregue el mínimo y admita modificaciones, típicamente una cola de prioridad. Los algoritmos de las últimas secciones del capítulo son de este tipo y su análisis es, por eso, más interesante.

4.1.4. Las dos propiedades. Falta lo esencial: ¿bajo qué condiciones el Algoritmo 4.1.2 produce una solución óptima y no simplemente una solución factible? Son dos condiciones y conviene mantenerlas separadas porque fallan de maneras distintas.

La primera dice que comprometerse con la elección voraz no cierra la puerta a la optimalidad.

Definición 4.1.4: Propiedad de elección voraz

Un criterio de selección $\preceq$ satisface la *propiedad de elección voraz* para un problema de optimización sii, para toda instancia con al menos una solución factible no vacía y para todo candidato c óptimo según $\preceq$ entre los disponibles, existe una solución óptima que contiene a c .

La segunda dice que, una vez tomada esa decisión, lo que queda es un problema de la misma naturaleza y basta resolverlo bien.

Definición 4.1.5: Subestructura óptima

Un problema de optimización tiene *subestructura óptima* con respecto a un conjunto de candidatos C sii, para todo candidato $c \in C$ que hace parte de alguna solución óptima, se cumple que $S \cup \{c\}$ es una solución óptima del problema siempre que S sea una solución óptima del problema que resulta de comprometerse con c .

Esta es la Definición 3.1.1 del Capítulo 3, particularizada a la descomposición propia de este capítulo: la que consiste en comprometerse con un candidato, de modo que la única subinstancia es el problema que queda después del compromiso. De las dos mitades que tiene aquella definición, aquí se enuncia únicamente la de composición, que es la que necesita la demostración del Teorema 4.1.1 por ser una inducción hacia adelante. El papel de la otra mitad, la de descomposición, lo cumple en este contexto la propiedad de elección voraz de la Definición 4.1.4: es ella la que garantiza que existe una solución óptima que contiene el candidato escogido.

Estas dos propiedades son exactamente lo que hace falta y esa es la razón de ser de todo el aparato anterior.

Teorema 4.1.1

Suponga que el criterio de selección $\preceq$ satisface la propiedad de elección voraz y que el problema tiene subestructura óptima, ambas con respecto al conjunto de candidatos C . Entonces el Algoritmo 4.1.2 retorna una solución óptima siempre que exista alguna solución factible.

Demostración

Se procede por inducción sobre $|C|$.

Si $C = \emptyset$, la única solución parcial es $\emptyset$ y, por suposición, existe alguna solución factible; luego $\emptyset \in \mathcal{F}$ y es la única, por lo que es óptima. El algoritmo retorna $\emptyset$.

Si $C \neq \emptyset$, considérese la primera iteración del ciclo. Si $\emptyset \in \mathcal{F}$ el ciclo no se ejecuta y el argumento del caso anterior aplica. De lo contrario, sea c el candidato escogido en el paso 2(a). Por la propiedad de elección voraz existe una solución óptima S^* con $c \in S^*$; en particular $\{c\}$ es una solución

parcial y el paso 2(b) efectivamente agrega c a S . Las iteraciones restantes del ciclo son una ejecución del mismo esquema sobre el problema que resulta de comprometerse con c , con conjunto de candidatos $C \setminus \{c\}$ y el mismo criterio de selección. Como $|C \setminus \{c\}| < |C|$ y ese problema tiene solución factible —la propia S^* lo atestigua— la hipótesis inductiva garantiza que dichas iteraciones producen una solución óptima S para él. Por la subestructura óptima, $S \cup \{c\}$ es una solución óptima del problema original, que es lo que el algoritmo retorna.

Nota 4.1.4: Optimización local y global

El Teorema 4.1.1 explica la organización que siguen todas las secciones de este capítulo. En cada una se demuestran dos teoremas:

- un *teorema de optimización local*, que enuncia la propiedad de elección voraz para el criterio propuesto, y es siempre la parte más retadora;
- un *teorema de optimización global*, que concluye que el algoritmo es correcto, y que no es más que la instancia del Teorema 4.1.1 para el problema en cuestión.

La subestructura óptima no suele enunciarse por separado porque en los problemas de este capítulo es evidente: comprometerse con una decisión deja un problema del mismo tipo sobre menos candidatos. Conviene, de todos modos, verificarla y no darla por descontada.

4.1.5. ¿Por qué falla? Con el vocabulario ya disponible se puede decir con precisión qué pasó en el Ejemplo 4.1.1. El problema de las monedas tiene subestructura óptima y la tiene con cualquier conjunto de denominaciones: si una solución óptima para pagar X usa una moneda de denominación d , entonces lo que resta de esa solución es necesariamente una solución óptima para pagar $X - d$ (¿por qué?). La propiedad que falla es la otra. Con $D = [4, 3, 1]$ y $X = 6$, el candidato óptimo según el criterio es la moneda de 4; la única solución óptima es $\{3, 3\}$ y no contiene ninguna moneda de 4. Es decir, el criterio no satisface la propiedad de elección voraz.

Nota 4.1.5: Un contraejemplo basta

Note la asimetría entre demostrar y refutar. Demostrar la propiedad de elección voraz exige un argumento que cubra todas las instancias del problema.

Refutarla exige exhibir una sola instancia, típicamente una diminuta: el contraejemplo de las monedas usa tres denominaciones y la cantidad 6. De ahí una recomendación práctica que este texto insiste en repetir: antes de intentar demostrar que un criterio funciona, dedíquele un rato honesto a intentar romperlo con instancias pequeñas. Es mucho más rápido y en la mayoría de los casos –recuerde que los algoritmos voraces son la excepción– va a funcionar. El Ejercicio 5 propone hacerlo de manera sistemática.

Nota 4.1.6: Voraces y programación dinámica

Las dos técnicas comparten la exigencia de subestructura óptima y se separan exactamente en la otra propiedad. La programación dinámica no supone nada parecido a la elección voraz: en cada paso considera todas las decisiones disponibles, resuelve el subproblema que resulta de cada una y se queda con la mejor. Por eso siempre funciona –el problema de las monedas con $D = [4, 3, 1]$ se resuelve sin dificultad con las técnicas del Capítulo 3– y por eso cuesta más.

La relación entre ambas se puede resumir así: un algoritmo voraz es una solución de programación dinámica en la cual se ha demostrado que no hace falta evaluar todas las alternativas porque siempre gana la misma. Esa demostración es el teorema de optimización local y el premio por conseguirla es pasar de una tabla a un recorrido.

4.1.6. Análisis. El análisis de un algoritmo voraz es directo y contrasta con el enfoque que tuvo el del capítulo anterior.

Teorema 4.1.2

Suponga que verificar si $S \cup \{c\}$ es una solución parcial toma tiempo $O(f)$ y que escoger el candidato óptimo entre los disponibles toma tiempo $O(g)$. Entonces el Algoritmo 4.1.2 toma tiempo $O(|C| \cdot (f + g))$. Si además el criterio de selección es estático, se puede ordenar C al comienzo y obtener tiempo $O(|C| \log |C| + |C| \cdot f)$.

Demostración

Se propone como ejercicio al lector.

No hay aquí nada parecido al factor $|V|$ que domina el costo de un algoritmo de reintento y que ningún análisis asintótico logra acotar con precisión. El ciclo del Algoritmo 4.1.2 ejecuta a lo sumo $|C|$ iteraciones siempre, para toda instancia, sin importar el criterio. Los algoritmos voraces son eficientes por construcción; lo que hay que demostrar no es que sean rápidos, sino que sean correctos.

Ejercicios

1. Demuestre el Teorema 4.1.2.
2. Justifique las siguientes afirmaciones hechas en esta sección:
 - a) El Algoritmo 4.1.1 termina siempre, bajo la suposición de que $D[M - 1] = 1$.
 - b) El problema de las monedas tiene subestructura óptima con cualquier conjunto de denominaciones.
 - c) El conjunto vacío es una solución parcial de todo problema de optimización con al menos una solución factible.
3. En la Definición 4.1.3 se pide que el criterio de selección sea un preorden *total*. Explique qué le ocurre al Algoritmo 4.1.2 si se admiten criterios que sean órdenes parciales, y por qué la demostración del Teorema 4.1.1 deja de funcionar.
4. El paso 2(b) del Algoritmo 4.1.2 descarta el candidato escogido cuando no puede agregarse a S .
 - a) Dé un ejemplo de un problema de esta sección o de la siguiente en el cual ese descarte efectivamente ocurra.
 - b) ¿Qué pasaría si, en lugar de descartarlo, el algoritmo lo devolviera a C ?
5. Implemente en Python el Algoritmo 4.1.1 y una función que calcule por fuerza bruta la mínima cantidad de monedas.
 - a) Escriba un programa que genere conjuntos de denominaciones al azar (siempre incluyendo la denominación 1) y cantidades pequeñas, y reporte la primera discrepancia entre las dos funciones.
 - b) ¿Qué tan pequeño es el menor contraejemplo que encuentra?
 - c) Repita el experimento restringiéndose a denominaciones de la forma $1, b, b^2, b^3, \dots$ para un $b \geq 2$ fijo. ¿Qué observa?
6. Demuestre que si las denominaciones son $1, b, b^2, \dots, b^{M-1}$ para algún $b \geq 2$, entonces el criterio de escoger la moneda de mayor denominación que quepa satisface la propiedad de elección voraz. *Ayuda:* demuestre primero que ninguna solución óptima puede usar b o más monedas de una misma denominación distinta de la mayor.

7. Considere el problema de escoger un subconjunto de peso máximo de un arreglo $A[0..N)$ de números, sujeto a la restricción de que no se pueden escoger dos posiciones consecutivas.
 - a) Identifique el conjunto de candidatos, las soluciones factibles y la función objetivo, según la Definición 4.1.1.
 - b) Proponga un criterio de selección y refútelo con un contraejemplo.
 - c) Proponga un segundo criterio y refútelo también.
8. 🌐 La Definición 4.1.2 exige que las soluciones factibles sean subconjuntos de un conjunto de candidatos, lo cual deja por fuera los problemas cuya solución es un ordenamiento, como el de la Sección 4.5. Proponga una manera de acomodarlos dentro del marco de esta sección y discuta si el Teorema 4.1.1 sigue siendo aplicable.

4.2. Agendamiento de actividades

Considere el problema de agendar procedimientos quirúrgicos en la sala de cirugías de un hospital o clínica. Cada cirugía tiene un horario, con horas de inicio y finalización. Suponiendo que entre más cirugías se hagan, mayor será el beneficio para los pacientes (acá nada tiene que ver el dinero, ¿cierto?), surge la siguiente pregunta: ¿cómo maximizar la cantidad de cirugías a realizar sin que haya conflicto de horario entre ellas?

De una forma más general, esta pregunta es formulada en el problema del agendamiento de actividades.

Problema 4.2.1: Agendamiento de actividades

- Entrada:** Un arreglo $A[0..N)$, $N \geq 0$, de parejas de números (s_n, e_n) tales que $0 \leq s_n < e_n$ indican el tiempo de inicio y finalización de la actividad $0 \leq n < N$, respectivamente.
- Salida:** Máxima cantidad de actividades en $A[0..N)$ que se pueden agendar sin conflicto.

Como convención para cualquier n , con $0 \leq n < N$, las expresiones $A[n][0]$ y $A[n][1]$ denotan, respectivamente, el tiempo de inicio y finalización de la actividad n : es decir, $s_n = A[n][0]$ y $e_n = A[n][1]$. Note que todos los tiempos se expresan con cantidades no negativas. Es necesario precisar qué significa actividades *compatibles* y en *conflicto* para entender completamente la especificación del problema de agendamiento de actividades.

Nota 4.2.1

Por convención, se entenderá que cada actividad (s, e) representa el intervalo cerrado-abierto de tiempo $[s..e)$. De esta forma, dos actividades (s_i, e_i) y (s_j, e_j) son *compatibles* sii $[s_i..e_i) \cap [s_j..e_j) = \emptyset$; de lo contrario, se dice que están en *conflicto*.

El primer paso es leer la especificación con el vocabulario de la Sección 4.1. Los tres elementos de la Definición 4.1.1 están todos en el enunciado del Problema 4.2.1:

- las *soluciones factibles* son los subconjuntos de actividades de $A[0..N)$ que no tienen conflictos entre sí, llamados *agendamientos*;
- la *función objetivo* de un agendamiento es su tamaño; y
- el *sentido* es maximizar.

El conjunto de candidatos, en el sentido de la Definición 4.1.2, son las N actividades, y una solución parcial es simplemente un agendamiento –cualquier conjunto sin conflictos se puede extender hasta uno maximal, en particular hasta uno de tamaño máximo o no.

Nota 4.2.2: Maximal y máximo

Conviene distinguir dos adjetivos que se parecen y no significan lo mismo. Un agendamiento es *maximal* sii no se le puede agregar ninguna actividad más sin producir un conflicto; es *máximo* sii ningún otro agendamiento tiene más actividades que él.

Todo agendamiento máximo es maximal, pero no al contrario. Con las tres actividades $(0, 10)$, $(1, 3)$ y $(4, 6)$, el agendamiento $\{(0, 10)\}$ es maximal –las otras dos están en conflicto con ella– y sin embargo $\{(1, 3), (4, 6)\}$ es más grande. El Problema 4.2.1 pide un agendamiento **máximo** y esa es la palabra que aparecerá en los enunciados de esta sección. La distinción no es una sutileza: encontrar un agendamiento maximal es trivial y encontrar uno máximo es el problema.

Ejemplo 4.2.1

Se ilustra cada cirugía como un rectángulo cuya coordenada izquierda en el eje horizontal denota el tiempo de inicio y la derecha el tiempo de finalización.



Las actividades que tienen su fondo resaltado hacen parte de un agendamiento máximo, sin conflictos entre ellas. Dada esta entrada, la salida es 6 para el problema de agendamiento.

El problema de agendamiento se puede resolver exhaustivamente enumerando todos los subconjuntos de actividades y, entre aquellos que no tienen conflictos, determinar el tamaño máximo posible. Es más, este problema exhibe las propiedades de la subestructura óptima y solapamiento (¿por qué?), y por ello es candidato a ser resuelto con programación dinámica (ver Ejercicio 15). El enfoque voraz es distinto al de una exploración exhaustiva o al de utilizar programación dinámica, pues evita apostarle a ‘todos los caballos’ y en cambio apuesta solo a uno de ellos, a uno que ganará (o que no perderá).

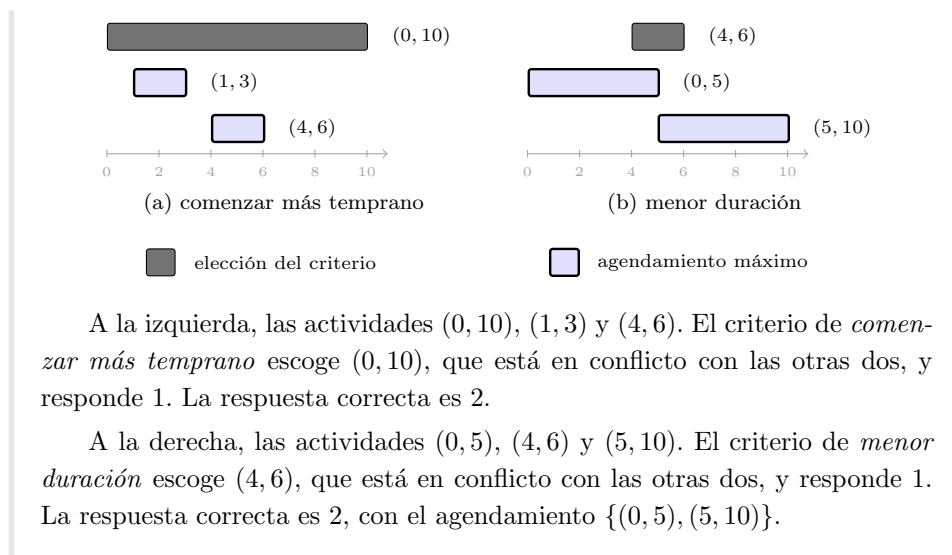
4.2.1. Escoger un criterio. Fijados los candidatos, lo único que falta para tener un algoritmo es un criterio de selección. Y aquí conviene resistir la tentación de proponer el primero que se le ocurra a uno y ponerse a demostrar que funciona, porque lo más probable –según se advirtió en la Nota 4.1.5– es que no funcione. La disciplina que este texto recomienda es la contraria: proponer varios criterios plausibles e intentar romperlos todos. Los que sobrevivan son los candidatos a demostración.

Para el agendamiento de actividades hay al menos cuatro criterios que se le ocurren a cualquiera:

1. escoger la actividad que *comienza* más temprano;
2. escoger la actividad de menor *duración*;
3. escoger la actividad que está en conflicto con la menor *cantidad* de las restantes;
4. escoger la actividad que *termina* más temprano.

En los cuatro casos se entiende que, tras escoger una actividad, se descartan las que estén en conflicto con ella y se repite el procedimiento. Los dos primeros criterios se rompen con tres actividades.

Ejemplo 4.2.2: Dos criterios que no funcionan



Note lo pequeños que son los contraejemplos y lo rápido que se encuentran: tres actividades bastan en ambos casos y en ambos la estrategia falla por la misma razón, que es haber escogido una actividad que consume demasiado tiempo del recurso.

El tercer criterio es más resistente y no se rompe con instancias tan pequeñas; encontrar un contraejemplo requiere algo de trabajo y se propone en el Ejercicio 9. Que un criterio sobreviva a los primeros intentos de refutación es información útil, pero no es una demostración: la única manera de saber si funciona es demostrarlo.

El cuarto criterio es el que sobrevive y la intuición que lo respalda es la siguiente: entre más extenso sea el rango que queda libre para agendar cirugías, más cirugías se podrán agendar. Suponga que se cuenta con un conjunto X de cirugías por agendar. La pregunta clave es, ¿cuál agendar primero? Bajo la intuición anterior, se puede elegir una cirugía $a \in X$ que termine lo más pronto posible dado que esto permite que, para una posterior elección, haya más tiempo para agendar otras cirugías bajo la misma estrategia.

Algoritmo 4.2.1

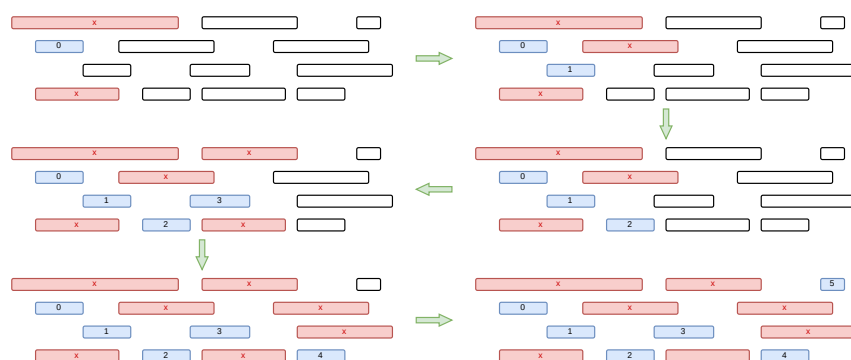
El Algoritmo 4.1.2 instanciado de la siguiente manera:

- el conjunto de candidatos C es el de las actividades en $A[0..N]$;
- el criterio de selección es $a \preceq b$ sii a no termina después de b ; y
- $S \cup \{a\}$ es una solución parcial sii a es compatible con todas las actividades en S .

Desenrollado, el Algoritmo 4.2.1 hace lo siguiente: escoge una actividad que termine lo más pronto posible, la agenda, descarta todas las que estén en conflicto con ella y repite el procedimiento con las que queden, hasta que no quede ninguna.

Ejemplo 4.2.3

El siguiente diagrama muestra, paso a paso, cómo el Algoritmo 4.2.1 selecciona un agendamiento máximo a partir del conjunto de actividades en el Ejemplo 4.2.1.



Cada elección hecha por el algoritmo se indica con un número consecutivo y aquellas actividades que van siendo descartadas porque tienen conflictos con al menos una de las actividades seleccionadas se marcan con una 'x'.

Lo que resta de la sección es demostrar que el Algoritmo 4.2.1 es correcto. De acuerdo con el Teorema 4.1.1, hay que establecer dos cosas: que el criterio satisface la propiedad de elección voraz y que el problema tiene subestructura óptima.

4.2.2. Optimización local: un argumento de intercambio. El teorema de optimización local enuncia, para este problema particular, la propiedad de elección voraz de la Definición 4.1.4.

Teorema 4.2.1: Optimización local

Sea X un conjunto finito y no vacío de actividades. Si $a \in X$ es tal que su tiempo de finalización es mínimo en X , entonces a hace parte de un agendamiento máximo de actividades de X .

Demostración

Suponga que $Y \subseteq X$ es un agendamiento máximo. Como $X \neq \emptyset$, necesariamente $Y \neq \emptyset$ (¿por qué?). Se procede por casos:

- Si $a \in Y$, entonces la conclusión del teorema es correcta.
- Si $a \notin Y$, entonces sea b aquella actividad en Y que tiene tiempo de finalización mínimo. Necesariamente $b \neq a$, porque de lo contrario $a \in Y$ (una contradicción). Considere el conjunto $Z = (Y \setminus \{b\}) \cup \{a\}$. Como $b \in Y$ y $a \notin Y$, se tiene que $|Z| = |Y|$. Además note que Z no tiene conflictos porque el tiempo de finalización de b no puede ser menor que el de a (a tiene tiempo de finalización mínimo entre todas las actividades de X , luego también de Y): al remover b de Y , las actividades que están en $Y \setminus \{b\}$ no tienen conflicto y como a no termina después de b , Z no tiene conflictos (por suposición, en Y no hay conflictos). Luego, Z es un agendamiento de actividades de X con $|Z| = |Y|$ que contiene a a ; como Y es máximo, Z también lo es, que es lo que se deseaba.

Nota 4.2.3: El argumento de intercambio

La demostración del Teorema 4.2.1 es el primer ejemplar de un patrón que reaparecerá varias veces en este capítulo y que conviene reconocer desde ahora. Se llama *argumento de intercambio* y su forma es siempre la misma:

1. se parte de una solución óptima *cualquiera* Y ;
2. si Y ya contiene la elección voraz a , no hay nada que hacer;
3. de lo contrario, se identifica en Y un elemento b que ocupa el “lugar” de a , se construye Z intercambiando b por a , y se demuestra que Z sigue siendo factible y que no es peor que Y .

Lo que hace funcionar al paso 3 es siempre alguna propiedad extrema (i.e., de minimalidad o maximalidad) de la elección voraz. Aquí es que a no termina después de b , lo cual garantiza que ninguna actividad que era compatible con b deja de serlo con a . Al diseñar un argumento de intercambio, la pregunta que hay que hacerse es: *¿qué le permite a la elección voraz reemplazar a cualquier otra sin causar daño?*

4.2.3. Optimización global. Con la propiedad de elección voraz demostrada, falta verificar la subestructura óptima. En este problema es directa.

Teorema 4.2.2: Subestructura óptima

Sea X un conjunto finito de actividades y $a \in X$ una actividad que hace parte de algún agendamiento máximo de X . Sea X_a el conjunto de las actividades de X compatibles con a . Si S es un agendamiento máximo de X_a , entonces $S \cup \{a\}$ es un agendamiento máximo de X .

Demostración

El conjunto $S \cup \{a\}$ es un agendamiento de X porque S no tiene conflictos y todas sus actividades son compatibles con a , por definición de X_a . Falta ver que es máximo.

Sea Y un agendamiento máximo de X que contiene a a ; existe por suposición. Entonces $Y \setminus \{a\}$ es un agendamiento de X_a (¿por qué?) y, como S es máximo en X_a , se tiene $|Y| - 1 = |Y \setminus \{a\}| \leq |S|$. Luego $|Y| \leq |S| + 1 = |S \cup \{a\}|$. Como Y es máximo en X y $S \cup \{a\}$ es un agendamiento de X , también $|S \cup \{a\}| \leq |Y|$. En conclusión $|S \cup \{a\}| = |Y|$ y $S \cup \{a\}$ es máximo.

Teorema 4.2.3: Optimización global

Sea X un conjunto finito de actividades. El Algoritmo 4.2.1 construye un agendamiento máximo para X .

Demostración

El criterio de selección satisface la propiedad de elección voraz por el Teorema 4.2.1 y el problema tiene subestructura óptima por el Teorema 4.2.2. El conjunto vacío es siempre un agendamiento, de modo que hay al menos una solución factible. Por el Teorema 4.1.1, el Algoritmo 4.2.1 retorna un agendamiento máximo.

Vale la pena detenerse en lo corta que resultó esta demostración. Todo el trabajo está en el Teorema 4.2.1, que es donde se decide si el criterio sirve o no; el Teorema 4.1.1 se encarga del resto de una vez y para siempre. Esta es exactamente la división de labores que anunció la Nota 4.1.4 y que se repetirá en todas las secciones del capítulo.

4.2.4. Otra ruta: el voraz va adelante. La demostración de la subsección anterior no es la única posible y conviene conocer una alternativa porque hay problemas para los cuales es la única que se consigue. En lugar de apoyarse en el

Teorema 4.1.1, se compara directamente lo que construye el algoritmo contra un agendamiento arbitrario y se demuestra que el primero *va adelante* del segundo en todo momento.

Teorema 4.2.4: El voraz va adelante

Sea X un conjunto finito de actividades, sean $a_0, \dots, a_{n-1}$ las actividades que selecciona el Algoritmo 4.2.1 y sean $b_0, \dots, b_{m-1}$ las de un agendamiento cualquiera de X , ambas colecciones ordenadas ascendentemente por tiempo de finalización. Entonces, para todo $0 \leq k < n \wedge m$, la actividad a_k no termina después de b_k .

Demostración

Se procede por inducción sobre k .

Si $k = 0$, la actividad a_0 es una de tiempo de finalización mínimo en todo X y $b_0 \in X$; luego a_0 no termina después de b_0 .

Si $k > 0$, por la hipótesis inductiva a_{k-1} no termina después de b_{k-1} . Como $b_0, \dots, b_{m-1}$ no tiene conflictos y está ordenado por tiempo de finalización, la actividad b_k comienza después de que b_{k-1} termina (¿por qué?) y, por lo tanto, después de que a_{k-1} termina. Es decir, b_k es compatible con todas las actividades en $a_0, \dots, a_{k-1}$ y, en consecuencia, estaba disponible cuando el algoritmo escogió a_k . Como el algoritmo escoge una actividad de tiempo de finalización mínimo entre las disponibles, a_k no termina después de b_k .

Demostración: Segunda demostración del Teorema 4.2.3

El Algoritmo 4.2.1 construye un agendamiento (¿por qué?), de modo que basta ver que $n \geq m$ para cualquier agendamiento $b_0, \dots, b_{m-1}$ de X .

Hacia una contradicción, suponga que $m > n$. Entonces la actividad b_n existe y, por el Teorema 4.2.4 con $k = n - 1$, la actividad a_{n-1} no termina después de b_{n-1} . Como b_n comienza después de que b_{n-1} termina, la actividad b_n es compatible con todas las actividades en $a_0, \dots, a_{n-1}$; es decir, seguía disponible cuando el algoritmo se detuvo. Esto contradice que el algoritmo se detiene únicamente cuando no quedan candidatos disponibles. Luego $n \geq m$.

Nota 4.2.4: ¿Cuál usar?

Los dos argumentos demuestran lo mismo y ninguno es superior al otro en general. El de intercambio es más corto cuando se consigue, porque delega la inducción en el Teorema 4.1.1, y es el que se ensaya primero. Depende de poder exhibir la solución óptima como algo que se puede modificar localmente sin dañar. El de *el voraz va adelante* es más laborioso, pero más flexible: no exige intercambiar nada, solo comparar. Es la herramienta indicada cuando la solución del problema es una secuencia y lo que se quiere comparar es el progreso paso a paso. Se volverá sobre él en la Sección 4.8, donde el argumento de intercambio no está disponible.

4.2.5. Una implementación eficiente. Hasta este punto, el diseño y el análisis del algoritmo voraz se ha hecho con base en una abstracción del problema original, suponiendo que la colección de actividades es un conjunto. De acuerdo con la especificación del problema, la colección de actividades es un arreglo $A[0..N]$ de parejas. Por ello, aún hace falta algo más de trabajo para llegar a tener un algoritmo que opere sobre arreglos, con la estrategia voraz diseñada.

El criterio de selección de este problema es *estático* en el sentido de la Nota 4.1.3: el tiempo de finalización de una actividad no cambia con lo que se haya agendado antes. Aplica entonces la receta de *ordenar y recorrer*, y lo único que falta es evitar hacer un cálculo costoso, en el caso recurrente, de aquellas actividades que son compatibles con la actividad seleccionada.

Suponga que $A[0..N]$ está ordenado ascendentemente por el tiempo de finalización de las actividades. Note que en este caso, si se ha hecho el mejor agendamiento posible para $A[0..n]$, con $0 \leq n \leq N$, las actividades candidatas para extender este mejor agendamiento parcial están en $A[n..N]$. Esto es porque el tiempo de finalización de estas actividades no es menor que el tiempo de finalización de la actividad $n - 1$ (y, por ende, de ninguna actividad antes de ella). Es decir, si se tiene registro de qué actividades han sido exploradas (e.g., $A[0..n]$) y el tiempo de finalización de la última tarea seleccionada allí, entonces se puede recurrir sobre las demás actividades en $A[n..N]$, en orden: se descarta $A[n]$ cuando tiene conflicto con alguna de las ya seleccionadas en $A[0..n]$ (esto se hace con base en el tiempo de finalización de la última actividad seleccionada) o se selecciona porque termina más temprano entre todas aquellas actividades en $A[n..N]$ que no generan conflicto con la selección hecha hasta ese punto.

La siguiente función objetivo, para $0 \leq n \leq N$ y $t \in \mathbb{R}_{\geq 0}$, ayuda a plasmar matemáticamente la idea elaborada en el párrafo anterior:

$\phi(n, t)$: “máxima cantidad de actividades compatibles en $A[n..N]$
que no tienen conflicto con el intervalo $[0..t]$.”

Para resolver el problema de agendamiento de actividades, el objetivo es calcular $\phi(0, 0)$ suponiendo que ninguna actividad tiene tiempos expresados con números negativos, como está estipulado en la especificación del problema.

La definición recurrente de ϕ se establece tomando a n como pivote, y seleccionando o descartando actividades en relación con el intervalo $[0..t)$. Formalmente, para $0 \leq n \leq N$ y $t \in \mathbb{R}_{\geq 0}$, se propone la siguiente definición recurrente de la función objetivo:

$$\phi(n, t) = \begin{cases} 0 & , \text{ si } n = N, \\ \phi(n+1, t) & , \text{ si } n \neq N \wedge A[n][0] < t, \\ 1 + \phi(n+1, A[n][1]) & , \text{ si } n \neq N \wedge A[n][0] \geq t. \end{cases}$$

El caso base corresponde a la situación en la cual no hay actividades para agendar; el agendamiento óptimo tiene tamaño 0. El caso inductivo se divide en dos, siempre suponiendo que hay al menos una actividad que potencialmente puede ser agendada (i.e., $n \neq N$). Si hay conflicto entre la actividad n y el intervalo “protegido” (i.e., $A[n][0] < t$), entonces esta se ignora y se recurre con el resto de las actividades. De lo contrario, si no hay conflicto entre la actividad n y el intervalo protegido (i.e., $A[n][0] \geq t$), entonces se selecciona la actividad y se recurre con las demás que están pendientes por explorar, actualizando a $[0..A[n][1])$ el intervalo protegido (i.e., con el cual se quiere evitar un conflicto).

Nota 4.2.5: Dónde quedó el esquema

Los dos pasos del Algoritmo 4.1.2 siguen ahí, aunque ya no se vean. El paso 2(a) –escoger el candidato óptimo entre los disponibles– lo realiza el ordenamiento previo junto con el recorrido de izquierda a derecha: la actividad n es siempre, entre las que quedan, una de tiempo de finalización mínimo. El paso 2(b) –verificar si el candidato se puede agregar– es la comparación $A[n][0] \geq t$, y el parámetro t es todo el estado que hace falta mantener para hacerla en tiempo constante.

Teorema 4.2.5

Suponga que las actividades en $A[0..N)$ están ordenadas ascendentemente por tiempo de finalización (i.e., $A[i][1] \leq A[j][1]$, para $0 \leq i < j < N$). Entonces:

1. Para $0 \leq n \leq N$ y $t \in \mathbb{R}_{\geq 0}$, el valor de $\phi(n, t)$ es la máxima cantidad de actividades compatibles en $A[n..N)$ que no tienen conflicto con el intervalo $[0..t)$.

2. El valor de $\phi(0, 0)$ es la máxima cantidad de actividades en $A[0..N)$ que se pueden agendar sin conflicto.

Demostración

Se propone como ejercicio al lector.

Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación. A continuación se presenta el Algoritmo 4.2.2, como la implementación de la función **act** que permite calcular ϕ .

Algoritmo 4.2.2

```

1 def act(A, N, n, t):
2     ans = None
3     if n==N: ans = 0
4     else:
5         if A[n][0]<t: ans = act(A, N, n+1, t)
6         else: ans = 1 + act(A, N, n+1, A[n][1])
7     return ans

```

A continuación se presenta un ejemplo con un llamado a la función **act** para un conjunto de actividades que, a escala, representan a aquellas en el Ejemplo 4.2.1.

Ejemplo 4.2.4

Considere el siguiente arreglo de parejas que identifica las actividades del Ejemplo 4.2.1:

```

1 A = [ (0, 14), (16, 24), (29, 31),
2       (2, 6), (9, 17), (22, 30),
3       (6, 10), (15, 20), (24, 32),
4       (2, 9), (11, 15), (16, 23), (24, 28) ]

```

Después de ordenar el arreglo ascendentemente por la segunda componente de las parejas, se obtiene:

```

1 >>> A.sort(key = lambda x: x[1])
2 >>> A
3 [ (2, 6), (2, 9), (6, 10), (0, 14), (11, 15),
4   (9, 17), (15, 20), (16, 23), (16, 24), (24, 28),

```



```
5 (22, 30), (29, 31), (24, 32) ]
```

Con base en la versión ordenada del arreglo, se tiene:

```
1 >>> act(A, len(A), 0, 0)
```

```
2 6
```

El Teorema 4.2.6 presenta los resultados de correctitud de la función `act`, junto con sus complejidades temporal y espacial.

Teorema 4.2.6

Si las actividades en $A[0..N)$ están ordenadas ascendentemente por tiempo de finalización, entonces el llamado `act(A, N, 0, 0)` calcula $\phi(0,0)$, y toma tiempo y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$ y $O(1)$, respectivamente. Si es necesario ordenar el arreglo, entonces la complejidad temporal es $O(N \log N)$.

Demostración

Note que la función `act` es una transcripción de la función ϕ , agregando los parámetros A y N que nunca son modificados. El tiempo de ejecución de `act` es linealmente proporcional a la cantidad de elementos en $A[0..N)$, i.e., es $O(N)$. El espacio es $O(1)$ sin tener en cuenta el espacio requerido para una pila de recurrencia sin optimizaciones al código. Teniendo en cuenta el espacio de la pila, el espacio es $O(N)$ debido a los llamados recurrentes.

El resultado coincide con lo que anticipa el Teorema 4.1.2 para un criterio estático: verificar la factibilidad de un candidato toma tiempo constante, de modo que el costo total es $O(N \log N + N) = O(N \log N)$ y queda dominado por el ordenamiento.

Ejercicios

1. Diseñe un algoritmo de búsqueda exhaustiva que enumere todos los subconjuntos de actividades en $A[0..N)$ para resolver el problema de agendamiento.
2. Enumere todos los agendamientos máximos del conjunto de actividades presentado en el Ejemplo 4.2.1.
3. Explique por qué el problema de agendamiento de actividades exhibe las propiedades de subestructura óptima y solapamiento. Ilustre la explicación con un ejemplo.
4. De acuerdo con la Nota 4.2.2, todo agendamiento máximo es maximal pero no al contrario.
 - a) Demuestre la primera afirmación.
 - b) Diseñe un algoritmo que, en tiempo $O(N)$ sobre un arreglo ya ordenado, produzca un agendamiento maximal.
 - c) ¿Produce el Algoritmo 4.2.1 un agendamiento maximal? Justifique.
5. Demuestre que cada una de las siguientes afirmaciones hechas en la demostración del Teorema 4.2.1 es cierta:
 - a) Si $X \neq \emptyset$, entonces $Y \neq \emptyset$.
 - b) Por suposición, Y no tiene conflictos.
 - c) El conjunto Z no tiene conflictos.
6. En la demostración del Teorema 4.2.2 se afirma que $Y \setminus \{a\}$ es un agendamiento de X_a . Demuestre que esta afirmación es correcta.
7. En la demostración del Teorema 4.2.4 se afirma que la actividad b_k comienza después de que b_{k-1} termina. Justifique por qué esta afirmación es cierta y señale exactamente dónde se usa que $b_0, \dots, b_{m-1}$ no tiene conflictos.
8. El Teorema 4.2.4 está enunciado para un agendamiento cualquiera, no necesariamente máximo.
 - a) ¿Se usa esa generalidad en la segunda demostración del Teorema 4.2.3?
 - b) ¿Seguiría siendo cierto el enunciado si las dos colecciones se ordenaran ascendentemente por tiempo de *inicio* en lugar de por tiempo de finalización?
9. 🌈 Considere el criterio de escoger la actividad que está en conflicto con la menor cantidad de las restantes. Construya un contraejemplo que demuestre que no satisface la propiedad de elección voraz. *Ayuda:* no existe ninguno con menos de 11 actividades; conviene pensar en una actividad “central” con pocos conflictos cuya selección arruine el agendamiento.

10. En el planteamiento recurrente de ϕ , el caso en el cual $n \neq N \wedge A[n][0] \geq t$ indica que la actividad n se considera como parte del agendamiento óptimo que se está construyendo. ¿Por qué no hay conflicto si $A[n][0]$ puede ser igual a t ?
11. Justifique con un ejemplo, por qué la suposición de ordenamiento de $A[0..N)$ es necesaria para que el Teorema 4.2.5 sea cierto.
12. Considere la siguiente función objetivo para el problema de agendamiento de actividades, con $0 \leq n \leq N$ y $t \in \mathbb{R}_{\geq 0} \cup \{+\infty\}$:

$$\bar{\phi}(n, t) : \quad \text{“máxima cantidad de actividades compatibles en } A[0..n) \\ \text{que no tienen conflicto con el intervalo } [t.. + \infty).”$$

Para resolver el problema de agendamiento de actividades, el objetivo es calcular $\bar{\phi}(N, +\infty)$. Proponga un planteamiento recurrente para $\bar{\phi}$, y diseñe un algoritmo voraz que la calcule en tiempo $O(N)$ y espacio $O(1)$.

13. Diseñe un algoritmo voraz que resuelva en tiempo $O(N)$ el problema de agendamiento bajo la suposición de que el arreglo $A[0..N)$ está ordenado:
 - a) Ascendentemente por tiempo inicio de las actividades.
 - b) Descendentemente por tiempo de inicio de las actividades.
14. La estrategia voraz diseñada en esta sección está basada en la idea de escoger aquella actividad que termine lo más pronto posible entre aquellas que no generan conflicto, para posteriormente recurrir. Para cada una de las siguientes estrategias voraces, o bien demuestre que sirve como regla de optimización local para resolver el problema de agendamiento de actividades (si hay empates, se debería poder escoger cualquiera de las posibilidades) o describa un pequeño contraejemplo que ilustre por qué no sirve:
 - a) Escoja aquella actividad a que *termina* lo más tarde posible, descarte las actividades que están en conflicto con a y recurra.
 - b) Escoja aquella actividad a que *comienza* lo más tarde posible, descarte las actividades que están en conflicto con a y recurra.
 - c) Si no hay conflicto, escoja todas las actividades; de lo contrario, descarte la actividad con la *mayor duración* y recurra.
 - d) Si no hay conflicto, escoja todas las actividades; de lo contrario, descarte la actividad con la *mayor cantidad de conflictos* y recurra.
15. Diseñe un algoritmo con programación dinámica para calcular ϕ , sin suponer el principio de optimalidad local, y que resuelva el problema de agendamiento de actividades, suponiendo que los tiempos de las actividades en $A[0..N)$ son números naturales.
16. Diseñe un algoritmo iterativo que, en tiempo $O(N)$ y espacio $O(1)$, permita calcular la función ϕ .

17. Suponga ahora que cada actividad n tiene además un *valor* $V[n] > 0$ y que el objetivo es maximizar la suma de los valores de las actividades agendadas, en lugar de su cantidad.
- Demuestre que el criterio de esta sección –escoger la que termine más temprano– deja de satisfacer la propiedad de elección voraz.
 - Refute también el criterio de escoger la actividad de mayor valor.
 - Explique por qué el Teorema 4.2.2 sí sigue siendo cierto para esta variante, y qué dice eso sobre la técnica que conviene usar.

4.3. Una metodología

Con base en las ideas de la Sección 4.1 y en el ejemplo desarrollado en la Sección 4.2, se propone una metodología para diseñar algoritmos voraces. Es deliberadamente análoga a la Metodología 3.3.1. Como aquella, no debe entenderse como una receta infalible, sino como una lista de las preguntas que conviene responder y en qué orden.

Hay, eso sí, una diferencia importante con las metodologías de los otros capítulos y conviene anunciarla de una vez. Allá, si se siguen los pasos con cuidado, se obtiene un algoritmo correcto. Acá no: hay un paso, el de *optimización local*, en el cual el diseño puede sencillamente fracasar, y entonces no hay más remedio que volver atrás y empezar de nuevo con otro criterio, o abandonar la técnica. Esa posibilidad de fracaso no es un defecto de la metodología, sino la naturaleza del asunto: los algoritmos voraces son la excepción y no la regla.

Metodología 4.3.1: Algoritmos voraces

Especificación: especificar el problema, indicando claramente su entrada y su salida, y reconocer en el enunciado los tres elementos de la Definición 4.1.1: cuáles son las soluciones factibles, cuál es la función objetivo y cuál es el sentido. Vale la pena insistir en separar la factibilidad de la optimalidad, pues confundirlas es la fuente más común de especificaciones equivocadas. Si de esta lectura no resulta un problema de optimización, la técnica de este capítulo no aplica.

Candidatos: determinar el conjunto C de candidatos y qué es una solución parcial, en el sentido de la Definición 4.1.2. Un candidato es aquello con lo que el algoritmo se compromete de una sola vez, de modo que este paso fija la granularidad de las decisiones. Casi nunca es único: el mismo problema admite descomposiciones distintas y no todas conducen a un criterio que funcione.

Criterio de selección: proponer *varios* criterios de selección, en el sentido de la Definición 4.1.3, y no solo uno. Cada uno debe ser un preorden total sobre C y debe poder evaluarse sin conocer la respuesta al problema. La heurística usual para inventarlos es preguntarse qué recurso escaso consume cada candidato –tiempo, capacidad, presupuesto– y ordenar los candidatos por cuánto consumen o aportan, o por la razón entre ambas cantidades.

Refutación: antes de intentar demostrar nada, intentar romper cada criterio propuesto con instancias pequeñas. Este paso es económico, rápido y casi siempre exitoso; la Nota 4.1.5 explica por qué conviene hacerlo primero. Un buen procedimiento es implementar el criterio y una solución de fuerza bruta, y compararlas sobre instancias aleatorias diminutas. Tenga presente que este paso nunca concluye afirmativamente: que un criterio sobreviva no demuestra nada, solo indica cuál merece el esfuerzo del paso siguiente.

Optimización local: demostrar que el criterio sobreviviente satisface la propiedad de elección voraz de la Definición 4.1.4. Es el paso difícil, el único que puede fracasar y aquel en el cual reside toda la corrección del algoritmo. La subsección siguiente cataloga los patrones de demostración disponibles.

Optimización global: verificar que el problema tiene subestructura óptima, en el sentido de la Definición 4.1.5, y concluir con el Teorema 4.1.1. La verificación suele ser rutinaria –comprometerse con un candidato deja un problema del mismo tipo sobre menos candidatos– pero no debe omitirse. Si el paso anterior se hizo con el patrón de *el voraz va adelante*, la conclusión global se obtiene directamente de la comparación y no hace falta invocar el teorema.

Función objetivo: proponer una función objetivo ϕ y definirla recurrentemente, tal y como se hace en el Capítulo 3. Este paso no aporta corrección –ya se tiene– pero convierte la estrategia abstracta en algo que se transcribe mecánicamente a código y, de paso, hace explícita la relación entre las dos técnicas.

Implementación: instanciar el Algoritmo 4.1.2 y escoger las estructuras de datos. Aquí decide todo la Nota 4.1.3: si el criterio es estático, basta ordenar C una vez y recorrerlo; si es dinámico, se necesita una estructura de datos auxiliar como una cola de prioridad, por ejemplo. Considere por separado qué estructura hace falta para verificar la factibilidad de un candidato, que puede ser distinta de la que entrega el mínimo o el máximo.

Análisis: establecer las complejidades temporal y espacial con el Teorema 4.1.2, identificando el costo $O(f)$ de verificar la factibilidad y el costo $O(g)$ de escoger el candidato óptimo. En la mayoría de los casos el resultado queda dominado por el ordenamiento inicial o por las operaciones sobre la estructura de datos auxiliar.

Los pasos anteriores se ilustrarán con cada uno de los ejemplos que se presentan en el resto del capítulo. De hecho, la Sección 4.2 ya los siguió todos, aunque sin nombrarlos.

Paso	En la Sección 4.2
Especificación	Agendamientos, su tamaño, maximizar
Candidatos	Las N actividades
Criterio de selección	Cuatro propuestas
Refutación	Dos rotas con tres actividades
Optimización local	Teorema 4.2.1, por intercambio
Optimización global	Teoremas 4.2.2 y 4.2.3
Función objetivo	$\phi(n, t)$ y su definición recurrente
Implementación	Ordenar por tiempo de finalización y recorrer
Análisis	$O(N \log N)$, dominado por el ordenamiento

4.3.1. Tres patrones de demostración. El paso de optimización local es el que concentra la dificultad y no existe un procedimiento que lo resuelva siempre. Lo que sí existe es un repertorio pequeño de formas de argumentar que cubre la gran mayoría de los casos. Esta subsección las describe y dice cuándo conviene ensayar cada una.

Argumento de intercambio. Es el patrón por defecto y el primero que hay que ensayar. Se parte de una solución óptima *arbitraria* Y y se demuestra que se puede transformar en otra, igual de buena, que contiene la elección voraz c . Si $c \in Y$ no hay nada que hacer; de lo contrario, se identifica en Y el elemento b al cual c puede sustituir, se define $Z = (Y \setminus \{b\}) \cup \{c\}$, y se demuestra que Z sigue siendo factible y que su valor objetivo no es peor que el de Y .

Lo que hace funcionar el intercambio es siempre alguna propiedad óptima de c : que termine lo más temprano posible, que pese lo menos posible, que se extienda lo más lejos posible. Al diseñar el argumento, la pregunta a responder es la que plantea la Nota 4.2.3: *¿qué le permite a la elección voraz reemplazar a cualquier otra sin causar daño?* Aparece en la Sección 4.2, en la Sección 4.6 y en la Sección 4.7.

Argumento de inversión. Es la variante del intercambio para problemas cuya solución no es un subconjunto sino un *ordenamiento* de todos los candidatos. Allí no hay nada que escoger —todos los candidatos van en la solución— y lo único

que se decide es en qué orden, de modo que la propiedad de elección voraz hay que reformularla.

La forma del argumento es la siguiente. Se define qué es una *inversión*: una pareja de elementos que aparecen en un orden contrario al que dicta el criterio. Se demuestra que intercambiar los dos elementos de una inversión mejora estrictamente el valor objetivo. Se concluye que una solución óptima no puede tener inversiones y que, por lo tanto, está ordenada según el criterio. El intercambio ocurre aquí entre dos elementos de la misma solución, y no entre una solución y un candidato externo; por lo demás, el espíritu es idéntico. Aparece en la Sección 4.5.

El voraz va adelante. Cuando ninguna de las dos formas anteriores se deja construir, queda comparar directamente lo que el algoritmo produce contra una solución arbitraria, decisión por decisión. Se ordenan ambas soluciones de acuerdo con el orden en que se tomaron las decisiones, se define una medida de progreso tras k decisiones y se demuestra, por inducción sobre k , que la medida del algoritmo voraz nunca es peor que la de la otra solución. De ahí se concluye que la solución voraz es al menos tan buena.

La ventaja de este patrón es que no exige modificar la solución óptima, sino únicamente compararla; por eso funciona en situaciones en las cuales el intercambio no está disponible. La desventaja es que hay que inventar la medida de progreso, que no siempre es evidente, y que la inducción es más laboriosa. Aparece en la Sección 4.2, como demostración alternativa y en la Sección 4.8 donde es la única disponible.

La siguiente tabla resume el repertorio.

Patrón	¿Qué se compara?	La solución es
Intercambio	Una óptima con ella misma modificada	un subconjunto
Inversión	Una óptima con ella misma reordenada	un ordenamiento
Va adelante	La voraz con una óptima, paso a paso	una secuencia

Nota 4.3.1: Cuando ninguno funciona

Si los tres patrones se resisten, hay tres explicaciones posibles y conviene descartarlas en este orden.

La primera, y con mucho la más probable, es que el criterio de selección sea sencillamente incorrecto y que el contraejemplo exista pero no se haya encontrado. Antes de insistir en la demostración, vuelva al paso de refutación y busque con más empeño o con instancias más grandes.

La segunda es que el criterio sea correcto pero requiera un argumento distinto. El repertorio de esta subsección es el usual, no el exhaustivo; en

particular, la teoría de matroides ofrece un cuarto camino, que caracteriza de manera abstracta una clase de problemas en los cuales toda estrategia voraz funciona (ver las notas al final del capítulo).

La tercera es que el problema sencillamente no admita una solución voraz. Esto no es una derrota: si en el paso de optimización global se verificó que hay subestructura óptima, entonces el problema es candidato para las técnicas del Capítulo 3 y el trabajo hecho hasta ahora –los candidatos, las decisiones, la función objetivo– se aprovecha íntegro. Si ni siquiera hay subestructura óptima, queda el Capítulo 5.

Ejercicios

1. Considere el problema de determinar la mínima cantidad de salas necesarias para realizar todas las actividades de un arreglo $A[0..N)$, de manera que dos actividades en conflicto nunca se programen en la misma sala.
 - a) Siga, paso a paso, la metodología propuesta en esta sección, indicando claramente en qué consiste cada uno de los pasos. Debe resultar código en Python ejecutable.
 - b) Para el paso de *criterio de selección*, proponga al menos tres criterios y refute los que pueda.
 - c) Identifique cuál de los tres patrones usó en el paso de *optimización local* y explique por qué los otros dos no se acomodan bien a este problema.
2. Considere el problema de llevar la mayor cantidad posible de objetos en un morral de capacidad W , dado un arreglo $P[0..N)$ con los pesos de los objetos. Note que interesa la *cantidad* de objetos y no su valor.
 - a) Aplique la metodología completa.
 - b) Compare el resultado con el problema del morral del Capítulo 3 y explique, en términos de la Definición 4.1.4, por qué aquel no admite solución voraz y este sí.
3. Para cada uno de los siguientes problemas, indique cuál de los tres patrones de demostración ensayaría primero y justifique brevemente su elección. No es necesario construir las demostraciones.
 - a) Dado un arreglo de números, ordenarlos de manera que la concatenación de sus representaciones decimales sea el número más grande posible.
 - b) Dado un conjunto de puntos sobre una recta, cubrirlos todos con la menor cantidad posible de intervalos de longitud fija L .
 - c) Dado un arreglo de tareas, cada una con una duración y una fecha límite, ordenarlas de manera que se minimice el máximo retraso.

4. La metodología de esta sección y la de la Sección 3.3 comparten varios pasos.
 - a) Empareje los pasos que se corresponden entre las dos.
 - b) Identifique los pasos que existen en una y no en la otra, y explique en cada caso por qué.
5. El paso de *refutación* se describe como económico y casi siempre exitoso. Escriba un programa que, dado un problema de optimización sobre arreglos pequeños de números, un criterio de selección y una solución de fuerza bruta, busque el contraejemplo más pequeño mediante enumeración exhaustiva de instancias. Úselo sobre alguno de los criterios refutados en la Sección 4.2.
6. 🌐 La Nota 4.3.1 afirma que si hay subestructura óptima pero el criterio falla, el problema es candidato para programación dinámica. Formule y demuestre esa afirmación con precisión, indicando qué hipótesis adicionales hace falta suponer sobre el conjunto de candidatos para que el algoritmo de programación dinámica resultante sea eficiente.

4.4. Cubrimiento de intervalos

Suponga que hay un evento de interés público a ser cubierto por una agencia de noticias. El evento cuenta con una hora de inicio y una de finalización. Además, la agencia de noticias conoce la disponibilidad de horarios de periodistas que podrían cubrir el evento, en donde cada periodista compromete una franja de trabajo determinada por una hora de inicio y otra de finalización. Con base en este contexto: ¿cuál es la mínima cantidad de periodistas que debe desplegar la agencia de noticias para cubrir la totalidad del evento?

Esta pregunta se puede formular a modo de problema algorítmico de la siguiente manera.

Problema 4.4.1: Cubrimiento mínimo de intervalos

Entrada: Un arreglo $A[0..N)$, $N \geq 0$, de parejas de números (s_n, e_n) tales que $s_n < e_n$ indican la hora de inicio y finalización que puede cubrir el periodista n , $0 \leq n < N$, y S y E que indican el tiempo de inicio y finalización del evento, $S < E$.

Salida: Mínima cantidad de periodistas con turnos en $A[0..N)$ requeridos para cubrir la totalidad del evento que inicia en S y termina en E .

Al igual que con el agendamiento de actividades (Sección 4.2), como convención para cualquier n , con $0 \leq n < N$, las expresiones $A[n][0]$ y $A[n][1]$ denotan, respectivamente, el tiempo de inicio y finalización del turno que puede asumir el

periodista n : es decir, $s_n = A[n][0]$ y $e_n = A[n][1]$. Note que todos los tiempos pueden incluir valores negativos.

Nota 4.4.1

Por convención, se entenderá que cada turno (s, e) representa el intervalo cerrado-abierto de números reales $[s..e)$. De esta forma, el evento a cubrir corresponde al intervalo $[S..E)$.

Al problema de cubrimiento de intervalos, como se plantea anteriormente, se le pueden asociar algunas suposiciones. Estas harán más fácil su abordaje y permitirán enfocar los esfuerzos en resolver la esencia del problema mismo, sin que estas cambien su naturaleza ni su complejidad intrínsecas (ver Ejercicio 3). Las suposiciones son las siguientes:

1. Los turnos en $A[0..N)$ cubren completamente el evento, es decir, los intervalos asociados a los turnos de los periodistas son un *cubrimiento* del intervalo determinado por el evento:

$$[S..E) \subseteq (\cup n \mid 0 \leq n < N : [s_n..e_n)).$$

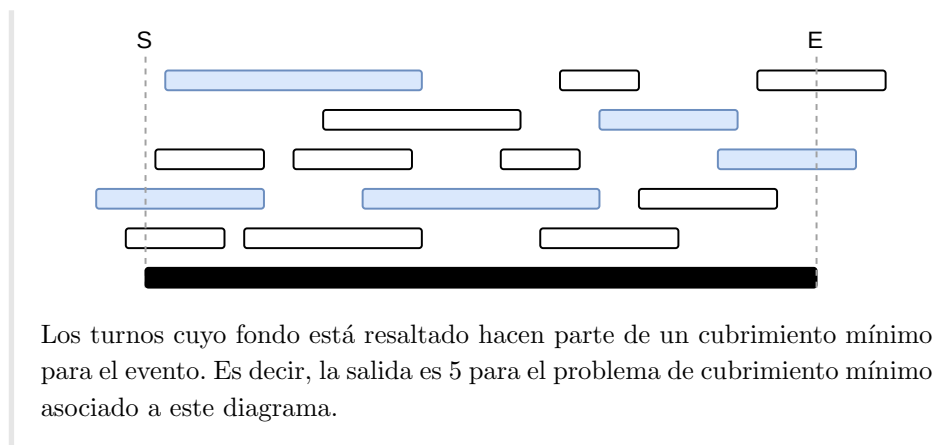
2. Los turnos asociados a los periodistas están relacionados con el evento; es decir, para cualquier $0 \leq n < N$ se cumple:

$$[s_n..e_n) \cap [S..E) \neq \emptyset.$$

La primera condición establece que los turnos de los periodistas son suficientes para cubrir la totalidad del evento. Si esta condición no se cumple, el problema especificado no tiene solución. La segunda condición establece que los turnos de los periodistas están relacionados con el evento. Es decir, que cada uno puede aportar a cubrir alguna porción del evento. Esta condición garantiza que ningún turno a ser considerado puede terminar antes del inicio del evento ni iniciar después de la finalización del evento.

Ejemplo 4.4.1

Se ilustra cada turno como un rectángulo cuya coordenada izquierda en el eje horizontal denota el tiempo de inicio y la derecha el tiempo de finalización; hay 15 turnos en el diagrama. El rectángulo cuyo fondo es negro representa el evento, con tiempos S y E de inicio y finalización, respectivamente.



Leído con el vocabulario de la Sección 4.1, los tres elementos de la Definición 4.1.1 están en el enunciado del Problema 4.4.1: las *soluciones factibles* son los subconjuntos de turnos que cubren $[S..E]$, la *función objetivo* de uno de ellos es su tamaño y el *sentido* es minimizar. El conjunto de candidatos son los N turnos.

Al igual que con el problema de agendamiento de actividades (Sección 4.2), el problema de cubrimiento mínimo se puede resolver exhaustivamente enumerando todos los subconjuntos de turnos y, entre aquellos que cubran $[S..E]$, identificar el tamaño mínimo posible. Es más, este problema también exhibe las propiedades de la subestructura óptima y solapamiento (¿por qué?), y por ello es candidato a ser resuelto con programación dinámica (ver Ejercicio 14).

4.4.1. Abstracción y estrategia voraz. Cualquier estrategia voraz que se plantee para resolver este problema debe considerar al menos dos aspectos. Primero, el de optimización: minimizar la cantidad de turnos. Segundo, garantizar que los turnos seleccionados cubran el evento completamente. Note que el primer aspecto, el de optimización, no tiene mucha gracia sin el segundo aspecto (¿por qué?).

Considere la situación general en la cual un conjunto de intervalos cerrados-abiertos A cubre completamente un intervalo $[s..e]$, con $s < e$. El objetivo es identificar un subconjunto de A , lo más pequeño posible, que cubra el mismo intervalo. Note que alguno de los intervalos en A debe incluir el extremo s (Suposición 1). Cualquier intervalo $[s_i..e_i]$ que incluya a s debe satisfacer $s_i \leq s$ (Suposición 2). Luego, alguno de estos intervalos debe ser parte de una solución óptima.

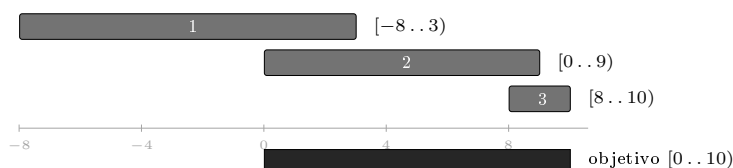
La pregunta es cuál y se responde con ayuda del criterio de selección. Note que apenas se hizo una observación que restringe severamente el conjunto de candidatos elegibles: solo tiene sentido escoger entre los intervalos que incluyen a s . Los tres criterios siguientes respetan esa restricción y difieren en cómo desempatan entre ellos. Escoger, entre los intervalos que incluyen a s , aquel que:

1. comienza más temprano;
2. es más largo; o
3. termina más tarde.

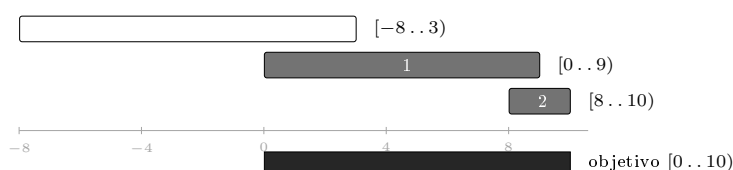
Siguiendo la recomendación de la Nota 4.1.5, conviene intentar romperlos antes de intentar demostrar alguno. Los dos primeros caen con la misma instancia y con solo tres intervalos.

Ejemplo 4.4.2: Dos criterios que no funcionan

(a) y (b): *comienza más temprano o el más largo*, 3 intervalos



(c): *termina más tarde*, 2 intervalos



Se trata de cubrir $[0..10]$ con los intervalos $[-8..3]$, $[0..9]$ y $[8..10]$.

Los dos primeros criterios escogen $[-8..3]$: es el que comienza más temprano entre los que incluyen a 0 y también el más largo de los dos. Hecha esa elección, queda por cubrir $[3..10]$ y hacen falta los otros dos intervalos, para un total de tres.

El tercer criterio escoge $[0..9]$, que es el que termina más tarde. Queda por cubrir $[9..10]$, que se resuelve con $[8..10]$, para un total de dos.

El contraejemplo señala con precisión qué mide cada criterio. La longitud de un intervalo y su extremo inferior son irrelevantes: lo único que importa es hasta dónde alcanza *hacia la derecha*, porque es lo único que reduce el subproblema que queda por resolver. Entre más se extienda el intervalo a elegir, más pequeño será ese subproblema. Es decir, la intuición sugiere que entre todos los intervalos que incluyen a s , es más conveniente seleccionar uno de los que termine más tarde (puede haber más de uno). Esta será la apuesta de una estrategia voraz para resolver el problema de cubrimiento mínimo de intervalos. Así, un algoritmo voraz utilizaría este principio de optimización local incluyendo el extremo menor del intervalo y

cubriendo lo que más se pueda después de este, cada vez resultando un subproblema más pequeño por resolver (i.e., un subintervalo más pequeño por cubrir), hasta agotarlo completamente.

Nota 4.4.2: Un criterio dinámico

Hay una diferencia importante con el criterio de la Sección 4.2 que conviene notar desde ahora. Allá, el criterio –el tiempo de finalización de una actividad– no dependía de nada de lo que el algoritmo hubiera hecho antes. Acá sí: cuál es el mejor intervalo depende de s y s cambia con cada selección. Se trata, en los términos de la Nota 4.1.3, del primer criterio *dinámico* del capítulo.

Note además que la restricción de elegibilidad –incluir a s – cumple aquí el papel que en la Sección 4.2 cumplía la verificación de factibilidad. En este problema, esa verificación sería vacía: por la Suposición 1, cualquier subconjunto de A se puede extender hasta un cubrimiento, de modo que ningún candidato es rechazable por infactible. Todo el trabajo lo hace el criterio.

A continuación se presenta la formalización y la demostración de este principio de optimización local para el problema de cubrimiento mínimo de intervalos.

Teorema 4.4.1: Optimización local

Sea A un conjunto finito de intervalos que cubren $[s..e)$. Si $[s_n..e_n) \in A$ es tal que incluye a s y e_n es máximo entre todos los intervalos en A que incluyen a s , entonces $[s_n..e_n)$ hace parte de un conjunto mínimo de intervalos en A que cubre $[s..e)$.

Demostración

Suponga que $Y \subseteq A$ es un conjunto mínimo de intervalos que cubren $[s..e)$. Como $s < e$, necesariamente $A \neq \emptyset$ y $Y \neq \emptyset$ (¿por qué?). Se procede por casos:

- Si $[s_n..e_n) \in Y$, entonces la conclusión del teorema es correcta.
- Si $[s_n..e_n) \notin Y$, entonces hay un intervalo, por ejemplo $[s_k..e_k) \in Y$, con $k \neq n$, que incluye a s . En efecto, $s \in [s..e)$ porque $s < e$ y Y cubre $[s..e)$. Considere el conjunto $Z = (Y \setminus \{[s_k..e_k)\}) \cup \{[s_n..e_n)\}$. Como $k \neq n$, necesariamente $|Z| = |Y|$, concluyendo que Z cumple con el requerimiento de minimalidad (dado que Y es mínimo). Falta ver que Z también cubre $[s..e)$, para lo cual basta ver que $[s_n..e_n)$ aporta al menos tanto como

$[s_k..e_k)$ dentro de $[s..e)$. Como ambos incluyen a s , se tiene $s_k \leq s$ y $s_n \leq s$; y como e_n es máximo en A con respecto a todos aquellos intervalos que incluyen a s , se tiene $e_k \leq e_n$ en particular. Luego:

$$[s_k..e_k) \cap [s..e) = [s..e_k \downarrow e) \subseteq [s..e_n \downarrow e) = [s_n..e_n) \cap [s..e).$$

Es decir, todo punto de $[s..e)$ que cubría $[s_k..e_k)$ lo cubre también $[s_n..e_n)$ y por lo tanto Z es un cubrimiento de $[s..e)$.

En conclusión, $[s_n..e_n)$ hace parte de un conjunto mínimo de intervalos en A que cubre $[s..e)$.

Nota 4.4.3: Otra vez el intercambio

La demostración del Teorema 4.4.1 es, paso por paso, la misma de la Nota 4.2.3: se parte de una solución óptima arbitraria, se identifica en ella el elemento al cual la elección voraz puede sustituir y se verifica que el reemplazo no hace daño. Lo único que cambia es la propiedad de optimalidad que hace legítimo el reemplazo. Allá era que la actividad escogida no termina después de ninguna otra; acá es que el intervalo escogido no termina antes que ningún otro de los que incluyen a s .

La estrategia planteada en el Teorema 4.4.1 constituye el principio de diseño para obtener una solución voraz al problema de cubrimiento mínimo de intervalos, como lo plantea recurrentemente el Algoritmo 4.4.1.

Algoritmo 4.4.1

Para un conjunto (finito) de intervalos cerrados-abiertos A y un intervalo $[s..e)$ cubierto por A :

1. Si $s \geq e$, entonces no seleccionar nada.
2. De lo contrario, sea $[s_n..e_n)$ un intervalo en A que incluye a s y con e_n máximo entre todos los intervalos en A que incluyen a s :
 - a) seleccionar $[s_n..e_n)$ como parte del cubrimiento mínimo y
 - b) recurrir con los intervalos en A para cubrir el intervalo $[e_n..e)$.

El Algoritmo 4.4.1 considera un caso base y uno inductivo para determinar un cubrimiento mínimo de $[s..e)$ con intervalos en A . El caso base está determinado por la condición $s \geq e$ y corresponde a la situación en la cual el intervalo a ser cubierto es vacío. Por ende, no debe hacerse nada. El caso inductivo, supone que hay algo que cubrir y se usa el Teorema 4.4.1 directamente. Al recurrir, se selecciona

a $[s_n..e_n)$ para cubrir el extremo de $[s..e)$ que incluye a s dado que e_n es máximo entre todos ellos y se recurre para cubrir el subintervalo $[e_n..e)$, con los mismos intervalos disponibles en A , teniendo así en cuenta la contribución hecha por el intervalo $[s_n..e_n)$ al cubrimiento de $[s..e)$. Note que no es necesario hacer casos sobre A (si es vacío o no); basta con hacer casos sobre la relación entre s y e (¿por qué?).

4.4.2. Optimización global. Con la propiedad de elección voraz ya demostrada, falta verificar la subestructura óptima para poder concluir.

Teorema 4.4.2: Subestructura óptima

Sea A un conjunto finito de intervalos que cubren $[s..e)$, con $s < e$, y sea $[s_n..e_n) \in A$ un intervalo que incluye a s y que hace parte de algún cubrimiento mínimo de $[s..e)$. Si $S \subseteq A$ es un cubrimiento mínimo de $[e_n..e)$, entonces $S \cup \{[s_n..e_n)\}$ es un cubrimiento mínimo de $[s..e)$.

Demostración

El conjunto $S \cup \{[s_n..e_n)\}$ cubre $[s..e)$: el intervalo $[s_n..e_n)$ cubre $[s..e_n)$ porque $s_n \leq s < e_n$ y S cubre lo que resta. Falta ver que es mínimo.

Sea Y un cubrimiento mínimo de $[s..e)$ con $[s_n..e_n) \in Y$; existe por suposición. Entonces $Y \setminus \{[s_n..e_n)\}$ cubre $[e_n..e)$ (¿por qué?) y, como S es mínimo, se tiene $|Y| - 1 \leq |S|$. Luego:

$$|S \cup \{[s_n..e_n)\}| \leq |S| + 1 \leq |Y|.$$

Como Y es mínimo y $S \cup \{[s_n..e_n)\}$ es un cubrimiento de $[s..e)$, las dos desigualdades son igualdades y $S \cup \{[s_n..e_n)\}$ es mínimo.

Teorema 4.4.3: Optimización global

Sea A un conjunto (finito) de intervalos cerrado-abiertos que cubren el intervalo $[s..e)$. El Algoritmo 4.4.1 identifica un cubrimiento (mínimo) para $[s..e)$ con intervalos en A .

Demostración

El criterio de selección satisface la propiedad de elección voraz por el Teorema 4.4.1 y el problema tiene subestructura óptima por el Teorema 4.4.2.

El conjunto A cubre $[s..e)$ por suposición, de modo que hay al menos una solución factible. Por el Teorema 4.1.1, el Algoritmo 4.4.1 identifica un cubrimiento mínimo.

Nota 4.4.4: La misma demostración dos veces

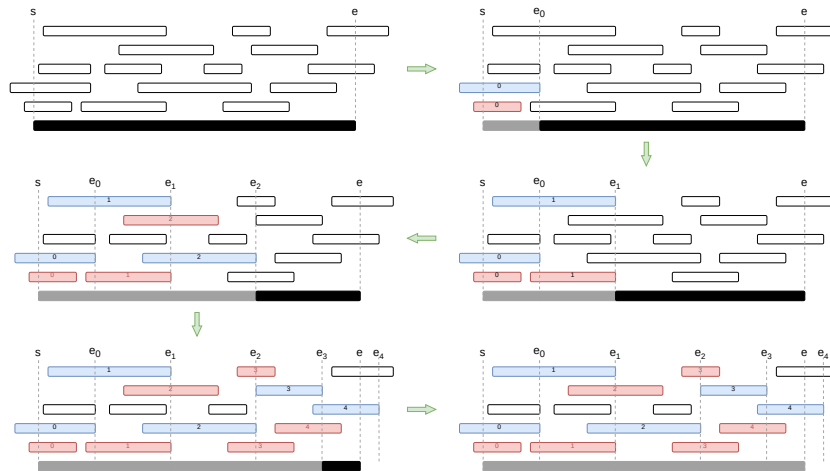
La demostración del Teorema 4.4.3 y la del Teorema 4.2.3 son, salvo por los nombres, idénticas: se invoca el teorema local, se invoca la subestructura óptima y se concluye con el Teorema 4.1.1. Eso no es una coincidencia ni una economía de la exposición, sino exactamente lo que la Nota 4.1.4 anunció: una vez demostrado el teorema de optimización local, el paso global ya está hecho de una vez y para siempre.

Vale la pena notar un detalle de la correspondencia con el Algoritmo 4.1.2. Allí, el candidato escogido se remueve de C en cada iteración, mientras que el Algoritmo 4.4.1 recurre con todos los intervalos de A . La diferencia es inofensiva: como $[s_n..e_n)$ no incluye a e_n , ese intervalo no vuelve a ser elegible en ninguna llamada posterior, y removerlo o no da lo mismo (ver Ejercicio 5).

El Ejercicio 4 propone una tercera demostración, esta vez con el patrón de *el voraz va adelante* de la Sección 4.2, que no necesita ninguno de los dos teoremas anteriores.

Ejemplo 4.4.3

El siguiente diagrama muestra, paso a paso, cómo el Algoritmo 4.4.1 selecciona un cubrimiento mínimo para el intervalo $[s..e)$, como en el Ejemplo 4.4.1.



En el primer paso, correspondiente al caso recurrente, se identifican dos intervalos que incluyen a s (con etiqueta 0); se escoge el intervalo que termina más tarde (color azul) y se descarta el otro (color rojo). Al recurrir, el intervalo a ser cubierto corresponde a $[e_0..e)$, lo cual se indica en la barra inferior con color negro (el color gris representa la parte del intervalo que ha sido cubierta). En la medida en que avanza el algoritmo, los intervalos seleccionados van siendo etiquetados con el paso correspondiente, y coloreados dependiendo de si son seleccionados o no. Además, el intervalo a cubrir se va reduciendo hasta ser cubierto completamente. Esta situación se observa en el último paso, que se resuelve con el caso base del algoritmo, dado que $e_4 > e$. Observe que hay intervalos que nunca son considerados (los que no tienen ningún color de relleno).

4.4.3. Una implementación eficiente. Una implementación directa del Algoritmo 4.4.1 resultaría en tiempo cuadrático en el tamaño de A (i.e., $O(N^2)$). Observe que para cada extremo inferior de s , se debe determinar la colección de intervalos en A que contienen a s y, entre ellos, seleccionar uno de los que tenga mayor extremo superior, lo cual tomaría tiempo $O(N)$. Como hay, en el peor de los casos, N extremos inferiores, entonces se deberían hacer $O(N^2)$ operaciones básicas en total.

Para una implementación eficiente se propone ordenar la colección de intervalos en $A[0..N)$ ascendentemente por los extremos inferiores, desempataando arbitrariamente cuando haya empates. Suponiendo, tal y como lo propone la estrategia voraz, ir cubriendo el intervalo objetivo $[S..E)$ del extremo inferior al superior, la situación general que se tiene es la siguiente: cubrir $[s..E)$ (con $s \geq S$) con intervalos en $A[n..N)$ (con $0 \leq n \leq N$) sabiendo que el mejor intervalo en $A[0..n)$ que incluye a s termina en a (con $a \geq s$). Conviene subrayar un aspecto del recorrido que será decisivo más adelante: ese mejor intervalo está identificado, pero todavía no está contado. El recorrido no puede saber que es el mejor mientras siga apareciendo algún intervalo que incluya a s , de modo que su cuenta queda pendiente y solo se cobra cuando aparece el primer intervalo que ya no incluye a s .

Si $s \geq E$, entonces no queda nada por cubrir y no hay nada más que hacer. Si algo queda por cubrir pero $n = N$, entonces se agotaron los intervalos disponibles y el único que puede aportar es justamente el que quedó pendiente, el que termina en a : hay que contarlo. De lo contrario, el intervalo $A[n] = (s_n, e_n)$ debe ser considerado y se dan, principalmente, dos opciones: $s \in [s_n..e_n)$ o $s \notin [s_n..e_n)$. Con la primera opción, necesariamente el intervalo $A[n]$ se debe considerar cuando $e_n > a$; de lo contrario no aportaría nada mejor y se podría descartar. Con la segunda opción, o bien el intervalo $A[n]$ termina antes de s (i.e., $e_n \leq s$) o inicia después de s (i.e., $s_n > s$). Como A está ordenado ascendentemente por tiempo de inicio, en la primera

situación se puede ignorar el intervalo y avanzar. En la segunda situación, por la misma razón, se concluye que el mejor cubrimiento para s termina en a , momento en el cual se cobra la cuenta pendiente, y $A[n..N)$ debe ser usado para cubrir $[a..E)$. Con este diseño, la mayoría de los intervalos se visitará una vez y como máximo dos veces. Es decir, la complejidad es lineal en N , pagando el “precio” de ordenarlo previamente (lo cual se puede hacer en $O(N \log N)$).

Nota 4.4.5: Dinámico sin cola de prioridad

La Nota 4.1.3 advierte que un criterio dinámico obliga, en general, a mantener una estructura que entregue el mejor candidato y admita modificaciones. El diseño anterior muestra que no siempre hace falta: acá basta un ordenamiento hecho una sola vez al comienzo y un recorrido de izquierda a derecha.

Lo que lo permite es una propiedad particular de este problema. El parámetro del que depende el criterio –el extremo s por cubrir– solo puede crecer, y los intervalos están ordenados por su extremo inferior. En consecuencia, los candidatos elegibles se recorren en el mismo orden en que van apareciendo y basta con acumular en a el mejor visto hasta el momento. Cuando esa monotonía no está disponible, no queda más remedio que usar una cola de prioridad, como ocurrirá en la Sección 4.7 y en la Sección 4.8.

La siguiente función objetivo, para $0 \leq n \leq N$, y $s, a \in \mathbb{R}$ con $s \geq S$ y $a \geq S$, plasma matemáticamente la idea elaborada en el párrafo anterior.

$\phi(n, s, a)$: “mínima cantidad de intervalos necesarios para cubrir $[s..E)$ disponiendo del intervalo $[s..a)$ y de los intervalos en $A[n..N)$.”

El intervalo $[s..a)$ que aparece en la especificación no es un elemento de $A[0..N)$ sino el resumen de lo que ya se recorrió: representa el mejor cubrimiento de s que ofrecen los intervalos en $A[0..n)$ y es la cuenta que quedó pendiente. Cuando $a = s$ ese intervalo es vacío y no aporta nada, que es precisamente la situación del llamado inicial. Así, para resolver el problema de cubrimiento mínimo de intervalos, el objetivo es calcular $\phi(0, S, S)$ bajo las condiciones establecidas en el enunciado del problema y las dos suposiciones hechas previamente: con $a = s = S$ no hay nada recorrido ni nada pendiente y todos los intervalos están disponibles.

La definición recurrente de ϕ se establece tomando a n como pivote, y seleccionando o descartando intervalos en $A[n..N)$ dependiendo de la relación de $A[n]$ con el intervalo $[s..E)$ a ser cubierto. Formalmente, para $0 \leq n \leq N$, y $s, a \in \mathbb{R}$ con

$s \geq S$ y $a \geq S$, se propone la siguiente definición recurrente de la función objetivo:

$$\phi(n, s, a) = \begin{cases} 0 & , \text{ si } s \geq E, \\ 1 & , \text{ si } s < E \wedge n = N, \\ \phi(n+1, s, a \uparrow A[n][1]) & , \text{ si } s < E \wedge n \neq N \wedge A[n][0] \leq s, \\ 1 + \phi(n+1, a, a \uparrow A[n][1]) & , \text{ si } s < E \wedge n \neq N \wedge A[n][0] > s. \end{cases}$$

Hay dos casos base y el orden en que se consideran importa. El primero corresponde a la situación en la cual el intervalo por cubrir es vacío: no hace falta nada más y la respuesta es 0. El segundo corresponde a la situación en la cual todavía queda algo por cubrir pero ya no hay intervalos por considerar: el único que puede aportar es el que quedó pendiente, el que se extiende hasta a y por eso la respuesta es 1. Que ese intervalo baste para terminar el trabajo –es decir, que necesariamente $a \geq E$ en esa situación– no es un supuesto adicional sino una consecuencia de la Suposición 1 y del orden del arreglo (ver Ejercicio 11). El orden importa porque el primer caso base debe cortar antes que el segundo: si el intervalo por cubrir ya es vacío, puede quedar un pendiente que no hace falta contar y contarlo daría una respuesta mayor que la mínima (ver Ejercicio 12). El caso inductivo consta de dos subcasos. El primero de ellos, especifica la situación en la cual el intervalo $A[n]$ puede contribuir al cubrimiento de s : si mejora lo que se ha encontrado anteriormente, entonces se recurre avanzando en los intervalos y actualizando el valor del mejor cubrimiento hasta ahora encontrado para s ; de lo contrario, el intervalo $A[n]$ se ignora y se avanza. El segundo de ellos, especifica la situación en la cual el intervalo $A[n]$ (y los intervalos de allí en adelante) no puede contribuir al cubrimiento de s : entonces, se ha encontrado el mejor intervalo que cubre a s , se cobra la cuenta pendiente con el término 1 y se avanza tratando de encontrar el mejor cubrimiento para $[a..E)$ con intervalos en $A[n+1..N)$ con un cubrimiento de a que se extiende hasta $a \uparrow A[n][1]$ (porque $A[n]$ puede ser parte de los intervalos que cubren a a).

Teorema 4.4.4

Suponga que los intervalos en $A[0..N)$ están ordenados ascendentemente por el extremo inferior (i.e., $A[i][0] \leq A[j][0]$, para $0 \leq i < j < N$). Entonces:

1. Para $0 \leq n \leq N$ y $s, a \in \mathbb{R}$ con $a \geq s$, el valor de $\phi(n, s, a)$ es la mínima cantidad de intervalos necesarios para cubrir $[s..E)$ disponiendo del intervalo $[s..a)$ y de los intervalos en $A[n..N)$.
2. El valor de $\phi(0, S, S)$ es la mínima cantidad de intervalos en $A[0..N)$ que cubren $[S..E)$.

Demostración

La segunda afirmación es una instancia de la primera: con $a = s = S$ el intervalo $[S..S)$ es vacío, de modo que los intervalos disponibles son exactamente los de $A[0..N)$.

La primera afirmación se demuestra por inducción sobre $N - n$, siguiendo los cuatro casos de la definición de ϕ y usando en dos de ellos el invariante del Ejercicio 11.

Si $s \geq E$, entonces $[s..E)$ es vacío, el conjunto vacío lo cubre y 0 es la respuesta, cualquiera que sea n . Si $s < E$ y $n = N$, entonces por la primera parte del invariante todo intervalo de $A[0..N)$ empieza en s o antes, de modo que el intervalo $A[j]$ cubre un punto $x \geq s$ si y solo si $A[j][1] > x$. Por la segunda parte del invariante, la porción de $[s..E)$ que cubren todos los intervalos entre sí es entonces $[s..a \downarrow E)$, y por la Suposición 1 esa porción es todo $[s..E)$; luego $a \geq E$. Así, el intervalo pendiente $[s..a)$ cubre $[s..E)$ por sí solo y 1 es la respuesta.

Suponga ahora $s < E$ y $n \neq N$, y suponga la afirmación para $N - (n + 1)$. Si $A[n][0] \leq s$, entonces $A[n]$ también incluye a s . Usar a la vez $[s..a)$ y $A[n]$ nunca es mejor que usar solo el que llega más lejos: dentro de $[s..E)$ la unión de los dos es $[s..a \uparrow A[n][1])$, que es exactamente lo que cubre por sí solo el que llega más lejos, y ninguno de los dos aporta nada a la izquierda de s , que no hay que cubrir. Luego reemplazar los dos por el pendiente $[s..a \uparrow A[n][1])$ no cambia la respuesta y la hipótesis de inducción da el resultado. Si $A[n][0] > s$, entonces $A[j][0] \geq A[n][0] > s$ para todo j con $n \leq j < N$, porque el arreglo está ordenado; es decir, ningún intervalo disponible distinto de $[s..a)$ incluye a s . Como $s \in [s..E)$, todo cubrimiento de $[s..E)$ debe contener a $[s..a)$, que cuesta 1 y aporta $[s..a \downarrow E)$; lo que resta por cubrir es $[a..E)$ y para ello se dispone de $A[n..N)$. Ahora, si $a \geq E$, entonces no resta nada por cubrir, el llamado recurrente cae en el primer caso base y responde 0; el valor de su tercer argumento es irrelevante. Y si $a < E$, entonces necesariamente $A[n][0] \leq a$: de lo contrario ningún intervalo de $A[n..N)$ cubriría el punto a , porque todos empiezan en $A[n][0] > a$ o después, y tampoco lo cubriría ninguno de $A[0..n)$, porque todos terminan en a o antes por la segunda parte del invariante; esto contradiría la Suposición 1, dado que $a \in [S..E)$. Luego $A[n]$ incluye a a y queda absorbido en el nuevo pendiente $[a..a \uparrow A[n][1])$, por el mismo argumento del caso anterior. La hipótesis de inducción da entonces el resultado.

Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación. A continuación se presenta el Algoritmo 4.4.2, como la implementación de la función `mic` que permite calcular ϕ .

Algoritmo 4.4.2

```

1 def mic(A, N, E, n, s, a):
2     ans = None
3     if s>=E: ans = 0
4     elif n==N: ans = 1
5     else:
6         if A[n][0]<=s: ans = mic(A, N, E, n+1, s, max(a, A[n][1]))
7         else: ans = 1 + mic(A, N, E, n+1, a, max(a, A[n][1]))
8     return ans

```

A continuación se presenta un ejemplo con un llamado a la función `mic` para un conjunto de intervalos que, a escala, corresponden al Ejemplo 4.4.1.

Ejemplo 4.4.4

El objetivo es cubrir el intervalo $[0..34]$ con el siguiente arreglo de parejas que especifica los intervalos del Ejemplo 4.4.1:

```

1 A = [ (1, 14), (21, 25), (31, 37),
2       (9, 19), (23, 30),
3       (1, 6), (7, 13), (18, 22), (29, 36),
4       (-3, 6), (11, 23), (25, 32),
5       (-1, 4), (5, 14), (20, 27) ]

```

Después de ordenar el arreglo ascendentemente por la primera componente de las parejas, se obtiene:

```

1 >>> A.sort()
2 >>> A
3 [ (-3, 6), (-1, 4), (1, 6), (1, 14), (5, 14), (7, 13), (9, 19),
4   (11, 23), (18, 22), (20, 27), (21, 25), (23, 30), (25, 32),
5   (29, 36), (31, 37) ]

```

Con base en la versión ordenada del arreglo, se tiene:

```

1 >>> mic(A, len(A), 34, 0, 0, 0)
2 5

```

El Teorema 4.4.5 presenta los resultados de correctitud de la función `mic`, junto con sus complejidades temporal y espacial.

Teorema 4.4.5

Si los intervalos en $A[0..N]$ están ordenados ascendentemente por el extremo inferior, entonces el llamado `mic(A, N, E, 0, S, S)` calcula $\phi(0, S, S)$, y toma tiempo y espacio $O(N)$ y $O(1)$, respectivamente. Si es necesario ordenar el arreglo, entonces la complejidad temporal es $O(N \log N)$.

Demostración

Note que la función `mic` es una transcripción de la función ϕ , agregando los parámetros A , N y E que nunca son modificados. El tiempo de ejecución de `mic` es linealmente proporcional a la cantidad de elementos en $A[0..N]$, i.e., es $O(N)$. El espacio es $O(1)$ sin tener en cuenta el espacio de la pila de la recurrencia. Finalmente, si es necesario ordenar el arreglo, se puede usar un algoritmo como *MergeSort* (Sección 2.5) con tiempo de ejecución $O(N \log N)$.

Ejercicios

1. Encuentre otro cubrimiento mínimo en el Ejemplo 4.4.1.
2. Ilustre, paso a paso, la ejecución de la función `mic` en el Ejemplo 4.4.4.
3. Para abordar el diseño de una solución algorítmica del problema de cubrimiento de intervalos se hicieron dos suposiciones, indicando que estas no cambiarían la naturaleza misma del problema. Una de ellas, la segunda, indica que se puede suponer que en $A[0..N]$ solo hay intervalos que pueden contribuir a cubrir $[S..E]$. Diseñe un algoritmo que en tiempo $O(N)$ preprocese a $A[0..N]$ e identifique cuáles de sus intervalos se pueden descartar por no cumplir con esta suposición. ¿Afectaría esta solución la complejidad definitiva del algoritmo diseñado para resolver el problema de cubrimiento mínimo de intervalos?
4. Desarrolle una demostración alternativa del Teorema 4.4.3 siguiendo el patrón de *el voraz va adelante*, tal y como se hace en el Teorema 4.2.4. *Ayuda:* sean $I_0, \dots, I_{n-1}$ los intervalos que selecciona el Algoritmo 4.4.1 y $J_0, \dots, J_{m-1}$ los de un cubrimiento cualquiera de $[s..e]$, ambos ordenados ascendentemente por extremo superior. Demuestre por inducción que el extremo superior de I_k no es menor que el de J_k .
5. La Nota 4.4.4 afirma que da lo mismo recurrir con A o con $A \setminus \{[s_n..e_n]\}$ en el paso inductivo del Algoritmo 4.4.1. Demuestre esa afirmación y explique qué pasaría si los intervalos fueran cerrados en ambos extremos.

6. Considere el criterio de escoger, entre los intervalos que incluyen a s , aquel que cubra la mayor porción todavía no cubierta del intervalo objetivo.
 - a) Demuestre que este criterio coincide con el de la Sección 4.4 cuando el algoritmo avanza de izquierda a derecha.
 - b) ¿Sigue coincidiendo si se permite escoger intervalos que no incluyan a s ? Justifique con un ejemplo.
7. Suponga que se obtiene la función $\hat{\phi}$ a partir de ϕ modificando el segundo caso inductivo de $1 + \phi(n+1, a, a \uparrow A[n][1])$ a $1 + \phi(n, a, a)$, manteniendo la misma condición. Explique cuál es el sentido de esta reformulación de la recurrencia, y explique si el algoritmo asociado serviría o no para resolver el problema del cubrimiento de intervalos.
8. Diseñe una versión iterativa de la función `mic`.
9. Modifique la función `mic` para que retorne una colección mínima de intervalos para cubrir $[S..E)$.
10. Modifique la función `mic` para que, además de calcular la cantidad mínima de intervalos, identifique cuándo dicha colección no existe, es decir, cuando no se cumple la suposición de que $A[0..N)$ necesariamente cubre $[S..E)$.
11. 🌈 Demuestre que todo llamado $\phi(n, s, a)$ alcanzable desde $\phi(0, S, S)$ satisface el siguiente invariante:
 - a) $A[j][0] \leq s$, para todo j con $0 \leq j < n$; y
 - b) $a = s \uparrow (\uparrow j \mid 0 \leq j < n : A[j][1])$.

Concluya, con la Suposición 1, que en el segundo caso base de ϕ —es decir, cuando $s < E$ y $n = N$ — necesariamente $a \geq E$, de modo que el intervalo pendiente alcanza a cubrir todo lo que falta y responder 1 es correcto.
12. Considere la versión de ϕ que tiene un único caso base, $n = N$, con valor 0, y que conserva los dos casos inductivos:
 - a) Exhiba una instancia con un solo intervalo que satisfaga las dos suposiciones de la sección y para la cual esa versión responda mal. ¿Cuál es el intervalo que se deja de contar?
 - b) Demuestre que esa versión nunca responde más que la cantidad mínima, y que cuando se equivoca lo hace exactamente en uno. *Ayuda:* caracterice el error en términos del valor de s en el momento en que la recurrencia alcanza $n = N$.
 - c) Considere ahora la versión con los dos casos base pero en el orden contrario: primero $n = N$, con valor 1 si $a \neq s$ y 0 en caso contrario, y después $s \geq E$. Exhiba una instancia con dos intervalos para la cual esta versión responda mal, y explique qué papel cumple la Suposición 2 en su construcción.
13. La estrategia voraz planteada en esta sección aborda el problema con un algoritmo que incrementalmente cubre el intervalo $[S..E)$ desde su extremo inferior. Diseñe un algoritmo cuya estrategia sea abordar el problema incrementalmente

cubriendo el intervalo desde su extremo superior. Especialmente, plantee el teorema de optimización local y una función recurrente que resuelva el problema de acuerdo con la estrategia planteada.

14. Diseñe un algoritmo usando la técnica de programación dinámica que resuelva el problema del cubrimiento mínimo de intervalos. Calcule sus complejidades temporal y espacial, y compárelas con las complejidades del algoritmo voraz diseñado en esta sección.

4.5. Tiempo en el sistema

Es común ver personas esperando a ser atendidas en una sucursal bancaria, en una fila, con un turno para realizar una consignación, un retiro o un reclamo, o para abrir/cerrar una cuenta. Los bancos saben cuánto tiempo toma, aproximadamente, cada servicio, en donde este tiempo puede ser distinto entre diferentes tipos de servicio. Un objetivo deseable es organizar los turnos de los clientes para minimizar el tiempo total que tardan todos en esperar y recibir el servicio. Este problema se conoce como el *tiempo en el sistema* y su propósito es identificar una organización óptima de turnos de espera. Tiene una gran cantidad de aplicaciones. Por ejemplo, para acceder a un archivo compartido en un servidor o en la nube, o para agendar procesos en un procesador.

Para plantear el problema, dado su carácter general, a continuación se abstrae el concepto de persona como agente.

Problema 4.5.1: Tiempo en el sistema

Entrada: Un arreglo $A[0..N)$, $N \geq 0$, de números positivos en donde $A[n]$ indica el tiempo que toma servir al agente n , con $0 \leq n < N$.

Salida: Mínimo tiempo alcanzable en el sistema al servir a los agentes con tiempos de servicio en $A[0..N)$.

Observe que si se minimiza el tiempo en que tardan todos los agentes en ser atendidos, directamente se está minimizando el tiempo promedio de atención porque la cantidad de agentes se supone fija.

Una forma de resolver el problema es por medio de una exploración exhaustiva de todas las posibilidades y quedarse con uno de los ordenamientos que tenga mínimo tiempo en el sistema, como se observa en el Ejemplo 4.5.1.

Ejemplo 4.5.1

Considere que hay tres agentes con tiempos $A = [4, 8, 3]$ de tal forma que el agente n es servido en tiempo $A[n]$, con $0 \leq n < 3$. Los turnos se pueden ordenar de 6 formas distintas, con los siguientes tiempos (totales) en el sistema:

$$0, 1, 2 : 4 + (4 + 8) + (4 + 8 + 3) = 31$$

$$0, 2, 1 : 4 + (4 + 3) + (4 + 3 + 8) = 26$$

$$1, 0, 2 : 8 + (8 + 4) + (8 + 4 + 3) = 35$$

$$1, 2, 0 : 8 + (8 + 3) + (8 + 3 + 4) = 34$$

$$2, 0, 1 : 3 + (3 + 4) + (3 + 4 + 8) = 25$$

$$2, 1, 0 : 3 + (3 + 8) + (3 + 8 + 4) = 29.$$

Observe que el tiempo total en el sistema se calcula tomando el tiempo que cada agente toma en ser atendido y el tiempo que espera antes de ser atendido. En el primer ordenamiento, el agente 0 es servido de primero (4 unidades de tiempo), el agente 1 es servido de segundo (4 unidades de tiempo esperando y 8 siendo servido) y el agente 2 es servido de último (4+8 unidades de tiempo esperando y 3 siendo servido). Entre todas las posibilidades, la mejor corresponde al quinto ordenamiento que resulta en 25 unidades de tiempo en el sistema: primero el agente 2, luego el agente 0 y finalmente el agente 1.

El tamaño del espacio de búsqueda de un algoritmo que exhaustivamente genere todos los ordenamientos de N agentes está acotado por las $N!$ permutaciones de $A[0..N]$; esto, sin calcular aún el tiempo en el sistema para cada ordenamiento. Suponiendo que el cálculo del tiempo en el sistema se puede hacer en tiempo $O(N)$ para cada ordenamiento, se tiene que un algoritmo exhaustivo (i.e., de fuerza bruta) toma tiempo $O(N \cdot N!) = O((N+1)!)$ en resolver este problema. La generación sistemática de las permutaciones de un arreglo, que es justamente lo que tal algoritmo necesitaría, se estudia en la Sección 5.2 del Capítulo 5.

Para un problema cuya solución es un ordenamiento, un criterio de selección es sencillamente una manera de ordenar los candidatos, y aquí solo hay dos que resulten naturales: atender primero a los agentes de menor tiempo de servicio, o atender primero a los de mayor tiempo de servicio. El Ejemplo 4.5.1 refuta el segundo sin necesidad de buscar más lejos. El ordenamiento 1, 2, 0, que atiende de primero al agente más lento, produce 34 unidades de tiempo en el sistema, contra las 25 del ordenamiento 2, 0, 1. La razón es clara: el tiempo de servicio del agente que se atiende de primero lo esperan *todos* los demás, de modo que conviene que sea lo más pequeño posible.

El Ejemplo 4.5.1 sugiere entonces que un mejor ordenamiento para la atención de agentes es uno en el cual se atiendan primero aquellos agentes cuyo tiempo de servicio sea menor. Uno que primero sirva al agente con menor tiempo de servicio, luego al agente con segundo menor tiempo de servicio y así hasta que, finalmente, sirva al agente con mayor tiempo de servicio. Es decir, intuitivamente, un ordenamiento es óptimo cuando los agentes que toman menos tiempo se atienden primero. Dicho de otra forma, uno en el cual los tiempos están ordenados ascendentemente.

Antes de continuar, se define una notación que permite referirse directamente al tiempo del sistema asociado a un arreglo de números.

Nota 4.5.1

Por convención, para cualquier arreglo de números $A[0..N)$, $N \geq 0$, la expresión $\sigma(A)$ se define como:

$$\begin{aligned}\sigma(A) &= N \cdot A[0] + (N-1) \cdot A[1] + \cdots + 1 \cdot A[N-1] \\ &= (+n \mid 0 \leq n < N : (N-n) \cdot A[n]).\end{aligned}$$

Note que $\sigma(A)$ representa el tiempo en el sistema asociado a los tiempos de acuerdo con el ordenamiento en $A[0..N)$.

4.5.1. Ordenar en lugar de escoger. Una motivación para abordar este problema es que el esquema que se plantea como estrategia voraz para su solución es distinto al que comparten los problemas de agendamiento de actividades (Sección 4.2) y de cubrimiento de intervalos (Sección 4.4). Conviene precisar en qué consiste esa diferencia antes de continuar.

Leído con el vocabulario de la Sección 4.1, las *soluciones factibles* de este problema son todos los ordenamientos posibles de los N tiempos de servicio, la *función objetivo* de uno de ellos es su tiempo en el sistema —es decir, σ — y el *sentido* es minimizar. Hasta ahí, nada nuevo.

La diferencia está en el conjunto de candidatos. En los dos problemas anteriores, el algoritmo tenía que decidir cuáles candidatos entraban en la solución y cuáles no, y por eso la Definición 4.1.2 describe una solución factible como un subconjunto del conjunto de candidatos. Acá no hay nada que escoger: todos los agentes van a ser atendidos, todos los tiempos de servicio aparecen en toda solución factible y, en consecuencia, la verificación de factibilidad del Algoritmo 4.1.2 resulta vacía, pues cualquier ordenamiento es factible. Lo único que se decide es en qué orden.

Esto obliga a reformular la propiedad de elección voraz de la Definición 4.1.4, que habla de un candidato que hace parte de alguna solución óptima y que aquí no dice nada, dado que todos los candidatos hacen parte de todas las soluciones. La reformulación adecuada, tal y como se anticipó en la Sección 4.3, se hace en

términos de *inversiones* y da lugar al segundo de los tres patrones de demostración del capítulo.

4.5.2. Optimización local: un argumento de inversión. Para continuar, es necesario recordar la noción de inversión de un arreglo (Ejercicio 7, Sección 2.5). Una *inversión* en un arreglo $A[0..N)$, $N \geq 0$, es una pareja de índices $0 \leq i < j < N$ tales que $A[i] > A[j]$. Por ejemplo, en el arreglo $[4, 8, 3]$ del Ejemplo 4.5.1 hay dos inversiones: $(0, 2)$ y $(1, 2)$.

Teorema 4.5.1: Optimización local

Sea $A[0..N)$, $N \geq 0$, un arreglo de números. Si en $A[0..N)$ hay una inversión con testigos $0 \leq i < j < N$, entonces el arreglo que resulta de intercambiar los valores en i y j tiene un tiempo de sistema estrictamente menor que el de $A[0..N)$.

Demostración

Suponga que $A[0..N)$ tiene una inversión con testigos $0 \leq i < j < N$. También suponga que $B[0..N)$ es una copia de $A[0..N)$ que resulta de intercambiar los valores en los índices i y j . Formalmente, $B[i] = A[j]$, $B[j] = A[i]$ y $B[k] = A[k]$ para cualquier $0 \leq k < N$ con $k \neq i$ y $k \neq j$. El objetivo es demostrar $\sigma(A) > \sigma(B)$. Basta con demostrar $\sigma(A) - \sigma(B) > 0$. Por la Nota 4.5.1, tanto $\sigma(A)$ como $\sigma(B)$ son sumas de los términos $(N - k) \cdot A[k]$ y $(N - k) \cdot B[k]$, respectivamente, para $0 \leq k < N$. Como $A[k] = B[k]$ para todo k distinto de i y de j , todos esos términos se cancelan en la resta y solo sobreviven los correspondientes a i y a j :

$$\begin{aligned} \sigma(A) - \sigma(B) &= (N - i) A[i] + (N - j) A[j] \\ &\quad - (N - i) B[i] - (N - j) B[j] \\ &= (N - i) A[i] + (N - j) A[j] \\ &\quad - (N - i) A[j] - (N - j) A[i] \\ &= [(N - i) - (N - j)] (A[i] - A[j]) \\ &= (j - i) (A[i] - A[j]) \\ &> 0. \end{aligned}$$

El último paso es válido porque $j > i$ y, por suposición, (i, j) es una inversión (i.e., $A[i] > A[j]$). En conclusión, $\sigma(B) < \sigma(A)$.

Nota 4.5.2: El argumento de inversión

La demostración del Teorema 4.5.1 es el ejemplar del segundo patrón del capítulo. Comparada con el argumento de intercambio de la Nota 4.2.3, la diferencia es dónde ocurre el intercambio: allá se cambiaba un elemento de una solución óptima por un candidato que estaba *fuera* de ella; acá se intercambian dos elementos que ya están *dentro* de la misma solución.

La consecuencia es que la conclusión también cambia de forma. Un argumento de intercambio concluye que existe una solución óptima que contiene la elección voraz; un argumento de inversión concluye que ninguna solución óptima puede tener inversiones y que, por lo tanto, toda solución óptima está ordenada según el criterio. Esa es la reformulación de la propiedad de elección voraz que exige un problema cuya solución es un ordenamiento.

4.5.3. Optimización global. La pregunta de qué ordenamiento es el mejor posible se puede responder, gracias al Teorema 4.5.1, de forma directa: aquel en donde no haya inversiones. Esto es porque si existe una inversión en un ordenamiento, entonces el tiempo en el sistema asociado disminuye necesariamente al intercambiar los elementos en dicha inversión. Como la cantidad de elementos en $A[0..N)$ es finita, necesariamente el proceso de disminuir la cantidad de inversiones allí presentes acabará eventualmente con un ordenamiento ascendente (más precisamente, no decreciente) de sus valores; note que intercambiar los valores de una inversión reduce estrictamente la cantidad de inversiones del arreglo (¿por qué?).

Teorema 4.5.2: Optimización global

Un ordenamiento de los tiempos de servicio minimiza el tiempo en el sistema si, y solo si, es no decreciente.

Demostración

Sea $A[0..N)$, $N \geq 0$, un arreglo de números que registra un ordenamiento de los tiempos de servicio. Se demuestran las dos implicaciones.

- Suponga que $A[0..N)$ minimiza el tiempo en el sistema y, hacia una contradicción, que no está ordenado no decrecientemente. Entonces, hay una inversión (i, j) con $0 \leq i < j < N$ y, por el Teorema 4.5.1, el arreglo que resulta de intercambiar los valores en i y j tiene un tiempo en el sistema

estrictamente menor. Esto contradice la minimalidad de $A[0..N]$. Luego, $A[0..N]$ está ordenado no decrecientemente.

- Recíprocamente, suponga que $A[0..N]$ está ordenado no decrecientemente. Dos ordenamientos no decrecientes de los mismos tiempos de servicio coinciden posición a posición (¿por qué?) y, por ende, tienen el mismo tiempo en el sistema. Por la implicación anterior, algún ordenamiento que minimiza el tiempo en el sistema es no decreciente; como todos los ordenamientos no decrecientes tienen el mismo tiempo en el sistema, $A[0..N]$ también lo minimiza.

Nota 4.5.3

El Teorema 4.5.2 no afirma que el ordenamiento óptimo sea único: cuando hay tiempos de servicio repetidos, hay varios ordenamientos *de los agentes* que producen el mismo arreglo de tiempos y, por lo tanto, el mismo tiempo en el sistema. Lo que sí es único es el arreglo de tiempos resultante. El Ejercicio 1 de esta sección ilustra la situación.

4.5.4. Una implementación eficiente. El criterio de selección de este problema es *estático* en el sentido de la Nota 4.1.3: el tiempo de servicio de un agente no cambia con lo que se haya decidido antes. Aplica entonces la receta de *ordenar y recorrer*. De hecho, aquí el ordenamiento no es una preparación para el algoritmo voraz sino que *es* el algoritmo voraz. Lo único que queda por hacer es evaluar la función objetivo sobre el arreglo ya ordenado.

Entendiendo que un ordenamiento no decreciente de $A[0..N]$ minimiza el tiempo en el sistema asociado, solo queda pendiente diseñar un algoritmo que calcule eficientemente este tiempo (para cualquier ordenamiento). Considere la siguiente función objetivo, con $0 \leq n \leq N$:

$\phi(n)$: “contribución acumulada de los primeros n agentes al tiempo total de los N agentes asociada al ordenamiento en $A[0..n]$.”

Se entiende que el ordenamiento en $A[0..n]$ corresponde a los primeros n agentes en ser atendidos. La definición recurrente de ϕ , para $0 \leq n \leq N$, consta de un caso base y uno inductivo, como se propone a continuación:

$$\phi(n) = \begin{cases} 0 & , \text{ si } n = 0, \\ \phi(n-1) + (N - (n-1)) \cdot A[n-1] & , \text{ si } n \neq 0. \end{cases}$$

El caso base corresponde a la situación en la cual no hay agentes por servir y el tiempo en el sistema es 0. El caso inductivo, el tiempo de servicio es el tiempo de servicio correspondiente a todos los agentes que se han atendido previamente

en adición al tiempo que toma servir al n -ésimo agente, acumulando este último tiempo para los agentes que hace falta atender (i.e., $N - n$).

Observe que $\phi(N)$ debe determinar el tiempo en el sistema para los N agentes de acuerdo con el ordenamiento establecido en $A[0..N)$. Entonces, debe ser cierto que $\phi(N) = \sigma(A)$, con $A[0..N)$.

Teorema 4.5.3

$$\phi(N) = \sigma(A).$$

Demostración

Note que N está fijo, tanto en la definición de σ como en la de ϕ , de modo que la inducción no puede hacerse sobre el tamaño del arreglo. Se demuestra, por inducción sobre n , que para todo $0 \leq n \leq N$:

$$\phi(n) = (+k \mid 0 \leq k < n : (N - k) \cdot A[k]).$$

Si $n = 0$, ambos lados valen 0: el izquierdo por definición de ϕ y el derecho por ser una suma vacía. Si $n \neq 0$, por definición de ϕ y por la Hipótesis Inductiva:

$$\begin{aligned} \phi(n) &= \phi(n-1) + (N - (n-1)) \cdot A[n-1] \\ &= (+k \mid 0 \leq k < n-1 : (N - k) \cdot A[k]) \\ &\quad + (N - (n-1)) \cdot A[n-1] \\ &= (+k \mid 0 \leq k < n : (N - k) \cdot A[k]). \end{aligned}$$

Instanciando la igualdad anterior con $n = N$ y comparándola con la Nota 4.5.1, se concluye $\phi(N) = \sigma(A)$.

Lo que resta es definir una función recurrente que implemente la función ϕ , como lo propone el Algoritmo 4.5.1.

Algoritmo 4.5.1

```

1 def sch(A, N, n):
2   ans = None
3   if n==0: ans = 0
4   else: ans = (N-(n-1))*A[n-1] + sch(A, N, n-1)
5   return ans

```

A continuación se presenta un ejemplo en el cual se usa la función `sch` con los tiempos en el Ejemplo 4.5.1.

Ejemplo 4.5.2

El objetivo es determinar el mejor tiempo de servicio (i.e., el mínimo) con los tiempos registrados en $A = [4, 8, 3]$ (Ejemplo 4.5.1).

```
1 A = [ 4, 8, 3 ]
```

Después de ordenar el arreglo ascendentemente, se obtiene:

```
1 >>> A.sort()
```

```
2 >>> A
```

```
3 [3, 4, 8]
```

Luego, usando la función `sch` se obtiene:

```
1 >>> sch(A, len(A), len(A))
```

```
2 25
```

El Teorema 4.5.4 presenta la correctitud de la función `sch`, junto con sus complejidades temporal y espacial.

Teorema 4.5.4

Si los valores en $A[0..N)$ están ordenados no decrecientemente, entonces el llamado `sch(A, N, N)` calcula $\phi(N)$, y toma tiempo y espacio $O(N)$ y $O(1)$, respectivamente. Si es necesario ordenar el arreglo, entonces la complejidad temporal es $O(N \log N)$.

Demostración

Se propone como ejercicio al lector.

El resultado coincide con lo que anticipa el Teorema 4.1.2 para un criterio estático. Note, sin embargo, que acá el ordenamiento no es un costo accesorio que se paga para poder aplicar la estrategia voraz, como ocurría en las dos secciones anteriores: es la estrategia voraz misma. Por eso el $O(N \log N)$ es, en este problema, insuperable con este enfoque.

Ejercicios

1. Suponga que hay 4 agentes en el sistema con tiempos 4, 1, 8 y 8, respectivamente. Enumere los tiempos en el sistema asociados a todos los ordenamientos de estos tiempos. ¿Cuál es el mejor?
2. Demuestre el Teorema 4.5.4.
3. Proponga una versión iterativa para `sch`.
4. Diseñe una función objetivo ψ , como alternativa a ϕ , de acuerdo con la siguiente especificación, con $0 \leq n \leq N$,

$\psi(n)$: “tiempo en el sistema asociado al ordenamiento en $A[n..N]$.”

con el objetivo de calcular $\psi(0)$ de tal manera que $\phi(N) = \psi(0)$. Justifique brevemente su diseño.

5. Suponga que, al contrario de lo que se busca en el problema del tiempo en el sistema, se desea calcular el máximo tiempo posible.
 - a) Proponga una estrategia voraz.
 - b) Enuncie y demuestre un teorema de optimización local.
 - c) Enuncie y demuestre un teorema de optimización global.
 - d) Proponga y defina recurrentemente una función objetivo ϕ .
 - e) Diseñe una función `schdual` en el lenguaje de programación Python que calcule ϕ .
 - f) Enuncie un teorema de correctitud para `schdual`, y establezca las complejidades temporal y espacial de la función.
 - g) Demuestre el teorema de correctitud de `schdual`.
6. La Nota 4.5.2 sostiene que el argumento de inversión es una variante del argumento de intercambio.
 - a) Explique por qué la Definición 4.1.4 no dice nada útil sobre este problema.
 - b) Proponga una manera de acomodar los problemas de ordenamiento dentro del marco de la Sección 4.1, de modo que el Teorema 4.1.1 sea aplicable.
Ayuda: tome como candidato la decisión de *cuál agente se atiende a continuación*.
7. 🌈 Suponga ahora que cada agente n tiene además un *peso* $W[n] > 0$, que mide su importancia, y que el objetivo es minimizar la suma de los productos entre el peso de cada agente y el instante en que termina de ser atendido.
 - a) Demuestre, con dos agentes, que atender primero al de menor tiempo de servicio deja de ser óptimo.
 - b) Proponga un criterio de selección para esta variante. *Ayuda:* el tiempo de servicio y el peso deben combinarse en una sola cantidad.

- c) Enuncie y demuestre el teorema de optimización local correspondiente, siguiendo el patrón de inversión de esta sección.
- d) Verifique que el problema original es el caso particular en el cual todos los pesos valen 1.

4.6. El morral fraccionario

El problema del morral, presentado en la Sección 3.6, pide escoger elementos de una colección –cada uno con un peso y un valor– de manera que el valor total sea máximo sin que el peso total exceda una capacidad dada. Es un problema de optimización, tiene subestructura óptima y su enunciado se parece bastante a los de este capítulo. Es razonable preguntarse si admite una solución voraz.

La respuesta es negativa y conviene ver por qué antes de continuar. El criterio de selección más plausible para este problema es la *densidad* de un elemento, es decir, el valor que aporta por unidad de peso; llevar primero los elementos de mayor densidad parece la manera natural de aprovechar una capacidad escasa.

Ejemplo 4.6.1: El morral resiste

Considere dos elementos con valores y pesos dados por, respectivamente:

$$V = [6, 10] \quad \text{y} \quad W = [3, 7],$$

y una capacidad $X = 7$. Las densidades son $6/3 = 2$ y $10/7 \approx 1,43$, de modo que el criterio prefiere el elemento 0. Al llevarlo quedan 4 unidades de capacidad, en las cuales el elemento 1 no cabe. El valor total es 6.

Sin embargo, llevar únicamente el elemento 1 produce un valor de 10.

Nota 4.6.1: Qué es lo que falla

El Ejemplo 4.6.1 no refuta la idea de que la densidad sea la medida correcta; lo que refuta es que llevar primero el elemento más denso deje siempre abierta la puerta a una solución óptima. Y la razón por la cual la cierra es la *indivisibilidad*: al llevar el elemento 0 quedan 4 unidades de capacidad que ya no sirven para nada, porque no hay manera de llevar cuatro séptimas partes del elemento 1.

Esa observación sugiere un experimento. ¿Qué pasa si se permite partir los elementos? El problema que resulta ya no es el mismo –nadie ha pedido resolverlo– pero es más fácil, y lo que se aprenda de él no será tiempo perdido: la Sección 5.7 lo utilizará para resolver el problema original.

Problema 4.6.1: El morral fraccionario

Entrada: Arreglos $V[0..N]$ y $W[0..N]$, con $N \geq 0$, de valores y pesos (números positivos), y un número $X \geq 0$.

Salida: Máximo valor de $(+n \mid 0 \leq n < N : x_n \cdot V[n])$ entre las asignaciones $x_0, \dots, x_{N-1}$ con $x_n \in [0..1]$ que satisfacen $(+n \mid 0 \leq n < N : x_n \cdot W[n]) \leq X$.

El número x_n indica qué fracción del elemento n se lleva: $x_n = 1$ significa llevarlo entero, $x_n = 0$ no llevarlo y $x_n = 1/3$ llevar una tercera parte, que pesa $W[n]/3$ y aporta $V[n]/3$. El Problema 4.6.1 con la restricción adicional $x_n \in \{0, 1\}$ es exactamente el problema del morral de la Sección 3.6; por eso se dice que es una *relajación* suya.

Leído con el vocabulario de la Sección 4.1, las *soluciones factibles* son las asignaciones que respetan la capacidad, la *función objetivo* de una de ellas es el valor total que produce y el *sentido* es maximizar. El conjunto de candidatos son los N elementos.

Nota 4.6.2: Asignar en lugar de escoger

Este problema tiene, respecto del marco de la Sección 4.1, una particularidad parecida a la de la Sección 4.5 aunque de otra naturaleza. Allá una solución factible era un ordenamiento de los candidatos; acá es una *asignación* que le da a cada candidato un número en $[0..1]$ y no un subconjunto como pide la Definición 4.1.2.

La adaptación es directa y no vale la pena formalizarla en el cuerpo del texto: donde las definiciones dicen que una solución *contiene* un candidato, aquí hay que leer que le *asigna un valor determinado*. Con esa lectura, la propiedad de elección voraz de la Definición 4.1.4 afirma que existe una solución óptima que le asigna al candidato preferido por el criterio exactamente lo que le asigna el algoritmo voraz y la subestructura óptima dice lo de siempre. Los subconjuntos son el caso particular en el cual las asignaciones solo pueden valer 0 o 1.

4.6.1. Escoger un criterio. Hay tres criterios de selección que parecen naturales: llevar primero el elemento de mayor *valor*, el de menor *peso* o el de mayor *densidad* $V[n]/W[n]$. Una sola instancia con tres elementos basta para descartar los dos primeros.

Ejemplo 4.6.2: Dos criterios que no funcionan

Considere tres elementos con valores y pesos dados por, respectivamente:

$$V = [1, 10, 20] \quad \text{y} \quad W = [1, 5, 100],$$

y una capacidad $X = 5$. Las densidades son 1, 2 y 0,2.

El criterio de *mayor valor* prefiere el elemento 2, del cual cabe una vigésima parte; el valor total es 1.

El criterio de *menor peso* prefiere el elemento 0, que se lleva entero y aporta 1; con las 4 unidades de capacidad restantes se llevan cuatro quintas partes del elemento 1, que aportan 8. El valor total es 9.

El criterio de *mayor densidad* prefiere el elemento 1, que se lleva entero, agota la capacidad y aporta 10. Ninguna otra asignación produce más (¿por qué?).

Los dos criterios refutados fallan por la misma razón y es instructiva: ambos miran una sola de las dos cantidades que definen a un elemento. El valor sin el peso no dice cuánta capacidad cuesta obtenerlo; el peso sin el valor no dice qué se obtiene a cambio. La densidad es la única de las tres que las combina y es la que sobrevive.

De aquí en adelante se supone que los elementos están ordenados descendientemente por densidad, es decir, que:

$$\frac{V[0]}{W[0]} \geq \frac{V[1]}{W[1]} \geq \dots \geq \frac{V[N-1]}{W[N-1]}.$$

Como los pesos son positivos, la densidad está bien definida y este ordenamiento siempre existe. Note que el criterio es *estático* en el sentido de la Nota 4.1.3: la densidad de un elemento no depende de lo que se haya llevado antes. Aplica entonces la receta de *ordenar y recorrer*.

Algoritmo 4.6.1

Para elementos ordenados descendientemente por densidad y una capacidad X : llevar los elementos enteros, en ese orden, mientras quepan; del primero que no quepa, llevar la fracción que alcance; no llevar nada de los restantes.

4.6.2. Optimización local. Lo que el Algoritmo 4.6.1 le asigna al primer elemento es la mayor fracción de él que cabe en el morral, que es 1 si el elemento entero cabe y $X/W[0]$ si no.

Teorema 4.6.1: Optimización local

Suponga $N \geq 1$, que los elementos están ordenados descendientemente por densidad y que $g = 1 \downarrow (X/W[0])$. Entonces, hay una solución óptima $y_0, \dots, y_{N-1}$ del Problema 4.6.1 con $y_0 = g$.

Demostración

Escríbase $d_n = V[n]/W[n]$ para la densidad del elemento n , de modo que $d_0 \geq d_n$ para todo n , y sea $y_0, \dots, y_{N-1}$ una solución óptima cualquiera. Note primero que $y_0 \leq g$: por un lado $y_0 \leq 1$ por ser una solución factible y por otro $y_0 \cdot W[0] \leq X$ por la restricción de capacidad, es decir, $y_0 \leq X/W[0]$.

Si $y_0 = g$, no hay nada que hacer. Suponga entonces $y_0 < g$ y considere $\delta = (g - y_0) \cdot W[0] > 0$, que es el peso que hace falta liberar para poder subir y_0 hasta g .

El primer paso es verificar que ese peso está disponible entre los demás elementos, i.e., que:

$$(+n \mid 1 \leq n < N : y_n \cdot W[n]) \geq \delta.$$

En efecto, $g \cdot W[0] \leq X$ (¿por qué?), y por lo tanto:

$$\delta = g \cdot W[0] - y_0 \cdot W[0] \leq X - y_0 \cdot W[0].$$

Ahora bien, si la solución y dejara capacidad sin usar podría aumentarse y_0 —lo cual es posible porque $y_0 < g \leq 1$ — y su valor crecería, contradiciendo que es óptima. Luego la capacidad está agotada, el peso dedicado a los elementos $1, \dots, N-1$ es exactamente $X - y_0 \cdot W[0]$ y este no es menor que δ .

El segundo paso es el intercambio. Constrúyase la asignación z tomando $z_0 = g$ y reduciendo los demás y_n , en cualquier orden, hasta haber liberado un peso total de exactamente δ ; el paso anterior garantiza que esto es posible. Sea $\Delta_n \geq 0$ la reducción aplicada a y_n , de manera que:

$$(+n \mid 1 \leq n < N : \Delta_n \cdot W[n]) = \delta.$$

La asignación z es factible: cada z_n sigue en $[0..1]$ y el peso total no cambió, pues se agregaron δ unidades al elemento 0 y se liberaron δ de los demás.

Falta comparar los valores. Lo que se gana es $(g - y_0) \cdot V[0] = \delta \cdot d_0$ y lo que se pierde es:

$$\begin{aligned} (+n \mid 1 \leq n < N : \Delta_n \cdot V[n]) &= (+n \mid 1 \leq n < N : \Delta_n W[n] \cdot d_n) \\ &\leq (+n \mid 1 \leq n < N : \Delta_n W[n] \cdot d_0) \\ &= \delta \cdot d_0. \end{aligned}$$

Es decir, el valor de z no es menor que el de y . Como y es óptima, z también lo es y $z_0 = g$.

Nota 4.6.3: Densidad y capacidad

El Teorema 4.6.1 es otro argumento de intercambio de los descritos en la Nota 4.2.3, y su propiedad óptima es que ninguna otra densidad supera a d_0 : por eso el peso que se le quita a cualquier otro elemento vale, medido en valor, a lo sumo lo que ese mismo peso vale puesto en el elemento 0.

Compare esta demostración con el Ejemplo 4.6.1, en el cual la misma idea fracasaba. La diferencia está en el paso en el cual se construye z reduciendo los y_n hasta liberar *exactamente* δ . Eso es lo que la divisibilidad permite y la indivisibilidad prohíbe: en el problema original solo se pueden liberar elementos completos y ningún peso intermedio.

4.6.3. Optimización global. Como en las secciones anteriores, una vez demostrada la propiedad de elección voraz solo falta verificar la subestructura óptima.

Teorema 4.6.2: Subestructura óptima

Suponga $N \geq 1$ y sea $g = 1 \downarrow (X/W[0])$. Suponga además que hay una solución óptima del Problema 4.6.1 que le asigna g al elemento 0. Si $z_1, \dots, z_{N-1}$ es una solución óptima del problema sobre los elementos $1, \dots, N-1$ con capacidad $X - g \cdot W[0]$, entonces $g, z_1, \dots, z_{N-1}$ es una solución óptima del problema original.

Demostración

La asignación $g, z_1, \dots, z_{N-1}$ es factible porque el peso que dedica a los elementos $1, \dots, N-1$ no excede $X - g \cdot W[0]$.

Sea y una solución óptima del problema original con $y_0 = g$, que existe por suposición. Entonces $y_1, \dots, y_{N-1}$ es una solución factible del problema

sobre los elementos $1, \dots, N-1$ con capacidad $X - g \cdot W[0]$ y, como z es óptima allí, el valor de $y_1, \dots, y_{N-1}$ no supera el de $z_1, \dots, z_{N-1}$. Sumando $g \cdot V[0]$ a ambos lados se concluye que el valor de y no supera el de $g, z_1, \dots, z_{N-1}$. Como y es óptima, los dos valores coinciden.

Teorema 4.6.3: Optimización global

Si los elementos están ordenados descendientemente por densidad, entonces el Algoritmo 4.6.1 resuelve el Problema 4.6.1.

Demostración

Se procede por inducción sobre N .

Si $N = 0$ no hay nada que llevar y la única asignación, que es la vacía, es óptima.

Si $N \geq 1$, el Teorema 4.6.1 garantiza que hay una solución óptima que le asigna al elemento 0 exactamente $g = 1 \downarrow (X/W[0])$, que es lo que le asigna el algoritmo. Lo que resta por resolver es el mismo problema sobre los elementos $1, \dots, N-1$ con capacidad $X - g \cdot W[0]$, cuyos elementos siguen ordenados descendientemente por densidad; por la Hipótesis Inductiva, el algoritmo lo resuelve de manera óptima. Por el Teorema 4.6.2, lo que produce en total es una solución óptima del problema original.

Nota 4.6.4: La misma inducción

La demostración anterior es, paso por paso, la del Teorema 4.1.1: optimización local para comprometerse con la primera decisión, hipótesis inductiva para el resto y subestructura óptima para pegar las dos partes. No se invocó aquel teorema directamente únicamente porque su enunciado habla de subconjuntos de candidatos y aquí las soluciones son asignaciones, según se advirtió en la Nota 4.6.2. El Ejercicio 7 propone enunciarlo en la generalidad necesaria.

4.6.4. Una implementación eficiente. Considere la siguiente función objetivo, para $0 \leq n \leq N$ y $x \geq 0$:

$\phi(n, x)$: “máximo valor que se puede obtener con los elementos en $V[n..N]$ y $W[n..N]$, permitiendo fracciones, con una capacidad de x .”

Para resolver el problema del morral fraccionario, el objetivo es calcular $\phi(0, X)$ sobre los elementos ya ordenados por densidad.

La definición recurrente de ϕ se establece tomando a n como pivote y preguntando si el elemento n cabe entero en la capacidad disponible:

$$\phi(n, x) = \begin{cases} 0 & , \text{ si } n = N, \\ V[n] + \phi(n+1, x - W[n]) & , \text{ si } n \neq N \wedge W[n] \leq x, \\ x \cdot V[n]/W[n] & , \text{ si } n \neq N \wedge W[n] > x. \end{cases}$$

El caso base corresponde a la situación en la cual no quedan elementos por considerar. El primer caso inductivo corresponde a la situación en la cual el elemento n cabe entero: se lleva completo y se recurre con la capacidad reducida. El segundo caso inductivo corresponde a la situación en la cual el elemento n no cabe: se lleva la fracción $x/W[n]$, que aporta $x \cdot V[n]/W[n]$, con lo cual la capacidad queda agotada y no hace falta recurrir.

Nota 4.6.5: Dos maneras de terminar

Vale la pena notar que la recurrencia tiene dos maneras de terminar y que corresponden a las dos maneras en las que el Algoritmo 4.6.1 se detiene: porque se acabaron los elementos o porque se acabó la capacidad. La segunda no aparece como un caso base sino escondida en el tercer caso, que no recurre. Esta es también la razón por la cual el algoritmo lleva a lo sumo un elemento partido (¿por qué?).

Algoritmo 4.6.2

```

1 def frac(V, W, N, X, n):
2     # items sorted in decreasing order of density V[i]/W[i]
3     ans = None
4     if n==N: ans = 0
5     else:
6         if W[n]<=X: ans = V[n] + frac(V, W, N, X-W[n], n+1)
```

```

7     else: ans = X*(V[n]/W[n])
8     return ans

```

Ejemplo 4.6.3

Considere los elementos del Ejemplo 4.6.2:

```

1 V = [1, 10, 20]
2 W = [1, 5, 100]

```

Después de ordenarlos descendientemente por densidad, se obtiene:

```

1 >>> P = sorted(zip(V, W), key = lambda p: -p[0]/p[1])
2 >>> V = [p[0] for p in P]
3 >>> W = [p[1] for p in P]
4 >>> V, W
5 ([10, 1, 20], [5, 1, 100])

```

Con base en la versión ordenada y con una capacidad de 5, se lleva únicamente el primer elemento:

```

1 >>> frac(V, W, len(V), 5, 0)
2 10.0

```

Con una capacidad de 7, en cambio, se llevan el primero y el segundo enteros, y una centésima parte del tercero:

```

1 >>> frac(V, W, len(V), 7, 0)
2 11.2

```

Teorema 4.6.4

Si los elementos están ordenados descendientemente por densidad, entonces el llamado `frac(V, W, N, X, 0)` calcula $\phi(0, X)$, y toma tiempo y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$ y $O(1)$, respectivamente. Si es necesario ordenar los elementos, entonces la complejidad temporal es $O(N \log N)$.

Demostración

Se propone como ejercicio al lector.

4.6.5. Una cota para el problema original. El morral fraccionario no fue el problema que se quiso resolver, sino una relajación del que sí interesaba. Lo que se gana con la relajación es lo siguiente.

Teorema 4.6.5

Para cualesquiera arreglos $V[0..N)$ y $W[0..N)$ de números positivos y cualquier capacidad $X \geq 0$, el valor óptimo del Problema 4.6.1 no es menor que el valor óptimo del problema del morral de la Sección 3.6 sobre los mismos datos.

Demostración

Sea $S \subseteq \{0, \dots, N-1\}$ una solución óptima del problema del morral. La asignación que toma $x_n = 1$ para $n \in S$ y $x_n = 0$ para $n \notin S$ es una solución factible del Problema 4.6.1 –tiene el mismo peso total, que no excede X – y produce el mismo valor. Luego el valor óptimo del problema relajado no puede ser menor.

Es decir, en tiempo $O(N \log N)$ se obtiene un número que acota por encima la respuesta de un problema que el Capítulo 3 tuvo que resolver por tabulación. Contar con una cota superior que se calcule rápido es exactamente lo que necesita la técnica de ramificación y poda, y en eso la empleará la Sección 5.7.

Ejercicios

1. Demuestre el Teorema 4.6.4.
2. Justifique las siguientes afirmaciones hechas en esta sección:
 - a) En el Ejemplo 4.6.2, ninguna asignación produce un valor mayor que 10.
 - b) El Algoritmo 4.6.1 lleva a lo sumo un elemento partido.
 - c) En la demostración del Teorema 4.6.1 se cumple $g \cdot W[0] \leq X$, sea cual sea de los dos valores el que alcanza el mínimo.
3. En el enunciado del Problema 4.6.1 se exige que los pesos sean positivos.
 - a) ¿Qué parte de esta sección deja de funcionar si se permite que algún peso sea 0?
 - b) Proponga una manera de tratar esos elementos y justifique que la solución resultante sigue siendo óptima.
4. Modifique la función `frac` para que, además del valor óptimo, retorne las fracciones $x_0, \dots, x_{N-1}$ que lo alcanzan.

5. Diseñe una versión iterativa de la función `frac`, en tiempo $O(N)$ y espacio $O(1)$ sobre elementos ya ordenados.
6. Suponga que el criterio de la densidad produce empates, es decir, que hay elementos $i \neq j$ con $V[i]/W[i] = V[j]/W[j]$.
 - a) ¿Afecta esto la corrección del Algoritmo 4.6.1?
 - b) ¿En qué paso de la demostración del Teorema 4.6.1 se usa que la desigualdad entre densidades no es estricta?
7. La Nota 4.6.2 sostiene que las definiciones de la Sección 4.1 se adaptan a este problema reemplazando “la solución contiene al candidato” por “la solución le asigna al candidato un valor determinado”.
 - a) Escriba las versiones generalizadas de la Definición 4.1.2, la Definición 4.1.4 y la Definición 4.1.5.
 - b) Enuncie y demuestre la versión correspondiente del Teorema 4.1.1.
 - c) Verifique que las definiciones originales son el caso particular en el cual las asignaciones solo toman los valores 0 y 1.
8. Considere la variante del morral fraccionario en la cual de cada elemento hay una cantidad ilimitada de copias.
 - a) Especifique el problema.
 - b) Proponga un criterio de selección, y enuncie y demuestre el teorema de optimización local correspondiente.
 - c) Diseñe un algoritmo que lo resuelva en tiempo $O(N)$, sin necesidad de ordenar.
9. 🌈 El Teorema 4.6.5 da una cota superior para el problema del morral, y se quiere saber qué tan buena es. Sean B el valor óptimo del morral fraccionario, O el del morral y V^* el mayor de los valores en $V[0..N)$.
 - a) Demuestre que $B < O + V^*$.
 - b) Concluya que llevar, en orden de densidad, los elementos enteros que quepan produce una solución del problema del morral cuyo valor es mayor que $O - V^*$.
 - c) Construya una familia de instancias en las cuales la diferencia entre B y O se acerque tanto como se quiera a V^* .

4.7. Árboles de cubrimiento mínimo

Identificar un árbol de cubrimiento mínimo es un problema de optimización sobre grafos. Dado un grafo no dirigido, conexo y con peso en sus arcos, el problema consiste en encontrar un subgrafo que cumpla con dos propiedades específicas: una estructural, de ser conexo y sin ciclos, y otra de optimalidad, en cuanto que tenga la menor suma posible de los pesos de sus arcos. Este problema, conocido en

inglés como *minimum spanning tree* (MST), ha sido ampliamente estudiado y se conocen soluciones algorítmicas *voraces* eficientes, además relativamente fáciles de implementar.

Antes de especificar el problema algorítmico es indispensable introducir formalmente algunos conceptos y términos útiles para su definición. En particular, se precisan las nociones de árbol de cubrimiento y de minimalidad en este contexto a partir de nociones de grafos que comúnmente se conocen.

Definición 4.7.1

Un *árbol* es un grafo no dirigido que es conexo y acíclico. Dado un grafo no dirigido $G = (V, E)$, se dice que $T = (V_T, E_T)$ es un *árbol de cubrimiento* de G si:

$$T \text{ es un árbol,} \quad V_T = V \quad \text{y} \quad E_T \subseteq E.$$

Dada además una función de peso $w : E \rightarrow \mathbb{R}$, se escribe $w(T)$ para la suma de los pesos de los arcos en E_T . Se dice que T es un *árbol de cubrimiento mínimo* de G (con respecto a w) si ningún otro árbol de cubrimiento T' de G satisface $w(T') < w(T)$.

Un árbol de cubrimiento mínimo de un grafo es un subgrafo que conecta todos los vértices, sin ciclos y lo menos pesado posible. Un escenario común en el cual se requieren árboles de cubrimiento mínimo es en el diseño de redes de comunicaciones. Considere diferentes ubicaciones que deben ser conectadas y que el costo de conectar dos ubicaciones es proporcional a su distancia. El objetivo es conectar todas las ubicaciones usando el presupuesto más modesto posible. La solución deberá ser un árbol de cubrimiento mínimo.

Problema 4.7.1: Árbol de cubrimiento mínimo

Entrada: Un grafo $G = (V, E)$ conexo con función de peso $w : E \rightarrow \mathbb{R}$.

Salida: Un árbol $T = (V, E_T)$ de cubrimiento mínimo de G (con respecto a w).

Nota 4.7.1: Sobre el signo de los pesos

Ni la Definición 4.7.1 ni el Problema 4.7.1 imponen restricción alguna sobre el signo de los pesos y es deliberado: *ningún* resultado de esta sección la

necesita. El teorema del corte y los dos algoritmos que se presentan más adelante funcionan igual con pesos negativos.

En las aplicaciones habituales –distancias, costos, capacidades– los pesos son positivos y hay enunciados que sí dependen de ello. El Ejercicio 1 es uno y por eso agrega la hipótesis explícitamente en lugar de heredarla del problema.

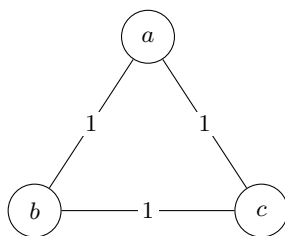
Vale la pena retener la observación porque el contraste con la Sección 4.8 es marcado. Allá la hipótesis sobre el signo de los pesos no es un adorno de la especificación sino el corazón de la demostración y el algoritmo deja de funcionar sin ella.

Leído con el vocabulario de la Sección 4.1, los tres elementos de la Definición 4.1.1 están en el enunciado del Problema 4.7.1: las *soluciones factibles* son los árboles de cubrimiento de G , la *función objetivo* de uno de ellos es la suma de los pesos de sus arcos y el *sentido* es minimizar. El conjunto de candidatos son los arcos en E y una solución factible es –identificando T con E_T – un subconjunto de ellos.

Una aproximación de fuerza bruta para resolver el problema consiste en enumerar todos los subconjuntos de E y determinar, entre aquellos que representen un árbol de cubrimiento de G , uno de suma mínima. Como hay un total de $\Theta(2^{|E|})$ subconjuntos de arcos de E , necesariamente este enfoque no es viable en la práctica. Aún observando que un árbol con vértices V tiene $|V| - 1$ arcos (Ejercicio 2), la enumeración de los subconjuntos de E de tamaño $|V| - 1$ sigue siendo exponencial en tamaño. Es decir, los enfoques de fuerza bruta no son una opción si se desea considerar grafos con, al menos, centenares de arcos.

Ejemplo 4.7.1

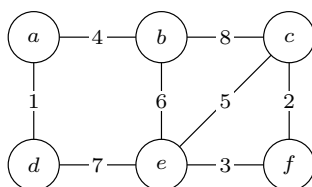
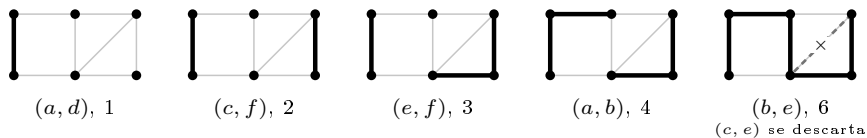
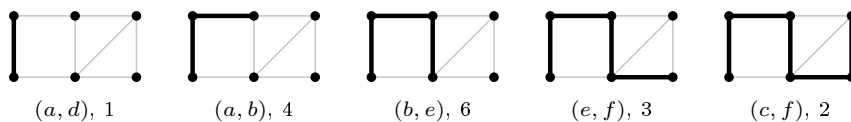
El MST de un grafo dado no es necesariamente único. Por ejemplo, considere el grafo de tres vértices que se dibuja a continuación:



Este grafo tiene tres árboles de cubrimiento mínimo, dados por los siguientes conjuntos de arcos, cada uno con suma de pesos 2: $\{(a, b), (b, c)\}$, $\{(a, b), (a, c)\}$ y $\{(a, c), (b, c)\}$.

Ejemplo 4.7.2: Kruskal y Prim sobre el mismo grafo

Los dos algoritmos voraces de esta sección construyen el mismo árbol de cubrimiento mínimo del grafo que se dibuja arriba, pero lo hacen en órdenes distintos. En cada panel, los vértices ocupan las mismas posiciones que en el grafo de referencia y los arcos ya seleccionados se resaltan.

**Kruskal****Prim**

El algoritmo de Kruskal escoge los arcos en orden ascendente de peso, sin importar dónde estén: durante buena parte de la ejecución lo construido no es un árbol sino un bosque de pedazos sueltos y el arco (c, e) se descarta porque cerraría un ciclo. El algoritmo de Prim, en cambio, mantiene siempre un único árbol que crece desde el vértice a y por eso los pesos que escoge no aparecen en orden ascendente.

Los dos terminan con el mismo conjunto de cinco arcos, de peso total 16. Que coincidan no es casualidad: en este grafo los pesos son distintos par a par y el MST resulta ser único (Ejercicio 26).

Nota 4.7.2: Un problema, dos criterios

Esta sección se aparta de la organización de las anteriores y conviene anunciarlo. En cada una de ellas se propuso un criterio de selección, se demostró para él un teorema de optimización local y se concluyó con uno de optimización

global. Aquí se presentan *dos* algoritmos voraces distintos –el de Kruskal y el de Prim– que resuelven el mismo problema con criterios de selección distintos.

Lo notable es que no hacen falta dos teoremas de optimización local. Los dos criterios son casos particulares de un mismo resultado, el teorema del corte y por eso el desarrollo tiene la forma de un tronco común –el de esta subsección y las dos siguientes– del que cuelgan dos ramas simétricas. Al llegar a ellas, cada una aportará su criterio de selección, su implementación y su análisis, mientras que la corrección estará ya demostrada.

4.7.1. El esquema genérico. La idea general para construir un MST de $G = (V, E)$ con función de peso w es la de siempre: partir del conjunto vacío de arcos y agregar arcos, uno a la vez, cuidando de no arruinar la posibilidad de llegar a una solución óptima. Lo primero es precisar en qué consiste ese cuidado.

Definición 4.7.2

Un conjunto de arcos $A \subseteq E$ es *factible* sii $A \subseteq E_T$ para algún MST $T = (V, E_T)$ de G . Un arco $e \in E \setminus A$ es *seguro* para un conjunto factible A sii $A \cup \{e\}$ es factible.

Nota 4.7.3: Factible no es lo mismo que ser parte de un MST

La Definición 4.7.2 exige que *todo* A esté contenido en un mismo MST y no basta con pedir que cada uno de sus arcos, por separado, haga parte de alguno.

El Ejemplo 4.7.1 lo muestra. Allí cada uno de los tres arcos hace parte de algún MST del triángulo, pero el conjunto formado por los tres no está contenido en ninguno: tiene un ciclo y ningún árbol lo tiene. La condición débil se satisface y la fuerte no.

La distinción importa porque lo que hay que preservar al construir la solución es la condición fuerte. Un algoritmo que solo garantice la débil puede terminar con un conjunto de arcos que no es un árbol.

Note, por último, que la factibilidad de un conjunto de arcos es más exigente que la de una solución. Un árbol de cubrimiento cualquiera es una solución factible del Problema 4.7.1, pero sus arcos forman un conjunto factible únicamente cuando el árbol es de peso mínimo.

Con esta noción, el esquema es inmediato: mantener a A factible y extenderlo con arcos seguros hasta que sea un árbol de cubrimiento.

Algoritmo 4.7.1: Esquema genérico para construir un MST

1. $A = \emptyset$.
2. Mientras $|A| < |V| - 1$:
 - a) encontrar un arco $e \in E \setminus A$ que sea seguro para A ;
 - b) extender A con e .
3. Retornar A .

Nota 4.7.4: Por qué esto no es un algoritmo voraz

Conviene ser claro sobre qué es y qué no es el Algoritmo 4.7.1. Ser seguro *no* es un criterio de selección en el sentido de la Definición 4.1.3: la Definición 4.7.2 está formulada en términos de los MST de G , que son exactamente lo que se está tratando de calcular. Un algoritmo no puede consultarla.

El Algoritmo 4.7.1 es, entonces, una plantilla de corrección y no un algoritmo. Su utilidad es la de fijar de una vez la obligación que cada algoritmo concreto debe cumplir: exhibir un criterio de selección *evaluable* y demostrar que los arcos que escoge son seguros. Los algoritmos voraces de esta sección son los de Kruskal y Prim; el esquema es lo que tienen en común.

4.7.2. Optimización local: el teorema del corte. Las siguientes nociones son las que permiten enunciar un criterio suficiente para que un arco sea seguro, sin mencionar los MST de G .

Definición 4.7.3: Cortes y arcos ligeros

Sea $G = (V, E)$ un grafo no dirigido con función de peso w .

- Un *corte* de V es una pareja (S_0, S_1) de subconjuntos no vacíos de V tales que $S_0 \cap S_1 = \emptyset$ y $S_0 \cup S_1 = V$.
- Un arco $e \in E$ *cruza* un corte (S_0, S_1) de V sii uno de los vértices en e está en S_0 y el otro está en S_1 .
- Un corte (S_0, S_1) de V *respet*a un subconjunto de arcos $E' \subseteq E$ sii ningún arco en E' cruza (S_0, S_1) .
- Un arco $e \in E$ es *ligero* para un corte (S_0, S_1) de V sii $w(e)$ es mínimo entre todos los arcos que cruzan (S_0, S_1) (note que puede haber más de un arco ligero para un corte (S_0, S_1) de V).

El resultado que se presenta a continuación es la base matemática sobre la cual funcionan los algoritmos voraces comúnmente utilizados para calcular el MST de un grafo. Es el teorema de optimización local de esta sección y, a diferencia de los de las secciones anteriores, va a servir para dos criterios de selección distintos.

Teorema 4.7.1: Optimización local: el teorema del corte

Sea $A \subseteq E$ factible y (S_0, S_1) un corte de V que respeta a A . Si $(u, v) \in E$ es un arco ligero para (S_0, S_1) , entonces (u, v) es seguro para A .

Demostración

Como A es factible, hay un MST $T = (V, E_T)$ de G con $A \subseteq E_T$. Note además que $(u, v) \notin A$, pues (u, v) cruza un corte que respeta a A . Se procede por casos.

- Si $(u, v) \in E_T$, entonces $A \cup \{(u, v)\} \subseteq E_T$ y el mismo T atestigua que $A \cup \{(u, v)\}$ es factible. Por ende, (u, v) es seguro para A .
- Si $(u, v) \notin E_T$, entonces note que $(V, E_T \cup \{(u, v)\})$ no es un árbol (¿por qué?). Como T es un árbol, hay un camino simple único (¿por qué?) entre u y v en T . Dado que u y v están en “lados” opuestos de (S_0, S_1) , al menos un arco en el camino de u a v cruza este corte. Sea (x, y) dicho arco: observe que (x, y) no está en A porque (S_0, S_1) respeta a A . Como el camino simple entre u y v es único en T , remover (x, y) de E_T desconecta T en dos componentes. Estas dos componentes se pueden volver a unir agregando el arco (u, v) . Es decir, $T' = (V, (E_T \setminus \{(x, y)\}) \cup \{(u, v)\})$ es un árbol de cubrimiento de G .

El objetivo ahora es demostrar que el árbol de cubrimiento T' de G es de costo mínimo. Para ello, note lo siguiente:

$$\begin{aligned}
 w(T') &= (+e \mid e \in (E_T \setminus \{(x, y)\}) \cup \{(u, v)\} : w(e)) \\
 &= (+e \mid e \in E_T : w(e)) + w(u, v) - w(x, y) \\
 &\leq (+e \mid e \in E_T : w(e)) \\
 &= w(T).
 \end{aligned}$$

El paso de la desigualdad es posible porque (u, v) es ligero para (S_0, S_1) y (x, y) también cruza ese corte, de modo que $w(u, v) \leq w(x, y)$. Por otra parte, se sabe que T es un MST de G . Luego $w(T) \leq w(T')$. De las dos desigualdades se concluye

$$w(T) = w(T').$$

Es decir, T' es un MST de G . Finalmente, $A \subseteq E_T \setminus \{(x, y)\}$ porque $(x, y) \notin A$, y por lo tanto $A \cup \{(u, v)\}$ está contenido en los arcos de T' . Es decir, $A \cup \{(u, v)\}$ es factible y (u, v) es seguro para A .

Nota 4.7.5: Otra vez el intercambio

La demostración del Teorema 4.7.1 es un argumento de intercambio de los descritos en la Nota 4.2.3, aunque de una variedad algo más elaborada que las de las secciones anteriores. Se parte de una solución óptima arbitraria que contiene a A , se identifica en ella el arco (x, y) al cual la elección voraz puede sustituir –que no es un arco cualquiera, sino uno que el camino entre u y v obliga a que exista– y se verifica que el reemplazo no aumenta el peso total.

La propiedad óptima que hace legítimo el intercambio es que (u, v) sea ligero: como (x, y) también cruza el corte, su peso no puede ser menor que el de (u, v) . Y el papel del corte que respeta a A es garantizar que el arco sacrificado no pertenezca a A , es decir, que el intercambio no deshaga ninguna decisión ya tomada.

4.7.3. Optimización global. Con el teorema del corte demostrado, la corrección del Algoritmo 4.7.1 genérico es una inducción rutinaria. Note que el enunciado es condicional: afirma que *si* el paso 2(a) siempre encuentra un arco seguro, entonces el resultado es correcto.

Teorema 4.7.2: Optimización global

Sea $G = (V, E)$ conexo con $V \neq \emptyset$. Si en cada iteración el paso 2(a) del Algoritmo 4.7.1 encuentra un arco seguro para A , entonces el algoritmo termina y el conjunto A que retorna es el conjunto de arcos de un MST de G .

Demostración

Se demuestra primero que “ A es factible” es un invariante del ciclo del paso 2.

Antes de la primera iteración, $A = \emptyset$. Como G es conexo y $V \neq \emptyset$, el grafo tiene al menos un árbol de cubrimiento y, como E es finito, tiene al menos uno de peso mínimo; sea $T = (V, E_T)$ uno de ellos. Entonces $\emptyset \subseteq E_T$ y A es factible. Si A es factible al comienzo de una iteración, por suposición el paso

2(a) encuentra un arco $e \in E \setminus A$ seguro para A y el paso 2(b) lo agrega. Por la Definición 4.7.2, $A \cup \{e\}$ es factible.

El ciclo termina: como $e \notin A$, cada iteración incrementa $|A|$ en exactamente 1, de modo que después de a lo sumo $|V| - 1$ iteraciones la guarda del paso 2 deja de cumplirse.

Al terminar el ciclo, A es factible y $|A| = |V| - 1$. Sea $T = (V, E_T)$ un MST de G con $A \subseteq E_T$. Por el Ejercicio 2, $|E_T| = |V| - 1 = |A|$ y un subconjunto de un conjunto finito que tiene su mismo tamaño es el conjunto entero. Luego $A = E_T$ y A es el conjunto de arcos de un MST de G .

Nota 4.7.6: Lo que le falta al esquema

El Teorema 4.7.2 deja pendiente exactamente una cosa y es la hipótesis: que el paso 2(a) *encuentre* un arco seguro cada vez que la guarda se cumple. Nada de lo dicho hasta ahora garantiza que exista.

Esa es, entonces, la obligación que cada concreción del esquema debe cumplir, y son dos cosas distintas:

1. exhibir un criterio de selección evaluable y demostrar, con ayuda del Teorema 4.7.1, que el arco que escoge es seguro; y
2. demostrar que, mientras $|A| < |V| - 1$, ese criterio siempre tiene algo que escoger.

Las dos subsecciones siguientes hacen ese trabajo dos veces, para dos criterios distintos.

Nota 4.7.7: Un problema anormalmente permisivo

Las secciones anteriores dedicaron un esfuerzo apreciable a refutar criterios de selección plausibles y la Nota 4.1.5 insistió en que romperlos es lo primero que hay que intentar. Con los árboles de cubrimiento mínimo ese ejercicio resulta curiosamente infructuoso: casi todo criterio razonable funciona.

Escoger repetidamente el arco más liviano que no cierre un ciclo funciona y es el algoritmo de Kruskal. Escoger el arco más liviano que conecte lo ya construido con lo que falta funciona y es el de Prim. Escoger simultáneamente, para cada componente, su arco incidente más liviano también funciona y es el algoritmo de Borůvka mencionado en la Nota 4.0.3. Incluso funciona proceder al revés, eliminando repetidamente el arco más pesado cuya remoción no desconecte el grafo (Ejercicio 27).

Esa abundancia no es casualidad ni suerte. Los árboles de cubrimiento son el ejemplo canónico de una estructura llamada *matroide* y el resultado central de esa teoría –al que aludía la Nota 4.0.3– caracteriza precisamente las situaciones en las cuales la estrategia voraz es óptima. Las notas al final del capítulo indican dónde estudiarla.

4.7.4. El algoritmo de Kruskal. El primero de los dos algoritmos concreta el esquema con el criterio de selección más simple que se le puede ocurrir a uno: recorrer los arcos de menor a mayor peso y quedarse con los que se pueda.

Algoritmo 4.7.2: Kruskal

El Algoritmo 4.7.1 con el siguiente criterio de selección: entre los arcos que no cierran un ciclo con A , escoger uno de peso mínimo.

A diferencia de ser seguro, esta condición sí se puede evaluar: para saber si un arco cierra un ciclo con A basta con mirar A y el peso de un arco viene dado. La obligación que impone la Nota 4.7.6 son dos resultados y el primero es una aplicación directa del teorema del corte.

Teorema 4.7.3: Optimización local para Kruskal

Sea $A \subseteq E$ factible y sea $(u, v) \in E \setminus A$ un arco de peso mínimo entre los que no cierran un ciclo con A . Entonces, (u, v) es seguro para A .

Demostración

Sea $S \subseteq V$ el conjunto de vértices de la componente conexa de (V, A) que contiene a u . Se verifica que $(S, V \setminus S)$ es un corte de V que respeta a A y para el cual (u, v) es ligero, con lo cual el Teorema 4.7.1 da la conclusión.

- $(S, V \setminus S)$ es un corte. El conjunto S no es vacío porque $u \in S$. Y $V \setminus S$ tampoco lo es porque $v \notin S$: si v estuviera en la componente de u , habría un camino entre u y v en (V, A) y el arco (u, v) cerraría un ciclo con A , contrario a la suposición.
- El corte respeta a A . Ningún arco de A cruza $(S, V \setminus S)$, pues los dos extremos de un arco de A están siempre en la misma componente conexa de (V, A) .
- (u, v) es ligero para el corte. El arco (u, v) cruza el corte, por lo dicho arriba. Sea (x, y) cualquier otro arco que lo cruce. Entonces x y y están

en componentes distintas de (V, A) y, por lo tanto, (x, y) no cierra un ciclo con A . Como (u, v) es de peso mínimo entre los arcos con esa propiedad, se tiene $w(u, v) \leq w(x, y)$.

Note dónde estuvo el trabajo. El criterio de Kruskal no menciona ningún corte, de modo que la demostración tuvo que fabricar uno –el inducido por la componente conexa de u – y verificar sus tres propiedades por separado. Esa construcción es la parte no evidente del argumento y conviene retenerla, porque es lo que permite que un criterio enunciado sobre ciclos se conecte con un teorema enunciado sobre cortes.

Queda la segunda obligación, que es garantizar que el criterio siempre tenga algo que escoger mientras la guarda del paso 2 se cumpla.

Teorema 4.7.4: Existencia

Sea $G = (V, E)$ conexo y $A \subseteq E$ con $|A| < |V| - 1$ tal que (V, A) es acíclico. Entonces, hay al menos un arco en $E \setminus A$ que no cierra un ciclo con A .

Demostración

Como (V, A) es acíclico y $|A| < |V| - 1$, el grafo (V, A) no es conexo (¿por qué?) y tiene al menos dos componentes. Sea S el conjunto de vértices de una de ellas, de modo que $S \neq \emptyset$ y $V \setminus S \neq \emptyset$. Como G es conexo, hay al menos un arco de E con un extremo en S y el otro en $V \setminus S$; ese arco no está en A y sus extremos están en componentes distintas de (V, A) , de modo que no cierra un ciclo.

Con las dos obligaciones cumplidas, la corrección del algoritmo se obtiene invocando el teorema del esquema genérico. El único detalle que falta es notar que la hipótesis de aciclicidad del resultado anterior se cumple siempre, pues todo conjunto factible está contenido en los arcos de un árbol.

Teorema 4.7.5: Optimización global para Kruskal

El Algoritmo 4.7.2 calcula los arcos de un MST de G .

Demostración

Todo conjunto factible es acíclico, pues está contenido en los arcos de un árbol. Por el Teorema 4.7.4, mientras la guarda del paso 2 se cumpla el criterio tiene al menos un arco entre los cuales escoger; y por el Teorema 4.7.3, el que escoge es seguro. Se satisface entonces la hipótesis del Teorema 4.7.2.

Representación del grafo. Para este algoritmo conviene representar el grafo como una *lista de arcos*: un arreglo $X[0..M]$ que contiene la tripleta $(u, v, w(u, v))$ por cada arco $(u, v) \in E$, en cualquier orden. Los vértices se identifican con los enteros en $[0..N]$. Es la representación natural aquí porque el criterio de selección ordena los arcos y los recorre uno por uno, sin preguntar nunca por los vecinos de un vértice.

Note además que el criterio es *estático* en el sentido de la Nota 4.1.3: el peso de un arco no cambia con lo que se haya seleccionado antes. Aplica entonces la receta de *ordenar y recorrer*, y lo único que falta es poder responder rápido si un arco cierra un ciclo.

Un tipo abstracto de datos. Decidir si el arco (u, v) cierra un ciclo con A equivale a decidir si u y v ya están en la misma componente conexa de (V, A) . Y las componentes conexas de un grafo son las clases de equivalencia de la relación “estar conectado con”, de modo que lo que hace falta es poder consultar y modificar una relación de equivalencia sobre los vértices.

Definición 4.7.4: El tipo abstracto disjoint-union

Sea U un conjunto finito. El tipo abstracto de datos *disjoint-union* sobre U mantiene una relación de equivalencia $\sim$ sobre U y ofrece tres operaciones:

- $make(U)$ construye la relación en la cual cada elemento es equivalente únicamente a sí mismo, en tiempo $O(|U|)$;
- $find(x)$, para $x \in U$, retorna el *representante canónico* de la clase de equivalencia de x , en tiempo $O(\log |U|)$;
- $union(x, y)$, para $x, y \in U$, reemplaza $\sim$ por la menor relación de equivalencia que la contiene y en la cual $x \sim y$, en tiempo $O(\log |U|)$.

El representante canónico es un elemento de la clase, el mismo para todos sus miembros; es decir, $find(x) = find(y)$ sii $x \sim y$.

La operación *union* merece un comentario. La “menor relación de equivalencia que contiene a $\sim$ y en la cual $x \sim y$ ” es exactamente la que resulta de fusionar en una sola las clases de x y de y , dejando las demás intactas (Ejercicio 8). Esa es la razón del nombre: el efecto de la operación es la unión de dos clases disjuntas.

Nota 4.7.8: Tipo abstracto y estructura de datos

Conviene separar dos nociones que suelen confundirse.

Un *tipo abstracto de datos* (en inglés, *abstract data type* o ADT) describe *qué* se puede hacer y a qué costo: un conjunto de operaciones, con sus parámetros, su comportamiento y una restricción de complejidad temporal sobre cada una. Nada dice sobre *cómo* se logra. La Definición 4.7.4 es un ADT.

Una *estructura de datos* (en inglés, *data structure* o DS) describe *cómo*: una representación concreta en memoria y unos algoritmos que operan sobre ella. Se dice que una DS *implementa* un ADT cuando ofrece todas sus operaciones, con el comportamiento especificado, dentro de las restricciones de complejidad exigidas.

La separación paga dos veces. Permite analizar el Algoritmo 4.7.2 antes de haber escogido una implementación, contando operaciones del ADT en lugar de instrucciones. Y permite reemplazar la implementación por otra sin revisar nada de lo anterior, siempre que la nueva respete las mismas restricciones.

Una estructura de datos: *dforest*. La estructura que implementa el ADT anterior representa cada clase de equivalencia como un árbol cuya raíz es el representante canónico de la clase; el conjunto de todas ellas es un bosque, de donde viene el nombre *dforest*. Se guarda en dos arreglos: $p[0..N)$ con el padre de cada elemento, donde una raíz es su propio padre, y $sz[0..N)$ con el tamaño del árbol que cada raíz encabeza.

Algoritmo 4.7.3: La estructura dforest

```

1 def dmake(N):
2     return [ list(range(N)), [1]*N ]
3
4 def dfind(D, x):
5     p = D[0]
6     while p[x]!=x: x = p[x]
7     return x
8
9 def dunion(D, x, y):
10    p, sz = D[0], D[1]
11    rx, ry = dfind(D, x), dfind(D, y)
12    if rx!=ry:
13        if sz[rx]<sz[ry]: rx, ry = ry, rx
14        p[ry] = rx

```

15 $sz[rx] = sz[rx] + sz[ry]$

La función `dfind` sube de padre en padre hasta llegar a la raíz, de modo que su costo es proporcional a la altura del árbol. La función `dunion` cuelga una raíz de la otra y ahí está la única decisión de diseño: cuelga siempre el árbol *más pequeño* del más grande. Esa regla, llamada *unión por tamaño*, es lo que impide que los árboles se vuelvan altos.

Teorema 4.7.6

En un *dforest* construido con `dmake` y modificado únicamente con `dunion`, todo árbol de altura h tiene al menos 2^h elementos.

Demostración

Se procede por inducción sobre la cantidad de llamados a `dunion` ejecutados.

Si no se ha ejecutado ninguno, cada árbol tiene un solo elemento y altura 0, y $1 \geq 2^0$.

Suponga que la propiedad vale antes de un llamado a `dunion` que efectivamente fusiona dos árboles, con raíces r_x y r_y , alturas h_x y h_y y tamaños $s_x \geq s_y$; el código cuelga r_y de r_x . Los demás árboles no cambian. El árbol resultante tiene tamaño $s_x + s_y$ y altura $h_x \uparrow (h_y + 1)$. Hay dos casos:

- Si $h_x \geq h_y + 1$, la altura sigue siendo h_x y el tamaño creció, de modo que $s_x + s_y > s_x \geq 2^{h_x}$ por la Hipótesis Inductiva.
- Si $h_x < h_y + 1$, la altura pasa a ser $h_y + 1$ y, usando $s_x \geq s_y$ y la Hipótesis Inductiva sobre el árbol de r_y :

$$s_x + s_y \geq 2 \cdot s_y \geq 2 \cdot 2^{h_y} = 2^{h_y+1}.$$

Teorema 4.7.7

El Algoritmo 4.7.3 implementa el ADT *disjoint-union* de la Definición 4.7.4 sobre $U = [0..N)$.

Demostración

Que las operaciones se comportan como exige la Definición 4.7.4 se propone como ejercicio al lector (Ejercicio 9). Para las restricciones de complejidad, note que `dmake` construye dos arreglos de tamaño N en tiempo $O(N)$.

Por el Teorema 4.7.6, un árbol de altura h tiene al menos 2^h elementos y, como ningún árbol tiene más de N elementos, se cumple $2^h \leq N$; es decir, $h \leq \log_2 N$. El ciclo de `dfind` ejecuta a lo sumo h iteraciones, cada una en tiempo constante, de modo que `dfind` toma tiempo $O(\log N)$. La función `dunion` hace dos llamados a `dfind` y una cantidad constante de trabajo adicional, de modo que también toma tiempo $O(\log N)$.

Nota 4.7.9: Compresión de caminos

La unión por tamaño no es la única técnica disponible. La *compresión de caminos* consiste en aprovechar cada llamado a `dfind` para reconectar directamente a la raíz todos los elementos que se visitaron en el ascenso, de manera que las consultas siguientes sobre ellos sean inmediatas. Combinada con la unión por tamaño, el costo amortizado de las dos operaciones deja de ser $O(\log N)$ y pasa a ser esencialmente constante; las notas al final del capítulo indican dónde estudiar el resultado preciso, que es notablemente difícil.

Para lo que sigue, sin embargo, la mejora es irrelevante. El Algoritmo 4.7.2 tiene que ordenar los arcos y ese ordenamiento va a dominar el costo total sin importar qué tan rápidas sean las operaciones sobre el *dforest*. Es un ejemplo instructivo de que optimizar la parte más sofisticada de un algoritmo no siempre mejora nada. El Ejercicio 12 propone implementarla y medir.

La implementación. Para llegar a un algoritmo hace falta un último ingrediente. El criterio del Algoritmo 4.7.2 pide, en cada paso, el arco de peso mínimo entre los que no cierran un ciclo, lo cual parece obligar a revisar todos los arcos disponibles cada vez. No es necesario: si $X[0..M)$ contiene los arcos de E ordenados ascendentemente por peso y ya se examinaron los de $X[0..m)$, entonces basta con mirar $X[m]$. En efecto, los arcos de $X[0..m)$ que no fueron seleccionados cierran un ciclo con lo construido y los de $X[m..M)$ no pesan menos que $X[m]$. Luego, si $X[m]$ no cierra un ciclo, es de peso mínimo entre los que no lo cierran.

Escríbanse $N = |V|$ y $M = |E|$. Considere entonces la siguiente función objetivo, definida para $0 \leq m \leq M$, un conjunto de arcos A y un valor D del ADT

disjoint-union sobre V , bajo tres suposiciones:

- Y_0 : $A \subseteq X[0..m)$ y A es factible;
- Y_1 : todo arco de $X[0..m)$ que no está en A cierra un ciclo con A ;
- Y_2 : las clases de D son las componentes conexas de (V, A) .

$\phi(A, m, D)$: “los arcos del árbol de cubrimiento mínimo que resulta de extender A con arcos de $X[m..M)$, según el criterio del Algoritmo 4.7.2.”

Para resolver el problema del árbol de cubrimiento mínimo, el objetivo es calcular $\phi(\emptyset, 0, \text{make}(V))$, llamado en el cual las tres suposiciones se cumplen trivialmente.

La definición recurrente de ϕ se establece tomando a m como pivote y preguntando, por medio del ADT, si el arco $X[m] = (u, v, w)$ cierra un ciclo con A . Para aligerar la notación se escriben $r_u = \text{find}(u)$ y $r_v = \text{find}(v)$ para los representantes canónicos de u y v en D , y $\text{end}(A, m)$ para la condición $m = M \vee |A| = N - 1$:

$$\phi(A, m, D) = \begin{cases} A & , \text{ si } \text{end}(A, m), \\ \phi(A, m+1, D) & , \text{ si } \neg \text{end}(A, m) \wedge r_u = r_v, \\ \phi(A \cup \{X[m]\}, m+1, \text{union}(u, v)) & , \text{ si } \neg \text{end}(A, m) \wedge r_u \neq r_v. \end{cases}$$

El caso base cubre dos situaciones distintas y conviene distinguirlas. La primera, $m = M$, es la habitual: no quedan arcos por examinar. La segunda, $|A| = N - 1$, es una parada temprana: por el Ejercicio 2, un conjunto factible de ese tamaño es ya el conjunto de arcos de un árbol de cubrimiento, de modo que ningún arco restante puede agregarse y recorrer lo que falta de $X[m..M)$ sería trabajo perdido. El primer caso inductivo corresponde a la situación en la cual u y v ya están en la misma componente conexa de (V, A) : el arco cerraría un ciclo, se descarta y se recurre sin modificar nada. El segundo caso inductivo corresponde a la situación en la cual u y v están en componentes distintas: el arco se agrega a A y, para que la suposición Y_2 siga cumpliéndose, hay que fusionar las dos clases correspondientes de D .

Nota 4.7.10: Las suposiciones son los invariantes

Las tres suposiciones sobre los parámetros de ϕ no son decoración. La suposición Y_0 es la que hace correcto al algoritmo y es el invariante del Teorema 4.7.2. La suposición Y_1 es la que permite mirar únicamente a $X[m]$ en lugar de recorrer todos los arcos disponibles. Y la suposición Y_2 es la que hace correcta la consulta al ADT: sin ella, comparar r_u con r_v no diría nada sobre ciclos.

Al transcribir ϕ a una versión iterativa, las tres se convierten en los invariantes del ciclo (Ejercicio 13). Esta correspondencia entre las suposiciones

de una función objetivo y los invariantes del ciclo que la calcula es general y conviene tenerla presente.

Teorema 4.7.8

Sea $G = (V, E)$ conexo con función de peso w y suponga que $X[0..M)$ contiene los arcos de E , con sus pesos, ordenados ascendentemente por w . Entonces $\phi(\emptyset, 0, \text{make}(V))$ es el conjunto de arcos de un MST de G .

Demostración

Basta con verificar que la secuencia de arcos que ϕ agrega a A es una ejecución del Algoritmo 4.7.2, pues entonces el Teorema 4.7.5 da la conclusión.

Las tres suposiciones se cumplen en el llamado inicial y se preservan en cada paso: Y_0 por el Teorema 4.7.3, que garantiza que el arco agregado es seguro; Y_1 porque el arco $X[m]$ se descarta únicamente cuando cierra un ciclo; y Y_2 por el llamado a *union*.

Por Y_2 , la condición $r_u \neq r_v$ equivale a que $X[m]$ no cierre un ciclo con A ; y por Y_1 y el ordenamiento de $X[0..M)$, según se argumentó arriba, ese arco es de peso mínimo entre los que no cierran un ciclo. Es decir, cada arco que ϕ agrega es uno de los que el criterio del Algoritmo 4.7.2 habría escogido.

Las dos maneras de terminar corresponden con la guarda del paso 2 del Algoritmo 4.7.1. Si ϕ se detiene porque $|A| = N - 1$, la guarda dejó de cumplirse y el algoritmo habría terminado también. Y si se detiene porque $m = M$ con $|A| < N - 1$, entonces todo arco de E cierra un ciclo con A y, por el Teorema 4.7.4, el grafo G no era conexo, contrario a la suposición.

Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación.

Algoritmo 4.7.4

```

1 def kruskal(X, N, M, A, m, D):
2     # X[0..M) holds edges (u, v, w) sorted by weight
3     ans = None
4     if m==M or len(A)==N-1: ans = A
5     else:
6         u, v, w = X[m]
```

```

7     if dfind(D, u)!=dfind(D, v):
8         A.append(X[m])
9         dunion(D, u, v)
10    ans = kruskal(X, N, M, A, m+1, D)
11    return ans
12
13 def mst_kruskal(X, N, M):
14     X.sort(key = lambda x: x[2])
15     return kruskal(X, N, M, [], 0, dmake(N))

```

Nota 4.7.11: Estado compartido

La definición de ϕ trata a A y a D como valores: el segundo caso inductivo recurre con $A \cup \{X[m]\}$ y con $union(u, v)$, que son objetos nuevos. El Algoritmo 4.7.4, en cambio, modifica en sitio la lista y el *dforest*, y los pasa a la recurrencia tal cual.

Las dos versiones coinciden porque ningún llamado vuelve sobre un estado anterior: la recurrencia avanza siempre de m a $m + 1$ y nunca deshace una decisión. Copiar A en cada paso sería correcto, pero costaría tiempo $O(N)$ por llamado y arruinaría la complejidad que se establece a continuación.

Ejemplo 4.7.3

Considere el grafo del Ejemplo 4.7.2, con los vértices $a, \dots, f$ identificados con los enteros $0, \dots, 5$:

```

1 X = [ (0,1,4), (1,2,8), (0,3,1), (1,4,6),
2       (2,5,2), (3,4,7), (4,5,3), (2,4,5) ]

```

El llamado a `mst_kruskal` produce los cinco arcos del MST, en el orden en el que fueron seleccionados:

```

1 >>> mst_kruskal(X, 6, len(X))
2 [(0, 3, 1), (2, 5, 2), (4, 5, 3), (0, 1, 4), (1, 4, 6)]

```

El resultado coincide con la fila superior del Ejemplo 4.7.2, incluido el descarte del arco $(2, 4, 5)$ —el arco (c, e) de peso 5— que es el único que la recurrencia examina sin agregar.

Teorema 4.7.9

Sea $G = (V, E)$ conexo con función de peso w , $|V| = N$ y $|E| = M$, representado por la lista de arcos $X[0..M)$. El llamado `mst_kruskal(X, N, M)` calcula los arcos de un MST de G y toma tiempo $O(M \log M)$ y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$.

Demostración

La corrección es el Teorema 4.7.8: al ordenar $X[0..M)$ ascendentemente por peso, `mst_kruskal` establece la hipótesis que ese teorema requiere y el resto es el cálculo de $\phi(\emptyset, 0, \text{make}(V))$.

El ordenamiento toma tiempo $O(M \log M)$ y el llamado a `dmake` toma tiempo $O(N)$. La recurrencia hace a lo sumo $M + 1$ llamados –menos, si la parada temprana se activa– y cada uno ejecuta a lo sumo tres operaciones del ADT, cada una en tiempo $O(\log N)$ por el Teorema 4.7.7, más una cantidad constante de trabajo adicional; la recurrencia toma entonces tiempo $O(M \log N)$. Como G es conexo se tiene $N - 1 \leq M$ y, por lo tanto, $\log N \in O(\log M)$: el costo de la recurrencia es $O(M \log M)$ y el total también.

El espacio adicional es el de los dos arreglos del *dforest*, que es $O(N)$, más el de la respuesta, que tiene $N - 1$ arcos. Teniendo en cuenta el espacio de la pila, el espacio es $O(M)$ debido a los llamados recurrentes.

El resultado confirma lo que anticipaba la Nota 4.7.9: el costo queda dominado por el ordenamiento y no por la estructura de datos. También ilustra el Teorema 4.1.2 con un caso en el cual la verificación de factibilidad no es gratuita: acá $O(f) = O(\log N)$ y no $O(1)$ como en las secciones anteriores; aun así, el ordenamiento se impone.

4.7.5. El algoritmo de Prim. El segundo algoritmo concreta el esquema con un criterio de selección distinto. En lugar de recorrer todos los arcos del grafo en orden de peso, mantiene un conjunto S de vértices ya conectados y se pregunta, en cada paso, cuál es el arco más liviano que sale de S .

Algoritmo 4.7.5: Prim

El Algoritmo 4.7.1 con el siguiente criterio de selección, a partir de un vértice inicial $r \in V$ y de $S = \{r\}$: escoger un arco ligero para el corte $(S, V \setminus S)$ y agregar a S el extremo del arco que aún no estaba allí.

Este criterio también se puede evaluar, pues solo menciona los pesos de los arcos y el conjunto S , que el algoritmo conoce. Las dos obligaciones de la Nota 4.7.6 son, en este caso, casi inmediatas.

Teorema 4.7.10: Optimización local para Prim

Sea $S \subseteq V$ con $S \neq \emptyset$ y $S \neq V$, y sea $A \subseteq E$ factible tal que todo arco de A tiene sus dos extremos en S . Si (u, v) es un arco ligero para el corte $(S, V \setminus S)$, entonces (u, v) es seguro para A .

Demostración

La pareja $(S, V \setminus S)$ es un corte de V , pues ambas partes son no vacías por suposición. El corte respeta a A , pues ningún arco con sus dos extremos en S puede cruzarlo. Y (u, v) es ligero para él, también por suposición. El Teorema 4.7.1 da la conclusión.

La brevedad de la demostración anterior no debe engañar: es la que tiene que ser, porque el criterio de Prim está enunciado directamente en términos de un corte y no hay nada que construir. Toda la dificultad la absorbió el Teorema 4.7.1.

Queda la segunda obligación, que es garantizar que el criterio siempre tenga algo que escoger. Es igual de directa.

Teorema 4.7.11: Existencia

Sea $G = (V, E)$ conexo y $S \subseteq V$ con $S \neq \emptyset$ y $S \neq V$. Entonces, hay al menos un arco de E que cruza el corte $(S, V \setminus S)$.

Demostración

De lo contrario, ningún camino de G podría ir de un vértice de S a uno de $V \setminus S$ y el grafo no sería conexo.

Con las dos obligaciones cumplidas, la corrección del algoritmo se obtiene invocando el teorema del esquema genérico. Lo único que hay que verificar es que las

hipótesis de los dos resultados anteriores se cumplen en cada iteración y para eso basta observar que el conjunto S crece exactamente en un vértice por cada arco seleccionado.

Teorema 4.7.12: Optimización global para Prim

El Algoritmo 4.7.5 calcula los arcos de un MST de G .

Demostración

Mientras la guarda del paso 2 se cumpla, se tiene $|A| < |V| - 1$ y, como el algoritmo agrega un vértice a S por cada arco que selecciona, se tiene $S \neq V$; por el Teorema 4.7.11 hay entonces algún arco que cruza el corte y el criterio tiene entre cuáles escoger. Todo arco seleccionado tiene sus dos extremos en S al momento de agregarse, de modo que el Teorema 4.7.10 garantiza que es seguro. Se satisface entonces la hipótesis del Teorema 4.7.2.

Nota 4.7.12: El mismo teorema, dos veces

Vale la pena comparar las demostraciones de los dos teoremas de optimización local. Ambas consisten en exhibir un corte que respete a A y para el cual el arco escogido sea ligero, y en las dos el trabajo está en encontrar ese corte.

En Prim el corte es explícito: es $(S, V \setminus S)$, el algoritmo lo mantiene y el criterio está enunciado en términos de él. En Kruskal está oculto: el criterio no lo menciona y la demostración del Teorema 4.7.3 tiene que fabricarlo a partir de la componente conexa del arco escogido. Esa es toda la diferencia entre los dos algoritmos y explica por qué Kruskal construye un bosque mientras Prim construye un árbol: Kruskal usa un corte distinto en cada paso y Prim usa siempre el mismo, que crece.

Representación del grafo. A diferencia de Kruskal, el criterio de Prim pregunta repetidamente por los arcos incidentes a un vértice y no recorre la colección de arcos en un orden fijo. Para este algoritmo conviene entonces representar el grafo con *listas de adyacencia*: un arreglo $G[0..N]$ en el cual $G[u]$ es la lista de las parejas (v, w) tales que (u, v) es un arco de peso w . Los vértices se identifican con los enteros en $[0..N]$ y, como el grafo es no dirigido, cada arco aparece dos veces: la pareja (v, w) en $G[u]$ y la pareja (u, w) en $G[v]$. En total, las listas ocupan espacio $O(N + M)$ y recorrerlas todas toma tiempo $O(N + M)$.

Note que el criterio es *dinámico* en el sentido de la Nota 4.1.3: cuál es el arco más liviano que sale de S depende de S , que cambia en cada paso. No aplica entonces la receta de *ordenar y recorrer* que sirvió en todas las secciones anteriores, y hace falta una estructura que entregue el mínimo y admita modificaciones.

Un tipo abstracto de datos. Antes de buscar esa estructura conviene precisar qué se le va a pedir. La observación que lo permite es la siguiente: en lugar de guardar los arcos que cruzan el corte, basta con guardar, para cada vértice $v \notin S$, el peso del arco más liviano que lo conecta con S . Ese número se llamará la *llave* de v y vale $+\infty$ cuando ningún arco conecta a v con S . El arco ligero del corte es entonces el que corresponde al vértice de llave mínima y al mover un vértice u de $V \setminus S$ a S solo hay que revisar si los arcos incidentes a u mejoran alguna llave.

Definición 4.7.5: El tipo abstracto minheap

Sea U un conjunto finito con $|U| = N$. El tipo abstracto de datos *minheap* sobre U mantiene un subconjunto $Q \subseteq U$ y una llave $key(x) \in \mathbb{R} \cup \{+\infty\}$ para cada $x \in Q$, y ofrece cuatro operaciones:

- *make*(U) construye el estado con $Q = U$ y $key(x) = +\infty$ para todo $x \in U$, en tiempo $O(N)$;
- *empty*() retorna un valor Booleano que indica si $Q = \emptyset$, en tiempo $O(1)$;
- *extract*() remueve de Q un elemento x de llave mínima y retorna la pareja $(x, key(x))$, en tiempo $O(\log N)$;
- *decrease*(x, k) reemplaza $key(x)$ por k y retorna verdadero cuando $x \in Q$ y $k < key(x)$; en cualquier otro caso no modifica nada y retorna falso, en tiempo $O(\log N)$.

Que *extract* retorne la llave junto con el elemento no es un capricho: esa llave es el peso del arco que se va a agregar a la solución y sin ella habría que consultarlo aparte.

Una estructura de datos: *bheap*. La estructura que implementa el ADT anterior es el *montón binario*. Consiste en mantener los elementos de Q en un árbol binario *casi completo* –todos los niveles llenos salvo quizás el último, que se llena de izquierda a derecha– sujeto a la condición de que la llave de cada nodo no supere la de sus hijos. Bajo esa condición, el elemento de llave mínima está siempre en la raíz.

Un árbol de esa forma no necesita apuntadores: se guarda en un arreglo $h[0..K)$ en el cual la raíz es $h[0]$ y los hijos de $h[i]$ son $h[2i + 1]$ y $h[2i + 2]$, cuando esos índices existen. La estructura completa consta de cuatro campos: ese arreglo, un

arreglo $key[0..N)$ con las llaves, un arreglo $pos[0..N)$ que indica en qué posición de h está cada elemento –o -1 si ya no está en Q – y el tamaño K de Q .

Esa correspondencia entre posiciones del arreglo y nodos del árbol es la única aritmética que la estructura necesita, y conviene aislarla en funciones propias para que el resto del código pueda escribirse hablando de padres y de hijos en lugar de sumas y productos.

Nota 4.7.13: ¿Para qué sirve el arreglo de posiciones?

El arreglo $pos[0..N)$ es la parte del diseño que no es evidente y conviene entender por qué está ahí.

La operación $decrease(x, k)$ tiene que modificar la llave de un elemento x dado y para eso necesita saber en qué posición del montón se encuentra. Sin $pos[0..N)$ habría que buscarlo recorriendo $h[0..K)$, lo cual costaría $O(N)$ y haría imposible cumplir la cota de la Definición 4.7.5. Mantenerlo cuesta poco: cada vez que dos elementos intercambian posiciones en h , se intercambian también sus entradas en pos .

Esta es la diferencia entre un montón binario y una simple cola de prioridad de las que solo insertan y extraen. La operación $decrease$ obliga a poder *entrar* al montón por un elemento cualquiera y no solo por la raíz.

Algoritmo 4.7.6: La estructura bheap

```

1 INF = float("inf")
2
3 def bis_root(i): return i==0
4
5 def bparent(i): return (i-1)>>1
6
7 def bleft(i): return 1+(i<<1)
8
9 def bright(i): return (1+i)<<1
10
11 def bswap(H, i, j):
12     h, pos = H[0], H[2]
13     h[i], h[j] = h[j], h[i]
14     pos[h[i]], pos[h[j]] = i, j
15
16 def bup(H, i):
```



```

17 if not(bis_root(i)):
18     h, key, p = H[0], H[1], bparent(i)
19     if key[h[p]]>key[h[i]]:
20         bswap(H, i, p)
21         bup(H, p)
22
23 def bdown(H, i):
24     h, key, K = H[0], H[1], H[3]
25     l, r, b = bleft(i), bright(i), i
26     if l<K and key[h[l]]<key[h[b]]: b = l
27     if r<K and key[h[r]]<key[h[b]]: b = r
28     if b!=i:
29         bswap(H, i, b)
30         bdown(H, b)

```

Las cuatro primeras funciones son la traducción entre las dos maneras de ver la estructura. La función `bis_root` responde si una posición es la raíz; `bparent`, `bleft` y `bright` entregan las posiciones del padre y de los dos hijos. Están escritas con corrimientos de bits $-i \ll 1$ es $2i$ y $(i-1) \gg 1$ es $\lfloor (i-1)/2 \rfloor$ — porque es la manera habitual de multiplicar y dividir por dos, pero lo que importa no es esa economía sino que, una vez definidas, ningún otro lugar del código vuelve a mencionar un índice calculado.

Note que `bparent` solo tiene sentido cuando la posición no es la raíz, razón por la cual `bup` consulta primero a `bis_root` y únicamente después calcula el padre (Ejercicio 17).

Las funciones `bup` y `bdown` restauran la condición del montón cuando una sola llave se ha alterado y las dos son recurrentes. La primera compara la llave del elemento con la de su padre y, si es menor, los intercambia y vuelve a llamarse sobre la posición del padre; se detiene al llegar a la raíz o al encontrar un padre que ya no la supera. La segunda escoge, entre la posición dada y las de sus hijos que existan, la de llave mínima. Si no es la propia, intercambia y vuelve a llamarse sobre ella; se detiene cuando el elemento ya no supera a ninguno de sus hijos. En ambos casos el llamado recurrente es la última instrucción que se ejecuta, de modo que las dos son recursivas de cola y se pueden convertir en ciclos sin esfuerzo (Ejercicio 18). Con ellas, las cuatro operaciones del ADT se escriben de inmediato.

Algoritmo 4.7.7: Las operaciones sobre un bheap

```

1 def bmake(N):
2     return [ list(range(N)), [INF]*N, list(range(N)), N ]
3

```

```

4 def bempty(H):
5     return H[3]==0
6
7 def bextract(H):
8     h, key, pos = H[0], H[1], H[2]
9     u = h[0]
10    H[3] = H[3]-1
11    bswap(H, 0, H[3])
12    pos[u] = -1
13    bdown(H, 0)
14    return u, key[u]
15
16 def bdecrease(H, x, k):
17     key, pos = H[1], H[2]
18     ans = pos[x]>=0 and k<key[x]
19     if ans:
20         key[x] = k
21         bup(H, pos[x])
22     return ans

```

La función `bmake` no tiene que hacer ningún trabajo de reacomodo porque todas las llaves valen $+\infty$ y la condición del montón se cumple trivialmente. La función `bextract` toma la raíz, trae el último elemento a ocupar su lugar y lo hace bajar. La función `bdecrease` baja la llave y hace subir al elemento desde la posición que le indica `pos`.

Teorema 4.7.13

Un montón binario con $K \geq 1$ elementos tiene altura a lo sumo $\log_2 K$.

Demostración

Considere un nodo a profundidad d , es decir, uno al cual se llega desde la raíz siguiendo d arcos. Como los hijos de $h[i]$ están en las posiciones $2i+1$ y $2i+2$, la posición de un nodo a profundidad d es al menos $2^d - 1$ (¿por qué?). Como las posiciones ocupadas son las de $[0..K)$, se tiene $2^d - 1 \leq K - 1$; es decir, $d \leq \log_2 K$.

Teorema 4.7.14

El Algoritmo 4.7.7 implementa el ADT *minheap* de la Definición 4.7.5 sobre $U = [0..N)$.

Demostración

Que las operaciones se comportan como exige la Definición 4.7.5 se propone como ejercicio al lector (Ejercicio 16). Para las restricciones de complejidad:

- **bmake** construye tres arreglos de tamaño N en tiempo $O(N)$;
- **bempty** consulta un campo, en tiempo $O(1)$;
- **bextract** hace una cantidad constante de trabajo y un llamado a **bdown**. Cada llamado a **bdown** hace una cantidad constante de trabajo y, cuando recurre, lo hace sobre la posición de un hijo, es decir, un nivel más abajo. Por el Teorema 4.7.13 la cadena de llamados tiene longitud a lo sumo $\log_2 N$. En total, $O(\log N)$;
- **bdecrease** hace una cantidad constante de trabajo y un llamado a **bup**, que por la misma razón produce una cadena de longitud a lo sumo $\log_2 N$; esta vez cada llamado recurre sobre la posición del padre, un nivel más arriba. En total, $O(\log N)$.

Note, por último, que **bup** y **bdown** consumen espacio de pila proporcional a la longitud de su cadena de llamados, es decir, $O(\log N)$. Es una cantidad despreciable frente a los $O(N)$ que ocupan los tres arreglos de la estructura, y desaparece del todo si se las escribe iterativamente.

Nota 4.7.14: Una estructura más simple

El montón binario no es la única manera de resolver el problema y no siempre es la mejor. La alternativa más simple consiste en guardar las llaves en un arreglo y no hacer ningún esfuerzo por mantenerlas ordenadas: entonces *decrease* toma tiempo $O(1)$, pero *extract* tiene que recorrer el arreglo completo y toma tiempo $O(N)$.

Esa estructura no cumple la Definición 4.7.5, de modo que no implementa el ADT y no puede sustituirse por el Algoritmo 4.7.7 sin más. Pero sería un error descartarla: como se verá en la Sección 4.7 más adelante, combinada con una matriz de pesos produce una versión de Prim que le gana al montón binario cuando el grafo es denso. El Ejercicio 21 propone desarrollarla.

La implementación. Escribáse $N = |V|$. El estado del algoritmo consta de cuatro piezas: el conjunto A de arcos ya seleccionados, el conjunto Q de los vértices que aún no se han extraído, un valor H del ADT *minheap* que asigna una llave a cada elemento de Q y un arreglo $P[0..N)$ que registra, para cada vértice, el vecino que realiza su llave. Como es usual, S denota el conjunto $[0..N) \setminus Q$ de los vértices ya extraídos.

El conjunto Q queda determinado por H , pues es su primera componente según la Definición 4.7.5. Se lleva de todos modos como un parámetro aparte porque es lo que cambia de un llamado al siguiente y porque las suposiciones que vienen a continuación se enuncian en términos suyos.

Considere la siguiente función objetivo, bajo tres suposiciones:

- Z_0 : A es factible y todo arco de A tiene sus dos extremos en S ;
- Z_1 : (S, A) es un árbol;
- Z_2 : para todo $v \in Q$, $key(v) = (\downarrow x \mid x \in S \wedge (x, v) \in E : w(x, v))$ y, cuando ese valor es finito, $P[v]$ es un vértice de S que lo alcanza.

$\phi(A, Q, H, P)$: “los arcos del árbol de cubrimiento mínimo que resulta de extender A con arcos incidentes a S , según el criterio del Algoritmo 4.7.5.”

Para resolver el problema del árbol de cubrimiento mínimo desde un vértice inicial r , el objetivo es calcular $\phi(\emptyset, V, decrease(r, 0), P_0)$, donde $P_0[v] = v$ para todo v . En ese llamado $S = \emptyset$ y las suposiciones Z_0 y Z_1 se cumplen trivialmente; la suposición Z_2 es la única que no y el efecto de haber bajado a 0 la llave de r es precisamente que r sea el primer vértice extraído, con lo cual Z_2 pasa a cumplirse desde entonces.

La definición recurrente de ϕ se establece tomando como pivote el vértice de llave mínima. Escribanse $(u, k) = extract()$ para ese vértice y su llave, H' para el estado del *minheap* después de extraerlo y Q' para $Q \setminus \{u\}$. La relajación de u modifica dos de los cuatro parámetros a la vez, de modo que se escribe:

$$(H'', P') = relax(u, H', P)$$

para la pareja que resulta de aplicar $decrease(v, w)$ a toda pareja (v, w) en $G[u]$, actualizando $P[v] = u$ cada vez que la operación retorne verdadero. Escribáse, por último, a_u para el arco $(P[u], u, k)$, que es el que se agrega a la solución cuando u no es el vértice inicial. Con esa notación:

$$\phi(A, Q, H, P) = \begin{cases} A & , \text{ si } empty(Q), \\ \phi(A, Q', H'', P') & , \text{ si } \neg empty(Q) \wedge P[u] = u, \\ \phi(A \cup \{a_u\}, Q', H'', P') & , \text{ si } \neg empty(Q) \wedge P[u] \neq u. \end{cases}$$

El caso base corresponde a la situación en la cual no quedan vértices por agregar a S . El primer caso inductivo corresponde al vértice inicial: es el único que se extrae sin que ningún arco lo haya conectado con S , lo cual se reconoce en que $P[u]$ sigue valiendo u . El segundo caso inductivo es el habitual: el arco $(P[u], u)$ es un arco ligero del corte, su peso es la llave k con la cual u fue extraído y se agrega a la solución. En ambos casos hay que relajar, para que la suposición Z_2 siga cumpliéndose tras haber crecido S con u .

Nota 4.7.15: ¿Por qué el mínimo de las llaves es un arco ligero?

El criterio del Algoritmo 4.7.5 pide un arco ligero para el corte $(S, V \setminus S)$ y la recurrencia extrae un vértice de llave mínima. Que las dos cosas coincidan es consecuencia de la suposición Z_2 .

En efecto, por Z_2 la llave de cada $v \in Q$ es el peso del arco más liviano entre v y S , de modo que el mínimo de las llaves sobre Q es el peso del arco más liviano que cruza el corte. Si u alcanza ese mínimo, entonces $(P[u], u)$ es un arco de ese peso que cruza el corte, es decir, un arco ligero.

Teorema 4.7.15

Sea $G = (V, E)$ conexo con $|V| = N$, representado por sus listas de adyacencia, y sea $r \in V$. Entonces $\phi(\emptyset, V, decrease(r, 0), P_0)$ es el conjunto de arcos de un MST de G .

Demostración

Basta con verificar que la secuencia de arcos que ϕ agrega a A es una ejecución del Algoritmo 4.7.5, pues entonces el Teorema 4.7.12 da la conclusión.

Tras la primera extracción se tiene $S = \{r\}$ y las tres suposiciones se cumplen. Se preservan en cada paso: Z_0 porque el arco agregado tiene un extremo en S y el otro es u , que pasa a estar en S ; Z_1 porque ese arco conecta u con el árbol ya construido sin crear ciclos, dado que $u \notin S$ antes de extraerlo; y Z_2 por el llamado a *relax*, que recorre $G[u]$ y por lo tanto revisa todos los arcos incidentes al único vértice que se agregó a S .

Por la Nota 4.7.15, el arco $(P[u], u)$ que se agrega en el segundo caso inductivo es un arco ligero del corte $(S, V \setminus S)$, que es exactamente lo que el criterio del Algoritmo 4.7.5 escoge.

Falta ver que el primer caso inductivo ocurre únicamente en la primera extracción. Como G es conexo y $S \neq \emptyset$ después de ella, el Teorema 4.7.11 garantiza que algún arco cruza el corte; por Z_2 , el vértice extraído tiene entonces llave finita y $P[u] \neq u$.

Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación.

Algoritmo 4.7.8

```

1 def prim(G, N, A, H, P):
2     # G[u] is the list of pairs (v, w) for the edges out of u
3     ans = None
4     if bempty(H): ans = A
5     else:
6         u, k = bextract(H)
7         if P[u] != u: A.append((P[u], u, k))
8         for v, w in G[u]:
9             if bdecrease(H, v, w): P[v] = u
10        ans = prim(G, N, A, H, P)
11    return ans
12
13 def mst_prim(G, N, r):
14     H = bmake(N)
15     bdecrease(H, r, 0)
16    return prim(G, N, [], H, list(range(N)))

```

El ciclo de la línea 8 es la operación *relax*: recorre la lista de adyacencia de u y deja que el ADT decida, en cada vecino, si la llave mejora. Note que no hace falta preguntar si v sigue en Q ni comparar llaves a mano, porque `bdecrease` ya hace las dos cosas y reporta el resultado. Como en el Algoritmo 4.7.4, la lista A y el estado H se modifican en sitio y se pasan a la recurrencia tal cual, lo cual es correcto por la razón que da la Nota 4.7.11.

Note también que el conjunto Q , que la definición de ϕ lleva como parámetro, no aparece en el código: vive dentro de H , que es precisamente lo que la estructura de datos oculta. Por eso `bempty` responde sin recibirlo aparte y `bextract` lo reduce en uno como efecto de su llamado.

Ejemplo 4.7.4

Considere el grafo del Ejemplo 4.7.2, con los vértices $a, \dots, f$ identificados con los enteros $0, \dots, 5$. Sus listas de adyacencia se construyen a partir de la lista de arcos:

```

1 >>> N = 6
2 >>> G = [ [] for u in range(N) ]
3 >>> for u, v, w in X: G[u].append((v,w)); G[v].append((u,w))
4 >>> G[4]
5 [(1, 6), (3, 7), (5, 3), (2, 5)]

```

El llamado a `mst_prim` desde el vértice a produce los cinco arcos del MST, en el orden en el que fueron seleccionados y orientados del vértice que ya estaba en S hacia el que se agrega:

```

1 >>> mst_prim(G, N, 0)
2 [(0, 3, 1), (0, 1, 4), (1, 4, 6), (4, 5, 3), (5, 2, 2)]

```

El resultado coincide con la fila inferior del Ejemplo 4.7.2. Compárelo con el del Ejemplo 4.7.3: el conjunto de arcos es el mismo, pero el orden en el que aparecen no lo es y, a diferencia de Kruskal, sus pesos no son ascendentes.

Teorema 4.7.16

Sea $G = (V, E)$ conexo con $|V| = N$ y $|E| = M$, representado por sus listas de adyacencia. El llamado `mst_prim(G, N, r)` calcula los arcos de un MST de G y toma tiempo $O((N + M) \log N)$ y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$.

Demostración

La corrección es el Teorema 4.7.15.

El llamado a `bmake` toma tiempo $O(N)$. La recurrencia hace $N + 1$ llamados, pues cada uno extrae un vértice y hay N de ellos. Cada llamado ejecuta una operación *extract*, en tiempo $O(\log N)$ por el Teorema 4.7.14; en total, $O(N \log N)$.

El ciclo de relajación recorre $G[u]$ una vez por cada vértice u y hace una operación *decrease* por cada pareja. Como cada arco aparece dos veces entre todas las listas de adyacencia, la cantidad total de operaciones *decrease* es $2M$ y el tiempo que consumen es $O(M \log N)$. Recorrer las listas cuesta, además, $O(N + M)$.

Sumando, el tiempo total es $O(N + N \log N + M \log N + N + M) = O((N + M) \log N)$.

El espacio adicional es el de los tres arreglos del *bheap* y el del arreglo $P[0..N]$, que es $O(N)$, más el de la respuesta, que tiene $N - 1$ arcos.

Como G es conexo se tiene $N - 1 \leq M$ y, por lo tanto, la cota se puede escribir simplemente como $O(M \log N)$. Vale la pena notar dónde quedó cada término: el $N \log N$ viene de las extracciones y el $M \log N$ de las relajaciones. Es el segundo el que domina y es también el que explica por qué la representación importa tanto como la estructura de datos: con una matriz de pesos las relajaciones serían N^2 en lugar de $2M$, y ninguna cola de prioridad podría arreglarlo.

4.7.6. ¿Cuál de los dos? Los dos algoritmos resuelven el mismo problema. Antes de compararlos conviene separar en qué difieren estructuralmente, que es lo que no depende de ninguna decisión de implementación.

	Kruskal	Prim
Criterio	el arco más liviano sin ciclo	el más liviano que sale de S
Es	estático	dinámico
Corte	uno distinto en cada paso	siempre el mismo, que crece
Construye	un bosque	un árbol

En cuanto al costo, la comparación no es entre dos algoritmos sino entre tres implementaciones, porque Prim admite dos que no son comparables entre sí. La tercera es la que propone el Ejercicio 21, con matriz de pesos y llaves en un arreglo sin ordenar.

Algoritmo	Representación	Estructura	Tiempo
Kruskal	lista de arcos	<i>dforest</i>	$O(M \log N)$
Prim	listas de adyacencia	<i>bheap</i>	$O(M \log N)$
Prim	matriz de pesos	arreglo	$O(N^2)$

La primera fila merece una aclaración: el Teorema 4.7.9 da $O(M \log M)$, pero en un grafo simple se tiene $M \leq N^2$ y por lo tanto $\log M \leq 2 \log N$, de modo que las dos expresiones son la misma. Es decir, las dos primeras implementaciones son asintóticamente equivalentes y la elección entre ellas se decide por otras razones: cuál representación del grafo ya se tiene a la mano o cuál de las dos estructuras de datos resulta más fácil de programar correctamente.

La tercera fila es la interesante. Como M está entre $N - 1$ y $\binom{N}{2}$, cuál gana depende de la densidad del grafo. Para un grafo disperso, con $M \in \Theta(N)$, las dos primeras toman tiempo $O(N \log N)$ y le ganan con holgura al $O(N^2)$ de la tercera.

Para un grafo denso, con $M \in \Theta(N^2)$, las dos primeras toman tiempo $O(N^2 \log N)$ y pierden. El punto de equilibrio está alrededor de $M \in \Theta(N^2/\log N)$.

Conviene, eso sí, no leer la tabla como si las tres columnas fueran independientes. Nada obliga a implementar Kruskal con lista de arcos ni Prim con listas de adyacencia; lo que ocurre es que cada criterio de selección hace natural una representación y esa representación hace natural una estructura de datos. Cambiar una de las tres decisiones obliga a revisar las otras dos y la tercera fila de la tabla es precisamente lo que resulta de cambiarlas todas a la vez.

Ejercicios

1. Sea $G = (V, E)$ un grafo no dirigido y conexo, y $w : E \rightarrow \mathbb{R}_{>0}$ una función de peso para los arcos; note que este ejercicio agrega la hipótesis de positividad, que el Problema 4.7.1 no impone. Demuestre que si $T = (V, E_T)$, con $E_T \subseteq E$, es un subgrafo conexo de G de costo mínimo (con respecto a w), entonces T es un árbol (de cubrimiento mínimo de G). ¿Dónde se usa la positividad? Exhiba un contraejemplo si se permite que algún peso sea 0.
2. Sea T un árbol con $N \geq 1$ vértices. Demuestre que T tiene $N - 1$ arcos.
3. Demuestre que la noción de árbol en la Definición 4.7.1 es equivalente a la siguiente definición:
Un *árbol* es un grafo en el cual existe exactamente un camino simple (i.e., sin repeticiones) entre cualquier par de vértices.
4. En el caso cuando $(u, v) \notin E_T$, en la demostración del Teorema 4.7.1, se hacen las siguientes afirmaciones. Demuestre que cada una de ellas es cierta:
 - a) El grafo $(V, E_T \cup \{(u, v)\})$ no es un árbol.
 - b) En T hay un camino simple único entre u y v .
 - c) Al menos un arco del camino entre u y v cruza (S_0, S_1) .
5. La Nota 4.7.3 distingue dos condiciones sobre un conjunto de arcos A : la fuerte, que es ser factible, y la débil, que es que cada arco de A haga parte de algún MST.
 - a) Demuestre que la primera implica la segunda.
 - b) Demuestre que un conjunto que satisface la débil puede tener un ciclo y por lo tanto no estar contenido en ningún árbol.
 - c) ¿Es invariante la condición débil bajo el paso 2(b) del Algoritmo 4.7.1? Justifique.
6. El Teorema 4.7.2 supone $V \neq \emptyset$.
 - a) ¿Qué hace el Algoritmo 4.7.1 cuando $|V| = 1$?
 - b) ¿Por qué la demostración necesita la suposición?
7. En el paso 2(a) del Algoritmo 4.7.1 se exige que el arco seguro esté en $E \setminus A$.

- a) ¿Qué parte de la demostración del Teorema 4.7.2 deja de funcionar si se omite esa exigencia?
 - b) Demuestre que la exigencia no le quita generalidad al esquema.
8. En la Definición 4.7.4, la operación $\text{union}(x, y)$ se define como la menor relación de equivalencia que contiene a $\sim$ y en la cual $x \sim y$.
 - a) Demuestre que esa relación existe y es única.
 - b) Demuestre que sus clases son las de $\sim$, salvo que las de x y y aparecen fusionadas en una sola.
9. Demuestre que las funciones del Algoritmo 4.7.3 se comportan como exige la Definición 4.7.4. *Ayuda:* formule primero el invariante que relaciona el contenido de $p[0..N)$ con la relación de equivalencia representada, y verifique que dmake lo establece y dunion lo preserva.
10. La cota del Teorema 4.7.6 es ajustada.
 - a) Construya, para cada $k \geq 0$, una secuencia de llamados a dunion sobre $N = 2^k$ elementos que produzca un árbol de altura exactamente k .
 - b) Concluya que dfind toma tiempo $\Theta(\log N)$ en el peor caso.
11. Suponga que dunion cuelga siempre r_y de r_x , sin comparar tamaños.
 - a) Exhiba una secuencia de llamados que produzca un árbol de altura $N - 1$.
 - b) ¿Qué complejidad temporal tendría entonces el Algoritmo 4.7.4?
12. Implemente la compresión de caminos descrita en la Nota 4.7.9.
 - a) Modifique dfind para que, después de encontrar la raíz, reconecte a ella todos los elementos visitados.
 - b) Verifique experimentalmente que el Algoritmo 4.7.4 no mejora de manera apreciable y explique por qué.
 - c) Diseñe un experimento en el cual la compresión de caminos sí produzca una mejora apreciable.
13. Diseñe una versión iterativa del Algoritmo 4.7.4.
 - a) Formule los invariantes del ciclo y verifique que son las suposiciones Y_0 , Y_1 y Y_2 de la función objetivo.
 - b) Enuncie y demuestre el teorema de correctitud correspondiente.
 - c) Compare el espacio adicional que usan las dos versiones. ¿Cuál es la ventaja de la iterativa y de dónde viene?
14. Modifique el Algoritmo 4.7.4 para que:
 - a) retorne únicamente el peso del MST, en espacio adicional $O(N)$;
 - b) no tenga la parada temprana $|A| = N - 1$. Mida, sobre grafos aleatorios, cuántos llamados ahorra esa condición. ¿Cambia la complejidad asintótica?
 - c) reporte que G no es conexo, en lugar de retornar un bosque.
15. Demuestre la afirmación que se hace en la demostración del Teorema 4.7.4: si (V, A) es acíclico y $|A| < |V| - 1$, entonces (V, A) no es conexo.

16. Demuestre que las funciones del Algoritmo 4.7.7 se comportan como exige la Definición 4.7.5. *Ayuda:* formule primero los invariantes que relacionan los cuatro campos de la estructura con la pareja (Q, key) ; después demuestre, por inducción sobre la longitud de la cadena de llamados, que **bup** y **bdown** restauran la condición del montón cuando una sola llave se ha alterado.
17. Las funciones **bparent**, **bleft** y **bright** del Algoritmo 4.7.6 están escritas con corrimientos de bits.
 - a) Verifique que **bleft**(*i*) vale $2i + 1$ y que **bright**(*i*) vale $2i + 2$.
 - b) Demuestre que **bparent** es inversa de las otras dos: el padre del hijo izquierdo de *i* y el del hijo derecho de *i* son ambos *i*.
 - c) ¿Qué retorna **bparent**(0)? ¿Por qué eso no causa problemas en **bup**, y qué ocurriría si el llamado a **bis_root** se hiciera después y no antes?
18. Escriba versiones iterativas de **bup** y de **bdown**.
 - a) Formule el invariante del ciclo de cada una y verifique que corresponde con la hipótesis inductiva de la demostración recurrente.
 - b) ¿Cambia alguna de las cotas del Teorema 4.7.14?
 - c) ¿Qué se gana y qué se pierde con el cambio?
19. Justifique las siguientes afirmaciones hechas sobre el Algoritmo 4.7.6:
 - a) En la demostración del Teorema 4.7.13, la posición de un nodo a profundidad *d* es al menos $2^d - 1$.
 - b) La función **bmake** no necesita reacomodar nada.
 - c) La función **bextract** supone que $Q \neq \emptyset$. ¿Por qué el Algoritmo 4.7.8 nunca la llama con $Q = \emptyset$?
20. Suponga que se elimina el arreglo **pos**[0..*N*) del Algoritmo 4.7.6 y que **bdecrease** busca el elemento recorriendo **h**[0..*K*).
 - a) ¿Qué complejidad temporal tendría **decrease**?
 - b) ¿Seguiría la estructura cumpliendo la Definición 4.7.5?
 - c) ¿Qué complejidad temporal tendría el Algoritmo 4.7.8?
21. 🌈 Desarrolle la variante de Prim que anuncia la Nota 4.7.14, con el grafo representado por su matriz de pesos $W[0..N][0..N]$, en la cual $W[u][v]$ es el peso del arco entre *u* y *v* y vale INF cuando no hay tal arco.
 - a) Implemente la estructura que guarda las llaves en un arreglo sin ordenar, y demuestre que **extract** toma tiempo $O(N)$ y **decrease** toma tiempo $O(1)$.
 - b) Explique por qué esa estructura no implementa el ADT de la Definición 4.7.5 y por qué, a pesar de eso, sirve.
 - c) Reescriba el Algoritmo 4.7.8 para que use la matriz y esa estructura, y demuestre que toma tiempo $O(N^2)$.
 - d) ¿Por qué usar la matriz de pesos con un **bheap** no produce nada mejor que $O(N^2 \log N)$?

22. Modifique el Algoritmo 4.7.8 para que retorne únicamente el peso del MST, sin construir la lista de arcos. ¿Cuánto espacio adicional ahorra?
 23. La Nota 4.7.12 sostiene que la diferencia entre los dos algoritmos está en el corte que usa cada uno.
 - a) Ejecute a mano el Algoritmo 4.7.4 sobre el grafo del Ejemplo 4.7.2 y, para cada arco seleccionado, exhiba el corte que la demostración del Teorema 4.7.3 construye.
 - b) Haga lo mismo con el Algoritmo 4.7.8.
 - c) ¿Coinciden en algún paso?
 24. El Algoritmo 4.7.5 depende de un vértice inicial r .
 - a) Demuestre que el peso del árbol que calcula no depende de cuál sea.
 - b) Exhiba un grafo y dos vértices iniciales para los cuales el *conjunto* de arcos sí sea distinto.
 25. Compare las tres implementaciones de la Sección 4.7, cuyos tiempos son $O(M \log N)$, $O(M \log N)$ y $O(N^2)$.
 - a) Determine para qué valores de M , en función de N , cada una es preferible.
 - b) Impleméntelas y compárelas experimentalmente sobre grafos aleatorios de densidad creciente. ¿Coincide el punto de equilibrio medido con el previsto?
 - c) Las dos primeras son asintóticamente equivalentes. ¿Lo son también en la práctica?
 26. Demuestre que si los pesos de los arcos de G son distintos par a par, entonces G tiene un único árbol de cubrimiento mínimo. *Ayuda:* suponga que hay dos y considere el arco de menor peso que está en uno y no en el otro.
 27. 🌈 Considere la siguiente estrategia, que procede al revés que el Algoritmo 4.7.1: comenzar con $A = E$ y remover repetidamente el arco más pesado cuya remoción no desconecte el grafo, hasta que no se pueda más.
 - a) Enuncie el resultado análogo al Teorema 4.7.1 para esta estrategia. *Ayuda:* el papel de los cortes lo cumplen los ciclos.
 - b) Demuéstrelo.
 - c) Concluya que la estrategia calcula un MST de G .
-

4.8. Caminos más cortos

La última sección del capítulo resuelve un problema de optimización sobre grafos que se parece al de la Sección 4.7 en la forma, pero no en el fondo. Dado un grafo con peso en sus arcos y un vértice de partida, se trata de determinar qué tan lejos está cada uno de los demás vértices, entendiendo la distancia como el peso total del camino más liviano que los conecta.

A diferencia de las secciones anteriores, aquí conviene trabajar con grafos *dirigidos*, pues los caminos más cortos rara vez son simétricos en las aplicaciones: una calle puede ser de un solo sentido, un vuelo puede costar distinto en cada dirección. Un grafo no dirigido es el caso particular en el cual cada arco aparece en ambos sentidos y con el mismo peso.

Definición 4.8.1

Sea $G = (V, E)$ un grafo dirigido con función de peso $w : E \rightarrow \mathbb{R}_{\geq 0}$. Un *camino* de u a v es una secuencia de vértices $u = x_0, x_1, \dots, x_k = v$, con $k \geq 0$, tal que $(x_i, x_{i+1}) \in E$ para todo $0 \leq i < k$. Su *peso* es la suma de los pesos de sus arcos.

La *distancia* de u a v , escrita $\delta(u, v)$, es el mínimo peso entre todos los caminos de u a v ; vale $+\infty$ cuando no hay ninguno.

Note que un camino de u a u de longitud $k = 0$ siempre existe y tiene peso 0, de modo que $\delta(u, u) = 0$ para todo vértice.

Problema 4.8.1: Caminos más cortos desde un origen

Entrada: Un grafo dirigido $G = (V, E)$ con función de peso $w : E \rightarrow \mathbb{R}_{\geq 0}$ y un vértice $s \in V$.

Salida: La distancia $\delta(s, v)$ para todo vértice $v \in V$.

Nota 4.8.1: Tres problemas, no uno

El Problema 4.8.1 es la variante llamada *de origen único*: se fija el vértice de partida y se pregunta por todos los destinos. Hay otras dos de uso corriente. La de *pareja única* fija también el destino y no se conoce ningún algoritmo que la resuelva asintóticamente más rápido que calcular todas las distancias desde el origen. La de *todas las parejas* pregunta por $\delta(u, v)$ para todo par de vértices y se resuelve con programación dinámica; las notas al final del capítulo indican dónde estudiarla.

Nota 4.8.2: Aquí el signo sí importa

La Nota 4.7.1 advirtió que la Sección 4.7 no usaba en ninguna parte la hipótesis sobre el signo de los pesos. Acá es al revés: la no negatividad que exige el Problema 4.8.1 es indispensable y conviene ver por qué antes de empezar.

Hay que separar dos preguntas que se confunden con facilidad, porque no tienen la misma respuesta.

La primera es si el *problema* está bien planteado, y lo que decide eso son los ciclos y no los arcos. Si hay un ciclo de peso total negativo al cual se puede llegar desde s , recorrerlo repetidamente baja el peso sin límite y $\delta(s, v)$ no existe para los vértices afectados. Mientras no haya ciclos negativos, en cambio, la distancia está perfectamente bien definida aunque haya arcos de peso negativo y hay algoritmos que la calculan (Ejercicio 3).

La segunda es si el *algoritmo* de esta sección la calcula, y lo que decide eso son los arcos y no los ciclos. Basta un solo arco de peso negativo para que falle, aunque el grafo no tenga ningún ciclo: el Ejercicio 2 exhibe una instancia de tres vértices y ningún ciclo en la cual reporta una distancia mayor que la correcta.

Es decir, la hipótesis que exige el Problema 4.8.1 es más fuerte de lo que el problema necesita y exactamente lo que el algoritmo requiere. El lugar donde la demostración la usa se señalará cuando aparezca; vale la pena estar atento, porque es un solo paso.

Leído con el vocabulario de la Sección 4.1, este problema tiene una forma algo distinta de los anteriores. Para un vértice v fijo, las *soluciones factibles* son los caminos de s a v , la *función objetivo* de uno de ellos es su peso y el *sentido* es minimizar. Lo que el problema pide, sin embargo, no es un camino, sino su peso, y no para un vértice, sino para todos a la vez. El algoritmo resolverá los $|V|$ problemas de optimización simultáneamente, aprovechando que comparten el origen.

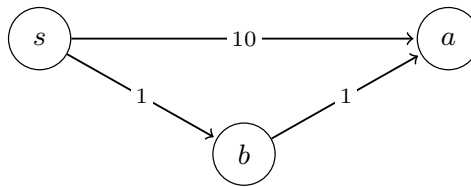
4.8.1. Escoger un criterio. La estrategia voraz mantendrá un conjunto S de vértices cuya distancia desde s ya se conoce con certeza y en cada paso agregará uno más. La pregunta es cuál.

Como en la Sección 4.7, conviene restringir de entrada los candidatos elegibles: solo tiene sentido agregar a S un vértice que esté conectado con S por algún arco. Los tres criterios siguientes respetan esa restricción. Escoger, entre los vértices que no están en S , aquel que:

1. se alcance desde S con la menor cantidad de arcos;
2. esté conectado con S por el arco más liviano; o
3. tenga la menor distancia acumulada desde s .

El segundo criterio merece atención especial, porque es exactamente el del Algoritmo 4.7.5. Si funcionara, el problema de esta sección se resolvería con el algoritmo de la anterior sin cambiarle una línea. No funciona y la misma instancia lo refuta junto con el primero.

Ejemplo 4.8.1: Dos criterios que no funcionan



Se trata de calcular las distancias desde s en el grafo dirigido de tres vértices que se dibuja arriba. Las respuestas correctas son $\delta(s, b) = 1$ y $\delta(s, a) = 2$, esta última por el camino que pasa por b .

El primer criterio agrega a S los vértices a y b de una vez, pues ambos se alcanzan desde s con un solo arco, y le asigna a a la distancia 10. Se queda corto: nunca revisa el camino de dos arcos que la mejora.

El segundo criterio agrega primero a b , cuyo arco desde s pesa 1, y luego agrega a a porque el arco (b, a) pesa 1, que es menos que los 10 del arco (s, a) . Le asigna entonces a a la distancia 1. Se pasa, porque suma mal: el peso del arco no es la distancia al origen.

Los dos errores van en direcciones opuestas y la comparación es instructiva. El primer criterio cuenta arcos y no pesos, de modo que ignora que un camino largo puede ser liviano. El segundo mira el peso del arco pero olvida lo que costó llegar hasta donde ese arco empieza. El criterio correcto es el que combina las dos cosas: lo que importa no es el peso del último arco sino la suma de todo el camino.

Nota 4.8.3: Prim y Dijkstra

El Ejemplo 4.8.1 deja ver que la diferencia entre el algoritmo de esta sección y el de la anterior es de una sola cantidad. Los dos mantienen un conjunto S que crece, los dos le asignan a cada vértice de afuera una llave y los dos extraen repetidamente el de llave mínima. Lo único que cambia es qué mide la llave:

	Prim	Dijkstra
Llave de v	$w(x, v)$	$D[x] + w(x, v)$
Minimiza	la suma de los arcos del árbol	cada camino desde s
Los pesos	pueden ser negativos	deben ser no negativos

Que dos algoritmos tan parecidos resuelvan problemas distintos, y que uno tolere pesos negativos y el otro no, es una buena razón para desconfiar de los parecidos superficiales. El Ejercicio 4 propone precisar la relación.

4.8.2. Optimización local: el voraz va adelante. Fijado el criterio, hay que demostrar que sirve. Aquí el argumento de intercambio no está disponible: una solución es un camino y no hay manera de sustituirle un vértice por otro conservando la factibilidad. Lo que sí se puede hacer es comparar directamente lo que el algoritmo afirma contra un camino arbitrario, que es el tercero de los patrones descritos en la Sección 4.3.

Teorema 4.8.1: Optimización local

Sea $S \subseteq V$ con $s \in S$ y sea D una asignación tal que $D[x] = \delta(s, x)$ para todo $x \in S$. Para $v \notin S$ defínase:

$$D[v] = (\downarrow x \mid x \in S \wedge (x, v) \in E : D[x] + w(x, v)),$$

con la convención de que el mínimo del conjunto vacío es $+\infty$. Si $u \notin S$ es tal que $D[u]$ es mínimo entre los vértices que no están en S , entonces $D[u] = \delta(s, u)$.

Demostración

Se demuestran las dos desigualdades.

Para $\delta(s, u) \leq D[u]$, note que si $D[u] = +\infty$ no hay nada que demostrar. De lo contrario, sea $x \in S$ el vértice que alcanza el mínimo, de modo que $D[u] = D[x] + w(x, u)$. Como $x \in S$ se tiene $D[x] = \delta(s, x)$ y por lo tanto hay un camino de s a x de peso $D[x]$; extendiéndolo con el arco (x, u) se obtiene un camino de s a u de peso $D[u]$. Luego $\delta(s, u) \leq D[u]$.

Para $D[u] \leq \delta(s, u)$, sea P un camino cualquiera de s a u ; basta ver que su peso no es menor que $D[u]$. Como $s \in S$ y $u \notin S$, en algún punto el camino sale de S : sea (x, y) el primer arco de P tal que $x \in S$ y $y \notin S$, y sea P_x el

tramo inicial de P , que va de s a x . Entonces:

$$\begin{aligned} w(P) &\geq w(P_x) + w(x, y) \\ &\geq \delta(s, x) + w(x, y) \\ &= D[x] + w(x, y) \\ &\geq D[y] \\ &\geq D[u]. \end{aligned}$$

La primera desigualdad descarta el tramo de P posterior al arco (x, y) ; la segunda vale porque P_x es un camino de s a x ; la igualdad porque $x \in S$; la tercera por la definición de $D[y]$, que toma el mínimo sobre todos los vértices de S y en particular sobre x ; y la última porque $y \notin S$ y $D[u]$ es mínimo entre los vértices que no están en S .

Nota 4.8.4: ¿Dónde se usa la no negatividad?

Toda la demostración anterior es aritmética elemental, salvo el primer paso: descartar el tramo de P posterior al arco (x, y) y afirmar que el peso no aumenta. Eso vale porque los arcos descartados tienen peso no negativo y es el *único* lugar de la sección donde la hipótesis se usa.

Con un arco de peso negativo el paso falla y el teorema deja de ser cierto, tal y como lo muestra el Ejercicio 2. Vale la pena notar lo poco que hace falta para romper un algoritmo: una desigualdad, en una línea de una demostración.

Note además que lo que se pierde es la corrección de *este* algoritmo y no la del problema. Como advirtió la Nota 4.8.2, con arcos negativos pero sin ciclos negativos las distancias siguen existiendo, y calcularlas exige renunciar a la estrategia voraz, y no a la especificación.

Nota 4.8.5: El voraz va adelante

El Teorema 4.8.1 es el ejemplar del tercero de los patrones catalogados en la Sección 4.3 y el único del capítulo en el cual se usa como demostración principal. La Nota 4.2.4 lo había anunciado.

Reconozca la forma. No se transforma ninguna solución óptima ni se intercambia nada; se toma un camino *arbitrario* P y se demuestra que lo que el algoritmo afirma, $D[u]$, no es peor que su peso. La medida de progreso es la asignación D restringida a S y lo que la cadena de desigualdades establece es que esa medida nunca queda por detrás de ninguna alternativa.

El intercambio no está disponible por una razón de fondo y no por falta de ingenio: en las secciones anteriores una solución era un conjunto o una asignación, objetos a los que se les puede cambiar una pieza dejando el resto intacto. Un camino no admite esa cirugía, pues cambiarle un vértice suele dejar de ser un camino.

4.8.3. Optimización global. El Teorema 4.8.1 justifica una sola extracción, y lo hace bajo una hipótesis fuerte: que D ya coincide con la distancia en todos los vértices de S . Encadenar las extracciones es entonces una inducción sobre el tamaño de S , en la cual esa hipótesis es justamente lo que hay que preservar.

Quedan dos cosas por verificar y ninguna la cubre el teorema anterior. La primera es cómo arranca la inducción, es decir, por qué el primer vértice extraído es el origen y por qué la distancia que se le asigna es la correcta. La segunda es que la hipótesis se preserva en cada paso, que es donde el teorema local hace su trabajo.

Teorema 4.8.2: Optimización global

Sea $G = (V, E)$ dirigido con pesos no negativos y $s \in V$. Repetir la extracción del Teorema 4.8.1, partiendo de $S = \emptyset$ y $D[s] = 0$, hasta que $S = V$, produce $D[v] = \delta(s, v)$ para todo $v \in V$.

Demostración

Se demuestra, por inducción sobre $|S|$, que $D[x] = \delta(s, x)$ para todo $x \in S$.

Si $S = \emptyset$ la afirmación es vacía. La primera extracción escoge a s , pues $D[s] = 0$ y ningún otro vértice tiene llave menor: los pesos son no negativos y por lo tanto ninguna llave lo es. Además $\delta(s, s) = 0$, ya que el camino de longitud cero tiene peso 0 y ningún camino puede pesar menos. Luego $D[s] = \delta(s, s)$ y la afirmación vale para $S = \{s\}$.

Si vale para S con $s \in S$ y $S \neq V$, el Teorema 4.8.1 garantiza que el vértice u que se extrae satisface $D[u] = \delta(s, u)$, de modo que sigue valiendo para $S \cup \{u\}$.

Al terminar se tiene $S = V$ y la afirmación es la conclusión del teorema.

Nota 4.8.6: Sin pasar por el esquema general

La demostración anterior no invoca el Teorema 4.1.1 y no es por descuido. Aquí una solución no es un subconjunto del conjunto de candidatos, como

pide la Definición 4.1.2, sino una asignación de distancias, de modo que el enunciado de aquel teorema no aplica literalmente; es la misma situación de la Nota 4.6.4.

Pero hay algo más y es lo que anticipó la Nota 4.2.4: cuando la optimización local se demuestra con el patrón de *el voraz va adelante*, la conclusión global sale de la propia inducción y no hace falta invocar nada. El teorema local ya comparó lo que el algoritmo produce contra una solución arbitraria; lo único que resta es encadenar las comparaciones.

4.8.4. Una implementación eficiente.

Lo que se hereda. Este algoritmo no necesita ninguna maquinaria nueva. El criterio es *dinámico* en el sentido de la Nota 4.1.3, igual que el de Prim, y las decisiones de la Sección 4.7 se aplican sin cambio alguno:

- el grafo se representa con *listas de adyacencia*, un arreglo $G[0..N]$ en el cual $G[u]$ es la lista de las parejas (v, w) tales que (u, v) es un arco de peso w ; a diferencia del caso no dirigido, ahora cada arco aparece una sola vez;
- las llaves se mantienen en el ADT *minheap* de la Definición 4.7.5, implementado por el Algoritmo 4.7.7.

Lo único que cambia respecto de Prim es el valor con el cual se relaja: donde aquel usaba el peso del arco, este usa la distancia acumulada.

La función objetivo. Escribase $N = |V|$. El estado del algoritmo consta de tres piezas: un arreglo $D[0..N]$ con las distancias ya establecidas, el conjunto Q de los vértices que aún no se han extraído y un valor H del ADT *minheap* que asigna una llave a cada elemento de Q . Como es usual, S denota el conjunto $[0..N] \setminus Q$ de los vértices ya extraídos.

El conjunto Q queda determinado por H , pues es su primera componente según la Definición 4.7.5. Se lleva de todos modos como un parámetro aparte porque es lo que cambia de un llamado al siguiente y porque las suposiciones que vienen a continuación se enuncian en términos suyos.

Considere la siguiente función objetivo, bajo dos suposiciones:

Z_0 : para todo $x \in S$, $D[x] = \delta(s, x)$;

Z_1 : para todo $v \in Q$,

$key(v) = (\downarrow x \mid x \in S \wedge (x, v) \in E : D[x] + w(x, v)).$

$\phi(D, Q, H)$: “el arreglo de distancias que resulta de establecer la de los vértices que aún están en Q .”

Escríbase D_0 para el arreglo que tiene $+\infty$ en todas sus posiciones y H_0 para el estado del *minheap* que resulta de aplicarle $decrease(s, 0)$ al recién construido. Para resolver el problema desde un origen s , el objetivo es calcular $\phi(D_0, V, H_0)$.

En ese llamado $S = \emptyset$ y la suposición Z_0 se cumple trivialmente; la suposición Z_1 es la única que no y el efecto de haber bajado a 0 la llave de s es precisamente que s sea el primer vértice extraído.

La definición recurrente de ϕ se establece tomando como pivote el vértice de llave mínima. Escribanse $(u, k) = extract()$ para ese vértice y su llave, H' para el estado del *minheap* después de extraerlo y $R = relax(u, k, H')$ para el resultado de aplicar $decrease(v, k+w)$ a toda pareja (v, w) en $G[u]$. Se escribe, además, $D[u := k]$ para el arreglo que coincide con D en todas sus posiciones salvo en la posición u , donde vale k :

$$\phi(D, Q, H) = \begin{cases} D & , \text{ si } empty(Q), \\ \phi(D[u := k], Q \setminus \{u\}, R) & , \text{ si } \neg empty(Q). \end{cases}$$

El caso base corresponde a la situación en la cual no quedan vértices por establecer. El caso inductivo registra en $D[u]$ la llave con la cual u fue extraído –que por el Teorema 4.8.1 es $\delta(s, u)$ – y relaja los arcos que salen de u , para que la suposición Z_1 siga cumpliéndose tras haber crecido S .

Nota 4.8.7: Más simple que Prim

Compare esta recurrencia con la de la Sección 4.7 y note que aquí no hay dos casos inductivos, sino uno. Allá el vértice inicial era una excepción, porque no había ningún arco que lo conectara con el árbol y por lo tanto no había arco que agregar a la solución. Acá no hay excepción que hacer: la distancia del origen a sí mismo es 0, que es exactamente la llave con la cual se extrae, y el caso general lo cubre.

Los vértices inalcanzables desde s tampoco necesitan tratamiento especial. Se extraen con llave $+\infty$, precisamente su distancia.

Teorema 4.8.3

Suponga que $G = (V, E)$ es dirigido, que sus pesos son no negativos y que $s \in V$. Entonces $\phi(D_0, V, H_0)[v] = \delta(s, v)$ para todo vértice v .

Demostración

La primera extracción escoge a s , por la razón que da la demostración del Teorema 4.8.2, y tras ella las dos suposiciones se cumplen. Se preservan en cada paso: Z_0 por el Teorema 4.8.1, que garantiza que la llave con la cual se extrae u es $\delta(s, u)$, y Z_1 por el llamado a *relax*, que recorre $G[u]$ y por lo tanto revisa todos los arcos que salen del único vértice que se agregó a S .

La secuencia de extracciones de ϕ es entonces la del Teorema 4.8.2 y su conclusión es la de este teorema.

El algoritmo. Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación.

Algoritmo 4.8.1

```

1 def dijkstra(G, N, D, H):
2     # G[u] is the list of pairs (v, w) for the arcs out of u
3     ans = None
4     if bempty(H): ans = D
5     else:
6         u, k = bextract(H)
7         D[u] = k
8         for v, w in G[u]:
9             bdecrease(H, v, k+w)
10        ans = dijkstra(G, N, D, H)
11    return ans
12
13 def sssp(G, N, s):
14     H = bmake(N)
15     bdecrease(H, s, 0)
16    return dijkstra(G, N, [INF]*N, H)

```

El ciclo de la línea 8 es la operación *relax* y es la única línea que distingue a este algoritmo del de Prim: en lugar de ofrecer al ADT el peso w del arco, le ofrece la distancia acumulada $k + w$. Note que no hace falta preguntar si v sigue en Q ni comparar llaves a mano, porque *bdecrease* ya hace las dos cosas.

Note también que el conjunto Q , que la definición de ϕ lleva como parámetro, no aparece en el código: vive dentro de H , que es precisamente lo que la estructura de datos oculta. Por eso *bempty* responde sin recibirlo aparte y *bextract* lo reduce en uno como efecto de su llamado.

Ejemplo 4.8.2

Considere el grafo dirigido con cinco vértices y siete arcos que se describe a continuación, con origen en el vértice 0:

```

1 >>> N = 5
2 >>> A = [ (0,1,4), (0,2,1), (2,1,2), (1,3,5),
3 ...      (2,3,8), (3,4,3), (2,4,12) ]
4 >>> G = [ [] for u in range(N) ]
5 >>> for u, v, w in A: G[u].append((v,w))
6 >>> sssp(G, N, 0)
7 [0, 3, 1, 8, 11]
```

Los vértices se extraen en el orden 0, 2, 1, 3, 4, con llaves 0, 1, 3, 8, 11. Vale la pena seguir lo que le ocurre al vértice 1. Al extraer el origen, su llave baja a 4 por el arco (0, 1). Al extraer el vértice 2, cuya distancia es 1, el arco (2, 1) de peso 2 ofrece la distancia acumulada $1 + 2 = 3$, que mejora la anterior. Es esa la que queda y es la correcta: el camino más corto de 0 a 1 no es el arco directo, sino el que da un rodeo por el vértice 2.

Teorema 4.8.4

Sea $G = (V, E)$ dirigido con pesos no negativos, $|V| = N$ y $|E| = M$, representado por sus listas de adyacencia. El llamado `sssp(G, N, s)` calcula $\delta(s, v)$ para todo $v \in V$, y toma tiempo $O((N + M) \log N)$ y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$.

Demostración

La corrección es el Teorema 4.8.2.

El llamado a `bmake` toma tiempo $O(N)$. La recurrencia hace $N + 1$ llamados, pues cada uno extrae un vértice y hay N de ellos. Cada llamado ejecuta una operación *extract*, en tiempo $O(\log N)$ por el Teorema 4.7.14; en total, $O(N \log N)$.

El ciclo de relajación recorre $G[u]$ una vez por cada vértice u y hace una operación *decrease* por cada pareja. Como cada arco aparece una sola vez entre todas las listas de adyacencia, la cantidad total de operaciones *decrease* es M y el tiempo que consumen es $O(M \log N)$. Recorrer las listas cuesta, además, $O(N + M)$.

Sumando, el tiempo total es $O((N + M) \log N)$. El espacio adicional es el de los tres arreglos del *bheap* y el del arreglo $D[0..N)$, que es $O(N)$.

La cota es la misma del Teorema 4.7.16, lo cual no sorprende: los dos algoritmos hacen la misma cantidad de operaciones sobre la misma estructura de datos y difieren únicamente en la aritmética de una línea. Es un buen cierre para el capítulo, porque muestra hasta qué punto la eficiencia de un algoritmo voraz depende de las decisiones de representación y de estructuras de datos, y no del criterio de selección que tanto trabajo costó demostrar.

Ejercicios

1. Justifique las siguientes afirmaciones hechas en esta sección:
 - a) $\delta(u, u) = 0$ para todo vértice u . ¿Dónde se usa que los pesos son no negativos?
 - b) En la demostración del Teorema 4.8.1, el camino P efectivamente tiene un primer arco (x, y) con $x \in S$ y $y \notin S$.
 - c) Los vértices inalcanzables desde s se extraen con llave $+\infty$.
2. Considere el grafo dirigido con vértices s , a y b y arcos (s, a) de peso 2, (s, b) de peso 3 y (b, a) de peso -2 .
 - a) Verifique que no tiene ciclos y que $\delta(s, a) = 1$.
 - b) Ejecute el Algoritmo 4.8.1 sobre él y explique qué responde.
 - c) Identifique el paso exacto de la demostración del Teorema 4.8.1 que falla, según la Nota 4.8.4.
3. 🌈 La Nota 4.8.2 distingue entre los arcos de peso negativo, que este algoritmo no tolera, y los ciclos de peso negativo, que hacen que el problema mismo deje de tener sentido.
 - a) Modifique el Algoritmo 4.8.1 para que verifique, antes de empezar, que todos los pesos son no negativos y reporte un error cuando no lo sean. ¿Cuánto cuesta la verificación?
 - b) Demuestre que si G no tiene ciclos de peso negativo alcanzables desde s , entonces para todo vértice v alcanzable hay un camino de peso $\delta(s, v)$ con a lo sumo $N - 1$ arcos.
 - c) Con base en lo anterior, diseñe un algoritmo que calcule $\delta(s, v)$ para todo v admitiendo pesos negativos y que reporte un error cuando haya un ciclo de peso negativo alcanzable desde s . *Ayuda:* relaje todos los arcos $N - 1$ veces; si una relajación más todavía mejora alguna distancia, hay un ciclo negativo.
 - d) Determine su complejidad temporal y compárela con la del Teorema 4.8.4. ¿Qué se paga por admitir pesos negativos?

4. La Nota 4.8.3 sostiene que Prim y Dijkstra difieren únicamente en qué mide la llave.
 - a) Verifique la afirmación comparando el Algoritmo 4.7.8 con el Algoritmo 4.8.1.
 - b) Explique, en términos de las dos demostraciones de optimización local, por qué Prim tolera pesos negativos y Dijkstra no.
 - c) Construya un grafo no dirigido con pesos no negativos en el cual el árbol de cubrimiento mínimo y el árbol de caminos más cortos desde un vértice sean distintos.
5. El Algoritmo 4.8.1 calcula las distancias pero no los caminos que las alcanzan.
 - a) Modifíquelo para que retorne además un arreglo $P[0..N]$ con el predecesor de cada vértice en un camino más corto desde s , como hace el Algoritmo 4.7.8.
 - b) Escriba una función que, dado ese arreglo y un vértice v , reconstruya el camino.
 - c) Demuestre que los arcos $(P[v], v)$, para v alcanzable y distinto de s , forman un árbol con raíz s .
6. Diseñe una versión iterativa del Algoritmo 4.8.1. Formule los invariantes del ciclo y verifique que son las suposiciones Z_0 y Z_1 de la función objetivo.
7. Suponga que solo interesa $\delta(s, t)$ para un vértice t fijo.
 - a) Modifique el Algoritmo 4.8.1 para que se detenga tan pronto sea posible, y justifique que la respuesta es correcta.
 - b) ¿Cambia la complejidad asintótica en el peor caso?
 - c) Mida, sobre grafos aleatorios, cuántas extracciones ahorra.
8. Suponga que todos los arcos tienen peso 1.
 - a) Demuestre que en ese caso el primero de los tres criterios de esta sección sí funciona.
 - b) Diseñe un algoritmo que resuelva el problema en tiempo $O(N+M)$, sin usar el ADT *minheap*. *Ayuda:* las llaves que se van generando nunca decrecen y difieren en a lo sumo 1.
9. Considere la variante en la cual, entre todos los caminos más cortos de s a v , se pide uno con la menor cantidad de arcos.
 - a) Proponga una manera de resolverla modificando el criterio de selección.
 - b) Enuncie y demuestre el teorema de optimización local correspondiente.
10. Un vértice puede ser extraído con llave $+\infty$ únicamente cuando es inalcanzable desde s .
 - a) Demuéstrelo.
 - b) Modifique el Algoritmo 4.8.1 para que se detenga en cuanto extraiga uno así, y explique por qué es correcto.

11. 🌈 Considere la variante del problema en la cual el peso de un camino no es la suma de los pesos de sus arcos sino el máximo, y se pide minimizarlo; es decir, se busca el camino cuyo arco más pesado sea lo más liviano posible.
- Adapte el Teorema 4.8.1 y demuéstrela.
 - ¿Hace falta suponer que los pesos son no negativos? Justifique con cuidado.
 - Modifique el Algoritmo 4.8.1 para que la resuelva.
 - Relacione la respuesta con el árbol de cubrimiento mínimo de la Sección 4.7.
-

Notas del capítulo y referencias

El planteamiento de los algoritmos voraces en este capítulo es cercano al de T. Cormen et al. [CLRS22] y al de J. Kleinberg y E. Tardos [KET06]. De estos últimos proviene el nombre del tercero de los patrones de demostración de la Sección 4.3: lo que aquí se llama *el voraz va adelante* aparece allá como *greedy stays ahead*, y es también en ese texto donde el argumento de intercambio se presenta como una técnica con nombre propio y no como un truco ocasional. J. Erickson [Eri19] insiste, más que los anteriores, en la fragilidad de los criterios voraces y en la disciplina de refutarlos antes de intentar demostrarlos; las tres referencias son fuentes abundantes de ejercicios. El Ejercicio 14 de la Sección 4.2 es una versión reducida de [Eri19, Problema 4.1].

Los algoritmos de este capítulo son considerablemente más antiguos que la idea de estudiarlos como una familia, tal y como lo anunció la Nota 4.0.3. El primer algoritmo conocido para el árbol de cubrimiento mínimo es el de O. Borůvka [Bor26], publicado en checo en 1926 y motivado por el diseño de una red eléctrica en Moravia. Le siguieron el de V. Jarník [Jar30] —que R. Prim [Pri57] redescubrió veintisiete años después, razón por la cual algunos autores lo llaman algoritmo de Jarník-Prim— y el de J. Kruskal [Kru56]. El algoritmo de caminos más cortos de la Sección 4.8 apareció en un artículo de tres páginas de E. Dijkstra [Dij59], que además contiene un segundo algoritmo para el árbol de cubrimiento mínimo. R. Graham y P. Hell [GH85] escribieron una historia detallada del problema del árbol de cubrimiento mínimo que vale la pena leer, entre otras cosas porque muestra cuántas veces se redescubrió lo mismo.

La explicación de por qué tantos criterios distintos funcionan para el árbol de cubrimiento mínimo, anunciada en la Nota 4.7.7 y en la Nota 4.3.1, está en la teoría de matroides. J. Edmonds [Edm71] demostró que la estrategia voraz es óptima sobre un sistema de independencia si, y solo si, ese sistema es un matroide, y los bosques de un grafo forman uno. J. Oxley [Oxl11] es la referencia estándar sobre la teoría, y B. Korte y J. Vygen [KV18] la presentan desde el punto de vista de la optimización combinatoria, que es el más cercano a los intereses de este

texto. Conviene, eso sí, no sobreestimar el alcance del resultado: de los problemas de este capítulo, únicamente el árbol de cubrimiento mínimo cae dentro de esa caracterización.

La estructura de conjuntos disjuntos de la Sección 4.7 se puede mejorar con la técnica de compresión de caminos que menciona la Nota 4.7.9. R. Tarjan [Tar75] demostró que, combinada con la unión por tamaño, el costo amortizado de cada operación es $O(\alpha(N))$, donde α es la inversa de la función de Ackermann y crece tan despacio que vale a lo sumo 4 para cualquier N que quepa en el universo observable. La demostración es difícil y se encuentra en [CLRS22, Capítulo 19].

El ADT *minheap* de la Definición 4.7.5 admite implementaciones muy distintas del montón binario, que se debe a J. Williams [Wil64]. M. Fredman y R. Tarjan [FT87] introdujeron los *montones de Fibonacci*, en los cuales *decrease* toma tiempo amortizado $O(1)$ y *extract* toma tiempo amortizado $O(\log N)$; sustituyéndolos en los algoritmos de Prim y de Dijkstra, sin cambiarles una línea, el tiempo baja a $O(M + N \log N)$. La constante oculta es grande y en la práctica el montón binario suele ganar, pero el ejercicio de sustitución es instructivo: es la separación entre ADT y estructura de datos rindiendo exactamente lo que la Nota 4.7.8 prometió.

En una dirección distinta, el ADT también se puede implementar sobre un *árbol de segmentos*: un árbol binario completo cuyas hojas son las llaves y en el cual cada nodo interno guarda el mínimo de su subárbol. Las dos operaciones toman tiempo $O(\log N)$ y, a diferencia del montón, se puede consultar el mínimo sobre cualquier subrango, lo cual resulta útil en otros contextos. Su pariente más económico es el *árbol binario indexado* (en inglés, *binary indexed tree*), también llamado *árbol de Fenwick* por P. Fenwick [Fen94], que ocupa un solo arreglo del tamaño del universo y cuyas operaciones son unas pocas líneas de aritmética sobre los bits del índice. Antes de emprender esa última implementación conviene advertir una sutileza. El árbol de Fenwick para mínimos soporta con naturalidad las actualizaciones *monótonas* –bajar una llave, que es exactamente lo que hace *decrease*– pero no las que aumentan un valor, y *extract* necesita subir a $+\infty$ la llave del elemento que remueve. Conseguirlo cuesta un factor logarítmico adicional o exige recurrir al árbol de segmentos, que sí admite actualizaciones arbitrarias. Es un buen ejercicio de diseño, y un buen recordatorio de que una estructura de datos implementa un tipo abstracto solo cuando ofrece *todas* sus operaciones dentro de las cotas exigidas.

La Nota 4.8.1 menciona las otras dos variantes del problema de caminos más cortos. La de todas las parejas se resuelve con el algoritmo de R. Floyd [Flo62], que es un ejercicio clásico de las técnicas del Capítulo 3. Para admitir pesos negativos sin ciclos negativos, como lo pide el Ejercicio 3, está el algoritmo que suele atribuirse a R. Bellman [Bel58] y a L. Ford, que renuncia a la estrategia voraz y paga por ello un factor N . Para la variante de pareja única, cuando se dispone de una estimación de la distancia que falta, el algoritmo A^* de P. Hart, N. Nilsson y B. Raphael [HNR68]

explora una porción mucho menor del grafo; es, en esencia, el algoritmo de la Sección 4.8 con las llaves corregidas por esa estimación.

Quedaron por fuera de este capítulo varios algoritmos voraces importantes. El de D. Huffman [**Huf52**], que construye códigos de longitud variable de redundancia mínima, es quizás el ejemplo más citado de la técnica y su demostración de optimización local es un argumento de intercambio particularmente elegante. También quedaron por fuera los algoritmos voraces que no resuelven un problema sino que lo *aproximan*, como el del cubrimiento de conjuntos; el Ejercicio 9 de la Sección 4.6 es una muestra de ese enfoque, y V. Vazirani [**Vaz01**] es una buena puerta de entrada al tema.

Reintento

Las tres técnicas presentadas hasta ahora responden, cada una a su manera, a una misma pregunta: ¿cómo evitar recorrer por completo el espacio de soluciones de un problema? Dividir, conquistar y combinar lo parte en subespacios independientes; la programación dinámica lo comprime aprovechando que muchas subinstancias se repiten; los algoritmos voraces lo colapsan a un único camino, aquel que dicta el criterio de selección. Hay problemas, sin embargo, para los cuales ninguna de las tres respuestas está disponible. No hay una partición natural que independice los subproblemas, la función objetivo no exhibe subestructura óptima explotable y todo criterio voraz que se proponga admite un contraejemplo. Para ellos queda una última alternativa, que es la que desarrolla este capítulo: recorrer el espacio de soluciones con una exploración sistemática.

La *exploración sistemática* se diseña con base en dos principios fundamentales. El primero es *construir*: las soluciones no se examinan una por una, ya terminadas, sino que se arman por partes, tomando una decisión a la vez. El segundo es *abandonar temprano*: apenas se sabe que lo construido hasta el momento no puede completarse hasta una solución, se deshace la última decisión y se intenta con otra. A esta forma de proceder se le llama *reintento*.

Nota 5.0.1: Sobre el nombre

En la literatura en inglés esta técnica se conoce como *backtracking*. En castellano circulan también los nombres *vuelta atrás* y *retroceso*. En este texto se prefiere *reintento* porque nombra la operación que caracteriza la técnica –intentar de nuevo con una decisión distinta– y no únicamente el desplazamiento que la hace posible. El lector debe reconocer, sin embargo, los otros tres nombres al consultar otras fuentes.

El reintento es la más elemental de las cuatro técnicas de diseño de este texto. También la que peor fama puede tener dado que, en el peor caso, un algoritmo de reintento puede llegar a examinar una cantidad exponencial de soluciones parciales. Esa fama es merecida, pero no necesariamente determinante. Por un lado, para muchos problemas de interés práctico no se conoce –ni se espera conocer, según lo que se estudiará en el Capítulo 8– nada esencialmente mejor. Por otro, la diferencia entre un algoritmo de reintento ingenuo y uno bien diseñado es, en la práctica, la diferencia entre un programa que no termina nunca y uno que responde en milisegundos, aun cuando ambos tengan la misma complejidad asintótica. El diseño de esa diferencia –la *poda*– es el asunto central del capítulo.

Nota 5.0.2: Lehmer, Walker, Golomb y Baumert

El procedimiento que este capítulo describe es mucho más antiguo que su nombre: recorrer sistemáticamente las posibilidades y retroceder ante un callejón sin salida es una idea que aparece en la solución de rompecabezas y laberintos desde el siglo XIX. Se le atribuye al matemático Derrick H. Lehmer haber acuñado, en la década de 1950, el término *backtrack* para nombrarla. En 1960, Robert J. Walker describió la técnica como un método general de enumeración para una clase de problemas combinatorios y, en 1965, Solomon W. Golomb y Leonard D. Baumert publicaron *Backtrack programming*, el artículo que la estableció como una técnica de diseño con derecho propio y que introdujo varias de las ideas de poda que se estudian en este capítulo (ver notas y referencias al final del capítulo).

Este capítulo presenta las ideas principales que respaldan el reintento, una metodología para su uso y varios ejemplos, incluyendo su extensión a problemas de optimización mediante ramificación y poda.

5.1. Ideas clave

Considere un arreglo $A[0..N]$ de números y el objetivo de determinar cuáles de sus subconjuntos de índices suman exactamente un número dado X . Una solución es, entonces, una decisión por cada índice: llevarlo o no llevarlo. Con $N = 3$ hay ocho decisiones posibles y todas caben en una página; con $N = 100$ hay más de 10^{30} y posiblemente no sea buena idea pensar en escribirlas en algún lado. La técnica que se estudia en este capítulo no elimina ese crecimiento, pero sí permite, en muchos casos, no tener que explorar el espacio de búsqueda completamente.

La primera observación es que las ocho soluciones del caso $N = 3$ no son ocho objetos sin relación entre sí: comparten decisiones iniciales. Las cuatro soluciones que llevan el índice 0 coinciden en su primera decisión, y las dos que llevan los

índices 0 y 1 coinciden en las dos primeras. Esta estructura compartida es lo que permite organizar el recorrido como un árbol y, sobre todo, lo que permite descartar muchas soluciones a la vez: al descartar una decisión inicial se descartan de un golpe todas las soluciones que la comparten.

5.1.1. Soluciones parciales. Para hacer precisas estas ideas conviene fijar qué tipo de problema se está resolviendo.

Definición 5.1.1: Problema de búsqueda combinatoria

Un *problema de búsqueda combinatoria* está dado por una longitud $K \geq 0$, una secuencia $C[0..K)$ de conjuntos finitos no vacíos, llamados *conjuntos de candidatos*, y un predicado *sol* sobre las tuplas de $C[0] \times \cdots \times C[K-1]$. El conjunto de *soluciones* del problema es

$$\{s \in C[0] \times \cdots \times C[K-1] \mid \text{sol}(s)\}.$$

Es decir, una solución es una tupla que asigna a cada una de K decisiones un candidato entre los disponibles para esa decisión, de manera que se satisfaga el predicado *sol*. En el problema con el que se abrió la sección, se tiene $K = N$, cada $C[i]$ es el conjunto $\{0, 1\}$ –que corresponde a la decisión de no llevar o llevar el índice i – y *sol*(s) es cierto exactamente cuando la suma de los elementos escogidos por s es X .

Nota 5.1.1: Sobre la longitud fija

La Definición 5.1.1 exige que todas las soluciones tengan la misma longitud K . Esto parece restrictivo –¿cómo representar, por ejemplo, los subconjuntos de $A[0..N)$, que tienen tamaños distintos?–, pero no lo es: basta con tomar una decisión por cada índice, incluyendo la decisión de no usarlo, como se acaba de hacer. Este es un patrón recurrente en el diseño de algoritmos de reintento y vale la pena enunciarlo. Los objetos de tamaño variable se representan como secuencias de decisiones de longitud fija. La representación alternativa, en la cual las soluciones tienen longitudes distintas, es a veces más natural y se explora en el Ejercicio 4.

Las tuplas incompletas, que registran algunas decisiones pero no todas, son el objeto central de la técnica.

Definición 5.1.2: Solución parcial

Sea un problema de búsqueda combinatoria con longitud K y candidatos $C[0..K)$. Una *solución parcial* de longitud k , con $0 \leq k \leq K$, es una tupla $s = \langle s_0, s_1, \dots, s_{k-1} \rangle$ tal que $s_i \in C[i]$ para $0 \leq i < k$. Se escribe $|s| = k$. La solución parcial de longitud 0 se denota $\langle \rangle$ y se llama *vacía*. Una solución parcial de longitud K se llama *completa*.

Si $|s| = k < K$ y $c \in C[k]$, la *extensión inmediata* de s con c es la solución parcial $s \cdot c = \langle s_0, \dots, s_{k-1}, c \rangle$, de longitud $k + 1$. Se dice que t *extiende* a s , y se escribe $s \preceq t$, si s es un prefijo de t .

Note que toda solución es una solución parcial completa, pero no toda solución parcial completa es una solución: para serlo debe además satisfacer *sol*.

Definición 5.1.3: Árbol de reintento

El *árbol de reintento* de un problema de búsqueda combinatoria es el árbol cuyos nodos son las soluciones parciales del problema, cuya raíz es $\langle \rangle$, y en el cual los hijos del nodo s , con $|s| = k < K$, son exactamente las extensiones inmediatas $s \cdot c$ para $c \in C[k]$. Los nodos de longitud K son las hojas del árbol.

El árbol de reintento es a este capítulo lo que el diagrama de necesidades fue al Capítulo 3: no es una estructura de datos que el algoritmo construya en memoria, sino un objeto de diseño que se dibuja en el papel para razonar acerca del algoritmo. De hecho, la característica que define a los algoritmos de esta sección es precisamente que el árbol nunca se construye: se recorre en profundidad y en todo momento el algoritmo tiene en memoria un único camino de la raíz al nodo que está visitando.

5.1.2. Poda. Recorrer el árbol de reintento completo equivale a examinar todas las soluciones parciales completas, es decir, a fuerza bruta. Lo que hace útil a la técnica es poder decidir, en un nodo interno, que ninguna de las hojas de su subárbol es una solución y no seguir descendiendo.

Definición 5.1.4: Predicado de factibilidad

Un predicado *feasible* sobre soluciones parciales es un *predicado de factibilidad* para el problema si y solo si para toda solución t y toda solución parcial s se

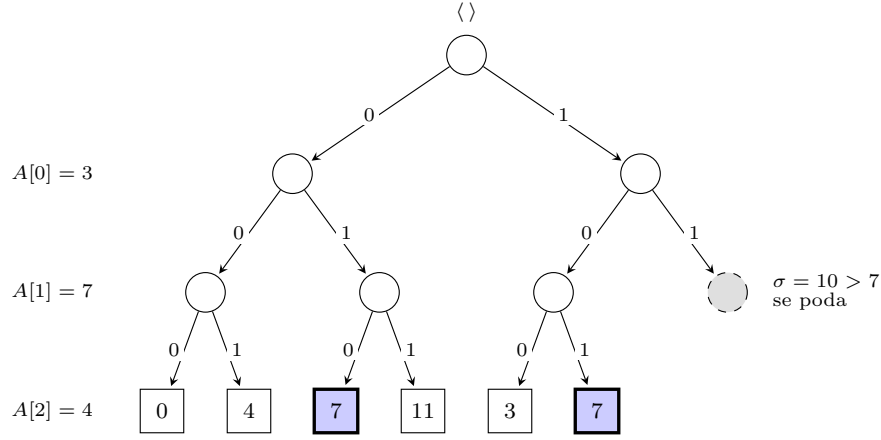


Figura 1. Árbol de reintento para las decisiones sobre $A[0..3)$, con $A = [3, 7, 4]$ y $X = 7$. Cada nivel corresponde a un índice y cada arco a la decisión de no llevarlo (0) o llevarlo (1). Las hojas están etiquetadas con la suma correspondiente y las soluciones aparecen resaltadas.

cumple

$$s \preceq t \implies \text{feasible}(s).$$

Un predicado de factibilidad es, entonces, una condición necesaria para poder llegar a una solución. Por contrapositiva, la definición dice lo que interesa en la práctica para realizar una poda: si $\text{feasible}(s)$ es falso, entonces ninguna extensión de s es una solución y, por tanto, todo el subárbol con raíz en s puede descartarse sin miedo a perder soluciones. A esto último se le llama *podar* el nodo s .

Nota 5.1.2: La poda debe ser conservadora

La Definición 5.1.4 impone una obligación en un solo sentido y esa asimetría es la fuente del error más común al diseñar algoritmos de reintento. El predicado *feasible* debe ser cierto en los prefijos de las soluciones; en cambio, no está obligado a ser falso en ningún otro lugar. Es decir, se le permite equivocarse por exceso de optimismo —dejar pasar soluciones parciales que no llevan a nada, con el único costo de trabajo desperdiciado— pero nunca por exceso de pesimismo, pues podar un nodo que sí tenía soluciones en su subárbol las pierde para siempre.

Dos predicados de factibilidad siempre existen. El primero es $\text{feasible}(s) = \text{true}$, que no poda nada y produce la fuerza bruta. El segundo es $\text{feasible}(s) = \text{“alguna extensión de } s \text{ es solución”}$, que poda todo lo podable pero cuya

evaluación es, en general, tan difícil como el problema original. Todo diseño de un algoritmo de reintento es la búsqueda de un punto intermedio entre estos dos extremos: un predicado que pide mucho y se evalúe barato.

Ejemplo 5.1.1

Considere el problema con el que se abrió la sección, con $A = [3, 7, 4]$ y $X = 7$, y suponga que todos los elementos de A son positivos. Sea $\sigma(s)$ la suma de los elementos escogidos por la solución parcial s . El predicado

$$\text{feasible}(s) : \sigma(s) \leq X$$

es un predicado de factibilidad. En efecto, si $s \preceq t$ y t es solución, entonces $\sigma(t) = X$. Además, como los elementos son positivos y s es un prefijo de t , se tiene $\sigma(s) \leq \sigma(t) = X$.

Con este predicado, el nodo $\langle 1, 1 \rangle$ del árbol de la Figura 1 se poda, pues $\sigma(\langle 1, 1 \rangle) = 3 + 7 = 10 > 7$, y con él se descartan de una vez sus dos hojas. Note que la hipótesis de que los elementos son positivos es indispensable: con $A = [3, 7, -3]$ el predicado ya no es de factibilidad, pues $\langle 1, 1 \rangle$ sí se extiende a la solución $\langle 1, 1, 1 \rangle$. En la Sección 5.5 se estudian predicados de factibilidad que no dependen de esta hipótesis.

5.1.3. El esquema general. Con el vocabulario anterior, el esquema de un algoritmo de reintento se enuncia en pocas líneas. La solución parcial se mantiene en un arreglo s que funciona como pila: se agrega un candidato al final antes de descender y se retira al regresar.

Algoritmo 5.1.1: Esquema general de un algoritmo de reintento

```

1 def backtrack(s):
2     if len(s)==K:
3         if sol(s): report(s)
4     else:
5         for c in C[len(s)]:
6             s.append(c)                # choose
7             if feasible(s): backtrack(s) # explore
8             s.pop()                   # undo

```

El cuerpo del ciclo tiene tres pasos, y esos tres pasos son toda la técnica. Se *elige* un candidato para la decisión siguiente, extendiendo la solución parcial. Se *explora* el subárbol correspondiente, pero únicamente si la extensión sigue siendo

factible. Y se *deshace* la elección, dejando a s como estaba, para poder intentar con el candidato siguiente. La invocación inicial es `backtrack([])`.

Los algoritmos de este capítulo son recurrentes y no iterativos, de modo que el método de diseño por invariantes de ciclo del Capítulo 2 no aplica directamente. Su lugar lo ocupa la especificación del llamado recurrente: lo que debe ser cierto al invocarlo y lo que debe ser cierto al retornar de él. Para el Algoritmo 5.1.1 se propone la siguiente.

Pre : s es una solución parcial.

*Post*₀ : se reportan exactamente las soluciones t tales que $s \preceq t$.

*Post*₁ : el estado es idéntico al que tenía inmediatamente antes de la invocación.

La precondition es deliberadamente débil: no exige *feasible*(s). Esto permite que la invocación inicial `backtrack([])` la satisfaga trivialmente, sin tener que suponer nada acerca de si el problema tiene o no soluciones. La poscondición *Post*₀ es la corrección propiamente dicha, y *Post*₁ es la que hace que el ciclo de la línea 5 pueda intentar con el candidato siguiente sin arrastrar el efecto de los anteriores. Con esta especificación, la corrección del esquema se demuestra por inducción sobre la cantidad de decisiones que faltan por tomar.

Teorema 5.1.1

Sea *feasible* un predicado de factibilidad para el problema. Entonces el Algoritmo 5.1.1 satisface *Post*₀ y *Post*₁ bajo la precondition *Pre*. En particular, el llamado `backtrack([])` reporta exactamente las soluciones del problema, cada una exactamente una vez.

Demostración

Se procede por inducción sobre $K - |s|$, la cantidad de decisiones que faltan por tomar.

- Caso $K - |s| = 0$: entonces $|s| = K$ y s es completa. La única solución parcial que extiende a s es s misma, de modo que las soluciones t con $s \preceq t$ son, o bien ninguna, o bien únicamente s , según si *sol*(s) es falso o cierto. Esto es exactamente lo que hace la línea 3, luego se cumple *Post*₀. Como el algoritmo retorna sin modificar s , también se cumple *Post*₁.
- Caso $K - |s| > 0$: entonces $|s| = k < K$. Sea t una solución con $s \preceq t$ y sea $c = t_k$, el candidato que t escoge para la decisión k ; note que $c \in C[k]$ y que $s \cdot c \preceq t$. Como *feasible* es un predicado de factibilidad y $s \cdot c$ es un prefijo

de la solución t , la Definición 5.1.4 garantiza que $feasible(s \cdot c)$ es cierto. Luego la línea 7 sí invoca $backtrack(s \cdot c)$ y, por hipótesis inductiva, ese llamado reporta a t . Es decir, toda solución que extiende a s es reportada.

Recíprocamente, todo lo que se reporta durante la ejecución lo reporta alguno de los llamados de la línea 7 y, por hipótesis inductiva, es una solución que extiende a $s \cdot c$ para algún $c \in C[k]$; en particular, es una solución que extiende a s . Además, las soluciones que extienden a $s \cdot c$ y las que extienden a $s \cdot c'$ son disjuntas cuando $c \neq c'$, pues difieren en la posición k . Como el ciclo de la línea 5 recorre cada candidato exactamente una vez, ninguna solución se reporta dos veces. Se cumple, entonces, $Post_0$.

Finalmente, para $Post_1$ basta observar que en cada iteración el comando `s.append(c)` de la línea 6 y el `s.pop()` de la línea 8 se corresponden exactamente, y que la invocación intermedia restaura el estado por hipótesis inductiva.

Note que en la demostración anterior la hipótesis de que $feasible$ es un predicado de factibilidad se usa en un solo lugar y se usa en una sola dirección: para garantizar que el algoritmo no deja de descender por un camino que llevaba a una solución. Esta es la razón formal de la asimetría señalada en la Nota 5.1.2.

La poscondición $Post_1$ merece atención aparte, porque en los algoritmos concretos de las secciones siguientes deja de ser evidente.

Nota 5.1.3: El invariante de restauración

La poscondición $Post_1$ se enuncia sobre *todo* el estado que el algoritmo modifique: no solamente la solución parcial s , sino también cualquier estructura auxiliar que se agregue para evaluar $feasible$ eficientemente.

Cuando en las secciones siguientes se agreguen estructuras auxiliares –por ejemplo, los arreglos de diagonales ocupadas en el problema de las N reinas o la suma acumulada en el de la suma exacta– cada una de ellas deberá actualizarse al elegir y restaurarse al deshacer. Omitir una de esas restauraciones es la falla más frecuente en las implementaciones de reintento, y tiene la característica desagradable de producir respuestas incorrectas sin producir errores de ejecución.

5.1.4. Enumerar, contar, optimizar. El Algoritmo 5.1.1 *enumera* las soluciones. Las otras dos tareas que se le piden usualmente a un algoritmo de reintento son pequeñas variaciones sobre el mismo esquema:

Existencia y búsqueda: determinar si hay alguna solución o encontrar una en particular. Basta con detener el recorrido al encontrar la primera, lo cual se

implementa haciendo que **backtrack** retorne un valor Booleano que se propaga hacia arriba.

Conteo: determinar cuántas soluciones hay. Se acumula, por ejemplo, un contador global en lugar de reportar.

Optimización: encontrar la solución que maximiza o minimiza una función objetivo. Este caso admite una forma de poda adicional más poderosa que la de la Definición 5.1.4, porque aprovecha el valor de la mejor solución hallada hasta el momento. Es el tema de la Sección 5.7.

5.1.5. Análisis. El análisis de un algoritmo de reintento se reduce a contar los nodos que efectivamente visita.

Teorema 5.1.2

Sea V el conjunto de nodos del árbol de reintento que el Algoritmo 5.1.1 visita, y suponga que evaluar *feasible* en un nodo toma tiempo $O(f)$ y que $|C[k]| \leq c$ para $0 \leq k < K$. Entonces el algoritmo toma tiempo $O(|V| \cdot c \cdot f)$ y espacio $O(K)$, sin contar el espacio necesario para reportar las soluciones.

Demostración

Se propone como ejercicio al lector.

El factor $|V|$ es el que domina y el que ningún análisis asintótico logra acotar con precisión: en el peor caso, cuando la poda no descarta nada, $|V| = \Theta(c^K)$. Esta cota es genuina y no un artefacto del análisis, pues para cualquier predicado de factibilidad se pueden construir instancias en las cuales no poda nada.

Nota 5.1.4: Sobre el peor caso y la práctica

La observación anterior tiene una consecuencia incómoda: dos algoritmos de reintento para un mismo problema, uno con una poda excelente y otro sin poda alguna, tienen la *misma* complejidad asintótica de peor caso. La herramienta de análisis desarrollada en el Capítulo 1 es, en este capítulo, ciega a la única diferencia que importa.

Esto no invalida el análisis asintótico ni justifica abandonarlo; señala únicamente que en este capítulo debe complementarse. La forma usual de hacerlo es reportar el tamaño $|V|$ del árbol efectivamente recorrido para instancias concretas, que es una medida empírica y no asintótica. Varios de los ejercicios

de este capítulo piden instrumentar los algoritmos con un contador de nodos visitados y comparar; se recomienda al lector no omitirlos, pues son la única manera de percibir el efecto de la poda.

Para cerrar la sección, la siguiente tabla ubica el reintento entre las técnicas de diseño estudiadas, según lo que cada una hace con el espacio de soluciones.

Técnica	Estrategia sobre el espacio de soluciones
Dividir, conquistar y combinar	Partirlo en subespacios independientes
Programación dinámica	Comprimirlo, reusando subinstancias repetidas
Algoritmos voraces	Colapsarlo a un único camino
Reintento	Recorrerlo en profundidad, podando

Ejercicios

- Demuestre el Teorema 5.1.2.
- Demuestre que el árbol de reintento de un problema con longitud K y conjuntos de candidatos de tamaño $c \geq 2$ tiene exactamente $\frac{c^{K+1}-1}{c-1}$ nodos. ¿Cuántos de ellos son hojas? ¿Qué fracción del árbol son hojas cuando $c = 2$ y K es grande?
- Sea *feasible* un predicado de factibilidad para un problema dado.
 - Demuestre que el predicado $feasible'(s) : feasible(s) \wedge true$ también lo es.
 - Sean $feasible_1$ y $feasible_2$ predicados de factibilidad. ¿Lo es $feasible_1 \wedge feasible_2$? ¿Y $feasible_1 \vee feasible_2$? Demuestre o dé un contraejemplo en cada caso.
 - 🌈 Concluya que los predicados de factibilidad para un problema dado forman un retículo bajo el orden $\Rightarrow$ y describa sus elementos extremos.
- La Nota 5.1.1 indica que los objetos de tamaño variable se pueden representar como secuencias de decisiones de longitud fija. Considere la representación alternativa en la cual un subconjunto de índices de $A[0..N)$ se representa directamente como la secuencia creciente de sus elementos, de modo que las soluciones parciales tienen longitudes distintas.
 - Adapte la Definición 5.1.3 a esta representación. ¿Cuáles son ahora los hijos de un nodo? ¿Cuáles son las hojas?
 - Adapte el Algoritmo 5.1.1.
 - Compare la cantidad de nodos de los dos árboles. ¿Cuál representación prefiere y por qué?
- Un predicado de factibilidad *feasible* se dice *perfecto* si para toda solución parcial s se cumple que *feasible*(s) es cierto si y solo si alguna extensión de s es solución.

- a) Demuestre que si se dispone de un predicado perfecto evaluable en tiempo $O(f)$, entonces el Algoritmo 5.1.1 encuentra una solución (o determina que no existe) en tiempo $O(K \cdot c \cdot f)$.
 - b) Explique por qué el numeral anterior *no* demuestra que todo problema de búsqueda combinatoria se resuelva en tiempo polinomial.
6. Considere el problema de generar todas las cadenas de longitud N sobre el alfabeto $\{a, b\}$ que no contienen dos a consecutivas.
- a) Especifique el problema como un problema de búsqueda combinatoria en el sentido de la Definición 5.1.1.
 - b) Proponga un predicado de factibilidad y demuestre que lo es.
 - c) Implemente el algoritmo en el lenguaje de programación Python.
 - d) Instrumente su implementación con un contador de nodos visitados. Compare, para $N = 20$, la cantidad de nodos visitados con y sin poda. ¿Qué sucesión reconoce en la cantidad de soluciones?
7. El Algoritmo 5.1.1 recorre el árbol de reintento en profundidad. Discuta qué ocurriría si se recorriera en amplitud. ¿El algoritmo seguiría siendo correcto? ¿Qué pasaría con el espacio utilizado? ¿Se podría seguir hablando de *deshacer*?
8. 🌈 Escriba una versión iterativa del Algoritmo 5.1.1 que use una pila explícita, proponga los invariantes del ciclo en el estilo del Capítulo 2 y demuestre que el diseño es correcto.

5.2. Generación de objetos combinatorios

Antes de aplicar el reintento a problemas de decisión y de optimización, conviene usarlo para la tarea más simple que admite: generar, uno por uno, todos los objetos de una familia combinatoria. Estos generadores son útiles por sí mismos—para ensayar hipótesis, para verificar por fuerza bruta algoritmos más sofisticados, para construir casos de prueba— y, sobre todo, sirven para fijar el esquema del capítulo en su forma más despojada, sin las complicaciones que introduce una poda interesante.

5.2.1. Subconjuntos. El problema de generación de subconjuntos se especifica a continuación.

Problema 5.2.1: Generación de subconjuntos

Entrada: Un arreglo $A[0..N)$.

Salida: Todos los subconjuntos de elementos de $A[0..N)$.

Siguiendo la Nota 5.1.1, un subconjunto se representa como una secuencia de N decisiones, una por cada índice del arreglo: la decisión i -ésima vale 1 si el elemento $A[i]$ se incluye y 0 si no. Es decir, $K = N$ y $C[i] = \{0, 1\}$ para $0 \leq i < N$. Toda solución parcial completa es una solución, de modo que $sol(s) = true$, y no hay nada que podar: se toma $feasible(s) = true$. El árbol de reintento correspondiente es el de la Figura 1, con N niveles en lugar de tres.

Con estas decisiones, el Algoritmo 5.1.1 se instancia directamente.

Algoritmo 5.2.1: Generación de subconjuntos

```

1 def subsets(A, s):
2     N = len(A)
3     if len(s)==N:
4         print([ A[i] for i in range(N) if s[i]==1 ])
5     else:
6         for c in [0, 1]:
7             s.append(c)
8             subsets(A, s)
9             s.pop()
```

La invocación inicial es `subsets(A, [])`. La línea 4 traduce la secuencia de decisiones al objeto que representa, que es el subconjunto de los elementos cuya decisión fue 1. Note que esa traducción pasa por referencia una extensión de `s` (con 0 o 1) y que luego se elimina, resultando en la versión inicial de `s`. De este modo, $Post_1$ se preserva.

Ejemplo 5.2.1

Considere la siguiente invocación:

```
1 subsets([4,6,5], [])
```

El resultado de esta ejecución es el siguiente:

```

1 []
2 [5]
3 [6]
4 [6, 5]
5 [4]
6 [4, 5]
7 [4, 6]
8 [4, 6, 5]
```

El orden en el cual aparecen los subconjuntos no es arbitrario: es el orden en que el recorrido en profundidad visita las hojas del árbol de reintento, con la decisión 0 explorada antes que la decisión 1. Leyendo las decisiones como dígitos binarios de derecha a izquierda, es el orden $0, 1, \dots, 7$.

Teorema 5.2.1

El llamado `subsets(A, [])`, con $|A| = N$:

1. Reporta exactamente los 2^N subconjuntos de elementos de $A[0..N)$, cada uno exactamente una vez.
2. Visita $2^{N+1} - 1$ nodos del árbol de reintento.
3. Toma tiempo $\Theta(N \cdot 2^N)$.
4. Usa espacio $O(N)$, sin contar el espacio de la salida.

Demostración

Se propone como ejercicio al lector.

El numeral 3 del Teorema 5.2.1 merece un comentario. El factor 2^N es inevitable, pues hay esa cantidad de objetos que reportar; el factor adicional N proviene de construir explícitamente cada subconjunto en la línea 4. Si lo único que se desea es *contar* los subconjuntos o acumular alguna función de ellos que pueda mantenerse incrementalmente, ese factor desaparece y el algoritmo toma tiempo $\Theta(2^N)$.

Nota 5.2.1: Subconjuntos y máscaras de bits

La Sección 3.9 presentó las máscaras de bits como representación de conjuntos. Con ellas, los subconjuntos de $A[0..N)$ se pueden generar sin recurrencia alguna, simplemente recorriendo los enteros de 0 a $2^N - 1$ e interpretando cada uno como el conjunto de índices de sus bits en 1:

```

1 def subsets_mask(A):
2     N = len(A)
3     for m in range(1<<N):
4         print([ A[i] for i in range(N) if (m>>i)&1 ])

```

Esta versión es más corta, más rápida por un factor constante y no consume pila. Cuando se desean todos los subconjuntos sin excepción, debería preferirse.

Su limitación aparece apenas se quiere podar. La generación por máscaras es inherentemente plana: los 2^N enteros se visitan uno por uno y no hay noción de subárbol que descartar, de modo que saber que un prefijo de decisiones no conduce a nada no ahorra ningún trabajo. El reintento, en cambio, paga la sobrecarga de la recurrencia a cambio de conservar la estructura de árbol, que es precisamente lo que la poda explota. Esta tensión –una representación eficiente que no se deja podar contra una menos eficiente que sí– reaparece en la Sección 5.5.

5.2.2. Permutaciones. El problema de generación de permutaciones se especifica a continuación.

Problema 5.2.2: Generación de permutaciones

Entrada: Un arreglo $A[0..N)$ de elementos distintos.

Salida: Todas las permutaciones de $A[0..N)$.

De nuevo hay que identificar qué es una decisión. La elección natural es que la decisión k -ésima determine qué índice de A ocupa la posición k de la permutación. Es decir, $K = N$ y $C[k] = \{0, 1, \dots, N - 1\}$ para todo k . Con esta representación no toda solución parcial completa es una permutación: la secuencia $\langle 0, 0 \rangle$ es una solución parcial legítima según la Definición 5.1.2, pero repite un índice. Entonces

$sol(s) :$ “los elementos de s son distintos par a par”,

y el problema, a diferencia del de los subconjuntos, sí admite poda.

Teorema 5.2.2

El predicado

$feasible(s) :$ “los elementos de s son distintos par a par”

es un predicado de factibilidad para el Problema 5.2.2. Más aún, $feasible$ es perfecto en el sentido del Ejercicio 5.

Demostración

Sean t una solución y s una solución parcial con $s \preceq t$. Como los elementos de t son distintos par a par y los de s son un subconjunto de ellos, los de s también lo son. Luego $feasible(s)$ es cierto. Además, $feasible$ es un predicado de factibilidad.

Para la perfección hay que demostrar, además, que si *feasible*(*s*) entonces alguna extensión de *s* es solución. Sea $|s| = k \leq N$ y suponga que los *k* elementos de *s* son distintos par a par. Entonces quedan exactamente $N - k$ índices sin usar y basta con ubicarlos, en cualquier orden, en las posiciones $k, k + 1, \dots, N - 1$. La secuencia completa resultante tiene sus *N* elementos distintos par a par y, por lo tanto, es una solución que extiende a *s*.

Evaluar *feasible* recorriendo *s* tomaría tiempo $O(N)$ en cada nodo. Como lo único que cambia entre un nodo y su hijo es un elemento, conviene mantener esa información en una estructura auxiliar: un arreglo *used*[0..*N*] de Booleanos tal que *used*[*i*] indica si el índice *i* ya aparece en la solución parcial. Con él, la factibilidad de extender *s* con el candidato *c* se decide en tiempo constante consultando *used*[*c*].

Esta es la primera vez en el capítulo que aparece estado auxiliar y con él la obligación anunciada en la Nota 5.1.3: el arreglo *used* debe actualizarse al elegir y restaurarse al deshacer, o la poscondición *Post*₁ deja de cumplirse.

Algoritmo 5.2.2: Generación de permutaciones

```

1 def permutations(A, s, used):
2     N = len(A)
3     if len(s)==N:
4         print([ A[i] for i in s ])
5     else:
6         for c in range(N):
7             if not used[c]:
8                 used[c] = True           # choose
9                 s.append(c)
10                permutations(A, s, used)
11                s.pop()                   # undo
12                used[c] = False

```

La invocación inicial es *permutations*(*A*, [], [*False*]*len(*A*)). Las líneas 8–9 y 11–12 se corresponden en orden inverso, como el llenado y vaciado de una pila; escribirlas así, y no en el mismo orden, es una costumbre que ayuda a no olvidar ninguna restauración cuando el estado auxiliar crece.

Nota 5.2.2: Dónde se evalúa la factibilidad

El Algoritmo 5.1.1 consulta *feasible* después de extender la solución parcial, mientras que el Algoritmo 5.2.2 lo hace antes, en la línea 7. Las dos formas

son correctas y equivalentes. La segunda se prefiere cuando la factibilidad de $s \cdot c$ se puede decidir mirando únicamente el candidato c y el estado auxiliar, pues evita el trabajo de extender y deshacer una solución parcial que se va a descartar de inmediato. La primera es preferible cuando el predicado se expresa más naturalmente sobre la solución parcial ya extendida. En este capítulo se usan ambas, según convenga.

Teorema 5.2.3

El llamado `permutations(A, [], [False]*N)`, con $|A| = N$ y elementos distintos:

1. Reporta exactamente las $N!$ permutaciones de $A[0..N)$, cada una exactamente una vez.
2. Visita $\left(+k \mid 0 \leq k \leq N : \frac{N!}{(N-k)!}\right)$ nodos del árbol de reintento.
3. Toma tiempo $\Theta(N \cdot N!)$.
4. Usa espacio $O(N)$, sin contar el espacio de la salida.

Demostración

Se propone como ejercicio al lector.

Vale la pena contrastar los dos generadores de esta sección. El de subconjuntos no poda nada y visita $2^{N+1} - 1$ nodos para producir 2^N objetos: cerca de la mitad de los nodos visitados son hojas útiles. El de permutaciones sí poda y lo hace con un predicado perfecto: por el Teorema 5.2.2, todo nodo que el algoritmo visita conduce a por lo menos una permutación, de modo que no hay un solo nodo de trabajo desperdiciado. Sin la poda, en cambio, el algoritmo recorrería las N^N secuencias de índices para quedarse con $N!$ de ellas; con $N = 10$ eso es la diferencia entre visitar diez mil millones de hojas y visitar tres millones y medio.

Un predicado de factibilidad perfecto es una situación excepcionalmente afortunada y no es la que se encuentra en los problemas de las secciones siguientes. Allí la poda descartará subárboles enormes, pero también dejará pasar soluciones parciales que no llevan a ninguna parte.

Ejercicios

1. Demuestre el Teorema 5.2.1.
 2. Demuestre el Teorema 5.2.3. Para el numeral 2, observe que la cantidad de nodos de longitud k es la cantidad de secuencias de k índices distintos tomados de N .
 3. Demuestre que $\left(+k \mid 0 \leq k \leq N : \frac{N!}{(N-k)!} \right) < e \cdot N!$ y concluya que el Algoritmo 5.2.2 visita $\Theta(N!)$ nodos.
 4. Modifique el Algoritmo 5.2.1 para que reporte únicamente los subconjuntos de tamaño exactamente M , con $0 \leq M \leq N$.
 - a) Proponga un predicado de factibilidad no trivial y demuestre que lo es. Ayuda: se puede podar tanto por exceso como por defecto.
 - b) ¿Es perfecto el predicado que propuso?
 - c) Implemente el algoritmo e instrumentelo con un contador de nodos visitados. Compare, para $N = 20$ y $M = 3$, la cantidad de nodos visitados con y sin poda.
 5. El Algoritmo 5.2.2 supone que los elementos de $A[0..N)$ son distintos. Si no lo son, reporta permutaciones repetidas. Modifíquelo para que cada permutación distinta se reporte exactamente una vez, sin generar y luego descartar las repetidas. Ayuda: ordene A previamente.
 6. Considere el problema de generar todas las cadenas bien balanceadas de N paréntesis de apertura y N de cierre.
 - a) Especifique el problema como un problema de búsqueda combinatoria.
 - b) Proponga un predicado de factibilidad y demuestre que lo es.
 - c) Demuestre que su predicado es perfecto.
 - d) Implemente el algoritmo. ¿Cuántas soluciones hay para $N = 1, 2, 3, 4, 5$? Investigue qué sucesión es esa.
 7. 🌈 Un *código de Gray* de N bits es una enumeración de los 2^N subconjuntos de un conjunto de N elementos en la cual dos subconjuntos consecutivos difieren en exactamente un elemento. Diseñe un algoritmo recurrente que genere uno y demuestre que es correcto. ¿Es un algoritmo de reintento?
 8. Use el Algoritmo 5.2.2 para resolver por fuerza bruta el problema del agente viajero de la Sección 3.9. Compare experimentalmente su tiempo de ejecución con el de la solución por memorización de esa sección, para $N = 5, 8, 10, 12$. Explique lo que observa a la luz de las complejidades $\Theta(N \cdot N!)$ y $O(2^N N^2)$.
-

5.3. Una metodología

Con base en los ejemplos de la sección anterior, se propone una metodología para diseñar algoritmos de reintento. Es deliberadamente análoga a las metodologías 3.3.1 y 4.3.1. Como aquella, no debe entenderse como una receta infalible, sino como una lista de las preguntas que conviene responder y en qué orden.

Metodología 5.3.1: Reintento

Especificación: especificar el problema, indicando claramente su entrada y su salida, y si lo que se desea es enumerar todas las soluciones, encontrar una, contarlas u optimizar sobre ellas.

Decisiones: identificar de qué decisiones se compone una solución. Concretamente, determinar la longitud K y los conjuntos de candidatos $C[0..K)$ de la Definición 5.1.1. Esta es la decisión de diseño más importante y casi nunca es única: el mismo problema admite representaciones distintas y con árboles de reintento de tamaños muy diferentes. Una representación desafortunada no necesariamente se compensa después con una buena poda.

Predicado de solución: caracterizar cuándo una solución parcial completa es efectivamente una solución, es decir, definir *sol*.

Predicado de factibilidad: proponer *feasible* y demostrar que lo es, en el sentido de la Definición 5.1.4. La manera usual de obtenerlo es *debilitar* el predicado de solución, ya sea recortando el rango de una cuantificación para que abarque únicamente las decisiones ya tomadas, como en la Sección 5.4, ya sea relajando alguna de sus restricciones, como en la Sección 5.5. Vale la pena averiguar además si es perfecto, pues de serlo el algoritmo no desperdiciará trabajo. Recuerde la Nota 5.1.2: la única obligación es no podar de más.

Estado auxiliar: determinar qué información conviene mantener para que evaluar *feasible* no sea costoso, típicamente en tiempo constante o lineal. Por cada estructura auxiliar que se introduzca, especificar cómo se actualiza al elegir y cómo se restaura al deshacer, de modo que se preserve $Post_1$.

Orden (opcional): escoger en qué orden se toman las decisiones y en qué orden se recorren los candidatos de cada una. Este paso no afecta la corrección –el conjunto de soluciones reportadas es el mismo– pero puede cambiar drásticamente la cantidad de nodos visitados. La heurística usual es tomar primero las decisiones más restringidas y ensayar primero los

candidatos más prometedores, de manera que la poda actúe lo más arriba posible en el árbol.

Implementación: instanciar el Algoritmo 5.1.1 con las decisiones anteriores, verificando que la guarda del algoritmo computa efectivamente *feasible* sobre la solución parcial extendida.

Análisis: acotar el peor caso y, dado lo señalado en la Nota 5.1.4, medir además el tamaño del árbol efectivamente recorrido en instancias concretas.

Nota 5.3.1: ¿Por qué *sol* antes que *feasible*?

Los dos predicados aparecen en pasos consecutivos y en ese orden por una razón que conviene explicitar: no son independientes. La Definición 5.1.4 enuncia la obligación de *feasible* cuantificando sobre las soluciones del problema, de modo que sin una caracterización previa de qué es una solución no hay manera de enunciar esa obligación, mucho menos de descargarla. Un *feasible* propuesto antes que *sol* se propone sin criterio de corrección.

Hay, además, una regularidad que se repite en todos los ejemplos de este capítulo y que vale la pena tener presente:

$$\text{sol}(s) \equiv \text{feasible}(s) \wedge \text{“}s \text{ es completa”}.$$

En la Sección 5.6 es literalmente la definición de *sol* y el Teorema 5.6.3 la lee al revés. En la Sección 5.5 es el Teorema 5.5.3, donde las sumas por sufijo se anulan al llegar al final; en la Sección 5.4 lo pide demostrar el Ejercicio 2.

La regularidad no es automática y por eso hay que verificarla en cada diseño. El predicado trivial $\text{feasible}(s) = \text{true}$ satisface la Definición 5.1.4 y no la cumple en absoluto; entre ese extremo y el otro, cuanto más ajustado sea *feasible*, más cerca estará de coincidir con *sol* sobre las soluciones parciales completas. Cuando la igualdad se alcanza hay una recompensa concreta: el algoritmo no necesita verificar nada en las hojas, porque haber llegado hasta allá por un camino de extensiones factibles es ya haber encontrado una solución.

Esto explica que *sol* parezca desvanecerse del código –con frecuencia no aparece por ninguna parte– y sin embargo siga siendo indispensable en el razonamiento. Es el único puente entre el enunciado del problema, escrito en el lenguaje del dominio, y el predicado que el programa evalúa, escrito en el lenguaje de las decisiones. Saltarse el paso es cómodo hasta que la representación escogida no captura bien el enunciado, y para entonces el error está escondido dentro de *feasible*.

Llama la atención que la lista no incluya un paso dedicado a demostrar que el algoritmo es correcto. La omisión es deliberada: si los pasos anteriores se cumplieron, esa demostración ya está hecha.

Teorema 5.3.1

Sea un problema de búsqueda combinatoria y sea $\mathcal{A}$ un algoritmo tal que

1. *feasible* es un predicado de factibilidad para el problema, en el sentido de la Definición 5.1.4;
2. $\mathcal{A}$ es una instancia del Algoritmo 5.1.1, es decir, difiere de él únicamente en cómo se computan *sol* y *feasible*, y en el estado auxiliar que se haya agregado para computarlos; y
3. en todo nodo, la guarda de $\mathcal{A}$ computa *feasible*($s \cdot c$) y el estado auxiliar satisface la poscondición $Post_1$.

Entonces, $\mathcal{A}$ reporta exactamente las soluciones del problema, cada una exactamente una vez.

Demostración

Por las hipótesis 2 y 3, la ejecución de $\mathcal{A}$ visita los mismos nodos y reporta las mismas soluciones parciales completas que el Algoritmo 5.1.1 con esos predicados: las dos guardas tienen el mismo valor en cada nodo y el estado auxiliar, al restaurarse, no arrastra efectos de una rama a otra. Por la hipótesis 1 se puede aplicar el Teorema 5.1.1 a ese algoritmo, que es lo que se afirma.

Nota 5.3.2: ¿Qué queda por demostrar y qué no?

El Teorema 5.3.1 no dice que no haya nada que demostrar; dice *qué* hay que demostrar y es notable que ninguna de las tres obligaciones se refiera al recorrido del árbol, que es la parte difícil de entender de un algoritmo de reintento. También es la única que la metodología permite dar por resuelta de una vez y para siempre.

Cada hipótesis se descarga en un paso distinto. La primera es exactamente lo que pide el paso de *predicado de factibilidad* y es la única que exige una demostración genuina, usualmente breve. La tercera se descarga en el paso de *estado auxiliar* y es la que suele descuidarse: quien agrega una estructura para acelerar la evaluación de *feasible* contrae dos obligaciones, la de que la guarda siga computando el predicado correcto –lo cual exige enunciar qué significa

que la estructura sea fiel a la solución parcial, como hacen las condiciones R_0 , R_1 y R_2 de la Sección 5.4– y la de restaurarla al deshacer. La segunda se descarga en el paso de *implementación* y suele ser inmediata, pero conviene no darla por sentada porque es justamente la que se rompe en las secciones finales de este capítulo.

En efecto, las tres hipótesis son reales y ninguna sobra. La Sección 5.6 presenta un algoritmo cuyas decisiones no tienen longitud fija y que acumula un conteo en lugar de reportar: incumple la hipótesis 2 y por eso su correctitud se demuestra directamente, por inducción. Las secciones 5.7 y 5.8 mantienen una variable con la mejor solución hallada que deliberadamente *no* se restaura al deshacer: incumplen la hipótesis 3 y por eso hace falta un teorema propio, el Teorema 5.7.2.

Lo que el Teorema 5.3.1 sí garantiza es que en los casos ordinarios –los de las secciones 5.4 y 5.5, y los de la mayoría de los ejercicios– la demostración de correctitud se reduce a verificar una lista y que el esfuerzo puede concentrarse donde de verdad hace falta, que es el análisis.

Estos pasos se ilustrarán con cada uno de los ejemplos que se presentan en el resto del capítulo.

Ejercicios

1. El problema de coloración de un grafo consiste en asignar uno de M colores a cada vértice de un grafo, de manera que dos vértices adyacentes nunca reciban el mismo color.
 - a) Especifique el problema.
 - b) Siga, paso a paso, la metodología propuesta en esta sección, indicando claramente en qué consiste cada uno de los pasos. Debe resultar código en Python ejecutable.
 - c) Para el paso de *orden*, ensaye dos criterios distintos para escoger el vértice siguiente: uno arbitrario y otro que prefiera el vértice con más vecinos ya coloreados. Compare experimentalmente la cantidad de nodos visitados.
 2. Considere de nuevo el problema de generar los subconjuntos de tamaño M del Ejercicio 4 de la Sección 5.2, y la representación alternativa del Ejercicio 4, en la cual una solución parcial es una secuencia creciente de índices.
 - a) Aplique la metodología a cada una de las dos representaciones.
 - b) Cuente, en cada caso, la cantidad de nodos del árbol de reintento.
 - c) Discuta el paso de *decisiones* de la metodología a la luz de esta comparación.
-

5.4. El problema de las N reinas

En el juego de ajedrez, una reina ubicada en una casilla ataca todas las casillas que comparten con ella una fila, una columna o una diagonal, como se ilustra en la Figura 2. El problema que se estudia en esta sección consiste en ubicar N reinas en un tablero de $N \times N$ casillas sin que ninguna ataque a otra.

	0	1	2	3	4
0		×	×	×	
1	×	×	●	×	×
2		×	×	×	
3	×		×		×
4			×		

Figura 2. Casillas atacadas por una reina ubicada en la fila 1 y la columna 2 de un tablero de 5×5 .

Problema 5.4.1: Ubicación de N reinas

Entrada: Un número entero N , $N \geq 1$.

Salida: Todas las formas de ubicar N reinas en un tablero de $N \times N$ casillas, de modo que ningún par de ellas se ataque.

Este problema es el ejemplo canónico del reintento y sirve bien para dos propósitos. El primero es mostrar que la decisión de *representación* de la metodología de la Sección 5.3 puede hacer, por sí sola, la mitad del trabajo: con la representación que se escoge a continuación, dos de las tres formas de ataque quedan descartadas por construcción y no hay que podarlas. El segundo es exhibir, por primera vez en el capítulo, un predicado de factibilidad que *no* es perfecto, con las consecuencias que ello tiene sobre el trabajo desperdiciado.

5.4.1. Decisiones. Una solución es un conjunto de N casillas del tablero, de modo que la representación más directa sería una lista de pares de coordenadas. Esa representación, sin embargo, ignora una restricción que la propia definición del problema impone y que conviene aprovechar desde el principio.

Teorema 5.4.1

En toda solución del Problema 5.4.1 cada fila del tablero contiene exactamente una reina, y lo mismo ocurre con cada columna. En consecuencia, la función que asigna a cada fila la columna de su reina es una permutación de $[0..N)$.

Demostración

Dos reinas ubicadas en una misma fila se atacan, de modo que en una solución cada fila contiene a lo sumo una reina. Como hay N reinas y N filas, cada fila contiene exactamente una. El argumento para las columnas es idéntico.

Sea entonces col la función que asigna a cada fila i la columna de la única reina que hay en ella. Está bien definida por lo anterior. Es inyectiva, pues si $col(i) = col(j)$ con $i \neq j$, las reinas de las filas i y j compartirían columna y se atacarían. Una función inyectiva de $[0..N)$ en sí mismo es una biyección, es decir, una permutación.

El Teorema 5.4.1 indica cómo representar una solución: la decisión k -ésima es la columna que ocupa la reina de la fila k . Es decir, $K = N$ y $C[k] = [0..N)$ para $0 \leq k < N$, exactamente la misma representación que se usó para generar permutaciones en la Sección 5.2.

La ganancia de esta decisión es que dos de las tres formas de ataque desaparecen. Los ataques por fila son imposibles, porque la representación ubica una sola reina por fila. Los ataques por columna se reducen a que la solución parcial repita un valor, que es precisamente la condición ya estudiada. Queda por tratar únicamente el ataque en diagonal.

Teorema 5.4.2

Sean dos reinas ubicadas en las casillas (i, s_i) y (j, s_j) , con $i \neq j$. Las dos reinas comparten una diagonal si y solo si

$$i - s_i = j - s_j \quad \text{o} \quad i + s_i = j + s_j.$$

Demostración

Dos casillas comparten una diagonal si y solo si el desplazamiento vertical entre ellas coincide en magnitud con el horizontal, es decir, si y solo si $|i - j| = |s_i - s_j|$. Ahora bien, para cualesquiera enteros a y b se tiene que $|a| = |b|$ si y solo si $a = b$ o $a = -b$. Tomando $a = i - j$ y $b = s_i - s_j$:

$$\begin{aligned} |i - j| = |s_i - s_j| &\equiv i - j = s_i - s_j \vee i - j = -(s_i - s_j) \\ &\equiv i - s_i = j - s_j \vee i + s_i = j + s_j, \end{aligned}$$

que es lo que se quería demostrar.

El Teorema 5.4.2 dice que las diagonales del tablero se organizan en dos familias, y que cada familia está indexada por una cantidad que es constante a lo largo de

sus diagonales: $i - j$ para las descendentes y $i + j$ para las ascendentes. La Figura 3 muestra ambas.

0	-1	-2	-3	-4
1	0	-1	-2	-3
2	1	0	-1	-2
3	2	1	0	-1
4	3	2	1	0

$i - j$

0	1	2	3	4
1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8

$i + j$

Figura 3. Las dos familias de diagonales de un tablero de 5×5 . Cada casilla está rotulada con el valor que la identifica dentro de su familia; se resaltan la diagonal descendente $i - j = 1$ y la ascendente $i + j = 4$.

Con esto, la condición de que las reinas de las filas i y j no se ataquen adopta una forma notablemente regular: las tres cantidades s_i , $i - s_i$ e $i + s_i$ deben ser distintas para reinas distintas.

$$ok(s, i, j) : s_i \neq s_j \wedge i - s_i \neq j - s_j \wedge i + s_i \neq j + s_j.$$

El predicado de solución exige que eso valga para todo par de reinas.

$$sol(s) : (\forall i, j \mid 0 \leq i < j < N : ok(s, i, j)).$$

5.4.2. Predicado de factibilidad. Una solución parcial describe las reinas de las primeras $|s|$ filas. La condición natural para podar es que esas reinas, las ya ubicadas, no se ataquen entre sí: es decir, el mismo predicado anterior restringido a las decisiones tomadas. Lo único que cambia respecto de sol es el rango de la cuantificación.

$$feasible(s) : (\forall i, j \mid 0 \leq i < j < |s| : ok(s, i, j)).$$

Teorema 5.4.3

El predicado *feasible* es un predicado de factibilidad para el Problema 5.4.1.

Demostración

Sean t una solución y s una solución parcial con $s \preceq t$. Como s es un prefijo de t , se tiene $|s| \leq N$ y $s_i = t_i$ para $0 \leq i < |s|$. El rango de la cuantificación de $feasible(s)$ está contenido en el de $sol(t)$ y el término es el mismo, de modo que $sol(t)$ implica $feasible(s)$.

Nota 5.4.1: Una poda que no es perfecta

El generador de permutaciones de la Sección 5.2 usaba un predicado perfecto: toda solución parcial factible se extendía hasta una solución, y por lo tanto ningún nodo visitado era trabajo perdido. Aquí eso deja de ser cierto, y es la primera vez que ocurre en el capítulo.

El contraejemplo más pequeño e interesante es $N = 4$. La solución parcial $\langle 0 \rangle$ es factible —una sola reina nunca se ataca a sí misma— pero ninguna solución la extiende: como muestra la Figura 4, las dos soluciones del tablero de 4×4 empiezan ubicando la reina de la primera fila en la columna 1 o en la columna 2, nunca en la 0. El algoritmo recorrerá entonces por completo el subárbol de $\langle 0 \rangle$ sin obtener nada.

Esto no es un defecto del predicado que se propuso, sino la situación normal. Detectar que $\langle 0 \rangle$ no lleva a ninguna parte exige, en el fondo, resolver el problema. Lo que un predicado de factibilidad ofrece es una condición necesaria barata, no una respuesta; y el trabajo desperdiciado que deja pasar es exactamente lo que la Nota 5.1.4 anticipaba que el análisis asintótico no logra medir.

5.4.3. Estado auxiliar. Evaluar *feasible* recorriendo la solución parcial completa costaría tiempo $O(|s|)$ en cada nodo. Como al descender solo se agrega una reina, conviene mantener registrado qué columnas y qué diagonales están ocupadas. Por el Teorema 5.4.2 hacen falta tres arreglos de *Booleanos*:

- $used[0..N)$, para las columnas;
- $desc[0..2N - 1)$, para las diagonales descendentes; y
- $asc[0..2N - 1)$, para las ascendentes.

Los tamaños se justifican solos. Para una reina en la casilla (i, c) se tiene $i - c \in [-(N - 1)..N - 1]$, que son $2N - 1$ valores; al sumarles $N - 1$ quedan indexados por $[0..2N - 1)$. Análogamente $i + c \in [0..2N - 2]$, que también son $2N - 1$ valores y no requiere desplazamiento.

Como en el Teorema 3.2.1, conviene enunciar explícitamente qué significa que estos arreglos sean correctos respecto de la solución parcial.

$$\begin{aligned}
 R_0 : \quad & (\forall c \mid 0 \leq c < N : \\
 & \quad \text{used}[c] \equiv (\exists i \mid 0 \leq i < |s| : s_i = c)). \\
 R_1 : \quad & (\forall d \mid 0 \leq d < 2N - 1 : \\
 & \quad \text{desc}[d] \equiv (\exists i \mid 0 \leq i < |s| : i - s_i + N - 1 = d)). \\
 R_2 : \quad & (\forall d \mid 0 \leq d < 2N - 1 : \\
 & \quad \text{asc}[d] \equiv (\exists i \mid 0 \leq i < |s| : i + s_i = d)).
 \end{aligned}$$

Bajo estas condiciones, la factibilidad de extender la solución parcial se decide con tres consultas.

Teorema 5.4.4

Sean s una solución parcial con $\text{feasible}(s)$ y $k = |s| < N$, y suponga que R_0 , R_1 y R_2 son ciertas. Entonces, para todo $c \in C[k]$,

$$\text{feasible}(s \cdot c) \equiv \neg \text{used}[c] \wedge \neg \text{desc}[k - c + N - 1] \wedge \neg \text{asc}[k + c].$$

Demostración

La cuantificación de $\text{feasible}(s \cdot c)$ recorre los pares (i, j) con $0 \leq i < j \leq k$. Los pares con $j < k$ son exactamente los de $\text{feasible}(s)$, que es cierto por hipótesis. Restan los pares de la forma (i, k) con $0 \leq i < k$, para los cuales el término, usando que $s_k = c$, es

$$s_i \neq c \wedge i - s_i \neq k - c \wedge i + s_i \neq k + c.$$

Que esto valga para todo $i < k$ equivale, respectivamente, a que ninguna reina ya ubicada use la columna c , a que ninguna use la diagonal descendente $k - c$ y a que ninguna use la ascendente $k + c$. Por R_0 , R_1 y R_2 , esas tres afirmaciones son $\neg \text{used}[c]$, $\neg \text{desc}[k - c + N - 1]$ y $\neg \text{asc}[k + c]$.

5.4.4. Implementación. Con las decisiones anteriores, el Algoritmo 5.1.1 se instancia como sigue. La guarda de la línea 7 es la del Teorema 5.4.4, y las líneas 8 y 12 mantienen las condiciones R_0 , R_1 y R_2 .

Algoritmo 5.4.1: Ubicación de N reinas

```

1 def queens(N, s, used, desc, asc):
2   if len(s) == N:

```

```

3     print(s)
4     else:
5         k = len(s)
6         for c in range(N):
7             if not used[c] and not desc[k-c+N-1] and not asc[k+c]:
8                 used[c] = desc[k-c+N-1] = asc[k+c] = True    # choose
9                 s.append(c)
10                queens(N, s, used, desc, asc)
11                s.pop()                                     # undo
12                used[c] = desc[k-c+N-1] = asc[k+c] = False

```

La invocación inicial debe partir de la solución parcial vacía y de tres arreglos que registren, correctamente, que no hay ninguna columna ni ninguna diagonal ocupada. Conviene encapsularla.

Algoritmo 5.4.2: Arranque

```

1 def all_queens(N):
2     queens(N, [], [False]*N, [False]*(2*N-1), [False]*(2*N-1))

```

Ejemplo 5.4.1

Considere la siguiente invocación:

```
1 all_queens(4)
```

El resultado de esta ejecución es el siguiente:

```

1 [1, 3, 0, 2]
2 [2, 0, 3, 1]

```

Los dos tableros correspondientes se muestran en la Figura 4. Note que uno es la imagen especular del otro.

Nota 5.4.2: Reportar contra acumular

El Algoritmo 5.4.1 imprime la solución parcial y no la guarda. Si en lugar de imprimir se quisiera acumular las soluciones en una lista, no bastaría con agregar `s` a esa lista: como `s` se modifica en cada paso y se restaura al deshacer,

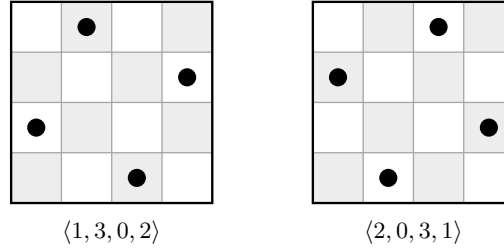


Figura 4. Las dos soluciones del problema de las 4 reinas, con la solución parcial que las representa.

todas las entradas de la lista terminarían apuntando al mismo arreglo, que al final de la ejecución estaría vacío. Hay que agregar una copia, con $s[:]$ o equivalente. Este error es una consecuencia directa de la poscondición $Post_1$: el algoritmo promete restaurar el estado, y quien quiera conservar una foto de ese estado debe tomarla por su cuenta.

Teorema 5.4.5

Para $N \geq 1$, el llamado `all_queens(N)` reporta exactamente las soluciones del Problema 5.4.1, cada una exactamente una vez.

Demostración

Se verifican las tres hipótesis del Teorema 5.3.1. La primera es el Teorema 5.4.3. La segunda es inmediata: el Algoritmo 5.4.1 es el Algoritmo 5.1.1 con $C[k] = [0..N)$, con la guarda de la línea 7 evaluada antes de elegir según la Nota 5.2.2, y con el estado auxiliar de las líneas 8 y 12. Para la tercera, que la guarda computa $feasible(s \cdot c)$ es el Teorema 5.4.4, cuyas hipótesis se satisfacen porque las condiciones R_0 , R_1 y R_2 valen inicialmente y se preservan, y que se cumple $Post_1$ se sigue de que las tres asignaciones de la línea 8 se deshacen exactamente en la línea 12. Los detalles se proponen como ejercicio al lector.

Falta observar que lo reportado son soluciones y no meras soluciones parciales completas, lo cual se sigue de que $feasible(s) \equiv sol(s)$ cuando $|s| = N$.

5.4.5. Análisis. El análisis se apoya en una propiedad que parece obvia pero conviene enunciar, porque es la que conecta el algoritmo con el conteo.

Teorema 5.4.6

Todo nodo que visita el Algoritmo 5.4.1 es una solución parcial factible.

Demostración

Se propone como ejercicio al lector. Ayuda: proceda por inducción sobre el orden en que se visitan los nodos. Para el caso base, observe que la cuantificación de $feasible(\langle \rangle)$ tiene rango vacío. Para el paso inductivo, use el Teorema 5.4.4, cuyas hipótesis se satisfacen por hipótesis inductiva, y porque las condiciones R_0 , R_1 y R_2 se preservan.

Teorema 5.4.7

Sea V el conjunto de nodos que visita el Algoritmo 5.4.1. Entonces

$$|V| \leq \left(+k \mid 0 \leq k \leq N : \frac{N!}{(N-k)!} \right) < e \cdot N!,$$

el algoritmo toma tiempo $O(N \cdot N!)$ y usa espacio $O(N)$, sin contar el espacio de la salida.

Demostración

Por el Teorema 5.4.6, todo nodo visitado satisface $feasible$ y, en particular, es una secuencia de valores de $[0..N)$ distintos par a par. Para $0 \leq k \leq N$, la cantidad de tales secuencias de longitud k es $N(N-1) \cdots (N-k+1) = N!/(N-k)!$, pues hay N valores disponibles para la primera posición, $N-1$ para la segunda, y así sucesivamente. Sumando sobre k se obtiene la primera desigualdad.

Para la segunda, sustituyendo $j = N - k$,

$$\begin{aligned} \left(+k \mid 0 \leq k \leq N : \frac{N!}{(N-k)!} \right) &= N! \left(+j \mid 0 \leq j \leq N : \frac{1}{j!} \right) \\ &< N! \left(+j \mid 0 \leq j : \frac{1}{j!} \right) \\ &= e \cdot N!, \end{aligned}$$

que es lo que pide demostrar el Ejercicio 3 de la Sección 5.2.

Para el tiempo basta acotar el trabajo de un nodo, sin contar los llamados recurrentes. En un nodo interno, el ciclo de la línea 6 da N vueltas, y en cada

una se evalúa la guarda en tiempo constante –tres consultas a arreglo, por el Teorema 5.4.4– y se ejecuta una cantidad constante de asignaciones. En una hoja, reportar la solución toma tiempo $O(N)$. En ambos casos el trabajo es $O(N)$ y multiplicando por $|V|$ se obtiene $O(N \cdot N!)$.

Para el espacio, cada invocación aumenta $|s|$ en uno y $|s| \leq N$, de modo que la profundidad de la recurrencia es a lo sumo N y cada marco ocupa espacio constante. Los cuatro arreglos suman $O(N)$ posiciones y se pasan por referencia, sin copiarse.

Vale la pena mirar de cerca qué se usó en esa demostración. De las tres condiciones que impone *feasible* solo se usó una, la de que los valores sean distintos par a par; las dos condiciones sobre las diagonales no intervienen en ninguna parte del conteo. Es decir, la cota del Teorema 5.4.7 es exactamente la cantidad de nodos del árbol de permutaciones del Teorema 5.2.3: la demostración acota el árbol de las reinas por el del generador de permutaciones y renuncia por completo a la poda por diagonales.

Esa renuncia es forzosa y es la ya señalada: la poda por diagonales descarta subárboles enormes sin cambiar el peor caso. La única forma de apreciar su efecto es medir. La siguiente tabla reporta, para varios valores de N , la cantidad de soluciones, la cantidad de nodos que visita el Algoritmo 5.4.1 y la cantidad que visitaría el generador de permutaciones de la Sección 5.2 si se le pidiera enumerar todas las permutaciones para filtrar al final las que no tienen ataques en diagonal.

N	Soluciones	Nodos (reinas)	Nodos (permutaciones)	Razón
4	2	17	65	3.8
6	4	153	1 957	12.8
8	92	2 057	109 601	53.3
10	724	35 539	9 864 101	277.6
12	14 200	856 189	1 302 061 345	1520.8

Las dos columnas centrales crecen exponencialmente, tal como predice el Teorema 5.4.7, pero a ritmos muy distintos: la razón entre ellas se multiplica por un factor cercano a 5 cada vez que N aumenta en 2. Para $N = 12$ la diferencia es la que hay entre recorrer menos de un millón de nodos y recorrer más de mil millones. Ese es, en concreto, el valor de podar temprano en lugar de filtrar al final.

Note ahora que la columna **Nodos (permutaciones)** no es únicamente el costo de una alternativa peor: es, además, el valor exacto de la cota que da el Teorema 5.4.7, por lo que se observó al terminar su demostración. Léida así, la última columna de la tabla mide precisamente cuánto sobreestima esa cota: un factor de 53 para $N = 8$ y de más de 1 500 para $N = 12$, y creciendo. Es la ilustración más concreta que ofrece este capítulo de lo advertido en la Nota 5.1.4: la cota es correcta, es la

mejor que el análisis asintótico permite obtener sin hipótesis adicionales sobre la instancia, pero es inservible para predecir lo que el algoritmo va a hacer.

Nota 5.4.3: Sobre la cantidad de soluciones

La segunda columna de la tabla forma parte de la sucesión 1, 0, 0, 2, 10, 4, 40, 92, 352, 724, ..., que da la cantidad de soluciones del problema de las N reinas. No se conoce una fórmula cerrada para ella, ni un algoritmo esencialmente mejor que la enumeración para calcular sus términos, que es la razón por la cual los valores conocidos llegan apenas hasta N del orden de unas pocas decenas. Llama la atención la irregularidad del comienzo: hay más soluciones para $N = 5$ que para $N = 6$.

Ejercicios

1. Demuestre el Teorema 5.4.6, es decir, que todo nodo visitado por el Algoritmo 5.4.1 es una solución parcial factible. Es el único punto que la demostración del Teorema 5.4.7 da por sentado.
2. Complete la demostración del Teorema 5.4.5. En particular:
 - a) Demuestre que las condiciones R_0 , R_1 y R_2 son ciertas para la invocación inicial.
 - b) Demuestre que el Algoritmo 5.4.1 preserva R_0 , R_1 y R_2 , y que satisface la poscondición $Post_1$.
 - c) Demuestre que si $|s| = N$, entonces $feasible(s) \equiv sol(s)$.
3. Demuestre, sin recurrir al algoritmo, que el Problema 5.4.1 no tiene soluciones para $N = 2$ ni para $N = 3$.
4. La Nota 5.4.1 afirma que $\langle 0 \rangle$ es factible para $N = 4$ pero no se extiende a ninguna solución. Determine cuántos nodos del árbol visita el algoritmo dentro de ese subárbol. ¿Qué fracción del total es?
5. Instrumente el Algoritmo 5.4.1 con un contador de nodos visitados y reproduzca la tabla de esta sección. Agregue las filas correspondientes a $N = 5, 7, 9, 11$ y 13 . ¿Observa alguna diferencia sistemática entre los valores de N pares e impares?
6. Modifique el Algoritmo 5.4.1 para que:
 - a) cuente las soluciones en lugar de reportarlas; y
 - b) reporte únicamente la primera solución que encuentre y termine.
 Para el segundo caso, mida cuántos nodos se visitan y compare con la enumeración completa.

7. El paso de *orden* de la metodología de la Sección 5.3 sugiere ensayar primero los candidatos más prometedores. Modifique el algoritmo para que recorra las columnas desde el centro del tablero hacia los bordes, en lugar de izquierda a derecha, y compare la cantidad de nodos visitados para encontrar la primera solución. ¿Cambia la cantidad de nodos visitados para la enumeración completa? Explique.
8. Considere la variante en la cual el tablero es un toro, es decir, las diagonales se continúan por el borde opuesto.
 - a) Adapte el Teorema 5.4.2 a esta variante.
 - b) Adapte el algoritmo.
 - c) Enumere las soluciones para $N \leq 8$. ¿Para cuáles valores de N hay soluciones?
9. 🌈 El grupo de simetrías del cuadrado, de orden 8, actúa sobre el conjunto de soluciones: rotar o reflejar una solución produce otra solución.
 - a) Verifique que las dos soluciones para $N = 4$ pertenecen a una misma órbita de esta acción.
 - b) Proponga una restricción sobre la primera decisión que permita recorrer solo una parte del árbol sin perder ninguna órbita, y demuestre que es correcta.
 - c) Implemente la mejora y mida el ahorro para $N = 8, 10, 12$.
10. 🌈 Investigue construcciones explícitas que produzcan una solución del Problema 5.4.1 para todo $N \geq 4$, sin recurrir a búsqueda alguna. Implementélas y verifique con el algoritmo de esta sección que lo que producen son, en efecto, soluciones.

5.5. Suma exacta de un subconjunto

El problema de esta sección ya apareció, informalmente, en el Ejemplo 5.1.1, que se usó allí para ilustrar la noción de poda. Aquel ejemplo terminaba con una advertencia: el predicado que se propuso solo era válido suponiendo que todos los elementos del arreglo eran positivos y esa hipótesis era indispensable. El propósito de esta sección es levantarla.

Problema 5.5.1: Suma exacta de un subconjunto

Entrada: Un arreglo $A[0..N)$ de números enteros y un número entero X .

Salida: Todos los subconjuntos de elementos de $A[0..N)$ cuya suma es X .

Este problema ocupa un lugar particular en el texto. Aparece en el Capítulo 3 resuelto por tabulación, se resuelve aquí por reintento y en el Capítulo 8 se demuestra que su versión de decisión es NP-completa. Vale la pena tener presente esa triple aparición mientras se lee lo que sigue.

5.5.1. Decisiones. La representación es la misma de la Sección 5.2: la decisión i -ésima vale 1 si el elemento $A[i]$ se incluye en el subconjunto y 0 si no, de modo que $K = N$ y $C[i] = \{0, 1\}$. Conviene nombrar la suma que una solución parcial acumula.

$$\sigma(s) = (+i \mid 0 \leq i < |s| : s_i \cdot A[i]).$$

Con esta notación, el predicado de solución es inmediato.

$$\text{sol}(s) : \sigma(s) = X.$$

5.5.2. Predicado de factibilidad. El Ejemplo 5.1.1 propuso podar cuando $\sigma(s) > X$, argumentando que agregar elementos positivos solo puede aumentar la suma. Con elementos de signo arbitrario ese argumento se cae: una solución parcial cuya suma ya excede a X puede corregirse más adelante con un elemento negativo y podarla perdería soluciones.

La pregunta correcta no es si $\sigma(s)$ se pasó de X , sino *qué puede llegar a aportar todavía* lo que falta por decidir. Si $|s| = k$, las decisiones pendientes son las de los elementos $A[k..N]$, y cada una de ellas contribuye $A[i]$ o 0. La contribución total, entonces, es a lo sumo la suma de los elementos positivos que quedan y por lo menos la suma de los negativos. Esto sugiere definir, para $0 \leq k \leq N$,

$$\text{pos}(k) = (+i \mid k \leq i < N : A[i] \uparrow 0),$$

$$\text{neg}(k) = (+i \mid k \leq i < N : A[i] \downarrow 0).$$

Teorema 5.5.1

Sea s una solución parcial con $|s| = k$ y sea t una solución parcial completa que extiende a s . Entonces

$$\sigma(s) + \text{neg}(k) \leq \sigma(t) \leq \sigma(s) + \text{pos}(k).$$

Más aún, ambas cotas se alcanzan para alguna extensión de s .

Demostración

Como t extiende a s , se tiene

$$\sigma(t) - \sigma(s) = (+i \mid k \leq i < N : t_i \cdot A[i]).$$

Para cada i se cumple $t_i \in \{0, 1\}$, de modo que $t_i \cdot A[i]$ es 0 o $A[i]$. En cualquiera de los dos casos

$$A[i] \downarrow 0 \leq t_i \cdot A[i] \leq A[i] \uparrow 0,$$

pues $A[i] \downarrow 0$ es el menor y $A[i] \uparrow 0$ el mayor de esos dos valores. Sumando sobre i se obtienen las dos desigualdades del enunciado.

Para ver que las cotas se alcanzan, considere la extensión que toma $t_i = 1$ exactamente cuando $A[i] > 0$: su contribución es $pos(k)$. Análogamente, la que toma $t_i = 1$ exactamente cuando $A[i] < 0$ contribuye $neg(k)$.

El Teorema 5.5.1 dice que las sumas alcanzables desde s están confinadas al intervalo $[\sigma(s) + neg(k), \sigma(s) + pos(k)]$, y que ese intervalo es el más ajustado posible. Si X queda por fuera, no hay nada que buscar en ese subárbol.

Teorema 5.5.2

El predicado

$$feasible(s) : \quad \sigma(s) + neg(|s|) \leq X \leq \sigma(s) + pos(|s|)$$

es un predicado de factibilidad para el Problema 5.5.1.

Demostración

Sean t una solución y s una solución parcial con $s \preceq t$. Por el Teorema 5.5.1, $\sigma(s) + neg(|s|) \leq \sigma(t) \leq \sigma(s) + pos(|s|)$. Como t es solución, $\sigma(t) = X$, y reemplazando se obtiene $feasible(s)$.

Note que en esta demostración no se usó en ningún momento el signo de los elementos de A , que era lo que se quería lograr.

Nota 5.5.1: Comparación con el predicado del Ejemplo 5.1.1

Cuando todos los elementos de A son no negativos se tiene $neg(k) = 0$ para todo k y el predicado de esta sección se reduce a

$$\sigma(s) \leq X \leq \sigma(s) + pos(|s|).$$

La desigualdad de la izquierda es exactamente la poda del Ejemplo 5.1.1. La de la derecha es nueva, y descarta las soluciones parciales que se quedaron cortas sin remedio: aquellas a las que, aun tomando todo lo que queda, no les alcanza para llegar a X .

Es decir, el predicado de esta sección no solo es más general, sino estrictamente más fuerte incluso en el caso positivo. Se podría llamar a las dos mitades *poda por exceso* y *poda por defecto*. La Figura 5 muestra ambas actuando sobre la instancia del Ejemplo 5.1.1 y las tablas del análisis, más adelante en esta sección, muestran que la diferencia entre tener una sola mitad y tener las dos es de casi un orden de magnitud.

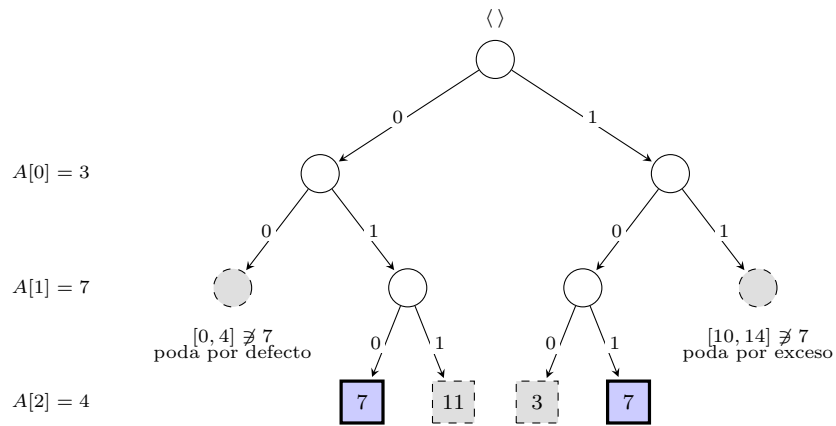


Figura 5. Árbol de reintento para $A = [3, 7, 4]$ y $X = 7$ con el predicado de factibilidad de esta sección. De los quince nodos del árbol completo, el algoritmo visita siete. Compare con la Figura 1, en la cual solo actuaba la poda por exceso.

Nota 5.5.2: Una relajación

El predicado tampoco aquí es perfecto. Para $A = [2, 2]$ y $X = 3$ la solución parcial vacía es factible, pues $0 \leq 3 \leq 4$, pero el problema no tiene soluciones.

La razón es instructiva. El conjunto de sumas alcanzables desde una solución parcial es, en general, un conjunto finito de valores con huecos; en el ejemplo, desde $\langle \rangle$ solo se alcanzan 0, 2 y 4. El predicado no trabaja con ese conjunto —calcularlo es tan difícil como resolver el problema— sino con el intervalo más pequeño que lo contiene, y los huecos se pierden. A esta idea, la de reemplazar un conjunto difícil de describir por otro más simple que lo contiene, se le llama *relajación*, y es una de las maneras más productivas de

obtener predicados de factibilidad. En la Sección 5.7 se usará de nuevo, con otro problema y con otra relajación.

5.5.3. Estado auxiliar. Hacen falta dos cosas. La primera es la suma acumulada $\sigma(s)$, que se mantiene como parámetro. La segunda son los valores de pos y neg , que no dependen de la solución parcial sino únicamente de la entrada y , que por lo tanto, se calculan una sola vez. Ambos satisfacen recurrencias inmediatas: $pos(N) = neg(N) = 0$ y, para $0 \leq k < N$,

$$\begin{aligned} pos(k) &= A[k] \uparrow 0 + pos(k+1), \\ neg(k) &= A[k] \downarrow 0 + neg(k+1). \end{aligned}$$

Calcularlas recorriendo el arreglo de derecha a izquierda es una tabulación en el sentido del Capítulo 3.

Algoritmo 5.5.1: Cotas por sufijo

```

1 def suffix_bounds(A):
2     N = len(A)
3     pos = [0]*(N+1)
4     neg = [0]*(N+1)
5     for k in range(N-1, -1, -1):
6         pos[k] = max(A[k], 0) + pos[k+1]
7         neg[k] = min(A[k], 0) + neg[k+1]
8     return pos, neg

```

Nota 5.5.3: Estado que no hay que restaurar

El estado auxiliar de esta sección contrasta con el de la Sección 5.4 en un punto que vale la pena notar. Allí los tres arreglos se modificaban al elegir y había que restaurarlos explícitamente al deshacer, so pena de perder la poscondición $Post_1$. Aquí no hay nada que restaurar: los arreglos pos y neg nunca se modifican, y la suma acumulada viaja como parámetro, de modo que cada invocación recibe su propia copia y la del invocador queda intacta.

Este es, en general, el recurso más seguro: cuando el estado auxiliar es pequeño, pasarlo por valor elimina de raíz la posibilidad de olvidar una restauración. Cuando es grande –como los $2N - 1$ Booleanos de las diagonales– copiarlo en cada nodo costaría más que mantenerlo, y no queda más remedio que modificar y restaurar.

5.5.4. Implementación. Conviene fortalecer la precondition del llamado recurrente respecto de la que se enunció para el Algoritmo 5.1.1, exigiendo que la solución parcial recibida ya sea factible y que el parámetro `sigma` sea correcto.

Pre : s es una solución parcial, $feasible(s)$ y $sigma = \sigma(s)$.

Fortalecerla tiene una recompensa concreta.

Teorema 5.5.3

Sea s una solución parcial con $|s| = N$. Entonces $feasible(s) \equiv sol(s)$.

Demostración

Las sumas $pos(N)$ y $neg(N)$ son vacías y por lo tanto valen 0. Luego $feasible(s)$ es $\sigma(s) \leq X \leq \sigma(s)$, que equivale a $\sigma(s) = X$, es decir, a $sol(s)$.

Es decir, bajo la precondition fortalecida no hace falta verificar nada en las hojas: toda solución parcial completa que el algoritmo alcance es, necesariamente, una solución.

Algoritmo 5.5.2: Suma exacta de un subconjunto

```

1 def subset_sum(A, X, s, sigma, pos, neg):
2     N = len(A)
3     if len(s) == N:
4         print([ A[i] for i in range(N) if s[i] == 1 ])
5     else:
6         k = len(s)
7         for c in [0, 1]:
8             t = sigma + c * A[k]
9             if t + neg[k+1] <= X <= t + pos[k+1]:
10                 s.append(c)                # choose
11                 subset_sum(A, X, s, t, pos, neg)
12                 s.pop()                    # undo

```

La guarda de la línea 9 es literalmente el predicado de factibilidad evaluado en $s \cdot c$, con t en el papel de $\sigma(s \cdot c)$. La invocación inicial debe calcular las cotas y establecer la precondition, lo cual incluye verificar que la solución parcial vacía sea factible.

Algoritmo 5.5.3: Arranque

```

1 def all_subset_sums(A, X):
2     pos, neg = suffix_bounds(A)
3     if neg[0] <= X <= pos[0]:
4         subset_sum(A, X, [], 0, pos, neg)

```

Ejemplo 5.5.1

Considere las siguientes invocaciones, la primera con la instancia del Ejemplo 5.1.1 y la segunda con la variante de signos mezclados que allí se señaló como problemática:

```

1 all_subset_sums([3,7,4], 7)
2 all_subset_sums([3,7,-3], 7)

```

El resultado de estas ejecuciones es el siguiente:

```

1 [7]
2 [3, 4]
3 [7]
4 [3, 7, -3]

```

La segunda instancia es la que el predicado del Ejemplo 5.1.1 habría manejado mal: allí se observó que $\langle 1, 1 \rangle$, con suma 10, se extiende a la solución $\langle 1, 1, 1 \rangle$, con suma 7. El predicado de esta sección no la poda, porque $neg(2) = -3$ y por lo tanto el intervalo alcanzable desde $\langle 1, 1 \rangle$ es $[7..10]$, que sí contiene a X .

Teorema 5.5.4

El llamado `all_subset_sums(A, X)` reporta exactamente los subconjuntos de elementos de $A[0..N)$ cuya suma es X , cada uno exactamente una vez.

Demostración

Se propone como ejercicio al lector.

5.5.5. Análisis. Como se ha planteado anteriormente, las podas permiten cambiar el valor de la constante que acompaña el comportamiento asintótico del algoritmo de exploración, pero no necesariamente revierte su naturaleza exponencial.

Teorema 5.5.5

El llamado `all_subset_sums(A, X)` con $|A| = N$ visita a lo sumo $2^{N+1} - 1$ nodos, toma tiempo $O(N \cdot 2^N)$ y usa espacio $O(N)$, sin contar el espacio de la salida.

Demostración

Se propone como ejercicio al lector.

Como en las secciones anteriores, la cota de peor caso es indiferente a la calidad de la poda y hay que medir. La siguiente tabla reporta la cantidad promedio de nodos visitados sobre cuarenta instancias aleatorias por fila, con X elegido siempre como la suma de un subconjunto aleatorio, de modo que todas las instancias tienen al menos una solución.

N	Elementos en $[1..100]$		
	Sin poda	Por exceso	Por exceso y defecto
12	8 191	4 808	1 089
16	131 071	88 426	16 208
20	2 097 151	1 102 915	218 810
24	33 554 431	20 442 316	3 406 942

N	Elementos en $[-100..100]$	
	Sin poda	Por exceso y defecto
12	8 191	1 179
16	131 071	17 048
20	2 097 151	208 907
24	33 554 431	3 578 645

Hay dos observaciones que hacer. La primera es que la poda por exceso, por sí sola, es sorprendentemente débil: para $N = 24$ apenas evita recorrer un tercio del árbol. La poda por defecto, agregada, baja el recorrido a una décima parte. La segunda es que la columna de la segunda tabla no es peor que la correspondiente de la primera, pese a que allí no hay poda por exceso posible en el sentido ingenuo: con elementos de ambos signos los intervalos alcanzables son más anchos, pero también lo es el rango de valores de X , y los dos efectos se compensan.

Ninguna de las dos tablas cambia el hecho de que el crecimiento sigue siendo exponencial. Para este problema eso no es un accidente del algoritmo: en el Capítulo 8 se demuestra que su versión de decisión es NP-completa, de modo que no se espera encontrar nada esencialmente mejor.

Nota 5.5.4: Reintento contra tabulación

El Capítulo 3 resuelve este mismo problema por tabulación, con un algoritmo cuya complejidad depende de N y de X , y no de N solamente. Ninguno de los dos domina al otro.

La tabulación es preferible cuando X es pequeño, pues su costo es proporcional a él, y es inaplicable cuando los elementos no son enteros o cuando X es enorme. El reintento es indiferente a la magnitud de X y a la naturaleza de los números –sirve igual con enteros negativos, y con racionales o reales, como se propone verificar en los ejercicios– pero su costo crece con N sin importar cuán pequeña sea la instancia en otros aspectos. Que un problema tenga dos algoritmos incomparables, cada uno mejor en una región distinta del espacio de instancias, es más frecuente de lo que sugiere la costumbre de resumir un algoritmo en una sola cota asintótica.

Ejercicios

1. Demuestre el Teorema 5.5.5.
2. Demuestre el Teorema 5.5.4 verificando las tres hipótesis del Teorema 5.3.1. Note que la segunda no se cumple literalmente, pues el Algoritmo 5.5.2 omite la verificación de *sol* en las hojas; justifique esa omisión con el Teorema 5.5.3 y verifique que el Algoritmo 5.5.3 establece la precondition fortalecida.
3. Demuestre que el Algoritmo 5.5.1 calcula correctamente *pos* y *neg*. Formule los invariantes del ciclo. ¿Cuál es su complejidad temporal?
4. La Nota 5.5.2 exhibe una instancia sin soluciones en la cual el predicado no poda. Construya una instancia *con* soluciones en la cual el algoritmo desperdicie trabajo, y cuantifique cuánto.
5. Demuestre formalmente la afirmación de la Nota 5.5.1: si todos los elementos de A son no negativos, entonces el predicado de esta sección implica el del Ejemplo 5.1.1, y la implicación recíproca es falsa.
6. Reproduzca las dos tablas de esta sección. Agregue una tercera con elementos en $[1..10]$, es decir, con muchos valores repetidos, y explique lo que observa.
7. Una heurística usual para este problema es ordenar A de mayor a menor magnitud antes de comenzar.
 - a) Explique, en términos del paso de *orden* de la metodología de la Sección 5.3, por qué cabría esperar que ayude.
 - b) Mídalo. ¿Ayuda?
8. Modifique el Algoritmo 5.5.2 para que:
 - a) cuente las soluciones en lugar de reportarlas;
 - b) reporte únicamente la primera y termine; y
 - c) reporte una solución de tamaño mínimo.
9. El problema de *partición* consiste en decidir si un arreglo de enteros positivos se puede partir en dos subconjuntos de igual suma. Redúzcalo al Problema 5.5.1 y resuélvalo con el algoritmo de esta sección.
10. El Teorema 5.5.1 y su demostración no usan en ningún momento que los elementos de A sean enteros.
 - a) Verifique que el algoritmo de esta sección funciona sin cambios con elementos racionales.
 - b) Discuta qué ocurre al ejecutarlo sobre números de punto flotante. ¿Sigue siendo correcto? Ayuda: piense en la línea 3 del Algoritmo 5.5.2 y en cómo se comparan dos valores.
11. 🌈 Existe un algoritmo para este problema que toma tiempo del orden de $2^{N/2}$, a costa de usar también ese espacio, y que consiste en partir el arreglo en dos

mitades, enumerar las sumas alcanzables de cada una y combinarlas. Investíguelo, impleméntelo y compare experimentalmente con el algoritmo de esta sección para $N = 24, 28, 32$.

5.6. Conteo de segmentaciones de texto

Considere una cadena de caracteres escrita sin espacios y la pregunta de en cuántas formas distintas puede partirse en palabras. El problema aparece al procesar textos en lenguas que se escriben sin separar las palabras, al interpretar identificadores o direcciones web que las concatenan, y al corregir la salida de sistemas de reconocimiento de escritura o de voz. En todos esos casos, si la cantidad de segmentaciones es mayor a uno es exactamente la señal de que el texto es ambiguo.

Problema 5.6.1: Conteo de segmentaciones

- Entrada:** Una cadena $A[0..N]$ de caracteres, con $N \geq 1$, y un predicado *isWord* tal que, para $0 \leq l < r \leq N$, *isWord*(l, r) indica si $A[l..r]$ es una palabra.
- Salida:** La cantidad de formas distintas de escribir $A[0..N]$ como concatenación de una secuencia de palabras.

El predicado *isWord* se supone dado y correcto; el problema no dice cómo se implementa ni cuánto cuesta evaluarlo. Note que solo se consulta sobre rangos no vacíos, por las razones que se explican más abajo.

El primer paso de la metodología de la Sección 5.3 pide precisar, además de la entrada y la salida, qué se le está pidiendo al algoritmo. Aquí no se pide enumerar las soluciones ni optimizar sobre ellas, sino contarlas: es el modo de *conteo* de los tres que anuncia la Sección 5.1 y es el único problema del capítulo que lo usa. La diferencia con los modos ya vistos no es cosmética y conviene anticiparla, porque condiciona todo lo que sigue.

- A diferencia de la enumeración, no hay que conservar las soluciones ni escribirlas; basta con un acumulador. El algoritmo podrá, por eso, prescindir por completo de la solución parcial.
- A diferencia de la búsqueda, no se puede parar temprano. Cada solución debe alcanzarse para ser contada, de modo que el árbol factible se recorre íntegro y no hay cortocircuito posible.

Conviene fijar formalmente qué se está contando.

Definición 5.6.1: Segmentación

Para $0 \leq l \leq r \leq N$, una *segmentación* de $A[l..r]$ es una secuencia de índices

$$l = p_0 < p_1 < \dots < p_k = r, \quad k \geq 0,$$

tal que $\text{isWord}(p_{t-1}, p_t)$ para todo $1 \leq t \leq k$. Los índices $p_1, \dots, p_{k-1}$ son los *puntos de corte* y cada $A[p_{t-1}..p_t]$ es una de las k palabras de la segmentación. La cadena $A[l..r]$ es *segmentable* si admite por lo menos una segmentación.

Dos segmentaciones son distintas cuando lo son como secuencias de índices. El caso $k = 0$ corresponde a la secuencia vacía de palabras y solo es posible cuando $l = r$.

Nota 5.6.1: Sobre la cadena vacía

Se supone que la cadena vacía es una palabra. La suposición es inocua: una palabra vacía no aporta nada a una concatenación, de modo que suprimir de una secuencia todas sus palabras vacías produce otra secuencia de palabras con la misma concatenación. En consecuencia, contar segmentaciones con palabras no vacías –que es lo que exige la Definición 5.6.1 al pedir cortes estrictamente crecientes– es contar todas las que importan; de otro modo la respuesta sería infinita, pues se podrían intercalar palabras vacías sin límite.

Esta es la razón por la cual isWord solo se consulta sobre rangos no vacíos. La cadena vacía tiene, con esta convención, exactamente una segmentación: la vacía. Ese hecho será el caso base del algoritmo.

Ejemplo 5.6.1

Suponga que las únicas palabras son ES, PARA, PARASOL, SAL, SALTA, SALTAMONTES, SOL, SOLES y MONTES.

- $A = \text{PARASOLES}$, con $N = 9$, tiene tres segmentaciones: $\text{PARA} \cdot \text{SOL} \cdot \text{ES}$, $\text{PARA} \cdot \text{SOLES}$ y $\text{PARASOL} \cdot \text{ES}$.
- $A = \text{PARASOLED}$, con $N = 9$, no tiene ninguna: los únicos prefijos que son palabras son PARA y PARASOL, y ni SOLED ni ED son segmentables.
- $A = \text{SALTAMONTES}$, con $N = 11$, tiene dos: la que no tiene cortes intermedios, pues la cadena completa es una palabra, y $\text{SALTA} \cdot \text{MONTES}$. Note que $\text{SAL} \cdot \dots$ no conduce a ninguna, pese a que SAL sí es una palabra: que un prefijo sea palabra no garantiza que el resto sea segmentable.

5.6.1. Decisiones. El paso siguiente de la metodología pide identificar de qué decisiones se compone una solución. Es, según se advirtió en la Sección 5.3, la decisión de diseño más importante y casi nunca es única. Este problema admite dos representaciones naturales que hay que comparar antes de escoger.

Una decisión por carácter. Una segmentación queda determinada por sus puntos de corte, que forman un subconjunto de las posiciones interiores $1, \dots, N - 1$. Representarla, entonces, es representar ese subconjunto, y la Sección 5.2 ya enseñó cómo: tomar $K = N - 1$ decisiones binarias, donde la decisión i -ésima vale 1 si hay un corte en la posición $i + 1$. Esta representación se ajusta literalmente a la Definición 5.1.1.

Una decisión por palabra. Alternativamente, se puede construir la segmentación de izquierda a derecha decidiendo, en cada paso, *dónde termina la palabra siguiente*. Una solución parcial es entonces una secuencia creciente de cortes

$$s = \langle p_1, p_2, \dots, p_k \rangle, \quad 0 < p_1 < p_2 < \dots < p_k \leq N,$$

con la convención $p_0 = 0$, y extenderla es escoger un j con $p_k < j \leq N$. La cantidad de alternativas de una decisión no es fija y las soluciones parciales tienen longitudes distintas, de modo que esta representación no es la de la Definición 5.1.1, sino la variante que propone explorar el Ejercicio 4.

Las dos son correctas y describen los mismos objetos. Lo que las separa es el tamaño del árbol que producen, y la diferencia se puede establecer con exactitud.

Teorema 5.6.1

Para $0 \leq l \leq N$, sea $f(l)$ la cantidad de segmentaciones de $A[0..l]$. Sea V_p el conjunto de soluciones parciales factibles bajo la representación por palabra y V_c el correspondiente bajo la representación por carácter. Entonces

$$\begin{aligned} |V_p| &= (+l \mid 0 \leq l \leq N : f(l)), \\ |V_c| &= (+l \mid 0 \leq l < N : f(l)(N - l)). \end{aligned}$$

Demostración

Bajo la representación por palabra, una solución parcial factible es una secuencia creciente de cortes cuyos bloques son todas palabras, es decir, exactamente una segmentación del prefijo $A[0..p_k]$; y recíprocamente, toda segmentación de un prefijo es una solución parcial factible. La correspondencia es biyectiva y agrupa por el valor de $p_k = l$, de donde $|V_p| = (+l \mid 0 \leq l \leq N : f(l))$.

Bajo la representación por carácter, una solución parcial factible de longitud k decide las posiciones $1, \dots, k$ y queda determinada por dos datos independientes: la segmentación del prefijo que sus cortes describen, digamos de $A[0..l)$ con $l \leq k$, y la propia k , que registra cuántos caracteres se han recorrido desde el último corte sin cortar. Para cada segmentación de $A[0..l)$ hay una solución parcial factible por cada k con $l \leq k \leq N-1$, esto es, $N-l$ de ellas. Sumando sobre l se obtiene la expresión.

Nota 5.6.2: Por qué se escoge una y no la otra

El Teorema 5.6.1 dice que la representación por carácter visita, por cada solución parcial que la otra visita, hasta N nodos en lugar de uno. La razón es transparente: gasta una decisión por cada posición de la cadena y la mayoría de esas decisiones –las de no cortar– no aportan información alguna ni pueden podarse, porque no cierran ningún bloque. La representación por palabra gasta una decisión por cada palabra de la solución y cada una de ellas se somete de inmediato a la prueba de factibilidad.

Medido sobre textos construidos concatenando palabras de un vocabulario aleatorio, el cociente $|V_c|/|V_p|$ pasa de 5 para $N = 13$ a cerca de 13 para $N = 37$, creciendo de manera aproximadamente lineal, como predice el teorema. En lo que sigue se usa la representación por palabra. La otra no queda descartada como ejercicio: el Ejercicio 3 pide desarrollarla completa y es la mejor manera de comprobar que una representación desafortunada no se compensa después con una buena poda.

Fijada la representación, el predicado de solución dice dos cosas: que todos los bloques son palabras y que el último corte llega al final de la cadena.

$$sol(s) : (\forall t \mid 1 \leq t \leq k : isWord(p_{t-1}, p_t)) \wedge p_k = N.$$

5.6.2. Predicado de factibilidad. Una solución parcial que ya contiene un bloque que no es palabra no tiene arreglo posible, pues los bloques ya decididos no cambian al extender la solución. Eso sugiere quedarse con la primera mitad del predicado de solución.

$$feasible(s) : (\forall t \mid 1 \leq t \leq k : isWord(p_{t-1}, p_t)).$$

Teorema 5.6.2

El predicado *feasible* es un predicado de factibilidad para el Problema 5.6.1.

Demostración

Sean $t = \langle q_1, \dots, q_m \rangle$ una solución y s una solución parcial con $s \preceq t$, digamos $s = \langle q_1, \dots, q_k \rangle$ con $k \leq m$. Los bloques de s son exactamente los primeros k bloques de t y todos ellos son palabras porque $\text{sol}(t)$. Luego $\text{feasible}(s)$.

Teorema 5.6.3

Sea $s = \langle p_1, \dots, p_k \rangle$ una solución parcial con $\text{feasible}(s)$. Entonces $\text{sol}(s)$ si y solo si $p_k = N$.

Demostración

El predicado $\text{sol}(s)$ es la conjunción de $\text{feasible}(s)$ con $p_k = N$. Si la primera se cumple por hipótesis, la conjunción equivale a la segunda.

El Teorema 5.6.3 tendrá una consecuencia agradable sobre la implementación: si el algoritmo solo desciende por soluciones parciales factibles, entonces llegar al final de la cadena es haber encontrado una solución y no hay nada que verificar.

Nota 5.6.3: Una poda que no es perfecta

El predicado no es perfecto en el sentido del Ejercicio 5, y el Ejemplo 5.6.1 ya exhibió el testigo. Para $A = \text{SALTAMONTES}$, la solución parcial $\langle 3 \rangle$ es factible, pues **SAL** es una palabra, pero ninguna solución la extiende: lo que queda, **TAMONTES**, no empieza por ninguna palabra del vocabulario.

Es la misma situación de la Nota 5.4.1 y por la misma razón de fondo: decidir si $\langle 3 \rangle$ lleva a alguna parte exige resolver el problema sobre lo que queda, que es tan difícil como el problema original. Lo que el predicado ofrece es una condición necesaria barata.

5.6.3. Estado auxiliar. El paso siguiente pregunta qué información conviene mantener para que evaluar feasible sea barato. Aquí la respuesta es sorprendentemente escueta. Suponga que $\text{feasible}(s)$ es cierto y que se quiere extender s con un corte j . Los bloques de $s \cdot j$ son los de s , que ya se sabe que son palabras, más el bloque nuevo $A[p_k..j]$. Luego

$$\text{feasible}(s \cdot j) \equiv \text{isWord}(p_k, j),$$

y para decidirlo no hace falta nada de s salvo su último corte.

Nota 5.6.4: El pasado se resume en un índice

Vale la pena detenerse en esto, porque es lo que distingue a este problema de los anteriores y lo que decide el resto del diseño. *Cuáles* palabras se hayan escogido hasta el momento no restringe en nada las que se pueden escoger de ahí en adelante: toda la información del pasado que el algoritmo necesita cabe en un único índice.

Compárese con las secciones anteriores. En el problema de las N reinas, la solución parcial completa determinaba qué columnas y qué diagonales quedaban disponibles, y hubo que mantener tres arreglos para registrarlo, actualizándolos al elegir y restaurándolos al deshacer. En la suma exacta hizo falta arrastrar la suma acumulada, aunque como parámetro y no como estructura modificable. Aquí basta con el último corte, que también viaja como parámetro, de modo que la poscondición $Post_1$ se satisface sin que haya nada que restaurar. Esta observación tiene además una consecuencia que se recoge al final de la sección.

De aquí y del modo de conteo se sigue que el algoritmo no necesita mantener la solución parcial en absoluto. Por un lado, el estado necesario para decidir la factibilidad es un solo índice. Por el otro, lo que se pide es un número, no una lista de soluciones, de modo que tampoco hay que conservar la solución parcial para reportarla. Todo lo que el algoritmo requiere saber en un nodo es dónde terminó la última palabra.

Eso justifica trabajar con la siguiente función objetivo, donde n ocupa el lugar del último corte y $n = 0$ corresponde a la solución parcial vacía.

$\phi(n)$: “la cantidad de segmentaciones de $A[n..N]$.”

El problema original se recupera como el caso particular $\phi(0)$. El dominio incluye $n = N$, es decir, el sufijo vacío. Por la Nota 5.6.1 se tiene $\phi(N) = 1$: es el caso base y es también lo que dice el Teorema 5.6.3 leído sobre esta función.

Definición 5.6.2: Función objetivo

Para $0 \leq n \leq N$,

$$\phi(n) = \begin{cases} 1 & , \text{ si } n = N, \\ (+j \mid n < j \leq N \wedge isWord(n, j) : \phi(j)) & , \text{ si } n < N. \end{cases}$$

La alternativa $j = N$, la que agota el sufijo con una sola palabra, no requiere tratamiento especial: para ella el término es $\phi(N) = 1$, de modo que aporta

una unidad exactamente cuando $isWord(n, N)$, como es de esperarse. Y si ninguna alternativa es factible, la cuantificación tiene rango vacío y $\phi(n) = 0$.

Todo llamado recurrente $\phi(j)$ ocurre con $j > n$, de modo que $N - j < N - n$: la recurrencia siempre avanza hacia el caso base y ϕ está bien definida sobre $0 \leq n \leq N$.

Nota 5.6.5: ¿Dónde quedó el reintento?

La Definición 5.6.2 es, en sí misma, el algoritmo: es ejecutable y cada caso indica exactamente qué calcular. Vista así puede costar reconocer en ella los tres pasos del Algoritmo 5.1.1, que están escondidos en la manera de evaluar la cuantificación.

Los términos se calculan uno tras otro, de izquierda a derecha: se *elige* una alternativa j , se *explora* el sufijo correspondiente, y al retornar se *des-hace* esa decisión para ensayar la siguiente. La poda está en el rango de la cuantificación: las alternativas con $\neg isWord(n, j)$ —que es, por lo visto arriba, exactamente $\neg feasible(s \cdot j)$ — no contribuyen ningún término y no generan llamado recurrente, tal como el Algoritmo 5.2.2 descartaba un candidato antes de extender la solución parcial. En el vocabulario de la Nota 5.2.2, la factibilidad se evalúa antes de elegir.

Lo que no aparece por ninguna parte es el cortocircuito. En una cuantificación existencial, hallar un testigo permite abandonar el resto; en una sumatoria hay que evaluar todos los términos. Es la diferencia entre buscar y contar, anunciada al comienzo de la sección.

5.6.4. Implementación. La transcripción de la Definición 5.6.2 es directa. El predicado se recibe como parámetro, del mismo modo que la memoria compartida se recibe como parámetro en el Algoritmo 3.2.2.

Algoritmo 5.6.1: Conteo de segmentaciones

```

1 def count(A, n, isWord):
2     N = len(A)
3     if n==N: ans = 1
4     else:
5         ans = 0
6         for j in range(n+1, N+1):
7             if isWord(n,j): ans = ans + count(A, j, isWord)
8     return ans

```

Algoritmo 5.6.2: Arranque

```

1 def segmentations(A, isWord):
2     return count(A, 0, isWord)

```

El ciclo de la línea 6 recorre las alternativas y la guarda de la línea 7 es el predicado de factibilidad. No hay ninguna verificación en el caso base, por el Teorema 5.6.3: llegar a $n = N$ por un camino de alternativas factibles es ya haber encontrado una solución, y contarla es sumar uno.

Nota 5.6.6: Sobre el orden

El paso de *orden* de la metodología de la Sección 5.3 es aquí vacío, por una razón que vale la pena hacer explícita: en el modo de conteo no hay nada que ganar reordenando. Las decisiones vienen forzadas de izquierda a derecha por la representación, y el orden en que se recorren las alternativas no puede cambiar la cantidad de nodos visitados, porque todos los nodos factibles se visitan de todas maneras.

Contrástese con la Nota 5.7.5, donde reordenar los candidatos cambiaba el recorrido en un factor de veinte. Allí el orden importaba porque encontrar pronto una buena solución permitía podar el resto; aquí no hay nada análogo. Es la contrapartida de lo observado en la Nota 5.6.5: sin cortocircuito no hay premio por acertar temprano.

Ejemplo 5.6.2

Con el vocabulario del Ejemplo 5.6.1, representado como un conjunto de cadenas:

```

1 WORDS = {'ES', 'PARA', 'PARASOL', 'SAL', 'SALTA',
2         'SALTAMONTES', 'SOL', 'SOLES', 'MONTES'}
3
4 for A in ['PARASOLES', 'PARASOLED', 'SALTAMONTES']:
5     isWord = lambda l, r: A[l:r] in WORDS
6     print(A, segmentations(A, isWord))

```

El resultado de esta ejecución es el siguiente:

```

1 PARASOLES 3
2 PARASOLED 0
3 SALTAMONTES 2

```

5.6.5. El árbol de búsqueda. Los llamados recurrentes que genera una ejecución forman un árbol: cada nodo es un llamado $\phi(n)$ y cada arco es una decisión, es decir, la escogencia de una alternativa j . Las figuras 6 y 7 muestran ese árbol para dos de las entradas del Ejemplo 5.6.1.

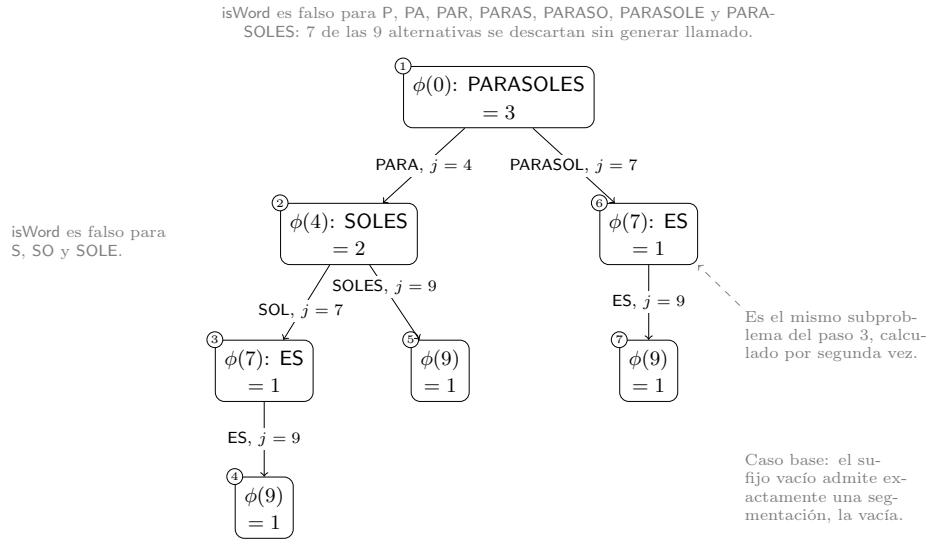


Figura 6. Árbol de búsqueda para $A = \text{PARASOLES}$. Los números indican el orden en que se visitan los nodos. El valor de un nodo interno es la suma de los valores de sus hijos.

En la Figura 6 se aprecian los tres fenómenos que caracterizan al reintento.

Poda: la mayoría de las alternativas se descartan al evaluar *isWord*, sin llegar a generar un llamado: en la raíz, 7 de las 9. El árbol que efectivamente se recorre es mucho menor que el de todas las formas de cortar la cadena.

Reintento: al retornar del paso 3 con el valor 1, la decisión $j = 7$ queda cerrada y se reintenta con la siguiente alternativa, $j = 9$, cuyo resultado se acumula sobre el anterior.

Repetición: los pasos 3 y 6 resuelven exactamente el mismo subproblema, $\phi(7)$, sin que el segundo aproveche nada de lo hecho por el primero.

Los dos primeros son la estrategia; el tercero es su costo, y de él se ocupa el análisis al final de la sección. Note que las tres hojas del árbol son llamados $\phi(9)$ que retornan 1, y que hay exactamente una por cada segmentación de la cadena. Esto no es casualidad y es lo que hará demostrable el algoritmo: las hojas del árbol de

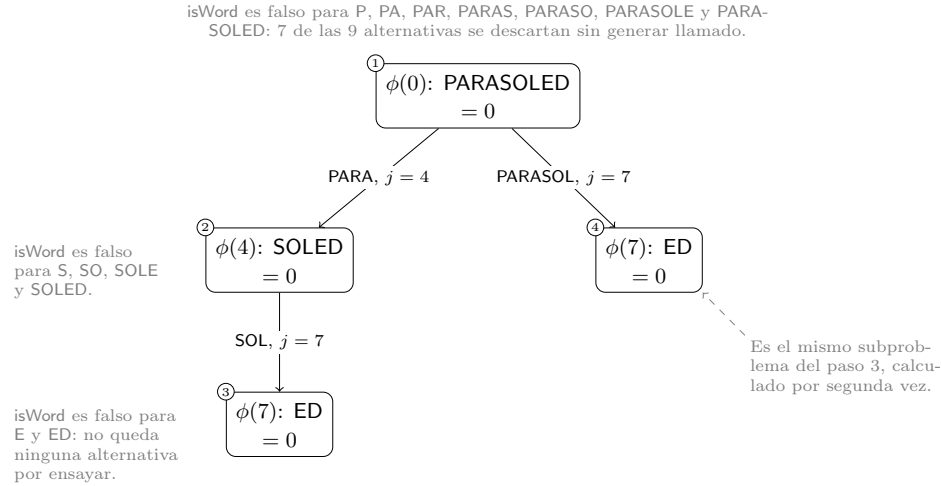


Figura 7. Árbol de búsqueda para $A = \text{PARASOLED}$. Los números indican el orden en que se visitan los nodos.

búsqueda están en biyección con las soluciones y el valor de la raíz es su cantidad. La Figura 7 es el mismo dibujo sin hojas: la cadena no es segmentable y todos los valores son 0.

5.6.6. Verificación de correctitud. El Algoritmo 5.6.1 se aparta del Algoritmo 5.1.1 en dos puntos: las soluciones parciales tienen longitudes distintas y el resultado es un número que se acumula hacia arriba en lugar de una enumeración. El Teorema 5.1.1 no aplica, entonces, directamente, y conviene demostrar la correctitud sin apoyarse en él.

Teorema 5.6.4

Sea n tal que $0 \leq n \leq N$. Entonces $\phi(n)$ es la cantidad de segmentaciones de $A[n..N]$.

Demostración

Sea $S(n)$ el conjunto de segmentaciones de $A[n..N]$. Se procede por inducción fuerte sobre la longitud del sufijo, $m = N - n \geq 0$.

Caso base ($m = 0$, es decir, $n = N$): el sufijo $A[N..N]$ es la cadena vacía.

Una segmentación suya es una secuencia $N = p_0 < \dots < p_k = N$, y

la única posible es la de $k = 0$, pues $k \geq 1$ obligaría a $p_0 < p_k$. Luego $|S(N)| = 1 = \phi(N)$.

Caso inductivo ($m \geq 1$, es decir, $n < N$): la hipótesis de inducción fuerte es que $\phi(n') = |S(n')|$ para todo n' con $N - n' < m$. Para cada j con $n < j \leq N$ y $\text{isWord}(n, j)$, sea $S_j(n)$ el conjunto de las segmentaciones de $A[n..N]$ cuya primera palabra termina en j , esto es, aquellas con $p_1 = j$.

Los conjuntos $S_j(n)$ son disjuntos dos a dos, pues una segmentación determina su p_1 . Además cubren a $S(n)$: toda segmentación de $A[n..N]$ tiene $k \geq 1$, porque $n < N$ impide $k = 0$, y por lo tanto tiene un p_1 bien definido, que satisface $n < p_1 \leq N$ y $\text{isWord}(n, p_1)$. Luego

$$|S(n)| = (+j \mid n < j \leq N \wedge \text{isWord}(n, j) : |S_j(n)|).$$

Falta ver que $|S_j(n)| = |S(j)|$. La función que a una segmentación $n < j = p_1 < \dots < p_k = N$ de $S_j(n)$ le asigna $j = p_1 < \dots < p_k = N$, es decir, la que resulta de suprimir el índice inicial, está bien definida –los bloques que quedan son los mismos y siguen siendo palabras– y tiene por inversa la que antepone n a una segmentación de $S(j)$, que es legítima porque $\text{isWord}(n, j)$. Es, entonces, una biyección entre $S_j(n)$ y $S(j)$.

Como $N - j < N - n = m$, la hipótesis de inducción da $|S(j)| = \phi(j)$. Reemplazando en la igualdad anterior se obtiene $|S(n)| = \phi(n)$, que era lo que se quería.

Conviene señalar qué papel juega cada mitad del argumento. Que los $S_j(n)$ cubran a $S(n)$ es lo que garantiza que la enumeración de alternativas es exhaustiva: ninguna segmentación se escapa, porque toda segmentación tiene una primera palabra y esa primera palabra es una de las alternativas consideradas. Que sean disjuntos es lo que garantiza que ninguna se cuenta dos veces. Son las dos mitades de la poscondición Post_0 del Algoritmo 5.1.1 –reportar todas las soluciones, cada una exactamente una vez– traducidas al modo de conteo, donde contar de más es tan grave como contar de menos.

Teorema 5.6.5

Para $N \geq 1$, el llamado `segmentations(A, isWord)` resuelve el Problema 5.6.1.

Demostración

Caso particular del Teorema 5.6.4, tomando $n = 0$.

5.6.7. Análisis. Como no se sabe cómo está implementado *isWord*, no se puede saber cuánto cuesta evaluarlo. El análisis se hace, entonces, contando la cantidad de evaluaciones de *isWord* en el peor caso.

Esta medida no deja por fuera nada relevante. Aparte de evaluar *isWord*, atender una alternativa cuesta una cantidad constante de operaciones –la aritmética sobre los índices, la suma y el llamado recurrente– y ningún llamado se genera sin que antes se haya evaluado *isWord*, de modo que la cantidad de llamados no excede en más de uno a la cantidad de evaluaciones. En consecuencia, si *isWord* costara $\Theta(1)$, el tiempo total del algoritmo sería del mismo orden que la cantidad que aquí se cuenta.

El peor caso es fácil de identificar en el modo de conteo y esa es una de sus ventajas para el análisis: como no hay cortocircuito, el recorrido no depende de cuándo se encuentren las soluciones sino únicamente de cuáles alternativas son factibles. Se recorre más árbol cuantas más lo sean, de modo que el peor caso es aquel en que todo subrango de A es una palabra.

Sea $T(m)$ la cantidad de evaluaciones sobre un sufijo de longitud $m = N - n$ en esa instancia. El caso base no evalúa *isWord*, luego $T(0) = 0$. Para $m \geq 1$ se hacen las m evaluaciones $isWord(n, j)$ para $n < j \leq N$, todas ciertas, y cada una genera un llamado recurrente sobre un sufijo de longitud $m - 1, m - 2, \dots, 0$. Es decir,

$$T(m) = (+l \mid 0 \leq l < m : T(l)) + m.$$

Nota 5.6.7: Recurrencias de historia completa

La recurrencia anterior no tiene la forma $T(m) = aT(m/b) + f(m)$ que exige el teorema maestro del Capítulo 2: el llamado no se descompone en una cantidad fija de subproblemas de tamaño proporcional a m , sino en subproblemas de todos los tamaños menores. A las recurrencias de esta forma se les llama *de historia completa*, y aparecen con naturalidad al analizar algoritmos de reintento en los cuales una decisión puede consumir una porción arbitraria de la entrada.

El recurso usual para resolverlas es escribir la misma recurrencia para $m - 1$ y restar, con lo cual la sumatoria desaparece. Para $m \geq 2$:

$$\begin{aligned} T(m) &= T(0) + \dots + T(m-2) + T(m-1) + m \\ T(m-1) &= T(0) + \dots + T(m-2) + (m-1) \end{aligned}$$

$$T(m) - T(m-1) = T(m-1) + 1$$

Es decir, $T(m) = 2T(m-1) + 1$ con $T(0) = 0$.

Teorema 5.6.6

En el peor caso, el llamado `segmentations(A, isWord)` con $|A| = N$ realiza exactamente $2^N - 1$ evaluaciones de `isWord`, y en esa instancia la respuesta es 2^{N-1} .

Demostración

Resolviendo la recurrencia $T(m) = 2T(m-1) + 1$ con $T(0) = 0$ se obtiene $T(m) = 2^m - 1$, lo cual se demuestra por inducción sobre m y se propone como ejercicio al lector; el resultado es el caso $m = N$. Para la segunda afirmación, si todo subrango es una palabra entonces toda secuencia creciente de cortes es una segmentación, de modo que hay tantas como subconjuntos de las $N - 1$ posiciones interiores.

El segundo enunciado del Teorema 5.6.6 pone el primero en perspectiva. En esa instancia el algoritmo hace algo menos de dos evaluaciones por cada segmentación que cuenta, lo cual es difícil de mejorar *si se insiste en visitar cada solución*. El problema es que no hay ninguna razón para insistir: la salida es un solo número, no una lista, y nada obliga a alcanzar cada solución por separado para saber cuántas hay. Es la enumeración disfrazada de conteo y es evitable.

Nota 5.6.8: La brecha

El costo es exponencial, pero los subproblemas distintos no lo son: ϕ solo se puede llamar sobre $N + 1$ argumentos distintos y `isWord` solo se puede evaluar sobre $\Theta(N^2)$ pares (n, j) distintos. Todo el costo exponencial proviene de recalculiar una y otra vez los mismos sufijos, como ya se veía en la Figura 6: en el árbol de búsqueda, $\phi(j)$ reaparece bajo cada secuencia de decisiones que consuma exactamente los primeros j caracteres.

El lector habrá reconocido la situación. Es la de la Sección 3.1: una función objetivo recurrente cuyos subproblemas se solapan. Y el remedio es el mismo, la memorización, que aquí baja el costo de $\Theta(2^N)$ a $\Theta(N^2)$ evaluaciones de `isWord`; hacerlo se propone como ejercicio al lector. Note que la mejora no es una aceleración marginal sino un cambio de naturaleza: con memorización se puede contar en tiempo polinomial una cantidad de soluciones que es exponencial, cosa imposible para cualquier algoritmo que las visite una por una.

Esta observación merece enunciarse en general, porque no es privativa de este problema. El reintento y la programación dinámica no son técnicas rivales sino consecutivas: la primera produce una función objetivo recurrente que explora el espacio de soluciones y la segunda la hace eficiente cuando esa exploración repite subproblemas. Lo que decide si la segunda es aplicable es precisamente el paso de *estado auxiliar* de la metodología, según se observó en la Nota 5.6.4: que el estado necesario para seguir adelante se resume en pocos parámetros, de modo que la cantidad de subproblemas distintos sea manejable. Cuando ese resumen no existe –como en el problema de las N reinas, donde el estado es el conjunto de columnas y diagonales ya ocupadas– no queda más que recorrer el árbol.

Ejercicios

1. Demuestre, por inducción sobre m , que la solución de la recurrencia $T(m) = 2T(m-1) + 1$ con $T(0) = 0$ es $T(m) = 2^m - 1$.
2. Verifique experimentalmente el Teorema 5.6.6. Instrumente el Algoritmo 5.6.1 con un contador de evaluaciones de *isWord* y ejecútelo, para $1 \leq N \leq 15$, sobre la instancia en la cual todo subrango es una palabra. Verifique también la cantidad de segmentaciones.
3. Desarrolle la representación por carácter descrita en la Sección 5.6, con $K = N - 1$ decisiones binarias.
 - a) Aplíquelo los pasos de la metodología de la Sección 5.3. En particular, formule el predicado de solución, el de factibilidad, y determine cuál es el estado auxiliar necesario. Ayuda: hace falta recordar dónde empieza el bloque que aún está abierto.
 - b) Explique por qué, en esta representación, el Teorema 5.6.3 deja de valer y hay que verificar algo al llegar a una hoja.
 - c) Impleméntela e instrumentela con un contador de nodos visitados. Verifique el Teorema 5.6.1 sobre las tres entradas del Ejemplo 5.6.1.
4. Demuestre que $f(N) = \phi(0)$ y, más generalmente, que la sucesión f del Teorema 5.6.1 satisface una recurrencia análoga a la de la Definición 5.6.2 pero sobre prefijos en lugar de sufijos. ¿Cuál de las dos conviene más para calcular la respuesta?
5. Modifique el Algoritmo 5.6.1 para que:
 - a) decida únicamente si la cadena es segmentable, deteniéndose apenas encuentre la primera segmentación;
 - b) reporte todas las segmentaciones; y

- c) reporte una segmentación con la menor cantidad posible de palabras.
Para el primer caso, explique qué le ocurre a la Nota 5.6.6: ¿sigue siendo vacío el paso de orden?
6. Agregue memorización al Algoritmo 5.6.1, en el estilo del Algoritmo 3.2.2.
 - a) Formule la condición de representación de la memoria compartida.
 - b) Demuestre que la versión con memorización es correcta.
 - c) Demuestre que realiza $O(N^2)$ evaluaciones de *isWord*.
 - d) Verifique experimentalmente la mejora sobre la instancia de peor caso del análisis, para $N = 20, 40, 80$.
 7. En el Ejemplo 5.6.2 el predicado *isWord* se implementa consultando un conjunto de cadenas.
 - a) ¿Cuánto cuesta esa evaluación? Tenga en cuenta que construir $A[l..r)$ toma tiempo proporcional a $r - l$.
 - b) ¿Cómo afecta esto al tiempo total del algoritmo?
 8. La Nota 5.6.6 afirma que en el modo de conteo el orden de las alternativas no cambia la cantidad de nodos visitados. Demuéstrelo.
 9. Considere la variante en la cual cada palabra tiene un costo y se busca la segmentación de costo mínimo.
 - a) Adapte la Definición 5.6.2.
 - b) En el sentido de la Definición 5.7.1, ¿qué papel jugaría aquí una cota optimista? Proponga una.
 - c) ¿Vuelve a tener sentido el paso de orden?
 10. 🌈 Una estructura de datos llamada *trie* permite, dado un índice n , recorrer directamente los índices j tales que $A[n..j)$ es una palabra, sin ensayar los demás. Investíguela e implemente con ella una versión del algoritmo que no evalúe *isWord* sobre alternativas que van a fallar. ¿Qué le ocurre al análisis de esta sección?

5.7. Ramificación y poda: el problema del morral

Todas las podas de las secciones anteriores son de la misma naturaleza: descartan una solución parcial porque no puede conducir a *ninguna* solución. Los problemas de optimización admiten una segunda clase de poda, más poderosa, que descarta una solución parcial porque no puede conducir a una solución *mejor que una ya encontrada*. A la combinación de las dos se le llama *ramificación y poda*.

Nota 5.7.1: Sobre el nombre

En inglés la técnica se conoce como *branch and bound*, donde *branch* alude a la ramificación del árbol de reintento y *bound* a la cota que permite descartar ramas enteras. En castellano circulan *ramificación y poda* y *ramificación y acotación*; se prefiere aquí la primera, que es la más difundida, aunque la segunda nombra mejor el ingrediente nuevo, que es la cota.

El problema que se usa para presentarla ya apareció en el Capítulo 3, resuelto por tabulación.

Problema 5.7.1: El morral

Entrada: Dos arreglos $W[0..N)$ y $V[0..N)$ de números enteros positivos, con los pesos y los valores de N elementos, y un número entero positivo C con la capacidad del morral.

Salida: Un subconjunto de elementos en $[0..N)$ de peso total a lo sumo C y de valor total máximo.

5.7.1. Decisiones y factibilidad. La representación es la de la Sección 5.2: la decisión i -ésima vale 1 si el elemento i se lleva y 0 si no, de modo que $K = N$ y $C[i] = \{0, 1\}$. Conviene nombrar el peso y el valor que una solución parcial acumula.

$$\omega(s) = (+i \mid 0 \leq i < |s| : s_i \cdot W[i]),$$

$$\nu(s) = (+i \mid 0 \leq i < |s| : s_i \cdot V[i]).$$

Una solución parcial completa es una solución cuando cabe en el morral y entre todas ellas se busca la de valor máximo.

$$sol(s) : \quad \omega(s) \leq C.$$

Teorema 5.7.1

El predicado $feasible(s) : \omega(s) \leq C$ es un predicado de factibilidad para el Problema 5.7.1 y, además, es perfecto en el sentido del Ejercicio 5.

Demostración

Sean t una solución y s una solución parcial con $s \preceq t$. Como los pesos son positivos y s es un prefijo de t , se tiene $\omega(s) \leq \omega(t) \leq C$.

Para la perfección, sea s con $\omega(s) \leq C$ y considere la extensión que no lleva ningún elemento más: su peso sigue siendo $\omega(s) \leq C$, de modo que es una solución que extiende a s .

Nota 5.7.2: Una poda perfecta que no sirve

El Teorema 5.7.1 parece una buena noticia y no lo es. En la Sección 5.2 se celebró que un predicado perfecto garantiza que ningún nodo visitado es trabajo perdido; aquí esa garantía es vacía, porque lo que se busca no es *una* solución sino la *mejor*. Que toda solución parcial factible se extienda a alguna solución no dice nada sobre si alguna de esas extensiones vale la pena.

El efecto práctico es que la poda por factibilidad, por sí sola, apenas ayuda: solo descarta las soluciones parciales que ya se pasaron de peso. La tabla del análisis, al final de la sección, muestra que para $N = 20$ eso deja sin recorrer poco más de la mitad del árbol. Para atacar un problema de optimización hace falta otra cosa.

5.7.2. Poda por optimalidad. La idea es llevar registro de la mejor solución encontrada hasta el momento y usar su valor para descartar ramas. Para ello se necesita poder afirmar, en un nodo, que nada de lo que cuelga de él supera ese valor. Eso es lo que hace una cota.

Definición 5.7.1: Cota optimista

Una función B de las soluciones parciales en los números reales es una *cota optimista* para un problema de maximización con función objetivo ν si y solo si, para toda solución t y toda solución parcial s ,

$$s \preceq t \implies \nu(t) \leq B(s).$$

La forma de la definición es deliberadamente la misma de la Definición 5.1.4, y la obligación que impone es igual de asimétrica: la cota debe ser *optimista*, es decir, nunca subestimar lo que se puede alcanzar. Sobreestimar solo cuesta trabajo desperdiciado; subestimar pierde soluciones.

Teorema 5.7.2

Sea B una cota optimista y considere el algoritmo de reintento que mantiene una variable *best* con el valor de la mejor solución hallada hasta el momento,

inicializada con el valor de alguna solución conocida y que poda toda solución parcial s con $B(s) \leq best$. Entonces, al terminar, $best$ es el valor óptimo del problema.

Demostración

Como $best$ solo se actualiza con valores de soluciones efectivamente halladas, en todo momento $best$ es el valor de alguna solución y por lo tanto $best \leq OPT$, donde OPT denota el valor óptimo. Resta demostrar la otra desigualdad.

Sea t^* una solución con $\nu(t^*) = OPT$. Hay dos casos. Si el algoritmo llegó a visitar la hoja t^* , entonces actualizó $best$ con $\nu(t^*)$ o ya tenía un valor mayor, y en cualquier caso el $best$ final es al menos OPT . Si no la visitó, es porque algún prefijo s de t^* fue podado. La poda por factibilidad no pudo ser la causa, pues *feasible* es cierto en todo prefijo de una solución; luego fue la poda por optimalidad, y en ese instante se cumplía $B(s) \leq best$. Como B es una cota optimista y $s \preceq t^*$, se tiene $OPT = \nu(t^*) \leq B(s) \leq best$. Y como $best$ nunca decrece, el valor final también es al menos OPT .

Nota 5.7.3: Estado que sobrevive al deshacer

Hasta ahora, todo el estado que un algoritmo de reintento modificaba al elegir se restauraba al deshacer; era la poscondición $Post_1$ del Algoritmo 5.1.1 y la Nota 5.1.3 insistía en no olvidar ninguna restauración. La variable $best$ rompe esa regla deliberadamente: no se restaura, y no debe restaurarse.

Esa es, de hecho, toda la diferencia entre el reintento y la ramificación y poda. En el reintento las ramas del árbol son independientes: lo que se aprende explorando una no sirve para ninguna otra y por eso el estado puede restaurarse íntegramente. En la ramificación y poda, en cambio, una rama informa a las siguientes y $best$ es el canal por el que viaja esa información. La poscondición correcta para esta sección es

$Post'_1$: al retornar, todo el estado es idéntico al de la invocación,
salvo $best$, que solo puede haber mejorado.

De aquí se sigue también que el orden en que se recorre el árbol deja de ser indiferente. En el reintento puro, cambiar el orden cambia la secuencia en que aparecen las soluciones pero no la cantidad de nodos visitados. Aquí sí la cambia, y mucho: encontrar pronto una buena solución hace que $best$ suba

pronto y que la poda actúe con más fuerza sobre todo lo que falta. Al final de la sección se cuantifica ese efecto.

5.7.3. La cota: relajación fraccionaria. Falta lo esencial, que es exhibir una cota optimista que sea a la vez ajustada y barata de calcular. La Nota 5.5.2 sugirió el camino: reemplazar el problema que falta por resolver por otro más simple que lo contenga.

El problema que falta por resolver, en un nodo s con $|s| = k$, es llenar una capacidad $R = C - \omega(s)$ con los elementos $[k..N)$. La relajación consiste en permitir llevar *fracciones* de un elemento.

Problema 5.7.2: El morral fraccionario

Entrada: Dos arreglos $W[0..M)$ y $V[0..M)$ de números positivos y un número positivo R .

Salida: Valores $x_i \in [0..1]$, para $0 \leq i < M$, que maximicen $(+i \mid 0 \leq i < M : x_i \cdot V[i])$ sujeto a $(+i \mid 0 \leq i < M : x_i \cdot W[i]) \leq R$.

Este problema, a diferencia del original, se resuelve con un algoritmo voraz: se llevan los elementos en orden decreciente de *densidad* $V[i]/W[i]$, completos mientras quepan, y del primero que no quepa se lleva la fracción que alcance. La Figura 8 lo ilustra.

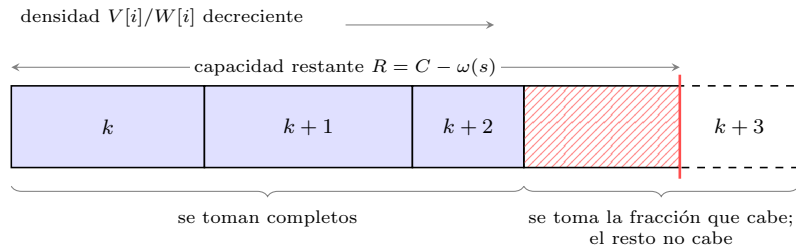


Figura 8. La solución del morral fraccionario para los elementos que faltan por decidir. Es esta cantidad, sumada a $\nu(s)$, la que sirve de cota optimista.

Teorema 5.7.3

Suponga que los elementos están ordenados de manera que $V[0]/W[0] \geq V[1]/W[1] \geq \dots \geq V[M-1]/W[M-1]$. Entonces la estrategia voraz descrita arriba resuelve el Problema 5.7.2.

Demostración

Sea $g = 1 \downarrow R/W[0]$ el valor que la estrategia voraz asigna a x_0 . Se demuestra primero que existe una solución óptima y con $y_0 = g$, y luego se concluye por inducción sobre M .

Sea y una solución óptima cualquiera. Si $y_0 = g$ no hay nada que hacer, y como g es el mayor valor que x_0 puede tomar sin violar la restricción, el único caso restante es $y_0 < g$. Sea entonces $\delta > 0$ el menor de los dos números siguientes: $(g - y_0)W[0]$ y la capacidad que y deja sin usar más el peso que y dedica a los elementos $1, \dots, M-1$. Constrúyase y' aumentando y_0 en $\delta/W[0]$ y disminuyendo los demás y_i hasta liberar un peso total de δ , lo cual es posible por la elección de δ .

La construcción no viola la restricción, pues el peso total no aumenta. Y el valor no disminuye: se ganan $\delta \cdot V[0]/W[0]$ y se pierden a lo sumo $\delta \cdot V[j]/W[j]$ repartidos entre elementos $j \geq 1$, todos ellos de densidad no mayor que la del elemento 0. Luego y' también es óptima y tiene $y'_0 > y_0$. Repitiendo el argumento se llega a una solución óptima con $y_0 = g$.

Fijado $y_0 = g$, lo que resta es el mismo problema sobre los elementos $1, \dots, M-1$ con capacidad $R - gW[0]$, que la hipótesis de inducción resuelve de manera voraz. El caso base $M = 0$ es inmediato.

Nota 5.7.4: De dónde viene esta cota

El Teorema 5.7.3 es un resultado del Capítulo 4: un criterio de selección –la densidad– y un argumento de intercambio que demuestra que seguirlo no cierra la puerta a ninguna solución óptima. Vale la pena detenerse en lo que esto significa para el libro. El problema del morral es resistente a la estrategia voraz, como se vio en su momento: llevar los elementos por densidad no resuelve el Problema 5.7.1. Relajado a fracciones, en cambio, sí cede. Y esa solución voraz de un problema que nadie pidió resolver es precisamente lo que permite podar el reintento sobre el problema que sí interesa, el mismo que el Capítulo 3 resolvió por tabulación.

Es un patrón general y conviene retenerlo: cuando un problema resiste a una técnica, relajarlo hasta que ceda ante otra suele producir la cota que la primera necesitaba.

Teorema 5.7.4

Sea s una solución parcial con $|s| = k$ y $\omega(s) \leq C$, y sea $F(k, R)$ el valor óptimo del Problema 5.7.2 sobre los elementos $[k..N]$ con capacidad R . Entonces

$$B(s) = \nu(s) + F(k, C - \omega(s))$$

es una cota optimista para el Problema 5.7.1.

Demostración

Sea t una solución con $s \preceq t$. Los valores $t_k, \dots, t_{N-1}$ pertenecen a $\{0, 1\} \subseteq [0..1]$ y satisfacen

$$(+i \mid k \leq i < N : t_i \cdot W[i]) = \omega(t) - \omega(s) \leq C - \omega(s),$$

de modo que constituyen una solución factible del Problema 5.7.2 sobre los elementos $[k..N]$ con capacidad $C - \omega(s)$. Su valor es entonces a lo sumo el óptimo de ese problema, es decir, $\nu(t) - \nu(s) \leq F(k, C - \omega(s))$, que reordenado es $\nu(t) \leq B(s)$.

5.7.4. Implementación. El Teorema 5.7.3 exige que los elementos estén ordenados por densidad decreciente. Conviene ordenarlos una sola vez, antes de comenzar: así, en cualquier nodo, los elementos que faltan por decidir –que son siempre un sufijo– ya están en el orden que la cota necesita, y calcularla es un recorrido lineal.

Algoritmo 5.7.1: Cota optimista

```

1 def bound(W, V, C, k, weight, value):
2     N = len(W)
3     b = value
4     r = C - weight
5     j = k
6     while j < N and W[j] <= r:
7         b = b + V[j]
8         r = r - W[j]
```

```

9     j = j+1
10    if j<N: b = b + V[j]*r/W[j]
11    return b

```

El ciclo de la línea 6 lleva elementos completos mientras quepan y la línea 10 agrega la fracción del primero que no cabe. El algoritmo principal mantiene *best* en una lista de dos posiciones, con el mejor valor y la mejor solución halladas; se pasa por referencia, como la memoria compartida del Algoritmo 3.2.2, para que las mejoras sobrevivan a los retornos.

Algoritmo 5.7.2: El morral por ramificación y poda

```

1 def knapsack(W, V, C, s, weight, value, best):
2     N = len(W)
3     if len(s)==N:
4         if value>best[0]: best[0], best[1] = value, s[:]
5     else:
6         k = len(s)
7         for c in [1, 0]:
8             w = weight + c*W[k]
9             v = value + c*V[k]
10            if w<=C and bound(W, V, C, k+1, w, v)>best[0]:
11                s.append(c) # choose
12                knapsack(W, V, C, s, w, v, best)
13                s.pop() # undo

```

La guarda de la línea 10 reúne las dos podas: la comparación $w \leq C$ es la factibilidad, y la de *bound* contra *best*[0] es la optimalidad. El ciclo de la línea 7 recorre los candidatos en el orden [1, 0], es decir, ensayando primero llevar el elemento; la razón se explica en el análisis.

Algoritmo 5.7.3: Arranque

```

1 def best_knapsack(W, V, C):
2     order = sorted(range(len(W)), key=lambda i: -V[i]/W[i])
3     Ws = [ W[i] for i in order ]
4     Vs = [ V[i] for i in order ]
5     best = [0, [0]*len(W)]
6     knapsack(Ws, Vs, C, [], 0, 0, best)
7     return best[0], [ order[i] for i in range(len(Ws)) if best[1][i]==1 ]

```

El arranque ordena por densidad decreciente, ejecuta la búsqueda y traduce la solución hallada de vuelta a los índices originales.

La inicialización de `best` merece un comentario, porque es más delicada de lo que parece. El Teorema 5.7.2 exige que `best` se inicialice con el valor de alguna solución conocida, y el subconjunto vacío siempre es una solución de valor 0. Pero `best` guarda dos cosas, el valor y el subconjunto que lo alcanza, y ambas deben corresponderse desde el principio: la línea 5 registra, entonces, el subconjunto vacío completo y no solamente su valor. Dejar la segunda componente sin inicializar sería un error, y uno que no se manifiesta en las instancias habituales. Considere una instancia en la cual ningún elemento cabe en el morral, por ejemplo $W = [2]$, $V = [1]$ y $C = 1$. La alternativa de llevar el elemento se descarta por la condición de peso de la línea 10; la de no llevarlo se descarta por la de optimalidad, pues su cota optimista vale 0 y la guarda exige superar estrictamente a `best`, que también vale 0. No se alcanza ninguna hoja, de modo que la línea 4 del Algoritmo 5.7.2 nunca se ejecuta y la segunda componente conserva el valor con el que se la haya inicializado.

Ejemplo 5.7.1

```
1 W = [12, 2, 1, 1, 4]
2 V = [4, 2, 2, 1, 10]
3 print(best_knapsack(W, V, 15))
```

El resultado de esta ejecución es el siguiente:

```
1 (15, [4, 2, 1, 3])
```

Es decir, se llevan los elementos 4, 2, 1 y 3, de pesos 4, 1, 2 y 1 y valores 10, 2, 2 y 1: peso total 8 y valor total 15. Note que el elemento 0, el más pesado, se queda por fuera pese a que aún sobra capacidad.

Teorema 5.7.5

El llamado `best_knapsack(W, V, C)` calcula el valor óptimo del Problema 5.7.1 y un subconjunto que lo alcanza.

Demostración

Se propone como ejercicio al lector. Ayuda: combine el Teorema 5.7.2, el Teorema 5.7.4 y el Teorema 5.7.3, y verifique que reordenar los elementos no cambia el conjunto de soluciones del problema.

5.7.5. Análisis. Ordenar cuesta $O(N \log N)$. En cada nodo, evaluar la cota cuesta $O(N)$ en el peor caso, de modo que el trabajo por nodo es $O(N)$.

Teorema 5.7.6

El llamado `best_knapsack(W, V, C)` con $|W| = N$ visita a lo sumo $2^{N+1} - 1$ nodos, toma tiempo $O(N \cdot 2^N)$ y usa espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

De nuevo la cota de peor caso no distingue entre podar bien y no podar nada, y hay que medir. La siguiente tabla reporta la cantidad promedio de nodos visitados sobre veinte instancias aleatorias por fila, con pesos y valores en $[1..100]$ y capacidad igual a la mitad del peso total.

N	Sin poda	Solo factibilidad	Con cota	Con cota y orden
12	8 191	4 779	172	22
16	131 071	76 754	406	27
20	2 097 151	1 191 407	653	48
24	33 554 431	—	1 248	52
28	536 870 911	—	2 126	53
32	8 589 934 591	—	3 338	74

Las casillas vacías corresponden a mediciones que no se hicieron porque tomaban demasiado tiempo, lo cual ya dice algo: la poda por factibilidad deja un algoritmo que no sirve. La tercera columna confirma lo anunciado en la Nota 5.7.2: para $N = 20$ apenas evita recorrer poco más de la mitad del árbol.

La cuarta columna es la que sorprende. Agregar la cota optimista lleva el recorrido de más de un millón de nodos a menos de setecientos, y a partir de ahí el crecimiento con N es casi lineal. Nada de esto contradice al Teorema 5.7.6: siguen existiendo instancias que obligan a recorrer todo el árbol, pero no son las que uno encuentra al azar.

Nota 5.7.5: Las dos mitades del orden

La última columna difiere de la anterior en el paso de *orden* de la metodología de la Sección 5.3, que tiene dos mitades: en qué orden se toman las decisiones

y en qué orden se recorren los candidatos de cada una. Aquí las dos están acopladas.

Ordenar los elementos por densidad decreciente hace que las decisiones importantes se tomen primero. Recorrer los candidatos en el orden $[1, 0]$ hace que la primera hoja que el algoritmo alcanza sea la solución voraz, que suele ser buena, con lo cual *best* arranca alto y la poda muere desde el principio. Juntas bajan el recorrido de 1 248 nodos a 52 para $N = 24$.

Lo interesante es qué pasa si se aplica solo una. Ordenar por densidad pero recorrer los candidatos en el orden $[0, 1]$ significa empezar descartando el elemento más prometedor y para $N = 24$ arroja 3 454 nodos: casi el triple que no ordenar nada. Las dos mitades del paso de orden no son independientes, y acertar en una mientras se yerra en la otra puede ser peor que no hacer nada.

Nota 5.7.6: Ramificación y poda contra tabulación

Como en la Nota 5.5.4, conviene comparar con la solución del Capítulo 3. La tabulación resuelve el problema en tiempo $\Theta(N \cdot C)$, que es excelente cuando la capacidad es pequeña y desastroso cuando es grande: con C del orden de 10^{12} la tabla no cabe en ninguna memoria, por pequeño que sea N . La ramificación y poda es indiferente a C , y también a que los pesos y valores sean enteros, pero su peor caso es exponencial en N .

Hay además una diferencia de naturaleza. La tabulación siempre paga su costo completo; la ramificación y poda paga lo que la instancia le exija, y la tabla anterior sugiere que las instancias aleatorias le exigen poco. Esa es también su debilidad: no ofrece ninguna garantía sobre cuánto tardará en una instancia concreta antes de ejecutarla.

Ejercicios

1. Demuestre el Teorema 5.7.6.
2. Demuestre el Teorema 5.7.5.
3. Demuestre que el Algoritmo 5.7.1 calcula $\nu(s) + F(k, C - \omega(s))$ cuando los elementos están ordenados por densidad decreciente. Formule los invariantes del ciclo.
4. El Teorema 5.7.2 poda cuando $B(s) \leq \text{best}$.
 - a) Demuestre que con esa condición el algoritmo encuentra una solución óptima, pero puede no encontrar todas.

- b) Modifíquelo para que reporte todas las soluciones óptimas. ¿Qué le ocurre a la cantidad de nodos visitados?
5. Considere la cota optimista trivial $B(s) = \nu(s) + (+i \mid |s| \leq i < N : V[i])$, que supone que todo lo que falta cabe en el morral.
 - a) Demuestre que es una cota optimista.
 - b) Demuestre que nunca es mejor que la del Teorema 5.7.4.
 - c) Mídala. ¿Cuánto se pierde por usarla?
6. Reproduzca la tabla de esta sección y la comparación de la Nota 5.7.5. Agregue una columna con instancias en las cuales el valor de cada elemento es proporcional a su peso. Explique lo que observa.
7. Una variante de la implementación evalúa la cota una sola vez por nodo, antes del ciclo de candidatos, en lugar de una vez por candidato. Impleméntela, explique por qué sigue siendo correcta y compare la cantidad de evaluaciones de la cota.
8. El Ejemplo 5.7.1 deja por fuera el elemento más pesado pese a que sobra capacidad. Construya una instancia en la cual la solución óptima deje por fuera el elemento de mayor densidad y explique por qué eso no contradice al Teorema 5.7.3.
9. Adapte la ramificación y poda al problema de la suma exacta de la Sección 5.5, en su versión de optimización: hallar el subconjunto cuya suma se acerque más a X sin pasarse.
 - a) Proponga una cota optimista y demuestre que lo es.
 - b) Impleméntela y compare con el algoritmo de la Sección 5.5.
10. 🌈 El recorrido en profundidad no es el único posible. La variante conocida como *el mejor primero* mantiene una cola de prioridad con los nodos vivos y expande siempre aquel cuya cota optimista sea mayor.
 - a) Impleméntela.
 - b) Demuestre que el primer nodo completo que expande es óptimo.
 - c) Compare la cantidad de nodos visitados y el espacio utilizado con los del algoritmo de esta sección. ¿Cuál prefiere?

5.8. El agente viajero, revisitado

La última sección del capítulo recoge casi todo lo anterior. El problema es el del agente viajero, ya resuelto en el Capítulo 3 por memorización sobre máscaras de bits; la representación es la de las permutaciones de la Sección 5.2; la técnica es la ramificación y poda de la Sección 5.7; y la cota, como allí, proviene de un algoritmo voraz del Capítulo 4, esta vez el del árbol de cubrimiento mínimo.

Problema 5.8.1: El agente viajero

Entrada: Una matriz $D[0..N][0..N]$ de números no negativos y simétrica, es decir, con $D[i][j] = D[j][i]$ para todo par de ciudades, donde $D[i][j]$ es el costo de ir de la ciudad i a la ciudad j .

Salida: Un recorrido que visite cada una de las N ciudades exactamente una vez y regrese a la de partida, de costo total mínimo.

La hipótesis de simetría es la misma que exige la Sección 3.9. No es inocua: de ella depende la validez de la cota que se construye más adelante, según se discute en la Nota 5.8.1.

5.8.1. Decisiones. Un recorrido es un ciclo que pasa por todas las ciudades, de modo que se puede describir dando el orden en que se visitan. Hay una redundancia que conviene eliminar de entrada.

Teorema 5.8.1

Todo recorrido puede describirse comenzando por la ciudad 0, y dos descripciones que difieran solo en la ciudad de partida tienen el mismo costo.

Demostración

Un recorrido es un ciclo, y rotar la secuencia que lo describe no cambia el conjunto de arcos recorridos: los pares de ciudades consecutivas, incluido el que cierra el ciclo, son los mismos. Como el costo es la suma de los costos de esos arcos, no cambia. Rotando hasta que la ciudad 0 quede al principio se obtiene la descripción buscada.

Fijar la ciudad de partida reduce el espacio de búsqueda de $N!$ descripciones a $(N-1)!$ recorridos distintos, sin perder ninguno. Hecho eso, la decisión k -ésima es qué ciudad se visita en el lugar $k+1$ del recorrido: $K = N-1$ y $C[k] = \{1, \dots, N-1\}$ para todo k . Una solución parcial s describe el camino

$$0 \rightarrow s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_{|s|-1},$$

y su costo es

$$\gamma(s) = D[0][s_0] + \sum_{0 \leq i < |s| - 1} D[s_i][s_{i+1}],$$

con $\gamma(\langle \rangle) = 0$. Una solución parcial completa describe un recorrido de costo $\gamma(s) + D[s_{N-2}][0]$.

Como en la Sección 5.2, el predicado de solución exige que las ciudades sean distintas par a par, y el predicado de factibilidad es el mismo, sostenido por un arreglo $used[0..N]$ de Booleanos. Esa poda, por sí sola, deja el algoritmo recorriendo el árbol de permutaciones completo: es la fuerza bruta que el Ejercicio 8 propone comparar con la memorización del Capítulo 3. Lo que sigue es lo que hace la diferencia.

5.8.2. La cota: relajación a un árbol. Se trata de un problema de minimización, de modo que la Definición 5.7.1 se dualiza: una cota optimista es ahora una cota *inferior* sobre el costo de cualquier recorrido que extienda a la solución parcial, y se poda cuando esa cota alcanza o supera el costo del mejor recorrido ya encontrado.

La cota más simple es $\gamma(s)$ misma: como los costos son no negativos, ningún recorrido que extienda a s cuesta menos que el camino ya construido. Es válida y gratuita, pero ignora por completo lo que falta por recorrer.

Para hacer algo mejor conviene mirar qué es, exactamente, lo que falta. Sea $v = s_{|s|-1}$ la última ciudad visitada y sea R el conjunto de ciudades que aún no se visitan. Completar la solución parcial es recorrer un camino que salga de v , pase por todas las ciudades de R y termine en 0. Ese camino es, en particular, un subgrafo conexo y sin ciclos que toca todos los vértices de $R \cup \{v, 0\}$: es un árbol de cubrimiento de ese conjunto. Y de todos los árboles de cubrimiento, ninguno pesa menos que el mínimo.

Es aquí donde entra la simetría de D . Hablar del árbol de cubrimiento de un conjunto de ciudades exige que el costo sea una propiedad del par $\{i, j\}$ y no del arco (i, j) ; de otro modo no hay grafo no dirigido sobre el cual calcular un árbol, ni queda claro qué peso tendría cada arista.

Teorema 5.8.2

Sea s una solución parcial no vacía, $v = s_{|s|-1}$, y R el conjunto de ciudades que no aparecen en s . Sea $acm(U)$ el peso de un árbol de cubrimiento mínimo del subgrafo no dirigido inducido por el conjunto de ciudades U , donde la arista $\{i, j\}$ pesa $D[i][j] = D[j][i]$. Entonces

$$B(s) = \gamma(s) + acm(R \cup \{v, 0\})$$

es una cota inferior del costo de todo recorrido que extienda a s .

Demostración

Sea t una solución parcial completa que extiende a s y sea c el costo del recorrido que describe. Ese recorrido se compone del camino $0 \rightarrow \dots \rightarrow v$, de costo $\gamma(s)$, y del camino que sale de v , recorre las ciudades de R en algún orden y regresa a 0. Llámese Q a este segundo camino y q a su costo, de modo que $c = \gamma(s) + q$.

El camino Q visita exactamente los vértices de $R \cup \{v, 0\}$, cada uno una vez, y sus arcos lo conectan sin formar ciclos. Por la simetría de D , cada arco (a, b) de Q cuesta lo mismo que la arista $\{a, b\}$ del subgrafo no dirigido inducido, de modo que Q , visto como conjunto de aristas, es un árbol de cubrimiento de $R \cup \{v, 0\}$ y su peso es exactamente q . Por definición de árbol de cubrimiento mínimo, $q \geq acm(R \cup \{v, 0\})$, y por lo tanto $c \geq \gamma(s) + acm(R \cup \{v, 0\}) = B(s)$.

Nota 5.8.1: Dónde se usa la simetría

La demostración anterior usa la simetría de D en un solo paso, pero es un paso del que depende todo: el de identificar el costo del arco (a, b) con el peso de la arista $\{a, b\}$. Conviene ver qué ocurre si se abandona esa hipótesis, porque el resultado no es una cota un poco peor sino un algoritmo incorrecto.

Sin simetría, el camino Q puede recorrer un par de ciudades en el sentido barato mientras el árbol de cubrimiento mínimo lo usa en el caro, de modo que acm puede *superar* a q . La cota deja entonces de ser inferior, poda soluciones parciales que llevaban al óptimo, y el algoritmo reporta un recorrido que no es el mejor. No se trata de un caso de borde: sobre matrices asimétricas con costos enteros aleatorios entre 1 y 20 y $N = 5$, el algoritmo de esta sección se equivoca en cerca de un tercio de las instancias.

El episodio deja una lección que excede a este problema. Una cota optimista no es válida en abstracto sino respecto de un problema con unas hipótesis determinadas, y relajar las hipótesis del problema sin revisar la demostración de la cota es una manera silenciosa de romper un algoritmo correcto. El Ejercicio 7 propone construir una cota que sí sirva para el caso dirigido.

La Figura 9 muestra por qué la cota es, en efecto, optimista y no exacta: el árbol de cubrimiento mínimo se ramifica, y un camino no puede hacerlo. La relajación consiste precisamente en haber olvidado esa restricción.

Nota 5.8.2: El mismo patrón, otra vez

Vale la pena poner las dos últimas secciones una al lado de la otra. En ambas, el problema resiste a la estrategia voraz. En ambas, relajar una restricción produce un problema que sí cede ante ella: permitir fracciones convierte el morral en un problema voraz, y permitir ramificaciones convierte el recorrido en un árbol de cubrimiento mínimo. Y en ambas, el óptimo del problema relajado es la cota que el reintento necesitaba.

La Nota 5.7.4 enunciaba el patrón; esta sección es la segunda evidencia de que no era una casualidad del morral.

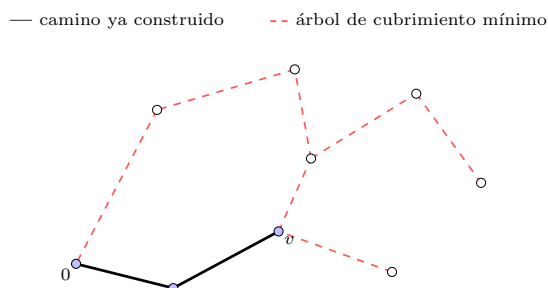


Figura 9. La cota inferior en un nodo del árbol de reintento. El camino ya construido va de 0 a v ; lo que falta por recorrer es un camino de v a 0 que pase por las demás ciudades, y todo camino de esos pesa al menos lo que el árbol de cubrimiento mínimo.

5.8.3. Implementación.

Algoritmo 5.8.1: Cota inferior por árbol de cubrimiento mínimo

```

1 def bound(D, cost, last, used):
2     N = len(D)
3     verts = [ v for v in range(1,N) if not used[v] ] + [last, 0]
4     if len(verts)>1:
5         inside = { verts[0] }
6         dist = { v: D[verts[0]][v] for v in verts[1:] }
7         while dist:
8             u = min(dist, key=dist.get)      # the one closest to the tree
9             cost = cost + dist.pop(u)
10            inside.add(u)
11            for v in dist:
12                if D[u][v]<dist[v]: dist[v] = D[u][v]
```

```
13 return cost
```

El Algoritmo 5.8.1 es el algoritmo de Prim del Capítulo 4, restringido al subgrafo inducido por las ciudades que interesan y acumulando el peso sobre el costo ya recorrido. Note que no recibe el conjunto de ciudades pendientes sino el arreglo *used*, y arma la lista de vértices por su cuenta; la Nota 5.8.3 explica por qué.

El algoritmo principal es el de la Sección 5.7 adaptado a minimización, con una diferencia en el orden de los pasos: la elección se hace *antes* de evaluar la cota y se deshace si esta no pasa. Es la variante que la Nota 5.2.2 llama evaluar la factibilidad después de elegir, y aquí es forzosa, porque la cota necesita saber que la ciudad candidata ya quedó visitada.

Algoritmo 5.8.2: El agente viajero por ramificación y poda

```
1 def tsp(D, s, used, last, cost, best):
2     N = len(D)
3     if len(s)==N-1:
4         total = cost + D[last][0]
5         if total<best[0]: best[0], best[1] = total, s[:]
6     else:
7         for c in range(1, N):
8             if not used[c]:
9                 used[c] = True           # choose
10                s.append(c)
11                if bound(D, cost+D[last][c], c, used)<best[0]:
12                    tsp(D, s, used, c, cost+D[last][c], best)
13                s.pop()                   # undo
14                used[c] = False
```

Algoritmo 5.8.3: Arranque

```
1 def best_tour(D):
2     N = len(D)
3     best = [ float('inf'), None ]
4     tsp(D, [], [False]*N, 0, 0, best)
5     return best[0], [0] + best[1]
```

Inicializar *best*[0] en infinito es correcto, aunque no es lo mejor que se puede hacer: cualquier recorrido conocido de antemano sirve como valor inicial y hace

que la poda actúe desde el primer nodo. Obtener uno con una heurística barata se propone como ejercicio al lector.

Teorema 5.8.3

El llamado `best_tour(D)` calcula el costo mínimo de un recorrido y un recorrido que lo alcanza.

Demostración

Se propone como ejercicio al lector. Ayuda: combine el Teorema 5.8.1, el Teorema 5.8.2 y la versión para minimización del Teorema 5.7.2.

5.8.4. Análisis. Todo nodo visitado es una solución parcial con ciudades distintas par a par, de modo que el árbol recorrido está contenido en el de las permutaciones de $N - 1$ elementos. Evaluar la cota cuesta $O(N^2)$.

Nota 5.8.3: Una lista de más cuesta un orden de magnitud

El espacio merece un cuidado que las secciones anteriores no exigían. Escribir la cota de manera que reciba el conjunto de ciudades pendientes es la opción natural, y conduce a reemplazar la línea 9 del Algoritmo 5.8.2 por

```

1      rest = [ v for v in range(1,N) if not used[v] and v!=c ]
2      if bound(D, cost+D[last][c], c, rest)<best[0]:

```

pasando *rest* en lugar de *used*. Esa versión visita exactamente los mismos nodos y reporta el mismo recorrido, pero su consumo de espacio no es el mismo.

La razón es que *rest* es una variable local que sigue viva mientras el llamado recurrente desciende. En un nodo de profundidad k la lista tiene $N - k - 2$ elementos, de modo que a lo largo de una rama completa coexisten listas de tamaños $N - 2, N - 3, \dots, 0$, y la cantidad de elementos vivos alcanza

$$(+k \mid 0 \leq k \leq N - 2 : N - k - 2) = \frac{(N - 1)(N - 2)}{2},$$

esto es, $\Theta(N^2)$. Medido sobre instancias euclidianas, el pico es de 10, 36 y 105 elementos para $N = 6, 10$ y 16 , que es exactamente esa fórmula.

La versión que aquí se presenta evita el problema sin pagar nada a cambio. La lista de vértices se arma dentro de *bound*, que retorna antes de que ocurra el llamado recurrente, de modo que nunca hay más de una viva a la vez y

el pico baja a N elementos. Lo único que hubo que ceder fue el orden de los pasos, anticipando la elección a la evaluación de la cota.

La diferencia entre $\Theta(N^2)$ y $\Theta(N)$ no cambia nada en la práctica para los tamaños que este algoritmo alcanza. Vale la pena señalarla de todos modos, por dos razones. La primera es que el espacio lineal es justamente lo que la Nota 5.8.4 pone en la balanza frente a la memorización del Capítulo 3, y conviene que la afirmación sea cierta del código que se escribió y no de uno parecido. La segunda es que la falla es del tipo que no se detecta leyendo: ninguna línea reserva memoria de más, y el exceso proviene únicamente de que una variable local sobrevive a un llamado recurrente.

Teorema 5.8.4

El llamado `best_tour(D)` visita a lo sumo $\sum_{k=0}^{N-1} \frac{(N-1)!}{(N-1-k)!}$ nodos, toma tiempo $O(N^2 \cdot N!)$ y usa espacio $O(N)$, sin contar el de la matriz de entrada.

Demostración

Se propone como ejercicio al lector.

La siguiente tabla reporta la cantidad promedio de nodos visitados sobre instancias aleatorias con ciudades ubicadas al azar en un cuadrado y costos iguales a las distancias euclidianas.

N	Recorridos	Solo factibilidad	Cota trivial	Cota por ACM
8	5 040	13 700	3 632	162
10	362 880	986 410	66 298	562
12	39 916 800	108 505 112	1 497 034	4 910
14	6 227 020 800	16 926 797 486	—	8 559
16	1 307 674 368 000	3 554 627 472 076	—	37 338
18	355 687 428 096 000	966 858 672 404 690	—	114 149

La segunda columna es la cantidad de recorridos distintos, $(N-1)!$, y la tercera la cantidad de nodos del árbol de permutaciones, que es lo que recorre el algoritmo si solo se poda por factibilidad; ambas son analíticas y se siguen del Teorema 5.2.3. Las dos últimas son medidas, y las casillas vacías corresponden a mediciones que no se hicieron porque tomaban demasiado tiempo.

La cota trivial, que no mira siquiera lo que falta por recorrer, ya reduce el trabajo en dos órdenes de magnitud. La cota por árbol de cubrimiento mínimo lo reduce en otros tres: para $N = 18$ visita cien mil nodos de un árbol que tiene casi 10^{15} .

Nota 5.8.4: Contra la memorización del Capítulo 3

La solución del Capítulo 3 toma tiempo $O(2^N N^2)$ y espacio $O(2^N N)$. La de esta sección tiene un peor caso mucho peor, $O(N^2 \cdot N!)$, y sin embargo la tabla muestra que sobre instancias aleatorias visita muchos menos nodos de los que la memorización tiene entradas en su tabla. ¿Cuál preferir?

La respuesta depende de qué se tema más. La memorización ofrece una garantía: nunca tardará más de $O(2^N N^2)$, cualquiera que sea la instancia. Lo que cobra por esa garantía es espacio, y lo cobra siempre: para $N = 30$ su tabla tiene más de 3×10^{10} entradas y no cabe en ninguna memoria, por fácil que sea la instancia. La ramificación y poda usa espacio $O(N)$ y suele ser rapidísima, pero no ofrece ninguna garantía: existen instancias que la obligan a recorrerlo todo, y no hay manera barata de saber de antemano si la instancia que se tiene enfrente es una de ellas.

Es el mismo dilema de la Nota 5.5.4 y de la Nota 5.7.6, y aparece una y otra vez al final de este capítulo por una razón de fondo: los problemas que aquí se atacan son, según lo que se estudiará en el Capítulo 8, problemas para los cuales no se espera una solución eficiente. Ante ellos no hay un algoritmo mejor, sino una elección entre garantías distintas.

Ejercicios

1. Demuestre el Teorema 5.8.4.
2. Demuestre el Teorema 5.8.3. En particular, enuncie y demuestre la versión para minimización del Teorema 5.7.2.
3. Demuestre que la cota trivial $B(s) = \gamma(s)$ es una cota inferior válida y que la del Teorema 5.8.2 nunca es menor que ella.
4. El Algoritmo 5.8.3 inicializa `best[0]` en infinito.
 - a) Diseñe una heurística barata que produzca algún recorrido, por ejemplo visitando siempre la ciudad más cercana entre las no visitadas.
 - b) Úsela para inicializar `best` y mida el efecto sobre la cantidad de nodos visitados.
 - c) ¿Sigue siendo correcto el algoritmo si la heurística produce un recorrido cualquiera, no necesariamente bueno?

5. Como D es simétrica, todo recorrido y su inverso tienen el mismo costo, de modo que la tabla de esta sección cuenta cada recorrido dos veces.
 - a) Proponga una restricción sobre las decisiones que elimine esa redundancia y demuestre que no pierde ninguna solución óptima.
 - b) Impleméntela y mida el ahorro.
6. Reproduzca la tabla de esta sección. Agregue una columna con instancias en las cuales los costos son enteros aleatorios simétricos sin relación con ninguna geometría, es decir, sin desigualdad triangular. ¿Siguiendo siendo buena la cota por árbol de cubrimiento mínimo? Explique.
7. Considere la variante *asimétrica* del problema, en la cual no se supone $D[i][j] = D[j][i]$; es la situación de una ciudad con calles de un solo sentido o de un terreno con pendientes.
 - a) Construya una instancia con $N = 4$ en la cual el algoritmo de esta sección reporte un recorrido que no es óptimo, y explique en qué paso de la demostración del Teorema 5.8.2 se rompe el argumento.
 - b) Demuestre que

$$B'(s) = \gamma(s) + (+u \mid u \in R \cup \{v\} : (\downarrow w \mid w \in (R \cup \{0\}) \setminus \{u\} : D[u][w]))$$
 es una cota inferior válida sin hipótesis de simetría. Ayuda: el camino que falta sale exactamente una vez de cada ciudad de $R \cup \{v\}$, y lo hace hacia una ciudad de $R \cup \{0\}$.
 - c) Demuestre que reemplazar cada costo por $\min(D[i][j], D[j][i])$ y aplicar la cota por árbol de cubrimiento mínimo también produce una cota válida.
 - d) Implemente las dos y compare la cantidad de nodos visitados sobre instancias asimétricas aleatorias. ¿Cuál prefiere?
8. El Algoritmo 5.8.2 evalúa la cota una vez por candidato, y cada evaluación recorre el arreglo `used` para armar la lista de vértices y ejecuta a Prim desde cero.
 - a) ¿Cuál es el costo total por nodo?
 - b) Proponga e implemente una versión que mantenga la lista de ciudades pendientes en una estructura compartida, actualizada al elegir y restaurada al deshacer, en vez de rearmarla en cada candidato. ¿Cuánto espacio usa ahora?
 - c) Mida si la mejora se nota. Compare la cantidad de nodos visitados y el tiempo de ejecución con los de la versión de esta sección, y explique lo que observa.
9. La Nota 5.8.3 afirma que la versión con `rest` usa $\Theta(N^2)$ de espacio.
 - a) Demuestre la fórmula $(N - 1)(N - 2)/2$ para la cantidad de elementos vivos en el peor caso.
 - b) Impleméntela e instruméntela para medir el pico de elementos vivos, y verifique la fórmula para $N = 6, 10, 16$.

- c) Verifique experimentalmente que las dos versiones visitan exactamente los mismos nodos. ¿Por qué tenía que ser así?
10. 🌐 Una cota inferior mejor que la del Teorema 5.8.2 se obtiene observando que el camino que falta no solo cubre $R \cup \{v, 0\}$ sino que además debe salir de v y llegar a 0.
- a) Demuestre que $\gamma(s) + acm(R) + m_v + m_0$ es una cota inferior válida, donde m_v y m_0 son los costos mínimos de un arco entre v y R , y entre 0 y R , respectivamente.
- b) Compárela experimentalmente con la de esta sección.
11. 🌐 Investigue la cota de Held y Karp, que refina la de esta sección ajustando iterativamente los costos de los arcos sin cambiar el conjunto de recorridos óptimos. Implementela y compare.

Notas del capítulo y referencias

El procedimiento que este capítulo describe es mucho más antiguo que su nombre. Bitner y Reingold [BR75] le atribuyen a Derrick H. Lehmer haber usado por primera vez el término *backtrack*, en la década de 1950, en el contexto de sus trabajos sobre problemas combinatorios en computadores digitales [Leh57]. La primera descripción sistemática de la técnica como método general de enumeración se debe a Walker [Wal60], y su establecimiento como técnica de diseño con derecho propio a Golomb y Baumert [GB65], quienes introdujeron buena parte del vocabulario que aún se usa, incluyendo la idea de podar con condiciones necesarias evaluadas sobre soluciones parciales. El artículo de Bitner y Reingold [BR75] es una buena lectura complementaria: presenta una historia breve de la técnica y varias mejoras de implementación.

La Sección 5.2 aborda la generación de objetos combinatorios por reintento, es decir, de manera recurrente. Hay una alternativa que este texto no desarrolla y que vale la pena conocer: generar los objetos *iterativamente*, mediante algoritmos que calculan el sucesor de un objeto dado en un orden fijado de antemano, o que traducen entre objetos y sus posiciones en ese orden. El texto de Kreher y Stinson [KS98] es una referencia recomendada. Sus capítulos 2 y 3 tratan la generación de subconjuntos, k -subconjuntos, permutaciones y otras familias con esta aproximación, y su capítulo 4 está dedicado a los algoritmos de reintento. La Nota 5.2.1 de este capítulo, que genera subconjuntos recorriendo los enteros de 0 a $2^N - 1$, es el caso más simple de esa familia de técnicas.

El problema de las N reinas de la Sección 5.4 nació como un pasatiempo. En 1848, el compositor de ajedrez Max Bezzel planteó en la *Berliner Schachzeitung* la pregunta por las ocho reinas, y en 1850 Franz Nauck publicó las 92 soluciones y

propuso la generalización a N . Por esos mismos años Gauss se ocupó del problema en su correspondencia con Schumacher, sin llegar a resolverlo. Que la lista de Nauck estuviera completa solo quedó demostrado en 1874. La cantidad de soluciones forma la sucesión A000170 de la *On-Line Encyclopedia of Integer Sequences* [OEI]; no se conoce una fórmula cerrada para ella y los valores exactos se han calculado únicamente hasta $N = 27$. El estudio de Bell y Stevens [BS09] recopila lo que se sabe sobre el problema y sus variantes, incluida la toroidal del Ejercicio 8 de esa sección. Vale la pena mencionar, por su cercanía con el estilo de este texto, que Dijkstra usó este mismo problema para ilustrar lo que llamó programación estructurada [Dij72], con una derivación detallada de un algoritmo de reintento en profundidad.

El problema de la Sección 5.5 y el de la Sección 5.7 son los dos únicos del capítulo que aparecen tres veces en el libro: resueltos por tabulación en el Capítulo 3, por reintento aquí, y demostrados NP-completos en el Capítulo 8. El algoritmo de tiempo $O(2^{N/2})$ que propone el Ejercicio 11 de la Sección 5.5 se debe a Horowitz y Sahni [HS74]; es un buen ejemplo de que reconocer un problema como NP-completo no cierra la puerta a mejoras significativas, aunque no las lleve hasta el terreno polinomial. Para el problema del morral en particular, el tratado de Martello y Toth [MT90] recoge las técnicas exactas y aproximadas conocidas hasta su publicación.

La Sección 5.6 no tiene una referencia obligada: el problema es un ejercicio clásico y su interés en este capítulo es estructural. Su función es exhibir el caso en que las decisiones tienen una cantidad variable de alternativas y, sobre todo, el caso en que el estado necesario para continuar se resume en un solo parámetro. Esa es la condición que hace aplicable la memorización del Capítulo 3, como se discute en la Nota 5.6.8, y el contraste con el problema de las N reinas —donde el estado es el conjunto de columnas y diagonales ocupadas, y no se deja resumir— es lo que da su lugar a la sección.

La ramificación y poda de la Sección 5.7 se le atribuye usualmente a Land y Doig [LD60], quienes la presentaron para programación entera; el nombre *branch and bound*, sin embargo, aparece por primera vez en el trabajo de Little, Murty, Sweeney y Karel [LSMK63], que la aplicó al problema del agente viajero. La reseña de Lawler y Wood [LW66] sistematizó la técnica pocos años después y sigue siendo una lectura provechosa.

La cota por árbol de cubrimiento mínimo de la Sección 5.8 es una versión simplificada de la que Held y Karp introdujeron en un par de artículos [HK70, HK71]. Su cota no usa árboles de cubrimiento sino 1-árboles, y sobre todo la refina ajustando iterativamente los costos de los arcos mediante una transformación que deja invariante el conjunto de recorridos óptimos pero cambia el árbol mínimo; es la que propone investigar el Ejercicio 9 de esa sección. Para el estado actual del

problema, el libro de Applegate, Bixby, Chvátal y Cook [ABCC06] documenta el diseño del programa Concorde, con el cual se han resuelto de manera exacta instancias de decenas de miles de ciudades; es una demostración notable de hasta dónde puede llegar la ramificación y poda cuando la cota es lo bastante buena.

Queda un asunto que este capítulo plantea sin resolver. La Nota 5.1.4 señala que el análisis asintótico es ciego a la calidad de la poda y propone medir empíricamente el tamaño del árbol recorrido, que es lo que hacen las tablas de las secciones siguientes. Medir, sin embargo, exige recorrer el árbol, que es precisamente lo que se querría evitar. Knuth [Knu75] propuso un método para *estimar* el tamaño de un árbol de reintento sin recorrerlo, mediante un descenso aleatorio desde la raíz que produce un estimador insesgado. Es una idea sorprendente y de lectura accesible, y permite decidir si vale la pena lanzar una búsqueda antes de lanzarla.

Computabilidad

Los computadores son máquinas fascinantes: posiblemente sean uno de los inventos más disruptivos de la humanidad después del lenguaje y la escritura. Desde su invención, han evolucionado tremendamente en relación con su poder de cómputo, y de la diversidad de tareas que pueden hacer o en las cuales pueden asistir. A pesar de esta notoria evolución, el potencial de lo que pueden hacer (o no hacer) no ha cambiado: hay un límite que no podrán rebasar aun teniendo a disposición más memoria, procesadores más veloces y almacenamiento casi ilimitado. Ese límite tampoco lo rebasa la inteligencia artificial. Por sofisticado que sea y por más datos con que haya sido entrenado, un sistema de este tipo es un programa que se ejecuta en un computador; como tal, está sujeto exactamente a las mismas restricciones que cualquier otro programa. Lo que este capítulo establece no depende de qué tan bien programado esté un computador, sino de qué es, en principio, programable.

Este capítulo resume una historia acerca del significado de la palabra “computabilidad”, de lo que significa computar y de los límites del poder de los computadores. Esta historia antecede por décadas a la aparición de los computadores y a la de las ciencias de la computación: se remonta a comienzos del siglo XX, cuando se plantearon en el terreno de las matemáticas preguntas de impacto profundo acerca de la existencia de *algoritmos* o *procedimientos computacionales efectivos* para resolver cierto tipo de problemas.

El capítulo se ocupa exclusivamente de *problemas de decisión*, es decir, de problemas computacionales que ante una entrada dada responden afirmativa o negativamente. Esa restricción no es una pérdida de generalidad y la primera sección se ocupa de mostrarlo: problemas de optimización, de búsqueda y de conteo se pueden resolver consultando repetidamente una solución al problema de decisión asociado, y la cantidad de consultas necesarias es, además, polinomial. Restringir el objeto de estudio simplifica el tratamiento sin renunciar a nada.

Sobre esa base, el capítulo construye una noción precisa de computabilidad. Los lenguajes formales sirven de marco uniforme para representar cualquier problema de decisión, y las funciones parciales y totales permiten definir, respectivamente, las nociones de aceptación y de decisión asociadas a esos lenguajes. Un algoritmo resulta ser una función computable, tal y como lo postula la tesis de Church y Turing. Para poder escribir los programas cuya existencia afirman las definiciones y no limitarse a describirlos, el texto fija un modelo de cómputo concreto; la tesis de Church y Turing es precisamente lo que garantiza que la noción resultante no dependa de esa elección. De ahí se sigue un resultado que conviene no dejar pasar: existe un solo programa capaz de ejecutar a todos los demás.

El capítulo concluye con dos resultados de signo opuesto. Por un lado, los problemas que sí se pueden resolver tienen una estructura ordenada: se reparten en tres categorías, se comportan bien frente a las operaciones usuales sobre lenguajes y son exactamente aquellos cuyos elementos se pueden listar mecánicamente. Por otro lado, son muy pocos. Un argumento basado en cardinales infinitos muestra que hay $\aleph_0$ problemas decidibles frente a $2^{\aleph_0}$ problemas que no lo son; a estos últimos se les denomina *indecidibles*. El argumento es no constructivo, de modo que demuestra que existen sin exhibir ninguno. Exhibir uno concreto requiere herramientas distintas y es el objeto de lo que sigue.

6.1. Problemas de decisión

Un problema de decisión es un problema algorítmico para el cual el conjunto de posibles respuestas es ‘sí’ o ‘no’. Generalmente, siguen un patrón de especificación. Todo problema de decisión abordado en este capítulo supone un conjunto universo U y un conjunto $X \subseteq U$ dados por el contexto del problema.

Problema 6.1.1: Especificación típica de un problema de decisión

Entrada: Un elemento $x \in U$.

Salida: ¿Está x en X (i.e., $x \in X$)?

Conviene insistir en el reparto de papeles del Problema 6.1.1, porque de él dependen las definiciones del resto del capítulo. El conjunto universo U sirve de referencia de X y se supone inferible del contexto; los dos se mantienen fijos para cada problema y únicamente el elemento x constituye realmente la entrada. La pregunta asociada a un problema de decisión consiste entonces en determinar, para cada elemento recibido x , si pertenece a X . Dicho de otro modo, un problema de decisión queda determinado por el conjunto X y nada más.

Note que la definición no dice nada acerca de la naturaleza de los elementos de U , y que tampoco necesita decirlo: la estructura interna de x es irrelevante para la pregunta que se formula. En unos problemas x será un número, en otros un grafo y en otros una pareja formada por los dos, y en todos los casos el Problema 6.1.1 se aplica sin cambios. Esa indiferencia es la que permite que la Sección 6.2 tome como U el conjunto de todas las cadenas binarias sin alterar nada de lo dicho hasta aquí.

A modo de ejemplo, se pueden enunciar algunos problemas de decisión conocidos usando el formato de especificación típica del problema de decisión presentado en el Problema 6.1.1.

Ejemplo 6.1.1

Considere los problemas que se enuncian a continuación, identificando versiones concretas de U , X y x .

Clique: determinar si un grafo (no dirigido) tiene un subgrafo completo de un tamaño dado. En este caso, U representa el conjunto de las parejas formadas por un grafo (finito) y un número natural, X el conjunto de aquellas parejas (G, k) en las cuales G tiene un subgrafo completo de tamaño k y x la pareja dada.

Primalidad: determinar si un número es primo. En este caso, U representa el conjunto de números naturales, X el conjunto de números primos y x el número sobre el cual se desea averiguar primalidad.

SAT: determinar si una proposición es satisfacible. En este caso, U corresponde al conjunto de proposiciones, X al conjunto de proposiciones satisfacibles y x a una proposición.

2Partition: determinar si un conjunto de números naturales puede ser dividido exactamente en dos subconjuntos tal que la suma de sus elementos coincida. En este caso, U corresponde al conjunto de todos los conjuntos finitos de números naturales, X al conjunto de aquellos que pueden ser particionados en dos subconjuntos con igual suma y x al conjunto dado.

Nota 6.1.1: Los problemas de Hilbert

En 1900, ante el Congreso Internacional de Matemáticos reunido en París, el matemático David Hilbert presentó una lista de problemas abiertos con la intención de orientar el trabajo del siglo que comenzaba. El décimo de ellos pide un procedimiento que determine, dada una ecuación diofántica con coeficientes enteros, si tiene o no una solución en los enteros.

Vale la pena reparar en la forma de ese enunciado. Es un problema de decisión en el sentido del Problema 6.1.1, planteado casi medio siglo antes de que existiera el primer computador y varias décadas antes de que la palabra “algoritmo” tuviera una definición precisa. Hilbert no pedía un algoritmo eficiente ni un algoritmo particular: pedía que se exhibiera uno cualquiera.

La respuesta llegó en 1970 y fue negativa: no existe tal procedimiento. Antes de poder demostrar algo así hubo que resolver una dificultad previa, que es la que ocupa buena parte de este capítulo. Para demostrar que *existe* un algoritmo basta exhibirlo; para demostrar que *no existe ninguno* hay que decir antes, con precisión matemática, qué es un algoritmo.

La importancia de los problemas de decisión radica en dos fenómenos fundamentales. Primero, los problemas de decisión son, a la vez, suficientemente fáciles de entender, y suficientemente complejos para plasmar la naturaleza de la mayoría de los problemas computacionales y abordar el estudio de la intratabilidad. Segundo, problemas de optimización, de búsqueda y de conteo pueden ser reformulados y resueltos a partir de un problema de decisión y una de sus soluciones. La investigación en el área de teoría de complejidad se enfoca típicamente en problemas de decisión, la cual ha sido ampliamente desarrollada en las últimas décadas, permeando diferentes campos del conocimiento dentro y fuera de la ingeniería y las ciencias.

El resto de esta sección desarrolla el segundo de estos fenómenos, que es el que sostiene la decisión de restringir el objeto de estudio. Lo hace en dos apartados que responden preguntas distintas. El primero pregunta si restringirse a problemas de decisión hace perder poder de resolución y concluye que no. El segundo pregunta cuánto cuesta esa restricción y concluye que el sobre costo es polinomial. La primera pregunta es la que le concierne a este capítulo; la segunda es un anticipo del Capítulo 8.

6.1.1. Restringirse a decidir no hace perder generalidad. La idea que hace funcionar todo lo que sigue es la misma en los tres casos. Un problema cuya respuesta no es ‘sí’ o ‘no’ tiene, sin embargo, una respuesta que pertenece a un rango conocido de antemano: un costo entre 0 y la suma de todos los pesos, una cantidad de soluciones entre 0 y la cantidad de candidatos, una solución entre las finitas posibles. Preguntarle a un problema de decisión asociado permite ir cercando esa respuesta hasta fijarla.

Ejemplo 6.1.2: Optimización

Considere el Problema 3.9.1 del agente viajero, con la salvedad de que aquí los pesos son enteros no negativos: dado un grafo $G = (V, E)$ completo, con función $w : E \rightarrow \mathbb{N}$ de peso sobre los arcos, determinar el costo mínimo (con respecto a w) de un circuito hamiltoniano en G . La Nota 6.1.2 explica por qué conviene esa salvedad.

Este problema de optimización cuenta con el siguiente problema de decisión asociado:

Entrada: Un grafo $G = (V, E)$ completo, con función $w : E \rightarrow \mathbb{N}$ de peso sobre los arcos y un número $k \in \mathbb{N}$.

Salida: ¿Existe en G un circuito hamiltoniano cuyo costo sea a lo sumo k ?

El problema de optimización original se puede resolver usando como caja negra (e.g., como una función que responde sí o no) un algoritmo que resuelva el problema de decisión asociado. El costo mínimo es un entero entre 0 y la suma W de todos los pesos, de modo que basta aplicar bisección sobre $[0..W + 1)$: se pregunta si existe un circuito de costo a lo sumo $\lfloor W/2 \rfloor$, y según la respuesta se repite la pregunta sobre la mitad inferior o sobre la superior del rango.

Como el rango es finito y la respuesta es uno de sus elementos, la bisección termina con el costo mínimo *exacto* al cabo de $\lceil \log_2(W + 1) \rceil$ llamados a la caja negra.

Nota 6.1.2: Por qué los pesos son enteros

El Problema 3.9.1 admite pesos reales no negativos y al estudiarlo en el Capítulo 3 esa generalidad no estorba. Aquí sí, por dos razones independientes.

La primera es que un número real no tiene, en general, representación finita, de modo que una instancia con pesos reales arbitrarios ni siquiera es una cadena y no cabe como entrada de un problema de decisión en el sentido del Problema 6.1.1.

La segunda sobrevive aun concediendo la primera. Con pesos reales la bisección no termina con el costo mínimo: lo acorrala en un intervalo tan pequeño como se quiera, pero nunca lo fija. Para que el procedimiento se detenga hay que fijar de antemano una tolerancia ε , y esa tolerancia no hace parte de la instancia. La cantidad de llamados a la caja negra deja entonces de ser una función de la entrada: se le puede exigir al procedimiento una precisión arbitrariamente fina sin cambiar un solo bit de la entrada.

Restringirse a pesos enteros elimina las dos dificultades de un golpe y no cuesta generalidad: una instancia con pesos racionales se convierte en una con pesos enteros multiplicando todos los pesos por un denominador común, lo cual no altera cuál es el circuito óptimo.

Encontrar el valor óptimo no es lo mismo que exhibir una solución que lo alcance y en la práctica lo segundo suele ser lo que interesa. El ejemplo que sigue muestra que la caja negra también sirve para eso.

Ejemplo 6.1.3: Búsqueda

Considere el problema de exhibir una asignación que satisfaga una proposición dada φ sobre las variables $p_1, \dots, p_n$, o de reportar que no existe ninguna. El problema de decisión asociado es SAT, en el Ejemplo 6.1.1.

Se dispone de una caja negra que responde si una proposición es satisfacible o no. Se comienza preguntándole por φ ; si responde que no, no hay nada que exhibir. Si responde que sí, se procede variable por variable. Para p_1 , se le pregunta a la caja negra si φ con p_1 reemplazada por verdadero es satisfacible. Si responde que sí, se fija p_1 en verdadero; si responde que no, se fija p_1 en falso, lo cual es correcto porque φ es satisfacible y toda asignación que la satisface debe entonces asignarle falso a p_1 . En cualquiera de los dos casos se reemplaza φ por la proposición que resulta de fijar p_1 y se repite con p_2 , y así sucesivamente.

Al cabo de n pasos no quedan variables por fijar y la asignación construida satisface φ . Se hicieron $n + 1$ llamados a la caja negra.

El caso del conteo es el que más se resiste a la intuición, porque la respuesta que se busca puede ser astronómica. La clave está en escoger bien el problema de decisión asociado.

Ejemplo 6.1.4: Conteo

Considere el problema de determinar cuántas asignaciones satisfacen una proposición dada φ sobre n variables. El problema de decisión asociado *no* es SAT, sino su versión con umbral:

Entrada: Una proposición φ y un número $k \in \mathbb{N}$.

Salida: ¿Hay al menos k asignaciones que satisfacen φ ?

La cantidad buscada está entre 0 y 2^n , de modo que la bisección sobre ese rango la fija exactamente al cabo de $n + 1$ llamados a la caja negra. Note

que los valores de k por los que se pregunta son enormes, pero la cantidad de preguntas no lo es.

Los tres ejemplos comparten la misma conclusión. La capacidad de resolver el problema original está contenida en la de resolver un problema de decisión asociado y por eso restringir el objeto de estudio a problemas de decisión no hace perder generalidad. Lo que falta por examinar es cuánto cuesta la traducción.

6.1.2. El precio de la restricción. En los tres ejemplos del apartado anterior se contaron los llamados a la caja negra: finitos en el primero, y $n + 1$ en el segundo y en el tercero. Vale la pena mirar esas cuentas con más cuidado, porque no solo son finitas sino *polinomiales* en el tamaño de la entrada.

En el Ejemplo 6.1.3 y en el Ejemplo 6.1.4 esto es inmediato: se hacen $n + 1$ llamados y n es la cantidad de variables de φ , que no supera el tamaño de la representación de φ . En el Ejemplo 6.1.2 el conteo es el clásico de la bisección: cada llamado reduce a la mitad la cantidad de valores que todavía pueden ser la respuesta, de modo que se hacen $\lceil \log_2(W + 1) \rceil$ llamados. Esa cantidad es polinomial en el tamaño de la entrada porque la entrada trae escritos todos los pesos, de manera que escribir su suma W no cuesta más que escribir la entrada. Note el patrón: en los tres casos la respuesta buscada vive en un rango de tamaño exponencial en el tamaño de la entrada y la bisección lo recorre en una cantidad de pasos proporcional al logaritmo de ese tamaño, es decir, polinomial. La Figura 1 ilustra el mecanismo sobre el rango del Ejemplo 6.1.2.

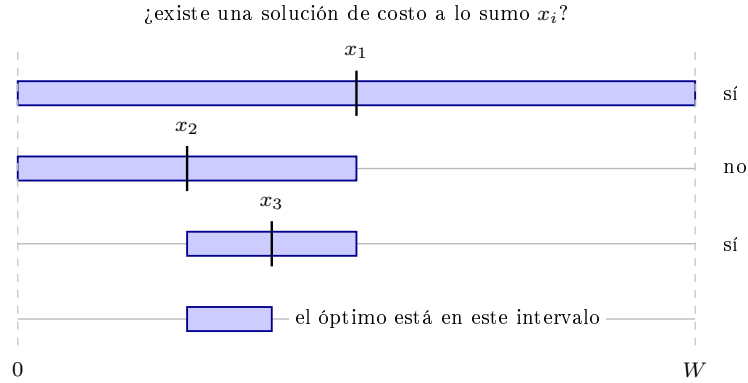


Figura 1. Bisección sobre el rango de la respuesta. La banda resaltada es el intervalo que todavía puede contener el óptimo y cada consulta a la caja negra lo reduce a la mitad. Por eso bastan del orden de $\log_2$ del tamaño del rango para fijar la respuesta.

La consecuencia es la que anuncia la introducción de esta sección. La dificultad computacional intrínseca de resolver el problema original a partir de una solución al problema de decisión asociado no es asintóticamente mayor que la de resolverlo directamente, salvo por un factor polinomial. Este hecho no se usa en el resto del capítulo, donde la pregunta es si un problema se puede resolver del todo y no a qué costo. Su lugar es el Capítulo 8, donde la distinción entre lo polinomial y lo que no lo es pasa a ser el objeto de estudio.

Ejercicios

1. Sea $N \in \mathbb{N}$ y considere una función $\phi : [0..N] \rightarrow \{0, 1\}$ no decreciente. Suponga que para cualquier $x \in [0..N]$, calcular $\phi(x)$ cuesta tiempo polinomial en el tamaño de la representación de x . Demuestre que un algoritmo de bisección sobre $[0..N]$ para determinar el mínimo valor x tal que $\phi(x) = 1$ toma tiempo polinomial en el tamaño de la representación de N . Explique por qué el enunciado sería mucho menos interesante si se fijara N de antemano, por ejemplo en 10^9 .
2. Investigue acerca de la lista de 21 problemas NP-completos publicada por Richard Karp en el artículo “Reducibility Among Combinatorial Problems” de 1972. Escoja 3 de ellos, preferiblemente entre aquellos que no conozca, y especifíquelos como problemas de decisión siguiendo el formato propuesto en esta sección.
3. Considere el Problema 3.6.1 del morral. Siguiendo el desarrollo del Ejemplo 6.1.2, reformule el problema de optimización original con base en un problema de decisión asociado. Explique claramente cómo una solución de la versión reformulada sirve para resolver el problema de optimización original.
4. El problema de satisfacibilidad de una proposición en forma normal conjuntiva (o CNFSAT) se puede formular como un problema de optimización en función de la cantidad de cláusulas. Explique cómo hacer esta formulación.
5. Suponga que se desea determinar la longitud máxima de un ciclo simple (i.e., sin repetir vértices) en un grafo dirigido. Defina un problema de decisión asociado y resuelva el problema original suponiendo que hay una solución para el problema de decisión. Calcule la complejidad temporal de la solución planteada como función de la complejidad temporal del algoritmo de decisión.
6. Investigue acerca de los científicos de la computación Stephen Arthur Cook y Leonid Levin, y explique brevemente cuál fue su principal aporte al estudio de problemas intratables.
7. En el Ejemplo 6.1.3 se afirma que si la caja negra responde que φ con p_1 en verdadero no es satisfacible, entonces se puede fijar p_1 en falso. Justifique esta afirmación con cuidado e indique exactamente dónde se usa que la primera consulta, sobre φ misma, respondió afirmativamente.
8. Adapte el Ejemplo 6.1.4 al problema de determinar cuántos circuitos hamiltonianos de costo a lo sumo k tiene un grafo dado. Especifique el problema de decisión asociado y calcule la cantidad de llamados a la caja negra.
9. La Nota 6.1.2 señala que con pesos reales la bisección no fija el costo mínimo y que la cantidad de llamados deja de ser una función de la entrada. Proponga una

manera de recuperar el enunciado: diga qué habría que agregarle a la entrada para que la cantidad de llamados vuelva a ser polinomial en su tamaño, y calcule esa cantidad. Explique además en qué sentido lo que se resuelve ya no es el problema de optimización original.

6.2. Un marco universal basado en lenguajes

Los conceptos que se trabajan en este capítulo se definen en términos de *lenguajes formales*. La razón para adoptar este marco es que ofrece una manera uniforme de representar cualquier problema de decisión, sin comprometerse todavía con ninguna manera particular de resolverlo. Qué significa resolver mecánicamente uno de estos problemas es asunto de la Sección 6.4; esta sección se ocupa únicamente de cómo se representan.

Esa separación no es solo un orden de exposición conveniente. Al no depender de ningún modelo de computación, las definiciones y los resultados de esta sección valen tal cual, cualquiera que sea el modelo que se adopte más adelante.

Nota 6.2.1: Modelos de computación

Postergar la elección de un modelo de computación es cómodo y, además, es inocuo. Los distintos modelos propuestos a lo largo del siglo XX identifican todos exactamente la misma colección de funciones computables. Entre ellos están las máquinas de Turing, el cálculo λ y las máquinas de acceso aleatorio (en inglés, *random access machine*), que son las que más se parecen a un computador moderno.

La coincidencia es, de hecho, más fuerte de lo que este capítulo necesita. Los modelos no solo computan lo mismo: se simulan unos a otros con un sobre costo moderado. Por ejemplo, cualquier algoritmo ejecutable por una máquina de acceso aleatorio puede ser simulado por una máquina de Turing de una sola cinta y una sola cabeza con un factor polinomial de costo de cómputo. Ese refinamiento no se usa aquí, donde la pregunta es qué se puede computar y no a qué costo, pero es el que hace que las clases del Capítulo 8 queden bien definidas con independencia del modelo.

Definición 6.2.1

Un *alfabeto* es un conjunto finito de símbolos. Un *lenguaje (formal)* L sobre un alfabeto Σ es un conjunto de cadenas formadas con los símbolos de Σ . Se

usan los símbolos λ para denotar la *cadena vacía*, $\emptyset$ el *lenguaje vacío* y Σ^* la *colección de todas las cadenas sobre Σ* .

De acuerdo con las convenciones introducidas en la Definición 6.2.1, cualquier lenguaje L sobre un alfabeto Σ es en realidad un subconjunto de Σ^* , i.e., $L \subseteq \Sigma^*$. Una ventaja de los lenguajes es que sobre ellos se pueden definir fácilmente operaciones de interés.

Definición 6.2.2

Sea Σ un alfabeto:

- El *complemento* $\bar{L}$ de un lenguaje $L \subseteq \Sigma^*$ es el conjunto

$$\bar{L} = \Sigma^* \setminus L.$$

- La *concatenación* $L_1 L_2$ de dos lenguajes $L_1, L_2 \subseteq \Sigma^*$ es el conjunto

$$L_1 L_2 = \{s_1 s_2 \mid s_1 \in L_1 \wedge s_2 \in L_2\}.$$

- La *concatenación generalizada* L^n , con $n \in \mathbb{N}$, de un lenguaje $L \subseteq \Sigma^*$ es el conjunto definido inductivamente de la siguiente manera:

$$L^0 = \{\lambda\}$$

$$L^{n+1} = L^n L.$$

Las operaciones de *unión* (i.e., $\cup$) e *intersección* (i.e., $\cap$) sobre lenguajes son las usuales sobre conjuntos. Se usa además el *conjunto potencia* $\mathcal{P}(-)$, heredado de la teoría de conjuntos. Note que $\mathcal{P}(\Sigma^*)$ es la colección de todos los lenguajes sobre Σ y no un lenguaje.

El complemento $\bar{L}$ de un lenguaje L es la colección de las cadenas que no están en L . La concatenación $L_1 L_2$ es el conjunto de cadenas que pueden ser partidas en exactamente dos partes: la primera parte está en L_1 y la segunda en L_2 . La concatenación generalizada L^n es el conjunto de cadenas que se construyen a partir de la concatenación de n cadenas en L . De acuerdo con la definición de conjunto potencia heredada de la teoría de conjuntos, las expresiones $L \subseteq \Sigma^*$ y $L \in \mathcal{P}(\Sigma^*)$ son equivalentes.

Ejemplo 6.2.1

Considere cada una de las siguientes situaciones.

- Si $\Sigma = \{0, 1, \dots, 9\}$ (i.e., el conjunto de los dígitos en base 10), entonces

$$L_1 = \{w \mid w \in \Sigma^* \wedge w \neq \lambda\}$$

es el conjunto de los números naturales en representación decimal, con repeticiones, pues los ceros a la izquierda dan varias representaciones de un mismo número; y

$$L_2 = \{w \mid w \in L_1 \wedge \text{"}w \text{ termina en } 0\text{"}\}$$

es el conjunto de números naturales múltiplos de 10 en representación decimal.

- Si Σ es el alfabeto del castellano, entonces

$$L_1 = \{w \mid w \in \Sigma^* \wedge w \neq \lambda \wedge \text{"}w \text{ está en la base de datos de la RAE"}\}$$

es la colección de palabras del castellano y

$$L_2 = \overline{L_1}$$

la colección de cadenas que no son palabras del castellano. Note que L_2 contiene la cadena vacía y muchas cadenas que no son palabras en ningún sentido.

Nota 6.2.2

La inclinación por usar Σ^* para denotar el conjunto de todas las cadenas sobre Σ no es caprichosa. Obedece al hecho de que este conjunto corresponde a la *clausura de Kleene* de Σ , definida como:

$$\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n,$$

es decir, el conjunto de todas las cadenas finitas sobre el alfabeto Σ .

La expresión Σ^n merece una aclaración. La concatenación generalizada de la Definición 6.2.2 está definida sobre lenguajes, y Σ es un alfabeto y no un lenguaje. Lo que se hace es identificar el alfabeto Σ con el lenguaje $\{a \mid a \in \Sigma\}$ de las cadenas de un solo símbolo. Bajo esa identificación, Σ^n es el conjunto de las cadenas de longitud exactamente n .

Aunque Σ^* es infinito, sus elementos se pueden disponer en una lista, uno tras otro, sin omitir ninguno. Basta ordenarlos primero por longitud y, entre los de igual longitud, alfabéticamente.

Definición 6.2.3: Orden canónico

Sea Σ un alfabeto con un orden total fijo sobre sus símbolos. El *orden canónico* sobre Σ^* es el orden total que resulta de comparar dos cadenas u y v de la siguiente manera: u precede a v si u es más corta que v ; y si ambas tienen

la misma longitud, u precede a v cuando u precede alfabéticamente a v . Se escribe w_n para denotar la cadena que ocupa la posición n en este orden, contando desde 0.

Ejemplo 6.2.2

Para $\Sigma = \{0, 1\}$, con 0 antes que 1, el orden canónico comienza así:

$\lambda, 0, 1, 00, 01, 10, 11, 000, 001, \dots$

Es decir, $w_0 = \lambda$, $w_1 = 0$, $w_4 = 01$ y $w_6 = 11$.

Note el contraste con el orden alfabético a secas, bajo el cual la lista comenzaría con $\lambda, 0, 00, 000, \dots$ y nunca alcanzaría la cadena 1. El orden canónico evita ese problema porque, para cada longitud, hay solo una cantidad finita de cadenas.

La observación importante acerca del orden canónico es que ninguna cadena queda infinitamente lejos del comienzo de la lista: para cualquier $w \in \Sigma^*$ existe un único $n \in \mathbb{N}$ tal que $w = w_n$. Dicho de otra forma, el mapa $n \mapsto w_n$ es una biyección entre $\mathbb{N}$ y Σ^* . Esta propiedad se usa en la Sección 6.6 y reaparece en la Sección 6.7.

El alfabeto de interés en el desarrollo de esta sección y este capítulo, sin pérdida de generalidad, es el binario (i.e., $\Sigma = \{0, 1\}$). Con ello se supone la existencia de una función inyectiva que identifica cualquier elemento o tupla de elementos (de un universo contable) con una secuencia binaria en $\{0, 1\}^*$. Se supone, además, que esa identificación se puede llevar a cabo mecánicamente en las dos direcciones: dado el objeto se puede escribir su código y dado el código se puede recuperar el objeto. Esta exigencia no es un tecnicismo. Los programas del capítulo reciben códigos y operan sobre lo que esos códigos representan, de modo que sin ella ni siquiera se podrían escribir.

Nota 6.2.3

Se usa la notación $\langle - \rangle$ para denotar una *función de codificación* con rango en Σ^* . Por ejemplo, $\langle 5 \rangle$ identifica el código binario que representa a 5, mientras que $\langle \text{“abracadabra”} \rangle$ el código binario que representa la cadena “abracadabra”. Esta notación se puede usar sobre otros tipos de estructuras como grafos, matrices, etc.

Cuando se necesite el sentido inverso se dice que un programa *decodifica* una cadena, es decir, que recupera de ella el objeto o la tupla que representa. En el código de este capítulo esa operación aparece con el nombre **decode**.

No toda cadena de Σ^* es el código de algo, pues cualquier codificación deja por fuera cadenas mal formadas. Reconocer si una cadena lo está es un análisis sintáctico y, por lo tanto, mecánico, de modo que todo programa que reciba códigos puede empezar por comprobarlo. Esa comprobación se da por supuesta de aquí en adelante; lo que el programa hace cuando falla depende de su oficio y se indica donde importa. Un algoritmo de decisión, por ejemplo, rechaza las cadenas mal formadas, con lo cual sigue siendo total sobre todo Σ^* .

La palabra “mecánicamente” se está usando aquí de manera informal, pues su significado preciso solo aparece en la Sección 6.4. Una vez disponible, la exigencia se enuncia sin rodeos: las funciones de codificación y de decodificación son computables.

Es importante anotar que existen varias funciones de codificación. Sin embargo, hacer distinciones no es relevante para el tratamiento que sigue. Además, note que elegir una función de codificación en particular es coherente con la situación que se tiene en los computadores que procesan y calculan volúmenes brutales de información representada en cadenas de bits: cada sistema de archivos puede usar un formato distinto. Tampoco es indispensable que los códigos sean binarios; por ejemplo, pueden ser ternarios o usar cualquier otro alfabeto, siempre que al convertir un código binario al alfabeto de elección la longitud del resultado quede acotada por un polinomio en la longitud del original. A las codificaciones que respetan esta restricción se les denomina *codificaciones razonables*. Por ejemplo, el alfabeto $\{1\}$ (i.e., el unario) no es razonable; ¿por qué?

Conviene precisar el alcance de esta última restricción. Ningún resultado de este capítulo depende de ella: en lo que concierne a la computabilidad, la codificación unaria es tan válida como cualquier otra y lo único que la exigencia polinomial evita es que la elección de la representación altere artificialmente el tamaño de la entrada. Eso pasa a ser determinante cuando el objeto de estudio es el costo y no la posibilidad, es decir, en el Capítulo 8.

Finalmente, una bondad de usar lenguajes formales como marco general de representación de problemas computacionales tiene que ver con la facilidad para formular problemas de decisión. Note que, de acuerdo con la especificación típica de un problema de decisión en la Sección 6.1, un problema de decisión es entonces una pregunta sobre la pertenencia o no de una cadena binaria a un lenguaje. El papel que allá cumplen el universo U y el conjunto X lo cumplen acá el conjunto de todas las cadenas binarias y un lenguaje $L \subseteq \{0, 1\}^*$ fijado por el contexto, de modo que la especificación se traduce sin cambios.

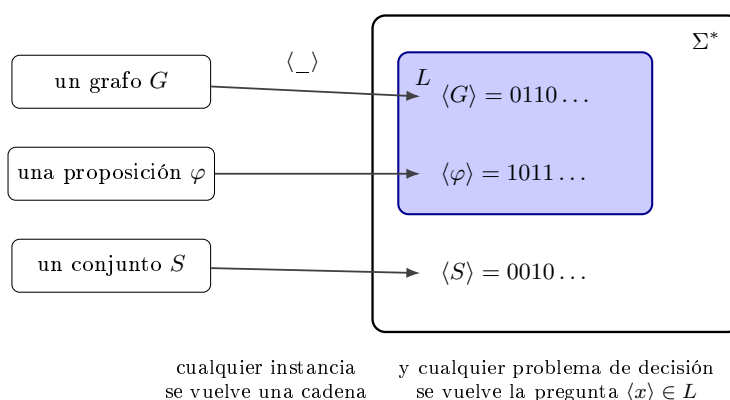
Problema 6.2.1: Especificación de un problema de decisión con lenguajes**Entrada:** Un elemento $x \in \{0, 1\}^*$.**Salida:** ¿ $x \in L$?

Figura 2. La función de codificación de la Nota 6.2.3 lleva instancias de cualquier tipo a cadenas de Σ^* y con ello convierte cualquier problema de decisión en una pregunta de pertenencia a un lenguaje.

Gracias al uso de lenguajes formales, un problema de decisión queda reducido a una pregunta de pertenencia, tal y como lo resume la Figura 2, y esa es toda la estructura que se necesita para lo que sigue. La Sección 6.3 recurre a funciones parciales y totales para precisar qué significa responder una pregunta de este tipo, y la Sección 6.4 precisa qué significa que esa respuesta se pueda obtener mecánicamente.

Conviene anotar que este mismo marco es el que sostiene, más adelante en el texto, el estudio de las clases P y NP , la noción de completitud y la técnica de reducción polinomial, que son objeto del Capítulo 8.

Ejercicios

1. Investigue acerca de máquinas de acceso aleatorio (en inglés, *random access machine*) y explique su relación con los computadores modernos.
2. Investigue acerca de máquinas de Turing. Elabore una justificación de la afirmación que aparece en la Nota 6.2.1: cualquier algoritmo ejecutable por una

- máquina de acceso aleatorio puede ser simulado por una máquina de Turing de una sola cinta y una sola cabeza con un factor polinomial de costo de cómputo.
3. Explique con ejemplos y justifique por qué el alfabeto unario no es razonable.
 4. Sea $\Sigma = \{0, 1\}$ y considere los lenguajes $L_1 = \{0, 01\}$ y $L_2 = \{\lambda, 1\}$. Calcule explícitamente L_1L_2 , L_2L_1 y L_1^2 . ¿Es la concatenación de lenguajes conmutativa?
 5. Sea $L \subseteq \Sigma^*$ y sean $m, n \in \mathbb{N}$. Demuestre que $L^{m+n} = L^mL^n$. Demuestre también que $L^0L = L$.
 6. Sea $L \subseteq \Sigma^*$. Demuestre que $\lambda \in L$ si y solo si $L^n \subseteq L^{n+1}$ para cualquier $n \in \mathbb{N}$.
 7. Determine cuáles de las siguientes afirmaciones son ciertas y justifique en cada caso:
 - a) $\emptyset$ es un lenguaje sobre Σ .
 - b) $\{\lambda\} = \emptyset$.
 - c) $\Sigma^* \in \mathcal{P}(\Sigma^*)$; $\Sigma^* \subseteq \mathcal{P}(\Sigma^*)$.
 - d) $\emptyset^0 = \emptyset$.
 8. Sea $L \subseteq \Sigma^*$ y sea Σ' un alfabeto con $\Sigma \subsetneq \Sigma'$. Explique por qué el complemento de L sobre Σ y el complemento de L sobre Σ' son distintos, y por qué la Definición 6.2.2 exige fijar el alfabeto antes de hablar de complemento.
 9. Considere $\Sigma = \{0, 1, 2\}$ con el orden 0, 1, 2 sobre sus símbolos. Liste las primeras doce cadenas en el orden canónico de la Definición 6.2.3. Deduzca una fórmula para la cantidad de cadenas de longitud a lo sumo k sobre un alfabeto de m símbolos y úsela para determinar el índice n de una cadena dada.
 10. Demuestre que el orden canónico de la Definición 6.2.3 es un orden total sobre Σ^* . Indique en qué punto se usa que Σ es finito.
 11. Proponga una función de codificación $\langle -, - \rangle$ para pares de cadenas binarias, es decir, una manera de representar un par (u, v) con $u, v \in \{0, 1\}^*$ mediante una única cadena binaria. Justifique que es inyectiva y describa cómo se recupera el par a partir del código. Ayuda: concatenar u y v no funciona; ¿por qué?

6.3. Aceptación, rechazo y decisión

El objetivo de esta sección es presentar y estudiar las nociones de aceptación y decisión asociadas a los lenguajes formales. Esto se hace con base en funciones parciales y totales, respectivamente, con dominio en las secuencias binarias $\{0, 1\}^*$ y rango en el alfabeto binario $\{0, 1\}$. En esta sección se fija el alfabeto binario $\Sigma = \{0, 1\}$. Mientras no se diga lo contrario, una función no se supone total: puede estar indeterminada en algunos puntos de su dominio. El término *parcial* se usa en

ese sentido, es decir, como “no necesariamente total”, de modo que toda función total es en particular parcial.

Haber adoptado el marco de los lenguajes formales permite expresar concisa y consistentemente la relación entre problemas de decisión y la forma en que estos se pueden resolver.

Definición 6.3.1: Aceptación y rechazo

Sea $A : \Sigma^* \rightarrow \{0, 1\}$ una función. Para $x \in \Sigma^*$, se dice que:

- A *acepta* x sii $A(x) = 1$ y
- A *rechaza* x sii $A(x) = 0$.

El *lenguaje aceptado* por A , denotado $L(A)$, es el conjunto:

$$L(A) = \{x \mid x \in \Sigma^* \wedge A(x) = 1\}.$$

La Definición 6.3.1 deja abierta una tercera posibilidad que conviene hacer explícita desde ahora. Ante una cadena $x \in \Sigma^*$, una función A hace exactamente una de tres cosas: la acepta si $A(x) = 1$; la rechaza si $A(x) = 0$, o ninguna de las dos si $A(x)$ está indeterminada. La Figura 3 ilustra las tres posibilidades. En particular, aceptar y rechazar no son alternativas complementarias: negar que A acepte x no equivale a afirmar que A la rechaza. Esa tercera posibilidad es la que separa las dos nociones centrales de la sección y la que desaparecería si toda función fuera total.

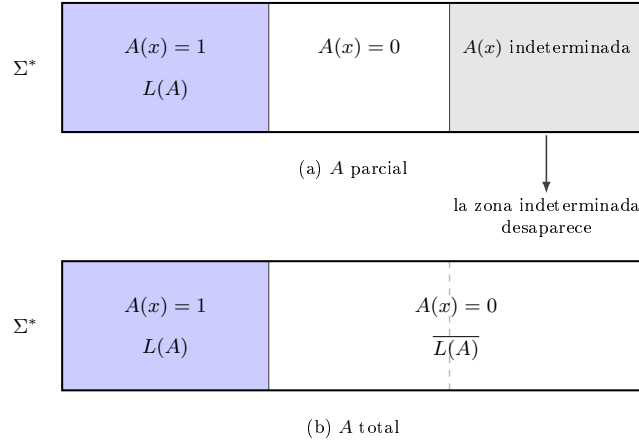


Figura 3. Las tres posibilidades ante una cadena $x \in \Sigma^*$ y lo que ocurre cuando A es total. En el panel (a) la zona indeterminada impide concluir que $L(A)$ sea la zona de rechazo; en el panel (b) sí lo es, tal y como lo establece el Teorema 6.3.1.

En el marco de los lenguajes formales, la inclinación es por representar una solución a un problema de decisión como una función que, dada una palabra binaria, calcula un valor binario. Por convención, aceptar una cadena se indica con el valor 1 y rechazarla con el 0, similar a lo que se tiene en la lógica Booleana implementada en los computadores de hoy en día. Dada una función A , se asocia a esta el lenguaje $L(A)$ que corresponde exactamente a las cadenas aceptadas por A . Evitar que las funciones sean necesariamente totales es clave, como se verá más adelante, para caracterizar las nociones de aceptación y decisión asociadas a los problemas de decisión (i.e., a lenguajes binarios).

Ejemplo 6.3.1

Considere las siguientes afirmaciones para una función $A : \Sigma^* \rightarrow \{0, 1\}$:

- Si $A(x) = 1$ para cualquier $x \in \Sigma^*$, entonces $L(A) = \Sigma^*$. Hacia una contradicción, suponga lo contrario: es decir, que $L(A) \neq \Sigma^*$. Entonces, necesariamente $L(A) \subsetneq \Sigma^*$. En consecuencia, hay una cadena $s \in \Sigma^* \setminus L(A)$. Por definición, $A(s) \neq 1$ dado que s no está en $L(A)$ (¿por qué no se puede afirmar que $A(s) = 0$?). Pero esto es una contradicción dado que se ha supuesto que $A(x) = 1$ para cualquier $x \in \Sigma^*$; en particular, para s . Luego, necesariamente $L(A) = \Sigma^*$.
- Si $A(x) = 0$ para cualquier $x \in \Sigma^*$, entonces $L(A) = \emptyset$. La justificación se propone como ejercicio al lector.

En la justificación de la primera parte del Ejemplo 6.3.1, se concluye que $A(s) \neq 1$ cuando $s \notin L(A)$, pero no que $A(s) = 0$. Este es un aspecto aparentemente irrelevante, pero es en realidad extremadamente importante porque no toda función parcial es total. Note que, en general, es posible que $A(s)$ esté indefinido, es decir, que s sea uno de los puntos de indeterminación de A (i.e., para el cual A no tiene asociado ni 0 ni 1). Por eso, sin información que garantice la totalidad de la función A , puede ser un error deducir $A(s) = 0$ cuando $A(s) \neq 1$.

Es usual diseñar un algoritmo con base en otro existente. Por ejemplo, construir nuevos procedimientos en un lenguaje de programación a partir de otros procedimientos existentes en una librería. Uno de estos casos corresponde al de funciones que invierten la salida de otra función.

Definición 6.3.2

Sea $A : \Sigma^* \rightarrow \{0, 1\}$ una función. La función *complementaria* de A , denotada $\overline{A}$, se define para cualquier $x \in \Sigma^*$ como:

- $\overline{A}(x) = 1$ si $A(x) = 0$,
- $\overline{A}(x) = 0$ si $A(x) = 1$ y
- $\overline{A}(x)$ es indeterminada si $A(x)$ es indeterminada.

Cuando A en la Definición 6.3.2 es total, coinciden el complemento del lenguaje determinado por A y el lenguaje determinado por su función complementaria $\overline{A}$.

Teorema 6.3.1

Si $A : \Sigma^* \rightarrow \{0, 1\}$ es una función total, entonces:

$$L(A) = \overline{L(\overline{A})}.$$

Demostración

Sea $x \in \Sigma^*$. El objetivo es demostrar:

$$x \in L(A) \iff x \in \overline{L(\overline{A})}.$$

Se procede por doble implicación.

($\implies$): Se supone $x \in L(A)$ con el objetivo de demostrar que $x \in \overline{L(\overline{A})}$ o, equivalentemente, que $x \notin L(\overline{A})$. Como $x \in L(A)$, se tiene que $A(x) = 1$. Por definición de función complementaria, se tiene que $\overline{A}(x) = 0$. Es decir, $x \notin L(\overline{A})$.

($\Leftarrow$): Se supone $x \in \overline{L(\bar{A})}$ con el objetivo de demostrar que $x \in L(A)$. Como $x \in \overline{L(\bar{A})}$, entonces $x \notin L(\bar{A})$. O bien $\bar{A}(x) = 0$ o $\bar{A}(x)$ es indeterminado. Por la definición de función complementaria y la suposición de que A es total, no se puede dar que $\bar{A}(x)$ sea indeterminado. Es decir, necesariamente $\bar{A}(x) = 0$. Por definición de $\bar{A}$, se tiene que $A(x) = 1$, como se deseaba.

En conclusión, $L(A) = \overline{L(\bar{A})}$.

El hecho de que una función A determine el lenguaje $L(A)$ no permite concluir que necesariamente A rechaza cualquier cadena $x \notin L(A)$. En la práctica, esto puede suceder cuando un ciclo no termina o cuando hay un error en la ejecución de un programa. Con este preámbulo y lo desarrollado anteriormente, se motiva la introducción de las nociones de aceptación y decisión asociadas a los lenguajes.

Definición 6.3.3: Aceptación y decisión de un lenguaje

Sea L un lenguaje y $A : \Sigma^* \rightarrow \{0, 1\}$ una función. Se dice que:

- A *acepta* L sii para cualquier $x \in L$ se tiene $A(x) = 1$ y para cualquier $x \notin L$ se tiene $A(x) \neq 1$.
- A *decide* L sii para cualquier $x \in L$ se tiene $A(x) = 1$ y para cualquier $x \notin L$ se tiene $A(x) = 0$.

Las dos condiciones de la Definición 6.3.3 coinciden sobre las cadenas de L y difieren únicamente fuera de él. Para aceptar basta con no aceptar lo que está afuera y la tricotomía anterior deja dos maneras de lograrlo: rechazando la cadena o quedando indeterminada sobre ella. Para decidir hay que rechazarla. Esa es toda la distancia entre las dos nociones.

Ejemplo 6.3.2

Sea $L = \{x \mid x \in \Sigma^* \wedge \text{"}x \text{ termina en } 0\text{"}\}$ y considere la función $A : \Sigma^* \rightarrow \{0, 1\}$ definida para cualquier $x \in \Sigma^*$ como:

- $A(x) = 1$ si x termina en 0 y
- $A(x)$ es indeterminada en caso contrario.

Entonces A acepta L , pues $A(x) = 1$ exactamente cuando $x \in L$. Pero A no decide L : la cadena 01 no pertenece a L y, sin embargo, $A(01)$ no vale 0 sino que está indeterminada.

Note que L sí es decidible por otra función. La función total que retorna 1 cuando la cadena termina en 0 y retorna 0 en caso contrario decide L . Es decir, que A no decida L es una limitación de A y no de L . Más adelante el capítulo exhibe lenguajes para los cuales esa limitación resulta insalvable.

La Definición 6.3.3 invita a varias observaciones importantes. Primero, toda función de decisión es una de aceptación, pero no necesariamente al contrario. Segundo, una función A acepta L si y solo si $L = L(A)$. Es decir, la noción de aceptación de un lenguaje y la de lenguaje aceptado de la Definición 6.3.1 son dos maneras de decir lo mismo. Tercero, un lenguaje puede ser aceptado por más de una función pero decidido solo por una única función (¿por qué?). Finalmente, cualquier función total necesariamente es una función de decisión de $L(A)$; el converso también es cierto, es decir, cualquier función de decisión es total. Esta última observación, en el marco de los lenguajes formales, es especialmente importante porque asocia unívocamente la noción de decisión de un lenguaje a la de totalidad de cierto tipo de funciones sobre lenguajes binarios.

Teorema 6.3.2

Si $A : \Sigma^* \rightarrow \{0, 1\}$ es una función, entonces:

$$A \text{ es total} \iff A \text{ decide } L(A).$$

Demostración

Se propone como ejercicio al lector.

Ejercicios

1. Proponga condiciones suficientes para que la siguiente igualdad sea cierta para $A : \Sigma^* \rightarrow \{0, 1\}$:

$$\overline{L(A)} = \{x \mid x \in \Sigma^* \wedge A(x) = 0\}.$$

2. Justifique formalmente la segunda afirmación en el Ejemplo 6.3.1.
3. Sea $A : \Sigma^* \rightarrow \{0, 1\}$. Demuestre o refute:
 - a) $L(A) \cup \overline{L(A)} = \Sigma^*$.
 - b) $L(A) \cup L(\overline{A}) = \Sigma^*$.
4. Presente los detalles de la siguiente afirmación, en el marco de la demostración del Teorema 6.3.1:

Por la definición de función complementaria y la suposición de que A es total, no se puede dar que $\overline{A}(x)$ sea indeterminado.

Sugerencia: proceder por contradicción.

5. Exhiba un lenguaje que sea aceptado por exactamente una función. Demuestre que, en efecto, dicha función es única.
6. Justifique brevemente por qué toda función que decide un lenguaje también lo acepta. El Ejemplo 6.3.2 muestra que el recíproco es falso; explique por qué.
7. Demuestre que A acepta L si y solo si $L = L(A)$.
8. Defina un lenguaje L y dos funciones distintas A_1 y A_2 tales que $L = L(A_1) = L(A_2)$.
9. Demuestre el Teorema 6.3.2.

6.4. Funciones computables

En la práctica, un problema computacional L puede ser resuelto cuando existe un programa o procedimiento que mecánicamente calcule los valores correspondientes a una función A tal que $L = L(A)$. A este tipo de funciones se les denomina *efectivamente computables* o *computables*. La noción de computabilidad no se refiere a eficiencia (en tiempo o espacio), sino más bien a una posibilidad de que exista cierto programa de computador (eficiente o no, en tiempo o espacio) que calcule los valores de una función dada: es una propiedad acerca de las funciones de Σ^* en $\Sigma = \{0, 1\}$. Esta sección presenta una definición de función computable que corresponde a (y se usará como) la noción de algoritmo. Estudia también algunas de sus propiedades, enuncia la tesis de Church y Turing acerca de lo que significa computar en el marco de los lenguajes formales, y cierra exhibiendo un programa capaz de ejecutar a cualquier otro.

Las definiciones que siguen hablan de la existencia de un “programa de computador”, de modo que conviene precisar antes qué se entiende por tal cosa. El texto adopta la siguiente convención.

Nota 6.4.1: Python como modelo de cómputo

En este texto, un *programa* es un programa de Python. Su *punto de entrada* es una función que recibe una cadena de Σ^* y retorna 1 o 0, o bien no termina. En el código de este capítulo ese punto de entrada suele llamarse **decide**, salvo cuando un nombre más descriptivo evita confusiones.

El modelo supone tres idealizaciones frente a un computador físico. Los enteros son de tamaño ilimitado, lo cual ya es cierto en Python. La memoria

disponible es ilimitada y la profundidad de recursión no está acotada, lo cual no es cierto en ninguna máquina real. Estas idealizaciones son las que permiten hablar de computabilidad sin referirse a los recursos de una máquina particular.

La elección de Python no restringe la noción que se está definiendo: por la tesis de Church y Turing (Nota 6.4.5), cualquier otro lenguaje de programación razonable determina exactamente la misma colección de funciones computables.

Definición 6.4.1: Funciones computables

Sea $A : \Sigma^* \rightarrow \Sigma$. Se dice que A es *computable* si y solo si existe un programa de computador que cumple las siguientes dos propiedades, para cualquier $x \in \Sigma^*$:

- Si $A(x)$ está definido, entonces el programa termina cuando se invoca con x y retorna el valor de $A(x)$.
- Si $A(x)$ está indefinido, entonces el programa no termina (nunca) cuando se invoca con x y no retorna ningún valor.

Las funciones computables son el análogo formal de la noción intuitiva de algoritmo: una función es computable si existe un algoritmo que permita calcular sus valores. Son, además, el objeto de estudio básico del área denominada teoría de la computabilidad.

Conviene precisar en qué sentido esta noción depende del modelo de cómputo. Fijar Python en la Nota 6.4.1 es una comodidad y no una restricción. Contar con un lenguaje concreto permite *escribir* los programas cuya existencia afirman las definiciones en lugar de limitarse a describirlos y de ello se saca provecho en el resto del capítulo. Pero la colección de funciones computables no depende de esa elección: en todos los modelos de computación aceptados actualmente las definiciones correspondientes identifican exactamente las mismas funciones y esa independencia es lo que afirma la tesis de Church y Turing. Lo que sí es relevante (y clave) es el hecho de aceptar como acuerdo que cualquier algoritmo se define a partir de una cantidad finita de símbolos que describen una secuencia de pasos que, por ejemplo, una persona con suficiente tiempo y recursos (como papel y lápiz) puede seguir.

Por su parte, la noción de función computable puede ser más general en cuanto a considerar más parámetros y otro tipo de dominios como, por ejemplo, los números. Sin embargo, para el propósito que anima esta sección y el capítulo en general, es suficiente enfocarse exclusivamente en funciones con dominio y rango en Σ^* y Σ , respectivamente, sin que los resultados asociados pierdan generalidad.

Con ayuda de las funciones computables, la noción de computabilidad puede ser extendida a problemas de decisión.

Definición 6.4.2: Lenguajes computablemente enumerables y computables

Sea $L \subseteq \Sigma^*$. Se dice que:

- L es *computablemente enumerable* si y solo si existe una función computable $A : \Sigma^* \rightarrow \Sigma$ tal que $L(A) = L$.
- L es *computable* si y solo si existe una función total computable $A : \Sigma^* \rightarrow \Sigma$ tal que $L(A) = L$.

Los lenguajes computablemente enumerables y computables coinciden con las nociones de aceptación y decisión en la Definición 6.3.3, respectivamente, cuando las funciones parciales y totales que los caracterizan son computables. De esta forma, un problema de decisión L es computablemente enumerable cuando existe un algoritmo A que retorna 1 al ser invocado con cualquier instancia $x \in L$. Además, si dicho algoritmo retorna 0 al ser invocado con cualquier instancia $x \notin L$, entonces L es computable. Note que, por la definición, cualquier lenguaje computable es computablemente enumerable. Sin embargo, el converso no es cierto: el problema de correspondencia de Post, en el Ejemplo 6.4.2, es computablemente enumerable pero no computable.

Ejemplo 6.4.1

Considere los siguientes ejemplos:

- Determinar si un número entero hace parte de un arreglo de números enteros. Considere el lenguaje

$$\text{ARRFIND} = \{\langle A[0..N], x \rangle \mid \text{"}A \text{ es un arreglo y } x \text{ está en } A\text{"}\}.$$

El algoritmo que sigue permite afirmar que ARRFIND es computable. Corresponde a explorar todas las posiciones del arreglo dado para determinar si el número dado está presente o no. Esto se admite porque la igualdad de dos números enteros es computable (¿por qué?).

- Determinar si una fórmula proposicional es una tautología. Considere el lenguaje

$$\text{TAUT} = \{\langle \varphi \rangle \mid \text{"}\varphi \text{ es una proposición tautológica"}\}.$$

El algoritmo que sigue permite afirmar que TAUT es computable. Corresponde a generar la tabla de verdad de φ y si todos sus renglones resultan

en verdadero, entonces la proposición es una tautología; de lo contrario, no lo es.

- Determinar si una fórmula de primer orden es un teorema. Considere el lenguaje

$$\text{FOLTHM} = \{\langle \varphi \rangle \mid \text{"}\varphi \text{ es un teorema en lógica de primer orden"}\}.$$

El algoritmo que sigue permite afirmar que FOLTHM es computablemente enumerable. Generar incrementalmente, en un orden arbitrario, las demostraciones en lógica de primer orden. Si aparece la fórmula dada como conclusión de una demostración, entonces dicha fórmula es un teorema de esta lógica. Note que este algoritmo no se detiene si dicha fórmula no es la conclusión de ninguna demostración.

Gracias a la Nota 6.4.1, la afirmación de que ARRFIND es computable no tiene que quedarse en una descripción: se puede exhibir el programa que la respalda.

Implementación 6.4.1: Un algoritmo de decisión para ArrFind

```

1 def decide(w):
2     ans,i = 0,0
3     if valid(w):                # does w encode a pair <A[0..N), x>?
4         A, x = decode(w)
5         while ans==0 and i!=len(A):
6             ans,i = (1 if A[i]==x else 0),i+1
7     return ans

```

La función `decode` invierte la función de codificación $\langle _ \rangle$ de la Nota 6.2.3 y la función `valid` comprueba que su argumento esté bien formado, es decir, que de verdad codifique un par de la forma $\langle A[0..N), x \rangle$.

Esa comprobación es la que permite afirmar que el programa de la Implementación 6.4.1 termina sobre *cualquier* entrada y no solo sobre las que son códigos. Si w está mal formada, el programa retorna 0 de inmediato, tal y como lo establece la convención de la Nota 6.2.3. Si está bien formada, el ciclo recorre una cantidad finita de posiciones y retorna 1 exactamente cuando x está en A . En los dos casos hay respuesta, de modo que la función que calcula es total y decide ARRFIND.

Nota 6.4.2

Más allá del contexto de los lenguajes formales, los problemas computablemente enumerables o computables reciben los nombres de *semidecidibles* o

decidibles, respectivamente. Por ejemplo, es un problema semidecidible determinar si una fórmula de primer orden es teorema, mientras que es un problema decidible determinar si una fórmula proposicional es tautología.

Ejemplo 6.4.2: Dos testigos

Los ejemplos del Ejemplo 6.4.1 no alcanzan a separar las tres posibilidades que deja abierta la Definición 6.4.2. Los dos que siguen sí lo hacen.

- El problema de correspondencia de Post (ver Ejercicio 5) es computablemente enumerable pero no es computable. Que sea computablemente enumerable se justifica con un algoritmo que genera, en un orden arbitrario, todas las secuencias finitas de índices y verifica si alguna de ellas es una solución; si la hay, el algoritmo eventualmente la encuentra y acepta. Note que este algoritmo no se detiene cuando el problema no tiene solución alguna. Que no sea computable es un resultado clásico de E. Post cuya demostración se propone investigar en el Ejercicio 5. En consecuencia, este problema es testigo de que hay lenguajes semidecidibles que no son decidibles.
- Determinar si una fórmula de primer orden no es un teorema. Considere el lenguaje $\overline{\text{FOLTHM}}$, es decir, el complemento del lenguaje FOLTHM . Este lenguaje no es computablemente enumerable. Para justificarlo, note que FOLTHM sí es computablemente enumerable, como se argumentó en el Ejemplo 6.4.1, pero no es computable: esto último es un resultado clásico de A. Church y A. Turing, y constituye la respuesta negativa al problema de decisión (en alemán, *Entscheidungsproblem*) planteado por D. Hilbert. Si $\overline{\text{FOLTHM}}$ fuera computablemente enumerable, entonces FOLTHM sería computable por el teorema de Post (Teorema 6.5.3), contradiciendo el resultado de Church y Turing. Luego, $\overline{\text{FOLTHM}}$ no es computablemente enumerable. Es decir, este problema es testigo de que hay lenguajes que ni siquiera son semidecidibles.

Nota 6.4.3

Un problema de decisión es *indecidible* si y solo si no es decidible, es decir, si no existe algoritmo alguno que lo decida en el sentido de la Definición 6.4.2. Los ejemplos 6.4.1 y 6.4.2 muestran que, con esta convención, los problemas de decisión se reparten en tres categorías disyuntas:

- los decidibles, como ARRFIND y TAUT ;

- los indecidibles que, sin embargo, son semidecidibles, como el problema de correspondencia de Post y FOLTHM ;
- los indecidibles que ni siquiera son semidecidibles, como $\overline{\text{FOLTHM}}$.

Dicho de otra forma, la colección de los lenguajes computables está contenida estrictamente en la de los computablemente enumerables, la cual a su vez está contenida estrictamente en la de todos los lenguajes. En particular, todo problema que no es semidecidible es indecidible, pero no al contrario. La Sección 6.7 presenta un argumento (no constructivo) de la existencia de problemas indecidibles, y la Sección 6.5 estudia las operaciones bajo las cuales se preservan la computabilidad y la enumerabilidad computable.

La Figura 4 presenta las relaciones de contención entre las categorías de la Nota 6.4.3.

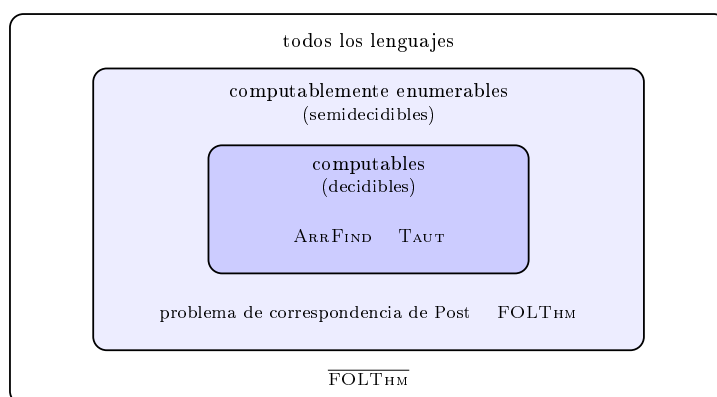


Figura 4. Las tres categorías de la Nota 6.4.3, con un testigo de cada una. Las contenciones son estrictas.

Nota 6.4.4: El programa de Hilbert y el Entscheidungsproblem

Durante la década de 1920, D. Hilbert impulsó un programa cuyo propósito era asentar las matemáticas sobre una base enteramente formal: una axiomatización que fuera completa, en el sentido de que toda afirmación verdadera pudiera demostrarse; consistente, en el sentido de que ninguna afirmación y su negación fueran ambas demostrables, y cuya consistencia se pudiera establecer por medios finitarios.

En 1928, junto con W. Ackermann, Hilbert formuló el *Entscheidungsproblem*, es decir, el *problema de decisión* por antonomasia: exhibir un procedimiento mecánico que determine si una fórmula de la lógica de primer orden es válida. En el vocabulario de este capítulo, se trata de preguntar si el lenguaje FOLTHM del Ejemplo 6.4.1 es computable.

En 1931, K. Gödel demostró sus teoremas de incompletitud, que descartaron el objetivo de completitud del programa. El *Entscheidungsproblem*, sin embargo, quedó en pie y para responderlo en un sentido o en otro había que superar antes el obstáculo señalado en la Nota 6.1.1: definir con precisión qué es un procedimiento mecánico. Esa es exactamente la tarea que resuelve la Definición 6.4.1.

Finalmente, se enuncia la tesis de Alonzo Church y Alan Turing acerca de lo que significa computar.

Nota 6.4.5: La tesis de Church y Turing

La *tesis de Church y Turing* afirma que las funciones computables son exactamente aquellas funciones que pueden ser calculadas mecánicamente con cantidades ilimitadas de tiempo y espacio.

Esta tesis es uno de varios intentos por formalizar la noción de computabilidad. Afirma que las funciones computables son aquellas para las cuales existe un algoritmo (y viceversa). Su carácter no es completamente formal o matemático dado que la noción de computabilidad asociada a las funciones es informal. Por esto, no es un postulado que se pueda demostrar.

Fijar un modelo de cómputo concreto, como se hace en la Nota 6.4.1, tiene una consecuencia que va más allá de la comodidad de poder escribir programas. Un programa de Python es, al mismo tiempo, un objeto que se ejecuta y una cadena de caracteres que se puede leer y manipular. Esa doble naturaleza permite escribir un programa que recibe a otro programa como entrada y lo ejecuta.

Teorema 6.4.1: Universalidad

Existe una función computable $U : \Sigma^* \rightarrow \Sigma$ tal que, para cualquier programa P y cualquier $x \in \Sigma^*$:

- si P termina al ser invocado con x y retorna v , entonces $U(\langle P, x \rangle) = v$; y
- si P no termina al ser invocado con x , entonces $U(\langle P, x \rangle)$ es indeterminada.

A una función con esta propiedad se le denomina *intérprete universal*.

Demostración

La Implementación 6.4.2, a continuación, exhibe un intérprete universal escrito en Python.

La invocación de `exec` ejecuta el texto del programa P en un espacio de nombres nuevo, con lo cual queda definida allí la función `decide` que P describe. La última línea la invoca sobre x . Si esa invocación termina y retorna v , entonces `universal` termina y retorna v ; y si no termina, entonces `universal` tampoco termina, pues está esperando su resultado. Es decir, el comportamiento de U sobre $\langle P, x \rangle$ coincide con el de P sobre x , tanto cuando este último está definido como cuando no lo está, que es exactamente lo que exige el enunciado.

Sobre las cadenas que no codifican un par de la forma $\langle P, x \rangle$, con P un programa, la función `decode` retorna un par fijo cuyo primer componente es un programa que retorna 0. El valor de U sobre esas cadenas es irrelevante para lo que sigue.

Implementación 6.4.2: Un intérprete universal

```
1 def universal(w):
2     code, x = decode(w)          # w encodes the pair <P, x>
3     env = {}
4     exec(code, env)              # defines decide inside env
5     return env["decide"](x)
```

El Teorema 6.4.1 es, quizás, el resultado más importante de este capítulo y conviene no dejarlo pasar como un tecnicismo. Afirma que *una sola máquina puede ejecutar todos los algoritmos*. Es decir, no hace falta construir un aparato distinto para cada tarea: basta construir uno solo y luego darle como dato la descripción de la tarea. Ese es, en esencia, el contenido matemático de la idea de computador de programa almacenado y la razón por la cual el aparato sobre el cual el lector ejecuta el código de este texto sirve igualmente para procesar texto, para entrenar modelos y para jugar ajedrez.

El Teorema 6.4.1 volverá a aparecer, de manera decisiva, cuando se busque exhibir un problema indecidible concreto. La posibilidad de que un programa reciba

programas como entrada es lo que permite formular preguntas de un programa acerca de sí mismo.

Nota 6.4.6: 1936

El *Entscheidungsproblem* de la Nota 6.4.4 fue resuelto en 1936, de manera independiente y con muy pocos meses de diferencia, por A. Church y por A. Turing. La respuesta fue negativa en ambos casos: no existe el procedimiento que Hilbert y Ackermann pedían.

Los caminos fueron distintos. Church formalizó la noción de procedimiento mecánico mediante el cálculo λ ; Turing lo hizo mediante las máquinas que hoy llevan su nombre y demostró además que las dos formalizaciones identifican la misma colección de funciones. Esa coincidencia, repetida después con cada nuevo modelo de cómputo propuesto, es la evidencia que sostiene la Nota 6.4.5.

El artículo de Turing de 1936 lleva por título *On Computable Numbers, with an Application to the Entscheidungsproblem*, y en él aparece también la máquina universal, es decir, el antepasado directo del Teorema 6.4.1. Conviene notar el orden de los hechos: la máquina universal fue concebida una década antes de que se construyera el primer computador de programa almacenado.

Ejercicios

1. Defina brevemente en qué consiste cada uno de los siguientes modelos de computación y enuncie la definición formal de computación correspondiente:
 - a) Máquinas de Turing.
 - b) Cálculo λ (de Alonzo Church).
 - c) Funciones μ -recursivas (de Kurt Gödel).
 - d) Máquinas abstractas (de Emil Post).
 - e) Algoritmos de Markov.
2. Demuestre que si $A : \Sigma^* \rightarrow \Sigma$ tiene dominio finito, entonces A es computable. Concluya que cualquier lenguaje finito es computable. El caso de los lenguajes cuyo complemento es finito se retoma en la Sección 6.5, una vez se disponga de la herramienta que hace falta.
3. Demuestre, exhibiendo un algoritmo como testigo, que:
 - a) Una función $A : \Sigma^* \rightarrow \Sigma$ constante es computable.
 - b) El conjunto vacío es computable.
 - c) El conjunto de números naturales es computable.
 - d) El conjunto de números primos es computable.
4. En el Ejemplo 6.4.1 se afirma que el problema `ARRFIND` es computable. Explique por qué este problema cuando se extiende a números reales deja de ser computable. Ayuda: fijarse en la computabilidad del problema de igualdad de dos números reales.
5. Considere el problema de correspondencia de Post mencionado en el Ejemplo 6.4.1.
 - a) Defina claramente en qué consiste este problema e ilustre la definición con un ejemplo.
 - b) Investigue acerca de una demostración de indecidibilidad de este problema y explique sus principales pasos.
6. Considere el décimo problema de Hilbert de la Nota 6.1.1. Explique qué es una ecuación diofántica, dé un ejemplo con solución entera y otro sin ella, e investigue a grandes rasgos en qué consistió el trabajo de M. Davis, H. Putnam, J. Robinson y Y. Matiyásevich que condujo a la respuesta de 1970.
7. Considere las tres idealizaciones de la Nota 6.4.1. Para cada una, describa un programa de Python que se comporte de manera distinta bajo el modelo idealizado y sobre una máquina real. Explique por qué, a pesar de ello, la colección de funciones computables no cambiaría si se acotara la profundidad de recursión.

8. Escriba, en Python, un algoritmo de decisión para el lenguaje de las cadenas binarias que tienen la misma cantidad de ceros que de unos. Justifique que termina sobre toda entrada.
9. Modifique la Implementación 6.4.2 para obtener un programa $U' : \Sigma^* \rightarrow \Sigma$ que, dado $\langle P, x \rangle$, retorne 1 si P termina al ser invocado con x y retorne 0 en caso contrario. ¿Qué dificultad encuentra? Guarde su respuesta: esta pregunta se retoma más adelante en el texto.
10. El Teorema 6.4.1 afirma la existencia de un intérprete universal para Python escrito en Python. Explique por qué no hay circularidad viciosa en ello. Ayuda: distinga entre el programa como texto y el programa en ejecución.
11. Investigue acerca de la máquina universal de Turing y explique su relación con el Teorema 6.4.1 y con la arquitectura de computador de programa almacenado.

6.5. Clausura y el teorema de Post

La Nota 6.4.3 reparte los problemas de decisión en tres categorías y le pone un ejemplo a cada una, pero no explica por qué son tres y no dos. Esta sección responde esa pregunta. La respuesta es estructural y cabe en una frase: los lenguajes computables son cerrados bajo complemento y los computablemente enumerables no lo son. Esa única asimetría es la que abre el espacio para la categoría intermedia.

El camino pasa por estudiar bajo cuáles operaciones se preservan la computabilidad y la enumerabilidad computable. Las operaciones son las que se introdujeron en la Definición 6.2.2: complemento, unión, intersección y concatenación. La sección cierra con el teorema de Post, que caracteriza exactamente cuándo un lenguaje computablemente enumerable llega a ser computable.

6.5.1. Simulación acotada. Varias de las demostraciones que siguen requieren ejecutar un programa sin correr el riesgo de quedarse esperando indefinidamente su resultado. La herramienta que lo permite es una variante del intérprete universal del Teorema 6.4.1 que ejecuta a lo sumo una cantidad prefijada de pasos.

Nota 6.5.1: Simulación acotada

Existe un programa `simulate` que, dada la codificación $\langle P, x, k \rangle$ de un programa P , una cadena $x \in \Sigma^*$ y un número $k \in \mathbb{N}$, ejecuta a lo sumo k pasos de P sobre x y:

- retorna el valor que P retorna, si P termina dentro de esos k pasos; y
- retorna **None**, si P no ha terminado al cabo de k pasos.

El programa `simulate` termina siempre, cualesquiera que sean P , x y k .

Conviene notar que `simulate` no calcula una función de Σ^* en Σ , pues su rango incluye el valor **None** y por lo tanto no es una función computable en el sentido de la Definición 6.4.1. Es un programa auxiliar en el sentido de la Nota 6.4.1, destinado a usarse como subrutina. Las funciones que se construyen con él en el resto de la sección sí tienen rango en Σ y a ellas sí les aplica la definición.

Para efectos de la Nota 6.5.1, un *paso* es la ejecución de una instrucción elemental del programa. La definición precisa de cuál es esa unidad no tiene ninguna importancia en lo que sigue; solo se necesitan tres propiedades. Primero, ejecutar un paso es un procedimiento mecánico. Segundo, una ejecución que termina lo hace después de una cantidad finita de pasos. Tercero, como consecuencia de las dos anteriores, ejecutar a lo sumo k pasos y detenerse es un procedimiento que siempre termina. Es esta última propiedad la que hace de `simulate` una herramienta segura: quien lo invoca recupera siempre el control.

Hay una cuarta propiedad que este capítulo no necesita y que conviene dejar registrada, porque el Capítulo 8 sí la usa: la simulación tiene *sobrecosto polinomial*. Es decir, ejecutar `simulate` sobre $\langle P, x, k \rangle$ cuesta a lo sumo un polinomio en k y en el tamaño de esa codificación. Para que así sea hace falta que un paso cueste una cantidad acotada de trabajo, lo cual es un supuesto sobre la unidad escogida y no un teorema; es el supuesto usual y es lo que uno espera de un intérprete, que reproduce cada paso pagando por él un costo fijo de administración.

La utilidad de `simulate` es que permite avanzar en la ejecución de dos o más programas *sin comprometerse* con ninguno. En lugar de invocar el primero y esperar, se ejecuta un tramo del primero, un tramo del segundo, un tramo más largo del primero, y así sucesivamente. A esta técnica se le denomina *ejecución entrelazada* (en inglés, *dovetailing*), se ilustra en la Figura 5 y aparece dos veces en lo que resta de la sección.

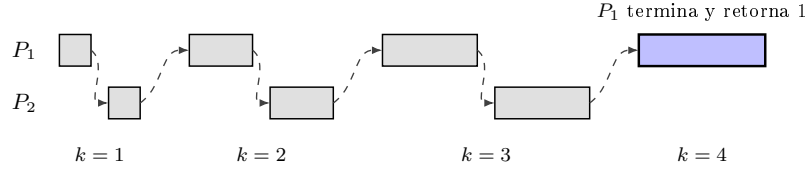


Figura 5. Ejecución entrelazada de dos programas sobre una misma entrada. Los bloques aparecen en el orden en que se ejecutan y su ancho es proporcional a la cantidad de pasos simulados. Cada ronda reinicia las dos simulaciones desde el primer paso y las lleva k pasos más lejos, de manera que ninguno de los dos programas puede acaparar la ejecución.

6.5.2. Clausura. Los lenguajes computables son cerrados bajo todas las operaciones introducidas en la Definición 6.2.2.

Teorema 6.5.1: Clausura de los lenguajes computables

Sean $L_1, L_2 \subseteq \Sigma^*$ lenguajes computables. Entonces los siguientes lenguajes también son computables:

1. $\overline{L_1}$;
2. $L_1 \cup L_2$;
3. $L_1 \cap L_2$; y
4. $L_1 L_2$.

Demostración

Sean A_1 y A_2 funciones totales computables tales que $L(A_1) = L_1$ y $L(A_2) = L_2$; existen por la Definición 6.4.2. En cada caso se exhibe una función total computable cuyo lenguaje asociado es el buscado. Note que en los cuatro casos las invocaciones a A_1 y A_2 terminan siempre, precisamente porque estas funciones son totales.

- (1): La función complementaria $\overline{A_1}$ de la Definición 6.3.2 es total, pues A_1 lo es, y es computable: basta invocar A_1 e invertir su resultado. Por el Teorema 6.3.1 se tiene $L(\overline{A_1}) = \overline{L_1}$.
- (2): Considere la función que, dada x , retorna 1 si $A_1(x) = 1$ o $A_2(x) = 1$, y retorna 0 en caso contrario. Es total y computable, y acepta exactamente las cadenas que están en L_1 o en L_2 .
- (3): Análogo al caso anterior, cambiando la disyunción por una conjunción.

(4): Recuerde que $x \in L_1 L_2$ si y solo si x se puede partir como $x = uv$ con $u \in L_1$ y $v \in L_2$. Si x tiene longitud n , hay exactamente $n + 1$ maneras de partirla de esa forma. Considere entonces la función que, dada x , recorre esas $n + 1$ particiones: retorna 1 si para alguna de ellas se tiene $A_1(u) = 1$ y $A_2(v) = 1$, y retorna 0 si ninguna partición cumple la condición. El recorrido es finito y cada verificación termina, luego la función es total y computable.

Con los lenguajes computablemente enumerables la situación es distinta. De las cuatro operaciones, tres se preservan y una no: la que falla es el complemento. Además, la demostración de una de las tres que sí se preservan ya no puede invocar los programas y esperar.

Teorema 6.5.2: Clausura de los lenguajes computablemente enumerables

Sean $L_1, L_2 \subseteq \Sigma^*$ lenguajes computablemente enumerables. Entonces $L_1 \cup L_2$, $L_1 \cap L_2$ y $L_1 L_2$ también son computablemente enumerables.

Demostración

Sean A_1 y A_2 funciones computables, no necesariamente totales, tales que $L(A_1) = L_1$ y $L(A_2) = L_2$. Sean P_1 y P_2 programas que las calculan.

Intersección: Considere el programa que, dada x , invoca P_1 sobre x y procede así con el resultado. Si esa invocación termina retornando 0, retorna 0. Si termina retornando 1, invoca P_2 sobre x y retorna lo que este retorne. Sea A la función que calcula.

Si $x \in L_1 \cap L_2$, ambas invocaciones terminan retornando 1 y por tanto $A(x) = 1$. Si $x \notin L_1 \cap L_2$, entonces $A_1(x) \neq 1$ o $A_2(x) \neq 1$, y en cualquiera de los dos casos $A(x) \neq 1$, bien porque el programa retorna 0, bien porque no termina. Luego $L(A) = L_1 \cap L_2$.

Unión: Aquí no sirve invocar P_1 y esperar: si $x \in L_2 \setminus L_1$, esa invocación podría no terminar nunca y la cadena x quedaría sin aceptarse, aun cuando pertenece a la unión. Se recurre entonces a la ejecución entrelazada de la Nota 6.5.1.

Considere el programa que, dada x , repite para $k = 1, 2, 3, \dots$ lo siguiente: simula k pasos de P_1 sobre x y retorna 1 si esa simulación terminó retornando 1, y luego simula k pasos de P_2 sobre x y retorna 1 si esa simulación terminó retornando 1. Sea A la función que calcula.

Si $x \in L_1 \cup L_2$, entonces $A_1(x) = 1$ o $A_2(x) = 1$. En el primer caso, la ejecución de P_1 sobre x termina retornando 1 después de una cantidad finita de pasos, digamos m ; entonces en la iteración $k = m$ el programa retorna 1. El segundo caso es análogo. Si $x \notin L_1 \cup L_2$, entonces ninguna de las dos simulaciones termina jamás retornando 1 y el ciclo no termina. Luego $A(x)$ está indeterminada y, en particular, $A(x) \neq 1$.

En conclusión, $L(A) = L_1 \cup L_2$.

Concatenación: Se propone como ejercicio al lector (ver Ejercicio 1).

Nota 6.5.2: Sobre la ausencia del complemento

El Teorema 6.5.2 no menciona el complemento y la omisión no es un descuido ni una limitación de la demostración. La Definición 6.3.2 construye la función complementaria $\bar{A}$ invirtiendo la salida de A , pero esa construcción no ayuda cuando A es parcial: si $A(x)$ está indeterminada, entonces $\bar{A}(x)$ también lo está y la cadena x no queda aceptada por ninguna de las dos. Por eso el Teorema 6.3.1 exige que A sea total.

El Ejemplo 6.4.2 exhibe el testigo concreto: FOLTHM es computablemente enumerable y $\overline{\text{FOLTHM}}$ no lo es. El Corolario 6.5.4, más adelante en esta misma sección, muestra que ese testigo no es una rareza: el complemento de *cualquier* lenguaje computablemente enumerable que no sea computable deja de ser computablemente enumerable.

6.5.3. El teorema de Post. El resultado que sigue caracteriza exactamente la distancia que separa a un lenguaje computablemente enumerable de uno computable. Se debe a E. Post, el mismo autor del problema de correspondencia del Ejemplo 6.4.2.

Teorema 6.5.3: Teorema de Post

Sea $L \subseteq \Sigma^*$. Entonces L es computable si y solo si L y $\bar{L}$ son computablemente enumerables.

Demostración

Se procede por doble implicación.

($\Rightarrow$): Suponga que L es computable. Por la Definición 6.4.2, L es computablemente enumerable. Y por la parte (1) del Teorema 6.5.1, $\bar{L}$ es computable y, por lo tanto, también computablemente enumerable.

($\Leftarrow$): Suponga que L y $\bar{L}$ son computablemente enumerables. Sean P_1 y P_2 programas que calculan funciones computables A_1 y A_2 tales que $L(A_1) = L$ y $L(A_2) = \bar{L}$.

Considere el programa que, dada x , repite para $k = 1, 2, 3, \dots$ lo siguiente: simula k pasos de P_1 sobre x y retorna 1 si esa simulación terminó retornando 1; y luego simula k pasos de P_2 sobre x y retorna 0 si esa simulación terminó retornando 1. Sea A la función que calcula.

La afirmación clave es que este programa *siempre* termina. En efecto, sea $x \in \Sigma^*$. O bien $x \in L$ o bien $x \in \bar{L}$, pues estos dos conjuntos particionan a Σ^* . Si $x \in L$, entonces $A_1(x) = 1$ y la ejecución de P_1 sobre x termina retornando 1 después de una cantidad finita de pasos, digamos m ; luego, el ciclo termina a más tardar en la iteración $k = m$. Si $x \in \bar{L}$, el mismo argumento aplica a P_2 . En ambos casos el ciclo termina, de donde A es total.

Resta verificar que A decide L . Si $x \in L$, la única simulación que puede terminar retornando 1 es la de P_1 , pues $A_2(x) \neq 1$ al ser $x \notin \bar{L}$; luego, $A(x) = 1$. Si $x \notin L$, la única que puede terminar retornando 1 es la de P_2 , de donde $A(x) = 0$.

En conclusión, A es total y computable con $L(A) = L$, es decir, L es computable.

La forma contrarrecíproca del teorema de Post es la que se usa en la práctica para demostrar que un lenguaje no es computablemente enumerable.

Corolario 6.5.4

Sea $L \subseteq \Sigma^*$ computablemente enumerable. Si L no es computable, entonces $\bar{L}$ no es computablemente enumerable.

Demostración

Hacia una contradicción, suponga que $\bar{L}$ es computablemente enumerable. Como L también lo es por hipótesis, el Teorema 6.5.3 implica que L es computable, lo cual contradice la hipótesis.

Con el Corolario 6.5.4 se cierra la pregunta con que abrió esta sección. Los lenguajes computables son cerrados bajo complemento por el Teorema 6.5.1. Los

computablemente enumerables no lo son y el Corolario 6.5.4 explica exactamente cuándo: falla para todo lenguaje que sea computablemente enumerable sin ser computable. Dicho de otra forma, cada problema de la categoría intermedia de la Nota 6.4.3 *produce* un problema de la tercera categoría, a saber, su complemento. Las tres categorías no son una clasificación arbitraria: la segunda no puede existir sin que exista la tercera.

Nota 6.5.3

La asimetría entre aceptar y decidir tiene una lectura práctica directa. Un algoritmo de aceptación para L responde afirmativamente cuando la respuesta es afirmativa, pero puede quedarse trabajando indefinidamente cuando no lo es. Quien lo ejecuta y ve que aún no ha respondido no puede distinguir entre “la respuesta es negativa” y “la respuesta es afirmativa y todavía no la ha encontrado”.

El teorema de Post dice que esa incertidumbre desaparece exactamente cuando se dispone también de un procedimiento que reconozca los casos negativos. Al ejecutar los dos de manera entrelazada, alguno de ellos responde y la espera deja de ser indefinida.

Ejercicios

1. Demuestre la parte del Teorema 6.5.2 que quedó pendiente: si L_1 y L_2 son computablemente enumerables, entonces L_1L_2 también lo es. Ayuda: combine el recorrido de particiones de la parte (4) del Teorema 6.5.1 con la ejecución entrelazada de la Nota 6.5.1.
 2. Sea $L \subseteq \Sigma^*$ computable. Demuestre que L^n es computable para cualquier $n \in \mathbb{N}$, donde L^n es la concatenación generalizada de la Definición 6.2.2.
 3. Demuestre que si L es computable, entonces la clausura de Kleene L^* es computable. Ayuda: proceda por inducción sobre la longitud de la cadena de entrada.
 4. En la demostración de la parte (2) del Teorema 6.5.1 se afirma que la función construida es total. Indique exactamente dónde se usa que A_1 y A_2 son totales, y explique qué sale mal si solo se supone que son computables.
 5. Considere la siguiente “demostración” de que los lenguajes computablemente enumerables son cerrados bajo complemento: dado un programa P que acepta L , constrúyase el programa que invoca P y retorna 1 si P retorna 0 y retorna 0 si P retorna 1. Explique con precisión en qué punto falla el argumento.
 6. Demuestre que si L es computablemente enumerable y $\bar{L}$ es computable, entonces L es computable. ¿Se puede debilitar la hipótesis sobre $\bar{L}$?
 7. Sea L un lenguaje finito. Demuestre que L es computable. Sea ahora L un lenguaje cuyo complemento es finito y demuestre que también es computable; esta es la parte que quedó pendiente en el Ejercicio 2 de la Sección 6.4 y la herramienta que hacía falta es la parte (1) del Teorema 6.5.1. Concluya que ningún lenguaje finito ni cofinito puede servir de testigo para la Nota 6.5.2.
 8. En la demostración del Teorema 6.5.3 se recorre $k = 1, 2, 3, \dots$ simulando k pasos de cada programa desde el principio, lo cual repite trabajo ya hecho. Proponga una variante que no lo repita y explique por qué la versión presentada, aunque más costosa, es suficiente para el resultado.
 9. Suponga que se cuenta con un algoritmo de aceptación para L y se sabe, además, que L es computable, pero no se conoce ningún algoritmo de decisión para L . ¿Permite el Teorema 6.5.3 construir uno? Justifique.
 10. Investigue acerca de la jerarquía aritmética y ubique en ella las tres categorías de la Nota 6.4.3. ¿Qué hay más allá de la tercera?
-

6.6. Enumeración

La Definición 6.4.2 bautiza como *computablemente enumerables* a los lenguajes que admiten un algoritmo de aceptación, pero hasta ahora nada ha justificado el adjetivo. La definición habla de aceptar cadenas, no de enumerarlas, y el lector tiene derecho a sospechar que la palabra es jerga heredada.

No lo es. Esta sección demuestra que un lenguaje admite un algoritmo de aceptación si y solo si sus elementos se pueden *listar* mediante un procedimiento mecánico. Es más, esa forma de verlos ya apareció dos veces, en los ejemplos 6.4.1 y 6.4.2: los algoritmos que justifican la enumerabilidad computable de FOLTHM y del problema de correspondencia de Post consisten, literalmente, en generar objetos uno tras otro hasta dar con el que interesa. Lo que sigue muestra que eso no es una coincidencia de esos dos ejemplos sino la forma general del asunto.

La sección cierra con una caracterización análoga para los lenguajes computables, que resulta ser sorprendentemente cercana: la diferencia entre decidir y aceptar es, exactamente, la diferencia entre listar *en orden* y listar en cualquier orden.

6.6.1. Listar todas las cadenas. El punto de partida es que Σ^* mismo se puede recorrer mecánicamente, siguiendo el orden canónico de la Definición 6.2.3.

Implementación 6.6.1: La enumeración canónica de Σ^*

```

1 def nth_string(n, alphabet):
2     m = len(alphabet)
3     w = ""
4     while n > 0:
5         n, r = divmod(n - 1, m)    # bijective base-m digits
6         w = alphabet[r] + w
7     return w

```

El programa de la Implementación 6.6.1 calcula w_n para cualquier alfabeto, que recibe como segundo argumento con sus símbolos listados en el orden que fija Σ ; en este capítulo se invoca con "01". Termina sobre cualquier entrada, pues cada iteración reemplaza n por $\lfloor (n-1)/m \rfloor$, que es estrictamente menor que n siempre que $n > 0$, de modo que calcula una función total. Es decir, el orden canónico no es solo un orden: es un orden que un programa puede recorrer.

Nota 6.6.1

La correspondencia que calcula la Implementación 6.6.1 es la *numeración biyectiva* en base m , en la cual los dígitos recorren $1, \dots, m$ en lugar de $0, \dots, m-1$. Al no haber dígito cero, ninguna cadena arrastra ceros a la izquierda que la hagan ambigua y, por lo tanto, cada número natural tiene exactamente una representación. El ciclo extrae esos dígitos de derecha a izquierda y la resta de 1 antes de la división es, precisamente, lo que desplaza el rango de los residuos.

Para $m = 2$ ocurre una coincidencia que esconde la diferencia: la representación biyectiva de n resulta ser la binaria ordinaria de $n + 1$ sin su primer dígito, que siempre es 1. Describir así el procedimiento es correcto para $\Sigma = \{0, 1\}$ y es, además, la forma en que suele presentarse. El riesgo está en concluir de ahí que para un alfabeto de m símbolos basta reemplazar la base 2 por la base m . Eso falla de inmediato: sobre $\Sigma = \{a, b, c\}$, escribir $n + 1$ en base 3 y descartar el primer dígito entrega la cadena vacía tanto para $n = 0$ como para $n = 1$, cuando el orden canónico pide a en el segundo caso. La correspondencia resultante ni siquiera es inyectiva. Lo que cambia al pasar de 2 a m no es solo la base sino también la transformación del índice.

Note que el programa tampoco excluye el caso $m = 1$: entrega $w_n = a^n$, que es el orden canónico del alfabeto unario.

La demostración del Teorema 6.7.4 usa una variante de la misma idea, aunque no la misma correspondencia: allí se lee una cadena que empieza en 1 como numeral binario y se le resta 1, de modo que el dominio no es todo Σ^* , sino el conjunto de las cadenas que empiezan en 1.

6.6.2. Enumeradores. Un enumerador es un programa cuyo oficio no es responder preguntas sino producir cadenas. Se trata de un programa en el sentido de la Nota 6.4.1, con la salvedad de que su punto de entrada no recibe argumento alguno y de que su resultado no es un valor retornado sino lo que va imprimiendo a lo largo de su ejecución.

Definición 6.6.1: Enumerador

Un *enumerador* es un programa E que se ejecuta sin recibir entrada alguna y que, durante su ejecución, imprime cadenas de Σ^* . El *lenguaje enumerado* por E , denotado $L(E)$, es el conjunto:

$$L(E) = \{w \mid w \in \Sigma^* \wedge \text{"E imprime } w \text{ en algún momento"}\}.$$

La Definición 6.6.1 es deliberadamente permisiva en tres aspectos y conviene notarlos porque las demostraciones que siguen se aprovechan de los tres. Primero, E puede imprimir una misma cadena varias veces; las repeticiones no cambian el

conjunto $L(E)$. Segundo, E puede imprimir las cadenas en cualquier orden. Tercero, E no tiene por qué terminar: si $L(E)$ es infinito, necesariamente no termina, pues debe alcanzar a imprimir infinitas cadenas. Note también que la notación $L(E)$ extiende la que se viene usando para funciones: tanto una función de aceptación como un enumerador determinan un lenguaje, aunque lo hagan de maneras distintas.

Ejemplo 6.6.1

Considere las siguientes situaciones.

- El programa que imprime $w_0, w_1, w_2, \dots$ usando la Implementación 6.6.1 es un enumerador de Σ^* .
- El programa que no imprime nada, por ejemplo el que entra de inmediato en un ciclo infinito vacío, es un enumerador de $\emptyset$.
- El programa que imprime 00, luego 11, luego 00 otra vez, y luego se detiene, es un enumerador de $\{00, 11\}$. Note que este lenguaje tiene dos elementos, cada uno una cadena de longitud dos, y no debe confundirse con el alfabeto Σ .

El resultado central de la sección afirma que enumerar y aceptar son capacidades equivalentes.

Teorema 6.6.1: Caracterización por enumeración

Sea $L \subseteq \Sigma^*$. Entonces L es computablemente enumerable si y solo si existe un enumerador E tal que $L = L(E)$.

Demostración

Se procede por doble implicación.

($\Rightarrow$): Suponga que L es computablemente enumerable y sea P un programa que calcula una función computable A con $L(A) = L$.

Considere el enumerador E que repite, para $m = 0, 1, 2, \dots$, lo siguiente: para cada n entre 0 y m , simula m pasos de P sobre w_n , y si esa simulación terminó retornando 1, imprime w_n . La simulación es la de la Nota 6.5.1 y la cadena w_n se obtiene con la Implementación 6.6.1; ambas terminan siempre, de modo que cada iteración del ciclo externo está bien definida y E avanza indefinidamente.

Se verifica que $L(E) = L$. Si E imprime w_n , es porque la ejecución de P sobre w_n terminó retornando 1, es decir, porque $w_n \in L$. Recíprocamente, sea $w \in L$. Como $n \mapsto w_n$ es una biyección, existe n tal que $w = w_n$; y como $A(w) = 1$, la ejecución de P sobre w termina retornando 1 después de una cantidad finita de pasos, digamos m_0 . Entonces, en la iteración $m = \max(n, m_0)$ del ciclo externo, el enumerador imprime w .

($\Leftarrow$): Suponga que existe un enumerador E con $L(E) = L$.

Considere el programa que, dada x , ejecuta E y compara cada cadena impresa con x , retornando 1 tan pronto como alguna coincida. Sea A la función que calcula.

Si $x \in L$, entonces E imprime x en algún momento de su ejecución, después de una cantidad finita de pasos, y el programa retorna 1; luego $A(x) = 1$. Si $x \notin L$, entonces E nunca imprime x y el programa nunca retorna; luego $A(x)$ está indeterminada y, en particular, $A(x) \neq 1$.

En conclusión $L(A) = L$, de donde L es computablemente enumerable.

Nota 6.6.2: ¿De dónde viene el nombre?

El Teorema 6.6.1 justifica el adjetivo de la Definición 6.4.2: los lenguajes computablemente enumerables son exactamente aquellos cuyos elementos se pueden enumerar mecánicamente.

Vale la pena releer con esta luz los dos algoritmos de los ejemplos 6.4.1 y 6.4.2. El que justifica que FOLTHM es computablemente enumerable genera demostraciones de primer orden una tras otra; el que justifica lo propio para el problema de correspondencia de Post genera secuencias de índices. En ambos casos lo que hay debajo es un enumerador y el algoritmo de aceptación se obtiene de él tal y como lo hace la segunda parte de la demostración del Teorema 6.6.1.

6.6.3. Enumerar en orden. La Definición 6.6.1 no le exige nada al orden en que el enumerador produce las cadenas. Exigirlo cambia todo.

Teorema 6.6.2: Caracterización de los lenguajes computables

Sea $L \subseteq \Sigma^*$. Entonces L es computable si y solo si existe un enumerador E tal que $L = L(E)$ y E imprime las cadenas de L en orden canónico creciente, sin repeticiones.

Demostración

Se procede por doble implicación.

($\Rightarrow$): Suponga que L es computable y sea A una función total computable con $L(A) = L$. Considere el enumerador que recorre $n = 0, 1, 2, \dots$ y, para cada n , calcula w_n e imprime esa cadena si $A(w_n) = 1$.

Cada invocación de A termina, por ser A total, de modo que el enumerador avanza indefinidamente. Imprime exactamente las cadenas de L , y las imprime en orden canónico creciente y sin repeticiones, pues recorre los índices en orden y cada cadena aparece una sola vez en la enumeración canónica.

($\Leftarrow$): Suponga que existe un enumerador E como en el enunciado. Se distinguen dos casos.

Si L es finito, entonces L es computable sin necesidad de recurrir a E : un programa que compare su entrada con cada uno de los finitos elementos de L decide L y termina siempre.

Si L es infinito, considere el programa que, dada x , ejecuta E y examina cada cadena que este imprime. Retorna 1 si la cadena impresa es x , y retorna 0 si la cadena impresa sucede a x en el orden canónico.

Este programa siempre termina. En efecto, como L es infinito y E lo imprime en orden creciente, E imprime cadenas arbitrariamente grandes en el orden canónico; en particular, imprime en algún momento una cadena que es x o que sucede a x . Además, decide L : si $x \in L$, la primera de esas cadenas es x y el programa retorna 1; y si $x \notin L$, ninguna cadena impresa es x , luego la primera que no precede a x lo sucede estrictamente y el programa retorna 0.

Nota 6.6.3

El contraste entre los teoremas 6.6.1 y 6.6.2 da una lectura muy concreta de la distancia entre aceptar y decidir. Los dos tipos de lenguaje se pueden listar; la diferencia está en si el listado sale ordenado.

La razón por la cual el orden lo cambia todo es que permite *darse por vencido con fundamento*. Ante un listado desordenado, no haber visto todavía la cadena x no dice nada, porque podría aparecer más adelante. Ante un listado ordenado, ver pasar una cadena que sucede a x es prueba concluyente de que x no va a aparecer nunca. Esa es, en este contexto, toda la diferencia entre un algoritmo de aceptación y uno de decisión.

Note que la distinción del caso finito en la demostración del Teorema 6.6.2 no es un tecnicismo evitable. Si L es finito, el enumerador puede quedarse trabajando indefinidamente sin imprimir nada más y quien lo observa no puede distinguir esa situación de la de un enumerador que va a imprimir algo más adelante.

Conviene advertir la consecuencia. En el caso finito, la demostración establece que el algoritmo de decisión *existe*, pero no lo construye a partir de E : sabe que L es finito y apela a que todo lenguaje finito es computable, sin poder exhibir la lista de sus elementos. El Teorema 6.6.2 no debe leerse entonces como una receta para convertir cualquier enumerador ordenado en un algoritmo de decisión. Es una equivalencia entre dos condiciones, no un procedimiento que lleve de una a la otra.

Ejercicios

1. Verifique la Implementación 6.6.1 calculando a mano w_n , sobre $\Sigma = \{0, 1\}$, para n entre 0 y 20. Compare los primeros nueve valores con el Ejemplo 6.2.2 y verifique que los restantes respetan el orden canónico de la Definición 6.2.3.
2. Escriba, en Python, un programa que calcule la función inversa de la Implementación 6.6.1 sobre $\Sigma = \{0, 1\}$. Es decir, que dada una cadena $w \in \{0, 1\}^*$ retorne el único n tal que $w = w_n$. Justifique que termina sobre toda entrada.
3. Sobre la generalización a un alfabeto de $m \geq 1$ símbolos:
 - a) Demuestre que la Implementación 6.6.1 es correcta, es decir, que retorna w_n para todo $n \in \mathbb{N}$ y todo alfabeto. Conviene razonar por inducción fuerte sobre n , notando que la última iteración del ciclo es la que produce el primer símbolo del resultado.
 - b) Programe la versión incorrecta que menciona la Nota 6.6.1, esto es, escribir $n + 1$ en base m y descartar el primer dígito, y compárela con la Implementación 6.6.1 sobre $\Sigma = \{a, b, c\}$ y $\Sigma = \{a, b, c, d\}$ para n entre 0 y 29. Verifique que en ambos alfabetos las dos coinciden únicamente en $n = 0$, mientras que para $\Sigma = \{0, 1\}$ coinciden en todos los valores.
4. En cada iteración de la Implementación 6.6.1, la asignación sobre w antepone un símbolo a lo ya acumulado. Esa operación no modifica la cadena existente: crea una nueva y copia en ella todo lo acumulado hasta el momento.
 - a) Sea k la cantidad de símbolos del resultado. Determine cuántos caracteres se copian en total a lo largo de la ejecución y exprese esa cantidad en notación asintótica.

- b) Reescriba el programa de manera que, en lugar de construir una cadena nueva por iteración, acumule los segmentos en una lista con `append` y construya el resultado una sola vez al final con `join`. Antes de escribirlo, responda: ¿en qué orden produce el ciclo los símbolos, y en qué orden deben aparecer en la cadena?
 - c) Determine el costo asintótico de su versión y compárelo con el del literal (a). Verifique además que ambas versiones producen el mismo resultado para n entre 0 y 20.
5. Demuestre que L es computablemente enumerable si y solo si $L = \emptyset$ o existe una función total computable $f : \Sigma^* \rightarrow \Sigma^*$ cuyo rango es exactamente L . Ayuda: para una de las direcciones necesitará una función total computable que recorra todos los pares $(n, m) \in \mathbb{N} \times \mathbb{N}$; investigue acerca de la función de emparejamiento de Cantor.
 6. En la primera parte de la demostración del Teorema 6.6.1, el ciclo externo recorre $m = 0, 1, 2, \dots$ y el interno recorre n entre 0 y m . Explique qué sale mal si se invierte el anidamiento, es decir, si se recorre n por fuera y m por dentro.
 7. Sea L un lenguaje computablemente enumerable e infinito. Demuestre que L tiene un subconjunto infinito computable. Ayuda: use el enumerador del Teorema 6.6.1 y quédese solo con las cadenas que superen a todas las impresas antes.
 8. Demuestre que si L_1 y L_2 son computablemente enumerables, entonces $L_1 \cup L_2$ también lo es, construyendo directamente un enumerador a partir de los enumeradores de L_1 y L_2 . Compare la longitud de este argumento con la de la demostración del Teorema 6.5.2.
 9. Demuestre el teorema de Post (Teorema 6.5.3) usando el Teorema 6.6.2 en lugar de la ejecución entrelazada. Ayuda: dados enumeradores de L y de $\bar{L}$, construya uno de L que imprima en orden canónico creciente.
 10. Un enumerador que imprime en orden canónico creciente no puede imprimir repeticiones. Justifique esta afirmación y explique por qué el enunciado del Teorema 6.6.2 menciona ambas condiciones de todos modos.
 11. Suponga que se dispone de un enumerador E de un lenguaje L y que, además, se sabe de antemano cuántos elementos tiene L . Demuestre que L es computable.

6.7. ¿Cuántos problemas se pueden decidir?

El principal resultado de la sección es negativo en el sentido en el cual se demuestra que existen funciones que no son computables. Muestra que, en el contexto de lo que significa computar hoy en día, hay funciones que no se pueden mecanizar en un computador. El principal argumento en la demostración es la existencia de

una cantidad tan grande de problemas de decisión, en el sentido de la Sección 6.2, que no hay suficientes programas de computador que los puedan resolver mecánicamente. Esto es equivalente a decir que no toda función con dominio en el conjunto de cadenas binarias Σ^* y rango en el conjunto binario Σ puede ser mecanizada en un computador (ver Teorema 6.3.2).

Una advertencia amigable acerca del desarrollo de la sección antes de entrar en materia. Se usarán conceptos y resultados de la teoría de conjuntos como, por ejemplo, cardinales y aritmética cardinal que generalmente no son estudiados en currículos de pregrado de ingeniería y computación. Por ello, la limitante potencial en esta sección es que algunos de los resultados empleados no se demuestran porque las definiciones o técnicas necesarias están fuera del alcance del texto; en consecuencia, esta sección no es autocontenida. Al final del capítulo se presentan referencias para profundizar en estos temas y conocer las demostraciones ausentes, en caso tal de que el lector quiera profundizar en ellos.

6.7.1. Conceptos básicos de cardinales. El plan general para lograr el objetivo de la sección es simple: comparar los tamaños del conjunto de lenguajes y del conjunto de funciones computables para determinar que el primero es mucho más grande que el segundo.

Para razonar sobre el tamaño de conjuntos infinitos es necesario desligar la noción de tamaño de la acción de contar uno a uno, tal y como sucede generalmente cuando los conjuntos son finitos. Por ejemplo, ¿se podrían contar, uno a uno, los números naturales? La respuesta es no: este conjunto no tiene una cota superior y, en consecuencia, cualquier intento de listarlos explícitamente nunca terminaría. En contraste, si se toma cualquier subconjunto finito de los números naturales, por grande que este sea, un proceso de conteo de uno tras otro seguro que terminará (posiblemente en varias generaciones de una familia si el subconjunto es muy grande, pero terminará).

Para referirse al tamaño de conjuntos infinitos, es necesario introducir la noción de cardinal y algunas propiedades asociadas a ella.

Definición 6.7.1: Número cardinal

El *cardinal* de un conjunto X se denota como $\text{card}(X)$. Sean X_1 y X_2 dos conjuntos. Entonces:

- $\text{card}(X_1) = \text{card}(X_2)$ si y solo si existe un función biyectiva entre X_1 y X_2 . En este caso, se dice que X_1 y X_2 son *equipotentes*, escrito $X_1 \sim X_2$.
- $\text{card}(X_1) \leq \text{card}(X_2)$ si y solo si existe una función inyectiva de X_1 a X_2 .

Además, si X_1 y X_2 son disyuntos, se tiene

$$\text{card}(X_1 \cup X_2) = \text{card}(X_1) + \text{card}(X_2).$$

La función $\text{card}(\cdot)$ asocia cualquier conjunto a un *ordinal*; para efectos prácticos, un *cardinal* es un tipo especial de ordinal que permite representar unívocamente el tamaño de un conjunto infinito. Intuitivamente, los ordinales son conjuntos que generalizan la noción de número y que coinciden con los números naturales cuando los objetos de estudio son los conjuntos finitos. De acuerdo con la Definición 6.7.1, para determinar la igualdad de los cardinales (o, equivalentemente, la equipotencia de dos conjuntos) es necesario exhibir una biyección entre los conjuntos en cuestión. Además, se extiende el orden total $\leq$ sobre los números naturales a los cardinales infinitos. Al igual que con conjuntos finitos, el tamaño de una unión disyunta de dos conjuntos es igual a la suma de los tamaños de las dos partes.

Ejemplo 6.7.1

El conjunto de los números naturales y el de los números naturales pares

$$2\mathbb{N} = \{2m \mid m \in \mathbb{N}\}$$

tienen el mismo tamaño (i.e., $\mathbb{N} \sim 2\mathbb{N}$). Esto es cierto porque la función determinada por el mapa

$$n \mapsto 2n$$

es biyectiva. Por una parte, es claramente inyectiva: si $2n_1 = 2n_2$, necesariamente $n_1 = n_2$. Para observar que es sobreyectiva, note que cualquier número en $2\mathbb{N}$ tiene la forma $2n$, cuya preimagen es $n \in \mathbb{N}$. En conclusión, los dos conjuntos tienen el mismo cardinal y son equipotentes.

Note que dado que hay una biyección entre $\mathbb{N}$ y $2\mathbb{N}$, también se tiene que $\text{card}(\mathbb{N}) \leq \text{card}(2\mathbb{N})$ y $\text{card}(2\mathbb{N}) \leq \text{card}(\mathbb{N})$.

El Ejemplo 6.7.1 exhibe una propiedad posiblemente sorprendente de los conjuntos infinitos: aun cuando $2\mathbb{N}$ es un subconjunto propio de $\mathbb{N}$, los dos conjuntos tienen la misma cantidad de elementos. Con conjuntos finitos esta situación es imposible porque cualquier subconjunto propio de un conjunto finito necesariamente tiene menos elementos. Por ejemplo, el axioma “el todo es mayor que cualquiera de sus partes” de Euclides es inválido para conjuntos infinitos.

Nota 6.7.1

El matemático Richard Dedekind usó la siguiente propiedad para caracterizar conjuntos finitos e infinitos:

Si no hay un subconjunto propio de X equipotente a X , entonces X es *finito*. Si hay un subconjunto propio de X equipotente a X , entonces X es *infinito*.

Nota 6.7.2

Usualmente, los cardinales infinitos se identifican con los ordinales:

$$\aleph_0, \aleph_1, \aleph_2, \dots$$

El símbolo $\aleph$ fue introducido por el matemático Georg Cantor para representar el tamaño de conjuntos infinitos y proviene del alfabeto hebreo; este símbolo se lee “alef” en castellano.

Los conjuntos infinitos usualmente se clasifican en dos tipos: aquellos que tienen el mismo tamaño de los números naturales y los demás. A los primeros se les denomina contables, mientras que a los segundos no contables. Históricamente, la distinción entre conjuntos contables y no contables ha permitido entender diferencias profundas entre conjuntos de interés como lo son, por ejemplo, los números racionales y los reales que hacen parte de categorías distintas en relación con el tamaño de los números naturales. El argumento que se desarrolla a lo largo de esta sección es un ejemplo más del uso de esta caracterización, como se terminará de ver en los próximos apartes.

Definición 6.7.2

Sea X un conjunto. Se dice que:

- X es *contable* si y solo si X es finito o $X \sim \mathbb{N}$;
- X es *no contable* de lo contrario.

Se adopta $\text{card}(\mathbb{N}) = \aleph_0$ por convención.

Como consecuencia de la Definición 6.7.2, $\mathbb{N}$ es contable, al igual que cualquiera de sus subconjuntos. Por eso, abusando de la convención, se dice que $\aleph_0$ es el único cardinal infinito contable, además de ser el primer cardinal infinito.

6.7.2. Aritmética cardinal. Antes de operar los cardinales entre sí, conviene precisar cómo se comparan. La Definición 6.7.1 introduce la relación $\leq$ entre cardinales a partir de la existencia de funciones inyectivas, pero de esa definición no

es evidente que dicha relación se comporte como el orden usual sobre los números naturales. El Teorema 6.7.1 afirma que sí lo hace.

Teorema 6.7.1: Orden de los cardinales

Sean X_1 y X_2 conjuntos. Entonces:

1. Si $\text{card}(X_1) \leq \text{card}(X_2)$ y $\text{card}(X_2) \leq \text{card}(X_1)$, entonces $\text{card}(X_1) = \text{card}(X_2)$.
2. $\text{card}(X_1) \leq \text{card}(X_2)$ o $\text{card}(X_2) \leq \text{card}(X_1)$.

En consecuencia, la relación $\leq$ es un orden total sobre los cardinales.

La primera parte del Teorema 6.7.1 se conoce como el teorema de Cantor, Schröder y Bernstein y es en la práctica la herramienta más cómoda para establecer la igualdad de dos cardinales: en lugar de exhibir una biyección, como lo exige la Definición 6.7.1, basta con exhibir dos funciones inyectivas, una en cada dirección. El comentario que cierra el Ejemplo 6.7.1 observa las dos desigualdades que se siguen de una biyección; esta primera parte afirma justamente el recíproco. La segunda parte establece que dos cardinales cualesquiera siempre son comparables, de manera que descartar una de las dos desigualdades equivale a afirmar la otra. Al igual que sucede con el Teorema 6.7.2, las demostraciones de estas dos partes están fuera del alcance de este texto.

Los cardinales, como números que son, se pueden operar entre sí. El Teorema 6.7.2 introduce una propiedad de la suma de cardinales que será útil posteriormente.

Teorema 6.7.2: Suma de cardinales infinitos

Sean X_1 y X_2 conjuntos, con X_1 infinito. Si $\text{card}(X_2) \leq \text{card}(X_1)$, entonces $\text{card}(X_1) + \text{card}(X_2) = \text{card}(X_1)$.

La suma de cardinales infinitos puede ser intuitiva si es interpretada, por ejemplo, con ayuda de la unión de conjuntos. Si un conjunto infinito se duplica, el cardinal del conjunto resultante es el mismo. De manera más general, si un conjunto infinito X se extiende con un conjunto que no sobrepasa su cantidad de elementos, el tamaño del conjunto resultante es el mismo de X , como se afirma en el Teorema 6.7.2. A nivel formal, la demostración del Teorema 6.7.2 está fuera del alcance de este texto (se propone como ejercicio al lector interesado en profundizar en el tema o con conocimiento previo acerca de cardinales).

Hay una forma mecánica de construir un conjunto infinito de mayor tamaño que cualquier conjunto infinito dado. Esta corresponde a la operación de construcción del conjunto potencia denotado en este texto como $\mathcal{P}(\cdot)$; este operador denota el conjunto de todos los subconjuntos de un conjunto dado. El Teorema 6.7.3 presenta resultados asociados al conjunto potencia de un conjunto, finito o infinito.

Teorema 6.7.3: Cardinal del conjunto potencia

Para cualquier conjunto X se tiene:

1. $\text{card}(\mathcal{P}(X)) = 2^{\text{card}(X)}$.
2. $\text{card}(X) < \text{card}(\mathcal{P}(X))$.

La demostración de la parte (1) del Teorema 6.7.3 está fuera del alcance de este texto, al igual que la del Teorema 6.7.2. Intuitivamente, afirma que el tamaño del conjunto partes de un conjunto infinito cumple la misma ley de exponenciación que en el caso finito.

La parte (2), en cambio, admite una demostración corta que vale la pena conocer. Se debe a G. Cantor y es el primer ejemplo de un *argumento diagonal*, una técnica que reaparece más adelante en el texto.

Demostración

Se demuestra la parte (2). Sea X un conjunto cualquiera.

Primero, $\text{card}(X) \leq \text{card}(\mathcal{P}(X))$: el mapa $x \mapsto \{x\}$ determina una función inyectiva de X en $\mathcal{P}(X)$, pues de $\{x_1\} = \{x_2\}$ se sigue $x_1 = x_2$.

Resta ver que los dos cardinales son distintos. Hacia una contradicción, suponga que $\text{card}(X) = \text{card}(\mathcal{P}(X))$, es decir, que existe una función biyectiva $f : X \rightarrow \mathcal{P}(X)$. Considere el conjunto:

$$D = \{x \mid x \in X \wedge x \notin f(x)\}.$$

Note que D está bien definido: para cada $x \in X$, el valor $f(x)$ es un subconjunto de X , de modo que la pregunta de si x pertenece o no a $f(x)$ tiene respuesta. Además $D \subseteq X$ y, en consecuencia, $D \in \mathcal{P}(X)$.

Como f es sobreyectiva, existe $d \in X$ tal que $f(d) = D$. La pregunta es si d pertenece o no a D :

- si $d \in D$, entonces por la definición de D se tiene $d \notin f(d)$; pero $f(d) = D$, luego $d \notin D$; y
- si $d \notin D$, entonces la condición que define a D falla para d , es decir, $d \in f(d) = D$.

Los dos casos son contradictorios, de donde no existe tal f . En conclusión:

$$\text{card}(X) \leq \text{card}(\mathcal{P}(X)) \quad \text{y} \quad \text{card}(X) \neq \text{card}(\mathcal{P}(X)),$$

es decir, $\text{card}(X) < \text{card}(\mathcal{P}(X))$.

Nota 6.7.3: El nombre del argumento

El calificativo de *diagonal* se entiende observando la Figura 6, una tabla cuyas filas están indexadas por los elementos x de X y cuyas columnas también. En la casilla de la fila x y la columna y se escribe 1 si $y \in f(x)$ y 0 en caso contrario, de manera que la fila x describe el conjunto $f(x)$.

El conjunto D de la demostración se construye recorriendo la diagonal de esa tabla, es decir, las casillas donde la fila y la columna coinciden, e invirtiendo lo que allí se encuentra. Por construcción, D difiere de la fila x en la casilla x , para cualquier x ; luego D no es ninguna de las filas y f no puede ser sobreyectiva.

La misma maniobra, aplicada a una tabla de programas contra entradas en lugar de conjuntos contra elementos, es la que permite exhibir un problema indecidible concreto. Ese es el asunto de lo que sigue en el texto.

6.7.3. ¿Cuántos lenguajes hay? Como cada lenguaje identifica un problema algorítmico de decisión, determinar el tamaño del conjunto de lenguajes también permite concluir cuántos problemas de decisión hay. Se establece un resultado clave en el camino hacia este objetivo intermedio al determinar el tamaño de $\{0, 1\}^*$.

Teorema 6.7.4: $\{0, 1\}^*$ es contable

El tamaño de Σ^* es el infinito contable, i.e., $\Sigma^* \sim \mathbb{N}$.

Demostración

Por la Definición 6.2.3, cada $n \in \mathbb{N}$ determina la cadena w_n , es decir, la que ocupa la posición n en el orden canónico de Σ^* . Basta verificar que el mapa $n \mapsto w_n$ es una biyección entre $\mathbb{N}$ y Σ^* .

	x_0	x_1	x_2	x_3	x_4	$\dots$
$f(x_0)$	1	0	1	1	0	$\dots$
$f(x_1)$	0	0	1	0	1	$\dots$
$f(x_2)$	1	1	0	0	1	$\dots$
$f(x_3)$	0	1	1	1	0	$\dots$
$f(x_4)$	1	0	0	1	1	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
diagonal	1	0	0	1	1	$\dots$
D	0	1	1	0	0	$\dots$

se lee la diagonal de la tabla
y se invierte cada una de sus casillas

Figura 6. El argumento diagonal de la demostración del Teorema 6.7.3. La fila $f(x)$ describe el conjunto $f(x)$, con un 1 en la columna y cuando $y \in f(x)$. El conjunto D se obtiene invirtiendo la diagonal, de modo que difiere de cada fila en al menos una casilla.

Es inyectivo, pues posiciones distintas del orden están ocupadas por cadenas distintas. Y es sobreyectivo, pues toda cadena ocupa alguna posición: como Σ es finito, para cada longitud hay una cantidad finita de cadenas, de modo que solo una cantidad finita de cadenas precede a una cadena dada en el orden canónico.

En conclusión, y por la Definición 6.7.1, $\Sigma^* \sim \mathbb{N}$.

El Teorema 6.7.4 establece que no hay más secuencias binarias que números naturales, lo cual puede parecer extraño: Σ^* contiene cadenas de toda longitud y, sin embargo, cabe en una sola lista. La Implementación 6.6.1 vuelve tangible esa lista, pues calcula w_n para cualquier n dado. Es más, el teorema se mantiene para cualquier alfabeto finito, e incluso si el tamaño del alfabeto es el infinito contable.

Dado que un problema algorítmico es subconjunto de Σ^* , para razonar acerca de la cantidad de lenguajes es necesario razonar acerca del tamaño del conjunto $\mathcal{P}(\Sigma^*)$, es decir, del conjunto partes de Σ^* . Para ello se usan los resultados del Teorema 6.7.3.

Teorema 6.7.5: Cantidad de lenguajes

Hay $2^{\aleph_0}$ lenguajes.

Demostración

Hay tantos problemas algorítmicos como elementos tiene $\mathcal{P}(\Sigma^*)$. Es decir, la cantidad de problemas algorítmicos es $\text{card}(\mathcal{P}(\Sigma^*))$. Observe que:

$$\begin{aligned} \text{card}(\mathcal{P}(\Sigma^*)) &= 2^{\text{card}(\Sigma^*)} && \text{(por el Teorema 6.7.3)} \\ &= 2^{\text{card}(\mathbb{N})} && \text{(por el Teorema 6.7.4)} \\ &= 2^{\aleph_0} && \text{(por el tamaño de } \mathbb{N} \text{).} \end{aligned}$$

Nota 6.7.4

El cardinal $2^{\aleph_0}$ es especial y se identifica con la letra c , dado que corresponde al cardinal del continuo: $2^{\aleph_0} = c = \text{card}(\mathbb{R})$. Por el Teorema 6.7.3 se tiene que $\aleph_0 < c$, es decir, hay (muchos) más números reales que naturales.

De manera aparte, pero como tema complementario, es importante llamar la atención a que c es interesante porque permite plantear uno de los problemas más desafiantes de las matemáticas, la Hipótesis del Continuo de Cantor (HCC):

No existe conjunto alguno cuyo tamaño esté estrictamente entre el tamaño de los naturales y el tamaño de los reales.

Dicho de otra manera, la HCC plantea la imposibilidad de que exista un cardinal entre $\aleph_0$ y $2^{\aleph_0}$. Si la HCC es cierta, entonces se tendría que

$$\aleph_1 = 2^{\aleph_0} = c.$$

6.7.4. ¿Cuántas funciones computables hay? Es necesario fijar un alfabeto de programación para determinar la cantidad de funciones computables. Por la Nota 6.4.1, ese alfabeto es el juego de caracteres con que se escribe un programa de Python; se denota Π y lo único que importa de él es que es finito. Nada de lo que sigue depende de esa elección: el argumento se sostiene para el alfabeto de cualquier lenguaje de programación, siempre que sea contable. De esta forma, se está tomando la dirección de tratar de identificar cada función computable con al menos una cadena de símbolos en Π que representa su codificación en un lenguaje de programación. Esto es lo que sucede en la realidad pues se pueden implementar tantos procedimientos de decisión como secuencias correctas de instrucciones escritas con símbolos en Π haya (en el lenguaje de programación de elección). De esta forma,

$\text{card}(\Pi^*)$ impone una cota superior sobre la cantidad de funciones implementables. El único supuesto que se necesita es que cada programa es finito, lo cual es cierto en la práctica y es además lo que exige la Definición 6.4.1: un programa es un objeto que se puede escribir.

Teorema 6.7.6: Cantidad de funciones computables

La cantidad de funciones computables es $\aleph_0$.

Demostración

La cantidad de funciones computables que se pueden programar con el alfabeto Π está acotada por $\text{card}(\Pi^*)$: el tamaño de todas las secuencias sobre Π (un programa tiene una descripción finita, luego debe ser una cadena en Π^*). Siguiendo un razonamiento similar al de la demostración del Teorema 6.7.4, se tiene que $\Pi^* \sim \mathbb{N}$. Luego, la cantidad de funciones computables es, a lo sumo, $\aleph_0$.

Para la cota inferior, escriba 1^n para denotar la cadena formada por n unos y considere, para cada $n \in \mathbb{N}$, la función total $A_n : \Sigma^* \rightarrow \Sigma$ definida para cualquier $x \in \Sigma^*$ como:

$$A_n(x) = \begin{cases} 1, & \text{si } x = 1^n, \\ 0, & \text{en caso contrario.} \end{cases}$$

Cada A_n es computable: basta un programa que compare su entrada con 1^n y que retorne 1 o 0 según coincidan o no. Además, si $n \neq m$, entonces $A_n \neq A_m$, pues $A_n(1^n) = 1$ mientras que $A_m(1^n) = 0$. Luego el mapa $n \mapsto A_n$ es inyectivo y, en consecuencia, $\aleph_0$ es cota inferior de la cantidad de funciones computables.

Las dos cotas y el Teorema 6.7.1 dan la igualdad buscada.

Saber que $\aleph_0$ es una cota superior sobre la cantidad de funciones computables es importante. Primero, esta cota se sostiene independientemente de cuál sea el alfabeto con que se implementen las soluciones algorítmicas, mientras este alfabeto sea contable. Segundo, es la última pieza que se requiere para demostrar que hay problemas algorítmicos para los cuales no existe programa de computador alguno que los resuelva. Antes de dar ese paso, conviene registrar una consecuencia inmediata sobre la cantidad de problemas que sí se pueden resolver.

Corolario 6.7.7: Cantidad de problemas decidibles y semidecidibles

La cantidad de problemas decidibles es $\aleph_0$, al igual que la cantidad de problemas semidecidibles.

Demostración

Sea D la colección de los lenguajes computables y E la de los lenguajes computablemente enumerables. Por la Definición 6.4.2, para cada $L \in E$ se puede escoger una función computable A_L tal que $L(A_L) = L$. La asignación $L \mapsto A_L$ es inyectiva, pues L se recupera de A_L como $L(A_L)$. Luego $\text{card}(E)$ no supera la cantidad de funciones computables, que es $\aleph_0$ por el Teorema 6.7.6. Como $D \subseteq E$, también $\text{card}(D) \leq \aleph_0$.

Para la cota inferior, considere para cada $n \in \mathbb{N}$ el lenguaje $L_n = \{1^n\}$. Cada L_n es computable, pues $L_n = L(A_n)$ para la función total computable A_n construida en la demostración del Teorema 6.7.6. Además, $L_n \neq L_m$ cuando $n \neq m$, de modo que el mapa $n \mapsto L_n$ es inyectivo y $\aleph_0 \leq \text{card}(D)$. Como $D \subseteq E$, también $\aleph_0 \leq \text{card}(E)$.

Por el Teorema 6.7.1 se concluye que $\text{card}(D) = \text{card}(E) = \aleph_0$.

6.7.5. Hay problemas que no se pueden decidir. Este aparte, finalmente, presenta el principal resultado de esta sección. Es decir, demuestra que existen problemas de decisión que no se pueden resolver mecánicamente con un computador, sin importar qué lenguaje de programación se use. Como se menciona al inicio de la sección, es un resultado negativo porque muestra la imposibilidad de usar los computadores como máquinas universales que resuelvan cualquier problema (de decisión). Sin embargo, como se puede comprobar con situaciones diarias, los computadores son extensamente utilizados para resolver problemas y asistir a las personas en la solución de otros tantos. Por ello, a pesar de que teóricamente se establece dicha imposibilidad de solución de problemas mecánicamente, en la práctica cada día se depende más de los computadores.

Teorema 6.7.8

Existe al menos un problema de decisión para el cual no hay una función computable que lo resuelva. Es decir, hay al menos un problema que un computador no puede resolver.

Demostración

Note que hay $2^{\aleph_0}$ lenguajes y $\aleph_0$ funciones computables, por los teoremas 6.7.5 y 6.7.6, respectivamente. Por el Teorema 6.7.3, se tiene que $\aleph_0 < 2^{\aleph_0}$. En consecuencia, hay al menos un problema de decisión para el cual no es posible diseñar un algoritmo que lo decida.

El Teorema 6.7.8 no riñe con el resultado del Teorema 6.3.2. Una cosa es entender que para cada problema hay exactamente una función total que lo decide y otra que dicha función se pueda mecanizar en un computador. Dicho de otra forma, en un computador no se pueden definir todas las funciones con dominio en Σ^* y rango en Σ . Así es como surge la existencia de problemas indecidibles: son aquellos problemas para los cuales no existe una implementación de su función de decisión asociada. La demostración del Teorema 6.7.8 no es constructiva y por ello no permite identificar concretamente uno de estos lenguajes indecidibles que sirva como testigo para corroborar su conclusión (no por ello deja de ser cierto).

Corolario 6.7.9: Cantidad de problemas indecidibles

Hay $2^{\aleph_0}$ problemas indecidibles. Más aún, hay $2^{\aleph_0}$ problemas que ni siquiera son semidecidibles.

Demostración

Sea E la colección de los lenguajes semidecidibles y sea $N = \mathcal{P}(\Sigma^*) \setminus E$ la colección de los lenguajes que no lo son. Estas dos colecciones son disjuntas y su unión es $\mathcal{P}(\Sigma^*)$, así que por la Definición 6.7.1 y el Teorema 6.7.5:

$$\text{card}(E) + \text{card}(N) = \text{card}(\mathcal{P}(\Sigma^*)) = 2^{\aleph_0}.$$

Suponga, hacia una contradicción, que $\text{card}(N) \leq \aleph_0$. Por el Corolario 6.7.7 se tiene $\text{card}(E) = \aleph_0$, que es el cardinal de un conjunto infinito, de modo que el Teorema 6.7.2 es aplicable con $X_1 = E$ y $X_2 = N$ y da $\text{card}(E) + \text{card}(N) = \text{card}(E) = \aleph_0$. Es decir, $2^{\aleph_0} = \aleph_0$, lo cual contradice el Teorema 6.7.3. Luego no se tiene $\text{card}(N) \leq \aleph_0$ y, por la comparabilidad del Teorema 6.7.1, necesariamente $\aleph_0 \leq \text{card}(N)$. En particular, N es infinito.

Ahora el Teorema 6.7.2 se puede aplicar en la dirección contraria, con $X_1 = N$ y $X_2 = E$, dado que $\text{card}(E) = \aleph_0 \leq \text{card}(N)$:

$$\text{card}(N) = \text{card}(N) + \text{card}(E) = 2^{\aleph_0}.$$

Finalmente, según la Nota 6.4.3, todo problema que no es semidecidible es indecidible, luego $N \subseteq I$, donde I denota la colección de los problemas indecidibles. Como además $I \subseteq \mathcal{P}(\Sigma^*)$, se tiene:

$$2^{\aleph_0} = \text{card}(N) \leq \text{card}(I) \leq 2^{\aleph_0},$$

de donde $\text{card}(I) = 2^{\aleph_0}$ por el Teorema 6.7.1.

El Corolario 6.7.9 refina el Teorema 6.7.8 en dos direcciones. Por una parte, no solo existe uno de estos problemas: hay tantos como lenguajes en total, mientras que los decidibles son apenas $\aleph_0$. Por otra, la conclusión se sostiene incluso para la categoría más fuerte, pues hay $2^{\aleph_0}$ problemas para los cuales no existe siquiera un algoritmo de aceptación. Los ejemplos concretos de la Sección 6.4, como $\overline{\text{FOLTHM}}$, son entonces representantes de una colección enorme y no rarezas aisladas.

Nota 6.7.5

Los resultados de esta sección admiten una lectura práctica que resulta aún menos alentadora que su enunciado.

Primero, el conteo del Teorema 6.7.6 admite programas de longitud finita pero arbitrariamente grande. En la práctica esa longitud está acotada: por la memoria de la máquina, por el tiempo de quien programa, por la duración de la civilización que lo emplea. Bajo cualquier cota fija sobre la longitud de los programas, la cantidad de funciones efectivamente implementables no es $\aleph_0$ sino finita. Note que esto no contradice el teorema: se está midiendo otra cantidad, a saber, la de las funciones computables dentro de un presupuesto fijo de recursos y no la de todas las funciones computables.

Segundo, ser decidible no es lo mismo que ser resoluble en la práctica. Entre los $\aleph_0$ problemas decidibles del Corolario 6.7.7 hay muchos para los cuales el único algoritmo conocido requiere un tiempo que excede cualquier presupuesto razonable. La distinción entre lo que se puede computar y lo que se puede computar de manera eficiente es, precisamente, el objeto del Capítulo 8.

Tercero, y en sentido contrario a los dos puntos anteriores, la indecidibilidad prohíbe menos de lo que parece. Lo que un resultado de indecidibilidad descarta es la existencia de un algoritmo *total* y *siempre correcto* sobre *toda* entrada. No descarta procedimientos que acierten sobre la mayoría de las entradas, ni procedimientos que respondan “no sé” cuando no logran decidir, ni procedimientos que funcionen bien sobre las entradas que de hecho aparecen en la práctica. Por eso los analizadores de programas existen, son útiles y conviven sin contradicción con los teoremas de esta sección. Lo mismo vale

para los sistemas de inteligencia artificial mencionados en la introducción del capítulo: el límite que no pueden rebasar es el de responder correctamente siempre, no el de ser útiles.

Con todo, conviene cerrar con una observación tranquilizadora: los problemas indecidibles pocas veces surgen al programar. El capítulo ya exhibió varios de ellos, como el problema de correspondencia de Post, pero en este texto no se ha necesitado diseñar un algoritmo de decisión para ninguno.

Ejercicios

1. Demuestre que $\sim$ es una relación de equivalencia. Es decir, reflexiva, simétrica y transitiva.
2. Demuestre que $\leq$ (sobre cardinales) es reflexiva y transitiva. Junto con la primera parte del Teorema 6.7.1, que provee la antisimetría, esto establece que $\leq$ es un orden parcial.
3. Use la propiedad de R. Dedekind en la Nota 6.7.1 para demostrar que $\mathbb{N}$ es infinito. Explique luego por qué esa misma propiedad, por sí sola, no permite concluir que hay una cantidad infinita de números primos, e indique qué haría falta para establecerlo. Finalmente, sabiendo que los primos son infinitos, determine si esa cantidad es contable o no contable.
4. Aplique el argumento diagonal de la demostración del Teorema 6.7.3 al caso $X = \mathbb{N}$ y concluya que la colección de las sucesiones infinitas de ceros y unos no es contable. Ayuda: identifique cada subconjunto de $\mathbb{N}$ con su función característica.
5. Demuestre que la colección de los lenguajes *finitos* sobre Σ tiene cardinal $\aleph_0$. Compare con el Teorema 6.7.5 y explique por qué no hay contradicción.
6. Elabore en más detalle la demostración del Teorema 6.7.6. En particular, diseñe una biyección entre Π^* y $\mathbb{N}$. Ayuda: el orden canónico de la Definición 6.2.3 está definido para cualquier alfabeto finito sobre el cual se fije un orden total de sus símbolos, y Π lo es.
7. Identifique el paso exacto de la demostración del Teorema 6.7.8 en el cual el argumento pierde la posibilidad de exhibir un testigo. Explique por qué un argumento de conteo, por sí solo, nunca puede señalar un objeto concreto.
8. El Corolario 6.7.9 afirma que hay $2^{\aleph_0}$ problemas indecidibles y el Corolario 6.7.7 que hay $\aleph_0$ decidibles. Explique por qué de estos dos hechos *no* se sigue que “la probabilidad de que un problema escogido al azar sea decidible es cero”, e indique qué habría que precisar para que una afirmación de ese tipo tuviera sentido.

Notas del capítulo y referencias

Los problemas de decisión han sido el principal objeto de estudio de la teoría de la computabilidad. Textos como el de M. Davis [Dav85] explican de manera detallada por qué es razonable restringir el estudio de la computabilidad e indecidibilidad a problemas de decisión. Explicaciones similares se pueden encontrar en [CLRS22, Eri19]. El tratamiento de la Sección 6.1, que separa la generalidad computacional del precio polinomial, distingue dos asuntos que la literatura suele presentar juntos. El primero es el que le concierne a este capítulo; el segundo es un anticipo del Capítulo 8. Basar el estudio de problemas de decisión en el marco de lenguajes formales es parte de la influencia del texto de Cormen et al. [CLRS22] sobre este texto.

La definición ‘genérica’ de problema de decisión, y las definiciones de aceptación y decisión de lenguajes basadas en funciones parciales y totales, no son las usuales (secciones 6.1 y 6.3). En particular, la relación entre función total y decisión sobre lenguajes (Teorema 6.3.2) es novedosa. Se optó por combinar lenguajes y funciones porque permite abordar el objeto de estudio en cuestión a alto nivel, identificando los conceptos clave y sin perder generalidad. Las funciones parciales y totales son conceptos básicos de matemáticas y son familiares para un grupo amplio de lectores. La tricotomía entre aceptar, rechazar y quedar indeterminada, que organiza la Sección 6.3, hace visible desde el comienzo que la negación de “aceptar” no es “rechazar”.

La noción de computabilidad que presenta este capítulo es la usual, y hay tratamientos similares en [Dav85, Har12, Sip12]. Incluir definiciones de computabilidad a nivel de funciones y de problemas es conveniente dado que permite entender el concepto de manera más general. De igual forma, es importante indicar las relaciones de conjunto computablemente enumerable y computable con las nociones de semidecidibilidad y decidibilidad, que son también comunes en literatura de ciencias de la computación. La decisión de fijar Python como modelo de cómputo (Nota 6.4.1) sí es propia de este texto. La mayoría de los tratamientos recurre a las máquinas de Turing o a las funciones recursivas, y paga por ello el costo de construir la maquinaria antes de poder decir nada. Fijar un lenguaje que el lector ya conoce permite escribir los programas cuya existencia afirman las definiciones, lo cual se aprovecha en el Teorema 6.4.1 y en la Sección 6.5. La contrapartida es que hay que argumentar que la elección no restringe la noción, y eso es exactamente lo que hace la tesis de Church y Turing.

Al respecto, hay tres categorías principales de modelos de computación: modelos secuenciales que incluyen las máquinas de Turing, las máquinas de Post y las máquinas de acceso aleatorio; modelos funcionales que incluyen el cálculo λ ,

las funciones recursivas generales, los sistemas de reescritura abstractos y la lógica combinatoria; y modelos concurrentes como el modelo de actores, las redes de Petri, la lógica de reescritura y los autómatas celulares. Material adicional de sistemas de reescritura abstractos se puede encontrar en [Ohl02] y de lógica de reescritura en [CDE⁺07]. Los demás modelos cuentan con literatura abundante que puede ser encontrada en textos clásicos o en internet. El Teorema 6.4.1, que establece la existencia de un intérprete universal, aparece por primera vez en el artículo de A. Turing de 1936 [Tur36] bajo la forma de la máquina universal. El texto de C. Petzold [Pet08] recorre ese artículo línea por línea y es una buena puerta de entrada para quien quiera leer el original.

Las propiedades de clausura de la Sección 6.5 y la caracterización por enumeración de la Sección 6.6 son resultados estándar de la teoría de la computabilidad, y se pueden consultar con más detalle en [Sip12, HR87]. El Teorema 6.5.3 se debe a E. Post [Pos44], artículo en el cual se introduce además buena parte del vocabulario que hoy se usa para hablar de conjuntos computablemente enumerables. La presentación que se hace aquí difiere de la habitual en dos aspectos. Primero, la simulación acotada (Nota 6.5.1) se introduce de manera explícita como herramienta, en lugar de aparecer camuflada dentro de cada demostración. Segundo, el orden de las dos secciones es deliberado: la clausura explica *por qué* hay tres categorías de problemas y no dos, y la enumeración explica *de dónde viene el nombre* de una de ellas. Las dos preguntas suelen quedar sin respuesta en tratamientos introductorios.

El tratamiento que se hace de la existencia de problemas indecidibles o funciones no computables no es el común, ni para pregrados de computación ni de matemáticas. Esto puede ser así por la necesidad de usar aritmética cardinal, que no es un tema estudiado, generalmente, en ninguno de estos currículos. La principal justificación para dar este tratamiento en el capítulo tiene que ver con la claridad del argumento (no constructivo): hay al menos una función que no puede ser calculada algorítmicamente. Los resultados acerca de cardinales utilizados en la Sección 6.7, incluyendo su aritmética, pueden ser constatados y profundizados en textos de teoría de conjuntos intuitiva como [Bre06, Hal11] y axiomática como [Kun11].

De los resultados sobre cardinales que se enuncian sin demostración conviene señalar una distinción. La antisimetría de la primera parte del Teorema 6.7.1 es el teorema de Cantor, Schröder y Bernstein y no requiere el axioma de elección; la comparabilidad de la segunda parte, en cambio, sí es equivalente a él, al igual que el Teorema 6.7.2. Ninguna de las dos salvedades altera lo que el capítulo demuestra, pero conviene que el lector sepa dónde están. La parte (2) del Teorema 6.7.3 sí se demuestra, y su argumento diagonal se debe a G. Cantor [Can91]. Es la misma maniobra que reaparece, con programas en lugar de conjuntos, cuando se busca exhibir un problema indecidible concreto. Los corolarios 6.7.7 y 6.7.9 no suelen

enunciarse de manera separada en la literatura, aunque se siguen sin dificultad del conteo. Se incluyen aquí porque convierten una intuición vaga, la de que los problemas decidibles son “muy pocos”, en dos cardinales concretos que se pueden comparar.

Las notas históricas distribuidas por el capítulo se apoyan en las fuentes originales. La lista de problemas que D. Hilbert presentó en París corresponde a [Hil00], y la respuesta negativa a su décimo problema se completó con el trabajo de Y. Matiyásevich [Mat70]. Los teoremas de incompletitud son de K. Gödel [Göd31]. La respuesta negativa al *Entscheidungsproblem* apareció en 1936 en dos trabajos independientes: el de A. Church [Chu36b, Chu36a], basado en el cálculo λ , y el de A. Turing [Tur36], basado en las máquinas que hoy llevan su nombre. La antología editada por M. Davis [Dav04] reúne estos y otros artículos fundacionales en un solo volumen y es la referencia recomendada para quien quiera ir a las fuentes.

Conviene no confundir las dos contribuciones de E. Post que aparecen en el capítulo. Una es el Teorema 6.5.3, que caracteriza cuándo un lenguaje semidecidible llega a ser decidible. La otra es el problema de correspondencia del Ejemplo 6.4.2, que es un problema indecidible concreto y cuya formulación aparece en [Pos46]. El texto [Ohl02] de E. Ohlebusch es una buena fuente de problemas indecidibles, como terminación y confluencia, en el marco de los sistemas de reescritura abstractos y de reescritura de términos, y del problema de correspondencia.

Finalmente, una advertencia sobre el alcance de lo que este capítulo demuestra. El argumento de la Sección 6.7 establece que los problemas indecidibles existen y que son muchos, pero no exhibe ninguno. Los ejemplos concretos que sí aparecen, como el problema de correspondencia de Post y $\overline{\text{FOLTHM}}$, se citan sin demostrar. Construir un problema indecidible desde cero, y obtener a partir de él otros, exige herramientas de naturaleza distinta a las de este capítulo.

Indecidibilidad

El Capítulo 6 termina con una deuda. Su argumento cardinal demuestra que los problemas indecidibles existen, y que de hecho son tantos como lenguajes hay en total, pero no exhibe ninguno. Los únicos ejemplos concretos que aparecen allí, como el problema de correspondencia de Post, se citan sin demostrar. Este capítulo paga esa deuda: construye un problema indecidible desde cero y, a partir de él, obtiene muchos otros.

La estrategia tiene dos etapas y conviene anunciarla de entrada, porque explica el orden de las secciones. La primera etapa consiste en disponer de una *semilla*, es decir, de un problema particular del cual se demuestre, sin apelar a ningún resultado previo de indecidibilidad, que ningún algoritmo lo decide. La segunda consiste en *transportar* esa indecidibilidad a otros problemas mediante una técnica que convierte instancias de un problema en instancias de otro. Ninguna de las dos etapas sirve por separado: sin la semilla no hay nada que transportar, y sin la técnica de transporte cada problema nuevo exigiría empezar de cero.

Las herramientas necesarias ya están disponibles. La semilla se obtiene del hecho, establecido en el Capítulo 6, de que un programa es a la vez un objeto que se ejecuta y una cadena que se puede leer, lo cual permite que un programa reciba programas como entrada y, en particular, que se formulen preguntas de un programa acerca de sí mismo. El argumento que explota esa posibilidad es el mismo argumento diagonal con el que G. Cantor comparó el tamaño de un conjunto con el de su conjunto potencia, aplicado ahora a una tabla de programas contra entradas.

El capítulo concluye con una colección de problemas indecidibles concretos y con un resultado que los explica a todos: ninguna propiedad semántica no trivial de los programas es decidible. Esa afirmación es la que traduce la teoría de este capítulo al oficio de quien programa, pues descarta de una vez la posibilidad de construir el analizador estático perfecto, el verificador de equivalencia entre programas o el detector infalible de ciclos infinitos.

7.1. Reducciones

El Teorema 6.7.8 y el Corolario 6.7.9 establecen que hay una cantidad enorme de problemas indecidibles, pero no exhiben ninguno. Este capítulo se propone corregir esa situación exhibiendo problemas concretos. La estrategia tiene dos etapas: primero se demuestra, desde cero, que un problema particular es indecidible (Sección 7.2); después, ese resultado se transporta a otros problemas mediante la técnica que ocupa esta sección.

La idea es la siguiente. Suponga que se quiere determinar si un problema L_2 es decidable y que ya se sabe que cierto problema L_1 no lo es. Si se logra exhibir un procedimiento mecánico que convierta cualquier instancia de L_1 en una instancia de L_2 , respetando la respuesta, entonces un algoritmo de decisión para L_2 produciría uno para L_1 : bastaría convertir la instancia y consultar. Como se sabe que L_1 no tiene algoritmo de decisión, tal algoritmo para L_2 no puede existir. En términos de ingeniería, exhibir la conversión es decir: *si supiera resolver L_2 , sabría resolver L_1* .

Formalizar esta idea requiere considerar funciones cuyo rango es Σ^* y no Σ , pues una conversión de instancias produce cadenas y no bits. Tal y como se anticipa en la Sección 6.4, la noción de computabilidad se extiende sin dificultad a este caso.

Definición 7.1.1: Funciones computables sobre cadenas

Sea $f : \Sigma^* \rightarrow \Sigma^*$ una función total. Se dice que f es *computable* si y solo si existe un programa, en el sentido de la Nota 6.4.1, que para cualquier $x \in \Sigma^*$ termina cuando se invoca con x y retorna el valor $f(x)$.

La Definición 7.1.1 es la Definición 6.4.1 restringida a funciones totales y con el rango ampliado de Σ a Σ^* . La restricción a funciones totales no es una pérdida para los propósitos de esta sección: una conversión de instancias que no terminara sobre alguna entrada no serviría para nada.

Note que el punto de entrada de un programa que calcula una función de este tipo retorna una cadena y no un bit, a diferencia de lo que establece la Nota 6.4.1. Es la misma situación de *simulate* en la Nota 6.5.1 y de los enumeradores de la Definición 6.6.1: el modelo de cómputo es el mismo y lo único que cambia es la forma del resultado.

Definición 7.1.2: Reducción computable

Sean $L_1, L_2 \subseteq \Sigma^*$. Se dice que L_1 se *reduce* a L_2 , escrito $L_1 \leq_c L_2$, si y solo si existe una función total computable $f : \Sigma^* \rightarrow \Sigma^*$ tal que, para cualquier

$x \in \Sigma^*$:

$$x \in L_1 \iff f(x) \in L_2.$$

En tal caso se dice que f es una *reducción* de L_1 a L_2 .

Conviene detenerse en la condición de la Definición 7.1.2. Exige una equivalencia y no una implicación: la función f debe enviar instancias positivas de L_1 en instancias positivas de L_2 e instancias negativas en instancias negativas. Una función que solo cumpla la implicación de izquierda a derecha podría enviar toda cadena a una misma instancia positiva de L_2 y no diría nada acerca de L_1 . Note también que f no necesita ser inyectiva ni sobreyectiva, y que nada obliga a que sea eficiente; solo se le pide terminar y respetar la pertenencia.

Nota 7.1.1: Las cadenas mal formadas

La Definición 7.1.2 exige la equivalencia para *toda* cadena de Σ^* y no solo para las que codifican instancias legítimas. Conviene fijar de una vez qué hace una reducción con las demás, porque la respuesta no es la que uno escribe por reflejo y porque el asunto reaparece en cada construcción del capítulo.

En todo lo que sigue L_1 es un lenguaje de códigos, de manera que una cadena mal formada no pertenece a L_1 y su imagen debe quedar *fuera* de L_2 . Basta entonces con fijar de antemano una cadena $y_0 \notin L_2$ y hacer que f la retorne cada vez que su entrada no sea una instancia bien formada. Reconocer si lo es es un análisis sintáctico, según la Nota 6.2.3, de modo que f sigue siendo total y computable.

Lo que hay que notar es que la elección de y_0 depende de L_2 y no de f . Cuando L_2 es a su vez un lenguaje de códigos, cualquier cadena mal formada sirve como y_0 y el asunto pasa inadvertido: esa es la elección refleja y en ese caso es correcta. Cuando L_2 es el complemento de uno de esos lenguajes la situación se invierte, pues las cadenas mal formadas están entonces *dentro* de L_2 y retornar una de ellas rompe la equivalencia. El Ejemplo 7.1.1 es de este segundo tipo.

Note, por último, que una cadena y_0 existe si y solo si $L_2 \neq \Sigma^*$, que es justamente lo que pide demostrar el Ejercicio 3.

Los dos resultados que siguen son los que le dan a la reducción todo su poder. El primero transporta decidibilidad en la dirección de la flecha.

Teorema 7.1.1: Transporte de decidibilidad

Sean $L_1, L_2 \subseteq \Sigma^*$ tales que $L_1 \leq_c L_2$. Si L_2 es decidible, entonces L_1 es decidible.

Demostración

Sea f una reducción de L_1 a L_2 . Como L_2 es decidible, la Definición 6.4.2 garantiza la existencia de una función total computable $A_2 : \Sigma^* \rightarrow \Sigma$ tal que $L(A_2) = L_2$. Considere la función $A_1 : \Sigma^* \rightarrow \Sigma$ definida, para cualquier $x \in \Sigma^*$, como $A_1(x) = A_2(f(x))$.

La función A_1 es total, pues f y A_2 lo son. Y es computable: un programa para A_1 consiste en invocar el programa de f sobre x , que termina y retorna $f(x)$, y luego invocar el programa de A_2 sobre ese resultado, que también termina.

Resta verificar que A_1 decide L_1 . Para cualquier $x \in \Sigma^*$:

$$\begin{aligned} A_1(x) = 1 &\iff A_2(f(x)) = 1 && \text{(definición de } A_1) \\ &\iff f(x) \in L_2 && (L(A_2) = L_2) \\ &\iff x \in L_1 && (f \text{ es reducción, Definición 7.1.2}). \end{aligned}$$

Luego $L(A_1) = L_1$ y, como A_1 es total y computable, L_1 es decidible.

El Teorema 7.1.1 se enuncia en términos de decidibilidad, pero en la práctica se usa siempre en su forma contrapositiva. Esa forma es la herramienta de trabajo del resto del capítulo y merece enunciado propio.

Corolario 7.1.2: Transporte de indecidibilidad

Sean $L_1, L_2 \subseteq \Sigma^*$ tales que $L_1 \leq_c L_2$. Si L_1 es indecidible, entonces L_2 es indecidible.

Demostración

Es la contrapositiva del Teorema 7.1.1.

El Corolario 7.1.2 indica cómo se usa una reducción para demostrar que un problema es indecidible: para concluir que L_2 es indecidible se debe exhibir una reducción *desde* un problema L_1 que ya se sepa indecidible *hacia* L_2 . La dirección es esencial y es la fuente de error más común al aplicar la técnica; la Nota 7.1.2 vuelve sobre este punto.

La misma construcción transporta semidecidibilidad.

Teorema 7.1.3: Transporte de semidecidibilidad

Sean $L_1, L_2 \subseteq \Sigma^*$ tales que $L_1 \leq_c L_2$. Si L_2 es semidecidible, entonces L_1 es semidecidible.

Demostración

Sea f una reducción de L_1 a L_2 y sea $A_2 : \Sigma^* \rightarrow \Sigma$ una función computable, no necesariamente total, tal que $L(A_2) = L_2$. Considere de nuevo $A_1(x) = A_2(f(x))$, que es computable por el mismo argumento de la demostración del Teorema 7.1.1. Como f es total, $A_1(x)$ está indeterminada exactamente cuando $A_2(f(x))$ lo está. Entonces:

$$\begin{aligned} L(A_1) &= \{x \mid x \in \Sigma^* \wedge A_2(f(x)) = 1\} \\ &= \{x \mid x \in \Sigma^* \wedge f(x) \in L_2\} \\ &= L_1, \end{aligned}$$

de donde L_1 es semidecidible.

Corolario 7.1.4

Sean $L_1, L_2 \subseteq \Sigma^*$ tales que $L_1 \leq_c L_2$. Si L_1 no es semidecidible, entonces L_2 no es semidecidible.

Demostración

Es la contrapositiva del Teorema 7.1.3.

El Corolario 7.1.4 es el que permite alcanzar la tercera categoría de la Nota 6.4.3, es decir, la de los problemas que ni siquiera admiten un algoritmo de aceptación.

La relación $\leq_c$ tiene, además, buenas propiedades estructurales.

Teorema 7.1.5: Propiedades de la reducción

Sean $L, L_1, L_2, L_3 \subseteq \Sigma^*$. Entonces:

1. $L \leq_c L$.

2. Si $L_1 \leq_c L_2$ y $L_2 \leq_c L_3$, entonces $L_1 \leq_c L_3$.

3. $L_1 \leq_c L_2$ si y solo si $\overline{L_1} \leq_c \overline{L_2}$.

En consecuencia, $\leq_c$ es un preorden sobre los lenguajes.

Demostración

Se procede por partes.

(1): La función identidad, definida por $x \mapsto x$, es total y computable, y claramente $x \in L$ si y solo si $x \in L$.

(2): Sean f_1 una reducción de L_1 a L_2 y f_2 una de L_2 a L_3 . La función $f_2 \circ f_1$ es total y computable, pues se calcula invocando los dos programas en secuencia. Además, para cualquier $x \in \Sigma^*$:

$$x \in L_1 \iff f_1(x) \in L_2 \iff f_2(f_1(x)) \in L_3.$$

(3): Sea f una reducción de L_1 a L_2 . La misma función f sirve como reducción de $\overline{L_1}$ a $\overline{L_2}$, pues para cualquier $x \in \Sigma^*$:

$$x \in \overline{L_1} \iff x \notin L_1 \iff f(x) \notin L_2 \iff f(x) \in \overline{L_2}.$$

El recíproco es el mismo argumento aplicado a $\overline{L_1}$ y $\overline{L_2}$, recordando que $\overline{\overline{L}} = L$.

Note que la parte (3) del Teorema 7.1.5 tiene una consecuencia útil: dado que $\leq_c$ respeta complementos, una reducción sirve simultáneamente para razonar sobre un problema y sobre su negación. Note también que $\leq_c$ no es antisimétrica: dos lenguajes distintos pueden reducirse mutuamente sin ser iguales. Por eso es un preorden y no un orden parcial, a diferencia de la relación $\leq$ sobre cardinales del Teorema 6.7.1.

El primer ejemplo ilustra el mecanismo sobre dos problemas conocidos.

Ejemplo 7.1.1

Considere los lenguajes

$$\text{TAUT} = \{\langle \varphi \rangle \mid \text{"}\varphi \text{ es una proposición tautológica"}\}$$

$$\text{SAT} = \{\langle \varphi \rangle \mid \text{"}\varphi \text{ es una proposición satisfacible"}\}$$

del Ejemplo 6.1.1 y el Ejemplo 6.4.1. Se afirma que $\text{TAUT} \leq_c \overline{\text{SAT}}$.

Considere la función f que, dada la codificación de una proposición φ , retorna la codificación de $\neg\varphi$. Cuando su entrada no codifica proposición alguna,

f retorna $\langle p_1 \rangle$, el código de una proposición satisfacible fija. Esta función es total y computable: negar una proposición es una operación sintáctica que siempre termina y verificar que una cadena codifica una proposición también.

La elección de $\langle p_1 \rangle$ es la que impone la Nota 7.1.1 y vale la pena detenerse en ella, porque aquí el blanco es un complemento. Una cadena mal formada no está en TAUT, de manera que su imagen tiene que quedar fuera de $\overline{\text{SAT}}$, es decir, tiene que ser satisfacible. Retornar otra cadena mal formada, que es lo que uno escribe por reflejo, la dejaría *dentro* de $\overline{\text{SAT}}$ y rompería la equivalencia.

Resta verificar la condición de la Definición 7.1.2. Una proposición φ es una tautología si y solo si es verdadera bajo toda asignación de sus variables, lo cual sucede si y solo si $\neg\varphi$ es falsa bajo toda asignación, es decir, si y solo si $\neg\varphi$ no es satisfacible. Luego $\langle\varphi\rangle \in \text{TAUT}$ si y solo si $f(\langle\varphi\rangle) \in \overline{\text{SAT}}$. Y si x está mal formada, entonces $x \notin \text{TAUT}$ y $f(x) = \langle p_1 \rangle \notin \overline{\text{SAT}}$, con lo cual la equivalencia también se cumple. La condición vale entonces sobre todo Σ^* .

El Ejemplo 7.1.1 no aporta información nueva, pues TAUT ya se sabe decidable por el Ejemplo 6.4.1. Su valor es mostrar el mecanismo en un caso donde todo es verificable a mano. El segundo ejemplo advierte sobre el alcance de la definición.

Ejemplo 7.1.2

Sea L_1 un lenguaje decidable y sea L_2 un lenguaje tal que $L_2 \neq \emptyset$ y $L_2 \neq \Sigma^*$. Entonces $L_1 \leq_c L_2$.

Como $L_2 \neq \emptyset$, existe una cadena $y_1 \in L_2$; como $L_2 \neq \Sigma^*$, existe una cadena $y_0 \notin L_2$. Además, como L_1 es decidable, existe una función total computable A_1 tal que $L(A_1) = L_1$. Considere:

$$f(x) = \begin{cases} y_1 & , \text{ si } A_1(x) = 1, \\ y_0 & , \text{ en caso contrario.} \end{cases}$$

Esta función es total y computable, pues A_1 lo es y y_0 e y_1 son cadenas fijas. Además,

$$x \in L_1 \iff A_1(x) = 1 \iff f(x) = y_1 \iff f(x) \in L_2.$$

El Ejemplo 7.1.2 muestra que la existencia de una reducción, por sí sola, no dice gran cosa: todo problema decidable se reduce a cualquier problema no trivial. Lo informativo no es que exista una reducción, sino *de qué problema* parte. Una reducción es valiosa exactamente cuando su lado izquierdo es difícil.

Nota 7.1.2: La dirección de la reducción

Al aplicar el Corolario 7.1.2 es fácil equivocarse la dirección. Para demostrar que L_2 es indecidible hay que exhibir una reducción $L_1 \leq_c L_2$, con L_1 indecidible conocido, y no una reducción $L_2 \leq_c L_1$.

La reducción en la dirección equivocada no permite concluir nada. Por el Ejemplo 7.1.2, cualquier problema decidable se reduce a cualquier problema no trivial; en particular, exhibir $L_2 \leq_c L_1$ con L_1 indecidible es compatible con que L_2 sea decidable y con que no lo sea.

Una forma de recordar la dirección es leer la reducción como una frase: $L_1 \leq_c L_2$ dice que L_1 *no es más difícil que* L_2 . Por eso la dificultad viaja hacia la derecha y la facilidad hacia la izquierda.

Ejercicios

1. Demuestre que si $L_1 \leq_c L_2$ y L_1 no es decidable, entonces L_2 tampoco lo es, sin invocar el Teorema 7.1.1. Es decir, elabore el argumento directamente a partir de la Definición 7.1.2.
2. Sea $f : \Sigma^* \rightarrow \Sigma^*$ una función total computable y suponga que solo se sabe que $x \in L_1$ implica $f(x) \in L_2$, sin la implicación recíproca. Exhiba lenguajes L_1 y L_2 y una función f que cumplan esta condición débil, con L_1 decidable y L_2 no decidable, y otros con L_1 no decidable y L_2 decidable. Concluya que la condición débil no sirve para transportar nada.
3. Demuestre que $\emptyset \leq_c L$ si y solo si $L \neq \Sigma^*$, y que $\Sigma^* \leq_c L$ si y solo si $L = \Sigma^*$.
4. Demuestre que $\leq_c$ no es antisimétrica exhibiendo dos lenguajes distintos L_1 y L_2 tales que $L_1 \leq_c L_2$ y $L_2 \leq_c L_1$.
5. Sean L_1 y L_2 lenguajes decidibles, ambos distintos de $\emptyset$ y de Σ^* . Demuestre que $L_1 \leq_c L_2$ y $L_2 \leq_c L_1$. Es decir, todos los problemas decidibles no triviales son equivalentes con respecto a $\leq_c$.
6. Suponga que $L_1 \leq_c L_2$ y que L_2 es decidable. ¿Se puede concluir algo acerca de $\overline{L_1}$? Justifique.
7. Considere el lenguaje CNFSAT de las proposiciones satisfacibles en forma normal conjuntiva. Explique por qué $\text{CNFSAT} \leq_c \text{SAT}$ es inmediato y describa cuál es la reducción.
8. Demuestre que si L es decidable, entonces $L \leq_c L_2$ para cualquier L_2 no trivial y $L \leq_c \overline{L_2}$ para cualquier L_2 no trivial. Explique por qué esto es coherente con la parte (3) del Teorema 7.1.5.

9. En la demostración del Teorema 7.1.3 se afirma que $A_1(x)$ está indeterminada exactamente cuando $A_2(f(x))$ lo está. Justifique esta afirmación e indique dónde se usa que f es total.
10. Investigue acerca de la diferencia entre las reducciones *many-one* y las reducciones de *Turing*. Explique cuál de las dos corresponde a la Definición 7.1.2 y dé una situación en la que la otra sea más conveniente.

7.2. El problema de la parada

Esta sección construye la semilla anunciada en la introducción del capítulo. Se trata de un problema de decisión que cualquier programador reconoce, del cual se demuestra sin apelar a ningún resultado previo de indecidibilidad, que ningún algoritmo lo resuelve. La pregunta es la siguiente: dado un programa y una entrada, ¿terminará ese programa al ser invocado con esa entrada? Formularla como un problema de decisión requiere únicamente la codificación de la Nota 6.2.3, pues tanto el programa como la entrada son cadenas.

Definición 7.2.1: El problema de la parada

El *problema de la parada* es el lenguaje:

$$\text{HALT} = \{\langle P, x \rangle \mid \text{"}P \text{ termina al ser invocado con } x\text{"}\}.$$

Conviene detenerse en lo que se está preguntando. No se trata de determinar si *cierto* programa termina, cosa que en muchos casos particulares es sencilla, sino de disponer de un procedimiento único que responda correctamente para *cualquier* par de programa y entrada. La diferencia entre las dos lecturas es la misma que hay entre resolver un caso y resolver un problema.

7.2.1. La parada es indecidible. Antes del enunciado conviene anticipar la forma del argumento, porque es corto pero no es obvio. Se supone que el procedimiento buscado existe, es decir, que hay un programa capaz de predecir, para cualquier par de programa y entrada, si la ejecución termina. Con ese predictor a la mano se construye un programa nuevo que hace una sola cosa: le pregunta al predictor qué va a pasar consigo mismo y luego se comporta al revés de lo predicho. Si el predictor anuncia que terminará, el programa se queda dando vueltas; si anuncia que no terminará, el programa termina de inmediato. Ninguna de las dos respuestas puede ser correcta, de modo que alguna de las suposiciones es falsa. Y la única suposición que se hizo fue la existencia del predictor.

Note que no hay paradoja en esto. La construcción se parece a la del mentiroso que afirma estar mintiendo, pero con una diferencia decisiva: aquí todos los

ingredientes, salvo uno, están contruidos y son indudables. El programa nuevo es un programa de Python corriente, su texto es una cadena y ejecutar un programa sobre una cadena es una operación legítima. Lo único dudoso es el predictor y por eso la contradicción cae sobre él.

Teorema 7.2.1: Indecidibilidad de la parada

HALT no es computable.

Demostración

Hacia una contradicción, suponga que HALT es computable. Existe entonces, por la Definición 6.4.2, una función total computable que lo decide y, en consecuencia, un programa que la calcula. Sea `halts` ese programa. Por hipótesis, la invocación `halts(p, x)` termina siempre: retorna 1 si el programa p termina al ser invocado con x y retorna 0 en caso contrario.

Considere el programa D de la Implementación 7.2.1, que se construye usando `halts` como subrutina. Sea d la cadena que codifica a D , es decir, su propio texto. La pregunta que produce la contradicción es si D termina al ser invocado con d .

Si `halts(d, d)` retorna 1: entonces, por la definición de `halts`, el programa D termina al ser invocado con d . Pero al ejecutar D sobre d , la condición del `if` resulta verdadera y la ejecución entra en el ciclo infinito, de modo que D no termina al ser invocado con d .

Si `halts(d, d)` retorna 0: entonces D no termina al ser invocado con d . Pero al ejecutar D sobre d , la condición del `if` resulta falsa, la ejecución salta el ciclo y retorna 1, de modo que D sí termina al ser invocado con d .

Las dos alternativas son contradictorias y, como `halts` termina siempre, alguna de las dos tiene que darse. Luego el programa `halts` no existe y HALT no es computable.

Implementación 7.2.1: El programa D

```
1 def decide(w):
2     if halts(w, w) == 1:           # does w halt on its own text?
3         while True: pass         # then loop forever
4     return 1                      # otherwise halt right away
```

Vale la pena recorrer el programa D línea por línea. Recibe una cadena w y la usa dos veces: como el programa cuya terminación se consulta y como la entrada de ese programa. Es decir, la consulta $\text{halts}(w, w)$ pregunta si w termina al recibir su propio texto. Si la respuesta es afirmativa, el **while** sin salida garantiza que D no termine; si es negativa, la ejecución salta el ciclo y el **return** hace que D termine de inmediato.

Hasta aquí D es un programa como cualquier otro y no hay nada extraño. Lo extraño aparece al invocarlo sobre su propio texto, pues en ese caso la cadena w pasa a ser d y la consulta $\text{halts}(d, d)$ predice el comportamiento de la ejecución que está ocurriendo en ese preciso momento. El programa D hace entonces exactamente lo contrario de lo predicho y esa inversión deliberada es la que no admite respuesta consistente.

Nota 7.2.1: La misma diagonal

La demostración del Teorema 7.2.1 es el argumento diagonal de la Nota 6.7.3 con otro contenido, como lo muestra la Figura 1. Las filas de la tabla están indexadas por los programas $P_0, P_1, P_2, \dots$ y las columnas por sus propias codificaciones $\langle P_0 \rangle, \langle P_1 \rangle, \dots$; la casilla de la fila i y la columna j lleva un 1 si P_i termina al ser invocado con $\langle P_j \rangle$ y un 0 en caso contrario. Que las filas y las columnas se puedan indexar de esa forma es consecuencia del Teorema 6.7.4: los programas son cadenas y las cadenas se pueden listar.

El programa D recorre la diagonal de esa tabla e invierte lo que encuentra: cuando la casilla (i, i) lleva un 1, D no termina sobre $\langle P_i \rangle$, y cuando lleva un 0 sí termina. Por construcción, el comportamiento de D difiere del de P_i en la entrada $\langle P_i \rangle$, para cualquier i , de modo que D no es ninguno de los programas de la lista. Pero D es un programa y la lista los contiene a todos.

La diferencia con el caso de Cantor es de dónde sale la contradicción. Allá la tabla existe y lo que se descarta es la sobreyectividad de f ; acá la lista de programas existe sin duda, de manera que lo que se descarta es la existencia de halts , que es lo único que se había supuesto.

7.2.2. ¿Dónde queda la parada? El Teorema 7.2.1 dice que HALT no es decidible, pero no dice si admite un algoritmo de aceptación. La respuesta es que sí y el algoritmo ya está construido: es el intérprete universal del Teorema 6.4.1.

La intuición detrás de este hecho es casi decepcionante. Para *aceptar* no hace falta predecir nada: basta con ejecutar el programa sobre la entrada y esperar. Si

	$\langle P_0 \rangle$	$\langle P_1 \rangle$	$\langle P_2 \rangle$	$\langle P_3 \rangle$	$\langle P_4 \rangle$	$\dots$	
P_0	1	1	0	1	0	$\dots$	1: termina 0: no termina
P_1	0	1	1	0	1	$\dots$	
P_2	1	0	1	1	0	$\dots$	
P_3	1	1	0	0	1	$\dots$	
P_4	0	0	1	0	1	$\dots$	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
diagonal	1	1	1	0	1	$\dots$	se lee la diagonal
D	0	0	0	1	0	$\dots$	y se invierte: así se comporta D

¿cuál de las filas es D ?

Figura 1. El argumento diagonal del Teorema 7.2.1. La casilla de la fila P_i y la columna $\langle P_j \rangle$ registra si P_i termina al ser invocado con $\langle P_j \rangle$. El programa D invierte la diagonal, de modo que su comportamiento difiere del de P_i en la entrada $\langle P_i \rangle$, para cualquier i . Compare con la Figura 6.

termina, se responde que sí y la respuesta es correcta. Si no termina, la espera no acaba nunca, pero eso es justamente lo que la Definición 6.3.3 permite en el caso de las cadenas que no pertenecen al lenguaje. Dicho de otro modo, toda la dificultad de HALT está del lado negativo. Reconocer que un programa termina es fácil, porque el propio programa lo demuestra al terminar. Reconocer que no termina es lo que ningún procedimiento puede hacer, porque no hay un momento en el cual la no terminación quede exhibida.

Teorema 7.2.2

HALT es computablemente enumerable.

Demostración

Considere el programa de la Implementación 7.2.2 y sea A la función que calcula.

Sea $w \in \Sigma^*$ y considere los tres casos posibles.

Si $w = \langle P, x \rangle \in \text{HALT}$, entonces P termina al ser invocado con x y, por el Teorema 6.4.1, la invocación de `universal` termina; luego el programa retorna 1 y $A(w) = 1$.

Si $w = \langle P, x \rangle \notin \text{HALT}$, entonces P no termina al ser invocado con x y, de nuevo por el Teorema 6.4.1, la invocación de `universal` tampoco termina; luego $A(w)$ está indeterminada y, en particular, no vale 1.

Si w está mal formada, entonces $w \notin \text{HALT}$ y la validación hace que el programa retorne 0, de modo que $A(w) = 0$ y tampoco vale 1. Note que para un algoritmo de aceptación basta con no retornar 1: la Definición 6.3.3 no distingue entre retornar 0 y no terminar.

En conclusión $L(A) = \text{HALT}$, de donde `HALT` es computablemente enumerable.

Implementación 7.2.2: Un algoritmo de aceptación para Halt

```

1 def decide(w):
2     ans = 0
3     if valid(w):                # does w encode a pair <P, x>?
4         universal(w)           # run P on x; may never return
5         ans = 1                # reached only if P halted
6     return ans

```

Salvo por la validación inicial, este programa no verifica nada: se limita a ejecutar P sobre x y a reportar que terminó, si es que termina. Es el ejemplo más puro de la asimetría de la Nota 6.5.3, pues quien lo ejecuta y ve que aún no ha respondido no puede saber si el programa va a responder más adelante o nunca.

La validación no es un adorno. Sin ella, una cadena mal formada haría que `universal` terminara retornando 0, según la convención de la Implementación 6.4.2, y el programa respondería 1 sobre una cadena que no está en `HALT`. Es el mismo cuidado de la Implementación 6.4.1, con una diferencia: allá el 0 significaba rechazar y acá significa, simplemente, no aceptar.

Si aceptar es fácil del lado positivo y la dificultad está del lado negativo, cabe preguntarse qué ocurre con el problema complementario, el de determinar si un programa *no* termina. La respuesta es que ese problema está aún peor: no solo carece de algoritmo de decisión, sino que tampoco tiene algoritmo de aceptación.

Corolario 7.2.3

$\overline{\text{HALT}}$ no es computablemente enumerable.

Demostración

Por el Teorema 7.2.2, HALT es computablemente enumerable y por el Teorema 7.2.1 no es computable. El Corolario 6.5.4 da entonces el resultado.

Nota 7.2.2: Testigos propios

La Nota 6.4.3 reparte los problemas de decisión en tres categorías y les asigna un testigo a cada una. Los testigos de las dos últimas, el problema de correspondencia de Post y $\overline{\text{FOLTHM}}$, se citan allí sin demostración, pues en ese punto del texto no había manera de construir ninguno.

Con lo demostrado en esta sección esa deuda queda saldada: HALT es un testigo de la segunda categoría y $\overline{\text{HALT}}$ lo es de la tercera, y los dos se obtienen sin importar resultado alguno. Todo lo que se usó fue el intérprete universal de la Sección 6.4, el teorema de Post de la Sección 6.5 y el argumento diagonal de la Sección 6.7.

7.2.3. Autorreferencia. La demostración del Teorema 7.2.1 ejecuta un programa sobre su propio texto. Vale la pena insistir en que eso no tiene nada de paradójico. El texto de D es una cadena como cualquier otra y D acepta cadenas como entrada, de modo que invocarlo sobre d es una operación tan legítima como invocarlo sobre cualquier otra cosa. El programa no se ejecuta a sí mismo: recibe una descripción de sí mismo, que es cosa distinta.

Aun así, la autorreferencia puede llevarse más lejos de lo que la demostración necesita y el resultado es sorprendente.

Nota 7.2.3: Un programa que se imprime a sí mismo

Existen programas que, al ejecutarse, imprimen exactamente su propio texto. A un programa con esta propiedad se le denomina *quine*, en honor al lógico W. V. Quine. La Implementación 7.2.3 exhibe uno escrito en Python, de dos líneas.

El truco consiste en separar el programa en dos partes, una que describe al texto y otra que lo usa. La primera línea guarda en `s` una plantilla del programa completo, con un hueco marcado por `\%r`; la segunda imprime esa

plantilla llenando el hueco con la plantilla misma. Como `\%r` produce la representación de una cadena, con sus comillas y sus caracteres de escape, lo que se imprime en el hueco es exactamente la primera línea.

La existencia de quines no es una curiosidad aislada. Es el caso más simple de un fenómeno general: cualquier programa puede obtener su propio texto y operar sobre él. Ese hecho, conocido como el teorema de recursión, permite reformular la demostración del Teorema 7.2.1 sin pasar la codificación como entrada y está en la base de construcciones más sofisticadas.

Implementación 7.2.3: Un programa que se imprime a sí mismo

```
1 s = 's = \%r\nprint(s %% s)'\n
2 print(s % s)
```

Ejercicios

1. Ejecute la Implementación 7.2.3 y verifique que su salida coincide carácter por carácter con su texto. Explique qué imprimiría el programa si en la segunda línea se usara `\%s` en lugar de `\%r`.
2. Escriba un quine en el lenguaje de su preferencia, distinto de Python.
3. En la demostración del Teorema 7.2.1, la última línea del programa D retorna 1. Explique por qué el argumento funciona igual si retornara 0, y por qué no funcionaría si esa línea se reemplazara por otro ciclo infinito.
4. Considere el lenguaje

$$\text{HALT_EMPTY} = \{ \langle P \rangle \mid \text{"}P \text{ termina sobre la cadena vacía"} \}.$$

Demuestre que HALT_EMPTY es computablemente enumerable. La indecidibilidad de este problema se retoma en la Sección 7.3.

5. Sea L un lenguaje computablemente enumerable cualquiera. Demuestre que $L \leq_c \text{HALT}$. Ayuda: use el algoritmo de aceptación de L para construir, a partir de x , un programa que termine exactamente cuando $x \in L$.
6. Concluya del ejercicio anterior que HALT es, en un sentido preciso, el problema más difícil de su categoría. Enuncie ese sentido con cuidado usando la relación $\leq_c$.
7. Suponga que alguien afirma haber escrito un programa que, dado cualquier programa P escrito en Python, determina si P termina sobre la cadena vacía. Sin examinar su código, explique con precisión por qué la afirmación es falsa.

Explique también por qué esto no impide que existan herramientas útiles que detecten ciclos infinitos en muchos casos particulares.

8. La Nota 7.2.1 afirma que los programas se pueden indexar como $P_0, P_1, P_2, \dots$. Justifique esa afirmación con cuidado, indicando qué resultado del Capítulo 6 la sostiene y qué hay que hacer con las cadenas que no son programas válidos.
9. Investigue acerca del teorema de recursión mencionado en la Nota 7.2.3 y explique, sin demostrarlo, qué afirma.

7.3. Más problemas indecibles

Con la semilla de la Sección 7.2 y el transporte de la Sección 7.1, la maquinaria del capítulo está completa. Esta sección la pone a trabajar tres veces. El procedimiento es siempre el mismo y conviene enunciarlo antes de usarlo. Para demostrar que un problema L es indecible se exhibe una reducción $\text{HALT} \leq_c L$ y se invoca el Corolario 7.1.2. La dirección es la que advierte la Nota 7.1.2: se parte de HALT , que ya se sabe indecible, hacia el problema que se quiere clasificar. Toda la dificultad está en construir la función de reducción.

7.3.1. Escribir programas sin ejecutarlos. Las tres reducciones de esta sección comparten una misma construcción, así que conviene presentarla una sola vez.

Dado un par $\langle P, x \rangle$, se construye el texto de un programa nuevo Q que ignora su propia entrada, ejecuta P sobre x y, si esa ejecución termina, retorna 1. La Implementación 7.3.1 muestra ese texto.

Implementación 7.3.1: El programa Q construido a partir de $\langle P, x \rangle$

```

1 def decide(z):           # the input z is ignored
2     universal("<P, x>")    # literal written in by the reduction
3     return 1             # reached only if P halted on x

```

El comportamiento de Q depende por completo de si P termina o no al ser invocado con x y no depende en absoluto de la entrada que reciba Q :

- si P termina al ser invocado con x , entonces la invocación de `universal` termina y Q retorna 1 sobre *cualquier* entrada, de donde $L(Q) = \Sigma^*$; y
- si P no termina al ser invocado con x , entonces la invocación de `universal` tampoco termina y Q no termina sobre *ninguna* entrada, de donde $L(Q) = \emptyset$.

Ahora bien, la construcción de Q es una operación sobre texto. La reducción toma la plantilla de la Implementación 7.3.1, reemplaza el literal por la cadena $\langle P, x \rangle$ que recibió como entrada y retorna el texto resultante. Nada más.

Nota 7.3.1: La reducción escribe programas, no los ejecuta

Este es el punto que más se malinterpreta al usar la técnica, así que conviene decirlo sin rodeos. La función de reducción *nunca* ejecuta P sobre x . Se limita a escribir el texto de un programa que, si alguien llegara a ejecutarlo, haría esa invocación.

La distinción es la que hace que la reducción sea total y computable, como lo exige la Definición 7.1.2. Concatenar cadenas termina siempre, sin importar qué tan mal se comporte el programa P que aparece en ellas. Una reducción que ejecutara P para decidir qué construir heredaría la no terminación de P y dejaría de ser una función total. Dicho de otro modo, la reducción trabaja con programas en tanto texto, tal y como lo permite la doble naturaleza señalada en el Teorema 6.4.1. La ejecución queda diferida a quien use el resultado.

7.3.2. Tres problemas indecidibles. El primero pregunta por la terminación sobre una entrada fija.

Teorema 7.3.1: Terminación sobre la cadena vacía

El lenguaje

$$\text{HALT_EMPTY} = \{\langle P \rangle \mid \text{"}P \text{ termina sobre } \lambda\text{"}\}$$

es indecidible.

Demostración

Se exhibe una reducción $\text{HALT} \leq_c \text{HALT_EMPTY}$.

Sea f la función que, dada una cadena w , procede así. Si w es de la forma $\langle P, x \rangle$ con P un programa, retorna $\langle Q \rangle$, donde Q es el programa de la Implementación 7.3.1 construido con ese par. En caso contrario, retorna la codificación de un programa fijo que no termina sobre ninguna entrada.

La función f es total y computable, pues construir el texto de Q consiste en reemplazar un literal dentro de una plantilla, tal y como lo señala la Nota 7.3.1.

Resta verificar la condición de la Definición 7.1.2. Sea $w \in \Sigma^*$.

Si $w = \langle P, x \rangle \in \text{Halt}$: entonces P termina al ser invocado con x , de modo que Q termina sobre cualquier entrada y, en particular, sobre λ . Luego $f(w) = \langle Q \rangle \in \text{HALT_EMPTY}$.

Si $w = \langle P, x \rangle \notin \text{HALT}$: entonces P no termina al ser invocado con x , de modo que Q no termina sobre ninguna entrada y, en particular, no termina sobre λ . Luego $f(w) = \langle Q \rangle \notin \text{HALTEMPTY}$.

Si w no es de la forma $\langle P, x \rangle$: entonces $w \notin \text{HALT}$ y $f(w)$ es la codificación de un programa que no termina sobre ninguna entrada, de donde $f(w) \notin \text{HALTEMPTY}$.

En conclusión, $\text{HALT} \leq_c \text{HALTEMPTY}$ y, por el Corolario 7.1.2 junto con el Teorema 7.2.1, el lenguaje HALTEMPTY es indecible.

El segundo no pregunta por la terminación sino por lo que el programa acepta.

Teorema 7.3.2: Lenguaje aceptado no vacío

El lenguaje

$$\text{NOTEMPTY} = \{\langle P \rangle \mid L(P) \neq \emptyset\}$$

es indecible, donde $L(P)$ denota el lenguaje aceptado por la función que P calcula.

Demostración

Se usa la misma función f de la demostración del Teorema 7.3.1, que es total y computable por la misma razón.

Si $w = \langle P, x \rangle \in \text{HALT}$, entonces $L(Q) = \Sigma^*$, que no es vacío, de donde $f(w) \in \text{NOTEMPTY}$. Si $w = \langle P, x \rangle \notin \text{HALT}$, entonces $L(Q) = \emptyset$, de donde $f(w) \notin \text{NOTEMPTY}$. Y si w no es de la forma $\langle P, x \rangle$, entonces $f(w)$ codifica un programa que no acepta nada, de donde $f(w) \notin \text{NOTEMPTY}$.

Luego $\text{HALT} \leq_c \text{NOTEMPTY}$ y el Corolario 7.1.2 da el resultado.

El tercero es de otra naturaleza, pues no pregunta por un programa sino por la relación entre dos.

Teorema 7.3.3: Equivalencia de programas

El lenguaje

$$\text{EQUIV} = \{\langle P_1, P_2 \rangle \mid L(P_1) = L(P_2)\}$$

es indecible.

Demostración

Se exhibe una reducción $\text{HALT} \leq_c \text{EQUIV}$.

Sea R el programa fijo que retorna 1 sobre cualquier entrada, de manera que $L(R) = \Sigma^*$. Sea g la función que, dada $w = \langle P, x \rangle$, retorna $\langle Q, R \rangle$ con Q el programa de la Implementación 7.3.1; y que, sobre cualquier otra cadena, retorna $\langle Q_0, R \rangle$ con Q_0 un programa fijo que no termina sobre ninguna entrada.

La función g es total y computable por la misma razón que f : el programa R es fijo y construir Q es reemplazar un literal.

Si $w \in \text{HALT}$, entonces $L(Q) = \Sigma^* = L(R)$ y por lo tanto $g(w) \in \text{EQUIV}$.

Si $w \notin \text{HALT}$ hay dos posibilidades. Cuando w es de la forma $\langle P, x \rangle$ se tiene $L(Q) = \emptyset$, y cuando w está mal formada se tiene $L(Q_0) = \emptyset$. En cualquiera de los dos casos el primer componente acepta el lenguaje vacío, que es distinto de $L(R) = \Sigma^*$, de donde $g(w) \notin \text{EQUIV}$.

Luego $\text{HALT} \leq_c \text{EQUIV}$ y el Corolario 7.1.2 da el resultado.

Nota 7.3.2: La receta

Las tres demostraciones anteriores siguen el mismo molde y vale la pena extraerlo, porque es el que se usa cada vez que se quiere clasificar un problema nuevo.

1. Se parte de $\langle P, x \rangle$, una instancia arbitraria de HALT .
2. Se *escribe* el texto de uno o más programas cuyo comportamiento dependa de si P termina sobre x , de manera que la propiedad que se está estudiando se cumpla exactamente en uno de los dos casos.
3. Se verifica que esa escritura es una operación que siempre termina, con lo cual la reducción es total y computable.
4. Se verifica la equivalencia en las dos direcciones, sin olvidar las cadenas mal formadas.

El paso interesante es el segundo y es donde está todo el ingenio. Los pasos restantes son verificaciones rutinarias que, sin embargo, no conviene saltarse: omitir el tercero es el error que produce “reducciones” que no son funciones totales.

7.3.3. Una observación sobre los tres resultados. Los dos primeros problemas de esta sección tienen algo en común que el tercero no comparte y esa diferencia se vuelve importante en la Sección 7.4.

Tanto HALTEMPTY como NOTEMPTY preguntan por una propiedad de *un* programa, y además por una propiedad de lo que el programa *hace* y no de cómo está escrito. Dos programas que calculen la misma función pertenecen ambos a HALTEMPTY o ninguno de los dos pertenece, sin importar qué tan distintos sean sus textos. Lo mismo vale para NOTEMPTY.

EQUIV no es de ese tipo. No es una propiedad de un programa sino una relación entre dos, de modo que la pregunta ni siquiera se formula en los mismos términos. La Sección 7.4 demuestra un teorema que cubre de un solo golpe todos los problemas del primer tipo, y por eso hace innecesarias las demostraciones de los teoremas 7.3.1 y 7.3.2. Al Teorema 7.3.3 no lo cubre de la misma manera, aunque tampoco lo deja por fuera: su indecidibilidad se obtiene de aquel teorema mediante una reducción corta, que allá se propone como ejercicio.

Eso no vuelve inútil ninguna de las tres demostraciones de esta sección. Hacer tres reducciones a mano antes de ver el teorema que las explica es la manera de entender qué es lo que el teorema está automatizando.

Ejercicios

1. En la demostración del Teorema 7.3.1 se atiende por separado el caso de las cadenas mal formadas. Explique qué sale mal si se omite ese caso y por qué no basta con decir que “en la práctica no ocurre”.
2. Considere la siguiente reducción propuesta para el Teorema 7.3.1: dada $\langle P, x \rangle$, ejecutar P sobre x y retornar la codificación de un programa que termina de inmediato si esa ejecución terminó, o la de uno que no termina en caso contrario. Explique con precisión por qué no es una reducción.
3. Demuestre que HALTEMPTY es computablemente enumerable. Concluya, usando el Corolario 6.5.4, que $\overline{\text{HALTEMPTY}}$ no lo es.
4. Demuestre que NOTEMPTY es computablemente enumerable. Ayuda: hay que examinar todas las cadenas y todos los presupuestos de pasos, de modo que la Nota 6.5.1 y la Implementación 6.6.1 son las herramientas indicadas.
5. Considere $\text{EMPTY} = \{ \langle P \rangle \mid L(P) = \emptyset \}$, el complemento de NOTEMPTY restringido a las codificaciones de programas. Demuestre que es indecible sin construir una reducción nueva.
6. Demuestre que el lenguaje $\{ \langle P \rangle \mid “P \text{ termina sobre toda entrada}” \}$ es indecible.

7. Demuestre que el lenguaje $\{\langle P, x \rangle \mid \text{"P ejecuta a lo sumo 100 pasos sobre } x\text{"}\}$ es decidable. Compare con el Teorema 7.2.1 y explique por qué no hay contradicción.
8. En la demostración del Teorema 7.3.3 se escogió R como el programa que acepta todas las cadenas. Demuestre que la reducción también funciona si R se escoge como el programa que no acepta ninguna, intercambiando el papel de los dos casos.
9. Sea $\text{EQUIVEMPTY} = \{\langle P_1, P_2 \rangle \mid L(P_1) = L(P_2) = \emptyset\}$. Determine si es decidable y demuestre su respuesta.
10. Un compilador que optimiza código quisiera reemplazar un fragmento de programa por otro más eficiente que haga lo mismo. Explique qué dice el Teorema 7.3.3 acerca de esa aspiración y por qué, a pesar de ello, los compiladores optimizantes existen y funcionan.

7.4. El teorema de Rice

La Sección 7.3 termina señalando que dos de sus tres problemas comparten una forma y que el tercero no. Esta sección demuestra el teorema que explica esa forma. El enunciado es de una generalidad inusual: afirma que *ninguna* propiedad interesante de lo que hace un programa es decidable. No es que las conocidas hasta ahora resulten serlo; es que no hay ninguna que lo sea.

Antes de poder enunciarlo hay que precisar qué es una propiedad de lo que hace un programa, por oposición a una propiedad de cómo está escrito.

7.4.1. Propiedades semánticas. Cada programa P calcula una función A_P , posiblemente parcial, en el sentido de la Definición 6.4.1. Programas distintos pueden calcular la misma función: basta cambiar el nombre de una variable o agregar una línea que no haga nada.

Definición 7.4.1: Propiedad semántica

Una *propiedad semántica* es una colección $\mathcal{S}$ de funciones computables. El *problema asociado* a $\mathcal{S}$ es el lenguaje:

$$L_{\mathcal{S}} = \{\langle P \rangle \mid A_P \in \mathcal{S}\}.$$

Se dice que $\mathcal{S}$ es *trivial* si y solo si $\mathcal{S}$ es vacía o si $\mathcal{S}$ contiene a todas las funciones computables.

Lo que la Definición 7.4.1 captura es que la pertenencia de $\langle P \rangle$ a $L_{\mathcal{S}}$ depende únicamente de la función que P calcula. Si dos programas calculan la misma función,

entonces o los dos están en L_S o ninguno lo está, sin importar cuán distintos sean sus textos. Esa es exactamente la diferencia entre una propiedad semántica y una sintáctica.

Ejemplo 7.4.1

Las siguientes son propiedades semánticas no triviales:

- “ A_P está definida sobre λ ”, que corresponde al HALTEEMPTY del Teorema 7.3.1;
- “ A_P vale 1 en al menos una cadena”, que corresponde al NOTEMPTY del Teorema 7.3.2;
- “ A_P es total”, es decir, que P termina sobre toda entrada; y
- “ A_P coincide con la función que calcula cierto programa fijo R ”.

Las siguientes, en cambio, *no* son propiedades semánticas:

- “el texto de P tiene más de cien líneas”;
- “ P contiene al menos un ciclo **while**”; y
- “ P termina en menos de mil pasos sobre λ ”.

Las tres últimas dependen de cómo está escrito P y no solo de lo que calcula. Note que las tres son, además, decidibles: basta examinar el texto en los dos primeros casos y simular mil pasos en el tercero, usando la Nota 6.5.1.

Nota 7.4.1: Una convención sobre la codificación

Para lo que sigue conviene acordar que *toda* cadena codifica algún programa. Las cadenas que no corresponden a un programa bien formado se toman como codificación de un programa fijo que no termina sobre ninguna entrada.

La convención es inocua y evita un caso especial en las demostraciones. Su efecto es que L_S y el problema asociado a la colección complementaria de S son complementos exactos el uno del otro, cosa que no sería cierta si hubiera cadenas que no pertenecieran ni a uno ni a otro.

7.4.2. El teorema.

Teorema 7.4.1: Teorema de Rice

Sea S una propiedad semántica no trivial. Entonces L_S es indecidible.

Antes de la demostración conviene anticipar la idea, que reutiliza la construcción de la Sección 7.3 con un solo cambio.

Allá el programa Q ejecutaba P sobre x y, si esa ejecución terminaba, retornaba 1. Acá hará lo mismo, salvo que en lugar de retornar 1 se comportará como un programa G escogido de antemano entre los que sí tienen la propiedad. El efecto es que Q calcula la función de G cuando P termina sobre x y no calcula nada en caso contrario. Como una de esas dos funciones tiene la propiedad y la otra no, decidir si Q la tiene equivaldría a decidir si P termina.

Demostración

Sea $\perp$ la función que no está definida sobre ninguna cadena, es decir, la que calcula un programa que nunca termina. Se distinguen dos casos según si $\perp$ pertenece o no a $\mathcal{S}$.

Caso $\perp \notin \mathcal{S}$: Como $\mathcal{S}$ no es vacía, existe una función $g \in \mathcal{S}$; sea G un programa que la calcula. Note que $g \neq \perp$, pues $\perp \notin \mathcal{S}$.

Sea f la función que, dada $w = \langle P, x \rangle$, retorna $\langle Q \rangle$ con Q el programa de la Implementación 7.4.1, construido reemplazando en la plantilla el literal por w y el texto de G donde corresponde. Sobre cualquier otra cadena, f retorna la codificación de un programa fijo que no termina.

La función f es total y computable, pues construir el texto de Q es reemplazar dos fragmentos dentro de una plantilla; vale aquí todo lo dicho en la Nota 7.3.1.

Resta verificar la condición de la Definición 7.1.2. Si $w = \langle P, x \rangle \in \text{HALT}$, entonces la invocación de `universal` termina para cualquier entrada z , de modo que Q se comporta como G y por lo tanto $A_Q = g \in \mathcal{S}$; en consecuencia $f(w) \in L_{\mathcal{S}}$. Si $w = \langle P, x \rangle \notin \text{HALT}$, entonces la invocación de `universal` no termina para ninguna entrada z , de modo que Q tampoco termina sobre ninguna entrada y $A_Q = \perp \notin \mathcal{S}$; y si w está mal formada, entonces $f(w)$ no es $\langle Q \rangle$ sino la codificación del programa fijo que no termina, cuya función es también $\perp$. En los dos casos $f(w) \notin L_{\mathcal{S}}$.

Luego $\text{HALT} \leq_c L_{\mathcal{S}}$ y, por el Corolario 7.1.2, el lenguaje $L_{\mathcal{S}}$ es indecidible.

Caso $\perp \in \mathcal{S}$: Sea $\overline{\mathcal{S}}$ la colección de las funciones computables que no están en $\mathcal{S}$. Esa colección tampoco es trivial. No es vacía, pues $\mathcal{S}$ no contiene a todas las funciones computables. No las contiene a todas, pues $\mathcal{S}$ no es vacía. Además, $\perp \notin \overline{\mathcal{S}}$.

Por el caso anterior, $L_{\overline{\mathcal{S}}}$ es indecidible. Y por la Nota 7.4.1 se tiene $L_{\overline{\mathcal{S}}} = \overline{L_{\mathcal{S}}}$. Si $L_{\mathcal{S}}$ fuera decidable, entonces $\overline{L_{\mathcal{S}}}$ también lo sería por la parte (1) del Teorema 6.5.1, lo cual es una contradicción.

Implementación 7.4.1: El programa Q de la demostración

```

1 def decide(z):
2     universal("<P, x>")      # literal written in by the reduction
3     return G(z)             # the text of G is written in as well

```

7.4.3. Lo que el teorema dice y lo que no. La generalidad del Teorema 7.4.1 vuelve innecesarias dos de las tres demostraciones de la Sección 7.3 y muchas otras que habría que hacer una por una.

Corolario 7.4.2

Los siguientes lenguajes son indecidibles:

1. $\{\langle P \rangle \mid \text{"}P \text{ termina sobre toda entrada"}\}$;
2. $\{\langle P \rangle \mid \text{"}P \text{ no termina sobre ninguna entrada"}\}$;
3. $\{\langle P \rangle \mid L(P) \text{ es finito}\}$; y
4. $\{\langle P \rangle \mid A_P = A_R\}$, con R un programa fijo.

Demostración

En los cuatro casos se trata de una propiedad de la función que calcula el programa y no de su texto, de modo que la Definición 7.4.1 aplica. Resta verificar que las cuatro colecciones son no triviales, y eso se hace exhibiendo en cada caso una función computable que la cumple y otra que no. Para la primera, por ejemplo, la función constante 1 es total y $\perp$ no lo es. Los demás casos son análogos y se proponen como ejercicio al lector.

La cuarta parte del Corolario 7.4.2 merece un comentario, porque se parece al Teorema 7.3.3 sin serlo. Aquí el programa R está fijo de antemano y la pregunta es sobre un solo programa, de manera que hay una propiedad semántica de por medio y el teorema aplica. En el Teorema 7.3.3 los dos programas son parte de la entrada y ninguna colección $\mathcal{S}$ corresponde a esa pregunta, porque $L_{\mathcal{S}}$ es siempre un conjunto de códigos de *un* programa.

Eso no significa que EQUIV quede fuera del alcance del teorema, sino que no es un caso particular suyo. Su indecidibilidad se obtiene del Teorema 7.4.1 mediante una reducción, y una muy corta, que se propone en el Ejercicio 8.

Conviene entonces distinguir tres situaciones, porque la diferencia entre ellas se pierde con facilidad.

Aplicación directa: La pregunta es “¿está A_P en $\mathcal{S}$?” para alguna colección no trivial $\mathcal{S}$. El teorema la resuelve sin más trámite y las cuatro partes del Corolario 7.4.2 son de este tipo.

Consecuencia por reducción: La pregunta es semántica pero no tiene la forma L_S , casi siempre porque involucra más de un programa de entrada. El teorema no aplica directamente, pero sirve como punto de partida de una reducción. EQUIV es el ejemplo.

Fuera del alcance: La pregunta no depende únicamente de la función calculada. El teorema no dice nada, ni a favor ni en contra, y hay que estudiar cada caso por separado: unos resultan decidibles, como los del Ejemplo 7.4.1, y otros indecidibles por una reducción desde HALT que no pasa por este teorema.

Nota 7.4.2: Lo que el teorema no dice

El Teorema 7.4.1 se presta a tres lecturas exageradas que conviene descartar.

No dice que todo sea indecible. Toda pregunta que dependa de algo más que la función calculada escapa a su alcance, como lo muestra el Ejemplo 7.4.1. Un analizador puede determinar sin dificultad cuántas líneas tiene un programa o qué funciones invoca, que son preguntas sobre el texto, y también si termina dentro de un presupuesto fijo de pasos, que no es una pregunta sobre el texto sino sobre una ejecución acotada y se responde simulando. Buena parte del trabajo que hacen los compiladores y las herramientas de análisis vive precisamente en ese territorio.

No dice que las triviales lo sean. La hipótesis de no trivialidad es esencial y no es un tecnicismo. La propiedad “ A_P es computable” la cumplen todas las funciones computables, de modo que el problema asociado se decide respondiendo siempre que sí. Lo que el teorema afirma es que apenas una propiedad distingue entre unos programas y otros, decidirla se vuelve imposible.

No dice que no se pueda hacer nada útil. Un resultado de indecidibilidad descarta un procedimiento que responda correctamente sobre *toda* entrada, tal y como lo advierte la Nota 6.7.5. No descarta procedimientos que respondan correctamente sobre muchas entradas, ni procedimientos que respondan “no sé” cuando no logran decidir. Un detector de ciclos infinitos que reporte algunos y calle sobre el resto es perfectamente posible y perfectamente útil; lo que no existe es el que los reporte todos sin equivocarse nunca.

Nota 7.4.3: La lectura para quien programa

Vale la pena traducir el Teorema 7.4.1 al oficio. Ninguna propiedad semántica no trivial de la función que calcula un programa se puede decidir sobre programas arbitrarios exigiendo una respuesta correcta siempre. Dicho de otro modo: si la pregunta depende únicamente de lo que el programa calcula y distingue entre unos programas y otros, no hay herramienta que la responda.

Eso cubre, por ejemplo, decidir si un programa termina sobre toda entrada o si el resultado de una optimización hace lo mismo que un programa de referencia fijo.

Buena parte de lo que uno quisiera automatizar queda cerca de ahí sin quedar exactamente ahí, y conviene saber por qué. Decidir si dos implementaciones de una misma especificación coinciden, o si una optimización preserva el comportamiento cuando ambos programas son entrada, es una pregunta semántica entre dos programas: es el caso de EQUIV y se resuelve por reducción, no por aplicación directa. Decidir si un programa accede a una posición inválida de un arreglo o si un fragmento de código es alcanzable no es siquiera una pregunta sobre la función calculada, pues dos programas que calculan lo mismo pueden diferir en ambas cosas; esos problemas son indecibles, pero el resultado exige una reducción propia desde HALT y no se sigue de este teorema.

La conclusión práctica es la misma en los tres casos y no es abandonar el intento sino cambiar la pregunta. La verificación de programas, el análisis estático y las pruebas automáticas conviven con este teorema restringiendo el dominio a programas de cierta forma, aceptando respuestas aproximadas o admitiendo la posibilidad de no responder. Ninguna de esas tres salidas contradice al teorema y las tres son la razón por la cual el área existe.

Ejercicios

1. Complete la demostración del Corolario 7.4.2 exhibiendo, para cada una de las cuatro colecciones, una función computable que la cumple y otra que no.
2. Determine cuáles de las siguientes son propiedades semánticas y justifique: “ P tiene exactamente tres variables”; “ A_P está definida sobre al menos diez cadenas”; “ P es el programa más corto que calcula A_P ”; “ A_P es la función constante 0”.
3. En la demostración del Teorema 7.4.1 se escoge $g \in \mathcal{S}$ arbitraria. Explique por qué el argumento no funcionaría si se escogiera $g = \perp$ y en qué punto exactamente se usa que $\perp \notin \mathcal{S}$.
4. Explique por qué el caso $\perp \in \mathcal{S}$ no se puede tratar directamente con la misma construcción e identifique qué paso de la verificación fallaría.
5. Sea $\mathcal{S}$ la colección de todas las funciones computables. Demuestre que $L_{\mathcal{S}} = \Sigma^*$ y que, por lo tanto, es decidable. Haga lo propio con $\mathcal{S} = \emptyset$. Concluya que la hipótesis de no trivialidad no se puede quitar.
6. Considere el lenguaje $\{\langle P \rangle \mid \text{“}P \text{ termina en menos de mil pasos sobre } \lambda\text{”}\}$. Demuestre que es decidable y explique por qué esto no contradice al Teorema 7.4.1.
7. Un programador propone la siguiente herramienta: dado un programa, reporta “puede colgarse” si encuentra algún ciclo cuya condición de salida no logra analizar y “no se cuelga” en caso contrario. Explique en qué sentido esta herramienta es correcta, en qué sentido no lo es y por qué su existencia no contradice al Teorema 7.4.1.
8. Sobre EQUIV y el Teorema 7.4.1:
 - a) Demuestre que EQUIV no se puede obtener como caso particular del teorema, precisando qué habría que escoger como $\mathcal{S}$ y por qué no se puede.
 - b) Demuestre, sin embargo, que su indecidibilidad sí se deduce del teorema. Ayuda: verifique primero que $\text{EMPTY} = \{\langle P \rangle \mid L(P) = \emptyset\}$ es el problema asociado a una propiedad semántica no trivial; fije después un programa R_0 que no termine sobre ninguna entrada y considere el mapa $\langle P \rangle \mapsto \langle P, R_0 \rangle$.
 - c) Explique por qué el literal anterior no necesita tratar las cadenas mal formadas por separado, a diferencia de las reducciones de la Sección 7.3. Ayuda: revise la Nota 7.4.1.
9. Investigue acerca de H. G. Rice y del contexto en que publicó este resultado.
10. El Teorema 7.4.1 habla de propiedades de funciones. Enuncie y demuestre la versión correspondiente para propiedades del lenguaje aceptado, es decir, para colecciones de lenguajes computablemente enumerables. Indique cuál de los problemas de la Sección 7.3 queda cubierto por esa versión y cuál no.

7.5. Dominós: el problema de correspondencia de Post

Todo lo que va del capítulo trata sobre programas. La semilla pregunta si un programa termina, las reducciones construyen programas y el teorema de Rice habla de propiedades de programas. Es razonable sospechar que la indecidibilidad sea un fenómeno propio de los objetos que se autorreferencian y que un problema formulado sobre objetos más inocentes no pueda padecerla.

Esta sección desmiente esa sospecha con un rompecabezas de fichas en el cual no aparece programa alguno y que, sin embargo, resulta ser indecidible.

7.5.1. El juego. Una ficha es un dominó con dos cadenas escritas, una arriba y otra abajo. Se dispone de una colección finita de tipos de ficha y de una cantidad ilimitada de copias de cada tipo. El juego consiste en colocar fichas una tras otra, en el orden que se quiera y repitiendo los tipos que haga falta, de manera que al final la cadena que se lee arriba coincida con la que se lee abajo.

Ejemplo 7.5.1: Una colección con solución

Considere las tres fichas siguientes:

<table><tr><td>1</td></tr><tr><td>111</td></tr></table>	1	111	<table><tr><td>10111</td></tr><tr><td>10</td></tr></table>	10111	10	<table><tr><td>10</td></tr><tr><td>0</td></tr></table>	10	0
1								
111								
10111								
10								
10								
0								
ficha 1	ficha 2	ficha 3						

La secuencia 2, 1, 1, 3 resuelve el juego. Colocando esas cuatro fichas en ese orden, arriba se lee

$$10111 \cdot 1 \cdot 1 \cdot 10 = 101111110$$

y abajo se lee

$$10 \cdot 111 \cdot 111 \cdot 0 = 101111110,$$

que es la misma cadena. Note que la ficha 1 se usó dos veces y que la cantidad de fichas usadas no tiene por qué coincidir con la cantidad de tipos disponibles.

La Figura 2 muestra la secuencia colocada.

Ejemplo 7.5.2: Una colección sin solución

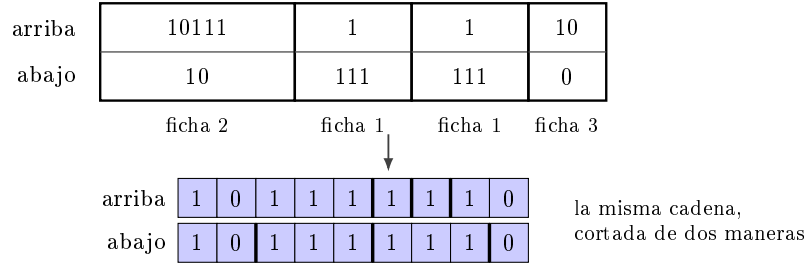
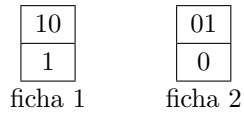


Figura 2. La secuencia 2, 1, 1, 3 que resuelve el Ejemplo 7.5.1. Abajo aparece la cadena resultante, que es la misma leída arriba y leída abajo. Las divisiones marcadas indican dónde termina el aporte de cada ficha: la solución existe porque una misma cadena admite los dos cortes.

Considere ahora estas dos fichas:



Ninguna secuencia resuelve el juego y la razón se ve de inmediato. En las dos fichas la cadena de arriba tiene longitud 2 y la de abajo tiene longitud 1. Cualquier secuencia de k fichas produce entonces una cadena de longitud $2k$ arriba y una de longitud k abajo, y estas no pueden coincidir para $k \geq 1$.

La formulación precisa es la siguiente.

Definición 7.5.1: Problema de correspondencia de Post

Una *instancia* del problema de correspondencia de Post es una secuencia finita de pares de cadenas no vacías

$$(u_1, v_1), (u_2, v_2), \dots, (u_n, v_n),$$

con $n \geq 1$. Una *solución* de esa instancia es una secuencia finita y no vacía de índices $i_1, i_2, \dots, i_k$, con $1 \leq i_j \leq n$ y repeticiones permitidas, tal que:

$$u_{i_1} u_{i_2} \cdots u_{i_k} = v_{i_1} v_{i_2} \cdots v_{i_k}.$$

El *problema de correspondencia de Post* es el lenguaje:

$$\text{PCP} = \{ \langle I \rangle \mid \text{"I es una instancia que tiene solución"} \}.$$

La exigencia de que la secuencia de índices sea no vacía no es un capricho. Sin ella, la secuencia vacía resolvería toda instancia, pues concatenar cero cadenas produce λ tanto arriba como abajo y el problema sería trivial.

7.5.2. Buscar la solución. Encontrar una solución es cuestión de buscarla. Las secuencias de índices se pueden generar una tras otra, primero las de longitud 1, luego las de longitud 2 y así sucesivamente; para cada una se concatena y se compara. Es la misma idea de la enumeración canónica de la Sección 6.6, aplicada ahora a secuencias de índices en lugar de cadenas.

Teorema 7.5.1

PCP es computablemente enumerable.

Demostración

La Implementación 7.5.1 recorre las secuencias de índices por longitud creciente y retorna la primera que resuelve la instancia. Ese programa es el solucionador y no el aceptador: recibe una instancia ya decodificada y retorna una secuencia de índices, no un bit. El aceptador es el de la Implementación 7.5.2, que lo envuelve; sea A la función que calcula.

Sea $w \in \Sigma^*$ y considere los tres casos posibles.

Si $w = \langle I \rangle \in \text{PCP}$, entonces I tiene alguna solución; sea k la longitud de la más corta. El solucionador la encuentra al llegar a las secuencias de longitud k , pues examina todas las de esa longitud, de modo que la invocación termina y $A(w) = 1$.

Si $w = \langle I \rangle \notin \text{PCP}$, entonces ninguna secuencia resuelve la instancia y el ciclo externo del solucionador no termina jamás. Luego $A(w)$ está indeterminada y, en particular, no vale 1.

Si w está mal formada, entonces $w \notin \text{PCP}$ y la validación hace que el aceptador retorne 0, de modo que $A(w)$ tampoco vale 1.

En conclusión $L(A) = \text{PCP}$.

Implementación 7.5.1: Búsqueda de una solución

```

1 def solve(tiles):
2     seqs = [[i] for i in range(len(tiles))]
3     ans = False
4     while ans == False:
5         k = 0
6         while ans == False and k != len(seqs):
7             top = "".join(tiles[i][0] for i in seqs[k])

```

```

8     bot = "".join(tiles[i][1] for i in seqs[k])
9     if top == bot:
10        ans = seqs[k]
11        k = k + 1
12    if ans == False:
13        seqs = [s + [i] for s in seqs for i in range(len(tiles))]
14    return ans

```

Implementación 7.5.2: Un algoritmo de aceptación para Pcp

```

1 def decide(w):
2     ans = 0
3     if valid(w):                # does w encode a PCP instance?
4         solve(decode(w))        # may never return
5         ans = 1                 # reached only if solve returned
6     return ans

```

Conviene separar los dos programas porque tienen oficios distintos. La Implementación 7.5.1 resuelve el juego y es donde está todo el contenido; la Implementación 7.5.2 se limita a traducir entre cadenas y bits, que es lo que exige la Definición 6.3.3. Es la misma división de trabajo de la Implementación 7.2.2 y, como allá, la validación inicial es lo que impide aceptar cadenas que no codifican instancia alguna.

La variable `ans` del solucionador arranca en `False` y solo cambia de valor cuando se encuentra una solución, de manera que sirve a la vez como respuesta y como condición de los dos ciclos. Note que ese centinela funciona porque una solución nunca es la secuencia vacía, tal y como lo exige la Definición 7.5.1; de lo contrario, el valor de `ans` no permitiría distinguir entre haber encontrado una solución y no haberla encontrado todavía.

Vale la pena ejecutar la Implementación 7.5.1 sobre las dos colecciones de esta sección. Sobre la del Ejemplo 7.5.1 responde de inmediato con la secuencia esperada. Sobre la del Ejemplo 7.5.2 no responde nunca: el ciclo externo sigue generando secuencias cada vez más largas sin encontrar ninguna que sirva.

Nota 7.5.1: Semidecidible no es lo mismo que difícil

El contraste entre los dos ejemplos merece atención, porque revela algo que suele confundirse.

Que el programa no termine sobre el Ejemplo 7.5.2 no se debe a que esa instancia sea difícil. Al contrario: el lector la resolvió en dos renglones contando longitudes. El programa no termina porque busca soluciones y la ausencia de solución no es algo que se encuentre buscando.

Esa es, otra vez, la asimetría de la Nota 6.5.3 y la misma que aparece en HALT. La respuesta afirmativa la certifica un objeto finito, la secuencia de índices, que se puede exhibir y verificar. La respuesta negativa no tiene ningún certificado de esa clase.

7.5.3. La indecidibilidad. El resultado que le da su lugar a este problema es el siguiente.

Teorema 7.5.2: Indecidibilidad de Pcp

PCP es indecidible.

La demostración es de E. Post y se encuentra en [Pos46]. Su estructura es la habitual del capítulo, una reducción desde un problema ya clasificado, pero la construcción es considerablemente más elaborada que las de la Sección 7.3 y por eso se desarrolla por etapas en los ejercicios de esta sección.

La idea se puede describir en pocas palabras. Una ejecución de un programa se puede escribir como una secuencia de configuraciones, cada una describiendo el estado completo del cómputo en un instante. A esa secuencia se le denomina la *historia* de la ejecución. La reducción construye, a partir de un programa y una entrada, una colección de fichas diseñada de manera que la única forma de hacer coincidir la cadena de arriba con la de abajo sea colocarlas en el orden que deletrea la historia de la ejecución. Las fichas obligan a que la cadena de abajo vaya siempre una configuración adelante de la de arriba, de modo que ambas solo se emparejan cuando la ejecución termina.

Corolario 7.5.3

$\overline{\text{PCP}}$ no es computablemente enumerable.

Demostración

Por el Teorema 7.5.1 el lenguaje PCP es computablemente enumerable y por el Teorema 7.5.2 no es computable. El Corolario 6.5.4 da el resultado.

Nota 7.5.2: ¿Por qué importa este problema?

Hasta esta sección, todos los problemas indecidibles del capítulo hablaban de programas. Eso podía sugerir que la indecidibilidad es un accidente de la autorreferencia y que basta con evitar hablar de programas para escapar de ella.

PCP muestra que no. En su enunciado no hay programas, ni ejecución, ni autorreferencia: hay fichas con cadenas escritas y una pregunta sobre concatenaciones. La indecidibilidad no es propiedad del vocabulario en que se formula un problema sino de la dificultad que el problema encierra.

Por esa razón PCP se ha vuelto una herramienta de trabajo. Es mucho más cómodo reducir desde fichas que desde programas cuando el problema de llegada también es combinatorio y buena parte de los resultados de indecidibilidad en teoría de lenguajes formales y en reescritura se demuestran así. Los de la Sección 7.6 están entre ellos.

Ejercicios

1. Determine si la colección de fichas $\begin{array}{|c|} \hline 1 \\ \hline 11 \\ \hline \end{array}, \begin{array}{|c|} \hline 11 \\ \hline 1 \\ \hline \end{array}, \begin{array}{|c|} \hline 0 \\ \hline 01 \\ \hline \end{array}$ tiene solución. Si la tiene, exhibala; si no, justifique por qué no.
2. Codifique la Implementación 7.5.1 y ejecútela sobre las colecciones de los ejemplos 7.5.1 y 7.5.2. Describa lo que observa en cada caso y relaciónelo con la Nota 7.5.1.
3. La Implementación 7.5.1 guarda en `seqs` todas las secuencias de una misma longitud, cuya cantidad crece de manera exponencial. Proponga una variante que use memoria acotada y explique qué se sacrifica a cambio.
4. Demuestre que si todas las fichas de una instancia cumplen $|u_i| > |v_i|$, entonces la instancia no tiene solución. Generalice el argumento del Ejemplo 7.5.2.
5. Demuestre que el problema de determinar si una instancia tiene solución *de longitud a lo sumo k*, con k dado como parte de la entrada, es decidable. Explique por qué esto no contradice al Teorema 7.5.2.
6. Considere el problema de correspondencia de Post *modificado*, denotado MPCP, que es idéntico a PCP salvo que la solución debe comenzar con la ficha 1. Demuestre que $\text{MPCP} \leq_c \text{PCP}$. Ayuda: introduzca dos símbolos nuevos que no aparezcan en las cadenas de la instancia y úselos para marcar el comienzo y el final, de manera que solo la ficha correspondiente a la número 1 pueda arrancar la construcción.

7. Escriba la historia de la ejecución de un programa sencillo de su elección sobre una entrada pequeña, como una secuencia de configuraciones. Decida primero qué información debe llevar una configuración para que la siguiente quede determinada.
8. Con base en los dos ejercicios anteriores, describa cómo se construirían las fichas de una instancia de MPCP a partir de un programa P y una entrada x , de manera que la instancia tenga solución si y solo si P termina al ser invocado con x . Identifique qué tipos de ficha hacen falta: la que arranca, las que copian símbolos sin cambio, las que ejecutan un paso y las que cierran al terminar.
9. Junte los ejercicios anteriores en una demostración completa del Teorema 7.5.2. Compare luego su construcción con la original de [Pos46].
10. Investigue acerca del problema de correspondencia de Post *acotado*, en el cual se fija de antemano la cantidad de fichas disponibles. ¿Para qué cantidades se sabe que es decidible y para cuáles se sabe que no?

7.6. Reescritura

Las secciones anteriores demuestran que ciertos problemas no tienen solución algorítmica. Esta cierra el capítulo desde el otro lado: muestra dónde está la frontera, presentando dos propiedades que no se pueden decidir junto a una tercera, muy cercana a ellas, que sí se puede y para la cual se exhibe el algoritmo. El terreno es el de los sistemas de reescritura, que formalizan la idea de transformar objetos aplicando reglas. Es un marco con aplicaciones directas en compiladores, demostración automática y lenguajes de especificación. Es también uno de los modelos de cómputo mencionados en las notas del Capítulo 6.

7.6.1. Sistemas de reescritura de cadenas. Los sistemas de reescritura de cadenas (en inglés, *string rewriting systems*) son la versión más austera de esa idea. El objeto que se transforma es una cadena y una regla dice qué subcadena puede reemplazarse por cuál: no hay variables, ni tipos, ni estructura de término. La austeridad es deliberada, pues las dos preguntas que ocupan la sección, si el proceso siempre acaba y si el resultado es único, ya resultan indecidibles en este marco mínimo y lo siguen siendo en los marcos más ricos que lo extienden, como la reescritura de términos.

Definición 7.6.1: Sistema de reescritura de cadenas

Un *sistema de reescritura de cadenas* sobre un alfabeto Σ es un conjunto finito R de *reglas*, cada una un par de cadenas que se escribe $l \rightarrow r$ con $l, r \in \Sigma^*$ y $l \neq \lambda$.

La *relación de reescritura* inducida por R , denotada $\rightarrow_R$, se define así: $u \rightarrow_R v$ si y solo si existen $x, y \in \Sigma^*$ y una regla $l \rightarrow r$ en R tales que $u = xly$ y $v = xry$.

Se escribe $\rightarrow_R^*$ para la cerradura reflexiva y transitiva de $\rightarrow_R$. Una cadena u es una *forma normal* si no existe v tal que $u \rightarrow_R v$.

En palabras, un paso de reescritura consiste en localizar dentro de la cadena una ocurrencia del lado izquierdo de alguna regla y reemplazarla por el lado derecho, dejando intacto lo que hay a su alrededor. Cuando la regla se puede aplicar en varias posiciones o cuando varias reglas son aplicables, hay más de un paso posible.

Ejemplo 7.6.1

Sobre el alfabeto $\{a, b\}$, considere $R_1 = \{ab \rightarrow ba\}$. La cadena $abab$ admite dos pasos distintos, según cuál de las dos ocurrencias de ab se reemplace, y los dos conducen a la misma forma normal $baaa$. En general, la única forma normal de una cadena es la que resulta de poner todas las b antes de todas las a .

Sobre $\{a, b, c\}$, considere $R_2 = \{a \rightarrow b, a \rightarrow c\}$. La cadena a admite dos pasos, uno hacia b y otro hacia c , y las dos son formas normales distintas.

Sobre $\{a\}$, considere $R_3 = \{a \rightarrow a\}$. La cadena a se reescribe en sí misma indefinidamente y no tiene forma normal.

7.6.2. Terminación y confluencia. El Ejemplo 7.6.1 ilustra dos propiedades que interesan para el propósito de la sección.

Definición 7.6.2: Terminación y confluencia

Sea $\rightarrow$ una relación binaria sobre un conjunto A y sea $\rightarrow^*$ su cerradura reflexiva y transitiva. Se dice que $\rightarrow$ es:

- *terminante* si y solo si no existe una secuencia infinita $u_0 \rightarrow u_1 \rightarrow u_2 \rightarrow \dots$;
- *confluente* si y solo si para cualesquiera $u, v, w \in A$ con $u \rightarrow^* v$ y $u \rightarrow^* w$, existe t tal que $v \rightarrow^* t$ y $w \rightarrow^* t$; y
- *localmente confluente* si y solo si la misma condición se cumple cuando los dos primeros pasos son de longitud uno, es decir, cuando $u \rightarrow v$ y $u \rightarrow w$.

De un sistema de reescritura de cadenas R se dice que tiene una de estas propiedades si y solo si la tiene su relación $\rightarrow_R$.

La terminación garantiza que el proceso de reescritura siempre acaba. La confluencia garantiza que el resultado no depende de las decisiones que se hayan tomado en el camino. Juntas dicen que toda cadena tiene exactamente una forma normal, lo cual es justamente lo que se le pide a un simplificador, a un optimizador o a un procedimiento de decisión por normalización. En el Ejemplo 7.6.1, R_1 es terminante y confluyente, R_2 es terminante pero no confluyente, y R_3 no es terminante.

La confluencia local es, en apariencia, apenas un debilitamiento técnico de la confluencia. La diferencia es mayor de lo que parece: la confluencia local se puede verificar mirando un solo paso en cada dirección, mientras que la confluencia obliga a considerar derivaciones de cualquier longitud. El resultado que sigue explica cuándo la primera basta. Como su demostración no usa nada de las cadenas, se enuncia para una relación cualquiera.

Teorema 7.6.1: Lema de Newman

Si $\rightarrow$ es terminante y localmente confluyente, entonces $\rightarrow$ es confluyente.

Demostración

Como $\rightarrow$ es terminante, la relación es bien fundada y se puede razonar por inducción sobre ella. La propiedad que se demuestra para un elemento u es que cualesquiera dos derivaciones que parten de u se pueden reunir. La Figura 3 acompaña lo que sigue. Sean $u \rightarrow^* v$ y $u \rightarrow^* w$. Si alguna de las dos derivaciones tiene longitud cero, entonces $u = v$ o $u = w$ y el resultado es inmediato. En caso contrario, escriba $u \rightarrow u_1 \rightarrow^* v$ y $u \rightarrow u_2 \rightarrow^* w$.

Por confluencia local aplicada a $u \rightarrow u_1$ y $u \rightarrow u_2$, existe t_1 con $u_1 \rightarrow^* t_1$ y $u_2 \rightarrow^* t_1$.

Ahora se usa la hipótesis de inducción, que es aplicable a u_1 y a u_2 porque ambos se obtienen de u en un paso. Aplicada a u_1 con las derivaciones $u_1 \rightarrow^* v$ y $u_1 \rightarrow^* t_1$, entrega t_2 con $v \rightarrow^* t_2$ y $t_1 \rightarrow^* t_2$. Aplicada a u_2 con las derivaciones $u_2 \rightarrow^* w$ y $u_2 \rightarrow^* t_1 \rightarrow^* t_2$, entrega t_3 con $w \rightarrow^* t_3$ y $t_2 \rightarrow^* t_3$.

Entonces $v \rightarrow^* t_2 \rightarrow^* t_3$ y $w \rightarrow^* t_3$, que es lo que se quería.

7.6.3. Lo que no se puede decidir. Las dos propiedades centrales son indecidibles.

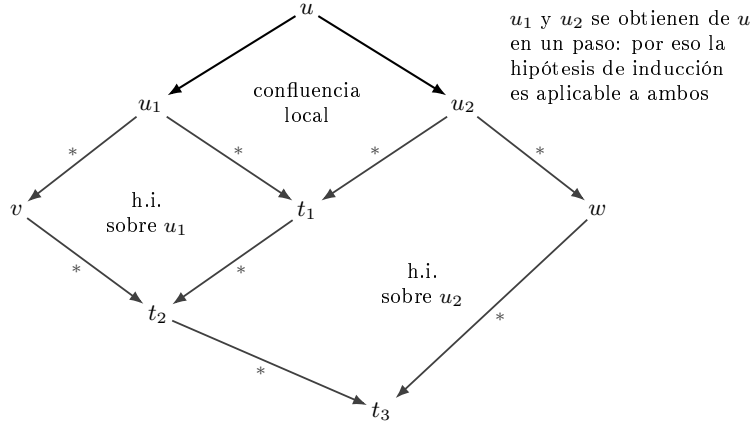


Figura 3. El paso inductivo de la demostración del Teorema 7.6.1. Las flechas gruesas son los dos primeros pasos y las delgadas son derivaciones de cualquier longitud. Cada una de las tres regiones se cierra con una hipótesis distinta: la de arriba con la confluencia local y las otras dos con la hipótesis de inducción, aplicada a u_1 y a u_2 respectivamente.

Teorema 7.6.2

Los siguientes lenguajes son indecidibles:

1. $\{\langle R \rangle \mid \text{"}R \text{ es terminante"}\}$; y
2. $\{\langle R \rangle \mid \text{"}R \text{ es confluyente"}\}$.

Las dos demostraciones son reducciones, pero de una complejidad que excede el alcance de este texto; se pueden consultar en [Ohl02]. La idea, en ambos casos, es la que ya se anticipó en la Sección 7.5: las configuraciones de un cómputo se codifican como cadenas y las transiciones como reglas, de manera que el sistema resultante hereda las propiedades de la ejecución que simula.

Nota 7.6.1: Las dos indecidibilidades no son independientes

Vale la pena leer el Teorema 7.6.2 a la luz del Teorema 7.6.1, porque la segunda parte no es un resultado separado de la primera.

Suponga por un momento que se supiera de antemano que el sistema es terminante. El lema de Newman reduce entonces la confluencia a la confluencia local, y la Sección 7.6.4 muestra que, bajo esa misma hipótesis, la confluencia local se decide. Es decir, sobre sistemas terminantes la confluencia es decidible. Conviene insistir en que la hipótesis se usa en los dos pasos: sin ella ni siquiera

la confluencia local se decide, pues verificar que un par se reúne obliga a recorrer un conjunto que puede ser infinito.

La indecidibilidad de la confluencia vive entonces enteramente en el territorio de los sistemas que no terminan. No son dos obstáculos independientes: la única manera de escapar del segundo es restringirse a los sistemas terminantes, y reconocerlos es justamente el primero.

7.6.4. Lo que sí se puede decidir. Falta justificar lo que afirma la Nota 7.6.1. La herramienta es la siguiente observación: para verificar la confluencia local no hace falta examinar todas las cadenas, sino apenas una cantidad finita de situaciones.

Considere dos pasos aplicables a una misma cadena u , digamos con las reglas $l_1 \rightarrow r_1$ y $l_2 \rightarrow r_2$. Las ocurrencias de l_1 y l_2 dentro de u pueden estar en tres posiciones relativas. Si son disyuntas, los dos pasos no se estorban y aplicar el otro después reúne las dos derivaciones de inmediato. Los casos interesantes son los otros dos, y ambos dependen solo de las reglas y no de u .

Definición 7.6.3: Par crítico

Sean $l_1 \rightarrow r_1$ y $l_2 \rightarrow r_2$ dos reglas de R , no necesariamente distintas. Los *pares críticos* que determinan son los siguientes:

- *por solapamiento*: si $l_1 = xw$ y $l_2 = wy$ con $x, w, y \neq \lambda$, el par (r_1y, xr_2) , que proviene de los dos pasos aplicables a la cadena xwy ; y
- *por inclusión*: si $l_1 = xl_2y$, el par (r_1, xr_2y) , que proviene de los dos pasos aplicables a la cadena l_1 .

De la segunda cláusula se excluye un único caso: cuando las dos reglas son la misma, en cuyo caso $x = y = \lambda$ forzosamente, los dos pasos son el mismo paso y el par resultante es (r, r) , que no aporta información.

Conviene notar tres cosas sobre esta definición. La primera es que una regla puede solaparse consigo misma de manera no degenerada, como ocurre con $bb \rightarrow d$ sobre la cadena bbb , con $x = w = y = b$.

La segunda es por qué la primera cláusula exige que x e y sean no vacías. Si $x = \lambda$, entonces l_1 es un prefijo de l_2 y la situación es una inclusión, que la segunda cláusula ya cubre al recorrer la misma pareja de reglas en el orden contrario; con $y = \lambda$ ocurre lo propio con los sufijos. Sin esa exigencia los mismos pares se contarían dos veces.

La tercera es que la exclusión de la segunda cláusula es estrecha a propósito: si las reglas son distintas, el par que produce $l_1 = l_2$ es legítimo, y es precisamente lo que ocurre con R_2 en el Ejemplo 7.6.1.

Note que la definición no menciona ninguna cadena de entrada: un par crítico se calcula mirando únicamente los lados izquierdos de dos reglas y buscando en qué posiciones uno puede solaparse con el otro o contenerlo. Como las reglas son finitas y sus lados izquierdos también, la cantidad de pares críticos de un sistema es finita y se calcula recorriendo todas las posiciones posibles.

Ejemplo 7.6.2

Considere $R = \{ab \rightarrow c, bb \rightarrow d\}$ sobre $\{a, b, c, d\}$.

El sufijo b del lado izquierdo de la primera regla coincide con el prefijo b del lado izquierdo de la segunda, de modo que hay un solapamiento con $x = a$, $w = b$ y $y = b$. La cadena en cuestión es abb , que admite dos pasos: usando la primera regla se obtiene cb y usando la segunda se obtiene ad . El par crítico es entonces (cb, ad) .

Ninguna de las dos cadenas admite pasos adicionales, de manera que son formas normales distintas y el par no se puede reunir. Luego R no es localmente confluente.

Este no es el único par crítico del sistema: la segunda regla se solapa consigo misma sobre la cadena bbb . Calcularlos todos se propone como ejercicio.

Teorema 7.6.3: Lema de los pares críticos

R es localmente confluente si y solo si todo par crítico (v, w) de R admite una cadena t con $v \rightarrow_R^* t$ y $w \rightarrow_R^* t$.

La demostración consiste en el análisis de los tres casos de posición relativa mencionados arriba y se desarrolla por partes en los ejercicios de esta sección. Una presentación detallada y su generalización a reescritura de términos se encuentran en [Ohl02].

Teorema 7.6.4: Decidibilidad de la confluencia sobre sistemas terminantes

Existe un algoritmo que, dado un sistema de reescritura de cadenas R del cual se sabe que es terminante, decide si R es confluente.

Demostración

El algoritmo calcula los pares críticos de R y verifica que cada uno se pueda reunir; responde afirmativamente si todos se reúnen y negativamente en caso contrario. Su corrección es el Teorema 7.6.3 junto con el Teorema 7.6.1. Resta ver que termina.

La cantidad de pares críticos es finita, pues hay finitas parejas de reglas y, para cada una, finitas posiciones de solapamiento o inclusión. Falta entonces verificar que la reunión de un par se decide.

Sea v una cadena cualquiera. El conjunto de las cadenas alcanzables desde v es finito. En efecto, el árbol de las derivaciones que parten de v es finitamente ramificado, ya que cada cadena admite una cantidad finita de pasos porque hay finitas reglas y finitas posiciones donde aplicarlas, y no tiene ramas infinitas, porque R es terminante. Un árbol con esas dos propiedades es finito, que es el lema de König leído en sentido contrario.

El conjunto de cadenas alcanzables desde v se calcula entonces por exploración exhaustiva, y el proceso termina. Dado un par (v, w) , basta calcular los dos conjuntos y verificar si se intersecan.

Nota 7.6.2: Una promesa que no se puede verificar

El Teorema 7.6.4 no afirma que cierto lenguaje sea decidible, y conviene ver por qué no puede afirmarlo.

Para hablar de decidir un lenguaje, la Definición 6.4.2 exige una función total sobre todo Σ^* . El algoritmo de la demostración anterior no calcula una: sobre un sistema que no termina, la exploración de las cadenas alcanzables desde v puede no acabar nunca y entonces el algoritmo tampoco acaba. Lo que el teorema afirma es de otra naturaleza y más débil: hay un procedimiento que responde correctamente *siempre que su entrada cumpla una promesa*, la de ser terminante, y del cual no se dice nada cuando la promesa falla.

La salida natural sería convertir la promesa en una verificación, haciendo que el procedimiento examine primero si R termina y responda negativamente cuando no. Eso es exactamente lo que la primera parte del Teorema 7.6.2 prohíbe. La promesa no se puede verificar y por eso el enunciado tiene la forma “existe un algoritmo que...” y no la forma “ L es decidible”.

Nota 7.6.3: ¿Dónde queda la frontera?

Conviene recoger lo que la sección establece, porque las tres afirmaciones son fáciles de confundir.

Determinar si un sistema arbitrario termina es indecidible. Determinar si un sistema arbitrario es confluente también lo es. Pero determinar si un sistema *del cual se sabe que termina* es confluente sí se decide y el Teorema 7.6.4 exhibe el procedimiento. La hipótesis de terminación es lo único que separa un caso del otro, y la Nota 7.6.2 explica por qué no se puede obtener por medios algorítmicos. En la práctica se suple con demostraciones de terminación hechas a mano o con técnicas que funcionan sobre familias restringidas de sistemas, que es exactamente la salida que la Nota 7.4.3 describe para el análisis de programas.

Ejercicios

1. Calcule todos los pares críticos del sistema $R = \{ab \rightarrow c, bb \rightarrow d\}$ del Ejemplo 7.6.2. Debe encontrar dos. Determine cuáles se pueden reunir.
2. Demuestre que el sistema $R_1 = \{ab \rightarrow ba\}$ del Ejemplo 7.6.1 no tiene pares críticos. Concluya que es localmente confluente y, usando el Teorema 7.6.1, que es confluente.
3. Demuestre que R_1 es terminante. Ayuda: a cada cadena asóciela la cantidad de parejas de posiciones $i < j$ tales que la posición i lleva una a y la posición j lleva una b , y examine cómo cambia ese número al aplicar la regla.
4. Calcule los pares críticos de $R_2 = \{a \rightarrow b, a \rightarrow c\}$ y verifique que el sistema no es localmente confluente. Explique por qué la exclusión del caso degenerado en la Definición 7.6.3 no elimina estos pares.
5. Demuestre la dirección fácil del Teorema 7.6.3: si R es localmente confluente, entonces todo par crítico se puede reunir. Ayuda: un par crítico proviene, por construcción, de dos pasos aplicables a una misma cadena.
6. Demuestre la dirección difícil del Teorema 7.6.3. Suponga que todo par crítico se reúne y sean $u \rightarrow_R v$ y $u \rightarrow_R w$ dos pasos sobre una misma cadena, con ocurrencias de los lados izquierdos en posiciones p_1 y p_2 . Distinga tres casos y resuelva cada uno:
 - a) las dos ocurrencias son disyuntas;
 - b) las dos ocurrencias se solapan sin que ninguna contenga a la otra; y
 - c) una ocurrencia está contenida en la otra.

Para el primer caso no se necesita ninguna hipótesis sobre pares críticos; explique por qué. Para los otros dos, identifique cuál par crítico corresponde a la situación y cómo el contexto que rodea a las ocurrencias se arrastra en la reunión.

7. Exhiba una relación que sea localmente confluente y que no sea confluente. Concluya que la hipótesis de terminación del Teorema 7.6.1 no se puede quitar. Ayuda: basta una relación sobre cuatro elementos.
8. En la demostración del Teorema 7.6.4 se afirma que el conjunto de cadenas alcanzables desde una cadena dada es finito. Identifique en qué punto exacto se usa que R es terminante y exhiba un sistema, necesariamente no terminante, para el cual ese conjunto sea infinito.
9. Implemente en Python el algoritmo del Teorema 7.6.4 y ejecútelo sobre los sistemas de esta sección. Su programa debe recibir un sistema que se sabe terminante; explique qué ocurre si se le entrega uno que no lo sea.
10. Investigue acerca del procedimiento de completación de Knuth y Bendix, que toma un sistema terminante y no confluente e intenta agregarle reglas hasta volverlo confluente. Explique por qué es un procedimiento y no un algoritmo, y relaciónelo con el Teorema 7.6.2.

Notas del capítulo y referencias

La técnica de reducción de la Sección 7.1 es estándar y aparece con este nombre en cualquier texto de computabilidad; se puede consultar en [Sip12, HR87, Dav85]. La versión que se presenta aquí es la que suele llamarse *many-one*, y su primera formulación sistemática es de E. Post [Pos44], el mismo artículo en el cual aparece el Teorema 6.5.3. La notación $\leq_c$ se escogió para que anticipe la de reducción polinomial del Capítulo 8. Esa elección no es solo tipográfica: la reducción polinomial es la misma noción con una exigencia adicional sobre el costo de calcular la función de reducción, de modo que el lector que llegue allá no encuentra un concepto nuevo sino un refinamiento de este.

El problema de la parada y su demostración por autorreferencia provienen del artículo de A. Turing de 1936 [Tur36], que también introduce la máquina universal usada en el Capítulo 6. La antología editada por M. Davis [Dav04] lo reproduce junto con los demás artículos fundacionales; el recorrido comentado de C. Petzold [Pet08] es una buena manera de leerlo. La presentación de la Sección 7.2 tiene una particularidad que conviene señalar. Como el texto fijó Python como modelo de cómputo en la Nota 6.4.1, el programa contradictorio de la demostración se puede *escribir* y no solo describir. La mayoría de los tratamientos debe conformarse con lo segundo, porque construir el programa exigiría antes construir la máquina.

Los programas que imprimen su propio texto, tratados en la Nota 7.2.3, son el caso más simple del teorema de recursión, cuya formulación general se encuentra en [HR87]. Vale la pena mencionar una aplicación que no es un juego: K. Thompson dedicó su conferencia de recepción del Premio Turing [Tho84] a mostrar cómo

un compilador puede reproducir una puerta trasera en cada compilación de sí mismo, de manera que el código malicioso no aparece en ningún archivo fuente. El mecanismo es exactamente el de un programa autorreplicante.

El orden de las secciones 7.3 y 7.4 es una decisión deliberada de este texto y no la habitual. Lo usual es demostrar el teorema de Rice y obtener los casos particulares como corolarios; aquí se hacen primero tres reducciones a mano y solo después aparece el teorema que las explica. La razón es que el paso difícil de una reducción es siempre el mismo, decidir qué programa construir, y ese paso no se aprende viendo un teorema general. El Teorema 7.4.1 se debe a H. G. Rice [Ric53]. Se enuncia aquí sobre propiedades de la función que calcula un programa y no sobre propiedades del lenguaje que acepta, que es la otra formulación frecuente. La razón es que HALTEEMPTY no es una propiedad del lenguaje aceptado, de manera que la versión sobre lenguajes no lo cubriría; el ejercicio final de la Sección 7.4 le pide al lector reconstruir esa otra versión y compararlas.

El problema de correspondencia de la Sección 7.5 fue formulado por E. Post en 1946 [Pos46], un año antes de que el mismo autor demostrara la indecidibilidad del problema de la palabra para sistemas de Thue [Pos47], que es el resultado fundacional de la indecidibilidad en reescritura de cadenas. La demostración de la indecidibilidad de PCP se deja en los ejercicios, desarrollada por etapas a través del problema modificado. Esa es la única indecidibilidad que el capítulo enuncia sin demostrar ni dejar demostrada por el lector en algún grado, y la decisión se tomó por costo: la construcción exige codificar historias de ejecución y, con ello, reintroducir el modelo de máquina que el Capítulo 6 evitó deliberadamente. Una exposición completa se encuentra en [Sip12].

La Sección 7.6 cambia de signo respecto del resto del capítulo: en lugar de demostrar que algo no se puede, exhibe un algoritmo. El Teorema 7.6.1 se debe a M. H. A. Newman [New42] y la caracterización de la confluencia local por pares críticos, junto con el procedimiento de completación que se menciona en los ejercicios, proviene del trabajo de D. Knuth y P. Bendix [KB70]. El tratamiento se restringe a reescritura de cadenas por una razón práctica que vale la pena explicitar: sobre cadenas, los pares críticos son solapamientos de subcadenas y no hace falta unificación, de modo que toda la maquinaria de términos y sustituciones se puede omitir sin perder la idea. El lector que quiera la versión general la encuentra en [BN98] y [Ohl02], y en [CDE⁺07] la verá funcionando como lenguaje de programación.

Conviene cerrar señalando que la indecidibilidad no es un fenómeno confinado a la computación. El décimo problema de Hilbert, mencionado en la Nota 6.1.1, es un enunciado de teoría de números que resultó indecidible; el problema de la palabra para grupos lo es también; y en física se conocen preguntas indecidibles sobre sistemas cuánticos de muchos cuerpos, entre ellas la de determinar si un modelo dado tiene o no una brecha espectral. Una revisión reciente de esa línea de trabajo

se encuentra en [PECG⁺25]. En todos esos casos el mecanismo es el mismo que el de este capítulo. Se muestra que el objeto de estudio, sean ecuaciones diofánticas, presentaciones de grupos o Hamiltonianos, es capaz de simular un cómputo arbitrario; desde ese momento hereda todo lo que el Capítulo 6 estableció sobre los límites de lo computable.

Intratabilidad: P vs. NP

Suponga por un momento que alguien recién graduado está en un proceso de selección con miras a una posición de programador en una red social famosa. Esta persona se ha preparado arduamente para la siguiente entrevista en la cual, como parte del proceso de selección, se valorarán sus conocimientos técnicos y prácticos en algoritmos. Por ello, el recién graduado ha estudiado algoritmos y practicado sus implementaciones, con una y mil variantes, en varios lenguajes de programación.

En la entrevista, se formula la siguiente pregunta: ¿se comprometería a resolver en un par de meses el problema de determinar el tamaño máximo de usuarios de la red social en el cual todos son amigos? La pregunta viene acompañada de un ofrecimiento con un jugoso salario de varios cientos de miles de dólares anuales y acceso privilegiado a asistentes basados en inteligencia artificial. Aprovechando sus conocimientos en grafos, el sujeto de la entrevista se da cuenta de que el problema computacional, básicamente, es determinar el tamaño máximo de un subgrafo completo (i.e., la red social es el grafo y el subgrafo es el grupo de amigos).

Problema 8.0.1: Grupo de amigos

Entrada: Un grafo G con N vértices, $N \geq 0$

Salida: El tamaño máximo de un subgrafo completo en G .

Se sabe que hay varios supercomputadores en la compañía, con capacidades de cómputo inimaginables para un recién graduado; hay un panorama prometedor. Todo parece estar dado para que el recién graduado acepte la pomposa vacante y para que, en un par de meses, el problema haya sido resuelto. Lamentablemente, el recién graduado decide no aceptar la propuesta argumentando que sería irresponsable comprometerse a resolver el problema formulado. Quien lidera el proceso de entrevista agradece al recién graduado por su honestidad, y le ofrece la posición

con un salario mayor y sin tener que comprometerse a resolver el problema de los amigos. Para muchas personas lo usual habría sido que la compañía optara por rechazar al candidato y terminar allí el proceso de selección; ¿qué sucedió?

Hace varias décadas se avanzó significativamente en la tarea de identificar problemas para los cuales no se conocen soluciones *eficientes*, es decir, para los cuales no se conocen algoritmos cuya complejidad temporal sea polinomial. Como parte de este grupo, fueron identificados problemas de lógica, optimización, combinatoria e inteligencia artificial. Sin embargo, la tarea ha sido muy difícil y, al día de hoy, el proceso se encuentra en el “peor” de dos mundos. Por una parte, para los problemas identificados no se conocen soluciones eficientes. Por otra parte, no hay demostraciones que garanticen que dichas soluciones no existen. Es decir, la tarea de identificación aún se encuentra en una zona gris.

Esta ha sido un área fértil para la investigación, sobre todo por las implicaciones que tienen estos problemas en la sociedad actual. Por ejemplo, el hecho de hacer transacciones en línea seguras depende, básicamente, de no contar con algoritmos que rápidamente puedan descomponer números en factores primos. A los problemas para los cuales no se conocen soluciones que corren en tiempo polinomial se les denomina *intratables*. El nombre viene de esa ausencia y de nada más: lo mejor que se conoce para ellos corre en tiempo *superpolinomial*, que no es lo mismo que exponencial. Entre lo polinomial y lo exponencial hay sitio, y el ejemplo de la factorización lo ocupa, pues el mejor algoritmo conocido para descomponer un número de n dígitos binarios no es polinomial en n ni tampoco exponencial. Para los problemas de este capítulo, en cambio, los algoritmos conocidos sí son exponenciales y eso basta para descartarlos en la práctica: si una red social tiene millones de usuarios, un algoritmo exponencial en su tamaño está lejos de terminar de ejecutarse en decenas de siglos.

Este capítulo cierra un recorrido de tres. El Capítulo 6 pregunta qué se puede computar y responde que casi nada, aunque sin exhibir un solo problema fuera del alcance de un computador. El Capítulo 7 exhibe varios y construye las herramientas para encontrar más. Queda una tercera pregunta, que es la que ocupa a este capítulo: de lo que sí se puede computar, ¿qué se puede computar en un tiempo razonable? La diferencia de tono entre los tres no es casual. Allá se trataba de lo que ningún computador podrá hacer jamás y los resultados son definitivos. Acá se trata de lo que nadie ha logrado hacer hasta ahora y los resultados son, en su mayoría, condicionales.

El desarrollo sigue de cerca las herramientas de los dos capítulos anteriores. Los problemas se estudian como lenguajes formales, tal y como se estableció en la Sección 6.2, y la técnica central vuelve a ser la reducción de la Sección 7.1, ahora con una exigencia adicional sobre el costo de calcular la función que transforma las instancias. Sobre esa base se definen las clases P, NP y coNP, y la noción de

completitud que identifica a los problemas más difíciles de NP. De ahí sale una de las preguntas abiertas más importantes de las ciencias exactas, la de determinar si el poder de cómputo de los algoritmos *determinísticos* polinomiales es igual al de los algoritmos *no determinísticos* polinomiales. Usualmente se formula como P vs. NP: es tan importante que quien la resuelva tiene derecho a un premio de un millón de dólares. A pesar de que numerosos intentos han fracasado desde los 1970s en reclamar este premio, se cree que $P \neq NP$.

El capítulo demuestra que cuatro problemas concretos son NP-completos, entre ellos el del grupo de amigos que abre esta introducción. Con eso queda explicado lo que ocurrió en la entrevista. El recién graduado reconoció el problema y supo que comprometerse a resolverlo en un par de meses equivalía a comprometerse a resolver, de paso, todos los problemas clásicos con los que han fracasado numerosas mentes durante medio siglo. Quien entrevistaba también lo reconoció y por eso el agradecimiento.

Encontrarse con un problema NP-completo es, entonces, una invitación a limitar las expectativas de encontrar un algoritmo eficiente que lo resuelva. No es, en cambio, una invitación a rendirse: la última sección del capítulo se ocupa de lo que sí se puede hacer cuando la respuesta exacta y eficiente está fuera de alcance.

8.1. La clase P

La clase P es la colección de problemas de decisión (i.e., lenguajes) que pueden ser decididos por un algoritmo (i.e., por una función total computable) en tiempo polinomial. Por razones filosóficas, los lenguajes en esta clase son denominados *tratables*. Esta sección presenta la clase P, con ella formaliza las nociones de problema aceptable y decidible en tiempo polinomial, y estudia algunas de sus propiedades. Se usan indistintamente las nociones de función computable y algoritmo para referirse al mismo objeto de estudio, tal y como se hace en el Capítulo 6.

Conviene ubicar de entrada esta clase dentro del panorama de los dos capítulos anteriores. Un lenguaje en P es, ante todo, un lenguaje decidible: la exigencia de tiempo polinomial se agrega a la de existir un algoritmo de decisión, no la reemplaza. Todo este capítulo transcurre, entonces, dentro de la región más interna de la Figura 4 y los problemas indecidibles del Capítulo 7 quedan por completo fuera de escena.

Definición 8.1.1: Algoritmo polinomial

Sea $L \subseteq \Sigma^*$ un lenguaje y $A : \Sigma^* \rightarrow \Sigma$ una función computable. Se dice que:

- A acepta L en tiempo polinomial si y solo si A acepta L y adicionalmente existe un $k \in \mathbb{N}$ tal que, para cualquier $x \in L$, la invocación $A(x)$ termina en tiempo $O(|x|^k)$.
- A decide L en tiempo polinomial si y solo si A decide L y adicionalmente existe un $k \in \mathbb{N}$ tal que, para cualquier $x \in \Sigma^*$, la invocación $A(x)$ termina en tiempo $O(|x|^k)$.

Note la asimetría entre las dos condiciones. La de decisión cuantifica sobre todas las cadenas, mientras que la de aceptación lo hace únicamente sobre las que están en L . La razón es la Definición 6.3.3: un algoritmo de aceptación puede quedar indeterminado sobre las cadenas que no pertenecen al lenguaje y sobre esas no tiene sentido hablar de cuánto tiempo toma en responder.

A un algoritmo que toma tiempo polinomial se le denomina *algoritmo polinomial*. Note que estos son los algoritmos cuyo comportamiento, en función de la cantidad de pasos que realizan para resolver un problema, está asintóticamente acotado por un polinomio.

Definición 8.1.2: La clase P

Sea $L \subseteq \Sigma^*$ un lenguaje. Se dice que L está en la clase P si y solo si existe un algoritmo polinomial que decide L . Formalmente,

$$P = \{L \mid L \subseteq \Sigma^* \wedge \text{“hay un algoritmo de decisión polinomial tal que } L = L(A)\text{”}\}.$$

De acuerdo con la Definición 8.1.2, la clase P es una colección de lenguajes, i.e., $P \subseteq \mathcal{P}(\Sigma^*)$. Entonces, para afirmar que un problema L está en P debe demostrarse que L puede ser decidido en tiempo polinomial. Pueden existir varios algoritmos de decisión para L ; es suficiente con que uno de ellos sea polinomial para afirmar que dicho lenguaje está en P.

Ejemplo 8.1.1

Son ejemplos de lenguajes en P:

- Buscar un elemento en un arreglo (si la operación de igualdad se calcula en tiempo polinomial; lo usual es hacerlo en tiempo constante). El lenguaje asociado es:

$$\text{ARRFIND} = \{\langle A[0..N], x \rangle \mid \text{“}A \text{ es un arreglo y } x \text{ está en } A\text{”}\}.$$

El algoritmo de la Implementación 6.4.1 lo decide recorriendo el arreglo una sola vez.

- Determinar si un grafo es acíclico. El lenguaje asociado es:

$$\text{ACYCLIC} = \{\langle G \rangle \mid \text{"}G \text{ es un grafo acíclico"}\}.$$

- Determinar si un número es primo. El lenguaje asociado es:

$$\text{PRIME} = \{\langle N \rangle \mid \text{"}N \text{ es un número primo"}\}.$$

A diferencia de los dos anteriores, este caso no es inmediato. Que PRIME esté en P es un resultado de 2002, debido a M. Agrawal, N. Kayal y N. Saxena, y estuvo abierto durante décadas. Note que la entrada es la *representación* de N , de modo que el tamaño de la entrada es del orden de $\log N$ y no de N ; el algoritmo ingenuo que ensaya todos los divisores hasta $\sqrt{N}$ es exponencial en ese tamaño.

Como se mencionó al inicio de la sección, un lenguaje que está en P se denomina *tratable*. Esto equipara problemas para los cuales hay soluciones que corren en tiempo lineal con otros para los cuales solo haya soluciones que toman tiempo $\Theta(N^{1000})$, en donde N es el tamaño del problema. La primera clase de algoritmos es útil en la práctica, mientras que claramente la segunda no lo es. A favor de esta identificación y categorización de “algoritmo eficiente” hay tres motivos, aún razonables a la luz de las dificultades anteriores:

- Primero, son pocos los problemas en P para los que no se conocen algoritmos acotados por un polinomio con grado razonable (e.g., que un grado no superior a 4 o 5). Esto sucede, principalmente, porque es común que las soluciones sean mejoradas (en relación con su comportamiento temporal asintótico) una vez se establece un primer algoritmo que resuelve un problema de interés.
- Segundo, esta noción de ejecución eficiente es compatible con diferentes modelos de computación, e.g., máquinas de Turing y máquinas de acceso aleatorio. Es una categorización homogénea y fácilmente interpretable en los modelos de computación más comunes. Esta es, precisamente, la segunda parte de la Nota 6.2.1: los modelos no solo computan lo mismo, sino que se simulan unos a otros con un sobre costo polinomial, de manera que la clase P resulta ser la misma sin importar cuál se adopte.
- Tercero, la clase P tiene propiedades interesantes en relación con los lenguajes que la conforman. Por ejemplo, es cerrada bajo complemento lo cual quiere decir que si un lenguaje se puede decidir eficientemente, lo mismo se puede hacer con su complemento. Lo mismo sucede con la unión, intersección, concatenación y clausura de Kleene. Esta tercera razón se retoma al final de la sección.

Un resultado importante es que las nociones de decisión y aceptación coinciden en P. Esto no es lo habitual en otras clases de complejidad computacional, lo cual hace de esta observación un resultado a resaltar.

Teorema 8.1.1

El siguiente conjunto es igual a P:

$$\{L \mid L \subseteq \Sigma^* \wedge \text{“hay un algoritmo de aceptación polinomial tal que } L = L(A)\text{”}\}.$$

Demostración

El objetivo es demostrar:

$$P = \{L \mid L \subseteq \Sigma^* \wedge \text{“hay un algoritmo de aceptación polinomial tal que } L = L(A)\text{”}\}.$$

Se procede por doble implicación.

($\Rightarrow$): Si $L \in P$, entonces hay un algoritmo de decisión polinomial A tal que $L(A) = L$ y, para cualquier $x \in \Sigma^* \setminus L$, se tiene que $A(x) = 0$ (por definición de decisión). Luego, A también es un algoritmo de aceptación para L . Es decir, hay un algoritmo polinomial que acepta L (en este caso, el mismo A).

($\Leftarrow$): Suponga que B es un algoritmo de aceptación polinomial para L , de modo que $L(B) = L$. No se puede concluir de inmediato que B decida L : para $x \notin L$ el algoritmo puede quedar indeterminado en lugar de retornar 0. Por la Definición 8.1.1 existen c y k tales que, para cualquier $x \in L$, la invocación $B(x)$ termina antes de $c|x|^k$ pasos. La idea es entonces darle a B ese presupuesto y, si no ha respondido, concluir que la cadena no está en L . La herramienta para hacerlo es la simulación acotada de la Nota 6.5.1. Sea P un programa que calcula B y considere el algoritmo A que, dada x , simula $c|x|^k$ pasos de P sobre x y retorna 1 si esa simulación terminó retornando 1; en cualquier otro caso retorna 0. El algoritmo A es total, pues `simulate` termina siempre, y es polinomial pues simular una cantidad polinomial de pasos toma tiempo polinomial. Resta ver que decide L . Si $x \in L$, entonces $B(x) = 1$ y la invocación termina dentro del presupuesto, de modo que $A(x) = 1$. Si $x \notin L$, entonces $B(x) \neq 1$ y ninguna simulación puede terminar retornando 1, de modo que $A(x) = 0$.

En conclusión, estos dos conjuntos son iguales.

Los lenguajes en P son cerrados bajo algunas operaciones de conjuntos, lo cual tiene una ventaja práctica para la construcción de algoritmos de decisión eficientes.

Por ejemplo, si se conocen soluciones polinomiales para dos lenguajes en P, automáticamente se sabe que hay un algoritmo eficiente para resolver la unión de los lenguajes dado que la clase P es cerrada bajo unión. Las propiedades de clausura más notorias sobre P se enuncian en el Teorema 8.1.2. Note que son exactamente las mismas operaciones bajo las cuales son cerrados los lenguajes computables, según el Teorema 6.5.1; lo que el enunciado agrega es que la propiedad se conserva al exigir además tiempo polinomial.

Teorema 8.1.2

Las siguientes afirmaciones son ciertas acerca de P:

1. Es cerrado bajo complemento; es decir, si $L \in P$, entonces

$$\overline{L} \in P.$$

2. Es cerrado bajo unión; es decir, si $L_1 \in P$ y $L_2 \in P$, entonces

$$(L_1 \cup L_2) \in P.$$

3. Es cerrado bajo intersección; es decir, si $L_1 \in P$ y $L_2 \in P$, entonces

$$(L_1 \cap L_2) \in P.$$

4. Es cerrado bajo concatenación; es decir, si $L_1 \in P$ y $L_2 \in P$, entonces

$$(L_1 \cdot L_2) \in P.$$

5. Es cerrado bajo la clausura de Kleene; es decir, si $L \in P$, entonces

$$L^* \in P.$$

Demostración

Se presenta una demostración de 2 y se proponen las demás demostraciones como ejercicios al lector. Suponga que $L_1 \in P$ y $L_2 \in P$; el objetivo es demostrar que $(L_1 \cup L_2) \in P$. Por las suposiciones, hay algoritmos polinomiales de decisión A_1 y A_2 tales que $L(A_1) = L_1$ y $L(A_2) = L_2$. Considere el siguiente algoritmo para cualquier $x \in \Sigma^*$:

- (i) Invocar $A_1(x)$ y guardar la respuesta.
- (ii) Invocar $A_2(x)$ y guardar la respuesta.
- (iii) Si al menos una de las respuestas obtenidas en (i) y (ii) es 1, entonces retornar 1; de lo contrario, retornar 0.

Note que el algoritmo propuesto retorna 1 sii $A_1(x) = 1 \vee A_2(x) = 1$, es decir, sii $x \in (L_1 \cup L_2)$. En cualquier otro caso retorna 0. Además, el algoritmo

es polinomial porque hace una cantidad constante de llamados a algoritmos polinomiales, según el Ejercicio 3. En conclusión, $(L_1 \cup L_2) \in P$ si L_1 y L_2 están en P.

Finalmente, un llamado a la precaución cuando se componen algoritmos polinomiales. En general, no es cierto que un algoritmo sea polinomial cuando hace una cantidad polinomial de llamadas a subrutinas que toman tiempo polinomial cada una (ver Ejercicio 4). Lo que sí es cierto es que si la cantidad de llamadas a las subrutinas está acotada por una constante, entonces necesariamente el algoritmo resultante corre en tiempo polinomial.

Ejercicios

1. Verdadero o falso: un algoritmo polinomial puede aceptar un lenguaje indecidible. Justifique su respuesta.
2. Complete la demostración del Teorema 8.1.2. Es decir, demuestre la clausura bajo complemento, bajo intersección, bajo concatenación y bajo la clausura de Kleene. Para la concatenación, recuerde que una cadena de longitud n admite $n+1$ particiones en dos, como en la parte (4) del Teorema 6.5.1. Para la clausura de Kleene, proceda por programación dinámica sobre los prefijos de la entrada.
3. Demuestre que si un algoritmo hace a lo sumo una cantidad constante de llamados a un algoritmo polinomial (a modo de subrutina) y adicionalmente realiza una cantidad de pasos en tiempo polinomial, entonces el algoritmo corre en tiempo polinomial.
4. Justifique con un ejemplo por qué si un algoritmo hace a lo sumo una cantidad polinomial de llamados a un algoritmo polinomial (a modo de subrutina), entonces no necesariamente el algoritmo corre en tiempo polinomial. Ayuda: apoyarse en el triángulo de Pascal.

8.2. La clase NP

La clase NP es la colección de problemas de decisión que pueden ser verificados en tiempo polinomial. Esta sección presenta esta clase, introduciendo el concepto de algoritmo de verificación, y obteniendo resultados relacionados con la clase P y otros de decidibilidad.

Definición 8.2.1: Algoritmo de verificación

Un *algoritmo de verificación* es una función total computable

$$V : \Sigma^* \times \Sigma^* \rightarrow \Sigma.$$

Se dice que V *verifica* una entrada $x \in \Sigma^*$ si y solo si existe un *certificado* $y \in \Sigma^*$ tal que $V(x, y) = 1$.

Un algoritmo de verificación es una función computable con dos argumentos, en la cual el primer argumento es la cadena de entrada a verificar, y el segundo el certificado o testigo para dicha verificación. Note que la Definición 6.4.1 define computabilidad para funciones de un solo argumento con rango en Σ . Un algoritmo de verificación se acomoda a ese marco de la manera usual, recibiendo la codificación $\langle x, y \rangle$ de la Nota 6.2.3; se escribe $V(x, y)$ por comodidad. Es la misma licencia que se toma con `simulate` en la Nota 6.5.1.

Habiendo definido qué es un algoritmo de verificación, se procede a definir el significado de verificar un lenguaje.

Definición 8.2.2: Verificación de lenguajes

Sea $V : \Sigma^* \times \Sigma^* \rightarrow \Sigma$ un algoritmo de verificación. El *lenguaje verificado* por V es el conjunto:

$$L(V) = \{x \mid x \in \Sigma^* \wedge \text{“existe } y \in \Sigma^* \text{ tal que } V(x, y) = 1\text{”}\}.$$

La notación $L(V)$ extiende la que se viene usando desde la Definición 6.3.1: tanto una función de aceptación como un algoritmo de verificación determinan un lenguaje, aunque lo hagan de maneras distintas.

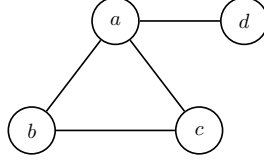
Un algoritmo V verifica un lenguaje L cuando para cualquier $x \in L$ existe un certificado y que el algoritmo puede utilizar para comprobar la pertenencia de x a L . Adicionalmente, no pueden existir certificados con los que V compruebe la pertenencia a L de cadenas que no hacen parte del lenguaje. Es decir, que al afirmar $L = L(V)$ se está entendiendo que no existen certificados y para los cuales $V(x, y) = 1$ cuando $x \notin L$. En el ejemplo de la red social que aparece en la introducción del capítulo, un certificado para comprobar que existe un grupo de 1000 amigos totalmente conectados es una lista de 1000 (puede ser una de varias) amigos que realmente están conectados, par a par. Si un grupo así no existe, es imposible contar con dicho certificado.

Ejemplo 8.2.1

Considere el siguiente problema de decisión:

$$\text{CLIQUE} = \{\langle G, k \rangle \mid \text{“}G \text{ tiene un subgrafo completo de tamaño } k\text{”}\}.$$

Considere el siguiente grafo $G = (V, E)$ con vértices $V = \{a, b, c, d\}$ y arcos $E = \{(a, b), (a, c), (a, d), (b, c)\}$:



Note que:

- $\langle G, 2 \rangle \in \text{CLIQUE}$ dado que $\langle \{a, b\} \rangle$ es un certificado para el cual un algoritmo de verificación A para CLIQUE responde

$$A(\langle G, 2 \rangle, \langle \{a, b\} \rangle) = 1.$$

- $\langle G, 3 \rangle \in \text{CLIQUE}$ dado que $\langle \{a, b, c\} \rangle$ es un certificado para el cual un algoritmo de verificación A para CLIQUE responde

$$A(\langle G, 3 \rangle, \langle \{a, b, c\} \rangle) = 1.$$

- $\langle G, 4 \rangle \notin \text{CLIQUE}$ dado que para cualquier $y \in \Sigma^*$ y algoritmo de verificación A para CLIQUE se tiene que

$$A(\langle G, 4 \rangle, y) = 0 \quad (\text{¿por qué?}).$$

La clase NP es la colección de lenguajes que pueden ser verificados en tiempo polinomial.

Definición 8.2.3: La clase NP

Un algoritmo de verificación V es *polinomial* si y solo si existe un $k \in \mathbb{N}$ tal que, para cualesquiera $x, y \in \Sigma^*$, la invocación $V(x, y)$ termina en tiempo $O((|x| + |y|)^k)$.

Sea $L \subseteq \Sigma^*$ un lenguaje. Se dice que L está en la clase NP si y solo si existen un algoritmo de verificación polinomial V y un polinomio p tales que, para cualquier $x \in \Sigma^*$:

$$x \in L \iff \text{“existe } y \in \Sigma^* \text{ con } |y| \leq p(|x|) \text{ y } V(x, y) = 1”}.$$

La Definición 8.2.3 impone dos condiciones que cumplen oficios distintos y conviene separarlos. La primera mide el tiempo en función de $|x| + |y|$, que es lo natural para un programa que recibe los dos argumentos y que, en principio, tiene que leerlos. La segunda acota la longitud del certificado, y es la que hace el trabajo de fondo: sin ella la clase resultante no sería NP sino una mucho más grande y ya conocida, como lo muestra la Nota 8.2.2.

Note que la cota no forma parte de la Definición 8.2.2, de manera que L y $L(V)$ no tienen por qué coincidir: lo que se pide es que L sea el conjunto de las cadenas que V verifica *con certificados cortos*. La diferencia se puede borrar exigiéndole a V que responda 0 cuando $|y| > p(|x|)$, con lo cual $L = L(V)$; es una comprobación que no le cuesta a V más que leer su entrada y se propone en el Ejercicio 9. De aquí en adelante se usa indistintamente una u otra forma.

De acuerdo con la Definición 8.2.3, la clase NP (al igual que P) es una colección de lenguajes, i.e., $\text{NP} \subseteq \mathcal{P}(\Sigma^*)$. Entonces, para afirmar que un problema L está en NP debe demostrarse que L puede ser verificado en tiempo polinomial. Pueden existir varios algoritmos de verificación para L ; es suficiente con que uno de ellos sea polinomial para afirmar que dicho lenguaje está en NP.

Nota 8.2.1

La sigla NP proviene de una definición alternativa de esta clase de complejidad: la de los problemas de decisión que pueden ser decididos en tiempo polinomial por un algoritmo sobre una máquina de Turing *no determinística* (ver Ejercicio 4).

Ejemplo 8.2.2

Son ejemplos de lenguajes en NP:

- El problema de buscar un elemento en un arreglo (si la operación de igualdad se calcula en tiempo polinomial; lo usual es hacerlo en tiempo constante):

$$\text{ARRFIND} = \{ \langle A[0..N], x \rangle \mid \text{“}A \text{ es un arreglo y } x \text{ está en } A\text{”} \}.$$

Un algoritmo de verificación tiene como certificado una posición n de x en $A[0..N]$. Note que la verificación se puede hacer en tiempo polinomial: se debe determinar si $0 \leq n < N$ y $A[n] = x$. Estas dos operaciones toman, a lo sumo, tiempo polinomial.

- El problema de determinar si un grafo dirigido es acíclico:

$$\text{ACYCLIC} = \{ \langle G \rangle \mid \text{“}G \text{ es un grafo dirigido acíclico”} \}.$$

Un algoritmo de verificación tiene como certificado una lista de vértices. Note que la verificación se puede hacer en tiempo polinomial: se debe determinar si la lista $v_0, \dots, v_{N-1}$ contiene exactamente una vez a cada vértice de G y si constituye un orden topológico, para lo cual basta revisar que cada arco (v_i, v_j) de G cumpla $i < j$. Note que la existencia de un

orden topológico equivale a la ausencia de ciclos, lo cual explica por qué esta lista sirve de certificado.

- El problema de determinar si un grafo tiene un subgrafo completo de un tamaño dado:

$$\text{CLIQUE} = \{ \langle G, k \rangle \mid \text{"}G \text{ tiene un subgrafo completo de tamaño } k\text{"} \}.$$

Un algoritmo de verificación tiene como certificado una lista de vértices. Note que la verificación se puede hacer en tiempo polinomial: se debe determinar si la lista $v_0, \dots, v_{k-1}$ corresponde a algunos vértices de G y si determinan un subgrafo completo de G . Esta última operación exige consultar las $\binom{k}{2}$ parejas de vértices de la lista, de modo que se puede hacer en orden $O(k^2)$ suponiendo que pertenencia al conjunto de arcos se puede hacer en tiempo constante (¿cómo?). Como k no supera el tamaño de la entrada, la verificación es polinomial.

De acuerdo con el Ejemplo 8.2.2, algunos problemas en P también están en NP. En realidad, como se establece en el Teorema 8.2.1, cualquier problema que puede ser decidido en tiempo polinomial también puede ser verificado en tiempo polinomial.

Teorema 8.2.1

$P \subseteq NP$.

Demostración

Sea L un lenguaje y suponga que $L \in P$. El objetivo es demostrar que $L \in NP$. Como $L \in P$, hay un algoritmo A que decide L en tiempo polinomial. Considere el algoritmo de verificación V de la Implementación 8.2.1, que ignora por completo el certificado y se limita a invocar a A .

El algoritmo V es total, pues A lo es, y es polinomial: A termina en tiempo polinomial en $|x|$, que lo es también en $|x| + |y|$. Tome como p el polinomio constante 0, de manera que el único certificado admisible es λ .

Resta verificar la condición de la Definición 8.2.3. Si $x \in L$, entonces $A(x) = 1$ y por lo tanto $V(x, \lambda) = 1$, de modo que existe un certificado admisible. Si $x \notin L$, entonces $A(x) = 0$ y por lo tanto $V(x, y) = 0$ para cualquier y ; luego no existe certificado alguno, admisible o no.

Como V es un algoritmo de verificación polinomial para L , se concluye que $L \in NP$.

Implementación 8.2.1: Verificación que ignora el certificado

```

1 def verify(x, y):          # the certificate y is ignored
2     return decide(x)       # decide is the polynomial algorithm for L

```

El Teorema 8.2.1 resuelve una mitad de la pregunta ¿P vs. NP? Establece que para responderla es necesario fijarse en la inclusión de NP en P, es decir, en la pregunta ¿NP $\subseteq$ P? Se cree que la respuesta es negativa, pero ninguna demostración ha podido ser construida en más de 50 años. Es decir, esta es la parte que ha ocupado muchas horas de muchos científicos y que aún hace falta por resolver, y así definitivamente completar el rompecabezas que dirimiría la relación exacta entre P y NP.

Nota 8.2.2: ¿Por qué se acota el certificado?

La cota $|y| \leq p(|x|)$ de la Definición 8.2.3 puede parecer un tecnicismo. No lo es: es lo único que separa a NP de una clase del Capítulo 6.

Considere el algoritmo de verificación que, dada la codificación $\langle P, x \rangle$ y un certificado y , simula $|y|$ pasos de P sobre x con la Nota 6.5.1 y responde 1 si esa simulación terminó retornando 1. El certificado es, literalmente, un reloj: lo único que aporta es cuánto tiempo conceder, y su contenido no se mira. Por el sobrecosto polinomial de la simulación, ese algoritmo de verificación es polinomial en $|x| + |y|$. Y existe un certificado que funciona si y solo si P termina aceptando a x , de manera que sin cota sobre $|y|$ el lenguaje verificado es HALT.

El fenómeno es general en las dos direcciones. Por un lado, quitarle la cota a la Definición 8.2.3 entrega siempre un lenguaje computablemente enumerable: basta recorrer los certificados en el orden canónico de la Definición 6.2.3 y aceptar al dar con uno que sirva, que es un algoritmo de aceptación en el sentido de la Definición 6.3.3. Por el otro, todo lenguaje computablemente enumerable se obtiene así, aplicándole el reloj del párrafo anterior a un programa que lo acepte. La versión sin cota de la Definición 8.2.3 define entonces, exactamente, la clase de los lenguajes computablemente enumerables.

Ninguna de las dos condiciones sobra, y se ve con precisión qué aporta cada una. Con la cota pero sin el límite de tiempo se obtienen todos los lenguajes decidibles; con el límite de tiempo pero sin la cota, todos los computablemente enumerables. La forma “existe un testigo que se comprueba mecánicamente” es la de la Sección 6.6 y por sí sola no dice nada acerca de eficiencia: lo que la

convierte en NP no es solo que la comprobación sea rápida, sino que el testigo sea corto.

Conviene cerrar la sección ubicando a NP dentro del panorama del Capítulo 6, tal y como se hizo con P al comienzo del capítulo.

Teorema 8.2.2

Si $L \in \text{NP}$, entonces L es decidable.

Demostración

Sean V un algoritmo de verificación polinomial y p un polinomio como los que la Definición 8.2.3 garantiza para L .

Considere el algoritmo que, dada x , recorre todas las cadenas y de longitud a lo sumo $p(|x|)$ usando el orden canónico de la Definición 6.2.3, evalúa $V(x, y)$ para cada una, y retorna 1 si alguna evaluación entrega 1 y 0 en caso contrario.

El recorrido es finito, pues hay $2^{p(|x|)+1} - 1$ cadenas de esa longitud o menos, y cada evaluación termina por ser V total. Luego el algoritmo es total y computable, y decide L por la Definición 8.2.3.

Nota 8.2.3: Intratable no es irresoluble

La demostración del Teorema 8.2.2 no solo establece que los problemas de NP son decidibles: exhibe un algoritmo concreto para cada uno de ellos. Ese algoritmo, sin embargo, examina una cantidad exponencial de certificados y por eso resulta inútil apenas la entrada crece un poco.

Ahí está la diferencia de fondo entre este capítulo y los dos anteriores. Los problemas del Capítulo 7 no tienen solución algorítmica alguna. Los de este capítulo sí la tienen y, de hecho, la solución acaba de ser exhibida; lo que no se conoce es una solución eficiente. Por eso se les llama *intratables* y no irresolubles.

Ejercicios

1. Diseñe algoritmos de verificación, respectivamente, para los lenguajes $L = \emptyset$ y $L = \Sigma^*$.
2. En el Ejemplo 8.2.1 se indica que $\{a, b\}$ es un certificado para determinar $\langle G, 2 \rangle \in \text{CLIQUE}$. ¿Cuáles son todos los testigos para $\langle G, 2 \rangle \in \text{CLIQUE}$? ¿Cuáles son todos los testigos para poder afirmar que $\langle G, 0 \rangle$ y $\langle G, 1 \rangle$ están en CLIQUE, respectivamente?
3. Suponga que se establece que $\langle G, n \rangle \in \text{CLIQUE}$. ¿Para qué valores de m automáticamente se tiene $\langle G, m \rangle \in \text{CLIQUE}$? Justifique su respuesta.
4. Hay una definición alternativa de la clase NP que se basa en algoritmos no determinísticos sobre máquinas de Turing. Investigue acerca de ella, enúnciela y explique por qué es equivalente a la de la Definición 8.2.3. El nombre NP proviene de la expresión inglesa *nondeterministic polynomial*.
5. En el Ejemplo 8.2.2 se demuestra que los lenguajes ARRFIND y ACYCLIC están en NP. En el espíritu de la demostración del Teorema 8.2.1, diseñe nuevas demostraciones para la pertenencia de estos dos lenguajes a NP en las cuales los algoritmos de verificación ignoren los certificados. El ejemplo menciona un tercer lenguaje, CLIQUE, para el cual la misma construcción no está disponible; la Sección 8.6 vuelve sobre el punto.
6. Demuestre que NP es cerrada bajo unión y bajo intersección. Ayuda: para la unión, un certificado puede indicar cuál de los dos lenguajes se pretende testificar; para la intersección, puede consistir en un par de certificados. Compare con el Teorema 8.1.2 y note cuál de las operaciones de aquel enunciado queda por fuera de este; la Sección 8.3 se ocupa de ese caso.
7. Precise el Teorema 8.2.2 determinando el orden de complejidad temporal del algoritmo que su demostración exhibe, en función de $|x|$, del polinomio p y del grado k del algoritmo de verificación. Concluya que todo lenguaje de NP se puede decidir en tiempo exponencial.
8. Considere el lenguaje de los números compuestos:

$$\text{COMPOSITE} = \{\langle N \rangle \mid \text{"}N \text{ no es primo y } N > 1\text{"}\}.$$

Demuestre que $\text{COMPOSITE} \in \text{NP}$ exhibiendo un certificado y justificando que la verificación es polinomial. Recuerde que el tamaño de la entrada es del orden de $\log N$, tal y como se advierte en el Ejemplo 8.1.1.

9. Sea $L \in \text{NP}$ y sean V y p como en la Definición 8.2.3. Construya un algoritmo de verificación polinomial V' que responda 0 cuando $|y| > p(|x|)$ y que coincida con V en los demás casos. Demuestre que $L = L(V')$ en el sentido de la Definición 8.2.2, con lo cual la cota queda absorbida dentro del verificador.

10. La Definición 8.2.3 mide el tiempo de verificación en función de $|x| + |y|$. Demuestre que medirlo en función de $|x|$ únicamente, conservando la cota sobre $|y|$, determina la misma clase NP. Ayuda: use que $|x| + |y| \leq |x| + p(|x|)$ y que la composición de dos polinomios es un polinomio. Compare este enunciado con la Nota 8.2.2 y note que lo que no se puede quitar es la cota, no la forma de medir el tiempo.
11. Exhiba algoritmos de verificación polinomiales para los dos lenguajes siguientes, con lo cual queda demostrado que ambos están en NP:
- a) el de los grafos que tienen un ciclo hamiltoniano, es decir, un ciclo que visita cada vértice exactamente una vez; y
 - b) el de los pares $\langle S, t \rangle$, con S un conjunto finito de números naturales y $t \in \mathbb{N}$, tales que algún subconjunto de S suma exactamente t .
- Los dos reaparecen más adelante en el capítulo.

8.3. La clase coNP

El Teorema 8.1.2 establece que P es cerrada bajo complemento: si un lenguaje se puede decidir en tiempo polinomial, basta invertir la respuesta para decidir su complemento en el mismo tiempo. La pregunta natural es si NP tiene la misma propiedad y la respuesta es que *nadie lo sabe*.

El obstáculo se ve al intentar el argumento. Un algoritmo de verificación para L certifica la pertenencia exhibiendo un certificado; para certificar la *no* pertenencia habría que convencer de que *ningún* certificado sirve y eso no es algo que un objeto finito exhiba de manera evidente. Es la misma asimetría entre aceptar y rechazar que aparece en la Nota 6.5.3, ahora con un presupuesto de tiempo de por medio.

Esta sección le da nombre a la clase que resulta de tomar complementos y muestra que el paralelo con el Capítulo 6 llega bastante más lejos de lo que parece.

8.3.1. Definición y ejemplos.

Definición 8.3.1: La clase coNP

Sea $L \subseteq \Sigma^*$ un lenguaje. Se dice que L está en la clase coNP si y solo si $\bar{L} \in \text{NP}$. Formalmente,

$$\text{coNP} = \{L \mid L \subseteq \Sigma^* \wedge \bar{L} \in \text{NP}\}.$$

Nota 8.3.1: coNP no es el complemento de NP

El nombre se presta a una confusión que conviene descartar de entrada. La clase **coNP** *no* es la colección de los lenguajes que no están en **NP**. Es la colección de los lenguajes *cuyo complemento* está en **NP**, que es cosa distinta. De hecho, las dos clases se traslapan. El Teorema 8.3.1 muestra que toda la clase **P** está contenida en ambas, de manera que la intersección $\mathbf{NP} \cap \mathbf{coNP}$ es, cuando menos, tan grande como **P**.

Ejemplo 8.3.1

Considere el lenguaje de las tautologías, introducido en el Ejemplo 6.4.1:

$$\mathbf{TAUT} = \{\langle \varphi \rangle \mid \text{"}\varphi \text{ es una proposición tautológica"}\}.$$

Se afirma que $\mathbf{TAUT} \in \mathbf{coNP}$, es decir, que $\overline{\mathbf{TAUT}} \in \mathbf{NP}$.

Hay dos razones por las cuales una cadena puede estar en $\overline{\mathbf{TAUT}}$: o no codifica proposición alguna o codifica una proposición que no es tautológica. Un algoritmo de verificación toma como certificado una asignación de valores de verdad a las variables y responde 1 cuando la entrada no codifica una proposición, o cuando la asignación del certificado hace falsa a la proposición codificada. Las dos comprobaciones son polinomiales: la primera es un análisis sintáctico y la segunda es evaluar una proposición bajo una asignación dada.

Ahora bien, no se sabe si $\mathbf{TAUT}$ está en **NP**. Para certificar que una proposición *sí* es tautológica habría que convencer de que las 2^n asignaciones posibles la hacen verdadera, y no se conoce ninguna manera de hacerlo en tiempo polinomial.

Ejemplo 8.3.2

La introducción de este capítulo menciona que la seguridad de las transacciones en línea descansa en no contar con algoritmos rápidos para descomponer números en factores primos. La versión de decisión de ese problema es:

$$\mathbf{FACTOR} = \{\langle N, k \rangle \mid \text{"}N \text{ tiene un factor } d \text{ con } 1 < d \leq k\}.$$

Este lenguaje está en **NP**: un certificado es el factor d , y verificar que $1 < d \leq k$ y que d divide a N es polinomial en el tamaño de la entrada.

Lo interesante es que también está en **coNP**. Un certificado de que N *no* tiene factores en ese rango es la descomposición completa $N = p_1 p_2 \cdots p_m$ en factores primos. Verificarlo consiste en comprobar que el producto da N , que cada p_i es primo, lo cual es polinomial por el Ejemplo 8.1.1, y que cada $p_i > k$.

Es decir, $\text{FACTOR} \in \text{NP} \cap \text{coNP}$, y sin embargo no se conoce ningún algoritmo polinomial que lo decida. Este es uno de los pocos problemas naturales en esa situación y es la razón por la cual se cree que no es NP-completo en el sentido de la Sección 8.5.

8.3.2. Propiedades. Los resultados de este apartado responden tres preguntas distintas. Conviene anticipar cuáles son, porque los enunciados son cortos y su importancia no salta a la vista.

¿Dónde queda P respecto de las dos clases nuevas? La respuesta es que queda dentro de las dos a la vez, y la razón es sencilla: un problema fácil lo es en los dos sentidos. Quien sabe decidir la pertenencia en tiempo polinomial sabe decidir también la no pertenencia, pues basta invertir la respuesta. Esa observación, que es todo el contenido del Teorema 8.3.1, es la que impide pensar en NP y coNP como dos mundos separados: comparten al menos todo P. La Figura 1 resume la situación.

¿Qué distingue entonces a una clase de la otra? El Teorema 8.3.2 responde que la diferencia está en un cuantificador. Pertenecer a NP significa que la respuesta afirmativa se certifica exhibiendo *algún* testigo; pertenecer a coNP significa que se certifica comprobando que *todos* los testigos funcionan. Esa lectura es útil en la práctica, porque permite ubicar un problema en una clase o en la otra con solo mirar cómo está formulada la pregunta: las que empiezan por “¿existe...?” suelen caer en NP y las que empiezan por “¿para todo...?” en coNP.

¿Y para qué sirve todo esto? El Teorema 8.3.3 y su corolario abren una ruta alternativa hacia la pregunta central del capítulo. En lugar de atacar directamente la igualdad $P = NP$, bastaría exhibir un solo lenguaje de NP cuyo complemento no esté en NP. Esa tarea parece más concreta que la original, y es una de las razones por las cuales el estudio de coNP no es un ejercicio de nomenclatura.

Teorema 8.3.1

$$P \subseteq \text{NP} \cap \text{coNP}.$$

Demostración

Sea $L \in P$. Que $L \in \text{NP}$ es el Teorema 8.2.1. Y como P es cerrada bajo complemento por la parte (1) del Teorema 8.1.2, se tiene $\bar{L} \in P$ y, aplicando de nuevo el Teorema 8.2.1, $\bar{L} \in \text{NP}$. Por la Definición 8.3.1 esto último significa que $L \in \text{coNP}$.

La caracterización que sigue explica en qué sentido NP y coNP son duales.

Teorema 8.3.2: Caracterización universal

Sea $L \subseteq \Sigma^*$. Entonces $L \in \text{coNP}$ si y solo si existe un algoritmo de verificación polinomial V tal que, para cualquier $x \in \Sigma^*$:

$$x \in L \iff V(x, y) = 1 \text{ para todo } y \in \Sigma^*.$$

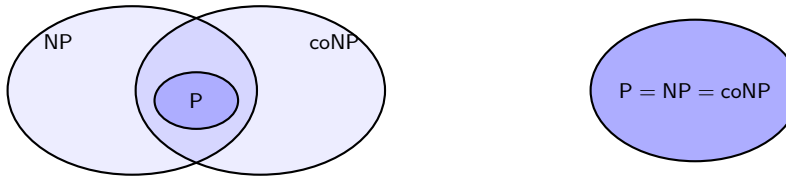
Demostración

Por la Definición 8.3.1, $L \in \text{coNP}$ si y solo si $\bar{L} \in \text{NP}$, es decir, si y solo si existe un algoritmo de verificación polinomial W tal que $x \in \bar{L}$ si y solo si $W(x, y) = 1$ para algún y .

Sea V el algoritmo que invierte la respuesta de W , es decir, $V(x, y) = 1$ si y solo si $W(x, y) = 0$. Es total y polinomial, pues W lo es. Entonces, para cualquier x :

$$\begin{aligned} x \in L &\iff x \notin \bar{L} && \text{(complemento)} \\ &\iff \text{no existe } y \text{ con } W(x, y) = 1 && (W \text{ verifica } \bar{L}) \\ &\iff W(x, y) = 0 \text{ para todo } y && \text{(negación)} \\ &\iff V(x, y) = 1 \text{ para todo } y && \text{(definición de } V\text{)}. \end{aligned}$$

El recíproco es el mismo cálculo recorrido en sentido contrario.



(a) lo que se sabe

(b) si $P = NP$

Figura 1. Contenencias conocidas entre P, NP y coNP. El panel (a) debe leerse con cuidado: ninguna de las contenencias que dibuja se sabe estricta, y tampoco se sabe si NP y coNP son distintas. El panel (b) muestra lo que ocurriría si $P = NP$, caso en el cual las tres clases coinciden por el Teorema 8.3.3.

Dicho en palabras, un lenguaje está en NP cuando la pertenencia se certifica exhibiendo *algún* testigo y está en coNP cuando se certifica verificando que *todos* los testigos son válidos. La diferencia entre las dos clases es, exactamente, la diferencia entre un cuantificador existencial y uno universal.

Teorema 8.3.3

Si $P = NP$, entonces $NP = \text{coNP}$.

Demostración

Suponga $P = NP$ y sea $L \in NP$. Entonces $L \in P$ y, por la parte (1) del Teorema 8.1.2, $\bar{L} \in P = NP$; luego $L \in \text{coNP}$ y se tiene $NP \subseteq \text{coNP}$.

Para la otra contención, sea $L \in \text{coNP}$. Entonces $\bar{L} \in NP = P$ y, de nuevo por la clausura bajo complemento, $L \in P = NP$.

Corolario 8.3.4

Si $NP \neq \text{coNP}$, entonces $P \neq NP$.

Demostración

Es el contrarrecíproco del Teorema 8.3.3.

El Corolario 8.3.4 señala una ruta posible hacia la respuesta de la pregunta central del capítulo: demostrar que NP y coNP son distintas bastaría para separar P de NP. Esa ruta tampoco ha dado resultado y hoy se cree que las dos clases son distintas sin que exista demostración.

8.3.3. Un paralelo que no cierra.

Nota 8.3.2: El análogo del teorema de Post

Vale la pena poner una al lado de la otra la situación de este capítulo y la de la Sección 6.5, porque la estructura es la misma y el desenlace no.

En aquella sección, los lenguajes computables resultaron cerrados bajo complemento y los computablemente enumerables no. La pregunta de cuándo

un lenguaje computablemente enumerable llega a ser computable quedó resuelta por el Teorema 6.5.3: ocurre exactamente cuando su complemento también es computablemente enumerable.

Aquí la estructura se repite. La clase P es cerrada bajo complemento y no se sabe si NP lo es. Y la pregunta análoga, la de si un lenguaje que está en NP junto con su complemento tiene que estar en P , es exactamente la pregunta de si $P = \text{NP} \cap \text{coNP}$.

La diferencia es que allá la respuesta es afirmativa y está demostrada, mientras que acá la pregunta lleva medio siglo abierta. El Ejemplo 8.3.2 muestra por qué no es una pregunta ociosa: FACTOR es un problema natural que vive en $\text{NP} \cap \text{coNP}$ y del cual no se conoce ningún algoritmo polinomial. Ese contraste resume bien la diferencia entre los dos capítulos. La teoría de la computabilidad llegó a respuestas definitivas; la teoría de la complejidad, con preguntas de la misma forma, todavía no.

Ejercicios

1. Demuestre que coNP es cerrada bajo unión y bajo intersección. Ayuda: use el ejercicio correspondiente de la Sección 8.2 junto con las leyes de De Morgan.
2. Demuestre que $L \in \text{NP}$ si y solo si $\bar{L} \in \text{coNP}$. Concluya que $\text{NP} = \text{coNP}$ si y solo si $\text{coNP} \subseteq \text{NP}$.
3. Determine si los siguientes lenguajes están en NP , en coNP , en ambas o si no se sabe, y justifique cada respuesta: $\emptyset$, Σ^* , SAT , $\overline{\text{SAT}}$ y el lenguaje de los grafos que *no* tienen un subgrafo completo de tamaño k .
4. Complete la demostración del Teorema 8.3.2 desarrollando el recíproco, es decir, suponiendo la existencia del algoritmo V con la propiedad universal y concluyendo que $\bar{L} \in \text{NP}$.
5. Suponga que se demuestra que algún lenguaje de NP no está en coNP . ¿Qué se puede concluir acerca de la pregunta P vs. NP ? ¿Y si se demostrara lo contrario, es decir, que todo lenguaje de NP está en coNP ?
6. En el Ejemplo 8.3.2 se afirma que verificar la descomposición en factores primos es polinomial. Explique por qué la cantidad de factores primos de N está acotada por $\log_2 N$ y por qué eso hace que el certificado tenga tamaño polinomial en el tamaño de la entrada.
7. Investigue acerca de la clase $\text{NP} \cap \text{coNP}$ y de por qué se considera poco probable que un problema NP -completo pertenezca a ella. Relacione su respuesta con la Sección 8.5.

8.4. Reducciones polinomiales

Una estrategia para resolver problemas consiste en transformar el problema de interés en otro para el cual exista una solución. Esa estrategia ya se usó a fondo en el Capítulo 7, donde la Definición 7.1.2 introduce la reducción computable y el Corolario 7.1.2 la convierte en la herramienta para transportar indecidibilidad.

Esta sección retoma esa misma noción y le agrega una exigencia. Allá bastaba con que la función que transforma las instancias fuera total y computable, porque la pregunta era si un problema se podía resolver del todo. Acá la pregunta es a qué costo, de manera que una transformación que tarde más que el problema que se quiere resolver no sirve de nada. La definición que sigue es, entonces, la de aquel capítulo con una condición adicional sobre el tiempo.

Definición 8.4.1: Reducción polinomial

Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$ problemas de decisión. Una *reducción polinomial* de L_1 a L_2 es una reducción $f : \Sigma^* \rightarrow \Sigma^*$ en el sentido de la Definición 7.1.2, es decir, una función total computable tal que para cualquier $x \in \Sigma^*$ se tiene

$$x \in L_1 \iff f(x) \in L_2,$$

y que además es computable en tiempo polinomial.

Se escribe $L_1 \leq_P L_2$ para indicar que hay una reducción polinomial de L_1 a L_2 .

Nota 8.4.1: Dos relaciones, una sola idea

La notación $\leq_P$ se escogió para que recuerde a la $\leq_c$ de la Sección 7.1, y la semejanza no es casual: toda reducción polinomial es, en particular, una reducción computable, de modo que $L_1 \leq_P L_2$ implica $L_1 \leq_c L_2$.

El recíproco no vale y ahí está todo el asunto de este capítulo. El Capítulo 7 construye reducciones sin preocuparse por el costo y de hecho varias de ellas escriben programas enteros a partir de sus entradas. Lo que se exige aquí es que la escritura misma sea barata.

Vale también la advertencia sobre la dirección de la Nota 7.1.2, que se aplica sin cambios: reducir L_1 a L_2 dice que L_1 no es más difícil que L_2 , de manera que la dificultad viaja hacia la derecha y la facilidad hacia la izquierda.

Una reducción polinomial de L_1 a L_2 es un algoritmo polinomial f que transforma instancias de L_1 a instancias de L_2 de tal manera que $x \in L_1$ y $f(x) \in L_2$ coincidan. Note, entonces, que la pregunta de si x pertenece o no a L_1 se puede

responder directamente averiguando la respuesta a la pregunta de si $f(x)$ pertenece o no a L_2 . Con una reducción de L_1 a L_2 , como f , nunca será el caso que $x \in L_1$ y $f(x) \notin L_2$ o que $x \notin L_1$ y $f(x) \in L_2$. De manera más general, una reducción preserva pertenencia y no pertenencia de sus imágenes en relación con sus respectivas preimágenes. Una reducción de L_1 a L_2 puede dibujarse como en la Figura 2.

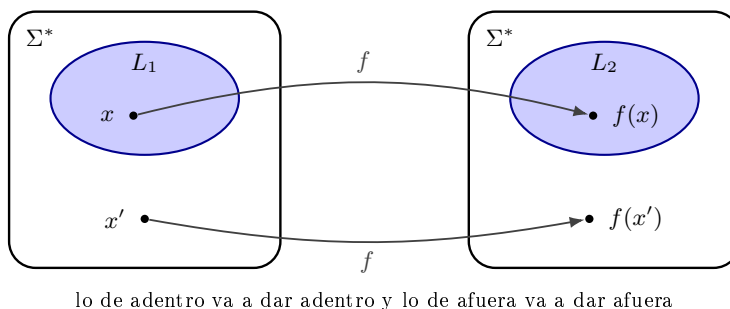


Figura 2. Asociaciones entre el lenguaje L_1 y el lenguaje L_2 cuando $L_1 \leq_P L_2$.

Note que no hay propiedades adicionales que se exijan a las reducciones. Por ejemplo, no deben ser, necesariamente, inyectivas o sobreyectivas en relación con los conjuntos de salida y llegada, como se explora en el Ejercicio 1.

Nota 8.4.2: Reducciones de Karp y Cook

A las reducciones presentadas en la Definición 8.4.1 se les conoce también como *reducciones de Karp* o *reducciones muchos a uno* en honor al científico de la computación Richard M. Karp. Hay otro tipo de reducciones, más generales, denominadas reducciones de Cook en honor al científico de la computación Stephen Cook (también son llamadas reducciones de Turing). Las reducciones de Cook se pueden definir para un grupo de problemas más general, como lo son los de optimización y construcción.

En este texto se prefieren las reducciones de Karp sobre las de Cook por dos razones principalmente. Primero, estas se formulan exclusivamente para reducciones entre problemas de decisión. Segundo, la clase NP es cerrada bajo reducciones de Karp.

Nota 8.4.3: Richard M. Karp y Stephen Cook

Richard M. Karp y Stephen Cook son reconocidos científicos de la computación.

Richard M. Karp recibió el premio Turing en 1985 por sus contribuciones a la teoría de algoritmos y complejidad. Estas contribuciones incluyen algoritmos eficientes para el problema de corte mínimo o flujo máximo en grafos (junto con Jack Edmonds), la identificación de computabilidad polinomial con la noción de eficiencia algorítmica y por sus contribuciones a la teoría de la NP-completitud. Por ejemplo, en 1972 publicó el artículo “Reducibility Among Combinatorial Problems” en donde demostró la existencia de 21 problemas NP-completos; este es uno de los artículos más citados en el área de algoritmos, computabilidad y complejidad. En 1973 publicó, junto con John Hopcroft, el algoritmo más eficiente hasta la fecha que resuelve el problema de encontrar la máxima cardinalidad de emparejamientos en grafos bipartitos, y en 1987, junto con Michael Rabin, un algoritmo eficiente para búsqueda en cadenas.

Stephen Cook es considerado uno de los padres fundadores de la teoría de complejidad computacional. En el artículo “The Complexity of Theorem Proving Procedures” que apareció en 1971, definió las nociones de reducción polinomial (à la Cook) y de NP-completitud, demostró que el problema de satisfacibilidad proposicional es NP-completo (resultado compartido con L. Levin) y formuló la pregunta de P vs. NP. Recibió el premio Turing en 1982.

Reducir un problema L_1 a otro L_2 significa diseñar un algoritmo que resuelva L_1 suponiendo que existe un algoritmo para resolver L_2 . Este tipo de enfoques se siguen desde, prácticamente, los primeros cursos de algoritmos y programación en donde se pueden hacer llamados desde funciones/procedimientos/módulos a otras funciones/procedimientos/módulos.

Las reducciones polinomiales permiten demostrar formalmente que un problema es al menos tan difícil como otro, módulo un factor de tiempo polinomial. Es decir, si $L_1 \leq_P L_2$, entonces L_1 no es más difícil que L_2 excepto por un factor polinomial. Por esta razón, principalmente, se usa la notación $\leq_P$.

Las reducciones polinomiales son una herramienta poderosa para demostrar que algunos problemas de decisión hacen parte de la clase P.

Teorema 8.4.1

Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$. Si $L_1 \leq_P L_2$ y $L_2 \in P$, entonces $L_1 \in P$.

Demostración

Se suponen $L_1 \leq_P L_2$ y $L_2 \in P$, y se demuestra $L_1 \in P$. Es decir, el objetivo es demostrar que existe un algoritmo, por ejemplo, A_1 que decide L_1 en tiempo polinomial. Por la primera suposición, existe una reducción polinomial $f : \Sigma^* \rightarrow \Sigma^*$ de L_1 a L_2 . Por la segunda suposición, hay un algoritmo, por ejemplo, A_2 que decide L_2 en tiempo polinomial. Considere el algoritmo de la Implementación 8.4.1, que calcula f y le entrega el resultado a A_2 .

Que A_1 corra en tiempo polinomial merece un comentario, porque no es tan inmediato como parece. Calcular $f(x)$ toma tiempo polinomial por hipótesis, pero el costo de la segunda invocación depende del tamaño de $f(x)$ y no del de x . Ahora bien, un algoritmo que corre en tiempo polinomial no alcanza a escribir más de una cantidad polinomial de símbolos, de modo que $|f(x)|$ está acotado por un polinomio en $|x|$. La invocación de A_2 toma entonces tiempo polinomial en $|x|$ y la composición de los dos pasos también.

Basta por demostrar que A_1 decide L_1 ; se procede por casos sobre $x \in \Sigma^*$:

($\Rightarrow$): Si $x \in L_1$, entonces $f(x) \in L_2$ dado que f es una reducción. Como A_2 decide L_2 , se tiene que $A_2(f(x)) = 1$. Es decir, $A_1(x) = 1$ cuando $x \in L_1$.

($\Leftarrow$): Si $x \notin L_1$, entonces $f(x) \notin L_2$ dado que f es una reducción. Como A_2 decide L_2 , se tiene que $A_2(f(x)) = 0$. Es decir, $A_1(x) = 0$ cuando $x \notin L_1$.

En conclusión, $L_1 \in P$.

Implementación 8.4.1: Composición de la reducción con el algoritmo

```

1 def decide(x):
2     return decide_L2(reduce(x))    # reduce computes f

```

El Teorema 8.4.1 se usa casi siempre en su forma contrarrecíproca, tal y como ocurre con el Teorema 7.1.1 en el capítulo anterior.

Corolario 8.4.2

Sean $L_1, L_2 \subseteq \Sigma^*$ tales que $L_1 \leq_P L_2$. Si $L_1 \notin P$, entonces $L_2 \notin P$.

Demostración

Es el contrarrecíproco del Teorema 8.4.1.

La clase NP se comporta de la misma manera y ese hecho es el que hace posible toda la Sección 8.5.

Teorema 8.4.3

Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$. Si $L_1 \leq_P L_2$ y $L_2 \in \text{NP}$, entonces $L_1 \in \text{NP}$.

Demostración

Se propone como ejercicio al lector. La construcción es análoga a la del Teorema 8.4.1, componiendo la reducción con el algoritmo de verificación de L_2 y usando el mismo certificado.

Nota 8.4.4

Las demostraciones que involucran reducciones polinomiales, generalmente siguen un mismo patrón. Para reducir un problema L_1 a un problema L_2 en tiempo polinomial, es necesario hacer tres cosas:

1. Diseñar un algoritmo, por ejemplo f , que en tiempo polinomial transforme cualquier cadena $x \in \Sigma^*$ en una cadena $f(x)$. Note que f debe estar definida sobre todas las cadenas y no únicamente sobre las que están en L_1 ; de lo contrario el tercer paso no tendría sentido.
2. Demostrar que si $x \in L_1$, entonces $f(x) \in L_2$.
3. Demostrar que si $f(x) \in L_2$, entonces $x \in L_1$.

Diseñar reducciones polinomiales y demostrar que en realidad funcionan es una tarea que demanda experiencia, por lo cual se sugiere al lector practicar.

El tercer paso en la Nota 8.4.4 puede ser confuso para algunos lectores: si ya se sabe que para cualquier $x \in L_1$ se cumple que $f(x) \in L_2$, ¿por qué es necesario este paso? La razón es que aún si se cumplen las condiciones (1) y (2) en la Nota 8.4.4, puede ser el caso que la reducción esté mal diseñada: es posible que $x \notin L_1$ y $f(x) \in L_2$. Es decir, que la reducción diseñada además de enviar instancias “buenas” de L_1 a instancias “buenas” de L_2 , también está enviando instancias “malas” de L_1 a instancias buenas de L_2 . De ser así, no se estaría respetando la definición de reducción.

Ejercicios

1. Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$, y $f : \Sigma^* \rightarrow \Sigma^*$ una reducción de L_1 a L_2 . Para cada uno de los siguientes casos, proponga testigos concretos para L_1 , L_2 y f tales que:
 - a) La reducción f es inyectiva.
 - b) La reducción f es sobreyectiva.
 - c) La reducción f es biyectiva.
 - d) La reducción f no es inyectiva.
 - e) La reducción f no es sobreyectiva.
 - f) La reducción f no es biyectiva.

¿En cuáles de los casos anteriores las reducciones son polinomiales?

2. Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$, y $f : \Sigma^* \rightarrow \Sigma^*$. Se define $f[L_1]$ como el conjunto imagen de L_1 bajo f , es decir,

$$f[L_1] = \{f(x) \mid x \in L_1\}.$$

Demuestre o refute:

- a) Si $f[L_1] \subseteq L_2$, entonces f es una reducción.
 - b) Si f es una reducción, entonces $f[L_1] \subseteq L_2$.
 - c) Si $f[L_1] = L_2$, entonces f es una reducción.
 - d) Si f es una reducción, entonces $f[L_1] = L_2$.
3. Demuestre que la relación $\leq_P$ es transitiva.
4. Demuestre que, para cualesquiera problemas de decisión L_1 y L_2 , la siguiente equivalencia es cierta:

$$L_1 \leq_P \overline{L_2} \iff \overline{L_1} \leq_P L_2.$$

5. Investigue acerca de los 21 problemas publicados por R. Karp en “Reducibility Among Combinatorial Problems”. Especifique 5 de ellos como problema de decisión.
6. Investigue acerca del problema de máxima cardinalidad de emparejamientos en grafos bipartitos y especifíquelo. Además, formule un ejemplo de una instancia del problema y una solución asociada.
7. Explique qué problema resuelve el algoritmo de Rabin y Karp para cadenas. Dé una breve explicación del algoritmo e ilustre su funcionamiento con un ejemplo sencillo.
8. Considere que está diseñando una calculadora proposicional: dada una fórmula, esta calculadora debe responder si la fórmula dada es satisfacible o no (i.e., si existe una asignación a sus variables proposicionales de tal manera que la

proposición resulte verdadera). El hardware de la calculadora solo permite construir proposiciones con base en negación ($\neg$) y disyunción ($\vee$). Sin embargo, la interfaz humano computador debe permitir analizar proposiciones que usen otros conectivos lógicos como equivalencia ($\equiv$), implicación ($\Rightarrow$) y conjunción ($\wedge$).

Como $\{\neg, \vee\}$ es un conjunto suficientemente completo de conectivos (i.e., cualquier proposición se puede expresar en términos de ellos), se entiende que la calculadora tiene futuro. Por ejemplo, la expresión $p \Rightarrow q$ (en donde p y q son variables proposicionales) se puede compilar en la proposición

$$(\neg p \vee q).$$

Algo similar puede hacerse para la equivalencia y la conjunción.

- a) Diseñe un algoritmo recurrente que dada una proposición ϕ la transforme en una proposición lógicamente equivalente, pero que solo usa negaciones y disyunciones, además de las mismas variables proposicionales.
 - b) ¿El algoritmo propuesto es una reducción?
 - c) ¿El algoritmo propuesto es una reducción polinomial?
9. Demuestre el Teorema 8.4.3, es decir, que si $L_1 \leq_P L_2$ y $L_2 \in \text{NP}$, entonces $L_1 \in \text{NP}$.
10. Enuncie y demuestre el resultado análogo al Teorema 8.4.3 para la clase coNP .

8.5. La clase NPC

La clase NPC es una colección nutrida de problemas prácticos en NP y que son intrínsecamente difíciles en el sentido en el cual no se conocen soluciones polinomiales (i.e., eficientes) para ellos. Una de sus principales propiedades radica en que si se encuentra una solución eficiente para cualquiera de ellos, entonces se tendría que $P = \text{NP}$. Sin embargo, se cree (pero no se ha demostrado) que no existen algoritmos eficientes para ninguno de ellos y, en consecuencia, $P \neq \text{NP}$. Esta sección presenta la clase NPC, una noción de “ser difícil” para los problemas en NP y un problema que hace parte de NPC.

En el área de la teoría de complejidad computacional, se identifica una colección de problemas que, informalmente, se pueden catalogar como al menos tan difíciles como cualquiera de los problemas más difíciles en NP. A esta clase se le denomina NP-hard.

Definición 8.5.1: NP-hard

Sea $L \subseteq \Sigma^*$. Se dice que L es NP-hard (léido, NP-difícil) si y solo si cualquier $L' \in \text{NP}$ es tal que $L' \leq_P L$. Alternativamente,

$$\text{NP-hard} = \{L \subseteq \Sigma^* \mid (\forall L' \in \text{NP}) L' \leq_P L\}.$$

Un problema NP-hard es un problema difícil relativamente en comparación con cualquier problema en NP (y, por ende, en P). Note que no necesariamente un problema de estos está obligado a ser parte de NP. Es decir, la definición de esta clase no excluye la posibilidad de que existan problemas difíciles fuera de NP.

Es importante llamar la atención del lector acerca de una dificultad práctica para el uso de la noción NP-hard: para que un problema de decisión sea admitido a esta clase, se debe demostrar que desde *cualquier* problema en NP hay una reducción polinomial a este. Como hay una cantidad infinita (¿contable o no contable?) de problemas en NP, ¿cómo lograr demostrar pertenencia a esta clase? Es más, no es obvio que esta clase no sea vacía. Afortunadamente, Stephen Cook (1971) y Leonid Levin (1973) de manera independiente demostraron que el problema CIRCUIT-SAT es NP-hard, uno de los principales logros de las ciencias de la computación (ver Nota 8.4.3), y que se conoce como el Teorema de Cook y Levin. Este teorema es acerca de la dificultad para resolver el problema de satisfacibilidad de circuitos (combinatorios) Booleanos.

Un *circuito combinatorio Booleano* es una colección no vacía de compuertas lógicas interconectadas por cables, en donde un cable conecta la salida de una compuerta con la entrada de otra, de tal forma que la salida de la primera se convierte en una de las entradas de la segunda. Los cables que no están conectados a la salida de compuerta alguna corresponden a las entradas del circuito y se exige que haya exactamente un cable que no alimente a ninguna compuerta, el cual es la salida del circuito. Se exigen además dos condiciones que las demostraciones de esta sección usan de manera esencial. La primera es que las conexiones no formen ciclos, lo cual es justamente lo que recoge el adjetivo *combinatorio* y es lo que permite propagar valores desde las entradas hasta la salida y hablar de la profundidad de una compuerta. La segunda es que las compuertas AND y OR tengan exactamente dos entradas, y la compuerta NOT exactamente una. Con base en esta caracterización (de alto nivel) de lo que es un circuito combinatorio Booleano, se puede definir el problema de satisfacibilidad de circuitos: dado un circuito combinatorio construido con compuertas AND, OR y NOT, ¿existe alguna asignación (encendido/apagado) para los cables de entrada del circuito que enciendan el cable de salida? El lenguaje CIRCUIT-SAT es, entonces, el siguiente:

$$\text{CIRCUIT-SAT} = \{\langle C \rangle \mid \text{"}C \text{ es un circuito combinatorio Booleano satisfacible"}\}.$$

Teorema 8.5.1: El Teorema de Cook y Levin

$\text{CIRCUIT-SAT} \in \text{NP-hard}$.

La demostración formal del Teorema 8.5.1 requiere detalles técnicos más allá del alcance de este manuscrito. Además, intencionalmente, se ha hecho un recuento vago del problema dado que no es importante entrar en tantos detalles.

Nota 8.5.1

No incluir una demostración del Teorema de Cook y Levin, no puede opacar su gran importancia. Gracias a la transitividad de las reducciones polinomiales, para demostrar que un problema L es NP-hard, basta con demostrar $\text{CIRCUIT-SAT} \leq_P L$.

El problema de satisfacibilidad de circuitos, CIRCUIT-SAT, surge naturalmente en el área de optimización de hardware: si un circuito siempre resulta apagado, entonces este puede ser eliminado del diseño y sustituido por una única compuerta que siempre resulte apagada. En la práctica es deseable resolver eficientemente este tipo de problemas.

La clase NPC caracteriza aquellos problemas en NP que son los más difíciles de resolver.

Definición 8.5.2: Problemas NP-completos

Sea $L \subseteq \Sigma^*$. Se dice que L es NP-completo si y solo si:

1. $L \in \text{NP}$.
2. $L \in \text{NP-hard}$.

La clase NPC es la colección de todos los problemas NP-completos:

$$\text{NPC} = \{L \subseteq \Sigma^* \mid "L \text{ es NP-completo}"\}.$$

Un problema es NP-completo si está en la intersección entre las clases NP y NP-hard. Una demostración de pertenencia a la clase NPC requiere dos comprobaciones: una, que el problema está en NP; otra, que el problema está en NP-hard.

Teorema 8.5.2

$\text{CIRCUIT-SAT} \in \text{NPC}$.

Demostración

Por la Definición 8.5.2 hay dos cosas que comprobar.

Que $\text{CIRCUIT-SAT} \in \text{NP}$ se obtiene con un algoritmo de verificación cuyo certificado es una asignación de valores a los cables de entrada. Verificarla consiste en propagar esos valores por el circuito, compuerta por compuerta, y comprobar que el cable de salida queda encendido. Cada compuerta se evalúa en tiempo constante y hay una cantidad de compuertas acotada por el tamaño de la entrada, de modo que la verificación es polinomial.

Que CIRCUIT-SAT sea NP-hard es el Teorema 8.5.1.

Como se muestra a continuación, la clase NPC suministra la esencia para atacar la pregunta P vs. NP : si existe un problema en NPC decidible eficientemente, necesariamente cualquier problema en NP puede ser decidido eficientemente.

Teorema 8.5.3

Si $\text{NPC} \cap \text{P} \neq \emptyset$, entonces $\text{P} = \text{NP}$.

Demostración

Se supone $\text{NPC} \cap \text{P} \neq \emptyset$ y se demuestra $\text{P} = \text{NP}$. Por el Teorema 8.2.1 se sabe que $\text{P} \subseteq \text{NP}$, luego basta con demostrar $\text{P} \supseteq \text{NP}$. Como $\text{NPC} \cap \text{P} \neq \emptyset$, hay un $L \subseteq \Sigma^*$ tal que $L \in \text{NPC}$ y $L \in \text{P}$. Sea $X \in \text{NP}$ cualquiera. Como L es NP-hard , se tiene $X \leq_{\text{P}} L$, y como $L \in \text{P}$, el Teorema 8.4.1 entrega directamente que $X \in \text{P}$.

Luego cualquier lenguaje en NP puede ser decidido en tiempo polinomial, es decir, $\text{P} \supseteq \text{NP}$. En conclusión, $\text{P} = \text{NP}$.

Basándose en el Teorema 8.5.3, durante los últimos 50 años se ha desarrollado una carrera estrepitosa por encontrar algoritmos eficientes para decidir problemas NP-completos . Además del noble interés por mover las fronteras del conocimiento humano, quizás también sea llamativo el premio de US \$1,000,000 ofrecido por el Instituto Clay de Matemáticas para quien resuelva la pregunta P vs. NP . Hasta la fecha, al menos públicamente, no se conoce una respuesta definitiva a esta pregunta, a pesar de que anualmente se publican “demostraciones” equivocadas, en cualquiera de las dos direcciones. En la comunidad científica hay una clara inclinación por creer que $\text{P} \neq \text{NP}$, pero sin una demostración no se puede dejar en firme dicha intuición.

Nota 8.5.2

Es importante retener la siguiente regla para trabajar con problemas NP-hard, y por lo tanto con problemas NP-completos:

Para demostrar que un problema de decisión L es NP-hard, basta reducir polinomialmente *algún* problema NP-hard ya conocido a L .

La palabra *algún* es esencial y conviene no confundirla con *cualquiera*. Reducir todos los problemas de NP a L es la definición, no el método; lo que el método aprovecha es la transitividad de $\leq_P$, que hace que baste con un solo problema.

Note que el cuantificador recorre NP y no los problemas NP-hard, y que la diferencia no es menor. Exigir que todos los problemas NP-hard se reduzcan a L sería una condición que ningún problema NP-completo cumple, pues HALT es NP-hard (ver Ejercicio 1) y reducirlo polinomialmente a un lenguaje decidable lo volvería decidable, en contra del Teorema 7.2.1. Y vale de nuevo la advertencia sobre la dirección de la Nota 7.1.2: la reducción va *desde* el problema que ya se sabe difícil *hacia* el que se quiere clasificar.

8.5.1. Un problema NP-completo en forma normal conjuntiva. Con el Teorema 8.5.2 se obtiene el primer problema NP-completo y con él la Nota 8.5.2 queda en condiciones de usarse. Sin embargo, CIRCUIT-SAT no es un punto de partida cómodo: reducir desde circuitos hacia problemas de grafos o de aritmética obliga a lidiar con la estructura de las compuertas en cada construcción.

Esta subsección obtiene un segundo problema NP-completo, mucho más manejable, que es el que servirá de semilla en el resto del capítulo.

Definición 8.5.3: Forma normal conjuntiva

Un *literal* es una variable proposicional o su negación. Una *cláusula* es una disyunción de literales. Una proposición está en *forma normal conjuntiva* si es una conjunción de cláusulas y está en *forma normal conjuntiva de tres literales* si además cada una de sus cláusulas tiene exactamente tres literales.

El problema de satisfacibilidad correspondiente es:

$$3\text{CNFSAT} = \{ \langle \varphi \rangle \mid \text{"}\varphi \text{ está en forma normal conjuntiva de tres literales y es satisfacible"} \}.$$

Ejemplo 8.5.1

La proposición

$$(p \vee \neg q \vee r) \wedge (\neg p \vee q \vee q)$$

está en forma normal conjuntiva de tres literales y es satisfacible; la asignación que hace verdaderas a p y a q la satisface. Note que la segunda cláusula repite el literal q , lo cual es admisible y resulta cómodo para completar cláusulas cortas.

Teorema 8.5.4

$3\text{CNFSAT} \in \text{NP}$.

Demostración

El certificado es una asignación de valores de verdad a las variables de φ . Verificarla consiste en evaluar cada cláusula y comprobar que todas resultan verdaderas, lo cual toma tiempo lineal en el tamaño de φ . Además, comprobar que la entrada codifica una proposición en la forma requerida es un análisis sintáctico, también polinomial.

El resultado que sigue es la pieza que faltaba.

Teorema 8.5.5

$\text{CIRCUIT-SAT} \leq_P 3\text{CNFSAT}$.

Antes de la demostración conviene anticipar la idea, porque la construcción es corta pero su motivación no es evidente.

El camino ingenuo sería escribir la proposición que describe la salida del circuito en términos de sus entradas y luego llevarla a forma normal conjuntiva. Ese camino no sirve: cuando la salida de una compuerta alimenta a varias, su descripción se repite, y la proposición resultante puede crecer de manera exponencial.

El truco consiste en no eliminar las compuertas intermedias sino en *darles nombre*. Se introduce una variable proposicional por cada cable del circuito y para cada compuerta se escriben unas pocas cláusulas que dicen “la variable de mi cable de salida vale lo que la compuerta calcula a partir de mis entradas”. Cada compuerta aporta una cantidad constante de cláusulas, de modo que el tamaño total crece linealmente.

Demostración

Sea C un circuito con compuertas $g_1, \dots, g_m$. Se asocia una variable proposicional a cada cable de C , tanto a los de entrada como a los de salida de cada compuerta.

Para cada compuerta se escriben las cláusulas siguientes, en las cuales z denota la variable del cable de salida de la compuerta y a y b las de sus cables de entrada. Que las compuertas AND y OR tengan exactamente dos entradas es lo que hace que ninguna de esas cláusulas pase de tres literales, es decir, lo que permite que el blanco de la reducción sea 3CNFSAT y no CNFSAT:

Compuerta NOT: $(z \vee a) \wedge (\neg z \vee \neg a)$.

Compuerta AND: $(\neg z \vee a) \wedge (\neg z \vee b) \wedge (z \vee \neg a \vee \neg b)$.

Compuerta OR: $(z \vee \neg a) \wedge (z \vee \neg b) \wedge (\neg z \vee a \vee b)$.

En los tres casos las cláusulas son satisfechas exactamente por las asignaciones en las cuales z toma el valor que la compuerta calcula a partir de a y b ; verificarlo es revisar las cuatro filas de la tabla de verdad correspondiente.

Sea w la variable del cable de salida del circuito. La proposición $f(C)$ es la conjunción de todas las cláusulas anteriores junto con la cláusula (w) . Finalmente, las cláusulas con menos de tres literales se completan repitiendo uno de los suyos, lo cual no altera su valor de verdad, tal y como se ilustra en el Ejemplo 8.5.1.

La función f es computable en tiempo polinomial: recorre las compuertas una sola vez y escribe a lo sumo tres cláusulas por cada una, de modo que $|f(C)|$ crece linealmente con el tamaño de C . Sobre las cadenas que no codifican circuito alguno, f retorna una proposición fija no satisfacible.

Resta verificar la condición de la Definición 8.4.1.

($\Rightarrow$): Suponga que C es satisfacible y sea α una asignación de sus cables de entrada que enciende la salida. Extienda α asignando a la variable de cada cable interno el valor que ese cable toma al propagar α por el circuito. Las cláusulas de cada compuerta quedan satisfechas por construcción y la cláusula (w) también, pues la salida queda encendida. Luego $f(C)$ es satisfacible.

($\Leftarrow$): Suponga que $f(C)$ es satisfacible y sea β una asignación que la satisface. La restricción de β a las variables de los cables de entrada define una asignación del circuito. Procediendo por inducción sobre la profundidad de las compuertas y usando que β satisface las cláusulas de cada una, se obtiene que el valor de β sobre la variable de cada cable coincide con el

valor que ese cable toma al propagar la asignación. Como β satisface la cláusula (w), la salida del circuito queda encendida y C es satisfacible.

Corolario 8.5.6

$3\text{CNFSAT} \in \text{NPC}$.

Demostración

Que $3\text{CNFSAT} \in \text{NP}$ es el Teorema 8.5.4. Para la dureza, sea $L \in \text{NP}$ cualquiera. El Teorema 8.5.1 entrega $L \leq_P \text{CIRCUIT-SAT}$ y el Teorema 8.5.5 entrega $\text{CIRCUIT-SAT} \leq_P 3\text{CNFSAT}$. Como la relación $\leq_P$ es transitiva, se concluye $L \leq_P 3\text{CNFSAT}$.

Nota 8.5.3: La semilla del resto del capítulo

El Corolario 8.5.6 es el que hace posible lo que sigue. De aquí en adelante, demostrar que un problema es NP-hard se reduce a exhibir una reducción polinomial desde 3CNFSAT , o desde cualquier otro problema que ya se haya clasificado de esa manera.

Las tres secciones siguientes hacen exactamente eso, encadenando una reducción tras otra. Vale la pena notar que solo el Teorema 8.5.1 se cita sin demostrar; todo lo demás se construye a partir de él.

Ejercicios

1. Sobre los problemas NP-hard que no hacen parte de NP:
 - a) Demuestre que HALT es NP-hard. Ayuda: sea $X \in \text{NP}$ y sea D un decisor para X , que existe por el Teorema 8.2.2; modifíquelo para que entre en un ciclo infinito cuando iba a responder 0.
 - b) Demuestre que HALT no se reduce polinomialmente a ningún lenguaje decidable. Concluya que ningún problema NP-completo admite que todos los problemas NP-hard se le reduzcan.
 - c) Investigue acerca de otro problema NP-hard fuera de NP, especifíquelo y dé un ejemplo breve de una de sus instancias.
2. ¿El cardinal $\text{card}(\text{NP})$ del conjunto NP es contable o no contable? Justifique su respuesta.

3. Verifique las tres familias de cláusulas de la demostración del Teorema 8.5.5 construyendo, para cada tipo de compuerta, la tabla de verdad correspondiente.
4. Sea CNFSAT el problema de satisfacibilidad de proposiciones en forma normal conjuntiva, sin restricción sobre la cantidad de literales por cláusula. Demuestre que $3\text{CNFSAT} \leq_P \text{CNFSAT}$ y concluya que $\text{CNFSAT} \in \text{NPC}$.
5. Considere el problema 2CNFSAT , idéntico a 3CNFSAT salvo que cada cláusula tiene exactamente dos literales. Investigue en qué clase se encuentra y explique por qué la demostración del Teorema 8.5.5 no se puede adaptar para reducir CIRCUIT-SAT a él.
6. Explique por qué el camino ingenuo mencionado antes de la demostración del Teorema 8.5.5, el de escribir la salida del circuito en términos de sus entradas, puede producir una proposición de tamaño exponencial. Ayuda: considere un circuito en el cual la salida de cada compuerta alimenta a dos compuertas de la capa siguiente.
7. Sea $L \in P$ un lenguaje con $L \neq \emptyset$ y $L \neq \Sigma^*$. Demuestre que L es NP-hard si y solo si $P = \text{NP}$.
 - a) Para la vuelta, adapte el Ejemplo 7.1.2 a reducciones polinomiales, usando que L tiene al menos una cadena adentro y al menos una afuera.
 - b) Demuestre que $\emptyset$ y Σ^* no son NP-hard, sin suponer nada acerca de P y NP . Concluya que las dos exclusiones del enunciado no se pueden quitar.
 - c) Explique por qué el enunciado se restringe a lenguajes de P . Ayuda: para un $L \notin P$, la implicación “si $L \in P$ entonces $P = \text{NP}$ ” se cumple sin que L tenga que ser NP-hard.
8. Investigue y enuncie los problemas por los cuales actualmente el Instituto Clay de Matemáticas ofrece premios de US \$1,000,000.
9. Un lenguaje L es *completo* para una clase de lenguajes C con respecto a reducciones polinomiales si y solo si $L \in C$ y $L' \leq_P L$ para cualquier $L' \in C$. Demuestre que $\emptyset$ y Σ^* son los únicos lenguajes en P que no son completos para P con respecto a reducciones polinomiales.

8.6. Clique es NP-completo

Esta es la primera aplicación de la receta de la Nota 8.5.2 con 3CNFSAT como semilla y es además la que salda la deuda de la introducción del capítulo. El problema del grupo de amigos del Problema 8.0.1, el que el recién graduado se negó a resolver en un par de meses, es exactamente el problema de hallar el subgrafo completo más grande de un grafo. Su versión de decisión es el lenguaje CLIQUE del Ejemplo 8.2.2:

$$\text{CLIQUE} = \{ \langle G, k \rangle \mid \text{“}G \text{ tiene un subgrafo completo de tamaño } k\text{”} \}.$$

Que CLIQUE $\in$ NP ya está establecido en el Ejemplo 8.2.2: el certificado es el conjunto de vértices y verificarlo consiste en comprobar que todos los pares están unidos por un arco. Falta establecer CLIQUE $\in$ NP-hard.

Satisfacer una proposición en forma normal conjuntiva consiste en escoger, de cada cláusula, un literal al cual asignarle verdadero. Esa elección está sujeta a una única restricción: no se pueden escoger a la vez un literal y su negación, porque no hay asignación que haga verdaderos a los dos. La construcción que sigue traduce esta intención al lenguaje de los grafos. Cada ocurrencia de un literal se vuelve un vértice y dos vértices se unen exactamente cuando escogerlos a ambos es admisible. Escoger un literal por cláusula, de manera compatible, se convierte entonces en hallar un subgrafo completo con tantos vértices como cláusulas tiene la proposición.

Conviene separar dos asuntos que suelen presentarse juntos. Primero se define el mapa que transforma instancias, sin afirmar todavía nada acerca de él; después se demuestra que ese mapa es, en efecto, una reducción polinomial. La Definición 8.6.1 se ocupa de lo primero y el Teorema 8.6.1 de lo segundo.

Definición 8.6.1: El mapa de cláusulas a grafos

Sea $f : \Sigma^* \rightarrow \Sigma^*$ el mapa definido de la siguiente manera.

Si w codifica una proposición $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ en forma normal conjuntiva de tres literales, con $C_i = (\ell_{i1} \vee \ell_{i2} \vee \ell_{i3})$, entonces $f(w) = \langle G, k \rangle$, donde $G = (V, E)$ es el grafo:

- V tiene un vértice v_{ir} por cada ocurrencia de literal, es decir, para $1 \leq i \leq k$ y $1 \leq r \leq 3$; y
- E une a v_{ir} con v_{js} si y solo si $i \neq j$ y los literales ℓ_{ir} y ℓ_{js} no son complementarios, es decir, no son una variable y su negación.

Si w no codifica una proposición de esa forma, entonces $f(w)$ es un par fijo que no está en CLIQUE.

De la Definición 8.6.1 se desprende de inmediato una propiedad que conviene tener presente, pues se usa en las dos direcciones de la demostración: dos vértices provenientes de una misma cláusula nunca están unidos. En consecuencia, cualquier subgrafo completo de G tiene a lo sumo un vértice por cláusula y, en particular, uno de tamaño k tiene exactamente uno de cada una.

Teorema 8.6.1

El mapa f de la Definición 8.6.1 es una reducción polinomial de 3CNFSAT a CLIQUE. En consecuencia, $3\text{CNFSAT} \leq_P \text{CLIQUE}$.

Demostración

Hay dos cosas que verificar: que f es computable en tiempo polinomial y que cumple la condición de la Definición 8.4.1.

Lo primero es directo. Reconocer si w codifica una proposición de la forma requerida es un análisis sintáctico y, en caso afirmativo, el grafo tiene $3k$ vértices y para cada uno de los pares, que son menos de $(3k)^2$, se decide en tiempo constante si va o no un arco. Como k no supera el tamaño de w , el trabajo total es polinomial.

Para lo segundo, sea w una cadena cualquiera. Si w no codifica una proposición de la forma requerida, entonces $w \notin 3\text{CNFSAT}$ y $f(w) \notin \text{CLIQUE}$ por construcción. Resta el caso en que w codifica una proposición φ , y se procede por doble implicación.

($\Rightarrow$): Suponga que φ es satisfacible y sea α una asignación que la satisface. Cada cláusula C_i tiene al menos un literal verdadero bajo α ; escoja uno y llámelo ℓ_{ir_i} .

Los k vértices $v_{1r_1}, \dots, v_{kr_k}$ están en cláusulas distintas y no hay dos de ellos complementarios, pues α los hace verdaderos a todos, y ninguna asignación hace verdaderos a un literal y a su negación. Luego todos los pares están unidos y los k vértices forman un subgrafo completo. Por lo tanto, $f(w) = \langle G, k \rangle \in \text{CLIQUE}$.

($\Leftarrow$): Suponga que G tiene un subgrafo completo de tamaño k . Por la propiedad señalada arriba, ese subgrafo tiene exactamente un vértice por cláusula, digamos $v_{1r_1}, \dots, v_{kr_k}$.

Considere la asignación que hace verdadero a cada literal ℓ_{ir_i} , extendida de cualquier manera sobre las variables restantes. Esta asignación está bien definida: si dos de los literales escogidos exigieran valores opuestos para una misma variable, serían complementarios y sus vértices no estarían unidos, contradiciendo que forman un subgrafo completo.

Bajo esa asignación cada cláusula C_i tiene al menos un literal verdadero, a saber ℓ_{ir_i} , de modo que φ resulta satisfecha y $w \in 3\text{CNFSAT}$.

Corolario 8.6.2

$\text{CLIQUE} \in \text{NPC}$.

Demostración

La pertenencia a NP es el Ejemplo 8.2.2. Para la dureza, el Corolario 8.5.6 establece que 3CNFSAT es NP-hard, y el Teorema 8.6.1 entrega:

$$3\text{CNFSAT} \leq_P \text{CLIQUE}.$$

La transitividad de $\leq_P$ da el resultado.

Conviene ahora poner a trabajar la Definición 8.6.1 sobre una instancia concreta. Seguir la construcción paso a paso, sobre una proposición pequeña, es la mejor manera de retenerla: el grafo que resulta es más fácil de recordar dibujado que enunciado y de paso deja ver por qué faltan justamente los arcos que faltan.

Ejemplo 8.6.1

Considere la proposición

$$\varphi = (p \vee q \vee r) \wedge (\neg p \vee \neg q \vee r) \wedge (\neg p \vee q \vee \neg r),$$

que tiene $k = 3$ cláusulas. El grafo que construye la reducción aparece en la Figura 3: tiene nueve vértices, agrupados de tres en tres según la cláusula de la cual provienen, y veintidós arcos.

Note que dentro de cada grupo no hay arcos y que entre grupos distintos faltan exactamente los seis arcos que unirían literales complementarios; por ejemplo, el p de C_1 no está unido con el $\neg p$ de C_2 ni con el $\neg p$ de C_3 .

El subgrafo completo resaltado en la figura está formado por el p de C_1 , el $\neg q$ de C_2 y el $\neg r$ de C_3 . La asignación correspondiente hace verdadero a p , falsos a q y a r , y en efecto satisface a φ : la primera cláusula por p , la segunda por $\neg q$ y la tercera por $\neg r$.

Nota 8.6.1: De vuelta a la entrevista

El Corolario 8.6.2 explica lo que ocurrió en la entrevista con que abre este capítulo.

El problema del grupo de amigos es CLIQUE, de modo que comprometerse a resolverlo eficientemente equivale, por el Teorema 8.5.3, a comprometerse a demostrar que $P = NP$. Es decir, el recién graduado no se negó por falta de

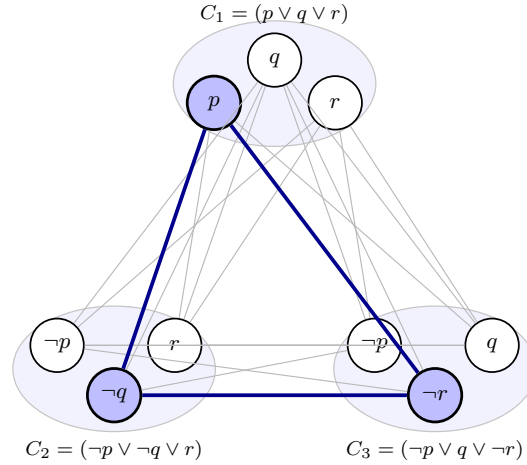


Figura 3. El grafo que la reducción del Teorema 8.6.1 construye a partir de la proposición del Ejemplo 8.6.1. Cada grupo corresponde a una cláusula y dentro de un grupo no hay arcos. El subgrafo completo resaltado entrega una asignación que satisface la proposición.

capacidad ni de tiempo: se negó porque la tarea incluía, sin anunciarlo, resolver una de las preguntas abiertas más importantes de las ciencias exactas.

Vale la pena notar qué tan lejos está ese hecho de ser evidente. La cadena que lo establece pasa por el teorema de Cook y Levin, por la reducción a 3CNFSAT de la Sección 8.5 y por la reducción de esta sección. Nada de eso se ve al leer el enunciado del Problema 8.0.1, y por eso reconocerlo es una destreza profesional y no una curiosidad teórica.

Ejercicios

1. Construya el grafo que la reducción del Teorema 8.6.1 produce a partir de la proposición $(p \vee \neg q \vee r) \wedge (\neg p \vee q \vee \neg r)$. Determine si tiene un subgrafo completo de tamaño 2 y, en caso afirmativo, exhiba la asignación correspondiente.
2. Aplique la reducción a una proposición no satisfacible de su elección y verifique que el grafo resultante no tiene un subgrafo completo del tamaño requerido.
3. En la demostración del Teorema 8.6.1 se observa que dos vértices de una misma cláusula nunca están unidos. Explique dónde se usa esa observación en cada una de las dos direcciones de la equivalencia.
4. La construcción no exige que los tres literales de una cláusula sean distintos. Verifique que la demostración sigue siendo válida cuando una cláusula repite un literal, como ocurre con las cláusulas completadas en la Sección 8.5.
5. Considere el problema de determinar si un grafo tiene un subgrafo completo de tamaño a *lo sumo* k . Determine en qué clase se encuentra y justifique. Compare con CLIQUE y explique la diferencia.
6. Un *conjunto independiente* de un grafo es un conjunto de vértices entre los cuales no hay ningún arco. Sea

$$\text{INDSET} = \{ \langle G, k \rangle \mid \text{"}G \text{ tiene un conjunto independiente de tamaño } k\text{"} \}.$$

Demuestre que $\text{CLIQUE} \leq_P \text{INDSET}$ y concluya que INDSET es NP-completo. Ayuda: considere el grafo complementario.

7. El Ejercicio 5 de la Sección 8.2 pide, para ARRFIND y ACYCLIC, algoritmos de verificación que ignoren el certificado, en el espíritu de la demostración del Teorema 8.2.1. El Ejemplo 8.2.2 menciona un tercer lenguaje, CLIQUE, que allá queda por fuera.
 - a) Explique por qué esa construcción no está disponible para CLIQUE, precisando qué habría que exhibir para llevarla a cabo.
 - b) Suponga que alguien la lleva a cabo. ¿Qué quedaría demostrado? Responda primero sin usar nada de esta sección y después usando el Corolario 8.6.2 y el Teorema 8.5.3, y compare las dos respuestas.
 8. El Problema 8.0.1 pide el tamaño *máximo* de un subgrafo completo, que es un problema de optimización y no de decisión. Explique, usando la Sección 6.1, cómo se resuelve ese problema consultando repetidamente a CLIQUE y calcule cuántas consultas hacen falta.
 9. Investigue acerca del tamaño de los grafos para los cuales los algoritmos actuales logran resolver CLIQUE en la práctica y compárelo con el tamaño de una red social real.
-

8.7. VCOVER es NP-completo

La sección anterior obtuvo su resultado reduciendo desde 3CNFSAT. Esta reduce desde CLIQUE, con lo cual se ilustra algo que la Nota 8.5.2 permite y que conviene ver funcionando: una vez un problema queda clasificado como NP-hard, pasa a ser un punto de partida tan legítimo como el original.

El problema de esta sección aparece con frecuencia en aplicaciones. Dado un grafo, se busca un conjunto pequeño de vértices que toque todos sus arcos. Por ejemplo, ubicar cámaras en las esquinas de una ciudad de manera que cada calle quede vigilada, o escoger un conjunto pequeño de servidores que cubra todos los enlaces de una red.

Definición 8.7.1: Cobertura de vértices

Sea $G = (V, E)$ un grafo. Un conjunto $C \subseteq V$ es una *cobertura de vértices* de G si y solo si todo arco de E tiene al menos uno de sus dos extremos en C . El problema de decisión asociado es el lenguaje:

$$\text{VCover} = \{ \langle G, k \rangle \mid \text{"}G \text{ tiene una cobertura de tamaño } k\text{"} \}.$$

Teorema 8.7.1

$\text{VCover} \in \text{NP}$.

Demostración

El certificado es el conjunto C de vértices. Verificarlo consiste en comprobar que C tiene k elementos y que cada arco de G tiene un extremo en C , lo cual toma tiempo lineal en el tamaño de la entrada.

Falta establecer $\text{VCover} \in \text{NP-hard}$. La conexión con CLIQUE descansa en una observación sencilla que conviene aislar, pues es la que hace todo el trabajo.

Teorema 8.7.2: Coberturas y conjuntos independientes

Sea $G = (V, E)$ un grafo y sea $C \subseteq V$. Entonces C es una cobertura de vértices de G si y solo si entre los vértices de $V \setminus C$ no hay ningún arco de E .

Demostración

Las dos condiciones son negaciones de la misma afirmación. Que C no sea una cobertura significa que algún arco tiene sus dos extremos fuera de C , es decir, que hay un arco entre dos vértices de $V \setminus C$. Negando ambos lados se obtiene el enunciado.

Con esa observación, la reducción se deja adivinar. Un subgrafo completo de G es un conjunto de vértices en el cual *todos* los pares están unidos; un conjunto sin arcos internos es uno en el cual *ningún* par lo está. Pasar de lo uno a lo otro es cuestión de invertir los arcos del grafo, es decir, de tomar el grafo complementario. Y el Teorema 8.7.2 traduce “sin arcos internos” en “complemento es cobertura”. El Ejercicio 6 de la Sección 8.6 pide construir la primera mitad de esta idea; la reducción en la Definición 8.7.2 la completa.

Definición 8.7.2: El mapa del grafo complementario

Sea $G = (V, E)$ un grafo. El *grafo complementario* de G , denotado $\overline{G}$, es el grafo con los mismos vértices V en el cual dos vértices distintos están unidos si y solo si no lo están en G .

Sea $g : \Sigma^* \rightarrow \Sigma^*$ el mapa definido de la siguiente manera. Si w codifica un par $\langle G, k \rangle$ con $G = (V, E)$ un grafo de n vértices y $0 \leq k \leq n$, entonces:

$$g(w) = \langle \overline{G}, n - k \rangle.$$

Si w no codifica un par de esa forma, entonces $g(w)$ es un par fijo que no está en VCOVER.

Teorema 8.7.3

El mapa g de la Definición 8.7.2 es una reducción polinomial de CLIQUE a VCOVER. En consecuencia, $\text{CLIQUE} \leq_P \text{VCOVER}$.

Demostración

Hay dos cosas que verificar.

Que g es computable en tiempo polinomial es directo: construir $\overline{G}$ consiste en recorrer los menos de n^2 pares de vértices y decidir para cada uno, en tiempo constante, si va o no un arco.

Para la condición de la Definición 8.4.1, sea w una cadena cualquiera. Si w no codifica un par de la forma requerida, entonces $w \notin \text{CLIQUE}$ y $g(w) \notin \text{VCOVER}$ por construcción. Resta el caso en que w codifica $\langle G, k \rangle$ con G de n vértices y $0 \leq k \leq n$.

Sea $S \subseteq V$ un conjunto de k vértices. Entonces:

$$\begin{aligned} S \text{ es completo en } G &\iff \text{todo par de } S \text{ está unido en } G \\ &\iff \text{ningún par de } S \text{ lo está en } \overline{G} \\ &\iff V \setminus S \text{ cubre a } \overline{G}, \end{aligned}$$

donde la primera equivalencia es la definición de subgrafo completo, la segunda es la definición de $\overline{G}$ y la tercera es el Teorema 8.7.2.

Como $\text{card}(V \setminus S) = n - k$, la cadena de equivalencias dice que G tiene un subgrafo completo de tamaño k si y solo si $\overline{G}$ tiene una cobertura de vértices de tamaño $n - k$. Es decir, $w \in \text{CLIQUE}$ si y solo si $g(w) \in \text{VCOVER}$.

Corolario 8.7.4

$\text{VCOVER} \in \text{NPC}$.

Demostración

La pertenencia a NP es el Teorema 8.7.1. Para la dureza, el Corolario 8.6.2 establece que CLIQUE es NP-hard y el Teorema 8.7.3 entrega:

$$\text{CLIQUE} \leq_P \text{VCOVER}.$$

La transitividad de $\leq_P$ da el resultado.

Conviene seguir la Definición 8.7.2 sobre un grafo concreto. Conviene el mismo del Ejemplo 8.2.2, porque se puede comparar gráficamente lo que allá se buscaba con lo que acá se obtiene.

Ejemplo 8.7.1

Considere el grafo G de la Figura 4, cuyos vértices son a, b, c y d y cuyos arcos son (a, b) , (a, c) , (a, d) y (b, c) . Es el mismo del Ejemplo 8.2.2, donde se observa que $\langle G, 3 \rangle \in \text{CLIQUE}$ gracias al subgrafo completo $\{a, b, c\}$.

Como G tiene $n = 4$ vértices y $k = 3$, la reducción entrega $\langle \overline{G}, 1 \rangle$. El grafo complementario tiene únicamente los arcos (b, d) y (c, d) , pues esos son los dos pares que no estaban unidos en G .

En efecto, $\{d\}$ es una cobertura de vértices de $\overline{G}$ de tamaño 1: los dos arcos tocan a d . Y es exactamente el complemento del subgrafo completo de partida, es decir, $\{a, b, c, d\} \setminus \{a, b, c\}$.

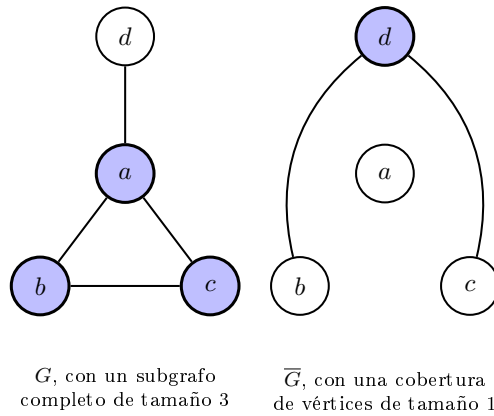


Figura 4. El grafo del Ejemplo 8.7.1 y su complementario. Los vértices resaltados a la izquierda forman un subgrafo completo de tamaño 3; el resaltado a la derecha es el complemento de ese conjunto y constituye una cobertura de vértices de tamaño 1.

Nota 8.7.1: Reducciones baratas y reducciones caras

Vale la pena comparar el esfuerzo de las dos últimas secciones.

La reducción de la Sección 8.6 tuvo que inventar un grafo a partir de una proposición, que son objetos de naturaleza distinta, y la construcción exigió decidir qué representaba cada vértice y cada arco. La de esta sección se limita a invertir los arcos y a restar dos números.

La diferencia no es casual. Cuando el problema de partida y el de llegada hablan de los mismos objetos, la reducción suele reducirse a una observación como la del Teorema 8.7.2. Cuando cambian de dominio, hay que construir. Por eso conviene escoger con cuidado desde cuál problema reducir: la lista de

problemas NP-completos conocidos es larga y casi siempre hay uno que habla del mismo tipo de objetos que el problema por clasificar.

Ejercicios

1. Aplique la Definición 8.7.2 al par $\langle G, 2 \rangle$, con G el grafo del Ejemplo 8.7.1. Determine si el resultado está en VCOVER y verifique que la respuesta coincide con la de $\langle G, 2 \rangle$ respecto de CLIQUE.
 2. La Definición 8.7.2 exige $0 \leq k \leq n$. Explique qué sale mal si se omite esa condición y cómo la trata el mapa definido.
 3. Demuestre que $\text{VCOVER} \leq_P \text{CLIQUE}$, es decir, que la reducción también funciona en la dirección contraria. Concluya que los dos problemas son igual de difíciles respecto de $\leq_P$.
 4. Un *conjunto dominante* de un grafo es un conjunto D de vértices tal que todo vértice fuera de D es adyacente a alguno de D . Determine si todo conjunto dominante es una cobertura de vértices y si toda cobertura es un conjunto dominante. Justifique con ejemplos.
 5. Demuestre que el problema de determinar si un grafo tiene una cobertura de vértices de tamaño a lo sumo 2 está en P. Generalice a tamaño a lo sumo k con k fijo y explique por qué esto no contradice al Corolario 8.7.4.
 6. Considere el siguiente algoritmo para hallar una cobertura de vértices pequeña: mientras queden arcos sin cubrir, escoger uno cualquiera y agregar sus *dos* extremos a la cobertura. Demuestre que el conjunto resultante es una cobertura y que su tamaño es a lo sumo el doble del de la cobertura más pequeña. Este algoritmo se retoma en la Sección 8.10.
 7. En grafos bipartitos, el tamaño de la cobertura de vértices más pequeña coincide con el de un emparejamiento máximo. Investigue acerca de ese resultado, conocido como el teorema de König, y explique qué dice sobre la dificultad de VCOVER restringido a grafos bipartitos.
-

8.8. SubsetSum es NP-completo

Las dos secciones anteriores se mueven entre proposiciones y grafos. Esta cambia de terreno por completo: el problema de llegada es aritmético y en su enunciado no aparecen ni variables ni vértices, sino números. Ese cambio tiene un propósito. Podría pensarse que la NP-completitud es un fenómeno de la teoría de grafos o de la lógica proposicional, y que basta con formular un problema en otro lenguaje

para escapar de ella. La reducción de esta sección muestra que no, del mismo modo que el problema de correspondencia de Post muestra en la Sección 7.5 que la indecidibilidad tampoco es un accidente del vocabulario.

En esta sección se estudia la versión de decisión del problema de suma exacta de un subconjunto presentado en la Sección 5.5.

Definición 8.8.1: Suma de subconjuntos

El problema de la *suma de subconjuntos* es el lenguaje:

$$\text{SUBSETSUM} = \{\langle S, t \rangle \mid \text{“algún subconjunto de } S \text{ suma } t\text{”}\},$$

donde S es un conjunto finito de números naturales y $t \in \mathbb{N}$.

Inicialmente, se determina que SUBSETSUM está en NP, como se indica en el Teorema 8.8.1. La demostración es directa.

Teorema 8.8.1

SUBSETSUM $\in$ NP.

Demostración

El certificado es el subconjunto. Verificarlo consiste en sumar sus elementos y comparar con t , lo cual toma tiempo polinomial en el tamaño de la entrada.

El siguiente paso es establecer SUBSETSUM $\in$ NP-hard y el plan es partir de 3CNFSAT. Para ello hay que resolver un problema de traducción que las secciones anteriores no enfrentaron: convertir una condición lógica en una condición aritmética.

La idea consiste en escribir los números en base 10 y reservar una *columna de dígitos* para cada cosa que se quiere controlar. Se reserva una columna por cada variable de la proposición y una por cada cláusula. Los números se diseñan de manera que cada columna imponga, por sí sola, una de las condiciones que la asignación debe cumplir. Las columnas de variable se encargan de que la asignación esté bien definida, obligando a escoger, para cada variable, exactamente uno de dos números: el que representa asignarle verdadero y el que representa asignarle falso. Las columnas de cláusula se encargan de que cada cláusula quede satisfecha, exigiendo una suma que no se puede alcanzar sin la contribución de al menos un literal verdadero. Para que el argumento por columnas funcione hace falta que las sumas nunca produzcan acarreo y por eso se trabaja en base 10 aunque los

dígitos que aparecen son pequeños: así cada columna se puede analizar de manera independiente de las demás.

Antes de escribir la construcción hay que atender un detalle. La Definición 8.5.3 permite que una cláusula repita literales, que es lo que hace cómodo completar cláusulas cortas, tal y como lo señala el Ejemplo 8.5.1. Esa licencia no les estorba a las reducciones de las dos secciones anteriores, pero sí a esta, y el motivo es que acá los números se entregan a un conjunto. Lo más sencillo es quitarla de en medio antes de empezar.

Definición 8.8.2: Proposición normalizada

Sea φ una proposición en forma normal conjuntiva de tres literales. Su *normalización* se obtiene aplicando, en orden, los tres pasos siguientes:

1. descartar las cláusulas *tautológicas*, es decir, las que contienen a la vez un literal y su negación;
2. suprimir las repeticiones de literales dentro de cada cláusula que quede; y
3. descartar las variables que ya no aparecen en ninguna cláusula.

Los tres pasos son mecánicos y cuestan a lo sumo un recorrido de la proposición. Note que ninguno altera la satisfacibilidad: una cláusula tautológica se satisface bajo cualquier asignación, repetir un literal no cambia el valor de una disyunción, y a una variable que no aparece se le puede asignar lo que sea. La normalización deja entonces cláusulas de uno, dos o tres literales, todos distintos, y ninguna cláusula vacía.

Definición 8.8.3: El mapa de cláusulas a números

Sea φ una proposición en forma normal conjuntiva de tres literales cuya normalización tiene variables $x_1, \dots, x_n$ y cláusulas $C_1, \dots, C_k$, con $k \geq 1$. Se consideran números de $n + k$ dígitos en base 10, en los cuales las primeras n posiciones se denominan *columnas de variable* y las últimas k *columnas de cláusula*.

El conjunto S tiene $2n + 2k$ números:

- Para cada variable x_i , dos números v_i y v'_i . El número v_i lleva un 1 en la columna de x_i y un 1 en la columna de cada cláusula que contiene al literal x_i . El número v'_i lleva un 1 en la columna de x_i y un 1 en la columna de cada cláusula que contiene al literal $\neg x_i$. Los demás dígitos son 0 en cada uno de los dos números.

- Para cada cláusula C_j , dos números de *holgura* s_j y s'_j . El primero, s_j , lleva un 1 en la columna de C_j . El segundo, s'_j , lleva un 2 en la columna de C_j . Los demás dígitos son 0 en los dos números de holgura.

El objetivo t es el número que lleva un 1 en cada columna de variable y un 4 en cada columna de cláusula.

Sea $h : \Sigma^* \rightarrow \Sigma^*$ el mapa definido así. Sobre una cadena que codifica una proposición φ de la forma requerida cuya normalización conserva alguna cláusula, h retorna $\langle S, t \rangle$. Si la normalización de φ no conserva ninguna cláusula, h retorna un par fijo que sí está en SUBSETSUM. Y sobre cualquier otra cadena, h retorna un par fijo que no está en SUBSETSUM.

El caso de las cláusulas que desaparecen no es un adorno defensivo. Si la normalización no deja ninguna, es porque toda cláusula de φ era tautológica, y entonces φ misma es una tautología y en particular es satisfacible; por eso el par fijo que h retorna allí debe ser uno afirmativo.

Los dos números asociados a una misma variable son ahora distintos, y conviene ver por qué, pues es lo único que la normalización estaba encargada de garantizar. Si x_i aparece solo de manera positiva, entonces v'_i no lleva ningún 1 en columnas de cláusula y v_i sí; si aparece solo de manera negativa, ocurre lo contrario. Y si aparece de las dos maneras, ninguna cláusula puede contener a x_i y a $\neg x_i$ al tiempo, pues sería tautológica y ya no está, de modo que los dos números llevan sus unos en columnas de cláusula distintas. El tercer paso de la Definición 8.8.2 es el que descarta el caso restante, el de una variable sin ocurrencias, para la cual v_i y v'_i serían ambos el número que lleva un único 1 en la columna de x_i .

Antes de la demostración conviene registrar la observación que hace funcionar todo el argumento.

Teorema 8.8.2: Las sumas no producen acarreo

Sea $T \subseteq S$ cualquier subconjunto. Al sumar los elementos de T , ninguna columna produce acarreo hacia la siguiente.

Demostración

Basta acotar la suma de los dígitos de cada columna sobre todo S .

En una columna de variable x_i solo llevan dígito distinto de 0 los números v_i y v'_i , cada uno con un 1, de modo que la suma no pasa de 2.

En una columna de cláusula C_j llevan un 1 los números asociados a los literales de C_j , que son a lo sumo tres por estar φ normalizada, y además s_j con un 1 y s'_j con un 2. La suma no pasa entonces de $3 + 1 + 2 = 6$.

Como en ningún caso se alcanza 10, no hay acarreos.

Con la construcción ya definida y los acarreos descartados, se puede enunciar el resultado principal de la sección. La demostración se apoya en las dos piezas anteriores y en nada más: la Definición 8.8.3 fija qué números se construyen y el Teorema 8.8.2 autoriza a mirar cada columna por separado, como si las demás no existieran. Esa autorización es lo que permite que un argumento sobre sumas de números grandes se reduzca a una cuenta de dígitos pequeños.

Teorema 8.8.3

El mapa h de la Definición 8.8.3 es una reducción polinomial de 3CNFSAT a SUBSETSUM. En consecuencia, $3\text{CNFSAT} \leq_P \text{SUBSETSUM}$.

Demostración

Hay dos cosas que verificar.

Que h es computable en tiempo polinomial se sigue de contar. Normalizar cuesta un recorrido de la proposición. Hecho eso, el conjunto S tiene $2n + 2k$ números, pues los números asociados a una misma variable son distintos, y cada uno tiene $n + k$ dígitos, de manera que escribirlos todos toma del orden de $(n + k)^2$ pasos. Como n y k no superan el tamaño de la entrada, el trabajo es polinomial. Note que los números son enormes, pero lo que cuenta es el tamaño de su representación, tal y como se advierte en el Ejemplo 8.1.1.

Para la condición de la Definición 8.4.1 hay tres casos. El de las cadenas mal formadas es inmediato por construcción. El de las proposiciones cuya normalización se queda sin cláusulas también lo es: esas proposiciones son tautologías y su imagen es un par afirmativo fijo. Resta el caso en que la entrada codifica una proposición φ cuya normalización conserva alguna cláusula, y se procede por doble implicación. Como la normalización preserva la satisfacibilidad, se puede suponer que φ ya está normalizada. En ambas direcciones se usa el Teorema 8.8.2, que permite razonar columna por columna.

($\Rightarrow$): Suponga que φ es satisfacible y sea α una asignación que la satisface.

Sea T el conjunto que contiene, para cada variable x_i , el número v_i si α hace verdadera a x_i y el número v'_i en caso contrario. En cada columna de variable, T aporta exactamente un 1, que es lo que exige t . En la columna

de una cláusula C_j , la suma que aporta T es la cantidad de literales de C_j que α hace verdaderos, pues cada uno de ellos contribuye con el único 1 del número que le corresponde. Aquí se usa que C_j no repite literales: de lo contrario la cuenta sería de ocurrencias y no coincidiría con la suma. Como α satisface a φ , esa cantidad está entre 1 y 3. Agregando números de holgura se completa hasta 4: con s_j y s'_j si la cantidad es 1, con s'_j si es 2 y con s_j si es 3.

El conjunto así formado suma exactamente t , de modo que $\langle S, t \rangle \in \text{SUBSETSUM}$.

($\Leftarrow$): Suponga que existe $T \subseteq S$ cuya suma es t .

Considere una columna de variable x_i . Los únicos números con dígito no nulo en esa columna son v_i y v'_i , que son distintos entre sí por estar φ normalizada, y el dígito de t allí es 1, de manera que T contiene exactamente uno de los dos. Defina la asignación α que hace verdadera a x_i si $v_i \in T$ y falsa si $v'_i \in T$. Es aquí donde hace falta que los dos números no hayan colapsado: si lo hubieran hecho, T contendría ese único número y la asignación quedaría sin determinar.

Considere ahora la columna de una cláusula C_j . La suma debe ser 4, y los números de holgura aportan a lo sumo $1 + 2 = 3$. Por lo tanto, al menos un número de los asociados a literales debe estar en T y aportar un 1 en esa columna. Ese número es v_i para algún literal x_i de C_j o v'_i para algún literal $\neg x_i$ de C_j . En cualquiera de los dos casos, α hace verdadero a ese literal y por lo tanto satisface a C_j .

Como esto vale para toda cláusula, α satisface a φ .

El resultado que da título a la sección se obtiene ahora encadenando lo anterior con la semilla de la Sección 8.5. Es la tercera vez en el capítulo que se usa ese mismo encadenamiento y vale la pena notar que en cada ocasión lo único nuevo ha sido la reducción: la pertenencia a **NP** es siempre un certificado evidente y el cierre es siempre la transitividad de $\leq_P$.

Corolario 8.8.4

$\text{SUBSETSUM} \in \text{NPC}$.

Demostración

La pertenencia a **NP** es el Teorema 8.8.1 y la dureza se sigue del Corolario 8.5.6, que establece que 3CNFSAT es **NP-hard**, junto con el Teorema 8.8.3 y la transitividad de $\leq_P$.

Conviene seguir la Definición 8.8.3 sobre una proposición pequeña, porque la construcción se entiende mucho mejor dispuesta en una tabla que enunciada.

Ejemplo 8.8.1

Considere la proposición

$$\varphi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3),$$

con $n = 3$ variables y $k = 2$ cláusulas. Los números tienen entonces cinco dígitos y la construcción entrega:

	x_1	x_2	x_3	C_1	C_2	
v_1	1	0	0	1	0	10 010
v'_1	1	0	0	0	1	10 001
v_2	0	1	0	0	1	01 001
v'_2	0	1	0	1	0	01 010
v_3	0	0	1	0	1	00 101
v'_3	0	0	1	1	0	00 110
s_1	0	0	0	1	0	00 010
s'_1	0	0	0	2	0	00 020
s_2	0	0	0	0	1	00 001
s'_2	0	0	0	0	2	00 002
t	1	1	1	4	4	11 144

La asignación que hace verdaderas a las tres variables satisface a φ : la primera cláusula por x_1 , y la segunda por x_2 y x_3 . El subconjunto correspondiente es $\{v_1, v_2, v_3\}$, que suma 11 112. La columna de C_1 va en 1 y necesita 3 más, que se obtienen con s_1 y s'_1 ; la de C_2 va en 2 y necesita 2 más, que se obtienen con s'_2 . En total:

$$10\,010 + 01\,001 + 00\,101 + 00\,010 + 00\,020 + 00\,002 = 11\,144 = t.$$

El ejemplo deja a la vista algo que conviene discutir antes de cerrar la sección: los números que la construcción produce son enormes, pues tienen tantos dígitos como variables y cláusulas juntas. Esa no es una torpeza de la reducción sino una condición para que funcione, y entenderlo previene una conclusión equivocada que a primera vista parece inevitable.

Nota 8.8.1: Pseudopolinomial no es polinomial

El Problema 3.6.1 del Capítulo 3 se resuelve por programación dinámica con un algoritmo cuyo tiempo es del orden de la cantidad de objetos multiplicada por la capacidad de la mochila. El mismo tipo de algoritmo resuelve

SUBSETSUM en tiempo del orden de $\text{card}(S)$ por t . Sería tentador concluir que $\text{SUBSETSUM} \in P$ y, con el Corolario 8.8.4 y el Teorema 8.5.3, que $P = NP$. El razonamiento es incorrecto y conviene ver exactamente dónde falla. Note que lo que falla es el razonamiento y no necesariamente su conclusión: si se supiera que $P \neq NP$, el argumento sobraría.

El tamaño de la entrada no es t sino la cantidad de dígitos que se necesitan para escribirlo, que es del orden de $\log t$. Un algoritmo que toma tiempo proporcional a t toma entonces tiempo exponencial en el tamaño de la entrada. A los algoritmos con esta característica se les denomina *pseudopolinomiales*: son polinomiales en el *valor* de los números de la entrada, pero no en su *representación*.

La reducción de esta sección explota esa distinción de manera deliberada. Los números que construye tienen $n + k$ dígitos, de modo que su valor es astronómico mientras que escribirlos cuesta poco. Una reducción que produjera números de valor acotado por un polinomio no podría existir, salvo que $P = NP$: con valores acotados por un polinomio también lo estaría el objetivo t , de manera que el algoritmo pseudopolinomial decidiría la instancia en tiempo polinomial y componerlo con la reducción decidiría a 3CNFSAT en tiempo polinomial.

Ejercicios

1. Aplique la Definición 8.8.3 a la proposición $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$ y exhiba un subconjunto que sume el objetivo.
2. En el Ejemplo 8.8.1 hay más de un subconjunto que suma t . Encuentre otro y determine qué asignación le corresponde.
3. Sobre la Definición 8.8.2 y los tres casos que atiende:
 - a) Aplique la Definición 8.8.3 a $(p \vee p \vee p)$ sin normalizar y verifique que la receta de la demostración del Teorema 8.8.3 prescribe agregar únicamente s_1 , con lo cual la suma no alcanza el objetivo. Determine cuál es la holgura correcta y qué cuenta hay que hacer para obtenerla.
 - b) Aplique la misma definición a $(p \vee \neg p \vee p)$ sin normalizar y verifique que $v_1 = v'_1$, de manera que S tiene tres elementos y no cuatro. Explique qué paso de la demostración deja de funcionar.
 - c) Exhiba una proposición normalizada en los dos primeros pasos pero no en el tercero para la cual todavía se tenga $v_i = v'_i$. Concluya que descartar las variables sin ocurrencias no es un trámite.
 - d) Demuestre que si la normalización de φ no conserva ninguna cláusula, entonces φ es satisfacible.
4. Explique por qué el dígito del objetivo en las columnas de cláusula es 4 y no otro número. En particular, determine qué sale mal si se usa 3, y qué sale mal si se usa 5.
5. Los números de holgura de una cláusula llevan un 1 y un 2. Explique por qué no bastaría con un solo número de holgura, y por qué no hacen falta tres.
6. Demuestre que la construcción sigue siendo correcta si se trabaja en base b para cualquier $b \geq 7$. Determine el menor valor de b para el cual el Teorema 8.8.2 sigue valiendo.
7. Considere el problema de la *partición*, que consiste en determinar si un conjunto finito de números naturales se puede dividir en dos partes con la misma suma. Demuestre que $\text{SUBSETSUM} \leq_P \text{PARTITION}$ y concluya que PARTITION es NP-completo. Ayuda: agregue a lo sumo dos números al conjunto.
8. El problema SUBSETSUM admite un algoritmo de programación dinámica cuyo tiempo es del orden de $\text{card}(S)$ por t . Escríbalo en Python y ejecútelo sobre la instancia del Ejemplo 8.8.1 y estime cuánto tardaría sobre la que produce la reducción a partir de una proposición con 50 variables y 50 cláusulas.
9. Investigue acerca de los problemas que admiten algoritmos pseudopolinomiales y de los que no, conocidos estos últimos como *fuertemente* NP-completos. Determine a cuál de los dos grupos pertenece CLIQUE y justifique su respuesta.

8.9. Tsp es NP-completo

El problema del agente viajero viene apareciendo desde hace varios capítulos. En el Capítulo 3 se resuelve por programación dinámica en el Problema 3.9.1 y en el Ejemplo 6.1.2 se usa como problema de optimización que se puede resolver consultando repetidamente su versión de decisión. Esta sección cierra ese recorrido clasificándolo.

La reducción de esta sección es, con diferencia, la más corta del capítulo. Eso no se debe a que el problema sea fácil, sino a que la dificultad ya está hecha: se parte de un problema sobre grafos que también es NP-hard y solo hay que vestirlo de pesos.

Definición 8.9.1: Ciclo hamiltoniano

Un *ciclo hamiltoniano* de un grafo es un ciclo que visita cada vértice exactamente una vez. El problema de decisión asociado es:

$$\text{HAMCYCLE} = \{ \langle G \rangle \mid \text{"}G \text{ tiene un ciclo hamiltoniano"} \}.$$

Teorema 8.9.1

HAMCYCLE es NP-hard.

Nota 8.9.1: Sobre la hipótesis anterior

El Teorema 8.9.1 se enuncia sin demostración y conviene decir con precisión qué se está aceptando.

La demostración usual es una reducción desde VCOVER, es decir, desde el problema de la Sección 8.7, de manera que la cadena de este capítulo no se interrumpe ni se apoya en nada externo. Lo que se omite es la construcción, que reemplaza cada arco del grafo de partida por un artefacto de doce vértices diseñado para que cualquier ciclo hamiltoniano lo recorra de una de dos maneras; esta verificación ocupa varias páginas.

Es la segunda y última vez en el texto que un resultado de dureza se cita sin demostrar; la primera es el Teorema 8.5.1. Las referencias aparecen en las notas del capítulo.

La versión de decisión del agente viajero es la del Ejemplo 6.1.2, con pesos enteros no negativos por las razones que allá expone la Nota 6.1.2: un peso real arbitrario no tiene representación finita y, aun concediendo eso, con pesos reales la cantidad de consultas deja de ser una función de la entrada.

Definición 8.9.2: El agente viajero

Sea $G = (V, E)$ un grafo completo, $w : E \rightarrow \mathbb{N}$ una función de peso sobre sus arcos y $k \in \mathbb{N}$. El problema de decisión del *agente viajero* es:

$$\text{TSP} = \{ \langle G, w, k \rangle \mid \text{"}G \text{ tiene un ciclo hamiltoniano de costo a lo sumo } k\text{"} \}.$$

Teorema 8.9.2

$\text{TSP} \in \text{NP}$.

Demostración

El certificado es la secuencia de vértices del ciclo. Verificarlo consiste en comprobar que cada vértice aparece exactamente una vez, que vértices consecutivos están unidos y que la suma de los pesos correspondientes no supera a k . Las tres comprobaciones son polinomiales en el tamaño de la entrada.

Falta establecer $\text{TSP} \in \text{NP-hard}$ y para ello se apela a una observación. Un ciclo hamiltoniano de G es un recorrido que usa únicamente arcos de G ; un recorrido de costo pequeño en un grafo con pesos es uno que evita los arcos caros. Si se completa G hasta volverlo un grafo completo, poniendo peso 0 sobre los arcos que ya estaban y peso 1 sobre los que se agregaron, entonces evitar los arcos caros y usar solo arcos de G pasan a ser la misma cosa.

Definición 8.9.3: El mapa de completar con pesos

Sea $q : \Sigma^* \rightarrow \Sigma^*$ el mapa definido de la siguiente manera. Si x codifica un grafo $G = (V, E)$ con $n = \text{card}(V)$ vértices, entonces $q(x) = \langle K_n, w, 0 \rangle$, donde $K_n = (V, E_n)$ es el grafo completo sobre V y la función de peso $w : E_n \rightarrow \mathbb{N}$

está dada, para cada $(u, v) \in E_n$, por:

$$w(u, v) = \begin{cases} 0, & \text{si } (u, v) \in E, \\ 1, & \text{en caso contrario.} \end{cases}$$

Note que el dominio de w es E_n y no E : la función debe asignarle peso también a los arcos que la construcción agrega, que son precisamente los del segundo caso. Si x no codifica un grafo, entonces $q(x)$ es una tripla fija que no está en TSP.

Teorema 8.9.3

El mapa q de la Definición 8.9.3 es una reducción polinomial de HAMCYCLE a TSP. En consecuencia, $\text{HAMCYCLE} \leq_P \text{TSP}$.

Demostración

Que q es computable en tiempo polinomial es inmediato: hay menos de n^2 pares de vértices y para cada uno se decide en tiempo constante si su peso es 0 o 1.

Para la condición de la Definición 8.4.1, el caso de las cadenas que no codifican un grafo es inmediato por construcción. Resta el caso en que la entrada codifica un grafo G y se procede por doble implicación.

($\Rightarrow$): Suponga que G tiene un ciclo hamiltoniano. Ese mismo ciclo es un ciclo hamiltoniano de K_n , pues K_n tiene todos los arcos de G . Y todos sus arcos están en E , de modo que cada uno pesa 0 y el costo total del recorrido es 0. Luego $q(x) \in \text{TSP}$.

($\Leftarrow$): Suponga que K_n tiene un ciclo hamiltoniano de costo a lo sumo 0. Como todos los pesos son 0 o 1, y por lo tanto no negativos, el costo total solo puede ser 0 si cada uno de sus arcos pesa 0. Por la definición de w , eso significa que todos sus arcos están en E , de manera que el ciclo es también un ciclo hamiltoniano de G . Luego $x \in \text{HAMCYCLE}$.

Corolario 8.9.4

$\text{TSP} \in \text{NPC}$.

Demostración

La pertenencia a NP es el Teorema 8.9.2 y la dureza se sigue del Teorema 8.9.1 junto con el Teorema 8.9.3 y la transitividad de $\leq_P$.

Conviene seguir la Definición 8.9.3 sobre un grafo pequeño, porque la construcción se ve de un golpe y deja clara la razón por la cual el umbral es 0.

Ejemplo 8.9.1

Considere el grafo G de la Figura 5, con cuatro vértices y los arcos (a,b) , (b,c) , (c,d) , (d,a) y (a,c) .

La reducción entrega el grafo completo K_4 , en el cual los cinco arcos anteriores pesan 0 y el único arco que faltaba, el que une a b con d , pesa 1. El umbral es 0.

El recorrido a, b, c, d, a visita cada vértice una vez y usa únicamente arcos de peso 0, de modo que su costo es 0 y por lo tanto $\langle K_4, w, 0 \rangle \in \text{TSP}$. Ese mismo recorrido es un ciclo hamiltoniano de G .

Note que cualquier recorrido que use el arco entre b y d tiene costo al menos 1 y por lo tanto no sirve, que es exactamente lo que la construcción busca: los arcos que no estaban en G quedan prohibidos por el precio.

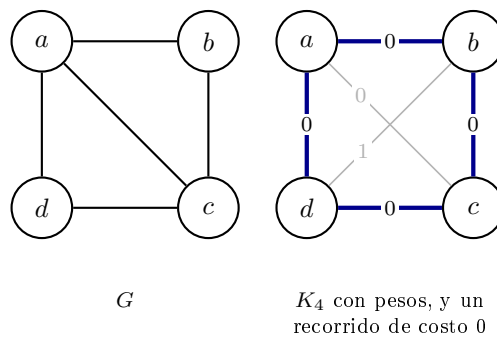


Figura 5. El grafo del Ejemplo 8.9.1 y el grafo completo con pesos que la reducción construye a partir de él. Los arcos que ya estaban en G reciben peso 0 y los que se agregan reciben peso 1. El recorrido resaltado tiene costo 0.

Nota 8.9.2: La optimización hereda la dificultad

El Corolario 8.9.4 habla de la versión de decisión, mientras que el Problema 3.9.1 del Capítulo 3 pide el costo mínimo, que es un problema de optimización. Conviene cerrar el recorrido explicando qué dice lo uno acerca de lo otro.

El Ejemplo 6.1.2 muestra que el problema de optimización se resuelve consultando repetidamente el de decisión, con una cantidad polinomial de consultas. En consecuencia, un algoritmo polinomial para el problema de optimización daría de inmediato uno para el de decisión y por el Teorema 8.5.3 eso implicaría $P = NP$.

Es decir, la intratabilidad se traslada de la versión de decisión a la de optimización. Ese traslado es precisamente lo que justifica, de manera retroactiva, la decisión de la Sección 6.1 de estudiar únicamente problemas de decisión: restringir el objeto de estudio no dejó nada por fuera.

Ejercicios

1. Aplique la Definición 8.9.3 a un grafo de cuatro vértices que *no* tenga ciclo hamiltoniano y verifique que el grafo con pesos resultante no admite recorrido de costo 0.
2. La Definición 8.9.3 usa los pesos 0 y 1 con umbral 0. Demuestre que la reducción también funciona con pesos 1 y 2 y umbral n , y explique cuál de las dos versiones le parece más clara.
3. Explique en qué punto de la demostración del Teorema 8.9.3 se usa que los pesos no son negativos y qué sale mal si se permitieran pesos negativos.
4. Demuestre que $\text{HAMCYCLE} \in \text{NP}$. Junto con el Teorema 8.9.1, concluya que $\text{HAMCYCLE} \in \text{NPC}$.
5. El problema del agente viajero se suele plantear sobre grafos cuyos pesos satisfacen la desigualdad triangular, es decir, $w(u, v) \leq w(u, z) + w(z, v)$ para cualesquiera tres vértices. Determine si la instancia que produce la Definición 8.9.3 la satisface.
6. Investigue acerca del problema del agente viajero con desigualdad triangular y determine si sigue siendo NP-completo. Compare con su respuesta al ejercicio anterior.
7. El Ejemplo 6.1.2 usa bisección para pasar del problema de optimización al de decisión. Calcule cuántas consultas hacen falta cuando los pesos son naturales acotados por W y compare con el caso de pesos reales discutido allá.

8.10. ¿Qué hacer con un problema NP-completo?

Las secciones anteriores clasifican cuatro problemas como NP-completos y la lista de los que se conocen supera los miles. Es razonable preguntarse qué hacer cuando el problema que uno tiene enfrente resulta ser uno de ellos.

La respuesta corta es que reconocerlo es una buena noticia y no una mala. Antes de saberlo, uno estaba buscando un algoritmo eficiente sin saber que buscaba, de paso, una demostración de que $P = NP$. Después de saberlo, uno puede dejar de buscar eso y empezar a buscar otra cosa. Esta sección recorre las opciones.

Conviene decir de entrada que ninguna de ellas resuelve el problema en el sentido estricto del capítulo. Todas renuncian a algo: al alcance, a la exactitud, a la garantía de terminar rápido o a las tres. Lo que las hace útiles es que la renuncia es explícita y está bajo control, en lugar de ser una sorpresa que aparece cuando el programa lleva tres días corriendo.

Cambiar la pregunta. La primera opción es verificar que el problema que uno tiene sea de verdad el que cree tener. La NP-completitud es una afirmación sobre el caso general y las instancias que aparecen en la práctica suelen tener estructura que el caso general ignora.

La diferencia puede ser enorme. El problema 3CNFSAT es NP-completo, pero su versión con dos literales por cláusula está en P. El problema VCOVER es NP-completo, pero restringido a grafos bipartitos se resuelve en tiempo polinomial. En ambos casos la restricción no es artificial: hay aplicaciones enteras que solo producen instancias de la clase fácil.

Por eso vale la pena preguntarse, antes que nada, si los grafos de la aplicación son planares, si las proposiciones tienen cláusulas cortas, si los números están acotados o si los datos llegan ordenados. Cambiar la pregunta es la única de las opciones de esta sección que no exige renunciar a nada.

Parametrizar. La segunda opción es aislar qué parte de la entrada causa la explosión. Con frecuencia el crecimiento exponencial no depende del tamaño total sino de una cantidad específica que en la práctica es pequeña.

El caso de VCOVER es ilustrativo. Determinar si un grafo de n vértices tiene una cobertura de tamaño a lo sumo k se puede resolver así: si no quedan arcos por cubrir, la respuesta es afirmativa; si $k = 0$ y aún quedan arcos, es negativa; y en cualquier otro caso se escoge un arco (u, v) y se resuelve el mismo problema, con $k - 1$, para el grafo sin u y para el grafo sin v . La corrección se sigue de que cualquier cobertura debe contener a u o a v .

El árbol de llamados tiene profundidad k y se ramifica en dos, de modo que el tiempo es del orden de 2^k multiplicado por un polinomio en n . Para n grande y k pequeño eso es perfectamente utilizable; note que el exponente ya no depende de n .

La versión que este algoritmo resuelve no es la de la Definición 8.7.1, que pide una cobertura de tamaño exactamente k . Las dos coinciden mientras $0 \leq k \leq n$, pues todo superconjunto de una cobertura es una cobertura y por lo tanto una de tamaño a lo sumo k se completa hasta tener exactamente k agregándole vértices cualesquiera. Difieren solo cuando $k > n$: allí la versión acotada responde que sí, porque V mismo cubre, y la exacta responde que no, porque no hay subconjuntos de ese tamaño. Una comparación inicial entre k y n resuelve la diferencia, y es la misma precaución que toma el mapa de la Definición 8.7.2.

Aproximar. La tercera opción es renunciar a la exactitud pero conservar una garantía sobre cuánto se pierde.

Definición 8.10.1: Factor de aproximación

Sea P un problema de minimización y A un algoritmo que, para cada instancia, produce una solución válida. Se dice que A es una *aproximación de factor* ρ si y solo si, para cualquier instancia, el costo de la solución que produce A es a lo sumo ρ veces el costo de la solución óptima.

El Ejercicio 6 de la Sección 8.7 propone un algoritmo para VCOVER que consiste en escoger un arco cualquiera sin cubrir y agregar sus *dos* extremos a la cobertura, hasta que no queden arcos. Agregar los dos parece un desperdicio evidente y, sin embargo, es lo que hace funcionar la garantía.

Teorema 8.10.1

El algoritmo anterior es una aproximación de factor 2 para el problema de hallar la cobertura de vértices más pequeña.

Demostración

Sea C el conjunto que produce el algoritmo y sea M el conjunto de los arcos que fueron escogidos durante su ejecución.

Primero, C es una cobertura: el ciclo solo termina cuando no quedan arcos sin cubrir.

Segundo, dos arcos de M no comparten extremos. En efecto, al escoger un arco se agregan sus dos extremos a C , con lo cual todos los arcos incidentes a ellos quedan cubiertos y ya no pueden ser escogidos después.

Sea ahora C^* una cobertura de tamaño mínimo. Como C^* cubre todos los arcos, contiene al menos un extremo de cada arco de M ; y como los arcos de M no comparten extremos, esos vértices son distintos entre sí. Luego $\text{card}(C^*) \geq \text{card}(M)$.

Por otra parte, C tiene exactamente dos vértices por cada arco de M , es decir, $\text{card}(C) = 2\text{card}(M)$. Combinando las dos desigualdades:

$$\text{card}(C) = 2\text{card}(M) \leq 2\text{card}(C^*),$$

que es lo que se quería.

Note que la demostración nunca calcula la cobertura óptima ni la conoce; se limita a acotarla por debajo usando los arcos escogidos. Esa es la maniobra característica de los algoritmos de aproximación: comparar contra una cota inferior que sí se puede calcular.

No todos los problemas se dejan aproximar igual de bien. Para TSP en su forma general no existe aproximación de factor constante, a menos que $P = NP$; pero si los pesos satisfacen la desigualdad triangular, sí las hay. La aproximabilidad es, en sí misma, una clasificación tan fina como la de este capítulo.

Renunciar a la garantía. La cuarta opción es abandonar toda garantía y quedarse con lo que funciona.

Puede sonar a rendición y es exactamente lo contrario. Los resolvers modernos de satisfacibilidad manejan de manera rutinaria proposiciones con cientos de miles de variables, y son una tecnología industrial que se usa en verificación de circuitos, en planificación y en análisis de programas.

Conviene ser preciso sobre a qué se renuncia, porque es fácil exagerarlo. Lo que se abandona es la garantía de un tiempo acotado por un polinomio, no la corrección: un resolver que termina entrega la respuesta exacta, y sobre cualquier instancia finita el recorrido exhaustivo de la Nota 8.2.3 termina. No hay instancias irresolubles, sino instancias cuya solución tomaría más tiempo del que nadie tiene. Y el resultado que lo explica no es el Corolario 8.5.6, que por sí solo no prohíbe nada: lo que ocurre es que no se conoce ningún algoritmo polinomial para 3CNFSAT y que exhibir uno resolvería la pregunta de este capítulo.

Lo que sí se sabe es que ciertos métodos tienen un techo demostrado. Los resolvers de uso corriente están contruidos sobre resolución, y para ella hay familias de proposiciones que exigen derivaciones de tamaño exponencial; las del principio del palomar son el ejemplo clásico [Hak85]. Note que esa es una cota

sobre el método y no sobre el problema: un resolutor que razone además sobre simetrías o sobre restricciones de cardinalidad escapa a ella.

Vale la pena contrastar esta situación con la que describe la Nota 7.4.3 para los analizadores de programas frente al teorema de Rice, porque el paralelo es menos directo de lo que parece. Allá el procedimiento que responde siempre y bien no existe, y las herramientas sobreviven restringiendo el dominio, aproximando o admitiendo la posibilidad de no responder. Acá ese procedimiento existe y es el que se está ejecutando; ninguna de las tres salidas hace falta. Lo único que se cede es la cota de tiempo. Lo que los dos casos comparten, y es lo que hace viable a la práctica, es que el peor caso no es el caso típico.

Pagar el exponencial, pero poderlo. La última opción es aceptar el costo exponencial y trabajar para que la constante y la base sean lo más pequeñas posible.

Esa es exactamente la idea del Capítulo 5: recorrer el espacio de soluciones de manera sistemática, descartando ramas enteras apenas se sabe que no contienen solución. Cuando además se dispone de una cota sobre el costo de las soluciones que una rama puede contener, la técnica recibe el nombre de *ramificación y acotamiento*. Aplicada al agente viajero, permite resolver de manera exacta instancias que el recorrido exhaustivo ni siquiera empieza [LD60, LMSK63].

Vale la pena notar que el comportamiento sigue siendo exponencial en el peor caso. Lo que cambia es que el peor caso deja de ser el caso típico.

Lo que no hay que hacer. Hay experiencias aprendidas a lo largo de los años que vale la pena tener en cuenta para no cometer errores de principiante.

Nota 8.10.1

Conviene cerrar señalando dos errores que este capítulo vuelve reconocibles.

El primero es proponer un algoritmo y afirmar, sin analizarlo, que resuelve eficientemente un problema NP-completo. Por el Teorema 8.5.3, tal afirmación equivale a haber demostrado que $P = NP$, y quien la hace debería estar reclamando el premio del Instituto Clay en lugar de escribiendo código.

El segundo es suponer que las instancias de la aplicación siempre serán pequeñas. Los tamaños crecen y un algoritmo exponencial que hoy responde en un segundo puede tardar más que la edad del universo con el doble de datos. Si esa es la apuesta, conviene al menos hacerla de manera consciente y medir dónde está el límite.

En ambos casos, el capítulo no ofrece una prohibición sino información. Saber que un problema es NP-completo no cierra puertas: indica cuáles vale la pena tocar.

Ejercicios

1. Implemente en Python el algoritmo de aproximación del Teorema 8.10.1 y ejecútelo sobre grafos generados al azar. Compare el tamaño de la cobertura que produce con el de la más pequeña, calculada por fuerza bruta, para grafos de a lo sumo quince vértices.
2. Exhiba un grafo para el cual el algoritmo del Teorema 8.10.1 produzca una cobertura de exactamente el doble del tamaño de la óptima. Concluya que el factor 2 no se puede mejorar para este algoritmo.
3. Considere la variante del algoritmo que, en lugar de agregar los dos extremos del arco escogido, agrega solo uno de ellos. Demuestre que el resultado puede no ser una cobertura, o que si se corrige para que lo sea, la garantía de factor 2 se pierde.
4. Implemente el algoritmo parametrizado para VCOVER descrito en esta sección y verifique experimentalmente que su tiempo depende mucho más de k que de n .
5. El Ejemplo 8.1.1 señala que PRIME está en P. Explique por qué ese resultado no ayuda en nada a resolver FACTOR y relacione su respuesta con el Ejemplo 8.3.2.
6. Un colega propone el siguiente razonamiento: “como SUBSETSUM se resuelve por programación dinámica en tiempo proporcional a $\text{card}(S)$ por t , y en mi aplicación t nunca pasa de un millón, mi algoritmo es eficiente”. Determine si el razonamiento es correcto y explique con precisión qué está afirmando y qué no.
7. Escoja un problema NP-completo de este capítulo y busque, en la literatura o en internet, cuál es el mejor factor de aproximación conocido y si se sabe que no se puede mejorar.
8. Investigue acerca de la clase de problemas tratables con parámetro fijo, conocida por su sigla en inglés FPT, y determine si CLIQUE con parámetro k pertenece a ella.

Notas del capítulo y referencias

La identificación de la clase P con la noción informal de “resoluble de manera eficiente” se conoce como la tesis de Cobham y Edmonds, y es el análogo, en el terreno de la complejidad, de lo que la tesis de Church y Turing es en el de la computabilidad. Las tres justificaciones que se dan en la Sección 8.1 siguen de cerca la discusión de [CLRS22]; tratamientos equivalentes se encuentran en [Sip12, Eri19]. La segunda de esas justificaciones, la de que la clase no depende del modelo

de cómputo, es la que la Nota 6.2.1 anticipa en el Capítulo 6: los modelos razonables se simulan unos a otros con sobrecosto polinomial, de manera que P resulta ser la misma en todos ellos. Esa propiedad es la que hace que la clase merezca un nombre.

El texto de T. Cormen et al. [CLRS22] define a NP por medio de algoritmos de verificación, que es la opción que sigue este capítulo. La formulación original, la que explica la sigla, se basa en máquinas de Turing no determinísticas. La equivalencia entre las dos se propone como ejercicio en la Sección 8.2. Hay un punto de la definición que conviene señalar porque suele presentarse de otra manera. Aquí el tiempo del algoritmo de verificación se mide en función de $\text{card}(x)$ únicamente, con lo cual la cota sobre el tamaño de los certificados se obtiene como consecuencia y no como una exigencia adicional. Es frecuente encontrar la versión que mide el tiempo en $\text{card}(x) + \text{card}(y)$ y acota $\text{card}(y)$ por separado; el ejercicio correspondiente pide demostrar que las dos determinan la misma clase.

La Sección 8.3 y su paralelo con el teorema de Post no son usuales en textos introductorios. La motivación para incluirlos es que la estructura de las dos situaciones es idéntica y el desenlace no: en el Capítulo 6 la pregunta de cuándo un lenguaje semidecidible llega a ser decidible está resuelta por el Teorema 6.5.3, mientras que su análogo de complejidad, la pregunta de si $P = NP \cap \text{co}NP$, lleva medio siglo abierta. Poner las dos una al lado de la otra es la manera más directa que encontramos de mostrar en qué se diferencian la teoría de la computabilidad y la de la complejidad. El Ejemplo 8.3.2 sobre el problema de la factorización descansa en que la primalidad se decide en tiempo polinomial, que es el resultado de M. Agrawal, N. Kayal y N. Saxena [AKS04].

Las reducciones polinomiales de la Sección 8.4 son las que R. Karp introduce en [Kar72] y por eso se les denomina reducciones de Karp. La presentación de este texto las define como las reducciones computables de la Definición 7.1.2 con una condición adicional sobre el costo, en lugar de introducirlas desde cero; la notación $\leq_P$ se escogió para que la relación con $\leq_c$ quede a la vista. Las reducciones de Cook, que permiten invocar varias veces el problema de llegada, aparecen en [Coo71].

El Teorema 8.5.1 se debe de manera independiente a S. Cook [Coo71] y a L. Levin [Lev73], este último publicado en la Unión Soviética y desconocido en occidente durante varios años. El texto lo cita sin demostrar, y la razón es la misma por la cual la Sección 7.5 difiere la indecidibilidad del problema de correspondencia de Post: la construcción exige codificar cómputos, es decir, reintroducir el modelo de máquina que el Capítulo 6 evitó deliberadamente al fijar Python. A partir de ahí, en cambio, todo se construye. La ruta que se sigue pasa por 3CNFSAT y evita el paso intermedio por CNFSAT gracias a la transformación que introduce una variable por compuerta, atribuida a G. Tseitin. Que esa transformación produzca cláusulas de a lo sumo tres literales no es una coincidencia afortunada sino consecuencia de que las compuertas tienen a lo sumo dos entradas.

Los cuatro problemas de las secciones 8.6 a 8.9 aparecen en la lista de veintidós problemas de [Kar72], que es el trabajo que convirtió a la NP-completitud en una herramienta de uso corriente. El catálogo de referencia sigue siendo el de M. Garey y D. Johnson [GJ79], cuyo apéndice recoge varios cientos de problemas clasificados. El orden que se escogió no es el único posible. Se prefirió ir de la lógica a los grafos, de los grafos a los grafos y de la lógica a la aritmética, para que el lector vea que la técnica no depende del dominio; la Nota 8.7.1 explica por qué unas reducciones resultan mucho más cortas que otras.

Hay entre esas reducciones un contraste que vale la pena señalar, porque explica una asimetría que al leerlas seguidas puede parecer arbitraria. La Definición 8.5.3 permite que una cláusula repita literales, licencia que resulta cómoda para completar cláusulas cortas y que no le estorba a la reducción de la Sección 8.6: allá la construcción pone un vértice por cada *ocurrencia* de literal, de modo que repetir un literal produce dos vértices distintos y el grafo no se entera. La reducción de la Sección 8.8 indexa, en cambio, por *variable*, y entrega sus números a un conjunto, que colapsa duplicados; por eso necesita normalizar la proposición antes de empezar, tal y como lo hace la Definición 8.8.2. El punto no es lógico sino estructural: lo que obliga a normalizar no es la forma de la proposición sino el hecho de que el problema de llegada hable de conjuntos. Una formulación de SUBSETSUM sobre multiconjuntos haría innecesaria la normalización, al precio de cambiar el problema para acomodar la reducción, que es la dirección equivocada.

La Nota 8.8.1 sobre los algoritmos pseudopolinomiales conecta con el Capítulo 3: la solución por programación dinámica del Problema 3.6.1 es eficiente en el valor de los números y no en su representación, distinción que la reducción de la Sección 8.8 explota de manera deliberada.

La dureza de HAMCYCLE, que la Sección 8.9 enuncia sin demostrar, se establece por reducción desde VCOVER; la construcción completa se encuentra en [CLRS22] y [GJ79]. Junto con el Teorema 8.5.1, es el único resultado de dureza que el texto acepta sin construir. Para el problema del agente viajero en particular, el estudio computacional de D. Applegate et al. [ABCC06] documenta hasta dónde se ha llegado resolviendo instancias reales de manera exacta, y es la mejor ilustración disponible de lo que la Sección 8.10 llama pagar el exponencial y podarlo.

Las estrategias de la Sección 8.10 corresponden, cada una, a un área de investigación con literatura propia. Para aproximación, el texto de V. Vazirani [Vaz01] es la referencia estándar; el algoritmo de factor $3/2$ para el agente viajero con desigualdad triangular se debe a N. Christofides [Chr76] y siguió siendo el mejor conocido durante más de cuarenta años. Para complejidad parametrizada, el tratamiento de R. Downey y M. Fellows [DF13] es el de referencia. Y para ramificación y acotamiento, los trabajos de A. Land y A. Doig [LD60], y de J. Little et al. [LMSK63] son los fundacionales, este último aplicado precisamente al agente viajero.

Una última precisión sobre el vocabulario de la introducción del capítulo. Allá se dice que los algoritmos conocidos para los problemas de este capítulo son exponenciales y así es; que no pueda haberlos mejores no es un teorema. Que 3CNFSAT no admita un algoritmo que corra en tiempo $2^{o(n)}$ es la *hipótesis del tiempo exponencial*, formulada por R. Impagliazzo y R. Paturi [IP01]. Es una conjetura estrictamente más fuerte que $P \neq NP$, pues la implica sin ser implicada por ella, y buena parte de las cotas inferiores que hoy se demuestran en complejidad parametrizada la suponen. Es decir, que un problema sea NP-completo no descarta por sí solo un algoritmo subexponencial; descartarlo exige una hipótesis adicional que también está abierta.

Conviene cerrar con una lectura que no es técnica. En su libro sobre el problema P frente a NP, L. Fortnow [For13] recorre su historia sin una sola fórmula, y dedica su segundo capítulo a imaginar con detalle un mundo en el cual alguien demuestra que las dos clases coinciden. En ese relato, un estudiante de doctorado de la Universidad de Illinois ejecuta su código en el supercomputador del centro nacional de la universidad, en Urbana, y obtiene un programa de cuarenta y dos millones de líneas de código ininteligible que resuelve problemas de NP muy rápido; un comunicado de prensa lo bautiza como el *algoritmo de Urbana* y el nombre queda. Lo que sigue en ese capítulo es una lista de consecuencias que vale la pena leer después de haber estudiado este: el cáncer se vuelve tratable, la predicción del clima se vuelve exacta, la producción industrial se optimiza sola, y de paso desaparece por completo la criptografía de llave pública y con ella buena parte de la infraestructura digital.

Ese contraste es, quizás, la mejor manera de terminar. Las tres últimas partes de este texto dicen, en orden, qué se puede computar, qué no se puede computar y qué se puede computar rápido. De las tres preguntas, las dos primeras están resueltas desde hace casi un siglo y la tercera sigue abierta. Que la más práctica de ellas sea también la única sin respuesta no es una anomalía: es un recordatorio de que la distancia entre lo posible y lo realizable es, todavía hoy, territorio sin cartografiar.

Bibliografía

- [AB98] Mohamad Akra and Louay Bazzi, *On the solution of linear recurrence equations*, Computational Optimization and Applications **10** (1998), no. 2, 195–210.
- [ABCC06] David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook, *The traveling salesman problem: A computational study*, Princeton Series in Applied Mathematics, Princeton University Press, 2006.
- [AKS04] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena, *PRIMES is in P*, Annals of Mathematics **160** (2004), no. 2, 781–793.
- [Bac94] Paul Bachmann, *Die analytische zahlentheorie*, Zahlentheorie, vol. 2, Teubner, Leipzig, 1894.
- [Bac03] Roland Backhouse, *Program construction: Calculating implementations from specifications*, Wiley, 2003.
- [BB88] Gilles Brassard and Paul Bratley, *Algorithms: Theory and practice*, Prentice Hall, 1988.
- [Bei13] Wolfgang Bein, *Advanced techniques for dynamic programming*, pp. 41–92, Springer, New York, NY, 2013.
- [Bel57] R. Bellman, *Dynamic programming*, Princeton University Press, Princeton, 1957.
- [Bel58] Richard Bellman, *On a routing problem*, Quarterly of Applied Mathematics **16** (1958), 87–90.
- [Bel62] R. Bellman, *Dynamic programming treatment of the travelling salesman problem*, Journal of the ACM **9** (1962), no. 1, 61–63.
- [Bel84] Richard Bellman, *Eye of the hurricane: an autobiography*, World Scientific, 1984.
- [Ben84] J. Bentley, *Programming pearls: Algorithm design techniques*, Communications of the ACM **27** (1984), no. 9, 865–873.
- [Ben00] Jon Bentley, *Programming pearls*, second ed., Addison-Wesley, 2000.
- [Bha15] Harsh Bhasin, *Algorithms: Design and analysis*, Oxford University Press, 2015.

- [BHS80] Jon Louis Bentley, Dorothea Haken, and James B. Saxe, *A general method for solving divide-and-conquer recurrences*, SIGACT News **12** (1980), no. 3, 36–44.
- [Bir89] R. S. Bird, *Algebraic identities for program calculation*, The Computer Journal **32** (1989), no. 2, 122–126.
- [BN98] Franz Baader and Tobias Nipkow, *Term rewriting and all that*, Cambridge University Press, 1998.
- [Boh06] Jaime Bohórquez, *Diseño efectivo de programas correctos*, Escuela Colombiana de Ingeniería, 2006.
- [Bor26] Otakar Borůvka, *O jistém problému minimálním*, Práce Moravské Přírodovědecké Společnosti (1926), 27–58.
- [BR75] James R. Bitner and Edward M. Reingold, *Backtrack programming techniques*, Communications of the ACM **18** (1975), no. 11, 651–656.
- [Bre06] Joseph Breuer, *Introduction to the theory of sets*, 1st ed., Dover Publications, 2006.
- [Bro82] Allan G. Bromley, *Charles Babbage’s Analytical Engine, 1838*, Annals of the History of Computing **4** (1982), no. 3, 196–217.
- [BS09] Jordan Bell and Brett Stevens, *A survey of known results and research areas for n -queens*, Discrete Mathematics **309** (2009), no. 1, 1–31.
- [Can91] Georg Cantor, *Über eine elementare Frage der Mannigfaltigkeitslehre*, Jahresbericht der Deutschen Mathematiker-Vereinigung **1** (1891), 75–78.
- [CDE⁺07] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Carolyn Talcott, *All about Maude*, 1st ed., Springer, 2007.
- [Chr76] Nicos Christofides, *Worst-case analysis of a new heuristic for the travelling salesman problem*, Tech. Report 388, Graduate School of Industrial Administration, Carnegie Mellon University, 1976.
- [Chu36a] Alonzo Church, *A note on the Entscheidungsproblem*, The Journal of Symbolic Logic **1** (1936), no. 1, 40–41, Corrección en el mismo volumen, páginas 101–102.
- [Chu36b] ———, *An unsolvable problem of elementary number theory*, American Journal of Mathematics **58** (1936), no. 2, 345–363.
- [CLRS22] Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein, *Introduction to algorithms*, 4th ed., MIT Press, 2022.
- [Coh90] Edward Cohen, *Programming in the 1990s: an introduction to the calculation of programs*, Springer-Verlag, 1990.
- [Coo71] Stephen A. Cook, *The complexity of theorem-proving procedures*, Proceedings of the Third Annual ACM Symposium on Theory of Computing, 1971, pp. 151–158.
- [CT65] James W. Cooley and John W. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Mathematics of Computation **19** (1965), no. 90, 297–301.

- [Dav85] Martin Davis, *Computability and unsolvability*, Dover Publications, 1985.
- [Dav04] ———, *The undecidable: Basic papers on undecidable propositions, unsolvable problems and computable functions*, Dover, 2004, Edición original: Raven Press, 1965.
- [DF13] Rodney G. Downey and Michael R. Fellows, *Fundamentals of parameterized complexity*, Texts in Computer Science, Springer, 2013.
- [Dij59] Edsger W. Dijkstra, *A note on two problems in connexion with graphs*, Numerische Mathematik **1** (1959), 269–271.
- [Dij72] ———, *Notes on structured programming*, Structured Programming (Ole-Johan Dahl, Edsger W. Dijkstra, and C. A. R. Hoare, eds.), Academic Press, London, 1972, pp. 1–82.
- [Dij76] Edsger Wybe Dijkstra, *A discipline of programming*, Prentice-Hall, 1976.
- [DM79] Nachum Dershowitz and Zohar Manna, *Proving termination with multiset orderings*, Communications of the ACM **22** (1979), no. 8, 465–476.
- [Dre02] S. E. Dreyfus, *Richard Bellman on the birth of dynamic programming*, Operations Research **50** (2002), no. 1, 48–51.
- [DS90] Edsger W. Dijkstra and Carel S. Scholten, *Predicate calculus and program semantics*, Springer-Verlag, 1990.
- [Edm71] Jack Edmonds, *Matroids and the greedy algorithm*, Mathematical Programming **1** (1971), 127–136.
- [Eri19] Jeff Erickson, *Algorithms*, Independently published, 2019.
- [Fen94] Peter M. Fenwick, *A new data structure for cumulative frequency tables*, Software: Practice and Experience **24** (1994), no. 3, 327–336.
- [Flo62] Robert W. Floyd, *Algorithm 97: Shortest path*, Communications of the ACM **5** (1962), no. 6, 345.
- [Flo67] ———, *Assigning meanings to programs*, Mathematical Aspects of Computer Science (J. T. Schwartz, ed.), Proceedings of Symposia in Applied Mathematics, vol. 19, American Mathematical Society, 1967, pp. 19–32.
- [For13] Lance Fortnow, *The golden ticket: P, NP, and the search for the impossible*, Princeton University Press, 2013.
- [FT87] Michael L. Fredman and Robert Endre Tarjan, *Fibonacci heaps and their uses in improved network optimization algorithms*, Journal of the ACM **34** (1987), no. 3, 596–615.
- [GB65] Solomon W. Golomb and Leonard D. Baumert, *Backtrack programming*, Journal of the ACM **12** (1965), no. 4, 516–524.
- [GH85] Ronald L. Graham and Pavol Hell, *On the history of the minimum spanning tree problem*, Annals of the History of Computing **7** (1985), no. 1, 43–57.
- [GJ79] Michael R. Garey and David S. Johnson, *Computers and intractability: A guide to the theory of NP-completeness*, W. H. Freeman, 1979.
- [GKP94] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik, *Concrete mathematics: A foundation for computer science*, second ed., Addison-Wesley, 1994.

-
- [Göd31] Kurt Gödel, *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I*, Monatshefte für Mathematik und Physik **38** (1931), 173–198.
 - [Gri81] David Gries, *The science of programming*, Springer-Verlag, 1981.
 - [Gri82] D. Gries, *A note on a standard strategy for developing loop invariants and loops*, Science of Computer Programming **2** (1982), no. 3, 207–214.
 - [GS93] David Gries and Fred B. Schneider, *A logical approach to discrete math*, Springer-Verlag, 1993.
 - [Hak85] Armin Haken, *The intractability of resolution*, Theoretical Computer Science **39** (1985), 297–308.
 - [Hal11] Paul R. Halmos, *Naive set theory*, 1st ed., Martino Fine Books, 2011.
 - [Har12] David Harel, *Algorithmics: The spirit of computing*, 3rd ed., Springer, 2012.
 - [HHE20] Steven Halim, Felix Halim, and Suhendry Effendy, *Competitive programming 4: Book I*, LuLu, 2020.
 - [Hil00] David Hilbert, *Mathematische Probleme*, Nachrichten von der Königlichen Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse (1900), 253–297.
 - [Hir75] D. S. Hirschberg, *A linear space algorithm for computing maximal common subsequences*, Communications of the ACM **18** (1975), no. 6, 341–343.
 - [HK62] M. Held and R. M. Karp, *A dynamic programming approach to sequencing problems*, Journal of the Society for Industrial and Applied Mathematics **10** (1962), no. 1, 196–210.
 - [HK70] Michael Held and Richard M. Karp, *The traveling-salesman problem and minimum spanning trees*, Operations Research **18** (1970), no. 6, 1138–1162.
 - [HK71] ———, *The traveling-salesman problem and minimum spanning trees: Part II*, Mathematical Programming **1** (1971), 6–25.
 - [HNR68] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael, *A formal basis for the heuristic determination of minimum cost paths*, IEEE Transactions on Systems Science and Cybernetics **4** (1968), no. 2, 100–107.
 - [Hoa62] C. A. R. Hoare, *Quicksort*, The Computer Journal **5** (1962), no. 1, 10–16.
 - [Hoa69] ———, *An axiomatic basis for computer programming*, Communications of the ACM **12** (1969), no. 10, 576–580.
 - [HR87] Jr. Hartley Rogers, *Theory of recursive functions and effective computability*, MIT Press, 1987.
 - [HS74] Ellis Horowitz and Sartaj Sahni, *Computing partitions with applications to the knapsack problem*, Journal of the ACM **21** (1974), no. 2, 277–292.
 - [Huf52] David A. Huffman, *A method for the construction of minimum-redundancy codes*, Proceedings of the IRE **40** (1952), no. 9, 1098–1101.
 - [Ing62] P. Z. Ingerman, *Algorithm 141: Path matrix*, Communications of the ACM **5** (1962), no. 11, 556.

- [IP01] Russell Impagliazzo and Ramamohan Paturi, *On the complexity of k -SAT*, Journal of Computer and System Sciences **62** (2001), no. 2, 367–375.
- [Jar30] Vojtěch Jarník, *O jistém problému minimálním*, Práce Moravské Přírodovědecké Společnosti (1930), 57–63.
- [Kad23] J. B. Kadane, *Two Kadane algorithms for the maximum sum subarray problem*, Algorithms **16** (2023), no. 11, 519.
- [Kal90] Anne Kaldewaij, *Programming: the derivation of algorithms*, Prentice-Hall, 1990.
- [Kar72] Richard M. Karp, *Reducibility among combinatorial problems*, Complexity of Computer Computations (Raymond E. Miller and James W. Thatcher, eds.), Plenum Press, 1972, pp. 85–103.
- [KB70] Donald E. Knuth and Peter B. Bendix, *Simple word problems in universal algebras*, Computational Problems in Abstract Algebra (John Leech, ed.), Pergamon Press, 1970, pp. 263–297.
- [KET06] Jon Kleinberg and Éva Tardos, *Algorithm design*, Pearson, 2006.
- [Kle56] S. C. Kleene, *Representation of events in nerve nets and finite automata*, Automata Studies (C. E. Shannon and J. McCarthy, eds.), Annals of Mathematics Studies, no. 34, Princeton University Press, Princeton, 1956, pp. 3–41.
- [Knu74] Donald E. Knuth, *Computer science and its relation to mathematics*, The American Mathematical Monthly **81** (1974), no. 4, 323–343.
- [Knu75] ———, *Estimating the efficiency of backtrack programs*, Mathematics of Computation **29** (1975), 121–136.
- [Knu76] Donald E. Knuth, *Big omicron and big omega and big theta*, SIGACT News **8** (1976), no. 2, 18–24.
- [Knu88] Donald Knuth, *The art of computer programming: Volume 3 / sorting and searching*, 2nd ed., Addison-Wesley, 1988.
- [Knu97] ———, *The art of computer programming: Volume 1 / fundamental algorithms*, 3rd ed., Addison-Wesley, 1997.
- [KO62] Anatolii Karatsuba and Yuri Ofman, *Multiplication of many-digit numbers by automatic computers*, Doklady Akademii Nauk SSSR **145** (1962), no. 2, 293–294.
- [Kru56] Joseph B. Kruskal, *On the shortest spanning subtree of a graph and the traveling salesman problem*, Proceedings of the American Mathematical Society **7** (1956), no. 1, 48–50.
- [KS98] Donald L. Kreher and Douglas R. Stinson, *Combinatorial algorithms: Generation, enumeration, and search*, Discrete Mathematics and Its Applications, CRC Press, 1998.
- [Kun11] Kenneth Kunen, *Set theory*, 2nd ed., College Publications, 2011.
- [KV18] Bernhard Korte and Jens Vygen, *Combinatorial optimization: Theory and algorithms*, 6 ed., Springer, 2018.
- [Lan09] Edmund Landau, *Handbuch der lehre von der verteilung der primzahlen*, Teubner, Leipzig, 1909.

- [LD60] Ailsa H. Land and Alison G. Doig, *An automatic method of solving discrete programming problems*, *Econometrica* **28** (1960), no. 3, 497–520.
- [Leh57] Derrick H. Lehmer, *Combinatorial problems with digital computers*, Proceedings of the Fourth Canadian Mathematical Congress, 1957, pp. 160–173.
- [Lev73] Leonid A. Levin, *Universal sequential search problems*, *Problemy Peredachi Informatsii* **9** (1973), no. 3, 115–116, En ruso. Traducción al inglés en *Problems of Information Transmission* **9** (1973), 265–266.
- [Lev12] Anany Levitin, *Introduction to the design and analysis of algorithms*, 3rd ed., Pearson, 2012.
- [LMSK63] John D. C. Little, Katta G. Murty, Dura W. Sweeney, and Caroline Karel, *An algorithm for the traveling salesman problem*, *Operations Research* **11** (1963), no. 6, 972–989.
- [LW66] Eugene L. Lawler and David E. Wood, *Branch-and-bound methods: A survey*, *Operations Research* **14** (1966), no. 4, 699–719.
- [Mat70] Yuri V. Matiyasevich, *Enumerable sets are Diophantine*, *Soviet Mathematics Doklady* **11** (1970), 354–358.
- [ML43] Luigi F. Menabrea and Ada A. Lovelace, *Sketch of the analytical engine invented by Charles Babbage, with notes by the translator*, *Scientific Memoirs* **3** (1843), 666–731.
- [MT90] Silvano Martello and Paolo Toth, *Knapsack problems: Algorithms and computer implementations*, Wiley-Interscience Series in Discrete Mathematics and Optimization, John Wiley and Sons, 1990.
- [New42] Maxwell H. A. Newman, *On theories with a combinatorial definition of “equivalence”*, *Annals of Mathematics* **43** (1942), no. 2, 223–243.
- [OEI] OEIS Foundation Inc., *Sequence A000170: Number of ways of placing n non-attacking queens on an $n \times n$ board*, The On-Line Encyclopedia of Integer Sequences, <https://oeis.org/A000170>.
- [Ohl02] Enno Ohlebusch, *Advanced topics in term rewriting*, 2nd ed., Springer, 2002.
- [Oxl11] James Oxley, *Matroid theory*, 2 ed., Oxford University Press, 2011.
- [PECG⁺25] Álvaro Perales-Eceiza, Toby Cubitt, Mile Gu, David Pérez-García, and Michael M. Wolf, *Undecidability in physics: A review*, *Physics Reports* **1138** (2025), 1–29, Acceso abierto. Preprint disponible como arXiv:2410.16532.
- [Pet08] Charles Petzold, *The annotated Turing: A guided tour through Alan Turing’s historic paper on computability and the Turing machine*, Wiley, 2008.
- [Pos44] Emil L. Post, *Recursively enumerable sets of positive integers and their decision problems*, *Bulletin of the American Mathematical Society* **50** (1944), no. 5, 284–316.
- [Pos46] ———, *A variant of a recursively unsolvable problem*, *Bulletin of the American Mathematical Society* **52** (1946), no. 4, 264–268.
- [Pos47] ———, *Recursive unsolvability of a problem of Thue*, *The Journal of Symbolic Logic* **12** (1947), no. 1, 1–11.

- [Pri57] Robert C. Prim, *Shortest connection networks and some generalizations*, The Bell System Technical Journal **36** (1957), no. 6, 1389–1401.
- [Pyt26] Python Software Foundation, *Time and space complexity reference*, Python Wiki, <https://wiki.python.org/moin/TimeComplexity>, 2026, Consultado en septiembre de 2026.
- [Ric53] Henry Gordon Rice, *Classes of recursively enumerable sets and their decision problems*, Transactions of the American Mathematical Society **74** (1953), no. 2, 358–366.
- [Rom15] D. Romik, *The surprising mathematics of longest increasing subsequences*, Institute of Mathematical Statistics Textbooks, no. 4, Cambridge University Press, New York, 2015.
- [Rou17] Tim Roughgarden, *Algorithms illuminated I: The basics*, Soundlikeyourself Publishing, 2017.
- [Roy59] B. Roy, *Transitivité et connexité*, Comptes Rendus de l'Académie des Sciences de Paris **249** (1959), 216–218.
- [Rus06] John Rust, *Dynamic programming*, pp. 1–26, Palgrave Macmillan, 12 2006.
- [Sch61] C. Schensted, *Longest increasing and decreasing subsequences*, Canadian Journal of Mathematics **13** (1961), 179–191.
- [Sch04] A. Schrijver, *Combinatorial optimization: Polyhedra and efficiency*, Springer, Berlin, 2004.
- [SF13] Robert Sedgewick and Philippe Flajolet, *An introduction to the analysis of algorithms*, 2nd ed., Addison-Wesley Professional, 2013.
- [Sip12] Michael Sipser, *Introduction to the theory of computation*, 3rd ed., Cengage Learning, 2012.
- [Ski08] Steven S. Skiena, *The algorithm design manual*, Springer London, 2008.
- [SW11] Robert Sedgewick and Kevin Wayne, *Algorithms*, fourth ed., Addison-Wesley, 2011.
- [Tar75] Robert Endre Tarjan, *Efficiency of a good but not linear set union algorithm*, Journal of the ACM **22** (1975), no. 2, 215–225.
- [Tar85] Robert Endre Tarjan, *Amortized computational complexity*, SIAM Journal on Algebraic and Discrete Methods **6** (1985), no. 2, 306–318.
- [Tho84] Ken Thompson, *Reflections on trusting trust*, Communications of the ACM **27** (1984), no. 8, 761–763, Conferencia de recepción del Premio Turing.
- [Tur36] Alan M. Turing, *On computable numbers, with an application to the Entscheidungsproblem*, Proceedings of the London Mathematical Society **42** (1936), 230–265, Corrección en el volumen 43 (1937), páginas 544–546.
- [Vaz01] Vijay V. Vazirani, *Approximation algorithms*, Springer, 2001.
- [Wal60] Robert J. Walker, *An enumerative technique for a class of combinatorial problems*, Combinatorial Analysis (Richard E. Bellman and Marshall Hall, Jr., eds.), Proceedings of Symposia in Applied Mathematics, vol. 10, American Mathematical Society, Providence, RI, 1960, pp. 91–94.

- [War62] S. Warshall, *A theorem on Boolean matrices*, Journal of the ACM **9** (1962), no. 1, 11–12.
- [WF74] R. A. Wagner and M. J. Fischer, *The string-to-string correction problem*, Journal of the ACM **21** (1974), no. 1, 168–173.
- [Wil64] John W. J. Williams, *Algorithm 232: Heapsort*, Communications of the ACM **7** (1964), no. 6, 347–348.

Índice alfabético

- agente viajero, 696
- al-Khwarismi, 34
- algoritmo, 34
 - de aproximación, 701
 - de Dijkstra, 312
 - de Karatsuba, 150
 - de Kruskal, 414
 - de Prim, 423
 - origen de la palabra, 34
 - que resuelve un problema, 36
 - verificación, 648
- algoritmo polinomial
 - aceptación, 643
 - decisión, 643
- algoritmos voraces, 341
 - argumento de intercambio, 358
 - argumento de inversión, 391
 - candidato, 346
 - criterio de selección, 346
 - el voraz va adelante, 360
 - metodología, 367
 - propiedad de elección voraz, 348
- análisis amortizado, 139
- análisis asintótico, 69
 - criterio del límite, 95
 - de algoritmos iterativos, 106
 - función simple, 122
 - regla de constantes, 90
 - regla de la suma, 94
 - regla del producto, 94
 - subsunción por suma, 92
- teorema maestro, 124
- transitividad, 93
- aproximación de Stirling, 102
- árbol
 - de reintento, 458
- árbol de recurrencia, 117
- argumento diagonal, 586, 607
- arreglo, 2
 - como función, 6
 - codominio, 6
 - dominio, 6
 - imagen, 6
 - concatenación, 4
 - en Python, 7
 - igualdad, 4
 - indexación, 3
 - índice, 2
 - inversión, 186, 390
 - ordenado, 14
 - permutación, 7
 - sección, 3
 - propiedades, 5
 - subarreglo, 3
 - tamaño, 2, 3
 - vacío, 3
- arreglo dinámico, 142
- autorreferencia, 610
- Babbage, Charles, 34
- bandera nacional holandesa, 197
- Baumert, Leonard, 456

- Bellman, Richard, 235, 237, 335
- bisección, 213
 - código Python, 214
 - punto flotante, 217
- Borůvka, Otakar, 343
- buena fundación, 53
- búsqueda binaria, 198
 - complejidad temporal, 127, 203
 - correctitud, 202
 - código Python, 201, 208
 - código Python iterativo, 203
 - diseño, 199, 201, 207
 - especificación del problema, 198
 - sobre predicados monótonos, 206
 - terminación, 202
- búsqueda ternaria, 218
- cadena segmentable, 497
- cadenas de adición, 231
- cardinal, 581
 - contable, 583
 - igualdad, 581
 - no contable, 583
 - orden, 584
 - suma, 581
- caso promedio, 78
- centinela, 16
- Church, Alonzo, 35, 564
- ciclo
 - invariante, 47
- ciclo hamiltoniano, 695
- ciclo iterativo
 - condición de terminación, 156
- clase
 - coNP, 656
 - NP, 650
 - NPC, 670
 - P, 644
- cobertura de vértices, 682
- coeficiente binomial, 255
- complejidad
 - espacial, 78
 - mejor caso, 78
 - peor caso, 77
 - temporal, 78
- conjunto ordenado, 51
- conjuntos
 - equipotencia, 581, 583
- corrección
 - parcial, 44
 - total, 44
- costo amortizado, 139
- cota optimista, 513
- cuantificador, 9
 - de conteo, 11
 - en Python, 17
 - forma general, 10
 - rango, 10
 - rango finito, 10
 - rango vacío, 12
 - reglas de manipulación, 13
 - separación de un término, 13
 - término, 10
 - variable ligada, 10
- descomposición, 152
- Dijkstra, Edsger, 46
- Dijkstra, Edsger, 343
- distancia de edición, 309
- dividir y conquistar, 149
- dividir y conquistar
 - _ y combinar, 149
 - análisis, 156
 - fases, 153
 - ideas clave, 152
 - independencia, 153
 - metodología, 164
- Edmonds, Jack, 343
- eficiencia algorítmica, 69
- elemento maximal, 53
- elemento minimal, 53
- Entscheidungsproblem*, 561
- enumerador, 575
- espacio auxiliar, 79
- especificación, 19
 - entrada, 19
 - lenguajes, 39
 - parámetro, 21
 - poscondición, 19

- precondición, 19
- salida, 19
- estructura de datos, 416
- exponenciación rápida, 226
 - complejidad temporal, 227
 - código Python, 226
 - modular, 229
- Floyd, Robert, 312
- Ford, Lester, 237
- forma normal conjuntiva, 672
- función
 - de Fibonacci, 243
 - implementación adhoc, 244
 - implementación con memorización, 247
 - implementación con tabulación, 251
 - simple, 122
- función unimodal, 218
- función
 - complementaria, 553
 - computable, 557
 - computable sobre cadenas, 598
- función de cota, 57
- generalización, 165
- Golomb, Solomon, 456
- grafo, 310
 - arcos, 310
 - camino, 310
 - ciclo, 310
 - circuito, 324
 - completo, 324
 - dirigido, 310
 - no-dirigido, 310
 - vértices, 310
- Grenander, Ulf, 263
- Held, Michael, 335
- Hilbert, David, 537, 561
- Hirschberg, Daniel, 308
- Huffman, David, 343
- inducción matemática, 149, 153
- inducción
 - bien fundada, 55
 - desde un valor inicial, 61
 - fuerte, 60
 - ordinaria, 60
- intérprete universal, 562
- invariante, 47, 155, 173
 - estabilidad, 156
 - iniciación, 156
 - justificación del método, 61
 - método de, 47
- inversión, *véase también* arreglo,
 - inversión
- jerarquía de crecimiento, 99
- Kadane, Jay, 263
- Karp, Richard, 335
- Knuth, Donald, 33
- Kruskal, Joseph, 343
- Lehmer, Derrick, 456
- lema de Newman, 632
- lenguaje
 - aceptación de, 554
 - computable, 558
 - caracterización, 577
 - clausura, 568
 - computablemente enumerable, 558
 - caracterización, 576
 - clausura, 569
 - decisión de, 554
 - verificado, 649
- lenguaje formal, 544
 - aceptación, 551
 - operaciones, 545
 - orden canónico, 546
 - rechazo, 551
- Leonardo de Pisa, 243
- logaritmo, 83
 - identidades, 101
- Lovelace, Ada, 34
- máscara de bits, 330, 467
 - conjunto unitario, 332
 - conjunto universo, 331
 - eliminación de un elemento, 331

- pertenencia, 331
- medida, 56
- mejor caso, 78
- Menger, Karl, 325
- mergesort, 178
 - código Python, 180
 - complejidad espacial, 185
 - complejidad temporal, 127, 184
 - correctitud, 182, 183
 - diseño, 179
- método de Newton, 219
- método del potencial, 141
- modelo de computación, 35
- modelo de costos, 75
- modelo de cómputo, 556
 - equivalencia, 544
- multiplicación del campesino ruso, 231
- notación asintótica
 - en expresiones, 97
 - O , 81
 - Ω , 87
 - Θ , 87
- notación de Eindhoven, 9
- operación barómetro, 75
- operación elemental, 75
- orden, 51
 - lineal, 52
- ordenamiento
 - estable, 197
 - por selección, 36
- ordenamiento por inserción, 76
- par crítico, 634
- paralelismo, 150
- peor caso, 77
- permutación, 7
- podar, 459
- poscondición, 19
- precondición, 19
- predicado de factibilidad, 458
- predicado monótono, 206
- Prim, Robert, 343
- principio de optimalidad, 239
- problema, 19
 - algorítmico, 19
 - árbol de cubrimiento mínimo, 406
 - caminos más cortos, 440
 - cálculo de números de Fibonacci, 243
 - de optimización
 - versión de valor, 30
 - de agendamiento de actividades, 353
 - de búsqueda combinatoria, 457
 - de búsqueda, 29
 - de caminos más cortos, 439
 - de caminos mínimos entre todos los pares, 309
 - de capacidad máxima, 323
 - de clausura transitiva, 323
 - de conteo, 29
 - de correspondencia de Post, 625
 - de cubrimiento mínimo de intervalos, 372
 - de decisión, 29
 - de enumeración, 29
 - de equivalencia, 614
 - de generación de permutaciones, 468
 - de generación de subconjuntos, 465
 - de la parada, 605
 - sobre la cadena vacía, 613
 - de la planeación de una travesía, 219
 - de la potencia, 226
 - de la raíz cuadrada entera, 206, 209
 - de la raíz de una función continua, 214
 - de la ruta más corta, 240
 - de la ruta simple más larga, 240
 - de la subcadena común más larga, 309
 - de la subsecuencia común más larga, 297
 - de la subsecuencia creciente más larga, 271
 - de la suma de subconjuntos, 29
 - de las N reinas, 24
 - de las 8 reinas, 23
 - de las N reinas, 476
 - de optimización, 29, 345
 - versión de testigo, 30
 - de ordenamiento, 20

- de ordenamiento de un arreglo, 169, 178, 186
- de segmentación de texto, 497
- de selección, 197
- de suma exacta de un subconjunto, 296, 487
- de suma máxima de un subarreglo, 261
- de teselación de un tablero
 - generalizado, 159
- de tiempo en el sistema, 387
- de un árbol de cubrimiento mínimo, 405
- decisión, 536
- decisión con lenguaje formal, 549
- del agente viajero, 324, 325, 522
- del morral, 281, 512
- del morral fraccionario, 397, 515
- indecidible, 560
- instancia, 21
- solución de una instancia, 22
- tamaño de la entrada, 73
- testigo, 29
- programa, 36
- programación dinámica
 - tabulación, 246
- programación dinámica, 235
 - memorización, 245, 246
 - metodología, 255
 - propiedad de solapamiento, 238
 - propiedad de subestructura, 238
 - tabulación, 250
- propiedad
 - semántica, 617
- Python, 556
- quicksort, 186
 - código Python, 189
 - complejidad espacial, 195
 - complejidad temporal, 192–194
 - correctitud, 190, 191
 - pivote aleatorio, 194
- ramificación y poda, 511
- recurrencia
 - de historia completa, 508
 - obtención, 113
- reducción
 - polinomial, 662
- reducción
 - computable, 598
- reintento, 455
 - metodología, 472
- relación, 50
 - bien fundada, 53
 - imagen, 51
- Roy, Bernard, 312
- Shamos, Michael, 263
- simulación acotada, 566
- sistema de reescritura
 - confluencia, 631
 - de cadenas, 630
 - terminación, 631
- solución parcial, 457
- solución, 22
- subarreglo, 70
- subestructura óptima, 238, 349
- subsecuencia, 271
 - común, 297
- suma de subconjuntos, 687
- suma máxima de un subarreglo, 70
 - dividir y conquistar, 128
- sumas
 - cotas por integrales, 104
- sumas prefijas, 79
- técnicas de diseño, 149
- teorema
 - del valor intermedio, 213
- teorema de Post, 570
- teorema de Rice, 618
- Teorema Maestro, 124, 157
- terminación, 57
- tesis de Church y Turing, 562
- tesis de Church–Turing, 35
- testigo, 29
- tiempo pseudo-polinomial, 289, 295
- tipo abstracto de datos, 416
- Torres de Hanói, 163

transformada rápida de Fourier, 150

Turing, Alan, 35, 564

variante, 57

Walker, Robert, 456

Warshall, Stephen, 312

Whitney, Hassler, 325

